МАШИННОЕ ОБУЧЕНИЕ

ОСНОВЫ

Теорема Байеса,
тест Тьюринга,
ирисы Фишера,
правило Лапласа,
проклятие размерности,
логистическая регрессия,
биномиальные обезьяны,
кризис воспроизводимости,
информационные критерии,
развитие пандемии COVID-19,
экспоненциальное семейство,
метод наименьших квадратов,
ретроспективные исследования
и многое другое...

Сергей Николенко

МАШИННОЕ ОБУЧЕНИЕ

ОСНОВЫ

Сергей Николенко

ПИТЕР®

Санкт-Петербург • Москва • Минск

2025

Сергей Николенко

Машинное обучение: основы

Серия «Библиотека программиста»

Руководитель дивизиона	*Ю. Сергиенко*
Ведущий редактор	*Н. Гринчик*
Литературный редактор	*Н. Терех*
Художественный редактор	*В. Мостипан*
Корректоры	*О. Андриевич, Е. Павлович*

ББК 32.973.2-018 +32.813
УДК 004.85

Николенко Сергей

Н63 Машинное обучение: основы. — СПб.: Питер, 2025. — 608 с.: ил. — (Серия «Библиотека программиста»).

ISBN 978-5-4461-4191-3

Машинное обучение давно уже стало синонимом искусственного интеллекта. Оно проникло во многие аспекты нашей жизни и стало одной из важнейших областей современной науки. Эта книга — путеводитель по ключевым идеям машинного обучения. Вы узнаете, как методы машинного обучения получаются из основных принципов теории вероятностей, пройдёте путь от теоремы Байеса до обобщённых линейных моделей и узнаете в лицо тех китов, на которых стоит весь современный искусственный интеллект. Множество увлекательных кейсов, практических примеров и интересных задач — от анализа ретроспективных научных исследований до эффекта «горячей руки» в баскетболе — помогут разобраться в самых сложных концепциях. Кроме того, книга может лечь в основу базовых курсов по машинному обучению.

16+ (В соответствии с Федеральным законом от 29 декабря 2010 г. № 436-ФЗ.)

ISBN 978-5-4461-4191-3

© ООО Издательство «Питер», 2025
© Серия «Библиотека программиста», 2025
© Сергей Николенко, 2025

Все права защищены. Никакая часть данной книги не может быть воспроизведена в какой бы то ни было форме без письменного разрешения владельцев авторских прав.

Информация, содержащаяся в данной книге, получена из источников, рассматриваемых издательством как надежные. Тем не менее, имея в виду возможные человеческие или технические ошибки, издательство не может гарантировать абсолютную точность и полноту приводимых сведений и не несет ответственности за возможные ошибки, связанные с использованием книги. Издательство не несет ответственности за доступность материалов, ссылки на которые вы можете найти в этой книге. На момент подготовки книги к изданию все ссылки на интернет-ресурсы были действующими.

Изготовлено в России. Изготовитель: ООО «Прогресс книга». Место нахождения и фактический адрес: 194044, Россия, г. Санкт-Петербург, Б. Сампсониевский пр., д. 29А, пом. 52. Тел.: +78127037373.

Дата изготовления: 03.2025. Наименование: книжная продукция. Срок годности: не ограничен.

Налоговая льгота — общероссийский классификатор продукции ОК 034-2014, 58.11.12 — Книги печатные профессиональные, технические и научные.

Импортер в Беларусь: ООО «ПИТЕР М», 220020, РБ, г. Минск, ул. Тимирязева, д. 121/3, к. 214, тел./факс: 208 80 01.

Подписано в печать 12.02.25. Формат 70×100/16. Бумага офсетная. Усл. п. л. 49,020. Тираж 1000. Заказ 0000.

Посвящается
маме,
Маше
и
Майлзу

Оглавление

Предисловие **9**
Весна искусственного интеллекта 9
Горький урок или повод для оптимизма? 13
План книги: о чём пойдёт речь далее 17
История книги и благодарности 20

Глава 1. Что такое машинное обучение

1.1 Искусственный интеллект от Адама до Франкенштейна **24**
1.1.1 Ранняя история 24
1.1.2 Механические автоматы Средних веков и Нового времени 27
1.1.3 Ранняя математическая логика 30

1.2 AI как наука: тест Тьюринга и три волны хайпа **34**
1.2.1 Тест Тьюринга 34
1.2.2 Дартмутский семинар и первая волна хайпа 39
1.2.3 Машинный перевод и первая «зима искусственного интеллекта» 42
1.2.4 Обратное распространение ошибки и вторая волна хайпа 47

1.3 Постановки задач искусственного интеллекта **50**
1.3.1 Классификация задач искусственного интеллекта 50
1.3.2 Обучение с учителем: регрессия и классификация 52
1.3.3 Обучение без учителя 54

1.4 Области искусственного интеллекта: какие бывают данные **57**
1.4.1 Извлечение признаков в машинном обучении 57
1.4.2 Оверфиттинг и разные части датасета 58
1.4.3 Табличные данные 60
1.4.4 Обработка последовательностей 61
1.4.5 Обработка изображений и более сложных типов данных 62

1.5 Как машинное обучение помогает другим наукам **64**
1.5.1 Физика и астрономия 64
1.5.2 Математика 66
1.5.3 Химия и биология 69
1.5.4 Науки о Земле 72
1.5.5 Социальные науки 73

Глава 2. Основы байесовского вывода

2.1 Основы теории вероятностей **77**
2.1.1 Введение: вероятностные пространства и распределения 77
2.1.2 Случайные величины и совместные распределения 82
2.1.3 Условные вероятности 85
2.1.4 Независимость и условная независимость 87
2.1.5 Моменты случайной величины: ожидание и дисперсия 91
**2.1.6 Как ещё можно ввести понятие вероятности* *94*

2.2 Вероятности в машинном обучении: теорема Байеса **100**
2.2.1 Теорема Байеса в машинном обучении 100
2.2.2 Медицинский тест с двусторонней ошибкой 104
2.2.3 Разные виды ошибок и метрики качества классификации 106
2.2.4 Задачи байесовского вывода 110

2.3 Байес в суде и сложности вероятностной интуиции **115**
2.3.1 Вероятностная интуиция — точнее, её отсутствие 115
2.3.2 Ошибка прокурора 117
2.3.3 Ошибка адвоката 120
2.3.4 Парадокс Монти Холла и когнитивные искажения 122

2.4 Монетка и сопряжённые априорные распределения **126**
2.4.1 Монетка с точки зрения байесовского вывода 126
2.4.2 Бета-распределения как апостериорные для монетки 129
2.4.3 Предсказательное распределение и правило Лапласа 132
2.4.4 Сопряжённые априорные распределения 134
2.4.5 Игральные кости и распределения, добавляющие разреженность 139

2.5 Case study: монетки, подброшенные «горячей рукой»* ***149*
**2.5.1 Разоблачение эффекта «горячей руки»* *149*
**2.5.2 Разоблачение разоблачения эффекта «горячей руки»* *151*
**2.5.3 Выводы* *155*

2.6 Кризис воспроизводимости **157**
2.6.1 Кризис воспроизводимости в психологии и социальных науках 157
2.6.2 Почему результаты не воспроизводятся: p-значения и p-хакинг 162
2.6.3 Сад расходящихся тропок и парапсихология 167
**2.6.4 Модель Иоаннидиса* *170*
2.6.5 Воспроизводимость в машинном обучении 173

Глава 3. Теория вероятностей и оптимизация

3.1 Важные дискретные распределения **177**
3.1.1 Равномерное, биномиальное и геометрическое распределения 177
3.1.2 Биномиальное распределение Пуассона 181
3.1.3 Распределение Пуассона и закон редких событий 183
3.1.4 Отрицательное биномиальное, гипергеометрическое распределения . . 186
**3.1.5 Закон Ципфа* *191*

3.2 Важные непрерывные распределения **196**
3.2.1 Непрерывное равномерное распределение 196
3.2.2 Нормальное распределение 197
3.2.3 Логнормальное распределение и распределение Стьюдента 204
3.2.4 Гамма-распределение и экспоненциальное распределение 207

3.3 Немецкие танки, датская камбала и биномиальные обезьяны **213**
3.3.1 Выборка без замещения и военная разведка 213
3.3.2 Байесовский анализ и некорректные априорные распределения 217
3.3.3 Как правильно ловить уток 224
**3.3.4 Пример Джейнса: урна с шарами и биномиальные обезьяны* *228*

3.4 Энтропия, KL-дивергенция и полное незнание **238**
3.4.1 Энтропия как мера неопределённости 238
3.4.2 Производные понятия: перекрёстная энтропия и KL-дивергенция . . . 243
3.4.3 KL-дивергенция в машинном обучении 246
3.4.4 Информация Фишера и принцип максимума энтропии 251
*3.4.5 *Априорные распределения Джеффриса* *258*
3.5 Оптимизация в машинном обучении **265**
3.5.1 Машинное обучение и невыпуклая оптимизация 265
3.5.2 Анализ градиентного спуска: проблемы с масштабом 267
3.5.3 Стохастический градиентный спуск 273
3.5.4 Свойства стохастического градиентного спуска 275

Глава 4. Линейная регрессия

4.1 История вопроса и метод наименьших квадратов **280**
4.1.1 История вопроса: почему «регрессия»? 280
4.1.2 Метод наименьших квадратов 285
4.1.3 Коэффициент детерминации 290
4.1.4 Функции признаков в линейной регрессии 293
4.1.5 Локальные признаки 298
4.2 Оверфиттинг и регуляризация **304**
4.2.1 Оверфиттинг в регрессии с полиномиальными признаками 304
4.2.2 Гребневая регрессия 309
4.2.3 Лассо-регрессия 313
4.2.4 Геометрия регуляризации и другие её формы 314
4.3 Вероятностная интерпретация линейной регрессии **321**
4.3.1 Правдоподобие линейной регрессии и гауссовский шум 321
4.3.2 Все вероятностные предположения линейной регрессии 324
4.3.3 Интерпретация коэффициентов: корреляция и причинность 329
*4.3.4 *Вероятностная робастная регрессия: другие распределения шума* *335*
*4.3.5 *Другая робастная регрессия: RANSAC и оценка Тейла — Сена* *338*
4.4 Байесовский вывод в линейной регрессии **346**
4.4.1 Априорное и апостериорное распределения 346
4.4.2 Пример байесовского вывода в линейной регрессии 351
4.4.3 Предсказательное распределение 357
*4.4.4 *Оценка Джеймса — Штейна* *362*
4.5 Case study: линейная регрессия и коронавирус **371**
4.5.1 Постановка задачи 371
4.5.2 Линейная регрессия на логарифмической шкале 372
4.5.3 Обучаем функцию распределения гауссиана 375
4.5.4 Анализ результатов и выводы 376

Глава 5. Классификация

5.1 Постановка задачи, геометрия и вероятности **381**
5.1.1 Геометрия классификации: разделяющие поверхности 381
5.1.2 Геометрия классификации для нескольких классов 385
5.1.3 Линейный дискриминант Фишера 389
5.1.4 Порождающие модели для классификации: LDA и QDA. 397
5.2 Логистическая регрессия **405**
5.2.1 Как из линейной функции получить вероятности 405
5.2.2 Максимизация правдоподобия 409
5.2.3 Другие сигмоиды и пробит-регрессия 413
5.3 Байесовский вывод в логистической регрессии **416**
5.3.1 Лапласовская аппроксимация 416
5.3.2 Обобщённое нормальное распределение и формула Стирлинга 418
5.3.3 *Предсказательное распределение в логистической регрессии* *421*
5.4 Ирисы Фишера **426**
5.4.1 Набор данных 426
5.4.2 Предварительный анализ данных 428
5.4.3 Сравнение классификаторов 431
5.5 Общие замечания о классификаторах **435**
5.5.1 Несбалансированные классы и перевзвешенная ошибка 435
5.5.2 Калибровка классификаторов 439
5.5.3 Как изменить классификатор в новых условиях 449
5.5.4 *Проспективные и ретроспективные исследования*. *455*
5.6 Порождающие модели и наивный Байес **460**
5.6.1 Порождающие и дискриминирующие модели 460
5.6.2 Вероятностные предположения наивного Байеса. 463
5.6.3 Правдоподобие и сравнение с логистической регрессией. 465
5.6.4 Пример классификации текстов 467

Глава 6. Несколько важных сюжетов

6.1 Ближайшие соседи и проклятие размерности **478**
6.1.1 Метод ближайших соседей. 478
6.1.2 В чём проблема с ближайшими соседями? Проклятие размерности . . 483
6.1.3 Эффект кожуры апельсина 489
6.2 Статистическая теория принятия решений. **494**
6.2.1 Функция регрессии 494
6.2.2 Анализ метода ближайших соседей 496
6.2.3 Минимизация ожидаемой ошибки предсказания 498
6.2.4 Разложение на смещение, дисперсию и шум 500
6.3 Эквивалентные ядра и ядерные методы **507**
6.3.1 Другой взгляд на предсказания линейной регрессии 507
6.3.2 Эквивалентное ядро в линейной регрессии 509
6.3.3 Ядерные методы 511

6.4 Case study: байесовский вывод для гауссиана **514**
6.4.1 Выводы для фиксированного среднего и фиксированной точности . . . 514
6.4.2 Вывод для среднего и точности одновременно 519
6.4.3 Маргинализация апостериорного распределения 523
6.4.4 Предсказательное распределение для гауссиана 525
6.5 Оценки $p(D)$: эмпирический Байес и сравнение моделей **528**
6.5.1 Маргинальное правдоподобие и подбор гиперпараметров 528
6.5.2 Оценка $p(D)$ как метод сравнения моделей 535
6.5.3 Байесовский информационный критерий 538
*6.5.4 *Информационные критерии Такеучи и Акаике* *540*
6.6 Экспоненциальное семейство **553**
6.6.1 Определение и примеры 553
6.6.2 Моменты достаточных статистик 557
6.6.3 Максимизация правдоподобия 559
6.6.4 Сопряжённые априорные и предсказательные распределения 561
6.7 Обобщённые линейные модели **563**
6.7.1 Определение GLM 563
6.7.2 Примеры и максимизация правдоподобия 565
6.7.3 Пуассоновская регрессия 567
6.7.4 Отрицательная биномиальная регрессия 574
Заключение **576**
Литература **580**

Предисловие

Весна искусственного интеллекта

> А вот теперь весна, так и мысли всё такие приятные, острые, затейливые, и мечтания приходят нежные; всё в розовом цвете. Я к тому и написал это всё; а впрочем, я это всё взял из книжки.
>
> *Ф. М. Достоевский. Бедные люди*

Я пишу эти строки в сентябре 2023 года, после нескольких месяцев, которые хочется назвать «весной искусственного интеллекта»[1]. За три-четыре месяца, с марта по июнь 2023 года, произошло столько новых событий, что хватило бы на несколько лет; давайте попробуем проследить последовательность событий и оценить, в каком мире мы сейчас оказались.

Непосредственные основы для моделей, которые прямо сейчас меняют мир, были заложены ещё в 2020 году, с выпуском очередной языковой модели компании OpenAI — GPT-3, где GPT означает Generative Pretrained Transformer («порождающий предобученный трансформер») [330]. Языковая модель — это метод искусственного интеллекта, в котором модель обучается естественному языку, предсказывая недостающую часть текста, обычно следующий токен (слово или часть слова), а трансформер — это разработанный в 2017 году вид архитектуры искусственных нейронных сетей [34], оказавшийся очень удачным и стремительно набравший популярность; сейчас основанные на трансформерах архитектуры применяются практически во всех областях искусственного интеллекта.

Языковая модель GPT-3 — это логическое продолжение семейства языковых моделей GPT, которые появились в 2018 году, практически сразу вслед за архитектурой трансформера в целом [275]; изначальный трансформер предназначался для машинного перевода и состоял из кодировщика и декодировщика, но очень быстро стало ясно, что хорошие архитектуры можно получить и разделив его на части: GPT — это декодировщик трансформера (не пугайтесь, если вы не знаете, что это такое, книга совсем не об этом). В третьей итерации модели были новые идеи, но они не меняли сути происходящего: GPT по-прежнему обучалась предсказывать, чем будет продолжаться текст, и делала это на огромном наборе текстов, написанных людьми; современные наборы данных для этой цели — это фактически весь доступный текстовый интернет.

Главным отличием стал масштаб: самый большой вариант GPT-3 содержал около 175 миллиардов обучаемых параметров, на два порядка больше, чем модель GPT-2 [331], предыдущая в линейке. Её обучение стало возможным благодаря прогрессу последних лет в разработке графических карт (GPU), в том числе специализированных GPU-подобных процессоров для тензорных вычислений, которые оказались очень полезными для обучения нейросетей. А в модели

[1] О «зимах искусственного интеллекта» мы поговорим в разделе 1.2.

GPT-4 [450] было уже 1,76 *триллиона* обучаемых параметров, то есть ещё в десять раз больше[1].

Выяснилось, что это почти механическое увеличение масштаба оказалось достаточным для того, чтобы GPT-3 вышла на новый уровень способностей: модель начала писать гораздо более длинные связные тексты на любые темы. По интернету пробежала волна примеров её поразительных способностей, однако в 2020 году GPT-3 казалась очередной вехой скорее академического развития искусственного интеллекта. Примером попытки найти практическое применение для GPT-3 стала модель OpenAI Codex [171], которая могла писать программный код по текстовому описанию, то есть тоже переписывать один текст в другой, только теперь на выходе получался текст не на естественном языке, а на языке программирования. Codex стала основой для GitHub Copilot и действительно набрала популярность, хотя были у неё и очевидные проблемы. Полагаться на то, что Codex сама напишет для вас программу, всё-таки было нельзя[2].

Следующим шагом стало развитие GPT-3 до выпущенных весной 2022 года моделей из семейства, которое OpenAI назвала GPT-3.5. Они были обучены на ещё большем объёме данных и имели некоторые дополнительные возможности по сравнению с «базовой» GPT-3, в частности умели не только продолжать текст, но и заполнять в нём пробелы. Кроме того, в семействе GPT-3.5 появились дополнительные модели: дообученные «понимать» программный код или специально дообученные на чатах для улучшения диалогов с человеком. Понимаю, что это звучит не слишком впечатляюще, но именно модели из семейства GPT-3.5 стали основой для главного прорыва.

Этот прорыв наступил с выпуском модели ChatGPT [602], которая представляет собой вариант GPT-3.5, дообученный таким образом, чтобы стать более полезным при разговоре с человеком. Модель, даже очень хорошо умеющая продолжать тексты, вовсе не обязательно будет отвечать на ваши вопросы и тем более стараться делать это полезным для пользователя образом: почему бы не продолжить вопрос другим вопросом или ответом в духе «Ах, отстаньте от меня»?

Поэтому исследователи из OpenAI придумали специальный метод обучения с человеческим участием — обучение с подкреплением на основе отзывов людей (reinforcement learning with human feedback, RLHF), — которым смогли дообучить GPT-3.5 так, чтобы она старалась вести диалог максимально полезным для пользователя образом. Сначала этот метод привёл к модели InstructGPT, в ноябре 2022 года вышел полноценный ChatGPT, а к началу февраля 2023 эта модель стала самым быстрорастущим программным продуктом в истории: более 100 миллионов зарегистрированных пользователей всего за два месяца.

---

[1] А сейчас, дополню я из осени 2024 года, OpenAI уже давно объявила о том, что обучает GPT-5, и новая цифра означает увеличение числа параметров ещё на порядок...

[2] А сейчас, дополню я из декабря 2024 года, полноценная модель o1, которую OpenAI только что выпустила, уже способна самостоятельно писать небольшие проекты от начала до конца, вести рефакторинг сложного кода, исправлять нетривиальные баги... В интересные времена мы живём.

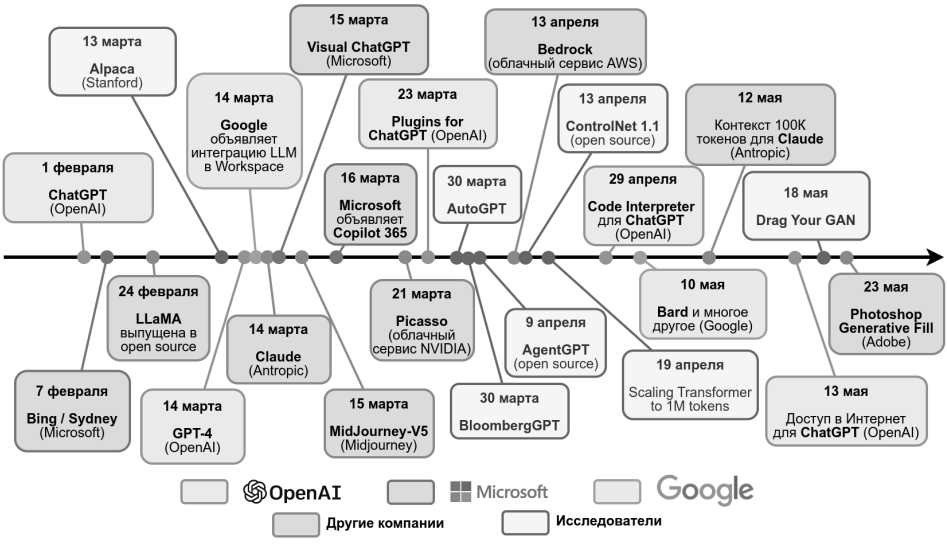


Рис. П.1. Развитие искусственного интеллекта весной 2023 года

И началось: весной 2023 года появилось огромное число моделей, либо созданных на основе ChatGPT, либо продолжающих и развивающих её идеи. Я собрал основные вехи прошедшей весны на рис. П.1; отмечу лишь самое главное:

- 7 февраля вышел аналог ChatGPT от Microsoft, языковая модель, призванная помочь поиску Bing; этот релиз оказался несколько преждевременным, помощник от Microsoft был быстро «взломан» пользователями — в частности, пользователи узнали, что помощника от Microsoft на самом деле «зовут» Sydney, и сейчас его обычно называют именно так — и часто выдавал странные, а порой и откровенно агрессивные ответы;
- 24 февраля вышла языковая модель LLaMA от исследователей из FAIR; сама по себе она не лучше GPT-3.5 ни в объективных тестах, ни с точки зрения пользовательского опыта, но, в отличие от OpenAI, учёные из FAIR выпустили её с открытым кодом и опубликовали веса обученной модели; обучить новую гигантскую языковую модель — это очень вычислительно затратный, дорогостоящий процесс, требующий миллионов долларов, даже если вы твёрдо знаете, что хотите сделать (что вряд ли; скорее всего, вам потребуется немало экспериментов), а вот использовать уже обученную модель можно и на домашнем десктопе; выход LLaMA уже привёл к огромному числу новых проектов от независимых исследователей;
- 14 марта стало, пожалуй, главным днём этой весны искусственного интеллекта: в один и тот же день Google объявил, что интегрирует языковые модели в свою экосистему (например, Google Docs), отделившийся от OpenAI

стартап Antropic выпустил свою языковую модель Claude, но оба этих события затмила OpenAI, объявившая о выпуске новой итерации своей базовой модели; GPT-4 [450] не просто ещё лучше научилась писать тексты, но и стала мультимодальной, то есть может теперь обрабатывать тексты и изображения одновременно;

- буквально на следующий день, 15 марта, вышла очередная серьёзная версия модели MidJourney, предназначенной для создания изображений по текстовому запросу; такие text-to-image модели тоже используют трансформеры, но это отдельная тема, так что про картинки мы пока (да и вообще в этой книге) говорить не будем;
- в апреле начали появляться проекты, использующие ChatGPT как основу для новых применений, через созданный OpenAI механизм плагинов; в частности, одним из самых интересных плагинов стал AutoGPT [641], эксперимент, в котором из языковой модели вполне успешно попытались сделать агента, самостоятельно достигающего своих целей;
- в мае вышел Google Bard, который оказался успешнее Sydney, а ещё раньше Google анонсировал поддержку больших языковых моделей в своей экосистеме, в том числе в Google Docs и Gmail;
- к концу мая этот первоначальный «большой взрыв языковых моделей» немного утих, но новые исследования продолжали (и всё ещё продолжают) появляться с огромной скоростью; книга никак не может угнаться за темпом этих новостей, так что остановлюсь здесь.

К чему привела эта «весна»? Прежде всего стало очевидным, что мы, человечество, если ещё не стоим на самом пороге, то уже вплотную подошли к созданию *сильного* искусственного интеллекта, интеллекта человеческого уровня[1].

Это выводит на первый план вопросы, связанные с потенциальной опасностью сильного искусственного интеллекта для людей: AI человеческого уровня почти наверняка тут же его и превзойдёт хотя бы за счёт способности обрабатывать больше информации и легко копировать себя, да и нет никакого закона природы, который бы устанавливал лимит интеллекта на человеческом уровне.

Иначе говоря, мы стоим на пороге создания сущностей «более умных», чем все остальные на Земле, — я ставлю «более умных» в кавычки, потому что опять же не умею это формализовать, но понимать «ум» здесь нужно не в смысле знания фактов или умения перемножать числа, а в смысле способности успешно достигать поставленных целей.

[1] Здесь много похожих друг на друга терминов: сильный AI (strong AI), AI человеческого уровня (human-level AI, HLAI), общий AI (artificial general intelligence, AGI), трансформативный AI (transformative AI, TAI), который не нужно путать с трансформерами — речь идёт о трансформации экономики и вообще нашего человеческого мира — и несколько других. Формализовать такие термины трудно, но сейчас и не обязательно — неточного, но обычно интуитивно понятного «человеческого уровня» нам будет вполне достаточно.

С одной стороны, с некоторой вероятностью сильный искусственный интеллект может решить для нас массу проблем и трансформировать человеческое общество в самую буквальную, неироничную утопию. С другой стороны, интеллектуальное превосходство вызывает и опасения: в последний раз самые умные существа на Земле появились, когда мы выделились из приматов, и судьба всех остальных приматов в наше время не кажется такой уж завидной...

Но безопасность сильного искусственного интеллекта — это отдельный большой разговор; я надеюсь, что когда-нибудь мы обсудим и это, но уж точно не в этой книге. Давайте попробуем понять, что говорит нам современное состояние искусственного интеллекта с инструментальной точки зрения. В дальнейшем будем предполагать, что занятия искусственным интеллектом всё ещё не вовсе бессмысленны для людей, то есть сам AI ещё не начал делать это лучше любого человека; это всё ещё верно и в тот момент, когда я пишу эти строки, и в момент финальной редактуры текста осенью 2024 года, но весна 2023 года заставляет такие предположения подчёркивать явно. Но если людям всё-таки ещё может пригодиться эта книга, то в предисловии нам с вами важно понять две вещи: во что наука об AI превращается в наше время и как её лучше изучать?

Горький урок или повод для оптимизма?

> Кваттроченто... показал кроме того — и этим он даёт нам урок — что человек, предоставленный своим собственным силам, отнятый от целого, опираясь только на самого себя и живя только для себя одного, не может совершить всего.
>
> *Филипп Монье. Опыт литературной истории Италии XV века: Кваттроченто*

В 2019 году Ричард Саттон[1], который с 2017 года разрабатывал лучшие в мире методы обучения с подкреплением в компании DeepMind, написал ставшее очень популярным небольшое эссе под названием «Горький урок» *(The Bitter Lesson)* [580]. В нём Саттон отмечает, что вся история искусственного интеллекта учит нас одному: сколько бы люди ни пытались придумать способы использовать свои знания о предметной области для того, чтобы улучшить модели искусственного интеллекта, все их попытки становятся совершенно бесполезными, как только в их распоряжении оказываются увеличившиеся на один-два порядка вычислительные ресурсы.

[1] *Ричард Саттон* (Richard S. Sutton, p. 1949) — американский специалист по искусственному интеллекту. Саттон считается одним из основоположников современного обучения с подкреплением — например, в своей диссертации он ввёл и проблему распределения вознаграждения (credit assignment), и методы типа actor-critic. Написанный им в соавторстве с его научным руководителем Эндрю Барто учебник *Reinforcement Learning: an Introduction* представляет собой уникальный случай в нашей области: первое его издание вышло в 1998 году [581] и 20 лет оставалось главным базовым источником по обучению с подкреплением, до 2018 года, когда вышло второе издание той же книги [582].

Так, например, люди несколько десятилетий пытались написать программу для игры в шахматы, используя знания людей о структуре игры, а потом оказалось, что Гарри Каспарова можно победить, по сути, обычным поиском с отсечениями, нужно лишь выделить на это достаточно много вычислительных ресурсов (мы немного поговорим об этом в разделе 1.2). Для игры в го поиск с отсечениями оказывается слишком сложным, но и там победа пришла не через использование человеческих знаний об игре, а через обучение нейросети с нуля: AlphaGo [380] ещё обучалась на партиях, сыгранных людьми, но уже следующая модель DeepMind для этой задачи, AlphaZero [214], использовала только правила игры, обучаясь через партии, сыгранные с самой собой. Десятилетия, потраченные исследователями в области распознавания речи на разработку кратких и информативных признаков для человеческой речи, например MFCC[1], оказались не нужны, как только признаки стало возможным обучать автоматически: ещё до разработки новых идей о самом распознавании акустическое моделирование глубокими нейросетями уже привело к существенным улучшениям [135], а сейчас, конечно, и само распознавание делается нейросетевыми архитектурами, причём это не специально разработанные для речи методы, а всё те же немного адаптированные трансформеры [111, 500, 553]. Аналогичные примеры есть в любой области: например, в распознавании изображений свёрточные сети вытеснили разработанные людьми SIFT-признаки[2], как только стало возможным обучить достаточно большие свёрточные архитектуры.

Получается, что заниматься искусственным интеллектом бессмысленно: если просто подождать несколько лет, любая задача решится сама собой? Конечно, Саттон не утверждает, что новые идеи совсем не нужны и что весь прогресс всегда сводится к «тупому» масштабированию числа параметров. Например, ChatGPT, о котором мы говорили в предыдущем разделе, был основан на модели GPT-3, которая вышла ещё в 2020 году. В ChatGPT не стало существенно больше параметров, они просто начали использоваться несколько иным образом, и этого оказалось достаточно для того, чтобы академический успех превратился в весьма практическую революцию. Однако основной аргумент Саттона действительно выглядит разумно: искусственные нейронные сети «не работали», пока не было

[1] Мел-кепстральные коэффициенты (mel-frequency cepstral coefficients, MFCC) [262] — это стандартный (до революции глубокого обучения) тип признаков, которые извлекали из человеческой речи для дальнейшего использования в задачах распознавания речи и аналогичных; идея MFCC в том, чтобы сначала взять преобразование Фурье от сигнала, потом преобразовать результат к шкале мелов, а потом ещё раз взять косинусное преобразование, получив «спектр спектра», который называют «кепстром» (cepstrum — это слово spectrum с перевёрнутой первой частью)... В общем, цифровая обработка сигналов — сложная и большая наука, которой мы в этой книге заниматься не будем [362, 451, 478].

[2] Масштабно-инвариантная трансформация признаков (scale-invariant feature transform, SIFT) — алгоритм, в котором из изображения выделяются особые точки (keypoints), предположительно соответствующие разным объектам [354, 355]; до революции глубокого обучения это был один из лучших способов выделения признаков из изображения. Но в компьютерное зрение мы в этой книге тоже углубляться не будем.

достаточно больших наборов данных и вычислительных мощностей для обучения на них, а потом вдруг «заработали» так, что многие кажущиеся более сложными вероятностные модели стали попросту не нужны.

Однако мне представляется, что такой взгляд чрезмерно пессимистичен: конечно, развитие науки не представляет собой прямую линию прогресса, на которую нанизаны аккуратно вытекающие друг из друга новые результаты. Некоторые направления оказываются принципиально тупиковыми: например, история медицины усеяна обломками теорий о гуморах[1], об elan vital[2] и многом другом, что сейчас уже невозможно принимать всерьёз (впрочем, см. подраздел 2.6.3). Некоторые оказываются тупиковыми не в принципе, а на практике: так, весь искусственный интеллект превратился в машинное обучение[3] не потому, что написать мыслящую программу без обучения принципиально невозможно — вы можете легко сесть и запрограммировать любую нейросеть, задав ей вручную любые значения весов — а потому, что у нас нет возможности придумать эти параметры «из головы» или из каких-то принципиальных соображений, приходится

[1] Гуморальная теория медицины предполагала, что функционирование человеческого тела обеспечивается течением четырёх разных жидкостей: крови, флегмы (слизи), жёлтой жёлчи и чёрной жёлчи. Обычно они в балансе, а разные нарушения баланса приводят к болезням. Теория сложилась ещё в Древней Греции, под влиянием пифагорейцев, и четыре жидкости были связаны с четырьмя стихиями и четырьмя темпераментами; затем гуморальную теорию развил великий римский врач Гален, и так она окончательно стала главенствующей [113, 327]. Полностью дискредитирован гуморизм был только в середине XIX века, когда развилась микробная теория болезней. Одна из главных тем этой книги — байесовский вывод и в целом правильный учёт полученных данных в том, чтобы изменять наши представления о мире. Гуморальная теория — очень интересный пример противоположного феномена: она была общепринятой две тысячи лет без всяких потверждающих данных, причём речь идёт не о загробном существовании или описаниях далёких стран, о которых данных нет никаких, а о повседневной работе тысяч врачей, которые лечили больных и постоянно наблюдали успехи и неудачи своих методов! Одного объяснения у этого феномена нет, это комбинация традиции почтения к авторитетам, холистического взгляда на мир (гуморальная теория — часть «теории всего», основанной на стихиях-элементах), некоторых действительно работающих вещей (например, гуморальная теория рекомендовала вполне разумные диеты и упражнения), эффекта плацебо (небольшое кровопускание не будет вредным для здоровья, если не занести инфекцию, но вызовет эффект плацебо по полной) и так далее; но пример интересный.

[2] Здесь я имею в виду теории, постулирующие существование некоей «жизненной силы»; в Древней Греции об этом писали Посидоний и Зенон Элейский, в восточной традиции корень «аюс» в аюрведе — это как раз «жизненный принцип», «конатус» у Декарта и Спинозы и та самая «воля» у Шопенгауэра — это тоже не просто когнитивные феномены, а некие «врождённые склонности» живой и даже неживой вещи к продолжению существования. А сам термин elan vital (жизненная сила) ввёл уже в XX веке Анри Бергсон в книге «Творческая эволюция» [672], в которой постулирует эту «энергию» как то, что позволяет переходить от простых форм организации материи к сложным.

[3] Терминологическое замечание, к которому мы не будем возвращаться далее: вообще говоря, машинное обучение — это раздел искусственного интеллекта. Существуют и другие разделы: экспертные системы, онтологии, логическое программирование и другие. Однако в последние десятилетия стало ясно, что хотя все эти другие разделы отнюдь не бесполезны, всё-таки главным содержанием искусственного интеллекта является именно обучение на опыте. Поэтому в этой книге мы говорим именно о машинном обучении, а не об искусственном интеллекте в целом, и в наше время эти термины уже почти синонимичны вне исторического контекста.

обучать их из данных. Но без многих результатов, полученных на этих тупиковых путях развития науки, идущий по магистральному пути паровоз «научного мейнстрима» тоже давно забуксовал бы.

Приведу конкретный пример из нашей области: в обработке естественных языков есть важный раздел, которому я и сам в своё время посвятил немало исследовательских проектов — *тематическое моделирование* (topic modeling) [67, 68, 436, 438, 489, 607]. Тематические модели могут обрабатывать большие наборы (корпусы) документов, выделяя из них темы в виде распределений на словах, что даёт возможность для интерпретации: вы можете записать самые вероятные слова в той или иной теме, прочесть их и понять, о чём «говорит» эта тема. В результате тематическая модель позволяет в краткой и понятной форме записать, о чём «говорит» и весь корпус, и каждый документ в отдельности; документы же в таких моделях представляются в виде распределения на темах.

Как ни печально мне будет видеть закат близких моему сердцу тематических моделей, я совершенно не уверен, что они останутся актуальными в ближайшие годы. Зачем прибегать к заведомо сильно упрощающим предположениям (классические тематические модели рассматривают документы как «мешки слов», то есть предполагают, что слова в теме появляются независимо друг от друга), зачем строить выводы, исходя из частотности слов в документах, если можно просто попросить языковую модель «прочитать» весь корпус документов и кратко его подытожить? Прямо сейчас такой подход, может быть, ещё не слишком эффективно бы работал, но здесь не верить в прогресс было бы странно.

Однако если мы с вами дойдём до соответствующих разделов машинного обучения (не в этой книге, но, будем надеяться, в одной из последующих), мы обязательно поговорим о тематических моделях! Дело в том, что основная тематическая модель *латентного размещения Дирихле* (latent Dirichlet allocation, LDA) представляет собой прекрасный пример сложной, но не слишком сложной вероятностной модели, для которой работают два основных приближённых метода вероятностного вывода: вариационные приближения к апостериорному распределению и сэмплирование по Гиббсу для его же аппроксимации [67, 173, 547]. Чтобы понять, как работают современные модели машинного обучения и что они делают «под капотом», требуется понимать и концепцию вариационных нижних оценок, и то, как вести вероятностный вывод через сэмплирование, а для этого тематическое моделирование остаётся идеально подходящим примером.

Более того, даже если сама модель LDA будет уже не нужна на практике, вариационные приближения, которые она призвана иллюстрировать, полезны пуще прежнего: современные порождающие модели для изображений основаны на вариационных автокодировщиках (variational autoencoders, VAE) [146, 307], и, например, DALL-E, одна из основных вех на пути развития text-to-image моделей, оптимизирует, по сути, гигантскую вариационную нижнюю оценку [659].

Вариационные автокодировщики являют собой хороший пример и другого эффекта: когда модель VAE появилась в 2013 году, она, конечно, не прошла неза-

меченной, но до конца 2010-х годов мало использовалась в реальной практике. Казалось, что это интересная и красивая идея, которая, однако, на практике не приводит к достаточно хорошим результатам, — таких идей всегда было много в машинном обучении. А лучшие модели для порождения изображений во второй половине десятых годов были основаны на состязательных порождающих сетях (generative adversarial networks, GAN) [215, 225, 479].

Но когда вариационные автокодировщики научились работать с дискретным латентным пространством [612] — то есть когда к ним добавили ещё одну математическую идею, — внезапно оказалось, что они более чем способны закодировать всё, что нужно. Тогда на первый план вышли их преимущества перед состязательными сетями, особенно более простое обучение, для которого не требуется настраивать состязательное обучение нескольких сетей, пытающихся обманывать друг друга.

Так что я не вполне согласен с Ричардом Саттоном: мне представляется, что современные модели, которые действительно могут показаться простым масштабированием давно известных подходов, были бы невозможны без тех идей, которые разрабатывались в машинном обучении в последние десятилетия. И чрезвычайно быстрый темп развития современного машинного обучения только укрепляет меня в твёрдом убеждении, которое было со мной на протяжении всей моей карьеры в AI: конкретные модели приходят и уходят, самое главное — понимать математические основы происходящего. Если вы понимаете, как устроен байесовский вывод и что делают модели машинного обучения на уровне пересчёта тех или иных распределений вероятностей, вам будет легко адаптироваться к любым изменениям, легко прочесть любую статью, даже выдвигающую совершенно новые идеи... если, конечно, сильный искусственный интеллект не заменит нас всех в ближайшем будущем, но от этого я вас вряд ли смогу застраховать.

И пусть в этой книге мы ещё не доберёмся ни до трансформеров, ни до больших языковых моделей, основы, на которых стоит современное машинное обучение, не имеют к ним прямого отношения. Эти основы, по сути, сводятся к творческому применению теоремы Байеса, то есть к математической формализации того, как новые данные влияют на наши представления о свойствах модели, её параметрах. Об этом мы в основном и будем говорить.

План книги: о чём пойдёт речь далее

> Колхоз непоколебимо спал на Оргдворе. В Оргдоме горел огонь безопасности — одна лампа на всю потухшую деревню; у лампы сидел активист за умственным трудом, он чертил графы ведомости, куда хотел занести все данные бедняцко-середняцкого благоустройства, чтоб уже была вечная, формальная картина и опыт как основа.
>
> *Андрей Платонов. Котлован*

Я
планов наших
люблю громадьё,
размаха
шаги саженьи.
Я радуюсь
маршу,
которым идём
в работу
и в сраженья.

В. В. Маяковский. Хорошо

Основная задача книги — дать введение в машинное обучение с вероятностной точки зрения. Мы начнём строить здание современного искусственного интеллекта прямо с фундамента, с теоремы Байеса и формулировок основных задач байесовского вывода, а затем постараемся рассмотреть самые простые модели машинного обучения — линейную и логистическую регрессию — с максимального числа разных углов, углубляясь во все возможные детали. Конкретный план в результате получается такой.

Глава 1 — вводная. Она начинается с краткого экскурса в историю искусственного интеллекта и как идеи, и как раздела компьютерных наук. А затем даёт несколько классификаций того, что происходит в искусственном интеллекте, с разных точек зрения: сначала классификацию постановок задач (обучение с учителем и без, регрессия и классификация), потом разных видов данных (табличные данные, последовательности токенов, изображения, видео и так далее), а потом применений машинного обучения. Поскольку дать обзор всех применений искусственного интеллекта в наше время уже вряд ли возможно, в последнем случае я сконцентрировался на том, как машинное обучение помогает другим наукам получать новые результаты.

В *главе 2* мы начинаем закладывать тот самый фундамент: сначала вспоминаем основы теории вероятностей (да, увы, без этого никак не обойтись), потом рассматриваем теорему Байеса как основу вероятностного вывода в искусственном интеллекте, а затем подробнейшим образом анализируем самую простую модель машинного обучения — обучение параметра одной нечестной монетки. Это позволит содержательно поговорить обо всех задачах байесовского вывода на простом примере, на котором все формулы легко вывести аналитически, а интуиция непосредственно ясна. Заканчивается глава несколькими развёрнутыми примерами того, где вероятностный вывод может пойти не так; в частности, мы обсудим кризис воспроизводимости, в последнее десятилетие затронувший многие разделы науки, и то, какие уроки мы можем из него извлечь.

Глава 3 продолжает разговор о тех разделах теории вероятностей, которые нужны в машинном обучении. После того как мы обсудим самые важные дискретные и непрерывные распределения, встречающиеся и в машинном обучении,

мы в этой главе рассмотрим пример байесовского анализа равномерного распределения, что позволит глубже понять априорные распределения. А затем перейдём к теоретико-вероятностным и теоретико-информационным понятиям, играющим важную роль в искусственном интеллекте: энтропии, KL-дивергенции и информации Фишера. Завершается глава 3 рассмотрением основных методов оптимизации, которые в машинном обучении тоже постоянно нужны.

Глава 4 полностью посвящена относительно простой, но уже совсем полноценной модели машинного обучения: линейной регрессии. Мы начнём с метода наименьших квадратов, обсудим оверфиттинг и регуляризацию «на пальцах», а затем перейдём к вероятностному анализу линейной регрессии, решив для неё все основные задачи байесовского вывода. Глава заканчивается подробно рассмотренным примером, который, надеюсь, убедит вас в пользе вероятностного рассмотрения и, в частности, в пользе получающихся из байесовского вывода предсказательных распределений.

В *главе 5* мы от задач регрессии переходим к задачам классификации. Здесь сначала стоит поговорить о геометрии задач классификации, о том, как выглядят разделяющие поверхности и как можно строить классификаторы, исходя из сугубо геометрических соображений. Затем обсудим оптимальный байесовский классификатор, которому требуются порождающие модели для классов, и перейдём к основному линейному методу классификации — логистической регрессии. Для неё мы тоже решим все основные задачи байесовского вывода, причём предсказательное распределение приведёт нас к аппарату лапласовских аппроксимаций. Затем я приведу развёрнутый пример датасета ирисов Фишера, мы обсудим несколько важных общих замечаний — перевзвешивание цен ошибок, калибровку классификаторов, априорные вероятности классов, — а закончим главу рассмотрением наивного байесовского классификатора.

Наконец, *глава 6* вводит несколько важных сюжетов, которые пронизывают всё машинное обучение. Проклятие размерности объясняет, почему вероятностные задачи, возникающие в машинном обучении, действительно сложны, а методы, прекрасно работающие в низких размерностях (например, метод ближайших соседей), могут не обобщаться на высокие. Статистическая теория принятия решений приведёт нас к разложению на смещение, дисперсию и шум, которое объясняет, что нужно сбалансировать при выборе хорошей модели машинного обучения. Ядерные методы, которые мы в этой книге затронем лишь очень кратко, дадут новый взгляд на линейную регрессию и другие модели. Полноценный байесовский вывод для обоих параметров одномерного гауссиана покажет, как при байесовском выводе могут появляться совсем новые, неожиданные распределения. Оценки функции свидетельства дадут эмпирический метод Байеса для подбора гиперпараметров, а также информационные критерии для выбора моделей. Наконец, последние разделы будут посвящены обобщениям линейных моделей: сначала мы вложим почти все представляющие интерес для машинного обучения распределения вероятностей в экспоненциальное семейство, а затем

рассмотрим обобщённые линейные модели, в которые вкладываются и линейная, и логистическая регрессия.

Конечно, эта книга могла бы выглядеть как обычный учебник по азам теории вероятностей и математической статистики, «развёрнутым» в сторону машинного обучения. Но, признаться честно, мне кажется, что такую книгу и читать, и писать было бы скучновато. Поэтому, хотя подробное обсуждение многих стандартных тем всё-таки неизбежно, я постарался разбавлять это классическое изложение разными интересными сюжетами: в последующих главах я буду пытаться по ходу изложения добавлять в стандартную и для многих читателей наверняка довольно скучную информацию что-нибудь интересненькое. Например, в разделе 3.1, в котором вводятся основные дискретные распределения, мы также обсудим закон Ципфа и его критику и значение, а в разделе 6.2 о статистической теории принятия решений поговорим о парадоксальной на первый взгляд оценке Джеймса — Штейна. Будут и целые разделы, посвящённые таким «дополнительным» темам, например раздел 2.5 об анализе эффекта «горячей руки» или раздел 3.3 о том, как обучить равномерное распределение. Сюжеты эти будут уводить нас немного в сторону от генеральной линии изложения, и они обычно будут несколько сложнее основного текста. Поэтому, во-первых, в тексте книги всегда будет отмечено, когда начинается такой раздел (звёздочками и курсивом в названиях и в оглавлении), а во-вторых, при первом или ознакомительном чтении книги их можно смело пропускать, для понимания дальнейшего они требоваться не будут.

История книги и благодарности

> Есть причины полагать, что выражение благодарности может быть связано — возможно, даже причинно-следственным образом — со счастьем и благополучием... И классические, и популярные источники о счастье согласны, что быть осознанно благодарным за подарки судьбы — это эффективный способ максимизировать удовольствие от жизни. Нам кажется, что приведённые здесь три экспериментальных исследования дают научные подтверждения этой расхожей мудрости.
>
> *R. A. Emmons, M. E. McCullogh. Counting Blessings Versus Burdens: An Experimental Investigation of Gratitude and Subjective Well-Being in Daily Life [168]*

Эта книга выросла из курса лекций по машинному обучению, который я читаю уже много лет.

Первая версия курса появилась ещё в 2007 году, в рамках Computer Science Club при ПОМИ РАН[1]. Здесь большое спасибо создателю и многолетнему руководителю клуба Александру Куликову, но главное спасибо — первому спонсору клуба Антону Лиходедову. Именно он в середине нулевых как-то раз сказал

[1] Санкт-Петербургское отделение Математического института им. В. А. Стеклова РАН.

нам, тогда сугубо теоретическим информатикам из лаборатории математической логики ПОМИ: «Machine learning is hot; нам нужен курс по машинному обучению». Я тогда совершенно ничего не знал о машинном обучении, занимался только немного вероятностными моделями, а в целом изучал в основном высшую алгебру, но курс составить всё-таки вызвался — и с тех пор машинным обучением и занимаюсь.

Конечно, мой курс в 2007 году был очень наивным, и те курсы, которые я читаю сейчас, имеют с ним очень мало общего. Поэтому спасибо и всем тем многочисленным организациям — и образовательным, и исследовательским, — в которых я преподавал с тех пор и до настоящего времени: Высшей школе экономики, Университету ИТМО, Академии больших данных Mail.Ru, Центру речевых технологий и многим другим. Главное спасибо здесь факультету математики и компьютерных наук Санкт-Петербургского государственного университета: как только ФМКН «отпочковался» от матмеха, руководители программы пригласили меня преподавать машинное обучение лучшим математикам-бакалаврам России и одним из лучших в мире. Не знаю, кого именно мне здесь лучше поблагодарить лично, так что назову основных руководителей и вдохновителей создания ФМКН: Станислава Смирнова, Сергея Владимировича Иванова, Николая Александровича Вавилова и Эдуарда Алексеевича Гирша.

Николай Александрович[1] был моим первым научным руководителем в университете (СПбГУ, 2000–2005), а Эдуард Алексеевич руководил моей кандидатской диссертацией (ПОМИ РАН, 2009), так что им благодарность особая и отдельная: они научили меня тому, как быть исследователем, и без Николая Александровича и Эдуарда Алексеевича не было бы у меня никакой научной карьеры. Отдельное спасибо также Александру Львовичу Тулупьеву, который с юных лет научил меня не бояться больших форм [686, 689, 690].

И наконец, самые свежие и самые конкретные благодарности хочется высказать в адрес моих коллег, которые читали ранние версии этой книги, высказывали замечания и помогли сделать её лучше. Это (в алфавитном порядке) Антон Алексеев, Ирина Пионтковская, Гарри Прошян, Маргарита Спиричева, Александр Тарасов, Алексей Шабанов и Михаил Широких. Самое большое здесь спасибо Максиму Николаеву, который выдвинул массу замечаний и предложений,

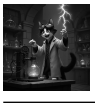
[1] *Николай Александрович Вавилов* (1952–2023) — российский математик, алгебраист, основатель научной школы, изучающей группы Шевалле над коммутативными кольцами. Вавилов был автором более 200 научных работ, в том числе открывающих целые новые области исследований в высшей алгебре [42, 559, 617, 618], но главное, наверное, не в этом, а в том, каким он был человеком: знал языков пятнадцать, включая китайский и фарси, обладал невероятной эрудицией как в математике, так и в большинстве других областей знания, был человеком высочайшей культуры. Эрудиция и культура Николая Александровича видны в его книгах; для непрофессиональных алгебраистов могу порекомендовать имеющиеся в открытом доступе учебники: «Не совсем наивная теория множеств» [675], «Не совсем наивная линейная алгебра» [674] и «Конкретная теория групп» [673]. К сожалению, эти учебники, как и множество других работ и планов, остались не вполне законченными; воспользуюсь этим случаем, чтобы посвятить книгу Николаю Александровичу, который в 2023 году скоропостижно ушёл из жизни.

приведших к целым новым параграфам в книге. Ну а котиков я просто люблю; все рисунки с котиками, которые иллюстрируют разделы в книге, сделаны при помощи современных версий модели Stable Diffusion [259], в частности Flux, и модели DALL-E 3, доступной от компании OpenAI [274]. И спасибо Майлзу, который был если не моделью, то вдохновением для чёрно-белых персонажей этой книги.

Сергей Николенко,
март и октябрь 2024 года

Глава 1
Что такое машинное обучение

TL;DR

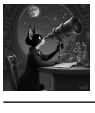


Раздел 1.1 открывает книгу кратким обзором истории идеи искусственного интеллекта, от робота-андроида Талоса через средневековую магию к поражающим воображение автоматонам Жаке-Дро; затронем и математическую сторону вопроса в виде развития логики.

В разделе 1.2 мы перейдём от идеи к собственно науке об искусственном интеллекте: от её начала в работах Тьюринга и на Дартмутском семинаре, через несколько последовательных волн хайпа, которые сменялись «зимами искусственного интеллекта», до революции глубокого обучения.

С раздела 1.3 история уступает место введению в сам искусственный интеллект. Здесь мы поговорим о разных постановках задач, а также подробно обсудим разницу между обучением с учителем и обучением без учителя.

Раздел 1.4 продолжает классификацию областей искусственного интеллекта, на этот раз с точки зрения данных. Мы обсудим извлечение признаков, а затем пройдёмся по основным типам данных: табличным данным, последовательностям, изображениям и так далее.



В разделе 1.5 мы снова классифицируем разделы искусственного интеллекта, теперь по его возможным применениям. Я решил сконцентрироваться на том, как искусственный интеллект помогает другим наукам: математике, физике и астрономии, химии и биологии, наукам о Земле и социальным наукам.

1.1. Искусственный интеллект от Адама до Франкенштейна

Первый профессор... сказал, что меня, быть может, удивит его работа над проектом усовершенствования умозрительного знания при помощи технических и механических операций... Затем он подвёл меня к раме, по бокам которой рядами стояли все его ученики... Поверхность её состояла из множества деревянных дощечек... Все они были сцеплены между собой тонкими проволоками. Со всех сторон каждой дощечки приклеено было по кусочку бумаги, и на этих бумажках были написаны все слова их языка в различных наклонениях, временах и падежах, но без всякого порядка... По его команде каждый ученик взялся за железную рукоятку, которые в числе сорока были вставлены по краям рамы, и быстро повернул её, после чего расположение слов совершенно изменилось. Тогда профессор приказал тридцати шести ученикам медленно читать образовавшиеся строки в том порядке, в каком они разместились в раме; если случалось, что три или четыре слова составляли часть фразы, её диктовали... Ученики занимались этими упражнениями по шести часов в день.

Джонатан Свифт. Путешествия в некоторые отдалённые страны света Лемюэля Гулливера, сначала хирурга, а потом капитана нескольких кораблей (пер. А. Франковского)

1.1.1. Ранняя история

Увидеть, как идея искусственного интеллекта развивалась в истории человеческой мысли, не так-то просто. Дело в том, что собственно искусственный интеллект как наука в её современном понимании сложился совсем недавно — обычно этим моментом считается создание теста Тьюринга, о котором мы поговорим ниже. До этого, конечно, были и другие рассуждения о мыслящих машинах, и попытки решать задачи, которые мы могли бы сейчас назвать задачами искусственного интеллекта, но грань между AI и другими науками провести сложно. Тем не менее давайте попробуем кратко пройтись по истории того, как люди мыслили себе искусственный разум (рекомендую также уже переведённую на русский язык книгу Клиффорда Пиковера [469, 688]).

Начнём с самого начала, прямо с книги Бытия. Привычные нам авраамические религии учат, что Бог создал человека по своему образу и подобию из глины, смешанной с разнообразными жидкостями; по всей видимости, эта идея восходит к вавилоно-аккадскому эпосу, в котором бог Мардук сотворил человека из глины, смешанной со слюной и кровью. Получается, все мы — пример искусственного интеллекта. И не так уж важно, создал ли ветхозаветный Бог людей сразу готовыми из глины, лично и самостоятельно разработав архитектуру нейронных сетей, или вольтеровский часовщик[1] создал и «завёл» механизм, а люди получились из него сами собой, — мы ведь сегодня не называем результат генетических алгоритмов или автоматического поиска нейросетевых архитектур[2] «естественным интеллектом».

В древнейшей авраамической традиции, иудейской, есть и другой яркий пример искусственного интеллекта. Как известно хотя бы из романа Густава Майринка, особо мудрые раввины могли создавать и оживлять *големов*, искусственных людей; в самой известной такой легенде творцом искусственного интеллекта выступает главный раввин Праги Йехуда бен Бецалель. Големы делались, что неудивительно, тоже из глины, и даже само слово «голем» означает «глина» или любой «необработанный материал».

Любопытно, что иудейская традиция различает задачи *распознавания речи* и её *синтеза*: големы не слишком умны, но могут слушать и понимать команды, хотя понимают их порой слишком буквально (примерно как в некоторых более современных историях о роботах). А вот отвечать и вообще говорить на человеческом языке — не могут, для этого, видимо, нужна душа, которую даже самый мудрый раввин вложить не может.

Греческие боги были не слишком склонны к научным занятиям, и разработку методов искусственного интеллекта в Древней Греции можно было ожидать

---

[1] *Франсуа-Мари Аруэ*, или *Вольтер* (François Marie Arouet, Voltaire, 1694–1778) — французский писатель и философ, один из главных деятелей Просвещения. В чисто художественной прозе Вольтер двигался от классицизма — он начинал как автор классической трагедии («Эдип», «Брут») и классической эпопеи («Генриада», «Поэма о битве при Фонтенуа») — к романтизму и сентиментализму («Танкред»), а порой и просто к сатире и фарсу («Орлеанская девственница»). Ещё более известна его философская проза: романы «Простодушный», «Кандид», «Задиг» и другие внесли важный вклад в Просвещение, популяризируя философские рассуждения и развивая новые интересные аргументы. Вольтера всегда хорошо знали в России: Екатерина II сначала вела с ним переписку, а после смерти философа выкупила его огромную библиотеку: 6814 книг и 37 томов с рукописями привезли в Петербург на специальном корабле; а просветительское движение, направленное против традиционной религии и феодализма, на защиту прав человека и равенства перед законом, в России во второй половине XVIII века стали буквально называть «вольтерьянством».

[2] Neural architecture search (NAS) — один из важных современных методов разработки архитектур глубоких нейронных сетей, в котором архитектуры получаются «сами собой» посредством направленного поиска либо генетическим алгоритмом, либо обучением с подкреплением. При помощи NAS уже было разработано множество архитектур, которые применяются сплошь и рядом, — функции активации Swish, семейства свёрточных сетей EfficientNet и EfficientDet и так далее — и не так уж ясно, чем наша роль в их создании принципиально отличается от роли того самого вольтеровского часовщика...

(а) (б)

Рис. 1.1. Средневековая магия: (а) папа Сильвестр II договаривается с дьяволом; (б) говорящая голова Роджера Бэкона пугает его слугу

разве что от великого кузнеца и изобретателя Гефеста. Гефест не подвёл — он создал гигантского человекоподобного робота по имени Талос ($Ταλως$)[1]. У Талоса была одна вена, по которой тёк божественный ихор; вена была заткнута гвоздём, который находился в пятке, по греческой традиции единственном уязвимом месте Талоса. Зевс подарил Талоса Европе для охраны острова Крит; андроид обегал остров по периметру трижды в день и бросал гигантские камни во вражеские корабли.

Здесь для нашего современного взгляда любопытно, что метко бросать гигантские камни человечество научилось довольно скоро: собственно, те же древние греки ещё и до Архимеда умели создавать вполне рабочие катапульты; а вот отличать вражеские корабли от своих, да и вообще различать корабли на фоне воды и понимать, где они находятся, — это непростая задача компьютерного зрения, решение которой до сих пор, наверное, можно улучшать.

Средневековые алхимики часто заключали договоры с дьяволом (рис. 1.1), по которым получали свои сверхъестественные способности; иногда эти способности были весьма похожи на искусственный интеллект.

[1] В наше время в популярной культуре Талос появляется реже, чем другие персонажи греческой мифологии, но забавно, что это обычно связано с компьютерными играми. Игра *Talos Principle* — интересная головоломка с философским сюжетом, в которой вы играете за искусственный интеллект... Пожалуй, не буду спойлерить дальше. А армия США с 2013 до 2019 года безуспешно разрабатывала *Tactical Assault Light Operation Suit* (TALOS), роботизированный экзоскелет, который должен был защищать носителя при помощи «жидкой керамической брони» из наночастиц. Как тут не вспомнить бессмертное «Nanomachines, son!» от Хидео Кодзимы (кстати, *Metal Gear Rising: Revengeance* вышла в том же 2013 году).

Так, через всё Средневековье проходит бродячий сюжет о говорящей голове, обычно бронзовой, которая могла самостоятельно отвечать на вопросы; видимо, магическая часть этого сюжета опиралась как раз на иудейскую традицию. В «Истории английских королей» Уильяма Мальмсберийского, опубликованной в 1125 году, говорящая голова упоминается в описании чудес папы Сильвестра II[1], который привёз из аль-Андалуса книгу с секретными знаниями арабов и смог избежать гнева владельца книги лишь с демонической помощью (см. рис. 1.1, *а*). Говорят, что известный алхимик Альберт Великий изготовил говорящую голову, чем немало расстроил своего учителя, Фому Аквинского, который в приступе гнева сломал дьявольскую машину; по одной из версий, эта говорящая голова скрывала в себе длинную трубку, посредством которой сам Альберт мог вещать из соседней комнаты. Подобные истории ходили о Боэции, Арнольде из Виллановы, Роберте Гроссетесте и многих других, но самой известной стала голова, созданная Роджером Бэконом[2] (см. рис. 1.1, *б*).

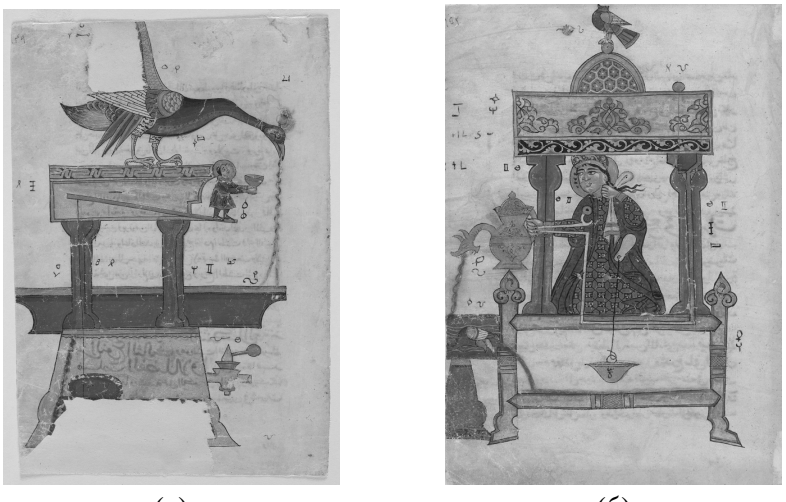
1.1.2. Механические автоматы Средних веков и Нового времени

В Средние века мы уже можем и перейти от мифов и легенд к настоящим созданиям человеческого гения. Конечно, об искусственном интеллекте в его современном понимании говорить было рано, но разные механические и гидравлические приспособления становились всё более изощрёнными и изобретатели часто придавали своим механизмам человекоподобный вид. Уже в XII веке арабский изобретатель аль-Джазари[3], которого иногда называют «отцом роботики»,

[1] *Сильвестр II* (Silvester PP. II, ок. 946–1003) — средневековый учёный и церковный деятель, папа римский с 999 по 1003 год, известный до избрания папой как Герберт Орильякский (Gerbertus Aureliacus, Gerbert d'Aurillac). Герберт долго жил и работал в Барселоне, где благодаря связям с Кордовой получил доступ к знаниям арабского мира; так, Герберт одним из первых европейцев познакомился с арабскими цифрами, осознал их удобство и стал пропагандировать их использование в Европе. Он популяризировал и другие достижения арабской науки, вернув в Европу абак, армиллярную сферу (модель небесной сферы, использовавшуюся для определения координат светил) и астролябию, которые не просто скопировал, а усовершенствовал. Разумеется, Герберту приписывали изучение магии и астрологии в Кордовском халифате, общение с дьяволом и связь с суккубом по имени Меридиана, а бронзовая голова, умевшая отвечать на вопросы «да» или «нет», якобы помогла ему подняться к папскому трону.

[2] *Роджер Бэкон* (Roger Bacon, ок. 1214–1292) — английский философ, теолог и естествоиспытатель, монах-францисканец и профессор богословия в Оксфорде. Его главная работа, *Opus majus* (1268), содержит новые результаты в математике, оптике, алхимии и астрономии, предлагает реформу календаря, которую через 300 лет проведёт папа Григорий XIII, а также первой в Европе описывает порох. История с говорящей головой — это взгляд на Бэкона из раннего Нового времени, когда его считали фаустианской фигурой, магом, обладающим тайными знаниями. Однако на самом деле Роджер Бэкон был провозвестником современной науки: за 350 лет до своего однофамильца Фрэнсиса он уже критиковал схоластику и закладывал основы эмпиризма, не боясь противоречить даже Аристотелю, а за 250 лет до Мартина Лютера критиковал распущенность духовенства, из-за чего 12 лет просидел в тюрьме.

[3] *Абу аль-Из ибн Исмаил ибн аль-Раззаз аль-Джазари* (1136–1206) — механик, математик, изобретатель и астроном из государства династии Артукидов. Он изобрёл коленчатый вал, конструировал клапанные насосы, машины для подъёма воды, фонтаны и многое другое. В 1206 году аль-Джазари составил трактат под названием «Книга знаний об остроумных механических устройствах»,



(а) (б)

Рис. 1.2. Автоматы аль-Джазари: (а) фонтан в виде павлина; (б) слуга, подающий воду для мытья рук

создал фонтан в виде павлина с автоматизированными человекоподобными слугами и робота-слугу, который подавал воду для мытья рук механизмом, подобным устройству современных унитазов (рис. 1.2). А самым известным его изобретением был музыкальный автомат в виде четырёх роботов-музыкантов, который считается одним из первых программируемых роботов: вероятно, музыканты могли играть в разных ритмах в зависимости от положения рычагов автомата.

Наука в Европе была тогда не так развита, но люди не оставляли мысли об искусственном интеллекте: так, в «Книге о Ланселоте Озёрном», созданной около 1230 года, Ланселот сражается с медными рыцарями-роботами. И вскоре европейцы тоже стали создавать механические автоматы. Например, в 1292 году Роберт II д'Артуа привёз в свой замок Эден (Hesdin) итальянского рыцаря Рено Куанье, которому поручил создать сад с механическими диковинками.

Куанье справился; местные мастера под его руководством создали механических птиц, обезьян, говорящую механическую сову, автоматический орган и массу увеселительных автоматов: статуи, поливающие гостей водой, мост, который разрушался, когда по нему шли люди, и тому подобное [603]. Были в эденском саду и просто шутки, отвечавшие, видимо, чувству юмора Роберта II; например, там было зеркало с надписью, приглашающей посмотреть, как вы будете выглядеть, если обсыпать вас мукой... и оно не обманывало!

где описал около 50 новых механизмов, в том числе конструкцию своих роботов. Исследователи считают, что музыкальный автомат аль-Джазари, представлявший собой фигуры четырёх музыкантов, мог содержать программируемую драм-машину.

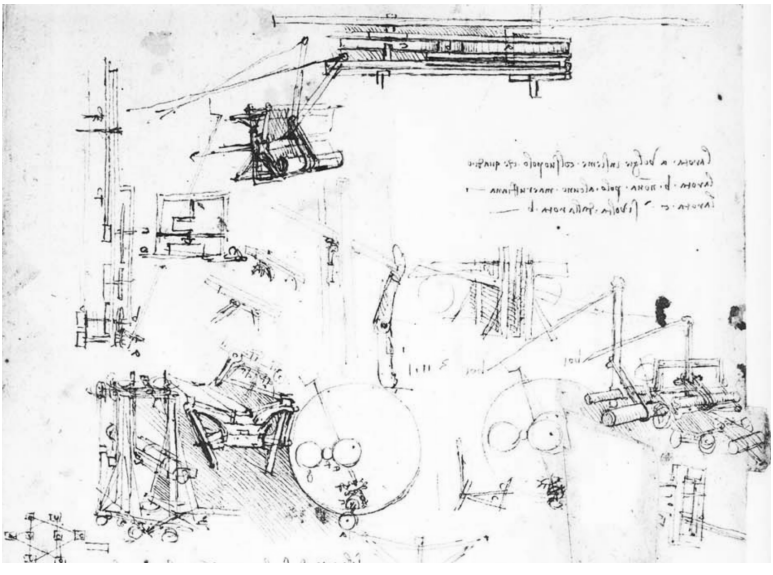


Рис. 1.3. Наброски Леонардо да Винчи из *Codex Atlanticus*

Спустя двести лет Леонардо да Винчи тоже придумал механического рыцаря, который работал на системе тросов и умел садиться, двигать руками и головой и открывать забрало [502]. На рис. 1.3 изображены, по всей видимости, наброски к устройству руки этого робота, которая должна была уметь поднимать тяжести, но разобраться в происходящем не так просто — известно, что Леонардо трепетно относился к своим изобретениям и часто намеренно запутывал записи, добавлял лишние, не относящиеся к делу рисунки и так далее.

Наверное, нет смысла подробно описывать историю чисто технического развития различных человеко- и звереподобных автоматонов, которые были весьма популярны в Новое время, но, по сути, представляли собой всего лишь механические игрушки. Упомяну лишь несколько примеров. Механическая утка, которую в 1738 году выставил в саду Тюильри французский часовщик Жак де Вокансон, стала одним из самых знаменитых «роботов» своего времени. Она могла крутить головой, хлопать крыльями, заглатывать еду из рук смотрителя и даже через несколько минут выводить её остатки в «переваренном» виде (на самом деле, конечно, это был всего лишь трюк, но давайте не будем останавливаться на его устройстве подробнее). Де Вокансон также создал автоматического флейтиста, который был уже полноценным «андроидом», человекоподобным роботом.

Гораздо более совершенных андроидов спустя 50 лет создал другой француз, тоже часовщик, Пьер Жаке-Дро (Jaquet Droz — такая марка часов существует и по сей день) и его сыновья Анри-Луи и Жан-Фредерик. Три их андроида —

(а) (б)

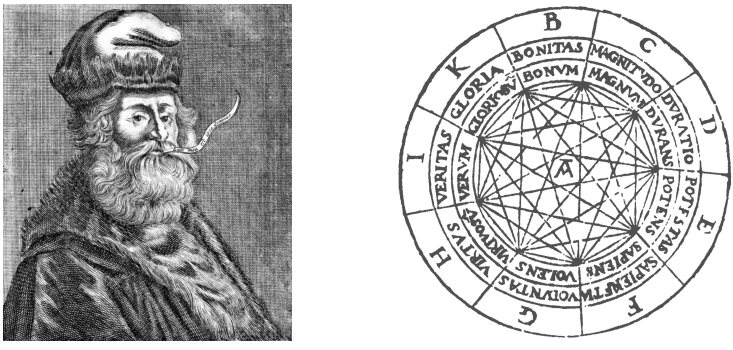
Рис. 1.4. Автоматоны Жаке-Дро: (а) рисовальщик; (б) каллиграф

«Музыкант Марианна» в виде девушки, играющей на органе посредством механизма, состоящего из 2500 элементов, «Рисовальщик Анри» из 2000 элементов (см. рис. 1.4, *а*) и «Каллиграф Шарль» из 6000 элементов (см. рис. 1.4, *б*) — до сих пор хранятся в Музее искусства и истории в Невшателе. Марианна умела играть несколько мелодий, нажимая на клавиши миниатюрного органа, и музыка не была записью, а действительно извлекалась из инструмента. Анри мог создать четыре разных рисунка: портрет Людовика XV, портрет королевской четы, собаку с подписью «мой пёсик» и едущего на колеснице Купидона. Но самым сложным и интересным андроидом был Шарль. Он мог писать тексты длиной до 40 символов, распределённые в четыре строки. Это был фактически первый в истории компьютер: текст сначала нужно было запрограммировать на специальном зубчатом диске, и программа транслировалась в движения мальчика при помощи специального заводного механизма, то есть у Шарля был самый настоящий механический программируемый контроллер. Существует даже версия, что слово «андроид», которое появилось в XVIII веке, происходит от имени Анри-Луи Жаке-Дро. Впрочем, это вряд ли: греческая этимология слишком уж очевидна, да и первые употребления слова «андроид» встречаются раньше, чем семья Жаке-Дро принялась за дело.

1.1.3. Ранняя математическая логика

Но это всё-таки чистая механика, и хотя на неподготовленного зрителя такие механизмы могли произвести сильное впечатление, ни о каком разуме для этих автоматов речи не шло[1]. Встречалось среди таких автоматонов и откровенное мошенничество. Так, шахматный автомат «Турок» Вольфганга фон Кемпелена гремел на всю Европу и обыграл даже самого Наполеона I, но в конце концов

[1] Хотя и сейчас ситуация со многими достижениями искусственного интеллекта примерно та же, не исключено, что положение дел вскоре может измениться...



(а) (б)

Рис. 1.5. Раймунд Луллий: (а) портрет самого философа; на ленточке во рту написано *Lux mea est ipse dominus* («Мой свет — сам Господь»); (б) простейшая логическая машина Луллия: атрибуты Бога и всевозможные связи между ними

выяснилось, что искусственный интеллект не такой уж искусственный: внутри автомата сидел сильный шахматист, который и вёл партию посредством действительно довольно сложного механизма управления механическим турком[1].

Ближе к теме собственно науки искусственного интеллекта подбирались другие средневековые учёные, которые пытались создать машины для проведения логических рассуждений или для порождения новых идей. Здесь нельзя не упомянуть Раймунда Луллия[2], который ещё в XIII веке в своём ars universalis дошёл фактически до панлогизма, то есть отождествления реального порядка вещей с порядком логическим. А это вполне, простите за каламбур, логично привело его к созданию логической машины, состоявшей из концентрических кругов,

[1] В наши дни в честь знаменитого автомата называется сервис *Amazon Mechanical Turk*, на котором живые люди выполняют небольшие и дешевые задания. Часто «туркеры» как раз и занимаются разметкой наборов данных для моделей, о которых мы будем говорить в этой книге; самый популярный русскоязычный аналог *Amazon Mechanical Turk* — это сервисы *«Толока»* и *«Задания»* от *«Яндекса»*.

[2] *Раймунд Луллий* (Raymundus Lullius на латыни, Ramon Llull на каталонском, 1235–1315) — каталонский философ, поэт, теолог, логик и в целом один из важнейших мыслителей высокого Средневековья. Он родился в городе Пальма-де-Майорка, там же и учился и с юности, что логично для испанца, изучал арабский язык и культуру; говорят, Луллий купил для этого раба, который преподавал ему язык, логику и суфизм. Любопытно, что первые его работы — «Компендиум логики аль-Газали» и «Книга размышлений о Боге», которая считается шедевром христианской литературы, — были написаны на арабском. На 32-м году жизни Луллию во время сочинения эротической песни явился распятый Христос, после чего Луллий поселился на пустынной горе Мирамар и пришёл к мысли об особом методе или искусстве, которое бы могло служить основой поиска истины и обращать неверных в христианство. Свой логический метод Луллий называл ars generalis, ars universalis или ars magna, что позже привело к ошибке: алхимики тоже называли искусство превращения металлов и получения жизненного эликсира ars universalis, и многие источники приписывают Луллию массу алхимических трудов, что вовсе не соответствует действительности.

разными вращениями которых можно было получать разные комбинации понятий (Луллий был по совместительству и основателем комбинаторики). Простейшая такая машина изображена на рис. 1.5, *б*; там ничего не движется, схема просто иллюстрирует всевозможные связи между атрибутами Бога, что помогает получить множество ортодоксальных истин: слава Бога вечна, а вечность Его славна и так далее.

Конечно, это совсем не автоматический логический вывод в современном понимании, но Луллий сделал первую попытку автоматического порождения текста и даже смысла. Эту попытку позже будет высмеивать Джонатан Свифт в лапутянской машине для производства знаний, которая описана в эпиграфе к этому разделу, а до логического завершения эту идею доведёт Хорхе Луис Борхес в своей «Вавилонской библиотеке». Но смех смехом, а говорят, что, например, арифмометр Готфрида Лейбница[1] был вдохновлён не только огромными объёмами вычислений, о которых Лейбниц узнал после знакомства с Гюйгенсом[2], но и панлогическими идеями Луллия.

А сам Лейбниц в своей «Монадологии» описал так называемую «мельницу Лейбница», машину размером с мельницу, которая симулирует чувства и восприятие, но внутрь которой можно зайти и осмотреть её движущиеся части, не найдя ни в одной из них восприятия или чувств. Лейбниц этим, конечно, объяснял дуализм и необходимость существования души. Точнее говоря, не совсем

[1] *Готфрид Вильгельм Лейбниц* (Gottfried Wilhelm von Leibniz, 1646–1716) — немецкий философ, логик, математик и механик, физик, юрист и дипломат, историк и не только. Лейбниц считается «завершителем философии XVII века», идеи которой он развил в учении об анализе и синтезе и учении о монадах, неделимых простых сущностях. В математике Лейбниц заложил основы математической логики и комбинаторики, в частности одним из первых описал двоичную систему счисления, изобрёл конструкцию арифмометра (куда лучше, чем у Паскаля), в физике ввёл понятие «живой силы», подозрительно похожей на кинетическую энергию (Лейбниц использовал именно произведение массы на квадрат скорости), а в истории первым обратился к вопросу о возникновении российской правящей династии и систематизировал источники о происхождении Рюрика. Кстати, Лейбниц был знаком с Петром I, который именно под влиянием Готфрида Вильгельма одобрил проект создания Академии наук в Петербурге. Но главной работой Лейбница нужно признать разработанное им дифференциальное исчисление. Вы наверняка слышали о долгом споре Ньютона и Лейбница о приоритете, который надолго омрачил отношения между английскими и европейскими математиками, но бесспорно то, что вариант Лейбница был гораздо доступнее (труды Ньютона знамениты своей сложностью для понимания), и именно он популяризировал новый подход к математике.

[2] *Христиан Гюйгенс ван Зёйлихем* (Christiaan Huygens, 1629–1695) — голландский физик, механик, астроном и изобретатель, первый президент Французской академии наук. Первая известность пришла к Гюйгенсу в астрономии: вместе с братом они усовершенствовали конструкцию телескопа, и Гюйгенс открыл и описал кольца Сатурна и его спутник Титан. Но главный вклад он оставил в математике и механике: Гюйгенс был тонким геометром и решил множество задач механики — описал циклоидальный маятник, развил теорию эволют и эвольвент, вывел законы равноускоренного движения — чисто геометрическими методами, ведь он работал ещё до изобретения математического анализа. И да, работы Гюйгенса сильно повлияли на Ньютона: «Маятниковые часы» были его настольной книгой. А для нас ещё интересно, что в 1657 году Гюйгенс написал приложение «О расчётах в азартной игре» к книге ван Схоотена «Математические этюды»; это было одним из первых изложений зарождавшейся теории вероятностей, и именно по этой книге с ней познакомился Якоб Бернулли.

так: Лейбниц и Декарт одними из первых вообще начали говорить о том, что в мире есть некая механистическая «материя», в которой всё должно объясняться на уровне чисто научном, в терминах, так сказать, соударения бильярдных шаров. Это потребовало от них поместить в какой-то другой тип сущего душу и мысли — сомневаться в существовании души для Лейбница и Декарта было бы немыслимо. Но это всё равно был огромный шаг к материализму: до этих рассуждений, в аристотелевской парадигме, вполне нормально было бы объяснить, скажем, рост деревьев тем, что «деревья имеют естественную склонность расти».

Но заметьте, что по сути это в точности знаменитый аргумент Джона Сёрла[1] о «китайской комнате»! В мысленном эксперименте Сёрла человек или робот находится в комнате, на стенах которой описан очень сложный алгоритм перекладывания карточек с непонятными символами, и он перекладывает эти карточки согласно алгоритму. Герой эксперимента не понимает, что он делает, и не знает ни одного иероглифа, но в итоге оказывается, что вся комната в целом разговаривает на китайском языке: на поступающих на вход карточках написаны реплики на китайском, а после выполнения алгоритма на выходе получаются разумные ответы на них, тоже на китайском языке. Сёрл задаёт не имеющий пока однозначного ответа вопрос: возникает ли здесь понимание китайского языка и если да, то «кто» его понимает — человек, комната, алгоритм?.. Так что в истории искусственного интеллекта, как и любой другой науки, идеи переплетаются и мотивируют друг друга в весьма причудливых сочетаниях.

Не будем углубляться в историю искусственного интеллекта в литературе и других видах искусства: начавшись с романа Мэри Шелли «Франкенштейн, или Современный Прометей», тема искусственного интеллекта, его опасностей и возможностей, особенностей его взаимодействия с людьми уже никуда не исчезала. А настоящей наукой искусственный интеллект стал в середине XX века, когда в литературе уже давно начался золотой век научной фантастики, непредставимой без роботов и мыслящих машин.

Давайте обсудим, с чего эта наука начиналась.

---

[1] *Джон Роджерс Сёрл* (John Rogers Searle, p. 1932) — американский философ. Сёрл получил известность благодаря своим исследованиям языка: он развил теорию речевых актов, ввёл понятие иллокутивного акта как минимальной единицы языкового общения, разделил иллокутивное (то есть функциональное) и пропозициональное (фактическое) содержание высказываний и так далее [519, 524]. Но нам, конечно, интереснее его работы о философии сознания и искусственном интеллекте: сначала теория иллокутивных актов привела его к теории *интенциональности*, то есть, упрощённо говоря, того, как ментальные состояния могут быть *о чём-то*, направлены на что-то в окружающем мире [521], а затем Сёрл стал развивать подход биологического натурализма к проблеме сознания, утверждая, что дихотомия между объективной реальностью и субъективным опытом ложна: сознание — это субъективный опыт, вызванный физическими процессами в мозге [523]. Здесь уже видна и «китайская комната», которую Сёрл придумал ещё в 1980 году [520]. Развивая философский взгляд на искусственный интеллект, Сёрл ввёл понятие «сильного AI» как сознания, возникающего в процессе работы компьютерной программы [523], но сам считал, что этого не будет, а разум не компьютерная программа [522]; порекомендую также работы Штефана Харнада, который много спорил с Сёрлом и в процессе спора, как и подобает философу, прояснял и развивал его идеи [241–244].

1.2. AI как наука: тест Тьюринга и три волны хайпа

Однажды, когда Джеральд Сассмен был ещё стажёром, к нему зашёл Минский и застал его в момент отладки очередной программы для PDP-6.
— Что ты делаешь? — спросил Минский.
— Обучаю случайно связанную нейросеть играть в крестики-нолики, — ответил Сассмен.
— А почему случайно связанную? — спросил Минский.
— Не хочу, чтобы у неё было заложенное заранее мнение о том, как играть, — сказал Сассмен.
Минский закрыл глаза.
— Зачем ты закрыл глаза? — спросил Сассмен учителя.
— Чтобы комната стала пустой.
Тут Сассмен стал просветлённым.

Коан о Марвине Минском,
приписываемый Дэнни Хиллису

1.2.1. Тест Тьюринга

Современному читателю может показаться, что нейронные сети отправили многие подходы из «классического искусственного интеллекта» на пенсию. Отчасти это, конечно, так: некоторые подходы устаревают и заменяются другими, и иногда можно с уверенностью сказать, что время той или иной конкретной модели прошло и вряд ли вернётся. Однако вероятностные основы машинного обучения — это фундамент, который, пожалуй, не устареет никогда. Именно с этим фундаментом вы и познакомитесь в этой книге; но сначала давайте сделаем действительно исторический экскурс и обсудим, откуда пошёл искусственный интеллект как наука с конкретными приложениями, а не как предмет умозрительных рассуждений и как он развивался со временем.

Можно сказать, что первыми шагами в искусственный интеллект были ранние работы по математическому моделированию биологии мозга. В знаменитой работе [386], вышедшей ещё в 1943 году, Уоррен Маккаллох (Warren McCulloch) и Уолтер Питтс (Walter Pitts) пытались понять, как мозг может производить такие сложные образы, если он состоит из относительно просто устроенных нейронов. Можно сказать, что эта работа стала основанием *коннекционизма*: базовой идеи о том, что сложное поведение и высокоуровневые способности можно получить при помощи большой композиции простых объектов. Математическая модель нейрона, которая вводилась в статье [386], была вполне современной: это та же линейная комбинация входов, за которой следует нелинейная функция,

разве что в те времена нелинейной функцией обычно служил порог активации, а не сигмоиды или ReLU (в этой книге мы до искусственных нейронных сетей не дойдём, но подозреваю, что многие читатели так или иначе об этом уже слышали). Сама же работа скорее относилась к математической логике и пыталась показать, что композициями таких нейронов можно реализовать широкий класс весьма сложных функций.

Однако обычно формальной отсечкой, началом искусственного интеллекта как науки считают формулировку *теста Тьюринга*. Этот тест впервые появился в знаменитом эссе *Computing Machinery and Intelligence*, которое Алан Тьюринг[1] опубликовал в 1950 году [606]. Эссе появилось не на пустом месте: возможность построения мыслящих машин, проблемы того, как снабдить компьютерные программы интеллектом, и последствия, к которым может привести появление такого интеллекта, обсуждались уже как минимум около десяти лет и самим Тьюрингом, и его коллегами в английском Ratio Club.

Наверное, вы уже слышали, как устроен тест Тьюринга: участвующий в нём компьютер должен успешно выдать себя за человека в диалоге на естественном языке с человеком-«судьёй». Здесь возникает вопрос о том, что это значит, ведь даже подбрасывающий монетку судья будет верно угадывать, человек перед ним или компьютер, в половине случаев. Тьюринг, будучи одним из основоположников теоретической информатики, смог формализовать, что значит «успешно выдать», при помощи достаточно стандартной техники из области вероятностных алгоритмов, которая тогда как раз появлялась. Будем говорить, что тест Тьюринга пройден, если человек-судья, разговаривающий с неизвестной сущностью, угадывает, человек это или компьютер, с вероятностью, неотличимой от $\frac{1}{2}$, то есть он не может угадывать чаще, чем бросая честную монетку. Диалог, конечно, предполагается письменным: мы не хотим требовать конструировать андроидов, неотличимых от человека на вид или на ощупь.

Кстати говоря, со временем в формулировке теста Тьюринга потерялась важная деталь. Обратите внимание на то, что тест крайне несправедлив по отношению к компьютерам, пытающимся его пройти, ведь человеку достаточно просто вести себя естественно, быть собой, а компьютеру надо стараться выдавать себя за того, кем он не является. Можно представить себе «обратный тест Тьюринга», в котором человек попробовал бы выдать себя за компьютер. Разумеется, он

[1] *Алан Тьюринг* (Alan Turing, 1912–1954) — английский математик и информатик. Тьюринг был одним из создателей современной теоретической информатики и теории вычислимости: он разработал конструкцию машины Тьюринга и доказал первые результаты о (не)вычислимости. Во время Второй мировой войны Тьюринг был одним из ведущих криптоаналитиков Блетчли-парка и расшифровал ряд важнейших сообщений, закодированных знаменитыми машинами «Энигма». Статьи Тьюринга о математике криптоанализа были рассекречены только в 2012 году. Тьюринг также стал основателем искусственного интеллекта, сформулировав основные положения AI как науки, в частности знаменитый тест Тьюринга. К сожалению, Тьюринг погиб, не дожив до 42-го дня рождения; мы до сих пор не знаем, было ли это самоубийством из-за назначенного Тьюрингу гормонального лечения гомосексуальности (увы, времена были отнюдь не толерантные), или цианид попал в пресловутое яблоко случайно (это, как ни странно, вполне возможно).

мгновенно был бы раскрыт вопросом вроде «Сколько будет $391^3 / 573$?». Тьюринг понимал эту тонкость и попытался устроить тест так, чтобы человеку тоже приходилось притворяться. Исходная формулировка теста Тьюринга была основана на популярной тогда имитационной игре[1], в которой мужчина и женщина письменно общаются с судьёй. Задача мужчины — выдать себя за женщину, задача женщины — помочь судье правильно разобраться, кто есть кто. Поэтому Тьюринг в своём тесте предложил сравнивать компьютерную программу, пытающуюся выдать себя за женщину, и мужчину, пытающегося выдать себя за женщину: тогда обе стороны вынуждены будут имитировать кого-то третьего. Думаю, читателю ясно, что такая формулировка не могла пройти проверку временем, и в современных описаниях теста Тьюринга эта тонкость теряется. Сам Тьюринг считал, что к 2000 году компьютеры с гигабайтом памяти (очень точное предсказание, кстати!) смогут играть в имитационную игру так, чтобы убеждать человека в 30 % случаев (наилучшим результатом, как я уже говорил, было бы 50 %).

Тьюринг предпринял первую попытку определить искусственный интеллект научным образом, и попытка оказалась очень удачной: тест Тьюринга до сих пор выглядит вполне актуальным с точки зрения своей исходной цели. Даже современные языковые модели пока в некотором смысле не прошли тест Тьюринга, хотя сейчас уже лучше уточнить, что речь идёт об антагонистическом тесте, в котором судья знает, что делает, и активно старается вывести «подделку» на чистую воду. На мой взгляд, дело здесь даже не столько в гигантской сложности естественных языков как формального объекта, сколько в интерактивной структуре теста Тьюринга. Фактически мы можем использовать тест Тьюринга как «рамку» для того, чтобы проверить способности собеседника к чему угодно: поговорить о литературе и искусстве, попросить описать окружающий собеседника трёхмерный мир, поместить разговор в контекст происходящего в мире или в контекст изменений во времени... Всё это даёт новые вызовы и задачи для искусственного интеллекта, не сводящиеся к умению порождать грамматически правильные тексты. Конечно, тест Тьюринга сложно использовать на практике; одна причина в том, что он требует человеческого участия, но главное здесь в том, что тест Тьюринга требует от машинного интеллекта сразу «всё или ничего» — нельзя «пройти тест Тьюринга на 10 %» (убедить 10 % судей — это уже 99 % пути к успеху), и непонятно, как выглядел бы тест Тьюринга в более ограниченной области, не требующей полноценного разговора на естественном языке.

Поэтому попытки придумать другие «лакмусовые бумажки» для искусственного интеллекта предпринимались часто и неизбежно оказывались не слишком удачными. Например, гуру искусственного интеллекта Аллен Ньюэлл в 1973 году выпустил статью под названием *You Can't Play 20 Questions with Nature and Win* («Нельзя выиграть у природы в “20 вопросов”») [431], в которой критиковал современную ему экспериментальную психологию и, в частности, утверждал,

[1] *The Imitation Game* — так называется и известный биографический фильм о Тьюринге.

что для развития и психологии, и искусственного интеллекта было бы полезно начать исследования некоторой промежуточной «сверхзадачи», для решения которой потребуются многие аспекты настоящего, сильного искусственного интеллекта. Ссылаясь на работы психологов, Ньюэлл предложил в качестве такой сверхзадачи... игру в шахматы: «Мы уже знаем из предыдущих исследований, что [шахматы] включают формы рассуждений и поиска, сложные процессы восприятия и памяти... планирование, оценку, анализ целей и средств и переопределение ситуации, а также несколько разновидностей обучения: краткосрочное, *post hoc* анализ, подготовительный анализ, обучение по книгам и так далее» [431].

Это предположение звучало абсолютно разумно в 1973 году, но уже 11 мая 1997 года созданный IBM суперкомпьютер Deep Blue[1] выиграл матч-реванш из шести партий у Гарри Каспарова (в первом матче чемпион мира уверенно победил), а последние матчи между людьми и лучшими компьютерами на равных сыграл Владимир Крамник, вничью завершив матч с Deep Fritz в 2002 году и проиграв фактически из-за одного знаменитого зевка в 2006-м; всё это проиллюстрировано на рис. 1.6. В последние 15 лет никто не сомневается, что у человека нет никаких шансов против лучших компьютерных программ: их рейтинги Эло уже давно перевалили за 3500, в то время как величайший шахматист современности Магнус Карлсен, скорее всего, так и не доберётся до 2900.

Но разве, когда Deep Blue победил Каспарова, мы признали, что искусственный интеллект создан? Вовсе нет: люди посмотрели на внутреннее устройство алгоритмов Deep Blue — по сути, он просто осуществлял поиск в глубину с α-β-отсечениями (см., например, [509]), а его функция оценки позиции была отчасти создана вручную, а отчасти обучена на партиях профессионалов, — и решили, что, пожалуй, нет, это не считается. Шахматы оказались слишком «простой» игрой, слишком легко оказалось использовать заведомо бóльшие вычислительные способности компьютеров. Вот го — это совсем другое дело, решили исследователи! И действительно, считать варианты в глубину в го на много порядков сложнее, потому что разных разумных ходов в обычной позиции гораздо больше; это значит, что дерево поиска даже после учёта всех отсечений и симметрий получается шириной не в 5–10 возможных ходов, как в шахматах, а в 30–50, а это уже совсем другая экспонента.

Когда Deep Blue обыгрывал Каспарова, лучшие программы для игры в го проигрывали средненьким любителям с гигантской форой в 15–20 камней, которая в шахматах была бы эквивалентна игре без ферзя и обеих ладей. Однако в 2005 году появился метод MCTS (Monte Carlo tree search, поиск по дереву методом Монте-Карло), который позволил го-программам «подтянуться» к лучшим игрокам-людям, сначала любителям.

[1] Сначала Deep Blue назывался Deep Thought, в честь того самого суперкомпьютера, который в произведениях Дугласа Адамса дал ответ на главный вопрос жизни, Вселенной и всего такого, а затем построил Землю как суперкомпьютер для поиска вопроса к этому ответу. Однако затем шахматный суперкомпьютер был переименован — говорят, из-за созвучия с сами догадайтесь чем...

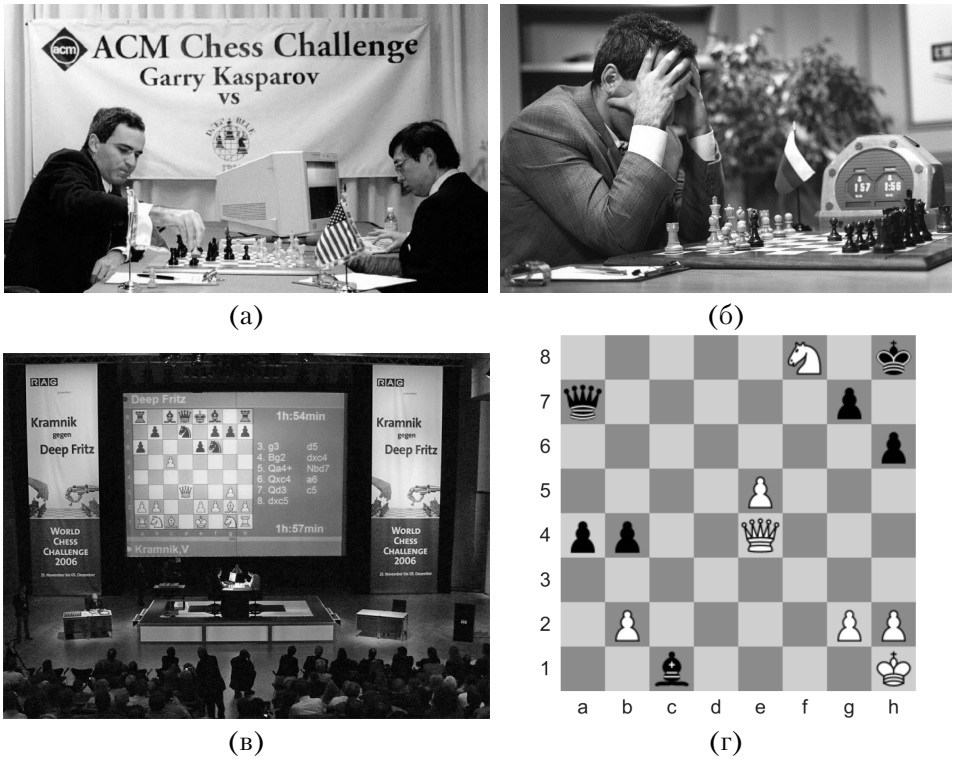


Рис. 1.6. Люди против машин в шахматах: (а) Гарри Каспаров во время первого (выигранного) матча против Deep Blue; (б) Гарри Каспаров во время второго (проигранного) матча против Deep Blue; (в) матч Владимира Крамника против Deep Fritz в 2006 году; (г) «зевок века»: в этой позиции Крамник сделал ход Фе3, не заметив мата на h7

А потом, весной 2016 года, основанная на глубокой нейронной сети программа AlphaGo (о ней и особенно о её последовательнице AlphaZero мы обязательно когда-нибудь поговорим, но не в этой книге) в историческом матче победила одного из ведущих профессионалов Ли Седоля, и в го, как и в шахматах, установилась гегемония компьютеров.

Признали ли мы, что искусственный интеллект был создан в лице AlphaGo? Вопрос риторический...

Так что с «лакмусовыми бумажками» пока не очень получается, и тест Тьюринга остаётся практически единственной вехой, которая с 50-х годов XX века дожила до наших дней. Современные большие языковые модели уже подходят очень близко к тому, чтобы пройти тест Тьюринга. Давать конкретные ссылки

опасно, ведь каждый год выходят новые модели. Когда я пишу эти строки, по миру уверенной поступью идёт GPT-4, пришедший на смену модели ChatGPT, которая является вариантом InstructGPT [602], в свою очередь основанной на архитектуре GPT-3 [330]. ChatGPT блестяще ведёт диалог и уже сейчас может помочь с автоматизацией различной интеллектуальной работы (думаю, мы ещё не до конца понимаем, насколько сильно может помочь), но всё-таки пока вряд ли обманет человека, который специально пытается выяснить, является ли собеседник компьютерной программой. Даже сейчас, в 2024 году, я не уверен, пройдёт ли GPT-4 настоящий состязательный (adversarial) тест Тьюринга[1]. Но я уверен, что, когда вы будете эти строки читать, главной новостью станет что-то другое, и полное прохождение антагонистического теста Тьюринга уже не за горами.

1.2.2. Дартмутский семинар и первая волна хайпа

Даже из умозрительного рассмотрения теста Тьюринга ясно, что задачи искусственного интеллекта очень разнообразны: здесь и обработка естественных языков, и компьютерное зрение, то есть умение понимать окружающее пространство и ориентироваться в нём по изображениям, и логический вывод, и в каком-то виде реализация памяти, и многое другое. Отсчёт науки об искусственном интеллекте, которая пытается решать все эти задачи, обычно ведётся от так называемого *Дартмутского семинара*, двухмесячной, как мы бы сейчас сказали, летней школы в Дартмутском колледже[2] на которой появились основные постановки задач искусственного интеллекта и сформировались первые конкретные направления исследований. Семинар провели летом 1956 года четыре отца-основателя искусственного интеллекта: основоположник функционального программирования и создатель языка Lisp Джон Маккарти (John McCarthy), будущий гуру искусственного интеллекта, лауреат премии Тьюринга Марвин Минский[3], главный архитектор первого промышленно выпускавшегося компьютера IBM 701

[1] Во многом, правда, потому, что авторы современных языковых моделей изо всех сил стараются этот тест не проходить: чтобы отличить саму GPT-4 от человека, достаточно спросить её, не языковая ли она модель...

[2] Несмотря на название, Дартмутский колледж (Гановер, штат Нью-Гэмпшир) — это полноценный исследовательский университет, входящий в Лигу плюща. Это один из девяти так называемых колониальных колледжей, созданных ещё до американской революции; колледж вырос из основанной в 1755 году индейской благотворительной школы «для обучения и воспитания молодёжи индейских племён на этой земле в чтении, письме и всех частях учёности, которая явится необходимой и целесообразной для цивилизации и христианизации детей язычников». На гербе колледжа изображён его девиз: *Vox clamantis in deserto* («Глас вопиющего в пустыне»).

[3] *Марвин Ли Минский* (Marvin Lee Minsky, 1927–2016) — американский информатик, один из отцов-основателей искусственного интеллекта. Его статьи *Steps Towards Artificial Intelligence* [399] и *Matter, Mind, and Models* [400] содержали постановки важнейших задач на пути к построению мыслящих машин и сыграли центральную роль в развитии искусственного интеллекта и когнитивистики. Минский разработал первый в мире шлем виртуальной реальности (в 1963 году!), а также изобрел конфокальный микроскоп, нашедший массу применений в биологии и медицине. Любопытно, что Минский консультировал Артура Кларка при написании и постановке его «Космической Одиссеи — 2001» и даже упоминается там как автор искусственного интеллекта, причём его описание

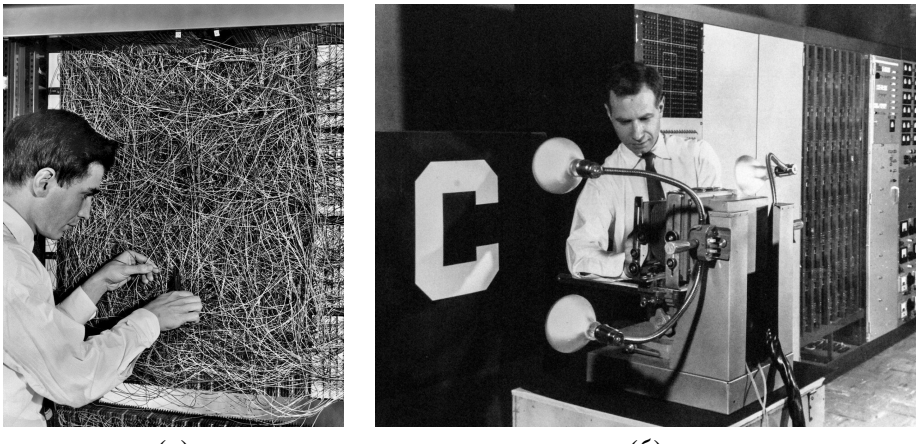
Натаниэль Рочестер (Nathaniel Rochester) и создатель теории информации Клод Шеннон (Claude Shannon).

Планы у них, правда, шли значительно дальше, чем просто сформулировать постановки задач, — их заявка на проведение семинара была одной из самых амбициозных заявок на грант в истории информатики. Вот что писал там Джон Маккарти [480]: «Мы предлагаем исследование искусственного интеллекта сроком на два месяца с участием десяти человек летом 1956 года в Дартмутском колледже, Гановер, Нью-Гемпшир. Исследование основано на предположении, что всякий аспект обучения или любое другое свойство интеллекта может в принципе быть столь точно описано, что машина сумеет его имитировать. Мы попытаемся понять, как обучить машины использовать естественные языки, формировать абстракции и концепции, решать задачи, сейчас подвластные только людям, и улучшать самих себя. Мы считаем, что существенное продвижение в одной или более из этих проблем вполне возможно, если специально подобранная группа ученых будет работать над этим в течение лета». Весьма оптимистично, правда?

Такой оптимизм был отличительной чертой ранней истории искусственного интеллекта. Это был золотой век научной фантастики, и три закона роботехники, которые тогда сформулировал Айзек Азимов, казались вполне реализуемыми в самом ближайшем будущем. Представлялось, что ещё немного, и вокруг нас будут ходить андроиды, разговаривать с нами приятным голосом и приносить кофе, а главными задачами искусственного интеллекта казались не естественные языки или распознавание образов, а проблемы логического вывода. Есть известная байка о том, что в 1966 году Марвин Минский выдал Джеральду Сассмену (Gerald Jay Sussman), тогда аспиранту, задачу решить проблему компьютерного зрения в качестве проекта на лето. Это, конечно, неправда — на самом деле в 1966 году коллега Минского Сеймур Пейперт (Seymour Papert) выдал целой группе студентов и аспирантов под руководством Сассмена летний проект, в котором предложил решить задачу сегментации изображений... ну ладно, не любых изображений, а сначала простых форм на однородном фоне, а затем простых форм со сложными текстурами: батарейки, пачки сигарет и так далее [454]. Но от этого уточнения история, честно говоря, менее фантастической не становится.

Оптимизм укрепился ещё больше, когда в 1958 году команда под руководством Фрэнка Розенблатта — по основной специальности, кстати, психолога — создала «Перцептрон», уже физически построенный компьютер, который реализовывал один искусственный нейрон, то есть фактически строил линейную модель для классификации (мы такие обсудим в главе 5). На рис. 1.7 показано, как выглядела романтика ранних компьютеров: Mark I Perceptron занимал два

звучит в книге очень современно: «К 80-м годам XX столетия... Минский и Гуд разработали методику автоматического зарождения и самовоспроизведения нервных цепей в соответствии с любой произвольно выбранной программой. Оказалось, что искусственный мозг можно “выращивать” посредством процесса, поразительно сходного с развитием человеческого мозга. Точные детали этого процесса в каждом отдельном случае так и оставались неизвестными; впрочем, будь они даже известны, человеческий разум не смог бы постичь всю их сложность».



(а) (б)

Рис. 1.7. Первый «Перцептрон» Розенблатта: (а) Розенблатт что-то исправляет в схеме; (б) демонстрация способностей «Перцептрона» [337]

шкафа с таким вот переплетением проводов, соединяющих вакуумные лампы. Он успешно решал задачи обучения соответствующего масштаба: например, на первой публичной демонстрации в 1958 году «Перцептрон» научился отличать левую сторону перфокарты от правой, то есть правильно определять, находятся ли отметки слева от середины или справа.

Но по итогам этой демонстрации *The New York Times* выпустила статью, в которой говорилось следующее: «Сегодня ВМФ США показал эмбрион электронного компьютера, который, как ожидается, сможет ходить, говорить, видеть, писать, воспроизводить себя и осознавать своё существование... Эмбрион — компьютер "704", принадлежащий Бюро метеорологии и стоивший $2 000 000, — в сегодняшней демонстрации для репортёров научился отличать левое от правого за 50 попыток. ВМФ утверждает, что на том же принципе сумеет построить первую мыслящую машину под названием "Перцептрон", которая сможет читать и писать. Работу над ней планируется закончить приблизительно за год, потратив ещё около $100 000. Доктор Фрэнк Розенблатт... сказал, что эта машина станет первым устройством, которое будет мыслить как человеческий разум... что "Перцептроны" можно будет запускать к другим планетам как механических исследователей космоса... что в принципе будет возможно построить искусственные мозги, которые смогут воспроизводить самих себя на конвейере и которые будут осознавать своё существование» [595].

Как вы уже догадались, такой безудержный оптимизм — а это не какой-то таблоид, это *The New York Times*, то есть вполне можно верить, что слова Розенблатта переданы верно, — в те годы никак не мог оказаться оправданным.

1.2.3. Машинный перевод и первая «зима искусственного интеллекта»

Одна из самых показательных историй первой шапкозакидательной волны искусственного интеллекта — это ранние попытки создать модели *машинного перевода* [270]. Как мы сейчас понимаем, в те времена было чрезмерным оптимизмом и вовсе говорить о машинном переводе как о задаче, имеющей какие-то практические перспективы, но время раннего расцвета искусственного интеллекта совпало с расцветом идей трансформационно-порождающих грамматик Ноама Хомского[1]. Казалось, что естественный язык скоро можно будет формально представить в виде единой универсальной грамматики, в которую нужно будет просто подставлять разные слова, а машинный перевод сведётся к поиску правил перевода одной грамматики в другую[2].

О машинном переводе как о практической задаче начали задумываться в конце 1940-х годов: Уоррен Уивер (Warren Weaver), директор Отделения естественных наук в Фонде Рокфеллера, в 1947 году написал об этой задаче Норберту Винеру[3], а в 1949-м опубликовал меморандум под названием «Перевод» *(Translation)*, который разослал ведущим лингвистам, логикам и математикам [639]. В 1952 году состоялась первая конференция по машинному переводу в Массачусетском технологическом институте, а в 1954-м состоялась первая публичная

[1] *Ноам Хомский* (Noam Chomsky, p. 1928) — американский лингвист, философ и социальный критик. Сам Хомский родился в Филадельфии, но оба его родителя происходили из Российской империи: отец уехал из украинского села Купель в 1913 году, а мать родилась на территории современной Беларуси. Ещё один эмигрант из России, Зеллиг Харрис, познакомил молодого Хомского с теоретической лингвистикой, и диссертация Хомского, защищённая в 1955 году, уже содержала все основные идеи трансформационно-порождающих грамматик. Хомский и сам работал над ранними проектами по машинному переводу, продолжал развивать теорию синтаксиса и порождающих грамматик; в 1971 году, после дебатов с Мишелем Фуко о природе человека, он стал одной из главных фигур аналитической философии. Политикой Хомский увлёкся во время войны во Вьетнаме: писал антивоенные эссе и книги, примкнул к «новым левым», критиковал американский империализм; начиная с 1980-х годов Хомский уже больше политический мыслитель и активист, чем лингвист и философ. Однако основополагающая важность его работ бесспорна. Хомский остаётся одним из самых высокоцитируемых учёных: в 2024 году Google Scholar насчитывает более 500 тысяч цитирований его работ и присваивает ему h-индекс 195.

[2] Идею о том, что у человеческих языков существует универсальная грамматика, которую человек получает при рождении и затем лишь наполняет содержанием его родного языка, Хомский защищает до сих пор. Рекомендую прочитать материалы его недавней публичной дискуссии с одним из выдающихся специалистов по искусственному интеллекту, Петером Норвигом, здесь защищающим статистический подход к анализу и синтезу естественных языков, который, собственно, мы все сейчас в машинном обучении и используем [305, 443]. Дискуссия началась с обработки естественных языков, но быстро перешла на искусственный интеллект в целом.

[3] *Норберт Винер* (Norbert Wiener, 1894–1964) — американский математик, основоположник кибернетики и искусственного интеллекта. В детстве Винер был классическим вундеркиндом: в 7 лет написал первый научный трактат (о дарвинизме), в 11 лет поступил в Тафтс-колледж, в 14 окончил его, получив степень бакалавра, а в 18 уже получил степень Ph. D. по математической логике в Корнелле и Гарварде. Винер создал фундаментальные труды по теории случайных процессов, обработке сигналов и теории информации (для непрерывных сигналов). Он создал кибернетику — науку об управлении и о процессах с обратной связью — и получил много результатов в роботике, теории управления и искусственном интеллекте. Винера часто приводят в пример как одного из умнейших людей XX века (наряду с Эйнштейном и Хокингом), оценивая его IQ около 170.

Таблица 1.1. Примеры, переведённые в Джорджтаунском эксперименте [271]

Вход (русский транслит)	Выход (английский)
Mi pyeryedayem mislyi posryedstvom ryechyi.	We transmit thoughts by means of speech.
Vyelyichyina ugla opryedyelyayetsya otnoshyenyiyem dlyini dugi k radyiusu.	Magnitude of angle is determined by the relation of length of arc to radius.
Myezhdunarodnoye ponyimanyiye yavlyayetsya vazhnim faktorom v ryeshyenyiyi polyityichyeskix voprosov.	International understanding constitutes an important factor in decision of political questions.

демонстрация программы для машинного перевода, которую написал Леон Достерт (Leon Dostert).

Эта демонстрация стала известна как Джорджтаунский эксперимент (Georgetown — IBM Experiment) [271]. В эксперименте демонстрировалась программа, содержащая всего шесть грамматических правил и 250 слов в словаре; тем не менее в демонстрационных примерах переведённые с русского на английский — а как же иначе, мир уже вовсю становился биполярным! — предложения выглядят вполне узнаваемо (табл. 1.1). В том же году аналогичную программу создали и советские учёные под руководством Дмитрия Панова и Изабеллы Бельской, запустив машинный перевод в Институте точной механики и вычислительной техники АН СССР на компьютере БЭСМ [687].

Первые успехи воодушевили как исследователей, так и государственные, так сказать, грантообразующие агентства в США, которые начали активно поддерживать исследования в области машинного перевода и искусственного интеллекта в целом. Однако шли годы, но ни роботы не начинали приносить кофе, ни даже автоматической вавилонской рыбки не появлялось.

В 1964 году правительство США организовало ALPAC (Automatic Language Processing Advisory Committee) — комитет, который был призван оценить прогресс вычислительной лингвистики и дать рекомендации о том, что делать с этой областью дальше. Хотя руководитель комитета, Джон Пирс, был не только специалистом по телекоммуникациям и теории информации, но и автором научно-фантастических книг, позиция комитета оказалась весьма скептичной. В 1966 году комитет выпустил отчёт под названием «Язык и машины: компьютеры в переводе и лингвистике» [121]; это была целая небольшая книга, которая рассматривала как научную сторону вопроса, так и экономическую целесообразность происходящего.

Выводы были неутешительными: ALPAC пришёл к выводу, что нужно ещё много исследований в фундаментальной лингвистике, прежде чем машинный перевод действительно заработает и станет экономически эффективным. Сейчас, глядя на те системы машинного перевода, которые действительно приносят

пользу на практике, мы, конечно, можем только согласиться с выводами комиссии ALPAC[1].

Отчёт ALPAC привёл к тому, что те самые государственные агентства, которые в США были основными спонсорами фундаментальных исследований (как, собственно, и в СССР, и в большинстве стран на тот момент), изрядно прикрутили кран, из которого в искусственный интеллект текли деньги. А самостоятельно зарабатывать новая наука ещё не могла — до по-настоящему применимых в жизни, трансформирующих целые индустрии приложений оставались ещё десятилетия. Поэтому конец 1960-х и 1970-е годы обычно называют первой «зимой искусственного интеллекта» (AI winter): в это время область существенно сократилась, и многим исследователям пришлось заняться чем-то другим.

Остались на плаву, в частности, *системы, основанные на знаниях* (knowledge-based systems), и *экспертные системы* (expert systems), расцвет которых приходится как раз на 1970-е годы. Суть дела здесь проще всего объяснить на примере. Давайте рассмотрим главный исторический пример такой системы: модель MYCIN, которая была разработана в начале 1970-х годов для задач медицинской диагностики [82, 539].

Представьте себе, что вам нужно поставить диагноз по анализу крови или рекомендовать, каким антибиотиком воспользоваться (то есть диагностировать, какая бактерия поселилась у больного); название MYCIN, кстати, не аббревиатура, а окончание «-мицин», которое встречается во многих антибиотиках. На входе у вас несколько десятков чисел или категориальных переменных, представляющих собой разные параметры самого больного, результаты его анализов и так далее. Опытный врач может посмотреть на эти данные и поставить диагноз, но мы не можем спросить у опытного врача, как он это делает. Точнее говоря, спросить можем, но ответить у врача не получится: наш мозг не очень приспособлен к тому, чтобы формализовать свой процесс принятия решений в виде математического описания функции от пяти десятков переменных. Что врач реалистично может сделать для формализации — это попытаться выделить некие эвристики, правила, соединяющие несколько входных переменных: если у пациента низкий гемоглобин и мало эритроцитов, это анемия, а если в крови полпроцента алкоголя, то пациенту срочно пора в реанимацию. Создатели экспертной системы могут собрать такие правила, подтвердив их валидность у опытных врачей; в системе MYCIN, например, было около 600 правил.

Задача экспертной системы состоит в том, чтобы научиться применять эти правила к новой ситуации. Параметры нового пациента могут соответствовать

[1] Поделюсь личным опытом: мне машинный перевод стал действительно помогать около 2016–2017 года. Только в это время *Google Translate* достиг того уровня, на котором переведённые тексты всё равно, конечно, нельзя было использовать напрямую, но уже стало быстрее и эффективнее прогнать длинный, не слишком литературный текст через *Google Translate* и затем исправить ошибки, чем напечатать всё самому с нуля (я сейчас говорю о переводах с русского на английский, в обратную сторону не проверял). По всей видимости, этот скачок качества был связан с переходом на нейросетевую модель *Google's Neural Machine Translation* [226].

сразу нескольким правилам, иногда с противоположными выводами. Более того, сформулированные людьми правила, скорее всего, будут нечёткими; например, что значит «низкий гемоглобин» в предыдущем абзаце, как это перевести в числа? Здесь экспертные системы смыкаются с *нечёткой логикой* (fuzzy logic) и *теорией нечётких множеств* (fuzzy set theory) [235, 311], которые тоже переживали свой расцвет в это время; но в эти области мы углубляться уж точно не будем. А чтобы совмещать друг с другом эвристические правила, нужно проводить нечто вроде вероятностного вывода: например, MYCIN вычисляла так называемые факторы уверенности (certainty factors), комбинируя уверенности в отдельных пропозициях.

Уже такая ранняя система, как MYCIN, ставила диагнозы очень хорошо, не хуже опытных врачей и лучше начинающих. Но на практике она всё-таки не взлетела, в основном по техническим причинам: в 1970-х годах запустить сложную компьютерную программу можно было только на большом мейнфрейме, время на котором нужно было резервировать заранее, а один сеанс работы с MYCIN занимал где-то полчаса, и в итоге для врачей этот процесс получался неудобным и ничего не улучшающим.

Ещё одна причина первой «зимы искусственного интеллекта» выглядит одновременно и более математически обоснованной, и более таинственной. В конце 1960-х годов уже упоминавшиеся Минский и Пейперт выпустили книгу «Перцептроны» [401], в которой немало места посвятили критике конструкции перцептрона Розенблатта. Основной пункт этой критики звучал так: один перцептрон со входами $x_1, \ldots, x_d$ реализует функцию $h(w_1 x_1 + \ldots + w_d x_d)$, где w_i — веса перцептрона, которые нужно обучать, а h — некоторая функция активации; например, если $h(x) = x$, то получится линейная регрессия (см. главу 4), а если $h(x) = \sigma(x) = \frac{1}{1+e^{-x}}$, то получится логистическая регрессия (см. раздел 5.2). А значит, как ни выбирай функцию активации h, разделяющая поверхность, которую реализует перцептрон в случае классификации на два класса, будет линейной: уравнение $h(w_1 x_1 + \ldots + w_d x_d) = 0$ превратится в $w_1 x_1 + \ldots + w_d x_d = c$ для такой константы c, что $h(c) = 0$[1]. Более подробно о геометрической стороне задач классификации мы поговорим в разделе 5.1.

Минский и Пейперт заключают из этого, что у перцептрона есть принципиальные ограничения, причём не абстрактные и недостижимые в реальной жизни (как, например, ограничения на логический вывод, накладываемые теоремами Гёделя о неполноте, — в принципе, можно найти конкретные недоказуемые утверждения, но вряд ли какая-то реально нужная людям теорема вдруг окажется недоказуемой, подозрения на этот счёт есть разве что в отношении вопроса о равенстве классов сложности P и NP [5–7]), а конкретные и осязаемые: перцептрон, например, никогда не сможет адекватно реализовать функцию XOR от

[1] Формально таких констант может быть несколько, но в реальности, конечно, функции активации h обычно монотонны, да и несколько параллельных гиперплоскостей здесь всё равно не помогут делу.

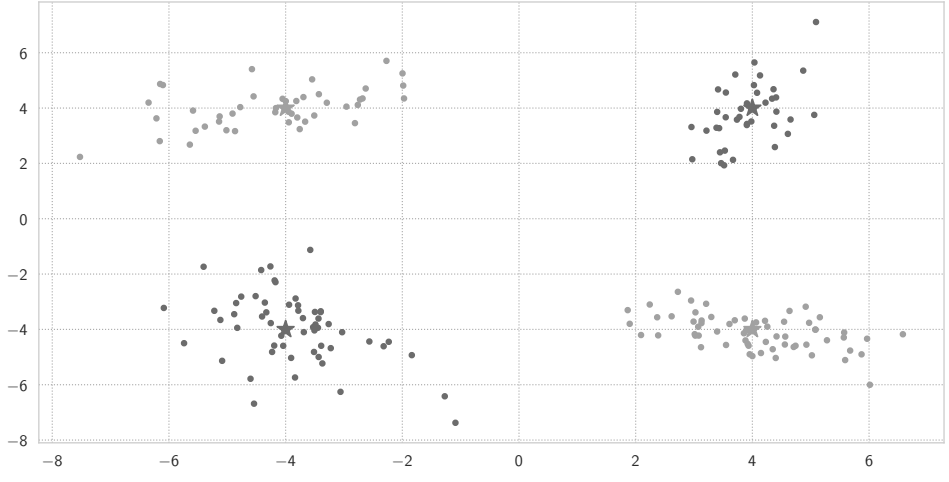


Рис. 1.8. Один перцептрон не сможет разделить точки этих двух классов, потому что не существует гиперплоскости (прямой), хорошо их разделяющей; но если разрешить провести две гиперплоскости, точки разделятся идеально

двух переменных, то есть не сможет разделить точки двух классов, показанных на рис. 1.8.

Разумеется, всё вышесказанное абсолютно верно: один перцептрон действительно задаёт линейную разделяющую поверхность, и он действительно не справится с примером на рис. 1.8. Но сейчас для нас очевидно решение этой проблемы: один перцептрон — это лишь линейная модель, а чтобы сделать что-то более выразительное, нужно построить *сеть* из перцептронов, то есть искусственную нейронную сеть. Конечно, Минский и Пейперт понимали это; идея искусственных нейронных сетей вообще является одной из старейших в искусственном интеллекте, она появилась ещё в упоминавшихся выше работах Маккаллоха и Питтса. Хотя Маккаллох и Питтс рассматривали нейронные сети скорее со стороны математической логики, чем не существовавшего тогда искусственного интеллекта, их конструкция в точности совпадает с перцептроном Розенблатта, а конструкцию глубокой нейронной сети для применений в искусственном интеллекте предложил ещё в 1948 году сам Тьюринг, вместе даже с алгоритмом обучения [605].

Но почему-то — здесь я и вижу некоторую странность — в конце 1960-х годов критика Минского и Пейперта была воспринята как принципиальный недостаток не только одного перцептрона, но и искусственных нейронных сетей вообще. Скорее всего, это было связано с тем, что для глубоких нейронных сетей тогда не было известно эффективных алгоритмов обучения. Но это тоже странно, ведь градиентный спуск применяли ещё отцы математического анализа в первой

половине XIX века, а алгоритм обратного распространения ошибки в глубокой нейронной сети — это, по сути, просто взятие производной от сложной функции. Тем не менее о нейронных сетях забыли до конца 1970-х...

1.2.4. Обратное распространение ошибки и вторая волна хайпа

Забыли до тех пор, пока не вспомнили вместе с тем самым алгоритмом обратного распространения ошибки. Его открывали несколько раз: первым был, по всей видимости, финский студент Сеппо Линненмаа, для которого обратное распространение ошибки стало темой магистерской диссертации [348], затем были работы Дрейфуса [153] и Вербоса [640], а окончательно популяризировали алгоритм американский психолог Дэвид Румельхарт и британский, а позже канадский математик Джеффри Хинтон[1], который вообще сыграл одну из ключевых ролей в истории искусственных нейронных сетей [507, 508]. После их работ, одна из которых появилась в журнале *Nature*, все поняли, что теоретически можно обучать нейронные сети какой угодно архитектуры, какой угодно глубины, с любым числом искусственных нейронов. И это тут же, конечно, породило вторую волну оптимизма насчёт возможностей нейронных сетей: во второй половине 1980-х многим исследователям снова казалось, что ещё немного — и андроиды начнут приносить кофе и разговаривать человеческим голосом.

На этот раз нельзя сказать, что оптимизм был столь же беспочвенным, как в 1960-х: в итоге оказалось, что нейронные сети действительно были способны революционизировать машинное обучение и изменить в том числе и нашу повседневную жизнь. Но в конце 1980-х не получилось: нейронные сети всё-таки не взлетели, и говорящие андроиды так и остались в мечтах.

Впрочем, вторая «зима искусственного интеллекта», накрывшая область во второй половине 1980-х и первой половине 1990-х годов, была связана не только с неудачами нейросетей, но и с вышеупомянутыми экспертными системами. К началу 1980-х они начали активно применяться в промышленности; первой

[1] *Джеффри Хинтон* (Jeffrey Everest Hinton, CC FRS FRSC, p. 1947) — британский, а затем канадский математик, информатик и когнитивист, один из «отцов» глубокого обучения. Хинтон — праправнук основоположника пропозициональной логики Джорджа Буля и известной обучающими книгами по математике Мэри Буль, а второе имя, Эверест, он получил в честь другого своего родственника, того самого сэра Джорджа Эвереста, именем которого названа высочайшая вершина мира. Хинтон продолжал заниматься искусственными нейронными сетями на протяжении всех «зим искусственного интеллекта», и его работа как раз приводила к тому, что «зимы» сменялись «вёснами»: в 1970-х Хинтон применил к нейросетям и популяризировал алгоритм обратного распространения ошибки, в начале 1980-х вместе с Дэвидом Румельхартом успешно применил его к глубоким нейронным сетям, тогда же изобрёл и долгое время развивал машины Больцмана, и многое, многое другое; в середине 2000-х именно он и его ученики произвели ту самую «революцию глубокого обучения». Хинтон является лауреатом премии Тьюринга (2018), награды Фрэнка Розенблатта IEEE (2014) и многих других важнейших научных наград. С 2023 года Джеффри Хинтон много высказывался о рисках, связанных с искусственным интеллектом, и даже ушёл из *Google*, чтобы иметь возможность свободно высказываться об опасностях развития AI. В октябре 2024 года Хинтон разделил с Джоном Хопфилдом Нобелевскую премию по физике — за «обучение искусственных нейронных сетей при помощи физики».

коммерческой экспертной системой был XCON (eXpert CONfigurer), разработанный Джоном Макдермоттом (John McDermott) в Университете Карнеги — Меллона для компании DEC. Как ясно из названия, система умела автоматически конфигурировать компьютеры по запросам пользователей, и она была очень успешной: по оценкам, за шесть лет применения она сэкономила для DEC 40 миллионов долларов (причём не сегодняшних, а долларов из начала 80-х). Однако этот пример так и остался одним из немногих положительных.

Росла популярность профильных конференций (главной из них тогда была AAAI, конференция Американской ассоциации искусственного интеллекта), и многим исследователям происходящее стало напоминать то, что они уже один раз пережили пятнадцатью годами ранее. На AAAI 1984 года была организована панельная дискуссия на тему «Тёмные века AI — можем ли мы их пережить?», где помнившие первую «зиму искусственного интеллекта» Роджер Шэнк и Марвин Минский предупредили представителей бизнеса, что AI-индустрия опять начинает выходить из-под контроля и скоро многие ожидания не оправдаются.

Через три года именно так и получилось: снова оказалось, что андроиды пока не очень готовы заменять живых людей в реальных интеллектуальных задачах, некоторые стартапы разорились, индустрия схлопнулась, на AAAI в 1991 году приехали всего 2000 участников вместо 6000, которые посетили AAAI — 1986, и началась вторая «зима искусственного интеллекта».

В 1990-е годы в нашей науке произошла важная смена парадигмы. Машинное обучение всегда, с самого Дартмутского семинара, считалось частью искусственного интеллекта, но именно частью, наряду с другими подходами. Например, экспертные системы вроде MYCIN — это искусственный интеллект, но в них нет никакого обучения, система просто комбинирует вероятности или другие оценки уверенности в разных логических высказываниях. Однако чем дальше, тем более очевидным становилось, что никто не сможет написать программу, способную разговаривать по-русски или уверенно отличать котиков от пёсиков: исследователи поняли, что эти задачи слишком сложны, чтобы напрямую придумать и запрограммировать их решения. Становилось ясно, что единственный путь к решению сложных задач искусственного интеллекта — это обучение на опыте.

А в 1990-е годы обучение ещё и начало становиться технически возможным: появлялись всё более мощные компьютеры, а главное, начали появляться достаточно большие наборы данных, на которых уже можно было обучить что-то интересное. Например, в 1988 году появился датасет TIMIT (точнее, DARPA-TIMIT Acoustic-Phonetic Continuous Speech Corpus), который распространялся на CD-ROM и содержал 6300 предложений (5,4 часа речи), по 10 от каждого из 630 дикторов [329, 669]. Раньше датасет такого размера было невозможно ни собрать, ни тем более распространить, и TIMIT фактически дал начало области автоматического распознавания речи. Машинное обучение стало занимать всё бóльшую и бóльшую часть искусственного интеллекта, и сейчас уже осталось не так много «живых» областей AI, которые не были бы напрямую связаны с обучением.

На ум приходят, пожалуй, только области представления и обработки знаний, извлечённых из людей, например те же экспертные системы или построение *онтологий* (ontology): как задать набор категорий, объектов, понятий и связей между ними в той или иной области, чтобы затем можно было представлять человеческие знания в этой области в более «автоматизированном» виде, пригодном для того же вероятностного вывода [206, 230, 447, 448, 570].

Другой важный тренд 1990-х и 2000-х годов — активное развитие вероятностного подхода к искусственному интеллекту и вероятностных моделей машинного обучения. Оказалось, что теория вероятностей — это тот самый язык, на котором задачи и методы искусственного интеллекта выражаются наиболее естественным образом и который может дать новые идеи и импульсы для развития моделей машинного обучения. Впрочем, здесь я не буду вдаваться в подробности — вся эта книга написана на языке теории вероятностей, так что без этого языка искусственного интеллекта мы точно не останемся.

С другой стороны, для искусственных нейронных сетей 1990-е годы и начало 2000-х оказались как раз второй «зимой»: хотя многие идеи и архитектуры нейросетей уже существовали, у исследователей не было ни достаточно больших датасетов, на которых нужно обучать глубокие нейронные сети, ни достаточно быстрых вычислительных устройств, чтобы это делать.

Ну а третья волна хайпа искусственного интеллекта, которая никак не хочет заканчиваться и сейчас, началась в середине 2000-х годов, когда в лабораториях Джеффри Хинтона, Йошуа Бенджи и Яна Лекуна вдруг заработали... те самые глубокие нейронные сети. Оказалось, что идеи, которые появились ещё в 1980-х, на самом деле могут прекрасно работать и открывать новые горизонты искусственного интеллекта. Нейросети требовали даже не столько новых идей (хотя их тоже было много), сколько новых технических возможностей: в частности, одним из главных двигателей прогресса оказалась идея перенести вычисления на видеокарты (GPU), чьи высокопараллельная архитектура и способность к матричным вычислениям как нельзя лучше подходят для обучения и применения нейросетей.

Но это уже совсем другая история, о которой в этой книге мы говорить не будем. Здесь я беспардонно сошлюсь на книгу «Глубокое обучение: погружение в мир нейронных сетей» [685], которая как введение в тему, думаю, вполне актуальна и до сих пор, и буду надеяться, что траектория этих книг выведет нас когда-нибудь на новый, сильно изменившийся даже за последние пять лет разговор о глубоком обучении.

1.3. Постановки задач искусственного интеллекта

Если есть вход, то есть и выход. Так устроено почти всё. Ящик для писем, пылесос, зоопарк, чайник... Но, конечно, существуют вещи, устроенные иначе. Например, мышеловка.

Харуки Мураками. Пинбол-1973
(пер. В. Смоленского)

1.3.1. Классификация задач искусственного интеллекта

В оставшихся разделах первой главы я попробую дать несколько разных классификаций того, чем занимается искусственный интеллект, какие задачи он решает (о том, *как* он это делает, мы поговорим во всей остальной книге). Цели и задачи искусственного интеллекта можно классифицировать разными способами, и каждый из них будет раскрывать предмет по-разному. Мы условно выделим три направления такого взгляда:

- в этом разделе мы обсудим *постановки задач* искусственного интеллекта, то есть будем классифицировать задачи по тому, какого типа данные у них на входе и выходе, а не по самой сути задач;
- раздел 1.4 будет посвящён *направлениям* приложения искусственного интеллекта; каждое направление объединено определённым типом входных данных — изображения, текст, звук, временные ряды и так далее — и, как следствие, имеет некоторую общность методов;
- а в разделе 1.5 мы поговорим о *приложениях* искусственного интеллекта, то есть конкретных областях науки и техники, где методы искусственного интеллекта позволяют решать разные интересные задачи, и дадим примеры таких задач; здесь, конечно, трудно будет претендовать на исчерпывающий обзор, так что мы ограничимся только научными применениями.

Слова «постановки задач», «области» и «применения», конечно, не являются индустриальным стандартом, мне просто нужно как-то обозначить эти разные взгляды. В результате каждую конкретную задачу мы сможем классифицировать с трёх разных сторон, не всегда строго укладываясь в одну-единственную корзинку. Например, задача распознавания рукописного текста состоит в том, чтобы по данному изображению рукописного текста выдать текст, там написанный, то есть последовательность символов. И вот что, забегая немного вперёд, получается:

- с точки зрения *постановки* это, скорее всего, либо задача классификации (выбрать верный символ, который подразумевался на данном участке текста), либо задача распознавания последовательности дискретных объектов

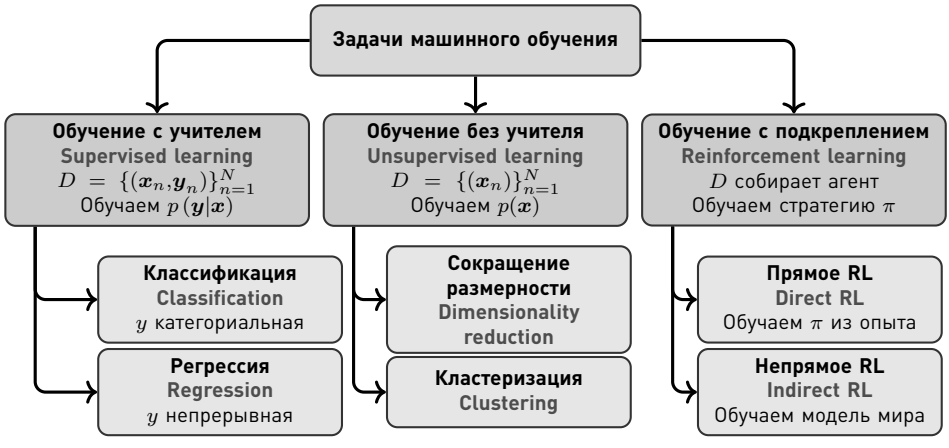


Рис. 1.9. Задачи машинного обучения

(если нужен сразу весь текст), но в любом случае почти наверняка задача обучения с учителем;

- с точки зрения *области приложений* это, очевидно, частный случай распознавания изображений, и основные методы здесь будут приходить из области компьютерного зрения (так, признаки из изображения в наше время почти наверняка будут выделяться нейронной сетью, либо свёрточной, либо на основе трансформера); но, возможно, здесь будет полезно некоторое вкрапление методов обработки естественных языков — например, языковая модель или даже простой словарь могут помочь в неоднозначных случаях, позволяя присвоить более высокие вероятности существующим словам по сравнению с несуществующими;
- а *применения* здесь могут быть и образовательные (автоматически оцифровывать и проверять контрольные работы), и медицинские (распознавать врачебные записи или рецепты), да и мало ли какие ещё — немного включив фантазию, вы легко придумаете миллион разнообразных применений.

Итак, в этом разделе давайте кратко пройдёмся по классификации задач машинного обучения, изображённой на рис. 1.9. Подобные картинки вы можете увидеть в любом учебном курсе по машинному обучению и даже в доброй половине презентаций, рассказывающих непосвящённым, что такое искусственный интеллект, за десять минут без регистрации и SMS.

Здесь важно понимать, что речь идёт именно о *задачах*, а не о методах их решения. Конечно, в быту можно неформально сказать: «Мы применили обучение с подкреплением», но на самом деле нет никакого единого «обучения с подкреплением», которое можно было бы «применить», можно только поставить задачу

обучения с подкреплением (и то, конечно, задач там великое множество и все разные), а потом попытаться решить её одним из многочисленных существующих методов, а то и придумать новый. А применить можно уже один из этих методов, конкретную модель. Например, в главе 4 мы обсудим, что значит «применить линейную регрессию»; там мы увидим, что это, конечно, тоже недоопределённое высказывание, вариантов у линейной регрессии много, но всё же некоторое базовое сходство и общие предположения у них есть. А вот «мы применили обучение с подкреплением» — примерно настолько же бессмысленная фраза, как «мы применили искусственный интеллект».

Тем не менее изображённая на рис. 1.9 классификация всё равно очень полезна. Просто классифицирует она не методы решения и модели, а *постановки задач*. Это классификация того, что наш чёрный ящик модели машинного обучения имеет на входе и выходе, а не его внутреннего устройства. Итак, давайте понемногу углубляться.

1.3.2. Обучение с учителем: регрессия и классификация

Обучение с учителем (supervised learning) — это задачи, в которых есть некоторый правильный ответ, целевая переменная, которую нужно предсказать. Датасет[1] здесь выглядит как $D = \{(\boldsymbol{x}_n, \boldsymbol{y}_n)\}_{n=1}^{N}$, то есть каждая точка датасета — это пара из входа $\boldsymbol{x}_n$ (вектора признаков) и выхода y_n, той самой целевой переменной. Забегая немного вперёд, скажу, что в вероятностном смысле мы здесь обучаем *условное* распределение $p(\boldsymbol{y}|\boldsymbol{x})$ и, даже если вдруг делаем это через порождающую модель, в итоге всё равно приводим всё к распределению целевой переменной при условии вектора признаков.

Задачи обучения с учителем условно делятся на два класса в зависимости от того, какой природы целевая переменная y: если она непрерывная, например $y \in \mathbb{R}$, то это задача *регрессии*, а если дискретная, то есть вход $\boldsymbol{x}$ нужно определить в один из нескольких классов, это задача *классификации*. Например, если вы пытаетесь предсказать рост ребёнка в зависимости от роста родителей, как сэр Фрэнсис Гальтон (см. раздел 4.1), то вы решаете задачу регрессии, а если различаете котиков и пёсиков на фотографиях (задача, кстати, отнюдь не простая, мы до неё в этой книге явно не доберёмся), вы решаете задачу классификации[2].

Разная природа целевых переменных приводит к тому, что задачи регрессии и классификации обычно требуют разного подхода даже на базовом уровне. В за-

[1] Он же набор данных, он же выборка; но я буду в этой книге иногда использовать и английскую кальку, очень уж прочно она вошла в (простите ещё раз) датасайентистский узус.

[2] Для въедливых читателей отмечу, что с поиском объектов на изображениях дело обстоит более тонким образом. Если вы хотите просто решить, котик на картинке или пёсик, — это, конечно, классификация. Если вы хотите ещё и обвести найденного котика или пёсика прямоугольничком, то есть решить задачу *распознавания объектов* (object detection) — это комбинация классификации и регрессии, ведь прямоугольник логично будет представлять четырьмя координатами. А если вы хотите решить задачу *сегментации* (segmentation), то есть выделить не грубый прямоугольник, а точный силуэт котика... то это опять классификация, но теперь вам нужно классифицировать каждый пиксель изображения: принадлежит ли он объекту и если да, то объекту какого класса.

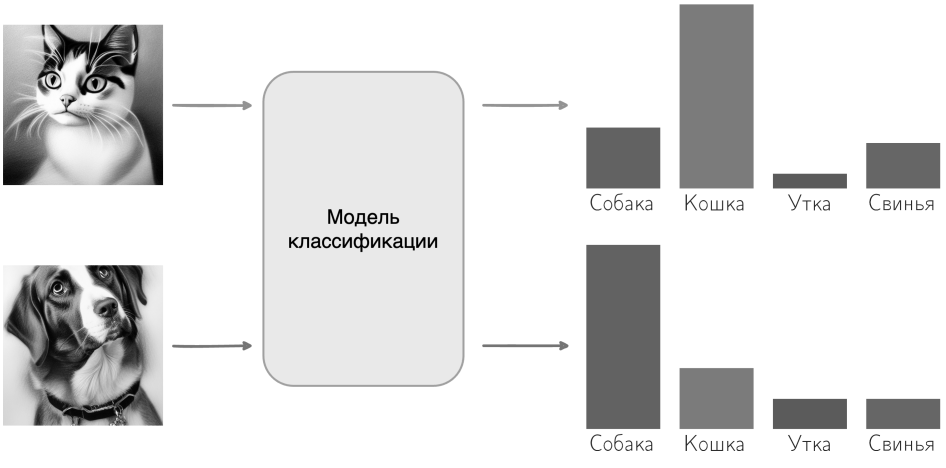


Рис. 1.10. Постановка задачи классификации

дачах регрессии мы строим модель значения целевой переменной $y \approx f(\boldsymbol{x})$, а затем пытаемся оценить, насколько хорошо у нас получилось, какое именно распределение шума скрывается в значке $\approx$, что можно сказать о новых значениях $f(\boldsymbol{x})$ и так далее. А в задачах классификации обычно проще напрямую строить модель вероятностей разных классов $p\,(C_k|\boldsymbol{x})$. Мы увидим, к чему приводит это различие, в главах 4 и 5, посвящённых соответственно линейной регрессии и линейным моделям для классификации, в частности регрессии логистической.

В целом, когда мы решаем задачу классификации, нам обычно хочется не просто получить дискретное значение наиболее вероятного класса, а оценить всё распределение вероятностей на различных классах. На рис. 1.10 показан пример постановки задачи классификации для четырёх классов: «Собака», «Кошка», «Утка» и «Свинья»; для каждого входа мы хотим получить распределение вероятностей на этих четырёх классах, и при этом, конечно, хочется, чтобы правильный ответ выигрывал у других. В реальности это значит, что функции ошибки (которые, как мы увидим в главах 2 и 4, обычно происходят в машинном обучении из задач максимизации правдоподобия или апостериорного распределения модели) у задач регрессии и классификации будут разными.

Есть и в некотором смысле промежуточные случаи между регрессией и классификацией. Например, в начале этого раздела я упомянул распознавание рукописного текста, где выходом обычно является распознанная *последовательность* слов или символов; пусть для определённости это будет одно предложение на русском языке. Конечно, текст — это дискретный объект, и формально можно сказать, что мы в этой задаче будем присваивать различные вероятности всевозможным предложениям на русском языке некоторой ограниченной длины. Но на

практике предложений явно слишком много, и хотя многие модели действительно смогут выдать некоторую оценку вероятности одной последовательности, перечислить их все, как в обычной задаче классификации, явно не получится. Здесь нужны другие методы, в которых последовательность на выходе получается символ за символом или слово за словом (слов тоже много, но это уже вполне подвластное современному машинному обучению «много»). В машинном обучении много разных задач, связанных с последовательностями.

Другой важный промежуточный случай — это *обучение ранжированию* (learning to rank). В этой постановке мы, казалось бы, обучаем непрерывную целевую переменную, но на самом деле её значение нам неважно и, скорее всего, неизвестно, а важен только относительный порядок этих переменных: когда $y_n > y_m$, а когда — наоборот. Здесь модельный пример — это интернет-поиск: когда *Google* выдаёт документы по запросу, он, конечно, сортирует их по значению некоторой функции релевантности. Но значений этой функции никто не знает, и никакого физического смысла эти значения не имеют, важно только, у какого документа релевантность больше. Размеченные людьми датасеты в этой области тоже не содержат значений функции релевантности: они либо бинарные («тут релевантно, тут не релевантно»), либо сравнительные («этот документ более релевантен запросу, чем тот» или «этот документ релевантен на три звёздочки из пяти»).

1.3.3. Обучение без учителя

Обучение без учителя (unsupervised learning) — это задачи, в которых никаких правильных ответов нет, а датасет представляет собой просто набор примеров $D = \{(\boldsymbol{x}_n)\}_{n=1}^{N}$; грубо говоря, есть набор фотографий, но непонятно, котики там, пёсики или лошадки. Что же тогда нужно сделать, спросите вы? Скорее всего, речь идёт о том, чтобы обучить некоторое *представление* (representation) этих данных, например сжать фотографии $\boldsymbol{x}$ размером 1024×1024 пикселя до вектора из 500 чисел. Это может быть нужно для того, чтобы потом использовать полученные признаки для обучения с учителем, а может быть нужно для того, чтобы потом исследовать структуру полученного представления и выделить какие-то общие черты разных подмножеств датасета.

В обучении без учителя тоже имеет смысл разделить непрерывный и дискретный случаи. В непрерывном случае, который обычно и называется *обучением представлений* (representation learning), речь идёт о том, чтобы обучить отображение из пространства, в котором обитают точки $\boldsymbol{x}_n$, в какое-то другое пространство, обычно меньшей размерности. Условный пример такого понижения размерности показан на рис. 1.11: изображения кошек и собак отображены в размерность 2 так, чтобы одна из осей отражала класс объекта (собака это или кошка), а другая ось отражала стиль изображения (рисунок или картина маслом). Разумеется, в реальных применениях размерность представлений будет гораздо больше — сотни или тысячи, но не миллионы, как в пространстве пикселей — и не все размерности будет так же легко интерпретировать, как на рис. 1.11.

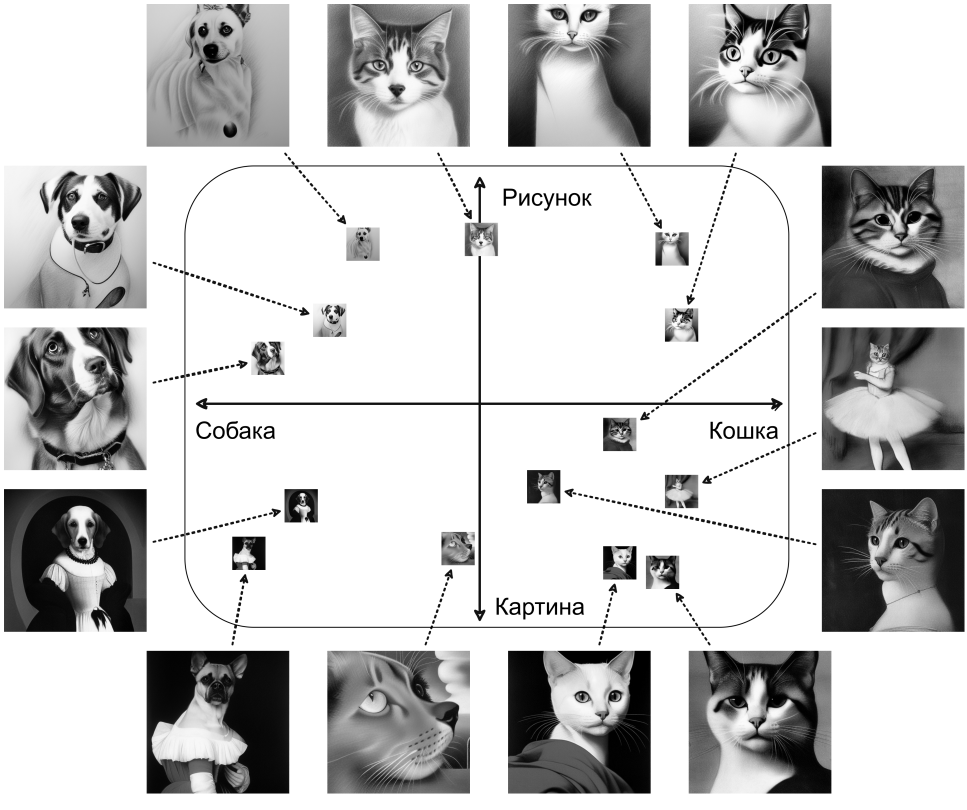


Рис. 1.11. Условный пример задачи обучения без учителя: понижение размерности

А дискретный случай здесь называется *кластеризацией* (clustering): мы отображаем каждую точку $\boldsymbol{x}_n$ в одну из нескольких дискретных меток, номеров кластеров, то есть, по сути, просто делим D на несколько подмножеств так, чтобы внутри каждого подмножества точки были похожи друг на друга, а между элементами разных подмножеств была бы как можно более существенная разница. Кластеризацию с вероятностной точки зрения мы в этой книге не обсудим, а обучение представлений в наших моделях предполагается уже сделанным: в линейных моделях, о которых мы в основном говорим на протяжении всей этой книги, мы предполагаем, что признаки уже выделены (см. раздел 1.4).

В жизни нередко встречаются и промежуточные варианты между обучением с учителем и без. Как правило, они возникают, когда неразмеченные данные получить относительно легко и/или дёшево, а вот разметить их уже гораздо сложнее. Представьте себе, например, задачу *сегментации*, в которой нужно не просто классифицировать фотографию как «котика» или «пёсика», а ещё и обвести силуэт этого котика. Мы уже обсудили выше, что это задача обучения с учителем,

но на практике получается неприятная ситуация: набрать много фотографий котиков и пёсиков не так уж сложно, а вот точно обвести на каждой из них вручную силуэт — это адский труд. Поэтому в такой ситуации часто встречаются датасеты, состоящие из двух частей: маленькой размеченной и большой неразмеченной. Если мы пытаемся использовать одновременно и размеченную, и неразмеченную части, такая постановка задачи называется *обучением с частичным привлечением учителя* (semi-supervised learning; перевод ужасно неуклюжий, но уже практически стандартный, и лучше я не придумал).

Третий вид машинного обучения на рис. 1.9, *обучение с подкреплением* (reinforcement learning) [582], требует отдельного изложения, которого в этой книге не будет. Основная идея такая: и обучение с учителем, и обучение без учителя подразумевают, что у нас перед началом обучения есть набор данных в той или иной форме, размеченный или не слишком. Но в жизни ведь обучение устроено совершенно не так! Когда мы в детстве учимся ходить или когда мы изучаем машинное обучение, мы вовсе не следуем описанной выше схеме: не занимаемся сначала сбором датасета и его разметкой, а затем загружаем полученный набор данных в какой-то «верховный алгоритм» в мозге. На самом деле мы учимся (хотя бы отчасти) методом проб и ошибок, собирая датасет за счёт удачных и не очень удачных попыток и параллельно с этим обучаясь делать попытки всё более удачными[1]. Именно этим занимаются методы обучения с подкреплением: они рассматривают агента, действующего в окружающей среде, который постепенно накапливает информацию об этой среде и обучает стратегию, которая даст наибольший выигрыш. Но более подробное обсуждение обучения с подкреплением нам придётся отложить до, надеюсь, других книг, а в этой мы будем предполагать, что датасет всё-таки существует.

[1] Кстати, вот вам интересный вопрос для самостоятельного размышления: когда вы просто читаете эту книгу, не пытаясь активно решать задачи или пробовать методы на практике, в каком смысле вы «обучаетесь»? Что на входе и что на выходе этой задачи обучения? И нет, я не просто пытаюсь намекнуть, что параллельно с чтением обязательно надо активно решать задачи и что-то программировать, хотя это было бы, конечно, полезно! Мне кажется, что ответ на эти вопросы вполне себе существует, и мы с вами действительно обучаемся, когда читаем книги. Но как это формально уложить в нашу науку?

1.4. Области искусственного интеллекта: какие бывают данные

Чем мы занимаемся в жизни? Собираем данные, приобретаем, разлагаем, сводим, поверяем, сравниваем, перерабатываем, испытываем и, так сказать, дистиллируем умом эти данные. Позвольте мне поговорить с вами о себе. Из всех данных самое главное я. Оно не приобретается, а даётся Богом, при создании человека; как чувство бытия, растёт, растёт и достигает до самосознания. Худо или лучше, однакоже это я – единственное наше орудие для собрания данных и обращения с ними. Не правда ли, что всякий из нас имеет право говорить о себе?.. Но далее я уже не пойду, и ждать более нечего: надобно осмелиться и по крайней мере выработать себе решительную уверенность в неспособности и потом спокойно, молча, добраться до могилы.

Гавриил Батеньков.
Данные: повесть собственной жизни

1.4.1. Извлечение признаков в машинном обучении

В предыдущем разделе мы (приблизительно) выяснили, какие бывают общие постановки задач машинного обучения, то есть в некотором смысле классифицировали, какие бывают целевые переменные y (непрерывные, дискретные, вообще не существующие...) и общие формы датасетов. Теперь давайте попробуем сделать следующий шаг и — опять же условно и приблизительно — классифицируем то, какие бывают типы данных, на которых обучаются модели искусственного интеллекта. Проще говоря, в паре $(\boldsymbol{x}, y)$, из которых состоят датасеты обучения с учителем, раздел 1.3 представлял классификацию y, а в этом разделе мы поговорим об $\boldsymbol{x}$.

В этой книге, где речь идёт в основном о линейных моделях (особенно линейной и логистической регрессии), мы обычно будем предполагать, что $\boldsymbol{x} \in \mathbb{R}^d$, то есть на входе у нас вектор признаков, состоящий из нескольких чисел. Но при этом, как только я начинаю приводить примеры задач из реального мира, там сразу же появляются самые разные данные. Например, выше я говорил о типичной задаче классификации, различении котиков и пёсиков на фотографиях. Конечно, фотографию тоже можно считать вектором чисел: каждый пиксель — это три числа, которые можно условно нормировать к отрезку $[0,1]$ и считать непрерывными. Но если вы попытаетесь обучить, скажем, логистическую регрессию (мы

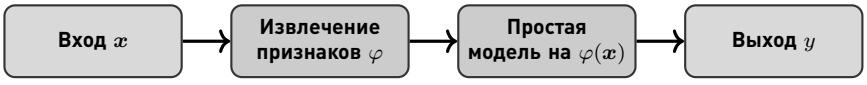


Рис. 1.12. Извлечение признаков для моделей машинного обучения

подробно обсудим эту модель в главе 5) на входных векторах из трёх миллионов чисел (скажем, один мегапиксель умножить на три цвета), у вас не то что котиков от пёсиков не получится отличить, но и куда более простые задачи вряд ли решатся.

Дело не в том, что модели, которые мы будем рассматривать в этой книге, слишком просты и потому бесполезны. Напротив, любой сколь угодно сложный метод машинного обучения, скорее всего, в конечном счёте использует какую-нибудь достаточно простую модель. Например, почти любая глубокая нейронная сеть, решающая задачу классификации, в качестве последнего слоя, предсказывающего вероятности классов, использует именно логистическую регрессию. Но эти простые модели получают на вход не сами данные, а их *скрытые представления* (latent representations), *признаки* (features), которые уже действительно обычно представляют собой векторы вещественных чисел, $\varphi(\boldsymbol{x}) \in \mathbb{R}^d$.

Соответственно, обработка сложных данных обычно устроена так, как показано на рис. 1.12. В этой книге мы будем в основном обсуждать то, что происходит в пункте «Простая модель на $\varphi(\boldsymbol{x})$». А самая сложная и интересная магия машинного обучения происходит в пункте «Извлечение признаков φ». В частности, революция глубокого обучения, которую я упоминал в разделе 1.2, в основном состояла в том, что для многих задач нейронные сети позволили обучать выделяющие признаки преобразования φ автоматически, а не пытаться их мучительно подбирать вручную или развивать смежные дисциплины, чтобы лучше научиться выделять признаки. Оказалось, что автоматически обученные признаки часто работают лучше и для многих типов входных данных их можно обучаться выделять одновременно для нескольких конкретных задач, на каком-нибудь большом глобальном датасете.

1.4.2. Оверфиттинг и разные части датасета

И ещё одно очень важное общее замечание о данных, которые используются в машинном обучении. Как правило, практическая цель всего происходящего состоит в том, чтобы научиться что-нибудь *предсказывать*; в обучении с учителем мы обычно предсказываем целевую переменную y по набору признаков $\boldsymbol{x}$. Как проверить, насколько хорошо у нашей модели это получается? На первый взгляд, можно просто подсчитать её предсказания и сравнить с правильными ответами, тоже содержащимися в данных.

Но здесь возникает проблема, с которой мы ещё много раз встретимся в этой книге: нельзя проверять модель на тех же данных, на которых она обучалась! Точнее, подсчитать метрику качества, конечно, можно, но её нельзя использовать для

того, чтобы сравнивать разные модели между собой: в результате выиграет модель, которая лучше всего «оближет» данные, предоставленные для обучения, а не та модель, которая лучше всего будет обобщаться на новые точки (в чём и состоит наша цель). Так, идеальной моделью машинного обучения с точки зрения такой проверки будет база данных, в которую мы поместим обучающий датасет: на любой точке из датасета ответ будет идеально точным, но, конечно, такая «модель» совершенно бесполезна для предсказаний. Этот эффект называется *оверфиттингом* (overfitting)[1]; он возникает тогда, когда у модели появляется слишком много параметров, она становится слишком выразительной для имеющихся данных. Однако, конечно, заранее никто вам не скажет, сколько это — «слишком много»...

Как же избежать оверфиттинга? Самый простой и постоянно встречающийся на практике способ — проверять модели не на тех данных, на которых они обучались, а на каких-нибудь других. Для этого обычно имеющийся датасет делят на две или три части:

- *обучающий набор*, или *обучающая выборка* (training set, training sample), — это обычно самая большая часть, на которой все участвующие в проекте модели будут собственно обучаться;
- *валидационный набор*, или *валидационная выборка* (validation set, validation sample), — это часть датасета, на которой мы будем выбирать конкретную модель из некоторого класса, то есть подбирать так называемые *гиперпараметры* (hyperparameters); например, в разделе 4.2 мы рассмотрим линейную регрессию с регуляризацией, и сила этой регуляризации (дисперсия априорного распределения) окажется некоторым числом α, которое нужно выбрать — вот именно для этого выбора и может пригодиться валидационный набор; если гиперпараметры подбирать не надо и модели существуют в единственном варианте, то валидационный набор можно и не выделять;
- *тестовый набор*, или *тестовая выборка* (test set, test sample), — это данные, на которых мы будем сравнивать модели между собой; здесь самое главное в том, что на тестовый набор никак нельзя смотреть при обучении или выборе моделей, эти данные должны остаться неприкосновенными.

В типичных проектах классического машинного обучения датасет делят на обучающий, валидационный и тестовый наборы в пропорции вроде 90:10:10; впрочем, бывает и так, что тестовый набор оказывается очень большим. А в соревнованиях по анализу данных, например, на самом известном соответствующем портале *Kaggle*, тестовый набор обычно просто скрывают от греха подальше —

[1] По-русски слово *overfitting* иногда переводят как «переобучение», но мне этот перевод кажется неудачным, сбивающим с толку: «переобучение» в русском языке — это давно существующее слово, которое означает обучение заново, обучение чему-то другому (re-training), а здесь речь идёт о «слишком сильном обучении». Поэтому здесь я предпочитаю заимствовать английское слово напрямую.

разумеется, если выдать его участникам, результатом будет сильнейший оверфиттинг на тестовое множество.

Начиная с главы 4, оверфиттинг станет одним из важнейших мотиваторов происходящего в этой книге. А сейчас давайте вернёмся к разным типам данных, встречающимся в машинном обучении.

1.4.3. Табличные данные

Классификация задач машинного обучения, которую я хочу обсудить в этом разделе далее, связана с разными типами входных данных. Самый простой случай — это $\boldsymbol{x} \in \mathbb{R}^d$ для не слишком большого d (десятки или сотни, а не миллионы), где разные компоненты вектора $\boldsymbol{x}$ можно считать не слишком зависимыми друг от друга (знаю, что это звучит не очень строго; «независимость» здесь, конечно, понимается не в буквальном вероятностном смысле). Этот случай мы будем обсуждать в разделах, посвящённых «простым» моделям машинного обучения. Обычно такие данные получаются либо в результате выделения признаков, либо в задачах, где размерность изначально была невелика.

Первый тип данных, которые часто встречаются в реальных практических задачах, — это *табличные данные*. Например, система, оценивающая вероятность возврата кредита, получает на вход некоторый набор параметров самого кредита и человека, который на этот кредит претендует. Скорее всего, этих параметров не миллионы, а десятки, максимум сотни, и они уже представлены в виде набора некоторых признаков: сумма, срок и процент кредита, доход, возраст и место проживания человека и так далее.

Казалось бы, это самая простая ситуация для машинного обучения: признаки за нас уже кто-то извлёк и свёл в табличку, можно просто обучать модели для классификации и регрессии напрямую. Но здесь тоже возникает большой простор и для исследований, и для выработки нетривиальных практических рецептов. Табличные данные почти всегда требуют некоторой *предобработки*: какие-то численные признаки лучше нормализовать, то есть привести к нулевому среднему и единичной дисперсии (эти понятия мы обсудим в главе 2), какие-то варианты дискретных признаков часто можно объединить в один. Например, на первый взгляд кажется, что возраст человека — это число, но на практике часто имеет смысл превратить возраст в дискретную, ординальную переменную с конечным небольшим набором значений вроде 0–17, 18–34, 35–54, 55+.

Такой конвертацией предобработка, конечно, не ограничивается: часто приходится изобретать нетривиальные способы извлекать из данных новые интересные признаки, и это важная часть искусства, простите за такое слово, датасайентистов.

Другая проблема, связанная с табличными данными, состоит в том, что в одной и той же таблице часто объединяются данные совершенно разной природы.

Например, в случае оценки кредитного риска почти наверняка на вход нам подадут в качестве признаков и числа (например, сумму кредита), и ординальные переменные (тот же возраст) одновременно. Хотя формально мы всегда можем, конечно, выразить ординальную переменную вектором чисел (просто запишем вектор из нулей с одной единицей в нужном классе — это называется прямым кодированием, one-hot encoding), далеко не всем моделям машинного обучения будет легко обработать такой смешанный вход. Отчасти этим объясняется популярность на практике моделей, основанных на деревьях принятия решений (decision trees), в частности методов бустинга (boosting) на деревьях [197, 198, 515] или случайных лесов (random forests) [76] — они как раз прекрасно обрабатывают разные типы данных; впрочем, до бустинга мы в этой книге не доберёмся.

1.4.4. Обработка последовательностей

Следующий шаг к увеличению размерности и сложности входных данных — это *анализ временных рядов* (time series analysis), то есть обработка *последовательностей*, состоящих из векторов признаков. В этом случае мы снова, скорее всего, предполагаем, что в каждый момент времени t вектор $\boldsymbol{x}_t$ имеет не слишком большую размерность, но теперь входом является целая последовательность таких векторов $\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_T$, и её нельзя просто разбить на составные части, предположив, что $\boldsymbol{x}_t$ и $\boldsymbol{x}_{t+1}$ независимы. Как правило, такие данные представляют собой последовательности значений одних и тех же переменных в разные моменты времени, отсюда и название.

Бывают и последовательности другой природы. Одна из важнейших областей искусственного интеллекта — это *обработка естественных языков* (natural language processing). Текст на естественном языке — это последовательность дискретных объектов, токенов; можно формализовать текст как последовательность слов, а иногда есть смысл спуститься на уровень ниже и рассматривать последовательности символов[1]. Кстати, в наше время из этих дискретных объектов очень часто извлекают непрерывные признаки, превращая каждое слово в вектор его *распределённого представления* (distributed representation, embedding), состоящий из вещественных чисел. Но в любом случае большинство задач обработки естественных языков — это задачи обработки последовательностей. В книге вы увидите много примеров из этой области.

Ещё один вид последовательностей, который может требовать особого отношения, — это *звук*. Конечно, по сути звук представляет собой дискретный временной ряд: амплитуда звуковой волны меняется во времени, и её значения записываются в звуковой файл с некоторой частотой дискретизации (те самые килогерцы, которых 44,1 в стандарте CD Audio). Но суть этого временного ряда и, как следствие, принципы работы с ним совсем не похожи на, скажем, финансовые

[1] В современных моделях обычно используют более сложные методы токенизации, выделяя часто повторяющиеся подпоследовательности символов; например, стандартный на сегодня метод byte-pair encoding делает это буквально кодированием по Хаффману [204, 526].

временные ряды. Обработка звука (sound processing) и извлечение признаков из аудио всегда требовали специальных методов *цифровой обработки сигналов* (digital signal processing); это большая наука, которую в этой книге мы не будем затрагивать. Впрочем, современные нейросетевые подходы автоматизируют извлечение признаков и из звука. А если соединить анализ звука и обработку естественных языков, получится ещё одна большая область приложений искусственного интеллекта — работа с человеческой речью. Это в основном *распознавание речи* (automatic speech recognition, ASR), то есть перевод звука в текст, и *синтез речи* (text-to-speech), то есть трансляция текста в звук. Современные подходы в этой области объединяют обработку текста и звука в единой модели [174, 342, 481, 500]; такие подходы называются *мультимодальными* (multimodal), потому что они объединяют разные *модальности* (modalities), то есть типы входных данных.

1.4.5. Обработка изображений и более сложных типов данных

В предыдущем разделе на вход подавались последовательности, то есть, хотя вход у модели не был одномерным (например, каждое слово текста по умолчанию представляется вектором длиной в размер словаря), сложные зависимости в нём предполагались только по одной размерности, по «времени». Давайте сделаем следующий шаг: от размерности 1 к размерности 2. Самые важные и часто встречающиеся двумерные данные — это, конечно же, картинки, так что после этого шага мы сразу же попадаем в область *обработки и анализа изображений* (image processing, image analysis).

Начнём с анализа. Одной из важнейших областей искусственного интеллекта всегда было и остаётся *компьютерное зрение* (computer vision), попытки научить компьютер видеть своё окружение и понимать, что там происходит [109, 477, 573, 584]. Собственно компьютерное зрение важно в первую очередь для роботов, которым нужно осуществлять навигацию в пространстве (роботов в широком смысле, автомобильный навигатор сюда тоже входит). В частности, движущемуся в пространстве роботу почти всегда приходится решать задачу SLAM — simultaneous localization and mapping, то есть понимать своё собственное положение в пространстве и строить карту окружающего мира [107, 574]; это сложная задача, состоящая из многих частей, но в первую очередь, конечно, включающая в себя анализ изображений с камер.

Сам анализ изображений и понимание того, что на них происходит (scene understanding) [134, 575], тоже очень важно, как напрямую (скажем, для автономного вождения), так и опосредованно: фотографии и другие картинки составляют огромную часть нашего мира, особенно мира виртуального, и научиться их понимать и обрабатывать может очень помочь, например, рекомендательным системам. Кстати, некоторые методы даже звук рассматривают как изображение, подавая на вход работающим с картинками моделям спектрограмму звукового сигнала [649].

А *обработка изображений* (image processing) или их порождение — это тип задач, где картинка не только на входе, но и на выходе; в таких задачах мы, скорее всего, пытаемся как-то изменить заданное изображение [27, 281, 289, 648]. Например, всевозможные instagram-фильтры или просто встроенные в смартфон приложения фотокамеры — это обычно результат обучения достаточно сложных моделей искусственного интеллекта. Есть и большая наука о *порождающих моделях* (generative models) для изображений [80, 105, 598]; такие модели уже сегодня умеют создавать весьма реалистичные фотографии и рисовать картины в любом стиле, и с каждым годом их результаты улучшаются. В частности, должен признаться, что рисовать я совершенно не умею, а все рисунки в этой книге, на источники которых нет конкретных ссылок, в частности иллюстрации в начале каждого раздела, созданы современными версиями моделей DALL-E [274] и Stable Diffusion [259].

Что если увеличивать размерность дальше? Во-первых, конечно, от двумерных изображений можно перейти к трёхмерным: лидары [501] и time-of-flight камеры [191] вполне могут предоставлять роботу трёхмерную информацию об окружающем мире, хоть порой и зашумлённую — но это лишь ещё одна причина использовать машинное обучение [20, 106, 576]. Другой важный пример трёхмерной информации — медицинские снимки (см. подраздел 1.5.3); компьютерная и магниторезонансная томография применяются всё шире, и автоматический анализ для медицинских снимков особенно важен, потому что врачу-человеку становится всё сложнее обрабатывать возрастающие объёмы трёхмерных данных [23, 141, 533, 662].

Во-вторых, третья координата может быть не пространственной; это опять может быть время, что приводит нас к *анализу видео* (video analysis). В этой области сходятся методы анализа изображений и методы анализа последовательностей или временных рядов. Видео можно просто анализировать [194, 623], например, чтобы рекомендовать вам новые ролики в интернете, можно искать там объекты, в частности на видео с камер наблюдения [548, 628, 654], а можно и модифицировать или даже создавать заново; порождающие модели для видео — это модная и тоже очень быстро развивающаяся область [102, 272, 549].

Это, конечно, далеко не всё, бывают и более специализированные типы данных. Например, входные данные для базовых *рекомендательных систем* (то есть тех, которые не знают, какие именно продукты рекомендуют, и не пытаются анализировать их контент) представляют собой матрицу с оценками или «лайками», которые пользователи выставили тем или иным продуктам; задача *коллаборативной фильтрации* (collaborative filtering) здесь состоит в том, чтобы предсказать значения оценок, которых в матрице не хватает, дозаполнить её [14, 315, 316, 572, 577]. Но в целом более специфические типы данных уже будут приводить нас к конкретным применениям искусственного интеллекта в реальной жизни; давайте об этом и поговорим.

1.5. Как машинное обучение помогает другим наукам

Студенты строгие сидят
В очках и с умными глазами
Серьёзно на меня глядят
И я как мышка сразу замер
Испуганно
Я — умный, одарённый, славный! —
О, Господи, для пущей славы
Твоей
Но и моей, в качестве презентанта Тебя
Убей их!
Ну, конечно, не окончательно, а так — припугни
Чтобы знали.

Дмитрий Александрович Пригов

1.5.1. Физика и астрономия

Думаю, на этом месте у вас уже не осталось сомнений, что искусственный интеллект — это отнюдь не только про роботов, чатботы и восстание машин. Но в этом разделе мы ещё более ясно увидим, насколько широки реальные приложения искусственного интеллекта (в основном, конечно, машинного обучения) и как много разных областей человеческой деятельности они затрагивают. Мы поговорим о том, как машинное обучение помогает другим наукам: как «царица наук» математика используется во всех остальных дисциплинах, так и искусственный интеллект уже сейчас позволяет получать массу новых научных результатов в самых разных областях. Давайте просто пойдём по списку... для начала хотя бы школьных предметов, хотя ближе к концу списка появятся и не совсем школьные дисциплины. Разумеется, изложенное ниже — это лишь небольшой набор примеров, а вовсе не исчерпывающий список.

Может показаться, что искусственный интеллект — это про что-то неточное, недоопределённое и «человеческое», но оказывается, что и в точных науках AI очень даже пригождается. Во-первых, конечно, искусственный интеллект неотделим от статистики, которая является основной частью любой обработки экспериментальных данных; например, линейная регрессия, которую мы обсудим в главе 4, применяется во всех экспериментальных дисциплинах для того, чтобы оценить влияние различных факторов на целевые переменные (когда это влияние предполагается линейным, конечно).

Но есть и более специальные и интересные приложения. Например, в последние десять лет глубокие нейронные сети нашли много применений в квантовой физике и физике элементарных частиц. Вы, скорее всего, слышали об открытии

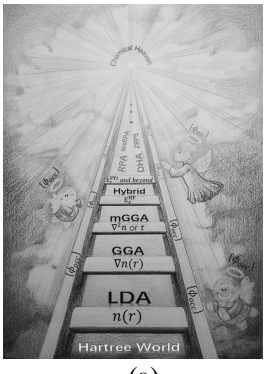
бозона Хиггса в Большом адронном коллайдере [3, 4, 46]. Как делаются такие открытия и где здесь машинное обучение?

В коллайдере есть огромное число различных датчиков, информация с которых должна либо «укладываться» в теоретические предсказания, либо нет, и если нет, то давать возможность корректировать параметры и форму модели. Машинное обучение помогает здесь в обоих направлениях: для прямых задач, то есть предсказания и симуляции процессов, можно обучать порождающие модели машинного обучения, которые будут аппроксимировать реальные данные. А обратные задачи физики, то есть задачи оценки параметров по данным эксперимента — это буквально задачи машинного обучения и есть! И современная физика элементарных частиц активно пользуется для этого самыми последними достижениями искусственного интеллекта [179, 363].

Здесь есть один большой класс похожих друг на друга приложений машинного обучения, который хочется выделить особо. Во многих областях физики (да и других наук, как мы увидим ниже) численные расчёты требуют огромных вычислительных мощностей, которых у людей в требующихся объёмах просто нет. В таких случаях машинное обучение может помочь аппроксимировать эти сложнейшие расчёты: учёные собирают датасет, состоящий из результатов сложных расчётов, и обучают модель, вычисления в которой будут гораздо проще, приближать результаты этих расчётов. Такой подход уже привёл к успеху в квантовой хромодинамике [527], физике конденсированного состояния [340], квантовой задаче многих тел [90] и многих других областях.

С другой стороны, астрономия всегда была наукой о больших данных: ещё Гиппарх и Птолемей изобретали свои эпициклы не исходя из философских или религиозных соображений, а чтобы объяснить данные астрономических наблюдений. В современной астрономии этих наблюдений накапливается огромное количество; например, строящийся сейчас самый большой в мире радиоинтерферометр SKA (Square Kilometer Array) будет производить около одного экзабайта данных в день, больше, чем производит весь интернет, вместе взятый; говорят, правда, что их можно будет сжать до десяти петабайт, но это тоже не так уж мало. Поэтому даже сама техническая обработка этих гигантских массивов данных — уже большая и сложная наука, тесно связанная с искусственным интеллектом [65].

После технической обработки наступает этап собственно анализа этих самых больших данных. Для этого тоже, конечно, используется машинное обучение, причём здесь это скорее исследование данных (exploratory analysis), поиск паттернов или просто поиск «чего-нибудь интересного». Один из важных примеров — поиск экзопланет, то есть в каком-то смысле поиск планет, похожих на Землю, у других звёзд. Экзопланеты обычно ищут на основе кривых блеска звёзд (light curves): когда планета проходит перед звездой, это вызывает небольшое снижение её видимого блеска. Методы машинного обучения позволяют идентифицировать кривые блеска с интересными артефактами, похожими на прохож-



(а)

# атомов	Графов	Углеводородов	Молекул
1	1	1	3
2	1	1	6
3	2	2	14
4	6	4	47
⋮	⋮	⋮	⋮
15	1 496 007 875	960 415	15 084 103 347
16	12 176 341 897	1 331 875	38 033 661 355
17	100 418 784 003	1 583 786	109 481 780 580
$\sum$	**114 304 569 097**	**5 422 154**	**166 443 860 262**

(б)

Рис. 1.13. Задачи для искусственного интеллекта в химии: (а) лестница Иакова в теории функционала плотности [223, 661]; (б) число молекул в базе GDB-17 в зависимости от числа тяжёлых атомов [94, 492]

дение экзопланет, то есть выделить из сотен тысяч звёзд, информация о которых собирается телескопами, самых перспективных кандидатов [460]. Это, конечно, лишь один пример: машинное обучение используется для анализа морфологии галактик [114], поиска галактик-спутников Млечного Пути [631], распознавания квазаров [126] и так далее.

1.5.2. Математика

Здесь мне, как математику по образованию (во введении я уже упоминал, что начинал свой путь в науке как алгебраист [209, 435, 439, 619]), хочется поговорить чуть подробнее. Результаты применения искусственного интеллекта в математике меня всегда удивляли, причём со знаком минус. Начиная с того самого момента, когда я впервые осознал основы математической логики и понял, что математические доказательства формализуются как последовательно применённые правила вывода в некоторых формальных системах, я никак не мог понять: почему математики всё ещё не потеряли работу? Казалось бы, самая формальная из всех наук (даже и не вполне наука в смысле science, потому что имеет дело не с реальным миром, а с абстрактными структурами) давно должна была покориться компьютерам, причём даже не человекоподобному искусственному интеллекту, а обычным алгоритмам поиска вывода.

Да, конечно, доказательства могут иметь огромную длину, а некоторые верные утверждения доказать в любой наперёд заданной формальной системе и вовсе не получится, как рассказал нам в теоремах о неполноте Курт Гёдель, но это же всё в худшем случае! А нам худшего случая не нужно, мы просто хотим доказывать новые теоремы. Если вдруг какая-то задача окажется действительно очень сложной логически — есть, например, мнение, что неравенство классов

сложности P и NP может оказаться недоказуемым [5, 193], — это всего лишь значит, что её мы решить автоматическим поиском не сможем, но должны смочь решить почти любую другую.

Начиналось всё действительно многообещающе. Одной из первых компьютерных программ, относящихся к искусственному интеллекту, был *Logic Theorist*, разработанный Алленом Ньюэллом, Гербертом Саймоном и Клиффом Шоу ещё в 1956 году [231, 432, 433]. Эта программа впервые применила на практике многие ставшие стандартными методы: формулы представлялись в виде деревьев, а сам поиск доказательства выглядел как дерево с исходной гипотезой в качестве корня и выводами в качестве ветвей; был разработан и метод отсечения ветвей для более эффективного поиска доказательств. Авторы протестировали *Logic Theorist* на знаменитой книге *Principia Mathematica* Альфреда Уайтхеда и Бертрана Рассела, в которой всё здание математики строится исходя из базовых аксиом теории множеств (например, там доказано, что $1 + 1 = 2$, но этот результат появляется лишь страниц через двести после начала книги, потому что сначала нужно определить, что такое $=$, $+$, 1 и 2). В результате были доказаны 38 из 52 предложенных программе теорем (почти три четверти!), причём иногда доказательства были короче и элегантнее, чем у самих Уайтхеда и Рассела!

В 1950-х годах компьютеры были медленными, а их память была маленькой, так что действительно новых теорем ожидать было трудно. Тем не менее, результаты были очень яркими, и неудивительно, что Герберт Саймон сначала говорил своим аспирантам, что «на Рождество Эл Ньюэлл и я изобрели мыслящую машину», а позже писал об этом так: «[мы] изобрели компьютерную программу, способную мыслить нечисловым способом, и тем самым решили проблему связи разума и тела (mind-body problem), показав, как система, состоящая из материи, может обладать свойствами разума» [125].

Но на этом успехи искусственного интеллекта в доказательстве теорем практически остановились. В 1970-е и 1980-е годы стали развиваться системы системы символьных вычислений, в частности система MACSYMA из Массачусетского технологического института [176, 456], которая стала предтечей *Maple* и *Matlab*. Эти системы могли сделать за вас ряд сложных вычислений, в том числе вывести результат в виде формулы, а не только числа; другой современный аналог таких систем — библиотека *SymPy* для *Python*. Но доказать теорему такие системы всё же не способны.

Для дальнейшей формализации математики развивались и системы помощи в доказательстве теорем (proof assistants): система Automath [429] одной из первых использовала соответствие Карри — Ховарда (Curry — Howard correspondence), которое стало основой для всех подобных систем, в том числе для до сих пор использующегося Mizar [428], прувера для логик высших порядков HOL [440] и ставшего стандартом индустрии Coq [384]. Здесь уже можно записать условие теоремы формально, проверить доказательство... но вот находить новые доказательства никак не получалось.

Разумеется, люди пытались. В 1994 году был опубликован так называемый *QED Manifesto* [36, 642], где ряд математиков и разработчиков систем автоматического доказательства призывал сделать всю математику формальной. В 1980-х годах был основан журнал *Formalized Mathematics*, где публикуются строгие, автоматически проверяемые (и уже проверенные!) доказательства известных математических фактов, в попытке довести здание формальной математики до уровня математики «настоящей», а там, глядишь, и превзойти его.

Системы автоматического доказательства теорем активно развиваются и сегодня: одним из первых успешных пруверов первого порядка был Otter [389], затем он был развит в EQP (Equational Prover, специализирующийся на переписывании уравнений) и Prover9 [464], а как пример современной системы можно привести до сих пор развивающийся проект Lean [336, 411].

Вот только очень уж скромны успехи на этом пути. Например, в номерах *Formalized Mathematics* за 2023 год публикуются среди прочего решения задач по теории чисел [317, 318] из сборника Вацлава Серпинского, вышедшего в 1960-х годах [542], причём это не новые теоремы, а именно сборник задач «со звёздочкой» для студентов-математиков. Использование автоматических доказателей теорем (automated theorem provers) до сих пор так и не стало даже полезным инструментом в арсенале профессионального математика [84].

Автоматический поиск срабатывает только для результатов, которые стоят совсем рядом с аксиомами. Так, одним из важных успехов стало доказательство гипотезы Роббинса о том, что некоторый сокращённый набор аксиом представляет собой полную аксиоматику булевых алгебр [128, 388]. Подробно вдаваться в этот пример я не буду, но смысл доказательства состоит в том, чтобы получить один набор аксиом булевой алгебры (логических формул) из другого, а само доказательство — это очень длинный список небольших лемм, полученных переписыванием формул друг из друга; с этим действительно прувер EQP справился лучше людей ещё в 1990-х годах.

Более значительными были успехи компьютерных программ не в полноценном доказательстве теорем, а в *помощи* живым математикам. Компьютеры быстро стали применять для разбора случаев, особенно когда разбор относительно легко формализовать и автоматизировать, но случаев слишком много для того, чтобы легко было перебрать это вручную. Первым большим успехом стала знаменитая теорема о четырёх красках: чтобы доказать, что любую карту можно раскрасить в четыре цвета так, чтобы граничащие друг с другом области имели разные цвета, нужно было перебрать полторы тысячи разных случаев; Аппелю и Хакену в этом помогла компьютерная программа [25], а сейчас доказательство теоремы о четырёх красках уже проверено в Coq, так что этот результат вошёл и в область формализованной математики [224].

Именно через «умный перебор случаев» в математику пришло и глубокое обучение. Здесь пальма первенства до сих пор принадлежит исследователям из той самой компании *DeepMind*, которая и Ли Седоля в го обыграла, и белки на-

училась сворачивать. В 2022 году разработанная ими модель *AlphaTensor* смогла улучшить алгоритмы умножения матриц, в частности впервые за 50 лет ускорив алгоритм Штрассена [145]; *AlphaTensor* ищет возможные алгоритмы примерно так же, как *AlphaZero* учится играть в шахматы или го, только пространство поиска становится ещё больше (что, конечно, требует дополнительных изменений в модели).

Другой модификацией той же идеи стала модель *AlphaDev*, которая в 2023 году смогла улучшить алгоритмы сортировки [175]; предложенные моделью изменения уже внесены в библиотеку *std*, то есть они уже ускоряют миллионы компьютерных программ. А модель *FunSearch* в том же 2023 году смогла получить и новые теоретические результаты за счёт того, что научилась порождать программы, функции, написанные фактически программным кодом [382].

Но это всё-таки пока выглядит как ещё более эффективный перебор вариантов. Другое направление предлагает работа от той же компании *DeepMind*, где машинное обучение используется для получения новой интуиции, идей о том, как могли бы выглядеть новые математические результаты [13]. Там модель машинного обучения пытается найти ранее не замеченные исследователями связи между математическими объектами — и действительно находит их! На основе предложений модели математики смогли доказать ряд интересных новых результатов в теории узлов и теории представлений. Но опять же «на основе» и «при помощи»; когда модели машинного обучения смогут доказывать теоремы полностью самостоятельно, мы пока не знаем, и работа математиков действительно оказалась куда сложнее для автоматизации, чем могло показаться поначалу.

1.5.3. Химия и биология

Химия — это ещё одна точная наука, в которой, как и в физике, не сразу очевидны применения искусственного интеллекта. Но здесь тоже машинное обучение в последние годы производит настоящую революцию во многих задачах [92]. Давайте поговорим только о самом главном направлении, зато чуть подробнее; см. также достаточно популярные обзоры [99, 596].

Всё началось с вычислительной химии, расцвет которой начался с 1980-х годов. По мере того как развивались методы квантовой химии, в том числе пригодные для компьютерных вычислений приближённые методы, химия и наука о материалах стали всё активнее использовать вычислительные симуляции для исследования свойств веществ и поиска новых соединений. Методы молекулярной динамики позволяют с высокой точностью понять, как будут взаимодействовать друг с другом те или иные вещества, применяя к их молекулярным структурам квантово-механические модели, и такие исследования *in silico*[1] уже давно стали важной частью современных химии и молекулярной биологии.

[1] В биологии уже много веков применяются устоявшиеся латинские термины *in vivo* («в живом»), что означает исследования, проведённые на живых клетках, тканях или целых организмах, *in vitro* («в стекле»), то есть исследования в пробирках или чашках Петри, и *in situ* («на месте»), то

Но на этом пути прогресса встали две проблемы. Во-первых, «честная» молекулярная динамика требует огромных вычислительных ресурсов, экспоненциально увеличивающихся с ростом размеров молекул, а также с переходом к более точным приближённым моделям. Последовательность этих аппроксимаций известна в квантовой химии как «лестница Иакова теории функционала плотности», от «бренного мира» метода Хартри — Фока до «химического рая» идеально точных моделей (рис. 1.13, *а*) [89, 223, 661].

С этой проблемой помогают методы, подобные которым я уже упоминал выше: собрать датасет из вычислительно сложной модели и попробовать аппроксимировать его результаты моделью машинного обучения в надежде, что её результаты будут обобщаться и на другие соединения. Более того, сейчас уже появляются результаты, в которых учёные делают следующий шаг: не просто пытаются обучиться предсказывать те или иные свойства квантово-механических систем из данных моделей (такие подходы всегда будут ограничены теми конкретными свойствами, которые есть в датасете), а пытаются напрямую строить модели молекулярных волновых функций на основе машинного обучения [423, 610].

Вторая проблема состоит в том, что не только каждая конкретная молекулярная симуляция требует очень сложных вычислений, но и само пространство возможных соединений огромно. Например, химическая база данных GDB-17 содержит молекулы из 17 и менее атомов C, N, O, S и галогенов и в ней насчитывается более 166 миллиардов соединений [94, 492]! На рис. 1.13, *б* показано, как быстро растёт число молекул в зависимости от числа тяжёлых атомов, то есть всех атомов, кроме водорода. Конечно, перебор такого размера невозможен, и требуются методы быстрого поиска многообещающих кандидатов среди этого огромного множества. И здесь тоже помогает машинное обучение: можно обучить модель предсказывать, будет ли то или иное соединение обладать нужными свойствами, и ранжировать молекулы в огромных базах не сложной молекулярной динамикой, а гораздо более быстрой обученной моделью. В результате множество из миллиардов кандидатов можно сократить до тысяч или сотен и только их исследовать более сложными методами (конечно, обойтись совсем без лабораторных исследований здесь не получится).

Такой подход уже не раз приводил к экспериментально подтверждённым успехам: виртуальный поиск молекул дал новые органические светодиоды [140], материалы для редокс-батарей [108], новые фазы в аморфных материалах [490] и другие интересные результаты [8].

есть исследования, проведённые там, где феномен, собственно, происходит. Термин *in situ* используется далеко не только в биологии (например, ресторан может готовить еду *in situ*, то есть на выезде и из ингредиентов, имеющихся на месте); есть и более специальные термины, например *in utero* («в матке»), что относится к этапу эмбрионального развития. А в последние десятилетия получили распространение два новых латинских словосочетания: сначала *in papyro* («на бумаге») — так называют исследования, проведённые без экспериментов вовсе, например метаисследования, обобщающие предыдущие работы, — а теперь и *in silico* («в кремнии»), то есть исследования с численными экспериментами, проведёнными на компьютерах.

Это, конечно, только главные направления применения машинного обучения в химии и науке о материалах. Упомяну только ещё одно любопытное применение — автоматическое извлечение химических реакций и инструкций: оказывается, что современного уровня обработки естественных языков уже вполне достаточно для того, чтобы «прочитать» научную статью по химии и конвертировать описание лабораторных процедур и реакций на естественном языке в формализованную последовательность шагов, которую потом можно хоть сразу лабораторному роботу на вход подавать [37].

Многие из этих направлений в полной мере можно отнести и к наукам о жизни. В биологии и медицине тоже возникают задачи, связанные со взаимодействием химических соединений. Например, поиск лекарств (drug discovery) — это во многом именно такая задача: нужно определить, какие вещества могут стать активными, а поиск обычно идёт среди десятков миллионов так называемых малых молекул (small molecules). Поэтому неудивительно, что для поиска лекарств тоже используются модели машинного обучения, примерно так же, как в поиске химических соединений выше; здесь особенно полезными оказываются порождающие модели (generative models), которые могут обучаться порождать кандидатов в молекулы, а в базе мы потом просто будем искать максимально похожие на них реально существующие вещества[1].

Современная молекулярная биология вообще опирается на машинное обучение гораздо больше, чем может показаться. Молекулярная биология научилась собирать очень большие и интересные наборы данных, искать в которых закономерности можно только автоматически. Например, в геномике возникает много разных задач, связанных с анализом последовательностей нуклеотидов, то есть фактически строчек над алфавитом из четырёх букв $\{A,C,G,T\}$. Здесь машинное обучение используется, чтобы искать гены, регуляторные участки ДНК и РНК, аннотировать ДНК и так далее [343, 476]. И дальше машинное обучение пронизывает молекулярную биологию вдоль её «центральной догмы» (ДНК — РНК — белки), сквозь все так называемые «омики» (omics; рис. 1.14, *а*), от геномики через транскриптомику, то есть науку об экспрессии генов [514], вплоть до протеомики [625] и даже дальше, с интересными недавними применениями в метаболомике для анализа метаболических профилей клеток живых организмов [136].

Нельзя не упомянуть и обработку медицинских изображений (medical imaging), которая, конечно, часто пользуется методами и результатами компьютерного зрения (computer vision) и «обычной» обработки изображений. Но методы и модели в медицине обычно немного отличаются, потому что различается характер данных: медицинские снимки часто бывают гораздо больше обычных изображений (десятки тысяч пикселей по одной стороне); такими, например, получаются результаты гистологических исследований, как на рис. 1.14, *б, сверху*.

[1] К этой деятельности в своё время приложил руку и я, и особенно мой друг, соавтор и бывший аспирант Артур Кадурин, который занимался этим в компании *Insilico Medicine* [1, 120, 154, 405, 406]; но подробный рассказ об использующихся там моделях я вынужден отложить.

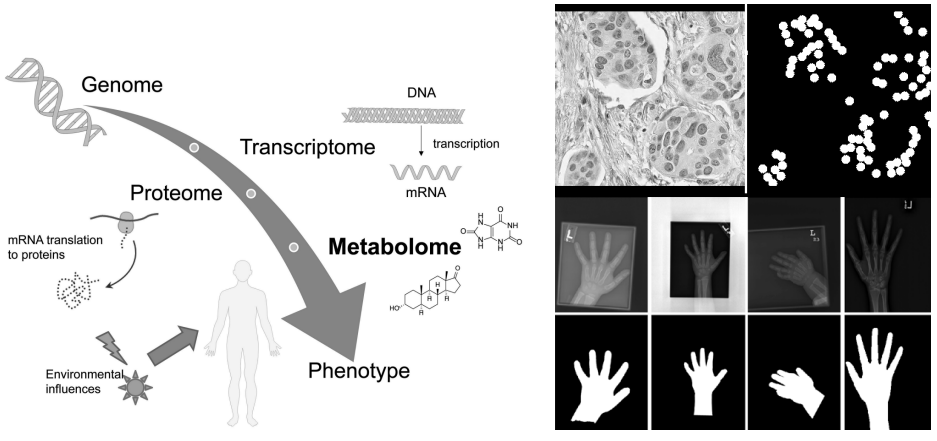


Рис. 1.14. Задачи для искусственного интеллекта в биологии: (а) структура «омик» [560]; (б) примеры медицинских изображений; сверху — участок гистологического изображения и соответствующие центры клеток [75]; снизу — рентгенограммы кистей рук людей и соответствующие маски сегментации [453].

Кроме того, медицинские изображения часто бывают трёхмерными (например, результаты компьютерной или магниторезонансной томографии), да и стандартные предобученные методы извлечения признаков здесь не работают, потому что медицинские снимки имеют мало общего с обычными фотографиями — например, показанные на рис. 1.14, *б, снизу* рентгенограммы кистей рук вряд ли получится обрабатывать теми же моделями, которые отличают котиков от пёсиков (хотя до какой-то степени предобучение всё равно можно проводить на обычных фотографиях).

Обработка медицинских изображений — это отдельная огромная область с тысячами ярких результатов и целыми отдельными конференциями; мы немного поговорим об этой области, когда будем касаться компьютерного зрения и обработки изображений в целом (не в этой книге, увы), а пока я просто дам ссылки на несколько обзоров [23, 141, 218, 357, 365, 533, 662].

1.5.4. Науки о Земле

Как и другие науки, существенно использующие большие наборы данных, науки о Земле применяли машинное обучение и другие разделы искусственного интеллекта практически с момента их появления; см., например, обзор [151]. Ещё в 1960-е годы в геологии применяли свежеизобретённый алгоритм кластеризации k-средних [475] и обучали марковские модели для анализа временных рядов, которых в науках о Земле великое множество [104, 127, 321].

С ранними применениями AI в науках о Земле связана забавная история: в геостатистике с тех же 1960-х годов широко применялись и развивались методы так называемого *кригинга* (kriging, он же предсказания Винера – Колмогорова), который, по сути, представляет собой регрессию на основе гауссовских процессов [15, 96, 124]. В этой книге мы не доберёмся до гауссовских процессов, так что просто оставлю несколько ссылок [160, 367, 488], но надеюсь, что в будущем поговорим и о них. Когда в 1970-е годы началась первая «зима искусственного интеллекта», которую мы обсуждали в разделе 1.2, многие применения AI для обработки данных в геостатистике тоже приостановили своё развитие... но на кригинге почему-то не было метки «искусственный интеллект», и гауссовские процессы спокойно продолжили своё победное шествие в науках о Земле.

Много приложений искусственного интеллекта в науках о Земле связано с *дистанционным зондированием Земли* (remote sensing), то есть с обработкой данных наблюдений поверхности нашей планеты при помощи самых разных средств: фотографирования, радаров, лидаров, тепловых датчиков в длинноволновом диапазоне, ультразвуковых и радиолокационных датчиков, объединения всего вышеперечисленного в мультиспектральных платформах и так далее и тому подобное. Такие данные можно собирать и с земли, но обычно речь идёт о спутниковых снимках. Как правило, дистанционное зондирование выдаёт цифровые данные в виде матрицы (двумерного массива) для каждого спектра, состоящей из замеров интенсивностей. Поэтому здесь сразу появляются для одних модальностей — цифровая обработка сигналов, а для других — методы компьютерного зрения [88, 366, 655].

Это, конечно, только два примера из многих. Так, в геологии машинное обучение постоянно используется для того, чтобы понять структуру и поведение земных недр. Это очень сложная задача, потому что под землёй проходит масса сложных и взаимодействующих друг с другом процессов, которые ещё и проходят в разных пространственных и временных масштабах, а данные о многих из этих процессов напрямую не получить. Но по мере того, как развиваются инструменты получения данных, а также численные модели всех этих процессов (здесь есть некая параллель с тем, что мы выше говорили о физике), многое постепенно становится ясным [151, 216, 364, 656, 656].

1.5.5. Социальные науки

Социология всегда была наукой о данных. Даже очень упрощённое и в плохом смысле слова «популярное» представление о социологии как о «науке об опросах» сразу же подразумевает сбор, обработку и анализ данных. Современная социология, конечно, сильно продвинулась со времён Гэллапа[1]. С одной стороны,

[1] *Джордж Гэллап* (George Horace Gallup, 1901–1984) — американский журналист, учёный и статистик, ещё в 1928 году защитивший диссертацию на тему «Объективный метод определения читательского интереса к материалам газеты». Настоящая слава пришла к Гэллапу в 1936 году: вопреки уверенному предсказанию журнала *Literary Digest*, который отдал победу республиканцу Альфу

социология — это во многом наука о *моделях*, которые описывают свойства сообществ людей, и, когда мы говорим «модели», мы сразу же попадаем в область статистики и машинного обучения [407]. Социологические модели, пытающиеся предсказать ту или иную зависимую переменную (для нас это обучение с учителем), могут быть о чём угодно: кому из пациентов будет полезнее замена коленного сустава при остеоартрите [474], между какими странами скорее возникнет вооружённый конфликт [51], что влияет на решения судей о выборе мер пресечения [308] и так далее и тому подобное.

С другой стороны, в последние десятилетия социологи получили огромные наборы данных, о которых раньше не могли и мечтать. Анализ социальных сетей методами теории графов и решение связанных с этим задач машинного обучения, обычно без учителя, — поиск сообществ, выделение инфлюенсеров, поиск «мостов» между сообществами и так далее — уже развился в отдельную большую науку [159, 240, 387]. Особенно интересно здесь расширять и углублять сугубо теоретико-графовые методы при помощи дополнительной информации, которой в социальных сетях тоже очень много, в основном текстов и фотографий. Здесь социологии помогает анализ естественных языков, в частности тематическое моделирование, которое позволяет искать сжатые представления больших корпусов текстов в виде неплохо интерпретируемых распределений на словах [144, 312, 438].

Но в общественных науках с применением статистики и машинного обучения нужно быть осторожными. Во-первых, человеку свойственно интерпретировать статистические зависимости как причинно-следственные связи, и настоящая задача социологического исследования обычно в том, чтобы найти *причины* того или иного явления, а не просто корреляты. В машинном обучении есть специальный раздел о *причинном выводе* (causal inference) [254, 457], который активно используется и в социологии, в частности в эконометрике [33, 413], но всё-таки найти именно *причину* обычно сложно и не то чтобы является предметом нашей науки.

Во-вторых, не всегда легко понять, имеет ли вообще решение стоящая перед социологами задача. Здесь я приведу один интересный пример: *Fragile Families Challenge*[1], большое междисциплинарное исследование, авторы которого пытались предсказать жизненный путь и достижения детей из проблемных семей. Учёные собрали очень большой набор данных о детях от рождения до девяти лет (почти 13 000 переменных про каждую из более чем 4000 семей) и попытались научиться предсказывать ключевые переменные о будущем этой семьи к тому времени, когда ребёнку исполнится 15 (его/её средний балл в школе, будет ли

Лэндону на основе выборки из 2,4 миллиона (!) опросников, выборы прошли в полном соответствии с предсказанием Гэллапа, сделанным на основе выборки из всего лишь 50 тысяч человек. С тех пор Гэллап стал синонимом опросов общественного мнения, а основной причиной такого результата считается необъективная выборка *Literary Digest*, смещённая в сторону людей с высоким доходом, автовладельцев и так далее — хотя есть на этот счёт и другие мнения [359].

[1] https://www.fragilefamilieschallenge.org/

семья выселена из дома и так далее). Исследование было организовано как большое соревнование по образцу Kaggle[1] и привлекло 160 команд-участниц, которые использовали много разных моделей и трюков для этого предсказания. В результате... практически ничего не получилось; оказалось, что ни одну из шести целевых переменных толком предсказать никто не смог [391].

Поскольку подобного рода исследований в социологии очень много[2], лично мне это говорит о том, что социологические модели часто задаются вопросами, на которые просто нельзя дать ответ, — слишком много случайностей в нашей жизни и слишком мало данных, чтобы весь этот шум из модели устранить. Более того, к социальным наукам полностью относится критика, связанная с малыми выборками, p-хакингом и невоспроизводимостью, которую мы обсудим в разделе 2.6, и кризис воспроизводимости нанёс по ним серьёзный удар. Но пытаться, конечно, всё равно нужно.

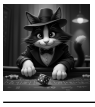
На этом давайте закончим краткий обзор применений искусственного интеллекта — он мог бы и сам по себе занять не одну книгу, а в наше время новые применения искусственного интеллекта находятся буквально еженедельно. Но наша цель всё-таки в том, чтобы разобраться, *как* работает искусственный интеллект, а не только *что* он может сделать. И для этого нам придётся вспомнить раздел математики, являющийся основой всего происходящего в машинном обучении, — теорию вероятностей.

[1] Kaggle — это главный портал, на котором проводятся соревнования по машинному обучению. Как правило, организаторы соревнования размещают обучающий набор, скрывают от участников тестовый набор (см. подраздел 1.4.2) и дают некоторое время на то, чтобы обучить модели, которые на этом скрытом наборе покажут наилучшие результаты. Многие новые методы и приёмы машинного обучения появились как «побочный эффект» таких соревнований.

[2] В том числе на основе, мягко скажем, не такого большого объёма данных: например, знаменитый «зефирный тест» пытается предсказывать даже более далёкие жизненные успехи ребёнка на основе фактически всего одной бинарной переменной... но об этом см. раздел 2.6.

Глава 2
Основы байесовского вывода

TL;DR

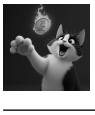


Раздел 2.1 посвящён основам теории вероятностей: начнём со стандартных определений, поговорим о распределениях, случайных величинах, условных вероятностях и моментах случайных величин, а закончим разделом «со звёздочкой», в котором построим теорию вероятностей с другой стороны.

В разделе 2.2 обсуждается главный вероятностный инструмент машинного обучения: теорема Байеса. Обсудим её смысл в машинном обучении, разберём примеры, поговорим о метриках качества в бинарной классификации и рассмотрим задачи байесовского вывода, вытекающие из применения теоремы Байеса.

Раздел 2.3 начинается с замечания о том, что вероятностная интуиция у людей развита не очень хорошо, что затем иллюстрируется так называемыми «ошибкой прокурора» и «ошибкой адвоката», а также парадоксом Монти Холла и другими когнитивными искажениями, связанными с оценкой вероятностей.

В разделе 2.4 мы наконец-то переходим к конкретному примеру байесовского вывода, полностью решая все задачи байесовского вывода для простейшего примера подбрасывания одной нечестной монетки. Это также позволит нам поговорить о сопряжённых априорных распределениях.



Раздел 2.5 посвящён примеру «со звёздочкой», тоже связанному с обучением параметра монетки: существует ли так называемый эффект «горячей руки», при котором у баскетболиста процент попадания после удачного броска растёт? Как узнать это из данных о бросках и был ли прав Амос Тверски, отрицая этот эффект?

Наконец, в разделе 2.6 мы обсудим кризис воспроизводимости, охвативший многие научные дисциплины в XXI веке. Почему результаты научных исследований не воспроизводятся, как это исправить и какое это имеет отношение к машинному обучению?

2.1. Основы теории вероятностей

Thus, the wit was not wrong who defined education in this way: “Education is that which remains, if one has forgotten everything he learned in school.” (Не так уж неправ был шутник, определивший образование как «то, что остаётся, когда мы забываем всё, чему нас учили».)

Albert Einstein. On Education

О святая математика, в общении с тобой хотел бы я провести остаток дней своих, забыв людскую злобу и несправедливость Вседержителя.

Лотреамон

2.1.1. Введение: вероятностные пространства и распределения

Когда я на первой лекции своего курса машинного обучения (из которого и выросла эта книга) говорю студентам, что машинное обучение практически полностью основано на теории вероятностей и математически мы будем в основном заниматься именно ею, студенты редко бывают вдохновлены такой перспективой. Они вспоминают курс теории вероятностей, который обычно начинается с аксиоматического построения теории вероятностей как раздела теории меры (по Колмогорову[1]), а затем состоит из интуитивно понятных, но довольно сложных в доказательстве теорем, опирающихся на эту самую теорию меры. Конечно, при изучении теории вероятностей от колмогоровской аксиоматики никуда не деться и лучше сразу дать несколько ссылок на классические учебники [503, 691, 693, 694].

Но, к счастью, в машинном обучении нам потребуется не сама теория, а то, что обычно остаётся в голове, когда все детали теоретических доказательств из неё

[1] *Андрей Николаевич Колмогоров* (1903–1987) — советский математик, один из величайших математиков XX века. Всего не перечислишь: Колмогоров ввёл современную аксиоматику теории вероятностей, обобщил закон больших чисел, ввёл критерий согласия Колмогорова, предложил новое понятие алгоритма и заложил основы теории колмогоровской сложности, создал теорию хаоса (теорию Колмогорова — Арнольда — Мозера), доказал теорему об универсальной аппроксимации (фактически основную теорему нейронных сетей) и многое другое. Кстати, и по нашей непосредственной теме Колмогоров занимал позицию весьма радикальную для 1964 года: «принципиальная возможность создания полноценных живых существ, построенных полностью на дискретных (цифровых) механизмах переработки информации и управления, не противоречит принципам материалистической диалектики» [684]. Кроме того, он реформировал школьное математическое образование, создал школу-интернат при МГУ и журнал «Квант». Но есть и другие страницы истории: так, Колмогоров принял активное участие в кампании против своего бывшего учителя Н. Н. Лузина (в 1936 году — сами понимаете, какие были ставки!) и в целом всегда колебался с линией партии, пока не был подвергнут критике за школьные учебники.

уже выветрились: базовые понятия, основные свойства и, главное, интуитивное понимание того, что же такое случайные величины и как с ними управляться. Давайте в этом разделе кратко и не особенно формально их повторим; читателям, твёрдо владеющим основами теории вероятностей, я рекомендую всё-таки не полностью пропускать этот раздел, а хотя бы посмотреть на альтернативный подход к формализации теории вероятностей в его конце.

В колмогоровской аксиоматике вся теория вероятностей стоит на понятии *элементарного события* (исход, outcome), из которых состоит *пространство элементарных событий* (sample space, possibility space). Элементарные события не пересекаются и представляют собой все возможные исходы тех случайных величин, о которых мы хотим рассуждать. Например, у монетки множество элементарных событий состоит из двух элементов $\Omega = \{\text{орёл}, \text{решка}\}$, а у случайной величины, определяющей время жизни человека, это $\Omega = [0, \infty)$ (пожалуй, пока такой правый предел звучит чересчур оптимистично, но формально именно так). *Событие* — это некоторое множество элементарных событий, то есть подмножество Ω; например, можно говорить о событии «человек проживёт от 50 до 100 лет», $[50{,}100] \subset \Omega$. Иначе говоря, элементарные события — это точки в вероятностном пространстве, а события — множества таких точек.

В конечном (и вообще дискретном) случае понятие вероятности ввести легко: для некоторого пространства элементарных событий $\mathcal{S} = \{x_1, x_2, \ldots\}$ *вероятность* (probability mass function, pmf) — это функция $p : \mathcal{S} \longrightarrow [0{,}1]$, для которой $p(x) \geq 0$ для любого $x \in \mathcal{S}$, и

$$\sum_{x \in \mathcal{S}} p(x) = 1.$$

Проще говоря, мы задаём неотрицательные и суммирующиеся в единицу вероятности каждого элементарного события и из них вероятность любого события получается простым сложением. Базовый и фактически единственный пример здесь — это бросание игральной кости[1] с разными неотрицательными вероятностями граней, которые суммируются в единицу:

$$p(X = i) \geq 0 \text{ для каждого } i, \quad \sum_{i=1}^{n} p(X = i) = 1.$$

Обратите внимание, что, хотя я пишу $X = i$ и обозначаю грани игральной кости числами, на самом деле это не числа, а дискретные, никак не связанные друг с другом объекты: пока что нет никакого смысла, в котором ⚀ + ⚁ = ⚃ или

[1] Правда же, от слов «игральная кость» веет чем-то архаическим? Лично я всю жизнь называл их исключительно «кубиками», даже когда близко познакомился с «кубиками», у которых было четыре, восемь или двадцать граней; но в книге, наверное, было бы несерьёзно оставлять такой очевидный когнитивный диссонанс, поэтому «кубики» будут только шестигранные, а в общем случае я буду стараться говорить «игральные кости». По-английски оно, конечно, сподручнее, в слове die никакое число граней не подразумевается.

⚅ > ⚂. Чтобы такой смысл появился, нужно определить случайную величину, о чём речь пойдёт ниже.

Формально может оказаться, что $n = \infty$ и сумма — это сумма ряда, но суть рассуждений от этого не изменится: все ряды в нашей науке заведомо будут сходиться и тонкости математического анализа вспоминать не потребуется.

А вот ввести вероятность на пространстве с континуальным множеством точек уже гораздо сложнее; здесь как раз и начинаются те самые сигма-алгебры из колмогоровской аксиоматики теории вероятностей. Для полноты картины я всё-таки кратко приведу определения, но теорией меры мы дальше всерьёз пользоваться не будем.

Рассмотрим некоторое множество элементарных событий $\mathcal{S}$. Тогда система его подмножеств $\mathcal{X}$ называется *алгеброй*, если оно включает $\mathcal{S}$ и замкнуто относительно конечных пересечений, объединений и дополнений:

- $\mathcal{S} \in \mathcal{X}$;
- если $X \in \mathcal{X}$ и $Y \in \mathcal{X}$, то $X \cup Y \in \mathcal{X}$ и $X \cap Y \in \mathcal{X}$;
- если $X \in \mathcal{X}$, то его дополнение $\bar{X} = \mathcal{S} \setminus X \in \mathcal{X}$.

А если система $\mathcal{X}$ замкнута относительно не только конечных, но и *счётных* объединений, то $\mathcal{X}$ называется *сигма-алгеброй*:

- если $X_1, X_2, \ldots, X_n, \ldots \in \mathcal{X}$, то $\bigcup_{n=1}^{\infty} X_n \in \mathcal{X}$.

Обратите внимание, что хотя формально сигма-алгебра замкнута относительно *счётных* объединений, но лишь *конечных* пересечений, на самом деле это не ограничение: счётное пересечение $\bigcap_{n=1}^{\infty} X_n$ — это дополнение к счётному объединению дополнений:

$$\bigcap_{n=1}^{\infty} X_n = \overline{\bigcup_{n=1}^{\infty} \bar{X}_n}.$$

Для целей этой книги достаточно будет ограничиться в непрерывном случае только евклидовыми пространствами, $\mathcal{S} = \mathbb{R}^d$, и сигма-алгеброй *борелевских*[1] подмножеств в евклидовом пространстве, то есть минимальной сигма-алгеброй, содержащей все открытые подмножества. Все «простые» множества являются борелевскими: например, на прямой открытый интервал — это открытое множество, замкнутый отрезок — это дополнение объединения двух открытых лучей, а дальше можно объединять отрезки и интервалы как душе угодно.

[1] *Феликс Эдуар Жюстен Эмиль Борель* (Félix Edouard Justin Émile Borel, 1871–1956) — французский математик и политический деятель. В математике Борель известен основополагающими работами по теории меры и её приложениями к теории вероятностей; его имя носит множество важных объектов: мера Бореля, алгебра Бореля, лемма Бореля — Кантелли, принцип Бореля — Лебега и так далее. Интересно, что Борель был и общественным деятелем: членом палаты депутатов в 1924–1936 годах, участником Сопротивления в годы немецкой оккупации, а в 1925 году даже морским министром в кабинете, возглавляемом другим известным математиком Полем Пенлеве. Вообще, во Франции профессиональные учёные ещё со времён революции принимали очень активное участие в общественной жизни; оставлю, пожалуй, читателю спекуляции о причинах этого феномена.

В любом случае *вероятность* теперь вводится не на точках пространства элементарных событий $\mathcal{S}$, а на множествах из сигма-алгебры $\mathcal{X}$, $p : \mathcal{X} \longrightarrow [0,1]$. Она по-прежнему должна удовлетворять набору аксиом, похожих на те, что мы вводили в дискретном случае, но теперь с дополнительной технической аксиомой непрерывности в нуле:

- $p(X) \geq 0$ для любого $X \in \mathcal{X}$;
- $p(\mathcal{S}) = 1$;
- если $X_1, \ldots, X_n, \ldots$ — не более чем счётное множество непересекающихся событий из $\mathcal{X}$, то есть все $X_i \in \mathcal{X}$ и $X_i \cap X_j = \emptyset$ для всех $i \neq j$, то

$$p\left(\bigcup_{n=1}^{\infty} X_n\right) = \sum_{n=1}^{\infty} p(X_n);$$

- если $X_1 \supseteq X_2 \supseteq \ldots \supseteq X_n \supseteq \ldots$ — последовательность вложенных событий из $\mathcal{X}$, стремящихся к пустому множеству, то есть все $X_i \in \mathcal{X}$ и $\bigcap_{n=1}^{\infty} X_n = \emptyset$, то $p(X_n) \to 0$ при $n \to \infty$.

Если эти условия выполняются, тройку $\langle \mathcal{S}, \mathcal{X}, p \rangle$ называют *вероятностным пространством*. Говоря языком теории меры, задание вероятностного пространства — это задание счётно-аддитивной неотрицательной меры на измеримом пространстве $\mathcal{S}$ с дополнительным свойством того, что мера $\mathcal{S}$ равна 1. Впрочем, на этом языке мы в книге говорить и не будем, нам достаточно будет случая $\mathcal{S} = \mathbb{R}^d$ и борелевской сигма-алгебры. В этом случае вероятность определена на борелевских подмножествах $X \subseteq \mathbb{R}^d$ с естественными аксиомами относительно пересечений, объединений и дополнений.

Переход к событиям в дискретном случае был самоочевиден: вероятность события равна сумме вероятностей составляющих его элементарных событий. Но машинное обучение непредставимо и без *непрерывных* вероятностей, то есть вероятностей, определённых на непрерывном, континуальном вероятностном пространстве $\langle \mathcal{S}, \mathcal{X}, p \rangle$. В континуальном случае вероятность каждой конкретной точки, скорее всего, равна нулю (хотя см. ниже), а смысл имеют только вероятности подмножеств пространства, $p(X \in A)$ для разных $A \subseteq \mathcal{S}$.

Чтобы не углубляться в теорию меры, обычно мы будем предполагать, что все непрерывные распределения вероятностей в этой книге имеют *плотность* (probability density function, pdf), то есть функцию $p(\boldsymbol{x})$[1], интегралы от которой дают вероятности подмножеств:

$$\int_A p(\boldsymbol{x})\, \mathrm{d}\boldsymbol{x} = p(X \in A). \tag{2.1}$$

[1] Важное замечание об обозначениях: во всей книге, как в формулах 2.1 и 2.2, жирным шрифтом будут обозначаться векторы (например, $\boldsymbol{x} \in \mathbb{R}^d$), а обычным шрифтом — одномерные числа, $x \in \mathbb{R}$. Матрицы будут обозначаться заглавными буквами.

Как и прежде, функция плотности и все её интегралы неотрицательны, а единице теперь равен интеграл по всему пространству:

$$\int_{\mathcal{S}} p(\boldsymbol{x})\,\mathrm{d}\boldsymbol{x} = 1. \tag{2.2}$$

В случае $\mathcal{S} = \mathbb{R}$ важную роль также играет *функция распределения* (cumulative density function, cdf), равная интегралу от плотности:

$$F(a) = \int_{-\infty}^{a} p(x)\,\mathrm{d}x.$$

Функция распределения монотонно возрастает от $F(-\infty) = 0$ до $F(\infty) = 1$.

Единственное обобщение, которое не совсем укладывается в понятие плотности, но всё же может пригодиться в машинном обучении, — это непрерывные распределения, присваивающие ненулевую вероятность отдельным точкам из $\mathcal{S}$. Например, нам вполне может встретиться распределение на $\mathcal{S} = \mathbb{R}$, которое полностью сосредоточено в точках $\{-1, 0, 1, \frac{5}{2}\}$, то есть по сути является дискретным. У такого распределения нет плотности, или, говоря другими словами, его плотность представляет собой *обобщённую функцию*.

Для серьёзного разговора об обобщённых функциях опять нужна теория меры, но нам в машинном обучении будет достаточно одной-единственной обобщённой функции — *дельта-функции Дирака*[1] (Dirac delta function). Она определяется как функция, полностью сосредоточенная в нуле и интегрирующаяся в единицу:

$$\delta(x) = 0 \text{ для всех } x \neq 0, \qquad \int_{-\infty}^{\infty} \delta(x)\mathrm{d}x = 1.$$

Теперь дискретное распределение, которое полностью сосредоточено в четырёх точках $\{-1, 0, 1, \frac{5}{2}\}$, можно представить как смесь (взвешенную сумму) четырёх дельта-функций. Например,

$$p(x) = \frac{1}{4}\delta(x+1) + \frac{1}{4}\delta(x) + \frac{1}{4}\delta(x-1) + \frac{1}{4}\delta(x-\frac{5}{2})$$

будет означать распределение, в котором этим четырём точкам присвоены равные вероятности, по $\frac{1}{4}$ каждой, а остальные точки не выпадают вовсе.

[1] *Поль Дирак* (Paul Dirac, 1902–1984) — английский физик-теоретик, один из главных физиков XX века, лауреат Нобелевской премии по физике 1933 года. Дирак был одним из основоположников квантовой механики, где разработал гамильтонов подход к квантовой механике на основе скобок Пуассона, сформулировал получившее его имя уравнение, которое предсказало существование антиматерии. Дельта-функция появилась в его общей теории преобразований, объединявшей волновую и матричную механики, в которой нужно было работать с непрерывными спектрами. Сергей Капица писал, что «по мнению многих, Дирак — ещё более великий физик, чем Эйнштейн» [680], а сам Эйнштейн отзывался о Дираке так: «у меня проблемы с Дираком; его балансирование на головокружительном пути между гением и безумством ужасно».

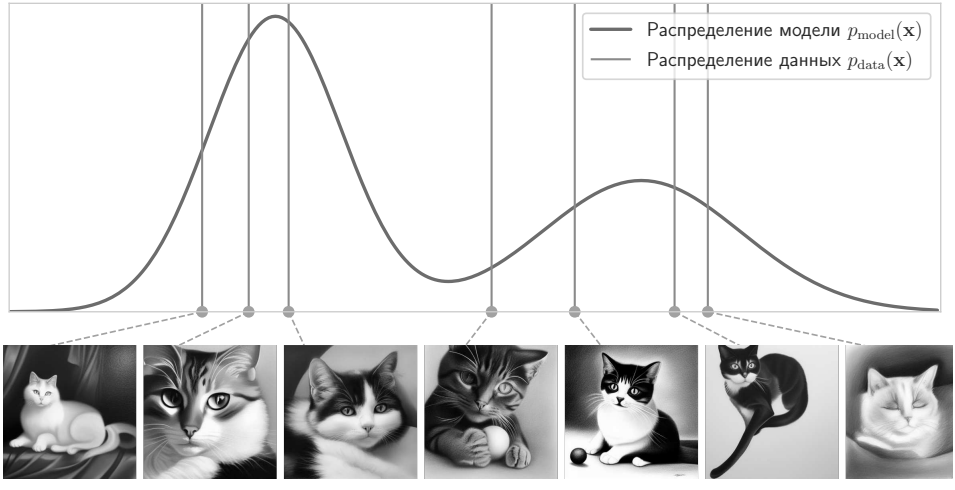


Рис. 2.1. Распределение данных (смесь дельта-функций) и распределение модели

В машинном обучении распределения такого вида часто встречаются в виде *распределения данных* (data distribution) p_{data}, которое определяется для датасета $D = \{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N\}$ как равномерное распределение на точках данных. Это значит, что формально мы можем записать:

$$p_{\text{data}}(\boldsymbol{x}) = \frac{1}{N} \sum_{n=1}^{N} \delta\left(\boldsymbol{x} - \boldsymbol{x}_n\right).$$

Абстрактный пример того, что мы пытаемся сделать в машинном обучении без учителя, показан на рис. 2.1: распределение данных p_{data} сосредоточено в конечном наборе точек (это и есть, собственно, наш датасет), а распределение модели p_{model}, которое мы обучаем, пытается аппроксимировать p_{data}, оставаясь при этом в некотором параметрическом семействе. Например, на рис. 2.1 распределение p_{model} представляет собой смесь двух гауссианов, но здесь мы уже изрядно забегаем вперёд.

2.1.2. Случайные величины и совместные распределения

Теперь, когда у нас есть вероятностные пространства и события в них, можно перейти к функциям, присваивающим событиям числовые значения, — к *случайным величинам*. Формально говоря, для вероятностного пространства $\langle \mathcal{S}, \mathcal{X}, p \rangle$ *случайная величина* X — это измеримая числовая функция $X : \mathcal{S} \to \mathbb{R}$, то есть функция, присваивающая числовое значение каждому элементарному исходу из множества $\mathcal{S}$. Измеримая функция — это такая функция, у которой прообразы измеримых множеств (то есть борелевских на $\mathbb{R}$) тоже будут измеримы (то есть будут лежать в $\mathcal{X}$); можно определить и случайные величины со значениями не

в $\mathbb{R}$, а в других измеримых пространствах, но в теорию меры я обещал не углубляться, так что этим определением и ограничимся.

Случайные величины позволяют присваивать событиям числовые значения и работать с ними. Вот теперь мы действительно можем присвоить численные значения граням игральной кости; например, если мы бросаем два шестигранных кубика, то сумма выпавших на них очков ξ — это классический пример *дискретной* случайной величины, то есть величины, заданной на дискретном вероятностном пространстве. Она определена на пространстве $\mathcal{S}$, являющемся декартовым произведением пространств исходов двух кубиков, то есть $|\mathcal{S}| = 36$, и каждой паре исходов соответствует число.

Теперь можно говорить о том, с какой вероятностью случайная величина примет то или иное значение, например

$$p(\xi = 7) = \frac{\#\{\text{исходов с суммой } 7\}}{\#\{\text{всех исходов}\}} = \frac{6}{36} = \frac{1}{6}.$$

Случайные величины можно комбинировать друг с другом, получая новые случайные величины. Например, если имеются две случайные величины X и Y на одном и том же вероятностном пространстве $\langle \mathcal{S}, \mathcal{X}, p \rangle$, то можно естественным образом определить их сумму, разность или произведение, и это тоже будут случайные величины на том же пространстве.

Но и разные вероятностные пространства тоже можно «скрещивать» между собой. Основной инструмент для этого выглядит так: если заданы две случайные величины $p_X(x)$ и $p_Y(y)$ из вероятностных пространств $\langle \mathcal{S}_X, \mathcal{X}, p_X \rangle$ и $\langle \mathcal{S}_Y, \mathcal{Y}, p_Y \rangle$, то их *совместное распределение* $p(x, y)$ будет частью вероятностного пространства $\langle \mathcal{S}_X \times \mathcal{S}_Y, S(\mathcal{X} \times \mathcal{Y}), p \rangle$, где пространство исходов является декартовым произведением $\mathcal{S}_X$ и $\mathcal{S}_Y$, а сигма-алгебра измеримых событий $S(\mathcal{X} \times \mathcal{Y})$ — это сигма-алгебра, порождённая декартовыми произведениями элементов $\mathcal{X}$ и $\mathcal{Y}$. При этом $p(X, Y)$ для некоторых событий $X \in \mathcal{X}$ и $Y \in \mathcal{Y}$ будет представлять собой вероятность того, что первая компонента исхода (x, y) попадёт в множество X *и одновременно* вторая компонента попадёт в Y: $p(X, Y) = p(x \in X \wedge y \in Y)$.

Например, в дискретном совместном распределении двух игральных кубиков

$$p(\{1,2,3\}, \{2,3,4\}) = p(x \in \{1,2,3\} \wedge y \in \{2,3,4\}) = \\ = \frac{\#\{\text{исходов с } x \in \{1,2,3\} \text{ и } y \in \{2,3,4\}\}}{\#\{\text{всех исходов}\}} = \frac{3 \cdot 3}{6 \cdot 6} = \frac{1}{4}.$$

Здесь я обозначаю через $\#\{\ldots\}$ число элементов, определённых внутри фигурных скобок; это обозначение будет встречаться во всей книге.

Определение совместного распределения $p(x, y)$ для двух случайных величин легко продолжить на любое конечное их число: совместное распределение вероятностей $p(x_1, x_2, \ldots, x_n)$ определено на вероятностном пространстве, представляющем собой декартово произведение $\langle \mathcal{S}_1 \times \mathcal{S}_2 \times \ldots \times \mathcal{S}_n, \mathcal{X}_1 \times \mathcal{X}_2 \times \ldots \times \mathcal{X}_n, p \rangle$,

и вероятность $p(X_1, X_2, \ldots, X_n)$ представляет собой вероятность того, что каждая компонента x_i вектора из реализаций случайных величин попадёт в соответствующее множество X_i.

Можно рассматривать совместные распределения случайных величин любой природы, в том числе соединять вместе дискретное и непрерывное распределения. В машинном обучении это иногда приходится делать потому, что разные входные данные могут иметь разную природу.

Представьте себе систему онлайн-рекомендаций, пользователи которой часто регистрируются в этой системе через социальные сети. Такой системе, скорее всего, будет доступен ряд категориальных атрибутов для каждого пользователя: страна проживания, пол и/или гендер, уровень образования и другие. Но будут и атрибуты, представлять которые в виде дискретной, категориальной переменной не хочется, потому что у них много разных числовых значений: возраст, число друзей в социальной сети, показатели активности в сети и так далее. В таком случае получается, что пользователь формализуется в системе как вектор атрибутов $\boldsymbol{x}$, в котором есть и дискретные, и непрерывные компоненты, и если мы захотим обучить распределение пользователей в системе $p(\boldsymbol{x})$, нам придётся обучать совместное распределение дискретных и непрерывных вероятностей.

И ещё одно важное замечание. Случайные величины по определению заданы на некотором пространстве исходов (сначала элементарных, потом на любом событии); эти исходы имеют обычно какой-то физический смысл — например, стороны монетки, грани игральной кости или рост человека. Но в реальности, говоря о монетках или людях, мы не будем всё время вспоминать этот физический смысл, а будем рассматривать соответствующие случайные величины как заданные на стандартных математических объектах: конечном наборе вариантов, натуральных числах, вещественных числах и так далее.

Формально говоря, чтобы это сделать, нам нужно добавить некоторое преобразование, которое называется *образом меры под действием отображения*, отобразив имеющие физический смысл исходы в соответствующие им математические объекты. Но, конечно, мы не будем больше ни разу об этом вспоминать, а просто будем сразу рассматривать случайные величины на стандартных математических пространствах.

Например, если при рассмотрении заявки на кредит нас о возможном заёмщике интересуют только возраст (дискретизированный в четыре возможных варианта, как в разделе 1.4.3) и уровень дохода (вещественное неотрицательное число), то формально мы должны сначала рассмотреть отображение из пространства исходов (потенциальных заёмщиков) в пространство $\{1, 2, 3, 4\} \times \mathbb{R}_+$, а затем перенести вероятностную меру на его образ; разумеется, мы не будем явно вводить такое отображение и загромождать им все формулы.

В разделе 3.1 я приведу важнейшие примеры дискретных случайных величин, в разделе 3.2 — непрерывных, а сейчас продолжу напоминать основные понятия.

2.1.3. Условные вероятности

Если нам дано некоторое вероятностное пространство, на котором заданы вероятности событий, то можно ввести и понятие *условной вероятности*. В колмогоровской аксиоматике условная вероятность события X при условии события Y, $p(X|Y)$ — это частное от деления вероятности их пересечения на вероятность события Y:

$$p(X|Y) = \frac{p(X \cap Y)}{p(Y)}. \tag{2.3}$$

Формула (2.3) — это определение, а не свойство или теорема, но можно было бы получить её и как результат, если начать с частотного (фриквентистского) подхода к вероятностям и искать частоту повторяющегося события X среди тех исходов, которые принадлежат событию Y.

В любом случае формула (2.3) показывает, что для определения условной вероятности важно, чтобы вероятность события Y не была равна нулю. Даже если вероятностное пространство непрерывно, и событие Y представляет собой множество меры ноль в этом пространстве, это может привести к странным последствиям.

Классический *парадокс Бореля — Колмогорова* выглядит так: представьте себе случайную величину в обычном трёхмерном евклидовом пространстве $\mathbb{R}^3$, которая распределена равномерно на единичной сфере. Каким будет условное распределение этой величины на большом круге сферы?

Известно, что равномерное распределение на сфере можно получить, выбирая сначала долготу λ равномерно из интервала $[-\pi, \pi]$, а потом широту φ из интервала $\left[-\frac{\pi}{2}, \frac{\pi}{2}\right]$ с плотностью $\frac{1}{2}\cos\varphi$[1]. Но затем оказывается, что перейти к большому кругу этой сферы можно по-разному:

- с одной стороны, можно зафиксировать широту и рассмотреть в качестве большого круга экватор $\varphi = 0$; тогда условное распределение будет равномерным:
$$p(\lambda|\varphi = 0) = \frac{1}{2\pi} \quad \text{на отрезке } [-\pi, \pi];$$
- с другой стороны, можно зафиксировать долготу, рассмотрев, например, «гринвичский меридиан» $\lambda = 0$, и тогда условное распределение будет совершенно неравномерным:
$$p(\varphi|\lambda = 0) = \frac{1}{2}\cos\varphi \quad \text{на отрезке } \left[-\frac{\pi}{2}, \frac{\pi}{2}\right].$$

Парадокс Бореля — Колмогорова можно разрешить, заметив, что условная вероятность при условии события вероятности ноль (большого круга сферы) на

[1] Если выбирать и широту, и долготу равномерно, то распределение на сфере не будет равномерным, потому что площадь при таком «накрытии» сферы не сохраняется; оставлю вам вывод нужной формулы в качестве упражнения.

самом деле не определена, и когда мы вводим эту вероятность, нам приходится получать её как предел некоторой последовательности условных вероятностей:

- когда мы сходимся к экватору, мы берём последовательность сужающихся горизонтальных колец, то есть фиксируем интервал, в котором должна быть заключена широта, $\varphi \in [a, b]$, и устремляем длину этого интервала к нулю: $a \to 0$, $b \to 0$;
- когда мы сходимся к нулевому меридиану, мы рассматриваем последовательность «клиньев», то есть фиксируем интервал, в котором должна быть долгота $\lambda \in [a, b]$, и устремляем к нулю его длину.

Парадокс возникает потому, что у этих двух процессов разные пределы — оставлю вам в качестве упражнения в этом убедиться. Нам же понадобится только самый общий вывод из этой неприятной ситуации: об условной вероятности $p(X|Y)$ лучше говорить только в том случае, когда $p(Y) > 0$.

И всё-таки совсем обойти парадокс Бореля — Колмогорова нам не удастся. Дело в том, что в машинном обучении нам потребуется сплошь и рядом говорить о *непрерывных* условных вероятностях, то есть об условных плотностях, которые определяются точно так же:

$$p(x|y) = \frac{p(x, y)}{p(y)}, \tag{2.4}$$

и точно так же определены, только если $p(y) > 0$. Формально говоря, здесь возникают некоторые дополнительные технические трудности. Например, тот самый парадокс Бореля — Колмогорова говорит нам, что условные плотности неустойчивы относительно замены координат: стоило повернуть сферу так, чтобы экватор стал меридианом, и условная плотность стала совсем другой. Однако в нужных нам приложениях эти трудности возникать не будут. В дальнейшем мы просто будем считать, что условные плотности определены формулой (2.4), и не будем пытаться слишком пристально в это определение вглядываться.

Вернёмся к определению условной вероятности. Если применить его два раза, разложив $p(X, Y)$ сначала в произведение $p(X|Y)\,p(Y)$, а затем в произведение $p(Y|X)\,p(X)$, мы получим так называемую *формулу полной вероятности*:

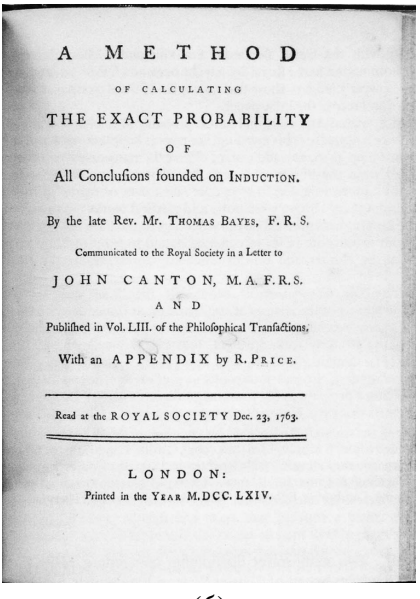
$$p(X, Y) = p(X|Y)\,p(Y) = p(Y|X)\,p(X). \tag{2.5}$$

Второе равенство в формуле (2.5) можно переписать как

$$p(X|Y) = \frac{p(Y|X)\,p(X)}{p(Y)}. \tag{2.6}$$

Уравнение (2.6) — это знаменитая *теорема Байеса*, которая, как вы увидите ниже, лежит в основе всего машинного обучения, да и не только его. Конечно, мы её получили настолько просто, что из формулы (2.6) трудно понять, в чём заслуга Томаса Байеса (рис. 2.2), вывод ведь был совершенно тривиальным. Но в том

(а)



A METHOD

OF CALCULATING

THE EXACT PROBABILITY

OF

All Conclusions founded on INDUCTION.

By the late Rev. Mr. THOMAS BAYES, F. R. S.

Communicated to the Royal Society in a Letter to

JOHN CANTON, M. A. F. R. S.

AND

Published in Vol. LIII. of the Philosophical Tranfactions,

With an APPENDIX by R. PRICE.

Read at the ROYAL SOCIETY Dec. 23, 1763.

LONDON:

Printed in the YEAR M.DCC.LXIV.

(б)

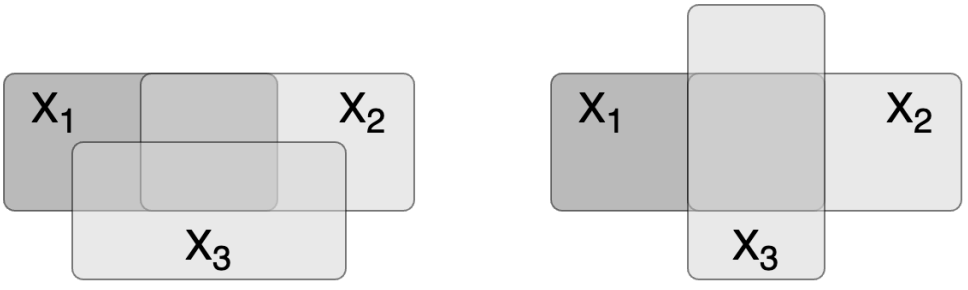
Рис. 2.2. Преподобный Байес: (а) собственной персоной (что, впрочем, сомнительно: портрет этот появился в книге 1936 года об истории страхования жизни без указания источника [445] и с тех пор копируется из книги в книгу; современные исследователи сомневаются, что на нём изображён действительно Томас Байес [55]); (б) в форме посмертно опубликованного эссе о теории вероятностей.

и заслуга, что во времена Байеса этот вывод провести было нельзя — не было ещё таких удобных базовых определений теории вероятностей, не было колмогоровской аксиоматики (дело было за 170 лет до Колмогорова), были только первые попытки что-то формализовать.

Но даже с учётом этого Байес решал куда более сложную задачу, которая в современных терминах звучала бы так: каким будет непрерывное распределение на вероятность орла на монетке θ, если дан набор результатов выпадения этой монетки? Понятно, что если монетка выпала десять раз орлом и один раз решкой, то распределение $p(\theta)$ должно хотя бы немного «перекоситься» в сторону значений θ, близких к единице, но насколько именно? Мы с вами решим эту задачу в разделе 2.4, но сначала нужно вспомнить ещё несколько важных понятий.

2.1.4. Независимость и условная независимость

В разделе 2.2 мы поговорим о теореме Байеса гораздо подробнее, ведь это главная формула всего машинного обучения, а детальным анализом бросков монетки с байесовской точки зрения займёмся в разделе 2.4. А сейчас давайте рассмотрим последнее базовое понятие теории вероятностей, которым машинное



(а) Независимые события (б) Попарно независимые события

Рис. 2.3. Примеры независимых и попарно независимых событий

обучение пользуется постоянно, — *независимость* (independence). Формально она определяется очень просто: два события X и Y называются *независимыми*, если их совместная вероятность равна произведению их вероятностей:

$$p(X,Y) = p(X)\,p(Y). \tag{2.7}$$

Понятие независимости можно продолжить на случайные величины: ξ и φ, определённые на одном и том же вероятностном пространстве с пространством исходов $\mathcal{S}$, называются *независимыми*, если события $\{\xi \le a\}$ и $\{\varphi \le b\}$ независимы в смысле определения (2.7) для любых $a,b \in \mathbb{R}$. Иначе говоря, совместная функция распределения ξ и φ должна раскладываться в произведение

$$F_{\xi,\varphi}(a,b) = F_{\xi}(a)F_{\varphi}(b).$$

В непрерывном случае, соответственно, плотность совместного распределения должна быть равна произведению плотностей $p(x)$ и $p(y)$ (мы снова предполагаем, что плотности существуют всегда, пусть иногда и в виде обобщённых функций):

$$p(x,y) = p(x)\,p(y).$$

Это определение естественным образом обобщается на несколько событий или несколько случайных величин: $X_1,\ldots,X_n$ называются *независимыми в совокупности* или просто *независимыми*, если для любых $1 \le i_1 < i_2 < \ldots < i_k \le n$

$$p\left(X_{i_1},X_{i_2},\ldots,X_{i_k}\right) = p\left(X_{i_1}\right)p\left(X_{i_2}\right)\ldots p\left(X_{i_k}\right).$$

Заметим, что попарная независимость (когда события X_i и X_j независимы для любых двух индексов $i \ne j$) не эквивалентна независимости всех случайных величин. Приведу простой пример, показывающий это; он известен как *тетраэдр Бернштейна* — в честь автора известного советского учебника теории вероятностей, где этот пример впервые появился [683]. Представьте себе игральную

кость в виде тетраэдра, с четырьмя элементарными исходами $\{1,2,3,4\}$, и три случайных события: $X_1 = \{1,4\}$, $X_2 = \{2,4\}$ и $X_3 = \{3,4\}$. Иначе говоря, три грани тетраэдра раскрашены в три разных цвета, а четвёртая грань считается «раскрашенной» во все три цвета одновременно; рисунок 2.4, *а* иллюстрирует этот пример. Тогда каждая пара событий здесь независима, у них $p(X_i) = p(X_j) = \frac{1}{2}$ и $p(X_i, X_j) = \frac{1}{4}$ для любых $i \neq j$. Но все три события в совокупности в данном случае не будут независимы, ведь $p(X_1,X_2,X_3) = \frac{1}{4}$, а вовсе не $\frac{1}{8}$: если мы знаем, что выпал исход из $X_1 \cap X_2$ (одновременно красный и синий цвета), то он уже неизбежно должен быть исходом 4 и, следовательно, попасть в X_3 тоже (то есть и жёлтый цвет тоже выпал). Этот пример для трёх событий показан на рис. 2.3 в виде диаграммы Венна.

Неформально говоря, независимость двух случайных величин значит, что если мы узнаем значение одной из величин, то это не даст нам никакой информации о значении другой, не меняет нашу оценку вероятностей различных её исходов; об информационной стороне теории вероятностей мы подробнее поговорим в разделе 3.4. Но зависимость не нужно путать с наличием причинно-следственных связей или вообще каким-то интуитивным понятием «связанности»: это совершенно разные вещи! Формально независимыми могут быть величины, которые «на самом деле» тесно связаны, и наоборот.

В качестве житейского примера давайте рассмотрим обычную электрическую лампочку B, управляющуюся двумя выключателями, S_1 и S_2, так, чтобы её можно было включить и выключить любым из них; в быту такая схема называется проходными выключателями, и часто это очень удобное решение. Пусть, например, лампочка горит, $B = 1$, если выключатели находятся в разных положениях, $S_1 \neq S_2$, и не горит, если в одном и том же, $S_1 = S_2$; тогда в терминах булевой логики получается, что лампочка реализует функцию XOR, $B = S_1 \oplus S_2$ (см. рис. 2.4, *б*).

Предположим, что положение выключателей представляет собой независимые случайные величины, полученные подбрасыванием двух независимых монеток. Тогда очевидно, что состояние лампочки имеет прямую причинно-следственную связь с положениями выключателей, причём даже не стохастическую, а вполне детерминированную: если я переключу любой выключатель, состояние лампочки изменится. Но случайные величины B и S_1 оказываются независимыми: не зная S_2, нельзя получить никакой информации о состоянии лампочки на основании положения выключателя S_1[1].

[1] На том же принципе основан простейший метод криптографии с закрытым ключом, *одноразовый блокнот* (one-time pad): если у вас есть возможность заранее передать своему собеседнику строчку случайных битов длиной в предполагаемое сообщение, можно будет взять XOR сообщения и этих битов, и в результате получится строчка, неотличимая от случайной, без всяких шансов на расшифровку. Достаточность ключа длиной в сообщение для идеальной зашифровки очевидна, а необходимость доказал Клод Шеннон, когда закладывал основы теории информации [530], но это уже другая история.

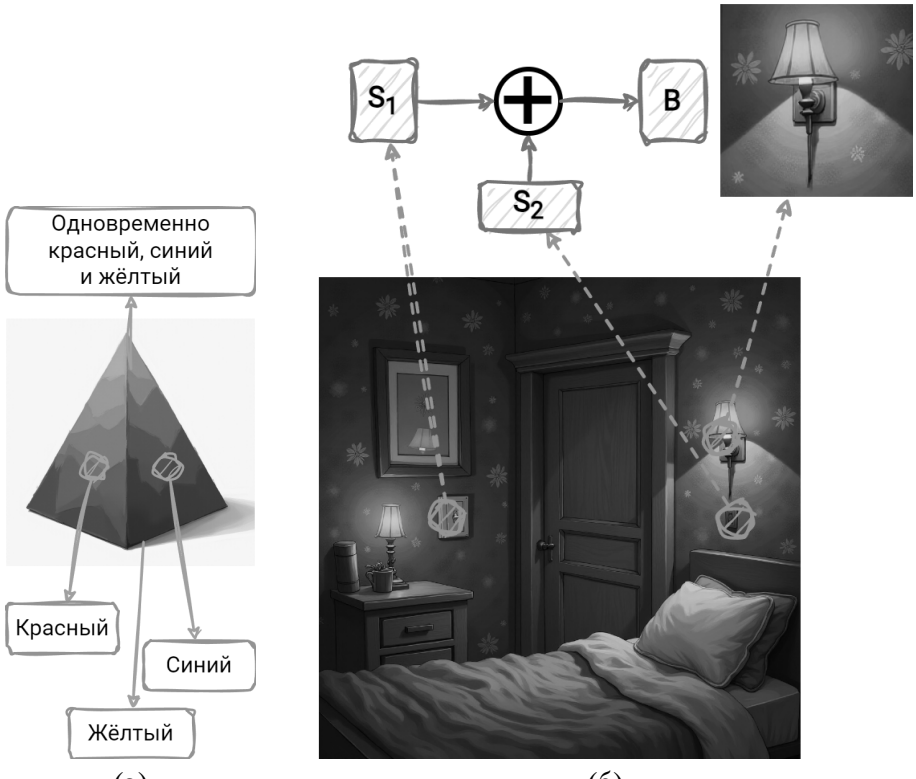


Рис. 2.4. Примеры независимых и зависимых величин: (а) тетраэдр Бернштейна; (б) проходные выключатели

Независимость можно естественным образом продолжить до понятия *условной независимости* (conditional independence): события X и Y *условно независимы при условии* Z, если

$$p(X,Y|Z) = p(X|Z)\,p(Y|Z)\,, \tag{2.8}$$

а случайные величины ξ и φ *условно независимы при условии* ψ, если для любых $a, b, c \in \mathcal{S}$

$$p(\xi = a, \varphi = b|\psi = c) = p(\xi = a|\psi = c)\,p(\varphi = b|\psi = c)\,. \tag{2.9}$$

Кстати, если говорить чуть более философски, все рассуждения о вероятностях, которые мы проводим в «обычной жизни», — это всегда рассуждения об условных вероятностях. Как минимум, чтобы начать рассуждать математически,

нужно сделать какие-то предположения о том, что математика действительно моделирует интересующую нас реальную жизнь, и с учётом этих предположений вести вывод.

В частности, условная независимость играет центральную роль и в машинном обучении. Все предположения о структурах разных моделей машинного обучения, которые мы будем делать, будут предположениями о независимости одних случайных величин при условии других случайных величин. Например, в разделе 2.2 мы поговорим о том, что основным вероятностным предположением в машинном обучении будет условная независимость точек данных при условии параметров модели — без этого предположения вообще ничего вычислить бы не получилось.

2.1.5. Моменты случайной величины: ожидание и дисперсия

Ещё одно важное понятие из теории вероятностей, без которого нам будет не обойтись, — это понятие *математического ожидания* (expectation, expected value). Поскольку случайные величины — это числовые функции, логично задаться вопросом, какие значения у них будут в среднем выпадать. Ожидание $\mathbb{E}[\xi]$ случайной величины ξ — это её среднее значение, результат, взвешенный вероятностями различных исходов. Для дискретной случайной величины это буквально взвешенная сумма: если случайная величина ξ задана на дискретном вероятностном пространстве $\langle \mathcal{S}, \mathcal{X}, p \rangle$ с $\mathcal{S} = \{x_1, x_2, \ldots, x_N\}$, то

$$\mathbb{E}[\xi] = p(x_1)\,\xi(x_1) + p(x_2)\,\xi(x_2) + \ldots + p(x_N)\,\xi(x_N).$$

Например, ожидание численного значения, выпавшего на одном шестигранном кубике, будет равно

$$\mathbb{E}[\xi] = \frac{1}{6} \cdot 1 + \frac{1}{6} \cdot 2 + \ldots + \frac{1}{6} \cdot 6 = \frac{7}{2}.$$

А для непрерывной случайной величины f сумма превращается в интеграл:

$$\mathbb{E}[f] = \int_{\mathcal{S}} p(x) f(x) \mathrm{d}x.$$

К примеру, ожидание случайной величины $f(x) = x$, равномерно распределённой на отрезке $[a,b]$, то есть взятой с распределением, плотность которого постоянна на этом отрезке, а значит, равна $p(x) = \frac{1}{b-a}$, будет равно

$$\mathbb{E}[f] = \int_a^b p(x) f(x) \mathrm{d}x = \int_a^b \frac{1}{b-a} x \mathrm{d}x = \frac{1}{b-a} \left(\frac{b^2}{2} - \frac{a^2}{2} \right) = \frac{b+a}{2},$$

то есть среднему арифметическому концов отрезка — вполне, простите за каламбур, ожидаемый результат.

Заметим, что поскольку ожидание случайной величины — это интеграл от неё (или сумма, что то же самое), ожидание представляет собой линейный оператор — для констант α и β и случайных величин ξ и φ

$$\mathbb{E}\left[\alpha\xi + \beta\varphi\right] = \alpha\mathbb{E}\left[\xi\right] + \beta\mathbb{E}\left[\varphi\right].$$

Ожидание не мультипликативно: в общем случае $\mathbb{E}\left[\xi \cdot \varphi\right] \neq \mathbb{E}\left[\xi\right]\mathbb{E}\left[\varphi\right]$. Однако если случайные величины ξ и φ, определённые на одном и том же вероятностном пространстве, независимы, то

$$\mathbb{E}\left[\xi \cdot \varphi\right] = \int_{\mathcal{S}} \xi(x)\varphi(y)p\left(x,y\right)\mathrm{d}x\mathrm{d}y = \int_{\mathcal{S}} \xi(x)\varphi(y)p\left(x\right)p\left(y\right)\mathrm{d}x\mathrm{d}y = \\ = \int_{\mathcal{S}} \xi(x)p\left(x\right)\mathrm{d}x \int_{\mathcal{S}} \varphi(y)p\left(y\right)\mathrm{d}y = \mathbb{E}\left[\xi\right] \cdot \mathbb{E}\left[\varphi\right].$$

Как и всё остальное в теории вероятностей, понятие математического ожидания не всегда соответствует нашей базовой интуиции. Например, нельзя рассчитывать на то, что ожидания будут конечными. В так называемом *санкт-петербургском парадоксе*, изобретённом ещё семьёй Бернулли[1], участнику азартной игры предлагают выигрыш 2^k долларов с вероятностью 2^{-k}: бросаем монетку, в случае орла выплачиваем 2 доллара и заканчиваем игру, в случае решки бросаем ещё раз и платим за орла уже 4 доллара, в случае двух решек подряд за орла на третьем месте заплатим 8 долларов и так далее. Математическое ожидание выигрыша в такой игре бесконечно:

$$\mathbb{E}\left[\xi\right] = \frac{1}{2} \cdot 2 + \frac{1}{4} \cdot 4 + \frac{1}{8} \cdot 8 + \ldots = 1 + 1 + 1 + \ldots = \infty.$$

Конечно, ничего парадоксального в бесконечном ожидании нет. «Парадоксом» этот пример называют потому, что, казалось бы, за игру с бесконечным ожиданием всякий был бы готов заплатить любую конечную сумму денег, но ещё Даниил Бернулли писал, что «...любой разумный человек с большим удовольствием продал бы свой шанс [сыграть в эту игру] за двадцать дукатов». Подумайте: сколько вы были бы готовы заплатить за то, чтобы сыграть в эту игру? Ясно, что больше двух долларов, ведь это минимальный гарантированный выигрыш, но согласитесь, что сотню долларов вы бы уже вряд ли отдали за такое заманчивое предложение, — но ведь ряд действительно расходится!

Есть несколько вариантов разрешения парадокса. Во-первых, ожидаемая полезность денег вряд ли линейна: второй миллион не так полезен человеку, как

[1] Николай Бернулли сформулировал парадокс в 1713 году в письме к своему коллеге Пьеру Ремону де Монмору, но название своё он получил благодаря анализу, проделанному Даниилом Бернулли, который в это время жил в Санкт-Петербурге и опубликовал свои результаты в 1738 году в трудах Императорской академии наук [62].

первый. Например, если игрок считает, что полезность выигранных денег для него логарифмическая, $U(k) = \ln k$ (это вполне разумное предположение), то полезность лотереи для человека, у которого уже есть c долларов и которому нужно заплатить за игру w долларов, составляет

$$\Delta\mathbb{E}\left[U\right] = \sum_{k=1}^{\infty} \frac{1}{2^k}\left(\ln\left(w + 2^k - c\right) - \ln\left(w\right)\right).$$

А этот ряд уже сходится: например, миллионеру, у которого $c = 10^6$, разумно будет в этих предположениях заплатить вплоть до 20,88 доллара за игру. Другое очевидное разрешение парадокса состоит в том, что бесконечных выплат в жизни не бывает, и выплаты ограничены капиталом нашего условного казино. Например, если за выплаты ручается Илон Маск со всем своим состоянием в 250 миллиардов долларов (данные на сентябрь 2024 года), то ожидаемый доход от такой игры составляет всего лишь 38 долларов; обратите внимание, как далеко в этом примере находится вся эта обещанная (и формально вполне реальная!) бесконечность.

Математическое ожидание — это первый пример так называемых *моментов* (moments) случайной величины; k-й момент случайной величины ξ определяется как ожидание k-й степени ξ:

$$\mu_k\left[\xi\right] = \mathbb{E}\left[\xi^k\right] = \int_{\mathcal{S}} p\left(x\right)\xi^k(x)\mathrm{d}x. \tag{2.10}$$

Очевидно, что первый момент — это само математическое ожидание и есть.

Нам также часто будет нужен второй момент, ожидание квадрата случайной величины $\mathbb{E}\left[\xi^2\right]$, и тесно с ним связанное понятие *дисперсии* (variance) случайной величины. По определению, дисперсия — это второй *стандартизированный* момент, то есть ожидание квадрата отклонения от среднего:

$$\mathrm{Var}\left[\xi\right] = \mathbb{E}\left[\left(\xi - \mathbb{E}\left[\xi\right]\right)^2\right] = \int_{\mathcal{S}} \left(\xi(x) - \mathbb{E}\left[\xi\right]\right)^2 p\left(x\right)\mathrm{d}x, \tag{2.11}$$

но также очень часто бывает полезным переписать это определение как

$$\begin{aligned}\mathrm{Var}\left[\xi\right] = \mathbb{E}\left[\left(\xi - \mathbb{E}\left[\xi\right]\right)^2\right] = \mathbb{E}\left[\xi^2 - 2\xi\mathbb{E}\left[\xi\right] + \mathbb{E}\left[\xi\right]^2\right] = \\ = \mathbb{E}\left[\xi^2\right] - 2\mathbb{E}\left[\xi\right]\mathbb{E}\left[\xi\right] + \mathbb{E}\left[\xi\right]^2 = \mathbb{E}\left[\xi^2\right] - \mathbb{E}\left[\xi\right]^2.\end{aligned} \tag{2.12}$$

В качестве примера давайте подсчитаем дисперсию равномерного распределения на отрезке $[a,b]$ — это уже не такой тривиальный пример! Сначала найдём ожидание квадрата

$$\mathbb{E}\left[x^2\right] = \int_{-\infty}^{\infty} x^2 p(x)\mathrm{d}x = \int_a^b \frac{x^2}{b-a}\mathrm{d}x = \frac{b^3 - a^3}{3(b-a)} = \frac{a^2 + ab + b^2}{3},$$

а затем подставим результат в формулу (2.12):

$$\operatorname{Var}[x] = \mathbb{E}\left[x^2\right] - \mathbb{E}[x]^2 = \frac{a^2+ab+b^2}{3} - \frac{(a+b)^2}{4} = \frac{(b-a)^2}{12}.$$

Для машинного обучения также потребуются многомерные аналоги всех этих понятий. Природа первого момента, математического ожидания, в многомерном случае не меняется — это по-прежнему интеграл

$$\boldsymbol{\mu} = \mathbb{E}[\boldsymbol{X}] = \int_{\mathcal{S}} \boldsymbol{x} p(\boldsymbol{x})\, \mathrm{d}\boldsymbol{x},$$

просто для многомерного $\boldsymbol{x}$. А дисперсия, что логично, превращается в *матрицу ковариаций* (covariance matrix), на диагонали которой находятся дисперсии компонентов ξ, а вне диагонали стоят *ковариации* (covariances) между этими компонентами:

$$\operatorname{Cov}(\xi, \eta) = \mathbb{E}[(\xi - \mathbb{E}\xi)(\eta - \mathbb{E}\eta)].$$

Матрицу ковариаций логично ввести для двух разных случайных величин:

$$\operatorname{Cov}(\boldsymbol{X}, \mathbf{Y}) = \mathbb{E}\left[(\boldsymbol{X} - \mathbb{E}\boldsymbol{X})(\mathbf{Y} - \mathbb{E}\mathbf{Y})^\top\right], \text{ то есть } \operatorname{Cov}(\boldsymbol{X}, \mathbf{Y})_{i,j} = \operatorname{Cov}\left(X_i, Y_j\right),$$

и тогда для одного и того же вектора мы получаем

$$\operatorname{Cov}(\boldsymbol{X}) = \operatorname{Cov}(\boldsymbol{X}, \boldsymbol{X}) = \mathbb{E}\left[(\boldsymbol{X} - \mathbb{E}\boldsymbol{X})(\boldsymbol{X} - \mathbb{E}\boldsymbol{X})^\top\right] = \\ = \mathbb{E}\left[\boldsymbol{X}\boldsymbol{X}^\top\right] - \mathbb{E}[\boldsymbol{X}] \cdot \mathbb{E}\left[\boldsymbol{X}^\top\right].$$

Примеры многомерных случайных величин, которые потребуются в этой книге, мы увидим позже, в разделах 3.1 и 3.2.

Итак, мы рассмотрели основные понятия теории вероятностей, которые понадобятся нам в этой книге. В табл. 2.1 показан краткий итог всего того, что мы здесь обсуждали. Скорее всего, все эти понятия уже были вам знакомы, и в дальнейшем я буду предполагать весьма творческое владение основами теории вероятностей, которое, конечно, нельзя было получить из этого параграфа; но повторить, думаю, всё-таки стоило.

*2.1.6. Как ещё можно ввести понятие вероятности

Завершу этот раздел небольшим лирическим отступлением. Стоит отметить, что колмогоровская аксиоматика — это хоть и общепринятый, но вовсе не единственный способ определить, что такое вероятность. Например, я рекомендую

Таблица 2.1. Основные понятия теории вероятностей

Понятие	**Краткое определение**	**Пример**
Вероятностное пространство	Тройка $\langle \mathcal{S},\mathcal{X},p\rangle$, где $\mathcal{S}$ — исходы, $\mathcal{X}$ — события, p — вероятность	Для честного кубика с шестью гранями $\mathcal{S}=\{x_1,\ldots,x_6\}$, $p(x_i)=\frac{1}{6}$
Плотность	Функция $p(x)$, для которой $\int_A p(\boldsymbol{x})\,\mathrm{d}\boldsymbol{x}=p(X\in A)$	Для равномерного распределения на отрезке $[a,b]$ $p(x)=\frac{1}{b-a}$
Функция распределения	Интеграл от плотности для $\mathcal{S}=\mathbb{R}$: $F(x)=\int_{-\infty}^{x} p(y)\,\mathrm{d}y$.	Для равномерного распределения на отрезке $[a,b]$ $F(x)=\begin{cases}0, & x\le a,\\ \frac{x-a}{b-a}, & x\in[a,b],\\ 1, & x\ge b\end{cases}$
Совместное распределение	$p(x,y)$ из вероятностного пространства $\langle \mathcal{S}_X\times\mathcal{S}_Y,\mathcal{X}\times\mathcal{Y},p\rangle$, $p(X,Y)=p(x\in X\wedge y\in Y)$	В совместном распределении двух честных кубиков $p(\{1,2\},\{5,6\})=\frac{4}{36}=\frac{1}{9}$
Условная вероятность	$p(X\|Y)=\frac{p(X\cap Y)}{p(Y)}$, $p(x\|y)=\frac{p(x,y)}{p(y)}$	Для честного кубика $p(\{1,2,3\}\|\{2,4,6\})=\frac{1}{3}$
Независимые события	$p(X,Y)=p(X)\,p(Y)$	Для честного кубика $\{1,2,3\}$ и $\{2,4,6\}$ зависимы, а $\{1,2\}$ и $\{2,4,6\}$ независимы
Условно независимые события	$p(X,Y\|Z)=p(X\|Z)\,p(Y\|Z)$	Для честного кубика $\{1,2,3\}$ и $\{2,4,6\}$ условно независимы при условии $\{1,2,5,6\}$
Случайная величина	Измеримая числовая функция $X:\mathcal{S}\to\mathbb{R}$	Значение на грани кубика: $f(x_i)=i$
Независимые случайные величины	$p(x,y)=p(x)\,p(y)$	Подбросим монетку десять раз; число орлов в первых пяти и во вторых пяти бросках независимы, а число орлов в бросках 1–6 и бросках 4–10 зависимы
Условно независимые случайные величины	$p(x,y\|z)=p(x\|z)\,p(y\|z)$	Подбросим монетку десять раз; число орлов в бросках 1–6 и 4–10 условно независимы при условии числа орлов в бросках 4–6
Математическое ожидание	$\mathbb{E}[f]=\int_{\mathcal{S}} f(x)p(x)\mathrm{d}x$	Для значения на грани кубика, $f(x_i)=i$, $\mathbb{E}[f]=\frac{7}{2}$
Дисперсия	$\mathrm{Var}[\xi]=\mathbb{E}\left[(\xi-\mathbb{E}[\xi])^2\right]=\mathbb{E}\left[\xi^2\right]-\mathbb{E}[\xi]^2$	Для значения на грани кубика, $f(x_i)=i$, $\mathrm{Var}[f]=\mathbb{E}\left[f^2\right]-\mathbb{E}[f]^2=\frac{1^2+\ldots+6^2}{6}-\left(\frac{7}{2}\right)^2=\frac{105}{36}$

прочесть изложение альтернативного способа ввести вероятность в книге Эдвина Джейнса [286]: этот способ начинается с интуитивных требований к численной формализации *достоверности* (plausibility) логических высказываний[1] и постепенно приходит к той же формальной системе. Материал отсюда и до конца раздела требует некоторой подготовки в математическом анализе, и его можно пропустить без ущерба для дальнейшего понимания.

Чтобы проиллюстрировать дух подхода Джейнса, приведу один пример его рассуждений: предположим, что мы хотим выразить достоверность конъюнкции $A \wedge B$ при условии C. Логика подсказывает, что для установления истинности $A \wedge B \mid C$ можно сначала установить истинность A при условии C, а затем истинность B при условии A и C, а можно, наоборот, начать с B и затем перейти к A, и в обоих случаях должен получиться один и тот же результат.

Таким образом, достоверность u для $A \wedge B \mid C$ можно определить:

- либо как функцию от достоверности x для $A \mid C$ и достоверности y для $B \mid A \wedge C$, $u = F(x, y)$;
- либо как функцию от достоверности w для $B \mid C$ и достоверности v для $A \mid B \wedge C$, $u = F(w, v)$.

Естественно предположить, что F — неубывающая функция от каждого из своих аргументов: если A становится более достоверным, а достоверность B не меняется, то $A \wedge B$ может стать только более достоверным. Для удобства также предположим, что F — дифференцируемая функция (существуют доказательства и без этого предположения, но они гораздо сложнее [9]).

Законы логики говорят, что F должна быть ассоциативной: если мы устанавливаем истинность $A \wedge B \wedge C$ при условии D, мы можем начать с того, что рассмотреть $B \wedge C$ как единое целое, а можем сначала отделить C, оставив вместе $A \wedge B$, и результат не должен измениться. Иными словами,

$$F(x, F(y, z)) = F(F(x, y), z). \tag{2.13}$$

Оказывается, что ассоциативности уже достаточно для того, чтобы существенно ограничить возможный вид функции F. Обозначим для краткости $u = F(x,y)$ и $v = F(y,z)$, то есть запишем уравнение (2.13) как $F(x, v) = F(u, z)$. Продифференцируем уравнение (2.13) по x и y как сложную функцию слева и справа:

$$\frac{\partial F(x,v)}{\partial x} = \frac{\partial F(u,z)}{\partial u}\frac{\partial F(x,y)}{\partial x},$$
$$\frac{\partial F(x,v)}{\partial v}\frac{\partial F(y,z)}{\partial y} = \frac{\partial F(u,z)}{\partial u}\frac{\partial F(x,y)}{\partial y}.$$

[1] Наверное, точнее было бы перевести здесь *plausibility* как «правдоподобие», но совсем другое правдоподобие (likelihood) скоро станет одним из центральных терминов этой книги, и лучше с ним ничего не путать.

Выразим $\frac{\partial F(u,z)}{\partial u}$ из этих уравнений и приравняем друг к другу:

$$G(x,v)\frac{\partial F(y,z)}{\partial y} = G(x,y), \qquad \text{где} \quad G(a,b) = \frac{\frac{\partial F(a,b)}{\partial b}}{\frac{\partial F(a,b)}{\partial a}}. \tag{2.14}$$

В последней формуле можно выразить $\frac{\partial F(y,z)}{\partial y}$ через G и получить

$$G(x,v)\frac{\partial F(y,z)}{\partial z} = G(x,y)\,G(y,z). \tag{2.15}$$

А теперь главный трюк: продифференцируем уравнения (2.14) и (2.15), первое по z, а второе по y. В формуле (2.14) вспомним, что $v = F(y,z)$, то есть $G(x,v)$ нужно дифференцировать, а вот правая часть даёт ноль:

$$0 = \frac{\partial G(x,y)}{\partial z} = \frac{\partial\left(G(x,v)\frac{\partial F(y,z)}{\partial y}\right)}{\partial z} = \\ = G(x,v)\frac{\partial^2 F(y,z)}{\partial y \partial z} + \frac{\partial F(y,z)}{\partial y}\frac{\partial G(x,v)}{\partial v}\frac{\partial F(y,z)}{\partial z};$$

аналогично из формулы (2.15) получается, что

$$\frac{\partial\left(G(x,y)\,G(y,z)\right)}{\partial y} = \frac{\partial\left(G(x,v)\frac{\partial F(y,z)}{\partial z}\right)}{\partial y} = \\ = G(x,v)\frac{\partial^2 F(y,z)}{\partial z \partial y} + \frac{\partial F(y,z)}{\partial z}\frac{\partial G(x,v)}{\partial v}\frac{\partial F(y,z)}{\partial y}.$$

Справа получилось одно и то же, а значит,

$$\frac{\partial\left(G(x,y)\,G(y,z)\right)}{\partial y} = 0, \tag{2.16}$$

то есть функция $G(x,y)\,G(y,z)$ не зависит от y!

Какой может быть функция G, удовлетворяющая свойству (2.16)? Если все участвующие в выводе функции дифференцируемы, это на самом деле несложный вопрос: второй аргумент функции $G(x,y)$ не должен встречаться в произведении $G(x,y)\,G(y,z)$, то есть то, что зависит от второго аргумента, должно сокращаться с тем, что зависит от первого. Самый общий вид функции, удовлетворяющей такому свойству, выглядит как

$$G(x,y) = r\frac{H(x)}{H(y)}, \tag{2.17}$$

где r — константа, а H — произвольная дифференцируемая функция.

Подставим теперь формулу (2.17) в уравнения (2.14) и (2.15):

$$\frac{\partial F\,(y,z)}{\partial y} = \frac{H(v)}{H(y)}, \qquad \frac{\partial F\,(y,z)}{\partial z} = r\frac{H(v)}{H(z)},$$

и теперь можно переписать $\mathrm{d}v = \mathrm{d}F(y,z) = \frac{\partial F(y,z)}{\partial y}\mathrm{d}y + \frac{\partial F(y,z)}{\partial z}\mathrm{d}z$ как

$$\frac{\mathrm{d}v}{H(v)} = \frac{\mathrm{d}y}{H(y)} + r\frac{\mathrm{d}z}{H(z)},$$

то есть

$$w(F\,(y,z)) = w(v) = w(y)w^r(z), \qquad \text{где} \quad w(x) = ce^{\int_{-\infty}^{x}\frac{\mathrm{d}x}{H(x)}}. \tag{2.18}$$

Взяв функцию w от равенства $F\,(x,v) = F\,(u,z)$, мы получим

$$w(x)w^r(v) = w(u)w^r(z),$$

и, применив результат (2.18) ещё раз, мы получим

$$w(x)w^r(y){w^r}^2(z) = w(x)w^r(y)w^r(z),$$

откуда сразу видно, что нетривиальное решение возможно только для $r = 1$.

Таким образом, после всех выкладок мы получили, что

$$w(F\,(x,y)) = w(x)w(y), \quad \text{или} \quad F\,(x,y) = w^{-1}\,(w(x)w(y))$$

для некоторой функции $w(x) = ce^{\int_{-\infty}^{x}\frac{\mathrm{d}x}{H(x)}}$, то есть, поскольку на $H(x)$ ограничений не было, для некоторой положительной непрерывной монотонной функции, возрастающей или убывающей в зависимости от знака $H(x)$. Функция w должна удовлетворять той самой ассоциативности:

$$w(A \wedge B \mid C) = w(A \mid B \wedge C)w(B \mid C) = w(B \mid A \wedge C)w(A \mid C). \tag{2.19}$$

Теперь подставим ещё одну интуитивную аксиому: пусть некоторое высказывание A достоверно известно при условии C. Тогда логика требует, чтобы достоверность $A \wedge B \mid C$ была равна достоверности $B \mid C$ для любого B, а также чтобы $A \mid B \wedge C$ было столь же достоверным. Ассоциативность (2.19) в терминах функции w будет означать, что

$$w(B \mid C) = w(A \wedge B \mid C) = w(B \mid C)w(A \mid B \wedge C) = w(B \mid C)w(A \mid C),$$

а значит, достоверно известные факты должны иметь $w(A \mid C) = 1$. С другой стороны, если $A \mid C$ невозможно, то $A \mid B \wedge C$ тоже должно быть невозможным, и аналогичным образом мы получаем

$$w(A \mid C) = w(A \mid C)w(B \mid C). \tag{2.20}$$

Любопытно, что для удовлетворения равенства (2.20) есть две альтернативные возможности:

- можно положить $w(A \mid C) = 0$ для невозможных высказываний; тогда w должна быть монотонно возрастающей функцией от 0 для невозможных высказываний до 1 для достоверных, и мы получим привычную нам теорию вероятностей;
- но можно и взять для невозможных высказываний $w(A \mid C) = \infty$; тогда w оказывается монотонно убывающей функцией от ∞ для невозможных высказываний до 1 для достоверных; это выглядит крайне контринтуитивно, но на самом деле это просто замена $w(x)$ на $\frac{1}{w(x)}$: формулы в такой экзотической репараметризации теории вероятностей будут непривычными, но вся теория получится совершенно эквивалентна обычной колмогоровской теории вероятностей.

Кажется, что это какие-то слишком уж сложные рассуждения для такого простого результата, который можно сразу же получить из колмогоровской аксиоматики. Но на самом деле это в некотором смысле дело вкуса: такой подход начинает с других базовых аксиом, которые могут показаться более интуитивными, а дополнительную работу, которую при этом требуется проделать для получения основных свойств вероятности, нужно сделать только один раз и потом всё равно можно не вспоминать.

Что ж, введение изрядно затянулось, но всё-таки без него было не обойтись как для полноты изложения, так и для того, чтобы напомнить читателю, возможно, слегка позабытые основы и успокоить насчёт того, что более глубоких экскурсов в теорию вероятностей в дальнейшем не будет. А теперь пора поговорить о том, как теория вероятностей пронизывает всю нашу науку, и о том, почему машинное обучение — это развёрнутый пример применения теоремы Байеса.

2.2. Вероятности в машинном обучении: теорема Байеса

I beseech you, in the bowels of Christ, think it possible that you may be mistaken. (Заклинаю вас всеми страстями Христовыми, задумайтесь о том, что можете ошибаться.)

Оливер Кромвель, письмо к Генеральной Ассамблее Церкви Шотландии от 5 августа 1650 г.

2.2.1. Теорема Байеса в машинном обучении

В предыдущем разделе мы пусть очень кратко и не слишком формально, но рассмотрели основные понятия теории вероятностей. Теперь нам нужно понять, как формальные постановки задач машинного обучения вытекают из теоретико-вероятностных соображений и как вероятностный вывод поможет нам их решать.

Для этого давайте вернёмся к теореме Байеса[1] (2.6) и перепишем её в других обозначениях:

$$p(\boldsymbol{\theta}|D) = \frac{p(\boldsymbol{\theta})\,p(D|\boldsymbol{\theta})}{p(D)}. \tag{2.21}$$

Смысл появляющихся здесь случайных величин таков:

- $\boldsymbol{\theta}$ — это параметры модели, те величины, которые мы хотим оценить; например, в разделе 2.4 мы будем обучать параметр монетки, то есть пытаться по результатам бросков оценить значение вероятности выпадения орла, а в главе 4 будем обучать параметры линейной регрессии, то есть коэффициенты в линейной комбинации признаков;
- D — это данные, результаты экспериментов, на которых мы, собственно, собираемся обучаться; в случае монетки это будут орлы и решки, которые на ней выпали, а в случае линейной регрессии — точки с известными значениями целевой переменной.

[1] *Томас Байес* (Thomas Bayes, 1701–1761) — английский статистик, философ и богослов. Байес учился в университете Эдинбурга, а затем всю жизнь проработал сначала в Лондоне, где помогал своему отцу-священнику, а потом в Mount Sion Chapel в графстве Кент. За всю жизнь Байес опубликовал всего две работы: изданную в 1731 году книгу «О божественной благодати» [47], и «Введение в доктрину флюксий» [48], в котором Байес защищал разработанный Ньютоном анализ от критики Джорджа Беркли. За последнюю работу Байеса, видимо, и приняли в Королевское общество. А основополагающий труд по теории вероятностей был на самом деле извлечён из записей Байеса уже после его смерти и представлен в Королевском обществе Ричардом Прайсом в 1763 году, через два года после смерти Байеса [49].

Теперь поговорим о распределениях, которые участвуют в формуле (2.21). Их в машинном обучении обычно называют и интерпретируют следующим образом.

1. Распределение $p(D|\boldsymbol{\theta})$ называется *правдоподобием* (likelihood); это сходится с обычным определением из теории вероятностей, где функцией правдоподобия называют условную вероятность, рассмотренную как функцию от условия, $f(\cdot) = p(x|\cdot)$. В машинном обучении модель обычно и определяют как её функцию правдоподобия, то есть вероятность породить набор данных D при условии параметров $\boldsymbol{\theta}$. Например, для монетки с вероятностью выпадения орла θ датасет D из n орлов и m решек будет иметь правдоподобие $\theta^n(1-\theta)^m$ (см. раздел 2.4, где мы об этом поговорим гораздо подробнее).
2. Распределение $p(\boldsymbol{\theta}|D)$ — это *апостериорное распределение* (posterior); интуитивно говоря, оно отражает то, что мы будем знать о параметрах модели $\boldsymbol{\theta}$ *после* того, как учтём полученные данные D. Отсюда и название — словосочетание *a posteriori* означает «из последующего», то есть «после опыта».
3. Распределение $p(\boldsymbol{\theta})$ — это *априорное распределение* (prior), то есть то представление о параметрах модели, которое у нас было *до* получения данных; это очень важная часть модели, и на протяжении всей этой главы мы будем возвращаться к понятию априорного распределения, а в разделе 2.4 увидим и формальные примеры.
4. Наконец, распределение $p(D)$, которое называют *вероятностью данных* или полной вероятностью (evidence probability), показывает, насколько вероятно получить набор данных D без фиксированного значения $\boldsymbol{\theta}$, то есть усреднённо по всем $\boldsymbol{\theta}$; обычно нет другого способа найти $p(D)$, кроме

$$p(D) = \int p(D, \boldsymbol{\theta})\, \mathrm{d}\boldsymbol{\theta} = \int p(\boldsymbol{\theta})\, p(D|\boldsymbol{\theta})\, \mathrm{d}\boldsymbol{\theta}, \tag{2.22}$$

   и это просто нормировочная константа для апостериорного распределения. Часто теорему Байеса (2.21) даже записывают без $p(D)$:

$$p(\boldsymbol{\theta}|D) \propto p(\boldsymbol{\theta})\, p(D|\boldsymbol{\theta}), \tag{2.23}$$

   где $\propto$ обозначает *пропорциональность*, то есть равенство функций (от $\boldsymbol{\theta}$) с точностью до мультипликативной константы.

Теперь, надеюсь, стало ясно, что теорема Байеса при всей своей тривиальности выражает главную суть машинного обучения: как учесть полученные данные и преобразовать наши *априорные* представления о мире (о параметрах модели), которые были до получения данных, в *апостериорные* представления, учитывающие полученную информацию.

На рис. 2.5 приведён абстрактный пример того, как мы интуитивно используем теорему Байеса в жизни: предположим, что мне предложили попробовать новое, незнакомое мне мороженое, скажем, со вкусом жареных каштанов. Будем

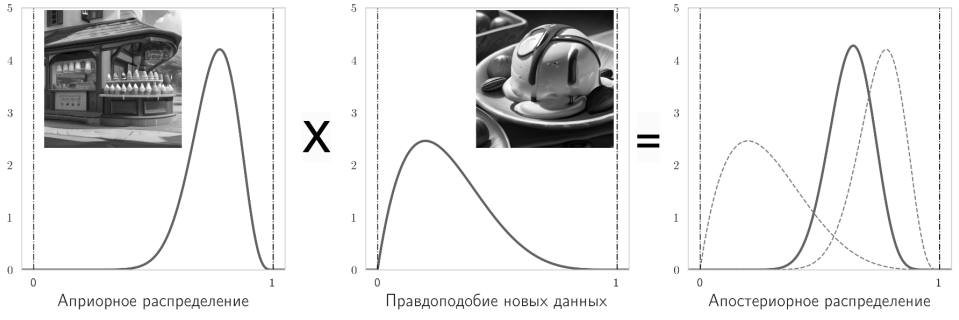


Рис. 2.5. Мороженое со вкусом жареных каштанов: плотности распределения положительного исхода

моделировать то, насколько мне понравится этот новый вкус, одним числовым параметром $\theta \in [0,1]$; можно считать это вероятностью положительного исхода в эксперименте с новым таким же мороженым, а можно — числовой оценкой предпочтения, это неважно. Я люблю мороженое, а каштанов никогда не пробовал, так что моё априорное распределение $p(\theta)$ происходит из истории разных мороженых в моей жизни. В целом мороженое мне чаще нравится, чем наоборот, так что априорное распределение в основном сосредоточено в области больших значений θ.

Но вот я попробовал новое мороженое, и оно оказалось весьма так себе. Конечно, я могу допустить, что именно этот конкретный бренд плохо умеет делать мороженое с каштанами или даже что этот конкретный контейнер хранили плохо и слишком долго. С другой стороны, может оказаться, что это мороженое было лучшим, что на мой вкус можно сделать с каштанами, а другие попытки понравятся мне ещё меньше. Один эксперимент — это всего лишь одна конкретная реализация случайной величины, а не точная информация о её параметрах.

Таким образом, правдоподобие одного эксперимента $p(d|\theta)$ — это не точечное распределение, не оставляющее места для дискуссий, а некоторое колоколообразное распределение вокруг той оценки, которую я сейчас мысленно выставляю этому конкретному мороженому (центральный график на рис. 2.5).

Когда априорное распределение умножается на правдоподобие, получается апостериорное распределение $p(\theta|d)$ (правый график на рис. 2.5; на нём ещё для наглядности пунктиром показаны априорное распределение и правдоподобие). Оно в том или ином виде (не в виде явной функции плотности, разумеется) отложится у меня в голове, и в следующий раз я, наверное, выберу другой вкус. А может быть, дам мороженому со вкусом каштанов ещё один шанс, решив, что неопределённости в $p(\theta|d)$ ещё много и в данном случае одного эксперимента было недостаточно.

Кстати, понятия априорного и апостериорного знания имеют долгую историю: выражения *a priori* и *a posteriori* появлялись ещё в латинских переводах «Начал» Евклида[1], противопоставлялись в философии Платона и Аристотеля, о них рассуждали средневековые схоласты. Но современный смысл этому противопоставлению придали сначала Лейбниц, а потом Иммануил Кант[2] в своей «Критике чистого разума»: он называет априорными суждения, не требующие эмпирической проверки (например, следующие из введённых нами же определений), а апостериорными — суждения, которым нужна верификация опытом (отражающие свойства окружающего нас мира).

А математически теорема Байеса действительно очень проста; она говорит нам, что апостериорное распределение получается умножением априорного распределения на правдоподобие (и, конечно, нормализацией посредством $p(D)$, чтобы опять получилось распределение вероятностей).

Мы гораздо подробнее поговорим обо всех этих понятиях ниже, ведь по сути вся эта книга, да и всё машинное обучение в целом, представляют собой разнообразные творческие применения теоремы Байеса. Упомяну сейчас только одно простое свойство: если априорное распределение $p(\boldsymbol{\theta}) = 0$ для некоторого $\boldsymbol{\theta}$, это значит, что и $p(\boldsymbol{\theta}|D) = 0$ для этого $\boldsymbol{\theta}$, какие бы данные D мы ни получили. А это обычно очень плохая идея: даже если что-то априори представляется невероятным, полностью исключать это вне зависимости от полученного далее опыта представляется разумным разве что для логически невозможных событий, например $2 + 2 = 5$.

---

[1] *Евклид* ($Ευκλιδης$, ум. ок. 300 года до н. э.) — древнегреческий математик, геометр, автор главного учебника геометрии на две тысячи лет вперёд — «Начала» ($Στοιχεια$). О жизни Евклида мало что известно достоверно — пара апокрифов («нет царского пути к геометрии» и «дай ему три обола») да и всё: одни источники отождествляют его с учеником Сократа Евклидом из Мегары, другие (кажется, общепринятые) пишут, что Евклид жил в Александрии при Птолемее I, а третьи считают, что Евклид — это и вовсе коллективный псевдоним вроде Бурбаков. Кроме «Начал», стилу Евклида принадлежит ещё ряд трактатов о геометрии и оптике, в частности о конических сечениях, теории зеркал и перспективе.

[2] *Иммануил Кант* (Immanuel Kant, 1724–1804) — немецкий философ, один из главных мыслителей в западной философии. Начал Кант с космогонии, в своей первой (весьма слабой) работе пытаясь примирить Декарта и Лейбница с точки зрения метафизики, а в книге «Всеобщая естественная история и теория небес» (1755) выдвигая теорию о происхождении Солнечной системы из пылевого облака. А главным делом Канта стали три фундаментальные работы его «критического» периода: «Критика чистого разума» [301, 678, 679], посвящённая эпистемологии, то есть возможностям и пределам познания, «Критика практического разума» [302, 678], посвящённая этике и моральной философии, и «Критика способности суждения» [303, 678], говорящая об эстетике и телеологии; даже просто назвать их вклад в философию в коротком примечании совершенно невозможно. А в жизни Кант был ипохондриком и меланхоликом: он действительно строго соблюдал расписанный по минутам распорядок дня, ритуализировал свои действия (например, ел раз в сутки, причём обязательно в компании, где сотрапезников не менее числа граций и не более числа муз), и находил у себя симптомы всех болезней. Такой строгий режим он разработал для себя сам, потому что не верил, что врачи могут ему помочь (как мы сейчас понимаем, в XVIII веке это было действительно так; во введении к этой книге вы можете найти сноску о гуморальной теории, которая тогда главенствовала в медицине), и прожил почти до 80 лет, ничем серьёзно не болея.

Поэтому обычно рекомендуют выбирать априорное распределение $p(\boldsymbol{\theta})$ так, чтобы оно по возможности нигде не было строго равным нулю. Этот принцип называется *правилом Кромвеля* (Cromwell's rule) благодаря вынесенной в эпиграф цитате Оливера Кромвеля[1]. Имя этому правилу дал статистик Деннис Линдли, который писал об этом так: «...оставьте небольшую вероятность и для того, что Луна сделана из зелёного сыра; эта вероятность может быть всего одной миллионной, но пусть она будет ненулевой, иначе даже армия астронавтов, возвращающихся с образцами сыра, не сможет вас переубедить» [347]. В машинном обучении правило Кромвеля особенно важно при обучении простых моделей вроде монеток и игральных костей: если даже вы никогда не видели, как на кубике выпадает шестёрка, не стоит присваивать этому событию строго нулевую вероятность (разве что если кубик на деле оказался тетраэдром); мы поговорим об этом в контексте наивного байесовского классификатора в разделе 5.6.

2.2.2. Медицинский тест с двусторонней ошибкой

Давайте рассмотрим простейший пример применения теоремы Байеса, который, возможно, набил уже оскомину некоторым читателям, потому что регулярно повторяется из книги в книгу; но потерпите, мы его немного, как говорил Михаил Сергеевич, расширим и углубим.

Этот пример из классической области применения статистики — медицины. Предположим, что некий тест на какую-нибудь болезнь (в наше время пусть это будет COVID-19) имеет вероятность успеха 95 %, причём это двусторонняя вероятность: если вы больны, то тест с вероятностью 95 % покажет, что вы больны, и с вероятностью 5 % — что здоровы, и, наоборот, если вы здоровы, то тест с той же вероятностью 5 % ошибочно назовёт вас больным. Предположим, что я пошёл в больницу, сдал тест и — о ужас! — получил позитивный результат, то есть тест говорит, что я болен. С какой вероятностью у меня действительно COVID-19? Прежде чем двигаться дальше, подумайте, как бы вы ответили на этот вопрос.

Подумали? Есть предположения? А теперь давайте запишем этот пример формально: пусть t (от слова «тест») — это событие «получен положительный тест» и соответственно $\bar{t}$ обозначает отрицательный тест, а d (от слова «болезнь») обозначает событие «я болен COVID-19» и соответственно $\bar{d}$ обозначает событие «я не болен COVID-19». Тогда нам нужно подсчитать $p(d|t)$, а данные в условиях задачи говорят о вероятностях $p(t|d)$, $p(\bar{t}|d)$ и так далее.

[1] *Оливер Кромвель* (Oliver Cromwell, 1599–1658) — лидер Английской революции, лорд-протектор Англии, Шотландии и Ирландии, протестант и вождь индепендентов и вообще очень важный человек в истории Великобритании. В таких сносках мы обычно обозреваем научные достижения, но у Кромвеля с этим негусто, он даже во время обучения в Кембридже предпочитал наукам спорт и разные шумные игры. Зато Кромвелю принадлежат много нововведений в военном деле: он создал «железнобокую» кавалерию (название Ironsides происходит от одного из прозвищ Кромвеля), а затем по её примеру реформировал парламентскую армию, создав весьма успешную «Армию нового образца». Умер Кромвель своей смертью на посту лорда-протектора, и в стране сразу начался хаос.

Запахло теоремой Байеса, так что давайте её и запишем:

$$p(d|t) = \frac{p(d)\,p(t|d)}{p(t)} = \frac{p(d)\,p(t|d)}{p(d)\,p(t|d) + p\left(\bar{d}\right)p\left(t\middle|\bar{d}\right)}. \tag{2.24}$$

Здесь мы встречаемся с первым интересным наблюдением: оказывается, исходный вопрос не имеет однозначного ответа. Мы не можем взять и «обратить» условную вероятность, то есть найти $p(d|t)$ по $p(t|d)$ без дополнительной информации, — для этого, как видно из формулы (2.24), нужно ещё знать $p(d)$, априорную вероятность болезни, то есть в данном случае общую долю заболевших в выборке, из которой я выбран равномерно (и это на самом деле ещё одно предположение, которое тоже может оказаться неверным).

В статистических рассуждениях, подобных приведённому в формуле (2.24), априорную вероятность иногда называют *превалентностью* (prevalence), или *базовой долей* (base rate), то есть речь идёт о распространённости «в общей популяции». С этим связана так называемая ошибка базовой доли (base rate fallacy), ошибочное представление о том, что условные вероятности можно «обращать». Итак, неверно, что если $p(t|d) = 0{,}95$, то и $p(d|t) = 0{,}95$; а что же верно?

Для нашего примера предположим, что всего болезнь имеется у 1 % людей, и отложим на время то, что они разного возраста и профессий, — будем считать, что априорная вероятность заболеть COVID-19 у меня как раз 1 %. Тогда апостериорная вероятность получается равной

$$p(d|t) = \frac{p(d)\,p(t|d)}{p(d)\,p(t|d) + p\left(\bar{d}\right)p\left(t\middle|\bar{d}\right)} = \frac{0{,}01 \times 0{,}95}{0{,}01 \times 0{,}95 + 0{,}99 \times 0{,}05} \approx 0{,}16.$$

Если вы не были заранее знакомы с рассуждениями по теореме Байеса, этот ответ наверняка вас удивил. В такой постановке он может удивить даже опытных врачей: в исследованиях, где действующим врачам-диагностам задавали этот вопрос, только около 15 % респондентов давали правильный ответ [93], и да, это исследование оказалось вполне устойчиво воспроизводимым [220]. Ответ, впрочем, можно сделать гораздо более интуитивным, если перейти от долей и вероятностей к целым числам живых людей. В формулировке «из тысячи пациентов тест выдаёт неправильный ответ пятидесяти, а в целом болезнь имеется у десяти» большинство людей с задачей уже успешно справляются.

Можно «на пальцах» подсчитать ответ и напрямую из вероятностей; для этого можно округлить 0,95 и 0,99 до единицы и прикинуть, что получается в знаменателе теоремы Байеса: вероятность того, что я болен и тест верно это отражает, — примерно одна сотая, а вероятность того, что я не болен, а тест ошибся, — примерно пять сотых. Вот и должны в ответе получаться шансы приблизительно один к пяти, то есть вероятность около одной шестой.

На рис. 2.6 изображён график, который показывает, как апостериорная вероятность оказаться больным после получения данных о результате теста меняется

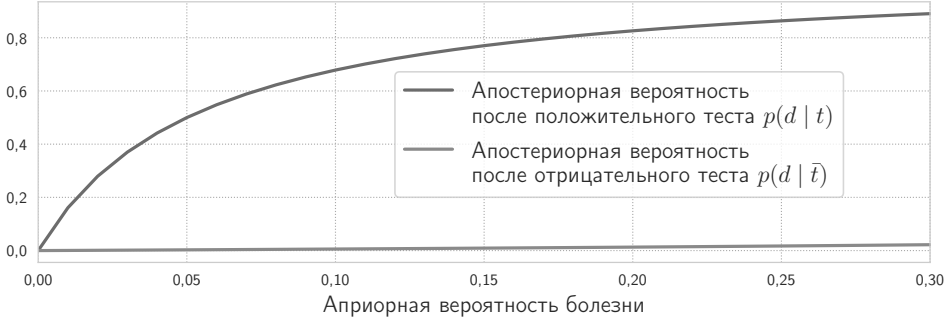


Рис. 2.6. Апостериорная вероятность $p(d \mid t)$ как функция от априорной вероятности $p(d)$ (превалентности) при фиксированных ошибках

по мере того, как возрастает априорная вероятность наличия болезни в популяции. Как видите, до половины вероятность оказаться больным при положительном тесте доходит только тогда, когда априорная вероятность достигает 0,05, то есть сравнивается с вероятностью ложного срабатывания.

2.2.3. Разные виды ошибок и метрики качества классификации

А теперь обещанный следующий шаг: что, если вероятность болезни в 16 % при положительном тесте нас не устраивает (конечно, не устраивает!) и мы хотим улучшить тест? Тест может сделать два разных вида ошибок:

- *ложные*, или *ложноположительные срабатывания* (false positives, FP), которые в классической традиции называют *ошибками первого рода* (type I errors)[1], то есть неверные положительные ответы (тест сказал, что я болен, а я на самом деле здоров);
- *пропуски событий*, или *ложноотрицательные срабатывания* (false negatives, FN), или *ошибки второго рода* (type II errors), то есть неверные отрицательные ответы (тест сказал, что я здоров, а я на самом деле болен).

В нашем изначальном примере вероятности обоих типов ошибок составляли 5 %. Предположим, что медики могут постараться улучшить (уменьшить) либо долю ложных срабатываний, либо долю пропусков событий — на чём им лучше сосредоточиться? Иными словами, какую характеристику теста нужно пытаться улучшать в первую очередь?

[1] Традицию эту можно объяснить, но трудно понять — какой смысл давать разного рода ошибкам названия, которые совершенно не отражают суть и которые вследствие этого, разумеется, все всегда будут путать?.. Я сам сейчас тоже погуглил, прежде чем писать, какой номер какой ошибке соответствует.

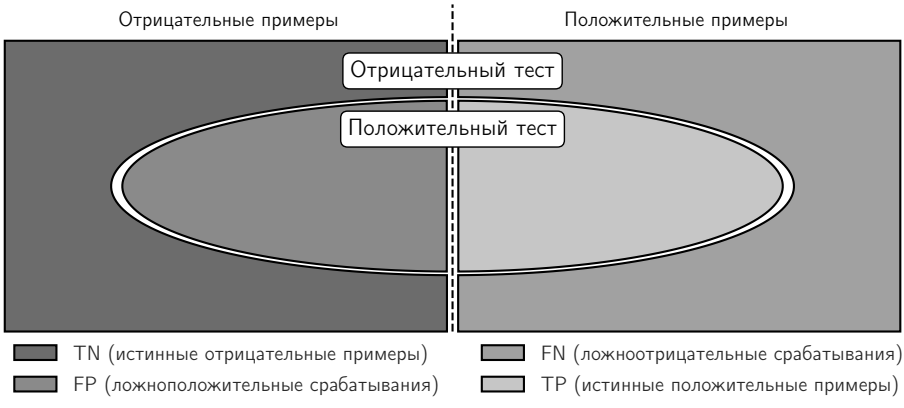


Рис. 2.7. Различные возможные исходы бинарного теста

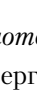
Для разных показателей качества классификации используется множество разных терминов, причём разбираться в них, к сожалению, в жизни всё-таки частенько приходится. Так что я попытался все наиболее часто употребляемые варианты собрать ниже и проиллюстрировать при помощи рис. 2.7. Рисунок нужно читать следующим образом: на схеме слева от вертикальной черты находятся отрицательные примеры (здоровые люди), а справа — положительные (больные). Эллипс показывает результаты теста: внутри эллипса тест положительный, снаружи — отрицательный.

Таким образом, рисунок определяет четыре части, на которые разбивается вся выборка возможных примеров:

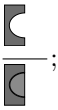
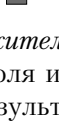
- TP (true positive, истинно положительные примеры), где тест корректно выдаёт положительный результат; на рис. 2.7 это часть ;
- TN (true negative, истинно отрицательные примеры), где тест корректно выдаёт отрицательный результат, то есть на рис. 2.7;
- FP (false positive, ложноположительные примеры), где тест выдаёт положительный результат, а должен быть отрицательный; это на рисунке;
- FN (false negative, ложноотрицательные примеры), где тест выдаёт отрицательный результат, а должен быть положительный; это .

Разные комбинации этих частей приводят к разным свойствам теста (классификатора); рис. 2.7 поможет нам проиллюстрировать их более наглядно:

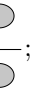
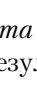
- *чувствительность* (sensitivity) — это доля выявленных истинно положительных больных (true positive rate):

$$\text{Sensitivity} = \frac{TP}{P} = \frac{TP}{TP + FN} = \frac{\text{ }}{\text{  }};$$

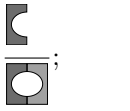
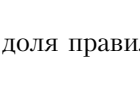
  в других контекстах эту величину ещё называют *полнотой* (recall);
- *специфичность* (specificity) — это доля корректно отвергнутых тестом здоровых людей (true negative rate):

$$\text{Specificity} = \frac{TN}{N} = \frac{TN}{TN + FP} = \frac{\text{  }}{\text{  }};$$

- *точность*, она же *прогностическая ценность положительного результата* (precision, positive predictive value, PPV), — это доля истинных положительных результатов среди всех положительных результатов теста:

$$\text{Precision} = \text{PPV} = \frac{TP}{TP + FP} = \frac{\text{  }}{\text{  }};$$

- *прогностическая ценность отрицательного результата* (negative predictive value, NPV) — это доля истинных отрицательных результатов среди всех отрицательных результатов теста:

$$\text{NPV} = \frac{TN}{TN + FN} = \frac{\text{  }}{\text{  }};$$

- *доля правильных ответов* (accuracy)[1] — это доля правильных ответов во всей выборке:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} = \frac{\text{ }}{\text{}};$$

[1] Здесь может быть ещё проще запутаться, чем обычно: эту величину по-русски тоже часто называют точностью! Впрочем, здесь можно сказать по-другому, а вот другого употребительного русского термина для метрики precision, кажется, нет, так что давайте зарезервируем слово «точность» для последней.

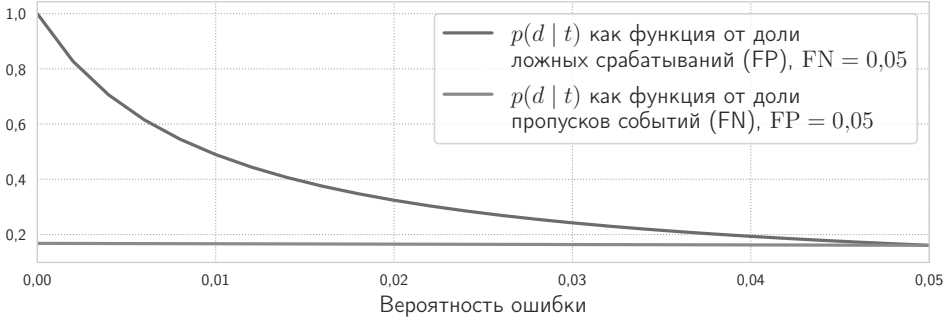


Рис. 2.8. Апостериорная вероятность $p(d \mid t)$ как функция от доли ложных срабатываний при априорной вероятности $p\,(d) = 0{,}01$

- *частота ложноположительных результатов* (false discovery rate, FDR) ясна из названия:

$$\mathrm{FDR} = \frac{FP}{TP + FP} = \frac{\bigcirc}{\bigcirc} = 1 - \mathrm{Precision} = 1 - \mathrm{FPV};$$

- *частота ложноотрицательных результатов* (false omission rate, FOR) тоже хорошо описывает сама себя:

$$\mathrm{FOR} = \frac{FN}{TN + FN} = \frac{\bigcirc}{\bigcirc} = 1 - \mathrm{NPV};$$

- кстати, и *превалентность* (prevalence, base rate, априорную вероятность болезни) тоже можно увидеть на картинке:

$$\mathrm{Prevalence} = \frac{TP + FN}{TP + TN + FP + FN} = \frac{\bigcirc}{\bigcirc}.$$

На вопрос о том, что из этого многообразия нужно улучшать, тоже поможет ответить теорема Байеса. Давайте для примера сравним влияния ложных срабатываний и пропусков событий, то есть попробуем понять, лучше ли будет сконцентрироваться на увеличении чувствительности теста или его специфичности. Интуитивно можно рассудить так: если мы будем уменьшать ложные срабатывания до нуля, числитель в формуле (2.24) не изменится, а из знаменателя постепенно пропадут примерно пять его шестых; а если уменьшим пропуски событий,

числитель увеличится на 5 % (знаменатель тоже немножко увеличится, но давайте этим даже пренебрежём). Ясно, что в этом примере ложные срабатывания должны оказать больше влияния.

Рисунок 2.8 показывает этот эффект уже точно, на графиках: при уменьшении доли ложных срабатываний от 0,05 до 0 апостериорная вероятность резко увеличивается до единицы, а вот уменьшение доли пропусков событий существенного эффекта не оказывает.

2.2.4. Задачи байесовского вывода

Задачи вроде рассмотренной выше составляют суть *вероятностного вывода* (probabilistic inference). Поскольку они обычно основаны на теореме Байеса, вывод часто называют *байесовским* (Bayesian inference). Но это не единственная причина: у байесовских вероятностей ещё и другой философский смысл.

В классической теории вероятностей вероятность часто вводится и интерпретируется как предел отношения числа некоторых интересующих нас результатов экспериментов к общему числу этих экспериментов. Избитый пример с бросанием монетки отлично это иллюстрирует: мы можем подбрасывать монетку миллион раз, и при этом отношение числа выпавших орлов к общему числу бросаний не будет осциллировать от нуля до единицы, а будет сходиться к некоторой величине, которая и представляет собой вероятность выпадения орла θ. Этот подход называют *фриквентистским* (frequentist), то есть основанным на частотах.

В этом взгляде на вероятности нет никакой ошибки — если мы рассматриваем некий повторяемый эксперимент, вероятность будет определяться именно так. Однако в жизни часто приходится говорить о «вероятностях» событий, с повторяемостью у которых плоховато:

- букмекерам нужно уметь оценивать, с какой вероятностью сборная Бразилии выиграет очередной чемпионат мира по футболу, хотя говорить о повторяемом событии здесь никак не получится: чемпионат пройдёт ровно один раз, и его результаты к предыдущим чемпионатам будут иметь лишь весьма косвенное отношение, ведь там играли совсем другие команды;
- историкам или криминалистам хотелось бы говорить о вероятностях единичных событий, произошедших в прошлом; например, рассуждать о том, с какой вероятностью «Одиссею» написала женщина (есть такая теория) или с какой вероятностью О. Джей Симпсон убил свою жену (об этом деле пойдёт речь в разделе 2.3);
- а всё машинное обучение, которым мы с вами занимаемся в этой книге, пытается делать выводы о параметрах модели на основе данных, которые получены один раз и зафиксированы, и никаких повторений здесь тоже не предвидится.

В таких случаях вероятности нужно понимать не в фриквентистском смысле, а как *степени доверия* (degrees of belief). Мы не до конца понимаем устройство

этого мира, не всё можем предсказать детерминированно и не все выводы о прошлом можем сделать; тёмные для нас места мы и формализуем посредством теории вероятностей.

Кстати, с броском монетки ведь тоже именно так обстоит дело: монетка летит по вполне ньютоновским законам, и если бы мы точно знали импульс, который мы придаём монетке при броске (это ещё можно было бы измерить на практике), а также состояние окружающего её воздуха (а это уже вряд ли), теоретически мы бы смогли подсчитать, сколько оборотов в воздухе она сделает и какой стороной кверху упадёт.

Настоящий источник неопределённости в мире вообще ровно один: измерения запутанных квантовых систем; там (по крайней мере, на нашем нынешнем уровне понимания физики) действительно происходит реализация некоторой случайной величины.

В качестве второго источника неопределённости можно ещё обсудить свободу воли, но это вопрос уже тёмный и философский: что такое свобода воли, есть ли она у нас, как реализуется... Вопрос свободы воли до сих пор остаётся не до конца решённой проблемой в нейробиологии, тесно связанной с проблемой сознания, к решению которой пока вообще непонятно как подступиться научным методом.

В нейробиологических экспериментах свободе воли уже практически не осталось места: начиная со знаменитых работ Бенджамина Либета начала 1980-х годов [344], исследователи всё точнее могли предсказать действия человека на основе паттернов нейронной активности заранее, не оставляя никакого времени для сознательного действия [273]. Другие эксперименты в то же время показывали, что сознательный выбор человека во многом реконструируется уже постфактум, на основе данных, полученных уже после совершения действия [43, 332]. Однако в наше время начинают появляться работы, которые пытаются примирить бытовое, философское и нейробиологическое понятия свободной воли [333].

О том, как рассматривал вероятности сам Томас Байес, у нас не очень много свидетельств. Вероятности называются байесовскими потому, что Байес в своём знаменитом эссе решал обратную задачу теории вероятностей и вводил условные вероятности, а при этом язык степеней доверия звучит более естественно: «Для двух следующих друг за другом событий, если вероятность второго b/N и вероятность того и другого вместе P/N, и сначала обнаруживается, что второе событие произошло, и из этого я делаю вывод, что первое событие также произошло, то вероятность того, что я прав, составляет P/b» [49]. Но лично мне нравится думать, что для преподобного Байеса, главным трудом жизни которого было вовсе не это эссе, а книга о божественной благодати, рассматривать вероятности как степени доверия было естественным подходом: Господь Бог наверняка знает заранее, орёл или решка выпадет на подброшенной мною монетке, но я своим слабым человеческим разумом не могу этого постичь и вынужден прибегать к вероятностному анализу.

Теперь вернёмся от базовой теоремы Байеса, применённой к вероятностям конкретных событий, к теореме Байеса, применённой к плотностям распределений точек данных и параметров модели, то есть к формуле (2.21):

$$p(\boldsymbol{\theta}|D) = \frac{1}{p(D)} p(\boldsymbol{\theta})\, p(D|\boldsymbol{\theta}) \propto p(\boldsymbol{\theta})\, p(D|\boldsymbol{\theta})\,.$$

Эта формула позволяет нам сформулировать основные *задачи байесовского вывода*, которые мы будем в этой книге последовательно решать в разных ситуациях и для разных моделей машинного обучения:

(1) максимизируя правдоподобие $p(D|\boldsymbol{\theta})$ по $\boldsymbol{\theta}$, мы находим *гипотезу максимального правдоподобия* (maximum likelihood hypothesis)

$$\boldsymbol{\theta}_{\mathrm{ML}} = \arg\max_{\boldsymbol{\theta}} p(D|\boldsymbol{\theta})\,;$$

это обычно самая простая, первая задача, которая ставится в машинном обучении для новой модели;

(2) умножая правдоподобие $p(D|\boldsymbol{\theta})$ на априорное распределение $p(\boldsymbol{\theta})$ и нормируя результат, мы находим *апостериорное распределение* $p(\boldsymbol{\theta}|D)$; если его максимизировать по $\boldsymbol{\theta}$, мы найдём *максимальную апостериорную гипотезу* (maximum a posteriori)

$$\boldsymbol{\theta}_{\mathrm{MAP}} = \arg\max_{\boldsymbol{\theta}} p(\boldsymbol{\theta}|D)\,;$$

(3) и, наконец, самой сложной задачей байесовского вывода является поиск *предсказательного распределения* (predictive distribution), то есть распределения результатов следующих экспериментов, ответов в новых точках:

$$p(\boldsymbol{x}|D) = \int p(\boldsymbol{x}, \boldsymbol{\theta}|D)\,\mathrm{d}\boldsymbol{\theta} = \int p(\boldsymbol{x}|\boldsymbol{\theta})\, p(\boldsymbol{\theta}|D)\,\mathrm{d}\boldsymbol{\theta}. \tag{2.25}$$

В конечном счёте всё машинное обучение нужно именно для того, чтобы попытаться решить третью задачу: *предсказать новые точки данных*. Когда вы, например, обучаете линейную регрессию (как мы будем делать в главе 4), вас на самом деле крайне редко интересуют собственно значения получающихся весов линейной комбинации. Что действительно важно — это то, хорошо ли они позволяют предсказывать новые значения функции. К сожалению, для сложных моделей машинного обучения решить третью задачу точно, аналитически вряд ли получится и придётся или прибегать к аппроксимациям, или ограничиваться решением первых двух, а затем использовать полученные точечные оценки $\boldsymbol{\theta}$.

Кроме того, обратите внимание, что в формуле (2.25) мы для второго равенства использовали предположение о том, что точки данных $\boldsymbol{x}$ условно независимы между собой при условии известных параметров модели $\boldsymbol{\theta}$; именно поэтому

мы могли записать, что $p(\boldsymbol{x}|\boldsymbol{\theta}, D) = p(\boldsymbol{x}|\boldsymbol{\theta})$. Это предположение всегда делается в машинном обучении, без него никакие разумные методы работать не будут, и мы всегда будем делать его и впредь. В частности, оно позволяет разложить правдоподобие датасета в произведение правдоподобий отдельных точек:

$$p(D|\boldsymbol{\theta}) = p(\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N|\boldsymbol{\theta}) = \prod_{n=1}^{N} p(\boldsymbol{x}_n|\boldsymbol{\theta}); \tag{2.26}$$

такое разложение мы будем использовать в каждой модели машинного обучения.

С точки же зрения реализации на практике большинство задач машинного обучения обычно сводятся к тем или иным задачам *оптимизации*. В списке выше это тоже видно: первые две задачи (поиск $\boldsymbol{\theta}_{\text{ML}}$ и $\boldsymbol{\theta}_{\text{MAP}}$) буквально представляют собой максимизацию функций, а после решения третьей мы тоже во многих случаях захотим максимизировать $p(\boldsymbol{x}|D)$, чтобы выдать точечную оценку результата. Оптимизации в машинном обучении будет посвящён раздел 3.5.

На этом месте пытливый читатель вполне может задаться вопросом: а зачем вообще нужны вероятностные модели? Зачем машинному обучению такие сложности, разве нельзя просто определять функции ошибки и минимизировать их, занимаясь многомерной оптимизацией и ни разу не произнося волшебного слова «вероятность»?

Во-первых, при таком подходе встаёт вопрос: откуда эти функции ошибки возьмутся? Как мы увидим в этой книге, берутся они главным образом из вероятностных предположений, которые мы в своих моделях будем делать, а также из вероятностных рассуждений, аналогичных проведённым выше в общем виде.

Но даже если предположить, что функцию ошибки мы сможем выбрать и сами, вероятностный подход имеет много плюсов и даёт много новых возможностей. Начну с чисто практических возможностей. Если мы, например, в задаче классификации знаем не просто бинарный ответ на вопрос, котик или пёсик изображён на картинке, а ещё и апостериорную вероятность котика, то мы можем, например:

- добавить опцию «я не знаю»: если эта вероятность близка к $\frac{1}{2}$, модель может честно признаться, что мало что понимает на этом входе;
- минимизировать риск, учитывая разные веса ошибок, если, например, принять котика за пёсика гораздо страшнее, чем наоборот: как мы скоро увидим, наличие апостериорных вероятностей этому крайне способствует;
- перебалансировать классы, если, например, котиков в датасете в сто раз больше, чем пёсиков: это частный случай задания априорных вероятностей;
- комбинировать модели, например объединить результаты распознавания котиков по картинке и по издаваемому ими звуку при помощи наивного байесовского подхода;
- и так далее и тому подобное...

Это, конечно, далеко не исчерпывающий список практических преимуществ. Но главное, по моему глубокому убеждению, не в этом, а в том, что теория вероятностей существенно расширяет наше *понимание* происходящего. Понимание смысла помогает, например:

- распознавать *предположения*, которые делают модели, в том числе неявные (какое предположение мы делаем, когда используем квадратичное отклонение в линейной регрессии?);
- тем самым понимать *границы применимости* моделей: когда эти предположения выполняются, а когда — нет и нужно модель модифицировать (когда квадратичное отклонение работает, а когда — нет?);
- затем собственно производить такие модификации; например, в главе 4 мы узнаем ответ на вопрос из первого пункта — линейная регрессия предполагает нормальное распределение ошибки (но как нужно изменить линейную регрессию, если это предположение не выполняется?);
- обобщать и переносить идеи моделей на другие задачи;
- ну и, наконец, содержательно интерпретировать происходящее, что, в свою очередь, позволяет отвечать на более сложные и интересные вопросы; это вы сможете увидеть на практике, например, в разделе 4.5, где проиллюстрированы вероятностные рассуждения в линейной регрессии на примере вспыхнувшей весной 2020 года пандемии коронавируса COVID-19.

Поэтому машинное обучение для нас и в этой книге, и вообще — это раздел прикладной теории вероятностей, и именно с вероятностных позиций мы будем рассматривать почти каждый метод машинного обучения в этой книге. Но пока мы только в начале пути, и нужно освоиться со смыслом и применениями теоремы Байеса.

2.3. Байес в суде и сложности вероятностной интуиции

Адвокат и прокурор, оба крашенные и очень похожие друг на друга (закон требовал, чтобы они были единоутробными братьями, но не всегда можно было подобрать, и тогда гримировались), проговорили с виртуозной скоростью те пять тысяч слов, которые полагались каждому. Они говорили вперемежку, и судья, следя за мгновенными репликами, вправо, влево мотал головой, и равномерно мотались все головы, — и только одна Марфинька, слегка повернувшись, неподвижно, как удивлённое дитя, уставилась на Цинцинната, стоявшего рядом с ярко-зелёной садовой скамьёй. Адвокат, сторонник классической декапитации, выиграл без труда против затейника прокурора, и судья синтезировал дело.

Владимир Набоков. Приглашение на казнь

— "Attorney Utterly Confused", end quote.
(«Адвокат совсем запутался», конец цитаты.)

Shu Takumi. Apollo Justice: Ace Attorney

2.3.1. Вероятностная интуиция — точнее, её отсутствие

Теория вероятностей — непростая наука для человека. Конечно, некоторая интуиция в оценке вероятностей человеку жизненно необходима, и появляется она достаточно рано. Хотя Жан Пиаже[1] считал, что способности к вероятностному выводу и проверке гипотез относятся уже к стадии формальных операций (11–15 лет), а первые их аналоги могут появляться только на стадии конкретных

[1] *Жан Вильям Фриц Пиаже* (Jean William Fritz Piaget, 1896–1980) — швейцарский философ и психолог, фактически создавший исследования в области психологии детей и психологии развития. Сын профессора средневековой литературы, Жан опубликовал свою первую научную статью (заметку о воробьях-альбиносах) в возрасте 11 лет, так что, учитывая, что дожил Пиаже до 84 лет, его карьера длилась очень долго. Пиаже разработал теорию развития интеллекта, в которой выделил такие три основные стадии: сенсомоторный интеллект (0–2 года), подготовку и организацию конкретных операций (2–11 лет) и стадию формальных операций (11–15 лет) [465–468]. В нашей области работы Пиаже использовал Сеймор Пейперт при разработке языка Logo, а для Алана Кея теории Пиаже стали основой концепции детского компьютера Dynabook, из которого позднее пришли многие идеи современных графических интерфейсов. Хотя современные исследователи с Пиаже уже не вполне согласны (одним из первых и главных его критиков был Лев Выготский), влияние Жана Пиаже на развитие психологии переоценить трудно.

операций (7–11 лет), современные исследователи в области психологии развития утверждают, что некоторые способности к вероятностному выводу у ребёнка можно заметить уже в возрасте 6–12 месяцев!

Например, в работе [650] восьмимесячные младенцы следили за тем, как исследователь доставал несколько мячиков для настольного тенниса из закрытой коробки, содержимое которой заранее показывали ребёнку. Когда исследователь доставал маловероятный набор шариков (четыре красных и один белый из коробки, в которой белых шариков было очевидно гораздо больше), младенец смотрел на получившуюся выборку значительно дольше, чем если получался набор с более высоким правдоподобием (четыре белых и один красный)[1].

Более того, оказывается, что к простому вероятностному выводу способны не только люди и даже не только высшие приматы. Исследования в этой области ведутся, например, в лаборатории Ирэн Пеппербергг[2]; в работе [278] исследователи показали, что серый попугай (жако) способен оценивать вероятности в выборке некоторых предметов на основе того, в какой пропорции они были в исходной популяции. Правда, здесь уже о доверительных интервалах и говорить не приходится: попугаев нужно обучать годами, и речь идёт о том, что один конкретный попугай в итоге достиг такого уровня.

Однако даже мы, взрослые, образованные представители вида *Homo Sapiens*, всё равно на удивление плохо себя показываем всякий раз, когда речь заходит об интуитивной оценке и использовании теории вероятностей. Лаплас и де Муавр разрабатывали теорию вероятностей для того, чтобы проанализировать азартные игры; если бы наша интуиция была хоть немного лучше, нам бы не потребовалось ждать до XVIII века, чтобы понять, что казино всегда выигрывает.

Особенно сложными для человека оказываются понятия, связанные с условными вероятностями. Самый простой пример здесь опять же приходит из азартных игр: известный психологический эффект заставляет людей верить, что вероятность того, что на рулетке выпадет чёрное, каким-то мистическим образом

---

[1] Авторы всех этих работ, безусловно, следят за статистической значимостью, проверяют, что $p < 0{,}05$, доверительные интервалы их оценок не пересекаются и так далее, но лично моя интуиция в исследованиях по психологии всё равно всегда напрягается от далекоидущих выводов на основе малых выборок. В работе [650] я размер выборки найти и вовсе не смог, но, например, в другой статье из той же лаборатории на ту же тему [139] авторы утверждают, что восьмимесячные младенцы способны учесть дополнительные параметры в своём вероятностном выводе, на основе выборки из аж целых 16 детей (восемь мальчиков и восемь девочек). Можете считать эту сноску моим старческим брюзжанием, но сначала прочтите раздел 2.6.

[2] *Ирэн Пеппербергг* (Irene Pepperberg, p. 1949) — американский учёный, исследователь интеллекта животных, знаменитая в основном своими исследованиями попугаев. Ирэн смогла обучить серого попугая по имени Алекс не только использовать достаточно большой словарь слов и выражений (150 слов), но и различать синтаксис и семантику, составлять запросы, считать до восьми и в целом владеть языком на уровне примерно двухлетнего ребёнка. Пеппербергг возглавляет The Alex Foundation, через которую спонсирует свои исследования, а также является советником METI (Messaging ExtraTerrestrial Intelligence), организации, которая в наше время продолжает дело SETI и отправляет сообщения к другим звёздным системам для цивилизаций, которые, возможно, живут в этих звёздных системах.

увеличивается после того, как красное выпало пять раз подряд. Понятно, что если нам байесовский вывод что-то и говорит в этой ситуации, то, наоборот, рекомендует повысить оценку вероятности выпадения красного цвета после многочисленных его наблюдений (то есть побуждает усомниться в исходном предположении о независимости исходов на рулетке).

Но такую ошибку читатели этой книги вряд ли сделают. А вот те две ошибки, которые станут основными примерами в этом разделе, часто допускают при обращении с условными вероятностями даже очень умные люди, даже профессионалы, даже в случаях, когда никакой спешки нет, а от их решений зависят судьбы людей.

2.3.2. Ошибка прокурора

Итак, представьте, что мы в суде. Слушается дело об убийстве. На месте убийства найдена кровь, которая явно принадлежит убийце (будем считать, что здесь вероятность ошибки равна нулю). Кровь принадлежит к редкой группе (условно назовём её X), которая присутствует только у 1 % населения. И так получилось, что эта группа крови нашлась в том числе и у подсудимого.

Первым встаёт прокурор; он обращается к суду: «Ваша честь, уважаемые господа присяжные! Шанс, что у подсудимого была бы именно такая группа крови, если бы он был невиновен, — всего 1 %; значит, с вероятностью 99 % он виновен». Подумайте, прежде чем читать дальше: в чём не прав прокурор?

Ошибка прокурора (prosecutor's fallacy) [593] здесь состоит в том, что прокурор оценил не ту вероятность. Он совершенно верно нашёл условную вероятность $p(\text{«кровь группы X»} \mid \text{«невиновен»}) = 1\,\%$. Но затем он почему-то решил приравнять её к совсем другой условной вероятности, а именно $p(\text{«невиновен»} \mid \text{«кровь группы X»})$, которую ему действительно нужно было оценить. Эту ошибку может быть нелегко заметить в бытовом рассуждении, но если написать хотя бы такие условные формулы, она становится очевидной. Разницу между этими условными вероятностями позволяет оценить всё та же теорема Байеса:

$$p(\text{«невиновен»} \mid \text{«кровь группы X»}) = \\ = p(\text{«кровь группы X»} \mid \text{«невиновен»}) \frac{p(\text{«невиновен»})}{p(\text{«кровь группы X»})}.$$

Отсюда видно, что особенно большая разница между оценкой прокурора и истинной оценкой нужной вероятности будет тогда, когда априорная вероятность невиновности высока, а вероятность иметь кровь группы X независимо от виновности мала. Именно такая ситуация складывалась в разделе 2.2, где мы обсуждали пример с медицинским тестом: вероятность ложного срабатывания теста была существенно больше априорной вероятности болезни, и в результате 95 % превратились в 16 %.

Ошибка прокурора может появиться и в настоящем суде [593] — не зря же она называется именно так, а не «ошибка диагноста» или «глупость мертвеца». Во-первых, ошибка прокурора снова возникает в том самом примере с ложным срабатыванием теста, только в суде речь идёт не о медицинских тестах (они обычно достаточно надёжны), а о детекторах лжи.

Хотя полиграф — очень интересное изобретение, на самом деле в судах его практически никогда не используют. Дело в том, что даже современные детекторы лжи очень много ошибаются: у них от 1 до 13 % ложных несрабатываний для людей, которые на самом деле лгут, и от 10 до 23 % (почти четверть!) ложных срабатываний для людей, которые говорят правду [583]. Не буду подробно повторять точно такие же выкладки, но для нас уже не слишком удивительно, что в результате, если учесть теорему Байеса и правильно подсчитать нужную вероятность (см. [290]), окажется, что детектор лжи в суде бесполезен чуть более чем полностью. Если принять для примера, что правду говорят 85 % испытуемых, то рассуждения, подобные примеру с тестом на COVID в разделе 2.2, покажут, что полиграф будет работать хуже монетки!

Есть и несколько других ярких случаев, когда ошибка прокурора приводила к настоящим судебным ошибкам. Самый известный трагический пример — дело Салли Кларк. Трагедия здесь началась ещё до суда: в декабре 1996 года новорождённый сын Салли умер, не прожив и нескольких недель. По всей видимости, он погиб от синдрома внезапной детской смерти[1]. Более того, ровно то же самое произошло в январе 1998 с её вторым сыном! Через месяц Салли арестовали; в ответ на аргументы защиты об СВДС обвинение пригласило эксперта (вполне настоящего профессора педиатрии сэра Роя Медоу), и он показал в суде, что шансы на то, что СВДС случится у двух детей в одной семье, составляют примерно один из 73 миллионов. Он получил это число, возведя в квадрат 1/8500 — именно таков был тогда в Британии риск СВДС.

Видите ошибку профессора? Их тут сразу две, и обе очень удивительные. Во-первых, профессор педиатрии перемножил вероятности, то есть решил, что случаи СВДС *у детей одной и той же матери* независимы. Это, конечно же, не так; мы не знаем наверняка, чем вызывается СВДС, но практически все известные факторы риска для этого синдрома (курение матери, неправильное укладывание, генетические факторы и так далее) будут с большой вероятностью присутствовать у всех детей в одной семье.

Но подождите: даже если допустить, что умножать нужно было не на 1/8500, а, скажем, на 1/500, число в знаменателе всё равно становится большим до полной невероятности, разве нет? А здесь как раз вступает ошибка прокурора: да, два случая СВДС подряд — это очень маловероятное событие, но двойное убийство

[1] СВДС (англ. SIDS, sudden infant death syndrome) — синдром, при котором совершенно здоровый младенец просто перестаёт дышать и умирает, без видимых причин, обычно во сне. Причины СВДС до сих пор остаются неясными; каждый год он уносит жизни около 20 000 младенцев и в развитых странах является одной из главных причин смерти младенцев возрастом от месяца до года.

своих собственных новорождённых детей — это тоже, слава богу, не слишком распространённый случай! А суду оценивать нужно, как и выше, сравнительную вероятность этих двух объяснений, и вероятности каждого по отдельности здесь недостаточно.

К сожалению, никто в суде этого не заметил, и история Салли Кларк закончилась совсем грустно: её осудили, и она провела за решёткой три очень, очень неприятных года — Салли была юристом, дочерью полицейского и (по крайней мере, в глазах окружающих) детоубийцей, что, мягко скажем, не расположило к ней других заключённых. Когда апелляцию всё же удовлетворили и Салли вышла на свободу, у неё в результате всех произошедших трагических событий развилось много психических проблем, и она вскоре умерла от алкогольного отравления.

После случая Салли Кларк были предприняты и попытки более корректно подсчитать статистику и сравнить вероятности нескольких случаев СВДС в одной семье и соответствующих детоубийств [261]. Такое вычисление кажется достаточно прямолинейным, и в случае СВДС вероятности подсчитать действительно несложно... но как быть с убийствами? Ведь мир, к счастью, устроен так, что если об одном убийстве становится достоверно известно, шансы совершить второе у убийцы резко уменьшаются.

Автор работы [261] попытался обойти эту проблему косвенными средствами и на британских данных оценил, что:

- одиночный случай СВДС примерно в 17 раз вероятнее одиночного случая убийства;
- двойной случай СВДС примерно в девять раз вероятнее двойного убийства; обратите внимание, что это значит, что зависимость между повторными случаями убийств гораздо сильнее, чем между случаями СВДС, иначе двойной СВДС был бы в $17^2 = 289$ раз вероятнее;
- соответственно, получается, что тройной СВДС в одной семье всего лишь вдвое вероятнее тройного убийства; учитывая неточность статистики и её косвенный характер, это уже и вовсе не даёт существенных оснований предпочесть одну гипотезу другой.

И всё это, конечно, в предположении, что случаи СВДС и убийств не зависят от других известных признаков (социального положения семьи, вредных привычек родителей и так далее), что на практике тоже вряд ли верно; учёт таких зависимостей ещё сильнее осложнил бы анализ.

Медицинские исследования нелегко проводить в том числе и потому, что дополнительные факторы можно придумывать бесконечно, а реальных данных для редких медицинских событий очень мало, на то они и редкие. Так, например, в собиравшейся годами статистике из [261] тройной случай СВДС встречается всего два раза, а тройное убийство — один раз; это, конечно, идеально соответствует теоретической оценке в 2:1, но не вселяет уверенности в том, что мы сможем её экспериментально подтвердить или опровергнуть.

2.3.3. Ошибка адвоката

Вернёмся в наш вымышленный зал суда. После речи прокурора выступает адвокат. Сначала он рассказывает всё то, что мы с вами только что выяснили об ошибке прокурора, а затем производит свой собственный расчёт. Адвокат говорит: «В городе живёт миллион человек, значит, примерно десять тысяч из них имеют группу крови X. Значит, всё, что говорит нам эта кровь, — это то, что подсудимый совершил убийство с вероятностью 0,01 %. Конечно, один к десяти тысячам — это гораздо больше, чем один к миллиону, но всё-таки это никак не может считаться доказательством; прошу суд исключить кровь подсудимого из числа улик». В чём не прав адвокат?

Ошибка адвоката (defense attorney's fallacy) [593] имеет иную природу, чем ошибка прокурора. Группу крови X действительно имеют десять тысяч человек, и адвокат в целом корректно подсчитал вероятность вины подсудимого по теореме Байеса. Но этот его подсчёт предполагал, что априорное распределение вероятности вины всех жителей города, включая стариков и грудных младенцев, было равномерным... что, конечно же, не так. Корректное априорное распределение здесь должно учитывать другие обстоятельства дела — например, если всего десять человек могли быть на месте преступления в нужное время и из них только подсудимый имеет группу крови X, арифметика теоремы Байеса приведёт к совсем другому результату. Собственно, в этом случае сомнений в вине подсудимого не остаётся вовсе — но только если предположить, что анализ крови безошибочен, найденная кровь точно принадлежит именно убийце и список из десяти человек действительно абсолютно исчерпывающий.

Такие ошибки тоже нередки в реальной практике. Наверное, самым широко известным таким примером стало дело О. Джей Симпсона[1], который в 1995 году подозревался в убийстве своей бывшей жены Николь Браун и её друга Рональда Голдмана. О его деле были написаны целые книги, но нас сейчас интересуют только два эпизода. Во-первых, ошибка адвоката появилась в точности в обсуждавшейся выше форме, речь даже шла именно об анализе крови: найденная на месте преступления кровь обладала теми же характеристиками, что и кровь Симпсона, причём такие характеристики встречались примерно в 1 из 400 случаев. Адвокаты заявили, что жителями Лос-Анджелеса, имеющими те же характеристики состава крови, «можно заполнить целый стадион» [504]. Суть этой ошибки мы с вами только что обсудили.

[1] *О. Джей Симпсон* (Oriental Jay Simpson, p. 1947) — великий раннингбек NFL, в 1985 году включённый в Зал Славы. О. Джей Симпсон единственным в истории пробежал с мячом 2000 ярдов за сезон из 14 игр; другие раннингбеки достигли этого рубежа только после того, как сезон в NFL удлинили до 16 матчей. После окончания футбольной карьеры Симпсон снимался в кино (играл детектива Нортберга в серии *Naked Gun*), но его жизнь, конечно, определила история с двойным убийством. В 1995 году он не без помощи обсуждающихся здесь статистических трюков был оправдан в убийстве, но найден «виновным в смерти» в гражданском процессе и приговорён к штрафу 33,5 миллиона долларов, с 2008 года сидел в тюрьме по другому делу (о вооружённом ограблении), вышел на свободу в 2017-м.

Рис. 2.9. События, которые нужно было рассмотреть в деле О. Джей Симпсона

Вторая ошибка адвокатов О. Джей Симпсона оказалась ещё интереснее. Дело в том, что сторона обвинения представила доказательства того, что между Симпсоном и Николь Браун была история семейного насилия — проще говоря, что О. Джей бил свою жену ещё до всех этих печальных событий. В ответ на это адвокаты отметили, что хотя семейное насилие — дело, безусловно, плохое и они его никоим образом не одобряют, это совсем не такой уж редкий случай, и статистика утверждает, что только в одном из 2500 случаев семейное насилие доходит до убийства. В результате суд согласился с этим доводом и указал присяжным не принимать во внимание свидетельства о семейном насилии в прошлом.

Статистика была приведена верная, но ход рассуждений всё равно был неправильным. Прежде чем читать следующий абзац, попробуйте сами сформулировать: в чём именно здесь ошибка?

Дело в том, что адвокаты в данном случае корректно оценили вероятность, но совсем не ту вероятность, которую должны были. Идея того, какие события нужно было рассматривать в этом случае и как они соотносятся между собой, абстрактно представлена на рис. 2.9. Адвокаты оценили вероятность

$$p(\text{муж убил жену} \mid \text{муж бил жену в прошлом}),$$

то есть соотношение между той частью события «Убийства женщин их мужьями», которая попадает в «Случаи семейного насилия» (на рис. 2.9 это незаштрихованная часть), и всем событием «Случаи семейного насилия».

Но ведь на самом деле суд интересует не эта вероятность, которая действительно составляет около $\frac{1}{2500}$, а вероятность

$$p(\text{муж убил жену} \mid [\text{муж бил жену в прошлом}] \;\; \text{И} \;\; [\text{кто-то убил жену}]),$$

ведь убийство уже произошло и оно как раз и расследуется! Согласитесь, второе условие здесь существенно меняет дело: разумеется, в подавляющем большинстве случаев женщин, страдающих от семейного насилия, никто не убивал. На рис. 2.9 это значит, что на самом деле нам нужно оценить соотношение между незаштрихованной частью события «Убийства женщин их мужьями» (она лежит внутри «Случаев семейного насилия») и частью события «Убийства женщин», лежащей внутри «Случаев семейного насилия» (левее пунктирной линии). Все

остальные случаи семейного насилия (которых, к счастью, подавляющее большинство) не имеют отношения к делу.

И действительно, если использовать ту же самую статистику для подсчёта нужной вероятности, то вместо $\frac{1}{2500}$ мы получим... около 90 %! Грубо говоря, из 100 000 замужних женщин, подвергавшихся семейному насилию, 99 955 не убивал никто, а 45 были убиты, и из этих 45 в 40 случаях убийцей оказался муж [219]. Хотя это по-прежнему не является доказательством вины, тем более доказательством, достаточным для обвинения в убийстве, думаю, присяжным было бы гораздо сложнее «не принимать во внимание» этот вероятностный вывод.

2.3.4. Парадокс Монти Холла и когнитивные искажения

Однако такие достаточно сложные для понимания ошибки — это далеко не единственный пример того, как сложно человеческому мышлению справляться с теорией вероятностей. Сейчас мы увидим, что проблемы с вероятностной интуицией заходят гораздо дальше. Начнём с классического примера уже не из зала суда, а со съёмок телешоу, очень похожего на «Поле чудес». На барабане выпадает соответствующий сектор, и ведущий, очень похожий на Леонида Якубовича, предлагает вам сделать выбор из шкатулок, только их теперь не две, а три. На вид шкатулки абсолютно одинаковы, и в одной из трёх лежит приз, так что сначала у вас нет никакой информации, и вы просто выбираете шкатулку случайным образом.

Но как только вы озвучиваете свой выбор, ведущий открывает одну из двух оставшихся шкатулок, показывает, что в ней ничего нет (ведущий знает, где приз, и всегда может выбрать пустую), и даёт вам возможность поменять выбор. Нужно ли его менять?

Пока вы думаете, расскажу, что эта ситуация известна как *парадокс Монти Холла* (Monty Hall problem); название своё она получила в честь канадского телеведущего, который вёл популярное шоу *Let's Make a Deal*. В течение шоу ведущий предлагал участникам разные необычные сделки. Шоу *Let's Make a Deal* появилось на NBC в конце 1963 года, и Монти Холл вёл его почти 30 лет; в 1990-х оно ушло с телеэкранов, но вернулось в 2009 году и продолжает выходить по сей день (уже без Монти Холла, который, увы, умер в 2017 году).

Парадокс Монти Холла — это ещё один пример того, что в теории вероятностей важно подходить к задаче формально и не слишком доверять интуиции. Действительно, на первый взгляд многим кажется, что менять выбор нет смысла: ведущий в любом случае мог выбрать пустую шкатулку, а значит, не добавил вам никакой информации тем, что её показал. Но на самом деле менять выбор очень даже полезно! Верная интуиция здесь в том, что, когда ведущий открывает одну из пустых шкатулок и предлагает поменять выбор, он фактически предлагает вам обменять *одну* шкатулку, которую вы выбрали изначально, на *две* оставшихся, ведь теперь из этих двух вы точно сделаете правильный выбор.

Эта интуиция становится очевидной, если увеличить число шкатулок: представьте, что вы выбрали одну шкатулку из 100, после чего ведущий открыл 98 пустых шкатулок и дал вам возможность поменять выбор на оставшуюся, 99-ю. Да и формально здесь всё несложно: с вероятностью $\frac{1}{3}$ вы изначально выбрали верную шкатулку, а с вероятностью $\frac{2}{3}$ ошиблись, так что, если предлагают заведомо правильный выбор из двух оставшихся, его и надо делать (можете расписать все условные вероятности совсем формально в качестве упражнения). Несложно, но поначалу весьма контринтуитивно.

В виде математической задачи парадокс Монти Холла появился в 1975 году, когда его в письме редакции журнала *American Statistician* изложил Стив Селвин [338], но настоящую популярность обрёл после того, как появился в колонке Мэрилин вос Савант[1] в журнале *Parade* в 1990 году [630]. Ответ вос Савант, который по сути совпадает с изложением выше, неожиданно вызвал очень бурную реакцию читателей: в 1990–1991 годах *Parade* получил буквально тысячи писем, авторы которых утверждали, что на самом деле вероятности остаются равными и менять выбор не нужно. Многие из этих читателей были профессиональными учёными, некоторые даже математиками...

О том, что нам, людям, сложно рассуждать о вероятностях, свидетельствуют и знаменитые эксперименты Даниэля Канемана[2] и Амоса Тверски, на которых была основана разработанная ими поведенческая экономика и теория когнитивных искажений (cognitive biases) [293, 295][3].

---

[1] *Мэрилин вос Савант* (Marilyn vos Savant, p. 1946) — американская писательница и колумнист. С 1985 по 1989 год вос Савант была занесена в Книгу рекордов Гиннесса как обладательница самого высокого IQ в мире: 186 по данным «Мега-теста» Рональда Хефлина или 228 по шкале Стэнфорд–Бине; впрочем, затем эту запись удалили из книги Гиннесса, решив, что тесты не позволяют однозначно определить лидера. Вос Савант ведёт колонку *Ask Marilyn* в журнале *Parade*, где отвечает на вопросы читателей. Кстати, хотя слово *savant* означает «мудрец», это не псевдоним, а настоящая фамилия матери Мэрилин: вос Савант считает, что девочки должны наследовать фамилию матери.

[2] *Даниэль Канеман* (Daniel Kahneman, p. 1934) — израильский, позже американский психолог и экономист, известный своими работами по психологии принятия решений и один из создателей поведенческой экономики (behavioural economics) [295], за которую и получил Нобелевскую премию (для педантов — премию по экономике памяти Альфреда Нобеля) 2002 года; сейчас Канемана регулярно включают в списки самых влиятельных экономистов мира. В его детстве семья Канемана жила в Париже и лишь чудом спаслась от преследований. Канеман рассказывал, что решил стать психологом, когда встретившийся ему на улице солдат в форме СС подозвал семилетнего Даниэля к себе и, не обращая внимания на жёлтую звезду, открыл бумажник, показал фотографию своего сына и дал Даниэлю денег; Канеман «отправился домой в полной уверенности, что мама была права: люди бесконечно сложны и интересны» [296]. Поведенческая экономика привела Канемана к гедонистической психологии, где он исследовал, как переживаемые нами ощущения становятся приятными или неприятными и как люди оценивают и предсказывают удовольствие (спойлер: предсказывают плохо) [294]. Широко известна книга Канемана «Думай медленно... решай быстро» *(Thinking, Fast and Slow)* [293, 677], которая представляет собой популярное изложение открытых им эффектов. Правда, кризис воспроизводимости не обошёл эксперименты Канемана и Тверски — см. раздел 2.6.

[3] Эта книга — яркий пример ошибки планирования (planning fallacy) [83, 298]: первоначально я обещал сдать книгу к концу 2022 года, но что-то случилось, и я пишу эти строки в начале октября 2023 года (а это замечание в скобках — в декабре 2024-го). Разумеется, я знал о существовании ошибки планирования, когда строил первоначальные планы, но...

Наиболее экономически релевантные их эксперименты, которые легли в основу *теории перспектив* (prospect theory) [297], с точки зрения теории вероятностей показывают скорее не недостатки человеческого мышления, а просто свойственное людям неприятие риска (risk aversion). Например, если предложить людям на выбор два варианта:

- получить 2500 шекелей[1] с вероятностью 0,33, 2400 шекелей с вероятностью 0,66 и не получить ничего с вероятностью 0,01;
- просто взять 2400 шекелей с вероятностью 1,

то люди скорее предпочтут второй вариант, с гарантированным выигрышем, хотя очевидно, что ожидаемый доход в первом случае больше.

Сам по себе этот факт можно было бы уложить в стандартную экономическую теорию, введя функцию полезности u, в которой ценность денег растёт нелинейно[2], и уход от риска не получить ничего стоил бы некоторой потери дохода. Но нет, предположение о наличии такой функции тоже приводит к парадоксам, которые несовместимы с наличием в наших головах машинки, считающей вероятности и ожидаемую доходность: если предложить

- 2500 шекелей с вероятностью 0,33 (и ноль с вероятностью 0,67) против
- 2400 с вероятностью 0,34,

те же самые люди выберут первый вариант, что не сходится с первым экспериментом — не получится присвоить значения функции полезности u так, чтобы одновременно выполнялось

$$0{,}33 \cdot u(2500) + 0{,}66 \cdot u(2400) < u(2400) \quad \text{и} \quad 0{,}33 \cdot u(2500) > 0{,}34 \cdot u(2400).$$

Однако ещё более показательны для нас эксперименты, в которых человеческая интуиция не сходится не просто с какими-то нетривиальными вычислениями функций полезности, а даже с самыми базовыми, чисто логическими вероятностными аксиомами. Один пример этого в работах Канемана и Тверски показывает, что даже очевидно вложенные друг в друга события всё равно могут получить противоречивые оценки вероятностей [299]. В одном эксперименте авторы показали участникам такое описание:

```
Линде 31 год, она не замужем, хорошо умеет говорить и очень умна.
Она специализировалась на философии. Будучи студенткой, Линда была
```

[1] Эксперименты проводились в Израиле; в то время средний месячный доход израильской семьи составлял около 3000 шекелей, так что это существенные деньги. Вообще, в такого рода экспериментах важно, чтобы ставки были достаточно высокими, иначе люди не будут серьёзно относиться к эксперименту и результаты будут не слишком достоверны.

[2] Она, кстати, и в действительности растёт скорее логарифмически — мы это уже обсуждали в разделе 2.1.5 в контексте санкт-петербургского парадокса. Пытаться приводить здесь конкретные примеры было бы неблагодарным занятием, но вы наверняка и в своей жизни можете проследить, что дополнительная полезность денег падает по мере того, как их становится больше.

```
глубоко озабочена вопросами дискриминации и социальной справедливости,
а также участвовала в демонстрациях против ядерного оружия.
```

Затем участников попросили проранжировать несколько высказываний согласно тому, какие из них представляются более вероятными:

```
A. Линда работает учителем в начальной школе.
B. Линда работает в книжном магазине и занимается йогой.
C. Линда активно участвует в феминистском движении.
D. Линда — психиатр, социальный работник.
E. Линда является членом Лиги женщин-избирательниц (League of Women Voters).
F. Линда работает операционистом в банке.
G. Линда работает страховым агентом.
H. Линда — операционист в банке, и она активно участвует в феминистском движении.
```

В результате вариант H оказался выше, чем вариант F, хотя здесь не нужно никакой теории вероятностей, чтобы понять, что H является строгим подмножеством F!

А в 1982 году Канеман и Тверски посетили международный конгресс по прогнозированию (sic!) и опросили 115 участников, большинство из которых были профессиональными аналитиками для различных компаний и университетов. Этим участникам (тоже, конечно, не совсем явно, как и в тесте выше варианты F и H были не единственными) предложили оценить вероятности двух вариантов развития событий:

(A) полный разрыв дипломатических отношений между СССР и США в течение 1983 года;

(B) вторжение СССР в Польшу с дальнейшим полным разрывом дипломатических отношений между СССР и США в течение 1983 года.

У профессиональных прогнозистов результаты оказались не лучше. Оба варианта, разумеется, получили низкие оценки вероятностей, но вариант B уверенно выиграл: среднее геометрическое предсказаний составило 0,47 % против 0,14 % у варианта A! Канеман и Тверски предлагают разные варианты психологического объяснения этого эффекта, но выглядит это всё в любом случае не слишком хорошо для перспектив наших интуитивных рассуждений о теории вероятностей.

Что же делать? Готового ответа у меня нет. Могу только привести слова известного физика Дэвида Мермина, сказанные по совершенно другому поводу: «Если бы мне нужно было подытожить в одном предложении то, что мне говорит копенгагенская интерпретация квантовой механики, это звучало бы так: “Заткнись и вычисляй!”» (Shut up and calculate!) [392]. Давайте этим и займёмся.

2.4. Монетка и сопряжённые априорные распределения

Розенкранц. Восемьдесят девять.
Гильденстерн. Должно же это означать что-нибудь ещё, кроме перераспределения капитала. (Размышляет.) Список возможных объяснений. Первое: я сам хочу этого. На дне моего подсознания я играю в орлянку против самого себя, используя монеты без решки во искупление своего невспоминаемого прошлого. (Бросает монету.)
Розенкранц. Орёл.
Гильденстерн. Второе: время остановилось намертво, и поэтому выпавший в тот миг орёл повторяется в девяностый раз... (Бросает монету, разглядывает, передаёт её Розенкранцу.) Но в целом сомнительно. Третье: божественное вмешательство; иными словами, благоволение свыше, ниспосланное ему, — см. притчу о детях Израилевых — или же кара свыше, ниспосланная мне, — см. притчу о жене Лота. Четвёртое: эффектное подтверждение принципа, согласно которому каждая отдельная монета, подброшенная в отдельности (бросает монету), с той же вероятностью упадёт как орлом, так и решкой, и поэтому нет оснований удивляться в каждую отдельную единицу времени, когда это происходит. (Это происходит — он кидает монету Розенкранцу.)
Розенкранц. Никогда не видел ничего подобного.

Том Стоппард. Розенкранц и Гильденстерн мертвы (пер. Иосифа Бродского)

2.4.1. Монетка с точки зрения байесовского вывода

В предыдущих разделах мы обсудили, что такое теорема Байеса, почему она лежит в основе всего машинного обучения и как байесовский вывод можно применить в бытовых рассуждениях. Теперь (наконец!) настало время привести конкретный математический пример. Мы будем рассматривать самую простую задачу вывода, которую только можно себе представить: обучение параметров обычной монетки, или, говоря математическим языком, *испытаний Бернулли*[1].

[1] *Якоб Бернулли* (Jakob Bernoulli, 1655–1705) — швейцарский математик, один из основоположников теории вероятностей и математического анализа. Бернулли происходил из семьи богатого

Давайте рассмотрим нечестную (точнее, не известной заранее честности) монетку, которая подброшена N раз, и результаты этих бросков известны. Это значит, что данные D в этой задаче обучения представляют собой последовательность исходов: орёл, решка, решка, орёл, орёл... А обучение здесь состоит в том, чтобы определить «нечестность» монетки, то есть её единственный параметр θ (пусть это будет вероятность выпадения орла), и предсказать, с какой вероятностью она выпадет орлом в следующий раз.

Давайте по порядку. Начнём с правдоподобия — вероятность получить данную последовательность D из n орлов и m решек при условии, что орёл выпадает с вероятностью θ, равна

$$p\left(D|\theta\right) = \theta^n \left(1-\theta\right)^m .$$

Отсюда легко получить гипотезу максимального правдоподобия: чтобы найти $\theta_{\text{ML}} = \arg\max_\theta p\left(D|\theta\right)$, можно просто взять производную и приравнять её к нулю:

$$\frac{\partial p\left(D|\theta\right)}{\partial \theta} = n\theta^{n-1}\left(1-\theta\right)^m - m\theta^n\left(1-\theta\right)^{m-1},$$

$$\theta_{\text{ML}}^{n-1}\left(1-\theta_{\text{ML}}\right)^{m-1}\left(n\left(1-\theta_{\text{ML}}\right) - m\theta_{\text{ML}}\right) = 0. \tag{2.27}$$

Пока всё, казалось бы, очень просто: третья скобка даёт нам линейное уравнение, из которого получается интуитивно понятный, ожидаемый результат

$$\theta_{\text{ML}} = \frac{n}{n+m}. \tag{2.28}$$

Но уже на этом этапе начинают возникать некоторые трудности. Во-первых, корнем уравнения (2.27) является не только всеми ожидаемое значение θ_{ML}, но и почему-то ещё $\theta = 0$ и $\theta = 1$ (если в данных есть по крайней мере два орла и две решки). Это, впрочем, не страшно: понятно, что в этих случаях мы получаем нулевое правдоподобие данных, а значит, это минимумы правдоподобия, а не максимумы.

А вот второй вопрос уже оказывается более интересным: какова будет гипотеза максимального правдоподобия, если на монетке выпали, скажем, два орла?

фармацевта и получил прекрасное образование: знал пять языков, ездил во Францию для изучения идей Декарта, в Нидерланды, чтобы познакомиться с Гюйгенсом, и в Англию — чтобы встретиться с Гуком и Бойлем. Бернулли очень заинтересовался основополагающими трудами Лейбница, познакомил с ними брата Иоганна (также выдающегося математика) и вместе с братом получил массу важных результатов. Вообще, семья Бернулли — поразительный пример, их генеалогическое древо, сплошь состоящее из блестящих математиков, в эту сноску точно не поместится. Якоб же был ещё и одним из основоположников теории вероятностей, ввёл многие современные понятия этой науки и сформулировал закон больших чисел. Перед смертью от туберкулёза Якоб Бернулли завещал выгравировать на своём надгробии логарифмическую спираль с девизом *Eadem mutata resurgo* («Изменённая, я воскресаю») как замечательный пример самоподобной кривой; выгравировали в итоге, разумеется, архимедову спираль, а не логарифмическую.

Казалось бы, мы должны увидеть $\theta_{\text{ML}} = \frac{2}{2} = 1$... но ведь формально так не получается! Правдоподобие в данном случае равно $p\,(D|\theta) = \theta^2$, и никакого максимума у этой параболы нет, она уходит на бесконечность. Wait, what?

Тоже мне проблема, скажете вы, это же монетка, а значит, параметр θ представляет собой вероятность одного из исходов и должен меняться от нуля до единицы. Но, собственно, кому должен?

Параметры в моделях машинного обучения далеко не всегда представляют собой вероятности; например, вес в линейной регрессии, о которой речь пойдёт в главе 4, вполне может быть и больше единицы, и вовсе отрицательным. Это значит, что нет никакой единой скрижали, в которой эта область определения параметра θ была бы записана для всех моделей сразу — нам нужно как-то формально ввести это ограничение в данном конкретном случае испытаний Бернулли. Как это сделать?

Оказывается, самый простой и одновременно концептуально прекрасно подходящий способ ввести это ограничение — сделать следующий шаг в нашем байесовском анализе, добавить к правдоподобию априорное распределение $p(\theta)$.

Как выбрать априорное распределение — это вопрос важный и нетривиальный, и о нём мы ещё не раз поговорим в этой книге и даже в этом же разделе. Сейчас давайте начнём с самого простого случая — равномерного распределения на отрезке [0,1]. Его плотность записать легко:

$$p\,(\theta) = \begin{cases} 1, & \text{если } \theta \in [0,1], \\ 0 & \text{в противном случае.} \end{cases}$$

Это распределение ничего не сообщает о параметре θ, кроме того, что это действительно вероятность, то есть число из отрезка [0,1]. Обратите внимание, кстати, что в машинном обучении никак не получится обойтись дискретной интуицией о вероятностях — даже для монетки сразу же появляются непрерывные распределения. Это вполне естественно, ведь, даже когда исходы (данные) дискретны, *параметры*, из которых эти исходы получаются, дискретными бывают очень редко.

В дальнейшем выводе пока что ровным счётом ничего не изменилось. Теперь мы хотим найти не θ_{ML}, а

$$\theta_{\text{MAP}} = \arg\max_{\theta} p\,(\theta|D) = \arg\max_{\theta} p\,(\theta)\,p\,(D|\theta)\,.$$

Но с равномерным априорным распределением это та же самая функция, оно добавляет только то самое ограничение $\theta \in [0,1]$, а внутри этого интервала умножает правдоподобие на единицу. Поэтому мы опять приходим к тому же уравнению (2.27) и получаем тот же ответ:

$$\theta_{\text{MAP}} = \frac{n}{n+m}.$$

А чтобы найти само апостериорное распределение, нужно будет ещё отнормировать правдоподобие в интервале $[0, 1]$. И это приводит нас к следующему важному понятию.

2.4.2. Бета-распределения как апостериорные для монетки

Чтобы найти нормировочную константу для правдоподобия бросаний монетки, придётся вспомнить важный интеграл из математического анализа, который в радикалах не берётся, а даёт нам *бета-функцию*

$$B(\alpha, \beta) = \int_0^1 x^{\alpha-1}(1-x)^{\beta-1}\mathrm{d}x. \tag{2.29}$$

Эту функцию называют ещё интегралом Эйлера I рода, потому что первым свойства этой функции изучал именно Эйлер[1]. Кстати, интеграл Эйлера II рода — это *гамма-функция*

$$\Gamma(\alpha) = \int_0^\infty x^{\alpha-1}e^{-x}\mathrm{d}x,$$

которая в натуральных точках превращается в факториал, $\Gamma(n+1) = n!$, и вообще во всех точках удовлетворяет свойству $\Gamma(x+1) = x\Gamma(x)$. Гамма- и бета-функции — одни из важнейших неэлементарных функций в математическом анализе, не в последнюю очередь из-за многочисленных приложений в теории вероятностей (но, конечно, не только — они появляются в решениях важных дифференциальных уравнений, преобразованиях Фурье и Лапласа для обработки сигналов, фейнмановских интегралах по траекториям и многих других областях).

Бета-функция выражается через гамма-функцию как

$$B(\alpha, \beta) = \frac{\Gamma(\alpha)\Gamma(\beta)}{\Gamma(\alpha+\beta)}, \tag{2.30}$$

[1] *Леонард Эйлер* (Leonhard Euler, 1707–1783) — швейцарский, а также прусский и российский математик, один из величайших в истории. Эйлер написал более 850 работ, в том числе два десятка монографий; статьи Эйлера продолжали выходить в течение нескольких десятилетий после его смерти, причём около половины своего наследия Эйлер создал уже будучи слепым и надиктовывая свои труды. С 1727 по 1741, а потом с 1766 по 1783 год Эйлер жил в Санкт-Петербурге, на Васильевском острове; он пользовался милостью Екатерины II и принимал участие в управлении Академией наук, хотя формальных постов не занимал (вероятно, из-за слепоты). Эйлер стал основоположником теории чисел, вариационного исчисления, развил множество новых методов в анализе и теории рядов, создал первый в мире учебник по аналитической и дифференциальной геометрии, заложил основы теории графов (да, те самые мосты), создал современную механику, основанную на решении дифференциальных уравнений, разработал многие численные методы, то есть был прекрасным алгоритмистом, и многое, многое другое. Классическая шутка утверждает, что в математике принято называть открытие в честь того, кто сделал его вторым, ведь иначе все объекты в математике были бы названы в честь Эйлера. Но даже с такими традициями статья Википедии «Список объектов, названных в честь Леонарда Эйлера» состоит из девяти разделов, в каждом из которых по десятку понятий или результатов.

а значит, для натуральных α и β это всего лишь

$$B(n,m) = \frac{\Gamma(n)\Gamma(m)}{\Gamma(n+m)} = \frac{(n-1)!(m-1)!}{(n+m-1)!}. \tag{2.31}$$

Впрочем, как вы увидите ниже, в машинном обучении и дробные параметры бета-функции могут пригодиться.

В любом случае для нас бета-функция важна как нормировочная константа для *бета-распределения*, которое выражает как раз правдоподобие бросков монетки:

$$\text{Beta}\,(\theta|\alpha,\beta) = \begin{cases} \frac{1}{B(\alpha,\beta)}\theta^{\alpha-1}\,(1-\theta)^{\beta-1}\,, & \text{если } \theta \in [0,1], \\ 0 & \text{в противном случае.} \end{cases} \tag{2.32}$$

В том, что мы рассказали выше, определением бета-функции является формула (2.29) (то есть бета-функция по определению является интересующей нас нормировочной константой), а результат (2.30), вообще говоря, ещё нужно из него вывести. Впрочем, сделать это совсем не сложно; давайте начнём с числителя дроби в формуле (2.29),

$$\Gamma(\alpha)\Gamma(\beta) = \int_0^\infty x_1^{\alpha-1}e^{-x_1}\mathrm{d}x_1 \int_0^\infty x_2^{\beta-1}e^{-x_2}\mathrm{d}x_2 = \\ = \int_0^\infty \int_0^\infty x_1^{\alpha-1}x_2^{\beta-1}e^{-x_1-x_2}\mathrm{d}x_1\mathrm{d}x_2$$

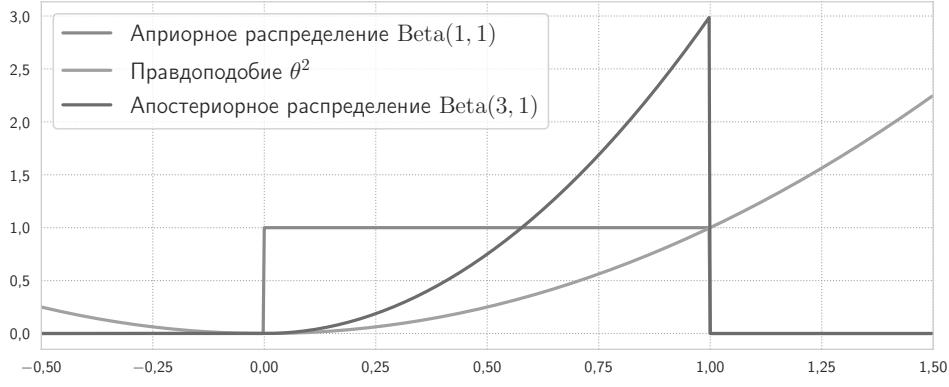
и сделаем замену переменных $x_1 = y_1y_2$, $x_2 = y_1(1-y_2)$, то есть $y_1 = x_1 + x_2$, $y_2 = \frac{x_1}{x_1+x_2}$.

В новых переменных пределы интегрирования для y_1 будут от 0 до ∞, а для y_2 — от 0 до 1, и новые переменные можно будет разделить:

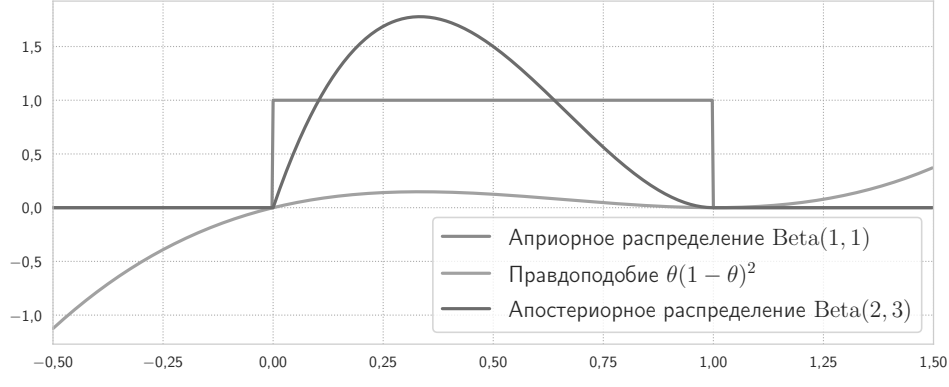
$$\Gamma(\alpha)\Gamma(\beta) = \int_{y_1=0}^\infty \int_{y_2=0}^1 e^{-y_1}\,(y_1y_2)^{\alpha-1}\,(y_1(1-y_2))^{\beta-1}\,y_1\mathrm{d}y_2\mathrm{d}y_1 = \\ = \int_{y_1=0}^\infty e^{-y_1}y_1^{\alpha+\beta-1}\mathrm{d}y_1 \int_{y_2=0}^1 y_2^{\alpha-1}\,(1-y_2)^{\beta-1}\,\mathrm{d}y_2 = \Gamma(\alpha+\beta)B(\alpha,\beta).$$

В случае монетки апостериорное распределение при равномерном априорном распределении и последовательности данных с n орлами и m решками выглядит именно как бета-распределение с параметрами $n+1$ и $m+1$:

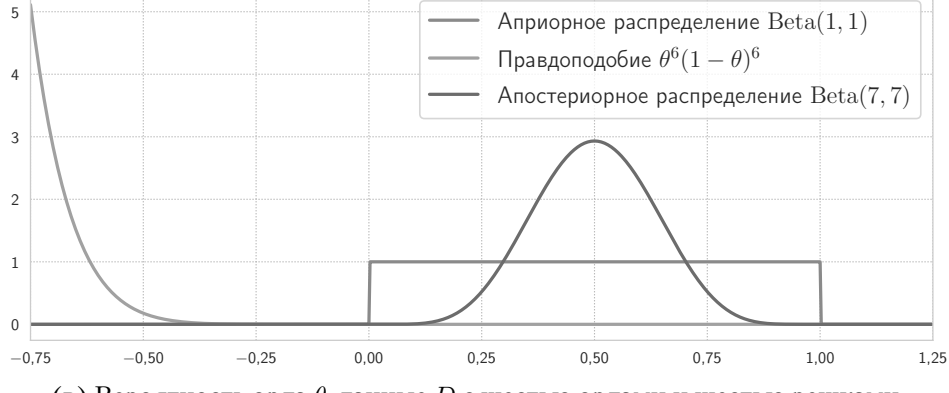
$$p\,(\theta|D) = \text{Beta}\,(\theta|n+1,m+1) = \\ = \begin{cases} \frac{1}{B(n+1,m+1)}\theta^n\,(1-\theta)^m = \frac{(n+m)!}{n!m!}\theta^n\,(1-\theta)^m\,, & \text{если } \theta \in [0,1], \\ 0 & \text{в противном случае.} \end{cases}$$



(а) Вероятность орла θ, данные D с двумя орлами



(б) Вероятность орла θ, данные D с одним орлом и двумя решками



(в) Вероятность орла θ, данные D с шестью орлами и шестью решками

Рис. 2.10. Примеры бета-распределений в анализе монетки

Давайте рассмотрим несколько примеров бета-распределений, которые могут получиться в результате такого вывода. Рисунок 2.10 показывает три таких примера с равномерным априорным распределением: на рис. 2.10, *а* данные состоят из двух орлов, на рис. 2.10, *б* — из одного орла и двух решек, а на рис. 2.10, *в* — из шести орлов и шести решек. Обратите внимание, что функция правдоподобия формально продолжается и за пределы отрезка $[0,1]$, причём может вести себя там как угодно, а «обрезает» её в нужный интервал априорное распределение; собственно, в случае монетки правдоподобие представляет собой многочлен, поэтому нет ничего удивительного в том, что оно в какой-то момент начинает уходить к бесконечности. А поскольку априорное распределение равномерное, форма апостериорного распределения на $\theta \in [0,1]$ повторяет форму функции правдоподобия, разница там только в нормировке.

2.4.3. Предсказательное распределение и правило Лапласа

Но это только начало. Мы решили только первую задачу байесовского вывода — поняли, как выглядит апостериорное распределение параметра θ, да и то только для равномерного априорного распределения. К нему мы ещё вернёмся, а пока давайте сделаем следующий шаг. Как мы обсуждали в разделе 2.2.4, параметры модели для нас обычно играют инструментальную роль, сама по себе оценка θ нам вряд ли нужна, а нужно нам на самом деле предсказать, какой стороной она выпадет в следующий раз, то есть найти вероятность выпадения орла в следующем эксперименте.

Для этого, как уже обсуждалось выше, нужно найти

$$p\,(\text{орёл}|D) = \int p\,(\text{орёл}|\theta)\,p\,(\theta|D)\,\mathrm{d}\theta.$$

Здесь $p\,(\theta|D)$ — это уже найденное нами выше апостериорное бета-распределение, а $p\,(\text{орёл}|\theta) = \theta$ буквально по определению, у нашей монетки довольно простой и понятный параметр.

Давайте подставим всё это в интеграл и подсчитаем:

$$p\,(\text{орёл}|D) = \int_{-\infty}^{\infty} p\,(\text{орёл}|\theta)\,p\,(\theta|D)\,\mathrm{d}\theta = \\ = \int_0^1 \theta \cdot \frac{(n+m)!}{n!m!}\theta^n\,(1-\theta)^m\,\mathrm{d}\theta = \frac{(n+m)!}{n!m!}\int_0^1 \theta^{n+1}\,(1-\theta)^m\,\mathrm{d}\theta. \quad (2.33)$$

Обратите внимание, что изначально мы интегрируем по всем возможным значениям θ, а ограничивает интеграл до $[0,1]$ апостериорное распределение. А последний интеграл мы уже видели — это опять бета-функция, только с изменившимся параметром! Получается, что

$$p\,(\text{орёл}|D) = \frac{(n+m)!}{n!m!}B(n+2,m+1) = \\ = \frac{(n+m)!}{n!m!}\frac{(n+1)!m!}{(n+m+2)!} = \frac{n+1}{n+m+2}. \quad (2.34)$$

Результат (2.34) называется *правилом Лапласа*[1] (Laplace's rule of succession), который изначально ставил задачу в следующей формулировке: если мы ничего не знаем о небесной механике, а просто в течение 5000 лет наблюдаем восход солнца каждое утро, с какой вероятностью мы можем ожидать, что завтра солнце взойдёт снова? Такая постановка звучит немного странно для одного из отцов-основателей небесной механики, но для нас равномерное априорное распределение означает всего лишь то, что мы ничего не знаем о вероятности выпадения орла. Впрочем, в разделе 3.4 мы обсудим и другие возможные формализации этой интуиции.

Кстати, вместе со значением предсказанной вероятности орла можно найти и точность этой оценки. Мы знаем, что $p\,(\text{орёл}|D)$ — это математическое ожидание правдоподобия одного орла (то есть параметра θ) по апостериорному распределению $p\,(\theta|D)$:

$$p\,(\text{орёл}|D) = \int p\,(\text{орёл}|\theta)\,p\,(\theta|D)\,\mathrm{d}\theta = \mathbb{E}_{p(\theta|D)}\left[\theta\right].$$

Но ведь можно найти не только среднее, но и дисперсию:

$$\mathbb{E}_{p(\theta|D)}\left[\theta^2\right] = \int_0^1 \theta^2 \cdot \frac{(n+m)!}{n!m!}\theta^n\,(1-\theta)^m\,\mathrm{d}\theta = \\ = \frac{(n+m)!}{n!m!}\frac{(n+2)!m!}{(n+m+3)!} = \frac{(n+1)(n+2)}{(n+m+1)(n+m+2)},$$

а значит,

[1] *Пьер-Симон, маркиз де Лаплас* (Pierre-Simon de Laplace, 1749–1827) — французский математик, механик, физик, астроном и почётный великий мастер Великого востока Франции. Знаменитым его сделало доказательство устойчивости Солнечной системы — к этому относится и вырванная из контекста фраза о том, что «в гипотезе о Боге он не нуждался»; на самом деле речь, скорее всего, шла о гипотезе о том, что Бог должен периодически вмешиваться и компенсировать накапливающиеся возмущения, как казалось Ньютону. Главным трудом Лапласа стала четырёхтомная «Небесная механика», которая выходила в течение 26 лет [378], а в «Изложении системы мира» Лаплас ещё в 1796 году фактически предсказал чёрные дыры. Кроме того, Лаплас стал одним из основоположников теории вероятностей (производящие функции, предельная теорема Муавра — Лапласа, метод наименьших квадратов) и математической физики (преобразование и уравнение Лапласа) — любопытно, что в физике и небесной механике он при этом был убеждённым детерминистом и, наверное, очень удивился бы принципиальной стохастичности механики квантовой. Лаплас руководил Бюро долгот, где предложил слово «метр» при переходе на метрическую систему, открыл барометрическую формулу и многое другое.

$$\mathrm{Var}_{p(\theta|D)}[\theta] = \mathbb{E}_{p(\theta|D)}\left[\theta^2\right] - \mathbb{E}_{p(\theta|D)}[\theta]^2 =$$
$$= \frac{(n+1)(n+2)}{(n+m+2)(n+m+3)} - \frac{(n+1)^2}{(n+m+2)^2} = \frac{(n+1)(m+1)}{(n+m+2)^2(n+m+3)}.$$

Если обозначить через $p = \frac{n+1}{m+2}$ предсказанное ожидаемое значение вероятности орла $p(\text{орёл}|D)$, полученный результат можно записать в довольно изящном виде, напоминающем о дисперсии обычной монетки:

$$p(\text{орёл}|D) = p \pm \sqrt{\frac{p(1-p)}{n+m+3}}. \tag{2.35}$$

Как и ожидалось, дисперсия предсказательного распределения падает с ростом числа экспериментов. Результатом (2.35) постоянно пользуются социологи: какой нужен размер выборки, если вам нужно оценить, например, поддержку одного из двух кандидатов на выборах? Если $p = \frac{1}{2}$ (это худший случай), то для выборки размера $n + m = N$ дисперсия нашего предсказания получается равной $\sqrt{1/(4(N+3))}$. Таким образом, если, например, мы хотим получить оценку с точностью $\pm 1\,\%$, нам нужна выборка размером около $\frac{100^2}{4} = 2500$ человек, что в целом действительно похоже на рекомендованный объём выборки в такого рода опросах[1].

2.4.4. Сопряжённые априорные распределения

Но и это ещё не всё, что можно сказать о байесовском анализе монетки. Мы до сих пор рассматривали только равномерное априорное распределение, то есть распределение, за которым стоит примерно следующая интуиция: монетка может иметь любую вероятность выпадения орла, причём заранее у нас нет никакого предпочтения; орёл может выпадать с вероятностью $\frac{1}{2}$, $\frac{1}{10}$ или 0 с одинаковым успехом. Выражаясь точнее (у нас ведь непрерывное распределение, и «вероятность» каждой конкретной точки в нём равна нулю), равномерное априорное распределение утверждает, что мы одинаково (не) удивимся, увидев вероятность орла θ как в интервале $[0{,}45, 0{,}55]$, так и в интервале $[0{,}85, 0{,}95]$, у них одинаковая априорная вероятность $\frac{1}{10}$.

Но разве это действительно соответствует нашей интуиции? Лично моей — не очень; доставая случайную монетку из кошелька, я, конечно, не ожидаю точного равенства $\theta = \frac{1}{2}$ (нет на свете совершенства), но ожидаю, что θ будет так или

[1] У социологов, к сожалению, проблемы на этом только начинаются: такой объём выборки подойдёт, если эти люди представляют собой действительно случайную выборку из генеральной совокупности, в которой ожидаемая поддержка никуда не смещена. Получить такую выборку обычно нелегко; просто подходить к людям на улице или обзванивать случайные номера телефонов — плохая идея как минимум из-за того, что выборка будет состоять только из людей, имеющих известный номер телефона (или часто ходящих по улицам), а также, что ещё важнее, готовых вам при таком подходе ответить.

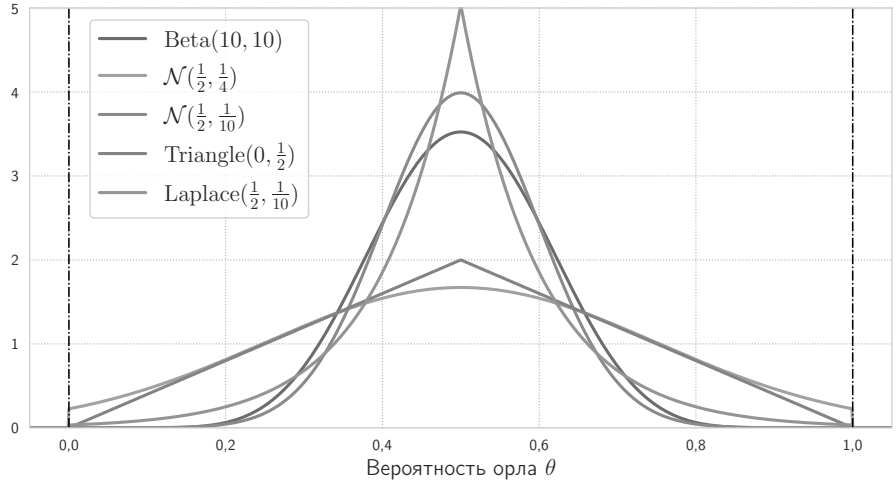


Рис. 2.11. Разные колоколообразные распределения с центром в $\theta = \frac{1}{2}$

иначе близка к $\frac{1}{2}$. Если попытаться нарисовать мою интуицию в виде графика, подобного рис. 2.10, она будет выглядеть как колоколообразное распределение с центром в точке $\theta = \frac{1}{2}$. Из примеров на рис. 2.10 ближе всего к моей интуиции будет, наверное, апостериорное распределение Beta $(\theta|7, 7)$ с рис. 2.10, *в*, но и его я бы, наверное, сделал более «острым» — я верю в лучшее и в людях, и в монетках.

А теперь главный вопрос: как определить это распределение? На свете миллиарды колоколообразных распределений с центром в $\frac{1}{2}$, причём миллиарды — это явная недооценка масштаба проблемы, на самом деле они образуют непрерывное и даже бесконечномерное множество. На рис. 2.11 показаны несколько вариантов того, как можно было бы формально определить такой колокол:

- треугольное распределение, которое поднимается от 0 в $\theta = 0$ до $p\left(\frac{1}{2}\right) = 2$ по прямой и затем убывает по прямой до $p\left(1\right) = 0$;
- уже знакомое нам бета-распределение Beta $(\theta|\alpha, \beta)$, в котором нужно задать $\alpha = \beta$, чтобы пик получился в $\theta = \frac{1}{2}$, а само значение α и β будет задавать «высоту» этого колокола, то есть дисперсию;
- нормальное распределение $\mathcal{N}\left(\theta\middle|\frac{1}{2}, \sigma^2\right)$; сам гауссиан нам не подходит, потому что он будет продолжаться влево и вправо до бесконечности, но ведь никто не мешает его обрезать, обнулив вне отрезка $[0,1]$ и перенормировав внутри; на рис. 2.11 показано в том числе $\mathcal{N}\left(\theta\middle|\frac{1}{2}, \frac{1}{4}\right)$, для которого это обрезание на краях интервала видно невооружённым глазом;
- распределение Лапласа

$$\text{Laplace}\left(x\middle|\frac{1}{2}, b\right) = \frac{1}{2b}e^{-\frac{1}{b}\left|x-\frac{1}{2}\right|};$$

как и гауссиан, распределение Лапласа тоже продолжается в обе стороны до бесконечности, но и с ним можно провернуть тот же трюк, обрезав на $[0, 1]$ и перенормировав результат.

И так далее и тому подобное — функций много, все функции хороши, выбирай на вкус. Но как выбрать? Разве у нас есть интуиция в отношении монеток, которая бы позволила различить показанные на рис. 2.11 распределения? Лично я точно не смог бы отличить, достал ли я из кармана монетку с априорным распределением $\mathcal{N}\left(\theta\middle|\frac{1}{2}, \frac{1}{10}\right)$ или $\text{Beta}\,(\theta|10, 10)$, а вы?

Чтобы всё-таки выбрать априорное распределение или хотя бы семейство, из которого брать априорные распределения, нужно привлечь какие-то дополнительные соображения. И здесь на первый план выходит *удобство* — довольно удивительно встретить такое соображение при выборе математической функции, но других в данном случае действительно обычно нет. Мы выбираем априорное распределение для того, чтобы использовать его при обучении, то есть подставить его в качестве $p\,(\boldsymbol{\theta})$ в теорему Байеса (2.21). Поэтому главная операция, которую нам придётся проводить с априорным распределением, — это умножение на правдоподобие $p\,(D|\boldsymbol{\theta})$.

Представьте, что мы выбрали в качестве априорного обрезанное нормальное распределение, то есть функцию

$$p\,(\theta) \propto e^{-\frac{1}{2\sigma^2}\left(\theta-\frac{1}{2}\right)}, \quad \theta \in [0{,}1].$$

Тогда при выводе по теореме Байеса будет происходить примерно следующее: для датасета D с n орлами и m решками

$$p\,(\theta|D) \propto p\,(\theta)\,p\,(D|\theta) \propto \theta^n\,(1-\theta)^m\,e^{-\frac{1}{2\sigma^2}\left(\theta-\frac{1}{2}\right)}$$

на интервале $[0, 1]$. Не очень удобно выглядит, правда? Выходит, что апостериорное распределение совсем не похоже ни на априорное распределение, ни на правдоподобие, да и вообще ни на что разумное не похоже. Нам придётся долго мучиться, если мы захотим сказать об этом распределении что-то содержательное аналитически; да и численно подсчитать здесь всё легко только потому, что монетка имеет один параметр. В высокой размерности и приближённый вывод представлял бы собой серьёзную проблему — о проклятии размерности мы поговорим в разделе 6.1.

Как сделать жизнь проще? Было бы замечательно, если бы априорное распределение *легко* умножалось на правдоподобие. В случае монетки правдоподобие выглядит как $\theta^n\,(1-\theta)^m$ — какое распределение было бы удобно умножать на эту функцию? Конечно же, бета-распределение, ведь оно выглядит точно так же! И байесовский вывод при таком подходе превращается в сложение показателей: для $p\,(\theta) = \text{Beta}\,(\theta|\alpha, \beta)$ мы получим

$$p(\theta|D) \propto p(\theta)\, p(D|\theta) \propto \theta^n (1-\theta)^m\, \theta^{\alpha-1} (1-\theta)^{\beta-1} = \\ = \text{Beta}(\theta|\alpha+n, \beta+m). \quad (2.36)$$

Совсем другое дело! При помощи результата (2.36) мы можем свести умножение априорного распределения на правдоподобие к простому сложению: параметр α превращается в $\alpha+n$, а параметр β — в $\beta+m$, но форма бета-распределения сохраняется.

Это частный случай более общего принципа, который в машинном обучении называют *сопряжёнными априорными распределениями* (conjugate priors). Говорят, что семейство распределений $\mathcal{P} = \{p(\boldsymbol{\theta})\}$ является семейством сопряжённых априорных распределений для семейства правдоподобий $\mathcal{L} = \{p(D|\boldsymbol{\theta})\}$, если апостериорные распределения, получаемые умножением априорных из $\mathcal{P}$ на правдоподобия из $\mathcal{L}$, по-прежнему лежат в $\mathcal{P}$:

$$\forall p(\boldsymbol{\theta}) \in \mathcal{P}, p(D|\boldsymbol{\theta}) \in \mathcal{L} \qquad p(\boldsymbol{\theta})\, p(D|\boldsymbol{\theta}) \propto p(\boldsymbol{\theta}|D) \in \mathcal{P}. \quad (2.37)$$

Если $\mathcal{P}$ — это параметрическое семейство, то есть у $p(\boldsymbol{\theta})$ есть общая форма с параметрами $\boldsymbol{\alpha}$ в виде $\mathcal{P} = \{p(\boldsymbol{\theta}|\boldsymbol{\alpha}) \mid \boldsymbol{\alpha} \in \mathcal{A}\}$, то это определение можно записать ещё проще: для любого $\boldsymbol{\alpha} \in \mathcal{A}$, если $p(D|\boldsymbol{\theta}) \in \mathcal{L}$, то

$$p(\boldsymbol{\theta})\, p(D|\boldsymbol{\theta}) \propto p(\boldsymbol{\theta}|D) = p\left(\boldsymbol{\theta}\middle|\boldsymbol{\alpha}'\right) \text{ для некоторого } \boldsymbol{\alpha}' \in \mathcal{A}. \quad (2.38)$$

Конечно, определение (2.37) можно довести до абсурда: например, семейство *всех* распределений будет сопряжённым к любому семейству правдоподобий. Но обычно в машинном обучении речь идёт о параметрическом семействе, как мы сразу же и заявили в формуле (2.38), и использование сопряжённого априорного распределения значит, что мы можем заменить сложную операцию — умножение распределений с перенормировкой — на куда более простую: пересчёт параметров $\boldsymbol{\alpha}$ в $\boldsymbol{\alpha}'$. Именно это мы проделали с монеткой, когда вывели формулу (2.36). В дальнейшем мы рассмотрим ещё несколько важных примеров; например, в главе 4 мы будем много говорить о сопряжённых априорных распределениях для линейной регрессии.

Обратите внимание, что при использовании сопряжённых априорных распределений можно очень легко делать байесовский вывод, последовательно добавляя новые данные. Если мы сначала подсчитали апостериорное распределение модели для некоторого датасета D,

$$p(\boldsymbol{\theta}|D) \propto p(\boldsymbol{\theta})\, p(D|\boldsymbol{\theta}), \quad (2.39)$$

а затем получили дополнительные данные D', нет нужды пересчитывать апостериорное распределение $p\left(\boldsymbol{\theta}\middle|D, D'\right)$ заново, можно использовать $p(\boldsymbol{\theta}|D)$ в качестве априорного распределения для нового вывода:

$$p\left(\boldsymbol{\theta}\middle|D,D'\right) \propto p(\boldsymbol{\theta}|D)\, p\left(D'\middle|\boldsymbol{\theta}\right). \quad (2.40)$$

Разумеется, формулы (2.39) и (2.40) верны всегда, для любого $p(\boldsymbol{\theta})$, но с сопряжёнными априорными распределениями их становится не сложно применить и на практике. Если $p(\boldsymbol{\theta}|\boldsymbol{\alpha})$ взято из семейства сопряжённых априорных распределений, то при получении на вход данных D, даже если мы ожидаем, что можем в будущем получить ещё и дополнительные данные D', мы можем сделать байесовский вывод согласно формуле (2.38) и запомнить только $\boldsymbol{\alpha}'$, а датасет D больше не хранить вовсе[1].

Заметим, что построение предсказательного распределения, которое мы для равномерного априорного распределения подытожили в правиле Лапласа (2.34), тоже остаётся несложным для любого априорного распределения $p(\theta)$ из семейства бета-распределений. Если $p(\theta) = \text{Beta}(\theta|\alpha,\beta)$, то по тем же соображениям, что в формуле (2.33), мы получим

$$\begin{aligned} p(\text{орёл}|D) &= \int_{-\infty}^{\infty} p(\text{орёл}|\theta)\, p(\theta|D)\, \mathrm{d}\theta \\ &= \int_{-\infty}^{\infty} p(\text{орёл}|\theta)\, \text{Beta}(\theta|\alpha+n,\beta+m)\, \mathrm{d}\theta \\ &= \int_{0}^{1} \theta \cdot \frac{1}{B(n+\alpha,m+\beta)} \theta^{n+\alpha-1} (1-\theta)^{m+\beta-1}\, \mathrm{d}\theta \\ &= \frac{B(n+\alpha+1,m+\beta)}{B(n+\alpha,m+\beta)} = \frac{n+\alpha}{n+\alpha+m+\beta}. \end{aligned}$$

Более того, всё это обсуждение приводит нас к ещё одному интересному понятию. Апостериорное распределение в данном случае оказывается похоже не только на априорное распределение, но и на правдоподобие, и это не случайно — действительно чаще всего будет получаться так, что удобнее всего умножать правдоподобие на функцию, которая выглядит точно так же. В таком случае у нас появляется не только арифметическое удобство умножения, но и некая дополнительная *интерпретируемость* априорного распределения: мы можем понять его «физический смысл» через сравнение с правдоподобием.

Например, априорное распределение $\text{Beta}(\theta|5,5) \propto \theta^4(1-\theta)^4$ для монетки пропорционально правдоподобию, которое бы получилось, если бы мы подбросили монетку восемь раз и получили четыре орла и четыре решки. Теперь вместо того, чтобы пытаться прикинуть точную форму колокола у своих предпочтений, мне достаточно сказать примерно следующее: «Я верю в честность взятой из кармана монетки примерно так же, как если бы подбросил её 50 раз и получил 25 решек и 25 орлов».

Здесь по-прежнему не до конца ясно, откуда взять конкретные числа, но это уже звучит гораздо более интуитивно понятно, чем «форма колокола должна

[1] Эта идея перекликается с идеей достаточных статистик, которые мы обсудим в разделе 6.6.

быть как у бета-распределения с $\alpha = 26$ и $\beta = 26$». Такой эффект называется *эквивалентным размером выборки* (equivalent sample size) и может помочь выбрать априорное распределение.

И последнее концептуально важное замечание. Как подсказывает нам правило Кромвеля, в качестве априорного распределения не нужно брать дельта-функцию $\delta(\theta - \frac{1}{2})$, то есть лучше не быть абсолютно уверенными в том, что монетка непременно окажется честной. Это значит, что даже если мы изначально выберем очень «уверенное» априорное распределение с большим эквивалентным размером выборки, достаточное число экспериментов сможет нас переубедить и существенно сместить наше представление о вероятности орла θ.

Это значит, что известная *ошибка игрока* (gambler's fallacy) — рассуждение о том, что если на рулетке десять раз подряд выпало чёрное, то вероятность выпадения красного цвета в следующий раз увеличивается, — не просто неверна, а верна в обратную сторону! Если вы играете на реальной физической рулетке и в наблюдаемой вами выборке её значений преобладает чёрное, стоит рассмотреть возможность того, что рулетка не вполне честна (например, слегка перекосилась) и чёрное действительно имеет бóльшую вероятность.

2.4.5. Игральные кости и распределения, добавляющие разреженность

В предыдущих разделах мы полностью обсудили байесовский вывод в случае монетки, то есть случайного события с двумя исходами, которое задаётся одной вероятностью $\theta = p(\text{орёл})$. Давайте начнём с того, что обобщим все наши конструкции на случай игральной кости с K гранями, у которой несколько возможных исходов $\{1, 2, \ldots, K\}$.

Соответствующее игральной кости мультиномиальное распределение задаётся уже, очевидно, не одним числом θ, а целым вектором

$$\boldsymbol{\theta} = \begin{pmatrix} \theta_1 & \theta_2 & \ldots \theta_K \end{pmatrix}^\top,$$

где $\theta_k \in [0, 1]$ и $\sum_{k=1}^{K} \theta_k = 1$.

Важное замечание: в машинном обучении вообще практически всегда удобнее выражать категориальные величины в виде векторов. Это называется *прямым кодированием* (one-hot encoding)[1]: если у рассматриваемой величины, будь то целевая переменная для классификатора или категориальный признак в линейной регрессии, K возможных значений, и они никак не упорядочены и не вы-

[1] Здесь я столкнулся с неожиданной проблемой: я никогда не слышал этого термина по-русски! Во всех курсах и источниках, которые я знаю, авторы так и говорили: *one-hot*, не переводя на русский. Словари предложили «прямое кодирование», «кодирование с одним активным состоянием» и «кодирование с одним "горячим" состоянием»; все варианты звучат разумно, так что я выбрал просто самый короткий перевод, но, честно говоря, ни одного из этих вариантов я в живой природе не встречал и сам дальше наверняка буду сбиваться на англоязычный термин.

ражаются числами[1], то разумнее всего присвоить каждому значению свою собственную размерность и кодировать их вектором, в котором все компоненты равны нулю, кроме одного, соответствующего ответу. Например, результаты броска обычного шестигранного кубика мы закодируем так (я продолжаю ставить знаки транспонирования, потому что привык к векторам-столбцам, но это не принципиально):

$$\begin{array}{rclrcl} \text{One-Hot}\,(\boxdot) &=& (1\,0\,0\,0\,0\,0)^\top, & \text{One-Hot}\,(\boxdot) &=& (0\,0\,0\,1\,0\,0)^\top, \\ \text{One-Hot}\,(\boxdot) &=& (0\,1\,0\,0\,0\,0)^\top, & \text{One-Hot}\,(\boxdot) &=& (0\,0\,0\,0\,1\,0)^\top, \\ \text{One-Hot}\,(\boxdot) &=& (0\,0\,1\,0\,0\,0)^\top, & \text{One-Hot}\,(\boxdot) &=& (0\,0\,0\,0\,0\,1)^\top. \end{array}$$

Прямое кодирование удобно в первую очередь тем, что позволяет работать с вероятностями: если мы обучаем вектор $\boldsymbol{\theta}$ на данных $D = \{d_1, d_2, \ldots, d_N\}$, полученных из игральной кости, целевой переменной для него будет как раз прямое кодирование.

Правдоподобие результатов броска игральной кости $D = \{n_1, n_2, \ldots, n_K\}$, где грань с номером i выпала n_i раз, таким образом, будет равно

$$p\,(D|\boldsymbol{\theta}) = \theta_1^{n_1}\theta_2^{n_2}\ldots\theta_K^{n_K}; \tag{2.41}$$

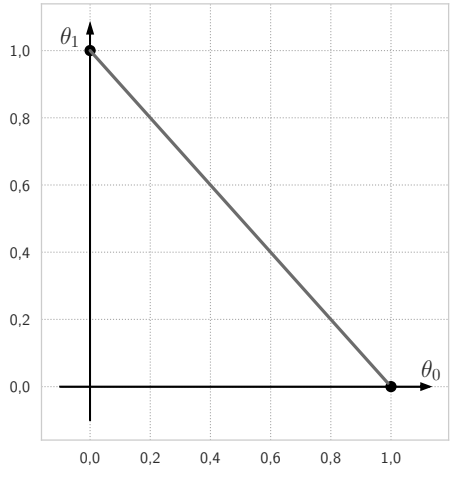
здесь мы сразу же использовали тот факт, что правдоподобие не зависит от порядка выпадения граней игральной кости, а только от числа тех или иных исходов.

Как выбрать априорное распределение? Идейно ничего нового здесь не будет; мы по-прежнему хотим выбрать сопряжённое априорное распределение, то есть сделать так, чтобы умножать на правдоподобие (2.41) было удобно, и снова получалось распределение той же формы, с которой мы начинали. Как и в случае монетки, здесь очевидно, как этого добиться: нужно взять $p\,(\boldsymbol{\theta})$, пропорциональное произведению θ_k в некоторых степенях. Такое распределение, конечно, тоже существует и называется *распределением Дирихле*: для некоторых гиперпараметров $\boldsymbol{\alpha} = (\,\alpha_1\ \cdots\ \alpha_K\,)$

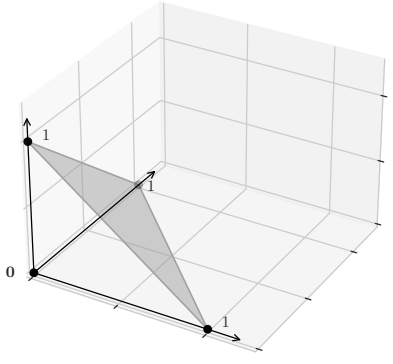
$$p\,(\boldsymbol{\theta}) = \text{Dir}\,(\boldsymbol{\theta}|\boldsymbol{\alpha}) = \begin{cases} \frac{1}{B(\boldsymbol{\alpha})}\theta_1^{\alpha_1-1}\theta_2^{\alpha_2-1}\ldots\theta_K^{\alpha_K-1}, & \theta_k \geq 0,\ \sum\limits_{k=1}^{K}\theta_k = 1, \\ 0 & \text{в противном случае.} \end{cases} \tag{2.42}$$

Здесь есть два момента, которые нужно обсудить. Во-первых, распределение Дирихле определено (точнее, не равно нулю) не на всём пространстве $\boldsymbol{\theta} \in \mathbb{R}^d$:

[1] Может показаться, что у игральной кости грани как раз выражаются числами, но это, конечно, иллюзия, которую мы обсуждали выше: числа — это лишь удобные метки, а в действительности мы рассматриваем каждую грань как отдельное элементарное событие. Выпавшая на кубике четвёрка ни в каком смысле не «ближе» к пятёрке, чем к единице.



(а) одномерный симплекс



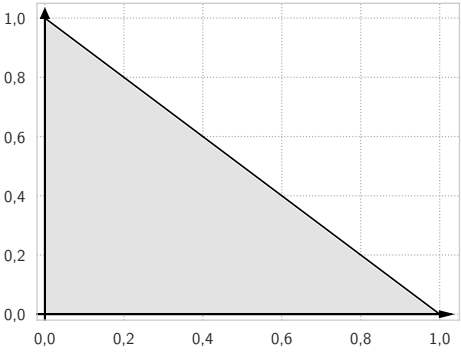
(б) двумерный симплекс

Рис. 2.12. Симплексы в низких размерностях: (а) одномерный симплекс в $\mathbb{R}^2$; (б) двумерный симплекс в $\mathbb{R}^3$

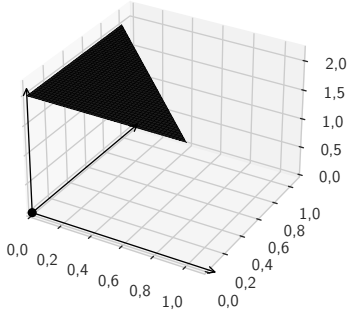
аналогично тому, как бета-распределение было определено на допустимых значениях параметра монетки $\theta \in [0,1]$, а остальные значения θ обнуляло, распределение Дирихле обнуляет все варианты, кроме допустимых вероятностей граней игральной кости. Это значит, что распределение Дирихле определено на *симплексе*, то есть на множестве вида $\theta_k \in [0,1]$, $\sum_{k=1}^{K} \theta_k = 1$:

- для $K = 2$ мы могли бы вместо одного параметра θ рассмотреть вектор из двух вероятностей $\boldsymbol{\theta} = \begin{pmatrix}\theta_0 & \theta_1\end{pmatrix}$, которые должны быть неотрицательными и суммироваться в единицу, $\theta_0 + \theta_1 = 1$, но для монетки это было бы излишним усложнением; соответствующий одномерный симплекс (отрезок прямой) в координатах (θ_0, θ_1) показан на рис. 2.12, *а*; когда мы рассматривали монетку, мы проецировали этот симплекс на одну из координат, скажем θ_0, а в качестве второй подставляли $\theta_1 = 1 - \theta_0$;
- для $K = 3$ уже действительно есть смысл рассматривать вектор из трёх вероятностей $\boldsymbol{\theta} = \begin{pmatrix}\theta_0 & \theta_1 & \theta_2\end{pmatrix}$, которые неотрицательны и удовлетворяют условию $\theta_0 + \theta_1 + \theta_2 = 1$; соответствующий двумерный симплекс (участок плоскости) в координатах $(\theta_0, \theta_1, \theta_2)$ показан на рис. 2.12, *б*;
- симплексы в размерностях выше 3 я уже изобразить для вас не смогу, но устроены они точно так же.

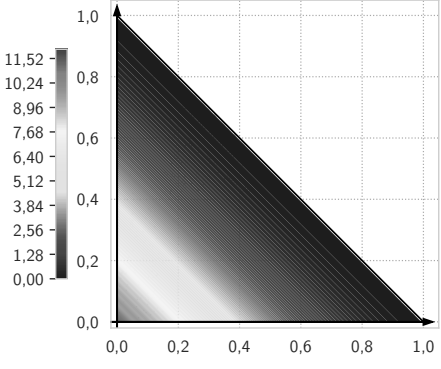
Во-вторых, нормировочная константа в распределении Дирихле, которая называется *многомерной бета-функцией* $B(\boldsymbol{\alpha})$, представляет собой результат интегрирования функции $\theta_1^{\alpha_1 - 1}\theta_2^{\alpha_2 - 1} \dots \theta_K^{\alpha_K - 1}$ по тому самому $(K-1)$-мерному



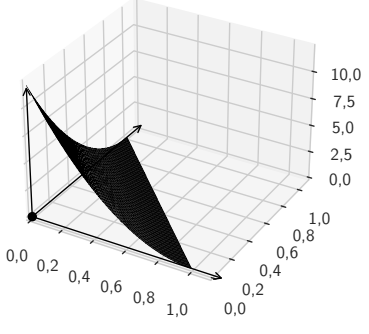
(а) Тепловая карта $\mathrm{Dir}\left(\boldsymbol{x}\middle|\begin{pmatrix}1 & 1 & 1\end{pmatrix}\right)$



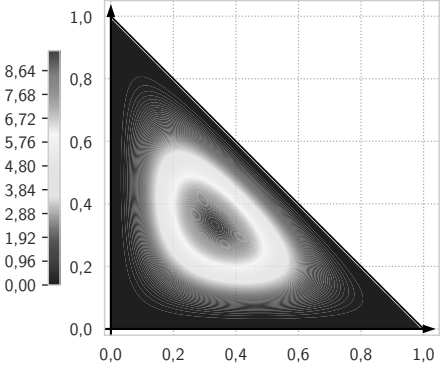
(б) График $\mathrm{Dir}\left(\boldsymbol{x}\middle|\begin{pmatrix}1 & 1 & 1\end{pmatrix}\right)$



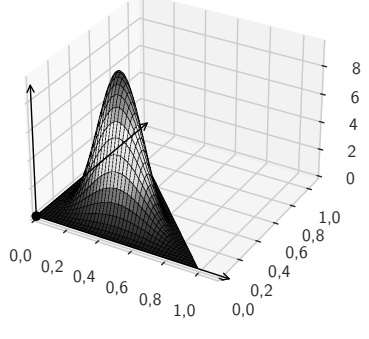
(в) Тепловая карта $\mathrm{Dir}\left(\boldsymbol{x}\middle|\begin{pmatrix}1 & 1 & 3\end{pmatrix}\right)$



(г) График $\mathrm{Dir}\left(\boldsymbol{x}\middle|\begin{pmatrix}1 & 1 & 3\end{pmatrix}\right)$



(д) Тепловая карта $\mathrm{Dir}\left(\boldsymbol{x}\middle|\begin{pmatrix}4 & 4 & 4\end{pmatrix}\right)$



(е) График $\mathrm{Dir}\left(\boldsymbol{x}\middle|\begin{pmatrix}4 & 4 & 4\end{pmatrix}\right)$

Рис. 2.13. Примеры распределений Дирихле для разных $\boldsymbol{\alpha}$

симплексу $S_K \subset \mathbb{R}^K$. Брать интеграл в подробностях мы не будем, но результат тоже определяется через гамма-функцию абсолютно аналогичным образом:

$$B(\boldsymbol{\alpha}) = \frac{\Gamma(\alpha_1)\Gamma(\alpha_2)\ldots\Gamma(\alpha_K)}{\Gamma(\alpha_1+\alpha_2+\ldots+\alpha_K)}.$$

В итоге распределение Дирихле для разных значений $\boldsymbol{\alpha}$ выглядит так, как показано на рис. 2.13 в случае двумерного симплекса; на рисунке слева в каждой строке (рис. 2.13, *а*, *в*, *д*) изображена тепловая карта, на которой мы видим значения двух первых координат θ_1 и θ_2, а θ_3 подразумевается равным $1-\theta_1-\theta_2$, то есть симплекс развёрнут в «смотрящий на нас» треугольник; справа в каждой строке (рис. 2.13, *б*, *г*, *е*) — соответствующий трёхмерный график. Здесь всё выглядит довольно ожидаемо. Распределение Дирихле с $\boldsymbol{\alpha} = \begin{pmatrix}1 & \ldots & 1\end{pmatrix}$ — это равномерное распределение (рис. 2.13, *а*, *б*), и дальше интуиция работает так же, как для монетки: значения параметров распределения Дирихле — это «виртуальные эксперименты» бросания игральной кости. Если наша виртуальная кость выпадала только одной стороной, например $\boldsymbol{\alpha} = \begin{pmatrix}3 & 1 & 1\end{pmatrix}$, то распределение Дирихле будет скошено в сторону этого значения (рис. 2.13, *в*, *г*), а если выпадала всеми сторонами поровну, то максимум окажется в середине, как на рис. 2.13, *д*, *е* максимум распределения $\mathrm{Dir}\left(\boldsymbol{x}\middle|\boldsymbol{\alpha} = \begin{pmatrix}4 & 4 & 4\end{pmatrix}\right)$ оказывается в точке $\begin{pmatrix}\frac{1}{3} & \frac{1}{3} & \frac{1}{3}\end{pmatrix}$.

Априорное распределение (2.42) очевидно является сопряжённым к правдоподобию (2.41), и апостериорное распределение здесь выглядит так:

$$\begin{aligned} p\left(\boldsymbol{\theta}|D,\boldsymbol{\alpha}\right) &\propto p\left(\boldsymbol{\theta}|\boldsymbol{\alpha}\right) p\left(D|\boldsymbol{\theta}\right) = \\ &= \frac{1}{B(\boldsymbol{\alpha})}\theta_1^{\alpha_1}\theta_2^{\alpha-2-1}\ldots\theta_K^{\alpha_K-1} \times \theta_1^{n_1}\theta_2^{n_2}\ldots\theta_K^{n_K} \propto \\ &\propto \theta_1^{n_1+\alpha_1-1}\theta_2^{n_2+\alpha_2-1}\ldots\theta_K^{n_K+\alpha_K-1} \propto \mathrm{Dir}\left(\boldsymbol{\theta}|\boldsymbol{\alpha}+\boldsymbol{n}\right), \end{aligned} \quad (2.43)$$

где $\boldsymbol{n} = \begin{pmatrix}n_1 & n_2 & \ldots & n_K\end{pmatrix}$ — это вектор данных.

Предсказательное распределение тоже получается аналогично (2.33):

$$\begin{aligned} p\left(k|D\right) &= \int_{S_K} p\left(k|\boldsymbol{\theta}\right) p\left(\boldsymbol{\theta}|D\right) \mathrm{d}\boldsymbol{\theta} = \int_{S_K} \theta_k \cdot \mathrm{Dir}\left(\boldsymbol{\theta}|\boldsymbol{\alpha}+\boldsymbol{n}\right) \mathrm{d}\boldsymbol{\theta} \\ &= \int_{S_K} \theta_k \cdot \frac{1}{B(\boldsymbol{n}+\boldsymbol{\alpha})}\theta_1^{n_1+\alpha_1-1}\theta_2^{n_2+\alpha_2-1}\ldots\theta_K^{n_K+\alpha_K-1}\mathrm{d}\boldsymbol{\theta} \\ &= \frac{B(n_1+\alpha_1,\ldots,n_k+\alpha_k+1,\ldots,n_K+\alpha_K)}{B(\boldsymbol{n}+\boldsymbol{\alpha})} = \frac{n_k+\alpha_k+1}{\sum_{i=1}^K\left(n_i+\alpha_i\right)}; \end{aligned}$$

здесь интеграл по симплексу S_K сворачивается в многомерную бета-функцию.

На этом можно было бы и завершить разговор, но переход в более высокие размерности приводит нас к ещё одному любопытному вопросу. В предыдущих разделах я приводил много примеров бета-распределения $B(\alpha,\beta)$ для разных зна-

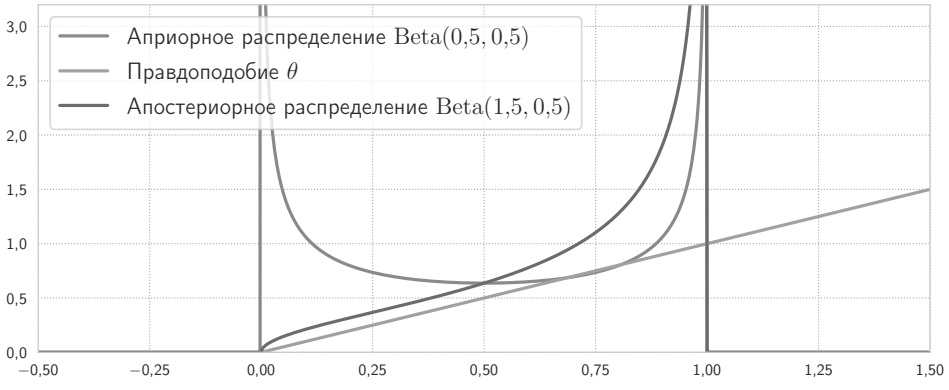


Рис. 2.14. Бета-распределение для $\alpha = \frac{1}{2}, \beta = \frac{1}{2}$

чений α и β (вспомните рис. 2.10). Все они укладывались в нашу концепцию «виртуальных бросков монетки с эквивалентным размером выборки»: в качестве параметров эти распределения имели натуральные числа, которые после вычитания единицы конвертировались в такие виртуальные эксперименты. Но определение бета-распределения $\text{Beta}(\theta|\alpha,\beta) = \frac{1}{B(\alpha,\beta)}\theta^{\alpha-1}(1-\theta)^{\beta-1}$ вполне позволяет рассматривать и дробные значения параметров, на то там и бета-функция, а не просто отношение факториалов.

Можем ли мы получить что-то новое и интересное, если возьмём нецелые значения α и β?

Если мы будем брать, как раньше, $\alpha \geq 1$ и $\beta \geq 1$, просто с дробными значениями, то ничего существенно нового не получится: априорное распределение с «десятью с половиной виртуальных бросков монетки» будет мало отличаться от априорного распределения, в котором результаты округлили и виртуальных бросков стало десять. А вот значения $\alpha < 1$ и $\beta < 1$ — это совсем другое дело.

Пример байесовского вывода в монетке с априорным распределением с дробными параметрами $\alpha = \beta = \frac{1}{2}$ показан на рис. 2.14. Посмотрите, к чему приводит такое распределение: если $\alpha < 1$ и $\beta < 1$, то распределение оказывается пропорциональным θ и $(1-\theta)$ в отрицательных степенях,

$$\text{Beta}\left(\theta\middle|\frac{1}{2},\frac{1}{2}\right) \propto \theta^{-\frac{1}{2}}(1-\theta)^{-\frac{1}{2}} = \frac{1}{\sqrt{\theta(1-\theta)}};$$

иначе говоря, распределение перевзвешено в сторону крайних значений 0 и 1.

Это значит, что результат буквально одного эксперимента, как и показано на рис. 2.14, производит апостериорное распределение, которое сильно перекошено в сторону того, что в этом эксперименте получилось.

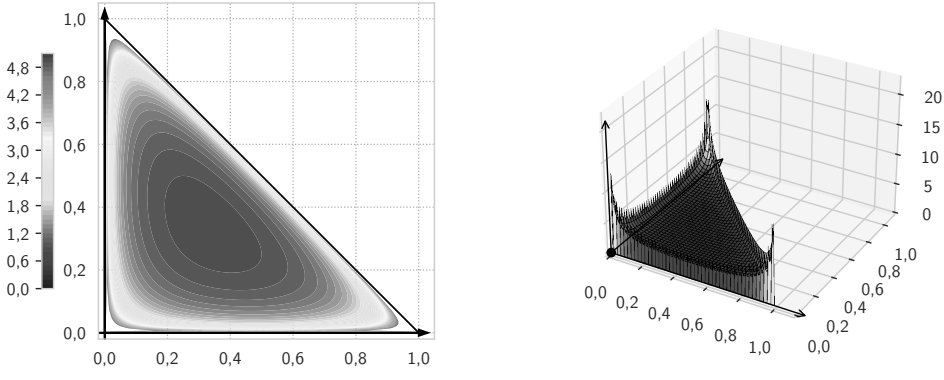


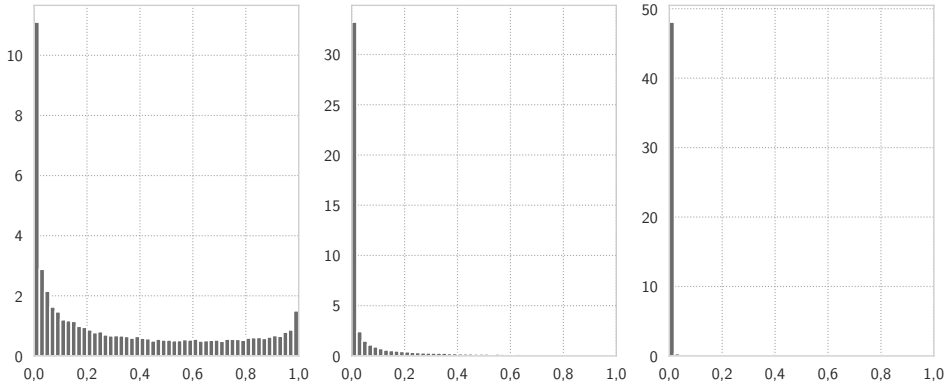
Рис. 2.15. Распределение Дирихле для $\boldsymbol{\alpha} = (\frac{1}{2}, \frac{1}{2}, \frac{1}{2})$

Иначе говоря, интуиция бета-распределения с дробными параметрами состоит в том, что распределение нашей монетки, скорее всего, сильно скошено в сторону либо орла, либо решки, но мы не знаем заранее, в какую именно сторону. Это как будто монетка из магазина фокусов: мы знаем, что это фальшивая монета для трюков, у которой с обеих сторон изображено одно и то же, но, пока мы на неё не посмотрим (не проведём хотя бы один эксперимент), мы не знаем, изображены ли там два орла или две решки.

Практический смысл такого априорного распределения в случае монетки сомнителен; лично я так и не смог придумать реального применения для априорного бета-распределения с дробными параметрами. Но давайте посмотрим, что будет происходить, когда мы обобщим эту конструкцию на случай игральной кости с несколькими гранями.

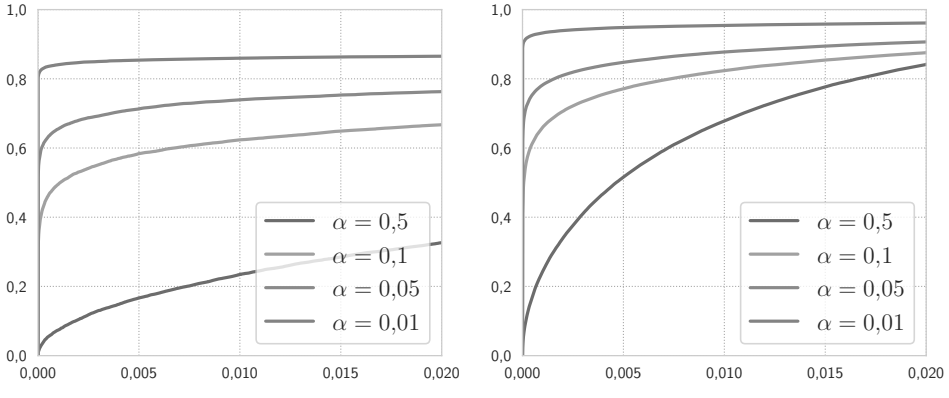
На рис. 2.15 изображён график распределения Дирихле в случае кости с тремя гранями (геометрически не очень реалистичная конструкция, хотя можно, конечно, представить себе обычный кубик, у которого на противоположных гранях изображено одно и то же число точек). Здесь мы снова видим тот же эффект: распределение Дирихле с дробными параметрами $\boldsymbol{\alpha} = (\frac{1}{2}, \frac{1}{2}, \frac{1}{2})$ перевзвешено в сторону рёбер и особенно углов треугольника, то есть в пользу значений, где одна, а лучше две из трёх вероятностей $\begin{pmatrix}\theta_1 & \theta_2 & \theta_3\end{pmatrix}$ очень близки к нулю.

Продолжая увеличивать число граней K, мы сможем добиваться похожего эффекта. Нарисовать графики самого распределения уже не получится, но давайте попробуем выяснить, какие распределения в целом предпочитает распределение Дирихле с дробными параметрами. Для этого посэмплируем много векторов $\boldsymbol{\theta}$ из распределения $\mathrm{Dir}(\boldsymbol{\theta}|\boldsymbol{\alpha})$ с разной размерностью и разными параметрами $\boldsymbol{\alpha}$ и посмотрим, какие значения вероятностей получаются; поскольку грани кубика для нас неразличимы, во всех примерах мы будем использовать симметричное распределение Дирихле с $\boldsymbol{\alpha} = \alpha \cdot \mathbf{1}$, где $\mathbf{1} = \begin{pmatrix}1 & 1 & \dots & 1\end{pmatrix}$.



(а) $K = 3, \boldsymbol{\alpha} = \frac{1}{3} \cdot \mathbf{1}$ (б) $K = 10, \boldsymbol{\alpha} = \frac{1}{10} \cdot \mathbf{1}$ (в) $K = 100, \boldsymbol{\alpha} = \frac{1}{100} \cdot \mathbf{1}$

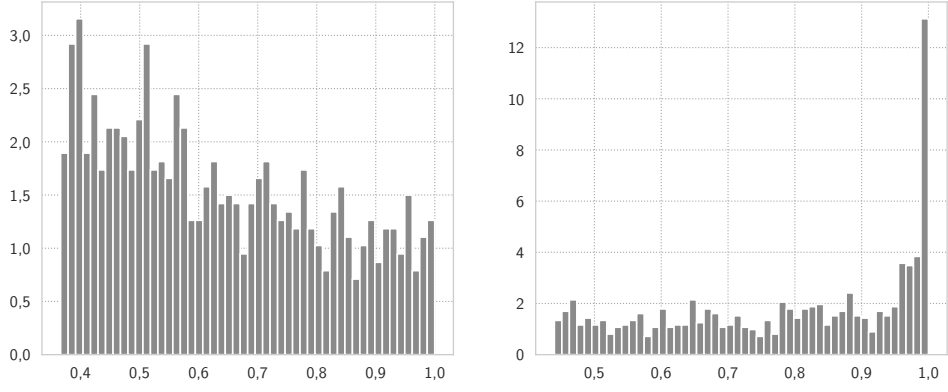
Рис. 2.16. Распределение значений вероятностей, порождённых из распределения Дирихле для разных K и $\boldsymbol{\alpha}$



(а) $K = 10$ (б) $K = 100$

Рис. 2.17. Эмпирические функции распределения значений вероятностей для распределения Дирихле с $\boldsymbol{\alpha} = \alpha \cdot \mathbf{1}$

Как видно из рис. 2.16, в больших размерностях подавляющее большинство вероятностей получаются очень близкими к нулю. Можно увидеть это ещё нагляднее, изобразив эмпирические функции распределения, то есть кумулятивную долю значений вероятностей, которые не превышают заданного порога. Такие функции для разных значений K и α изображены на рис. 2.17. Как видите, хотя функции распределения рано или поздно дойдут до единицы, на практике у них есть довольно большая зона «насыщения»: например, для $K = 100$



(а) $K = 100, \alpha = 0{,}01$ (б) $K = 10, \alpha = 0{,}05$

Рис. 2.18. Гистограммы значений наибольшего 1 % вероятностей для распределения Дирихле с $\boldsymbol{\alpha} = \alpha \cdot \mathbf{1}$

и $\alpha = 0{,}01$ 95 % всех вероятностей будут меньше чем приблизительно 0,005, а среди остальных 5 % будут уже попадаться всякие значения. На рис. 2.18 видно, что распределение топ-1 % вероятностей при $\alpha \approx \frac{1}{K}$ выглядит почти что равномерным, а при дальнейшем уменьшении α у этого распределения начинает появляться второй пик больших значений, близких к единице.

О чём нам всё это говорит? О том, что распределение Дирихле с дробными параметрами в качестве априорного добавляет нашей игральной кости *разреженности*[1]. Неверно, что в получающихся распределениях все вероятности, кроме одной, близки к нулю — но те, что не близки, составляют лишь малую долю общего числа исходов K. И если для монетки такой эффект было сложно применить на практике, то для игральных костей с большим числом граней он становится очень полезным.

Например, в разделе 5.6 мы поговорим о наивном байесовском классификаторе, который в случае классификации текстов делает предположение о том, что все слова в текстах на заданную тему (из заданного класса) порождаются независимо (в разделе 5.6 мы это предположение обсудим самым подробным образом). Иначе говоря, в стандартном варианте наивного байесовского классификатора каждый класс документов представляется игральной костью, у которой столько граней, сколько слов в словаре языка, — этаким «игральным шариком».

Следующий шаг развития моделей, основанных на этом предположении, — уже упоминавшиеся во введении *тематические модели* (topic models) [67, 68,

[1] Впрочем, разреженности в полном смысле слова, то есть строго нулевых вероятностей, получаться здесь не может, да и не должно — вспомните правило Кромвеля из раздела 2.2. Речь идёт о вероятностях, очень близких к нулю, но не строго нулевых.

436, 438, 489, 607], которые тоже обучают темы документов в виде таких «игральных шариков», но теперь каждый документ может содержать несколько разных тем. А значит, каждую отдельную тему хорошо было бы обучать в виде разреженного распределения: так она будет лучше интерпретироваться и скорее будет представлять собой действительно одну тему, а не смесь многих.

И действительно, главная тематическая модель — *латентное размещение Дирихле* (latent Dirichlet allocation, LDA) — получила своё название именно потому, что использует распределения Дирихле с дробными параметрами в качестве априорных распределений для «игральных шариков» распределений слов в темах. В целом, если в вашей модели появляется мультиномиальное распределение с большим числом исходов, исчисляемым сотнями или тысячами, присмотритесь к распределениям Дирихле с дробными параметрами: разреженность часто может помочь.

На этом мы завершаем обязательную вероятностную программу этой главы. Следующий раздел будет посвящён опциональному, но, надеюсь, любопытному сюжету «со звёздочкой», в котором мы поговорим о том, действительно ли баскетболисты могут «поймать фарт» и забрасывать много мячей подряд, или это всего лишь когнитивное искажение.

*2.5. Case study: монетки, подброшенные «горячей рукой»

Научный подход есть форма защиты от чистого чувства страха... Душевное равновесие обычного игрока зависит от закона, или, пускай, тенденции, или, скажем, вероятности, или, во всяком случае, математически вычислимых шансов, которые гарантируют, что у него не сядет психика от слишком большого проигрыша — и что он не надавит на психику своему партнёру слишком большим выигрышем... В конце концов, солнце вставало примерно так же часто, как и садилось, и монета падала примерно столько же решкой, сколько и орлом. А потом прибыл посланец. За нами послали. И больше ничего не случилось. Но девяносто две монеты, подброшенные одна за другой, упали девяносто два раза орлом... И в течение последних трех минут на ветру этого безветренного дня я слышу звуки барабана и флейты...

Том Стоппард. Розенкранц и Гильденстерн мертвы (пер. Иосифа Бродского)

Мёртвая змея не шипит,
Не щебечет дохлый щегол...

Запрещённые барабанщики

*2.5.1. Разоблачение эффекта «горячей руки»

Вы когда-нибудь чувствовали себя на коне, на вершине мира, ощущали, как именно сегодня всё складывается наиболее удачным образом? Наверняка такое бывало; но как вы распоряжались этим ощущением? Если сегодня «мой день», значит ли это, что нужно стараться использовать его по максимуму, предпринимать более рискованные попытки добиться более амбициозных результатов? Или лучше напомнить себе, что шарик рулетки, пять раз попавший на красное, всё равно имеет равную вероятность красного и чёрного в следующий раз, и стоит остановиться и порадоваться тому, что уже сложилось удачно?

В этом разделе я приведу любопытный пример, который показывает, что даже в самых основных, базовых вещах, вроде изучения бросаний монетки (испытаний Бернулли), теории вероятностей ещё есть чем удивить даже в наше, казалось бы, просвещённое время. Тему для этого примера я обнаружил в необычном для этой книги месте — на сайте sports.ru [671], хотя это был, конечно, пересказ недавно вышедшей популярной книги [103]. Замечу, что, хотя изложенные

в этом разделе математические выкладки уверенно превращают его в «раздел со звёздочкой», история получилась интересная и сама по себе, а доказательства при первом чтении можно пропустить.

Удачные и неудачные дни бывают у каждого: даже если представить жизнь как абсолютно случайный процесс, в некотором подмножестве дней монетка будет гораздо чаще выпадать орлом, а в другом подмножестве — решкой. И если бы можно было предсказать, какой день окажется удачным, или хотя бы экстраполировать начало дня на весь его остаток, многие люди могли бы использовать это к своей выгоде.

Хороший пример таких людей — профессиональные спортсмены. Во многих видах спорта игрок может до известной степени сам решать, брать ли на себя больше или меньше ответственности по ходу игры: футболист может чаще или реже идти в обводку и бить по воротам, шахматист может свернуть в сторону быстрой ничьей или пойти в осложнения, баскетболист — чаще или реже бросать по кольцу. Если игрок сможет предсказать, хороший у него сегодня будет день или плохой, это даст ему или его команде осязаемое преимущество.

Поэтому многие спортсмены пытаются оценить, как идёт игра сегодня, и принимать дальнейшие решения на основании этого. Кому-то это сделать сложно: например, в шахматных турнирах с классическим контролем почти никогда не играют больше одной партии в день. Но, к примеру, баскетболист довольно быстро может сделать несколько попыток и почувствовать, «залетает» у него сегодня или нет. И если «залетает», значит, нужно этим пользоваться: бросать с разных позиций, брать на себя игру, вести команду к победе. В профессиональном баскетболе это называется «горячей рукой» (hot hand).

В середине 1980-х годов за феномен «горячей руки» взялись учёные. Не какие-то там абстрактные британские учёные, а группа под руководством того самого Амоса Тверски[1]. Феномен «горячей руки» отлично иллюстрировал главную идею исследований Тверски, ведь он выглядит как типичная когнитивная иллюзия. Более того, этот эффект легко объяснить с психологической точки зрения: если после трёх удачных бросков подряд ты забиваешь и четвёртый, и пятый, это запоминается куда лучше, чем обратные случаи, и в результате создаётся впечатление, что таких случаев было непропорционально много. Поэтому группа Тверски попыталась исследовать феномен «горячей руки» статистически.

Исследователи провели простой эксперимент: набрали баскетболистов и попросили их бросать мяч в кольцо. Позицию для броска выбирали так, чтобы каж-

[1] *Амос Тверски* (Amos Tversky, 1937–1996) — израильский психолог, который вместе с Даниэлем Канеманом фактически создал современное понимание когнитивных искажений и иррациональности человеческой психологии [295, 300, 608]. Кроме чисто экономических эффектов, они также описали основные «когнитивные иллюзии» (cognitive biases) и показали, что люди склонны искать закономерности там, где их нет, делать слишком далекоидущие выводы на основе недостаточной информации, придавать разное значение рискам и возможным выгодам и так далее и тому подобное; всё это было Канеманом и Тверски систематизировано и тщательно изучено. К сожалению, в 1996 году Тверски умер от рака, иначе наверняка бы разделил с Канеманом Нобелевскую премию 2002 года.

дый баскетболист в среднем совершал около половины успешных бросков с этой позиции (точное равенство здесь непринципиально, такой выбор нужен только для того, чтобы броски сообщали побольше информации). Затем они взяли выборки получившихся результатов и посмотрели на то, как часто, например, четвёртый бросок после трёх успешных подряд также попадал в кольцо. Оказалось, что нет никакой статистической разницы: четвёртый в серии бросок залетает примерно так же успешно, как и первый, с точностью плюс-минус 2 %. Это было опубликовано в статье Гиловича, Валлоне и Тверски [221], которая вошла в число основополагающих работ о когнитивных искажениях.

Конечно, этот эксперимент можно покритиковать с интуитивно-баскетбольной точки зрения. Например, вполне возможно, что в настоящем матче эмоции и концентрация баскетболиста оказывают влияние на успешность его бросков куда сильнее, чем в бесстрастном эксперименте, проводящемся в пустом зале без всякого спортивного азарта. Тогда в эксперименте мы никакой «горячей руки» не увидим, но в реальности эффект появится.

Может случиться и обратное, ведь баскетболист на паркете не просто бросает мяч в кольцо с заданной точки. В настоящем матче (статистику баскетбольных матчей Гилович с соавторами тоже, разумеется, привлекали) эффект «горячей руки» или просто ощущение хорошо складывающегося матча могут побудить игрока, у которого сейчас всё залетает, бросать больше. Но больше — значит, в среднем из менее выгодных позиций! И тогда в эксперименте «горячая рука» проявится сильнее, чем в реальной баскетбольной статистике.

Так что, когда в середине 1980-х об этой работе написали журналисты, представители баскетбольного мира восприняли этот результат более чем прохладно. Так, знаменитый тренер Ред Ауэрбах отреагировал в своём стиле: «Да кто такой этот парень? Ну провёл он исследование, мне-то какое дело?»

Но давайте попробуем провести эксперимент ещё более холодный и непредвзятый: давайте повторим результаты Гиловича с соавторами без всяких баскетболистов, бросками обычной монетки, которая будет настолько честна, насколько нам позволит порождение псевдослучайных чисел в *Python*...

**2.5.2. Разоблачение разоблачения эффекта «горячей руки»*

Код для повторения экспериментов будет очень простой:

```
streak, N, num_experiments = 3, 20, 10000
results, results_streak = [], []
for _ in range(num_experiments):
    x = np.random.randint(2, size=N)
    after_streak = np.array([ i+streak for i in range(N) \
               if np.all(x[i:i+streak] == 1) and (i+streak < N) ])
    if len(after_streak) > 0:
        results_streak.append(np.sum(x[after_streak]) / len(after_streak))
    results.append(np.sum(x) / float(N))
```

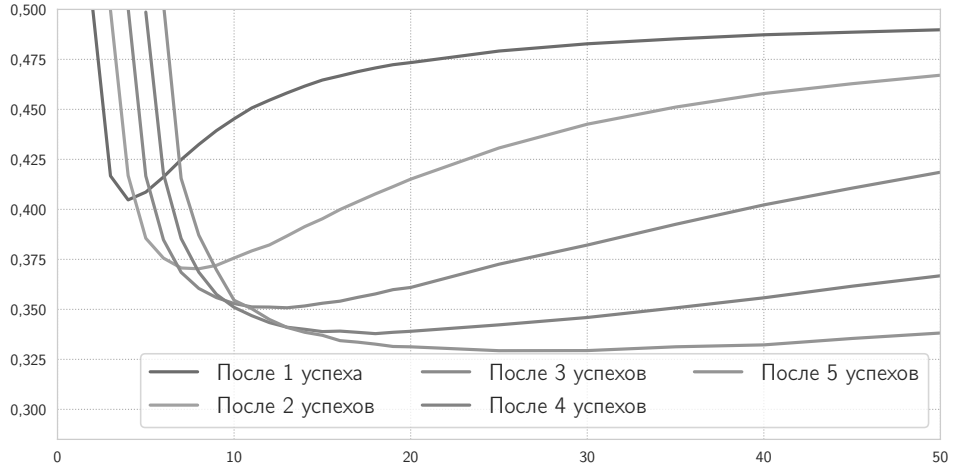


Рис. 2.19. Развенчание развенчания «горячей руки»: средняя оценка вероятности успеха после «удачной полосы» (streak) от 1 до 5 успехов подряд как функция от числа экспериментов N

Здесь написано, что мы 10 000 раз проводим такой эксперимент: бросаем честную монетку $N = 20$ раз, в массив `results` записываем полученное среднее число решек, а в массив `results_streak` записываем среднее число решек, выпавших после трёх подряд решек (не записываем ничего, если таких последовательностей нет, чтобы не делить на ноль). Массив `results` нужен скорее для самопроверки, после этого эксперимента в нём получится что-то вроде

```
> np.mean(results)
0.5002949999999999
```

А вот в массиве `results_streak` получится что-то вроде

```
> np.mean(results_streak)
0.3600021010477037
```

Что здесь происходит? Мы где-то ошиблись? На самом деле нет. Можно провести эти эксперименты для разных длин этих самых «удачных полос» (ох, не соответствует русскому духу слово streak, с ходу даже и не переведёшь) и нарисовать графики. То, что у меня получилось, показано на рис. 2.19, где вероятность успеха после «удачной полосы» изображена как функция от числа экспериментов N. Чем больше мы возьмём длину удачной полосы, тем сильнее «недовес» по сравнению с 50 %, хотя при увеличении N эффект постепенно слабеет и на бесконечности пропадает. Этот эффект был замечен совсем недавно, в работах исследователей из Университета Аликанте, которые появились как препринты около 2015–2016 годов, а опубликованы были и вовсе только в 2018 году [395, 396].

Таблица 2.2. Полный анализ последовательностей из трёх бросаний монетки; h — орёл, t — решка

Результат	Исходов после орла	Из них орлов	Доля
hhh	2	2	1
hht	2	1	$\frac{1}{2}$
hth	1	0	0
thh	1	1	1
htt	1	0	0
tht	1	0	0
tth	0	0	—
ttt	0	0	—

Стоп, но как же это так получается? Неужели это значит, что на рулетке действительно надо ставить на красное после трёх чёрных подряд и все состояния, проигранные верившими в это азартными людьми, на самом деле были когнитивной аберрацией?..

На самом деле нет, на рулетке, разумеется, так не выиграть. Этот результат связан с тем, что мы выбираем *конечную* последовательность опытов и фиксируем её заранее, а потом уже считаем удачные полосы в данных. Результат становится более интуитивно понятным, если полностью разобрать случай $N = 3$. Здесь всего восемь возможных результатов, и в этом самом маленьком случае мы будем рассматривать результаты после полосы из одного успеха. Давайте для каждой возможной последовательности запишем, в какой доле этих случаев после орла идёт ещё один орёл; таблица 2.2 делает это максимально подробно (там, как обычно, h обозначает орла, а t — решку).

И вот что получается: два исхода из восьми не содержат интересующих нас событий, их мы считать не будем; а если усреднить оценки из последнего столбца в остальных шести случаях, то получится вовсе не $\frac{1}{2}$, а $\frac{5}{12}$. При этом, конечно, из восьми раз, когда мы бросали монетку после выпадения орла, в полном наборе событий выпало ровно четыре орла... но два из них пришлись на одну и ту же последовательность, и поэтому они вдвоём дают только одно слагаемое в усреднённую оценку. А вот обратная ситуация — когда в одной последовательности сразу два раза выпала решка после орла — в последовательности длины 3 встречаться не может по определению. В этом и суть эффекта: если мы видим решку после орла, это автоматически означает, что полосы из нескольких орлов подряд в этой последовательности данных встречаются в среднем реже, и поэтому средняя доля удачных полос внутри последовательности заданной длины будет меньше, чем можно было бы ожидать из честности монетки.

В общем виде это уже не очень простой результат, но давайте попробуем выработать интуицию и проследить за основными шагами доказательства. Рассмо-

трим последовательность испытаний Бернулли $X_1, \ldots, X_n$ с вероятностью успеха π. Пусть D — это наши данные (последовательность из n орлов или решек), $\hat{p}_k(D)$ — оценка вероятности успеха после k успехов подряд, полученная из данных, а $I_k(D)$ — множество бросков сразу после k успехов подряд, то есть

$$I_k(D) = \left\{ t \middle| \prod_{i=t-k}^{t-1} X_i = 1 \right\}.$$

Нам нужно оценить условное ожидание $\mathbb{E}\left[\hat{p}_k(D) | I_k(D) \neq \emptyset\right]$. Доказательство состоит из трёх шагов (это только набросок доказательства!). На первом шаге нужно формально доказать, что

$$\mathbb{E}\left[\hat{p}_k(D) | I_k(D) \neq \emptyset\right] = p\left(X_\tau = 1 | I_k(D) \neq \emptyset\right),$$

где τ — это номер броска монетки, выбранный равномерно из $I_k(D)$. Это вполне интуитивный и логичный результат — мы утверждаем, что ожидаемая оценка вероятности успеха совпадает с собственно вероятностью успеха, — так что доказательство я опущу.

Второй шаг проходит по нашей любимой теореме Байеса:

$$\begin{aligned} p\left(X_\tau = 1 | I_k(D) \neq \emptyset\right) &= p\left(X_t = 1 | \tau = t, I_k(D) \neq \emptyset\right) = \\ &= p\left(X_t = 1 \middle| \tau = t, \prod_{i=t-k}^{t-1} X_i = 1\right) \propto \\ \propto p\left(\tau = t \middle| X_t = 1, \prod_{i=t-k}^{t-1} X_i = 1\right) &\, p\left(X_t = 1 \middle| \prod_{i=t-k}^{t-1} X_i = 1\right) = \\ &= p\left(\tau = t \middle| X_t = 1, \prod_{i=t-k}^{t-1} X_i = 1\right) \times \pi. \end{aligned}$$

Иначе говоря, теорема Байеса говорит нам, что вероятность получить решку на броске τ, $X_\tau = 1$, пропорциональна произведению вероятности выбрать момент $\tau = t$ равномерно из $I_k(D)$ на вероятность получить решку на шаге t вообще, которая уже действительно равна π без всяких парадоксов и искажений.

Осталось сравнить две вероятности:

$$p\left(\tau = t \middle| X_t = 1, \prod_{i=t-k}^{t-1} X_i = 1\right) \quad \text{и} \quad p\left(\tau = t \middle| X_t = 0, \prod_{i=t-k}^{t-1} X_i = 1\right).$$

И вот здесь-то и кроется секрет этого на первый взгляд парадоксального результата. Дело в том, что если $X_t = 0$, то следующие k бросков точно не попадут

в $I_k(D)$, а значит, в среднем $I_k(D)$ будет меньше размером. Но если множество, из которого мы равномерно выбираем t, стало меньше, значит, вероятность выбрать конкретный t из него будет выше! Иначе говоря,

$$p\left(\tau = t \middle| X_t = 1, \prod_{i=t-k}^{t-1} X_i = 1\right) < p\left(\tau = t \middle| X_t = 0, \prod_{i=t-k}^{t-1} X_i = 1\right),$$

а это значит, что

$$p\left(X_\tau = 1 | I_k(D) \neq \emptyset\right) < \pi.$$

Насколько именно меньше — вопрос более сложный, и в него мы углубляться не будем.

После этих рассуждений становится ясно, что этот эффект не может работать вперёд во времени — вся его суть в том, что мы добавляем «перекос» в выборке из уже проведённых экспериментов, когда рассматриваем последовательности успехов. Также понятно, что на бесконечности эффект пропадает: все кривые на рис. 2.19 асимптотически стремятся к $\frac{1}{2}$. Но в реальности, разумеется, выборки будут конечными, а если речь идёт о настоящих экспериментах с живыми баскетболистами, то и весьма небольшими.

*2.5.3. Выводы

Надеюсь, что было интересно; но к чему был этот пример, в чём мораль этой басни? Во-первых, только что сделанный вывод прекрасно иллюстрирует стандартную ситуацию в машинном обучении. «На бесконечности», когда данных неограниченно много, весь байесовский вывод обычно становится не очень-то и нужен: в произведении $p\left(\boldsymbol{\theta}\right) p\left(D|\boldsymbol{\theta}\right)$ первый сомножитель не меняется, а второй становится всё важнее с ростом D, и в конечном счёте всё сходится[1] к максимизации правдоподобия.

Но ведь наша жизнь — это малые выборки! Может показаться, что мы живём в эпоху огромных датасетов, которые с каждым годом становятся всё больше, и уже давно должны существенно продвинуться к тому, чтобы байесовский вывод стал не нужен. Но на самом деле всё наоборот: как только появляется датасет побольше, люди тут же начинают пытаться обучать на нём более сложные модели. Да, конечно, современные датасеты были бы более чем достаточны для обучения линейной регрессии без всякого байесовского вывода — вот только обучаем мы на них не линейную регрессию, а нейронные сети с миллиардами параметров. Машинное обучение всегда работает на «фронтире», где данных в обрез хватает, чтобы обучить модель, и связанные с этим проблемы, например тот же оверфиттинг, всегда с нами.

[1] Редактор книги — пользуясь случаем, передаю ей большое спасибо! — предложила здесь исправить «сходится» на «сводится»; действительно, оба слова прекрасно подходят.

Во-вторых, я хотел проиллюстрировать, насколько непростая штука теория вероятностей. В этом примере оказалось, что ответ и вообще вся суть происходящего существенно зависят от, казалось бы, незначительных деталей в постановке задачи. Эти детали сами по себе не очень сложны — когда мы разобрались, в какой момент происходит усреднение вероятностей, стало понятно, в чём дело. Но проблема в том, что эти вопросы не так-то просто заметить и поставить: как видите, в данном случае не справились великие учёные, лауреаты Нобелевской премии, в чьём блестящем владении статистическими методами сложно сомневаться. Это типичная ситуация для теории вероятностей: вспомните хотя бы пример с двумя детьми из раздела 2.1, где тоже ответ радикально менялся в зависимости от деталей постановки задачи и разницу легко можно было пропустить; да и ошибки прокурора и адвоката из раздела 2.3 тоже это иллюстрируют.

Как мы уже выяснили в разделе 2.3, у нас, людей, обычно не очень хорошо с вероятностной интуицией. Мы плохо «чувствуем» теорему Байеса и не можем быстро проводить рассуждения с её использованием так же легко, как мгновенно сравниваем размеры объектов или распознаём лица — хотя и математически, и с точки зрения практического искусственного интеллекта распознавание человеческих лиц является несравненно более сложной задачей. Каждый раз приходится хотя бы приблизительно, но считать, и я хочу порекомендовать вам не забывать это делать не только при решении математических задач, но и в жизни.

2.6. Кризис воспроизводимости

Первая любовь, школьные года,
В лужах голубых стекляшки льда.
Не повторяется, не повторяется,
Не повторяется такое никогда!

ВИА «Самоцветы»

2.6.1. Кризис воспроизводимости в психологии и социальных науках

В начале 2010-х годов социальные науки и психология пережили кризис, который затронул самые интересные результаты этих наук и стал причиной серьёзной трансформации, можно сказать, самих методов научного познания, принятых в этих дисциплинах. История этого *кризиса воспроизводимости* (replication crisis), во-первых, во многом относится к теме нашей книги и поучительна сама по себе, а во-вторых, находит неожиданные параллели с тем, что сейчас происходит в машинном обучении. Поэтому в этом разделе мы обсудим кризис воспроизводимости и его основные причины, особенно связанные с неправильным применением статистических методов.

Начнём с примеров. Одной из первых ласточек кризиса в психологии стали знаменитые исследования Джона Барга и его коллег о прайминге, то есть эффекте, при котором некий стимул продолжает оказывать влияние и на реакцию на последующие, никак с ним напрямую не связанные стимулы. Возможно, вы слышали о самом знаменитом результате Барга: люди, которым зачитывали слова, связанные со старостью (дело было в США, так что это были слова вроде «Флорида» или «бинго»), потом уходили из лаборатории медленнее, чем люди, которым читали случайные слова.

Этот результат был опубликован в 1996 году [44], быстро стал широко известен, попал в учебники и цитировался уже более 6000 раз. Однако попытки его воспроизвести, предпринятые в начале 2010-х годов, закончились неудачей [53]. Вслед за этим исследователи попробовали воспроизвести и другие результаты о прайминге, и оказалось, что они тоже не воспроизводятся; например, учёные не смогли воспроизвести известный эксперимент о прайминге социальной дистанции (social distance priming), когда участников просили нарисовать две точки на графике (по заданным экспериментаторами координатам), а затем якобы оказывалось, что их ответы на вопрос о том, насколько они близки с другими членами своей семьи, зависят от того, близки или далеки были эти точки [455].

Но проблема оказалась куда глубже, чем один недобросовестный исследователь. Джон Барг вовсе не мошенник, и в своих исследованиях он не делал никаких осознанных подтасовок, а честно использовал общепринятые методы научного поиска — но оказалось, что сами эти методы не так уж надёжны. В последние 10–15 лет очень многие исследования, особенно в психологии и социальных науках, не смогли пройти испытания воспроизведением.

Иногда это вовсе не вина исследователей, которых могли просто неверно прочитать. Так, например, вы наверняка слышали о так называемом зефирном тесте (marshmallow test), в котором четырёх-пятилетним детям предлагается либо съесть зефирку прямо сейчас, либо получить две зефирки, подождав несколько минут. Результат здесь состоит в том, что дети, которые оказались способны выдержать испытание отложенной награды, потом показывали гораздо лучшие результаты в учёбе и жизни. Точнее говоря, результат зефирного теста хорошо предсказывал результаты стандартизированных тестов SAT, которые дети сдавали более чем через десять лет после теста, будучи уже абитуриентами! Статья об этом была опубликована в 1990 году Юичи Шодой с соавторами [537], и с тех пор зефирный тест вошёл в популярную культуру и стал руководством к действию для ряда образовательных инициатив.

Однако в 2018 году оказалось, что зефирный тест всё-таки не работает [637]. Новое, более масштабное исследование показало, что корреляция между способностями ребёнка отложить вознаграждение зефиркой и дальнейшими образовательными успехами почти полностью пропадает, если учесть социальный статус его семьи, её финансовое благополучие и аналогичные факторы, которые, скорее всего, действительно во многом определяют результаты абитуриентов. Иначе говоря, более скучные факторы являются действительно важными, а зефирный тест с ними просто оказался тоже скоррелирован, что вполне логично: ребёнку из богатой семьи и сами эти зефирки не так уж нужны, и меньше привычки к тому, что хорошее быстро пропадает, а люди могут обманывать.

В данном случае, впрочем, исследователи совсем ни в чём не виноваты, ведь в исходной статье именно так и было сказано! Шода с соавторами пишут так: «Нужно подчеркнуть необходимость осторожной интерпретации результатов... особенно интерпретации ассоциаций между отложенным вознаграждением у дошкольников и оценками SAT... Даже самые большие корреляции объясняют всего лишь около 25 % дисперсии... По крайней мере, дальнейшая репликация на других популяциях, когортах и в других условиях тестирования представляется необходимым следующим шагом» [537]. Разумеется, этому мудрому совету никто и не подумал последовать до тех пор, пока кризис воспроизводимости не разгорелся в полную силу.

Иногда исследования действительно оказывались или сфальсифицированными, или весьма натянутыми даже без учёта деталей статистического анализа. Так произошло, например, со знаменитым Стэнфордским тюремным экспериментом (Stanford prison experiment), который в 1971 году провёл социальный

психолог Филипп Зимбардо [239]. Он построил в Стэнфорде аналог настоящей тюрьмы, для чего даже привлёк в качестве консультанта много лет проведшего в тюрьме Карло Прескотта. Затем группа набранных по объявлению студентов была разбита на «заключённых» и «охранников», и эксперимент начался.

В результате многие «охранники» стали издеваться над «заключёнными»: физическое насилие было запрещено, но «заключённых» заставляли спать на голом полу, отжиматься, проводили длительные унизительные досмотры и перекличку, не пускали в туалет и так далее; Зимбардо писал, что около трети «охранников» проявили садистские наклонности. Основным выводом из Стэнфордского тюремного эксперимента Зимбардо считал то, что садизм и унижения в тюрьмах обусловлены не «естественными наклонностями» людей, а социальными ролями, в которых они оказываются. И хотя сам Зимбардо позже признал эксперимент неэтичным, на его результатах он построил всю свою карьеру, а сам Стэнфордский тюремный эксперимент вошёл в учебники и стал классическим примером — вы тоже наверняка о нём слышали.

Однако вы, возможно, не слышали о том, что, хотя от эксперимента сохранились даже подлинные видеозаписи, на самом деле Зимбардо многого недоговаривал. Во-первых, он утверждал, что никак специально не инструктировал «заключённых», кроме объяснения правил «тюрьмы», но про «охранников» обычно скромно умалчивал. На самом же деле (здесь я опираюсь на работу Тибо ле Тексье [335], который изучил все публикации и видеозаписи Стэнфордского тюремного эксперимента):

- «охранники» знали заранее, к каким результатам должно прийти исследование: ещё на ориентации перед началом эксперимента Зимбардо рассказал им о том, что это исследование «условий, которые приводят к жестокости, потери идентичности, чувству анонимности... и мы хотим воссоздать такую психологическую среду в нашей тюрьме»; уже одного этого достаточно, чтобы считать результаты крайне ненадёжными;
- более того, Зимбардо инструктировал их и напрямую: «Мы можем создать скуку... чувство фрустрации... до некоторой степени страх... У [«заключённых»] совсем не будет приватности, они будут под постоянным наблюдением... мы лишим их индивидуальности разными способами»; ежедневная программа, которой должны были придерживаться «охранники», включала пересчёт «заключённых» посреди ночи, а на ориентации Карло Прескотт объяснял им из собственного опыта, как «правильно» унижать «заключённых» во время подсчёта;
- студент Зимбардо Дэвид Джаффе, который играл роль главного надзирателя, объяснял «охранникам», что они должны быть «жёсткими», мотивируя это тем, что главная цель эксперимента — сделать происходящее похожим на тюрьму, чтобы привлечь внимание прессы и, возможно, способствовать тюремной реформе (да-да, прямо так и говорил во время эксперимента);

- а сам Зимбардо старался создать у «охранников» впечатление, что они помогают исследователям изучать «заключённых», а не сами являются подопытными; в целом, конечно, обманывать подопытных в психологических экспериментах не запрещено, но здесь это приводило к тому, что «охранники» осознанно старались играть свою роль «хорошо», то есть максимально жёстко. Один из охранников позже признавался: «Я думал, что для исследования будет лучше, если я представлю то, что мне казалось реалистичными отношениями между охранником и заключённым... В течение всего эксперимента я был актёром»; аналогично высказывался и другой «охранник», один из самых жестоких.

А самый яркий результат эксперимента — нервный срыв одного из «заключённых», о котором много раз позже вспоминал Зимбардо, — оказался полным фейком. Участник эксперимента признался, что хотел выйти из игры, но Зимбардо отказался выпускать «заключённого» и сказал, что единственные допустимые причины выйти на свободу — это медицинские или психические показания. Тогда участник решил, что самым простым решением будет сымитировать нервный срыв; позже он в нескольких независимых интервью подтвердил, что сам себя накрутил и разыграл сцену. В целом Стэнфордский тюремный эксперимент был больше похож на специфическую форму иммерсивного театра, чем на научный эксперимент, соответствующий своим заявленным целям.

Другие примеры детально разбирать не буду, не в этом наша цель сейчас, но чтение источников производит потрясающий эффект: когда начинаешь разбираться в деталях практически любого знаменитого эксперимента, на который тысячи раз ссылались в самой разнообразной литературе, часто оказывается, что эксперимент или подделан, или неверно представлен, или сделан на малой выборке и не воспроизводится, или просто неверно интерпретируется при цитировании. Например:

- в 1954 году Музафер Шериф поставил знаменитый эксперимент в парке Robbers Cave, в котором одиннадцатилетние мальчики, разделённые на две группы, устроили эскалацию конфликта и дошли до настоящей ненависти и дегуманизации по отношению к другой группе[1]; однако Шериф забыл рассказать, как исследователи искусственно подталкивали мальчиков к конфликту, а главное — не любил рассказывать, что его предыдущий точно такой же эксперимент, в 1953 году, полностью провалился и две группы таких же мальчиков оставались друзьями в течение всего эксперимента, несмотря на все усилия психологов;

[1] Можно встретить мнение, что под впечатлением от этого эксперимента Уильям Голдинг написал своего «Повелителя мух»; на самом деле это не могло быть так, потому что Голдинг начал писать свою книгу ещё в 1951 году и в 1952 году уже был готов первый вариант. Обратное влияние, впрочем, тоже вряд ли возможно, потому что путь «Повелителя мух» к публикации был сложным и книга вышла только в сентябре 1954 года. Очевидно, идея поиздеваться над подростками в то время просто носилась в воздухе...

- в 1951 году Соломон Аш опубликовал результаты экспериментов о конформизме в группах (Asch conformity experiments) [31], в которых людям задавали вопрос с очевидным ответом (какая из трёх линий имеет ту же длину, что и данная); более 99 % людей отвечали правильно в одиночку, но в группе, где другие участники (актёры) уверенно отвечали неверно, в 36 % случаев люди давали «социально ожидаемый» ответ, а не очевидно верный, а 5 % участников всегда следовали за мнением группы; к Ашу здесь претензий нет, и его результаты были неоднократно воспроизведены, хотя уровень конформизма со временем снижался [72]; но даже в учебниках по психологии они обычно описываются некорректно [200, 229], преувеличивая конформизм, а не независимость суждений, на которую указывал сам Аш, и не упоминая проведённые Ашем интервью, в которых люди подтверждали, что знали правильный ответ и подчинялись мнению группы осознанно, не испытывая никаких иллюзий;
- читатель ждёт уж пункта об экспериментах Стэнли Милгрэма (Milgram), в которых испытуемые по указанию экспериментатора давали сильнейший электрический шок человеку, неправильно отвечающему на вопросы (шок был поддельный, но актёр играл хорошо) [394]; но нет, эти эксперименты были воспроизведены много раз (хотя обычно с очень маленькими группами подопытных), и всё подтверждается [85]. Единственная серьёзная критика результатов Милгрэма состоит в том, что в таком эксперименте очень трудно сказать, действительно ли участники могут поверить, что научный эксперимент может привести к прямому и серьёзному вреду для живого человека[1]: так, психолог Джина Перри утверждает, что только половина испытуемых действительно верила в реальность происходящего, и из этой половины две трети отказались продолжать эксперимент [461].

Разумеется, всю психологию никто отрицать не собирается. Например, мы начали раздел с того, что эксперименты Барга о прайминге не подтвердились, но это не значит, что эффекта прайминга не существует вовсе. Работы Канемана и Тверски, с которых началась эта наука [293, 295, 677], в целом никто под сомнение не ставит (хотя отдельные результаты могут уточняться), и базовый эффект действительно присутствует: если назвать человеку большое число, его последующие оценки неизвестной величины станут выше. Этот эффект имеет вполне реальное применение в торговле: если назвать высокую цену и предложить скидку,

[1] Я не специалист, но лично мне, честно говоря, это соображение кажется очень существенным. Например, как можно воспроизвести эксперимент Милгрэма в наше время, когда его потенциальные участники с высокой вероятностью уже слышали о подобных экспериментах? Одна из современных «репликаций» эксперимента была показана во французском документальном фильме *Le Jeu de la Mort* («Игра смерти»), вышедшем в 2010 году. В нём эксперимент Милгрэма был оформлен как съёмки пилотного эпизода реалити-шоу — но как могут участники такого «эксперимента» всерьёз поверить, что отметки напряжения настоящие и съёмочная группа «реалити-шоу» предлагает им наносить серьёзный вред здоровью людей прямо в эфире?..

это будет смотреться куда приятнее, чем если бы ту же самую цену установили изначально. Не воспроизводятся здесь многие из вторичных экспериментов, которые пытались распространить эффект прайминга на более «мутные» области. Так, если переходить от чисел к словам (как и делал Барг), то возникает масса новых проблем: слова в естественных языках имеют широкие семантические поля значений, которые не вполне совпадают ни в разных языках, ни у разных людей, а также имеют тенденцию немного меняться со временем. И всё это на воспроизводимость экспериментов о прайминге влияет не очень хорошо [280, 485, 486].

Но в целом, если вы где-то услышали о работе, в которой некий яркий результат получен при помощи экспериментов над небольшими группами людей, действительно имеет смысл уточнить, воспроизводится ли этот результат, а также заглянуть в исходную статью и проверить, что там, собственно, утверждается. Казалось бы, наука — это лучшее, что есть у человечества в эпистемологическом плане. Если даже научным результатам нельзя доверять, то ситуация становится совсем неприятной; почему же всё-таки приходится давать такие советы?

2.6.2. Почему результаты не воспроизводятся: p-значения и p-хакинг

Выше мы рассматривали яркие примеры, обсуждали эксперименты, о которых многие слышали. Но кризис воспроизводимости не ограничивается несколькими важными примерами, он пронизывает всю психологию и другие экспериментальные социальные науки. Соответственно, достаточно печальные выводы получаются и в отношении того, можно ли ожидать воспроизводимость от опубликованных результатов. Сами современные учёные, согласно опросам, считают, что воспроизведётся около половины всех опубликованных в приличных местах исследований: от 58 % в экономике до 42 % в психологии и педагогике [28].

Были и попытки проверить не опросами, а делом. Так, в 2015 году в журнале *Science* было опубликовано большое мета-исследование, то есть исследование, объединяющее и интерпретирующее выводы многих других работ, проведённое группой из почти 300 авторов [449]. Оно было посвящено воспроизводимости в психологии и изучало, насколько хорошо воспроизводятся результаты из ведущих психологических журналов — были выбраны *Psychological Science*, *Journal of Personality and Social Psychology* и *Journal of Experimental Psychology: Learning, Memory, and Cognition* — и как воспроизводимость связана с тем, что можно найти в исходной статье или подсчитать из исходных данных, если они доступны. Исследователи попытались воспроизвести 158 экспериментальных исследований из этих журналов; в итоге получилось завершить эксперименты для 100 из них. Оказалось, что среди воспроизведённых исследований статистически значимые результаты получились в 36 % случаев, а в исходных статьях таких было 97 % (а иначе не было бы и статей, в этом и смысл эффекта); а если механически объединить выборки в исходных и воспроизведённых исследованиях, статистически значимый эффект сохраняется в 68 % из них (что, конечно, не случайно находится примерно на полпути между 36 и 97 %).

Если обнаруженный учёными эффект не воспроизводится, значит, либо учёные где-то ошиблись, намеренно или случайно, либо сделали неверные выводы из корректно полученных данных. Давайте попробуем разобраться, как из верно описанных экспериментов могут получиться ложные выводы [195, 449].

Главная причина — формальные критерии научного открытия, ведущие к частым ложным срабатываниям (false positives); здесь мы наконец-то переходим к математическому содержанию этого параграфа. Предположим, что вы измерили некоторые экспериментальные данные: скажем, вы дали сотне больных вашу новую таблетку и 60 из них выздоровели, а в контрольной, получавшей плацебо группе выздоровели только 40 из 100. Является ли это значимым результатом? Интуитивно кажется, что скорее да. Но если бы больных в каждой группе было всего по десять, и выздоровели бы шесть в тестовой группе против четырёх в контрольной, это вряд ли дало бы нам какую-то уверенность в том, что таблетка чем-то помогает. Как это формализовать?

Традиционным критерием значимости открытий в науке много лет были так называемые *p-значения* (p-values). P-значение определяется как вероятность того, что имеющиеся экспериментальные данные будут получены при условии *нулевой гипотезы*, то есть в вероятностных предположениях, соответствующих отсутствию эффекта. Формально говоря, для некоторой статистики (случайной величины) T, полученного её экспериментального значения t и нулевой гипотезы, которую традиционно обозначают через H_0:

- $p(t) = p\left(T \geq t | H_0\right) = 1 - F_T(t)$ при проверке правосторонней альтернативы (one-sided right-tail test-statistic distribution), то есть если статистика T для интересующего нас эффекта должна быть велика;
- $p(t) = p\left(T \leq t | H_0\right) = F_T(t)$ при проверке левосторонней альтернативы (one-sided left-tail test-statistic distribution), то есть если статистика T для нужного эффекта должна быть мала;
- $p(t) = 2\min\left(p\left(T \geq t | H_0\right), p\left(T \leq t | H_0\right)\right) = 2\min\left(1 - F_T(t), F_T(t)\right)$ при проверке двусторонней альтернативы (two-sided test-statistic distribution), то есть если статистика T просто может принимать крайние значения, неважно, в какую сторону.

Чем меньше p-значение, тем «более невероятным» является случайное совпадение с таким значением исследуемой статистики; если p-значение становится совсем маленьким, то это можно расценивать как подтверждение того, что нулевая гипотеза неверна.

Например, если вы проверяете, честная ли монетка вам попалась, и из десяти бросков шесть выпали орлом, то:

- нулевая гипотеза H_0 состоит в том, что орёл и решка равновероятны, $\theta = \frac{1}{2}$;
- статистика T — это число орлов из десяти бросков и $t = 6$;
- p-значение для правосторонней (или двусторонней, так как здесь очевидно, в какую сторону отклонение) альтернативы составляет

$$p(t) = p\left(T \geq t | H_0\right) = p\left(T \geq 6 \middle| \theta = \frac{1}{2}\right) = \frac{1}{2^{10}}\left(\binom{10}{6} + \ldots + \binom{10}{10}\right) \approx 0{,}377.$$

Полученное нами p-значение в 0,377 — это очень большая величина, она нулевую гипотезу совершенно не отвергает. Но если бы мы получили аналогичные результаты на большем числе экспериментов, ситуация бы изменилась:

- для $t = 60$ орлов из $N = 100$ бросков

$$p(t) = \frac{1}{2^{100}}\left(\binom{100}{60} + \ldots + \binom{100}{100}\right) \approx 0{,}028;$$

- для $t = 600$ орлов из $N = 1000$ бросков

$$p(t) = \frac{1}{2^{1000}}\left(\binom{1000}{600} + \ldots + \binom{1000}{1000}\right) \approx 1{,}364 \times 10^{-10}.$$

В терминах классической науки это значит, что для шести орлов из десяти мы не сможем отвергнуть гипотезу о честной монетке и останемся без эффекта, а для 600 орлов из 1000 будем практически наверняка убеждены, что эффект есть и монетка действительно нечестна. А 60 орлов из 100 — это ситуация «на грани»: есть два традиционных пороговых p-значения, $p = 0{,}05$ и $p = 0{,}01$, и у нас получилась величина как раз между ними. Замечу здесь, впрочем, что эти пороговые значения являются традиционными для социальных наук, биологии и медицины, а, например, в физике элементарных частиц принят куда более жёсткий стандарт: «открытие» там требует выхода за пять дисперсий (five sigma criterion), что соответствует p-значению в 3×10^{-7} [360, 361]!

Исчезающе малое p-значение — это действительно хороший аргумент против нулевой гипотезы. Он используется уже более 300 лет, с работы Джона Арбетнота[1], в которой он увидел, что в течение 82 лет, с 1629 по 1710 год, в Лондоне рождалось больше мальчиков, чем девочек, и заключил, что шанс на случайное совпадение составляет лишь $\frac{1}{2^{82}}$, а значит, это работа божественного провидения [26].

Уже эта первая в истории работа, использующая фактически современный тест на значимость, вызвала много споров и сомнений [538]. Во-первых, Арбетнот использовал только бинарный факт того, что мальчиков больше; Лаплас позже провёл расчёты с численными данными, используя биномиальное распределение, и снова, конечно же, получил исчезающе малое p-значение. Николай Бернулли возражал против вывода Арбетнота о божественном провидении в рождении детей, утверждая, что появление мальчика всё ещё может быть броском мо-

[1] *Джон Арбетнот* (John Arbuthnot, 1667–1735) — шотландский врач, сатирик и в целом человек эпохи Просвещения. Был членом «клуба Мартина Писаки» (Scriblerus Club) вместе с Джонатаном Свифтом и Александром Поупом, и написанные Арбетнотом «Мемуары Мартина Писаки» во многом вдохновили Лоренса Стерна на «Тристрама Шенди»; кстати, Арбетнот был ещё и автором Джона Булля, образа типичного англичанина.

нетки, просто нечестной, с вероятностью $\pi > \frac{1}{2}$. С этой критикой мы сегодня согласимся, тем более что попутно Николай Бернулли вывел очень точное приближение для вероятности, с которой такая монетка за N бросаний покажет от $\pi N - l$ до $\pi N + l$ орлов.

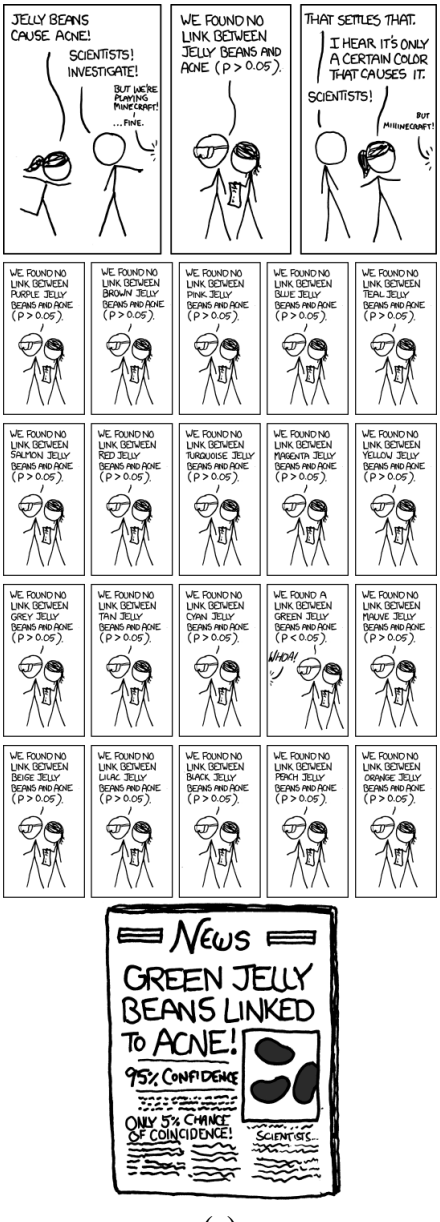
Но любопытно, что в высшей степени релевантная критика работы Арбетнота появилась сразу же, на заседании Королевского общества 26 апреля 1711 года. Услышав доклад о работе, казначей общества Абрахам Хилл отметил, что, по слухам, у короля Марокко уже 900 сыновей и ни одной дочери, однако же сокрытое в этом факте провидение отнюдь не божественно: просто король Марокко, обладатель обширного гарема, приказал убивать всех девочек сразу же при рождении!

Это возражение актуально до сих пор: аргументы о p-значении часто (но, как вы увидите ниже, не всегда!) действительно довольно убедительно отвергают нулевую гипотезу, но это далеко не всегда значит, что нужно принять гипотезу, выдвинутую учёными. Вполне возможно, что выдвинутая гипотеза не является строгим отрицанием нулевой и есть объяснение, которое отвергает обе гипотезы. С работой Арбетнота это произошло дважды: Николай Бернулли указал, что случайность может быть не равновероятной, а Абрахам Хилл предложил альтернативное объяснение, которое увязывает равновероятное случайное рождение мальчиков и девочек и наблюдаемое превосходство числа мальчиков.

В современную науку p-значения формально ввёл Карл Пирсон в своём непараметрическом критерии согласия (критерий ξ^2), а популяризировал их использование Рональд Фишер (о нём мы подробно поговорим в главе 5). В своей книге *The Design of Experiments* [183] Фишер использовал знаменитый пример о «леди, дегустирующей чай» (lady tasting tea): его знакомая Мюриэль Бристоль, любившая чай с молоком, утверждала, что может отличить, в какой последовательности напитки смешивали: наливали ли чай в молоко или молоко в чай. Фишер предложил ей определить последовательность вслепую для восьми чашек чая с молоком и определил, что вероятность полного успеха (а у Мюриэль получилось верно определить все чашки) составляет лишь $1/\binom{8}{4} = \frac{1}{70} \approx 0{,}014$. Там же Фишер предложил использовать $p = 0{,}05$ как стандартный уровень значимости и показал, что правильно определить три чашки из шести было бы недостаточно для того, чтобы превзойти этот уровень.

С лёгкой руки Фишера p-значения стали стандартом науки. Многие исследователи в самых разных науках использовали p-значения как критерий успеха... и часто делали это некорректно. Самая простая ошибка — продолжать получать, анализировать и подправлять данные до тех пор, пока не будет достигнуто нужное p-значение.

Сам смысл утверждения $p < 0{,}05$ состоит в том, что, даже если данные получены в соответствии с нулевой гипотезой, в среднем в одном эксперименте из двадцати встретятся результаты с $p < 0{,}05$; это иллюстрирует знаменитый xkcd-комикс на рис. 2.20, *а* [415]. Именно такой «анализ» данных называют p-хакингом (p-hacking) в узком смысле, и он, увы, часто встречается в жизни.



(а) (б)

Рис. 2.20. Иллюстрации к приёмам p-хакинга: (а) XKCD-комикс *Significant* [415]; (б) диаграмма p-хакинга из [410]

Но этим ошибки, связанные с p-значениями, не исчерпываются. Во-первых, здесь опять возможна ошибка прокурора: p-значение показывает вероятность получения данных при условии нулевой гипотезы, а не наоборот! Иначе говоря, p-значение имеет дело с правдоподобием, а не апостериорной вероятностью. Во-вторых, p-значение подсчитывается для конкретной модели случайного порождения данных и ничего не говорит о других моделях (именно это замечание сделал Николай Бернулли о работе Джона Арбетнота). В-третьих, p-значение ничего не говорит о размере наблюдаемого эффекта (effect size). В-четвёртых... в общем, подводных камней очень много; см., например, официальное объяснение Американской статистической ассоциации (ASA) [636].

2.6.3. Сад расходящихся тропок и парапсихология

Смежными с p-хакингом в широком смысле являются и другие манипуляции с данными. В исследованиях часто удаляют выбросы в данных, или добавляют новые переменные, или применяют математические преобразования... приёмов много, и в каждом из них нет ничего плохого или некорректного, но в сумме они дают много «степеней свободы», нередко позволяющих сделать из данных чуть ли не любой желаемый вывод. На рис. 2.20, *б* я перевёл «диаграмму p-хакинга» из работы Харви Мотульски [410] — все эти методы действительно используются в реальной жизни.

Предположим, что я хочу проверить, действительно ли русскоязычные люди, работающие в области искусственного интеллекта и изучившие мою предыдущую книгу «Глубокое обучение» [685], в среднем зарабатывают больше, чем люди, которые её не читали; это была бы неплохая реклама для следующей книги! Казалось бы, нужно просто подсчитать среднее по одной группе, среднее по другой, а затем сравнить их, получив p-значение. Но тут открываются большие возможности.

- Что значит «изучившие»? В какую группу отнести людей, дочитавших книгу до середины? А бросивших после первой вводной главы?
- Что значит «работающие в области AI»? Будем ли добавлять в данные, например, тестировщиков в AI-компаниях, которые и зарабатывают в среднем меньше, и реже имеют околоматематическое образование?
- Как будем сравнивать средние? Предположим, что распределения в выборках нормальные, или используем непараметрический тест? Какой именно?
- Как будем очищать данные от выбросов? Русский язык — родной для одного из основателей *Google* Сергея Брина, и, будем честны, вряд ли он читал книгу «Глубокое обучение»; но если включить его в выборку, то данные потеряют всякий смысл, ведь его доход затмит все остальные.
- Кстати, а не значит ли пример Сергея Брина, что нас интересуют не средние, а *медианы* в выборках? Точно-точно, для сравнения медиан есть ведь и совершенно другие статистические тесты, подождите, сейчас посчитаем...

В данном случае вполне очевидно, к какому выводу я хотел бы прийти в таком исследовании, и манипуляции с данными почти наверняка позволят мне сделать «нужный» вывод с любой наперёд заданной значимостью в виде p-значения.

В такие ловушки легко попасть и без всякого злого умысла. Эндрю Гельман и Эрик Локен придумали термин «сад расходящихся тропок» [212] для ситуации, когда действия исследователя существенно зависят от исследуемых данных. Дело в том, что первые публикации самого же Гельмана [211] и других исследователей [543] об этом эффекте в основном говорили о проблеме *множественных сравнений* (multiple comparisons): если достаточно долго перебирать варианты различных статистических тестов, переходя от проверки одних эффектов к другим и деля данные на группы по разным признакам и критериям, почти всегда можно будет найти что-нибудь статистически значимое. Согласитесь, это звучит как обвинение в крайней научной недобросовестности, и учёные, разумеется, обиделись: они сказали, что ничего не перебирали, а выбрали для данного датасета один конкретный метод статистического анализа и именно он и дал нужный результат.

«Сад расходящихся тропок» (куда менее обвинительно звучащее название) — это эффект, когда даже один тест всё равно может оказаться «нечестным». Дело в том, что этот тест выбирается уже после того, как данные получены, и если бы данные были другими, то и тест мог бы стать другим; проще говоря, выбранный метод статистического анализа зависит от данных, что тоже может приводить к тому же эффекту множественных сравнений.

Этот эффект можно было бы полностью исключить, если потребовать *пререгистрацию* (preregistration) статистического анализа: принимать только работы, авторы которых заранее, до сбора данных объявили, какие именно гипотезы будут проверять. Технически это несложно реализовать, но здесь легко выплеснуть вместе с водой и ребёнка: как пишет сам Гельман, «для хороших учёных абсолютно разумно уточнять свои гипотезы после полученных данных... и если желаемый результат не появляется как основной эффект, вполне естественно будет посмотреть на взаимодействия между переменными» [212]. Иначе говоря, нет ничего плохого или странного в том, что ваша гипотеза не подтвердилась, зато подтвердилась какая-то другая, похожая на неё или чуть более слабая — это тоже научный результат.

На мой взгляд, проблемы здесь начинаются в том случае, когда вы формулируете свои гипотезы на основе *тех же самых* данных, которые потом используете для их подтверждения. В терминах машинного обучения это звучит абсолютно естественно: как мы обсуждали в подразделе 1.4.2, не надо использовать тренировочный набор в качестве валидационного, ведь это прямой путь к оверфиттингу!

В работе [212] Гельман и Локен вспоминают и один из самых интересных примеров на эту тему; см. также подробное его обсуждение в моём любимом блоге, у Скотта Александра [22]. Этот пример — серьёзные научные статьи, посвящённые... парапсихологии!

Самые яркие примеры здесь связаны с феноменом *экстрасенсорного предвидения* (precognition). Дэрил Бем, абсолютно «настоящий» социальный психолог и почётный профессор в Корнеллском университете, заинтересовался парапсихологическими эффектами и провёл полноценное исследование экстрасенсорного предвидения по всем законам экспериментальной психологии [56]. В его работе приводятся результаты девяти экспериментов с более чем 1000 участников, и все они подтверждают пусть не очень сильный, но, несомненно, статистически значимый эффект предвидения.

Например, в основном эксперименте участникам показывали изображения двух одинаковых занавесов и говорили, что компьютер уже случайным образом выбрал, за каким из этих занавесов окажется содержательное изображение, а за каким — пустая стена. Задачей участника было определить, где изображение, так что для участников это выглядело как тест на ясновидение. Но на самом деле эксперимент был ещё более чистым: только после того, как участник нажимал на кнопку, компьютер действительно выбирал следующее случайное (ну хорошо, псевдослучайное — использованные псевдослучайные генераторы в статье тоже подробно обсуждаются) число; согласитесь, такая постановка эксперимента действительно выглядит как максимально честный способ проверить возможность предвидения ещё не наступившего события. Результаты эксперимента были весьма интересны: оказалось, что для нейтральных изображений никакого эффекта предвидения не обнаружилось, правильный ответ получался с вероятностью 49,8 %... но если помещать за занавес эротические картинки, вызывающие соответствующие эмоциональные и физиологические реакции, то участники (обоих полов) верно распознавали их с не слишком большой, но статистически значимо отличающейся от половины вероятностью 53,1 %!

Результаты этой работы были опубликованы во вполне приличном журнале, *Journal of personality and social psychology*. Но на этом Бем не остановился. Через несколько лет он выпустил мета-анализ 90 разных экспериментов о предвидении, выполненный согласно самым строгим научным стандартам [178]. К этому мета-анализу очень трудно придраться... но он тоже подтверждает существование экстрасенсорного предвидения! Как правило, в экспериментах получается небольшой размер эффекта (effect size), примерно как 53,1 % по сравнению с нулевой гипотезой в 50 % в эксперименте выше, но этот размер часто оказывается статистически значимым с очень маленькими p-значениями, а совокупно мета-анализ нашёл эффекты, p-значения и факторы Байеса (Bayes factors), по всем параметрам соответствующие научным стандартам решающих доказательств, по которым обычно оцениваются «нормальные» публикации.

О чём это говорит? Лично я, прочитав работы Бема, не начал верить в экстрасенсорное предвидение. Скорее всего, с экспериментами всё-таки что-то не так. Но как же после этого верить всем остальным научным результатам? Почему результаты Бема не нужно принимать всерьёз, а подавляющее большинство статей по медицине, в которых статистика сделана гораздо хуже и результаты

гораздо менее убедительны, — нужно? Скотт Александр называет парапсихологию «контрольной группой науки», и похоже, что очень мало какие результаты экспериментальных наук выдерживают сравнение с таким «плацебо»...

*2.6.4. Модель Иоаннидиса

Кризис воспроизводимости начался в медицине, и одной из главных действующих фигур кризиса стал профессор медицины из Стэнфорда Джон Иоаннидис. В 2005 году он опубликовал одну из первых работ, привлекающих внимание к воспроизводимости научных исследований: эссе под скромным названием «Почему большая часть научных результатов неверна» *(Why Most Published Research Findings Are False)* [279]. В этом разделе «со звёздочкой» мы обсудим его основные идеи.

Иоаннидис рассматривает очень простую модель: предположим, что мы проводим исследование в той или иной области, цель которого — установить некоторую зависимость или отношение. Тогда:

- обозначим априорную вероятность того, что такая зависимость есть, через $\frac{R}{R+1}$, то есть R можно понимать как шансы, отношение «числа существующих зависимостей» к «числу несуществующих»;
- вероятность её найти в исследовании, если она действительно существует, обозначим через $1-\beta$, то есть β здесь означает долю ложноотрицательных результатов;
- вероятность найти зависимость, если её нет, то есть вероятность ложного срабатывания, — через α.

Если в этой области всего исследуются c различных зависимостей, то ожидаемые числа различных исходов будут такими, как показаны в табл. 2.3. Апостериорная вероятность того, что выдавшее положительный результат исследование действительно нашло некоторую существующую зависимость, в этой модели составляет

$$\frac{cR(1-\beta)}{R+1} \Big/ \frac{c(R+\alpha-\beta R)}{R+1} = \frac{R(1-\beta)}{R(1-\beta)+\alpha}, \tag{2.44}$$

Иначе говоря, такое исследование будет скорее верным, чем неверным, если $R(1-\beta) > \alpha$, а в идеале мы бы, конечно, хотели, чтобы $R(1-\beta)$ было гораздо больше, чем α. В реальности, скорее всего, $\alpha = 0{,}05$, и в этой модели пока что ничего страшного не происходит.

Но давайте сделаем модель более реалистичной. Во-первых, в настоящих исследованиях, скорее всего, присутствует систематическая ошибка (bias), которая делает положительные результаты более вероятными. В неё входят все те приёмы p-хакинга, которые мы обсуждали выше, а также построение выборки для эксперимента и так далее и тому подобное. Мы промоделируем всё это одним числовым параметром: пусть доля u исследований, которые не должны были выдавать положительный результат, всё-таки его выдают из-за таких ошибок.

Таблица 2.3. Ожидаемые числа различных исходов в базовой модели Иоаннидиса [279]

Результат исследования	Истинный результат: Зависимость есть	Истинный результат: Зависимости нет	Итого
положительный	$\frac{cR(1-\beta)}{R+1}$	$\frac{c\alpha}{R+1}$	$\frac{c(R+\alpha-\beta R)}{R+1}$
отрицательный	$\frac{cR\beta}{R+1}$	$\frac{c(1-\alpha)}{R+1}$	$\frac{c(1-\alpha+\beta R)}{R+1}$
Итого	$\frac{cR}{R+1}$	$\frac{c}{R+1}$	c

Таблица 2.4. Ожидаемые числа различных исходов в модели Иоаннидиса с систематической ошибкой [279]

Результат исследования	Истинный результат: Зависимость есть	Истинный результат: Зависимости нет	Итого
положительный	$\frac{cR(1-\beta)+uc\beta R}{R+1}$	$\frac{c\alpha+uc(1-\alpha)}{R+1}$	$\frac{c(R+\alpha-\beta R+u-u\alpha+u\beta R)}{R+1}$
отрицательный	$\frac{(1-u)cR\beta}{R+1}$	$\frac{(1-u)c(1-\alpha)}{R+1}$	$\frac{c(1-u)(1-\alpha+\beta R)}{R+1}$
Итого	$\frac{cR}{R+1}$	$\frac{c}{R+1}$	c

Таблица 2.5. Ожидаемые числа различных исходов в модели Иоаннидиса с n независимыми исследованиями [279]

Результат исследования	Истинный результат: Зависимость есть	Истинный результат: Зависимости нет	Итого
положительный	$\frac{cR(1-\beta^n)}{R+1}$	$\frac{c(1-(1-\alpha)^n)}{R+1}$	$\frac{c(R+1-(1-\alpha)^n-\beta^n R)}{R+1}$
отрицательный	$\frac{cR\beta^n}{R+1}$	$\frac{c(1-\alpha)^n}{R+1}$	$\frac{c((1-\alpha)^n+\beta^n R)}{R+1}$
Итого	$\frac{cR}{R+1}$	$\frac{c}{R+1}$	c

Тогда таблица вероятностей превратится в показанную в табл. 2.4; здесь апостериорная вероятность истинности положительного результата, полученного в рамках одного исследования, составляет

$$\frac{cR\left(1-\beta\right)+uc\beta R}{R+1}\Big/\frac{c\left(R+\alpha-\beta R+u-u\alpha+u\beta R\right)}{R+1}=$$
$$=\frac{\left(1-\beta\right)R+u\beta R}{\left(1-\beta\right)R+u\beta R+\left(1-\alpha\right)u+\alpha}. \quad (2.45)$$

С ростом u эта вероятность уменьшается[1], причём довольно быстро.

Более того, есть и другой вариант, как можно получить больше ложных результатов. Предположим, что для каждого из c возможных результатов проводится не одно исследование, а n: например, n команд исследователей проверяют одни и те же гипотезы. Это вполне реалистично; более того, если действительно

[1] Формально она уменьшается, только если $1-\beta \geq \alpha$, но исследованиями *настолько* плохими, что в них вероятность заметить истинный эффект меньше 0,05, пожалуй, всё-таки можно пренебречь.

находится перспективное направление, эти команды, скорее всего, будут соперничать друг с другом в своём стремлении получить положительный результат — но не отрицательный, ведь опубликовать отсутствие результата не получится. Таким образом, теперь *любой* положительный результат из n будет засчитываться, и вероятности (без учёта систематической ошибки) приобретут показанный в табл. 2.5 вид. Здесь опять же апостериорная вероятность

$$\frac{cR\left(1-\beta^n\right)}{R+1} \Big/ \frac{c\left(R+1-(1-\alpha)^n-\beta^n R\right)}{R+1} = \frac{R\left(1-\beta^n\right)}{R+1-(1-\alpha)^n-\beta^n R} \qquad (2.46)$$

быстро убывает с ростом n.

Конкретные численные соотношения зависят от значения априорной вероятности $\frac{R}{R+1}$, но в любом случае получается, что во многих областях науки у исследований очень мало шансов оказаться верными. Иоаннидис приводит такой пример из своей области: предположим, что учёные проверяют, какие из 100 000 генетических полиморфизмов могут быть ассоциированы с шизофренией. Тогда:

- из общих соображений можно ожидать, что должны найтись около десяти полиморфизмов, ассоциированных с шизофренией с шансами около 1,3, то есть $R \approx 10^{-4}$, и $\frac{R}{R+1} \approx 10^{-4}$;
- предположим, что наше исследование находит такую ассоциацию с вероятностью $1-\beta = 0{,}6$ для $\alpha = 0{,}05$; это реалистичные числа, так как ассоциации в этой науке достаточно слабые и найти их с уверенностью нелегко.

Тогда для одного исследования, проведённого без систематических ошибок, апостериорная вероятность того, что положительный результат — это действительно нужный полиморфизм, составляет по формуле (2.44) $\frac{10^{-4}\cdot 0{,}6}{10^{-4}\cdot 0{,}6+0{,}05}$, то есть около $1{,}2\cdot 10^{-3}$. Более того:

- если исследователи вносят некоторую систематическую ошибку, пусть небольшую, $u = 0{,}1$, то по формуле (2.45) апостериорная вероятность составит лишь около $4{,}4\cdot 10^{-4}$, то есть положительный результат исследования уменьшит размер «стога сена», в котором мы ищем иголку, менее чем в пять раз;
- а если этой задачей занимаются $n = 10$ независимых команд исследователей, то формула (2.46) даст нам апостериорную вероятность того, что результат подтвердится, всего лишь около $1{,}5\cdot 10^{-4}$; с таким ожидаемым результатом не стоило и деньги тратить.

В целом Иоаннидис заключает, что во многих областях науки большинство исследований неверны, то есть находят несуществующие зависимости в данных, а в поисковых исследованиях вроде описанного выше апостериорные вероятности и вовсе могут быть исчезающе малы. Более того, для этого даже не обязательно предполагать какую бы то ни было недобросовестность: $u = 0{,}1$ может получиться из общепринятых подходов к обработке данных, да и с $u = 0$, чтобы

апостериорная вероятность сильно уменьшилась, достаточно лишь «натравить» на задачу несколько конкурирующих команд исследователей.

2.6.5. Воспроизводимость в машинном обучении

Итак, пессимизм Иоаннидиса был вполне оправдан: воспроизводимость результатов во многих естественных и социальных науках оставляет желать много лучшего. Какие уроки мы можем извлечь из этого кризиса?

С одной стороны, во многом эта книга посвящена как раз тому, как избежать ошибок, ведущих к невоспроизводимым экспериментам и чересчур далекоидущим выводам; например, в главе 4 мы будем говорить о том, насколько большой может получиться дисперсия предсказательного распределения, и о том, насколько действительно можно доверять выводам даже такой простой модели, как линейная регрессия. Поэтому не исключаю, что добравшийся до этого места читатель полагает, что главная мысль всего этого раздела состоит в том, что статистикой нужно пользоваться правильно, и наша наука, искусственный интеллект, как раз может в этом помочь, ведь по сути машинное обучение — это развитие идей математической статистики.

Но на деле всё наоборот! Оказывается, можно найти немало интересных параллелей между кризисом воспроизводимости в экспериментальных науках, например в социальной психологии, и тем, что в наши дни происходит на переднем крае машинного обучения! Так, работа Джессики Халлман и других учёных из Северо-Западного университета, Принстона и Коламбии [647] обсуждает сходство между основными причинами и симптомами кризиса воспроизводимости в социальной психологии и неприятными тенденциями в машинном обучении (см. также [54]). Параллелей они находят довольно много:

- в психологии данных зачастую слишком мало; в машинном обучении датасеты гораздо больше, но появляется другой эффект: поскольку собирать новые датасеты чрезвычайно дорого и сложно, большая часть исследований в машинном обучении направлена на дальнейшие улучшения моделей на уже известных датасетах, что приводит к тому, что их недостатки (недостаточное покрытие и вообще любые несоответствия теоретическому — то есть никому не известному — распределению данных) становятся предметом для оверфиттинга;
- в психологии применяются слишком простые модели, предположения которых зачастую не выполняются (а для обучения сложных моделей недостаточно данных); а в машинном обучении частенько не выполняется самое главное предположение: о том, что все данные порождаются из одного и того же распределения; распределение данных в стандартном датасете почти никогда не будет точно соответствовать распределению данных в дальнейшем реальном применении модели, но про этот вид систематической ошибки обычно забывают;

- социальные науки долгое время страдали и продолжают страдать от p-хакинга, но в машинном обучении есть практически точный его аналог: подбор *гиперпараметров* (алгоритмов оптимизации, скорости обучения, параметров априорных распределений и так далее); точно так же, как в социальных науках и психологии можно сгруппировать данные по-разному 20 раз и найти вариант с $p < 0{,}05$, в машинном обучении часто можно подобрать гиперпараметры новой (предлагаемой) модели так, что она на заданном датасете сработает лучше другой фиксированной модели; стандартные датасеты опять же побуждают искать модели, дающие небольшие, инкрементальные улучшения именно на них, а обобщающая способность полученных моделей обычно мало кого интересует;
- перекос, вызываемый требованиями к публикации (publication bias), в обеих областях тоже вполне аналогичен: в психологии, скорее всего, опубликован будет только положительный результат, причём лучше, если он окажется неожиданным, контринтуитивным, а в машинном обучении для публикации непременно нужно улучшить результаты других моделей, причём желательно на стандартном датасете;
- когда результат добирается до публикации, проявляются похожие неприятные эффекты: в психологии авторы пишут о «присутствующих» или «отсутствующих» эффектах на основании p-значений, а затем строят теории и делают далекоидущие выводы на этой достаточно зыбкой основе; а в машинном обучении результаты часто показываются как точечные оценки, без оценок неопределённости (например, доверительных интервалов);
- и, наконец, собственно *воспроизводимость* тоже оказывается не так легко проверить: в психологии для этого нужно провести эксперимент с достаточно большой выборкой живых людей, что обычно дорого и сложно, а в машинном обучении в последние годы модели стали настолько большими, а эксперименты настолько дорогими, что воспроизвести обучение лучших современных моделей могут только весьма богатые корпорации.

Что же делать? Простого ответа на этот вопрос нет, но столетия научного поиска уже не раз приводили учёных к таким затруднениям. Так что делать можно и нужно ровно то же, что в итоге решили делать учёные в других науках:

- публиковать данные и код, при помощи которого можно воспроизвести результаты экспериментов; по соображениям приватности и коммерческой тайны публикацию данных трудно сделать обязательным требованием, но статьи на лучших конференциях по машинному обучению уже сейчас считается хорошим тоном сопровождать ссылками на репозитории с реализациями представленных моделей;
- полностью публиковать все гиперпараметры и все детали процесса обучения моделей — это помогает понять, в частности, сколько «степеней свободы» было у исследователей;

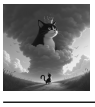
- использовать пререгистрацию исследований (preregistration), то есть объявлять заранее и публично, какие именно эксперименты будут проводиться и как именно будут анализироваться данные; это позволяет существенно сократить возможности для p-хакинга;
- вместо p-хакинга использовать практики «p-усердия» (p-diligence), которые используют схожие методы во благо: например, хорошей практикой будет повторить анализ данных с другими критериями исключения выбросов или другим вероятностным предположением в нулевой гипотезе не для того, чтобы найти «тот самый» вариант, а для того, чтобы проверить, что эффект сохраняется при всех разумных вариантах обработки данных;
- наконец, не преуменьшать важность нулевых результатов, а также репликаций положительных результатов; встретить в одном из ведущих журналов статью, в которой показан отрицательный результат или просто реплицированы предыдущие исследования, пока ещё сложно, но уже появляются места для таких публикаций, например *Journal of Comments and Replications in Economics* в экономике или *Royal Society Open Science* в психологии.

Это сложный путь, который и социальные науки ещё не прошли до конца, и машинное обучение, конечно, не пройдёт за год или два. Но стараться поддерживать высокие стандарты воспроизводимости нужно, и инициативы в этом направлении уже появляются и становятся стандартом индустрии [232, 276].

А в заключение хочется привести цитату из той самой статьи в *Science*, которая с переменным успехом пыталась воспроизвести многие психологические работы: «После всей этой сложной работы по воспроизводству целой выборки опубликованных психологических исследований, сколько найденных ими эффектов мы доказали и можем объявить истинными? Ноль. А сколько эффектов мы опровергли и можем объявить ложными? Ноль. Является ли это ограничением дизайна нашего проекта? Нет. Это реальность научной работы, даже если в ежедневной практике её не всегда замечают. Люди стремятся к определённости, а наука редко может её предоставить. И как бы нам ни хотелось обратного, одно исследование никогда не даст однозначного ответа за или против рассматриваемого эффекта и его объяснения. Исходные исследования, которые мы проанализировали, предоставили некоторые предварительные данные; воспроизведённые нами эксперименты дали дополнительные, подтверждающие данные. В некоторых случаях наши воспроизведения увеличили уверенность в надёжности исходных результатов; в других случаях они дали понять, что требуются дополнительные исследования, чтобы подтвердить эти находки. Научный прогресс — это кумулятивный процесс сокращения неопределённости, который может увенчаться успехом только в том случае, если сама наука останется главным скептиком для своих объяснительных утверждений» [449].

Глава 3
Теория вероятностей и оптимизация

TL;DR

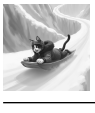


Открывает главу раздел 3.1, посвящённый важным дискретным распределениям: равномерному, биномиальному, геометрическому, распределению Пуассона, отрицательному биномиальному и гипергеометрическому. В качестве раздела «со звёздочкой» обсудим подробно закон Ципфа.

А раздел 3.2 говорит, напротив, о непрерывных распределениях: равномерном, нормальном, логнормальном, распределении Стьюдента, гамма-распределении и экспоненциальном распределении.

Раздел 3.3 посвящён неожиданным глубинам, которые обнаруживаются при обучении параметров равномерного распределения. Мы поговорим о том, как оценить общее число деталей по выборке серийных номеров, а также обсудим кольцевание птиц и рыб.

Раздел 3.4 продолжает знакомство с важнейшими понятиями теории вероятностей. Мы обсудим энтропию, перекрёстную энтропию, KL-дивергенцию и матрицу информации Фишера. А закончим мы разделом «со звёздочкой», в котором мы попробуем построить распределения полного незнания, которые бы, насколько это возможно, не делали никаких предположений.



В разделе 3.5 мы обсудим машинное обучение с точки зрения задач оптимизации: почему оптимизация в машинном обучении всегда сложная, как работает градиентный спуск и стохастический градиентный спуск, какие у них основные свойства и проблемы.

3.1. Важные дискретные распределения

Стрепсиад: Кто ж навстречу друг другу их гонит, скажи? Ну не Зевс ли, колеблющий тучи?
Сократ: Да нимало, не Зевс. Это — Вихрь.
Стрепсиад: Ну и ну! Значит, Вихрь! И не знал я, деревня,
Что в отставке уж Зевс и на месте его нынче Вихрь управляет Вселенной.

Аристофан. Облака

...Некоторым принцип относительных частот не нравится уже тем, что он статистический. Ибо статистика противна человеческому разуму; она болезненно определена для группы, но не слишком определена для индивида. Конечно, первичный закон, не знающий внутри себя колебаний, был бы приятнее. Если бы природа посоветовалась с человеком в этом вопросе, мы все предложили бы первичные законы уже из того инстинктивного страха, что иначе Вихрь может начать управлять Вселенной. Мы бы забыли при этом, однако, что даже в статистическом законе Вихрю трудно управлять безраздельно, а также что во многих случаях Вихрь, пожалуй, действительно стоит во главе.

George Kingley Zipf. Selected Studies of the Principle of Relative Frequency in Language [664]

3.1.1. Равномерное, биномиальное и геометрическое распределения

Надеюсь, предыдущие главы уже убедили вас в том, что машинное обучение — это раздел прикладной теории вероятностей. И в нём, конечно же, возникает много стандартных, хорошо изученных распределений, свойства которых желательно понимать. В этом разделе мы поговорим о тех распределениях, которые чаще всего появляются именно в машинном обучении; со многими из них нам придётся столкнуться в этой книге. Для каждого распределения мы найдём его ожидание и дисперсию, а для некоторых обсудим и другие важные свойства.

Начнём с *дискретных* распределений, то есть распределений с конечным или счётным числом исходов. Если число исходов N конечно (будем идентифицировать их с числами от 1 до N), то никаких сюрпризов не ожидается: это просто распределение броска игральной кости с тем или иным числом граней; оно

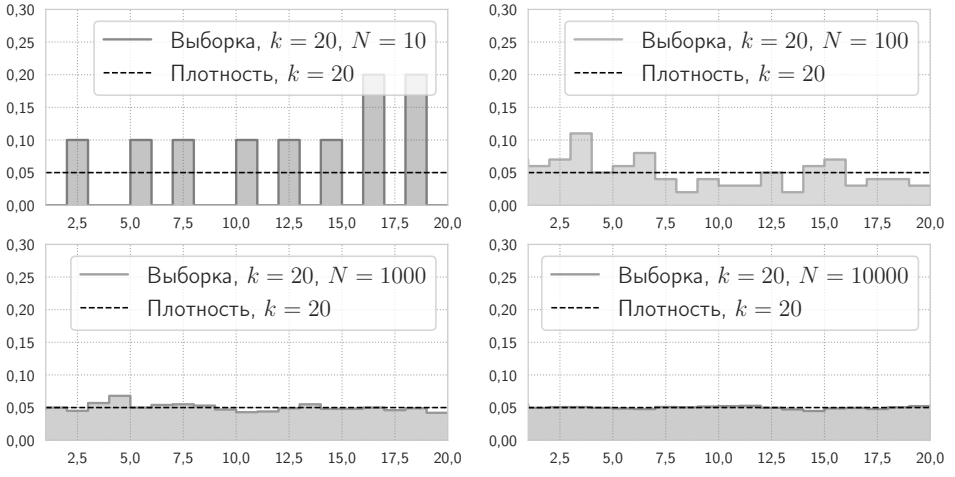


Рис. 3.1. Примеры выборок из равномерного распределения

полностью задаётся вероятностями каждого исхода $p_1, \ldots, p_N$. Здесь особенно важны два частных случая: если все вероятности равны, $p_i = \frac{1}{N}$ для всех i, то это *дискретное равномерное распределение* $\mathrm{Unif}(N)$. Математическое ожидание $\mathbb{E}[X]$ для $X \sim \mathrm{Unif}(N)$ равно

$$\mathbb{E}[X] = \sum_{i=1}^{N} i \frac{1}{N} = \frac{N+1}{2},$$

а дисперсия получается в виде

$$\mathrm{Var}[X] = \mathbb{E}\left[X^2\right] - \mathbb{E}[X]^2 = \sum_{i=1}^{N} i^2 \frac{1}{N} - \frac{(N+1)^2}{4} = \\ = \frac{1}{N} \frac{N(N+1)(2N+1)}{6} - \frac{(N+1)^2}{4} = \frac{N^2-1}{12}.$$

Если $N = 2$, то получается *распределение Бернулли*, то есть распределение исходов броска монетки; у него один параметр, вероятность выпадения орла p (а решки соответственно $1-p$), среднее p и дисперсия $p(1-p)$ (оставлю проверку этого вам в качестве упражнения).

На рис. 3.1 показаны несколько примеров выборки из равномерного распределения с 20 исходами; каждая из них порождена буквально в одну строчку с использованием `np.random` для разных `N`:

```
sample = np.random.randint(k, size=N) + 1
```

Здесь мы, конечно, могли бы углубиться в рассмотрение различных методов семплирования, но не будем; методы семплирования, применяющиеся в машинном обучении, выходят за пределы этой книги, а порождение выборок из стандартных распределений — это совсем уж не наша тема [511]. Для целей этой книги достаточно предполагать, что по любому рассматриваемому нами распределению можно достаточно эффективно взять выборку значений.

На рис. 3.1 и последующих рисунках выборки нормализованы, то есть на графиках показаны гистограммы счётчиков тех или иных взятых по этому распределению значений, делённые на общее число семплов; четыре графика показывают, как такие гистограммы постепенно сходятся к плотности самого распределения с ростом числа семплов в выборке.

Многие другие полезные дискретные распределения связаны с испытаниями Бернулли, то есть с бросками монетки. Так, если сложить результаты нескольких бросков одной и той же монетки, то получится *биномиальное распределение* $\mathrm{Binom}(n, p)$; у него два параметра, число испытаний n и вероятность выпадения орла в одном испытании p (хотя биномиальным оно называется не потому, что у него два параметра, а потому, что у монетки два исхода).

За n бросков может выпасть от 0 до n орлов, и вероятность выпадения k орлов равна

$$\mathrm{Binom}(k \mid n, p) = \binom{n}{k} p^k (1-p)^{n-k}.$$

Эта формула получается потому, что каждая конкретная последовательность орлов и решек с k орлами и $n-k$ решками выпадает с вероятностью $p^k (1-p)^{n-k}$, а всего их как раз $\binom{n}{k}$.

Математическое ожидание биномиального распределения, $\mathbb{E}[X]$ для случайной величины $X \sim \mathrm{Binom}(n, p)$, равно np, потому что ожидание линейно, а распределение $\mathrm{Binom}(n,p)$ — это сумма n испытаний Бернулли с ожиданием p у каждого.

С другой стороны, тот же самый результат можно было с тем же успехом получить и через бином Ньютона; для этого можно заметить, что $k\binom{n}{k} = n\binom{n-1}{k-1}$, что позволит вынести n за скобку, и внутри останется сумма по всему распределению $\mathrm{Binom}(n-1, p)$, которая равна единице:

$$\mathbb{E}[X] = \sum_{k=0}^{n} k \binom{n}{k} p^k (1-p)^{n-k} = \sum_{k=1}^{n} n \binom{n-1}{k-1} p^k (1-p)^{n-k} =$$
$$= np \sum_{j=0}^{n-1} \binom{n-1}{j} p^j (1-p)^{n-1-j} = n(p+1-p)^{n-1} = np.$$

Чтобы найти дисперсию биномиального распределения, нужно сначала подсчитать второй момент $\mathbb{E}\left[X^2\right]$:

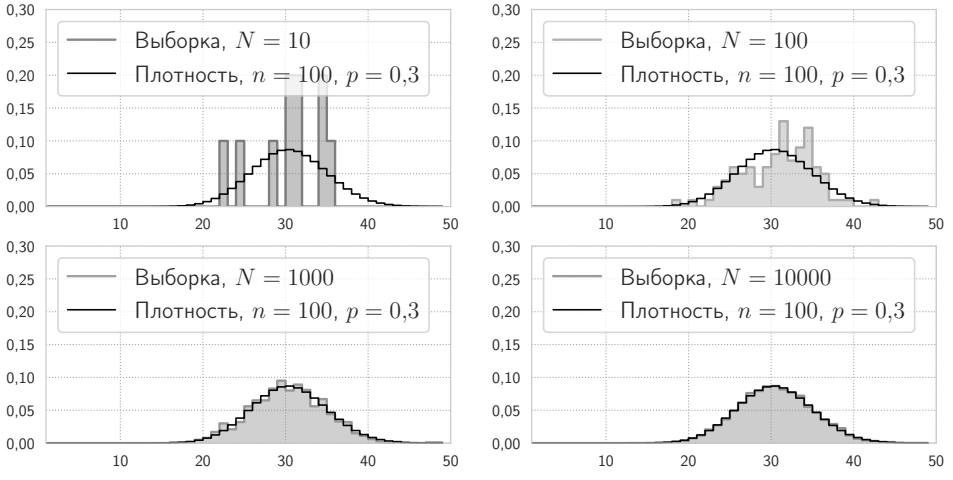


Рис. 3.2. Примеры выборок из биномиального распределения с $n = 100$ и $p = 0.3$

$$\begin{aligned}\mathbb{E}\left[X^2\right] &= \sum_{k=0}^{n} k^2 p_k = \sum_{k=0}^{n} k\,\mathrm{Binom}(k \mid n,p) + \sum_{k=0}^{n} (k-1)\,k\,\mathrm{Binom}(k \mid n,p) \\ &= np + \sum_{k=0}^{n} (k-1)\,k \frac{n!}{(n-k)!k!} p^k (1-p)^{n-k} \\ &= np + n(n-1)p^2 \sum_{k=2}^{n} \frac{(n-2)!}{((n-2)-(k-2))!(k-2)!} p^{k-2} (1-p)^{(n-2)-(k-2)} \\ &= np + n(n-1)p^2 = n^2p^2 + np(1-p);\end{aligned}$$

здесь мы выделили из второго слагаемого сумму по всей плотности распределения $\mathrm{Binom}(n-2,p)$, которая равна единице. Теперь дисперсию можно найти как

$$\mathrm{Var}\,[X] = \mathbb{E}\left[X^2\right] - \mathbb{E}\,[X]^2 = n^2p^2 + np(1-p) - (np)^2 = np(1-p). \qquad (3.1)$$

На рис. 3.2 показаны несколько примеров выборок из биномиального распределения для $n = 100$ и $p = 0{,}3$; аналогично рис. 3.1 мы видим, как нормализованная гистограмма выборки постепенно сходится к теоретической плотности распределения. Код для получения таких выборок тоже тривиален:

```
sample = np.random.binomial(n, p, size=N)
```

Геометрическое распределение — это ещё более простая вещь: вероятность получить ровно k решек подряд на монетке с вероятностью орла p, и легко видеть, что эта вероятность составляет $\mathrm{Geom}\,(k|p) = (1-p)^k\,p$.

Среднее геометрического распределения можно подсчитать изящным способом, в котором внезапно появляется дифференцирование:

$$\begin{aligned}\mathbb{E}[k|p] &= \sum_{k=0}^{\infty} k\,(1-p)^k\,p = p(1-p)\sum_{k=0}^{\infty} k\,(1-p)^{k-1} \\ &= p(1-p)\sum_{k=0}^{\infty}\frac{\mathrm{d}\left(-(1-p)^k\right)}{\mathrm{d}p} = -p(1-p)\frac{\mathrm{d}}{\mathrm{d}p}\left(\sum_{k=0}^{\infty}(1-p)^k\right) \\ &= -p(1-p)\frac{\mathrm{d}\frac{1}{p}}{\mathrm{d}p} = \frac{1-p}{p}.\end{aligned}$$

Оставлю вам в качестве упражнения доказать аналогичным образом, что

$$\mathrm{Var}[k|p] = \frac{1-p}{p^2}.$$

3.1.2. Биномиальное распределение Пуассона

А что, если сложить результаты бросков *разных* монеток? Получится *биномиальное распределение Пуассона*[1] (binomial Poisson distribution); его не надо путать с обычным распределением Пуассона, о котором речь пойдёт ниже. Вероятность получить k положительных исходов (орлов) из N монеток с вероятностями $p_1,\ldots,p_N$ записывается как

$$p(k|p_1,\ldots,p_N) = \sum_{A}\prod_{i\in A} p_i \prod_{i\in\{1,\ldots,N\}\setminus A}(1-p_i),$$

где суммировать приходится по всем возможным подмножествам $A\subseteq\{1,\ldots,N\}$ из k элементов. Иначе говоря, это сумма из $\binom{N}{k}$ слагаемых, и подсчитать её напрямую быстро становится невозможным чисто вычислительно. Есть разные более эффективные методы подсчёта этой вероятности, в том числе на основе дискретного преобразования Фурье, но я здесь не буду в них углубляться, а просто

[1] *Симеон Дени Пуассон* (Siméon Denis Poisson, 1781–1840) — французский математик, физик и механик. Способности Пуассона выяснились случайно: когда ему в детстве попали в руки тетради журнала École Polytechnique, Пуассон начал находить верные решения, после чего отец поместил его в центральную школу, а затем Пуассон в ту самую École Polytechnique и поступил. Там его заметили Лаплас и Лагранж, и первые научные статьи появились у Пуассона уже в 20 лет. В итоге Пуассон стал автором важных трудов по небесной механике, где доказал устойчивость планетарных движений и развил теорию притяжения, получил важные результаты по электростатике, магнетизму, теории упругости, гидромеханике и оптике, а в математике занимался теорией определённых интегралов и рядов Фурье. Список объектов, названных в честь Пуассона, конечно, уступает эйлеровскому, но всё равно выглядит крайне внушительно: здесь не только распределения и случайные процессы, но и многочисленные уравнения Пуассона, скобка Пуассона в гамильтоновой механике, пятно Пуассона в оптике, ядро Пуассона и соответствующая интегральная формула, алгебра Пуассона и многое, многое другое.

оставлю ссылку [95]. Зато со средним и дисперсией всё ясно, они в данном случае получаются по аддитивности:

$$\mathbb{E}[X] = \sum_{i=1}^{N} p_i, \qquad \operatorname{Var}[X] = \sum_{i=1}^{N} p_i (1 - p_i).$$

От бросков монеток естественным образом можно перейти к броскам игральных костей. Для n бросков игральной кости с k гранями и вероятностями граней $\boldsymbol{p} = (\, p_1 \; p_2 \; \cdots \; p_k \,)$, где $\sum_{i=1}^{k} p_i = 1$, набор возможных исходов представляет собой вектор

$$\boldsymbol{x} = \begin{pmatrix} x_1 & x_2 & \dots & x_k \end{pmatrix}^\top$$

счётчиков того, сколько раз выпала каждая из этих граней. Вероятность получить тот или иной вектор составляет *мультиномиальное распределение*

$$\operatorname{Mult}(\boldsymbol{x} | n, p_1, p_2, \dots, p_k) = \binom{n}{\boldsymbol{x}} p_1^{x_1} p_2^{x_2} \dots p_k^{x_k} = \frac{n!}{x_1! x_2! \dots x_k!} p_1^{x_1} p_2^{x_2} \dots p_k^{x_k}$$

для векторов $\boldsymbol{x}$ с $x_i \geq 0$ и $\sum_{i=1}^{k} x_i = n$ и ноль в противном случае.

Математическое ожидание здесь, разумеется, равно

$$\begin{aligned} \mathbb{E}[x_i] = \sum_{\boldsymbol{x}:\, \sum_i x_i = n} x_i \frac{n!}{x_1! \dots x_k!} p_1^{x_1} \dots p_k^{x_k} = \\ = n p_i \sum_{\boldsymbol{y}:\, \sum_y y_i = n-1} \frac{n!}{y_1! \dots y_k!} p_1^{y_1} \dots p_k^{y_k} = n p_i, \end{aligned} \tag{3.2}$$

где мы сделали замену переменных $\boldsymbol{y} = (\, x_1 \; \dots \; x_i - 1 \; \dots \; x_k \,)^\top$; обратите внимание, что у второй суммы в (3.2) меньше слагаемых, чем у первой, но оставшиеся слагаемые имеют $x_i = 0$ и, следовательно, вносят нулевой вклад в ожидание.

А матрица ковариаций выглядит так: её диагональные элементы — это дисперсии биномиальных распределений с вероятностями p_i, которые мы нашли выше в (3.1), а внедиагональные элементы представляют собой ковариации

$$\operatorname{Cov}(x_i, x_j) = \mathbb{E}\left[x_i x_j\right] - \mathbb{E}[x_i]\, \mathbb{E}\left[x_j\right].$$

Чтобы их найти, давайте представим x_i как сумму случайных величин-индикаторов, показывающих, выпала ли игральная кость i-й гранью вверх во время каждого из бросаний:

$$x_i = \sum_{s=1}^{n} I_i^{(s)}, \text{ где } I_i^{(s)} = \begin{cases} 1, & \text{если в } s\text{-м броске выпала грань } i, \\ 0 & \text{в противном случае.} \end{cases}$$

Теперь можно отметить, что индикаторы $I_i^{(s)}$ и $I_j^{(s')}$ независимы для $s \neq s'$ (это исходы разных независимых бросков), а для одного и того же броска $I_i^{(s)}$ и $I_j^{(s)}$ не могут произойти одновременно для $i \neq j$, то есть $\mathbb{E}\left[I_i^{(s)} I_j^{(s)}\right] = 0$. Так что остаётся только воспользоваться линейностью математического ожидания:

$$\operatorname{Cov}(x_i, x_j) = \mathbb{E}\left[\left(\sum_{s=1}^{n} I_i^{(s)}\right)\left(\sum_{s'=1}^{n} I_j^{(s')}\right)\right] - n^2 p_i p_j =$$

$$= \sum_{s=1}^{n} \mathbb{E}\left[I_i^{(s)} I_j^{(s)}\right] + \sum_{s \neq s'} \mathbb{E}\left[I_i^{(s)} I_j^{(s')}\right] - n^2 p_i p_j = \sum_{s \neq s'} p_i p_j - n^2 p_i p_j = -n p_i p_j.$$

Обратите внимание, что ковариация получилась отрицательной, потому что увеличение числа одного из исходов уменьшает «пирог», приходящийся на все остальные грани игральной кости.

3.1.3. Распределение Пуассона и закон редких событий

Из распределений со счётным числом исходов (будем считать, что они заданы на множестве натуральных чисел $\mathbb{N}$) в машинном обучении чаще других возникает одно: *распределение Пуассона* $\operatorname{Pois}(\lambda)$. Это дискретное распределение, которое показывает, с какой вероятностью за единицу времени произойдёт то или иное число событий, если они происходят независимо друг от друга с некоторой фиксированной интенсивностью. Иначе говоря, предположим, что некоторые события удовлетворяют предположениям так называемого *пуассоновского процесса*:

- наступление одного события не влияет на вероятность того, что произойдёт другое, то есть они происходят независимо друг от друга;
- интенсивность наступления событий, то есть то, сколько событий в среднем происходит за единицу времени, остаётся постоянной и не зависит от наступления событий.

Тогда случайная величина X, равная числу раз k, которое это событие произойдёт за фиксированный интервал времени, имеет *распределение Пуассона*:

$$p\left(X = k|\lambda\right) = \frac{1}{k!}\lambda^k e^{-\lambda}. \tag{3.3}$$

В этой параметризации λ представляет собой ту самую интенсивность, ожидаемое число событий за единицу времени; и действительно,

$$\mathbb{E}\left[X\right] = \sum_{k=0}^{\infty} k \frac{1}{k!}\lambda^k e^{-\lambda} = \sum_{k=0}^{\infty} \frac{1}{(k-1)!}\lambda^k e^{-\lambda} = \lambda e^{-\lambda} \sum_{j=0}^{\infty} \frac{\lambda^j}{j!} = \lambda,$$

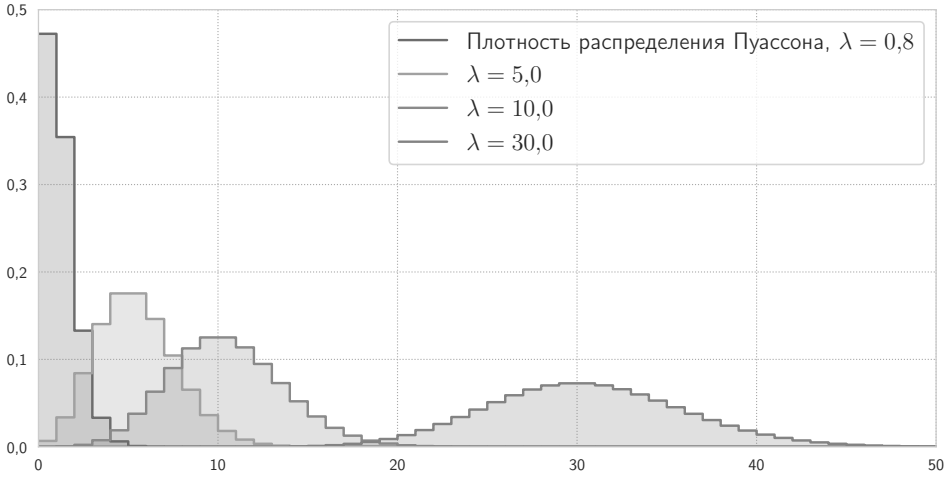


Рис. 3.3. Плотность распределения Пуассона с разными значениями λ

где мы сделали замену переменных $j = k - 1$ и заметили, что $\sum_{j=0}^{\infty} \frac{\lambda^j}{j!}$ — это как раз ряд Тейлора для экспоненты e^{λ}.

Любопытно, что и дисперсия распределения Пуассона тоже равна λ:

$$\begin{aligned}\operatorname{Var}[X] &= \mathbb{E}\left[X^2\right] - \mathbb{E}[X]^2 = \sum_{k=0}^{\infty} k^2 \frac{1}{k!}\lambda^k e^{-\lambda} - \lambda^2 = \lambda e^{-\lambda} \sum_{k=1}^{\infty} k \frac{1}{(k-1)!}\lambda^{k-1} - \lambda^2 \\ &= \lambda e^{-\lambda}\left(\sum_{k=1}^{\infty}(k-1)\frac{1}{(k-1)!}\lambda^{k-1} + \sum_{k=1}^{\infty}\frac{1}{(k-1)!}\lambda^{k-1}\right) - \lambda^2 \\ &= \lambda e^{-\lambda}\left(\lambda\sum_{i=0}^{\infty}\frac{\lambda^i}{i!} + \sum_{j=0}^{\infty}\frac{\lambda^j}{j!}\right) - \lambda^2 = \lambda^2 + \lambda - \lambda^2 = \lambda,\end{aligned}$$

где мы применили тот же трюк для $j = k - 1$ и $i = k - 2$.

На рис. 3.3 показаны графики плотности распределения Пуассона с разными значениями λ; их можно было бы изобразить и гладкими красивыми кривыми, но в этом разделе я рисую графики плотности в виде кусочно-постоянных функций, чтобы подчеркнуть, что речь идёт о дискретных распределениях.

Плотности распределений на рис. 3.3 очень напоминают плотности биномиального распределения на рис. 3.2. И действительно, кажется интуитивно ясным, что здесь есть связь: события, происходящие независимо друг от друга, можно представить себе как подбрасывание большого числа монеток. Формально этот результат известен как *закон редких событий* (law of rare events), или теорема

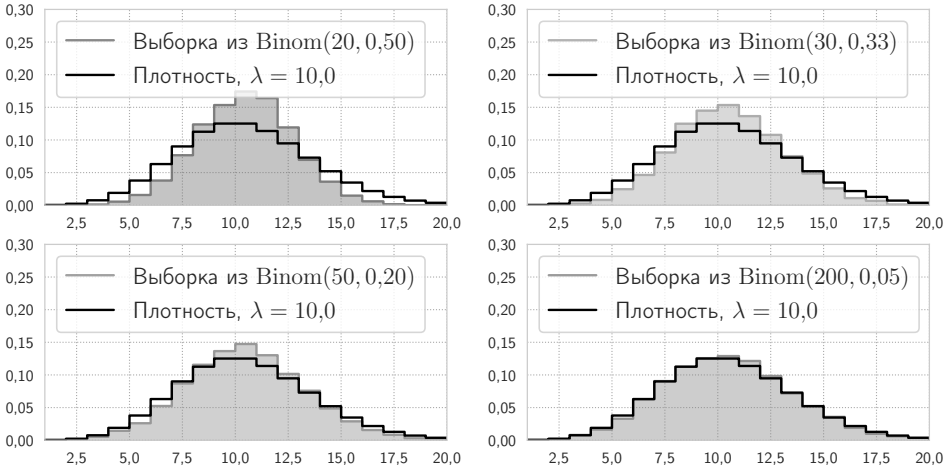


Рис. 3.4. Закон редких событий: как биномиальное распределение сходится к распределению Пуассона с $\lambda = np$

Пуассона: если для последовательности вероятностей p_n существует конечный предел последовательности np_n, $\lambda = \lim_{n\to\infty} np_n$, то для возрастающих n распределение $\mathrm{Binom}(n, p)$ сходится к распределению $\mathrm{Pois}(\lambda)$. Это легко доказать, если вспомнить один замечательный предел из курса математического анализа: для каждого возможного исхода k

$$\lim_{n\to\infty} \binom{n}{k} p_n^k (1-p_n)^{n-k} \approx$$

$$\approx \lim_{n\to\infty} \frac{n(n-1)\cdot\ldots\cdot(n-k+1)}{k!} \left(\frac{\lambda}{n}\right)^k \left(1-\frac{\lambda}{n}\right)^{n-k}$$

$$= \lim_{n\to\infty} \frac{n^k + O(n^{k-1})}{k!} \frac{\lambda^k}{n^k} \left(1-\frac{\lambda}{n}\right)^{n-k} = \frac{\lambda^k}{k!} \lim_{n\to\infty} \left(1-\frac{\lambda}{n}\right)^{n-k} = \frac{\lambda^k}{k!} e^{-\lambda},$$

потому что $\lim_{x\to\infty} \left(1+\frac{1}{x}\right)^x = e$.

На практике закон редких событий означает, что если вы подбрасываете достаточно много (хотя бы несколько сотен) монеток с одинаковой вероятностью успеха, вы можете не возиться с биномиальными коэффициентами, а достаточно точно аппроксимировать биномиальное распределение при помощи распределения Пуассона с соответствующей интенсивностью. И называется он законом *редких* событий, потому что для того, чтобы аппроксимация была хорошей,

вероятность каждого события должна быть достаточно мала, а n соответственно достаточно велико. На рис. 3.4 проиллюстрирована сходимость биномиального распределения к распределению Пуассона для $\lambda = 10$: как видите, выборка из $\text{Binom}(20, \frac{1}{2})$ не слишком похожа на $\text{Pois}(10)$, а вот $\text{Binom}(200, \frac{1}{20})$ уже вполне можно так аппроксимировать.

Самый естественный пример распределения Пуассона — это срабатывания счётчика Гейгера, то есть события распада радиоактивных атомов за некоторый период времени: это архетипический пример случайных и независимых событий, лучше не найти. Сам Пуассон в книге 1837 года, в которой обсуждал это распределение [472, 473], писал о числе неверных приговоров в судебной системе той или иной страны. Впрочем, ровно тот же результат для тех же самых данных о неверных приговорах получил Абрахам де Муавр более чем за сто лет до Пуассона, в работе 1711 года [403]; так что распределение Пуассона считается классическим примером закона Стиглера[1], и есть мнение, что его нужно переименовать в распределение де Муавра.

3.1.4. Отрицательное биномиальное и гипергеометрическое распределения

У распределения Пуассона есть важный недостаток, который проявляется при использовании в машинном обучении. Дело в том, что ему зачастую немного не хватает параметров: в реальных данных не всегда будет разумно моделировать и среднее, и дисперсию одним и тем же числом λ, хочется всё-таки иметь дополнительную степень свободы, которая разделила бы эти два ключевых параметра.

Поэтому наряду с распределением Пуассона часто рассматривают так называемое *отрицательное биномиальное распределение*. Оно определяется следующим образом: предположим, что мы проводим испытания Бернулли, то есть бросаем монетку с вероятностью орла p и ждём, пока не выпадет некоторое фиксированное число орлов r. Тогда число решек, которые выпадут до r-го орла, имеет отрицательное биномиальное распределение. Вероятность получить k решек в таком эксперименте ясна из обычного биномиального распределения:

[1] Закон Стиглера, предложенный в 1980 году профессором статистики Стивеном Стиглером [561], утверждает, что ни одно научное открытие не названо в честь человека, который его действительно сделал первым. Примерам несть числа в самых разных областях. В статистике и машинном обучении, например, де Муавр ввёл не только распределение Пуассона, но и распределение Гаусса, сам Пуассон ввёл и изучил функции Дирака и распределение Коши, оценки Чернова первым доказал Герман Робин, модели Брэдли — Терри были разработаны великим логиком Эрнстом Цермело, алгоритм Метрополиса — Гастингса принадлежит Августе Теллер и Маршаллу и Арианне Розенблют, а про закон Ципфа мы поговорим чуть позже. Классическая математика тоже подчиняется закону Стиглера: диаграммы Венна ввёл Леонард Эйлер, пять платоновых тел построил Теэтет Афинский, а арабские цифры пришли из Индии. Далее — везде: идею буриданова осла ввёл арабский философ аль-Газали, первый и второй законы Ньютона были сформулированы Галилеем, Гуком и Гюйгенсом (третий зато действительно принадлежит сэру Исааку), а, например, Элисон Бекдел всегда настаивала, что идея её теста принадлежит её знакомой Лиз Уоллес. Без метаиронии здесь тоже не обошлось: сам Стиглер ссылался на более ранние работы социолога Роберта Мертона, который задолго до Стиглера сформулировал тот же принцип.

$$\text{NegBinom}\,(k|r,p) = \binom{k+r-1}{r-1}(1-p)^k\,p^r. \tag{3.4}$$

В отличие от предыдущих распределений, здесь нормировочная константа $\binom{k+r-1}{r-1}$ не вполне очевидна; давайте её получим, вспомнив заодно важное тождество о биномиальных коэффициентах:

$$\binom{k+m-1}{m-1} + \binom{k+m-1}{m} = \frac{(k+m-1)!}{(m-1)!k!} + \frac{(k+m-1)!}{m!(k-1)!} =$$
$$= \frac{(k+m-1)!}{(m-1)!(k-1)!}\left(\frac{1}{k}+\frac{1}{m}\right) = \frac{(k+m-1)!}{(m-1)!(k-1)!}\,\frac{k+m}{km} = \binom{k+m}{m}. \tag{3.5}$$

Теперь давайте введём функцию, слегка обобщающую (3.4):

$$f_m(x) = \sum_{k=0}^{\infty}\binom{k+m}{m}x^k, \quad \text{то есть, в частности,}$$
$$p^r f_{r-1}(1-p) = p^r\sum_{k=0}^{\infty}\binom{k+r-1}{r-1}(1-p)^k = \sum_{k=0}^{\infty}\text{NegBinom}\,(k|r,p)\,.$$

Мы можем найти рекуррентное условие на функцию $f_m(x)$, воспользовавшись тождеством (3.5):

$$f_m(x) = \sum_{k=0}^{\infty}\binom{k+m}{m}x^k = \sum_{k=0}^{\infty}\binom{k+m-1}{m-1}x^k + \sum_{k=0}^{\infty}\binom{k+m-1}{m}x^k =$$
$$= f_{m-1}(x) + x\sum_{k=1}^{\infty}\binom{k-1+m}{m}x^{k-1} = f_{m-1}(x) + xf_m(x),$$

а значит,

$$f_m(x) = \frac{f_{m-1}(x)}{1-x}, \quad \text{то есть} \quad f_m(x) = \frac{f_0(x)}{(1-x)^m}.$$

Начальное значение функции подсчитать легко:

$$f_0(x) = \sum_{k=0}^{\infty}x^k = \frac{1}{1-x},$$

и мы получаем, что

$$f_m(x) = \frac{1}{(1-x)^{m+1}}. \tag{3.6}$$

Осталось только подставить в (3.6) значения $m = r - 1$ и $x = 1 - p$:

$$f_{r-1}(1-p) = \frac{1}{p^r}, \quad \text{а значит,} \quad \sum_{k=0}^{\infty} \text{NegBinom}\,(k|r,p) = 1.$$

Это распределение по форме похоже на пуассоновское, но у него два параметра. Меняя эти параметры, мы можем изменить по отдельности и среднее, и дисперсию отрицательного биномиального распределения:

$$\begin{aligned}\mathbb{E}\,[k|r,p] &= \sum_{k=0}^{\infty} k\binom{k+r-1}{r-1}(1-p)^k\,p^r = \sum_{k=0}^{\infty} k\frac{(k+r-1)!}{k!(r-1)!}(1-p)^k\,p^r = \\ &= \sum_{k=0}^{\infty} r\frac{(k+r-1)!}{(k-1)!r!}(1-p)^{k-1}(1-p)\,\frac{p^{r+1}}{p} = \\ &= \frac{(1-p)r}{p}\sum_{k=1}^{\infty}\binom{(k-1)+(r+1)-1}{(r+1)-1}(1-p)^{k-1}\,p^{r+1} = \frac{r(1-p)}{p}.\end{aligned}$$

Чтобы подсчитать дисперсию, заметим, что

$$\mathbb{E}\left[k^2\middle|r,p\right] = \sum_{k=0}^{\infty} k^2 p\,(k|r,p) = \sum_{k=0}^{\infty} k(k-1)p\,(k|r,p) + \mathbb{E}\,[k|r,p]\,,$$

а первое слагаемое в данном случае можно подсчитать как

$$\begin{aligned}&\sum_{k=0}^{\infty} k(k-1)\frac{(k+r-1)!}{k!(r-1)!}(1-p)^k\,p^r = \sum_{k=0}^{\infty}\frac{(k+r-1)!}{(k-2)!(r-1)!}(1-p)^k\,p^r = \\ &= \sum_{k=0}^{\infty}\frac{((k-2)+(r+2)-1)!r(r+1)}{(k-2)!((r+2)-1)!}(1-p)^{k-2+2}\,\frac{p^{r+2}}{p^2} = \frac{r(r+1)(1-p)^2}{p^2}.\end{aligned}$$

И дисперсия теперь получается как

$$\text{Var}\,[k|r,p] = \frac{r(r+1)(1-p)^2}{p^2} + \frac{r(1-p)}{p} - \left(\frac{r(1-p)}{p}\right)^2 = \frac{r(1-p)}{p^2}.$$

Я не отказал себе в удовольствии напомнить вам эти довольно изящные рассуждения с биномиальными коэффициентами, но точно такие же результаты можно было бы получить гораздо проще. Дело в том, что отрицательное биномиальное распределение — это обобщение геометрического распределения, а точнее сумма r геометрических распределений: чтобы получить r орлов, нужно r раз

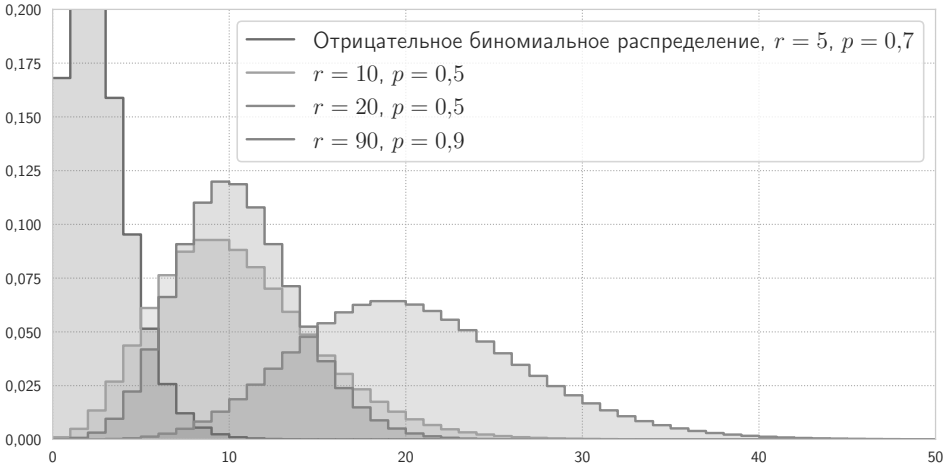


Рис. 3.5. Плотность отрицательного биномиального распределения с разными значениями r и p

последовательно получить по одному орлу. Таким образом, ожидание отрицательного биномиального распределения складывается из r ожиданий геометрического распределения, которое равно $\frac{1-p}{p}$, а дисперсия — из r дисперсий геометрического распределения, которая равна $\frac{1-p}{p^2}$.

На рис. 3.5 показаны примеры отрицательного биномиального распределения. В частности, там показаны плотности двух распределений, $\text{NegBinom}\left(10, \frac{1}{2}\right)$ и $\text{NegBinom}\left(90, \frac{9}{10}\right)$, у которых одинаковое среднее $\frac{10\cdot(1-0.5)}{0.5} = \frac{90\cdot(1-0.9)}{0.9} = 10$, но разные дисперсии: $\frac{10\cdot(1-0.5)}{0.5^2} = 20$ и $\frac{90\cdot(1-0.9)}{0.9^2} = \frac{100}{9}$ соответственно.

Кстати, откуда такое странное название — «отрицательное биномиальное»? Что здесь отрицательного? Дело в том, что биномиальный коэффициент в определении (3.4) можно переписать как

$$\binom{k+r-1}{r-1} = \frac{(k+r-1)(k+r-2)\cdot\ldots\cdot r}{k!} = \\ = (-1)^k \frac{(-r)(-r-1)\cdot\ldots\cdot(-r-k+1)}{k!} = (-1)^k \binom{-r}{k},$$

где мы искусственно «продолжили» определение биномиальных коэффициентов на отрицательные числа.

Ещё одно дискретное распределение, которое пригодится нам в книге, — *гипергеометрическое распределение* (hypergeometric distribution). Интуиция здесь происходит из простого эксперимента: предположим, что у нас есть N объектов

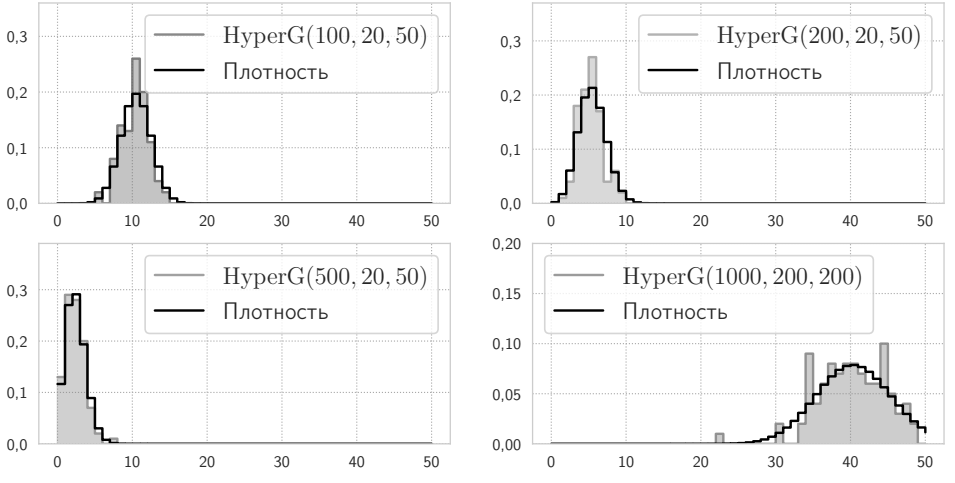


Рис. 3.6. Плотность гипергеометрического распределения с разными значениями параметров M, n и K

и K из них «особенные», то есть обладают некоторым свойством; например, в колоде из $N = 52$ карт есть $K = 13$ пик. Далее предположим, что мы берём некоторую выборку размера n случайным образом из этих N объектов, например набираем руку из $n = 2$ карт. Гипергеометрическое распределение отвечает на вопрос о том, сколько из этих объектов будут обладать нужным свойством, то есть в данном случае с какой вероятностью на руке окажутся 0, 1 или 2 карты пиковой масти.

Математически здесь нет ничего сложного, но формулы получаются не очень приятные. В нашем эксперименте $\binom{N}{n}$ возможных исходов, то есть выборок по n элементов из N. Чтобы в выборку попали k элементов с нужным свойством из K, можно выбрать сначала эти k элементов, что можно сделать $\binom{K}{k}$ вариантами, а затем выбрать остальные из не обладающих этим свойством, что даёт $\binom{N-K}{n-k}$ вариантов.

Итого вероятность получить k нужных элементов получается равной

$$p\left(k|N,n,K\right) = \frac{\binom{K}{k}\binom{N-K}{n-k}}{\binom{N}{n}}. \tag{3.7}$$

На рис. 3.6 показаны несколько примеров выборок из гипергеометрического распределения и соответствующих его плотностей. Примеры с картами здесь тоже очень хорошо подходят, и, например, игрокам в покер гипергеометрическое распределение желательно хорошо чувствовать. К примеру, предположим, что

вы играете в техасский холдем и видите у себя на руках пикового туза и трефовую девятку, а на столе — три пиковые карты. С какой вероятностью вы соберёте старший флеш (пять пиковых карт с тузом, который у вас уже есть) к концу раздачи?

Для этого вам достаточно получить хотя бы одну пику из двух оставшихся карт; предположим, что карт соперников вы совсем не знаете и даже предположить не можете, так что, по сути, вам нужно, чтобы выпала хотя бы одна из оставшихся $K = 13 - 4 = 9$ пик в совокупности из $N = 52 - 5 = 47$ карт в колоде за выборку из $n = 2$ оставшихся карт. Гипергеометрическое распределение здесь даёт

$$\begin{aligned} p\,(k=0|M,n,K) &= 1\cdot\tbinom{47-9}{2}/\tbinom{47}{2} \approx 0{,}65,\\ p\,(k=1|M,n,K) &= \tbinom{9}{1}\tbinom{47-9}{1}/\tbinom{47}{2} \approx 0{,}317,\\ p\,(k=2|M,n,K) &= \tbinom{9}{2}\cdot 1/\tbinom{47}{2} \approx 0{,}033, \end{aligned}$$

то есть вероятность собрать флеш составляет около 0,35.

*3.1.5. Закон Ципфа

И закончим этот раздел на том, что обсудим ещё одно интересное дискретное распределение — хотя напрямую мы с ним в этой книге работать не будем, в некотором смысле оно происходит именно из машинного обучения без учителя, точнее говоря, из статистического анализа данных. Это *распределение Ципфа*[1], или *закон Ципфа* (Zipf's law, Zipf's distribution), в котором вероятность получить исход k из конечного множества $1,\ldots,N$ обратно пропорциональна некоторой постоянной степени числа k:

$$p\,(k|s,N) = \frac{1}{H_{N,s}}\frac{1}{k^s},$$

где $H_{N,s}$ — это нормировочная константа распределения Ципфа, N-е обобщённое гармоническое число

$$H_{N,s} = 1 + \frac{1}{2^s} + \ldots + \frac{1}{N^s}.$$

[1] *Джордж Кингсли Ципф* (George Kingsley Zipf, 1902–1950) — американский лингвист и филолог. Ципф родился в семье выходцев из Германии, учился в Гарварде, Берлине и Бонне, а потом вернулся в Гарвард и стал преподавать немецкий, продолжая свои исследования. Помимо лингвистики, Ципф находил новые приложения найденной им закономерности в других науках. Например, оказалось, что размер городов в стране тоже удовлетворяет закону Ципфа, а в 1941 году вышла книга Ципфа *National unity and disunity: The nation as a bio-social organism*, в которой он обратил внимание на то, что тот же закон описывает распределение доходов людей, и предположил, что нарушения этого закона приводят к социальной напряжённости и в итоге к революционным ситуациям. К сожалению, у Ципфа слишком поздно обнаружили злокачественную опухоль и он умер молодым.

Ципф обнаружил[1], что этому закону подчиняется распределение встречаемости слов в языке: частота появления слова обратно пропорциональна его номеру по порядку этих самых частот [665, 666]. Иначе говоря, второе по популярности слово будет встречаться примерно вдвое реже, чем самое популярное, третье — втрое реже, чем первое, и так далее.

Исследователи показали, что закон Ципфа выполняется для разных языков мира, и не только языков, но и других объектов — доходов [513], размеров городов, нот в музыке [657], размеров корпораций [38] и так далее; а в языке закон Ципфа ещё лучше выполняется для коротких фраз, то есть более естественных единиц смысла [667]. Мимо этой красивой закономерности не прошёл даже Бенуа Мандельброт[2], который обобщил закон Ципфа до параболического фрактального распределения [373], в котором логарифм частоты встречаемости объекта представляет собой не линейную, а квадратичную функцию от логарифма его порядкового номера.

Долгое время считалось, что закон Ципфа представляет собой какую-то достаточно глубокую закономерность, отражающую свойства созданных людьми объектов, и смысл этой закономерности было бы важно объяснить. Но в последние десятилетия появились работы, которые убедительно показывают, что это не так. Например, в 1991 году Вэньтян Ли показал [341], что закон Ципфа возникает даже в случайно порождённых текстах! Это, кстати, очень простое рассуждение: давайте порождать слова русского алфавита как случайные сочетания символов. Всего этих символов будет 34: 33 буквы и пробел, и мы будем рассматривать последовательности букв между пробелами как слова. Тогда вероятность получить любое «слово» из одной буквы, например *_а_*, составит $(1/34)^3$, любое «слово» из двух букв, например *_аа_* или *_аб_*, — $(1/34)^4$ и так далее, то есть для алфавита из M букв вероятность породить любое конкретное слово x длины L составит

$$p\,(x|M,L) = \frac{1}{Z}\frac{1}{(M+1)^{L+2}}.$$

Нормировочную константу Z можно найти из соображения о том, что всего слов длины L будет M^L:

[1] Впрочем, на самом деле Ципф не претендовал на приоритет и сам ссылался на более ранние работы Жан-Батиста Эсту (Jean-Baptiste Estoup) [170], который описал ту же закономерность для естественных языков ещё в 1908 году, а для размеров городов этот закон заметил Феликс Ауэрбах в 1913 году [35]; вот и ещё один пример закона Стиглера.

[2] *Бенуа Мандельброт* (Benoît B. Mandelbrot, 1749–1827) — французский и американский математик. Главное детище Мандельброта — теория фракталов, множеств, обладающих свойством самоподобия, то есть таких множеств, которые по форме совпадают с частями самих себя. В математике фракталы — это множества с дробной метрической размерностью; в жизни точных фракталов, конечно, не найти, но приближённо их свойствам удовлетворяют такие объекты, как кроны деревьев, снежинки, сосуды в системе кровообращения и так далее; например, первая работа Мандельброта о фракталах называлась «Какова длина побережья Великобритании?» [374]. Мандельброт сам добавил себе инициал «В», и он не соответствует никакому второму имени; весьма уместная шутка утверждает, что этот инициал расшифровывается как Benoît B. Mandelbrot.

$$Z = \sum_{L=1}^{\infty} \frac{M^L}{(M+1)^{L+2}} = \frac{M}{(M+1)^2},$$

то есть

$$p\,(x|M,L) = \frac{(M+1)^2}{M} \frac{1}{(M+1)^{L+2}} = \frac{1}{M(M+1)^L}. \tag{3.8}$$

В таком случайно порождённом тексте более короткие слова всегда популярнее более длинных, а слов длины L всего M^L. Это значит, что ранг $\mathrm{rank}(L)$ любого слова длины L составит

$$\sum_{l=1}^{L-1} M^l < \mathrm{rank}(L) \le \sum_{l=1}^{L} M^l,$$

то есть

$$\frac{M(M^{L-1}-1)}{M-1} < \mathrm{rank}(L) \le \frac{M(M^L-1)}{M-1}. \tag{3.9}$$

Здесь мы уже видим, что получается экспоненциальная зависимость между длиной слов и их рангом, что естественно, ведь с длиной слов экспоненциально растёт и их число. Но можно пойти и дальше и найти более точную форму вероятности появления того или иного слова как функцию от его ранга. Для этого нужно преобразовать (3.9) так, чтобы выделить там вероятности слов, данные в (3.8). Давайте умножим (3.9) на $\frac{M-1}{M}$, прибавим единицу ко всем частям и возьмём логарифм по M:

$$L - 1 < \log_M \left(\frac{M-1}{M} \mathrm{rank}(L) + 1 \right) \le L.$$

Теперь, чтобы получить слева и справа выражения из формулы (3.8), возведём $1/(M+1)$ в эти степени (неравенства перевернутся) и умножим на $1/M$:

$$\frac{1}{M(M+1)^{L-1}} > \frac{1}{M} \left(\frac{1}{\frac{M-1}{M}\mathrm{rank}(L) + 1} \right)^{\frac{\log(M+1)}{\log M}} \ge \frac{1}{M(M+1)^L}.$$

И вот мы получили, что

$$p\,(x|M,L) \le \frac{C}{(\mathrm{rank}(L)+B)^\alpha} < p\,(x|M,L-1)\,, \tag{3.10}$$

где

$$\alpha = \frac{\log(M+1)}{\log M}, \quad B = \frac{M}{M-1}, \quad C = \frac{1}{M}\left(\frac{M-1}{M}\right)^\alpha = \frac{M^{\alpha-1}}{(M-1)^\alpha}.$$

Уравнение (3.10) представляет собой *обобщённый закон Ципфа*, который ввёл в 1950-х годах тот самый Бенуа Мандельброт [372]. Более того, оценки констант в законе Ципфа для, например, английского языка давали $C \approx 0{,}1$ и $\alpha \approx 1$,

а если подставить в формулу (3.10) число английских букв $M = 26$, получится как раз $\alpha \approx 1{,}01158$ и $C \approx 0{,}037$, то есть экспонента получается очень точная! Расхождение в константе C легко объяснить тем, что в настоящем языке далеко не все последовательности букв представляют собой слова; впрочем, нет ничего удивительного в том, что мы получили близкое к единице число из формулы $\alpha = \frac{\log(M+1)}{\log M}$, экспонента для не совсем маленьких M всегда будет близка к единице.

Позволю себе глобальный, почти философский вывод из этой истории. Как мы увидели выше, закон Ципфа оказался не то чтобы совсем пустышкой, но чисто математическим фактом, вовсе не представляющим собой какое-то глубокое свойство естественных языков. Это достаточно часто встречающийся эффект: закономерности, которые на первый взгляд кажутся интересными, выражающими важные свойства окружающего мира, на поверку оказываются почти тривиальными фактами, которые можно получить или совсем без каких бы то ни было предположений, или, наоборот, с очень сильными и, очевидно, не выполняющимися на практике предположениями. А выполнять эту поверку хочется не всегда: человеческий мозг постоянно ищет закономерности, на которые можно было бы опереться в нашем стохастическом мире, и частенько занимается оверфиттингом, то есть находит закономерности, которых на самом деле нет.

Самый простой пример такого оверфиттинга известен как *закон малых чисел* (law of small numbers, названный так в противоположность закону больших чисел): люди слишком уверены в том, что малые выборки будут похожи на большие, то есть на теоретическое распределение. Если подбросить честную монетку два раза, вероятность получить одного орла и одну решку составит всего лишь $\frac{1}{2}$ (а если один раз, то 0, но это мы уже обсуждали в разделе 2.4). В реальности это приводит к тому, что люди, в том числе даже проводящие эксперименты учёные, слишком быстро делают далеко идущие выводы из небольших выборок. Этот термин ввели знаменитые психологи и экономисты Амос Тверски и Даниэль Канеман в вышедшей в 1971 году работе *Belief in the law of small numbers* [609]. Они писали в основном об экспериментальных исследованиях, размеры выборок в которых часто слишком малы, но и простые житейские примеры здесь тоже уместны. С одной стороны, вера в закон малых чисел приводит к «ошибке игрока» (gambler's fallacy): мнению, что если честная монетка пять раз подряд выпала решкой, то на шестой раз должен с большей вероятностью выпасть орёл, чтобы как будто «скорректировать» эту малую выборку. Как мы с вами обсуждали в разделе 2.4, после пяти решек подряд можно слегка засомневаться в предположении, что монетка честная, но это сомнение может только *уменьшить* ожидаемую вероятность орла, а не увеличить её.

С другой стороны, закон малых чисел приводит к слишком быстрым обобщениям: людям кажется, что закономерность, встретившуюся им на малой выборке, можно ожидать и дальше. Канеман и Тверски приводят такой (надеюсь, что всё-таки не совсем реальный) пример: «Предположим, что [исследователь]

изучает, какую из двух игрушек будут предпочитать младенцы. Из пяти первых младенцев четыре предпочли одну и ту же игрушку. После этого многие психологи уже будут до некоторой степени уверены, что нулевая гипотеза об отсутствии предпочтений неверна». А панчлайн этой истории, как мне кажется, в том, что в разделе 2.5 мы приводили другой пример закона малых чисел из работ Амоса Тверски... и там выяснилось, что правильное применение теории вероятностей не так легко даётся даже учёным, специализирующимся на некорректном обращении с теорией вероятностей!

Подобный ментальный оверфиттинг лежит в основе всех предрассудков, примет и гаданий нашего мира. Закон Ципфа здесь, конечно, не самый плохой пример, он хотя бы действительно выполняется. Да и настоящих, даже великих математиков чаша сия не всегда минует — вспомним хотя бы Джона Нэша[1]. Простого выхода здесь нет; идеи своего собственного мозга критиковать непросто, но, к сожалению, необходимо. Могу только скромно надеяться, что эта книга сможет добавить ещё пару кирпичиков в фундамент критического мышления моих читателей.

[1] *Джон Форбс Нэш* (John Forbes Nash Jr., 1928–2015) — американский математик, один из создателей теории игр, лауреат Нобелевской премии по экономике 1994 года. Диссертация, которую Нэш защитил в 1950 году [427], состояла из всего лишь 26 страниц и содержала основные положения его теории некооперативных игр и доказательство существования равновесия Нэша; кроме того, Нэш много сделал в алгебраической геометрии [426] и предвосхитил современную основанную на вычислительной сложности криптографию (эти его работы, впрочем, были засекречены аж до 2011 года). То главное, что мы знаем о Нэше из фильма *A Beautiful Mind*, к сожалению, тоже правда: в 1959 году ему диагностировали шизофрению и в течение 1960-х лечили, в том числе шоковой терапией (инсулиновой комой). Лечение было неэффективным, и Нэшу в итоге, к счастью, хватило решимости от него отказаться: в 1970 году он прекратил ложиться в госпитали и принимать лекарства, после чего постепенно восстановился и в 1980-х годах вернулся к работе! Джон Нэш погиб в возрасте 86 лет в автокатастрофе, возвращаясь из аэропорта после того, как получил премию Абеля.

3.2. Важные непрерывные распределения

Отсюда прямо два вывода: первый — упорство в накоплении, даже копеечными суммами, впоследствии даёт громадные результаты (время тут ничего не значит), и второй — что самая нехитрая форма наживания, но лишь непрерывная, обеспечена в успехе математически.

Фёдор Достоевский. Подросток

3.2.1. Непрерывное равномерное распределение

В предыдущем разделе мы говорили о распределениях, заданных на дискретном носителе, то есть на конечном либо счётном числе точек. Теперь пора перейти к непрерывным распределениям, заданным на континуальном множестве; в этой книге мы всегда будем говорить только о распределениях, заданных на евклидовом пространстве $\mathbb{R}^d$ или его подмножествах. Связанные с непрерывными распределениями формальности мы уже кратко обсудили в разделе 2.1, так что здесь будем сразу говорить о плотностях непрерывных распределений. Для каждого распределения мы будем обсуждать его плотность и функцию распределения (если оно одномерное, то есть задано на $\mathbb{R}$), а также получим его среднее и дисперсию (в многомерном случае — матрицу ковариаций).

Нам уже знакомо дискретное равномерное распределение, но равные вероятности можно задать и на континуальном множестве. *Непрерывное равномерное распределение* на одномерном отрезке $[a,b] \subset \mathbb{R}$ — это распределение с плотностью

$$p(x) = \begin{cases} \frac{1}{b-a}, & \text{если } x \in [a,b], \\ 0 & \text{в противном случае.} \end{cases} \tag{3.11}$$

Форма функции распределения в данном случае тоже очевидна — это ломаная линия, которая поднимается от нуля в точке a до единицы в точке b:

$$F(y) = \begin{cases} \frac{y-a}{b-a}, & \text{если } y \in [a,b], \\ 0 & \text{в противном случае.} \end{cases} \tag{3.12}$$

На рис. 3.7 показаны примеры плотностей (сплошные линии) и функций распределения (пунктирные линии) для двух равномерных распределений: на отрезке $[0,1]$ и на отрезке $[-1,3]$.

Среднее здесь тоже вряд ли кого-то удивит:

$$\mathbb{E}[x] = \int_{-\infty}^{\infty} xp(x)\mathrm{d}x = \int_a^b \frac{x}{b-a}\mathrm{d}x = \frac{b^2-a^2}{2(b-a)} = \frac{a+b}{2}.$$

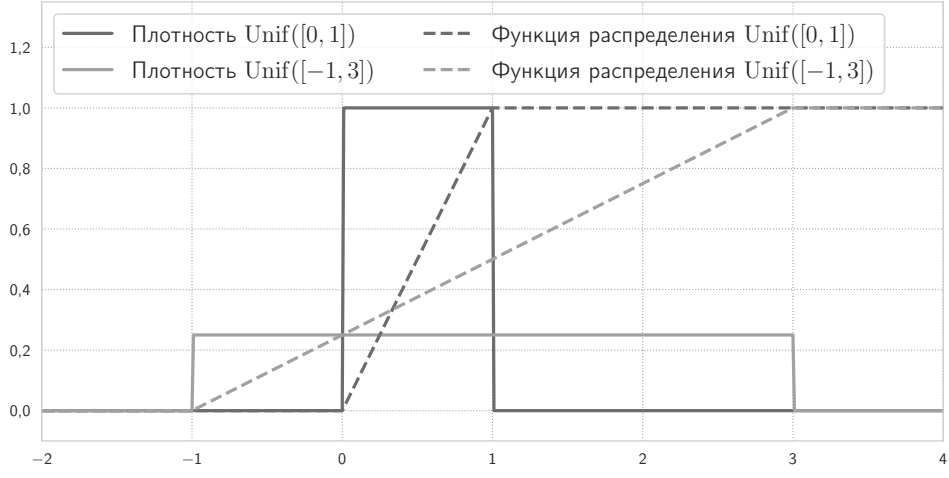


Рис. 3.7. Примеры плотности и функции распределения для равномерных распределений Unif([0,1]) и Unif([−1,3])

Дисперсия равномерного распределения — это уже менее тривиальная формула, которую мы получили в разделе 2.1:

$$\operatorname{Var}[x] = \frac{(b-a)^2}{12}.$$

Обучение же параметров равномерного распределения — это довольно сложная и интересная задача, даже две разные задачи, соответствующие разным возможным процессам обучения; этим задачам будет посвящён раздел 3.3.

3.2.2. Нормальное распределение

Наверное, самое главное непрерывное распределение, да и вообще центральный объект и всей теории вероятностей, и этой книги — это *нормальное*, или *гауссовское*, *распределение* (normal distribution, Gaussian distribution, а часто и просто Gaussian — я тоже иногда буду использовать слово «гауссиан»). В одномерном случае оно задаётся функцией плотности, которая носит название *функции Гаусса* (Gaussian function):

$$p(x) = \mathcal{N}\left(x \middle| \mu, \sigma^2\right) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{1}{2\sigma^2}(x-\mu)^2}. \tag{3.13}$$

С функцией распределения мы пока ничего осмысленного сделать не можем; это неэлементарная функция, которую нам придётся оставить в виде интеграла:

$$F(y) = \int_{-\infty}^{y} \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{1}{2\sigma^2}(x-\mu)^2} \mathrm{d}x. \tag{3.14}$$

Часто оказывается полезным частный случай (3.14), функция распределения *стандартного* нормального распределения $\mathcal{N}(x|0,1)$:

$$\Phi(y) = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^{y} e^{-\frac{x^2}{2}} \mathrm{d}x. \tag{3.15}$$

Она называется *пробит-функцией* (probit) и ещё пригодится нам значительно позже, в разделе 5.2.

Как видите, у нормального распределения два параметра: *среднее* μ и *дисперсия* σ^2. Чтобы убедиться, что они действительно соответствуют своим названиям, давайте сначала докажем факт, который пригодится нам и сам по себе: если случайная величина X имеет нормальное распределение с параметрами μ и σ^2, то случайная величина $Y = aX + b$, где a и b — некоторые константы, имеет нормальное распределение с параметрами $a\mu+b$ и $a^2\sigma^2$. Доказать это не сложно, если обратиться к функции распределения:

$$F_Y(y) = p\left(Y \le y\right) = p\left(aX + b \le y\right) = p\left(X \le \frac{y-b}{a}\right) = F\left(\frac{y-b}{a}\right),$$

и теперь её можно продифференцировать:

$$p_Y(x) = \frac{1}{a} p\left(\frac{x-b}{a}\right) = \frac{1}{a\sigma\sqrt{2\pi}} e^{-\frac{1}{2\sigma^2}\left(\frac{x-b}{a}-\mu\right)^2} = \frac{1}{a\sigma\sqrt{2\pi}} e^{-\frac{1}{2a^2\sigma^2}(x-(a\mu+b))^2},$$

и мы получили в точности плотность нормального распределения с параметрами $a\mu + b$ и $a^2\sigma^2$.

Этот результат означает, что среднее и дисперсию достаточно будет найти для стандартного нормального распределения $\mathcal{N}(x|0,1)$. В этом случае

$$\mathbb{E}\left[x\right] = \int_{-\infty}^{\infty} x p(x) \mathrm{d}x = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^{\infty} x e^{-\frac{x^2}{2}} \mathrm{d}x = -\frac{1}{\sqrt{2\pi}} e^{-\frac{x^2}{2}} \Big|_{-\infty}^{\infty} = 0,$$

и

$$\mathrm{Var}\left[x\right] = \mathbb{E}\left[x^2\right] - \mathbb{E}\left[x\right]^2 = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^{\infty} x^2 e^{-\frac{x^2}{2}} \mathrm{d}x =$$

$$[\text{интегрируем по частям с } u = x \text{ и } \mathrm{d}v = x e^{-\frac{x^2}{2}}]$$

$$= \frac{1}{\sqrt{2\pi}} \left(-x e^{-\frac{x^2}{2}} \Big|_{-\infty}^{\infty} + \int_{-\infty}^{\infty} e^{-\frac{x^2}{2}} \mathrm{d}x \right) = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^{\infty} e^{-\frac{x^2}{2}} \mathrm{d}x = 1.$$

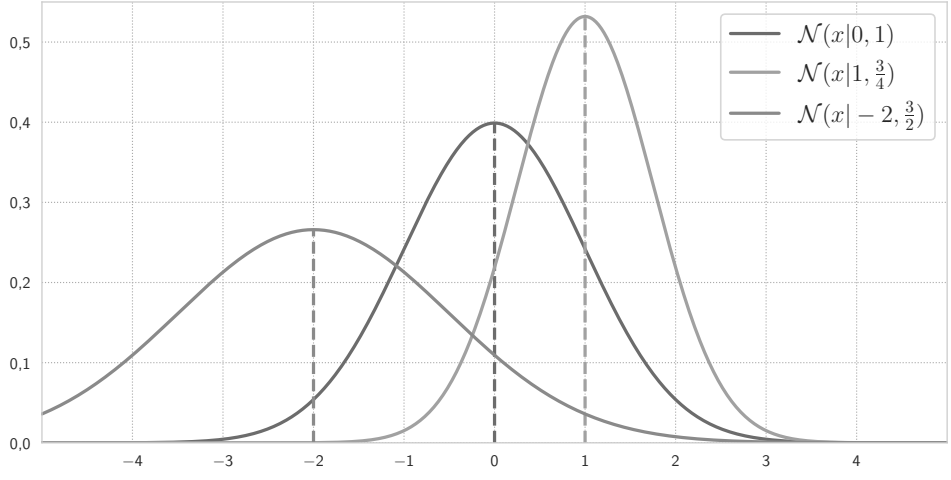


Рис. 3.8. Плотность одномерного нормального распределения с разными значениями μ и σ

Нормальное распределение $\mathcal{N}\left(x|\mu,\sigma^2\right)$, как мы уже выясняли, является распределением случайной величины $X = \sigma Z + \mu$, где Z имеет стандартное нормальное распределение. Так что для X автоматически получается

$$\mathbb{E}\left[x\right] = \mu + \sigma\mathbb{E}\left[z\right] = \mu, \qquad \mathrm{Var}\left[x\right] = \sigma^2\mathrm{Var}\left[z\right] = \sigma^2.$$

На рис. 3.8 показаны несколько примеров одномерного нормального распределения, а на рис. 3.9 — несколько примеров его функции распределения, в том числе пробит-функция $\Phi(x)$.

Полноценный байесовский вывод для нормального распределения потребует некоторых усилий, и мы его отложим до раздела 4.4 в контексте линейной регрессии и раздела 6.4 в общем виде. Однако давайте хотя бы найдём гипотезы максимального правдоподобия для параметров μ и σ. Для этого запишем логарифм правдоподобия гауссиана (3.13) для набора точек $D = \{x_1, \ldots, x_n\}$:

$$\begin{aligned}\log p\left(x_1,\ldots,x_n\middle|\mu,\sigma^2\right) &= \sum_{i=1}^{n}\log\mathcal{N}\left(x_i\middle|\mu,\sigma^2\right) = \\ &= \sum_{i=1}^{n}\left(-\frac{1}{2}\log 2\pi\sigma^2 - \frac{1}{2\sigma^2}\left(x_i-\mu\right)^2\right) = \\ &= -\frac{n}{2}\log 2\pi - \frac{n}{2}\log\sigma^2 - \frac{1}{2\sigma^2}\sum_{i=1}^{n}\left(x_i-\mu\right)^2. \quad (3.16)\end{aligned}$$

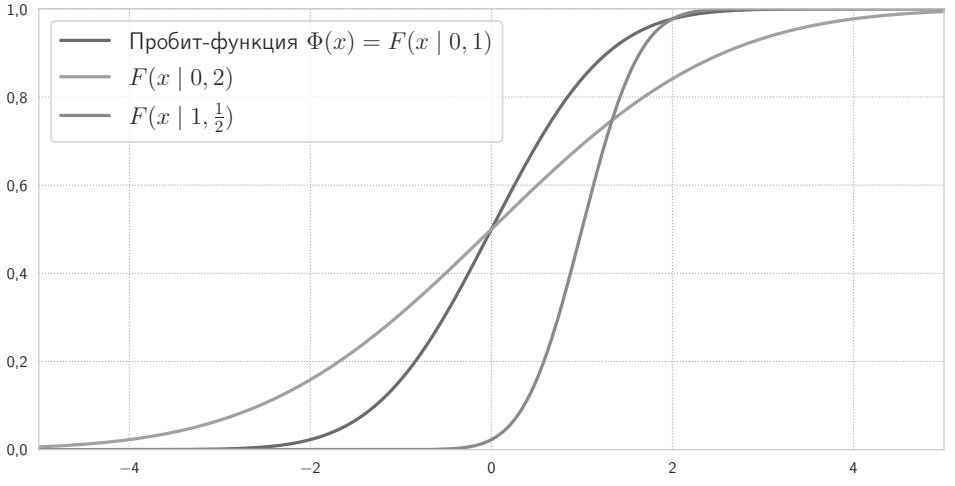


Рис. 3.9. Функции распределения одномерного нормального распределения с разными значениями μ и σ

Чтобы найти гипотезу максимального правдоподобия, нужно найти максимум $\log p\left(x_1, \ldots, x_n \middle| \mu, \sigma^2\right)$ по μ и σ^2. Возьмём для этого частные производные и приравняем их нулю; сначала по μ:

$$\frac{\partial \log p\left(x_1, \ldots, x_n \middle| \mu, \sigma^2\right)}{\partial \mu} = \frac{1}{2\sigma^2} \sum_{i=1}^{n} 2\left(x_i - \mu\right).$$

Нам повезло: оказывается, решить уравнение $\frac{\partial \log p(x_1, \ldots, x_n | \mu, \sigma^2)}{\partial \mu} = 0$ относительно μ можно независимо от σ, и получится просто среднее значение x_i:

$$\mu_{\mathrm{ML}} = \bar{x} = \frac{1}{n} \sum_{i=1}^{n} x_i. \tag{3.17}$$

Теперь возьмём производную по σ^2 (да, прямо по σ^2, а не по σ — так удобнее):

$$\frac{\partial \log p\left(x_1, \ldots, x_n \middle| \mu, \sigma^2\right)}{\partial \sigma^2} = -\frac{n}{2\sigma^2} + \frac{1}{2\sigma^4} \sum_{i=1}^{n} \left(x_i - \mu\right)^2.$$

Теперь в это уравнение нужно подставить найденную выше $\mu_{\mathrm{ML}} = \bar{x}$ и решить:

$$\frac{1}{2\sigma^2_{\mathrm{ML}}} \left(\frac{1}{\sigma^2_{\mathrm{ML}}} \sum_{i=1}^{n} \left(x_i - \bar{x}\right)^2 - n \right) = 0,$$

то есть

$$\sigma^2_{\mathrm{ML}} = \frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})^2 . \tag{3.18}$$

Результат (3.18) можно переписать так:

$$\sigma^2_{\mathrm{ML}} = \frac{1}{n}\sum_{i=1}^{n}\left(x_i^2 - 2x_i\bar{x} + \bar{x}^2\right) = \frac{1}{n}\sum_{i=1}^{n}x_i^2 - 2\bar{x}\frac{1}{n}\sum_{i=1}^{n}x_i + \frac{n}{n}\bar{x}^2 =$$

$$= \frac{1}{n}\sum_{i=1}^{n}x_i^2 - 2\bar{x}^2 + \bar{x}^2 = \frac{1}{n}\sum_{i=1}^{n}x_i^2 - \left(\frac{1}{n}\sum_{i=1}^{n}x_i\right)^2 , \tag{3.19}$$

и правдоподобие нормального распределения (3.16) тоже можно записать как

$$\log p\left(x_1,\ldots,x_n\middle|\mu,\sigma^2\right) =$$

$$= -\frac{n}{2}\log 2\pi - \frac{n}{2}\log\sigma^2 - \frac{1}{2\sigma^2}\left(\sum_{i=1}^{n}x_i^2 - 2\mu\sum_{i=1}^{n}x_i + n\mu^2\right). \tag{3.20}$$

Формулы (3.19) и (3.20) показывают, что и правдоподобие, и оценки максимального правдоподобия у гауссиана зависят не от каждой из n точек данных по отдельности, а от двух агрегированных оценок: $\sum_{i=1}^{n}x_i$ (или $\bar{x}$) и $\sum_{i=1}^{n}x_i^2$. Такой набор величин называется *достаточными статистиками* (sufficient statistics): их достаточно знать, чтобы оценить все параметры, а сами точки запоминать не нужно. Тот или иной набор достаточных статистик есть у любого параметрического семейства распределений — на худой конец множество $\{x_1,\ldots,x_n\}$ можно считать достаточными статистиками, хотя обычно этот термин употребляется, когда число статистик постоянное, не зависящее от n. Например, для описания испытаний Бернулли достаточно знать, сколько выпало орлов и сколько решек. Мы подробно поговорим о достаточных статистиках в разделе 6.6.

Любопытно, что оценка максимального правдоподобия для дисперсии (3.18) является *смещённой* (biased). Это значит, что если мы породим множество разных наборов данных $D = \{x_1,\ldots,x_n\}$ из распределения $\mathcal{N}\left(\mu,\sigma^2\right)$ и подсчитаем оценки σ^2_{ML} по формуле (3.18), то среднее этих оценок не будет стремиться к σ^2, даже если число датасетов будет стремиться к бесконечности:

$$\mathbb{E}\left[\sigma^2_{\mathrm{ML}}\right] = \mathbb{E}\left[\frac{1}{n}\sum_{i=1}^{n}x_i^2\right] - \mathbb{E}\left[\bar{x}^2\right] = \mathbb{E}\left[x_i^2\right] - \left(\mathbb{E}\left[\bar{x}\right]^2 + \mathrm{Var}\left[\bar{x}\right]\right) =$$

$$= \left(\mu^2 + \mathrm{Var}\left[x_i\right]\right) - \left(\mu^2 + \frac{1}{n^2}\mathrm{Var}\left[\sum_{i=1}^{n}\right]\right) = \sigma^2 - \frac{n\sigma^2}{n^2} = \frac{n-1}{n}\sigma^2, \tag{3.21}$$

где мы несколько раз воспользовались тем, что $\mathrm{Var}[x] = \mathbb{E}\left[x^2\right] - \mathbb{E}[x]^2$, и вынесли константный сомножитель $\frac{1}{n}$ из дисперсии, возведя его при этом в квадрат.

Теперь формула (3.21) сама же и подсказывает, как модифицировать оценку (3.18), чтобы она стала несмещённой:

$$\hat{\sigma}^2_{\text{unbiased}} = \frac{n}{n-1}\sigma^2_{\text{ML}} = \frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2\,. \tag{3.22}$$

И последнее замечание: нам часто будут нужны не только одномерные непрерывные распределения, но и *многомерные*, заданные на евклидовом пространстве $\mathbb{R}^d$ и его подмножествах. К счастью, нам не потребуется детально вникать в многочисленные новые распределения, возникающие в многомерном случае.

Но в машинном обучении мы точно должны творчески владеть *многомерным нормальным распределением*, обобщением гауссиана в более высоких размерностях. Его параметры теперь превращаются в вектор среднего $\boldsymbol{\mu}$ и матрицу ковариаций Σ, а плотность задаётся выражением

$$\mathcal{N}(\boldsymbol{x}|\boldsymbol{\mu}, \Sigma) = \frac{1}{(2\pi)^{\frac{d}{2}}\sqrt{\det\Sigma}} e^{-\frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu})^\top \Sigma^{-1}(\boldsymbol{x}-\boldsymbol{\mu})}. \tag{3.23}$$

Базовые свойства здесь ожидаемые: математическое ожидание многомерного гауссиана равно $\boldsymbol{\mu}$, а Σ — это действительно матрица ковариаций. Главное, что нужно здесь понимать на будущее, — это то, что логарифм плотности гауссиана представляет собой квадратичную форму от $\boldsymbol{x}$; и наоборот, если где-то мы в логарифме плотности видим квадратичную функцию от $\boldsymbol{x}$, это, скорее всего, значит, что мы можем выделить там полный квадрат, превратив функцию в квадратичную форму вида $-\frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu})^\top \Sigma^{-1}(\boldsymbol{x}-\boldsymbol{\mu})$, и тем самым найти параметры этого многомерного гауссиана. Сразу несколько таких рассуждений мы проведём в разделе 4.4 для линейной регрессии. Кроме того, многомерные гауссианы тоже остаются нормальными распределениями и под действием линейных преобразований, и под действием, например, маргинализации: если проинтегрировать многомерный гауссиан по части переменных, то опять получится гауссиан.

Почему же нормальное распределение так важно? Главная причина — это результат, известный как *центральная предельная теорема* (central limit theorem). Она показывает, что сумма достаточно большого числа случайных величин будет иметь нормальное распределение.

Классическая центральная предельная теорема говорит о том, что для любой последовательности одинаково распределённых независимых случайных величин $X_1, X_2, \ldots, X_n, \ldots$ с конечным ожиданием μ и дисперсией σ^2

$$\frac{\sum_{i=1}^{n} X_i - \mu n}{\sigma\sqrt{n}} \to_{n\to\infty} \mathcal{N}(0,1)\,.$$

На самом деле центральные предельные теоремы — это целый класс результатов в теории вероятностей, в которых условия независимости и одинаковой распределённости случайных величин постепенно ослаблялись; не будем углубляться не только в их доказательства, но даже и в формулировки, я просто оставлю ссылки на стандартные курсы теории вероятностей [691, 693].

Для нас важно, что из-за этого свойства нормальное распределение появляется каждый раз, когда значение случайной величины является суммой большого числа отдельных воздействий. Те самые школьные физические эксперименты, в которых шарик скатывается с наклонной плоскости и проходит после этого некоторое расстояние, — идеальные примеры таких случайных величин.

Кстати, исторически нормальное распределение впервые появилось именно из ранних вариантов центральной предельной теоремы. Кого считать первооткрывателем, здесь дело вкуса: формально говоря, первым был де Муавр, который в своём труде «Доктрина случайностей» [404] обсуждал биномиальные коэффициенты. Де Муавр доказал, что средний член разложения $(a+b)^n$ имеет асимптотику $\frac{2^n}{\sqrt{2\pi n}}$, а также что логарифм отношения члена на расстоянии l от середины к среднему члену составляет $-\frac{2l^2}{n}$. Это обычно считается первым упоминанием нормального распределения, де Муавр в некотором смысле ввёл его как предел результатов испытаний Бернулли. Но сам де Муавр не рассматривал это как распределение вероятностей, он просто искал асимптотику биномиальных коэффициентов.

А полноценно нормальное распределение ввёл и подробно рассмотрел именно Карл Фридрих Гаусс в своём труде *Theoria combinationis observationum erroribus minimis obnoxiae* («Теория комбинаций наблюдений, подверженных наименьшим ошибкам») [208]. Он там много всего ввёл и описал: и метод наименьших квадратов, и вообще метод максимального правдоподобия, и собственно нормальное распределение.

Гаусс получил нормальное распределение именно из метода максимального правдоподобия, несколько необычным для нас образом. Предположим, что мы получили наблюдения $a_1, a_2, \ldots, a_n$ для некоторой неизвестной величины (числа) x, и задача состоит в том, чтобы найти наиболее вероятное значение x.

Мы хотели бы сделать это методом максимального правдоподобия, то есть максимизируя произведение некоторых плотностей φ:

$$\varphi(x-a_1)\varphi(x-a_2)\ldots\varphi(x-a_n)\longrightarrow_x \max.$$

И здесь Гаусс делает следующее предположение: самый естественный способ оценить величину по нескольким измерениям — это взять их арифметическое среднее. Так давайте потребуем, чтобы точкой максимума правдоподобия было именно среднее $\frac{1}{n}\sum_{i=1}^n a_i$, и посмотрим, какие условия это накладывает на φ.

Гаусс показал, что для ошибки измерения Δ единственный закон, который обеспечивает это свойство, выглядит как (в обозначениях Гаусса)

$$\varphi(\Delta) = \frac{h}{\sqrt{\pi}} e^{-hh\Delta\Delta},$$

где h — «мера точности наблюдений», то есть в наших современных обозначениях $h = \sqrt{\frac{\tau}{2}} = \sqrt{\frac{1}{2\sigma^2}}$.

Именно Гаусс первым начал рассматривать нормальное распределение как распределение для ошибок экспериментов, отклонений конкретных элементов выборки от теоретического значения — к этому мы вернёмся в разделе 4.4.

3.2.3. Логнормальное распределение и распределение Стьюдента

Выше мы обсуждали, что нормальное распределение по закону больших чисел получается в результате суммирования или усреднения большого числа случайных величин. А если случайные величины не суммируются, а перемножаются, то получается так называемое *логнормальное распределение* (log-normal distribution), то есть распределение случайной величины, чей логарифм распределён нормально:

$$X = e^{\mu+\sigma Z}, \quad \text{где} \quad Z \sim \mathcal{N}(0,1).$$

Логнормальное распределение ещё иногда называют распределением Гальтона, о котором мы ещё поговорим в разделе 4.1. Оно естественным образом возникает в результате мультипликативной центральной предельной теоремы: геометрическое среднее независимых одинаково распределённых случайных величин будет распределено логнормально; этот результат также известен как закон Гибрата, который в начале 1930-х годов исследовал это распределение в контексте пропорционального роста размера компаний или населения городов [217].

Чтобы найти плотность логнормального распределения, достаточно переписать её через функцию распределения:

$$p(x) = \frac{\mathrm{d}F(x)}{\mathrm{d}x} = \frac{\mathrm{d}}{\mathrm{d}x} p\left(X \le x\right) = \frac{\mathrm{d}}{\mathrm{d}x} p\left(\log X \le \log x\right) = \\ = \frac{\mathrm{d}}{\mathrm{d}x} \Phi\left(\frac{\log x - \mu}{\sigma}\right) = \frac{1}{\sigma x} \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2\sigma^2}(\ln x - \mu)^2}.$$

Несколько примеров плотности логнормальных распределений показаны на рис. 3.10; но оно нам в книге больше не понадобится, так что и обсуждать его подробно не будем.

А вот другое распределение, тесно связанное с гауссовским, нам ещё не раз пригодится. Это *распределение Стьюдента* (Student's t-distribution, или просто

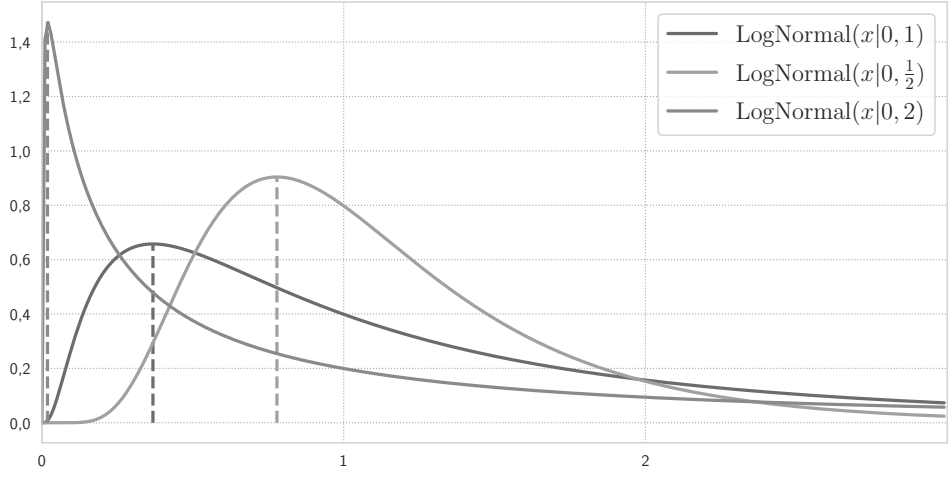


Рис. 3.10. Плотность одномерного логнормального распределения с разными μ и σ

t-distribution). У него один гиперпараметр — число степеней свободы ν, а плотность задаётся полиномиальной функцией степени $-(\nu+1)/2$:

$$p\left(x|\nu\right)=\frac{\Gamma\left(\frac{\mu+1}{2}\right)}{\sqrt{\mu\pi}\Gamma\left(\frac{\mu}{2}\right)}\left(1+\frac{t^2}{\nu}\right)^{-\frac{\nu+1}{2}}. \tag{3.24}$$

Это распределение, конечно, тоже можно сдвинуть (изменив среднее) и перемасштабировать (изменив дисперсию) стандартным преобразованием $t\to\frac{t-\mu}{\sigma}$:

$$p\left(x|\mu,\sigma,\nu\right)=\frac{\Gamma\left(\frac{\mu+1}{2}\right)}{\sqrt{\mu\pi\sigma^2}\Gamma\left(\frac{\mu}{2}\right)}\left(1+\frac{(t-\mu)^2}{\sigma^2\nu}\right)^{-\frac{\nu+1}{2}}. \tag{3.25}$$

Распределения из этого семейства возникают при оценке среднего по малой выборке из нормального распределения с неизвестной дисперсией. Если взять выборку из n величин $x_1,\ldots,x_n\sim\mathcal{N}\left(\mu,\sigma^2\right)$ и подсчитать статистики

$$\bar{x}=\frac{1}{n}\sum_{i=1}^{n},\qquad s^2=\frac{1}{n-1}\sum_{i=1}^{n}\left(x_i-\bar{x}\right)^2,$$

то величина

$$t=\frac{\bar{x}-\mu}{s/\sqrt{n}}$$

будет иметь распределение Стьюдента с $n-1$ степенью свободы.

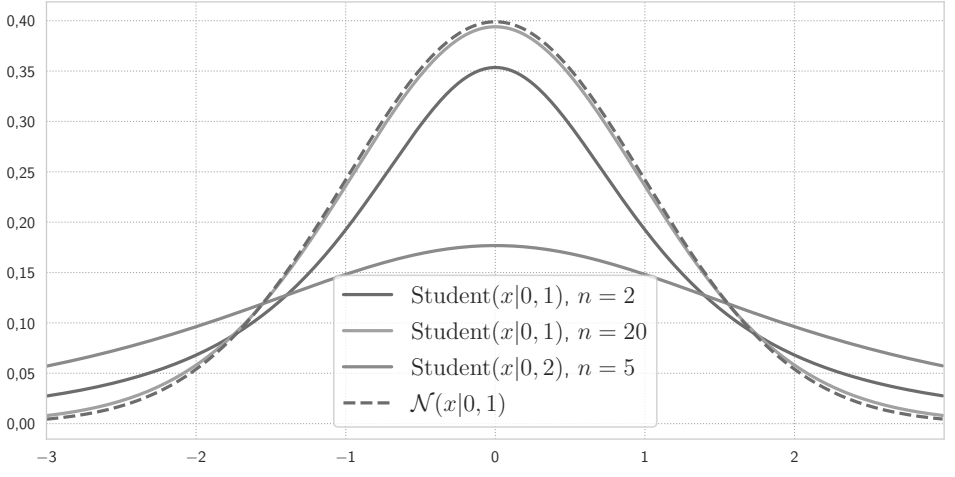


Рис. 3.11. Плотность распределения Стьюдента с разными значениями n, μ и σ

Практический смысл этого результата в том, что при оценке параметров нормального распределения в этой формуле мы не знаем μ, и распределение Стьюдента позволяет оценить доверительный интервал для оценки среднего нормального распределения μ по малой выборке. Например, если a — это константа, для которой $p\left(-a < x < a\right) = 0{,}9$ для распределения Стьюдента с n степенями свободы (скажем, для $n = 5$ это будет $a \approx 2{,}015$), то мы получим, что

$$p\left(\bar{x} - a\frac{s}{\sqrt{n}} < \mu < \bar{x} - a\frac{s}{\sqrt{n}}\right) = 0{,}9,$$

что и будет соответствующим доверительным интервалом для среднего μ по выборке из n элементов. В докомпьютерную эпоху таблицы квантилей распределения Стьюдента были верными спутниками протодатасайентистов.

На рис. 3.11 показаны несколько примеров распределений Стьюдента; при росте ν распределение Стьюдента стремится к стандартному нормальному распределению, которое там тоже изображено для сравнения. Но видно, что, даже если попытаться уравнять максимумы распределения Стьюдента и гауссиана, у последнего за счёт экспоненциального убывания будут гораздо более быстро затухающие «хвосты»; мы ещё увидим такие сравнения в разделе 6.4.

Многие знают историю появления распределения Стьюдента в работах британского статистика Уильяма Госсета[1], который публиковал свои исследования

[1] *Уильям Сили Госсет* (William Sealy Gosset, 1876–1973) — британский математик и статистик. Жизнь Госсета не была отмечена бурными событиями: он учился в Оксфорде, в 1899 году поступил на работу на пивоваренный завод Arthur Guinness Son & Co в Дублине, да так и работал там на

под псевдонимом Student [568]. Работы Госсета были связаны с тем, что он работал на пивоварне *Guinness* в Дублине, и данные исследований, которые ему приходилось обрабатывать, были очень маленькими: химические свойства ячменя, например, приходилось оценивать по выборке из трёх семплов.

А псевдоним Student появился потому, что работодатель запретил Госсету использовать своё настоящее имя: то ли сотрудникам вообще запрещались публикации в открытой печати, то ли компания *Guinness* не хотела, чтобы конкуренты поняли, что они используют разработанные Госсетом статистические методы. Название «распределение Стьюдента» и букву t (от слова *test*), которая участвует в английском названии, ввёл популяризировавший и развивший работы Госсета Рональд Фишер.

Однако любопытно, что на самом деле распределение Стьюдента впервые появилось не из статистических тестов, а именно из байесовского вывода для гауссиана. Этот вывод ещё в 1870-х годах провели немецкие учёные Фридрих Хельмерт [252, 253, 534] и Якоб Люрот [358]; мы с вами его подробно разберём в этой книге, в разделе 6.4.

3.2.4. Гамма-распределение и экспоненциальное распределение

Осталось обсудить ещё одно непрерывное распределение, которое будет задано не на всём $\mathbb{R}$, а только на положительной полуоси $x \in (0, \infty)$. В разделе 2.4 мы уже рассматривали бета-распределение, так что не буду повторяться, однако давайте обсудим другое распределение, нормировочная константа к которому представляет собой похожую неэлементарную функцию: *гамма-распределение* (gamma distribution). Оно задано на положительной полуоси, $x \in (0, \infty)$, у него два параметра — $\alpha > 0$ и $\beta > 0$, и его плотность выглядит как

$$\text{Gamma}\,(x|\alpha, \beta) = \frac{\beta^\alpha}{\Gamma(\alpha)} x^{\alpha-1} e^{-\beta x}.$$

Несколько вариантов этой плотности для разных α и β показаны на рис. 3.12.

Среднее и дисперсию гамма-распределения легко подсчитать, учитывая, что умножение плотности на x не выводит нас за пределы гамма-распределений; это значит, что можно применить тот же трюк, что с бета-распределением в разделе 2.4, воспользовавшись той же нормировочной константой с новыми параметрами:

различных должностях до самой смерти. По соображениям сохранения коммерческой тайны Гиннесс запретил публиковать результаты исследований, и почти все свои работы Госсет, включая свой главный труд *The Probable Error of a Mean* [568], опубликовал под псевдонимом *Student*. Впрочем, вряд ли он страдал от этого: Уильям Госсет был очень скромным человеком, умудрился стать другом и Рональду Фишеру, и Карлу Пирсону (что было очень нелегко!), а однажды на комплименты своим открытиям ответил так: «Фишер всё равно бы всё это открыл и сам».

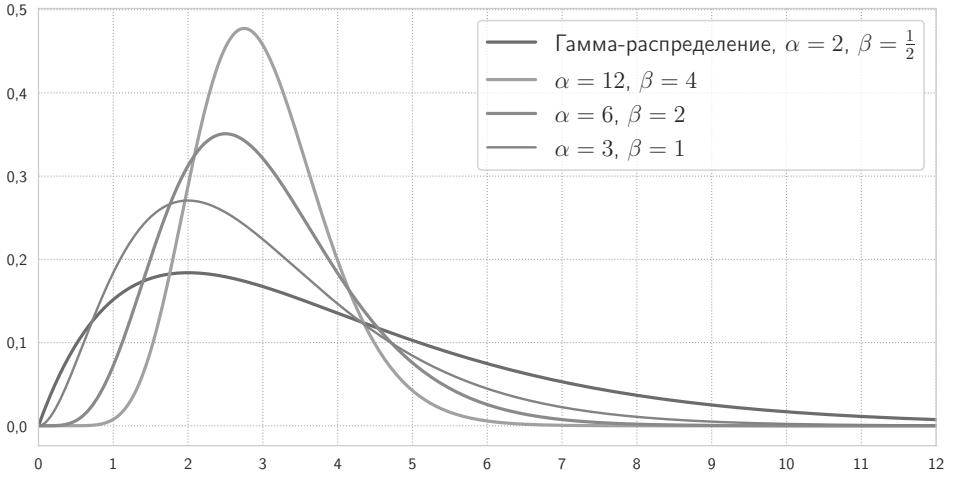


Рис. 3.12. Плотность гамма-распределения с разными значениями α и β

$$\mathbb{E}[x] = \int_0^\infty x \cdot \frac{\beta^\alpha}{\Gamma(\alpha)} x^{\alpha-1} e^{-\beta x} \mathrm{d}x = \frac{\beta^\alpha}{\Gamma(\alpha)} \int_0^\infty x^\alpha e^{-\beta x} \mathrm{d}x = \frac{\beta^\alpha}{\Gamma(\alpha)} \frac{\Gamma(\alpha+1)}{\beta^{\alpha+1}} = \frac{\alpha}{\beta},$$

$$\mathrm{Var}[x] = \mathbb{E}\left[x^2\right] - \mathbb{E}[x]^2 = \frac{\beta^\alpha}{\Gamma(\alpha)} \int_0^\infty x^{\alpha+1} e^{-\beta x} \mathrm{d}x - \frac{\alpha^2}{\beta^2} =$$

$$= \frac{\beta^\alpha}{\Gamma(\alpha)} \frac{\Gamma(\alpha+2)}{\beta^{\alpha+2}} - \frac{\alpha^2}{\beta^2} = \frac{\alpha(\alpha+1)}{\beta^2} - \frac{\alpha^2}{\beta^2} = \frac{\alpha}{\beta^2}.$$

В частности, на рис. 3.12 показаны несколько вариантов гамма-распределения с одним и тем же средним $\frac{\alpha}{\beta} = 3$, но разными соотношениями α и β. Обратите внимание, что меняется не только дисперсия, но и *мода* (mode) распределения, то есть максимум плотности: оставлю вам в качестве упражнения убедиться, что мода гамма-распределения находится в точке $\frac{\alpha-1}{\beta}$.

Важным частным случаем гамма-распределения является *экспоненциальное распределение*, плотность которого равна

$$\mathrm{Exp}(x|\lambda) = \lambda e^{-\lambda x}; \tag{3.26}$$

иначе говоря, это гамма-распределение с $\alpha = 1$. Оно всегда достигает максимума в $x = 0$ и равномерно экспоненциально убывает при росте x; несколько вариантов экспоненциального распределения показаны на рис. 3.13.

Экспоненциальное распределение моделирует время между двумя наступлениями одного и того же события при условии, что эти события происходят с некоторой интенсивностью независимо друг от друга. Формально говоря, рассмотрим последовательность событий, наступающих в случайные моменты времени *(поток событий)*, и предположим три естественных свойства:

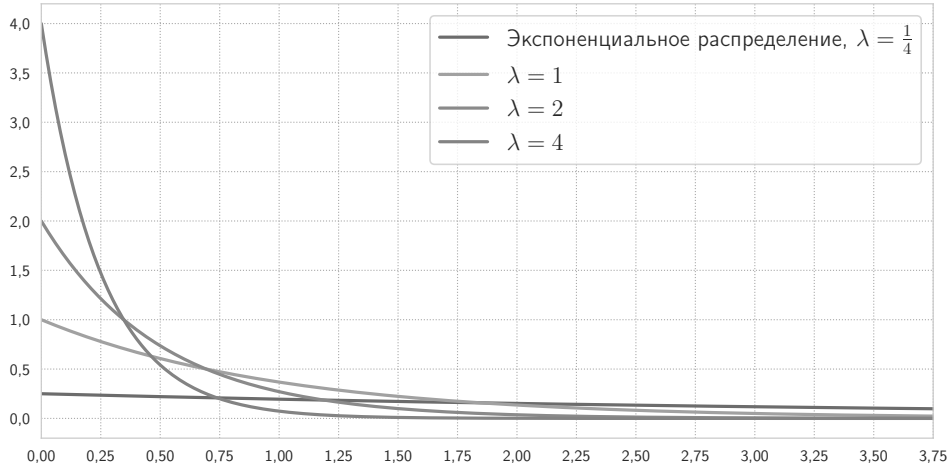


Рис. 3.13. Плотность экспоненциального распределения с разными значениями λ

- *стационарность:* вероятность появления k событий в некотором временном интервале $t \in [t_1, t_2]$ зависит от k и от длины интервала $t_2 - t_1$, но не зависит от того, когда этот интервал начинается;
- *ординарность:* два события не появляются одновременно в один и тот же момент времени t (точнее говоря, они могут появиться одновременно с вероятностью ноль, но нельзя сосредотачивать в этом событии ненулевую вероятность);
- *отсутствие последействия:* вероятность появления k событий в некотором промежутке времени $t \in [t_1, t_2]$ не зависит от того, сколько событий появилось вне промежутка, то есть до t_1 или после t_2.

Если эти предположения выполняются, перед нами *стационарный пуассоновский поток*, тот самый, который мы обсуждали в подразделе 3.1.3. Вероятность появления k событий потока за промежуток времени длительностью t определяется тем самым распределением Пуассона из подраздела 3.1.3

$$p\,(k|t) = \frac{1}{k!}\,(\lambda t)^k\, e^{-\lambda t},$$

где λ — параметр, который называют *интенсивностью* потока.

Оказывается, что в стационарном пуассоновском потоке время наступления следующего события распределено экспоненциально. Действительно, если обозначить через T время первого события в потоке, случайное событие «в пуассоновском потоке во время t не произошло ни одного события» — это ровно то же самое случайное событие, что и «$t < T$». Поэтому его функция распределения,

то есть вероятность $F(T)$ того, что следующее событие в потоке наступит не позже T, составляет

$$F(T) = 1 - p\,(0|T) = 1 - e^{-\lambda T}, \tag{3.27}$$

и, дифференцируя $F(T)$, мы получаем как раз плотность экспоненциального распределения (3.26).

Пуассоновские потоки возникают в жизни довольно часто: при моделировании реальных потоков входящих заявок в системах массового обслуживания (queueing systems), анализе сетевого трафика и так далее. Углубляться в теорию массового обслуживания (queueing theory) я в этой книге не буду, а просто оставлю несколько ссылок [309, 310, 670, 681, 682].

Если немного обобщить этот результат, мы снова вернёмся к гамма-распределению. Для этого обобщения давайте посмотрим на пуассоновский процесс с другой стороны: вместо того чтобы спрашивать, сколько раз событие произойдёт за фиксированный интервал времени, как в формуле (3.3), давайте спросим, сколько нужно ждать, прежде чем событие в пуассоновском процессе с интенсивностью λ произойдёт k раз.

Логика опять та же: если время T теперь обозначает момент наступления k-го события в потоке, то

$$F(T) = 1 - p\,(\text{событий до момента } T \text{ было } \leq k-1) = 1 - \sum_{i=0}^{k-1} \frac{1}{i!} e^{-\lambda T} (\lambda T)^i.$$

И теперь можно продифференцировать эту функцию распределения, получив телескопическую сумму, в которой сокращается всё, кроме последнего и первого члена, а первый оказывается нулевым:

$$\begin{aligned} p\,(T|k,\lambda) &= -\sum_{i=0}^{k-1} \left(-\lambda e^{-\lambda T} \frac{(\lambda T)^i}{i!} + \lambda i e^{-\lambda T} \frac{(\lambda T)^{i-1}}{(i-1)!} \right) = \\ &= \lambda e^{-\lambda T} \sum_{i=0}^{k-1} \left(\frac{(\lambda T)^{i-1}}{(i-1)!} - \frac{(\lambda T)^i}{i!} \right) = \\ &= \lambda \frac{1}{(k-1)!} e^{-\lambda T} \left(\lambda T\right)^{k-1}. \end{aligned} \tag{3.28}$$

Как видите, получилась в точности плотность гамма-распределения с параметрами $\alpha = k$ и $\beta = \lambda$.

Гамма-распределение напоминает бета-распределение и формой, и нормировочной константой, и подсчётом моментов — может быть, оно может иметь примерно те же функции в байесовском выводе? И действительно, семейство гамма-

распределений представляет собой семейство сопряжённых априорных распределений для распределения Пуассона $\text{Pois}(\lambda)$, которое мы рассмотрели в разделе 3.1. Если мы предполагаем, что точки в некотором датасете $D = \{x_n\}_{n=1}^N$ представляют собой натуральные числа, получившиеся из распределения Пуассона с параметром λ, то есть правдоподобие выглядит как

$$p\left(D|\lambda\right) = \prod_{n=1}^{N} p\left(x_n|\lambda\right) = \prod_{n=1}^{N} \frac{1}{x_n!}\lambda^{x_n}e^{-\lambda},$$

то мы можем ввести априорное гамма-распределение

$$p\left(\lambda\right) = \text{Gamma}\left(\lambda|\alpha_0,\beta_0\right) = \frac{\beta_0^{\alpha_0}}{\Gamma(\alpha_0)}\lambda^{\alpha_0-1}e^{-\beta_0\lambda},$$

и после умножения на правдоподобие апостериорное распределение будет выглядеть как

$$p\left(\lambda|D,\alpha_0,\beta_0\right) \propto \frac{\beta_0^{\alpha_0}}{\Gamma(\alpha_0)}\lambda^{\alpha_0-1}e^{-\beta_0\lambda}\prod_{n=1}^{N}\frac{1}{x_n!}\lambda^{x_n}e^{-\lambda} \propto$$

$$\propto \lambda^{\alpha_0+\sum_{n=1}^{N}x_n-1}e^{-(\beta_0+n)\lambda}.$$

Как видите, мы опять получили гамма-распределение, но с новыми параметрами:

$$p\left(\lambda|D,\alpha_0,\beta_0\right) = \text{Gamma}\left(\lambda\middle|\alpha_0+\sum_{n=1}^{N}x_n, \beta_0+n\right).$$

Похожий вывод мы ещё встретим в разделе 6.4.

Аналогично можно получить и ещё одно важное свойство, устанавливающее связь между рассмотренными выше дискретными распределениями. Оказывается, что отрицательное биномиальное распределение получается как *непрерывная смесь* пуассоновских распределений, если проинтегрировать интенсивность пуассоновского распределения по гамма-распределению; фактически это значит, что мы опять используем гамма-распределение в качестве априорного для пуассоновского.

Этот результат можно сначала обсудить «на пальцах», заодно выработав важную интуицию, которая может помочь окончательно разобраться с пуассоновскими процессами. Представим, что поток событий состоит из двух независимых пуассоновских процессов, один из которых выдаёт «успешные» события с некоторой интенсивностью $p \leq 1$, а другой выдаёт «неудачи» с интенсивностью $1-p$. Вместе эти два процесса создают поток событий, эквивалентный одному пуассоновскому потоку с интенсивностью 1, события которого представляют собой результаты бросков монетки с вероятностью орла (успеха) p.

Дальше рассуждения идут так:

- как мы обсуждали в подразделе 3.1.4, распределение числа успехов перед k-й неудачей имеет отрицательное биномиальное распределение;
- с другой стороны, это число событий в пуассоновском процессе интенсивностью p, которое случилось до момента времени T, когда у другого независимого пуассоновского процесса с интенсивностью $1-p$ набралось k событий; по результату (3.28) это гамма-распределение с параметрами k и $1-p$;
- значит, отрицательное биномиальное распределение — это пуассоновское распределение со средним pT, в котором параметр T имеет гамма-распределение с параметрами k и $1-p$, а значит, pT имеет гамма-распределение с параметрами k и $\frac{1-p}{p}$.

Формально это тоже легко доказать: непрерывную смесь, представляющую собой распределение на число случившихся событий i, можно подсчитать как

$$\int_0^\infty \text{Pois}\,(i|\lambda)\,\text{Gamma}\left(\lambda\middle|k,\frac{p}{1-p}\right)\mathrm{d}\lambda = \\ = \int_0^\infty \frac{1}{i!}\lambda^i e^{-\lambda}\frac{1}{\Gamma(k)}\left(\frac{p}{1-p}\right)^k \lambda^{k-1}e^{-\frac{p}{1-p}\lambda}\mathrm{d}\lambda = \\ = \left(\frac{p}{1-p}\right)^k \frac{1}{i!\Gamma(k)}\int_0^\infty \lambda^{i+k-1}e^{-\lambda\left(\frac{p}{1-p}+1\right)}\mathrm{d}\lambda.$$

В последнем выражении можно выделить гамма-распределение под интегралом, умножив и разделив на его нормировочную константу, и затем плотность проинтегрируется в единицу:

$$\int_0^\infty \text{Pois}\,(i|\lambda)\,\text{Gamma}\left(\lambda\middle|k,\frac{p}{1-p}\right)\mathrm{d}\lambda = \\ = \left(\frac{p}{1-p}\right)^k \frac{1}{i!\Gamma(k)}\Gamma(i+k)\,(1-p)^{i+k}\int_0^\infty \text{Gamma}\left(\lambda\middle|i+k,\frac{1}{1-p}\right)\mathrm{d}\lambda = \\ = \frac{\Gamma(i+k)}{i!\Gamma(k)}\,(1-p)^i\,p^k = \binom{i+k}{k}(1-p)^i\,p^k,$$

то есть в точности плотность (3.4) отрицательного биномиального распределения $\text{NegBinom}\,(i|k,p)$.

Итак, в этом разделе вы познакомились с основными непрерывными распределениями и даже рассмотрели кое-какие из них с точки зрения байесовского вывода. На этом распределений, наверное, хватит. Дальше мы рассмотрим первый развёрнутый пример этой главы: поговорим о том, как обучить, казалось бы, самое простое из распределений — равномерное. Оказывается, что даже для этого случая — а может быть, особенно для этого случая! — в обучении мы сможем найти много подводных камней и интересных результатов.

3.3. Немецкие танки, датская камбала и биномиальные обезьяны

Я поставил своей целью внушить как ветеранам, так и молодым солдатам, чтоб они задумались над этими вопросами, чтобы разобрались в них как можно глубже, а затем перешли к целенаправленным действиям; а ещё я надеюсь, что мой труд даст возможность нашей здоровой молодёжи мысленно представить образ наших танковых войск, а также научит их тому, как подчинить себе технические достижения настоящего времени и поставить их на службу Отечеству.

Гейнц Гудериан. Achtung — Panzer!

Однажды... израильское военное руководство разрешило мне осмотреть производственную линию танков «Меркава». Я спросил, сколько всего было произведено танков, и мне ответили, что эта информация засекречена. Это показалось мне забавным, ведь на каждом шасси стоял серийный номер.

Тревор Дюпюи, 1991, цит. по [291]

3.3.1. Выборка без замещения и военная разведка

Вы уже видели в разделе 2.4, что даже в байесовском анализе обычной монетки можно найти немало интересного. В этом разделе мы рассмотрим обычное дискретное равномерное распределение, то есть речь пойдёт даже не просто про обычную игральную кость, а про *честную* игральную кость. Может показаться, что здесь нет никакого простора для дискуссий и тем более для обучения, но мы попробуем поискать скрытые глубины. И достичь этих глубин нам помогут... датская камбала, дикие утки Соединённых Штатов и танки Третьего рейха. Столь разные примеры отражают два разных способа построить эксперимент, по которому мы оцениваем параметры равномерного распределения.

Первая часть этого раздела — *выборка без замещения*, но с серийными номерами. Здесь у нас очень серьёзный пример, из военной разведки. Одной из важнейших аналитических задач во время Второй мировой войны была оценка сил противника. Если удастся понять, сколько танков произвела Германия в этом году, станет ясно, можно ли атаковать имеющимися силами, какие направления Германия сможет удержать и так далее. Как это понять? Один путь — традиционные средства разведки: пытаться оценить число немецких танков там, где мы смогли

их увидеть, и как-то экстраполировать на всё остальное. Этим способом, конечно же, пользовались и СССР, и союзники.

Но нашёлся и другой, несколько неожиданный путь: предположим, что вы захватили немецкий танк (например, вывезли подбитый с поля боя, как это обычно и бывало). Оказалось, что на разных деталях танка дотошные немецкие производители... наносят серийные номера! Иначе говоря, мы можем посмотреть на, условно говоря, коленвал очередной «Пантеры» и узнать, что это коленвал «Пантеры» номер 273. Скорее всего, можно предполагать, что это случайный номер из текущей серии, так что кажется, что из этого мы должны суметь сделать какие-то выводы о том, сколько всего таких «Пантер» было произведено. Не будем углубляться в то, какие именно детали танков оказались для этого полезными, а с какими дело обстояло сложнее, но, в общем, нашлись и детали с неразрывными последовательностями номеров [506].

В результате мы приходим к задаче, которая на первый взгляд не выглядит сложной: даны несколько чисел, взятых из равномерного распределения на интервале $1, \ldots, N$, и требуется оценить N. Давайте начнём с самого простого случая, когда число только одно; иначе говоря, мы захватили немецкий танк с серийным номером m и хотим оценить, сколько всего их могло бы быть. Функция правдоподобия здесь выглядит очень просто:

$$p\left(m|N\right) = \begin{cases} \frac{1}{N}, & \text{если } N \geq m, \\ 0, & \text{если } N < m. \end{cases}$$

Получается, что гипотеза максимального правдоподобия — это $N = m$, и правдоподобие исхода m (и всех остальных) равно $\frac{1}{m}$. Естественно, точно такой же результат будет справедлив для любого числа наблюдений: оценка максимального правдоподобия будет равна максимуму из значений в данных.

И снова, как и в разделе 2.4, получается, что гипотеза максимального правдоподобия нас не радует даже чисто интуитивно: она представляет собой крайний случай, минимально возможное число N, и очевидно, что на самом деле мы не хотим предсказывать N как максимум из увиденных серийных номеров, истинное значение N должно быть больше. Такие оценки в математической статистике называют *смещёнными* (biased estimators): максимум из наблюдаемых значений даёт оценку параметра N, но ожидание этой оценки не равно N; в подразделе 3.2.2 вы уже видели смещённую оценку дисперсии нормального распределения.

Ошибка может быть достаточно большой; я проиллюстрирую это эмпирически. На рис. 3.14 показаны функции плотности максимума выборок размером k, состоящих из чисел от 1 до $N = 100$. Этот график можно породить примерно таким кодом (как обычно, я не буду приводить «обвязку» из импортируемых библиотек, создания `plt.figure` и так далее):

```
N, num_samples = 100, 1000000
sample = np.array([np.random.choice(N, size=10, replace=False) \
```

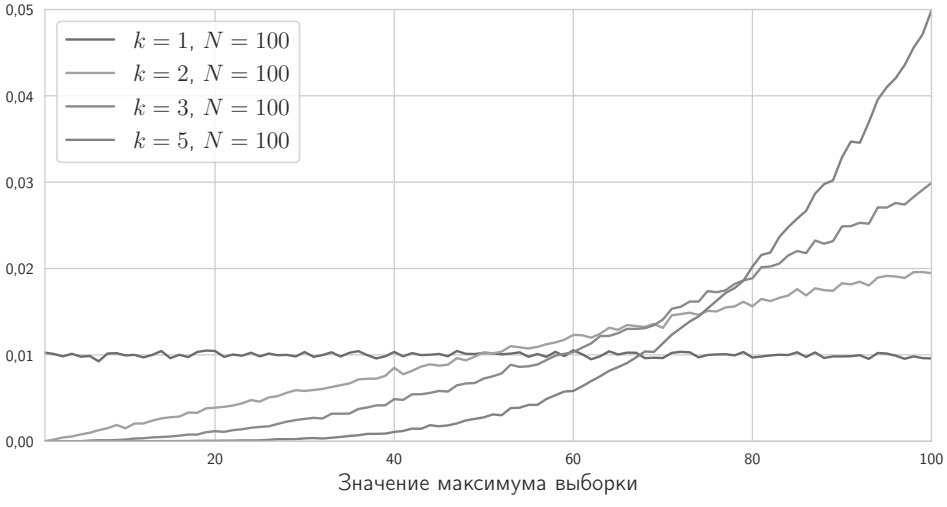


Рис. 3.14. Распределения максимумов выборки из равномерного распределения на числах от 1 до N; k — размер выборки

```
  + 1 for _ in range(num_samples)])
for k in (1, 2, 3, 5):
    hist = np.histogram(np.max(sample[:, :k], axis=1), bins=range(1, N+2))
    ax.plot(range(1, N+1), hist[0] / num_samples, \
      label=r'$k=%d$, $N=%d$' % (k, N))
```

Как видите, с ростом k ожидаемое значение максимума выборки действительно будет стремиться к N (в пределе это очевидно: если захватить *все* танки данной серии, то максимум выборки будет всегда равен N), но не очень быстро, и если мы смогли захватить всего несколько танков, то отклонение будет существенным. Средние значения для показанного на рис. 3.14 эксперимента получились такие: 50,49 для $k = 1$ (здесь очевидно, что теоретическое значение равно $\frac{101}{2} = 50{,}5$), 67,19 для $k = 2$, 75,5 для $k = 3$ и 83,8 для $k = 5$, то есть ожидаемое значение результата такой оценки существенно меньше, чем $N = 100$.

Что же делать? Ответ, который предлагает классическая статистика, состоит в том, чтобы найти *несмещённую* оценку для числа N чисто комбинаторным образом; для нашей дискретной задачи это вполне возможно и даже не так уж сложно. Чтобы это сделать, нужно найти распределение максимума M из выборки, то есть оценить $\mathbb{E}\,[M|N]$, а затем подставить в качестве этого ожидания эмпирический максимум $m = \max_{i=1}^{k} x_i$ и решить уравнение относительно N. Тем самым мы найдём несмещённую оценку $\hat{N}$.

Для $k = 1$ (одно число в выборке) это совсем просто: ожидание максимума совпадает с ожиданием одного x_i и равно $\frac{N+1}{2}$, а значит, $\hat{N} = 2m - 1$.

Это звучит вполне интуитивно: мы ожидаем, что равномерно взятое случайное число в среднем попадёт в середину интервала, значит, мы ожидаем, что сам интервал будет примерно вдвое шире этого числа.

Для $k > 1$ придётся всё-таки заняться комбинаторикой; давайте разберём этот случай как пример таких рассуждений. Начнём с распределения $p(M|N)$:

$$p(M = m|N) = \frac{\text{число выборок из } k \text{ чисел с максимумом } m}{\text{общее число выборок из } k \text{ чисел}}.$$

В формуле выше в знаменателе стоит число сочетаний из N по k, $\binom{N}{k}$, а в числителе, соответственно, число сочетаний из $m-1$ по $k-1$: мы уже зафиксировали одно из чисел m, и это максимум, то есть остальные $k-1$ число должны быть взяты от 1 до $m-1$. Итого получается, что

$$p(M = m|N) = \frac{\binom{m-1}{k-1}}{\binom{N}{k}}. \tag{3.29}$$

А ожидание максимума, значит, равно

$$\mathbb{E}[M|N] = \sum_{m=k}^{N} mp(M = m|N) = \sum_{m=k}^{N} m\frac{\binom{m-1}{k-1}}{\binom{N}{k}} = \frac{1}{\binom{N}{k}} \sum_{m=k}^{N} m\binom{m-1}{k-1} =$$
$$= \frac{1}{\binom{N}{k}} \sum_{m=k}^{N} m\frac{(m-1)!}{(k-1)!(m-k)!} = \frac{k}{\binom{N}{k}} \sum_{m=k}^{N} \frac{m!}{k!(m-k)!} = \frac{k}{\binom{N}{k}} \sum_{m=k}^{N} \binom{m}{k}.$$

В рассуждении выше мы в предпоследнем равенстве умножили и разделили на k, чтобы привести сумму к более красивой форме, выделив биномиальный коэффициент под суммой. Осталось воспользоваться широко известным тождеством о биномиальных коэффициентах:

$$\sum_{a=c}^{b} \binom{a}{c} = \binom{b+1}{c+1}. \tag{3.30}$$

Его легко доказать индукцией по b. База индукции состоит в том, что для $b = c$ слева и справа стоит единица, а если мы уже доказали тождество для некоторого b, то индукционный переход выглядит так:

$$\sum_{a=c}^{b+1} \binom{a}{c} = \sum_{a=c}^{b} \binom{a}{c} + \binom{b+1}{c} = \binom{b+1}{c+1} + \binom{b+1}{c} = \binom{b+2}{c+1},$$

где последнее равенство — это базовое свойство треугольника Паскаля (проверьте его самостоятельно, если не знаете).

Применяя доказанное тождество (3.30), получим, что

$$\mathbb{E}[M|N] = \frac{k}{\binom{N}{k}}\binom{N+1}{k+1} = k\frac{k!(N-k)!}{N!}\frac{(N+1)!}{(k+1)!(N-k)!} = \frac{k(N+1)}{k+1}. \quad (3.31)$$

Получилась очень простая формула, которую легко теперь и обратить:

$$\hat{N} = m\left(1+\frac{1}{k}\right) - 1 = m + \left(\frac{m}{k} - 1\right). \quad (3.32)$$

Этот результат опять же достаточно интуитивен: к максимуму нужно добавить «бонус», который уменьшается с ростом k. И с экспериментальными результатами на рис. 3.14 формулы (3.31) и (3.32) тоже сходятся: для $k = 2$ получается $\mathbb{E}[M|N] = 101 \cdot \frac{2}{3} = 67\frac{1}{3}$, для $k = 3$ $\mathbb{E}[M|N] = 101 \cdot \frac{3}{4} = 75{,}75$, а для $k = 5$ $\mathbb{E}[M|N] = 101 \cdot \frac{5}{6} = 84\frac{1}{6}$, что вполне соответствует приведённым выше цифрам.

На рис. 3.15 полученные оценки показаны на примере того же эксперимента, что на рис. 3.14, только вместо самих значений максимумов изображены значения $\hat{N}$ согласно формуле (3.32):

```
N, num_samples = 100, 1000000
sample = np.array([np.random.choice(N, size=10, replace=False) \
  + 1 for _ in range(num_samples)])
for k in (1, 2, 3, 5):
    max_hist = Counter(np.max(sample[:, :k], axis=1)*(k+1)/k - 1)
    xs = sorted(max_hist.keys())
    ys = [max_hist[i]/ num_samples for i in xs]
    ax.plot(xs + [xs[-1]+1], ys + [0], label=r'$k=%d$, $N=%d$' % (k, N))
```

Как видите, теперь можно поверить, что среднее всех этих распределений действительно похоже на 100; если подсчитать это среднее эмпирически, для данных четырёх значений k получится 99,96, 99,76, 99,67 и 99,58 соответственно.

3.3.2. Байесовский анализ и некорректные априорные распределения

А что же наш любимый байесовский подход? Его здесь, конечно, тоже можно применить: мы хотим найти

$$p(N|M,k) = \frac{p(M|N,k)\,p(N|k)}{p(M|k)}.$$

Здесь мы уже подсчитали в формуле (3.29), что $p(M = m|N) = \binom{m-1}{k-1}/\binom{N}{k}$. Знаменатель мы будем считать как нормировочную константу

$$p(M|k) = \sum_{N=M}^{\infty} p(M|N,k)\,p(N|k).$$

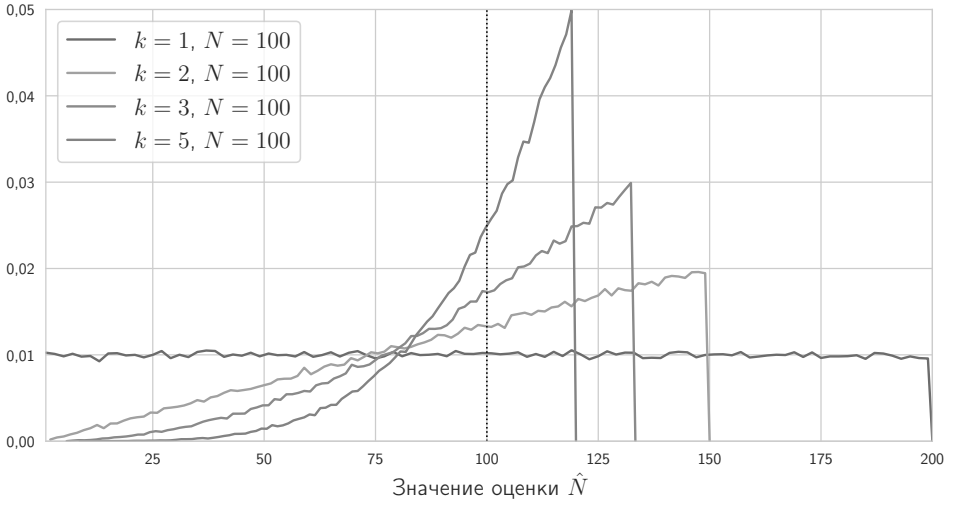


Рис. 3.15. Распределения оценки $\hat{N}$ на числах от 1 до N; k — размер выборки

Чтобы довести рассуждение до конца, придётся выбрать то или иное априорное распределение $p(N|k)$. Здесь появляется нетривиальный вопрос: какое априорное распределение взять? Мы бы хотели отразить в нём отсутствие информации о значении N, но равномерное распределение, как для параметра монетки в разделе 2.4, выбрать не получится, ведь нам нужно распределение на всех натуральных числах. Более серьёзный разговор о том, как выразить своё незнание, отложим до раздела 3.4, а пока просто рассмотрим несколько вариантов.

Во-первых, отметим, что приведённые выше рассуждения тоже можно выразить в виде применения теоремы Байеса. Для этого нужно взять *равномерное* априорное распределение $p(N|k)$ на большом объемлющем интервале $k, \ldots, \Omega$ (меньше k номеров при k наблюдениях получиться не может):

$$p(N|k) = \frac{1}{\Omega - k + 1} \quad \text{для всех} \quad k \le N,\, N \le \Omega.$$

Соответственно, получается, что для $k \le N$ и $N < \Omega$

$$p(N|M,k) = \frac{p(M|N,k)\,p(N|k)}{\sum_{n=M}^{\Omega} p(M|n,k)\,p(n|k)} = \frac{\frac{1}{\Omega-k+1} p(M|N,k)}{\sum_{n=M}^{\Omega} \frac{1}{\Omega-k+1} p(M|n,k)} =$$

$$= \frac{\binom{M-1}{k-1} / \binom{N}{k}}{\sum_{n=M}^{\Omega} \binom{M-1}{k-1} / \binom{n}{k}} = \frac{\binom{N}{k}^{-1}}{\sum_{n=M}^{\Omega} \binom{n}{k}^{-1}}, \quad (3.33)$$

а для других значений N эта вероятность равна нулю.

Здесь возникает ещё один интересный вопрос: каким выбрать верхний предел Ω? Конечно, в практической задаче о немецких танках можно было бы просто взять некоторую заведомо чрезмерно большую границу вроде $\Omega = 10^6$, но нельзя ли решить этот вопрос более универсально и концептуально? Оказывается, что технически препятствий к тому, чтобы положить $\Omega = \infty$, в формулах выше нет: если $k \geq 2$ (то есть мы пронаблюдали хотя бы два серийных номера — давайте сделаем это не слишком смелое предположение), то ряд $\sum_{n=M}^{\infty} \binom{n}{k}^{-1}$ сходится, ведь он по порядку величины похож на сумму величин, обратных n^k. А значит, мы можем подставить $\Omega = \infty$ в формулу (3.33). Но возникает проблема именно концептуального характера: какое же мы взяли априорное распределение? Получается, что мы выбрали в качестве $p(N|k)$ равномерное распределение на счётном множестве чисел от k до бесконечности, что, конечно, не будет настоящим, корректным распределением вероятностей.

Тем не менее такие ситуации встречаются в машинном обучении и байесовской статистике сплошь и рядом; они называются *некорректными априорными распределениями* (improper priors) [50, 87]. В данном примере некорректность выражается в том, что мы присваиваем бесконечному числу точек равную вероятность, то есть суммарная вероятность получается бесконечной. Можно рассмотреть и непрерывный аналог такого распределения: что, если мы хотим обучить какой-нибудь непрерывный параметр так, чтобы априори он мог равновероятно оказаться в любой точке вещественной оси? Для этого можно рассмотреть некорректное априорное распределение, которое присваивает равный вес каждому $x \in \mathbb{R}$; о нём можно думать, например, как о пределе нормального распределения, дисперсия которого уходит на бесконечность. Но эта интуиция всё равно не слишком поможет в вычислениях: по сути, такое некорректное априорное распределение просто означает, что мы считаем апостериорное распределение совпадающим с правдоподобием (с точностью до нормализации).

Конечно, не все некорректные априорные распределения обязаны быть равномерными, смысл здесь просто в том, что некорректное распределение не удаётся нормализовать так, чтобы оно суммировалось в единицу. Например, вспомним раздел 2.4, в котором мы обсуждали байесовский вывод для монетки. Самым «честным» априорным распределением мы тогда полагали равномерное, $B(1,1)$, но оно всё-таки предполагает некоторую дополнительную информацию для вывода, что отражается и в физическом смысле эквивалентного размера выборки, который мы обсуждали там же: априорное распределение $B(1,1)$ эквивалентно предварительным экспериментам с монеткой, в которых выпали один орёл и одна решка. Но что, если мы не хотим добавлять ничего лишнего к результатам, отражённым в данных, то есть к правдоподобию (и да, хотим получить все те странные эффекты на малых выборках, которые не нравились нам в разделе 2.4)? Тогда, казалось бы, нужно выбрать априорное распределение $B(0,0)$, то есть

$$p(\theta|0,0) \propto \frac{1}{\theta}\frac{1}{1-\theta} \quad \text{для} \quad \theta \in (0,1).$$

Это было бы обычным бета-распределением, но это некорректное распределение, интеграл от него по отрезку [0,1] бесконечен. Тем не менее для байесовского вывода его вполне можно использовать: при умножении на правдоподобие с n орлами и m решками получится апостериорное распределение

$$p(\theta|D) \propto \theta^{n-1}(1-\theta)^{m-1},$$

то есть как раз обычное бета-распределение $B(n,m)$, как мы и хотели. Есть и более сложные примеры некорректных априорных распределений, и попытки ослабить аксиомы классической теории вероятностей так, чтобы некорректные распределения стали «корректными» [587], но они нам пока не понадобятся.

Итак, вернёмся к нашим танкам. Применив некорректное априорное распределение с $\Omega = \infty$, мы приходим к апостериорному распределению

$$p(N|M,k) = \frac{\binom{N}{k}^{-1}}{\sum_{n=M}^{\infty}\binom{n}{k}^{-1}}.$$

Как его подсчитать? Оказывается, что найти сумму ряда в знаменателе не так-то просто и требует экскурса в теорию гипергеометрических функций, которые нам в этой книге больше никогда не понадобятся. Поэтому я просто оставлю ссылку на подробный вывод [269], а здесь приведу результат:

$$\sum_{n=M}^{\infty}\binom{n}{k}^{-1} = \binom{M}{k}^{-1}\frac{M}{k-1}, \quad \text{а значит,}$$

$$p(N|M,k) = \frac{k-1}{M}\binom{M}{k}\binom{N}{k}^{-1} \quad \text{для} \quad N = M, M+1, \ldots. \tag{3.34}$$

Как мы уже обсуждали выше, результат имеет смысл только для $k > 1$. Распределение (3.34) — это не слишком часто встречающееся, но всё-таки известное *факториальное распределение* (точнее, здесь $N-k$ имеет факториальное распределение) [376], и его среднее и дисперсия известны:

$$\mathbb{E}[N|M,k] = \frac{k-1}{k-2}(M-1) \quad \text{для} \quad k > 2,$$
$$\mathrm{Var}[N|M,k] = \frac{(k-1)(M-1)(M-k+1)}{(k-2)^2(k-3)} \quad \text{для} \quad k > 3.$$

Распределения $p(N|M,k)$ для разных значений M и k показаны на рис. 3.16.

Давайте проверим полученный результат (3.34) на практике. Для этого нам нужно породить выборку, которая бы отражала распределение $p(N|M,k)$. Мы не можем сделать это напрямую, но легко можем породить «обратную» выборку так

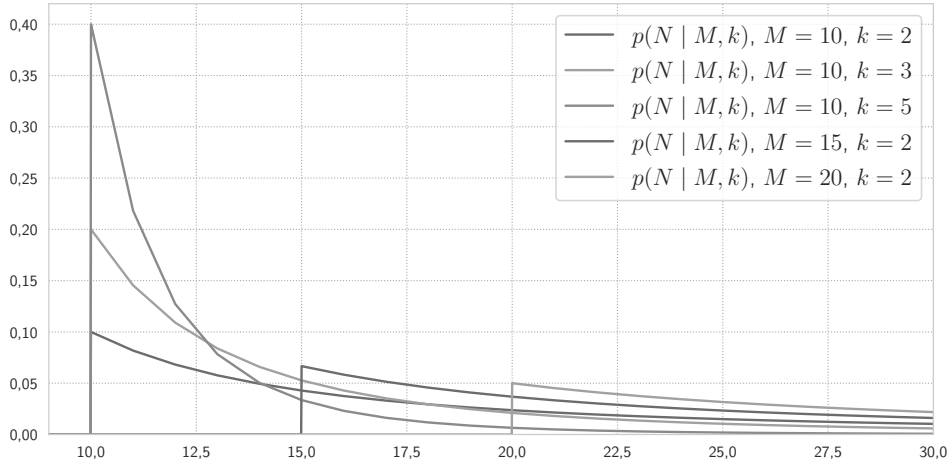


Рис. 3.16. Распределения $p\,(N|M,k)$ для разных значений M и k

же, как мы это делали для рис. 3.14: нужно зафиксировать N, M и k, породить выборку из чисел от 1 до N без замещения и выбрать максимум этой выборки. Но теперь нам нужно рассмотреть результат как распределение N для фиксированных M и k, поэтому давайте зафиксируем $M = 10$ и $k = 2$, породим такие выборки для разных N (в этом примере — для $N = 10, \ldots, 50$) и «перевернём» результат:

```
M, k, Nmax, num_samples = 10, 2, 50, 5000
counter = {}
for N in range(10, Nmax+1):
    sample = np.array([np.random.choice(N, size=k, replace=False) + 1 \
      for _ in range(num_samples)])
    sample_max = np.max(sample, axis=1)
    for i in sample_max:
        counter[(i, N)] = counter.get((i, N), 0) + 1
```

Теперь словарь `counter` по ключу (M, N) содержит число раз, которое в нашей выборке для данного N получился максимум M, из общего числа в `num_samples` семплов с данным N. Чтобы получить из него нужное распределение, нужно выбрать разные N для фиксированного M и нормировать результат:

```
result = {x[1]:v for x,v in counter.items() if x[0] == M}
result_total = np.sum([v for v in res.values()])
xs = list(np.arange(M, Nmax+1))
ys = [result.get(x, 0) / result_total for x in xs]
```

Теперь можно нарисовать график по точкам `xs` и `ys`; он показан на рис. 3.17. Выглядит похоже... но всё-таки не совсем похоже, кажется, будто эмпирическое

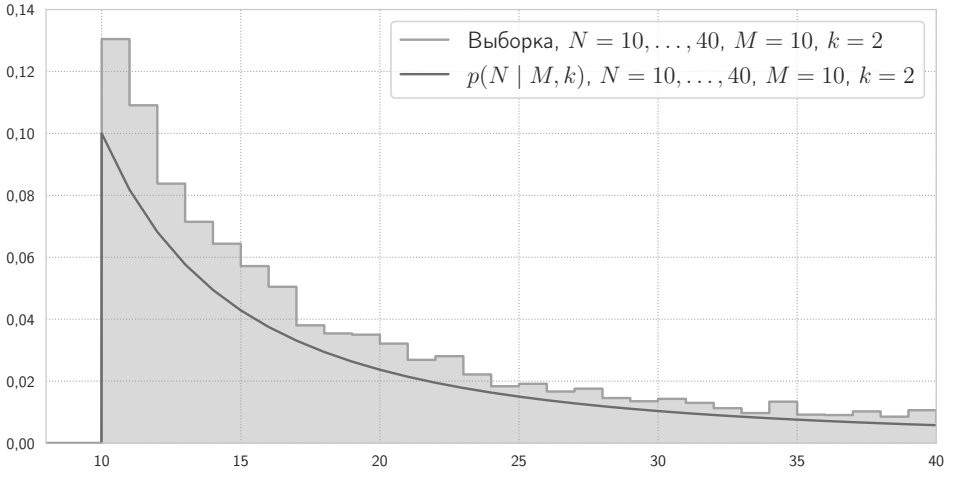


Рис. 3.17. Экспериментальная проверка (3.34): гистограмма выборочных значений и график плотности $p\,(N|M,k)$

распределение не совпадает с теоретической плотностью, и если вы будете увеличивать `num_samples`, выборочные значения действительно не сойдутся к теоретической кривой $p\,(N|M,k)$. Сможете догадаться, в чём дело?

Дело в том, что наш метод порождения выборки не совсем соответствует кривой $p\,(N|M,k)$. Теоретическая кривая соответствует случаю, когда N может принимать значения от 10 до бесконечности, а мы порождали данные для N от 10 до 40. Это значит, что в нашей выборке не могло встретиться никакое значение $N > 40$, а кривая $p\,(N|M,k)$ идёт до бесконечности — разумеется, выборочное распределение на интервале $[10, 40]$ будет больше, оно ведь должно суммироваться в единицу на этом интервале! Как исправить это несоответствие? Мы не сможем довести цикл в приведённом выше коде до бесконечности; но можем зайти с другой стороны: давайте обрежем распределение $p\,(N|M,k)$ и нормируем его на интервал $N \in [10, 40]$, то есть давайте нарисуем $\frac{p(N|M,k)}{\sum_{N=10}^{40} p(N|M,k)}$ на этом интервале. Результат показан на рис. 3.18, и вот здесь уже выборочные значения действительно весьма точно соответствуют теоретическим.

Во время Второй мировой эти методы сработали великолепно. Разведка регулярно поставляла существенно завышенные цифры немецкого производства, причём эти оценки были неточны и среди союзников, и в Советском Союзе [629]. Тогда вышеописанные статистические методы применили к данным о серийных номерах деталей захваченных танков. И выяснилось, что статистика на протяжении всей войны давала куда более точные результаты, чем методы традиционной разведки! Например [506]:

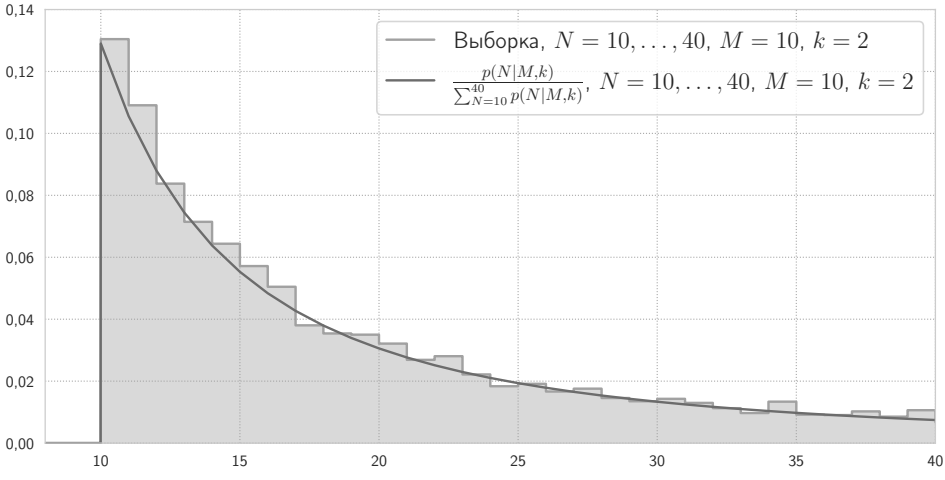


Рис. 3.18. Экспериментальная проверка (3.34): выборочные значения и плотность $p\,(N|M,k)$, нормированная на интервале $[10, 40]$

- в июне 1940 года разведданные союзников оценивали немецкое производство в 1000 танков; статистическая оценка по серийным номерам дала результат 169 танков (разница в шесть раз!); а уже после войны в архивах министра вооружения Альберта Шпеера нашлись настоящие цифры: было произведено 122 танка;
- в июне 1941 года разведка сообщала о 1550 произведённых танках, аналитики называли число 244, а по архивным данным выяснилось, что был произведён 271 танк;
- в августе 1942 года оценка разведки не изменилась — 1550 танков, предсказания по серийным номерам давали оценку в 327, а истинное число составило 342 танка.

И так далее; вам могло показаться, что дисперсия у приведённых выше статистических оценок довольно велика, но на самом деле другие методы оценивания давали результат, отличающийся в разы, и ошибка в 20–30 % на этом фоне была практически пренебрежимо малой.

Разумеется, до того, как война закончилась и союзники получили данные об истинном положении дел, невозможно было узнать наверняка, насколько точны оказались статистические оценки, так что оценки разведки тоже приходилось принимать во внимание. Но, как видите, апостериорно стало ясно, что оценки равномерного распределения уверенно победили.

3.3.3. Как правильно ловить уток

Теперь давайте перейдём ко второму методу, более простому. Он появился в биологических исследованиях конца XIX — начала XX века (сейчас мы назвали бы их экологическими, но тогда такие слова употребляли редко): как оценить популяцию того или иного вида в данном регионе? Например, сколько уток водится в Соединённых Штатах Америки? Мы можем найти или даже поймать несколько уток, возможно даже несколько сотен... но что это нам даст? Если бы мы знали, что нашли каждую сотую утку, ответ было бы легко получить, но нам же это неизвестно; что делать?

Биологи придумали для этого метод «отметить и снова поймать» (mark and recapture). Давайте будем ловить уток не один раз, а *два*: сначала поймаем некоторое подмножество уток (выборку из генеральной совокупности), окольцуем их и выпустим, а потом, спустя какое-то время, поймаем другую выборку и посмотрим, сколько среди них попадётся окольцованных. Кольцевание здесь можно заменить на любую отметку, в том числе и вовсе не требующую никого ловить: например, львов можно весьма надёжно идентифицировать по расположению точек, где растут их усы, так что для подсчёта повторяющихся львов нужно будет просто сделать достаточно хорошие фотографии. И тогда можно будет построить оценку общего числа, составив простую пропорцию:

$$\frac{\text{Число снова пойманных}}{\text{Размер второй выборки}} = \frac{\text{Число отмеченных}}{\text{Полный размер выборки}},$$

то есть

$$\text{Полный размер выборки} = \frac{\text{Размер второй выборки} \times \text{Число отмеченных}}{\text{Число снова пойманных}}. \qquad (3.35)$$

Эта нехитрая идея носит название *метода Линкольна — Петерсена* (Lincoln — Petersen method), потому что впервые его применили сначала датский морской биолог Карл Георг Йоханнес Петерсен, который в 1896 году подсчитал этим методом популяцию камбалы [463], а затем и американский орнитолог Фредерик Чарльз Линкольн, который организовывал всеамериканскую программу кольцевания птиц и, в частности, в 1930 году оценил этим методом популяцию диких уток в США [346].

Кажется, что тут уже трудно что-то добавить, но давайте хотя бы вкратце проанализируем этот метод. Попробуем перечислить предположения, которые здесь явно и неявно делаются.

1. Популяция предполагается *замкнутой*, то есть мы предполагаем, что генеральная совокупность, из которой выбираются особи, остаётся постоянной между двумя экспериментами. По сути это значит, что мы предполагаем, что птицы не рождаются и не умирают во время эксперимента. Точнее говоря, смерти могут и не помешать — если отметки не мешают жить, то отмеченные и неотмеченные особи будут умирать с одинаковой вероятностью

и пропорция не изменится; а вот рождаться новые особи точно будут без наших отметок.

2. Предполагается, что обе выборки делаются *равномерно*, то есть и в первой выборке, и во второй особи берутся из всей генеральной совокупности с одинаковыми вероятностями. Обратите внимание, что на практике это требование противоречит первому: чтобы выборка, скажем, перелётных птиц снова стала равномерной, нужно, чтобы прошло достаточно много времени и окольцованные птицы «перемешались» с остальными. Но чем дольше мы ждём, тем больше птиц за это время погибнут, родятся или ещё как-то изменят выборку (например, улетят за пределы региона измерений).
3. Равномерность выборки означает и ещё одно предположение: даже в уже «хорошо перемешанной» совокупности мы ещё должны предположить, что каждая особь попадётся в наши сети с одной и той же вероятностью. Но в жизни одних особей легче поймать, чем других: они медленнее, или глупее, или более доверчивы. Это значит, что такие особи будут непропорционально представлены и в первой выборке, и во второй — а значит, наша оценка окажется заниженной; интуитивно это можно представить как оценку неравномерной смеси двух выборок, маленькой и большой.
4. Отметки предполагаются постоянными, то есть птица не может взять пример с Фродо Бэггинса и сбросить или уничтожить своё кольцо.

Немало получилось предположений, правда? Большая их часть касается скорее практически-биологической стороны вопроса, и в реальности приходится тщательно исследовать конкретный способ отмечать особей на предмет соответствия этим предположениям.

Но есть и математическая проблема. Дело в том, что метод Линкольна — Петерсена сразу несколько раз использует оценку вероятности через отношение двух выборок $p \approx \frac{n}{N}$. Это хорошо работает, если выборки большие, но для малых n и N работать перестаёт. Как обычно, эффект легко увидеть в крайнем случае: предположим, что во второй выборке мы не нашли *ни одной* окольцованной птицы. Наверное, это должно нам что-то говорить о том, что общее число птиц достаточно велико, но формула (3.35) будет давать буквально бесконечный ответ, что очевидно неверно.

Чтобы решить эту проблему, можно попробовать сделать примерно то же, что мы делали в разделе 2.4 с монеткой: перейти к полноценному байесовскому выводу и найти апостериорное распределение. Сейчас мы поговорим только о базовой постановке задачи, известной с 1970-х годов [91, 207], но есть у неё и различные обобщения, например на множественные выборки, когда мы несколько лет подряд окольцовываем птиц и заодно подсчитываем, сколько и каких именно окольцованных птиц нам попалось [210].

Итак, пусть у нас есть популяция из неизвестного числа N особей, и K из них были отмечены. Во второй раз мы поймали n особей, и k из них оказались отмеченными. Задача состоит в том, чтобы по известным K, n и k оценить N.

Во-первых, давайте запишем правдоподобие. Здесь мы в точности попадаем в определение гипергеометрического распределения: нам нужно получить k успехов в выборке из n элементов без замещения из конечной популяции размера N с K успешными вариантами. Значит,

$$p\left(k|N,K,n\right) = \frac{\binom{K}{k}\binom{N-K}{n-k}}{\binom{N}{n}}.$$

Какое здесь выбрать априорное распределение, совершенно неясно, так что давайте попробуем снова воспользоваться неинформативным априорным распределением и просто предположить, что апостериорное распределение $p\left(N|k\right)$ при фиксированных K и n будет пропорционально правдоподобию:

$$p\left(N|k,K,n\right) = \frac{p\left(k|N\right)}{\sum_{M=k+n-k}^{\infty} p\left(k|M\right)} = \frac{\binom{K}{k}/\binom{N-K}{n-k}\binom{N}{n}}{\sum_{M=k+n-k}^{\infty}\binom{K}{k}\binom{M-K}{n-k}/\binom{M}{n}}. \quad (3.36)$$

Ряд в знаменателе здесь сходится для $k \geq 2$, и моменты апостериорного распределения опять можно подсчитать точно при помощи теории гипергеометрических функций, в которую мы углубляться не будем. В итоге получается ([283]; см. также [510]), что

$$\mathbb{E}\left[N|k,K,n\right] = \frac{(K-1)(n-1)}{(k-2)} \quad \text{для} \quad k > 2,$$
$$\mathrm{Var}\left[N|k,K,n\right] = \frac{(K-1)(n-1)(K-k+1)(n-k+1)}{(k-2)^2(k-3)} \quad \text{для} \quad k > 3.$$

Как и в случае танков, мы можем проиллюстрировать наш результат экспериментом. Теперь мы немного по-другому порождаем выборку: для популяции из N особей нужно сначала зафиксировать K отмеченных, потом сделать новую выборку из n особей без замещения, а потом подсчитать сколько. Давайте сделаем это для всех N от $K = 100$ до 10 000:

```
Nmax, K, n, num_samples = 10000, 100, 100, 100
counter = {}
for N in range(K, Nmax):
    marked = [ set(np.random.choice(N, size=K, replace=False)) \
      for _ in range(num_samples)]
    new_sample = [ set(np.random.choice(N, size=n, replace=False)) \
      for _ in range(num_samples)]
    sample_k = [ len(marked[i].intersection(new_sample[i])) \
      for i in range(num_samples) ]
    for k in sample_k:
        counter[(k, N)] = counter.get((k, N), 0) + 1
```

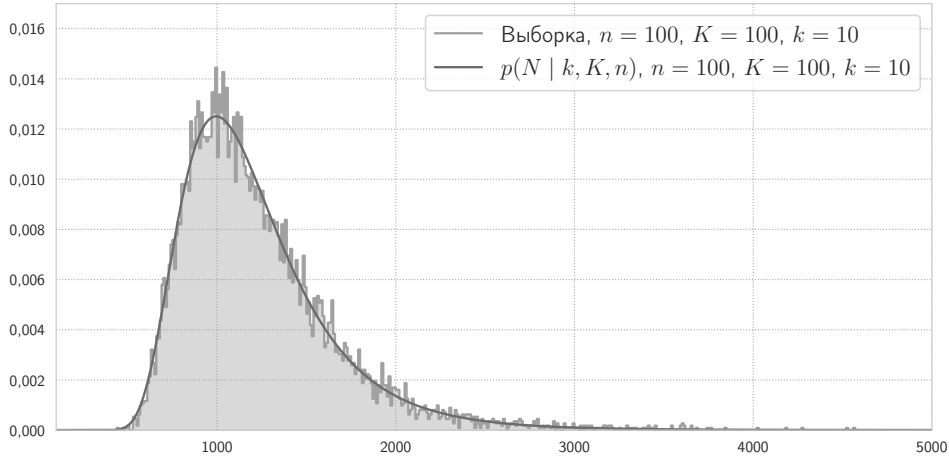


Рис. 3.19. Экспериментальная проверка (3.36): выборочные значения и плотность $p\,(N|k, K, n)$ для $n = 100$, $K = 100$ и $k = 10$, нормированная на интервале $[100, 10\,000]$

Как и раньше, у нас получился счётчик, в котором указано, сколько раз в нашем эксперименте встретилась та или иная пара (k, N), и эти счётчики теперь можно суммировать и перенормировать как нам угодно, в частности подсчитывать значения $p\,(N|k, K, n)$ для фиксированного k. В коде ниже мы ещё просуммировали значения N так, чтобы они шли через десять, для удобства рисования; в итоге переменные `xs` и `ys` будут содержать как раз требующиеся координаты по осям X и Y.

```
k = 10
counter_binned = {}
for x,v in counter.items():
    counter_binned[ ( x[0], 10*(x[1] // 10) ) ] = \
      counter_binned.get( ( x[0], 10*(x[1] // 10) ), 0 ) + v
result = {x[1]:v for x,v in counter_binned.items() if x[0] == k}
result_total = np.sum([v for v in result.values()])
xs = list(np.arange(K, Nmax+1, 10))
ys = [result.get(x, 0) / result_total for x in xs]
```

Кроме того, давайте нарисуем теоретическую кривую, отнормировав формулу (3.36) на интервале $N \in [100, 10\,000]$.

```
distrib = np.array([ sp.special.binom(K, k) * \
  sp.special.binom(N-K, n-k) / sp.special.binom(N, n) for N in range(K, Nmax)])
distrib = distrib / np.sum(distrib)
distrib_binned = np.array([sum(distrib[i:i+10]) \
  for i in range(0, len(distrib)+1, 10) ])
```

Результат показан на рис. 3.19: теоретическое апостериорное распределение $p(N|k,K,n)$ действительно отлично сходится с эмпирическим результатом. На практике это значит, что если мы нашли $k = 10$ отмеченных особей в новой выборке из $n = 100$ особей, а в первой части исследования отметили $K = 100$ особей, то хотя, конечно, ответ $N = \frac{nK}{k} = 1000$, полученный по методу Линкольна — Петерсена, действительно похож на моду апостериорного распределения, не стоит забывать о том, что у этого распределения получилась очень немаленькая дисперсия. Например, в нашем эксперименте интервал $N \in [500, 2000]$ покрывает лишь около 94,7 % всего распределения, а это разница в два раза в обе стороны.

Итак, в этом разделе вы ещё раз увидели, как делать байесовский вывод, на примере обучения параметров дискретного равномерного распределения. Хотя на первый взгляд это кажется очень простым случаем, оказалось, что и здесь возникают некоторые технические трудности, а полноценный вывод требует достаточно глубокого понимания вероятностной стороны дела.

В следующих главах мы обсудим и другие примеры интересного с математической точки зрения обучения параметров различных распределений, а сейчас вспомним ещё несколько ключевых понятий теории вероятностей, очень важных для машинного обучения. Но сначала — ещё одно рассуждение «со звёздочкой», которое, надеюсь, поможет несколько систематизировать интуитивное понимание априорных распределений.

*3.3.4. Пример Джейнса: урна с шарами и биномиальные обезьяны

Когда мы в разделе 2.4 говорили о равномерном априорном распределении, мы как само собой разумеющееся приняли, что именно равномерное распределение выражает полное незнание того, какой стороной монетка выпадает чаще. Но действительно ли это так? В этом разделе мы обсудим альтернативные взгляды на этот вопрос, которые развивал уже упомянутый выше основоположник байесовского подхода Эдвин Джейнс [285, 286].

Давайте разберём пример, который Джейнс в своей книге [286] рассматривает очень подробно (мы будем следовать изложению Джейнса с некоторыми сокращениями); он очень похож по духу на оценки, которые мы получали в подразделах 3.3.1 и 3.3.3.

Рассмотрим очень простую задачу байесовского вывода: предположим, что перед нами урна, в которой N шаров, R из которых красные, а $N - R$ — белые. Мы достаём из урны n шаров, r из которых оказываются красными. В разделе 3.1 мы обсуждали дискретные распределения, которые могут помочь нам решить прямую задачу: найти распределение пар (n, r) по заданным N и R. Это не сложно: вероятность вынуть из урны сначала r красных шаров подряд, а затем w белых, составляет

$$\frac{R}{N}\frac{R-1}{N-1}\cdot\ldots\cdot\frac{R-r+1}{N-r+1}\cdot\frac{N-R}{N-r}\frac{N-R-1}{N-r-1}\cdot\ldots\cdot\frac{N-R-w+1}{N-r-w+1}=$$
$$=\frac{R!(N-r)!}{(R-r)!N!}\cdot\frac{(N-R)!(N-r-w)!}{(N-R-w)!(N-r)!}=\frac{R!(N-R)!(N-n)!}{(R-r)!(N-R-w)!N!}. \quad (3.37)$$

Эта вероятность получена для конкретной последовательности исходов, и она по симметрии не зависит от последовательности, так что для того, чтобы получить ответ, достаточно умножить (3.37) на число возможных таких последовательностей, которое составляет $\binom{n}{r}$:

$$p\left(r|N,R,n\right)=\frac{R!(N-R)!(N-n)!}{(R-r)!(N-R-w)!N!}\frac{n!}{r!(n-r)!}=\frac{\binom{R}{r}\binom{N-R}{n-r}}{\binom{N}{n}}, \quad (3.38)$$

то есть то самое гипергеометрическое распределение (3.7), которое мы обсуждали в подразделе 3.1.4.

Но теперь перед нами стоит обратная задача: мы получили данные $D=(n,r)$ и хотим что-то сказать о значениях N и R. Байесовский вывод проходит точно так же, как в подразделе 3.3.2:

$$p\left(N,R|D\right)=\frac{p\left(D|N,R\right)p\left(R|N\right)p\left(N\right)}{p\left(D\right)},$$

и здесь в знаменателе стоит нормировочная константа

$$p\left(D\right)=\sum_{N=0}^{\infty}\sum_{R=0}^{N}p\left(D|N,R\right)p\left(R|N\right)p\left(N\right),$$

а правдоподобие представляет собой гипергеометрическое распределение (3.38).

Самый интересный вопрос здесь стоит так: как выбрать априорное распределение $p\left(N,R\right)$, которое мы в формуле выше уже разложили в произведение $p\left(N,R\right)=p\left(R|N\right)p\left(N\right)$? В целом мы в байесовском выводе выбираем априорное распределение как захотим, согласно своей интуиции, но чего мы можем захотеть от этого априорного распределения, как формализовать интуицию? Оказывается, в ответе на этот вопрос есть много развилок и неожиданных выводов.

Во-первых, у нас есть некоторое интуитивное представление о том, что апостериорное распределение на величину N должно зависеть от числа выбранных шаров n (и это, казалось бы, именно та задача, которую мы рассматривали в подразделе 3.3.2), но не должно зависеть от того, сколько из них оказались красными: с какой стати пропорция красных шаров должна влиять на их общее число? Более того, в данном случае хотелось бы, чтобы полученные данные «обрезали» невозможные значения, то есть сообщали нам, что $N\geq n$, но не меняли наше априорное представление о том, сколько именно шаров может быть в урне, если это число больше n. Иными словами, хотелось бы обеспечить равенство

$$p(N|D) = \begin{cases} c \cdot p(N), & \text{если } N \geq n, \\ 0 & \text{в противном случае,} \end{cases} \tag{3.39}$$

где c — нормировочная константа, не зависящая от N.

Оказывается, что это требование вовсе не тривиально и уже накладывает достаточно серьёзные ограничения на $p(N, R)$. Давайте запишем его формально. Апостериорное распределение на N равно

$$p(N|D) = \frac{p(N)\,p(D|N)}{p(D)} = \frac{p(N)\sum_{R=0}^{N} p(D|N,R)\,p(R|N)}{p(D)},$$

и требование (3.39) значит, что

$$p(D|N) = \sum_{R=0}^{N} p(D|N,R)\,p(R|N) = \begin{cases} f(n,r), & \text{если } N \geq n, \\ 0 & \text{в противном случае,} \end{cases} \tag{3.40}$$

где $f(n, r)$ — это некоторая функция, зависящая от данных n и r, но не от N.

Более того, можно уточнить результат (3.40), вспомнив, что мы говорим о гипергеометрическом распределении (3.38): для $N \geq n$

$$\sum_{R=0}^{N} \binom{R}{r}\binom{N-R}{n-r} p(R|N) = f(n,r)\binom{N}{n}. \tag{3.41}$$

Уравнение (3.41) — это весьма нетривиальное ограничение на априорное распределение $p(R|N)$! Обратите внимание, что мы получили его из базовой гипотезы о том, что данные не должны менять наше мнение об N, и это условие (в некотором смысле условие независимости) привело нас к ограничению на математическое ожидание некоторой величины по распределению $p(R|N)$; согласитесь, что мы вряд ли смогли бы записать такое условие интуитивно, без вышеприведённых рассуждений. Впрочем, как мы сейчас увидим, разумные априорные распределения на R удовлетворяют условию (3.41), так что байесовский вывод по N в этой задаче будет тривиальным: апостериорное распределение будет пропорционально априорному, за исключением начального отрезка $N < n$, который полученные данные делают невозможным.

Значит, осталось выбрать $p(R|N)$; давайте рассмотрим несколько вариантов. Первый вариант — равномерное распределение $p(R|N) = \frac{1}{N+1}$ на интервале $0, \ldots, N$. Условие (3.41) здесь легко получить из свойств суммирования биномиальных коэффициентов: мы знаем, что

$$\sum_{R=0}^{N} \binom{R}{r}\binom{N-R}{n-r} = \binom{N+1}{n+1}, \tag{3.42}$$

потому что слева мы сначала выбираем один элемент из $N+1$, фиксируя R, а потом выбираем r из R слева от него и $n-r$ из $N-R$ справа, то есть по сути выбираем $n+1$ элемент из $N+1$. А значит,

$$\sum_{R=0}^{N}\binom{R}{r}\binom{N-R}{n-r}p\left(R|N\right)=\binom{N+1}{n+1}\frac{1}{N+1}=$$
$$=\frac{(N+1)!(N+1-n-1)!}{(n+1)!(N+1)}=\frac{1}{n+1}\binom{N}{n},$$

то есть $f(n,r)=\frac{1}{n+1}$, и эта функция действительно не зависит от N. Апостериорное распределение тогда получается таким:

$$p\left(R|D,N\right)=\binom{N+1}{n+1}^{-1}\binom{R}{r}\binom{N-R}{n-r}; \tag{3.43}$$

заметьте, что это уже не гипергеометрическое распределение, потому что переменная теперь R, а не N.

Полученное апостериорное распределение (3.43) удовлетворяет нескольким естественным требованиям «здравого смысла»:

- для $n=r=0$ это распределение превращается в априорное распределение $p\left(R|N\right)=\frac{1}{N+1}$;
- для $n=r=1$ (вытащили один шар, и он оказался красным) получается

$$p\left(R|D=(1,1),N\right)=\frac{2R}{N(N+1)},$$

  что пропорционально R; это тоже естественно, потому что в правдоподобии вероятность вытащить красный шар $p\left(r=1|n=1,R,N\right)=\frac{R}{N}$, и мы умножаем это правдоподобие на равномерное распределение;
- кроме того, $\frac{2R}{N(N+1)}$ равно нулю при $R=0$ (что логически невозможно: получается, что мы вытащили красный шар из урны, где их не было), и это свойство сохраняется и для любых других $n\geq r\geq 1$.

Можно и продолжить байесовский вывод, почти в точности как мы делали для монетки в разделе 2.4. В данном случае апостериорная вероятность (3.43) достигает максимума при (проверьте!)

$$R_{\text{MAP}}=\left\lfloor\frac{r}{n}\left(N+1\right)\right\rfloor,$$

что тоже звучит вполне естественно. Но этот максимум не совпадает с математическим ожиданием:

$$\mathbb{E}[R] = \sum_{R=0}^{N} R \cdot p(R|D,N) = \binom{N+1}{n+1}^{-1} \sum_{R=0}^{N} R\binom{R}{r}\binom{N-R}{n-r} =$$
$$= \binom{N+1}{n+1}^{-1} \left(\sum_{R=0}^{N} (R+1)\binom{R}{r}\binom{N-R}{n-r} - \sum_{R=0}^{N} \binom{R}{r}\binom{N-R}{n-r} \right) =$$
$$= \binom{N+1}{n+1}^{-1} \left(\sum_{R=0}^{N} (r+1)\binom{R+1}{r+1}\binom{N-R}{n-r} - \sum_{R=0}^{N} \binom{R}{r}\binom{N-R}{n-r} \right) =$$
$$= (r+1)\binom{N+1}{n+1}^{-1}\binom{N+2}{n+2} - 1 = \frac{(N+2)(r+1)}{n+2} - 1,$$

где мы воспользовались известным тождеством о биномиальных коэффициентах: $(R+1)\binom{R}{r} = (r+1)\binom{R+1}{r+1}$.

Для больших N, r и n этот результат, конечно, похож на R_{MAP}, но поправка существенна: ожидаемая доля красных шаров, остающихся в урне, составляет

$$\frac{\mathbb{E}[R] - r}{N-n} = \frac{(N+2)(r+1) - (n+2) - r(n+2)}{(N-n)(n+2)} = \frac{r+1}{n+2}, \tag{3.44}$$

то есть мы опять получили в точности правило Лапласа. Можно и предсказательное распределение в виде правила Лапласа получить (здесь я оставлю выкладки с биномиальными коэффициентами читателю в качестве упражнения):

$$p(\text{Red}|D,N) = \sum_{R=0}^{N} R \cdot p(\text{Red}, R|D,N) = \sum_{R=0}^{N} R \cdot p(\text{Red}|R,D,N)\, p(R|D,N) =$$
$$= \binom{N+1}{n+1}^{-1} \sum_{R=0}^{N} \frac{R-r}{N-n}\binom{R}{r}\binom{N-R}{n-r} = \frac{r+1}{n+2}. \tag{3.45}$$

Аналогично (снова оставлю вывод как упражнение) можно найти дисперсию апостериорного распределения (3.43), и результат тоже будет похож на (2.35):

$$\hat{R} = r + (N-n)p \pm \sqrt{\frac{p(1-p)}{n+3}(N+2)(N-n)}, \tag{3.46}$$

где мы обозначили $p = \frac{r+1}{n+2}$, а затем выразили дисперсию оценки $\hat{R}$ из дисперсии оценки вероятности $\frac{R-r}{N-n}$.

Заметим, что оценка (3.46) говорит нам о том, что оценка вероятности становится точнее с ростом n, то есть с ростом полученной выборки, причём она падает до нуля, если $n = N$. Это очень хорошо сходится с интуицией: если мы вытащили из урны *все* шары, то оценки числа красных шаров и разнообразных вероятностей становятся идеально точными! С другой стороны, если вернуться

обратно к оценке доли шаров, как в (3.44), то нам нужно будет разделить оценку (3.46) на $N-n$, и мы получим

$$\frac{\hat{R}-r}{N-n} = p \pm \sqrt{\frac{p(1-p)}{n+3}\frac{N+2}{N-n}}. \tag{3.47}$$

Теперь ситуация обратная: чем больше шаров мы вынули из урны, тем менее точной будет оценка доли оставшихся. Ну а в пределе $N \to \infty$ останется как раз $p \pm \sqrt{\frac{p(1-p)}{n+3}}$, что в точности соответствует результату (2.35) про монетку.

Но мы здесь не только для этих рассуждений, хотя они уже довольно содержательно расширили наши результаты из раздела 2.4, а для того, чтобы исследовать и другие формы априорных распределений. Давайте сначала сделаем совсем небольшое изменение: предположим, что нам заранее известно, что в урне есть хотя бы один красный и хотя бы один белый шар. А в остальном оставим распределение равномерным: $p(R|N) = \frac{1}{N-1}$ на интервале $1,\ldots,N-1$ и 0 в других точках. Тогда в равенстве (3.42) нужно выбросить слагаемые, соответствующие $R=0$ и $R=N$. Поскольку для $R=0$ $\binom{R}{r} = [\![r=0]\!]$, а для $R=N$ $\binom{N-R}{n-r} = [\![r=n]\!]$, тождество (3.42) превращается в

$$\sum_{R=1}^{N-1}\binom{R}{r}\binom{N-R}{n-r} = \binom{N+1}{n+1} - \binom{N}{n}[\![r=n]\!] - \binom{N}{n}[\![r=0]\!],$$

и, соответственно, апостериорное распределение (3.43) можно записать как

$$p(R|D,N) = \frac{\binom{R}{r}\binom{N-R}{n-r}}{\binom{N+1}{n+1} - \binom{N}{n}[\![r=n]\!] - \binom{N}{n}[\![r=0]\!]}. \tag{3.48}$$

Во-первых, заметим, что если мы получили данные с $0 < r < n$ (среди выбранных были и красные, и белые шары), то вся разница между формулами (3.43) и (3.48) пропадает, и апостериорное распределение остаётся тем же. Это кажется контринтуитивным: мы изменили вероятностное пространство, а распределение осталось тем же самым! Джейнс пишет об этом так: «И всё же после некоторого размышления мы видим, что результат верен, поскольку в этом случае из данных можно дедуктивно умозаключить, что R не может быть равным нулю или N; поэтому неважно, говорит ли нам то же самое ещё и априорное распределение: состояние наших знаний об R в этих двух случаях одно и то же, на что указывают и теория вероятностей, и логика» [286]. А, например, для $r=0$, когда мы не вытащили ни одного красного шара, результат изменится:

$$p(R|D=(0,n),N) = \binom{N-R}{n}\left(\binom{N+1}{n+1} - \binom{N}{n}\right)^{-1} = \binom{N-R}{n}\binom{N}{n+1}^{-1}$$

для $R = 1, \ldots, N-1$ и 0 вне этого интервала. Заметим, что это в точности совпадает с апостериорным распределением (3.43), лишь перенормированным на константу $\frac{N+1}{N-n}$. Мы видим, что хотя на этот раз изменение в априорном распределении исключило аж максимальную апостериорную гипотезу $R = 0$, общий вид распределения на оставшейся части носителя всё равно не изменился; разные априорные распределения не обязательно приводят к разным выводам.

Следующий шаг даёт другой интересный результат. Правило Лапласа (3.44) говорит нам, что предсказание при равномерном распределении сглаживается, причём это сглаживание в точности соответствует правдоподобию данных, в которых мы достали из урны два шара, и один из них оказался красным (в разделе 2.4 мы называли это эквивалентным размером выборки). Иначе говоря, равномерное априорное распределение даёт при выводе некоторую информацию, соответствующую выборке из $n = 2$ шаров. Можно ли придумать полностью неинформативное (uninformative) априорное распределение, такое, чтобы никакой добавки не получалось?

Можно попробовать поискать это распределение так: с какого распределения нужно начать, чтобы после получения данных $D = (n, r) = (2, 1)$ прийти к равномерному? Иначе говоря, на какое априорное распределение нужно умножить правдоподобие (3.38), которое в данном случае равно

$$p\,(r = 1|N, R, n = 2) = \frac{\binom{R}{1}\binom{N-R}{1}}{\binom{N}{2}} = \frac{R(N-R)}{N(N-1)},$$

чтобы на выходе получилась константа?

На первый взгляд кажется, что ничего не получится, ведь для $R = 0$ и $R = N$ получается $p\,(r = 1|N, R, n = 2) = 0$, и ноль никак не обратить. Но мы только что придумали, как с этим справиться: можно отрезать крайние случаи в априорном распределении, и апостериорное не изменится, если $1 \le r \le n-1$, а у нас данные именно такие! Таким образом, можно определить априорное распределение

$$p\,(R|N) = \frac{\text{Const}}{R(N-R)} \quad \text{для} \quad R = 1, \ldots, N-1. \tag{3.49}$$

При этом априорном распределении апостериорное распределение, которое можно найти аналогично формуле (3.43), будет равно

$$p\,(R|D = (n, r), N) = \frac{\text{Const}}{R(N-R)}\binom{R}{r}\binom{N-R}{n-r} = \\ = \frac{\text{Const}}{r(n-r)}\binom{R-1}{r-1}\binom{N-R-1}{n-r-1},$$

а из этого по формуле суммирования (3.42) можно и нормировочную константу получить: для $R = 1, \ldots, N-1$

$$p\,(R|D=(n,r),N)=\binom{N-1}{n-1}^{-1}\binom{R-1}{r-1}\binom{N-R-1}{n-r-1}. \tag{3.50}$$

Если подставить в уравнение (3.50) $n=2$ и $r=1$, мы получим равномерное распределение $\frac{1}{N-1}$. Этот раздел уже и так затягивается, так что оставлю читателю в качестве упражнения проделать дальнейшие рассуждения об априорном распределении (3.49): доказать, что свойство (3.40) выполняется, то есть можно вести вывод отдельно на R, а также получить формулы ожидания и дисперсии апостериорного распределения, аналогичные (3.47).

А мы перейдём к последнему, особенно любопытному случаю. Рассмотрим такое, на первый взгляд вполне естественное априорное распределение: предположим, что урна наполнялась шарами, каждый из которых имел некоторую вероятность g быть красным, причём цвет шаров выбирался независимо друг от друга. Джейнс называет этот случай «распределением биномиальных обезьян» (binomial monkey prior), описывая его как урну, наполняемую обезьянами, швыряющими туда случайно выбранные мячи (видимо, это обезьяны-дальтоники). Тогда априорное распределение будет биномиальным:

$$p\,(R|N)=\binom{N}{R}g^R(1-g)^{N-R}. \tag{3.51}$$

Это значит, что априори мы можем оценить число красных шаров в урне, исходя из дисперсии монетки, как $\hat{R}=Ng\pm\sqrt{Ng(1-g)}$.

Теперь проведём байесовский вывод с априорным распределением (3.51). Во-первых, нужно проверить свойство (3.40):

$$\begin{aligned}
p\,(D|N)&=\sum_{R=0}^{N}p\,(D|N,R)\,p\,(R|N)=\sum_{R=0}^{N}\frac{\binom{R}{r}\binom{N-R}{n-r}}{\binom{N}{n}}\binom{N}{R}g^R(1-g)^{N-R}=\\
&=\binom{n}{r}\sum_{R=0}^{N}\binom{N-n}{R-r}g^R(1-g)^{N-R}=\\
&=\binom{n}{r}g^r(1-g)^{n-r}\sum_{R=0}^{N}\binom{N-n}{R-r}g^{R-r}(1-g)^{N-n-R}=\\
&=\binom{n}{r}g^r(1-g)^{n-r},
\end{aligned}$$

где мы воспользовались тем, что (проверьте!)

$$\binom{R}{r}\binom{N-R}{n-r}\binom{N}{R}=\binom{N}{n}\binom{n}{r}\binom{N-n}{R-r}, \tag{3.52}$$

а затем просуммировали по биномиальному распределению, в котором выбираются $R-r$ красных шаров из $N-n$ попыток. Результат не зависит от N, так что свойство (3.40) выполняется.

Подсчитаем апостериорное распределение:

$$p\,(R|D,N) = \text{Const}\cdot\binom{N}{R}g^R(1-g)^{N-R}\binom{R}{r}\binom{N-r}{n-r},$$

и нормировочную константу можно подсчитать, опять воспользовавшись (3.52):

$$1 = \sum_{R=0}^{N} p\,(R|D,N) = \text{Const}\binom{N}{n}\binom{n}{r}\sum_{R=0}^{N}\binom{N-n}{R-r}g^R(1-g)^{N-R} = \\ = \text{Const}\binom{N}{n}\binom{n}{r}g^r(1-g)^{n-r} \quad (3.53)$$

для $R = r,\ldots,N-n+r$.

Теперь из результата (3.53) можно выразить Const и найти апостериорное распределение уже полностью, вместе с константой:

$$p\,(R|D,N) = \binom{N-n}{R-r}g^{R-r}(1-g)^{N-R-n+r}. \quad (3.54)$$

Отсюда можно найти и среднее, и дисперсию апостериорной оценки $\hat{R}$ точно так же, как мы выводили оценку (3.46):

$$\hat{R} = r + (N-n)g \pm \sqrt{g(1-g)(N-n)}, \quad (3.55)$$

и аналогично формуле (3.47) мы оценим долю оставшихся красных шаров как

$$\frac{\hat{R}-r}{N-n} = g \pm \sqrt{\frac{g(1-g)}{N-n}}. \quad (3.56)$$

На первый взгляд кажется, что полученные нами результаты (3.55) и (3.56) практически неотличимы от (3.46) и (3.47) — зачем было и выводить? Но есть очень существенная разница: в оценках (3.46) и (3.47) апостериорное распределение на p определялось в первую очередь данными $D=(n,r)$, а в оценках (3.55) и (3.56) данные вообще практически не участвуют! Значение r для оценки (3.56) вообще неважно, а значение n просто постепенно уменьшает дисперсию, пока мы не получим точный результат g с нулевой дисперсией, когда вынем из урны все шары, $n=N$.

Получилось, что биномиальное априорное распределение *отменяет влияние данных*: апостериорная оценка совпадает с априорной и никак не зависит ни от r, ни от n! Это выглядит крайне странно, но опять же вполне соответствует нашей

интуиции. Априорное распределение с «биномиальными обезьянами» предполагало, что цвет шаров выбирается независимо друг от друга — вот и получилось, что знание цвета части шаров ничего не сообщает нам о цвете остальных, всё логично. Обратите внимание, как эта независимость оказалась «пропущена через» байесовский вывод и сохранилась в апостериорном результате.

Давайте подведём итог. В этом разделе мы рассмотрели одно из простейших дискретных вероятностных пространств — обычную урну, из которой в различных экспериментах и постановках задачи достают красные и белые шары. Мы последовательно провели байесовский вывод для нескольких разных априорных распределений:

- обычного равномерного распределения на $R = 0, \ldots, N$;
- обрезанного равномерного, в котором исключены крайние случаи только красных и только белых шаров в урне, то есть равномерного распределения на $R = 1, \ldots, N - 1$;
- неинформативного распределения $\frac{\text{Const}}{R(N-R)}$, которое не даёт дополнительного сглаживания в апостериорном распределении;
- распределения биномиальных обезьян $p(R|N) = \binom{N}{R} g^R (1-g)^{N-R}$, которое, как выяснилось, полностью отменяет влияние данных и сохраняет независимость цвета шаров в урне друг от друга.

Тем самым мы рассмотрели несколько важных частных случаев, которые дают представление о том, насколько разных и интересных эффектов можно добиться выбором априорного распределения. В следующем разделе вы, в частности, увидите, как перенести аналогичные результаты на непрерывный случай, но на самом деле мы будем вести гораздо более общий разговор об энтропии и производных от неё понятиях.

3.4. Энтропия, KL-дивергенция и полное незнание

Или нет: я лучше на твоём языке, так ты скорее поймёшь. Вот: две силы в мире — энтропия и энергия. Одна — к блаженному покою, к счастливому равновесию; другая — к разрушению равновесия, к мучительно-бесконечному движению. Энтропии — наши или, вернее, — ваши предки, христиане, поклонялись как Богу. А мы, антихристиане, мы...

Евгений Замятин. Мы

Уходя оттуда, я рассуждал сам с собою, что этого-то человека я мудрее, потому что мы с ним, пожалуй, оба ничего в совершенстве не знаем, но он, не зная, думает, что что-то знает, а я коли уж не знаю, то и не думаю, что знаю. На такую-то малость, думается мне, я буду мудрее, чем он, раз я, не зная чего-то, и не воображаю, что знаю эту вещь.

Платон. Апология Сократа (пер. М. С. Соловьёва)

3.4.1. Энтропия как мера неопределённости

Мы уже почти закончили с воспоминаниями из курсов теории вероятностей (к концу третьей главы пора бы уже!). Нам осталось рассмотреть только ещё один набор понятий, которые возникли даже не в классической теории вероятностей, а в теории информации и потом оказались очень полезными для машинного обучения; все они так или иначе связаны с понятием *энтропии* распределения вероятностей. Наверное, всякий слышал, что энтропия — это мера неопределённости; но как это формализовать математически? Давайте попробуем развить здесь некоторую интуицию.

Начнём даже не с априорных распределений для параметров монеток и кубиков, а просто с поиска дискретных распределений вероятностей, которые выражают наше незнание. Предположим, например, что мы должны выбрать из шести гипотез, шести граней кубика, и нам нужно выразить полное незнание о том, какие грани более вероятны. Такое состояние полного и абсолютного незнания действительно описывается равномерным распределением, здесь вряд ли есть место для разногласий.

Но что если какая-то априорная информация всё же известна? Например, предположим (это один из примеров из книги [286]), что мы знаем, что у шестигранного кубика среднее число выпадающих точек равно не 3,5, как было бы в равномерном распределении, а 4,5. Какое тогда лучше выбрать распределение

вероятностей на гранях кубика $(p_1, p_2, p_3, p_4, p_5, p_6)$, которое отражало бы степень нашего незнания, но при этом учитывало бы имеющуюся информацию? Какую целевую функцию лучше оптимизировать при выборе такого распределения?

Первый возможный вариант выглядит так: мы знаем среднее значение 4,5 для некоторой величины (в данном случае для ожидания числа точек) и хотим выразить тот факт, что не знаем ничего больше; так давайте же максимизировать разброс этой величины, то есть дисперсию числа точек. Но этот вариант оказывается не слишком подходящим — дисперсия мультиномиального распределения равна

$$\mathrm{Var}(X) = \mathbb{E}\left[\left(X - \bar{X}\right)^2\right] = \sum_{i=1}^{6} p_i \left(i - 4{,}5\right)^2,$$

и очевидно, что для максимизации дисперсии нужно просто поместить весь вес в крайние значения 1 и 6; это сразу даёт ответ, потому что нам нужно удовлетворить условия $p_1 + p_6 = 1$ и $p_1 + 6p_6 = 4{,}5$, и получается $p_1 = 0{,}3$, $p_6 = 0{,}7$. Это решение максимизирует дисперсию числа точек, но, согласитесь, не слишком отвечает нашей интуиции полного незнания — например, почему вдруг значения от двух до пяти точек совсем никогда выпасть не могут?

Таким образом, нам нужно выбрать какой-то другой, более корректный функционал, который отражал бы «степень нашего незнания», меру неопределённости, содержащуюся в распределении вероятностей, а затем его максимизировать. Оказывается, что в выборе такого функционала совсем не так много свободы, как может показаться. Первым его форму нашёл Клод Шеннон[1] в самом главном результате теории информации — *теореме Шеннона* [529].

Давайте рассмотрим рассуждения из теоремы Шеннона по крайней мере для дискретного случая. Начнём с нескольких вполне естественных требований, аксиом, которым должен удовлетворять этот функционал:

- для некоторого дискретного распределения $(p_1, \ldots, p_n)$ должно существовать число $H_n(p_1, \ldots, p_n)$, которое собственно и выражает эту меру неопределённости;

[1] *Клод Элвуд Шеннон* (Claude Elwood Shannon, 1916–2001) — американский математик и инженер, основоположник теории информации и современной криптографии. В 1937 году он прошёл стажировку в Bell Labs и в результате написал магистерскую диссертацию, в которой показал, что на переключающихся схемах можно реализовать всю булеву алгебру [528]; эта работа получила премию имени Нобеля IEEE и стала, наверное, самой известной в мире магистерской диссертацией. Ph. D. диссертация Шеннона в Массачусетском технологическом институте была посвящена теоретической генетике и могла бы стать основополагающей, но задержалась с публикацией; затем он работал в Bell Labs и Institute for Advanced Study. Главными работами Шеннона стали *A Mathematical Theory of Communication* [529], в которой он ввёл понятие энтропии и фактически создал теорию информации, и *Communication Theory of Secrecy Systems* [530], в которой Шеннон заложил основы современной криптографии, проанализировав (опять же с теоретико-информационной точки зрения) метод одноразовых блокнотов (one-time pad). Когда в 1985 году Шеннон неожиданно посетил Международный симпозиум по теории информации, один из участников описал это так: «Это было, как если бы сэр Исаак Ньютон появился на конференции по физике».

- H_n должна быть непрерывной функцией от p_i, чтобы сколь угодно малое изменение в распределении не приводило к большим скачкам в нашей мере неопределённости;
- функция неопределённости равномерного распределения

$$h(n) = H_n\left(\frac{1}{n}, \frac{1}{n}, \ldots, \frac{1}{n}\right)$$

должна монотонно возрастать с ростом n; это выражает нашу естественную интуицию о том, что много разных возможностей дают больше неопределённости, чем мало, и даёт некоторое «направление» функционалу H_n;

- мера неопределённости H_n должна быть *непротиворечивой*, то есть давать один и тот же результат при подсчёте H_n несколькими разными способами; в данном случае это значит, что неопределённость в выборе из трёх альтернатив $H_3(p_1, p_2, p_3)$ должна оставаться той же самой, если мы сложим её из двух бинарных выборов, сначала бросив монетку с вероятностями p_1 и $p_2 + p_3$, а затем с вероятностью q бросив ещё одну монетку с вероятностями $p_2/(p_2 + p_3)$ и $p_3/(p_2 + p_3)$ для выбора из второй и третьей альтернатив:

$$H_3(p_1, p_2, p_3) = H_2(p_1, p_2 + p_3) + qH_2\left(\frac{p_2}{p_2 + p_3}, \frac{p_3}{p_2 + p_3}\right). \tag{3.57}$$

Разумеется, самую большую часть работы здесь выполняет последнее предположение (3.57); помимо собственно непротиворечивости, оно ещё и требует аддитивность функции H_n: мы предположили, что неопределённости двух последовательных выборов складываются (и даже складываются с весами, как в математическом ожидании, если эти выборы делаются не всегда). Это, впрочем, тоже выглядит вполне естественно.

Теперь, когда аксиомы введены, дальнейшее практически неизбежно. Условие непротиворечивости и аддитивности (3.57) можно в более общем виде переписать так (далее будем опускать индекс при H, когда он ясен из контекста). Рассмотрим некоторое разбиение исходов $1, \ldots, n$ на r подмножеств $I_1, \ldots, I_r$ из $|I_1| = k_1, |I_2| = k_2, \ldots, |I_r| = k_r$ исходов, где $k_1 + \ldots + k_r = n$, и обозначим общий вес подмножества исходов (вероятность в него попасть) через $q_s = \sum_{i \in I_s} p_i$. Тогда, применяя несколько раз условие непротиворечивости, мы получим

$$\begin{aligned} H(p_1, \ldots, p_n) = & H(q_1, \ldots, q_r) + q_1 H\left(\frac{p_{1,1}}{q_1}, \ldots, \frac{p_{1,k_1}}{q_1}\right) + \\ & + q_1 H\left(\frac{p_{2,1}}{q_2}, \ldots, \frac{p_{2,k_2}}{q_2}\right) + \ldots + q_r H\left(\frac{p_{r,1}}{q_r}, \ldots, \frac{p_{r,k_r}}{q_r}\right), \end{aligned}$$

где $p_{s,i}$ пробегает исходы из подмножества I_s; формально ещё не доказано, что значение H не зависит от порядка аргументов, но уже скоро доказано будет, а пока можно считать, что мы зафиксировали произвольный порядок исходов.

Теперь самый интересный шаг: заметим, что вычисление значений энтропии $H(p_1,\ldots,p_n)$, по крайней мере для рациональных p_i (на иррациональные можно будет продолжить по непрерывности), можно свести к выражению, использующему только функцию H на равномерных распределениях; обозначим её через

$$h(n) = H\left(\frac{1}{n},\frac{1}{n},\ldots,\frac{1}{n}\right).$$

Чтобы это сделать, нужно рассмотреть разложение на исходы с вероятностями $(p_1,\ldots,p_n)$ как первый шаг на пути к разложению на элементарные исходы с вероятностями $\left(\frac{1}{N},\frac{1}{N},\ldots,\frac{1}{N}\right)$, где N — наименьшее общее кратное знаменателей p_i, и «дополнить» его, используя разложения на равномерные части.

Иначе говоря, мы приводим p_i к общему знаменателю N, $p_i = n_i/N$, а затем разделяем множество из N элементарных исходов на части соответствующего размера. Например, если мы хотим вычислить

$$H\left(\frac{1}{3},\frac{1}{3},\frac{1}{7},\frac{4}{21}\right),$$

мы можем рассмотреть это как разложение $N = 21$ элементарного исхода:

$$h(21) = H_{21}\left(\frac{1}{21},\ldots,\frac{1}{21}\right) = H\left(\frac{1}{3},\frac{1}{3},\frac{1}{7},\frac{1}{21}\right) +$$
$$+\frac{1}{3}H_7\left(\frac{1}{7},\ldots,\frac{1}{7}\right)+\frac{1}{3}H_7\left(\frac{1}{7},\ldots,\frac{1}{7}\right)+\frac{1}{7}H_3\left(\frac{1}{3},\frac{1}{3},\frac{1}{3}\right)+\frac{4}{21}H_4\left(\frac{1}{4},\ldots,\frac{1}{4}\right),$$

что сразу же даёт нам

$$H\left(\frac{1}{3},\frac{1}{3},\frac{1}{7},\frac{1}{21}\right) = h(21)-\frac{1}{3}h(7)-\frac{1}{3}h(7)-\frac{1}{7}h(3)-\frac{4}{21}h(4).$$

В общем виде всё проходит точно так же, и для $p_i = n_i/N$ мы получаем

$$H\left(p_1,\ldots,p_n\right) = h\left(N\right) - \sum_{i=1}^{n} p_i h\left(n_i\right). \tag{3.58}$$

Более того, если в ту же самую формулу (3.58) подставить $n_i = m$ и соответственно $N = nm$, то получится уравнение

$$h(mn) = h(m) + h(n). \tag{3.59}$$

У этого функционального уравнения, которое должно выполняться для любых натуральных m и n, есть очевидное решение — для некоторой константы c положим $h(n) = c\log(n)$.

Вопрос единственности здесь чуть более тонкий. Единственность доказывалась бы несложно, если бы m и n были произвольными вещественными числами: из уравнения (3.59) сразу ясно, что $h(1) = 0$, и можно продифференцировать (3.59) по m, положить $m = 1$ и получить дифференциальное уравнение

$$nh'(n) = h'(1),$$

которое с начальным условием $h(1) = 0$ других решений не имеет. Но у нас уравнение (3.59) дано только для целых чисел, так что здесь возникает вывод скорее из области теории чисел, в который мы уже не будем углубляться (см., например, [286]). Как бы то ни было, на выходе мы получаем $h(n) = \log n$ (выбор константы c эквивалентен выбору основания логарифма), а значит, из (3.59) следует, что

$$H(p_1, \ldots, p_n) = -\sum\nolimits_{i=1}^{n} p_i \log p_i. \tag{3.60}$$

Это и есть определение *энтропии*, фундаментальной меры неопределённости, содержащейся в распределении вероятностей. С точки зрения теории информации энтропия $H(p(\boldsymbol{x}))$ — это среднее число битов, которое требуется, чтобы оптимальным образом закодировать элемент $\boldsymbol{x}$, который порождается по распределению $p(\boldsymbol{x})$. Например, энтропия честной монетки равна

$$H\left(\frac{1}{2}, \frac{1}{2}\right) = -\frac{1}{2}\log\frac{1}{2} - \frac{1}{2}\log\frac{1}{2} = -\log\frac{1}{2},$$

что для двоичного логарифма составит ровно один бит (впрочем, дальше логарифмы обычно будут натуральные, в машинном обучении так удобнее).

Это очень важный объект для машинного обучения, и энтропия и производные от неё понятия ещё не раз встретятся в этой книге. Мы рассмотрели его только в дискретном случае, но те же рассуждения обобщаются и на непрерывный случай, где энтропия превращается в функционал

$$H(p) = -\int p(\boldsymbol{x}) \log p(\boldsymbol{x})\, \mathrm{d}\boldsymbol{x} = -\mathbb{E}_{p(\boldsymbol{x})}\left[\log p(\boldsymbol{x})\right].$$

Например, энтропия одномерного нормального распределения составляет

$$\begin{aligned} H\left(\mathcal{N}\left(x\middle|\mu,\sigma^2\right)\right) &= -\mathbb{E}_{\mathcal{N}}\left[\log\mathcal{N}\left(x\middle|\mu,\sigma^2\right)\right] = \\ &= -\mathbb{E}_{\mathcal{N}}\left[-\frac{1}{2}\log(2\pi\sigma^2) - \frac{1}{2\sigma^2}(x-\mu)^2\right] = \\ &= \frac{1}{2}\log(2\pi\sigma^2) + \frac{1}{2\sigma^2}\mathbb{E}_{\mathcal{N}}\left[(x-\mu)^2\right] = \frac{1}{2}\log(2\pi\sigma^2) + \frac{1}{2}, \end{aligned} \tag{3.61}$$

потому что $\mathbb{E}_{\mathcal{N}}\left[(x-\mu)^2\right]$ — это и есть дисперсия, практически по определению.

3.4.2. Производные понятия: перекрёстная энтропия и KL-дивергенция

От энтропии буквально один шаг до понятия *перекрёстной энтропии* (кросс-энтропии, cross-entropy), теоретико-информационной меры неопределённости, связывающей два распределения:

$$H\left(p\left(\boldsymbol{x}\right), q\left(\boldsymbol{x}\right)\right) = -\mathbb{E}_{p(\boldsymbol{x})}\left[\log q\left(\boldsymbol{x}\right)\right] = \int p\left(\boldsymbol{x}\right) \log q\left(\boldsymbol{x}\right) \mathrm{d}\boldsymbol{x}. \tag{3.62}$$

Её теоретико-информационный смысл состоит в следующем: представим, что мы построили оптимальную схему кодирования для объектов $\boldsymbol{x}$ согласно распределению $q\left(\boldsymbol{x}\right)$. Но вот незадача: распределение $q\left(\boldsymbol{x}\right)$ оказалось всего лишь моделью реальности, а на самом деле новые объекты $\boldsymbol{x}$ приходят к нам по распределению $p\left(\boldsymbol{x}\right)$. Тогда $H\left(p\left(\boldsymbol{x}\right), q\left(\boldsymbol{x}\right)\right)$ — это число битов, которые нам на самом деле потребуются; если $p\left(\boldsymbol{x}\right) \neq q\left(\boldsymbol{x}\right)$, то битов потребуется больше, чем для $p\left(\boldsymbol{x}\right) = q\left(\boldsymbol{x}\right)$ (докажу это чуть ниже). Чувствуете, как в этом объяснении сразу запахло машинным обучением?

И действительно, перекрёстная энтропия сплошь и рядом используется в машинном обучении. В частности, она представляет собой одну из стандартных функций ошибки для задач классификации. Если мы классифицируем объекты $\boldsymbol{x}$ на несколько классов $C_1, \ldots, C_K$, то правильный ответ C_k разумно выразить при помощи уже знакомого нам прямого кодирования (one-hot encoding), унарной схемы, где правильный ответ представляется как вектор $\boldsymbol{y}$ длины K, в котором все элементы равны нулю, кроме правильного ответа, который равен единице: $y_i = [\![i = k]\!]$. А выходом модели машинного обучения будет, скорее всего, вектор вероятностей различных классов: $\hat{\boldsymbol{y}} = (\,\hat{y}_1 \cdots \hat{y}_K\,)$, где $\hat{y}_i = q\left(C_i\right)$ — предсказанная вероятность класса C_i. Тогда логично будет определить функцию ошибки как перекрёстную энтропию между уверенным в себе дискретным распределением $\boldsymbol{y}$ и нашей моделью реальности $\hat{\boldsymbol{y}}$:

$$\mathcal{L}_{\mathrm{CE}} = -\sum_{i=1}^{K} y_i \log \hat{y}_i.$$

Для единственного правильного ответа C_k в прямом кодировании получается, что $\mathcal{L}_{\mathrm{CE}} = -\log \hat{y}_k$, то есть перекрёстная энтропия в этом случае просто просит вероятность правильного ответа быть побольше (что абсолютно естественно), но обратите внимание, что определение $\mathcal{L}_{\mathrm{CE}}$ годится и для того, чтобы уравнивать полноценные распределения вероятностей. Это позволяет использовать в практике машинного обучения, например, такие трюки, как *сглаживание меток* (label smoothing), когда вероятность правильного ответа «сглаживается» и устанавливается меньше единицы [138, 422].

В непрерывном случае перекрёстная энтропия тоже может пригодиться. Продолжим пример выше и подсчитаем кросс-энтропию между двумя одномерными гауссианами:

$$H\left(\mathcal{N}\left(x\middle|\mu_p,\sigma_p^2\right),\mathcal{N}\left(x\middle|\mu_q,\sigma_q^2\right)\right) = -\mathbb{E}_{\mathcal{N}(x|\mu_p,\sigma_p^2)}\left[\log\mathcal{N}\left(x\middle|\mu_q,\sigma_q^2\right)\right] =$$
$$= -\mathbb{E}_{\mathcal{N}(x|\mu_p,\sigma_p^2)}\left[-\frac{1}{2}\log(2\pi\sigma_q^2) - \frac{1}{2\sigma_q^2}\left(x-\mu_q\right)^2\right] =$$
$$= \frac{1}{2}\log(2\pi\sigma_q^2) + \frac{\mathbb{E}_{\mathcal{N}(x|\mu_p,\sigma_p^2)}\left[\left(x-\mu_q\right)^2\right]}{2\sigma_q^2} =$$
$$= \frac{1}{2}\log(2\pi\sigma_q^2) + \frac{\mathbb{E}_{\mathcal{N}(x|\mu_p,\sigma_p^2)}\left[\left(x-\mu_p\right)^2 + 2x\left(\mu_p-\mu_q\right) + \mu_q^2 - \mu_p^2\right]}{2\sigma_q^2} =$$
$$= \frac{1}{2}\log(2\pi\sigma_q^2) + \frac{\sigma_p^2 + 2\mu_p\left(\mu_p-\mu_q\right) + \mu_q^2 - \mu_p^2}{2\sigma_q^2} =$$
$$= \frac{1}{2}\log(2\pi\sigma_q^2) + \frac{\sigma_p^2 + \left(\mu_p-\mu_q\right)^2}{2\sigma_q^2}. \quad (3.63)$$

Разумеется, она тем меньше, чем меньше разница между параметрами двух гауссианов, и достигает минимума при $\mu_q = \mu_p$ и $\sigma_q = \sigma_p$.

Ещё одно понятие, связанное с энтропией и при этом очень важное для машинного обучения, — *дивергенция Кульбака — Лейблера* (Kullback — Leibler divergence), или просто KL-дивергенция, которую иногда также называют *относительной энтропией* (relative entropy). Она определяется как

$$\mathrm{KL}\left(p\left(\boldsymbol{x}\right)\|q\left(\boldsymbol{x}\right)\right) = \int p\left(\boldsymbol{x}\right)\log\frac{p\left(\boldsymbol{x}\right)}{q\left(\boldsymbol{x}\right)}\mathrm{d}\boldsymbol{x}. \quad (3.64)$$

Теоретико-информационное объяснение $\mathrm{KL}\left(p\|q\right)$ можно извлечь из приведённых выше мотиваций для энтропии и кросс-энтропии, ведь

$$\mathrm{KL}\left(p\left(\boldsymbol{x}\right)\|q\left(\boldsymbol{x}\right)\right) = H(p\left(\boldsymbol{x}\right), q\left(\boldsymbol{x}\right)) - H(p\left(x\right)). \quad (3.65)$$

Выходит, что KL-дивергенция — это мера «добавочного удивления», которое мы получаем, используя $q\left(\boldsymbol{x}\right)$ как модель для истинного распределения $p\left(\boldsymbol{x}\right)$; иначе говоря, это среднее *дополнительное* число битов, которое нужно потратить, чтобы закодировать наблюдения $\boldsymbol{x}$, поступающие из $p\left(\boldsymbol{x}\right)$, если наш код оптимизирован для наблюдений, поступающих из $q\left(\boldsymbol{x}\right)$.

Вопреки закону Стиглера, дивергенция Кульбака — Лейблера действительно была введена Соломоном Кульбаком и Ричардом Лейблером в 1951 году; в их работе она вводится как «средняя информация для различения H_1 и H_2, приходящаяся на одно наблюдение из μ_1» [324].

Однако на самом деле они ввели её как промежуточный шаг на пути к симметричной мере $J(p\|q) = \mathrm{KL}\left(p\|q\right) + \mathrm{KL}\left(q\|p\right)$, которую за несколько лет до того

ввёл Гарольд Джеффрис[1] [287]; её теперь называют *дивергенцией Джеффриса* (Jeffreys divergence). Кстати, книга Джеффриса «Теория вероятности» [287] вышла ещё в 1939 году, но до сих пор остаётся одним из главных текстов о байесовском выводе; вместе с ней можно порекомендовать уже упоминавшуюся книгу Джейнса [286], а также работу [495], в которой к изложению Джеффриса добавляют несколько недостающей там математической строгости.

Очевидно, что дивергенция Кульбака — Лейблера равна нулю в случае, если $p(\boldsymbol{x})$ и $q(\boldsymbol{x})$ совпадают почти всюду, то есть если $p(\boldsymbol{x}) = q(\boldsymbol{x})$ везде, кроме, возможно, подмножества меры ноль. Чуть менее очевидно, что она всегда неотрицательна, $\mathrm{KL}(p\|q) \geq 0$. Чтобы это доказать, вспомним, что логарифм — это выпуклая функция, а затем воспользуемся *неравенством Йенсена* (Jensen's inequality):

$$\mathbb{E}_{p(\boldsymbol{x})}[f(\boldsymbol{x})] \leq f\left(\mathbb{E}_{p(\boldsymbol{x})}[\boldsymbol{x}]\right), \quad \text{если } f \text{ выпукла.} \tag{3.66}$$

За формальными доказательствами неравенства Йенсена я отсылаю к курсам математического анализа, но его легко интуитивно понять как «непрерывную версию» определения выпуклости: по определению, для выпуклой функции верно, что

$$f\left(\sum_{i=1}^{n}\alpha_i\boldsymbol{x}_i\right) \geq \sum_{i=1}^{n}\alpha_i f(\boldsymbol{x}_i), \quad \text{где} \quad \alpha_i \geq 0,\ \sum_{i=1}^{n}\alpha_i = 1,$$

и это в точности формула (3.66) для дискретного распределения, сосредоточенного в точках $\boldsymbol{x}_i$ и присваивающего им вероятности α_i.

Применяя (3.66) к функции $\log\frac{q(\boldsymbol{x})}{p(\boldsymbol{x})}$ и распределению $p(\boldsymbol{x})$, мы получаем, что

$$\int p(\boldsymbol{x})\log\frac{q(\boldsymbol{x})}{p(\boldsymbol{x})}\mathrm{d}\boldsymbol{x} \leq \log\int p(\boldsymbol{x})\frac{q(\boldsymbol{x})}{p(\boldsymbol{x})}\mathrm{d}\boldsymbol{x} = \log\int q(\boldsymbol{x})\,\mathrm{d}\boldsymbol{x} = \log 1 = 0,$$

и слева здесь стоит в точности $-\mathrm{KL}(p(\boldsymbol{x})\|q(\boldsymbol{x}))$.

Завершая серию примеров с гауссианами, отмечу, что KL-дивергенцию между двумя нормальными распределениями можно легко получить из формул (3.61) и (3.63), воспользовавшись теоретико-информационным определением (3.65):

[1] *Сэр Гарольд Джеффрис* (Sir Harold Jeffreys, FRS, 1891–1989) — английский математик, геофизик и астроном. Основные результаты Джеффриса посвящены движению, строению и развитию Земли, а также происхождению Солнечной системы; в частности, он развил теорию приливной эволюции Солнечной системы и оценил эволюцию системы Земля — Луна. Обработка астрономических и геофизических данных привела Джеффриса к байесовской статистике, где он разработал теорию некорректных априорных распределений и в целом популяризировал байесовский способ рассуждений; Джеффрису посвящён знаменитый учебник Эдвина Джейнса [286]. Любопытно, что, несмотря на большой объём накопившихся подтверждений, Джеффрис до конца жизни так и не принял теорию дрейфа материков Альфреда Вегенера.

$$\mathrm{KL}\left(\mathcal{N}_p \| \mathcal{N}_q\right) = H\left(\mathcal{N}\left(x \middle| \mu_p, \sigma_p^2\right), \mathcal{N}\left(x \middle| \mu_q, \sigma_q^2\right)\right) - H\left(\mathcal{N}\left(x \middle| \mu_p, \sigma_p^2\right)\right) =$$
$$= \frac{1}{2}\log(2\pi\sigma_q^2) + \frac{\sigma_p^2 + \left(\mu_p - \mu_q\right)^2}{2\sigma_q^2} - \frac{1}{2}\log(2\pi\sigma_p^2) - \frac{1}{2} =$$
$$= \log\frac{\sigma_q}{\sigma_p} + \frac{\sigma_p^2 + \left(\mu_p - \mu_q\right)^2}{2\sigma_q^2} - \frac{1}{2}.$$

Из этой формулы снова легко видеть, что KL-дивергенция убывает по мере того, как параметры гауссианов становятся ближе друг к другу, и достигает нуля для $\mu_q = \mu_p$ и $\sigma_q = \sigma_p$.

3.4.3. KL-дивергенция в машинном обучении

В этом разделе мы рассмотрим два важных для машинного обучения минисюжета, связанных с дивергенцией Кульбака — Лейблера. Во-первых, я уже говорил в разделе 2.2 о том, что одной из первых и важнейших задач машинного обучения является максимизация правдоподобия $p\left(D|\boldsymbol{\theta}\right)$ по параметрам модели $\boldsymbol{\theta}$ для некоторого датасета $D = \{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N\}$, из которой получается гипотеза максимального правдоподобия

$$\theta_{\mathrm{ML}} = \arg\max_{\boldsymbol{\theta}} p\left(D|\boldsymbol{\theta}\right).$$

Оказывается, что максимизацию правдоподобия можно представить как минимизацию KL-дивергенции $\mathrm{KL}\left(p_{\mathrm{data}}\left(\boldsymbol{x}\right) \| p_{\mathrm{model}}\left(\boldsymbol{x}\right)\right)$; здесь $p_{\mathrm{model}}\left(\boldsymbol{x}\right)$ — это распределение, задаваемое моделью, то есть функция правдоподобия $p_{\mathrm{model}}\left(\boldsymbol{x}\right) = p\left(\boldsymbol{x}|\boldsymbol{\theta}\right)$, а $p_{\mathrm{data}}\left(\boldsymbol{x}\right)$ — это распределение данных, которое мы уже обсуждали в разделе 2.1: для набора данных $D = \{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N\}$ оно определяется как равномерное распределение на точках данных, то есть

$$p_{\mathrm{data}}\left(\boldsymbol{x}\right) = \frac{1}{N}\sum_{n=1}^{N}\delta\left(\boldsymbol{x} - \boldsymbol{x}_n\right).$$

Действительно:

$$\mathrm{KL}\left(p_{\mathrm{data}}\left(\boldsymbol{x}\right) \| p_{\mathrm{model}}\left(\boldsymbol{x}\right)\right) = \int p_{\mathrm{data}}\left(\boldsymbol{x}\right)\log\frac{p_{\mathrm{data}}\left(\boldsymbol{x}\right)}{p_{\mathrm{model}}\left(\boldsymbol{x}\right)}\mathrm{d}\boldsymbol{x} =$$
$$= \mathrm{Const} - \int p_{\mathrm{data}}\left(\boldsymbol{x}\right)\log p_{\mathrm{model}}\left(\boldsymbol{x}\right)\mathrm{d}\boldsymbol{x} = \mathrm{Const} - \mathbb{E}_{p_{\mathrm{data}}(\boldsymbol{x})}\left[\log p_{\mathrm{model}}\left(\boldsymbol{x}\right)\right],$$

и здесь константа — это не зависящая от $\boldsymbol{\theta}$ энтропия p_{data}, а ожидание по линейной комбинации дельта-функций — это линейная комбинация значений в соответствующих точках,

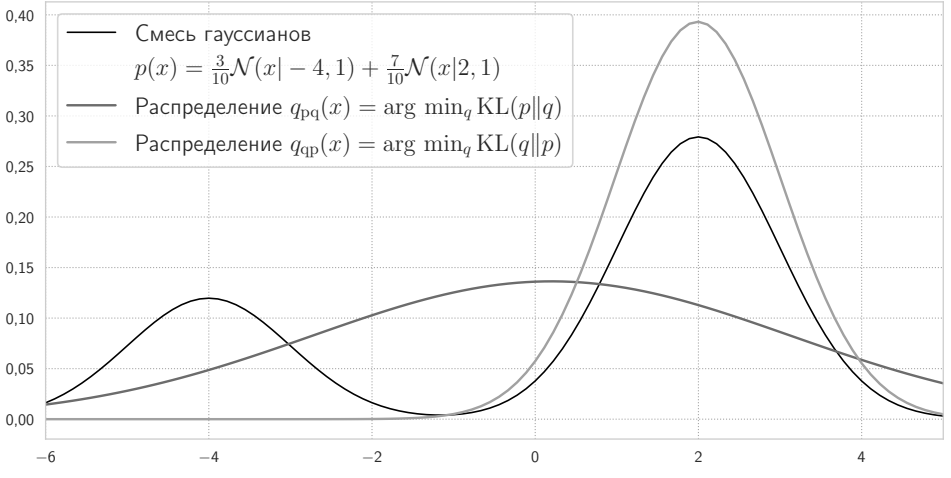


Рис. 3.20. Иллюстрация разницы между $\text{KL}(p\|q)$ и $\text{KL}(q\|p)$: распределения, минимизирующие разные KL-дивергенции

$$\mathbb{E}_{p_{\text{data}}(\boldsymbol{x})}\left[\log p_{\text{model}}(\boldsymbol{x})\right] = \frac{1}{N}\sum_{n=1}^{N}\log p_{\text{model}}(\boldsymbol{x}_n) = \frac{1}{N}\sum_{n=1}^{N}\log p(\boldsymbol{x}_n|\boldsymbol{\theta}).$$

Получилось, что минимизация $\text{KL}(p_{\text{data}}(\boldsymbol{x})\|p_{\text{model}}(\boldsymbol{x}))$ эквивалентна максимизации (логарифма) правдоподобия данных; это наблюдение оказывается полезным для порождающих моделей в машинном обучении [225].

Второй мини-сюжет связан с тем, что хотя дивергенция Кульбака — Лейблера представляет собой естественную меру похожести двух распределений, её нельзя использовать как метрику, которой можно было бы задавать расстояние между распределениями, ведь она несимметрична! Если в определении (3.64) поменять местами p и q, значение дивергенции изменится. Более того, разница между $\text{KL}(p\|q)$ и $\text{KL}(q\|p)$ и весьма существенна, и очень важна для машинного обучения; давайте я проиллюстрирую её классическим примером.

Предположим, что мы хотим приблизить одномерное распределение $p(x)$, которое является смесью двух нормальных распределений

$$p(x|\alpha,\mu_1,\sigma_1,\mu_2,\sigma_2) = \alpha\mathcal{N}\left(x\middle|\mu_1,\sigma_1^2\right) + (1-\alpha)\mathcal{N}\left(x\middle|\mu_2,\sigma_2^2\right), \tag{3.67}$$

посредством одного одномерного гауссиана $q(x)$. Вам наверняка уже ясно, что мы будем строить эту аппроксимацию, минимизируя дивергенцию Кульбака — Лейблера; но какую именно: $\text{KL}(p\|q)$ или $\text{KL}(q\|p)$?

Есть два варианта, проиллюстрированные на рис. 3.20: для бимодальной (то есть имеющей два максимума) плотности $p(x)$

- более пологая кривая, «накрывающая» оба максимума, показывает аппроксимацию $q_{pq}(x)$, полученную минимизацией $\mathrm{KL}(p\|q)$, то есть

$$q_{pq}(x) = \arg\min_q \mathrm{KL}(p(\boldsymbol{x})\|q(\boldsymbol{x}));$$

- а более «острая» кривая, которая выделяет только один максимум из двух, показывает аппроксимацию $q_{qp}(x)$, полученную минимизацией $\mathrm{KL}(q\|p)$, то есть

$$q_{qp}(x) = \arg\min_q \mathrm{KL}(q(\boldsymbol{x})\|p(\boldsymbol{x})).$$

Форму кривой $q_{pq}(x)$ на рис. 3.20 легко получить формально. Для аппроксимации $q_{pq}(x)$ можно вывести даже более общее утверждение: для любого $p(x)$

$$\begin{aligned}
q_{pq}(x) &= \arg\min_q \mathrm{KL}(p(\boldsymbol{x})\|q(\boldsymbol{x})) = \arg\min_{\mu,\sigma} \int p(x)\log\frac{p(x)}{\mathcal{N}(x|\mu,\sigma^2)}\mathrm{d}x = \\
&= \arg\max_{\mu,\sigma} \int p(x)\log\mathcal{N}\left(x\middle|\mu,\sigma^2\right)\mathrm{d}x = \\
&= \arg\max_{\mu,\sigma} \int p(x)\left(-\frac{1}{2}\log\sigma^2 - \frac{1}{2\sigma^2}(x-\mu)^2\right)\mathrm{d}x = \\
&= \arg\min_{\mu,\sigma}\left(\log\sigma^2 + \frac{\mu^2}{\sigma^2} - 2\frac{\mu}{\sigma^2}\int xp(x)\,\mathrm{d}x + \frac{1}{\sigma^2}\int x^2p(x)\,\mathrm{d}x\right) = \\
&= \arg\min_{\mu,\sigma}\left(\frac{1}{\sigma^2}\left(\mu - \mathbb{E}_{p(x)}[x]\right)^2 - \frac{\mathbb{E}_{p(x)}[x]^2}{\sigma^2} + \log\sigma^2 + \frac{\mathbb{E}_{p(x)}\left[x^2\right]}{\sigma^2}\right) = \\
&= \arg\min_{\mu,\sigma}\left(\frac{1}{\sigma^2}\left(\mu - \mathbb{E}_{p(x)}[x]\right)^2 + \log\sigma^2 + \frac{\mathrm{Var}_{p(x)}[x]}{\sigma^2}\right).
\end{aligned}$$

Поскольку от μ зависит только первое слагаемое, очевидно, что минимум достигается при $\mu_* = \mathbb{E}_{p(x)}[x]$. А для минимизации по σ^2 можно взять по этой величине производную и получить, что

$$\sigma_*^2 = \mathrm{Var}_{p(x)}[x].$$

Иначе говоря, минимизация дивергенции $\mathrm{KL}(p\|q)$ для гауссовского $q(x)$ превращается в *метод моментов* (moment matching): нужно просто найти среднее и дисперсию распределения $p(x)$ и использовать их для гауссиана. На рис. 3.20 это выражается тем, что кривая $q_{pq}(x)$ «накрывает» оба пика $p(x)$ и представляет собой некое их усреднение.

С обратной дивергенцией $\mathrm{KL}(q\|p)$ и оптимизирующим её распределением $q_{qp}(x)$ дело совсем другое. В этом случае мы уже не сможем получить ни общего аналитического результата, ни даже какого-то конкретного ответа для смеси двух гауссианов

$$p\left(x|\alpha,\mu_1,\sigma_1,\mu_2,\sigma_2\right) = \alpha\mathcal{N}\left(x\middle|\mu_1,\sigma_1^2\right) + (1-\alpha)\,\mathcal{N}\left(x\middle|\mu_2,\sigma_2^2\right).$$

Если записать $\mathrm{KL}\left(q\|p\right)$ в этом случае:

$$\mathrm{KL}\left(q\left(x\right)\|p\left(x\right)\right) = \int q\left(x\right)\log q\left(x\right)\mathrm{d}x - \int q\left(x\right)\log p\left(x\right)\mathrm{d}x,$$

то в первом выражении мы легко узнаем энтропию гауссиана (3.61), но вот со вторым ничего не поделать — это ожидание логарифма суммы, и никакого аналитического ответа не получится.

Но давайте попробуем получить ответ численно и проанализировать результат. Для этого я снова буду проводить все вычисления в максимально явном виде, прямыми численными операциями, реализованными в пакете `scipy`. Определим сначала искомую смесь (3.67); её параметры в нашем примере будут равны $\alpha = \frac{3}{10}$, $\mu_1 = -4$, $\mu_2 = 2$, $\sigma_1 = \sigma_2 = 1$:

```
from scipy.stats import norm
def gauss_mixture(x, alpha=.3, mu1=-4, sigma1=1, mu2=2, sigma2=1):
    return alpha * norm.pdf(x, mu1, sigma1) + \
      (1-alpha) * norm.pdf(x, mu2, sigma2)
```

Затем определим функцию, которая будет подсчитывать интересующую нас KL-дивергенцию. Для этого нужно численно взять интеграл от $q\left(x\right)\log p\left(x\right)$ и не забыть добавить энтропию гауссиана $\frac{1}{2}\log(2\pi\sigma^2)+\frac{1}{2}$, то есть просто $\log\sigma$, потому что аддитивная константа нас не интересует:

```
import scipy.integrate as integrate
def f(params, lims=(-20, 20)):
    return -np.log(params[1]) - integrate.quad( lambda x : \
      norm.pdf(x, params[0], params[1]) * \
      np.log(gauss_mixture(x)), lims[0], lims[1])[0]
```

Здесь функция `integrate.quad` выполняет численное интегрирование функции на заданном интервале, а `params` — это вектор из среднего и дисперсии гауссиана $q\left(x\right)$. Я не стал выбирать бесконечные пределы интегрирования, чтобы точно не натолкнуться на численные ошибки, — интервала $[-20, 20]$ для нашей смеси гауссианов будет более чем достаточно.

И теперь можно оптимизировать:

```
from scipy.optimize import minimize
res = minimize(f, [1, 1])
```

Функция `minimize` выполняет численную оптимизацию заданной функции; она может использовать любой из довольно большого числа разных алгоритмов (в данном случае по умолчанию будет выбран BFGS, но есть и метод сопряжённого градиента, и алгоритм Пауэлла *dog leg*, и многое другое, в том числе много

алгоритмов оптимизации с ограничениями — о них мы в книге говорить не будем, я просто оставлю пару ссылок на хорошие учебники по теории оптимизации [64, 356, 441]), начиная с заданного начального приближения, которое мы здесь задали как `[1, 1]`. Именно для соответствия сигнатуре функции `minimize`, кстати, у функции `f` был один векторный параметр, а не отдельно μ и σ.

В результате в переменной `res` должно получиться что-то в таком духе:

```
      fun: 1.77297248169507
 hess_inv: array([[ 1.03918546, -0.0533979 ],
       [-0.0533979 ,  0.58178219]])
      jac: array([-8.94069672e-08,  2.68220901e-07])
  message: 'Optimization terminated successfully.'
     nfev: 21
      nit: 6
     njev: 7
   status: 0
  success: True
        x: array([1.99053966, 1.01466874])
```

Нас из этого многообразия интересует главным образом `res.x`, в котором и записана полученная точка максимума. Как видите, оптимальные параметры получились почти точно равными $\mu = 2$ и $\sigma = 1$, то есть $q_{qp}(x)$ оказался фактически в точности равен *одному из* компонентов смеси.

Всё это показано на рис. 3.20. Теперь, глядя на рисунок, мы можем сформулировать полученный результат и в более интуитивно ясном виде. Посмотрим на формулу KL-дивергенции (3.64):

- для $\mathrm{KL}(p\|q) = \int p(\boldsymbol{x}) \log \frac{p(\boldsymbol{x})}{q(\boldsymbol{x})} \mathrm{d}\boldsymbol{x}$ можно заметить, что самые большие значения подинтегральная функция принимает, когда плотность $p(\boldsymbol{x})$ имеет заметно ненулевые значения, а $q(\boldsymbol{x})$ очень близка к нулю; в таких случаях «штраф» в KL-дивергенции максимален, а значит, минимизация $\mathrm{KL}(p\|q)$ будет избегать таких случаев, то есть $q(\boldsymbol{x})$ должна будет «накрывать» все относительно большие значения $p(\boldsymbol{x})$; именно этот эффект мы видим в кривой $q_{pq}(x)$;
- а для $\mathrm{KL}(q\|p) = \int q(\boldsymbol{x}) \log \frac{q(\boldsymbol{x})}{p(\boldsymbol{x})} \mathrm{d}\boldsymbol{x}$ ситуация будет обратной: самый большой «штраф» в KL-дивергенции появится в случаях, когда $q(\boldsymbol{x})$ ненулевая, а $p(\boldsymbol{x})$ очень близка к нулю; значит, оптимальное с точки зрения $\mathrm{KL}(q\|p)$ распределение $q_{qp}(x)$ должно будет «выбрать» один из локальных пиков.

Можно и ещё более явно проиллюстрировать этот эффект, использовав определённую нами выше функцию `f(params)`. Давайте зафиксируем дисперсию, скажем `params[1]=1`, и построим зависимость `f`, то есть KL-дивергенции $\mathrm{KL}(q\|p)$, от первого аргумента, то есть от среднего гауссиана $q(x)$. Результат показан на рис. 3.21: он имеет очевидные локальные минимумы в точках $\mu = -4$ и $\mu = 2$,

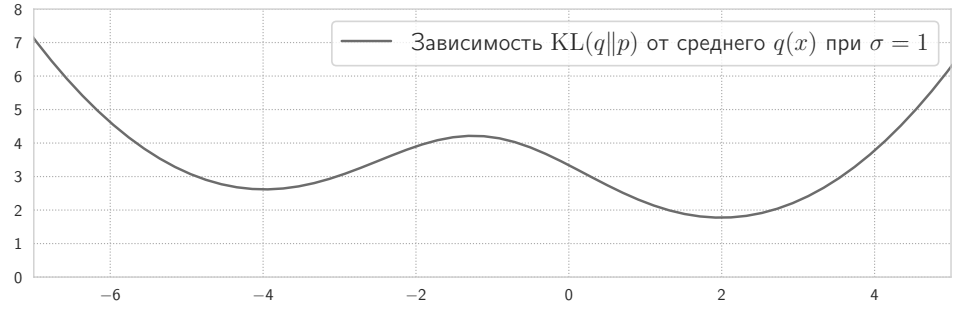


Рис. 3.21. Зависимость $\mathrm{KL}\,(q\|p)$ от среднего гауссиана $q\,(x)$ при $\sigma = 1$

соответствующие двум компонентам смеси $p\,(x)$, причём минимум в точке $\mu = 2$ глубже, потому что на эту компоненту приходится больше веса в $p\,(x)$.

3.4.4. Информация Фишера и принцип максимума энтропии

Ещё одно связанное с теорией информации понятие, которое нам понадобится в этой книге, происходит из анализа гипотез максимального правдоподобия. Вспомним раздел 2.2: одна из первых задач, которую мы решаем в машинном обучении при встрече с новой моделью, — это максимизация правдоподобия $p\,(D|\boldsymbol{\theta})$. Предположим, что мы это сделали и получили гипотезу максимального правдоподобия $\boldsymbol{\theta}_{\mathrm{ML}}$. Более того, давайте предположим, что наша модель полностью соответствует действительности и данные действительно берутся из распределения модели с некоторым параметром $\boldsymbol{\theta}_{\mathrm{true}}$: $p_{\mathrm{data}}\,(\boldsymbol{x}) = p\,(\boldsymbol{x}|\boldsymbol{\theta}_{\mathrm{true}})$. В разделе 6.5 мы подробно обсудим взаимосвязь между $\boldsymbol{\theta}_{\mathrm{true}}$ и $\boldsymbol{\theta}_{\mathrm{ML}}$, которая станет основой для информационного критерия Акаике.

А сейчас нас интересует теоретико-информационный вопрос, возникающий даже в этой идеальной ситуации: как много информации об истинном значении параметров $\boldsymbol{\theta}_{\mathrm{true}}$ сообщают нам данные D, точнее говоря, одна точка данных $\boldsymbol{x}$?

Это зависит от формы функции правдоподобия $p\,(\boldsymbol{x}|\boldsymbol{\theta})$. Если $p\,(\boldsymbol{x}|\boldsymbol{\theta})$ резко меняется при изменении $\boldsymbol{\theta}$, то есть правдоподобие образует острый пик вокруг значения $\boldsymbol{\theta}_{\mathrm{true}}$ и быстро убывает при удалении от него, то точка данных сообщает довольно много информации: при других значениях $\boldsymbol{\theta}$ мы ожидаем увидеть совсем другие значения $\boldsymbol{x}$. А если распределение $p\,(\boldsymbol{x}|\boldsymbol{\theta})$ более «пологое» и конкретную точку $\boldsymbol{x}$ можно с немалой вероятностью ожидать увидеть при многих разных значениях $\boldsymbol{\theta}$, то информации она нам сообщит мало, и потребуется достаточно большой набор данных, чтобы отличить друг от друга разные значения $\boldsymbol{\theta}$.

В ожидании по точке, взятой из истинного распределения данных $\boldsymbol{x} \sim p_{\mathrm{data}}$, где $p_{\mathrm{data}}\,(\boldsymbol{x}) = p\,(\boldsymbol{x}|\boldsymbol{\theta}_{\mathrm{true}})$, максимумом правдоподобия должна быть точка $\boldsymbol{\theta}_{\mathrm{true}}$.

Давайте это проверим сначала для простоты в размерности один, $\theta \in \mathbb{R}$, продифференцировав логарифм правдоподобия $\log p(\boldsymbol{x}|\theta)$:

$$\mathbb{E}_{\boldsymbol{x}}\left[\frac{\partial \log p(\boldsymbol{x}|\theta)}{\partial\theta}\right] = \int \frac{\frac{\partial}{\partial\theta}p(\boldsymbol{x}|\theta)}{p(\boldsymbol{x}|\theta)} p(\boldsymbol{x}|\theta)\,\mathrm{d}\theta = \frac{\partial}{\partial\theta}\int p(\boldsymbol{x}|\theta)\,\mathrm{d}\theta = \frac{\partial}{\partial\theta}1 = 0,$$

где, как всегда, мы предполагаем, что все интегралы сходятся, все условия регулярности выполняются и дифференцировать под знаком интеграла можно.

Тогда разумная мера того, насколько быстро в ожидании правдоподобие убывает вокруг своего максимума, — это ожидаемая дисперсия производной логарифма правдоподобия. Её называют *информацией Фишера* (Fisher information):

$$\mathcal{I}(\theta) = \mathbb{E}_{\boldsymbol{x}\sim p(\boldsymbol{x}|\theta)}\left[\left(\frac{\partial}{\partial\theta}\log p(\boldsymbol{x}|\theta)\right)^2\right] = \int\left(\frac{\partial}{\partial\theta}\log p(\boldsymbol{x}|\theta)\right)^2 p(\boldsymbol{x}|\theta)\,\mathrm{d}\boldsymbol{x}. \quad (3.68)$$

Если логарифм правдоподобия дважды дифференцируем по θ (как у нас всегда и будет), то можно заменить квадрат на вторую производную, ведь

$$\frac{\partial^2 \log p(\boldsymbol{x}|\theta)}{\partial\theta^2} = \frac{\partial}{\partial\theta}\frac{\frac{\partial}{\partial\theta}\log p(\boldsymbol{x}|\theta)}{p(\boldsymbol{x}|\theta)} = \frac{\frac{\partial^2}{\partial\theta^2}\log p(\boldsymbol{x}|\theta)}{p(\boldsymbol{x}|\theta)} - \left(\frac{\frac{\partial}{\partial\theta}\log p(\boldsymbol{x}|\theta)}{p(\boldsymbol{x}|\theta)}\right)^2 =$$
$$= \frac{\frac{\partial^2}{\partial\theta^2}\log p(\boldsymbol{x}|\theta)}{p(\boldsymbol{x}|\theta)} - \left(\frac{\partial}{\partial\theta}\log p(\boldsymbol{x}|\theta)\right)^2.$$

По тем же причинам, что и выше, первое слагаемое обращается в ноль:

$$\mathbb{E}_{p(\boldsymbol{x}|\theta)}\left[\frac{\frac{\partial^2}{\partial\theta^2}\log p(\boldsymbol{x}|\theta)}{p(\boldsymbol{x}|\theta)}\right] = \frac{\partial^2}{\partial\theta^2}\int p(\boldsymbol{x}|\theta)\,\mathrm{d}\theta = \frac{\partial^2}{\partial\theta^2}1 = 0,$$

и мы приходим к альтернативному определению информации Фишера:

$$\mathcal{I}(\theta) = -\mathbb{E}_{\boldsymbol{x}\sim p(\boldsymbol{x}|\theta)}\left[\frac{\partial^2}{\partial\theta^2}\log p(\boldsymbol{x}|\theta)\right] = \int\frac{\partial^2}{\partial\theta^2}\log p(\boldsymbol{x}|\theta)\,p(\boldsymbol{x}|\theta)\,\mathrm{d}\boldsymbol{x}. \quad (3.69)$$

Определение (3.69) легко интерпретировать как ожидаемую кривизну логарифма правдоподобия в окрестности точки максимума — ровно то, чего мы изначально и пытались добиться. В подразделе 3.4.5 я приведу несколько примеров вычисления информации Фишера — сейчас пока не до конца понятно, зачем они нам могут пригодиться, а вот в подразделе 3.4.5 мы их сразу же и применим.

Как и многое другое в математической статистике, это понятие ввёл Рональд Фишер в 1920-х годах [189]; о самом сэре Рональде мы подробно поговорим позже, в подразделе 3.4.4, да и вообще он много раз будет появляться в этой книге.

И в завершение раздела вернёмся к началу и посмотрим на понятие энтропии с другой стороны, примерно как в разделе 2.1 мы с другой стороны смотрели на понятие вероятности. На данный момент мы выяснили, что понятие энтропии с неизбежностью следует из аксиомы (3.57). Конечно, может показаться, что эта аксиома не так уж очевидна и обязательна для хорошей меры неопределённости[1], но, к счастью, к тому же самому понятию энтропии можно прийти и другими способами.

Например, в [286] приводится другое рассуждение, на которое Эдвину Джейнсу[2] указал физик Грэм Уоллис (ссылки, увы, нет): пусть у нас имеется некоторая информация I, которой должно соответствовать распределение вероятностей $(p_1, \ldots, p_n)$. «Информация» здесь понимается в самом общем виде: это просто значит, что не любое распределение нам подойдёт.

Будем порождать распределения таким процессом: разобьём общую вероятность (единицу) на N очень маленьких частей (квантов) размера $\delta = \frac{1}{N}$ и будем случайно распределять их между n исходами: N раз бросать честную игральную кость с n гранями. Тогда, если $p_i = \frac{N_i}{N}$, нам нужно будет распределить ровно N_i этих квантов в i-й исход. Каждая последовательность из N бросаний выпадает с вероятностью n^{-N}, и вероятность получить распределение $(p_1, \ldots, p_n)$ получится из соответствующего числа сочетаний:

$$n^{-N} \frac{N!}{N_1! \cdot \ldots \cdot N_n!}. \tag{3.70}$$

А теперь зададимся таким вопросом: если мы будем запускать этот процесс и выбрасывать распределения, не соответствующие информации I, какое распределение будет чаще всего нам попадаться? Выражение (3.70) говорит, что это будет то из распределений, соответствующих I, которое максимизирует величину $\frac{N!}{N_1! \cdot \ldots \cdot N_n!}$. Если устремить $N \to \infty$, то есть выбирать всё более мелкие кванты, в пределе мы можем оценить величины факториалов по формуле Стирлинга

$$\log N! = N \log N - N + \sqrt{2\pi N} + \frac{1}{12N} + O\left(\frac{1}{N^2}\right).$$

[1] Кстати, есть ничем особенно не подтверждённая версия, что свой пятый постулат Евклид специально записал максимально туманно и длинно, чтобы исследователям хотелось от него так или иначе избавиться, а первые четыре, напротив, сформулированы лапидарно и никакого внутреннего протеста не вызывают.

[2] *Эдвин Томпсон Джейнс* (Edwin Thompson Jaynes, 1922–1988) — американский физик, прославившийся работами по статистической механике и байесовским методам. В 1957 году Джейнс развил термодинамику максимальной энтропии, показав, что это частный случай байесовских методов (и ссылаясь при этом на более ранние работы Гиббса). Всю жизнь Джейнс работал над логическими основаниями теории вероятностей, и его главную книгу, вышедший уже посмертно учебник *Probability Theory: The Logic of Science* [286], нельзя не порекомендовать каждому, кто интересуется байесовскими методами и машинным обучением.

Вспомнив, что $N_i = p_i N$ и $\sum_{i=1}^n N_i = N$, получим:

$$\begin{aligned}\log \frac{N!}{N_1! \cdot \ldots \cdot N_n!} &= \log N! - \sum_{i=1}^n \log N_i! = \\ &= N \log N - N - \sum\nolimits_{i=1}^n (N_i \log N_i - N_i) + O\left(\sqrt{N}\right) = \\ &= N \log N - N - \sum_{i=1}^n p_i N (\log N + \log p_i) + \sum_{i=1}^n N_i + O\left(\sqrt{N}\right) = \\ &= -N \sum_{i=1}^n p_i \log p_i + O\left(\sqrt{N}\right).\end{aligned}$$

Иначе говоря, при $N \to \infty$ логарифм числа сочетаний растёт как раз как N умножить на энтропию, то есть из такого (вполне естественного) порождающего процесса с наибольшей вероятностью будет получаться именно распределение, максимизирующее энтропию. Мы получили совсем другое обоснование понятия энтропии, не опирающееся на аксиомы; более того, в этом подходе ещё и вполне естественно получается, что энтропию нужно именно *максимизировать*.

Мы пришли к *принципу максимума энтропии* (maximum entropy principle): для некоторой заданной априорной информации I наиболее характерным распределением вероятностей является распределение с максимальной энтропией среди тех распределений, которые удовлетворяют этой информации. Мы видели самый простой пример этого принципа: если информации никакой нет, то лучший выбор априорного распределения — это равномерное распределение, потому что оно максимизирует энтропию среди всех распределений.

Но его можно применить и к куда более сложным ситуациям. Вернёмся к примеру с кубиком, у которого среднее значение составляет $m = 4{,}5$ вместо 3,5. Давайте применим принцип максимума энтропии, то есть максимизируем

$$H(p_1, \ldots, p_6) = -\sum\nolimits_{i=1}^n p_i \log p_i$$

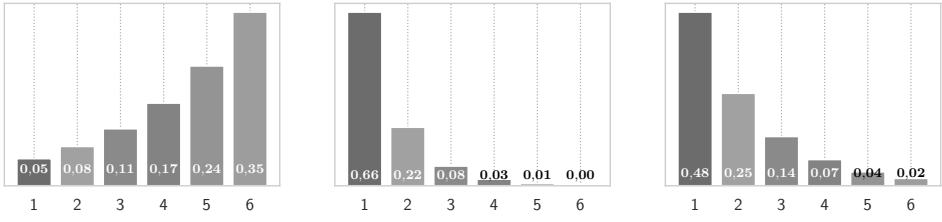
при двух условиях: $p_1 + \ldots + p_6 = 1$ и $\mathbb{E}[i] = \sum_{i=1}^6 i p_i = m$. Мы можем, как обычно, записать функцию Лагранжа для этой задачи условной оптимизации:

$$L = -\sum\nolimits_{i=1}^n p_i \log p_i - \lambda \sum\nolimits_{i=1}^6 i p_i - \mu \sum\nolimits_{i=1}^6 p_i,$$

взять от неё производные по p_i и приравнять их к нулю:

$$\frac{\partial L}{\partial p_i} = -\log p_i - 1 - \lambda i - \mu = 0,$$

то есть $p_i = e^{-\lambda i - \mu - 1}$.



(а) $m = 4{,}5$ (б) $m = 1{,}5$ (в) $m = 2$

Рис. 3.22. Распределения вероятностей кубиков, максимизирующие энтропию для разных фиксированных средних: (а) $m = 4{,}5$; (б) $m = 1{,}5$; (в) $m = 2$

Чтобы теперь найти множители Лагранжа λ и μ, нужно применить известные нам условия. Во-первых, $p_1 + \ldots + p_6 = 1$, то есть

$$\sum_{i=1}^{6} e^{-\lambda i - \mu - 1} = e^{-\mu - 1} \sum_{i=1}^{6} e^{-\lambda i} = 1.$$

Во-вторых, $\sum_{i=1}^{6} i p_i = m$, то есть

$$\sum_{i=1}^{6} i e^{-\lambda i - \mu - 1} = e^{-\mu - 1} \sum_{i=1}^{6} i e^{-\lambda i} = m.$$

Отсюда мы можем легко выразить m и μ через λ:

$$m = \frac{\sum_{i=1}^{6} i e^{-\lambda i}}{\sum_{i=1}^{6} e^{-\lambda i}}, \qquad \mu = \log \sum_{i=1}^{6} e^{-\lambda i} - 1.$$

Теперь осталось найти λ из первого уравнения, подставив туда $m = 4{,}5$ (записать решение аналитически, видимо, вряд ли получится), и подставить полученное значение λ во второе уравнение и в выражения для p_i. Давайте для нашего примера сделаем это численно:

```
from scipy.optimize import fsolve
func = lambda x : 4.5 - np.sum([(i+1)*np.exp(-x*(i+1)) for i in range(6)]) \
                  / np.sum([np.exp(-x*(i+1)) for i in range(6)])
lmb = fsolve(func, 0)[0]
mu = np.log(np.sum([ np.exp(-lmb*(i+1)) for i in range(6) ])) - 1
p = [ np.exp(-lmb*(i+1)-mu-1) for i in range(6) ]
print("\t".join(["%.4f" % x for x in p]))
```

Этот код выдаёт набор вероятностей

```
0.0544        0.0788        0.1142        0.1654        0.2398        0.3475
```

В сумме они дают единицу, и заметьте, что это распределение скошено в сторону больших значений, как и следовало ожидать из имеющейся априорной информации о среднем. На рис. 3.22 показаны этот и ещё два примера распределений, максимизирующих энтропию для разных значений m; привычное нам равномерное распределение получится при $m = 3{,}5$.

А в качестве характерного непрерывного примера давайте попробуем найти распределение на $\mathbb{R}$, достигающее максимальной энтропии с заданным средним μ и дисперсией σ^2. Мы хотим максимизировать функционал $\int_{-\infty}^{\infty} p(x) \log p(x)\, \mathrm{d}x$, причём среднее и дисперсия задают нам два условия на эту задачу оптимизации:

$$\int_{-\infty}^{\infty} xp(x)\, \mathrm{d}x = \mu, \qquad \int_{-\infty}^{\infty} (x-\mu)^2 p(x)\, \mathrm{d}x = \sigma^2;$$

кроме того, есть ещё базовое условие о том, что $p(x)$ представляет собой плотность вероятности, то есть $\int_{-\infty}^{\infty} p(x)\, \mathrm{d}x = 1$.

Давайте запишем функцию Лагранжа для этой задачи оптимизации, с множителями Лагранжа, соответствующими условиям выше:

$$L(x,\lambda_1,\lambda_2) = \int_{-\infty}^{\infty} p(x) \log p(x)\, \mathrm{d}x - \lambda_0 \left(1 - \int_{-\infty}^{\infty} p(x)\, \mathrm{d}x\right) - \\ - \lambda_1 \left(\mu - \int_{-\infty}^{\infty} xp(x)\, \mathrm{d}x\right) - \lambda_2 \left(\sigma^2 - \int_{-\infty}^{\infty} (x-\mu)^2 p(x)\, \mathrm{d}x\right).$$

Теперь можно применить метод вариационного исчисления[1]: давайте добавим небольшую (стремящуюся к нулю) вариацию $\delta p(x)$ к плотности $p(x)$ и посмотрим, что произойдёт с функционалом:

$$\delta L = \int_{-\infty}^{\infty} \delta p(x) \left(\ln p(x) + 1 + \lambda_0 + \lambda_1 (x-\mu) + \lambda_2 (x-\mu)^2\right).$$

В точке максимума вариация функционала должна быть равна нулю для любого малого $\delta p(x)$, то есть нулю должно быть равно выражение в скобках, и мы получаем, что

$$0 = \ln p(x) + 1 + \lambda_0 + \lambda_1 (x-\mu) + \lambda_2 (x-\mu)^2, \quad \text{то есть}$$
$$p(x) = e^{-1-\lambda_0-\lambda_1(x-\mu)-\lambda_2(x-\mu)^2}.$$

Вы наверняка уже догадываетесь, к чему мы пришли: плотность распределения максимальной энтропии с заданными средним и дисперсией — это экспонента с квадратичной функцией в показателе, то есть нормальное распределение!

[1] Вариационные нижние оценки — один из главных инструментов машинного обучения в целом, но сейчас речь не о них.

Осталось только подставить полученную плотность в условия, чтобы найти значения λ_0, λ_1 и λ_2. Это я оставлю вам в качестве упражнения, потому что результат очевиден: из условий о том, что $p(x)$ должно быть плотностью распределения со средним μ и дисперсией σ^2, получится, что это гауссиан с именно таким средним и именно такой дисперсией:

$$p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{1}{2\sigma^2}(x-\mu)^2}.$$

Метод вариационного исчисления на этом не вполне заканчивается, нужно ещё подтвердить, что результат получился действительно тот, который нам нужен (например, вдруг это минимум энтропии, а не максимум?). Для лучшей иллюстрации происходящего давайте сделаем это другим способом. Вариационное исчисление дало нам толстый намёк, что ответом должен быть гауссиан; воспользуемся этим намёком и запишем дивергенцию Кульбака — Лейблера между произвольным распределением $q(x)$ и гауссианом $p(x)$:

$$0 \le \mathrm{KL}(q\|p) = \int_{-\infty}^{\infty} q(x) \log \frac{q(x)}{p(x)} \mathrm{d}x = \\ = -H(q) - \int_{-\infty}^{\infty} q(x) \log p(x)\, \mathrm{d}x. \quad (3.71)$$

Второе слагаемое — перекрёстную энтропию между $q(x)$ и $p(x)$ — можно подсчитать примерно так же, как мы выше делали для двух гауссианов:

$$\int_{-\infty}^{\infty} q(x) \log p(x)\, \mathrm{d}x = -\int_{-\infty}^{\infty} q(x) \log \sqrt{2\pi\sigma^2} \mathrm{d}x - \int_{-\infty}^{\infty} q(x) \frac{(x-\mu)^2}{2\sigma^2} \mathrm{d}x = \\ = -\frac{1}{2} \log\left(2\pi\sigma^2\right) - \frac{1}{2\sigma^2} \int_{-\infty}^{\infty} q(x) \frac{(x-\mu)^2}{2\sigma^2} \mathrm{d}x = -\frac{1}{2} \log\left(2\pi\sigma^2\right) - \frac{\sigma_q^2}{2\sigma^2},$$

ведь у нас получилось буквально определение дисперсии распределения $q(x)$. Но на дисперсию у нас как раз есть условие: $\sigma_q = \sigma$. А значит, можно воспользоваться результатом (3.61):

$$\int_{-\infty}^{\infty} q(x) \log p(x)\, \mathrm{d}x = -\frac{1}{2} \log\left(2\pi\sigma^2\right) - \frac{1}{2} = -H(p),$$

и теперь дивергенция Кульбака — Лейблера в выражении (3.71) равна

$$0 \le \mathrm{KL}(q\|p) = -H(q) + H(p), \quad \text{то есть} \quad H(p) \ge H(q).$$

В итоге мы получили, что для любого распределения $q(x)$ с заданной дисперсией σ^2 его энтропия $H(q)$ не превышает энтропии нормального распределения $H(p)$, что и требовалось. Рассуждение выше — это уже полноценное формальное доказательство. Обратите внимание, что оно вовсе не апеллирует к вариационному исчислению; последнее мы использовали для того, чтобы при помощи формально не вполне законченных рассуждений догадаться, что ответом будет гауссиан, а в рассуждении выше мы начали с этой гипотезы и уже действительно её доказали.

*3.4.5. Априорные распределения Джеффриса

И ещё одно рассуждение «со звёздочкой», опять посвящённое любопытному и несколько неожиданному развитию темы, которая на первый взгляд кажется очевидной. В подразделе 3.3.4 мы уже немного поговорили о неинформативных априорных распределениях. Мы видели, что равномерное распределение для шаров, которые достают из урны, оказывается не таким уж неинформативным: оно на самом деле соответствует данным $D = (2, 1)$, в которых мы уже достали два шара и увидели, что ровно один из них красный.

Такие «виртуальные данные» даже позволили нам в подразделе 3.3.4 исключить крайние случаи, когда в урне нет ни одного красного или, наоборот, белого шара, и получить «действительно неинформативное» априорное распределение, пропорциональное $\frac{1}{R(N-R)}$ для числа красных шаров $R = 1, \ldots, N-1$. Аналогично равномерное распределение на параметр монетки θ, которое мы рассматривали в разделе 2.4, можно рассматривать как результат «виртуального эксперимента» с двумя подбрасываниями, давшими разные результаты: это как раз будет соответствовать байесовскому сглаживанию в правиле Лапласа.

Как обобщить эти рассуждения? Нам уже ясно, что, поскольку выводы машинного обучения зависят не только от данных, но и от априорных распределений, нам в любой задаче машинного обучения нужно чётко задать последние. Чтобы перейти к математически строгим рассуждениям, нужно сделать следующий шаг: задаться целью получить *объективные* априорные распределения, то есть добиться того, чтобы два субъекта с одной и той же априорной информацией задали одни и те же априорные распределения. Для этого нужно будет придумать какую-то целевую функцию, некое условие, которому должны такие априорные распределения удовлетворять, — и мы сейчас увидим какое.

Вернёмся для начала к монетке. Пытаясь выразить своё незнание, мы в разделе 2.4 вводили равномерное распределение $p(\theta) = \mathrm{Unif}([0,1])$, мотивируя его тем, что мы «не предпочитаем никакого значения θ другому». Но ведь это не единственный способ параметризовать монетку! Давайте, к примеру, перейдём к так называемому *логарифму шансов* (log-odds)

$$\eta = \log \frac{\theta}{1-\theta}, \quad \text{то есть} \quad \theta = \frac{1}{1+e^{-\eta}}.$$

Логарифм шансов позволяет параметризовать монетку величиной, изменяющейся от $-\infty$ до ∞; мы подробно поговорим об этом в подразделе 5.2.1, а в подразделе 6.6.1 увидим, что это как раз более естественный способ параметризации для испытаний Бернулли.

Сейчас нам важно только одно небольшое замечание: так же как мы «не предпочитали никакого значения θ другому», мы, по идее, не предпочитаем и никакого значения η другому. Но если мы сделаем в априорном распределении $p_\theta(\theta) = \mathrm{Unif}([0,1])$ замену переменных $\theta = \frac{1}{1+e^{-\eta}}$, мы получим вовсе не равномерное распределение:

$$p_\eta(\eta) = p_\theta\left(\frac{1}{1+e^{-\eta}}\right)\left(\frac{1}{1+e^{-\eta}}\right)' = \frac{e^\eta}{(1+e^\eta)^2}.$$

Получается, что если мы решаем «не предпочитать» разные значения θ, то мы тем самым очень даже предпочитаем разные значения η; и наоборот, если мы постараемся определить равномерное априорное распределение на η (это уже формально сложнее, потому что η меняется на всей числовой оси $\mathbb{R}$, так что в детали углубляться мы здесь не будем), обратное преобразование приведёт к вовсе не равномерному распределению θ на отрезке $[0,1]$.

Выходит, что мы не можем на самом деле определить распределение, которое не предпочитает одни значения параметров другим; точнее, можем, но только для конкретной параметризации — а это ведь всё-таки одна и та же монетка, и у неё один и тот же параметр, как его ни преобразуй; по крайней мере, монотонные преобразования вроде перехода от θ к η и наоборот уж точно должны быть разрешены. Долгое время это замечание считалось серьёзным аргументом против байесовского подхода к статистике в целом; сам сэр Рональд Фишер критиковал байесовский подход в том числе с этих позиций [597].

Решение этой проблемы предложил тот самый сэр Гарольд Джеффрис [288], о котором мы уже говорили. Он предложил просто считать это свойство требованием к «неинформативному априорному распределению»! Рассмотрим некоторую модель машинного обучения с правдоподобием $p_\theta(\boldsymbol{x}|\theta)$ (пока ограничимся одним параметром $\theta \in \Theta$) и сделаем в ней репараметризацию: применим к параметру θ некое (произвольное) гладкое монотонное преобразование $\eta = h(\theta)$. Тогда заменой переменных мы получим новую модель $p_\eta(\boldsymbol{x}|\eta) = p_\theta\left(D\middle|h^{-1}(\eta)\right)$.

Предположим, что мы так или иначе придумали априорное распределение $p_\theta(\theta)$ для исходной модели. Тогда у нас есть два способа задать априорное распределение после репараметризации:

- мы можем напрямую применить тот же принцип и взять то же распределение $p_\eta(\eta)$, но с параметром η;
- а можем сделать замену переменных $\eta = h(\theta)$ в априорном распределении $p_\theta(\theta)$ и получить распределение

$$\tilde{p}_\eta(\eta) = \frac{p_\theta\left(h^{-1}(\eta)\right)}{|h'\left(h^{-1}(\eta)\right)|}. \tag{3.72}$$

Джеффрис предложил потребовать, чтобы для любой гладкой монотонной функции h полученные этими двумя способами распределения совпадали:

$$p_\eta(\eta) = \tilde{p}_\eta(\eta). \tag{3.73}$$

Чтобы это проверить, проще будет, используя (3.72), переписать (3.73) в виде

$$\tilde{p}_\eta(h(\theta)) = \frac{p_\theta(\theta)}{|h'(\theta)|} \quad \text{для всех} \quad \theta \in \Theta. \tag{3.74}$$

Джеффрис не только предложил такое определение, но и дал конструктивный ответ на вопрос, как построить такое априорное распределение. Оказалось, что для этого нужно использовать ту самую информацию Фишера, которую мы только что обсуждали в подразделе 3.4.4. В случае одного параметра *априорное распределение Джеффриса* (Jeffreys prior) для модели $p_\theta(\boldsymbol{x}|\theta)$ определяется как

$$p^J(\theta) = \text{Const} \cdot \sqrt{\mathcal{I}(\theta)}. \tag{3.75}$$

Проверить свойство (3.74) оказывается не сложно. Давайте запишем определение $\mathcal{I}(\theta)$ и продифференцируем в этом определении $\log p_\eta(\boldsymbol{x}|h(\theta))$ как сложную функцию:

$$\begin{aligned}
\mathcal{I}_\theta(\theta) &= \int \left(\frac{\partial}{\partial\theta}\log p_\theta(\boldsymbol{x}|\theta)\right)^2 p_\theta(\boldsymbol{x}|\theta)\,\mathrm{d}\boldsymbol{x} \\
&= \int \left(\frac{\partial}{\partial\theta}\log p_\eta(\boldsymbol{x}|h(\theta))\right)^2 p_\eta(\boldsymbol{x}|h(\theta))\,\mathrm{d}\boldsymbol{x} \\
&= \int \left(h'(\theta)\frac{\partial}{\partial\eta}\log p_\eta(\boldsymbol{x}|\eta)\right)^2 p_\eta(\boldsymbol{x}|h(\theta))\,\mathrm{d}\boldsymbol{x} = h'(\theta)^2\mathcal{I}_\eta(h(\theta)),
\end{aligned}$$

а значит,

$$p^J_\eta(h(\theta)) = \text{Const} \cdot \sqrt{\mathcal{I}_\eta(h(\theta))} = \text{Const} \cdot \frac{\mathcal{I}_\theta(\theta)}{|h'(\theta)|} = \frac{p^J_\theta(\theta)}{|h'(\theta)|},$$

что и требовалось.

Важное замечание: выражение (3.75) будет полноценным распределением вероятностей только в том случае, если интеграл $\int_\Theta \sqrt{\mathcal{I}(\theta)}$ конечен; тогда этот интеграл даст нам нормировочную константу для плотности, и всё будет хорошо. Но во многих даже вполне практических случаях, как мы увидим ниже, этот интеграл будет расходиться. Что же делать?

Если интеграл расходится, распределение Джеффриса будет задавать так называемое *некорректное априорное распределение* (improper prior), то есть такое априорное распределение, которое само по себе не является распределением вероятностей, но при подстановке в формулу Байеса даёт для любого непустого набора данных полноценное апостериорное распределение. Грубо говоря, в таких случаях мы не будем обращать внимания на то, что интеграл расходится, а будем просто умножать правдоподобие данных на функцию $\sqrt{\mathcal{I}(\theta)}$ и нормировать результат — и вот у этого результата уже действительно интеграл обязательно должен будет сходиться.

Например, для бросания монетки модель в терминах θ выглядит как

$$p\left(x|\theta\right) = \theta^{[\![x=1]\!]}(1-\theta)^{[\![x=0]\!]}$$

для данных, где $x = 1$ означает выпадение орла, а $x = 0$ — решки. Таким образом, априорное распределение Джеффриса можно найти как

$$p^J\left(\theta\right) \propto \sqrt{\mathcal{I}(\theta)} = \sqrt{\mathbb{E}\left[\left(\frac{\partial}{\partial\theta}\log p_\theta\left(D|\theta\right)\right)^2\right]} =$$

$$\sqrt{\mathbb{E}\left[\left(\frac{[\![x=1]\!]}{\theta} - \frac{[\![x=0]\!]}{1-\theta}\right)^2\right]} = \sqrt{\theta\left(\frac{1}{\theta}\right)^2 + (1-\theta)\left(\frac{1}{1-\theta}\right)^2} = \frac{1}{\sqrt{\theta(1-\theta)}}.$$

Получилось уже знакомое вам по подразделу 2.4.5 бета-распределение, которое очень удачно является сопряжённым для бросания монетки, с дробными параметрами $\alpha = \beta = \frac{1}{2}$:

$$p^J\left(\theta\right) = \mathrm{Beta}\left(\theta\middle|\frac{1}{2},\frac{1}{2}\right) \propto \theta^{-\frac{1}{2}}\left(1-\theta\right)^{-\frac{1}{2}}.$$

Аналогично можно получить и априорное распределение Джеффриса для бросания игральной кости с k гранями: это будет распределение Дирихле с дробными параметрами $\boldsymbol{\alpha} = (\,\frac{1}{2}\;\ldots\;\frac{1}{2}\,)$.

В качестве другого примера рассмотрим одномерный гауссиан $\mathcal{N}\left(x|\mu,\sigma^2\right)$; у него два параметра, но здесь, как и в линейной регрессии (см. главу 4), мы предположим, что дисперсия σ^2 фиксирована, а вывод ведётся только по среднему μ. Тогда информация Фишера для одной точки данных x равна

$$\mathcal{I}(\mu) = \mathbb{E}_{x|\mu}\left[\left(\frac{\partial}{\partial\mu}\log p\left(x|\mu\right)\right)^2\right] = \mathbb{E}_{x|\mu}\left[\left(\frac{x-\mu}{\sigma^2}\right)^2\right] =$$

$$= \frac{1}{\sigma^4}\mathbb{E}_{x|\mu}\left[(x-\mu)^2\right] = \frac{\sigma^2}{\sigma^4} = \frac{1}{\sigma^2};$$

а значит, информация Фишера для $D = \{x_1, \ldots, x_N\}$ равна N/σ^2, и априорное распределение Джеффриса получается пропорциональным $p^J(\mu) \propto \sqrt{N/\sigma^2}$. Но для нас в этом выводе σ^2 представляет собой константу, так что получается, что распределение Джеффриса просто равно одной и той же константе для всех μ.

Это характерный пример некорректного априорного распределения: нельзя взять равномерное распределение на всей прямой $\theta \in (-\infty, \infty)$, но для байесовского вывода это не страшно, потому что при умножении на правдоподобие пусть даже одной точки будет получаться суммируемая функция.

А что в многомерном случае? Распределение Джеффриса (3.75) обобщается и на векторный параметр $\boldsymbol{\theta} \in \mathbb{R}^d$:

$$p^J(\boldsymbol{\theta}) = \text{Const} \cdot \sqrt{\det \mathcal{I}(\boldsymbol{\theta})}. \tag{3.76}$$

Базовое свойство проверяется так же, просто в многомерном случае замена переменных добавляет определитель матрицы частных производных; в выводе ниже мы воспользовались тем, что определитель мультипликативен:

$$\begin{aligned}
p^J(\boldsymbol{\eta}) = p^J(\boldsymbol{\theta}) \left|\det \frac{\partial \theta_i}{\partial \eta_j}\right| &\propto \sqrt{\det \mathcal{I}(\boldsymbol{\theta}) \left(\det \frac{\partial \theta_i}{\partial \eta_j}\right)^2} \\
&= \sqrt{\det \frac{\partial \theta_k}{\partial \eta_i} \det \mathbb{E}\left[\frac{\partial \log p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{\theta})}{\partial \theta_k} \frac{\partial \log p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{\theta})}{\partial \theta_l}\right] \det \frac{\partial \theta_l}{\partial \eta_j}} \\
&= \sqrt{\det \mathbb{E}\left[\sum_{k,l} \frac{\partial \theta_k}{\partial \eta_i} \frac{\partial \log p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{\theta})}{\partial \theta_k} \frac{\partial \log p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{\theta})}{\partial \theta_l} \frac{\partial \theta_l}{\partial \eta_j}\right]} \\
&= \sqrt{\det \mathbb{E}\left[\frac{\partial \log p_{\boldsymbol{\eta}}(\boldsymbol{x}|\boldsymbol{\eta})}{\partial \eta_i} \frac{\partial \log p_{\boldsymbol{\eta}}(\boldsymbol{x}|\boldsymbol{\eta})}{\partial \eta_j}\right]} = \sqrt{\det \mathcal{I}(\boldsymbol{\eta})}.
\end{aligned}$$

Для примера давайте найдём распределение Джеффриса $p^J(\boldsymbol{\theta}) \propto \sqrt{\det \mathcal{I}(\boldsymbol{\theta})}$ для одномерного гауссиана с двумя параметрами. Мы находимся в двумерном случае, где $\boldsymbol{\theta} = (\,\mu\ \sigma^2\,)^\top$, то есть матрица информации Фишера задаётся как

$$\mathcal{I}(\mu, \sigma^2) = -\mathbb{E}_{D|\mu,\sigma^2}\left[\begin{pmatrix} \frac{\partial^2}{\partial \mu^2} \log p\left(D\middle|\mu, \sigma^2\right) & \frac{\partial^2}{\partial \mu \partial \sigma^2} \log p\left(D\middle|\mu, \sigma^2\right) \\ \frac{\partial^2}{\partial \sigma^2 \partial \mu} \log p\left(D\middle|\mu, \sigma^2\right) & \frac{\partial^2}{\partial (\sigma^2)^2} \log p\left(D\middle|\mu, \sigma^2\right) \end{pmatrix}\right].$$

Для нормального распределения все эти производные легко подсчитать. Для $D = \{x_1, \ldots, x_n\}$ сначала преобразуем логарифм правдоподобия к достаточным статистикам $\bar{x} = \frac{1}{n}\sum_i x_i$, $s^2 = \frac{1}{n-1}\sum_i (x_i - \bar{x})^2$:

$$\log p\left(D\middle|\mu, \sigma^2\right) = \text{Const} - \frac{n}{2}\log \sigma^2 - \frac{1}{2\sigma^2}\left((n-1)s^2 + n(\bar{x} - \mu)^2\right),$$

а затем возьмём производные:

$$\frac{\partial \log p\left(D\middle|\mu,\sigma^2\right)}{\partial \mu} = \frac{n(\bar{x}-\mu)}{\sigma^2},$$
$$\frac{\partial \log p\left(D\middle|\mu,\sigma^2\right)}{\partial \sigma^2} = -\frac{n}{2\sigma^2} - \frac{(n-1)s^2 + n(\bar{x}-\mu)^2}{4\sigma^4},$$

а значит,

$$\frac{\partial^2}{\partial \mu^2}\log p\left(D\middle|\mu,\sigma^2\right) = -\frac{n}{\sigma^2},$$
$$\frac{\partial^2}{\partial \mu \partial \sigma^2}\log p\left(D\middle|\mu,\sigma^2\right) = \frac{\partial^2}{\partial \sigma^2 \partial \mu}\log p\left(D\middle|\mu,\sigma^2\right) = -\frac{n(\bar{x}-\mu)}{\sigma^4},$$
$$\frac{\partial^2}{\partial (\sigma^2)^2}\log p\left(D\middle|\mu,\sigma^2\right) = \frac{n}{2\sigma^4} - \frac{(n-1)s^2 + n(\bar{x}-\mu)^2}{\sigma^2}.$$

Теперь нужно взять ожидания; поскольку $\mathbb{E}[\bar{x}] = \mu$, $\mathbb{E}\left[(\bar{x}-\mu)^2\right] = \sigma^2/n$ и $\mathbb{E}\left[\sum_i (x_i - \bar{x})^2\right] = (n-1)\sigma^2$, матрица информации Фишера и её определитель равны

$$\mathcal{I}(\mu,\sigma^2) = \begin{pmatrix} \frac{n}{\sigma^2} & 0 \\ 0 & \frac{n}{2\sigma^4} \end{pmatrix}, \qquad \det \mathcal{I}(\mu,\sigma^2) = \frac{n^2}{2\sigma^6},$$

и априорное распределение Джеффриса оказывается равным

$$p^J\left(\mu,\sigma^2\right) \propto \sqrt{\det \mathcal{I}(\boldsymbol{\theta})} \propto \left(\sigma^2\right)^{-3/2}.$$

Обратите внимание, что это распределение совсем не содержит среднее μ — по μ опять получается некорректное равномерное априорное распределение на всей прямой. А вот по σ^2 это полноценное распределение, интеграл по $(0,\infty)$ сходится, и нормировочную константу можно подсчитать.

Из априорного распределения Джеффриса получается такое апостериорное распределение:

$$p\left(\mu,\sigma^2\middle|x_1,\ldots,x_n\right) \propto \left(\sigma^2\right)^{-3/2} \cdot \left(\sigma^2\right)^{-n/2} \cdot e^{-\frac{1}{2\sigma^2}\left((n-1)s^2 + n(\bar{x}-\mu)^2\right)} =$$
$$= \left(\sigma^2\right)^{-(n+3)/2} e^{-\frac{1}{2\sigma^2}\left((n-1)s^2 + n(\bar{x}-\mu)^2\right)} =$$
$$= \left(\frac{1}{\sqrt{\sigma^2}} e^{-\frac{(\bar{x}-\mu)^2}{2(\sigma/\sqrt{n})^2}}\right) \cdot \left(\left(\sigma^2\right)^{-(n+1)/2} e^{-\frac{(n-1)s^2}{2\sigma^2}}\right). \quad (3.77)$$

Первый сомножитель в формуле (3.77) — это (с точностью до константы) нормальное распределение на μ со средним $\bar{x}$ и дисперсией $(\sigma/\sqrt{n})^2$. Получился

вполне логичный результат: среднее постепенно уточняется с ростом n, то есть при получении новых данных.

А вот второй сомножитель, распределение на σ^2, выглядит интереснее — это так называемое *обратное распределение хи-квадрат* (inverse-chi-squared distribution), точнее, его прямое обобщение с дополнительным параметром масштабирования (scaled inverse-chi-squared distribution), плотность которого в общем виде выглядит как

$$p\left(x\middle|\nu,\tau^2\right) = \frac{(\tau^2\nu/2)^{\nu/2}}{\Gamma(\nu/2)}x^{-\nu/2-1}e^{-\frac{\nu\tau^2}{2x}},$$

где параметр ν называется числом *степеней свободы* (degrees of freedom), а τ^2 — параметр масштабирования (scale parameter). В случае (3.77) значения параметров получились равными $\nu = n-1$ и $\tau^2 = s^2$; всё остальное не будет зависеть от σ^2 и уйдёт в нормировочную константу.

Больше мы в этой книге об этом распределении говорить не будем — оно только тем и примечательно, что даёт сопряжённое априорное распределение (и распределение Джеффриса) для параметра σ^2, но байесовский вывод, как вы увидите в разделе 6.4, окажется удобнее вести для обратного параметра $\tau = \frac{1}{\sigma^2}$. В разделе 6.4, особенно начиная с подраздела 6.4.3, мы ещё встретим неожиданные распределения (а именно, распределения Стьюдента), когда будем рассматривать байесовский вывод для гауссиана с параметрами μ и τ. А сейчас давайте двигаться дальше; о теории вероятностей мы уже говорили много, пора немного осветить и другой столп практического машинного обучения — задачи оптимизации и методы их решения.

3.5. Оптимизация в машинном обучении

Кавказ подо мною. Один в вышине
Стою над снегами у края стремнины;
Орёл, с отдалённой поднявшись вершины,
Парит неподвижно со мной наравне.
Отселе я вижу потоков рожденье
И первое грозных обвалов движенье.

Александр Пушкин. Кавказ

3.5.1. Машинное обучение и невыпуклая оптимизация

Мы уже не раз обсуждали основные задачи машинного обучения, которые связаны с теоремой Байеса:

$$p(\boldsymbol{\theta}|D) = \frac{p(D|\boldsymbol{\theta})\,p(\boldsymbol{\theta})}{p(D)}.$$

Среди этих основных задач центральное место занимает поиск тех или иных оптимальных гипотез, который по сути представляет собой задачу оптимизации. Очень часто в машинном обучении мы решаем одну из двух задач — ищем

- либо *гипотезу максимального правдоподобия* $\boldsymbol{\theta}_{\text{ML}} = \arg\max_{\boldsymbol{\theta}} p(D|\boldsymbol{\theta})$,
- либо *максимальную апостериорную гипотезу* $\boldsymbol{\theta}_{\text{MAP}} = \arg\max_{\boldsymbol{\theta}} p(\boldsymbol{\theta}|D)$.

Таким образом, даже после того, как мы завершили все свои математические рассуждения, сделали все нужные предположения и записали все участвующие в байесовском выводе распределения, в результате мы получаем ещё не готовый ответ, а функцию от $\boldsymbol{\theta}$ (скорее всего, логарифм правдоподобия или логарифм апостериорного распределения), которую нужно будет оптимизировать.

С численно-алгоритмической точки зрения задача на этом ещё не решена, экстремум ещё нужно найти. Иногда (как, например, в линейной регрессии — этот случай мы рассмотрим в подразделе 4.1.2) этот экстремум можно найти аналитически. Но в более сложных задачах найти аналитическое решение часто оказывается невозможно (мы это увидим уже при переходе от линейной регрессии к логистической в главе 5) и нам потребуются численные методы оптимизации. Поэтому эти методы занимают очень важное место в машинном обучении.

С другой стороны, в машинном обучении не так часто удаётся применить всё богатство методов и результатов теории оптимизации. Дело в том, что функции правдоподобия и апостериорные распределения у моделей машинного обучения обычно имеют очень сложную и совсем не выпуклую форму, с огромным (обычно экспоненциально большим) числом локальных максимумов. В такой ситуации классическая теория оптимизации не так уж много что может предложить,

и наша задача, по сути, сводится к тому, чтобы найти хотя бы *локальный* максимум апостериорного распределения. Методы оптимизации тоже так или иначе превращаются в итеративные методы поиска локальных экстремумов.

С другой стороны, хоть функции правдоподобия и имеют сложную форму, но всё же они обычно гладкие, так что мы обычно можем подсчитать и первые, и вторые производные этих функций. Это значит, что центральными методами оптимизации в машинном обучении становятся:

- *градиентный спуск* (gradient descent), оптимизация первого порядка, при которой мы в каждый момент времени итеративно смещаемся в направлении, обратном градиенту (если это спуск, то есть минимизация функции) или совпадающем с ним (если это подъём, то есть максимизация функции);
- *метод Ньютона*, то есть метод второго порядка, в котором мы в каждый момент времени фактически строим аппроксимацию по формуле Тейлора до второго порядка и сдвигаемся в экстремум соответствующей квадратичной функции.

Давайте запишем это чуть более формально и обсудим. Предположим, что после построения модели мы для каждого элемента данных получили некоторую функцию ошибки $f(\boldsymbol{\theta}, d)$, где d — это единица данных (нам здесь неважно даже, с учителем это обучение или без), а $\boldsymbol{\theta}$ — параметры модели. А задача оптимизации состоит в том, чтобы минимизировать общую функцию ошибки по некоторому датасету D относительно $\boldsymbol{\theta}$:

$$F(\boldsymbol{\theta}) = \sum_{d \in D} f(\boldsymbol{\theta}, d) \rightarrow_{\boldsymbol{\theta}} \min .$$

Как правило, в задачах машинного обучения $F(\boldsymbol{\theta})$ — это взятый с обратным знаком логарифм правдоподобия $-\ln p\,(D|\boldsymbol{\theta})$ или логарифм апостериорного распределения $-\ln p\,(\boldsymbol{\theta}|D)$. А предположение о том, что $F(\boldsymbol{\theta})$ представляет собой сумму функций ошибки для каждого примера $f(\boldsymbol{\theta}, d)$ по $d \in D$, — это одно из главных вероятностных предположений, которое делается в машинном обучении буквально всегда: при условии известных параметров модели $\boldsymbol{\theta}$ каждая точка данных d порождается независимо от других точек.

Алгоритм 1 показывает обычный, «ванильный» градиентный спуск: на каждом шаге мы вычисляем градиент минимизируемой функции в текущей точке и движемся в сторону, противоположную градиенту. В обычном градиентном спуске многое зависит от скорости обучения α. Например, можно вполне разумно предположить, что скорость обучения должна уменьшаться со временем, и тогда первой мыслью будет выбрать некоторое расписание, по которому мы будем постепенно уменьшать α. Оно может быть

- линейным:

$$\alpha = \alpha_0 \left(1 - \frac{t}{T}\right)$$

Алгоритм 1. Градиентный спуск

Выберем $\boldsymbol{\theta}_0$, положим $k := 1$;
repeat
  $\boldsymbol{\theta}_{k+1} := \boldsymbol{\theta}_k - \alpha \sum_{d \in D} \nabla_{\boldsymbol{\theta}} F(\boldsymbol{\theta}_k, d)$;
  $k := k + 1$;
until *выполнено некоторое условие остановки*;

- или экспоненциальным: $\alpha = \alpha_0 e^{-\frac{t}{T}}$.

В обоих случаях константа T обычно называется *температурой*, потому что действительно играет роль, похожую на роль температуры в статистической механике: чем она больше, тем медленнее скорость обучения убывает со временем.

3.5.2. Анализ градиентного спуска: проблемы с масштабом

В предыдущем разделе мы обсудили всего лишь самые первые интуитивные идеи, которые можно значительно улучшить. Здесь как раз теория оптимизации может сказать нам очень много интересного; есть ряд результатов, исследующих свойства градиентного спуска и поиска оптимального значения α на каждом шаге. По причинам, которые станут ясны уже скоро, мы обсудим эти результаты только очень вкратце, а для более подробного ознакомления с ними и вообще с основными идеями теории оптимизации рекомендую, например, книги и обзоры [81, 430, 442, 444].

Основная идея здесь состоит в том, что, если мы минимизируем $f(\boldsymbol{\theta})$ градиентным спуском и на шаге k мы уже нашли направление $\boldsymbol{p}_k$, вдоль которого собираемся двигаться, — например, в обычном градиентном спуске это направление $\boldsymbol{p}_k = \nabla_{\boldsymbol{\theta}} f(\boldsymbol{\theta}_k)$, — задача превращается в одномерную задачу оптимизации:

$$\min_\alpha f(\boldsymbol{\theta}_k + \alpha \boldsymbol{p}_k),$$

и её теперь можно пытаться решать, находя оптимальное расстояние, на которое хочется сдвинуться вдоль вектора $\boldsymbol{p}$.

В частности, ещё с 1960-х годов известны *условия Вульфа* (Wolfe conditions) и *правило Армихо* (Armijo rule). Эти результаты выглядят так:

- для $\phi_k(\alpha) = f(\boldsymbol{\theta}_k + \alpha \boldsymbol{p}_k)$ мы получаем, что $\phi_k'(\alpha) = \nabla f(\boldsymbol{\theta}_k + \alpha \boldsymbol{p}_k)^\top \boldsymbol{p}_k$, и если $\boldsymbol{p}_k$ является направлением спуска, то $\phi_k'(0) < 0$;
- размер шага оптимизации α должен удовлетворять правилу Армихо:

$$\phi_k(\alpha) \le \phi_k(0) + c_1 \alpha \phi_k'(0) \text{ для некоторого } c_1 \in (0, 1/2);$$

- или более сильным условиям Вульфа, которые добавляют требование

$$|\phi_k'(\alpha)| \le c_2 |\phi_k'(0)|;$$

иначе говоря, условия Вульфа говорят, что нам хочется уменьшить проекцию градиента на текущий вектор смещения $\boldsymbol{p}_k$, сделать градиент в новой точке как можно меньше и как можно «более перпендикулярным» $\boldsymbol{p}_k$.

При таком градиентном спуске процесс оптимизации нужно останавливать согласно условию на L_2-норму градиента, то есть мы будем прекращать градиентный спуск в тот момент, когда начинают выполняться условия $\|\nabla_{\boldsymbol{\theta}} f(\boldsymbol{\theta}_k)\|^2 \leq \epsilon$ или $\|\nabla_{\boldsymbol{\theta}} f(\boldsymbol{\theta}_k)\|^2 \leq \epsilon \|\nabla_{\boldsymbol{\theta}} f(\boldsymbol{\theta}_0)\|^2$ для некоторой константы ϵ.

Однако у методов первого порядка есть важные недостатки. В частности, методы вроде градиентного спуска начинают плохо работать, когда встречаются с задачами, переменные в которых имеют разный *масштаб*. Это часто бывает в машинном обучении: разные параметры из вектора $\boldsymbol{\theta}$ могут иметь совершенно разный смысл. Проблема здесь в том, что градиентный спуск выбирает единую скорость для движения вдоль градиента и, скорее всего, будет эту скорость со временем только уменьшать. А эта единая скорость может оказаться неподходящей для разных масштабов переменных, и подобрать подходящую может быть нелегко.

Этот эффект можно увидеть даже на самых простых примерах. Давайте воспользуемся этим поводом, чтобы ввести в наши программные примеры библиотеку *PyTorch* [483]. Обучение искусственных нейронных сетей практически всегда происходит при помощи того или иного варианта градиентного спуска. Нейронная сеть представляет собой граф вычислений, и для того, чтобы провести градиентный спуск, нужно уметь считать градиент по весам; мы не будем обсуждать искусственные нейронные сети в этой книге, но когда-нибудь, надеюсь, дойдём и до них, а пока сошлюсь на свою предыдущую книгу [685].

Поэтому современные библиотеки для глубокого обучения — *PyTorch*, *Tensorflow* и так далее — по сути представляют собой библиотеки *автоматического дифференцирования*: их основная функция состоит в том, чтобы уметь быстро и эффективно подсчитать градиент от очень сложной функции. Конечно, заодно они ещё и реализуют массу разных алгоритмов оптимизации, готовые конструкции для нейронных сетей и так далее, но главное — это именно дифференцирование.

Давайте применим стандартные методы из *PyTorch* для оптимизации градиентным спуском очень простой квадратичной функции вида

$$f(x,y) = x^2 + \rho y^2.$$

```
import torch
import torch.optim as optim
x0, y0, rho = 1., 1.5, 0.5
def quadratic_function(x, y):
    return x**2 + rho * (y**2)

def compute_sgd(f, n=100, lrs=[0.01, 0.001, 0.0001], x0=-2.0, y0=-2.0):
```

```
    results = {}
    for lr in lrs:
        x = torch.tensor([x0], requires_grad=True)
        y = torch.tensor([y0], requires_grad=True)
        x_hist, y_hist = [x0], [y0]
        optimizer = optim.SGD([x, y], lr=lr)

        def closure():
            optimizer.zero_grad()
            ff = f(x, y)
            ff.backward()
            return ff

        for _ in range(n):
            optimizer.step(closure)
            x_hist.append(x.detach().numpy()[0])
            y_hist.append(y.detach().numpy()[0])
        results[lr] = (x_hist, y_hist)
    return results
```

В приведённом выше коде происходит следующее: мы объявляем две переменные, по которым будет производиться оптимизация:

```
x = torch.tensor([x0], requires_grad=True)
y = torch.tensor([y0], requires_grad=True)
```

Параметр `requires_grad` как раз и значит, что по x и y мы потом будем брать производные, они войдут в вектор градиента функции f. Дальше мы объявляем стандартный оптимизатор, который реализует алгоритм градиентного спуска (точнее, *стохастического* градиентного спуска, но об этом ниже; в этом примере разницы нет):

```
optimizer = optim.SGD([x, y], lr=lr)
```

Обратите внимание, что оптимизатор пока ещё не знает, какую функцию оптимизировать, он инициализируется набором переменных, по которым будет идти оптимизация, и скоростью обучения `lr`.

Функция `closure` — это функция, в которой реализован один шаг подсчёта градиента. Здесь *PyTorch* делает всю содержательную работу за нас: мы должны всего лишь обнулить градиенты и запустить функцию `backward`, которая реализует обратное распространение ошибки, то есть подсчитывает $\frac{\partial f}{\partial x}$ и $\frac{\partial f}{\partial y}$ в их текущих значениях. Эту функцию мы подаём на вход каждому шагу оптимизатора:

```
for _ in range(n):
    optimizer.step(closure)
```

И всё, остальное здесь просто обвязка для того, чтобы сохранять полученные результаты по ходу оптимизации. В результате значения переменных x и y будут

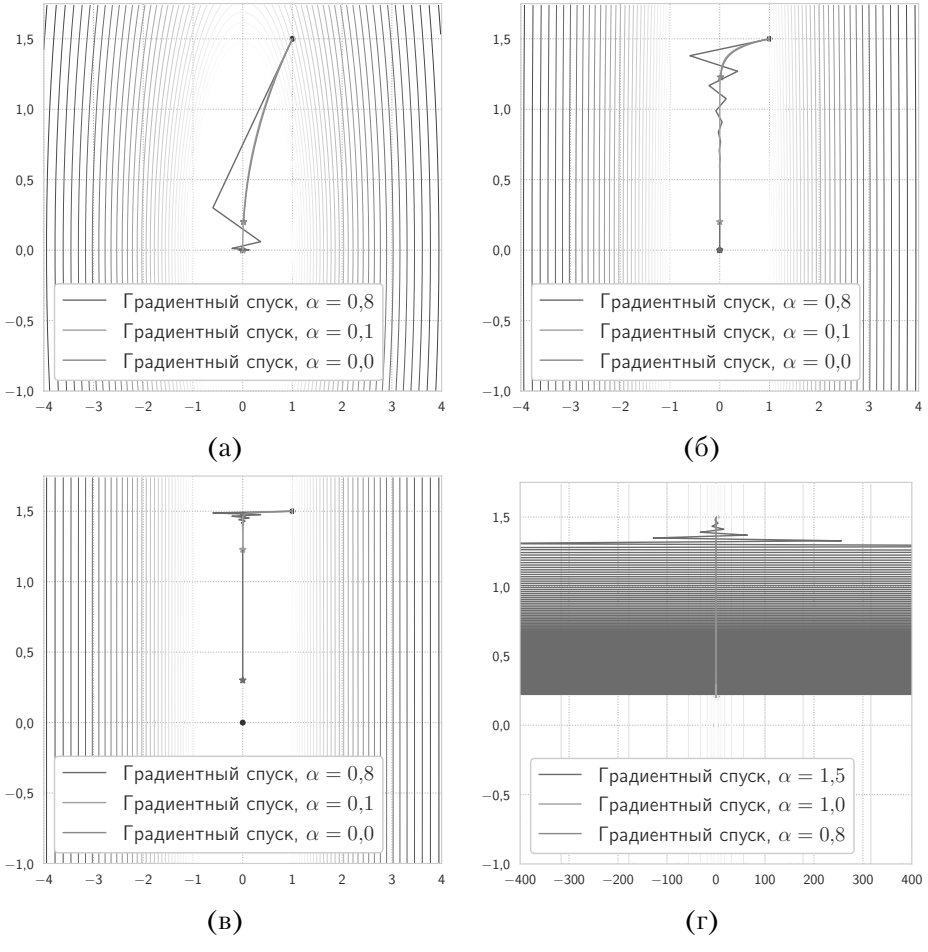


Рис. 3.23. Пример градиентного спуска с разным масштабом переменных: (а) $f(x,y) = x^2 + \frac{1}{2}y^2$; (б) $f(x,y) = x^2 + \frac{1}{10}y^2$; (в) $f(x,y) = x^2 + \frac{1}{100}y^2$; (г) $f(x,y) = x^2 + \frac{1}{100}y^2$ для больших значений α

постепенно меняться (менять их будет `optimizer`) согласно алгоритму 1 со скоростью обучения `lr`. Теперь можно вызывать функцию `compute_sgd` и получать историю промежуточных результатов оптимизации градиентным спуском, причём можно задавать и сравнивать сразу несколько скоростей обучения.

На рис. 3.23 показаны три графика, иллюстрирующие оптимизацию градиентным спуском для трёх квадратичных функций, имеющих вид $x^2 + \rho y^2$ для разных значений ρ: $x^2 + \frac{1}{2}y^2$ на рис. 3.23, *а*, $x^2 + \frac{1}{2}y^2$ на рис. 3.23, *б* и $x^2 + \frac{1}{2}y^2$ на рис. 3.23, *в*, *г*. Каждая кривая построена для 200 шагов градиентного спуска.

Казалось бы, функцию мы выбрали проще не придумаешь: $f(x,y)$ является выпуклой функцией с единственным минимумом в нуле, все дороги ведут в него и всё должно работать как часы. На рис. 3.23, *а*, где масштаб переменных примерно одинаковый, так и есть. А дальше начинаются проблемы: на рис. 3.23, *б* мы видим, что то же самое значение скорости обучения уже приводит к оптимуму не так быстро, а на рис. 3.23, *в* градиентный спуск замедляется уже совсем радикально и кривые, соответствующие $\alpha = 0{,}1$ и $\alpha = 0{,}01$, вполне разумных скоростей обучения в других случаях, доползут до нуля ещё ой как не скоро — а это ведь 200 (!) итераций для такой тривиальной, казалось бы, функции.

Более того, эта проблема не решается простым увеличением скорости обучения: в какой-то момент скорость обучения станет слишком большой и градиентный спуск перестанет сходиться вообще. В данном случае ясно, что это произойдёт при $\alpha > 1$: тогда проекция $\alpha\nabla f$ на горизонтальную ось станет по модулю больше, чем текущее значение координаты x, и процесс разойдётся. Конечно, теоретически можно сказать, что надо выбрать скорость обучения $1 - \epsilon$, и при достаточно малом ϵ градиентный спуск будет вести себя хорошо... но как же нам такое волшебное значение найти в общем случае, когда f — это очень сложная функция, скорее всего, логарифм какого-нибудь сложного правдоподобия? А просто «разумно большие» значения не помогут: на рис. 3.23 я «угадал» значение $\alpha = 0{,}8$, но даже оно спасает далеко не полностью и всё равно требует сотен итераций для $\rho = \frac{1}{100}$, а это ещё далеко не худший случай. И самое печальное, что задачи с такими плохими ситуациями часто встречаются в машинном обучении: достаточно, чтобы параметры модели, входящие в вектор $\boldsymbol{\theta}$, имели разный смысл или разную природу или чтобы у входных данных были компоненты разного масштаба. Простое нормирование входов тут не поможет, потому что речь идёт в том числе и о масштабе переменных «внутри» модели, который мы заранее не знаем. Что же делать?

В таких случаях хочется сделать градиентный спуск *адаптивным*, чтобы он мог по-разному выбирать скорость обучения для разных направлений в пространстве параметров $\boldsymbol{\theta}$. И здесь есть единый ответ о том, как сделать это масштабирование максимально соответствующим форме функции F: для этого нужно перейти к *методам второго порядка*, например к *методу Ньютона*. Давайте разложим функцию $F(\boldsymbol{\theta})$ до второго члена ряда Тейлора в текущей точке $\boldsymbol{\theta}_k$:

$$F(\boldsymbol{\theta}) \approx F(\boldsymbol{\theta}_k) + \nabla_{\boldsymbol{\theta}} F(\boldsymbol{\theta}_k)^\top (\boldsymbol{\theta} - \boldsymbol{\theta}_k) + \frac{1}{2} (\boldsymbol{\theta} - \boldsymbol{\theta}_k)^\top H_k (\boldsymbol{\theta} - \boldsymbol{\theta}_k),$$

где через

$$H_k = \nabla^2_{\boldsymbol{\theta}} F(\boldsymbol{\theta}_k) = \begin{pmatrix} \frac{\partial^2 F(\boldsymbol{\theta}_k)}{\partial {\theta_1}^2} & \frac{\partial^2 F(\boldsymbol{\theta}_k)}{\partial \theta_1 \partial \theta_2} & \cdots & \frac{\partial^2 F(\boldsymbol{\theta}_k)}{\partial \theta_1 \partial \theta_d} \\ \frac{\partial^2 F(\boldsymbol{\theta}_k)}{\partial \theta_2 \partial \theta_1} & \frac{\partial^2 F(\boldsymbol{\theta}_k)}{\partial {\theta_2}^2} & \cdots & \frac{\partial^2 F(\boldsymbol{\theta}_k)}{\partial \theta_2 \partial \theta_d} \\ \vdots & \vdots & & \vdots \\ \frac{\partial^2 F(\boldsymbol{\theta}_k)}{\partial \theta_d \partial \theta_1} & \frac{\partial^2 F(\boldsymbol{\theta}_k)}{\partial \theta_d \partial \theta_2} & \cdots & \frac{\partial^2 F(\boldsymbol{\theta}_k)}{\partial {\theta_d}^2} \end{pmatrix}$$

мы обозначили матрицу вторых производных, которую часто называют *гессианом* (Hessian; отсюда и буква H)[1].

Полученную методом второго порядка аппроксимацию сразу можно минимизировать точно, аналитически (тем же самым рассуждением мы сразу получим точный ответ для метода наименьших квадратов в подразделе 4.1.2). Если взять градиент по $\boldsymbol{\theta}$, получится

$$\nabla_{\boldsymbol{\theta}} F(\boldsymbol{\theta}_k) + H_k\boldsymbol{\theta} - H_k\boldsymbol{\theta}_k,$$

и если его теперь приравнять к нулю и умножить полученное равенство слева на H_k^{-1}, мы получим правило обновления весов для метода Ньютона:

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - H_k^{-1}\nabla_{\boldsymbol{\theta}} F(\boldsymbol{\theta}_k).$$

В методе Ньютона матрица H_k^{-1} служит как раз «правильной» перенормировкой весов, которая учитывает соотношение между требующимися скоростями движения по разным компонентам входа. Для метода Ньютона тоже есть своя большая теория, свой вариант правила Армихо и так далее.

Но мы не будем углубляться в теорию методов оптимизации второго порядка, потому что, к сожалению, в машинном обучении их очень редко удаётся применить. В этой книге вы увидите метод Ньютона в главе 4 для линейной регрессии и в главе 5 для логистической регрессии, где матрицу H удастся подсчитать аналитически... но этим всё и ограничится. Дело в том, что в реальности, чтобы решить сложную задачу машинного обучения, нам, скорее всего, потребуется модель с достаточно большим числом параметров, то есть размерность вектора $\boldsymbol{\theta}$ будет велика. И это обычно означает, что попытки на каждом шаге обучения численно подсчитать матрицу H, в которой элементов квадратично больше, вычислительно будут слишком сложны.

Здесь, впрочем, теория оптимизации тоже предлагает разумный ответ: если точно подсчитать гессиан слишком сложно, то можно попробовать подсчитать его приближённо. Это происходит в так называемых *квазиньютоновских методах* (quasi-Newton methods), где строится аппроксимация гессиана H при помощи значений самой функции и её градиента.

И конечно, квазиньютоновские методы было бы замечательно использовать в машинном обучении... но тоже не получается. Дело в том, что для того, чтобы эта оценка гессиана была хоть сколько-нибудь разумной, нужно всё-таки знать

[1] *Людвиг Отто Гессе* (Ludwig Otto Hesse, 1811–1874) — немецкий математик. В жизни Гессе было не так много ярких событий: он родился в Кёнигсберге и учился там же (в частности, у Фридриха Бесселя), затем переехал через Гейдельберг в Мюнхен, где преподавал в Политехнической школе. Основные результаты Гессе относятся к теории алгебраических инвариантов и геометрии; в его честь назван не только гессиан, но и нормальная форма Гессе для гиперплоскостей, конфигурация Гессе (реализуемая на комплексной проективной плоскости, но не на евклидовой), группа Гессе и пучок Гессе.

хотя бы значения градиента целевой функции. Но даже этого мы в машинном обучении сделать не можем! О том, почему так и как с этим бороться, мы поговорим в следующем подразделе.

3.5.3. Стохастический градиентный спуск

Предыдущий раздел закончился на небольшом cliffhanger'е: казалось бы, что может быть проще, чем подсчитать градиент функции ошибки? В нейронных сетях, например, для этого есть обратное распространение ошибки, и все библиотеки для нейронных сетей дифференцируют любые графы вычислений совершенно автоматически.

Однако на самом деле проблема гораздо серьёзнее. Мы действительно без труда можем подсчитать градиент функции ошибки $\nabla_{\boldsymbol{\theta}} f(\boldsymbol{\theta}, d)$ для одной точки данных d; например, при максимизации правдоподобия обычно легко подсчитать градиент $f(\boldsymbol{\theta}, d) = -\log p(d|\boldsymbol{\theta})$. Но на самом деле полностью функция ошибки представляет собой сумму по всему датасету:

$$F(\boldsymbol{\theta}) = \sum\nolimits_{d \in D} f(\boldsymbol{\theta}, d), \qquad \nabla_{\boldsymbol{\theta}} F(\boldsymbol{\theta}) = \sum\nolimits_{d \in D} \nabla_{\boldsymbol{\theta}} f(\boldsymbol{\theta}, d).$$

И выходит, что в реальности, для настоящих датасетов, которых будет достаточно для решения задач в реальной жизни, нет никакой возможности подсчитывать градиент *всей* функции ошибки на каждом шаге оптимизации: таких шагов много и каждый раз проходить по всему датасету было бы крайне неэффективно. В реальности мы можем предполагать, что на каждом шаге оптимизации можем рассчитывать только на градиент в отдельной точке данных или на небольшом их подмножестве. Это значит, что не только методы второго порядка, но и квазиньютоновские методы нам не светят... а что вообще светит? Казалось бы, даже градиентный спуск теряет смысл — неужели совсем всё пропало?

К счастью, не всё. В машинном обучении обычно используется так называемый *стохастический градиентный спуск* (stochastic gradient descent, SGD), общая схема которого показана в алгоритме 2. Разница с алгоритмом 1 состоит в том, что на каждом шаге градиент подсчитывается не по всему датасету D, а по некоторой подвыборке D_k из этого датасета. В машинном обучении такие подвыборки называются *мини-батчами* (mini-batches). Обычно их размер определяется теми или иными практическими соображениями эффективности; например, в глубоком обучении часто размер мини-батча выбирается так, чтобы соответствующее число копий нейронной сети помещалось в память имеющегося у нас устройства (например, видеокарты).

Но ведь это совсем не то же самое, что градиентный спуск, который мы обсуждали выше? Формально это действительно не совсем то, но оказывается, что навести между ними мосты всё-таки можно. В терминах теории оптимизации стохастический градиентный спуск представляет собой частный случай алгоритма для решения *задачи стохастической оптимизации*:

Алгоритм 2. Стохастический градиентный спуск с мини-батчами

Инициализировать $\boldsymbol{\theta}_0$, $k := 0$;
repeat
  $D_k := \text{Sample}(D)$;
  $\boldsymbol{\theta}_{k+1} := \boldsymbol{\theta}_k - \alpha \sum_{d \in D_k} \nabla_{\boldsymbol{\theta}} f(\boldsymbol{\theta}_k, d)$;
  $k := k + 1$;
until *выполнено некоторое условие остановки*;

$$F(\boldsymbol{\theta}) = \mathbb{E}_{q(\boldsymbol{y})} \left[f(\boldsymbol{\theta}, \boldsymbol{y}) \right] \to \min_{\boldsymbol{\theta}},$$

где $q(\boldsymbol{y})$ — это некоторое заранее известное распределение. Основной пример здесь — минимизация *эмпирического риска* (empirical risk):

$$F(\boldsymbol{\theta}) = \frac{1}{N} \sum_{i=1}^{N} f_i(\boldsymbol{\theta}) = \mathbb{E}_{i \sim \text{Unif}(1,\ldots,N)} \left[f_i(\boldsymbol{\theta}) \right] \to \min_{\boldsymbol{\theta}}.$$

Другой важный пример — это минимизация вариационной нижней оценки, но об этом мы сможем всерьёз поговорить только в последующих книгах.

Такая формализация позволяет нам понять, что такое мини-батчи с математической точки зрения. Усреднение по мини-батчу — это фактически подсчёт эмпирической оценки для целевой функции стохастической оптимизации на некоторой подвыборке:

$$\hat{F}(\boldsymbol{\theta}) = \frac{1}{m} \sum_{i=1}^{m} f(\boldsymbol{\theta}, \boldsymbol{y}_i), \qquad \hat{\boldsymbol{g}}(\boldsymbol{\theta}) = \frac{1}{m} \sum_{i=1}^{m} \nabla_{\boldsymbol{\theta}} f(\boldsymbol{\theta}, \boldsymbol{y}_i),$$

где m — размер мини-батча.

С одной стороны, мы знаем, что у таких оценок (это оценка математического ожидания через среднее) масса приятных для нас свойств: они несмещённые, всегда сходятся к истинному значению ожидания, их легко вычислить и так далее. С другой стороны, дисперсия этих оценок всё-таки огромна: размер мини-батча обычно на несколько порядков меньше, чем размер всего датасета, и трудно ожидать, что у настоящего градиента целевой функции будет много общего с каждой из этих стохастических оценок.

Именно поэтому, хотя стохастический градиентный спуск можно рассматривать по сути как стохастический вариант градиентного спуска, аналогичных стохастических вариантов у тех же квазиньютоновских методов построить не получается: слишком велика дисперсия оценок градиентов и после преобразований

метода BFGS гессиан получается совсем уж случайным. Разработка стохастических вариантов квазиньютоновских методов остаётся интересной и потенциально революционной, но, по всей видимости, слишком сложной открытой задачей.

Но и у самого SGD, метода первого порядка, есть несколько проблем:

- он никогда не движется в точности в верном направлении (в направлении градиента);
- более того, оценки градиента из SGD не станут нулевыми, даже если нам повезёт попасть ровно в точку минимума функции $F(\boldsymbol{\theta})$, то есть мы не заметим, что попали в минимум, и алгоритм 2 никогда не сойдётся, если не уменьшать со временем скорость обучения;
- да и со скоростью обучения не всё ясно: поскольку мы не знаем ни $F(\boldsymbol{\theta})$, ни $\nabla_{\boldsymbol{\theta}} F$, только их стохастические оценки с огромными дисперсиями, мы не можем применять правило Армихо или другие результаты классической теории оптимизации, чтобы найти оптимальную длину шага.

3.5.4. Свойства стохастического градиентного спуска

Давайте проанализируем стохастический градиентный спуск более тщательно и попробуем выяснить некоторые его свойства.

Рассмотрим одну итерацию SGD для некоторой целевой функции

$$F(\boldsymbol{\theta}) = \mathbb{E}_{q(\boldsymbol{y})}\left[f(\boldsymbol{\theta}, \boldsymbol{y})\right] \to_{\boldsymbol{\theta}} \min.$$

Далее в этом разделе мы обозначим через $\boldsymbol{g}_k$ градиент F в точке $\boldsymbol{\theta}_k$, а через $\hat{\boldsymbol{g}}_k$ — его эмпирическую оценку на шаге k, то есть стохастический градиентный спуск будет выглядеть как

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \alpha_k \hat{\boldsymbol{g}}_k, \quad \mathbb{E}\hat{\boldsymbol{g}}_k = \boldsymbol{g}_k = \nabla_{\boldsymbol{\theta}} F(\boldsymbol{\theta}_k).$$

Попробуем оценить *невязку* (residue) на шаге k, то есть остающуюся разницу между текущим значением функции $F(\boldsymbol{\theta}_k)$ и настоящим оптимальным значением. Обозначая через $\boldsymbol{\theta}_*$ точку настоящего оптимума, получаем

$$\|\boldsymbol{\theta}_{k+1} - \boldsymbol{\theta}_*\|^2 = \|\boldsymbol{\theta}_k - \alpha_k \hat{\boldsymbol{g}}_k - \boldsymbol{\theta}_*\|^2 = \|\boldsymbol{\theta}_k - \boldsymbol{\theta}_*\|^2 - 2\alpha_k \hat{\boldsymbol{g}}_k^\top (\boldsymbol{\theta}_k - \boldsymbol{\theta}_*) + \alpha_k^2 \|\hat{\boldsymbol{g}}_k\|^2.$$

Возьмём слева и справа ожидания по выборке из $q(\boldsymbol{y})$ на шаге k:

$$\mathbb{E}\left[\|\boldsymbol{\theta}_{k+1} - \boldsymbol{\theta}_*\|^2\right] = \|\boldsymbol{\theta}_k - \boldsymbol{\theta}_*\|^2 - 2\alpha_k \boldsymbol{g}_k^\top (\boldsymbol{\theta}_k - \boldsymbol{\theta}_*) + \alpha_k^2 \mathbb{E}\left[\|\hat{\boldsymbol{g}}_k\|^2\right]. \tag{3.78}$$

Для дальнейшего анализа нам потребуется предположение, которое может показаться странным: далее мы будем предполагать, что целевая функция F выпуклая. Иногда целевые функции в этой книге действительно будут выпуклыми, но только в самых простых случаях; разве что в линейной регрессии в главе 4 так

действительно получится. Начиная с хоть сколь-нибудь сложных моделей машинного обучения, соответствующие логарифмы правдоподобий и логарифмы апостериорных распределений уже никак не смогут считаться выпуклыми.

Но на самом деле для исследования *локальных* свойств часто и в невыпуклой оптимизации можно предполагать, что целевая функция выпукла, ведь в некоторой окрестности нужного нам экстремума это действительно так. Поэтому просто надо понимать, что всё наше дальнейшее рассуждение касается сугубо *локальной* оптимизации в некоторой *окрестности* минимума функции F и никаких *глобальных* гарантий эти рассуждения дать не могут.

С этим пониманием давайте всё-таки предположим выпуклость, то есть предположим, что

$$F(\boldsymbol{\theta}_*) \geq F(\boldsymbol{\theta}_k) + \boldsymbol{g}_k^\top(\boldsymbol{\theta}_k - \boldsymbol{\theta}_*).$$

Теперь соединим это с формулой (3.78) для $\mathbb{E}\left[\|\boldsymbol{\theta}_{k+1} - \boldsymbol{\theta}_*\|^2\right]$:

$$\begin{aligned}\alpha_k(F(\boldsymbol{\theta}_k) - F(\boldsymbol{\theta}_*)) \leq \alpha_k \boldsymbol{g}_k^\top(\boldsymbol{\theta}_k - \boldsymbol{\theta}_*) = \\ = \frac{1}{2}\|\boldsymbol{\theta}_k - \boldsymbol{\theta}_*\|^2 + \frac{1}{2}\alpha_k^2 \mathbb{E}\left[\|\hat{\boldsymbol{g}}_k\|^2\right] - \frac{1}{2}\mathbb{E}\left[\|\boldsymbol{\theta}_{k+1} - \boldsymbol{\theta}_*\|^2\right].\end{aligned}$$

Возьмём ожидания от левых частей и просуммируем их:

$$\begin{aligned}\sum_{i=0}^{k} \alpha_i(\mathbb{E}\left[F(\boldsymbol{\theta}_i)\right] - F(\boldsymbol{\theta}_*)) \leq \frac{1}{2}\|\boldsymbol{\theta}_0 - \boldsymbol{\theta}_*\|^2 + \frac{1}{2}\sum_{i=0}^{k}\alpha_i^2 \mathbb{E}\left[\|\hat{\boldsymbol{g}}_i\|^2\right] - \\ - \frac{1}{2}\mathbb{E}\left[\|\boldsymbol{\theta}_{k+1} - \boldsymbol{\theta}_*\|^2\right] \leq \frac{1}{2}\|\boldsymbol{\theta}_0 - \boldsymbol{\theta}_*\|^2 + \frac{1}{2}\sum_{i=0}^{k}\alpha_i^2 \mathbb{E}\left[\|\hat{\boldsymbol{g}}_i\|^2\right].\end{aligned}$$

Таким образом, мы получили сумму значений функции в разных точках с весами α_i. Воспользуемся теперь предположением о выпуклости:

$$\begin{aligned}\mathbb{E}\left[F\left(\frac{\sum_i \alpha_i \boldsymbol{\theta}_i}{\sum_i \alpha_i}\right) - F(\boldsymbol{\theta}_*)\right] \leq \\ \leq \frac{\sum_i \alpha_i(\mathbb{E}\left[F(\boldsymbol{\theta}_i)\right] - F(\boldsymbol{\theta}_*)}{\sum_i \alpha_i} \leq \frac{\frac{1}{2}\|\boldsymbol{\theta}_0 - \boldsymbol{\theta}_*\|^2 + \frac{1}{2}\sum_{i=0}^{k}\alpha_i^2 \mathbb{E}\left[\|\hat{\boldsymbol{g}}_i\|^2\right]}{\sum_i \alpha_i}.\end{aligned}$$

Итак, мы получили оценку на ожидаемую невязку $\mathbb{E}\left[F\left(\frac{\sum_i \alpha_i \boldsymbol{\theta}_i}{\sum_i \alpha_i}\right) - F(\boldsymbol{\theta}_*)\right]$ в некоторой промежуточной точке $\hat{\boldsymbol{\theta}}_k = \frac{\sum_i \alpha_i \boldsymbol{\theta}_i}{\sum_i \alpha_i}$, представляющей собой линейную комбинацию $\boldsymbol{\theta}_i$. Это тоже часто встречающаяся в теории оптимизации ситуация: несмотря на то что теоретические гарантии мы получили только для точки $\hat{\boldsymbol{\theta}}_k$, на практике обычно нет разницы между этой промежуточной точкой и последней точкой $\boldsymbol{\theta}_k$ и невязка в точке $\boldsymbol{\theta}_k$ обычно может быть только лучше.

Другими словами, мы доказали, что если начальное отклонение от оптимума ограничено константой R, то есть $\|\boldsymbol{\theta}_0 - \boldsymbol{\theta}_*\| \leq R$, и если дисперсия стохастической оценки градиента ограничена константой G, то есть $\mathbb{E}\left[\|\hat{\boldsymbol{g}}_k\|^2\right] \leq G^2$, то

$$\mathbb{E}\left[F(\hat{\boldsymbol{\theta}}_k) - F(\boldsymbol{\theta}_*)\right] \leq \frac{R^2 + G^2 \sum_{i=0}^{k} \alpha_i^2}{2 \sum_{i=0}^{k} \alpha_i}. \tag{3.79}$$

Это главный результат базового теоретического анализа стохастического градиентного спуска. В частности, для постоянной скорости обучения $\alpha_i = h$ мы получим из формулы (3.79), что

$$\mathbb{E}\left[F(\hat{\boldsymbol{\theta}}_k) - F(\boldsymbol{\theta}_*)\right] \leq \frac{R^2}{2h(k+1)} + \frac{G^2 h}{2} \xrightarrow[k\to\infty]{} \frac{G^2 h}{2},$$

то есть невязка сходится не к нулю, а к конечной величине.

Это вполне естественно: если мы продолжаем делать стохастический градиентный спуск в окрестности глобального оптимума с той же скоростью, каждый мини-батч всё равно будет давать некоторое случайное направление градиента и процесс никогда не сойдётся. Даже если нам повезёт попасть в точности в глобальный оптимум, всё равно текущая стохастическая оценка градиента не будет равна нулю и мы этого попросту не заметим.

С другой стороны, из оценки (3.79) можно получить некоторые условия на то, чтобы стохастический градиентный спуск всё-таки сошёлся. Чтобы оценка сходилась к нулю при $k \to \infty$, очевидно, нужно, чтобы ряд из α_i в знаменателе расходился: $\sum_{i=0}^{k} \alpha_i = \infty$. А с другой стороны, нужно, чтобы ряд из α_i^2 в числителе не расходился ещё быстрее, то есть для простоты пусть $\sum_{i=0}^{k} \alpha_i^2 < \infty$.

Этим условиям удовлетворяет, например, линейное уменьшение скорости обучения: при $\alpha_i \propto \frac{1}{i}$ гармонический ряд в знаменателе будет расходиться, а ряд из обратных квадратов в числителе — нет.

Давайте кратко подытожим основные свойства стохастического градиентного спуска, которые следуют из нашего анализа:

- при постоянной длине шага алгоритм приходит в «зону неопределённости» (uncertainty region) радиусом $\frac{1}{2}G^2 h$; важно отметить, что этот радиус пропорционален длине шага;
- таким образом, чем быстрее мы движемся, тем быстрее придём в эту зону неопределённости, но тем больше будет сама зона; в целом, имеет смысл уменьшать длину шага в течение оптимизационного процесса;
- кроме того (этого мы не доказывали, но всё же отметим), стохастический градиентный спуск сходится достаточно медленно по сравнению с полным, использующим всю функцию целиком: известно, что полный градиентный спуск для выпуклых функций сходится с асимптотикой $O(1/k)$, а стохастический — лишь $O(1/\sqrt{k})$;

- с другой стороны, вдали от зоны неопределённости скорость сходимости для стохастического градиентного спуска тоже превращается в $O(1/k)$ (это как раз видно из оценки (3.79)) и сокращается, лишь когда мы в неё приходим;
- но всё вышеперечисленное на практике существенно зависит от G, а его мы никаким разумным образом обычно оценить не можем; это тоже важная проблема, которая часто осложняет дело в машинном обучении, особенно в высоких размерностях (то есть для моделей с большим числом весов, например современных глубоких нейронных сетей).

Что всё это значит на практике? Выходит, что обычный стохастический градиентный спуск не всегда лучший выбор для оптимизации, чёткого ответа на вопрос о том, как менять скорость спуска со временем, тоже нет, а в высоких размерностях вдобавок появляется проблема перемасштабирования градиентов, ведь разные веса часто имеют совсем разные масштабы и разные оптимальные скорости обучения.

Все эти вопросы остаются предметом исследований в машинном обучении, и уже было разработано множество улучшений и возможных подходов к решению этих проблем. Оставлю ссылки на ряд классических алгоритмов, в том числе вариантов метода моментов [81, 430, 442, 444], но, конечно, главные современные исследования здесь ведутся относительно глубоких нейронных сетей. Там активно развиваются методы *адаптивного* градиентного спуска, в котором скорости обучения разные для разных компонентов вектора $\boldsymbol{\theta}$, и они зависят от текущего ландшафта оптимизируемой функции, то есть от истории значений функции и её градиентов.

Углубляться в эти результаты мы сейчас не будем; оставлю несколько ссылок на классические источники [156, 306, 351, 658], а также, простите за самоцитирование, сошлюсь на разделы 4.4 и 4.5 книги «Глубокое обучение: погружение в мир нейронных сетей» [685], повествующие как раз о методах стохастического градиентного спуска в глубоком обучении. А нас уже ждёт следующая глава, в которой мы наконец-то поговорим о нашем первом настоящем, «серьёзном» методе машинного обучения: линейной регрессии.

Глава 4
Линейная регрессия

TL;DR

Раздел 4.1 начинает знакомство с линейной регрессией; здесь мы рассмотрим историю вопроса, метод наименьших квадратов, коэффициент детерминации как меру качества регрессии, а также разные виды функций признаков, которые делают линейную регрессию не такой уж безнадёжно линейной.

Раздел 4.2 посвящён одной из главных проблем машинного обучения: оверфиттингу. Вы увидите, как оверфиттинг может возникнуть даже в линейной регрессии; мы также рассмотрим основные виды регуляризаторов и их геометрический смысл.

Только в разделе 4.3 мы наконец-то начинаем рассматривать линейную регрессию с вероятностной точки зрения; здесь вы увидите, откуда берётся метод наименьших квадратов, какие вероятностные предположения делает линейная регрессия и как их можно было бы при необходимости изменить.

А уже в разделе 4.4 мы это рассмотрение и завершим, решив все задачи байесовского вывода для линейной регрессии; мы даже сможем аналитически построить предсказательное распределение. Здесь же приводится раздел «со звёздочкой» об оценке Джеймса — Штейна.

Раздел 4.5 — это один большой развёрнутый пример, в котором мы рассматриваем, как линейная регрессия может (или не может) помочь предсказать развитие болезни на примере ранних стадий развития пандемии COVID-19. Что можно сделать линейной регрессией и поможет ли это?

4.1. История вопроса и метод наименьших квадратов

I have no great faith in political arithmetic, and I mean not to warrant the exactness of either of these computations. (Не слишком я верю в политическую арифметику и за точность этих [вышеприведённых] вычислений ручаться не буду.)

Adam Smith. An Inquiry into the Nature and Causes of the Wealth of Nations

Ему пришёл на помощь брат:
«Не плачь, мой маленький Квадрат!
Ты улыбнись, братишка, шире.
У нас с тобой угла — четыре!
Мы все — отличная семья:
Четырёхугольники-друзья.
Вот ты, хотя и маловат,
Вполне квадратненький Квадрат.
Все стороны твои равны —
Что по бокам, что со спины».

Из стихотворения Л. М. Чурбановой, выдвинутого на конкурс «Детская литература — памяти Николая Носова»

4.1.1. История вопроса: почему «регрессия»?

Может показаться, что посвящать линейной регрессии целую главу — это чересчур. Что там обсуждать: линейная регрессия минимизирует сумму квадратов отклонений точек данных от линейной функции, из этого получаются наилучшие возможные веса, да и дело с концом. Однако, как вы увидите в этой главе, изучение даже такой не очень сложной модели, как линейная регрессия, таит в себе немало глубин и в ней можно увидеть практически все аспекты машинного обучения «в миниатюре». Этим мы и займёмся.

Начать, как водится, предлагаю с истории вопроса, которая здесь оказывается довольно занимательной [45, 535, 557]. Например, как вы думаете, почему линейная регрессия (а от неё и соответствующая задача машинного обучения в целом) называется регрессией? Что за регресс здесь имеется в виду, почему предсказание непрерывной переменной — это вдруг «регресс», а не «прогресс»? И откуда вообще такое странное название?

Термин «регрессия» появляется в статистике... ох, we need to go deeper. На этом месте придётся сделать ещё одно вложенное лирическое отступление: а от-

куда взялось собственно слово «статистика»? Ведь корень *stat* означает «государство» (state, etat, Staat) и имеет, казалось бы, мало общего с математикой? Как получилось так, что «статистика» — это раздел математики, а не политологии?

И действительно, названия эти не слишком логичные. Первой работой, посвящённой статистике в современном понимании этого слова, считается вышедшая ещё в 1661 году *Natural and Political Observations upon the Bills of Mortality* Джона Граунта[1]. Статистическими методами занимались ещё во второй половине XVIII века и Лаплас с Гауссом, которые создали многое из того, что мы обсуждаем в первых главах этой книги, и Уильям Плейфэр[2], который изобрёл многие современные виды визуализации статистических данных (рис. 4.1).

Однако само слово «статистика» появилось во второй половине XVIII — начале XIX века, и мотивация у него была другой. Это слово происходит от названия популярной в Германии дисциплины, которая занималась описанием государственных дел в цифрах: как ранее Макиавелли в Италии, в эпоху раздробленности Германии многие пытались обобщать принципы управления государствами, под которыми обычно понимались крохотные герцогства. Например, труд барона Карла-Августа Малхуса, министра финансов в королевстве Вестфальском, который вряд ли доказал в жизни хоть одну захудалую теорему, так и называется: *Statistik und Staatenkunde, ein Beitrag zur Staatenkunde von Europa*. Аналогичные попытки численного описания государственных дел в Великобритании в то время назывались «политической арифметикой» (political arithmetic), но этот термин в итоге проиграл.

---

[1] *Джон Граунт* (John Graunt, 1620–1674) — английский галантерейщик, по совместительству основоположник современной демографии и один из первых эпидемиологов. Граунт впервые в мире построил таблицу дожития; это было особенно любопытно тем, что основной его источник — бюллетени смертей и рождений в разных лондонских приходах под названием *Bills of Mortality* — не содержал возрастов умерших, так что здесь Граунту пришлось применить как раз статистические методы. А благодаря той самой книге *Natural and Political Observations*... Граунта приняли в Королевское общество по рекомендации собственно короля Карла II, который отметил профессию Граунта и с редкой для того времени демократичностью порекомендовал не стесняться принимать в научные круги «и других таких торговцев».

[2] *Уильям Плейфэр* (William Playfair, 1759–1823) — шотландский инженер, политэконом, монтажник, чертёжник, счетовод, изобретатель, серебряных дел мастер, торговец, инвестиционный брокер, статистик, памфлетист, переводчик, публицист, спекулянт земельными участками, банкир, журналист, редактор и в целом весьма колоритная фигура. В 18 лет стал персональным ассистентом Джеймса Уатта, в 1787 году переехал в Париж и принимал участие во взятии Бастилии, в 1793 году вернулся в Лондон и открыл банк. В истории остался своими блестящими образчиками инфографики, а также как человек, победивший Французскую революцию: в 1793 году Плэйфер предложил британскому правительству напечатать и распространить сто миллионов фальшивых ассигнатов (французские бумажные деньги времён революции). План был реализован самим Плейфэром и блестяще удался: к 1795 году ассигнаты полностью обесценились, и это во многом послужило причиной для краха революционного правительства. Хотя во Франции Уильям Плейфэр был секретным британским агентом, его не следует путать с Лионом Плейфэром, первым бароном Плейфэром (Lyon Playfair, 1st Baron Playfair, GCB PC FRS), в честь которого назван известный шифр Плейфэра, основанный на заменах биграмм.

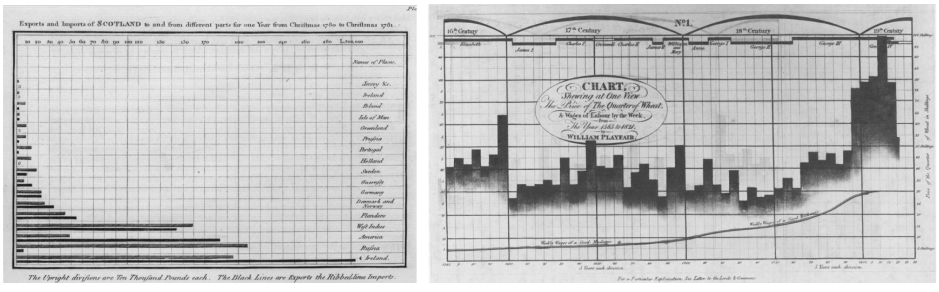


Рис. 4.1. Зарождение инфографики: схемы Уильяма Плейфэра. *Слева*: диаграмма экспорта и импорта Шотландии в 1780 году. Обратите внимание, что больше всего Шотландия импортировала из России; эх, были времена... *Справа*: сравнение цен на хлеб и заработной платы «хорошего механика» в течение 250 лет, иллюстрирующее несправедливость «грабительских наценок», начавшихся в XIX веке. Даже немного жаль, что максимум пришёлся на 1810–1815 годы, а не на 1805–1810-й — тогда обращённый к рабочему классу посыл стал бы ещё нагляднее

Благодаря трудам всех этих учёных, от галантерейщика до барона (даже Томас Байес до кардинала, увы, не дотянул), статистика в начале XIX века переживала расцвет. Волна хайпа привела к тому, что уже в 1834 году было основано Лондонское статистическое общество (Statistical Society of London), позже ставшее Королевским (Royal Statistical Society). Это было очень важной вехой для такой относительно узкой дисциплины: к тому времени, кроме основного Лондонского королевского общества (Royal Society of London), уже существовали Королевское географическое, медицинское и астрономическое общества, но, например, Лондонское математическое общество появилось лишь в 1865 году и Королевским так и не стало, Лондонское общество физиков — в 1874-м, Химическое общество (вошедшее в состав новообразованного Королевского химического общества в 1980 году) было образовано в 1841 году, а историческое — в 1868-м.

Но популярны тогда были не столько математические методы статистики как науки, сколько сам процесс сбора данных; в область того, как делать выводы из этих данных, хайп пришёл значительно позже[1]. А сначала все просто ужасно любили всевозможные цифры. Важной целью новообразованного общества было отгородиться от политической экономии, большой области, в которой цифры тогда использовались без особого порядка, а главную роль играли вполне гуманитарные идеи о том, как следует управлять государством; см., например, эпиграф к этому параграфу. Поэтому в первом постановочном «Проспекте» общества говорилось прямо: «Лондонское статистическое общество создано в целях сбора,

[1] Нет ничего нового под солнцем. В 2010-х годах мне доводилось слышать от вполне уважаемых потенциальных заказчиков из крупных корпораций вопросы вроде такого: «Не пора ли нам переходить с big data на machine learning?»...

упорядочения и публикации “Фактов, призванных иллюстрировать Состояние и Перспективы Общества” <...> общество считает первым и самым важным правилом своей деятельности полностью исключить Мнения из всех своих трудов и публикаций и строго ограничиться исключительно фактами, причём по мере возможности такими фактами, которые могут быть выражены численно и организованы в виде таблиц». На официальной печати общества был изображён сноп пшеницы, перевязанный лентой с девизом *Aliis Exterendum* — «Пусть молотят другие»; подразумевалось, что общество будет лишь собирать факты.

Такой подход, конечно, продержался недолго. Но не будем далее углубляться в историю статистики, а вернёмся к исходному вопросу: откуда есть пошла линейная регрессия? Линейные зависимости изучали давно, но считается, что первым статистиком, детально разработавшим идею линейной регрессии и назвавшим её этим именем, был Фрэнсис Гальтон[1].

Впрочем, нет, не совсем её. В 1886 году Гальтон выпустил работу *Regression Towards Mediocrity in Hereditary Stature* [205], в которой исследовал данные о наследственности в росте людей; это была, наверное, одна из самых политически корректных его работ о наследственности. Он использовал такой метод анализа данных: сначала привёл рост родителей к единой шкале, умножив рост матерей на коэффициент 1,08 (заметим, что уже здесь, по сути, кроется некоторая линейная регрессия), а затем разбил их на группы по среднему росту в паре родителей. На рис. 4.2 это показано прямой AB (mid-parents). Затем он изобразил на том же графике средний рост детей родителей из соответствующей группы; на рис. 4.2 это прямая CD (children). Получилось, что хотя знак отклонений от среднего оставался тем же, дети в среднем были ближе к средним значениям, чем родители. Гальтон назвал этот эффект «регрессией к среднему» (regression towards mediocrity; современный термин — regression to the mean), и здесь слово «регрессия» действительно вполне на месте: речь идёт о том, что экстремальные

[1] *Фрэнсис Гальтон* (Sir Francis Galton, FRS, 1822–1911) — английский статистик, социолог, психолог, антрополог, евгеник, исследователь тропиков, географ, изобретатель, метеоролог, психометрист, внук Эразма Дарвина (а значит, двоюродный брат Чарльза). В возрасте двух лет научился читать, в пять лет неплохо знал латынь, греческий и деление в столбик, а в восемьдесят семь был произведён в рыцари. Гальтон успел очень многое: сформулировал понятие регрессии и исследовал свойства двумерных гауссианов, разработал первую карту погоды и создал теорию антициклонов, основал психометрию и разработал лексическую гипотезу о том, что словарь и язык человека отражает его личность, изобрёл композитную фотографию (усреднение нескольких снимков) и организовал двухлетнюю экспедицию в юго-западную Африку (ныне Намибию). Однако, к сожалению, сейчас он больше известен с другой стороны. Гальтон принял близко к сердцу труды своего кузена и переложил идеи Дарвина о выведении новых пород животных на людей; кстати, сам Дарвин предусмотрительно ничего о людях не писал, по крайней мере напрямую. В результате Гальтон посвятил много времени всевозможным измерениям и классификациям людей, разработал расовую теорию, выпустил труд *Hereditary Genius* о том, что у великих людей и среди родственников много незаурядных личностей, создал и пропагандировал евгенику, написал статью в *The Times* под названием «Африка для китайцев» (Africa for the Chinese), в которой предлагал заместить темнокожих африканцев более высокоразвитой расой китайцев, а перед смертью работал над романом-утопией *Kantsaywhere* о чудесном обществе, в котором разводят умных и приспособленных для жизни людей (большую часть книги сожгла племянница Гальтона, возмущённая любовными сценами). Но любим мы его не за это.

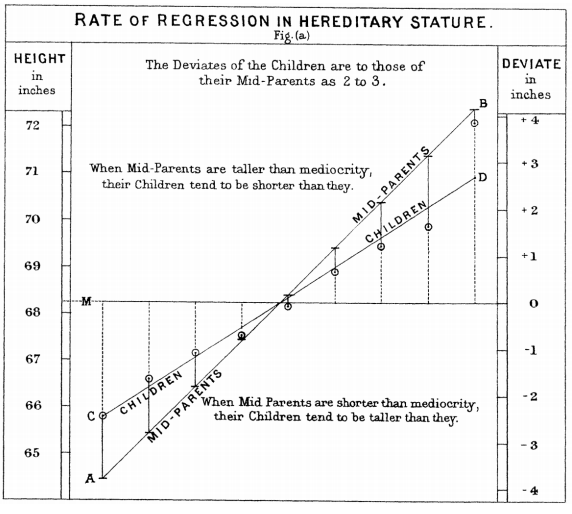


Рис. 4.2. Иллюстрация из работы Гальтона *Regression Towards Mediocrity in Hereditary Stature* [205]

значения имеют тенденцию «в следующем поколении» регрессировать к среднему, а наклон прямой, приближающей точки, регрессирует к нулю. Для роста детей Гальтон оценил коэффициент этой регрессии к среднему: наклон прямой, показывающей рост детей, составляет приблизительно $2/3$ от наклона прямой, показывающей рост родителей.

Эффект регрессии к среднему легко объяснить математически, и мы это сделаем чуть позже. Однако так получилось, что сам термин «регрессионный анализ», а затем и просто «регрессия», постепенно распространился на вообще всю науку о поиске линейных зависимостей между случайными переменными. Приоритет Гальтона здесь относится лишь к эффекту регрессии к среднему; саму идею метода наименьших квадратов для обучения параметров линейных зависимостей разработали ещё Гаусс и Лежандр[1] лет за 80 до работ Гальтона. Они прекрасно понимали и всю сопутствующую теорию, связанную с нормальными распределениями; не случайно они называются гауссовскими. К этой теории мы сейчас и перейдём.

[1] *Адриен Мари Лежандр* (Adrien-Marie Legendre, 1752–1833) — французский математик. Лежандр знаменит многими работами: в теории чисел он ввёл символы Лежандра и доказал теорему Ферма для $n = 5$, в анализе ввёл многочлены Лежандра и преобразование Лежандра, написал учебник *Éléments de géométrie* («Начала геометрии»), популярности которого не повредило даже то, что в разных изданиях Лежандр приводил аж три разных доказательства пятого постулата Евклида. Однако с научным приоритетом Лежандру не везло: методом наименьших квадратов он очень гордился, но оказалось, что его придумал Гаусс (и не опубликовал, как это за Карлом Фридрихом часто водилось), а работы Лежандра по эллиптическим функциям были поглощены более ранними трудами Абеля и Якоби, которые Лежандру известны не были.

4.1.2. Метод наименьших квадратов

Что ж, пора переходить к собственно математическому рассмотрению происходящего. Начнём с самой простой постановки вопроса: предположим, что перед нами одна-единственная численная переменная $y \in \mathbb{R}$, которую надо научиться предсказывать по нескольким входным численным переменным $\boldsymbol{x}^\top = (x_1, \ldots, x_d) \in \mathbb{R}^d$. Задача линейной регрессии — построить линейную функцию

$$y(\boldsymbol{x}, \boldsymbol{w}) = w_0 + w_1 x_1 + \ldots + w_d x_d,$$

которая бы наилучшим образом описывала данные. Сразу начнём с нехитрого трюка, упрощающего запись: давайте переопределим вектор $\boldsymbol{x}$, добавив в него фиктивную переменную, всегда равную единице: $\boldsymbol{x}^\top = (1, x_1, \ldots, x_d)$. Тогда нам не придётся всё время выписывать w_0, и мы сможем записать модель как

$$y(\boldsymbol{x}, \boldsymbol{w}) = \sum\nolimits_{j=0}^{d} w_j x_j = \boldsymbol{w}^\top \boldsymbol{x}.$$

Этот приём мы будем применять в книге постоянно.

Параметры модели — это, очевидно, веса линейной комбинации $\boldsymbol{w}$. Когда мы обучим параметры $\hat{\boldsymbol{w}}$ из данных, мы сможем по вектору входов $\boldsymbol{x}^\top = (x_0, \ldots, x_p)$ предсказывать выход y как

$$\hat{y}(\boldsymbol{x}) = \sum\nolimits_{j=0}^{d} \hat{w}_j x_j = \hat{\boldsymbol{w}}^\top \boldsymbol{x}.$$

От того, чтобы начать применять эту модель на практике, нас отделяет только один вопрос: как найти оптимальные параметры $\boldsymbol{w}^*$ по тренировочным данным вида $(\boldsymbol{x}_i, y_i)_{i=1}^N$? Очевидно, мы бы хотели, чтобы ошибка модели на тренировочном наборе была как можно меньше. Но какую именно функцию ошибки лучше оптимизировать?

Думаю, большинство моих читателей уже знают ответ: метод наименьших квадратов потому так и называется, что в нём минимизируется сумма квадратов отклонений предсказанных значений $\hat{y} = \boldsymbol{w}^\top \boldsymbol{x}$ от истинных значений переменной y, записанных в датасете:

$$\text{RSS}(\boldsymbol{w}) = \sum\nolimits_{n=1}^{N} (y_n - \boldsymbol{w}^\top \boldsymbol{x}_n)^2.$$

Как минимизировать такую функцию? На самом деле линейная регрессия — это настолько простая модель, что многое здесь можно подсчитать точно, не прибегая к задачам оптимизации или итеративным процессам. В методе наименьших квадратов функция ошибки представляет собой поверхность второго порядка, и её минимум легко найти аналитически. Чтобы это сделать, нужно продифференцировать сумму квадратов отклонений $\text{RSS}(\boldsymbol{w})$ по $\boldsymbol{w}$ и приравнять полученный градиент к нулю.

Для этого давайте сначала перепишем всё в матричном виде:

$$\text{RSS}(\boldsymbol{w}) = \sum_{n=1}^{N}(y_n - \boldsymbol{w}^\top \boldsymbol{x}_n)^2 = (\boldsymbol{y} - \boldsymbol{X}\boldsymbol{w})^\top(\boldsymbol{y} - \boldsymbol{X}\boldsymbol{w}), \tag{4.1}$$

где $\boldsymbol{X}$ — это матрица размера $N \times d$, в строках которой записаны d-мерные векторы $\boldsymbol{x}_n$, а $\boldsymbol{y}$ — вектор правильных ответов:

$$\boldsymbol{X} = \begin{pmatrix} \boldsymbol{x}_1 \\ \vdots \\ \boldsymbol{x}_N \end{pmatrix} = \begin{pmatrix} x_{11} & x_{12} & \ldots & x_{1d} \\ \vdots & \vdots & & \vdots \\ x_{N1} & x_{N2} & \ldots & x_{Nd} \end{pmatrix}, \qquad \boldsymbol{y} = \begin{pmatrix} y_1 \\ \vdots \\ y_N \end{pmatrix}.$$

Теперь можно считать градиент; чтобы дифференцировать было удобно, давайте выведем сначала несколько стандартных формул в векторной форме. Во-первых, рассмотрим скалярное произведение на константу $\boldsymbol{a}^\top\boldsymbol{x} = a_1x_1 + \ldots + a_dx_d$; ясно, что частная его производная по x_i равна a_i, а значит,

$$\nabla_{\boldsymbol{a}^\top\boldsymbol{x}}\boldsymbol{x} = \boldsymbol{a}.$$

Во-вторых, для квадрата нормы вектора $\boldsymbol{x}^\top\boldsymbol{x} = x_1^2 + \ldots + x_d^2$ ответ тоже очевиден по аналогичным соображениям:

$$\nabla_{\boldsymbol{x}^\top\boldsymbol{x}}\boldsymbol{x} = 2\boldsymbol{x}.$$

Кроме того, нам ещё понадобится градиент квадратичной формы $\boldsymbol{x}^\top A\boldsymbol{x}$ с матрицей $A = (a_{ij})_{i,j=1}^d$. Для того, чтобы его найти, проще всего записать требующиеся формулы в координатах и выделить в сумме $\sum_{i=1}^d\sum_{j=1}^d a_{ij}x_ix_j$ зависящие от x_k члены:

$$\frac{\partial \boldsymbol{x}^\top A\boldsymbol{x}}{\partial x_k} = \frac{\partial \sum_{i=1}^d\sum_{j=1}^d a_{ij}x_ix_j}{\partial x_k} =$$

$$= \frac{\partial \sum_{i\neq k} a_{ik}x_ix_k + \sum_{j\neq k} a_{kj}x_kx_j + x_k^2}{\partial x_k} =$$

$$= \sum_{i\neq k} a_{ik}x_i + \sum_{j\neq k} a_{kj}x_j + 2x_k = \sum_{i=1}^d a_{ik}x_i + \sum_{j=1}^d a_{kj}x_j,$$

то есть, возвращаясь в векторную форму,

$$\frac{\partial \boldsymbol{x}^\top A\boldsymbol{x}}{\partial x_k} = A\boldsymbol{x} + A^\top\boldsymbol{x}.$$

Ну а если матрица A симметрична, как всегда и будет в нашей практике, то получится естественная формула

$$\frac{\partial \boldsymbol{x}^\top A\boldsymbol{x}}{\partial x_k} = 2A\boldsymbol{x}.$$

Раскрыв скобки в выражении (4.1), мы сразу сможем применить эти стандартные формулы:

$$\nabla_{\boldsymbol{w}}\text{RSS}(\boldsymbol{w}) = \nabla_{\boldsymbol{w}}\left(\boldsymbol{y}^\top\boldsymbol{y} - 2\boldsymbol{w}^\top\boldsymbol{X}^\top\boldsymbol{y} + \boldsymbol{w}^\top\boldsymbol{X}^\top\boldsymbol{X}\boldsymbol{w}\right) = -2\boldsymbol{X}^\top\boldsymbol{y} + 2\boldsymbol{X}^\top\boldsymbol{X}\boldsymbol{w}.$$

Как и ожидалось, мы получили линейную функцию от $\boldsymbol{w}$, и теперь осталось только приравнять градиент к нулю и решить систему линейных уравнений:

$$\boldsymbol{X}^\top\boldsymbol{X}\boldsymbol{w}^* = \boldsymbol{X}^\top\boldsymbol{y},$$

и

$$\boldsymbol{w}^* = \left(\boldsymbol{X}^\top\boldsymbol{X}\right)^{-1}\boldsymbol{X}^\top\boldsymbol{y}, \tag{4.2}$$

если матрица $\boldsymbol{X}^\top\boldsymbol{X}$ невырожденная, то есть если столбцы матрицы $\boldsymbol{X}$ линейно независимы.

Матрица $\left(\boldsymbol{X}^\top\boldsymbol{X}\right)^{-1}\boldsymbol{X}^\top$, которая у нас получилась, называется *псевдообратной матрицей Мура — Пенроуза* (Moore — Penrose pseudo-inverse) для матрицы $\boldsymbol{X}$; это «правильный» способ обобщить понятие обратной матрицы на прямоугольные матрицы. Разумеется, на практике при обучении линейной регрессии никто обратных матриц не считает; во-первых, решить систему линейных уравнений даже методом Гаусса будет гораздо быстрее, чем искать обратную матрицу, а во-вторых, в линейной регрессии нас интересует численное, приближённое решение, а для этого есть масса ещё более эффективных методов, например через разложение Холецкого[1] [58], метод сопряжённых градиентов [257] и другие итеративные методы [228].

Давайте рассмотрим конкретный пример. Пример будет учебным и иллюстративным, поэтому первый вопрос звучит так: откуда взять для него «идеальные» данные? Для этого давайте загадаем правильный ответ, то есть линейную функцию от x, например $y = \frac{1}{2}x - \frac{1}{2}$ (пример будет с одномерным x, чтобы результаты можно было легко изобразить), и выберем абсциссы точек данных:

```
orig = lambda x : 0.5*x - 0.5
```

[1] *Андре-Луи Холецкий* (André-Louis Cholesky, 1875–1918) — французский военный офицер и математик. Он окончил École Polytechnique и работал геодезистом, в частности составлял карты Крита и Северной Африки перед Первой мировой. Там он и разработал свой метод матричного разложения, который использовал для триангуляции. Холецкий погиб на самом излёте войны, во время прорыва линии Гинденбурга, а разложение было опубликовано в *Bulletin Géodésique* уже после его смерти.

```
xd = np.array([-3, -2, -1, -0.5, 0, 0.5, 1, 1.5, 2.5, 3, 4]) / 2
num_points = len(xd)
```

Если теперь просто применить функцию `orig` к точкам `xd`, результаты выстроятся в точности на прямой без всяких ошибок и от задачи линейной регрессии ничего не останется, можно будет просто проводить прямую через точки. Поэтому давайте добавим шум, прибавив к результату нормально распределённую величину $\epsilon \sim \mathcal{N}\left(\epsilon|0, \frac{1}{4}\right)$:

```
data = orig(xd) + np.random.normal(0, .25, num_points)
```

В результате получится примерно такой набор данных (я немного округлил результаты для удобства):

$$\text{data} = (\,-1{,}21\ -0{,}79\ -1{,}18\ -0{,}75\ -0{,}72\ -0{,}53\ -0{,}45\ -0{,}10\ -0{,}19\ 0{,}27\ 0{,}47\,).$$

Теперь можно строить линейную регрессию; воспользуемся для этого сначала стандартным классом из `scikit-learn`:

```
model = linear_model.LinearRegression(fit_intercept=True).fit(\
    xd.reshape(-1, 1), data )
```

Здесь `fit_intercept=True` означает, что модель будет выглядеть не как $y = w_1 x$, а как $y = w_0 + w_1 x$. Свободный член у линейной функции или многочлена по-английски называется *intercept*, потому что это та точка, где ось Y пересекается с кривой, «перехватывает» её; если быть совсем точным, свободный член называется Y-intercept, потому что есть ещё, с другой стороны, и X-intercept. А функцию `reshape(-1, 1)` приходится написать, чтобы сделать из переменной `xd` вектор-столбец, и библиотека `numpy` смогла понять, что это регрессия с одним признаком и 11 точками, а не, наоборот, регрессия с 11 признаками и одной точкой данных[1].

В результате получится что-то вроде того, что изображено на рис. 4.3, *а*. Можно проверить, насколько полученные коэффициенты похожи на загаданные — изображённая на рис. 4.3, *а* модель задаётся уравнением

$$y = 0{,}479x - 0{,}601,$$

так что совпадение в целом достаточно точное.

А можно попробовать понабрасывать ещё датасетов, то есть добавлять разные реализации случайной величины ϵ и смотреть, что будет получаться. Пример результата изображён на рис. 4.3, *б*, где проведены 20 разных прямых для разных значений шума ϵ, взятых из одного и того же нормального распределения. К тому, насколько разные получаются прямые и как их оценить, мы ещё вернёмся позже.

Метод наименьших квадратов можно объяснить и геометрически. Для этого давайте вернёмся к исходной задаче: приблизить некоторый вектор целевых

[1] Лучше всего, конечно, одиннадцать пирогов и одна свечка.

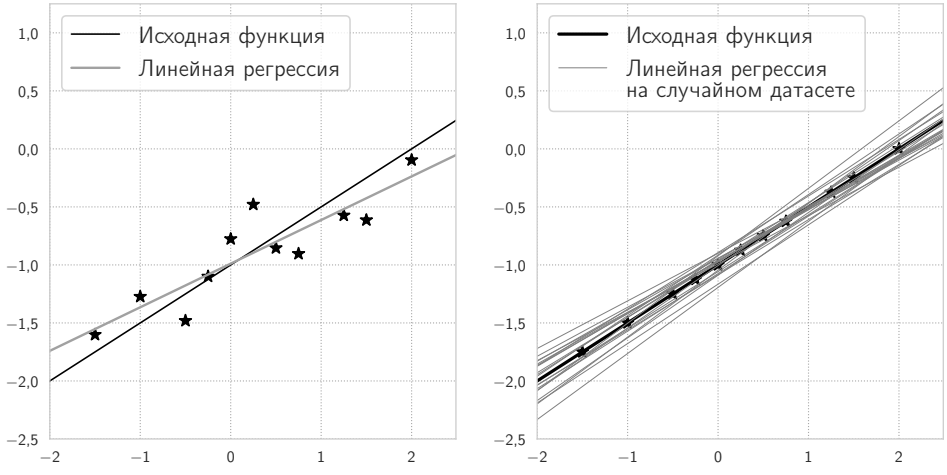


Рис. 4.3. Два примера линейной регрессии для исходной функции $y = \frac{1}{2}x - \frac{1}{2}$: (а) результат линейной регрессии; (б) 20 результатов на случайных датасетах

переменных $\boldsymbol{y}$ вектором $\boldsymbol{X}\boldsymbol{w}$, в котором матрица $\boldsymbol{X}$ нам дана, а вектор $\boldsymbol{w}$ нужно подобрать. Результат вычисления $\boldsymbol{X}\boldsymbol{w}$ — это некоторая линейная комбинация столбцов матрицы $\boldsymbol{X}$:

$$\boldsymbol{X}\boldsymbol{w} = \begin{pmatrix} x_{11} & x_{12} & \dots & x_{1d} \\ \vdots & \vdots & & \vdots \\ x_{N1} & x_{N2} & \dots & x_{Nd} \end{pmatrix} \begin{pmatrix} w_1 \\ \vdots \\ w_d \end{pmatrix} = \begin{pmatrix} x_{11} \\ \vdots \\ x_{N1} \end{pmatrix} w_1 + \ldots + \begin{pmatrix} x_{d1} \\ \vdots \\ x_{dN} \end{pmatrix} w_d.$$

Иначе говоря, выбором вектора $\boldsymbol{w}$ мы меняем только коэффициенты линейной комбинации, но при этом всё равно остаёмся в линейном пространстве, «натянутом» на векторы $\boldsymbol{a}_i = \begin{pmatrix} x_{1i} & \dots & x_{iN} \end{pmatrix}^\top$.

Это проиллюстрировано на рис. 4.4, а: вектор $\boldsymbol{y}$ проецируется на подпространство, порождённое векторами $\boldsymbol{a}_1, \dots, \boldsymbol{a}_d$. И тогда становится геометрически очевидным, какая аппроксимация будет наилучшей с точки зрения квадрата расстояния: это будет проекция вектора $\boldsymbol{y}$ на подпространство, порождённое $\boldsymbol{a}_1, \dots, \boldsymbol{a}_d$! На рис. 4.4, *а* эта проекция обозначена через $\hat{\boldsymbol{y}}$, а вектор $\mathbf{r} = \hat{\boldsymbol{y}} - \boldsymbol{y}$ — это та самая невязка (residue), длину которой мы хотим минимизировать.

Геометрический смысл линейной регрессии позволит нам вывести результат (4.2) и без всякого дифференцирования. Действительно, мы хотели бы найти проекцию $\hat{\boldsymbol{y}} = \boldsymbol{X}\boldsymbol{w}^*$, то есть добиться того, чтобы вектор $\mathbf{r}$ был ортогонален подпространству, порождённому $\boldsymbol{a}_1, \dots, \boldsymbol{a}_d$ (на рис. 4.4, *а* это плоскость, порождённая $\boldsymbol{a}_1$ и $\boldsymbol{a}_2$).

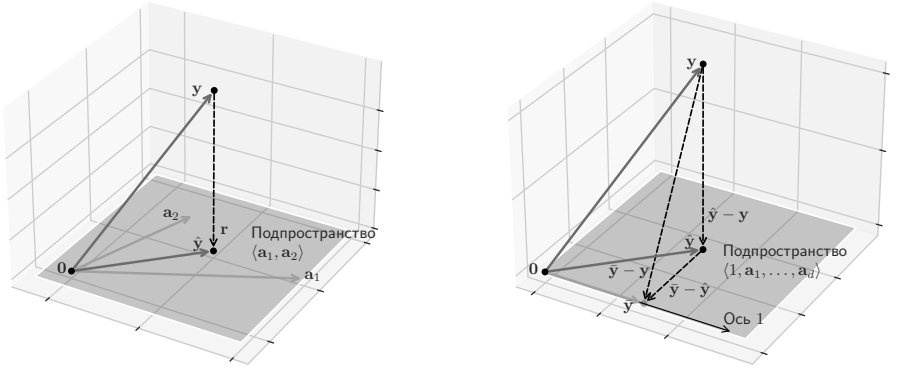


Рис. 4.4. Геометрический смысл линейной регрессии: (а) проекция на подпространство, порождённое векторами $\boldsymbol{a}_i$; (б) теорема Пифагора для коэффициента детерминации

Это значит, что невязка $\mathbf{r}$ должна быть ортогональна любой линейной комбинации векторов $\boldsymbol{a}_1, \ldots, \boldsymbol{a}_d$. Но мы уже обсуждали выше, что такие линейные комбинации — это векторы вида $\boldsymbol{X}$ для некоторого вектора коэффициентов $\boldsymbol{u}$. Получается, что для любого вектора $\boldsymbol{u}$ должно быть верно, что

$$(\boldsymbol{X}\boldsymbol{u})^\top = (\boldsymbol{X}\boldsymbol{u})^\top (\hat{\boldsymbol{y}} - \boldsymbol{y}) = (\boldsymbol{X}\boldsymbol{u})^\top (\boldsymbol{X}\boldsymbol{w}^* - \boldsymbol{y}) = 0.$$

Раскроем в последнем выражении транспонирование:

$$\boldsymbol{u}^\top \boldsymbol{X}^\top (\boldsymbol{X}\boldsymbol{w}^* - \boldsymbol{y}) = \boldsymbol{u}^\top \left(\boldsymbol{X}^\top \boldsymbol{X}\boldsymbol{w}^* - \boldsymbol{X}^\top \boldsymbol{y}\right) = 0.$$

Получилось, что скалярное произведение $\boldsymbol{u}$ на какой-то другой вектор должно быть равно нулю для любого вектора $\boldsymbol{u}$, что возможно, только если этот самый другой вектор сам тождественно равен нулю.

Вот мы и пришли к нужному результату:

$$\boldsymbol{X}^\top \boldsymbol{X}\boldsymbol{w}^* - \boldsymbol{X}^\top \boldsymbol{y} = 0, \quad \text{то есть} \quad \boldsymbol{w}^* = \left(\boldsymbol{X}^\top \boldsymbol{X}\right)^{-1} \boldsymbol{X}^\top \boldsymbol{y}.$$

4.1.3. Коэффициент детерминации

И ещё одно замечание. Теперь, когда мы научились рисовать прямые, можно подумать о том, как оценить, насколько хорошо наш метод наименьших квадратов сработал. Для этого принято использовать так называемый *коэффициент*

детерминации R^2 (coefficient of determination): ту часть дисперсии зависимой переменной y, которую сумела объяснить наша регрессия. Формально, обозначая среднее значение y_n через $\bar{y} = \frac{1}{N}\sum_{n=1}^{N} y_n$, мы должны подсчитать

$$R^2 = \frac{\sum_{n=1}^{N}(\hat{y}_n - \bar{y})^2}{\sum_{n=1}^{N}(y_n - \bar{y})^2}. \tag{4.3}$$

Эту величину можно переписать, используя специальную матрицу $A = \mathbf{I}_N - \frac{1}{N}\mathbf{1}\mathbf{1}^\top$, где $\mathbf{I}_N$ — единичная матрица размера $N \times N$, а $\mathbf{1}$ — вектор, состоящий из единиц. Матрица A позволяет кратко записать знаменатель дроби:

$$\sum\nolimits_n (y_n - \bar{y})^2 = (\boldsymbol{y} - \frac{1}{N}\mathbf{1}\mathbf{1}^\top \boldsymbol{y})^\top (\boldsymbol{y} - \frac{1}{N}\mathbf{1}\mathbf{1}^\top \boldsymbol{y}) = \boldsymbol{y}^\top A \boldsymbol{y}. \tag{4.4}$$

А чтобы числитель тоже переписать через A, давайте вспомним, что такое $\hat{\boldsymbol{y}}$, и введём матрицу $\mathbf{P}$ (её логично обозначить именно так, потому что это матрица оператора проектирования на линейное подпространство, порождённое столбцами матрицы $\boldsymbol{X}$):

$$\hat{\boldsymbol{y}} = \boldsymbol{X}\boldsymbol{w}^* = \boldsymbol{X}\left(\boldsymbol{X}^\top \boldsymbol{X}\right)^{-1}\boldsymbol{X}^\top \boldsymbol{y} = \mathbf{P}\boldsymbol{y}. \tag{4.5}$$

Теперь объединим формулы (4.4) и (4.5), вспомним, что $\bar{y} = \bar{\hat{y}}$, и подставим всё это в выражение (4.3):

$$R^2 = \frac{\left(\mathbf{P}\boldsymbol{y} - \frac{1}{N}\mathbf{1}\mathbf{1}^\top \mathbf{P}\boldsymbol{y}\right)^\top \left(\mathbf{P}\boldsymbol{y} - \frac{1}{N}\mathbf{1}\mathbf{1}^\top \mathbf{P}\boldsymbol{y}\right)}{\boldsymbol{y}^\top A \boldsymbol{y}} = \frac{\boldsymbol{y}^\top \mathbf{P}^\top A \mathbf{P}\boldsymbol{y}}{\boldsymbol{y}^\top A \boldsymbol{y}} =$$
$$= 1 - \frac{\boldsymbol{y}^\top \mathbf{M}^\top A \mathbf{M}\boldsymbol{y}}{\boldsymbol{y}^\top A \boldsymbol{y}} = 1 - \frac{\mathrm{RSS}(\boldsymbol{w}^*)}{\mathrm{TSS}} = 1 - \frac{\mathrm{Var}\,[\boldsymbol{y} \mid \boldsymbol{X}]}{\mathrm{Var}\,[\boldsymbol{y}]}, \tag{4.6}$$

где $\mathbf{M} = \mathbf{I}_N - \mathbf{P}$ — это матрица, которая превращает шум в исходных данных в ошибки регрессии, которые мы обозначим через

$$\hat{\epsilon} = y - \hat{y} = \mathbf{M}\boldsymbol{y} = \mathbf{M}(\boldsymbol{y} - \boldsymbol{X}\boldsymbol{w}^*),$$

а через TSS (total sum of squares) мы обозначили общую сумму квадратов отклонений в данных $\boldsymbol{y}$ от их собственного среднего.

Иначе говоря, коэффициент детерминации R^2 показывает, какую долю от общей дисперсии $\boldsymbol{y}$ мы смогли убрать, приблизив данные линейной регрессией. Он называется R^2, потому что для линейной регрессии он является квадратом множественного коэффициента корреляции между y и $\boldsymbol{x}$; не будем сейчас в это углубляться, но в качестве упражнения проверьте, что действительно получается корреляция, хотя бы для одномерного случая $d = 1$.

У вычислений, связанных с коэффициентом детерминации, тоже есть понятный геометрический смысл. Он проиллюстрирован на рис. 4.4, *б*; обратите внимание, что этот рисунок иллюстрирует заодно и наш стандартный трюк с переходом в линейное пространство на единицу большей размерности: там выделена отдельная ось «единиц», на которой отложено число $\bar{y}$ в виде вектора, а горизонтальная плоскость теперь иллюстрирует подпространство, порождённое векторами $\boldsymbol{a}_i$ и единичным вектором чисел.

При таком изображении хорошо видно, как здесь работает теорема Пифагора. Применим её к изображённым пунктиром векторам:

$$\|\hat{\boldsymbol{y}} - \bar{\boldsymbol{y}}\|^2 + \|\hat{\boldsymbol{y}} - \boldsymbol{y}\|^2 = \|\boldsymbol{y} - \bar{\boldsymbol{y}}\|^2 .$$

Это значит, что общая сумма квадратов отклонений от среднего TSS

$$\text{TSS} = \|\boldsymbol{y} - \bar{\boldsymbol{y}}\|^2 = \sum\nolimits_{n=1}^{N} (y_n - \bar{y})^2$$

складывается из суммы квадратов отклонений истинных значений от обученной регрессии $\hat{\boldsymbol{y}}$, которую здесь называют RSS (residual sum of squares) и которую мы минимизировали при обучении линейной регрессии,

$$\text{RSS}(\boldsymbol{w}^*) = \|\boldsymbol{y} - \hat{\boldsymbol{y}}\|^2 = \sum\nolimits_{n=1}^{N} \left(y_n - \boldsymbol{x}_n^\top \boldsymbol{w}^*\right)^2 ,$$

и «объяснённой» с помощью регрессии суммы квадратов отклонений проекции от среднего ESS (explained sum of squares)

$$\text{ESS}(\boldsymbol{w}^*) = \|\bar{\boldsymbol{y}} - \hat{\boldsymbol{y}}\|^2 = \sum\nolimits_{n=1}^{N} \left(\boldsymbol{x}_n^\top \boldsymbol{w}^* - \bar{y}\right)^2 .$$

Выражение TSS = ESS + RSS часто можно встретить в лекциях по эконометрике и других аналогичных источниках, где линейную регрессию объясняют со статистической точки зрения. А коэффициент детерминации в этих обозначениях представляет собой как раз долю объяснённой суммы квадратов отклонений:

$$R^2 = 1 - \frac{\text{RSS}(\boldsymbol{w}^*)}{\text{TSS}} = \frac{\text{ESS}(\boldsymbol{w}^*)}{\text{TSS}}.$$

Итак, мы разработали метод, позволяющий обучать параметры линейной функции (гиперплоскости), которая неплохо описывает имеющиеся данные. Но пока всё, что мы сделали, как будто построено на песке: мы совершенно не понимаем, откуда что берётся.

Ясно, что ошибка должна быть тем меньше, чем ближе предсказанное значение $\hat{y}$ к правильному ответу y. Но почему именно квадрат разности? На свете есть много других замечательных монотонных функций от модуля разности: сам модуль разности $|y - \hat{y}|$, любая его степень, например $(y - \hat{y})^4$, экспонента $e^{y-\hat{y}}$,

логарифм $\ln|y-\hat{y}|$... откуда берётся квадрат? Об этом — в разделе 4.3, но сначала давайте обсудим, каким может быть в линейной регрессии вектор признаков $\boldsymbol{x}$.

4.1.4. Функции признаков в линейной регрессии

В этой главе мы всё время говорим о *линейной* регрессии: предполагаем, что целевая переменная зависит от поступающих на вход признаков линейно: $y = \boldsymbol{w}^\top \boldsymbol{x} + \epsilon$. Вся наша теория была построена на этом предположении и на предположении о том, что шум распределён нормально. О нормальном распределении шума мы поговорим позже, а пока обсудим первое предположение: значит ли это, что линейная регрессия безнадёжно ограничена только теми случаями, когда y зависит от $\boldsymbol{x}$ именно линейно?

Давайте попробуем сделать шаг в сторону. Что, если мы хотим предположить, что переменная y зависит от входа x (пусть он для простоты пока будет одномерным) не линейно, а, скажем, квадратично? Формально говоря, мы должны от модели вида

$$y = w_0 + w_1 x + \epsilon \tag{4.7}$$

перейти к модели вида

$$y = w_0 + w_1 x + w_2 x^2 + \epsilon. \tag{4.8}$$

Посмотрите на эти записанные друг под другом формулы (4.7) и (4.8): они не так уж и сильно различаются! Выше мы уже применяли трюк, при котором добавляли к переменной x ещё одну размерность, всегда равную единице, преобразуя $x \mapsto \binom{1}{x}$, чтобы не таскать за собой всё время отдельную переменную w_0, а спрятать её в скалярное произведение. Давайте теперь сделаем примерно то же самое, но добавим не только единицу, а ещё и x^2. Перейдём от вектора признаков $(1\ x)^\top$ к вектору признаков $(1\ x\ x^2)^\top$, а затем применим к нему линейную регрессию:

$$y = \begin{pmatrix} w_0 \\ w_1 \\ w_2 \end{pmatrix}^\top \begin{pmatrix} 1 \\ x \\ x^2 \end{pmatrix} + \epsilon = w_0 + w_1 x + w_2 x^2 + \epsilon.$$

У нас получилась ровно та самая квадратичная модель, которую мы изначально хотели обучить! И обучать её теперь можно как обычную линейную регрессию, методом наименьших квадратов. Мы абсолютно ничего не изменили и ничего не потеряли: ошибка по-прежнему представляет собой среднеквадратичное отклонение от предсказанных значений, изменился только способ, которым мы эти предсказания получили. Естественно, эта идея не ограничивается квадратичными признаками; как минимум уж точно можно добавить любую степень x в признаки, получив так называемую *полиномиальную регрессию*:

$$y = \begin{pmatrix} w_0 \\ w_1 \\ \dots \\ w_d \end{pmatrix}^\top \begin{pmatrix} 1 \\ x \\ \dots \\ x^d \end{pmatrix} + \epsilon = w_0 + w_1 x + w_2 x^2 + \ldots + w_d x^d + \epsilon.$$

Давайте посмотрим, как это может выглядеть на практике. Вернёмся к примеру одномерной линейной регрессии, который мы рассматривали выше, добавим

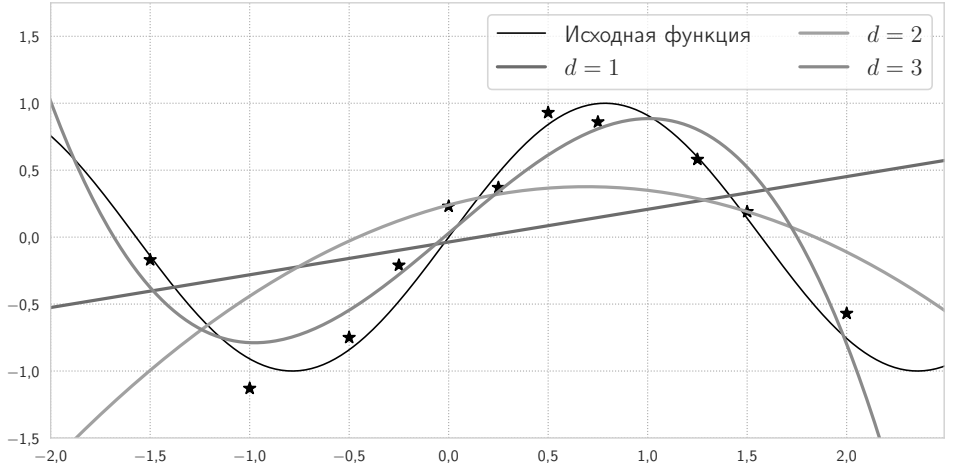


Рис. 4.5. Полиномиальная регрессия степени 1, 2 и 3

признаки в виде многочленов до определённой степени, а затем обучим линейную регрессию на новых признаках:

```
xs_d = np.vstack([xs ** i for i in range(1, d+1)]).transpose()
xd_d = np.vstack([xd ** i for i in range(1, d+1)]).transpose()

model = linear_model.LinearRegression(fit_intercept=True).fit( xd_d, data )
```

На рис. 4.5 показаны три кривые такой полиномиальной регрессии: для степеней $d = 1$ (обычная линейная регрессия), $d = 2$ (квадратичная) и $d = 3$ (кубическая). Они выглядят примерно так, как мы и ожидали: полиномиальная регрессия успешно подобрала параболу и кубический многочлен, которые наилучшим образом описывают данные. Заметьте, что, поскольку исходные данные получались из одного периода синусоиды, кубическая кривая уже очень хорошо их описывает: выразительности многочлена третьей степени как раз хватает для того, чтобы описать один период синусоиды. С другой стороны, обратите внимание, что *экстра*поляция по-прежнему совершенно не получается, причём имеет даже меньше смысла, чем в прошлый раз: многочлены более высокой степени ещё быстрее убегают на бесконечность, чем прямая линия.

Конечно, для того, чтобы полиномиальная регрессия имела смысл, не обязательно что-то знать наверняка о форме зависимости. Достаточно просто считать, что функция, которую мы хотим описать, на интересующем нас интервале «достаточно хорошая» для того, чтобы хорошо описываться первыми несколькими слагаемыми своего разложения в ряд Тейлора. Но опять же обратите внимание, что ряд Тейлора — это именно *локальная* аппроксимация, которая по определению работает лишь в окрестности некоторой точки и при отдалении от неё будет

неизбежно расходиться с приближаемой функцией (если, конечно, она сама не является многочленом).

С другой стороны, не стоит забывать о том, что можно применять преобразования не только к признакам $\boldsymbol{x}$, но и к целевой переменной y. Например, если у нас есть основания полагать, что целевая переменная зависит от признаков экспоненциально,

$$y \approx e^{\boldsymbol{w}^\top \boldsymbol{x}}, \tag{4.9}$$

то для того, чтобы найти $\boldsymbol{w}$, достаточно просто прологарифмировать целевую переменную и обучать линейную регрессию вида

$$\log y \approx \boldsymbol{w}^\top \boldsymbol{x}. \tag{4.10}$$

Здесь есть, правда, одна маленькая тонкость: преобразования целевой переменной меняют и распределение шума. Так, если мы обучим линейную регрессию из формулы (4.10), это будет значить, что мы предположили *аддитивный* шум в данных, то есть случайную ошибку ϵ, которая прибавляется к данным как

$$\log y = \boldsymbol{w}^\top \boldsymbol{x} + \epsilon.$$

И если теперь перейти обратно от $\log y$ к изначальной экспоненциальной зависимости (4.9), шум внезапно превратится в *мультипликативный*:

$$y = e^{\boldsymbol{w}^\top \boldsymbol{x} + \epsilon} = e^{\boldsymbol{w}^\top \boldsymbol{x}} \cdot e^{\epsilon}.$$

Если вы обучаете экспоненциальную зависимость, мультипликативный шум, скорее всего, подходит лучше. В разделе 4.3 мы поговорим о конкретных распределениях шума в линейной регрессии, а в разделе 4.5 подробно рассмотрим как раз пример с логарифмированием целевой переменной.

С другой стороны, выбором признаков можно добавить в линейную регрессию и входы, которые на первый взгляд для неё нехарактерны. Например, представим, что у нас есть некоторая линейная зависимость, которая при этом может меняться в зависимости от *бинарного* признака (binary feature), принимающего только два возможных значения: «пользователь получил промокод или нет», «клиент уже имеет продукт в нашем банке или нет», «получил студент зачёт или нет» и так далее. Тогда можно добавить в линейную регрессию ещё один признак, который будет принимать только значения 0 и 1 в зависимости от значения этого признака.

Так же можно сделать и если у признака несколько значений, то есть он не бинарный, а *категориальный* (categorical feature): «группа крови у пациента», «тип кузова машины», «оценка студента на экзамене» и так далее. В таком случае пригодится уже не раз встречавшееся в этой книге *прямое кодирование* (one-hot

encoding): добавим столько новых размерностей, сколько существует возможных вариантов, и будем кодировать каждое значение вектором, в котором есть ровно одна единица и несколько нулей.

Например, оценки студентов на экзамене можно закодировать так:

$$\begin{aligned} \text{«Отлично»}: &\quad [1,0,0,0],\\ \text{«Хорошо»}: &\quad [0,1,0,0],\\ \text{«Удовлетворительно»}: &\quad [0,0,1,0],\\ \text{«Неудовлетворительно»}: &\quad [0,0,0,1]. \end{aligned}$$

Такой подход практически всегда будет лучше, чем пытаться найти численные значения, кодирующие разные значения категориальной переменной. Прямое кодирование очень полезно, и мы к нему ещё не раз вернёмся в этой книге. Например, в главе 5, посвящённой задачам классификации, уже целевая переменная станет категориальной, и там в случае нескольких классов мы будем кодировать целевую переменную именно так, чтобы на выходе из вероятностной модели естественным образом получался вектор вероятностей классов.

Но важно заметить, что линейная регрессия с категориальными признаками всё-таки остаётся *линейной* регрессией, что может приводить к неожиданным результатам. Проиллюстрирую это на примере: добавим к данным, которые мы рассматривали выше (на рис. 4.3), ещё одну бинарную размерность x_2, принимающую только значения 0 и 1. Предположим, что для значения $x_2 = 0$ мы получаем те же данные, что выше, с исходной функцией $y = \frac{1}{2}x_1 - \frac{1}{2}$, а для $x_2 = 1$ данные тоже описываются линейной функцией, но другой, $y = -\frac{1}{2}x - \frac{1}{2}$. Добавим к ним такой же нормально распределённый шум с дисперсией $\sigma = \frac{1}{2}$ и получим, например, такой набор данных:

$$\text{data}_2 = (\,0{,}46\ 0{,}13\ -0{,}16\ -0{,}31\ -0{,}45\ -0{,}71\ -0{,}84\ -0{,}72\ -0{,}90\ -1{,}74\ -1{,}37\,).$$

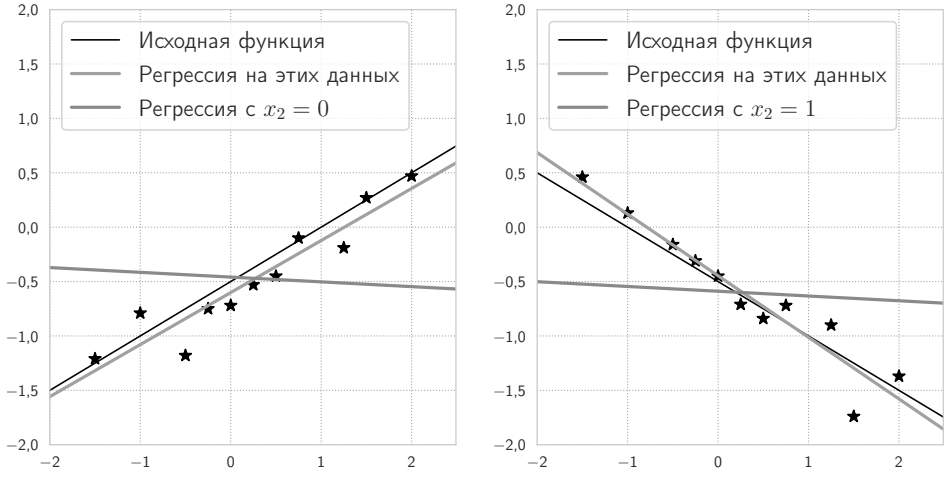
Линейная регрессия с бинарным признаком будет обычной линейной регрессией, у которой на входе два признака, один из которых бинарный, а на выходе то data, то data_2 в зависимости от значения бинарного признака. В целом получается, что мы обучаем линейную регрессию с такими матрицей признаков и вектором данных:

$$\boldsymbol{X} = \begin{pmatrix} \boldsymbol{x} & \boldsymbol{x} \\ \mathbf{O} & \mathbf{1} \end{pmatrix}^\top, \quad \boldsymbol{y} = \begin{pmatrix} \text{data} & \text{data}_2 \end{pmatrix}^\top,$$

где

$$\begin{aligned} \boldsymbol{x} &= (\,-1{,}5\ -1\ -0{,}5\ -0{,}25\ 0\ 0{,}25\ 0{,}5\ 0{,}75\ 1{,}25\ 1{,}5\ 2\,),\\ \text{data} &= (\,-1{,}21\ -0{,}79\ -1{,}18\ -0{,}75\ -0{,}72\ -0{,}53\ -0{,}45\ -0{,}10\ -0{,}19\ 0{,}27\ 0{,}47\,),\\ \text{data}_2 &= (\,0{,}46\ 0{,}13\ -0{,}16\ -0{,}31\ -0{,}45\ -0{,}71\ -0{,}84\ -0{,}72\ -0{,}90\ -1{,}74\ -1{,}37\,), \end{aligned}$$

вектор $\mathbf{O}$ состоит из 11 нулей, а вектор $\mathbf{1}$ — из 11 единиц.



(а) $x_2 = 0$ (б) $x_2 = 1$

Рис. 4.6. Линейная регрессия с бинарным признаком: (а) предсказания для $x_2 = 0$; (б) предсказания для $x_2 = 1$

Давайте обучим такую регрессию и посмотрим, что она предсказывает для разных значений бинарной переменной. Регрессия получится примерно такая:

$$\hat{y} = -0{,}044x_1 - 0{,}13x_2 - 0{,}46. \tag{4.11}$$

На рис. 4.6 показаны результаты предсказаний этой линейной регрессии для разных значений x_1 и двух возможных значений x_2: слева для $x_2 = 0$, а справа для $x_2 = 1$. Как видите, предсказания имеют очень мало общего с тренировочным множеством! Если на этих же данных обучить две отдельные линейные регрессии, результаты получатся куда лучше; они тоже показаны на рис. 4.6.

Дело здесь в том, что бинарный признак в линейной регрессии не представляет собой «точку выбора»; линейная регрессия с категориальными признаками не превращается в дерево принятия решений, которое могло бы принимать то или иное решение в зависимости от значения категориальной переменной. Это просто ещё одно число, которое может внести свой линейный вклад в предсказанное значение переменной.

Коэффициент при x_1 не может меняться в зависимости от значения x_2, и в уравнении (4.11) получается, что он принимает некоторое «среднее» значение между $-\frac{1}{2}$ и $\frac{1}{2}$, то есть оказывается близок к нулю.

В реальности мы «просим» у линейной регрессии решить задачу, изображённую на рис. 4.7, *а*: нужно приблизить изображённый набор точек в трёхмерном

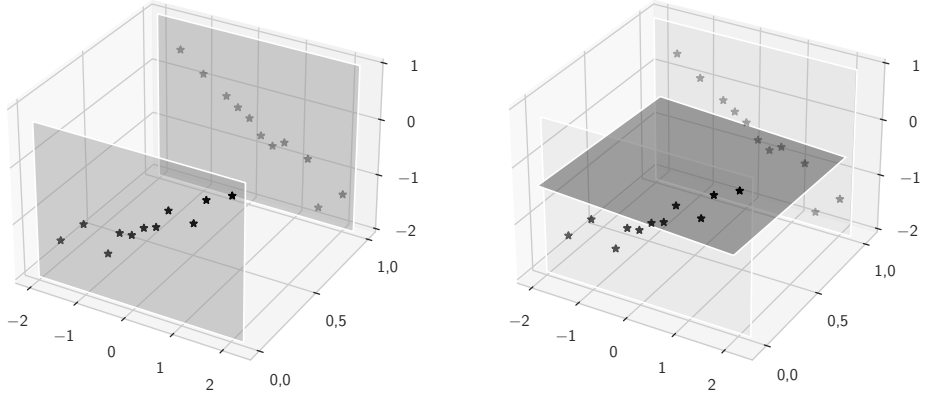


Рис. 4.7. Линейная регрессия с бинарным признаком: (а) исходные данные; (б) обученная плоскость

пространстве при помощи линейной функции от двух переменных, то есть плоскости. Как видите, хорошо это сделать никак не получится, и полученный нами ответ, изображённый на рис. 4.7, *б* — это лучший из имеющихся плохих вариантов.

Поэтому в случаях, когда значения категориальной переменной не просто вносят независимый друг от друга вклад в целевую переменную, а могут изменить форму зависимости от других признаков, нужно использовать не линейную регрессию, а какую-нибудь более выразительную модель — например, те же деревья принятия решений.

4.1.5. Локальные признаки

В предыдущих разделах мы обсудили многочлены и представление категориальных признаков — а можно ли обучить что-то другое? Конечно, да! Таким же способом можно подставлять в обычную линейную регрессию какие угодно признаки и получать множество разных моделей, которые можно условно разделить на два класса. И полиномиальная регрессия, и прямое кодирование категориальных переменных — это примеры так называемых *глобальных* признаков: добавление такого признака оказывает существенное влияние на вид всей кривой, которую мы обучаем.

Использование глобальных признаков подразумевает, что у нас есть какая-то интуиция насчёт того, как может выглядеть зависимость: например, чтобы решить, что мы хотим обучить квадратичную функцию, нужно каким-то образом

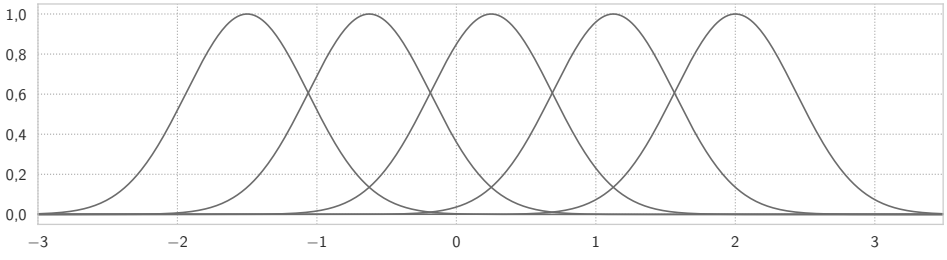


Рис. 4.8. Пять локальных гауссовских признаков

понимать, что квадратичные зависимости между признаками (их попарные произведения и квадраты) имеют смысл в этой задаче.

Но бывают и часто применяются и *локальные* признаки. Слово «локальный» здесь значит, что «действие», «влияние» каждого из этих признаков распространяется не слишком далеко. Чаще всего в качестве локальных признаков используются *радиальные базисные функции* (radial basis functions, RBF), у каждой из которых есть некий центр $\boldsymbol{\mu}$, а вокруг него функция убывает в зависимости от расстояния от $\boldsymbol{\mu}$ до текущей точки.

Давайте посмотрим, как это работает, на примере *гауссовских базисных функций*, которые выглядят так:

$$\phi(\boldsymbol{x};\boldsymbol{\mu}) = e^{-\frac{1}{2s}\|\boldsymbol{x}-\boldsymbol{\mu}\|^2}.$$

Обратите внимание, что здесь нет нужды добавлять нормировочную константу: да, гауссовские базисные функции копируют форму нормального распределения, но интегрироваться в единицу им незачем, всё равно смысл происходящего в том, чтобы подбирать коэффициенты их линейной комбинации.

На рис. 4.8 изображены пять таких базисных функций, распределённых равномерно на отрезке $[-1{,}5, 2]$, где находятся данные в нашем примере. Под равномерным распределением здесь имеется в виду распределение их центров μ, то есть на рисунке изображены $\phi(\boldsymbol{x};\mu_j)$, где $\mu_j = -1{,}5 + \frac{2+1{,}5}{5}(j-1)$ для каждого $j = 1,\ldots,5$.

С точки зрения линейной регрессии это значит, что мы моделируем y в виде

$$y = \sum_{i=1}^{M} w_i \phi(\boldsymbol{x};\boldsymbol{\mu}_i) + \epsilon, \text{ где } \epsilon \sim \mathcal{N}(0,\sigma^2).$$

Теперь определим гауссовские базисные функции в коде (здесь я адаптирую пример из [615]), а заодно и введём важную вспомогательную концепцию из па-

кета scikit-learn: определим пайплайн[1], который будет последовательно сначала выделять признаки, а затем обучать линейную регрессию. Для этого нужно будет определить выделение гауссовских локальных признаков как отдельный класс, реализующий методы fit и transform — именно из таких объектов можно составить пайплайн в scikit-learn. Вот как это может работать.

```
from sklearn.pipeline import make_pipeline
from sklearn.base import BaseEstimator, TransformerMixin

nums_gauss = [2, 5, 10]

class GaussianFeatures(BaseEstimator, TransformerMixin):
    def __init__(self, N, width_factor=.5):
        self.N = N
        self.width_factor = width_factor

    @staticmethod
    def _gauss_basis(x, y, width, axis=None):
        arg = (x - y) / width
        return np.exp(-0.5 * np.sum(arg ** 2, axis))

    def fit(self, X, y=None):
        self.centers_ = np.linspace(X.min(), X.max(), self.N)
        self.width_ = self.width_factor * (self.centers_[1]-self.centers_[0])
        return self

    def transform(self, X):
        return self._gauss_basis(X[:, :, np.newaxis], \
                self.centers_, self.width_, axis=1)

for num_gauss in nums_gauss:
    g_m = make_pipeline(GaussianFeatures(num_gauss),
                        linear_model.LinearRegression())
    g_m.fit(xd[:, np.newaxis], data)
    yfit = g_m.predict(xs[:, np.newaxis])
    print("Коэффициенты с~%d признаками: %s" % (num_gauss," ".join(
      ["%.4f" % x for x in g_m.get_params()['linearregression'].coef_])))
```

Запуская это на тех же xd и data, что выше, получим коэффициенты вроде таких (у вас, конечно, получатся другие, ведь данные были порождены случайно):

```
Коэффициенты с 2 признаками: 9.6902 10.6803
Коэффициенты с 5 признаками: -0.4874 -1.5486 0.2145 0.5596 -1.4100
```

[1] Шишков, прости: слово «конвейер» здесь прекрасно подходит, но, во-первых, в реальности так никто не говорит, а во-вторых, честно говоря, слово «конвейер» образовалось ровно тем же способом, прямым копированием conveyor из английского. Так почему бы не взять и пайплайн тоже...

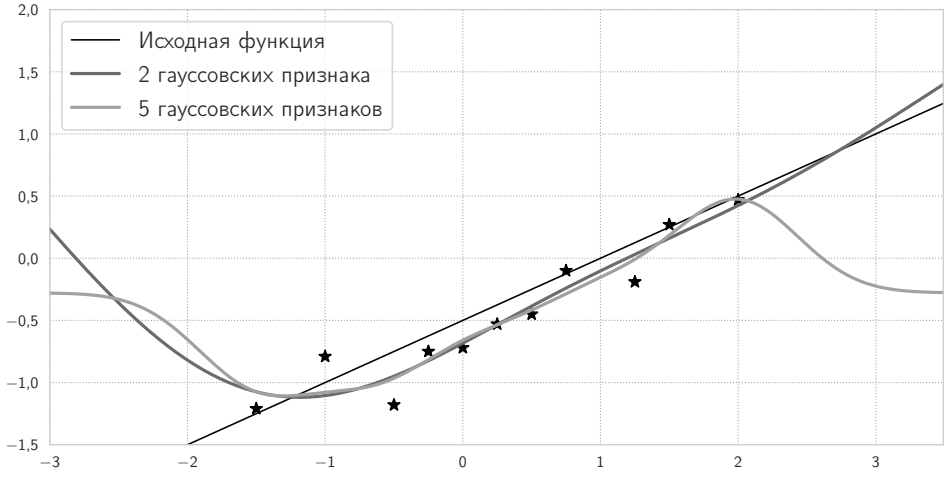


Рис. 4.9. Регрессия с локальными гауссовскими признаками

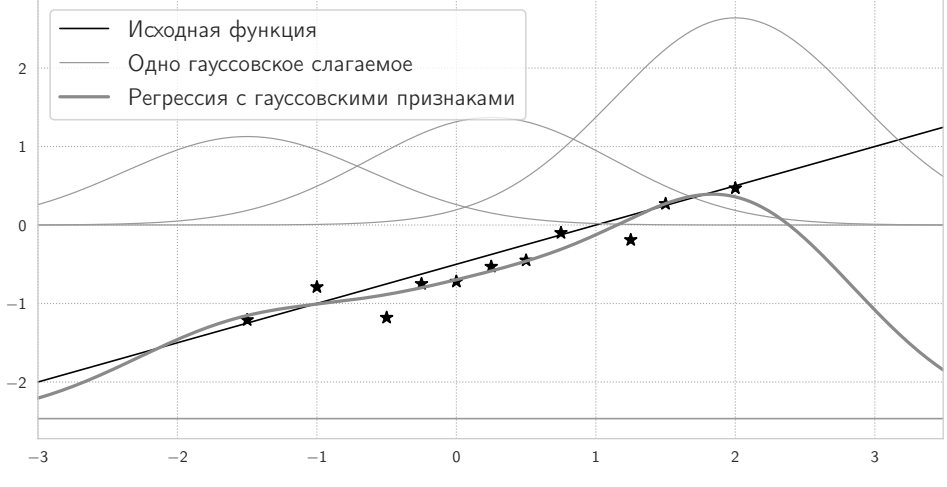


Рис. 4.10. Регрессия с тремя локальными гауссовскими признаками в деталях

```
Коэффициенты с 10 признаками: -10.7818 -10.4922 -10.8983 -10.0297 -9.7361
                               -8.7612 -9.0050 -9.0168 -9.5279 -11.7543
```

А если всё происходящее нарисовать, то получится нечто вроде рис. 4.9. Как видите, получаются красивые гладкие кривые, которые действительно следуют

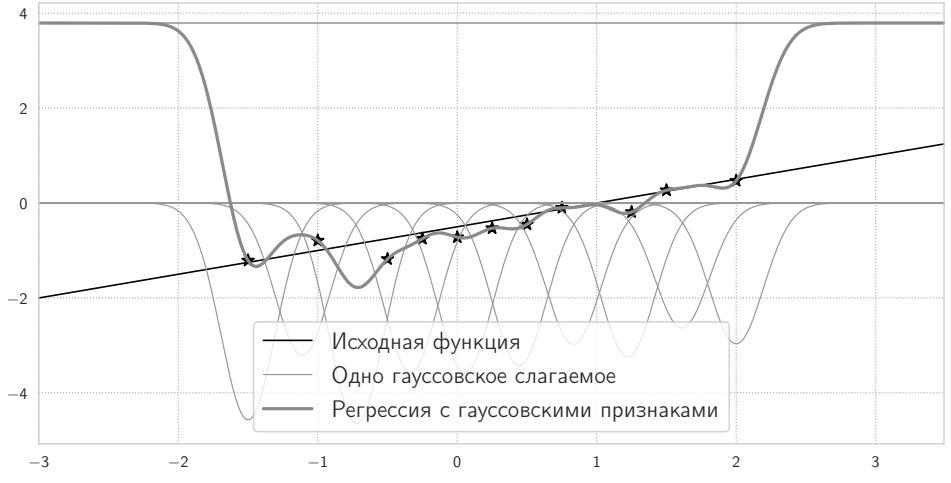


Рис. 4.11. Регрессия с десятью локальными гауссовскими признаками на большом датасете

за данными и неплохо их описывают. В данном случае очевидно, как и с полиномиальной регрессией, что свободы у модели с 10 гауссовскими признаками получилось многовато: модель с 11 свободными переменными опять может пройти через все точки точно, и её не стоит принимать всерьёз. Но с десятью признаками было ясно, что получится совсем перебор, а вот, например, пять признаков работают вполне неплохо.

На рис. 4.10 показана модель с 5 гауссовскими признаками в деталях: серые тонкие кривые показывают отдельные компоненты получающейся смеси (с учётом коэффициентов), а синяя кривая — это их сумма, то есть получающаяся в результате модель.

Если же добавить больше данных, то и десять признаков перестанут давать оверфиттинг: на рис. 4.11 мы видим кривую, которая неплохо следует за данными, а её «волнообразный» вид является просто следствием самой природы модели как суммы гауссианов.

Обратите внимание, как устроена эта модель: все гауссовские функции направлены в одну и ту же сторону, а хорошее приближение получается потому, что они вместе компенсируют большой положительный свободный член (горизонтальная серая черта; на этой картинке она проходит на на уровне около −12). Это достаточно типичная картина, и большой по модулю свободный член в такого рода регрессии сам по себе не является признаком оверфиттинга.

Как видите, линейная регрессия может работать с самыми разными функциями признаков, и на собственно ход рассуждений в регрессии форма признаков

никакого влияния не оказывает. Но она оказывает большое влияние на результат — это мы увидим уже в следующем разделе. Поэтому в дальнейшем я не буду возвращаться к обозначениям вроде $\phi(\boldsymbol{x})$ и буду просто предполагать, что линейная регрессия строится на входе, который мы обозначаем $\boldsymbol{x}$, предполагая, что этот вход уже представляет собой результат выделения признаков.

А в следующем разделе мы поговорим о том, что же делать с оверфиттингом, когда добавить больше данных не получается.

4.2. Оверфиттинг и регуляризация

Скажем несколько слов о тех правилах, которых необходимо надлежит держаться, переводя стихи стихами. Первое: излишнюю верность почитаю излишнею неверностию. Например, это выражение в оригинале благородно; выражение, соответствующее ему в вашем языке, низко: будучи слишком точным, вы унижаете благородство подлинника и заменяете его низостию.

Жак Делиль. О переводах вообще и в особенности о переводах стихов (пер. В. А. Жуковского)

4.2.1. Оверфиттинг в регрессии с полиномиальными признаками

Оверфиттинг — это одна из главных проблем в машинном обучении. Как мы обсуждали в подразделе 1.4.2, этот эффект возникает, когда у модели становится слишком много параметров по сравнению с объёмом имеющихся в обучающей выборке данных. В результате модель слишком хорошо обучается «облизывать» данные из тренировочного датасета, и от этого страдает её обобщающая способность. Грубо говоря, если пытаться обучить модель машинного обучения отличать котиков от пёсиков, имея всего три фотографии в каждом классе, скорее всего, модель просто заметит, что, например, верхний левый пиксель у всех трёх котиков случайно вышел светлее, чем у всех трёх собачек, — это будет идеально работать на обучающей выборке, но окажется совершенно бесполезным для дальнейшего обобщения. Замечу, что модели всё равно, какой пиксель использовать, а их миллионы, и среди них наверняка найдётся подходящий даже по чистой случайности.

Слова «слишком много параметров» нас не удивляют, когда речь идёт о современной нейронной сети, у которой число параметров может исчисляться миллиардами. Но как может быть «слишком много параметров» у линейной регрессии?! Казалось бы, куда уж проще и куда уж меньше параметров, чем по одному на размерность данных.

Однако в предыдущем разделе мы увидели, что «размерность данных» — понятие растяжимое: мы можем сами её увеличить, придумывая новые признаки. Кстати, именно этим и занимаются нейронные сети: пытаются автоматически обучиться выделять из данных признаки, которые будут максимально удобны для решения нужной задачи на последнем слое, где у нейронной сети в подавляющем большинстве случаев находится как раз либо линейная, либо логистическая регрессия.

Таблица 4.1. Коэффициенты многочленов полиномиальной регрессии для примера, изображённого на рис. 4.12

d	**Многочлен**
0	$0{,}0306$
1	$-0{,}0365 + 0{,}2462x$
2	$0{,}2411 + 0{,}3973x - 0{,}2863x^2$
3	$0{,}0295 + 1{,}2717x + 0{,}0196x^2 - 0{,}4313x^3$
4	$0{,}1820 + 1{,}6629x - 0{,}5965x^2 - 0{,}6819x^3 + 0{,}2348x^4$
5	$0{,}1722 + 1{,}6964x - 0{,}5463x^2 - 0{,}7377x^3 + 0{,}2119x^4 + 0{,}0175x^5$
...	...
9	$0{,}1370 + 1{,}1876x + 0{,}8948x^2 + 2{,}2403x^3 - 7{,}0611x^4 - 1{,}7442x^5$ $+8{,}0992x^6 - 1{,}5797x^7 - 2{,}2884x^8 + 0{,}8084x^9$
10	$0{,}2380 + 0{,}4192x - 3{,}1878x^2 + 15{,}4055x^3 + 11{,}2409x^4 - 53{,}4733x^5$ $-0{,}6596x^6 + 52{,}1244x^7 - 13{,}5341x^8 - 13{,}9678x^9 + 5{,}2790x^{10}$

Давайте разберём классический, но от этого не менее наглядный пример. Будем обучать регрессию с полиномиальными признаками на одном и том же датасете, постепенно увеличивая степень многочлена. Точки породим из функции $\sin(2x)$, распределив 11 точек на интервале $[-2, 2]$ приблизительно равномерно и добавив нормально распределённый шум с дисперсией $\frac{1}{4}$:

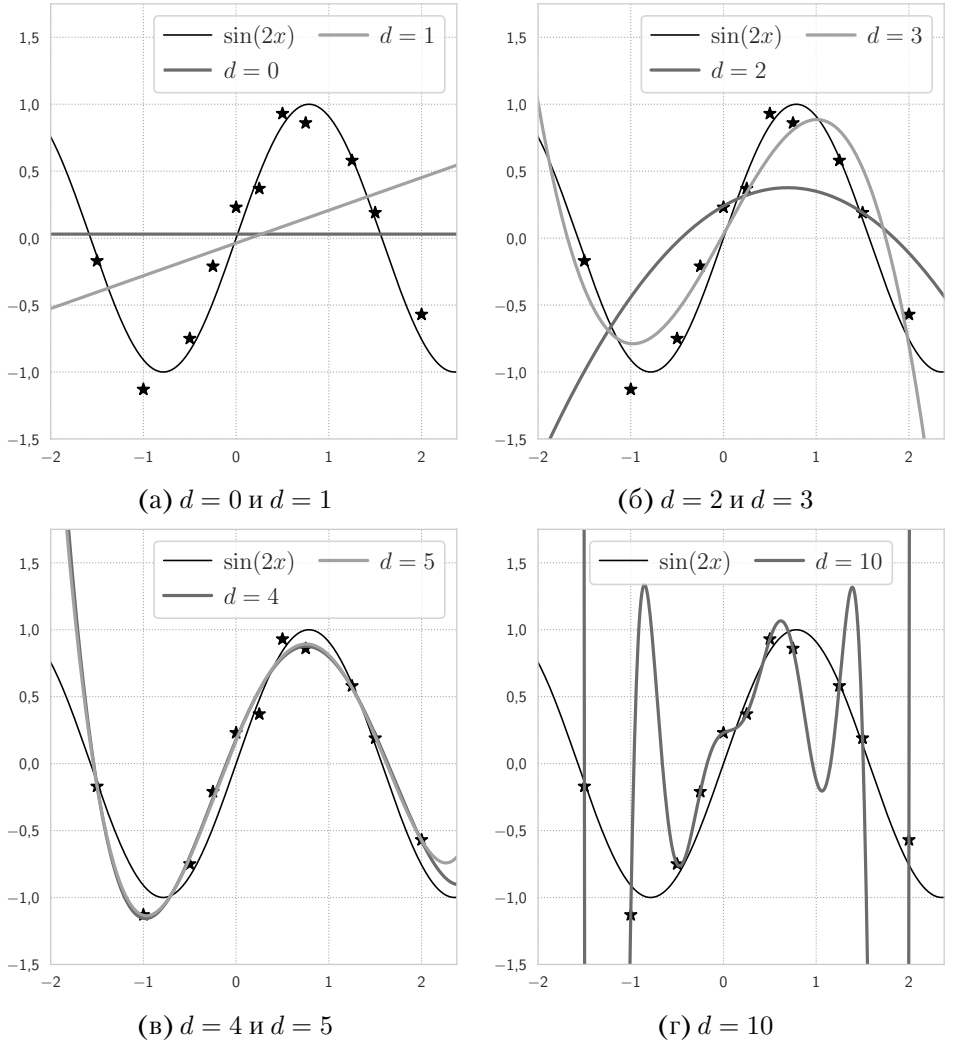
```
orig = lambda x : np.sin(2*x)
xd = np.array([-3, -2, -1, -0.5, 0, 0.5, 1, 1.5, 2.5, 3, 4]) / 2
num_points = len(xd)
data = orig(xd) + np.random.normal(0, .25, num_points)
```

В этом примере массив значений получился такой (разумеется, у вас при запуске получатся другие числа); в списке ниже приведены пары (x_n, y_n):

$$D = \{(-1{,}5, -0{,}172), (-1, -1{,}131), (-0{,}5, -0{,}759), (-0{,}2, -0{,}214), (0, 0{,}238),$$
$$(0{,}2, 0{,}379), (0{,}5, 0{,}930), (0{,}8, 0{,}860), (1{,}2, 0{,}585), (1{,}5, 0{,}192), (2, -0{,}570)\}.$$

Теперь выделим полиномиальные признаки для всех степеней многочлена от 1 до 11 и будем обучать линейные регрессии с разными степенями:

```
def train_model(xs, ys):
    return linear_model.LinearRegression(fit_intercept=True).fit( xs, ys )

xd_d = np.vstack([xd ** i for i in range(1, num_points+1)]).transpose()

trained_models = []
for d in range(1, num_points + 1):
       trained_models.append( train_model( xd_d[:, :d], data ) )
```



(а) $d = 0$ и $d = 1$ (б) $d = 2$ и $d = 3$

(в) $d = 4$ и $d = 5$ (г) $d = 10$

Рис. 4.12. Результаты обучения полиномиальной регрессии с возрастающей степенью многочлена d

Изобразим то, что получается (код для рисования я опущу); на рис. 4.12 показаны кривые многочленов разной степени вместе с точками данных и исходной синусоидой. И вот что можно заметить:

- на рис. 4.12, *а* всё просто: регрессия степени $d = 0$ — это не настоящая регрессия, а просто константа, равная среднему $\bar{y} = \frac{1}{N}\sum_{n=1}^{N} y_n$, а для $d = 1$ мы получаем уже хорошо нам знакомую линейную регрессию;

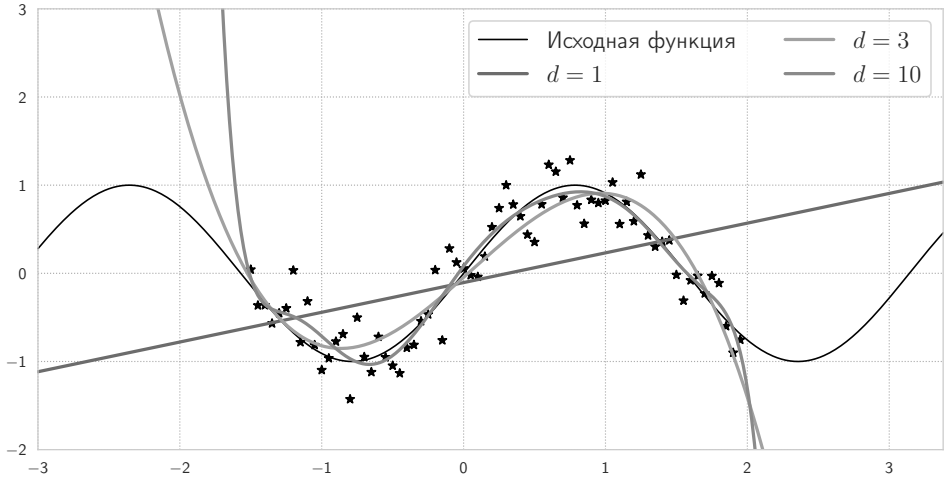


Рис. 4.13. Полиномиальная регрессия на большом датасете

- рис. 4.12, *б* показывает параболу для $d = 2$, которая не выглядит значительным улучшением по сравнению с $d = 1$, а также кубическую кривую при $d = 3$; интуитивно кажется, что кубическая кривая очень хорошо описывает данные — они у нас взяты из приблизительно одного периода синусоиды, а один период синусоиды как раз весьма неплохо описывается кубической кривой;
- рис. 4.12, *в* подтверждает эту интуицию: когда мы переходим от $d = 3$ к более высоким степеням $d = 4$ и $d = 5$, графики почти не меняются: добавление четвёртой и пятой степеней оказывается практически ненужным;
- но всё меняется на рис. 4.12, *г*, где показана полиномиальная регрессия для $d = 10$; теперь многочлен десятой степени может *точно* пройти через 11 точек данных, и, конечно, именно это он и делает, становясь интерполяционным многочленом; ясно, что дальше увеличивать степень уже точно нет никакого смысла.

Таблица 4.1 показывает коэффициенты этих многочленов.

Рисунок 4.12, *г* — это и есть классический, дистиллированный пример оверфиттинга: ошибка на тренировочном множестве упала буквально до нуля, но очевидно, что для предсказаний использовать многочлен с рис. 4.12, *г* было бы весьма опрометчиво. Разумеется, как и гауссовские признаки в предыдущем разделе, полиномиальная регрессия начинает вести себя хорошо, если добавить данных: рис. 4.13 показывает те же модели, обученные на новом, увеличенном наборе данных. Как видите, теперь многочлен степени 10 замечательно следует за точками

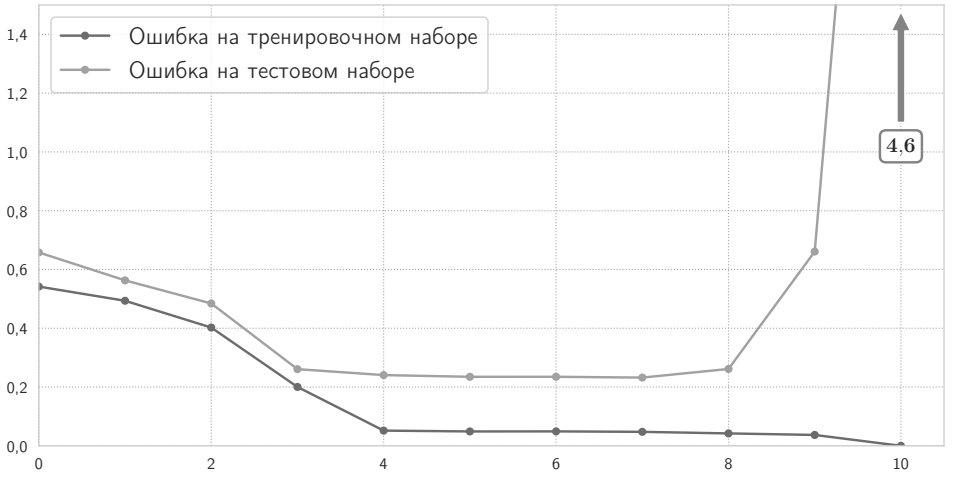


Рис. 4.14. Ошибки на тренировочном и тестовом множествах для полиномиальных регрессий разной степени в примере на рис. 4.12

данных, разве что по-прежнему бесполезен для экстраполяции: как только точки заканчиваются, он мгновенно убегает на бесконечность и слева, и справа. Но такова судьба многих регрессионных моделей, и мы увидим разницу между *интер*поляцией, то есть предсказанием в точках, окружённых данными, и *экстра*поляцией, то есть попытками продолжить найденную закономерность за пределы имеющегося датасета, и в разделе 4.4, где мы увидим, как быстро растёт дисперсия предсказательного распределения в таких случаях, и в разделе 4.5, где мы столкнёмся с этим эффектом на практике.

Чтобы увидеть эффект оверфиттинга ещё более наглядно, давайте добавим тестовое множество точек (в нём пусть точек будет много, здесь не жалко) и подсчитаем ошибку предсказания каждого из многочленов:

```
num_test_points = 100
xtest = np.random.rand(num_test_points) * 4 - 2
xtest_d = np.vstack([xtest ** i for i in range(1, num_points+1)]).transpose()
ytest = orig(xtest) + np.random.normal(0, .25, num_test_points)

models = [ train_model( xd_d[:, :d], data ) for d in range(1, 11) ]
predictions = [np.ones(num_test_points) * np.mean(data)] + \
       [ models[d-1].predict(xtest_d[:, :d]) for d in range(1, 11) ]
train_predictions = [np.ones(num_points) * np.mean(data)] + \
       [ models[d-1].predict(xd_d[:, :d]) for d in range(1, 11) ]
test_errors = [ np.mean(np.abs(pred - ytest)) for pred in predictions ]
train_errors = [ np.mean(np.abs(pred - data)) for pred in train_predictions ]
```

И теперь можно нарисовать результат; он показан на рис. 4.14, где по горизонтальной оси отложена степень многочлена, а по вертикальной — среднее значение отклонения $\frac{1}{N}\sum_{n=1}^{N}\left|\boldsymbol{w}^\top \boldsymbol{x}_n - y_n\right|$ для тренировочного и тестового наборов. Как видите, ошибка на тренировочном наборе монотонно убывает до нуля; это логично, потому что в данном случае все менее выразительные модели являются просто частными случаями более выразительных (парабола не может лучше описывать данные, чем кубический многочлен, потому что это частный случай кубического многочлена).

А вот ошибка на тестовом множестве ведёт себя совсем по-другому: она действительно сначала убывает до $d = 3$, потом остаётся приблизительно постоянной до $d = 7$ (небольшие флуктуации связаны, скорее всего, просто с тем, как случайно легли точки в обучающей выборке), но начиная с $d = 8$ ошибка на тестовом множестве начинает расти очень быстро. Ошибка интерполяционного многочлена для $d = 10$ уже не просто не помещается на график, а превышает его на порядок (на рис. 4.14 значение для $d = 10$ указано под стрелочками в правом верхнем углу)! Это и есть классический оверфиттинг.

Разумеется, этот эффект не ограничивается интерполяционными многочленами, здесь дело в том, что свободных параметров постепенно становится почти столько же, а затем и ровно столько же, сколько точек в обучающей выборке. Так, на рис. 4.9 была изображена регрессия с десятью гауссовскими признаками, у которой наблюдается ровно тот же эффект: кривая точно проходит через обучающую выборку, что делает её не слишком полезной для предсказаний.

Конечно, в данном конкретном случае особой проблемы нет: мы можем посмотреть на график и сразу заметить, что что-то пошло не так; мы даже можем сразу заметить, что обучающая выборка похожа именно на кубический многочлен, а не какой-то другой. Но это только потому, что данные имеют размерность 1, и нам легко их нарисовать. В размерности 10 или 100 мы уже ничего подобного заметить не смогли бы, а оверфиттинг более чем возможен и там. Что же делать? Есть ли какой-то другой способ заметить оверфиттинг?

4.2.2. Гребневая регрессия

Начну с ответа, подробную мотивацию которого смогу объяснить только в следующем разделе. Ответ здесь можно заметить не на графиках, а в табл. 4.1. Там видно, что коэффициенты обученных многочленов постепенно растут и становятся особенно большими, когда начинается оверфиттинг (в таблице это строки $d = 9$ и $d = 10$). Да и для интуитивного восприятия это звучит странно: если у нас есть десять точек в размерности 1 и значения (x_n, y_n) все находятся около нуля, не превышая по модулю 2, то кажется немного удивительным, что описывать их нужно многочленом с коэффициентами около 50.

Давайте попробуем прямолинейно и простодушно бороться с этим: добавим в целевую функцию метода наименьших квадратов (а мы пока что всё время

пользовались для обучения именно им) штраф за слишком большие коэффициенты многочлена. Например, вместо $\text{RSS}(\boldsymbol{w}) = \sum_{n=1}^{N}(y_n - \boldsymbol{w}^\top \boldsymbol{x}_n)^2$ рассмотрим

$$L(\boldsymbol{w}) = \sum_{n=1}^{N}(y_n - \boldsymbol{w}^\top \boldsymbol{x}_n)^2 + \alpha \left\|\boldsymbol{w}\right\|^2, \tag{4.12}$$

где α — некоторая константа, которая называется *коэффициентом регуляризации* (его надо будет как-нибудь выбрать), а всё слагаемое $\alpha \left\|\boldsymbol{w}\right\|^2$ называется *регуляризатором* (regularizer). В этом разделе мы не будем менять обозначения, но отметим, что часто регуляризуют не весь вектор весов, а только коэффициенты при элементах исходного вектора признаков, а свободный член w_0 регуляризовать не обязательно.

Как оптимизировать функцию ошибки (4.12)? Да точно так же — она всё ещё представляет собой квадратичную функцию от $\boldsymbol{w}$. Перепишем целевую функцию в матричном виде, аналогично формуле (4.1):

$$\text{RSS}(\boldsymbol{w}) = \frac{1}{2}(\boldsymbol{y} - \boldsymbol{X}\boldsymbol{w})^\top (\boldsymbol{y} - \boldsymbol{X}\boldsymbol{w}) + \frac{\alpha}{2}\boldsymbol{w}^\top \boldsymbol{w}.$$

Затем возьмём градиент по $\boldsymbol{w}$; после вычислений, полностью аналогичных проведённым в разделе 4.1.2, получится

$$\boldsymbol{w}^* = \left(\boldsymbol{X}^\top \boldsymbol{X} + \alpha \mathbf{I}\right)^{-1} \boldsymbol{X}^\top \boldsymbol{y}. \tag{4.13}$$

Этот метод носит название *гребневой регрессии* (ridge regression), и это один из классических методов *регуляризации* (regularization) в машинном обучении; оба эти термина требуют обсуждения.

Начнём с регуляризации, которая полностью называется *регуляризацией по Тихонову*[1] (Tikhonov regularization). Смысл её состоит в следующем: предположим, что мы хотим решить систему линейных уравнений вида

$$A\boldsymbol{x} = \boldsymbol{b}.$$

Метод наименьших квадратов, минимизирующий $\|A\boldsymbol{x} - \boldsymbol{b}\|_2^2$, здесь обычно работает прекрасно, гораздо быстрее, скажем, метода Гаусса.

[1] *Андрей Николаевич Тихонов* (1904–1993) — советский математик и геофизик, основатель факультета вычислительной математики и кибернетики МГУ (1979), директор Института прикладной математики АН СССР (1978–1989, позже почётный директор). Тихонов начинал как тополог и специалист по функциональному анализу, где ввёл понятия, позднее названные тихоновским произведением топологических пространств и функциональными пространствами Тихонова. Во время Великой Отечественной Тихонов разрабатывал математические модели движения газовых облаков и функционирования противогазов, в 1948 году организовал вычислительную лабораторию для расчёта процессов, происходящих при взрыве атомной бомбы, а затем создал математические методы разведки полезных ископаемых, при помощи которых в СССР нашли немало нефти, и так далее. Тихонов был одним из главных математиков Советского Союза, что неизбежно вовлекало его и в политику: в 1955 году он подписал «письмо трёхсот» против Трофима Лысенко, а в 1973-м подписал письмо в газету «Правда» с осуждением Андрея Сахарова, от чего не отказался и десять лет спустя, став одним из четырёх авторов открытого письма 1983 года против Сахарова «Когда теряют честь и совесть».

Но он работает, только если задача *корректно поставлена* (well-posed), то есть её решение существует и единственно. А если определитель A равен нулю и решений уравнения $A\boldsymbol{x} = \boldsymbol{b}$ много, то задача становится *некорректной* (ill-posed, ill-conditioned) и метод наименьших квадратов приводит либо к переопределённой, либо (чаще) к недоопределённой системе уравнений. Кроме того, в недоопределённых задачах непонятно, какое из многочисленных решений выбрать.

Для таких случаев Тихонов предложил следующий метод: давайте добавим к методу наименьших квадратов дополнительное слагаемое и будем минимизировать

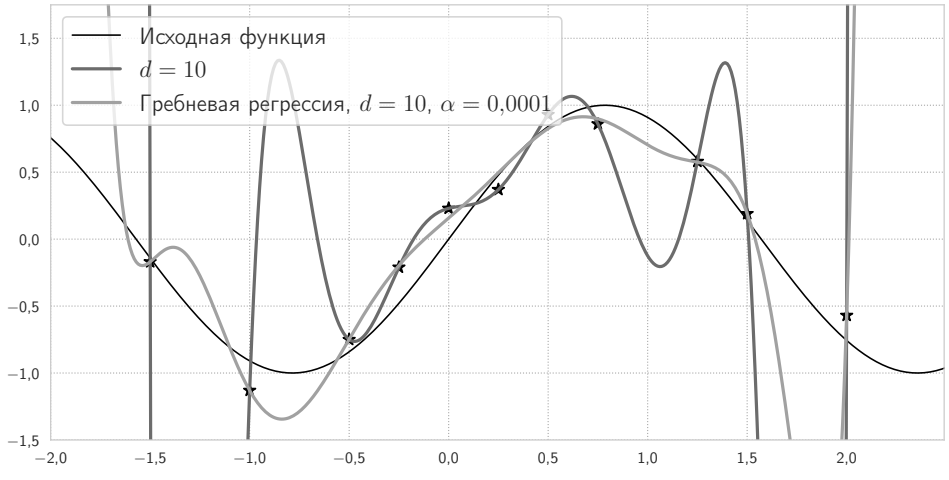
$$\|A\boldsymbol{x} - \boldsymbol{b}\|_2^2 + \|\Gamma\boldsymbol{x}\|_2^2,$$

где Γ — некоторая *матрица Тихонова*. Частный случай этого метода, в котором $\Gamma = \alpha\mathbf{I}$, даёт гребневую регрессию. Отсюда и сам термин «регуляризация»: это значит, что мы добавляем к матрице $A^\top A$, которая является вырожденной, нерегулярной (то есть $\det A = 0$), слагаемое $\Gamma^\top\Gamma$, например $\alpha\mathbf{I}$, и полученная матрица $A^\top A + \Gamma^\top\Gamma$ становится невырожденной, регулярной, то есть обратимой.

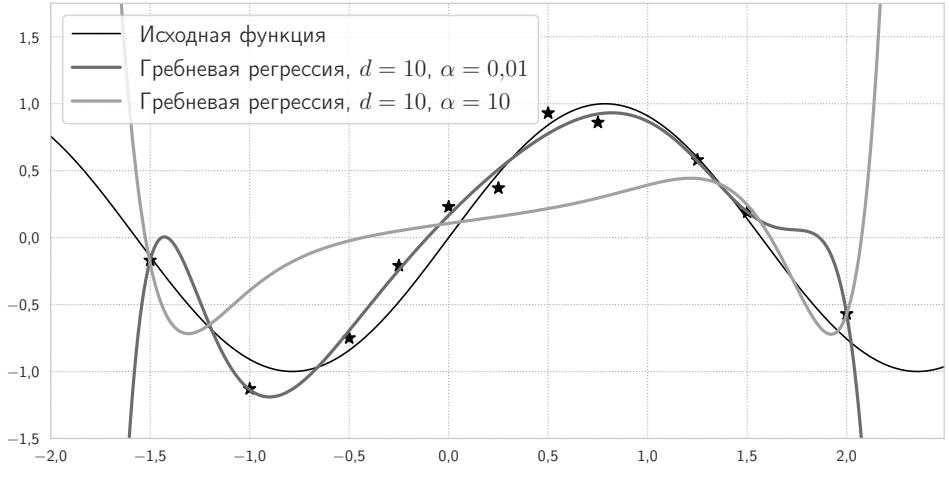
Смысл термина «гребневая регрессия» (ridge regression) объяснить уже сложнее, да и не слишком это объяснение для нашей темы релевантно. Первоисточником гребневой регрессии был *метод Бокса — Уилсона* (Box — Wilson method) для задач оптимизации, который также называется *методом поверхностей отклика* (response surface methodology) [74]. Смысл здесь такой: предположим, что нам нужно максимизировать некоторую функцию от нескольких переменных; мы не умеем считать значение этой функции в любой точке, но имеются некоторые данные, по которым можно обучить модель, приближающую эту функцию, а также можно ставить (не бесплатные) эксперименты в новых точках. Тогда логично будет использовать эту модель для того, чтобы двигаться в сторону, где мы ожидаем максимум этой функции, ставить там новые эксперименты и обновлять модель. В простейшем случае можно двигаться в сторону градиента модели, а когда линейное движение перестаёт работать, можно перейти к моделям более высокого порядка. Подробнее об этом можно прочитать, например, в книге [420].

Гребневый анализ (ridge analysis) — это как раз метод Бокса — Уилсона, в котором обучаются модели второго порядка; он был разработан Артуром Хорлом (Arthur Hoerl) в конце 1950-х годов [264, 265], а затем Хорл и Роберт Кеннард (Robert Kennard) применили его к задачам регрессии [266, 267]. Последние две статьи и считаются теми основополагающими работами, которые ввели гребневую регрессию в оборот, хотя к тому времени уже существовали и результаты Тихонова, и многие другие похожие публикации... но пора завершать исторический экскурс.

Вернёмся к нашему примеру. Рисунок 4.15 показывает результаты гребневой регрессии в нашем примере для нескольких разных значений α. В частности, для $\alpha = 0$ мы получаем ту же полиномиальную регрессию, что выше, а затем, по мере увеличения α, форма кривой постепенно начинает сглаживаться, а коэффициенты многочленов — уменьшаться (эти коэффициенты приведены в табл. 4.2).



(а)



(б)

Рис. 4.15. Результаты обучения гребневой регрессии со степенью многочлена $d = 10$: (а) $\alpha = 0$ и $\alpha = 0{,}0001$; (б) $\alpha = 0{,}01$ и $\alpha = 10{,}0$

И если мы нарисуем ошибки на тренировочном и тестовом множестве по аналогии с рис. 4.14, мы получим график, изображённый на рис. 4.16, в котором хорошо виден «фазовый переход» от жёсткого оверфиттинга к более адекватным моделям. Мы обсудим такие графики гораздо подробнее в разделе 6.2, где увидим, из чего складывается ожидаемая ошибка предсказания моделей машинного обучения, а сейчас перейдём к другим формам регуляризации.

Таблица 4.2. Коэффициенты многочленов гребневой регрессии для примера, изображённого на рис. 4.15

d	α	**Многочлен**
10	0	$0{,}2380 + 0{,}4192x - 3{,}1878x^2 + 15{,}4055x^3 + 11{,}2409x^4 - 53{,}4733x^5$ $-0{,}6596x^6 + 52{,}1244x^7 - 13{,}5341x^8 - 13{,}9678x^9 + 5{,}2790x^{10}$
10	0,0001	$0{,}1633 + 1{,}2902x - 0{,}0024x^2 + 2{,}1602x^3 - 3{,}0072x^4 - 4{,}5374x^5$ $+4{,}4054x^6 + 2{,}4218x^7 - 2{,}0933x^8 - 0{,}4153x^9 + 0{,}3216x^{10}$
10	0,01	$0{,}1710 + 1{,}5168x - 0{,}5419x^2 - 0{,}0235x^3 - 0{,}0219x^4 - 0{,}3901x^5$ $+0{,}2735x^6 - 0{,}2253x^7 + 0{,}0332x^8 + 0{,}1248x^9 - 0{,}0485x^{10}$
10	10,0	$0{,}1067 + 0{,}2064x - 0{,}0275x^2 + 0{,}1184x^3 - 0{,}0357x^4 + 0{,}0796x^5$ $-0{,}0376x^6 + 0{,}0340x^7 - 0{,}0299x^8 - 0{,}0435x^9 + 0{,}0235x^{10}$

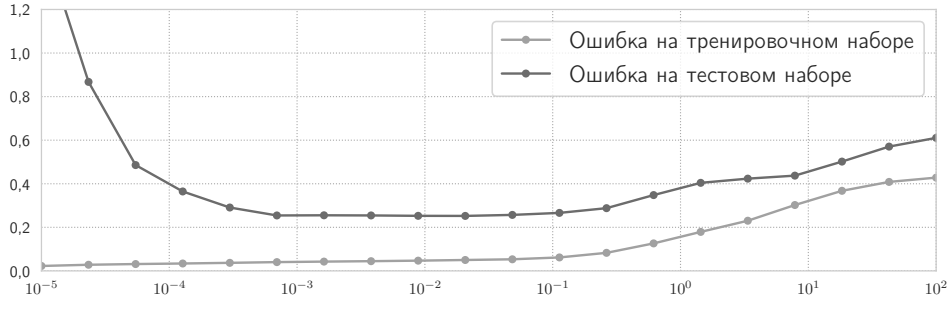


Рис. 4.16. Ошибки на тренировочном и тестовом множествах для гребневых регрессий степени $d = 10$ в примере на рис. 4.15

4.2.3. Лассо-регрессия

Хотя в разделе 4.3 мы мотивируем гребневую регрессию с вероятностной точки зрения, это далеко не единственный возможный способ регуляризации. Мы добавили к целевой функции регуляризатор $\alpha\|\boldsymbol{w}\|^2$, чтобы уменьшить веса, — но могли бы добавить с той же целью и какую-нибудь другую функцию, правда? В частности, гребневую регрессию называют *L_2-регуляризатором*, потому что он представляет собой L_2-норму вектора $\boldsymbol{w}$, но можно использовать и другие варианты. Например, L_1-норму, то есть сумму модулей компонентов $\boldsymbol{w}$:

$$L(\boldsymbol{w}) = \sum_{n=1}^{N} (y_n - \boldsymbol{w}^\top \boldsymbol{x}_n)^2 + \alpha \sum_{i=1}^{d} |w_i| \,,$$

где α — коэффициент регуляризации.

Такой подход называется *лассо-регрессией* (lasso regression). Слово «лассо» ввёл Роберт Тибширани в работе [594], и оно появилось как аббревиатура от

Таблица 4.3. Коэффициенты многочленов лассо-регрессии для примера, изображённого на рис. 4.17

d	α	**Многочлен**
10	0	$0{,}2380 + 0{,}4192x - 3{,}1878x^2 + 15{,}4055x^3 + 11{,}2409x^4 - 53{,}4733x^5$ $-0{,}6596x^6 + 52{,}1244x^7 - 13{,}5341x^8 - 13{,}9678x^9 + 5{,}2790x^{10}$
10	0,0001	$0{,}1663 + 1{,}6475x - 0{,}4931x^2 - 0{,}6323x^3 + 0{,}1566x^4 - 0{,}0254x^5$ $+0{,}0131x^6 + 0{,}0011x^7 + 0{,}005x^8 + 0{,}005x^9$
10	0,01	$0{,}1079 + 1{,}2651x - 0{,}0944x^2 - 0{,}0823x^3 - 0{,}1633x^4 - 0{,}1540x^5$ $+0{,}0373x^6 - 0{,}0073x^7 + 0{,}0140x^8 - 0{,}0041x^9 - 0{,}0024x^{10}$
10	10,0	$0{,}0868 - 0{,}005x^{10}$

least absolute shrinkage and selection operator (оператор сжатия и выбора на основе наименьших абсолютных значений). Но смысл этого слова тоже не случаен: Тибширани пишет, что идея лассо-регуляризации основана на более ранней работе Лео Бреймана (создателя метода случайных лесов [76]), который в 1993 году предложил вариант регуляризации линейной регрессии с мультипликативным сокращением весов неотрицательными коэффициентами и назвал его «неотрицательной гарротой» (non-negative garrote) [668].

Давайте сделаем для лассо-регрессии то же самое, что мы уже сделали выше для гребневой регрессии. Мы видим примерно тот же эффект, на этот раз показанный на рис. 4.17 и в табл. 4.3, а также на графике на рис. 4.18. Но обратите внимание, что теперь коэффициенты многочленов имеют совсем разный размер, а в крайне зарегуляризованном случае $\alpha = 10{,}0$ почти все коэффициенты попросту строго равны нулю. Да и для более маленьких значений α уже становится ясно, куда ветер дует, то есть какие коэффициенты останутся важны, а какие при дальнейшем увеличении α обнулятся первыми.

Это и есть тот самый эффект лассо-регрессии — но откуда он берётся?

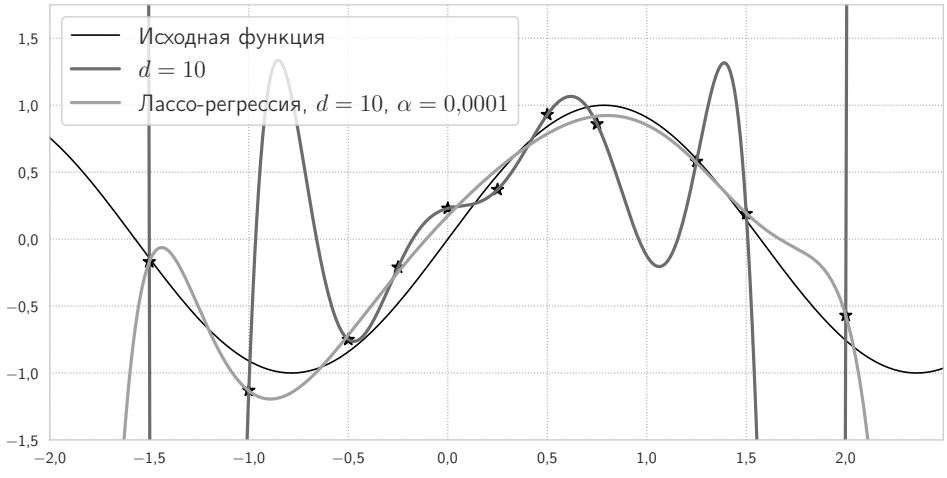
4.2.4. Геометрия регуляризации и другие её формы

Чтобы лучше понять, к чему стремятся и к чему приводят разные формы регуляризации, рассмотрим немного с другой стороны задачу оптимизации метода наименьших квадратов с регуляризатором $R(\boldsymbol{w})$:

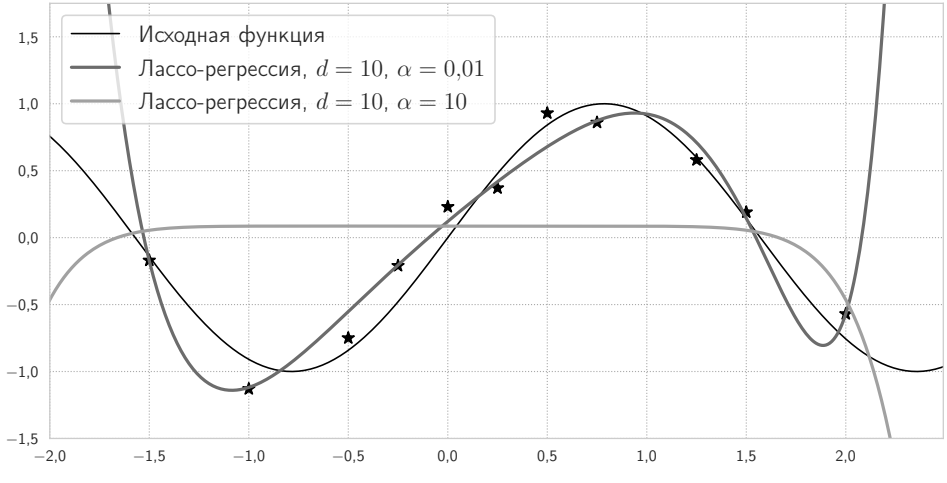
$$\sum_{n=1}^{N}(y_n - \boldsymbol{w}^\top \boldsymbol{x}_n)^2 + R(\boldsymbol{w}). \tag{4.14}$$

Начнём с того, что запишем задачу условной оптимизации

$$\sum_{n=1}^{N}(y_n - \boldsymbol{w}^\top \boldsymbol{x}_n)^2 \longrightarrow \min \quad \text{при условии} \quad R(\boldsymbol{w}) \leq \beta \tag{4.15}$$



(а)



(б)

Рис. 4.17. Результаты обучения лассо-регрессии со степенью многочлена $d = 10$: (а) $\alpha = 0$ и $\alpha = 0{,}0001$; (б) $\alpha = 0{,}01$ и $\alpha = 10{,}0$

для некоторой константы β. Тогда лагранжиан для этой задачи условной оптимизации[1] будет выглядеть как

$$\mathcal{L}(\boldsymbol{w}, \lambda) = \sum\nolimits_{n=1}^{N} (y_n - \boldsymbol{w}^\top \boldsymbol{x}_n)^2 + \lambda \left(R(\boldsymbol{w}) - \beta \right). \tag{4.16}$$

[1] Чтобы проследить за рассуждениями здесь, придётся вспомнить базовый курс математического анализа; давать здесь такое введение я не буду, а просто сошлюсь на классические учебники [505, 692].

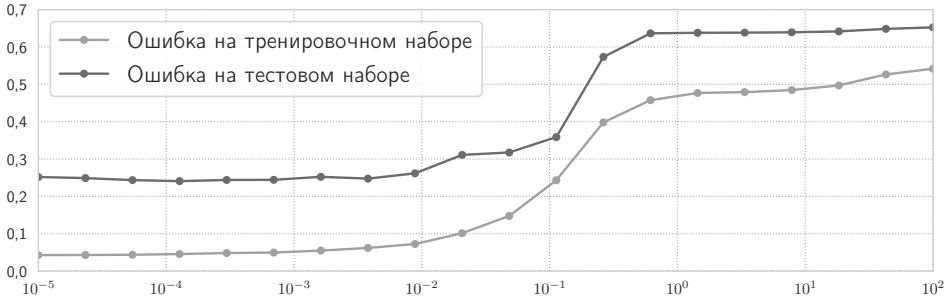


Рис. 4.18. Ошибки на тренировочном и тестовом множествах для лассо-регрессий степени $d = 10$ в примере на рис. 4.17

Теперь принцип двойственности говорит нам, что при некоторых дополнительных условиях[1] задача условной минимизации (4.15) эквивалентна задаче максимизации двойственной функции

$$g(\lambda) = \inf_{\boldsymbol{w}} \mathcal{L}(\boldsymbol{w}, \lambda).$$

А это значит, что для каждого β существует такое λ (и наоборот, для каждого λ существует такое β), что, минимизируя лагранжиан (4.16) с этим λ, мы на самом деле минимизируем сумму квадратов отклонений с условием $R(\boldsymbol{w}) \leq \beta$; а лагранжиан — это и есть наша целевая функция с регуляризатором. Правда, явной формулы для соответствия между λ и β мы тут не получим, это соответствие скрыто за нетривиальной задачей оптимизации.

Иначе говоря, оказывается, что регуляризация фактически добавляет некоторые геометрические условия на ответ: вместо того чтобы искать глобальный минимум суммы квадратов отклонений, мы ищем этот минимум внутри области, заданной неравенством $R(\boldsymbol{w}) \leq \beta$. И здесь очень интересно посмотреть на то, как выглядят эти области для разных $R(\boldsymbol{w})$. Несколько примеров изображены на рис. 4.19:

- для гребневой регрессии $R(\boldsymbol{w}) = \alpha\|\boldsymbol{w}\|_2^2$, и область $R(\boldsymbol{w}) \leq \beta$ представляет собой просто шар радиуса β (рис. 4.19, *а*);
- для лассо-регрессии $R(\boldsymbol{w}) = \alpha\|\boldsymbol{w}\|_1 = \alpha\sum_{j=1}^{d}|w_j|$, и интересующая нас область $R(\boldsymbol{w}) \leq \beta$ представляет собой многогранник, симплекс, отражённый

[1] Эта фраза, как часто бывает, скрывает довольно важные детали. Так, например, базовый результат о лагранжевой двойственности требует, чтобы все эти функции были непрерывно дифференцируемы в точке минимума, что, конечно, выполняется для гребневой регрессии, но уже для лассо-регрессии не гарантировано. Впрочем, все эти технические сложности можно преодолеть, и для целей этой книги они не слишком интересны, так что предлагаю поверить слову джентльмена.

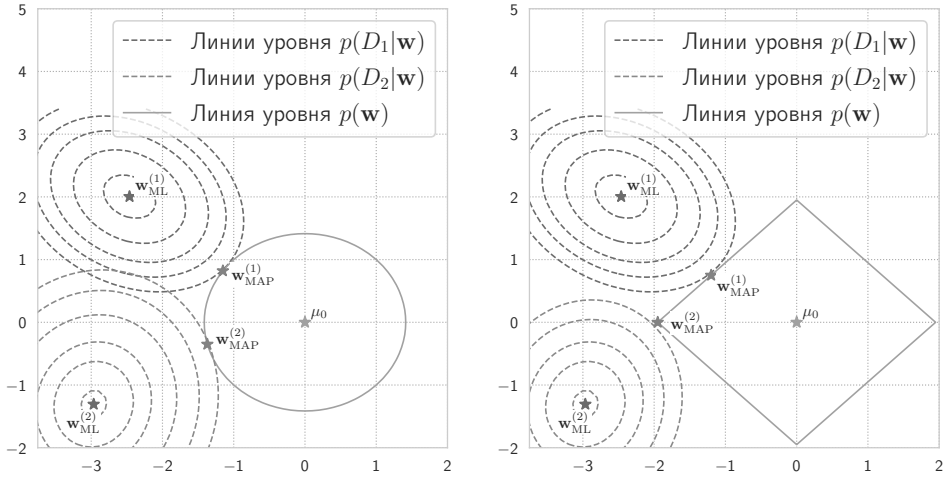


Рис. 4.19. Линии уровня регуляризаторов и соответствующая условная оптимизация: (а) гребневая регрессия, $R(\boldsymbol{w}) = \alpha\|\boldsymbol{w}\|_2^2$; (б) лассо-регрессия, $R(\boldsymbol{w}) = \alpha\|\boldsymbol{w}\|_1$

от всех осей; в размерности 2 это получается квадрат[1] с углами в точках $\pm\beta$ по обеим осям (рис. 4.19, *б*).

Если глобальный минимум попадает внутрь этой области, то ответ не меняется, а если лежит вне её, то нам придётся увеличивать целевую функцию до тех пор, пока мы не пересечёмся с границей области, то есть до первой линии уровня суммы квадратов отклонений (эти линии уровня, естественно, представляют собой эллипсы), которая будет касаться границы области $R(\boldsymbol{w}) = \beta$. Это тоже проиллюстрировано на рис. 4.19.

И теперь мы можем понять, откуда берётся обнуляющий эффект лассо-регрессии: граница области $\alpha\|\boldsymbol{w}\|_1 = \beta$ не симметрична со всех сторон, как в случае гребневой регрессии, а имеет углы, вершины. Вершина симплекса — это точка, имеющая непропорционально большую вероятность оказаться условным оптимумом: линии уровня, начинающиеся из целого большого куска пространства, первыми коснутся симплекса именно в вершине. Ещё больше вероятность того,

[1] Любопытное математико-психологическое замечание: я регулярно спрашиваю студентов на лекциях, какая фигура получится в размерности 2 для лассо-регрессии, и всегда, буквально всегда, получаю ответ «ромб», хотя на самом деле очевидно, что это не просто ромб, а именно квадрат. Это, по всей видимости, связано с тем, как мы воспринимаем окружающий мир: чтобы назвать эту фигуру квадратом, нам приходится мысленно её повернуть на 45°, то есть совершить некое нетривиальное усилие. Это один из аргументов в пользу того, что у нас в голове есть некая «система координат по умолчанию», с вертикальной и горизонтальной осями.

что они коснутся симплекса или в вершине, или на ребре, то есть на грани размерности 0 или 1 и так далее. А грани малой размерности — это именно те векторы весов $\boldsymbol{w}$, в которых много нулей! Например, вершина многогранника, заданного неравенством $\alpha\|\boldsymbol{w}\|_1 \leq \beta$, — это точка, в которой обнуляются все веса, кроме одного, ведь она лежит на одной из осей координат.

Отсюда и получается эффект разреженности: лассо-регрессия начнёт не просто равномерно уменьшать веса, как гребневая, а сначала строго обнулит некоторые из них. Как следствие, лассо-регрессия используется также как метод *выбора переменных* (feature subset selection), то есть как способ понять, какие из факторов в регрессии действительно важны, оказывают влияние на переменную отклика, а какие можно опустить [246, 349].

На этом месте можно задаться вопросом: мы уже поняли, что регуляризатор $R(\boldsymbol{w})$ может быть L_2-нормой вектора $\boldsymbol{w}$, а может быть L_1-нормой, и оба варианта имеют свой собственный смысл и полезны в народном хозяйстве. Что если мы попробуем найти нечто среднее между ними или, наоборот, попытаемся усилить эффект лассо-регрессии? Это можно сделать как минимум двумя разными способами.

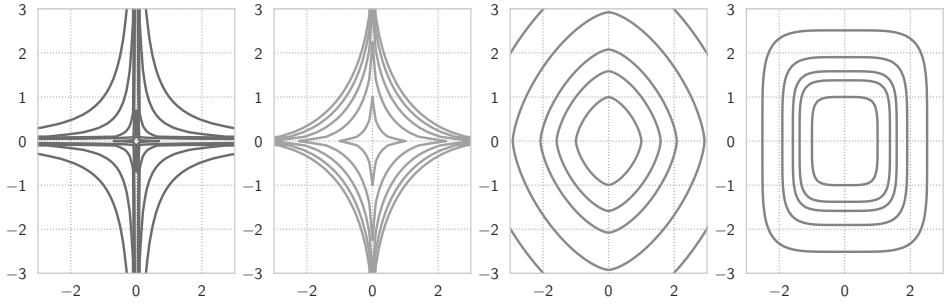
Первый способ — бридж-регрессия (bridge regression), предложенная Илдико Фрэнком и Джеромом Фридманом (с последним мы ещё встретимся в последующих книгах, ведь он был одним из создателей градиентного бустинга) в 1993 году [196]. Происхождение названия *bridge regression* (а значит, и его перевод на русский язык), честно говоря, осталось для меня загадкой: в исходной работе 1993 года [196] слово *bridge* не упоминается буквально ни разу, но все последующие источники, например Тибширани в работе 1996 года [594], ничтоже сумняшеся пишут что-нибудь вроде «Фрэнк и Фридман предложили класс методов регуляризации под названием bridge»... Могу только предположить, что к мостам это слово имеет мало отношения, а скорее добавляет что-то к слову *ridge*, поэтому по-русски пусть будет «бридж».

По сути, бридж-регрессия — это обобщение гребневой и лассо-регрессий на случай произвольной L_q-нормы; в бридж-регрессии мы минимизируем

$$\sum_{n=1}^{N}(y_n - \boldsymbol{w}^\top \boldsymbol{x}_n)^2 + \alpha\,\|\boldsymbol{w}\|_q = \sum_{n=1}^{N}(y_n - \boldsymbol{w}^\top \boldsymbol{x}_n)^2 + \alpha\sum_{j=1}^{d}|w_j|^q.$$

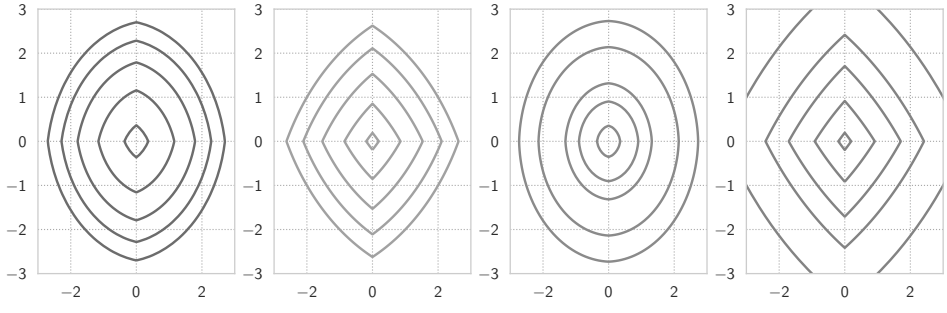
Варьируя q, мы можем получать разные эффекты: значения $1 < q < 2$ будут давать нечто промежуточное между гребневой и лассо-регрессией, а при $q < 1$ эффект разреженности будет усиливаться; это проиллюстрировано на рис. 4.20. С другой стороны, при $q > 2$ линии уровня регуляризатора будут «раздуваться» и при $q \to \infty$ стремиться к квадрату $\max w_j \leq \beta$.

Второй способ объединить лассо и гребневую регрессию — это не пытаться найти математически изящное недостающее звено между ними, а просто добавить в целевую функцию и то и другое; давайте минимизировать



(а) $q = 0{,}1$ (б) $q = 0{,}5$ (в) $q = 1{,}5$ (г) $q = 5$

Рис. 4.20. Линии уровни регуляризатора в бридж-регрессии для разных q: (а) $q = 0{,}1$; (б) $q = 0{,}5$; (в) $q = 1{,}5$; (г) $q = 5$



(а) $\alpha_1 = 2, \alpha_2 = 2$ (б) $\alpha_1 = 5, \alpha_2 = 1$ (в) $\alpha_1 = 1, \alpha_2 = 5$ (г) $\alpha_1 = 5, \alpha_2 = \frac{1}{2}$

Рис. 4.21. Примеры линий уровня elastic net регрессии для разных α_1 и α_2: (а) $\alpha_1 = 2, \alpha_2 = 2$; (б) $\alpha_1 = 5, \alpha_2 = 1$; (в) $\alpha_1 = 1, \alpha_2 = 5$; (г) $\alpha_1 = 5, \alpha_2 = \frac{1}{2}$

$$\sum_{n=1}^{N}(y_n - \boldsymbol{w}^\top \boldsymbol{x}_n)^2 + \alpha_1 \sum_{j=1}^{d}|w_j| + \alpha_2 \sum_{j=1}^{d} w_j^2.$$

Такой вариант регрессии в 2005 году предложили Чжоу и Хасти [668], назвав его elastic net[1], и здесь тоже можно добиваться разных эффектов, варьируя α_1 и α_2. Линии уровня таких регуляризаторов проиллюстрированы на рис. 4.21.

Подводя итог разговора об оверфиттинге, хочется снова вернуться к более общему взгляду на происходящее. В самом начале книги, в подразделе 1.4.1, мы говорили о том, что модели машинного обучения обычно устроены так, как было показано на рис. 1.12: сначала из входа $\boldsymbol{x}$ извлекают признаки $\varphi(\boldsymbol{x})$, а затем на

[1] «Эластичная сеть», хотя по-русски практически никто так не говорит; это опять, скорее всего, обыгрыш слова «лассо».

этих признаках обучается относительно простая модель машинного обучения. Линейная регрессия — типичный пример такой «простой модели», и обратите внимание, что весь оверфиттинг, о котором мы говорим в этом разделе, сосредоточен не в самом обучении линейной регрессии, а именно на этапе извлечения признаков. Это общая ситуация: когда оверфиттинг случается в больших нейронных сетях с миллионами параметров, он тоже происходит вовсе не на последнем слое, который обычно реализует линейную или логистическую регрессию в зависимости от природы целевой переменной.

В этом разделе я приводил пример оверфиттинга, связанного с тем, что мы извлекли слишком много признаков: довели зачем-то до десяти степень многочлена, описывающего 11 точек. Зачем тогда вся эта наука, ведь можно просто не делать таких очевидных глупостей? К сожалению, это не всегда возможно.

Когда вход одномерный, мы, конечно, вольны не выбирать слишком высокую степень многочлена. Но представьте, что вход $\boldsymbol{x}$ имеет размерность 200. Мы обучаем обычную линейную регрессию... но по итогам её тестирования становится очевидным, что этого недостаточно: линейная зависимость для нашей задачи слишком проста, и нужно повышать степень извлекаемых признаков. Но как только мы сделаем даже один следующий шаг — извлечём квадратичные признаки — из 200 переменных сразу же получатся $\frac{200 \cdot 201}{2} = 20100$ разных мономов! А если искать кубический многочлен, то мономов будет уже $\binom{202}{3} = 1\,353\,400$...

Без дополнительной информации (которой часто нет) мы не можем оставить только часть из этих мономов: да, скорее всего, важными окажутся только некоторые взаимодействия (interactions) между переменными, но мы не знаем заранее, какие именно. Так что для входа достаточно высокой размерности оверфиттинг может получиться и в степени 2, и все разработанные нами методы оказываются очень даже нужны. Более того, если подняться на ещё более общий концептуальный уровень, признаки оверфиттинга можно увидеть и без всяких повышающих размерность формальных преобразований. В разделе 2.6 мы обсуждали, как p-хакинг и даже вполне добросовестные методы анализа данных часто приводят к невоспроизводимым результатам. Это тоже оверфиттинг, только происходит он в голове исследователя: например, учёный видит данные и выбирает «подходящие» для этих данных гипотезу и метод статистического анализа; в подразделе 2.6.3 мы видели, что в результате может получиться всё что угодно, даже экстрасенсорное предвидение. Так что оверфиттинг — неизбежный спутник анализа данных, и мы ещё не раз поговорим и о методах регуляризации, и о методах байесовского вывода, которые во многом тоже призваны избавить модели от оверфиттинга, и о теоретических свойствах этого явления.

Но пока всё, что мы сказали о линейной регрессии, звучит как раздел теории оптимизации. Где же вероятности, о которых мы так много говорили раньше? Конечно же, и саму линейную регрессию, и рассмотренные нами здесь методы регуляризации можно исследовать с вероятностной стороны, и это выведет нас на новый уровень понимания происходящего. Давайте к этому и перейдём.

4.3. Вероятностная интерпретация линейной регрессии

Сообразно позитивной тенденции деструкции ближайшим образом подлежит постановке вопрос, была ли в течение истории онтологии вообще интерпретация бытия и могла ли быть — и в какой мере — тематически приведена в связь с феноменом времени, а необходимая для этого проблематика темпоральности — принципиально разработана.

Мартин Хайдеггер. Бытие и время (пер. В. В. Бибихина)

Речь Хайдеггера самообъяснительна, в ней достаточно света для прояснения себя самой и того, о чём она. Не случайно он не хотел к своим изданиям указателей и комментариев. Но, как всякая речь, эта тоже требует вслушивания, тем более что она не продолжение привычного дискурса. На первых страницах привыкающего чтения не нужно стремиться сразу к полному пониманию, достаточно одной увлечённости, без которой дело всё равно никуда не пойдёт.

Владимир Бибихин. Примечания переводчика к трактату «Бытие и время»

4.3.1. Правдоподобие линейной регрессии и гауссовский шум

Ответ на вопрос о происхождении метода наименьших квадратов был ясен уже Гауссу и Лежандру. Чтобы его получить, давайте (наконец-то!) поговорим о линейной регрессии на вероятностном языке. Линейная регрессия — это пример так называемой *дискриминирующей модели* (discriminative model), которая моделирует не совместное распределение $p(\boldsymbol{x}, y)$ (как делают *порождающие* модели), а только условное распределение $p(y \mid \boldsymbol{x})$. Так что мы будем предполагать, что точка $\boldsymbol{x}$ для предсказания и точки данных $\mathcal{X}$ фиксированы и всегда находятся «справа от черты» в условной вероятности.

Дальше нужно сделать главное: зафиксировать вероятностные предположения. В линейной регрессии мы предполагаем, что зависимая переменная y действительно порождается как линейная функция от вектора признаков $\boldsymbol{x}$, но она,

конечно, не строго равна $\boldsymbol{w}^\top \boldsymbol{x}$, а является случайной переменной, распределённой вокруг этого значения. Иначе говоря, мы моделируем y как

$$y \sim \boldsymbol{w}^\top \boldsymbol{x} + \epsilon,$$

где ϵ — это случайная величина, соответствующая шуму в данных, отклонению получаемых данных от «теоретического» значения. Вся наша вероятностная модель пока что сводится к тому, чтобы выбрать распределение для величины ϵ. Какое же распределение шума выбрать для численной целевой переменной?

Основное предположение, которое делает линейная регрессия, да и большинство других моделей с непрерывными целевыми переменными, состоит в том, что шум (ошибка в данных) распределён *нормально*, то есть переменная t, которую мы наблюдаем, получается как

$$t = y(\boldsymbol{x},\boldsymbol{w}) + \epsilon, \quad \epsilon \sim \mathcal{N}(0,\sigma^2).$$

Иными словами,

$$p\left(t\middle|\boldsymbol{x},\boldsymbol{w},\sigma^2\right) = \mathcal{N}\left(t \mid y(\boldsymbol{x},\boldsymbol{w}),\sigma^2\right).$$

Откуда берётся это предположение? Главным образом из *центральной предельной теоремы*, одного из основных результатов теории вероятностей; мы обсуждали его в подразделе 3.2.2. Напомню, что по этой теореме для любой последовательности независимых одинаково распределённых случайных величин X_1, $X_2, \ldots$ с конечным ожиданием μ и дисперсией σ^2 их сумма $S_n = \sum_{i=1}^n X_i$ сходится к нормальному распределению со средним $n\mu$ и дисперсией $n\sigma^2$:

$$\frac{S_n - \mu n}{\sigma\sqrt{n}} \to_{n\to\infty} \mathcal{N}(0,1).$$

Именно благодаря этому результату нормальное распределение, которое ещё называют гауссовским[1], играет такую важную роль в теории вероятностей, да

[1] *Иоганн Карл Фридрих Гаусс* (Johann Carl Friedrich Gauß, 1777–1855) — «король математиков». За свою долгую жизнь Гаусс развил или основал массу разделов математики, начиная от классической геометрии — карьера Гаусса началась с работ по построению правильных многоугольников циркулем и линейкой — и продолжая теорией чисел, где Гаусс доказал закон взаимности квадратичных вычетов и изучил их свойства, анализом, где Гаусс открыл много свойств эллиптических функций, алгеброй — Гаусс впервые доказал основную теорему алгебры, — современной геометрией, где Гаусс фактически основал дифференциальную геометрию, и так далее и тому подобное. Гаусс мало публиковал свои результаты, стараясь довести их до совершенства, и многое им открытое так и осталось неопубликованным; в черновиках Гаусса можно найти и неевклидову геометрию, и даже кватернионы. Более того, Гаусс занимался ещё и аналитической механикой, и оптикой, и астрономией, где с видимой лёгкостью проводил сложнейшие вычисления; например, он предсказал положение «потерянной» астрономами Цереры. Гаусс вообще прекрасно считал в уме, хотя известная история про сумму чисел от 1 до 100 — дело очень тёмное [247].

и в жизни появляется постоянно. Если взять много случайных величин — например, случайных воздействий на некоторый физический процесс, — каждая из которых может иметь абсолютно любое распределение, в сумме или, что то же самое, в среднем всё неизбежно сойдётся к нормальному распределению. Отсюда и происходит популярность предположения о нормально распределённом шуме, ведь в физических процессах шум часто получается как огромная сумма маленьких случайных воздействий.

Теперь давайте вернёмся к вычислениям. Рассмотрим набор данных $\boldsymbol{X} = \{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N\}$ с правильными ответами $\boldsymbol{t} = \{t_1, \ldots, t_N\}$. Будем предполагать, что данные взяты независимо по одному и тому же распределению с гауссовским шумом; тогда правдоподобие всего датасета выглядит так:

$$p\left(\boldsymbol{t}\middle|\boldsymbol{X},\boldsymbol{w},\sigma^2\right) = \prod_{n=1}^{N} \mathcal{N}\left(t_n \mid \boldsymbol{w}^\top \boldsymbol{\phi}(\boldsymbol{x}_n),\sigma^2\right).$$

Прологарифмируем это правдоподобие (здесь и далее я буду опускать зависимость от $\boldsymbol{X}$, потому что в линейной регрессии координаты точек данных всегда стоят справа от черты и их мы не моделируем):

$$\ln p\left(\boldsymbol{t}\middle|\boldsymbol{w},\sigma^2\right) = -\frac{N}{2}\ln(2\pi\sigma^2) - \frac{1}{2\sigma^2}\sum_{n=1}^{N}\left(t_n - \boldsymbol{w}^\top \boldsymbol{x}_n\right)^2. \tag{4.17}$$

В наших предположениях $\frac{N}{2}\ln(2\pi\sigma^2)$ представляет собой константу, и дисперсия шума σ^2 тоже оказывается неважна — её можно вынести за скобку. И вот мы получили, что для максимизации правдоподобия (4.17) по $\boldsymbol{w}$ нам нужно как раз минимизировать среднеквадратичную ошибку! Теперь можно опять взять градиент, приравнять его к нулю и получить ровно то же самое, что в предыдущем параграфе.

А после этого можно уже и относительно σ^2 максимизировать то же самое правдоподобие (4.17). Для этого давайте подставим $\boldsymbol{w} = \boldsymbol{w}_{\text{ML}}$, продифференцируем (4.17) по σ^2 (да-да, прямо вместе с квадратом, это полезный трюк: с точки зрения максимизации от этого ничего не изменится, а формулы станут проще) и приравняем к нулю:

$$-\frac{N}{2\sigma^2} + \frac{1}{2\sigma^4}\sum_{n=1}^{N}\left(t_n - \boldsymbol{w}_{\text{ML}}^\top \boldsymbol{x}_n\right)^2 = 0,$$

то есть

$$\sigma_{ML}^2 = \frac{1}{N}\sum_{n=1}^{N}\left(t_n - \boldsymbol{w}_{ML}^\top \boldsymbol{x}_n\right)^2.$$

И снова, казалось бы, ничего нового мы не нашли: получилась как раз выборочная дисперсия имеющихся данных вокруг предсказанного значения.

4.3.2. Все вероятностные предположения линейной регрессии

Но если нет разницы, зачем же платить больше и заниматься всей этой вероятностной обвязкой? Во-первых, в машинном обучении и статистике всегда важно понимать, какие именно предположения используются, ведь иначе мы не сможем заметить, не нарушаются ли они в каждом конкретном применении модели. Давайте кратко подытожим, какие вероятностные предположения мы использовали в рассуждении выше:

1) прежде всего мы предположили, что *точки данных условно независимы* при условии известных параметров модели: $p\left(D|\boldsymbol{w}\right) = \prod_{d\in D} p\left(d|\boldsymbol{w}\right)$;
2) затем мы предположили, что *шум в данных гауссовский*, то есть правдоподобие одной точки записывается как $p\left(t|\boldsymbol{w},\boldsymbol{x}\right) = \mathcal{N}\left(t \mid \boldsymbol{w}^\top\boldsymbol{x}, \sigma^2\right)$;
3) а в последнем переходе уже непосредственно к методу наименьших квадратов мы вынесли за скобку $\frac{1}{2\sigma^2}$ в логарифме правдоподобия; иначе говоря, мы предположили, что *дисперсия шума в данных постоянна* и не зависит от точки $\boldsymbol{x}$.

Немало! Давайте теперь обсудим, что у нас получилось. Первый пункт — это важное предположение, которое мы в машинном обучении будем делать практически всегда, ибо без него совсем ничего не получится. Если вдруг оказывается, что это предположение неверно, значит, надо изменить модель так, чтобы оно выполнилось; скорее всего, менять придётся определение того, что такое «одна точка данных». Так, например, если на входе у модели текст, то есть последовательность токенов (слов или других единиц), то одной точкой данных, скорее всего, придётся считать не одно слово, а целый текст или его достаточно содержательную часть, ведь если считать слова независимыми друг от друга, анализ текста тривиализуется, и ничего разумного сделать не получится[1].

Второй пункт мы уже мотивировали выше, но теперь можем задуматься о том, всегда ли разумно предполагать, что шум в данных гауссовский. Краткий ответ — нет, не всегда! В частности, у гауссовского шума есть два свойства, которые могут оказаться слишком ограничивающими.

Во-первых, гауссовский шум *симметричен*, но иногда могут возникнуть ситуации, когда мы заранее знаем, что шум симметричным быть не должен. Классический пример здесь представляют финансовые модели, в которых часто встречаются несимметричные распределения; например, несимметрична доходность большинства финансовых активов [12], а простейшая модель для построения портфеля из активов — это как раз линейная регрессия.

[1] Даже наивный байесовский классификатор, который мы рассмотрим в разделе 5.6, всё-таки будет считать слова независимыми только при условии метки класса.

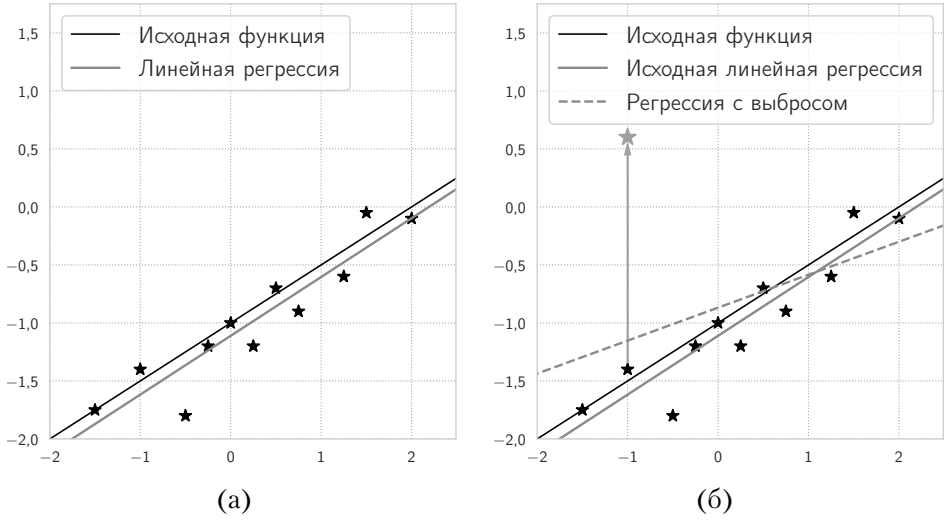


Рис. 4.22. Влияние выбросов на линейную регрессию с гауссовским шумом: (а) исходная регрессия; (б) регрессия с выбросом

Во-вторых, нормальное распределение *убывает экспоненциально быстро*, что зачастую оказывается слишком быстрым убыванием в практических ситуациях. Говорят, что у нормального распределения *лёгкие хвосты* (light-tailed distribution); это значит, что из нормального распределения очень сложно получить точки, отстоящие от среднего на много значений дисперсии. В статистике широко известны численные значения процентилей нормального распределения: у распределения $\mathcal{N}(x|\mu,\sigma)$

- 68 % значений будут лежать в пределах одной дисперсии от среднего:

$$p(\mu - \sigma \leq x \leq \mu + \sigma) \approx 0{,}6827;$$

- 95,5 % значений будут лежать в пределах двух дисперсий от среднего:

$$p(\mu - 2\sigma \leq x \leq \mu + 2\sigma) \approx 0{,}9545;$$

- 99,7 % значений будут лежать в пределах трёх дисперсий от среднего:

$$p(\mu - 3\sigma \leq x \leq \mu + 3\sigma) \approx 0{,}9973.$$

Именно из этих процентилей происходит и стандартное p-значение в 95 %, значение которого для социальных наук мы обсуждали в разделе 2.6: у нормального распределения ошибка 5 % соответствует попаданию в интервал двух дисперсий от среднего. А ещё это приводит к так называемому правилу трёх сигм

(three sigma rule): на практике обычно можно округлять 0,9973 до единицы и считать, что все значения будут лежать в пределах трёх стандартных отклонений.

Что это значит для обучения? Оказывается, что на обучение параметров гауссиана (полное его рассмотрение последует в разделе 6.4, здесь пока обсудим «на пальцах») сильное влияние оказывают *выбросы* (outliers), то есть точки данных, которые отстоят от «идеального» значения гораздо дальше остальных. Даже если десять точек данных лежат недалеко от среднего и только одна отстоит на большое расстояние, обученное на близких друг к другу точках нормальное распределение будет присваивать слишком низкое правдоподобие для единственного выброса и при обучении параметров ему придётся существенно сдвинуть среднее и/или увеличить дисперсию, чтобы попытаться этот эффект скомпенсировать.

Тот же эффект выбросов переносится и на линейную регрессию с гауссовским шумом: каждый выброс будет оказывать большое влияние на то, как пройдёт обученная прямая. На рис. 4.22 показан пример такого влияния: рис. 4.22, *а* показывает линейную регрессию, обученную на некотором наборе точек, а на рис. 4.22, *б* показана регрессия, обученная на том же наборе с одной сдвинутой вверх точкой; как видите, прямая развернулась к выбросу очень существенно.

В реальной же жизни часто встречаются ситуации, где такой выброс лучше если не вовсе проигнорировать, то уж точно не так сильно к нему адаптироваться. Например, в реальных экспериментальных данных необычно большие ошибки могут свидетельствовать о каких-то сбоях в сборе или передаче данных и могут быть вовсе не так редки, как предполагает нормально распределённый шум [482].

Более того, и без всяких сбоев многие распределения вероятностей в реальной жизни убывают отнюдь не экспоненциально, то есть являются распределениями с *тяжёлыми хвостами* (heavy-tailed distributions). В подразделе 3.1.5 вы видели классический пример этого: закон Ципфа и его обобщения, в которых вероятность убывает полиномиально, появляются и в статистике встречаемости слов, и в распределениях доходов, размеров городов и многого другого, причём в подразделе 3.1.5 мы даже обсудили нехитрый математический механизм того, откуда он всё время появляется.

В таких ситуациях могут помочь варианты линейной регрессии, в которой шум задаётся распределением с тяжёлыми хвостами, убывающим не экспоненциально. Для таких распределений правило трёх сигм уже не будет работать, хотя большая часть примеров всё равно должна укладываться в одну-две дисперсии от среднего, для этого, собственно, дисперсия и нужна. На этот счёт в теории вероятностей есть классический результат — *неравенство Чебышёва*[1]: для *любой* случайной величины X с конечным математическим ожиданием μ и конечной

[1] *Пафнутий Львович Чебышёв* (1821–1894) — русский математик и механик, основоположник петербургской математической школы, автор многих важных результатов в математическом анализе, геометрии, теории приближения функций и многом другом. Слава его началась с работ по теории чисел, но для нас важнее работы Чебышёва по теории вероятностей: он доказал неравенство, ограничивающее вероятность больших отклонений случайной величины от своего ожидания (сейчас этот

дисперсией σ^2 вероятность попасть в пределы k дисперсий от среднего ограничена так:

$$p\left(\mu - k\sigma \leq X \leq \mu + k\sigma\right) \geq 1 - \frac{1}{k^2}.$$

Для попадания в интервал в плюс-минус две дисперсии неравенство Чебышёва даёт вероятность $\frac{3}{4}$, а для плюс-минус трёх дисперсий — $\frac{8}{9}$. Значения, превышающие 95 %, появятся только для интервалов плюс-минус пять дисперсий и больше; это и есть разница между распределениями.

Подробное рассмотрение этих пунктов, особенно второго, давайте пока отложим: в подразделе «со звёздочкой» 4.3.4 мы рассмотрим так называемую *робастную* (robust) линейную регрессию с шумом из распределения Стьюдента, а в подразделе 4.3.5 поговорим совсем о другом способе сделать линейную регрессию робастной.

Третье предположение — то, что мы можем вынести за скобку $\frac{1}{\sigma^2}$, то есть то, что дисперсия шума везде в данных одна и та же. В статистике это предположение носит громкое название *гомоскедастичность* (homoscedasticity), и в статистике есть ряд тестов на гомоскедастичность, которые мы здесь подробно обсуждать не будем; в частности, для регрессионной модели, которую мы сейчас рассматриваем, предназначен тест Бройша — Пагана [77].

Отмечу только главное: если мы можем явно сформулировать вероятностное предположение, это сразу же позволяет нам распознавать ситуации, в которых оно точно нарушается, даже и без всяких вероятностных тестов. Давайте рассмотрим совсем игрушечный, но очень наглядный пример. Породим точки в данных с двумя разными дисперсиями шума:

```
orig = lambda x : .5*x - 1
xd = np.array([-3, -2.5, -2, -1, -0.5, 0, 1, 1.5, \
    1.75, 2.25, 2.5, 3, 4, 4.5]) / 2
num_points_1, num_points_2 = 6, 8

data = orig(xd)
data[:num_points_1] += np.random.normal(0, .1, num_points_1)
data[-num_points_2:] += np.random.normal(0, .5, num_points_2)
```

результат поглощён неравенством Маркова) и использовал его для очень общей формулировки закона больших чисел; а затем он обобщил и теорему Муавра — Лапласа, сформулировав центральную предельную теорему в почти современном виде. Работу Чебышёва продолжили его многочисленные ученики, в том числе А. А. Марков и А. М. Ляпунов; именно ученики Чебышёва в 1890 году основали Санкт-Петербургское математическое общество. Пафнутий Львович много работал в прикладной механике и синтезе механизмов, и писал о практических приложениях так: «Сближение теории с практикою даёт самые благотворные результаты, и не одна только практика от этого выигрывает: сами науки развиваются под влиянием её: она открывает им новые предметы для исследования или новые стороны в предметах давно известных... Если теория много выигрывает от новых приложений старой методы или от новых развитий её, то она ещё более приобретает открытием новых метод, и в этом случае науки находят себе верного руководителя в практике». Немного подвела наследие Чебышёва разве что сложная русская фамилия: когда Джон Дербишир попытался найти работы самого Чебышёва и о нём, он насчитал 32 (тридцать два!) варианта транскрипции фамилии Чебышёв.

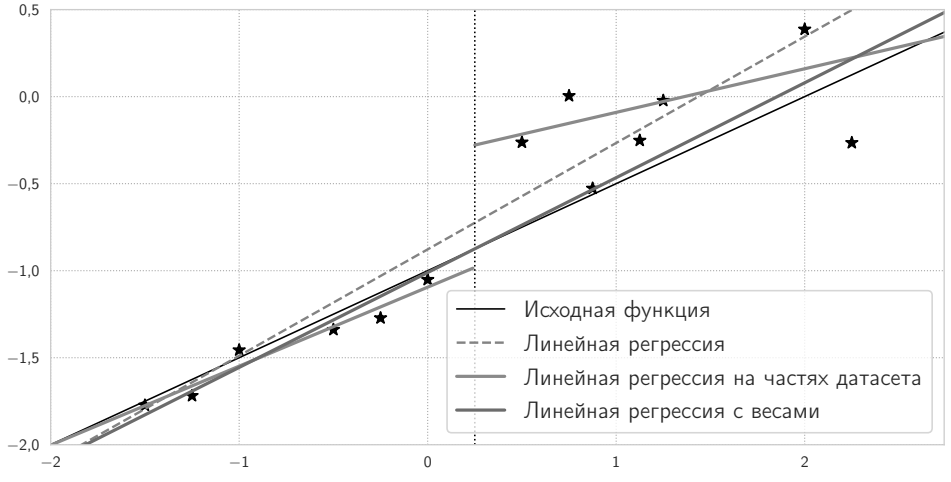


Рис. 4.23. Пример линейной регрессии с разными дисперсиями в разных частях датасета

Полученные точки и обученные регрессии изображены на рис. 4.23. Как видите, точки с координатами меньше $\frac{1}{4}$ в этом примере имеют маленький шум, с дисперсией $\sigma = 0{,}1$, а правее $\frac{1}{4}$ — большой, с дисперсией $\sigma = 0{,}5$. Все точки порождены вокруг одной и той же прямой $y = \frac{1}{2}x - 1$, но эта разница в дисперсиях приводит к тому, что обычная линейная регрессия, обученная на всех точках сразу (синяя прямая), слишком подвержена большому случайному шуму в правой части данных и как результат не очень похожа на правильный ответ (на чёрную прямую).

Даже если мы знаем, где проходит водораздел между двумя типами данных, с ходу неочевидно, как использовать это знание: можно построить две линейные регрессии слева и справа от $x = \frac{1}{4}$ (пунктирные прямые на рис. 4.23), но эти регрессии, конечно, будут иметь разный наклон и не будут давать одно и то же значение в точке $x = \frac{1}{4}$; как же их объединить?

Вероятностный взгляд на ситуацию сразу даёт ответ — благодаря тому, о чём мы говорили выше, мы понимаем, что изменится в методе наименьших квадратов, если дисперсии станут разными. Ведь метод наименьших квадратов происходит из максимизации правдоподобия нормально распределённого шума

$$\ln p\left(\boldsymbol{t}\middle|\boldsymbol{w},\sigma^2\right) = -\frac{1}{2}\sum_{n=1}^{N}\frac{1}{\sigma_n^2}\left(t_n - \boldsymbol{w}^\top \boldsymbol{x}_n\right)^2 + \text{Const},$$

и если σ_n, как в этом примере, не все одинаковые, это просто значит, что σ нельзя вынести за знак суммы и у ошибок в разных точках появятся разные веса.

В нашем примере разных весов будет два:

$$\ln p\left(\boldsymbol{t}\middle|\boldsymbol{w},\sigma^2\right) = \\ = -\frac{1}{2\sigma_1^2}\sum_{n:\boldsymbol{x}_n<\frac{1}{4}}\left(t_n-\boldsymbol{w}^\top\boldsymbol{x}_n\right)^2 - \frac{1}{2\sigma_2^2}\sum_{n:\boldsymbol{x}_n>\frac{1}{4}}\left(t_n-\boldsymbol{w}^\top\boldsymbol{x}_n\right)^2 + \text{Const}.$$

Более того, раз уж мы знаем дисперсии, мы знаем и эти самые веса. Значение имеет только соотношение между ними, $\sigma_2 = 5\sigma_1$, то есть вес ошибки в точках слева от черты $x = \frac{1}{4}$ должен быть в 25 раз больше веса ошибки в точках справа от неё. В пакете `sklearn` легко можно задать веса ошибок в разных точках:

```
modelw = linear_model.LinearRegression(fit_intercept=True).fit( \
    xd.reshape(-1, 1), data, sample_weight=[25]*num_points_1 + [1]*num_points_2 )
```

Полученная модель (красная прямая на рис. 4.23) гораздо больше похожа на правильный ответ, чем всё остальное. Конечно, это игрушечный пример, в котором мы предположили, что знаем всё на свете — и точку изменения данных $x = \frac{1}{4}$, и сами дисперсии слева и справа от неё, — но он хорошо показывает, как можно использовать вероятностное понимание происходящего для того, чтобы проводить небольшие обобщения того или иного метода машинного обучения на нестандартные случаи.

4.3.3. Интерпретация коэффициентов: корреляция и причинность

Теперь мы готовы к тому, чтобы обсудить один из ключевых и для практического применения, и для понимания сути происходящего вопросов: что, собственно, показывает нам линейная регрессия? О чём говорят веса линейной регрессии пытливому исследователю, который запустил её на только что собранных данных?

В идеале все эти модели пытаются ответить на вопрос о том, *как связаны* исследуемые переменные, как они *влияют* друг на друга. Если исследователь хочет оценить, насколько уровень образования людей влияет на их доход в той или иной стране, он может собрать данные о некоторой выборке людей из этой страны, а затем построить линейную регрессию:

$$\text{«Доход»} \sim w_1 \cdot \text{«Уровень образования»} + w_0.$$

В этой формуле я сразу же заметаю под ковёр очень многое; например, доход разумно измерять деньгами за единицу времени, но как превратить уровень образования в число? Возможно, здесь понадобится one-hot представление, о котором мы говорили выше, а возможно, исследователь придумает какое-то отображение уровня образования в число, ведь эти уровни в целом можно упорядочить (то есть это не категориальная переменная, а скорее *ординальная*).

Но о проблемах извлечения и представления признаков мы уже говорили, а сейчас речь о другом, так что предположим, что отображение нашлось.

Более того, представим, что и регрессия обучилась с коэффициентом w_1, который достаточно далёк от нуля, то есть какая-то зависимость между случайными величинами «Доход» и «Уровень образования», похоже, действительно есть. Предположим, что w_1 оказался положительным — значит ли это, что гипотезу о том, что уровень образования позитивно влияет на доход, можно считать доказанной и можно переходить к численной оценке силы этого эффекта?

Конечно же, нет. Обучив линейную регрессию, мы действительно нашли зависимость между случайными величинами. Более того, мы нашли *линейную* зависимость между этими величинами; если мы обучаем регрессию $y \sim w_1 x + w_0$ на одном численном признаке x, то фактически коэффициент w_1 оценивает уровень *корреляции* между величинами x и y.

Коэффициент корреляции Пирсона r на основе выборки значений случайных величин $D = \{(x_n, y_n)\}_{n=1}^N$ определяется как

$$r = \frac{\sum_{n=1}^N (x_n - \bar{x})(y_n - \bar{y})}{\sqrt{\sum_{n=1}^N (x_n - \bar{x})^2 \sum_{n=1}^N (y_n - \bar{y})^2}},$$

где $\bar{x} = \frac{1}{N}\sum_n x_n$, $\bar{y} = \frac{1}{N}\sum_n y_n$.

Корреляция показывает силу линейной зависимости между x и y; величина r не является напрямую оценкой коэффициента w_1 (как минимум коэффициент корреляции ограничен интервалом от -1 до 1), но связь между ними самая прямая и очевидная.

В методе наименьших квадратов с одним признаком $y \sim w_1 x + w_0$ мы минимизируем (здесь проще вывести заново, чем обращать матрицу 2×2, да и преобразования довольно характерные получаются, эти идеи нам ещё пригодятся)

$$\text{RSS}(w_0, w_1) = \sum_{n=1}^N (y_n - w_1 x_n - w_0)^2.$$

Если взять производную по w_0 и приравнять её к нулю, мы получим, что

$$\sum_{n=1}^N (y_n - w_1^* x_n - w_0^*) = 0,$$

то есть

$$w_0^* = \frac{1}{N}\sum_{n=1}^N (y_n - w_1^* x_n) = \bar{y} - w_1^* \bar{x}.$$

Теперь возьмём производную по w_1, приравняем к нулю и подставим туда этот результат, а затем проведём пару нехитрых преобразований:

$$0 = \sum_{n=1}^{N} \left(y_n - w_1^* x_n - w_0^*\right) x_n = \sum_{n=1}^{N} \left(y_n - w_1^* x_n - (\bar{y} - w_1^* \bar{x})\right) x_n$$
$$= \sum_{n=1}^{N} x_n \left(y_n - \bar{y}\right) - w_1^* \sum_{n=1}^{N} x_n \left(x_n - \bar{x}\right)$$
$$= \sum_{n=1}^{N} \left(x_n - \bar{x} + \bar{x}\right) \left(y_n - \bar{y}\right) - w_1^* \sum_{n=1}^{N} \left(x_n - \bar{x} + \bar{x}\right) \left(x_n - \bar{x}\right) =$$
$$= \sum_{n=1}^{N} \left(x_n - \bar{x}\right) \left(y_n - \bar{y}\right) - w_1^* \sum_{n=1}^{N} \left(x_n - \bar{x}\right) \left(x_n - \bar{x}\right),$$

потому что

$$\sum\nolimits_{n=1}^{N} \left(x_n - \bar{x}\right) = 0 \qquad \text{и} \qquad \sum\nolimits_{n=1}^{N} \left(y_n - \bar{y}\right) = 0.$$

В итоге получается

$$w_1^* = \frac{\sum_{n=1}^{N} \left(x_n - \bar{x}\right) \left(y_n - \bar{y}\right)}{\sum_{n=1}^{N} \left(x_n - \bar{x}\right)^2}.$$

Если мы обозначим через

$$\hat{\sigma}_x = \sqrt{\sum\nolimits_{n=1}^{N} \left(x_n - \bar{x}\right)^2} \quad \text{и} \quad \hat{\sigma}_y = \sqrt{\sum\nolimits_{n=1}^{N} \left(y_n - \bar{y}\right)^2}$$

выборочные оценки дисперсий для x и y соответственно, мы увидим, что

$$w_1^* = \frac{\sum_{n=1}^{N} \left(x_n - \bar{x}\right) \left(y_n - \bar{y}\right)}{\sigma_x^2} \quad \text{и} \quad r = \frac{\sum_{n=1}^{N} \left(x_n - \bar{x}\right) \left(y_n - \bar{y}\right)}{\sigma_x \sigma_y},$$

то есть

$$w_1^* = r \frac{\sigma_x}{\sigma_y}.$$

Иначе говоря, коэффициент линейной регрессии, обученной методом наименьших квадратов, — это в точности коэффициент корреляции, перемасштабированный согласно выборочным дисперсиям x и y.

И коэффициент корреляции, и линейная регрессия призваны оценивать силу линейной зависимости между переменными. Но здесь есть сразу несколько подводных камней, которые мешают нашему воображаемому исследователю просто взять и заключить, что людям, желающим больше зарабатывать, нужно срочно поступать в аспирантуру. На самом деле *корреляция не означает причинности*; по-английски эта фраза звучит особенно хорошо: correlation does not imply

causation, так что она стала крылатой и заслужила даже отдельную статью в Википедии [643]. Почему так?

Во-первых, даже если корреляция сигнализирует о причинно-следственной связи, она ничего не говорит о том, в каком направлении эта связь. Тривиальный пример здесь звучит примерно так: чем быстрее вращаются лопасти мельниц, тем более сильный ветер мы наблюдаем, следовательно, мельницы вызывают ветер. Здесь нам непосредственно очевидно, что рассуждение неверно; столь же очевидной будет эта ошибка, если выяснится, например, что «причина» на самом деле произошла после своего «следствия».

Но заметить такую ошибку становится гораздо сложнее, когда механизм взаимодействия между переменными неизвестен. Обратите внимание, что в науке обычно именно так и происходит, ведь если бы мы хорошо понимали механизм, нам не нужно было бы строить регрессионные модели и определять причинность через данные. Так, в Средние века многие европейцы верили, что вши полезны для здоровья, потому что на больных людях вшей практически не находили [645]. Впоследствии выяснилось, что причинно-следственная связь направлена в другую сторону: вши очень чувствительны к температуре тела, поэтому сразу покидают человека, у которого температура поднялась. Кстати, с порядком на оси времени у средневековых европейцев здесь ошибки как раз не было: в Средние века термометры ещё не изобрели и поднявшуюся температуру трудно было заметить сразу, а другие симптомы появляются обычно позже, когда вши уже сбежали с «корабля».

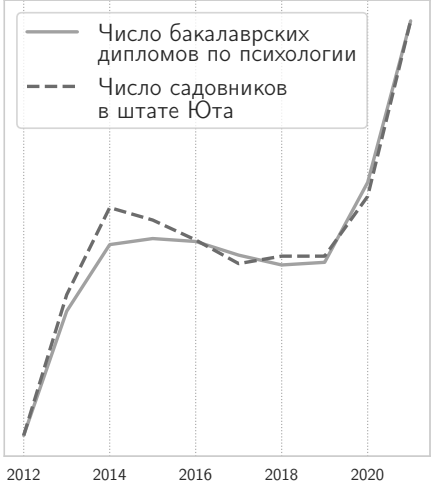
Во-вторых, причинно-следственного направления может и вовсе не быть. Вы, возможно, видели яркие примеры ложных корреляций (spurious correlations), найденных поиском по большой базе временных рядов; недавно даже появился сайт *Spurious Scholar*, посвящённый таким «исследованиям» [624]; несколько примеров найденных там корреляций приведены на рис. 4.24. Если порыться в достаточно большой базе, совпадения можно найти очень точные, причём не только вида «обе переменные растут с одинаковой скоростью», как на рис. 4.24, *б*, но и повторяющие более сложные формы графиков, как на рис. 4.24, *а*, *в*.

Но даже чаще наличие корреляции в исследовании оказывается не вовсе случайным, а указывающим на то, что у двух скоррелированных событий есть общая причина. Так, один из классических примеров ложной корреляции — связь между потреблением мороженого и числом инцидентов на воде (например, числом утонувших людей). Разумеется, мороженое не ухудшает способности к плаванию, просто и потребление мороженого, и число купающихся людей увеличиваются в тёплое время года и особенно в жаркую погоду.

Такую ошибку не всегда легко заметить. Один знаменитый пример ложной причинности появился в 1999 году в журнале *Nature* [421]; исследователи обнаружили, что у маленьких детей, которые спали при свете (не обязательно ярком, обычно просто с ночником), позже гораздо чаще развивается близорукость. Результат звучит просто и логично, и исследование привлекло внимание прессы;

(а) Маргарин вреден для брака?

(б) Пришельцы приносят идеи?



(в) Психологи не нужны?

(г) А здесь я сдаюсь

Рис. 4.24. Примеры ложных корреляций [624]

многие семьи приняли его к сведению и стали следить за тем, чтобы их дети спали в темноте. Однако последующие исследования не подтвердили зависимость, и стал ясен механизм общей причины: близорукость — наследственная болезнь (это было известно давно), а близорукие родители чаще включают свет для своего собственного удобства, когда укладывают детей спать [626, 627].

Другой известный пример — исследования о заместительной гормональной терапии. В 1991 году вышла ставшая широко известной статья, в которой устанавливалось, что такая терапия снижает у женщин риск ишемической болезни сердца [556]. Однако последующие более масштабные исследования [192, 487] показали, что такой связи на самом деле нет, а обнаруженная корреляция, скорее всего, происходит оттого, что заместительной гормональной терапией в основном пользуются женщины из более высоких слоёв общества, которые и питаются лучше, и в целом ведут более здоровый образ жизни.

Здесь важно отметить, что исходное исследование 1991 года было *наблюдательным* (observational study), то есть эффект был замечен в пусть контролируемой, но уже существующей выборке женщин, а более поздние представляли собой полноценные *рандомизированные контролируемые исследования* (randomized controlled trials, RCT). Авторы статьи 2004 года, описывающей эту историю, даже задались вопросом, не приведёт ли это к смерти наблюдательной эпидемиологии [334]. И действительно, рандомизированные контролируемые исследования, безусловно, предпочтительнее — в них можно проконтролировать куда больше различных переменных, которые могли бы «загрязнить» результаты. Впрочем, о смерти наблюдательной медицины тоже говорить рано, ведь не всегда такое исследование можно провести[1].

Принципом «корреляция не означает причинности» можно и злоупотребить. Так, например, уже хорошо нам знакомый сэр Рональд Фишер в 1950-е годы много раз выступал с публикациями, в которых опровергал появлявшиеся тогда результаты о связи курения с развитием онкологических заболеваний [186–188]. Фишер был консультантом табачных компаний, так что это выглядит как очевидное заказное исследование, но здесь я склонен доверять его биографам Фрэнку Йейтсу и Кеннету Матеру: «Это было бы несправедливо. Фишер не был выше того, чтобы получать деньги за свои услуги, но причинами его интереса были, безусловно, неприязнь и недоверие ко всякого рода пуританским тенденциям, а также, возможно, то удовольствие, которое он сам всегда находил в табаке» [651]. Завзятый курильщик Фишер считал пропаганду вреда курения пропагандой в плохом смысле слова[2] до самой смерти. На этом месте обычно добав-

[1] Так, например, я уже упоминал своё сотрудничество с компанией *Insilico Medicine* [1, 154, 405, 406], главная миссия которой — победить старение. Однако свои соединения компания выводит на рынок как БАДы, ведь провести рандомизированное контролируемое исследование для препарата, призванного замедлять старение, сложно: сколько оно должно продолжаться?.. По той же причине нельзя сказать, что у компании получилось работающее решение; сама *Insilico Medicine* не скрывает, что их препараты — это скорее пари Паскаля, чем лекарства, имеющие доказанный эффект.

[2] С этим словом в русском языке происходят любопытные изменения. Слово «пропаганда» изначально было нейтральным и означало попытку донести информацию до широкого круга людей или убедить в чём-то этот широкий круг; когда Владимир Ленин писал «О пропаганде и агитации», он никакой негативной окраски в это слово не вкладывал. Да и сегодня «пропаганда здорового образа жизни» звучит вполне нейтрально. Но, видимо, под влиянием английского языка, где слово *propaganda* давно уже окрашено негативно, «пропаганда» и в русском всё чаще используется в более узком значении: это всё ещё попытка убедить в чём-то широкий круг людей, но на этот раз с использованием

ляют «до смерти от рака», что звучит очень эффектно, но я бы как раз поостерёгся в данном случае говорить о каузальности: рак был колоректальный (его риск у курильщиков тоже повышен, но, конечно, не так сильно, как рака лёгких или гортани), да и дожил Фишер до 72 лет, а в этом возрасте вероятность онкологических заболеваний уже весьма заметна и без всякого курения.

Существует отдельная область вероятностного вывода, в которой исследователи пытаются установить не просто зависимости между случайными переменными, а именно причинно-следственные связи (каузальность, causality). Она тесно примыкает к графическим вероятностным моделям и выходит за пределы нашего рассмотрения в этой книге, так что я просто оставлю здесь несколько ссылок [254, 409, 458, 459, 462] и буду надеяться, что найдётся возможность вернуться к этой теме в последующих книгах.

*4.3.4. Вероятностная робастная регрессия: другие распределения шума

А теперь давайте вернёмся к заявленному выше рассмотрению робастной регрессии и поговорим о том, какие варианты линейной регрессии можно разработать, если использовать другие распределения случайного шума.

Самый употребительный такой альтернативный вариант — это линейная регрессия с шумом, задаваемым распределением Стьюдента, которое мы обсуждали в разделе 3.2. Для фиксированного числа степеней свободы ν можно считать, что плотность распределения шума (3.24) имеет вид

$$p(\epsilon|\nu) = \text{Const} \cdot \left(\nu + \epsilon^2\right)^{-\frac{\nu+1}{2}},$$

то есть логарифм плотности шума составляет

$$\log p(\epsilon|\nu) = \text{Const} - \frac{\nu+1}{2} \log\left(\nu + \epsilon^2\right).$$

Это значит, что логарифм правдоподобия для датасета $D = \{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N\}$, на котором мы обучили линейную регрессию с шумом $\epsilon = y - \boldsymbol{w}^\top \boldsymbol{x}_n$, равен соответственно

$$\log p(D|\boldsymbol{w}, \nu) = \text{Const} - \frac{\nu+1}{2} \sum\nolimits_{n=1}^{N} \log\left(\nu + \left(y - \boldsymbol{w}^\top \boldsymbol{x}_n\right)^2\right). \tag{4.18}$$

Следовательно, в линейной регрессии с шумом, распределённым по Стьюденту с ν степенями свободы, нужно максимизировать величину (4.18), то есть минимизировать вот такую функцию ошибки:

заведомо негодных средств — специального подбора примеров (cherry-picking), неверных умозаключений (вроде как раз превращения корреляции в причинность), эмоционально окрашенных терминов, откровенной лжи и так далее. Фишер, разумеется, высказывался по-английски.

$$\sum_{n=1}^{N} \log\left(\nu + \left(y - \boldsymbol{w}^\top \boldsymbol{x}_n\right)^2\right) \underset{\boldsymbol{w}}{\longrightarrow} \min. \tag{4.19}$$

По сути, мы по-прежнему используем квадраты отклонений, но пропускаем их через специальное преобразование $f(a) = \log(\nu + a)$. Это логарифмическое преобразование смягчает эффект от больших значений a, как нам и требовалось, а при росте числа степеней свободы на бесконечности сходится к стандартной линейной регрессии — при больших значениях ν

$$f(a) = \log(\nu + a) = \log\nu + \log\left(1 + \frac{a}{\nu}\right) \approx \log\nu + \frac{a}{\nu},$$

то есть для больших ν нужно минимизировать сумму квадратов отклонений.

Как решить задачу минимизации (4.19)? Аналитически не получится, сумма логарифмов дело непростое. Поэтому придётся прибегать к тем или иным численным методам.

В данном случае проще всего взять градиент от выражения (4.19) и приравнять его к нулю:

$$\nabla_{\boldsymbol{w}} \log p\left(D|\boldsymbol{w}\right) = \text{Const} \cdot \sum_{n=1}^{N} \frac{y - \boldsymbol{w}^\top \boldsymbol{x}_n}{\nu + \left(y - \boldsymbol{w}^\top \boldsymbol{x}_n\right)^2} \boldsymbol{x}_n = 0. \tag{4.20}$$

И эту систему уравнений теперь можно решать итеративно, например методом Ньютона, ведь матрицу вторых производных здесь тоже легко найти:

$$\begin{aligned}
\frac{\partial^2 \log p\left(D|\boldsymbol{w}\right)}{\partial w_i \partial w_j} &= \text{Const} \cdot \partial \left[\sum_{n=1}^{N} \frac{y - \boldsymbol{w}^\top \boldsymbol{x}_n}{\nu + \left(y - \boldsymbol{w}^\top \boldsymbol{x}_n\right)^2} x_{ni}\right] / \partial w_j \\
&= \text{Const} \cdot \sum_{n=1}^{N} \frac{-x_{nj}\left(\nu + \left(y - \boldsymbol{w}^\top \boldsymbol{x}_n\right)^2\right) + 2\left(y - \boldsymbol{w}^\top \boldsymbol{x}_n\right)^2 x_{nj}}{\left(\nu + \left(y - \boldsymbol{w}^\top \boldsymbol{x}_n\right)^2\right)^2} x_{ni} \\
&= -\text{Const} \cdot \sum_{n=1}^{N} \frac{\nu - \left(y - \boldsymbol{w}^\top \boldsymbol{x}_n\right)^2}{\left(\nu + \left(y - \boldsymbol{w}^\top \boldsymbol{x}_n\right)^2\right)^2} x_{ni} x_{nj}.
\end{aligned}$$

А мы для нашего простого численного примера не будем мучаться с реализацией итеративного алгоритма обучения, а просто воспользуемся готовой процедурой `scipy.optimize.fsolve`, чтобы решить уравнение (4.20).

Для этого нужно выразить его в виде функции; в коде ниже мы сначала считаем отклонения `rs` (от слова residuals), затем веса при $\boldsymbol{x}_n$ в (4.20), а потом вычисляем собственно левую часть равенства (4.20):

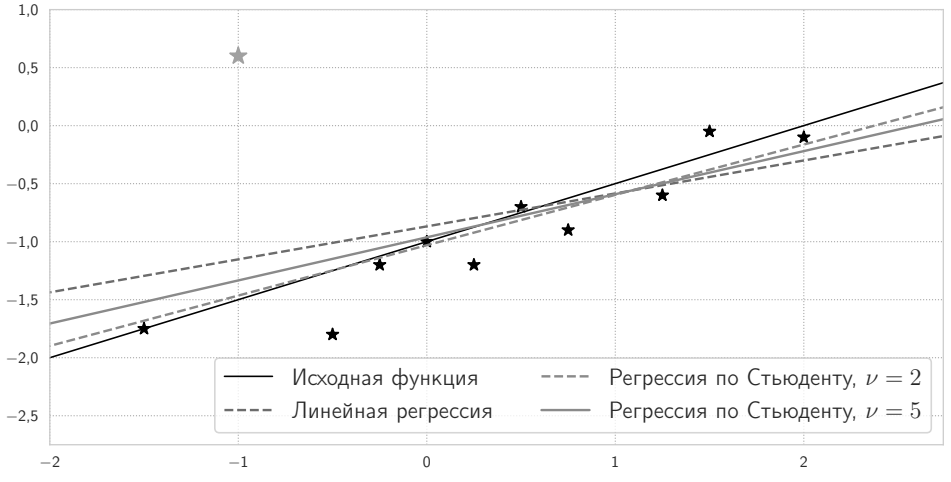


Рис. 4.25. Сравнение гауссовского шума и шума с распределением Стьюдента: линейная регрессия с одним выбросом

```
def loglike_grad(w, nu, y, X):
    rs = (y - X.dot(w))
    grad_ws = rs / (nu + rs**2)
    return X.T.dot(grad_ws)
```

Теперь можно решать эту систему уравнений, подставляя вместо `y` и `X` имеющиеся данные и признаки; например, чтобы получить веса линейной регрессии с шумом, распределённым по Стьюденту с $\nu = 1$, можно запустить такой код:

```
from scipy.optimize import fsolve
tw = fsolve(lambda x : loglike_grad(x, 1, data, X), np.zeros(X.shape[1]))
```

Здесь написано, что мы решаем уравнение $\text{loglike_grad}(\boldsymbol{x}) = 0$ (мы предполагаем, что в функцию подставлены признаки `X` и правильные ответы `data`) численно, с начальной точкой в нуле.

Результат показан на рис. 4.25. Мы видим, что регрессия, в которой шум имеет распределение Стьюдента, действительно более устойчива к выбросам, причём эту устойчивость, как мы обсуждали выше, можно контролировать при помощи параметра ν: уже для $\nu = 5$ прямая нашей новой регрессии становится весьма похожа на прямую обычной линейной регрессии с гауссовским шумом, и при больших значениях ν они сольются.

Разумеется, ровно то же самое можно сделать не только для линейных функций, но и для каких угодно других признаков. Например, на рис. 4.26 проведён тот же самый эксперимент, но для уже знакомой нам полиномиальной регрессии с $d = 3$ и точками, распределёнными вокруг одного периода синусоиды, с одним

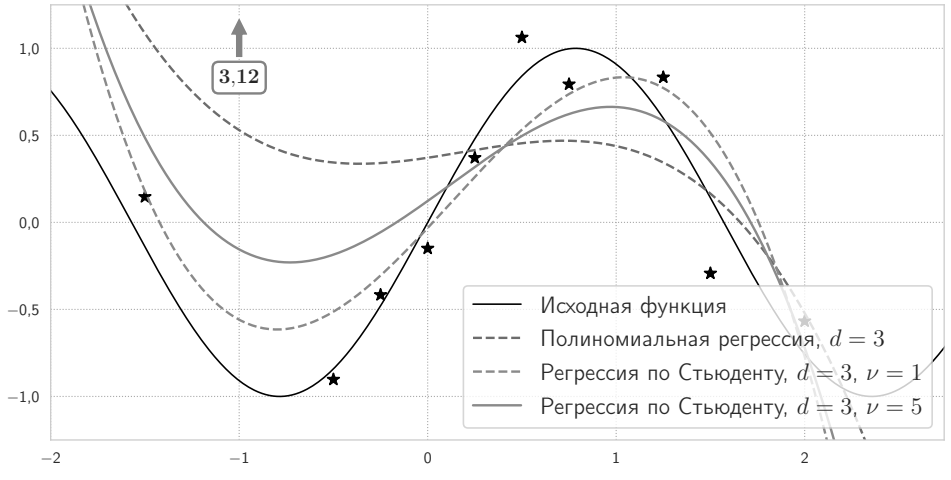


Рис. 4.26. Сравнение гауссовского шума и шума с распределением Стьюдента: полиномиальная регрессия с одним выбросом

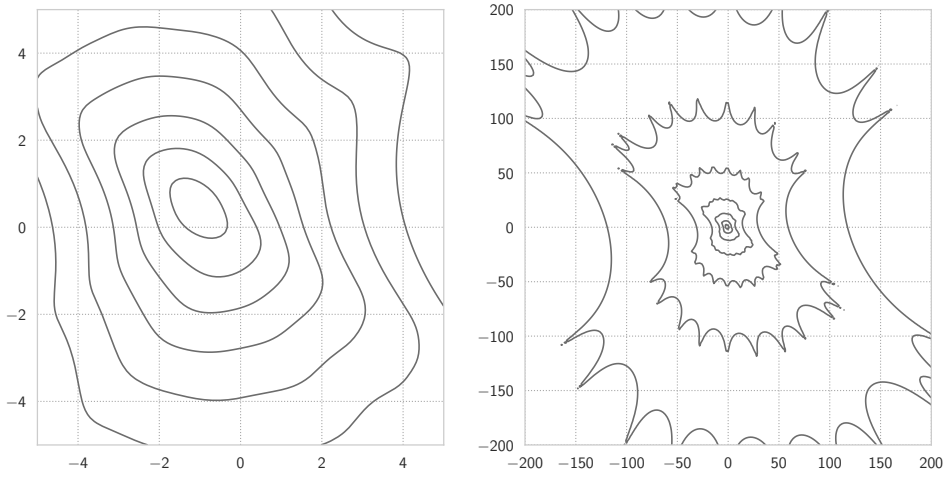
искусственно введённым выбросом. Поведение полученных кривых здесь точно такое же: регрессия со стьюдентовским шумом гораздо устойчивее к выбросу, и при увеличении ν стремится к обычной полиномиальной регрессии. Замечу, однако, что задача оптимизации, которую мы замели под ковёр, для регрессии со стьюдентовским шумом может оказаться сложнее и интереснее, чем в случае обычной линейной регрессии. Мы не будем углубляться, но я проиллюстрирую это тем, что нарисую линии уровня той функции, которую мы оптимизируем, а именно логарифма правдоподобия (точнее говоря, пропорциональной ему функции):

```
def loglike_t(w, nu, data, X):
    rs = (data - X.dot(w))
    return np.sum(np.log(nu + rs**2))
```

На рис. 4.27 показаны получающиеся линии уровня логарифма правдоподобия в случае обычной линейной регрессии с $\nu = 1$ (как на рис. 4.25). Как видите, хотя в данном случае целевая функция действительно имеет один оптимум, она далеко не выпукла, и задача оптимизации (4.19), особенно в высокой размерности или со сложными признаками, может оказаться совсем не так проста.

*4.3.5. Другая робастная регрессия: RANSAC и оценка Тейла — Сена

А в завершение этого раздела давайте обсудим другие способы сделать регрессию робастной. Конечно, в некотором высшем вероятностном смысле все



(а) (б)

Рис. 4.27. Линии уровня логарифма правдоподобия для регрессии со стьюдентовским шумом: (а) рядом с точкой оптимума; (б) в более мелком масштабе

они сводятся к тому, чтобы изменить распределение шума, но на практике алгоритмы и оценки, которые мы сейчас рассмотрим, получаются совсем из других соображений.

Первый метод, который хочется обсудить, — RANSAC (random sample consensus, случайный консенсус выборки). Идея метода такова: RANSAC условно предполагает, что точки данных делятся на «нормальные» (inliers), на которых мы и хотим обучить регрессию в итоге, и выбросы (outliers), которые хочется просто проигнорировать. Задача, таким образом, состоит в том, чтобы отличить одни от других. RANSAC делает это, итеративно запуская следующий алгоритм:

- выбрать случайное подмножество входных данных;
- обучить на них модель;
- посмотреть, насколько хорошо модель подходит имеющимся данным; подмножество точек, которые подходят «хорошо» (что бы это ни значило), называется *консенсусным множеством* (consensus set) для текущей модели;
- если консенсусное множество S слишком маленькое (опять же что бы это ни значило), значит, нам не повезло с моделью и лучше вернуться к первому шагу;
- а если достаточно большое, то можно подсчитать, насколько хорошо модель подходит точкам из S, и если ошибка оказалась меньше, чем на предыдущих попытках, то сохранить модель как лучшую и тогда уже перезапускать процесс;

Алгоритм 3. Метод RANSAC

Вход: датасет D, модель M, порог ℓ, порог d;
$\boldsymbol{\theta}_*, \epsilon_* := \emptyset, \infty$;
repeat
  $D' \sim D$ // семплируем подмножество датасета
  $\boldsymbol{\theta} := M.\text{fit}(D')$;
  $S := \{\boldsymbol{x} \in D \mid > M.\text{error}(\boldsymbol{\theta}, \boldsymbol{x}) \leq \ell\}$ // выберем консенсусное множество
  if $|S| > d$ **then**
    $\epsilon := \frac{1}{|S|} \sum_{\boldsymbol{x} \in S} M.\text{error}(\boldsymbol{\theta}, \boldsymbol{x})$;
    if $\epsilon < \epsilon_*$ **then**
      $\boldsymbol{\theta}_*, \epsilon_* := \boldsymbol{\theta}, \epsilon$;
    end
    $\boldsymbol{\theta}' := M.\text{fit}(S)$;
    $\epsilon' := \frac{1}{|S|} \sum_{\boldsymbol{x} \in S} M.\text{error}(\boldsymbol{\theta}', \boldsymbol{x})$;
    if $\epsilon' < \epsilon_*$ **then**
      $\boldsymbol{\theta}_*, \epsilon_* := \boldsymbol{\theta}', \epsilon'$;
    end
  end
until *выполнено некоторое условие остановки*;

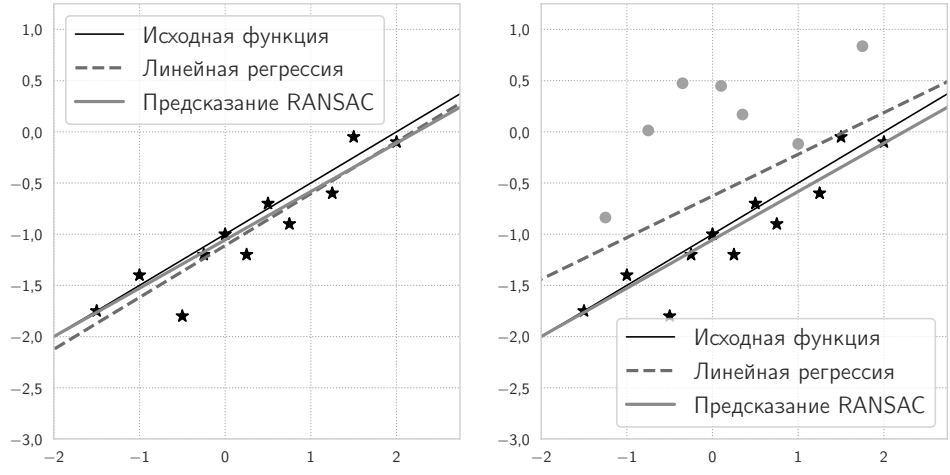
- кроме того, можно попробовать улучшить модель, переобучив её только на точках из S, чтобы исключить выбросы, попавшие в изначальное случайное подмножество.

Условный псевдокод метода RANSAC показан в алгоритме 3. Для того, чтобы он работал, нужно, чтобы модель могла обучаться на подмножествах датасета (метод fit, выдающий обученные параметры модели $\boldsymbol{\theta}$), а также подсчитывать некоторую ошибку на точке данных (метод error, который получает на вход параметры модели $\boldsymbol{\theta}$ и точку данных $\boldsymbol{x}$ и выдаёт ошибку предсказания).

На рис. 4.28, *а* показан метод RANSAC, применённый к стандартному датасету, который мы уже не раз видели в этой главе; здесь, конечно, никакой ощутимой разницы между RANSAC и обычной линейной регрессией не видно. А вот на рис. 4.28, *б* я добавил несколько точек с искусственно порождёнными «выбросами» (на рис. 4.29, *б* эти точки отмечены кружками), причём этот случай я сделал довольно сложным:

- выбросов много — новых точек я добавил целых семь, а исходных было всего одиннадцать;
- все выбросы смещены в одну и ту же сторону (вверх).

Как видите, RANSAC блестяще справляется с этой сложной ситуацией, в то время как обычная линейная регрессия уходит далеко наверх.



(а) Стандартный датасет (б) Датасет с выбросами

Рис. 4.28. Метод RANSAC: (а) оценка на стандартном датасете; (б) оценка на датасете с искусственно добавленными выбросами

Впервые RANSAC появился в вышедшей в 1981 году работе Мартина Фишера и Роберта Болля [182], в которой они применяли новый метод робастной регрессии к задаче определения местоположения (location determination problem, LDP): по фотографии, на которой присутствуют объекты (landmarks) с известными координатами, определить, из какой точки эта фотография была снята.

В дальнейшем RANSAC получил большую популярность именно в компьютерном зрении; в 2006 году в честь 25-летия исходной работы на главной конференции по компьютерному зрению CVPR (International Conference on Computer Vision and Pattern Recognition) был даже организован специальный семинар, посвящённый новым вариантам и дальнейшему развитию метода RANSAC. В наше время, правда, RANSAC для компьютерного зрения уже не слишком актуален, глубокие нейронные сети (сначала свёрточные, а теперь основанные на архитектуре трансформера) вытеснили остальные методы почти полностью.

Многие из этих вариантов посвящены тому, как правильно выбирать гиперпараметры: как видно из алгоритма 3, для применения RANSAC нужно выбрать два порога, ℓ и d, выбрать размер семплируемого подмножества D', а также перезапускать весь процесс какое-то число раз или до «сходимости», которую тоже нужно так или иначе определить. Важнее всего здесь правильно выбрать порог для того, чтобы считать точку «нормальной»: можно оценивать правдоподобие или апостериорную вероятность консенсусного множества [600, 601], можно использовать априорные знания [98, 599] и так далее. Другие работы ускоряют

RANSAC, например переходят от оценки консенсусного множества на всём датасете к оценке на случайном подмножестве [381]; третьи обобщают RANSAC на ещё более «испорченные» данные, где, например, заранее не известен масштаб шума [633]; но во всю эту науку мы углубляться не будем.

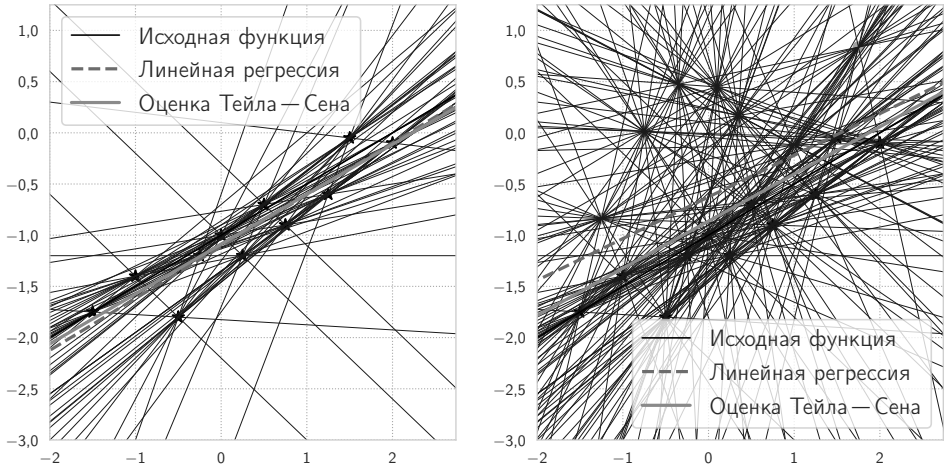
Второй метод, позволяющий линейной регрессии успешно обрабатывать выбросы, мотивирован самой простой робастной статистической оценкой, которую вы наверняка знаете, а именно использованием *медианы* (значения из середины упорядоченного списка данных) вместо среднего. Действительно, если вам нужно, например, оценить доход людей в некоторой выборке одним числом, медиана будет для этого куда более подходящей оценкой, чем среднее арифметическое, потому что доход существенно ограничен снизу (нулём), а сверху ограничен куда хуже. Если в вашу выборку из тысячи человек, доход которых нормально распределён вокруг \$1000 в месяц, случайно попадёт один богач с доходом \$1 000 000 в месяц, среднее арифметическое сразу же вырастет вдвое, а медиана практически не изменится — это как раз и есть то самое свойство робастности, устойчивости к выбросам, которое мы хотим получить; мы сделаем это рассуждение точнее чуть ниже. Для случая оценки доходов это очевидно, потому что существенное расслоение в доходах можно ожидать не только в выборке по всему миру или всей стране, но даже среди доходов, например, сотрудников одного и того же предприятия: вполне естественно, что директор и совладелец зарабатывают куда больше, чем только что нанятые практиканты, и если их доходы просто усреднить между собой, скорее всего, получится ни о чём не говорящее промежуточное число. Но как применить медиану к линейной регрессии?

В 1950 году Анри Тейл [589–591], а в 1968 году Пранаб Сен [525][1] предложили метод, который получил название *оценки Тейла — Сена* (Theil — Sen estimator): давайте проведём прямую через каждую пару точек данных $(x_i, y_i), (x_j, y_j) \in D$, а потом возьмём медиану m полученных углов наклона! Ну а когда m уже определено, свободный член тоже получится как медиана остатков от вычитания mx_i:

$$m = \text{Median}_{i\neq j=1}^{N} \left(\frac{y_i - y_j}{x_i - x_j} \right), \quad b = \text{Median}_{i=1}^{N} \left(y_i - mx_i \right).$$

У оценки Тейла — Сена очень хорошие статистические свойства. В частности, важным показателем, которым можно оценить качество робастной оценки, является так называемая *точка срыва* (breakdown point) [133, 150]: какую долю данных можно испортить (заменить на недостоверные), прежде чем оценка потеряет смысл, то есть прежде чем этой заменой можно будет внести в оценку сколь угодно большие изменения? Для метода наименьших квадратов точка срыва, очевидно, составляет $\frac{1}{N}$: изменяя одну точку, можно сделать среднее арифметическое сколь угодно большим или сколь угодно маленьким.

[1] Эх, вот были времена: можно было опубликовать практически ту же идею на 18 лет позже и всё равно попасть в название... Впрочем, это я не всерьёз: на самом деле в работе Пранаба Сена проделан существенный дополнительный анализ.



(а) Стандартный датасет (б) Датасет с выбросами

Рис. 4.29. Оценка Тейла — Сена: (а) оценка на стандартном датасете; (б) оценка на датасете с выбросами

Оказывается [142, 143, 644], что для оценки Тейла — Сена точка срыва составляет $1 - \frac{1}{\sqrt{2}} \approx 0{,}293$, то есть можно исправлять как угодно почти 30 % данных, прежде чем эта оценка действительно безвозвратно испортится. И даже более того, есть разработанный Эндрю Сигелом вариант оценки Тейла — Сена, который называется *повторяющейся медианной оценкой* (repeated median estimator) [540]:

- сначала для каждой точки выборки (x_i, y_i) определить медиану коэффициентов наклона прямых, проходящих через эту точку:

$$m_i = \text{Median}_{j \neq i} \left(\frac{y_i - y_j}{x_i - x_j} \right);$$

- а затем выбрать медиану из этих медиан, $m = \text{Median}_i(m_i)$.

Для повторяющейся медианной оценки точка срыва составляет аж $\frac{1}{2}$: половину данных можно испортить, не потеряв смысла оценки!

Это можно увидеть и на практике. Рисунок 4.29 показывает практический пример применения оценки Тейла — Сена для тех же данных, которые мы видели на рис. 4.28; тонкие серые линии показывают всевозможные прямые, соединяющие точки данных попарно. Как видите, несмотря на то, что пучок соединяющих точки попарно прямых на рис. 4.29, *б* выглядит уже совсем не так приятно, как на рис. 4.29, *а*, оценка Тейла — Сена всё равно даёт результат, весьма близкий к «настоящей» прямой (хотя с этим примером, пожалуй, RANSAC справился лучше).

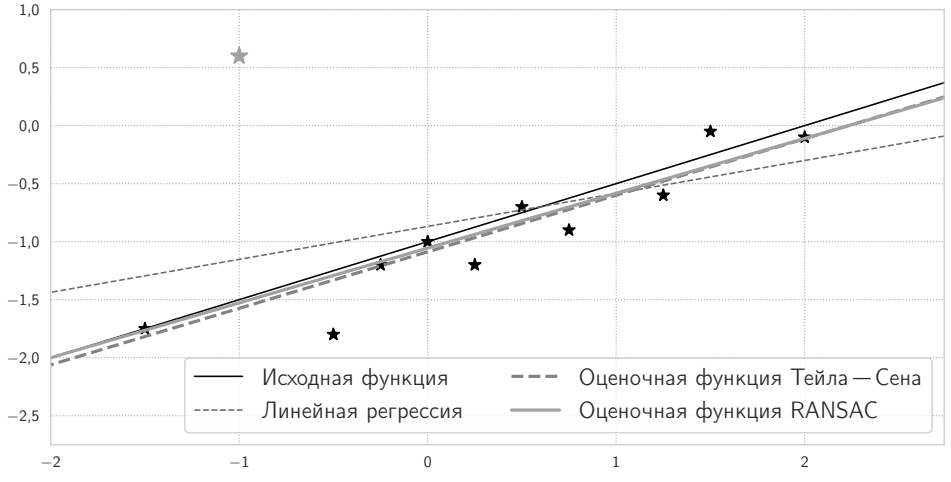


Рис. 4.30. Методы робастной регрессии на примере линейной регрессии с одним выбросом

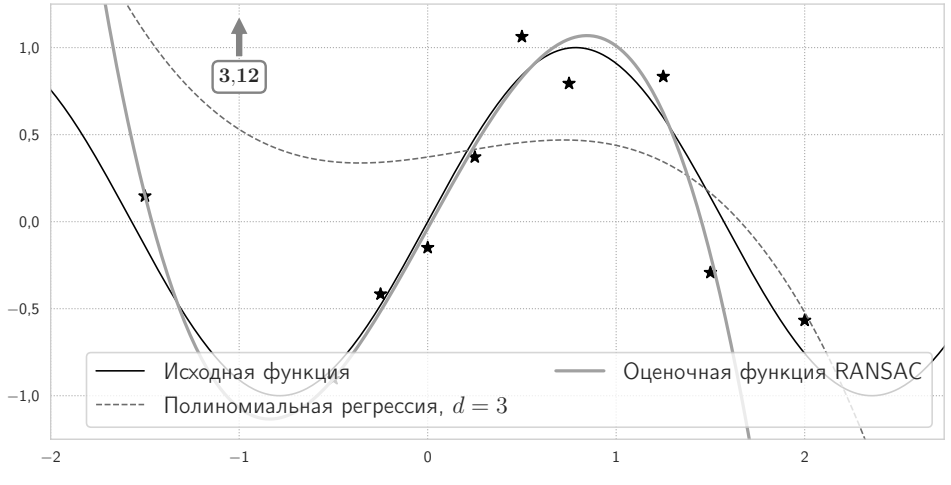


Рис. 4.31. Метод RANSAC на примере полиномиальной регрессии с одним выбросом

Практическим недостатком оценки Тейла — Сена является то, что для её вычисления нужно перебрать все пары точек, то есть сложность её вычисления при наивном подходе составляет $O(N^2)$. Однако существуют алгоритмы, сокращающие её до $O(N \log N)$ [70, 79, 304], да и сложность повторяющейся медианной оценки можно свести к тому же [383].

На рис. 4.30 и 4.31 рассмотренные нами методы применены к примерам, которые мы уже рассматривали в подразделе 4.3.4. Для простой линейной регрессии с одним выбросом (рис. 4.30) заметной разницы, конечно, нет: и RANSAC, и оценка Тейла — Сена прекрасно справляются. Для полиномиальной регрессии с одним выбросом (рис. 4.31) RANSAC работает по-прежнему, а вот с оценкой Тейла — Сена возникает проблема: как обобщить её на несколько признаков? Для этого нужно обобщить понятие медианы, и оказывается, что это не так просто — есть разные варианты определения многомерной медианы (multivariate median), работать с ними совсем не просто, и многомерные обобщения начали развиваться только в 2000-х годах, через 50 лет после исходных работ Тейла и Сена, поэтому я просто оставлю здесь ссылки на эти обобщения [592, 663].

Лично для меня самое интересное в истории методов робастной регрессии — это то, насколько малое распространение на практике они получили. Казалось бы, почти в любой науке, где нужно обрабатывать данные, бывают выбросы, которые лучше бы не учитывать, а шум часто имеет тяжёлые хвосты по тем же уже упомянутым причинам, которые и к закону Ципфа приводят. Тем не менее, если вы посмотрите даже на современные статьи со статистической обработкой данных, вы не так уж часто увидите там как оценку Тейла — Сена или RANSAC, так и регрессию с шумом, порождённым из распределения Стьюдента. Причины мне не ясны: более высокая вычислительная сложность в подавляющем большинстве случаев никому не помешает. Так что мне кажется, что во многом это происходит всего лишь от недостаточной осведомлённости исследователей о методах робастной регрессии. Давайте я осторожно порекомендую вам всё-таки пользоваться робастной регрессией, особенно в случаях относительно небольшой размерности; ну а теперь нам с вами пора двигаться дальше.

4.4. Байесовский вывод в линейной регрессии

...Самостоятельная жизнь, являющаяся для учения несомненным недостатком, представляет для пытливого исследователя определённые преимущества. В моём случае она была тем более необходимой, что я не мог следовать собственно научной методе, то есть не мог использовать работы предшественников и вступать в контакт с современными исследователями. Я был полностью предоставлен самому себе, начал с самых азов и с пониманием — окрыляющим в юности и удручающим в преклонные лета пониманием — того, что и случайный вывод, к коему я приду, окажется выводом окончательным. А на самом ли деле я так одинок в своих исследованиях, прежде и теперь? И да, и нет. Нельзя ведь исключать того, что отдельные псы там и сям оказывались или оказываются в моем положении.

Франц Кафка. Исследования одной собаки
(пер. Ю. Архипова)

4.4.1. Априорное и апостериорное распределения

В предыдущем разделе мы рассмотрели вероятностные предположения, которые делают линейная регрессия и метод наименьших квадратов. Но всё, о чём мы до сих пор говорили, — это максимизация правдоподобия $p(\boldsymbol{t}|\boldsymbol{X}, \boldsymbol{w})$. А где же байесовский вывод, который мы обсуждали в главе 2 как основу всего машинного обучения? Давайте посмотрим, что будет получаться.

Во-первых, как мы уже знаем, от максимизации правдоподобия можно перейти к подсчёту (и, если нужно, максимизации) апостериорного распределения, домножив правдоподобие на априорное распределение на весах модели $p(\boldsymbol{w})$:

$$p(\boldsymbol{w}|\boldsymbol{t}, \boldsymbol{X}) \propto p(\boldsymbol{w})\, p(\boldsymbol{t}|\boldsymbol{X}, \boldsymbol{w}).$$

С точки зрения задач оптимизации поиск максимальной апостериорной гипотезы — это максимизация

$$\ln p(\boldsymbol{w}|\boldsymbol{t}, \boldsymbol{X}) = \ln p(\boldsymbol{t}|\boldsymbol{X}, \boldsymbol{w}) + \ln p(\boldsymbol{w}) + \text{Const},$$

то есть в случае линейной регрессии

$$\ln p(\boldsymbol{w}|\boldsymbol{t}, \boldsymbol{X}) = -\frac{1}{2\sigma^2} \sum\nolimits_{n=1}^{N} \left(t_n - \boldsymbol{w}^\top \boldsymbol{x}_n\right)^2 + \ln p(\boldsymbol{w}) + \text{Const}.$$

Теперь можно в качестве $p(\boldsymbol{w})$ подставлять априорные распределения, выражающие нашу интуицию по поводу того, какими должны быть веса $\boldsymbol{w}$. На первый взгляд кажется, что никакой интуиции у нас нет — это же просто абстрактные веса, откуда нам знать про них что-то? Но на самом деле мы уже обсуждали в разделе 4.2, что объяснением для введения регуляризаторов может стать предположение о том, что веса должны быть не слишком велики по модулю.

На вероятностном языке это значит, что мы хотим выбрать $p(\boldsymbol{w})$ так, чтобы оно было в основном сосредоточено вокруг нуля, а к бесконечности убывало. Думаю, вы уже догадались, что первым из таких распределений мы попробуем гауссиан с нулевым средним и простейшей матрицей ковариаций, пропорциональной единичной:

$$p(\boldsymbol{w}) = \mathcal{N}\left(\boldsymbol{w}\middle|\mathbf{O}, \sigma_0^2\mathbf{I}\right).$$

Тогда логарифм правдоподобия априорного распределения будет равен

$$p(\boldsymbol{w}) = -\frac{d}{2}\ln\sigma_0^2 - \frac{1}{2\sigma_0^2}\boldsymbol{w}^\top\boldsymbol{w},$$

а логарифм плотности апостериорного распределения превращается в

$$\ln p(\boldsymbol{w}|\boldsymbol{t}, \boldsymbol{X}) = -\frac{1}{2\sigma^2}\sum_{n=1}^{N}\left(t_n - \boldsymbol{w}^\top\boldsymbol{x}_n\right)^2 - \frac{1}{2\sigma_0^2}\boldsymbol{w}^\top\boldsymbol{w} + \text{Const}, \tag{4.21}$$

и максимизировать его по $\boldsymbol{w}$ — это то же самое, что минимизировать

$$\sum_{n=1}^{N}\left(t_n - \boldsymbol{w}^\top\boldsymbol{x}_n\right)^2 + \frac{\sigma^2}{\sigma_0^2}\boldsymbol{w}^\top\boldsymbol{w}.$$

Узнаёте? Это и есть та самая гребневая регрессия, которую мы подробно обсуждали в разделе 4.2! Иначе говоря, мы только что дали вероятностную интерпретацию гребневой регрессии: она эквивалентна выбору априорного распределения в виде гауссиана с нулевым средним и матрицей ковариаций $\sigma_0\mathbf{I}$. Более того, это объяснение, как обычно и бывает с вероятностными интерпретациями, даёт нам ещё и понимание смысла коэффициента регуляризации α: оказывается, $\alpha = \frac{\sigma^2}{\sigma_0^2}$. Пока что это понимание не очень много добавляет к нашему арсеналу: если мы всё равно выбираем σ_0^2 сами, то и α мы фактически выбираем произвольно, а логика «чем больше α, тем больше мы стягиваем веса к нулю» была ясна и так. Но если вдруг σ_0 можно будет как-то оценить, этот результат станет ещё более полезным.

Замечу, однако, что, варьируя только σ_0, мы не сможем получить семейство сопряжённых априорных распределений (вспомните раздел 2.4). После первой

же точки данных максимальная апостериорная гипотеза перестанет быть нулевым вектором. Но в целом мы на верном пути: в уравнении (4.21) справа стоит квадратичный многочлен от $\boldsymbol{w}$, а значит, $p(\boldsymbol{w}|\boldsymbol{t}, \boldsymbol{X})$ опять будет гауссианом. Чтобы получить сопряжённое априорное распределение, нужно просто обобщить семейство до всех многомерных гауссианов:

$$p(\boldsymbol{w}) = \mathcal{N}(\boldsymbol{w}|\boldsymbol{\mu}_0, \Sigma_0).$$

И теперь можно подсчитать апостериорное распределение. Для этого нужно подставить в формулу (4.21) логарифм правдоподобия априорного распределения в общем виде:

$$\ln p(\boldsymbol{w}|\boldsymbol{t}, \boldsymbol{X}) = -\frac{1}{2\sigma^2}\sum_{n=1}^{N}\left(t_n - \boldsymbol{w}^\top \boldsymbol{x}_n\right)^2 - \\ - \frac{1}{2}(\boldsymbol{w} - \boldsymbol{\mu}_0)^\top \Sigma_0^{-1}(\boldsymbol{w} - \boldsymbol{\mu}_0) + \text{Const}, \quad (4.22)$$

а затем привести результат снова к виду

$$\ln p(\boldsymbol{w}|\boldsymbol{t}, \boldsymbol{X}) = -\frac{1}{2}(\boldsymbol{w} - \boldsymbol{\mu}_N)^\top \Sigma_N (\boldsymbol{w} - \boldsymbol{\mu}_N) + \text{Const}$$

для каких-то других $\boldsymbol{\mu}_N$ и Σ_N. Константу здесь можно будет не считать — она автоматически станет нормировочной константой гауссиана. В итоге нам нужно просто выделить полный квадрат в правой части уравнения (4.22). Это дело сугубо техническое, но давайте всё-таки проведём это вычисление подробно — выделять полный квадрат в подобных формулах нам ещё понадобится не раз.

В правой части уравнения (4.21) нужно записать сумму квадратов в матричном виде, затем раскрыть все скобки, убрать константы, не зависящие от переменной $\boldsymbol{w}$, распределение по которой мы ищем, и привести подобные:

$$\begin{aligned}
&\ln p(\boldsymbol{w}|\boldsymbol{t}, \boldsymbol{X}) = \\
&\quad = -\frac{1}{2\sigma^2}\sum_{n=1}^{N}\left(t_n - \boldsymbol{w}^\top \boldsymbol{x}_n\right)^2 - \frac{1}{2}(\boldsymbol{w} - \boldsymbol{\mu}_0)^\top \Sigma_0^{-1}(\boldsymbol{w} - \boldsymbol{\mu}_0) + \text{Const} \\
&= -\frac{1}{2\sigma^2}(\boldsymbol{t} - \boldsymbol{X}\boldsymbol{w})^\top (\boldsymbol{t} - \boldsymbol{X}\boldsymbol{w}) - \frac{1}{2}(\boldsymbol{w} - \boldsymbol{\mu}_0)^\top \Sigma_0^{-1}(\boldsymbol{w} - \boldsymbol{\mu}_0) + \text{Const} = \\
&\quad = -\frac{1}{2\sigma^2}\left(\boldsymbol{t}^\top \boldsymbol{t} - 2\boldsymbol{w}^\top \boldsymbol{X}^\top \boldsymbol{t} + \boldsymbol{w}^\top \boldsymbol{X}^\top \boldsymbol{X}\boldsymbol{w}\right) - \\
&\quad - \frac{1}{2}\left(\boldsymbol{w}^\top \Sigma_0^{-1}\boldsymbol{w} - 2\boldsymbol{w}^\top \Sigma_0^{-1}\boldsymbol{\mu}_0 + \boldsymbol{\mu}_0 \top \Sigma_0^{-1}\boldsymbol{\mu}_0\right) + \text{Const} \\
&\quad = -\frac{1}{2}\boldsymbol{w}\left(\frac{1}{\sigma^2}\boldsymbol{X}^\top \boldsymbol{X} + \Sigma_0^{-1}\right)\boldsymbol{w} + \boldsymbol{w}^\top \left(\frac{1}{\sigma^2}\boldsymbol{X}^\top \boldsymbol{t} + \Sigma_0^{-1}\boldsymbol{\mu}_0\right) + \text{Const}.
\end{aligned}$$

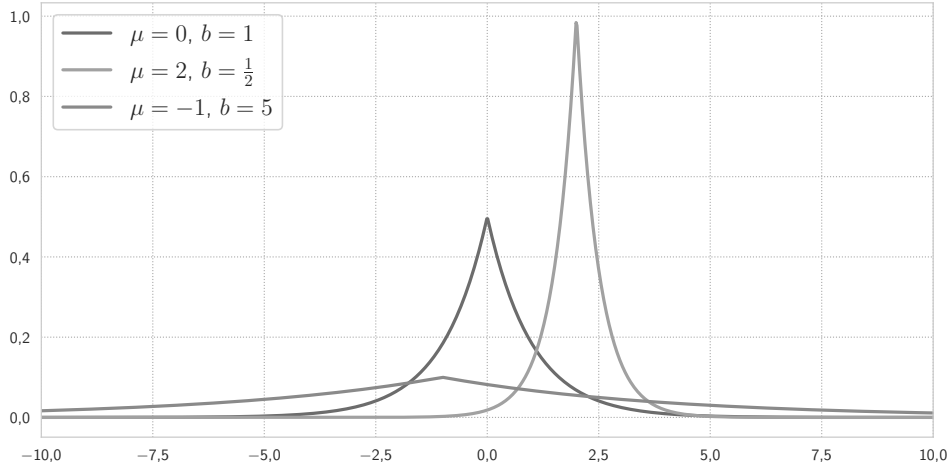


Рис. 4.32. Плотность распределения Лапласа $p\left(w|\mu, b\right) = \frac{1}{2b}e^{-\frac{1}{b}|x-\mu|}$ для нескольких разных значений μ и b

Итого из квадратичного слагаемого мы видим, что

$$\Sigma_N^{-1} = \frac{1}{\sigma^2}\boldsymbol{X}^\top\boldsymbol{X} + \Sigma_0^{-1}. \tag{4.23}$$

А матрица, на которую умножается линейная часть, — это $\Sigma_N^{-1}\boldsymbol{\mu}_N$, так что мы получаем

$$\boldsymbol{\mu}_N = \Sigma_N\left(\frac{1}{\sigma^2}\boldsymbol{X}^\top\boldsymbol{t} + \Sigma_0^{-1}\boldsymbol{\mu}_0\right). \tag{4.24}$$

Таким образом, мы получили формулы, которые позволяют теперь забыть обо всех муторных вычислениях и просто напрямую пересчитывать $\boldsymbol{\mu}_N$ и Σ_N, когда приходят новые точки данных. Формулы (4.23, 4.24) позволяют делать всё то, что мы обсуждали в разделе 2.4 о сопряжённых априорных распределениях: мы можем начать с некоторого $p\left(\boldsymbol{w}\right) = \mathcal{N}\left(\boldsymbol{w}|\boldsymbol{\mu}_0, \Sigma_0\right)$, получить набор точек, найти $\boldsymbol{\mu}_N$ и Σ_N, а затем забыть эти точки и использовать $\mathcal{N}\left(\boldsymbol{w}|\boldsymbol{\mu}_N, \Sigma_N\right)$ как новое априорное распределение для нового набора точек. Апостериорное распределение $\mathcal{N}\left(\boldsymbol{w}|\boldsymbol{\mu}_N, \Sigma_N\right)$ будет содержать всю информацию, которую мы в наших предположениях можем извлечь из исходного датасета о параметрах $\boldsymbol{w}$.

Итак, мы увидели, что гребневая регрессия соответствует частному случаю сопряжённого априорного распределения в виде гауссиана с нулевым средним. Но ведь гребневая регрессия — это только одна из форм регуляризаторов, которые мы видели в разделе 4.2. Можно ли другие регуляризаторы тоже представить в виде априорных распределений?

Конечно, можно. Любой регуляризатор $\alpha R(\boldsymbol{w})$ из раздела 4.2, добавляемый к сумме квадратов отклонений, можно по тем же соображениям рассматривать как минус логарифм плотности соответствующего априорного распределения. Например, лассо-регрессия, в которой минимизируется

$$\sum_{n=1}^{N}(y_n - \boldsymbol{w}^\top \boldsymbol{x}_n)^2 + \alpha \sum_{j=1}^{d} |w_j|,$$

соответствует априорному распределению с плотностью, пропорциональной

$$p(\boldsymbol{w}) \propto e^{-\sum_{j=1}^{d}|w_j|} = \prod_{j=1}^{d} e^{-|w_j|}.$$

Как видите, в данном случае (как и в случае гребневой регрессии, кстати) $p(\boldsymbol{w})$ раскладывается в произведение независимых распределений $p(w_j)$, а каждое из них представляет собой *распределение Лапласа*. В общем виде распределение Лапласа выглядит как

$$p(x|\mu, b) = \frac{1}{2b} e^{-\frac{1}{b}|x-\mu|}$$

и, по сути, представляет собой два экспоненциальных распределения, «сшитых» друг с другом в точке μ. Несколько графиков плотностей распределений Лапласа с разными параметрами показаны на рис. 4.32.

Точно так же можно рассмотреть и бридж-регрессию, и elastic net. Например, уже в той самой статье, где метод elastic net был предложен [668], Чжоу и Фридман рассматривают априорное распределение

$$p(\boldsymbol{w}) \propto e^{-\lambda\left(\alpha\sum_{j=1}^{d} w_j^2 + (1-\alpha)\sum_{j=1}^{d}|w_j|\right)},$$

хотя никаких интересных вероятностных свойств не обнаруживают.

А вот распределения, соответствующие бридж-регрессии, то есть

$$p(\boldsymbol{w}) \propto e^{-\lambda\sum_{j=1}^{d}|w_j|^q},$$

как раз довольно интересны. Формула выше тут же распадается на одномерные независимые сомножители $p(w_j) \propto e^{\lambda|w_j|^q}$, каждый из которых представляет собой частный случай так называемого *обобщённого нормального распределения*

$$p(x|\mu, \alpha, \beta) = \frac{\beta}{2\alpha\Gamma(1/\beta)} e^{-\left(\frac{|\boldsymbol{x}-\mu|}{\alpha}\right)^{\beta}},$$

семейства, которое включает в себя и обычные гауссианы, и распределение Лапласа. Обобщённое нормальное распределение находит интересные применения в анализе аудио- и видеосигналов [97, 531]. Но ещё раз подчеркну, что, хотя все эти варианты линейной регрессии по-своему интересны и могут иметь важные и полезные свойства, сопряжённым априорным распределением для линейной регрессии с гауссовским шумом является только семейство гауссианов, так что с вероятностной точки зрения все другие варианты рассматривать не слишком удобно.

4.4.2. Пример байесовского вывода в линейной регрессии

Проиллюстрирую происходящее на максимально подробном и, надеюсь, наглядном примере. Вернёмся к самому простому случаю — обычной линейной регрессии с одним признаком на входе и, соответственно, двумя весами, $\boldsymbol{w} = \begin{pmatrix} w_0 & w_1 \end{pmatrix}$. Данные я, как обычно, породил случайно, а затем зафиксировал, чтобы пример сохранялся неизменным:

```
true_mu = [.5, -.5]
def orig(x):
    return true_mu[0] + true_mu[1]*x

xd = np.array([-3, -2, -1, -0.5, 0, 0.5, 1, 1.5, 2.5, 3, 4]) / 2
num_points = len(xd)
data = np.array([1.4, 0.9, 1.5, 1.0, 0.3, 0.15, 0.18, 0.15, -0.20, 0.28, -0.17])

model = linear_model.LinearRegression( \
  fit_intercept=True).fit(xd.reshape(-1, 1), data)
```

Если теперь нарисовать предсказания обученной выше модели, получится линейная регрессия, изображённая на рис. 4.33. Впрочем, мы будем работать почти исключительно с двумя точками данных, специально выделенными на рисунке: это `data[1]` и `data[7]`.

Теперь давайте попробуем ввести и изобразить распределения на весах $\boldsymbol{w}$, которые получаются по мере добавления данных и соответствующего обучения линейной регрессии. Чтобы понять графики на рис. 4.34–4.37, которые составляют основу примера, очень важно понимать, в каких пространствах эти графики построены. Там, где на графиках изображены двумерные тепловые карты (то есть на рис. 4.34, *а*, 4.35, *а*, *б*, 4.36, *а* и 4.37, *а*), мы находимся в пространстве *весов* $\boldsymbol{w}$, где по горизонтальной оси отложен вес w_0, а по вертикальной — вес w_1. А на остальных графиках (то есть на рис. 4.34, *б*, 4.36, *б* и 4.37, *б*) мы видим прямые, проведённые уже в пространстве (x, y); здесь веса $\boldsymbol{w}$ становятся коэффициентами в уравнениях этих прямых.

Итак, начнём. Первый шаг, ещё до получения каких бы то ни было данных, — зафиксировать априорное распределение. Давайте возьмём в качестве него стандартный гауссиан, $p(\boldsymbol{w}) = \mathcal{N}(\boldsymbol{w}|\mathbf{O}, \mathbf{I})$. На рис. 4.34, *а* показаны тепловые карты

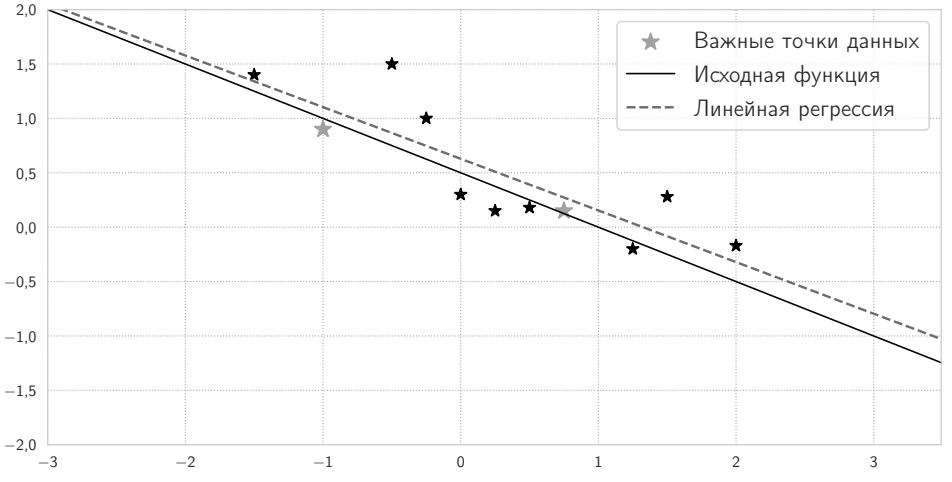
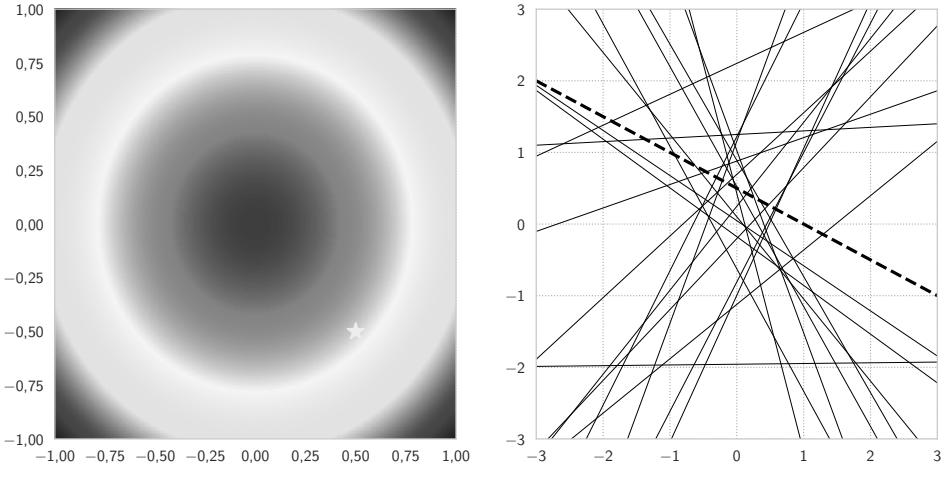


Рис. 4.33. Линейная регрессия для примера байесовского вывода



(а) Априорное распределение $p(\boldsymbol{w})$ (б) Прямые с параметрами $\boldsymbol{w} \sim p(\boldsymbol{w})$

Рис. 4.34. Априорное распределение параметров линейной регрессии и примеры соответствующих прямых

этого гауссиана (разумеется, это просто симметрично убывающая от нуля функция с круглыми линиями уровня); звёздочка на этом графике показывает истинное значение весов $\boldsymbol{w}_{\text{true}} = \begin{pmatrix} \frac{1}{2} & -\frac{1}{2} \end{pmatrix}$, о котором у нас пока никакой информации нет.

А на рис. 4.34, *б* показаны примеры прямых, чьи параметры просемплированы из этого распределения, $\boldsymbol{w} \sim p(\boldsymbol{w})$. Как видите, между этими прямыми мало общего; разве что близкие к вертикали или проходящие слишком далеко от нуля прямые мы тут вряд ли встретим, потому что коэффициенты будут получаться не слишком большими по модулю. На рис. 4.34, *а* звёздочка соответствует истинной прямой, вокруг которой были порождены данные, а на рис. 4.34, *б* истинная прямая показана пунктирной жирной линией.

Теперь давайте понемногу добавлять данные. Возьмём первую из выделенных на рис. 4.33 точек, `data[1]` с координатами $(-1, 0{,}9)$; в формулах я буду называть её d_1. Формула для правдоподобия точек данных в линейной регрессии нам уже хорошо знакома, это выражение (4.17), в которое нужно подставить лишь одну точку.

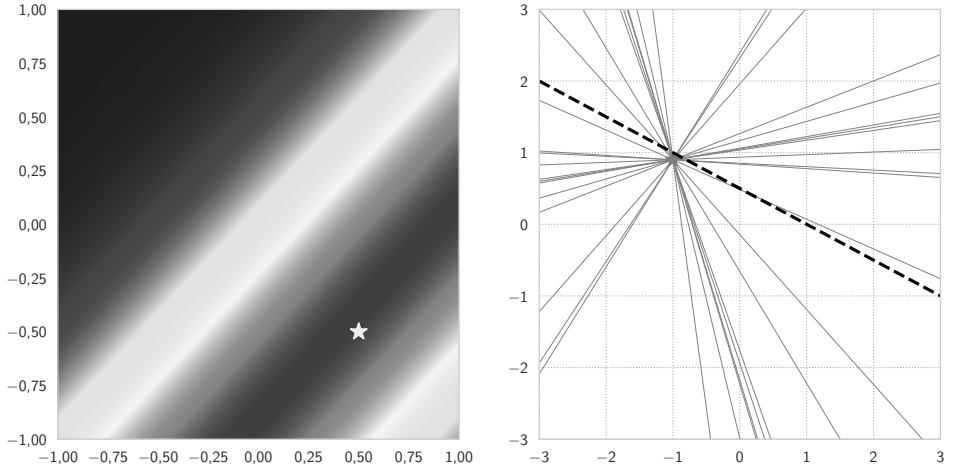
Давайте реализуем формулу для вычисления правдоподобия в коде на *Python*; мы знаем, что конкретное значение дисперсии шума для вывода в обычной линейной регрессии неважно, поэтому для определённости зафиксируем значение $\sigma = \frac{1}{2}$:

```
def get_likelihood(px, py, sigma=.5):
    return lambda w : np.exp(-(w[0] + w[1]*px - py) ** 2) / \
      (2 * sigma * np.sqrt(2.*np.pi))

cur_likelihood = get_likelihood(xd[1], data[1])
```

Теперь `cur_likelihood` — это функция от двух переменных, двух компонент вектора, и мы можем изобразить правдоподобие точки `data[1]` в том же двумерном пространстве весов; это сделано на рис. 4.35, *а*. Обратите внимание, что правдоподобие как функция от $\boldsymbol{w}$ — это не распределение вероятностей! В частности, максимум правдоподобия одной точки d_1 достигается, естественно, не в одном векторе $\boldsymbol{w}$, а во всех векторах, для которых соответствующая прямая проходит через эту точку. На рис. 4.35, *б* изображены несколько примеров таких прямых (они проходят через точку данных, показанную звёздочкой), но это не «семплы из правдоподобия», такая фраза вообще лишена смысла.

Как мы обсуждали в подразделе 4.4.1, апостериорное распределение $p(\boldsymbol{w}|d_1)$ можно найти аналитически; собственно, мы это уже сделали: $p(\boldsymbol{w}|d_1)$ — это гауссиан, чьи параметры задаются формулами (4.23) и (4.24). Давайте реализуем эти формулы в коде напрямую:

```
def bayesian_update(mu, sigma, x, y, sigma_noise=.5):
    x_matrix = np.array([[1, x]])
    sgm_n = np.linalg.inv(np.linalg.inv(sigma) + \
      (1/(sigma_noise**2)) * np.matmul(np.transpose(x_matrix), x_matrix))
    mu_n = np.matmul(sgm_n, np.matmul(np.linalg.inv(sigma), np.transpose(mu))+\
      (1/(sigma_noise**2)) * np.matmul(np.transpose(x_matrix), np.array([y])))
    return mu_n, sgm_n
```



(а) Правдоподобие одной точки $p(d_1|\boldsymbol{w})$

(б) Прямые с весами максимального правдоподобия

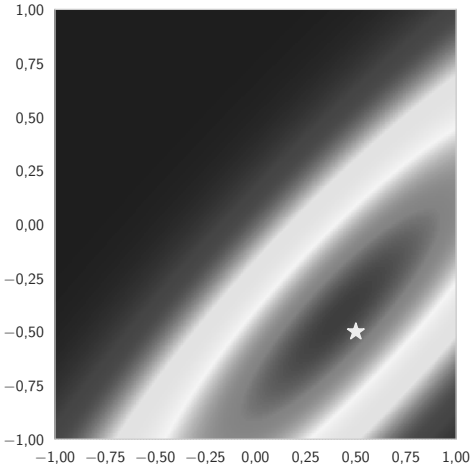
Рис. 4.35. Правдоподобие одной точки $d_1 = (-1, 0{,}9)$ и примеры прямых, максимизирующих его

```
mu1, sigma1 = bayesian_update(np.array([0, 0]), \
  2*np.array([[1, 0], [0, 1]]), xd[1], data[1])
```

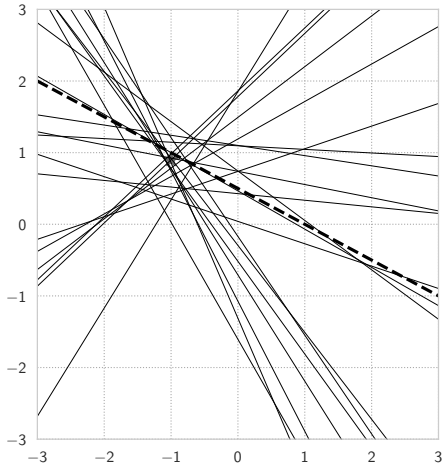
В результате получатся новые параметры апостериорного распределения. Теперь мы можем и нарисовать его, и посемплировать из него значения $\boldsymbol{w}$, то есть прямые в пространстве данных; обратите внимание, что апостериорное распределение $p(\boldsymbol{w}|d_1)$ — это уже самое настоящее распределение, и брать из него семплы вполне можно[1]. Результат показан на рис. 4.36. Слева, на рис. 4.36, *а*, показано пространство весов, в котором апостериорное распределение $p(\boldsymbol{w}|d_1)$ представляет собой гауссиан; этот гауссиан является перенормированным произведением $p(\boldsymbol{w})$ (см. рис. 4.34, *а*) и $p(d_1|\boldsymbol{w})$ (см. рис. 4.35, *а*). Это, наверное, самая наглядная иллюстрация: посмотрите на тепловые карты и убедитесь, что у нас действительно получилось произведение двух функций. А если посемплировать из него точки и нарисовать соответствующие прямые, то проходить последние будут где-то недалеко от точки d_1, как показано на рис. 4.36, *б*.

Пока с одной точкой мы больше ничего сделать не сможем, да и интуитивно понятно, что на одной точке регрессию не построишь даже в этом простейшем случае. Так что давайте добавим ещё одну, вторую выделенную точку `data[7]` с рис. 4.33; далее я буду называть её $d_2 = (0{,}75, 0{,}15)$.

[1] А иногда, когда аналитически подсчитать ответ не получается, даже нужно, но это уже совсем другая история, до которой мы в этой книге не дойдём...

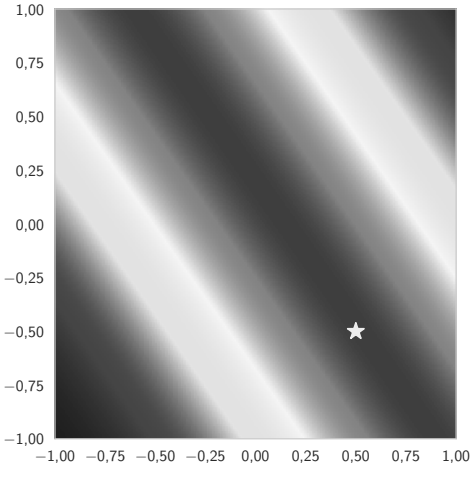


(а) Апостериорное распределение $p(\boldsymbol{w}|d_1)$

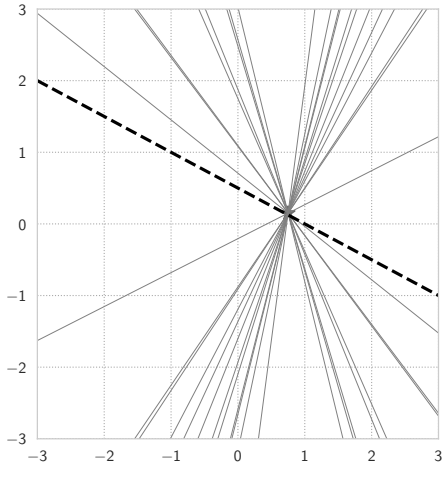


(б) Прямые с параметрами $\boldsymbol{w} \sim p(\boldsymbol{w}|d_1)$

Рис. 4.36. Апостериорное распределение на весах после добавления одной точки данных и примеры соответствующих прямых

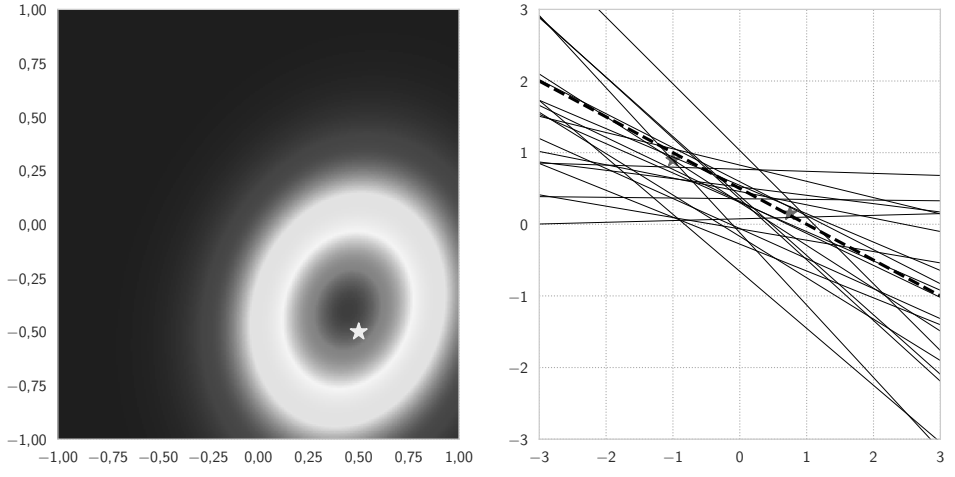


(а) Правдоподобие одной точки $p(d_2|\boldsymbol{w})$



(б) Прямые с весами максимального правдоподобия

Рис. 4.37. Правдоподобие одной точки $d_2 = (0{,}75, 0{,}15)$ и примеры прямых, максимизирующих его



(а) Апостериорное распределение $p(\boldsymbol{w}|d_1, d_2)$

(б) Прямые с параметрами $\boldsymbol{w} \sim p(\boldsymbol{w}|d_1, d_2)$

Рис. 4.38. Апостериорное распределение на весах после добавления двух точек данных и примеры соответствующих прямых

Правдоподобие одной точки $p(d_2|\boldsymbol{w})$ и прямые, максимизирующие это правдоподобие (то есть проходящие через d_2), показаны на рис. 4.37, который полностью аналогичен рис. 4.35.

Теперь можно ещё раз запустить процедуру `bayesian_update`:

```
mu2, sigma2 = bayesian_update(mu1, sigma1, xd[7], data[7])
```

Обратите внимание, что здесь мы используем эффект, который подробно обсуждали в подразделе 2.4.4, добавляя точки одну за другой и используя апостериорное распределение одной точки как априорное при добавлении следующей.

Полученное апостериорное распределение $p(\boldsymbol{w}|d_1, d_2)$ показано на рис. 4.38. Слева, на рис. 4.38, *а*, показано пространство весов, в котором $p(\boldsymbol{w}|d_1, d_2)$ представляет собой гауссиан уже гораздо более сконцентрированный, чем распределение $p(\boldsymbol{w}|d_1)$ на рис. 4.36, *а*, ведь две точки на плоскости задают прямую однозначно: гипотеза максимального правдоподобия здесь уже ровно одна и представляет собой прямую, проходящую через d_1 и d_2. Так что и пучок прямых, просемплированных из распределения $p(\boldsymbol{w}|d_1, d_2)$ (см. рис. 4.38, *б*), будет гораздо более плотным.

Итак, в этом разделе мы увидели на практике, как выглядит байесовский вывод для линейной регрессии, как априорное распределение постепенно умножается на правдоподобие точек данных и превращается в апостериорное распределение. Пора сделать следующий шаг.

4.4.3. Предсказательное распределение

Теперь нас ждёт следующая задача байесовского вывода. Давайте попробуем сделать последний шаг из тех, что мы обсуждали в главе 2, и построить предсказательное распределение: для новой точки $\boldsymbol{x}$ мы хотим понять, какое будет распределение у возможного ответа y,

$$p\left(y|\boldsymbol{x}, D\right) = \int_{\boldsymbol{w}} p\left(y, \boldsymbol{w}|\boldsymbol{x}, D\right) \mathrm{d}\boldsymbol{w} = \int_{\boldsymbol{w}} p\left(y|\boldsymbol{x}, \boldsymbol{w}\right) p\left(\boldsymbol{w}|D\right) \mathrm{d}\boldsymbol{w}. \tag{4.25}$$

В случае гребневой регрессии, то есть линейной регрессии с сопряжённым априорным распределением в виде гауссиана, здесь снова везде получаются гауссианы, и в правдоподобии одной точки $p\left(y|\boldsymbol{x}, \boldsymbol{w}\right)$, и в апостериорном распределении $p\left(\boldsymbol{w}|D\right)$ (которое мы вывели выше):

$$p\left(y|\boldsymbol{x}, D\right) = \int_{\boldsymbol{w}} p\left(y|\boldsymbol{x}, \boldsymbol{w}\right) p\left(\boldsymbol{w}|D\right) \mathrm{d}\boldsymbol{w} = \int_{\boldsymbol{w}} \mathcal{N}\left(y\middle|\boldsymbol{w}^\top \boldsymbol{x}, \sigma^2\right) \mathcal{N}\left(\boldsymbol{w}|\boldsymbol{\mu}_N, \Sigma_N\right) \mathrm{d}\boldsymbol{w}.$$

Значит, опять речь пойдёт о том, чтобы выделять полный квадрат в показателе экспоненты. Теперь, правда, задача чуть более хитрая: нам нужно взять интеграл по $\boldsymbol{w}$. Интеграл такого вида мы умеем брать только один: интеграл по гауссиану равен единице. Значит, нам нужно привести показатель экспоненты под интегралом к виду

$$-\frac{1}{2}(\boldsymbol{w} - \boldsymbol{\mu}')^\top \Sigma'^{-1}(\boldsymbol{w} - \boldsymbol{\mu}') + \mathrm{Const}(y),$$

и тогда интеграл от первой части свернётся в нормировочную константу гауссиана, а вторую часть, не зависящую от $\boldsymbol{w}$, можно будет вынести за скобку. Технически это чуть посложнее, потому что теперь мы не можем просто забывать все константы — нам из них потом нужно будет получить распределение на y, так что те части, где встречается y, придётся таскать за собой. Читатель, не желающий выделять полные квадраты, может смело пропустить приведённый ниже вывод и перейти к формуле (4.28) и её обсуждению ниже.

А с теми, кто остался, давайте выпишем логарифм подынтегрального выражения:

$$\begin{aligned}
&\ln \mathcal{N}\left(y\middle|\boldsymbol{w}^\top \boldsymbol{x}, \sigma^2\right) \mathcal{N}\left(\boldsymbol{w}|\boldsymbol{\mu}_N, \Sigma_N\right) = \\
&\quad = -\frac{1}{2\sigma^2}\left(\boldsymbol{w}^\top \boldsymbol{x} - y\right)^2 - \frac{1}{2}\left(\boldsymbol{w} - \boldsymbol{\mu}_N\right)^\top \Sigma_N^{-1}\left(\boldsymbol{w} - \boldsymbol{\mu}_N\right) + \mathrm{Const} = \\
&= -\frac{1}{2}\boldsymbol{w}^\top \left(\frac{1}{\sigma^2}\boldsymbol{x}\boldsymbol{x}^\top + \Sigma_N^{-1}\right)\boldsymbol{w} + \boldsymbol{w}^\top \left(\frac{y}{\sigma^2}\boldsymbol{x} + \Sigma_N^{-1}\boldsymbol{\mu}_N\right) - \frac{1}{2\sigma^2}y^2 + \mathrm{Const}.
\end{aligned}$$

Таким образом, мы получили (точно так же, как в выводе апостериорного распределения выше), что

$$\Sigma'^{-1} = \frac{1}{\sigma^2}\boldsymbol{x}\boldsymbol{x}^\top + \Sigma_N^{-1},$$
$$\Sigma'^{-1}\boldsymbol{\mu}' = \frac{y}{\sigma^2}\boldsymbol{x} + \Sigma_N^{-1}\boldsymbol{\mu}_N.$$

Казалось бы, можно сворачивать гауссиан, но здесь есть дополнительная тонкость. Поскольку $\boldsymbol{\mu}'$ зависит от y, нам нельзя просто посчитать свободный член $-\frac{1}{2}\boldsymbol{\mu}'^\top\Sigma'^{-1}\boldsymbol{\mu}'$ константой, а придётся выписать из него всё, что зависит от y. Получается, что в показателе экспоненты нам нужно вычесть и прибавить

$$\frac{1}{2}\boldsymbol{\mu}'^\top\Sigma'^{-1}\boldsymbol{\mu}' = \frac{1}{2}\left(\frac{y}{\sigma^2}\boldsymbol{x} + \Sigma_N^{-1}\boldsymbol{\mu}_N\right)^\top \Sigma'^\top\Sigma'^{-1}\Sigma'\left(\frac{y}{\sigma^2}\boldsymbol{x} + \Sigma_N^{-1}\boldsymbol{\mu}_N\right) =$$
$$= \frac{1}{2}\left(\frac{y}{\sigma^2}\boldsymbol{x} + \Sigma_N^{-1}\boldsymbol{\mu}_N\right)^\top \left(\frac{1}{\sigma^2}\boldsymbol{x}\boldsymbol{x}^\top + \Sigma_N^{-1}\right)^{-1}\left(\frac{y}{\sigma^2}\boldsymbol{x} + \Sigma_N^{-1}\boldsymbol{\mu}_N\right)$$

(здесь мы пользовались тем, что матрицы Σ_N и Σ' симметричны).

Теперь то, что не зависит от $\boldsymbol{w}$, можно вынести за знак интеграла, а сам интеграл сворачивается в нормировочную константу гауссиана:

$$\int e^{-\frac{1}{2}(\boldsymbol{w}-\boldsymbol{\mu}')^\top\Sigma'^{-1}(\boldsymbol{w}-\boldsymbol{\mu}')}\mathrm{d}\boldsymbol{w} = \left(2\pi\det\Sigma'\right)^{\frac{d}{2}}.$$

К счастью, Σ' не зависит от y, и этот сомножитель можно считать константой.

В итоге у нас остаётся следующее выражение:

$$\ln p\left(y|\boldsymbol{x},D\right) = \mathrm{Const} - \frac{1}{2\sigma^2}y^2 +$$
$$+ \frac{1}{2}\left(\frac{y}{\sigma^2}\boldsymbol{x} + \Sigma_N^{-1}\boldsymbol{\mu}_N\right)^\top \left(\frac{1}{\sigma^2}\boldsymbol{x}\boldsymbol{x}^\top + \Sigma_N^{-1}\right)^{-1}\left(\frac{y}{\sigma^2}\boldsymbol{x} + \Sigma_N^{-1}\boldsymbol{\mu}_N\right). \quad (4.26)$$

Узнаёте? Конечно, это опять экспонента от квадратичной функции от y, то есть в ответе опять получится гауссиан! Осталось только найти его параметры.

Для этого сначала заметим промежуточный результат, который, по сути, говорит нам, что $\boldsymbol{x}$ является собственным вектором матрицы $\Sigma'^{-1}\Sigma_N$:

$$\Sigma'^{-1}\Sigma_N\boldsymbol{x} = \left(\frac{1}{\sigma^2}\boldsymbol{x}\boldsymbol{x}^\top + \Sigma_N^{-1}\right)\Sigma_N\boldsymbol{x} = \frac{1}{\sigma^2}\boldsymbol{x}\boldsymbol{x}^\top\Sigma_N\boldsymbol{x} + \boldsymbol{x} = \left(\frac{1}{\sigma^2}\boldsymbol{x}^\top\Sigma_N\boldsymbol{x} + 1\right)\boldsymbol{x},$$

где мы можем менять местами сомножители в последнем равенстве, потому что $\boldsymbol{x}^\top\Sigma_N\boldsymbol{x}$ — это число, матрица размера 1×1. Если теперь это выражение умножить слева на Σ', а потом на Σ_N^{-1}, получатся два полезных равенства:

$$\Sigma'\boldsymbol{x} = \frac{1}{\frac{1}{\sigma^2}\boldsymbol{x}^\top\Sigma_N\boldsymbol{x}+1}\Sigma_N\boldsymbol{x}, \qquad \Sigma_N^{-1}\Sigma'\boldsymbol{x} = \frac{1}{\frac{1}{\sigma^2}\boldsymbol{x}^\top\Sigma_N\boldsymbol{x}+1}\boldsymbol{x}. \tag{4.27}$$

Теперь выделим в формуле (4.26) коэффициент при y^2; он равен

$$-\frac{1}{2}\left(\frac{1}{\sigma^2} - \frac{1}{\sigma^4}\boldsymbol{x}^\top\Sigma'\boldsymbol{x}\right),$$

что по формуле (4.27) равно

$$-\frac{1}{2}\left(\frac{1}{\sigma^2} - \frac{1}{\sigma^4}\frac{1}{\frac{1}{\sigma^2}\boldsymbol{x}^\top\Sigma_N\boldsymbol{x}+1}\boldsymbol{x}^\top\Sigma_N\boldsymbol{x}\right) =$$

$$= -\frac{1}{2\sigma^2}\left(1 - \frac{\boldsymbol{x}^\top\Sigma_N\boldsymbol{x}}{\boldsymbol{x}^\top\Sigma_N\boldsymbol{x}+\sigma^2}\right) = -\frac{1}{\boldsymbol{x}^\top\Sigma_N\boldsymbol{x}+\sigma^2}.$$

Это значит, что мы нашли дисперсию предсказательного распределения:

$$\sigma_{\text{pred}}(\boldsymbol{x})^2 = \boldsymbol{x}^\top\Sigma_N\boldsymbol{x} + \sigma^2.$$

А чтобы получить среднее $\mu_{\text{pred}}(\boldsymbol{x})$, нужно найти коэффициент при y первой степени; это оказывается ещё проще, коэффициент сразу виден из уравнения (4.26) и равен

$$\frac{1}{\sigma^2}\boldsymbol{x}^\top\Sigma'\Sigma_N^{-1}\boldsymbol{\mu}_N.$$

Этот коэффициент равен, как мы знаем, $\frac{\mu_{\text{pred}}(\boldsymbol{x})}{\sigma^2_{\text{pred}}(\boldsymbol{x})}$, и чтобы его найти, можно воспользоваться формулой (4.27). Ниже мы сначала меняем местами сомножители в скалярном произведении, потом пользуемся симметричностью Σ' и Σ_N, а затем подставляем результат из (4.27):

$$\frac{\mu_{\text{pred}}(\boldsymbol{x})}{\sigma^2_{\text{pred}}(\boldsymbol{x})} = \frac{1}{\sigma^2}\boldsymbol{x}^\top\Sigma'\Sigma_N^{-1}\boldsymbol{\mu}_N = \frac{1}{\sigma^2}\left(\Sigma'\Sigma_N^{-1}\boldsymbol{\mu}_N\right)^\top\boldsymbol{x} =$$

$$= \frac{1}{\sigma^2}\boldsymbol{\mu}_N^\top\Sigma_N^{-1}\Sigma'\boldsymbol{x} = \frac{1}{\sigma^2}\boldsymbol{\mu}_N^\top\frac{1}{\frac{1}{\sigma^2}\boldsymbol{x}^\top\Sigma_N\boldsymbol{x}+1}\boldsymbol{x} = \frac{1}{\sigma^2_{\text{pred}}(\boldsymbol{x})}\boldsymbol{\mu}_N^\top\boldsymbol{x}.$$

В итоге получается достаточно простой на вид ответ:

$$\begin{aligned} p(y|\boldsymbol{x},D) &= \mathcal{N}\left(y\middle|\mu_{\text{pred}}(\boldsymbol{x}),\sigma_{\text{pred}}(\boldsymbol{x})\right), \quad \text{где} \\ \mu_{\text{pred}}(\boldsymbol{x}) &= \boldsymbol{\mu}_N^\top\boldsymbol{x}, \\ \sigma^2_{\text{pred}}(\boldsymbol{x}) &= \boldsymbol{x}^\top\Sigma_N\boldsymbol{x} + \sigma^2. \end{aligned} \tag{4.28}$$

Обратите внимание, что собственно предсказанное значение, среднее этого гауссиана, по-прежнему равно $\boldsymbol{\mu}_N^\top \boldsymbol{x}$ и совпадает с максимальной апостериорной гипотезой. Иначе говоря, если нам нужно только точечное предсказание, эти рассуждения проводить не обязательно, предсказанное значение y в новых точках не изменится.

Но знание распределения (4.28) помогает узнать *дисперсию* нашего предсказания $\sigma_N^2(\boldsymbol{x})$, то есть откалибровать, насколько точными мы можем их ожидать. Во-первых, формула (4.28) показывает, что эта дисперсия всегда не меньше дисперсии случайного шума, $\sigma_N^2(\boldsymbol{x}) \geq \sigma^2$, — это очень естественно и ожидаемо. А во-вторых, формула (4.28) показывает точную зависимость этой самой дисперсии от $\boldsymbol{x}$, то есть показывает, как быстро наша уверенность падает по мере того, как мы начинаем удаляться от данных.

Предсказательное распределение на практике можно было бы получить и по-другому. Давайте вернёмся к уравнению (4.25) и вспомним, что на самом деле предсказательное распределение представляет собой математическое ожидание по апостериорному распределению:

$$p\left(y|\boldsymbol{x}, D\right) = \int_{\boldsymbol{w}} p\left(y, \boldsymbol{w}|\boldsymbol{x}, D\right) \mathrm{d}\boldsymbol{w} = \\ = \int_{\boldsymbol{w}} p\left(y|\boldsymbol{x}, \boldsymbol{w}\right) p\left(\boldsymbol{w}|D\right) \mathrm{d}\boldsymbol{w} = \mathbb{E}_{p(\boldsymbol{w}|D)}\left[p\left(y|\boldsymbol{x},\boldsymbol{w}\right)\right].$$

Это значит, что мы можем получить выборку из значений $p\left(y|\boldsymbol{x}, D\right)$, семплируя значения $\boldsymbol{w} \sim p\left(\boldsymbol{w}|D\right)$ из апостериорного распределения и подставляя их в модель. При этом нужно ещё добавить и дисперсию шума из модели, а то дисперсия предсказаний может оказаться даже ниже дисперсии шума, что было бы, мягко говоря, странно.

Так можно оценивать дисперсию предсказаний и в более сложных вероятностных моделях, где подсчитать предсказательное распределение аналитически, как в формуле (4.28), уже никак не получится. Именно поэтому для машинного обучения так важны алгоритмы *семплирования* из сложных вероятностных распределений... но до этого мы в этой книге ещё не доберёмся.

Проиллюстрирую полученный результат на примере, который мы уже рассматривали на рис. 4.33–4.38. Рисунок 4.39 показывает истинную «загаданную» прямую и максимальную апостериорную гипотезу, которая получается после добавления одной точки данных с $x_1 = -1$. Но главное — он показывает дисперсию предсказательного распределения: в точке $x = -1$ она равна дисперсии случайного шума, а потом начинает достаточно быстро расти в обе стороны.

Рисунок 4.39 показывает ту же дисперсию, но после добавления двух точек данных; как видите, максимальная апостериорная гипотеза почти точно проходит через эти две точки (почти, потому что априорное распределение всё же было), а дисперсия предсказательного распределения мала между этими точками

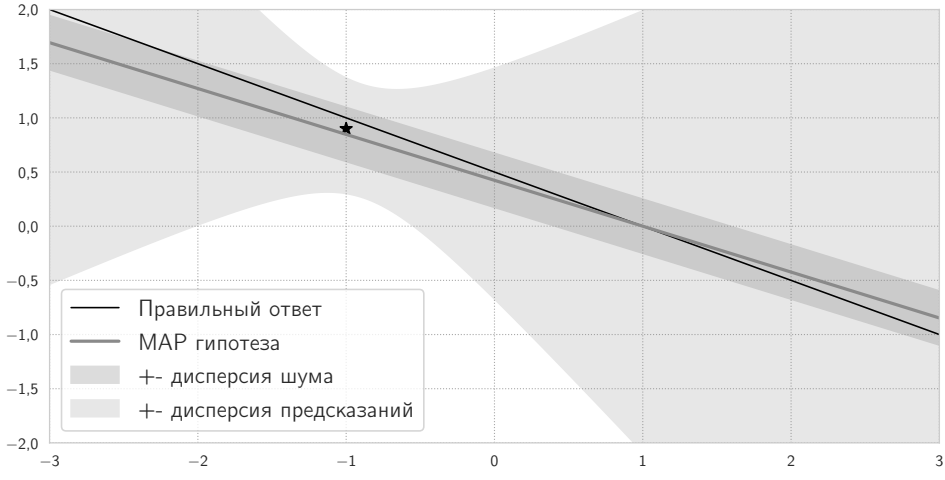


Рис. 4.39. Предсказательное распределение после добавления одной точки данных $d_1 = (-1, 0{,}9)$ (отмечена звёздочкой)

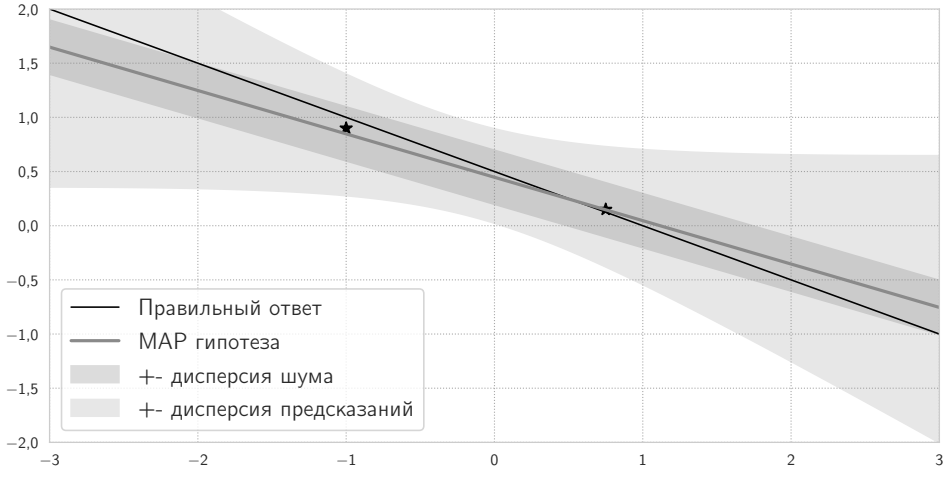


Рис. 4.40. Предсказательное распределение после добавления двух точек данных $(x_1, y_1) = (-1, 0{,}9)$ и $d_2 = (0{,}75, 0{,}15)$ (отмечены звёздочкой)

(там, где мы решаем задачу *интер*поляции) и опять начинает быстро расти по краям, там, где мы переходим к задаче *экстра*поляции.

*4.4.4. Оценка Джеймса — Штейна

В качестве сюжета «со звёздочкой» в этом разделе давайте рассмотрим интересный парадокс, который возникает при оценке ожидаемой ошибки при обучении обычного гауссиана, причём даже не при полноценном байесовском выводе (до него мы доберёмся только в разделе 6.4), а при поиске гипотезы максимального правдоподобия по μ с фиксированной дисперсией σ^2. Казалось бы, эту задачу мы полностью решили в формуле (3.17) — но давайте разберёмся подробнее...

Рассмотрим самую простую ситуацию: предположим, что у нас есть несколько нормально распределённых случайных величин $X_1, X_2, \ldots, X_n$ с одинаковой (единичной) дисперсией и разными неизвестными средними: $X_i \sim \mathcal{N}(\mu_i, 1)$. Для каждой величины дано ровно *одно* наблюдение, то есть у нас есть вектор данных $\boldsymbol{x} = (x_1, \ldots, x_n)$, и мы хотим построить оценку $\hat{\boldsymbol{\mu}}$ для $\boldsymbol{\mu} = (\mu_1, \ldots, \mu_n)$.

Качество этой оценки мы будем устанавливать при помощи вполне естественной в данном случае квадратичной функции потерь:

$$L(\hat{\boldsymbol{\mu}}, \boldsymbol{\mu}) = \|\hat{\boldsymbol{\mu}} - \boldsymbol{\mu}\|^2 .$$

Обратите внимание, что функция потерь $L(\hat{\boldsymbol{\mu}}, \boldsymbol{\mu})$ — это случайная величина, которая принимает разные значения на разных наборах данных $\boldsymbol{x}$. Поэтому сравнивать потери нужно либо на конкретном датасете, что в данном случае не очень осмысленно — точка данных всего одна, и оценка $\hat{\boldsymbol{\mu}} = \boldsymbol{x}$ будет на таком «тренировочном наборе» всегда иметь нулевую ошибку, а никакого «тестового набора» нам не предлагается, — либо в ожидании.

В математической статистике это называется *функцией риска* (risk function): для оценки $\hat{\boldsymbol{\mu}}(D)$, построенной по набору данных D,

$$R(\hat{\boldsymbol{\mu}}, \boldsymbol{\mu}) = \mathbb{E}_D\left[L(\hat{\boldsymbol{\mu}}, \boldsymbol{\mu})\right] = \mathbb{E}_D\left[\|\hat{\boldsymbol{\mu}}(D) - \boldsymbol{\mu}\|^2\right]. \tag{4.29}$$

Это почти то же самое, что ожидаемая ошибка предсказания, которую мы подробно обсудим в подразделе 6.2.3; разница только в том, что функция риска обычно определяется как ошибка оценки параметра модели, а не как ошибка предсказания нового значения в модели обучения с учителем. В нашем случае выражение (4.29) — это не что-то сверхсложное, а всего лишь ожидание по $\boldsymbol{x} \sim \mathcal{N}(\boldsymbol{x}|\boldsymbol{\mu}, \mathbf{I})$, и ниже мы вполне справимся с тем, чтобы его подсчитать в нужных нам случаях.

В статистике говорят, что одна оценка $\hat{\boldsymbol{\mu}}$ *строго доминирует* (strictly dominates) над другой оценкой $\hat{\boldsymbol{\mu}}'$, если для всех истинных значений $\boldsymbol{\mu}$ функция риска у неё не больше, а иногда строго меньше:

$$\forall \boldsymbol{\mu}\ R(\hat{\boldsymbol{\mu}}, \boldsymbol{\mu}) \le R(\hat{\boldsymbol{\mu}}', \boldsymbol{\mu}), \quad \text{и} \quad \exists \boldsymbol{\mu}\ R(\hat{\boldsymbol{\mu}}, \boldsymbol{\mu}) < R(\hat{\boldsymbol{\mu}}', \boldsymbol{\mu}).$$

Например, для одного гауссиана с фиксированной дисперсией $x \sim \mathcal{N}(\mu, \sigma^2)$ вполне разумно было бы рассмотреть оценку $\hat{\mu} = x$. Её риск составляет

$$R(\hat{\mu}, \mu) = \mathbb{E}_x \left[\|x - \mu\|^2 \right] = \mathrm{Var}\,[x] = \sigma^2. \tag{4.30}$$

А риск, например, константной оценки $\hat{\mu} = c$, которая вообще не зависит от данных, составляет

$$R(c, \mu) = \mathbb{E}_x \left[\|c - \mu\|^2 \right] = \int_{-\infty}^{\infty} \|c - \mu\|^2 \, p(x) \mathrm{d}x = \|c - \mu\|^2 . \tag{4.31}$$

Это значит, в частности, что при $\mu = c$ у этой оценки нулевой риск: стоящие часы дважды в сутки всё-таки показывают правильное время.

Оценка $\hat{\boldsymbol{\mu}}$ называется *допустимой* (admissible), если никакая другая оценка не строго доминирует над ней, и *недопустимой* (inadmissible) в противном случае. Действительно, зачем нам рассматривать оценки, которые можно строго улучшить, ничего не потеряв?

На первый взгляд кажется, что понятие допустимой оценки — это барьер, который не очень сложно преодолеть. Например, в нашем случае константная оценка $\boldsymbol{c} = (c_1, c_2, \ldots, c_n)$, которая вообще не зависит от данных, будет допустимой: совершенно аналогично (4.31) мы видим, что её риск составляет

$$R(\boldsymbol{c}, \boldsymbol{\mu}) = \mathbb{E}_{\boldsymbol{x}} \left[\|\boldsymbol{c} - \boldsymbol{\mu}\|^2 \right] = \|\boldsymbol{c} - \boldsymbol{\mu}\|^2 ,$$

то есть в случае $\boldsymbol{\mu} = \boldsymbol{c}$ она окажется абсолютно права и её риск будет равен нулю. Значит, какая-то другая оценка $\hat{\boldsymbol{\mu}}$, которая могла бы строго доминировать над $\boldsymbol{c}$, должна тоже иметь нулевой риск в точке $\boldsymbol{\mu} = \boldsymbol{c}$:

$$R(\hat{\boldsymbol{\mu}}, \boldsymbol{c}) = \mathbb{E}_{\boldsymbol{x}} \left[\|\hat{\boldsymbol{\mu}}(\boldsymbol{x}) - \boldsymbol{c}\|^2 \right] = \int_{-\infty}^{\infty} \|\hat{\boldsymbol{\mu}}(\boldsymbol{x}) - \boldsymbol{c}\|^2 \, p(\boldsymbol{x}) \mathrm{d}\boldsymbol{x} = 0.$$

Но, поскольку $p(\boldsymbol{x}) > 0$ на всём пространстве $\mathbb{R}^n$, это возможно только в том случае, когда $\hat{\boldsymbol{\mu}}(\boldsymbol{x}) = \boldsymbol{c}$ (кроме, возможно, меры ноль), так что никакой другой оценки, строго доминирующей над $\boldsymbol{c}$, у нас не получится. Выходит, что всё, что нам нужно для допустимости, — это одна удачная точка.

Вернёмся теперь к исходной задаче — набору из n гауссианов с единичной дисперсией и неизвестными средними, для каждого из которых дана ровно одна точка. Для одного гауссиана, $n = 1$, всё получается ровно так, как мы и ожидали с самого начала: оценка $\hat{\mu} = x$ — это оценка максимального правдоподобия, которая является несмещённой, то есть $\mathbb{E}\,[\hat{\mu}] = \mu$, а её риск, как мы видели в (4.30), равен 1. Доказать, что это допустимая оценка, то есть что её нельзя улучшить для всех μ, не так просто, и мы этим заниматься не будем, но ещё в начале 1950-х годов появились доказательства этого вполне естественного факта [71, 222, 263].

Но в 1956 году появился результат совершенно неожиданный: Чарльз Штейн доказал [284, 558], что для $n = 2$ «обычная» оценка $\hat{\boldsymbol{\mu}} = \boldsymbol{x}$ ещё работает, то есть является допустимой, а вот для $n \geq 3$ уже работать перестаёт! Поскольку доказательства *не*допустимости — это, по сути, предъявления контрпримера, поиск

функции риска для какой-нибудь другой оценки, они обычно выглядят гораздо проще, чем доказательства допустимости, и пример Штейна мы вполне можем разобрать (см. также обсуждения этого парадокса в [166, 512]).

Оценка Джеймса — Штейна (James — Stein estimator), которая строго доминирует над $\hat{\boldsymbol{\mu}} = \boldsymbol{x}$, выглядит как

$$\hat{\boldsymbol{\mu}}_{\text{JS}}(\boldsymbol{x}) = \left(1 - \frac{n-2}{\|\boldsymbol{x}\|^2}\right)\boldsymbol{x}. \tag{4.32}$$

Иначе говоря, это вектор, который по-прежнему направлен в ту же сторону, что $\boldsymbol{x}$, но при этом немного «ужат» по сравнению с $\boldsymbol{x}$, его длина уменьшена умножением на число меньше единицы, которое зависит от длины вектора и от размерности.

Поскольку наши гауссианы независимы, риск обычной оценки максимального правдоподобия $\hat{\mu} = \boldsymbol{x}$ складывается из n рисков оценки $\hat{\mu}_i$ для каждого гауссиана, то есть её риск равен n. Давайте попробуем оценить риск оценки Джеймса — Штейна (4.32). По линейности ожидания и потому, что $\mathbb{E}_{\boldsymbol{x}}\left[\|\boldsymbol{x} - \boldsymbol{\mu}\|^2\right] = n$, можно расписать

$$\begin{aligned} R(\hat{\boldsymbol{\mu}}_{\text{JS}}, \boldsymbol{\mu}) &= \mathbb{E}_{\boldsymbol{x}}\left[\left\|\boldsymbol{x} - \frac{n-2}{\|\boldsymbol{x}\|^2}\boldsymbol{x} - \boldsymbol{\mu}\right\|^2\right] = \mathbb{E}_{\boldsymbol{x}}\left[\left\|(\boldsymbol{x} - \boldsymbol{\mu}) - (n-2)\frac{\boldsymbol{x}}{\|\boldsymbol{x}\|^2}\right\|^2\right] \\ &= n - 2(n-2)\mathbb{E}_{\boldsymbol{x}}\left[\frac{\boldsymbol{x}^\top(\boldsymbol{x} - \boldsymbol{\mu})}{\|\boldsymbol{x}\|^2}\right] + (n-2)^2\mathbb{E}_{\boldsymbol{x}}\left[\frac{1}{\|\boldsymbol{x}\|^2}\right]. \end{aligned} \tag{4.33}$$

По линейности математического ожидания

$$\mathbb{E}_{\boldsymbol{x}}\left[\frac{\boldsymbol{x}^\top(\boldsymbol{x} - \boldsymbol{\mu})}{\|\boldsymbol{x}\|^2}\right] = \sum_{i=1}^{n}\mathbb{E}_{\boldsymbol{x}}\left[\frac{x_i(x_i - \mu_i)}{\|\boldsymbol{x}\|^2}\right].$$

Теперь давайте подсчитаем один элемент суммы, например первый:

$$\mathbb{E}_{\boldsymbol{x}}\left[\frac{x_1(x_1 - \mu_1)}{\|\boldsymbol{x}\|^2}\right] = \frac{1}{(2\pi)^{n/2}}\int_{-\infty}^{\infty}\cdots\int_{-\infty}^{\infty}\frac{x_1(x_1 - \mu_1)}{\|\boldsymbol{x}\|^2}e^{-\frac{1}{2}\|\boldsymbol{x}-\boldsymbol{\mu}\|^2}\mathrm{d}x_1\ldots\mathrm{d}x_n.$$

Возьмём внутренний интеграл по частям; внутри интеграла по $\mathrm{d}x_1$ от экспоненты можно оставить только $e^{-\frac{1}{2}(x_1-\mu_1)^2}$:

$$\begin{aligned} \int_{-\infty}^{\infty}\frac{x_1(x_1 - \mu_1)}{\|\boldsymbol{x}\|^2}e^{-\frac{1}{2}(x_1-\mu_1)^2}\mathrm{d}x_1 &= \left[u = \frac{x_1}{\|\boldsymbol{x}\|^2}, v = -e^{-\frac{1}{2}(x_1-\mu_1)^2}\right] \\ &= \frac{x_1 e^{-\frac{1}{2}(x_1-\mu_1)^2}}{\|\boldsymbol{x}\|^2}\bigg|_{-\infty}^{\infty} + \int_{-\infty}^{\infty}\frac{\|\boldsymbol{x}\|^2 - 2x_1^2}{\|\boldsymbol{x}\|^4}e^{-\frac{1}{2}(x_1-\mu_1)^2}\mathrm{d}x_1, \end{aligned}$$

и в последнем выражении первое слагаемое обращается в ноль. Повторив то же самое для всех n интегралов, получим, что

$$\mathbb{E}_{\boldsymbol{x}}\left[\frac{\boldsymbol{x}^\top(\boldsymbol{x}-\boldsymbol{\mu})}{\|\boldsymbol{x}\|^2}\right] = \frac{1}{(2\pi)^{n/2}}\sum_{i=1}^{n}\int_{-\infty}^{\infty}\dots\int_{-\infty}^{\infty}\frac{\|\boldsymbol{x}\|^2-2x_i^2}{\|\boldsymbol{x}\|^4}e^{-\frac{1}{2}\|\boldsymbol{x}-\boldsymbol{\mu}\|^2}\mathrm{d}x_1\dots\mathrm{d}x_n =$$

$$= \sum_{i=1}^{n}\mathbb{E}_{\boldsymbol{x}}\left[\frac{\|\boldsymbol{x}\|^2-2x_i^2}{\|\boldsymbol{x}\|^4}\right] = n\mathbb{E}_{\boldsymbol{x}}\left[\frac{1}{\|\boldsymbol{x}\|^2}\right] - 2\mathbb{E}_{\boldsymbol{x}}\left[\frac{\sum_{i=1}^{n}x_i^2}{\|\boldsymbol{x}\|^4}\right] =$$

$$= (n-2)\mathbb{E}_{\boldsymbol{x}}\left[\frac{1}{\|\boldsymbol{x}\|^2}\right]. \quad (4.34)$$

Мы уже близки к победе — давайте подставим (4.34) в (4.33):

$$R(\hat{\boldsymbol{\mu}}_{\mathrm{JS}},\boldsymbol{\mu}) = n - 2(n-2)\mathbb{E}_{\boldsymbol{x}}\left[\frac{\boldsymbol{x}^\top(\boldsymbol{x}-\boldsymbol{\mu})}{\|\boldsymbol{x}\|^2}\right] + (n-2)^2\mathbb{E}_{\boldsymbol{x}}\left[\frac{1}{\|\boldsymbol{x}\|^2}\right] =$$

$$= n - (n-2)\mathbb{E}_{\boldsymbol{x}}\left[\frac{1}{\|\boldsymbol{x}\|^2}\right]. \quad (4.35)$$

Результат (4.35) говорит нам, что риск оценки Джеймса — Штейна $R(\hat{\boldsymbol{\mu}}_{\mathrm{JS}},\boldsymbol{\mu})$ равен n за вычетом некоторого зависящего от μ, но в любом случае всегда положительного числа. А это значит, что $\hat{\boldsymbol{\mu}}_{\mathrm{JS}}$ строго доминирует над оценкой $\boldsymbol{x}$, у которой риск, как мы знаем, равен n. Более того, получилось, что чем больше размерность n, тем сильнее нужно сдвигать оценку к нулю, и тем больше будет разница между результатами оценки максимального правдоподобия и оценки Джеймса — Штейна.

Обратите внимание, что все гауссианы здесь от начала до конца предполагаются независимыми. Иначе говоря, результат Штейна говорит следующее: если вы оцениваете одну величину с нормальным распределением, например рост мужчин в Финляндии, по одной точке, то лучшей оценкой будет рост встреченного вами мужчины. Но если вы одновременно оцениваете рост мужчин в Финляндии, среднюю температуру в Йоханнесбурге в октябре и число ожидаемых голов по системе xG, забитых за сезон лидером английской премьер-лиги по футболу, вам нужно сдвинуть все эти оценки вниз, чтобы уменьшить ожидаемую ошибку!

Этот результат выглядит настолько удивительно, что хочется убедиться в нём собственными глазами. Давайте проведём эксперимент:

```
def JS(x, n):
    return (1 - (n-2) / np.sum(x ** 2)) * x

def JS_experiment(mu, n, N=1000):
    xs = np.random.normal(mu, 1, size=(N, 5))
```

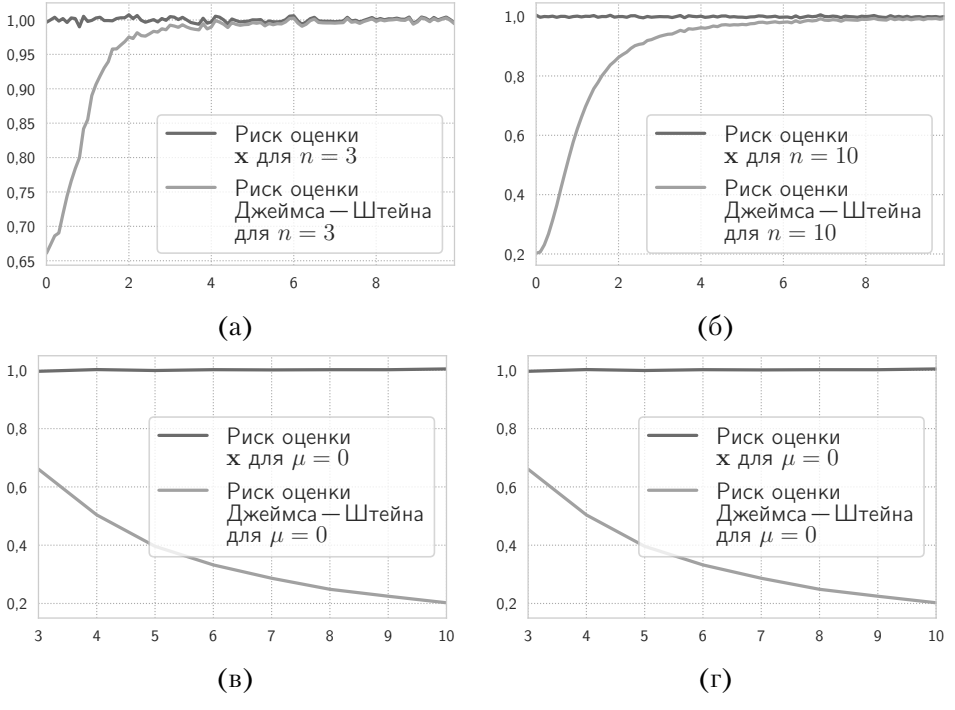
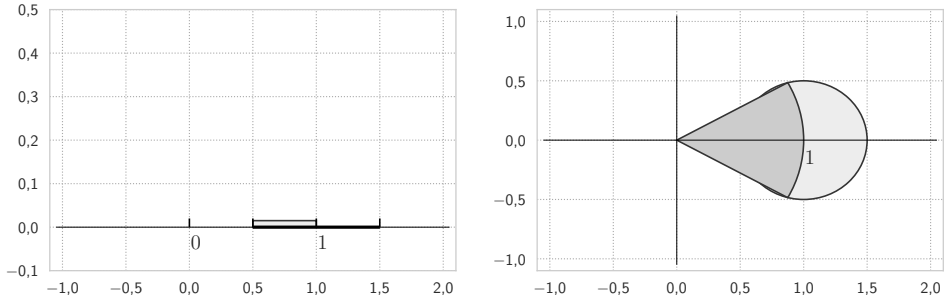


Рис. 4.41. Оценка Джеймса — Штейна: (а) зависимость среднего риска от μ для $n = 3$; (б) зависимость среднего риска от μ для $n = 10$; (в) зависимость среднего риска от n для $\mu = 0$; (г) зависимость среднего риска от n для $\mu = 2$

```
    js = np.array([JS(xs[i,:], n) for i in range(N)])
    avg_risk_ml = np.mean((xs - mu)**2)
    avg_risk_js = np.mean((js - mu)**2)
    return [avg_risk_ml, avg_risk_js]

mus = np.arange(0, 10, 0.1)
results = np.array([[JS_experiment(mu, n, N=50000) \
  for mu in mus ] for n in range(3, 10)])
```

В этом очень простом кусочке кода я реализовал оценку Джеймса — Штейна и провёл ряд экспериментов: для каждого n от 3 до 9 (меньше нет смысла: при $n = 2$ оценка Джеймса — Штейна совпадает с $\boldsymbol{x}$, а при $n = 1$ и вовсе начинает делать контрпродуктивные вещи) и для каждого $\mu \in [0, 10]$ с шагом 0,1 мы породили выборку из 10 000 векторов размера n из независимых нормально распределённых семплов со средним μ и дисперсией 1 (иначе говоря, здесь я выбрал для простоты $\boldsymbol{\mu} = (\mu, \ldots, \mu)$), а затем подсчитали среднеквадратичное отклонение



(а) $d = 1$ (б) $d = 2$

Рис. 4.42. Геометрическое объяснение оценки Джеймса — Штейна: (а) в размерности $d = 1$; (б) в размерности $d = 2$

истинного значения μ от этих семплов и от оценки Джеймса — Штейна. Некоторые результаты этого эксперимента показаны на рис. 4.41. И мы видим здесь на практике именно то, что предсказывала теория: для фиксированного n эффект понемногу пропадает с ростом μ, а для фиксированного μ становится сильнее с ростом n.

Результат Штейна произвёл небольшую революцию в математической статистике в 1950-е годы; теория допустимых и недопустимых оценок стала развиваться куда активнее, появились обобщения и продолжения этого результата, байесовские подходы к анализу таких оценок и так далее. Но можно ли всё-таки понять этот парадокс, выработать интуицию, почему вдруг нам нужно сдвигаться в сторону меньших значений оценки, даже несмотря на независимость всех участвующих случайных величин?

Конечно, можно. Во-первых, можно дать геометрическое объяснение [562]; оно имеет прямое отношение к проклятию размерности, которое мы обсудим в разделе 6.1. Представьте себе круг, который расположен на некотором расстоянии μ от центра координат; это проиллюстрировано на рис. 4.42, где $\mu = 1$, а радиус круга равен $\frac{1}{2}$. Какая доля точек в этом круге будет на расстоянии от начала координат, меньшем, чем расстояние от начала координат до его центра?

В размерности $d = 1$ — ровно половина, конечно (см. рис. 4.42, *а*); но уже в размерности $d = 2$, как видно на рис. 4.42, *б*, бóльшая часть точек лежит дальше, чем центр; с ростом размерности этот эффект будет усиливаться экспоненциально, но, с другой стороны, эффект будет ослабевать с увеличением расстояния от центра до начала координат. А теперь представьте, что это не просто круг, а линия уровня нормального распределения вокруг центра круга, из которого берутся точки нашей выборки. Геометрически говоря, оценка Джеймса — Штейна делает поправку на этот эффект: типичный элемент выборки будет лежать дальше от центра, чем истинное среднее, поэтому оценку среднего лучше сдвинуть

немного к началу координат. Ещё один способ посмотреть на этот эффект мы изучим в подразделе 6.2.4: оценка Джеймса — Штейна — это способ существенно сократить дисперсию оценки за счёт некоторого (меньшего) увеличения её смещения.

Во-вторых, оценку Джеймса — Штейна можно получить и из байесовских соображений, причём довольно просто [164, 165]. Представьте, что у средних μ_i есть нормальные априорные распределения с центрами в нуле и одинаковыми дисперсиями: $p(\mu_i) = \mathcal{N}(0, \sigma^2)$. Тогда мы можем найти апостериорное распределение (мы будем подробно обсуждать такой байесовский вывод для гауссианов в разделе 6.4, считайте это небольшим тизером)

$$p(\mu_i|x_i) \propto p(\mu_i)\, p(x_i|\mu_i) = \mathcal{N}\left(\mu_i \middle| 0, \sigma^2\right) \mathcal{N}(x_i|\mu_i, 1).$$

Переходя к логарифмам и убирая в константу все слагаемые, не зависящие от μ_i, получаем, что

$$\log p(\mu_i|x_i) = \text{Const} - \frac{1}{2\sigma^2}\mu_i^2 - \frac{1}{2}(x_i - \mu_i)^2 = \\ = \text{Const} - \frac{1}{2}\left(\left(1 + \frac{1}{\sigma^2}\right)\mu_i^2 - 2x_i\mu_i\right),$$

а значит, апостериорное распределение тоже будет нормальным:

$$p(\mu_i|x_i) = \mathcal{N}\left(\mu_i \middle| \left(1 - \frac{1}{1+\sigma^2}\right) x_i, 1 - \frac{1}{1+\sigma^2}\right), \tag{4.36}$$

так как

$$\left(1 + \frac{1}{\sigma^2}\right)^{-1} = 1 - \frac{1}{1+\sigma^2}.$$

Более того, распределение самого x_i в этой модели тоже ясно: мы сначала выбираем среднее μ_i вокруг нуля с дисперсией σ^2, а затем выбираем x_i вокруг полученного μ_i с дисперсией 1, и при этом (проверьте!) дисперсии складываются: $x_i \sim \mathcal{N}(x_i|0, 1+\sigma^2)$. Значит, $\mathbb{E}\left[x_i^2\right] = 1 + \sigma^2$.

Мы уже знаем, что распределение χ^2 с n степенями свободы — это сумма n стандартных гауссовских величин, а значит, такое распределение в нашем случае имеет случайная величина $\frac{1}{1+\sigma^2}\|\boldsymbol{x}\|^2$.

Соответственно, случайная величина $\frac{1+\sigma^2}{\|\boldsymbol{x}\|^2}$ имеет обратное распределение χ^2, математическое ожидание которого мы знаем:

$$\mathbb{E}\left[\frac{1+\sigma^2}{\|\boldsymbol{x}\|^2}\right] = \frac{1}{n-2}, \quad \text{то есть} \quad \mathbb{E}\left[\frac{n-2}{\|\boldsymbol{x}\|^2}\right] = \frac{1}{1+\sigma^2}.$$

Вот и получилось, что если мы не знаем значения σ^2, как в исходных условиях задачи, то вместо $\frac{1}{1+\sigma^2}$ хочется подставить имеющийся у нас семпл $\frac{n-2}{\|\boldsymbol{x}\|^2}$, и тогда (4.36) превратится в точности в оценку Джеймса — Штейна (4.32). Это тоже своего рода эмпирический Байес, подход, который мы обсудим в разделе 6.5.

У внимательного читателя на этом месте может возникнуть вопрос: почему же мы не используем оценку Джеймса — Штейна повсеместно в линейной регрессии? Ответ на этот вопрос прост: в некотором смысле используем! Гребневая регрессия, которую мы обсуждали в подразделе 4.2.2, делает примерно то же, что происходит в формуле (4.32): сжимает веса линейной регрессии, причём довольно похожим на оценку Джеймса — Штейна образом.

Чтобы это увидеть, нужно посмотреть на линейную регрессию со статистической точки зрения: если мы рассмотрим модель

$$y \sim \mathcal{N}\left(y \middle| \boldsymbol{w}_*^\top \boldsymbol{x}, \sigma^2\right)$$

и построим для неё оценку максимального правдоподобия

$$\boldsymbol{w}_{\text{ML}} = \left(\boldsymbol{X}^\top \boldsymbol{X}\right)^{-1} \boldsymbol{X}^\top \boldsymbol{t},$$

то $\boldsymbol{w}_{\text{ML}}$ будет несмещённой оценкой $\boldsymbol{w}$ с матрицей ковариаций $\sigma^2 \left(\boldsymbol{X}^\top \boldsymbol{X}\right)^{-1}$:

$$\boldsymbol{w}_{\text{ML}} \sim \mathcal{N}\left(\boldsymbol{w} \middle| \boldsymbol{w}_{\text{ML}}, \sigma^2 \left(\boldsymbol{X}^\top \boldsymbol{X}\right)^{-1}\right). \tag{4.37}$$

Это классический результат из статистики, но его можно увидеть и в наших результатах о байесовском выводе, если подставить в формулы (4.23) и (4.24) некорректное распределение с $\boldsymbol{\Sigma}_0^{-1} = \mathbf{O}$ в качестве априорного.

Получилось, что мы оцениваем нормально распределённую величину, и оценка Джеймса — Штейна (4.32) предлагает сжать предсказание (4.37) некоторым константным мультипликативным фактором. Если применить формулу (4.32) к (4.37), мы получим оценку

$$\boldsymbol{w}_{\text{JS}} = \left(1 - \frac{(d-2)\sigma^2}{\boldsymbol{w}_{\text{ML}}^\top \left(\boldsymbol{X}^\top \boldsymbol{X}\right) \boldsymbol{w}_{\text{ML}}}\right) \boldsymbol{w}_{\text{ML}}.$$

А классическая гребневая регрессия, с другой стороны, выдаёт оценку (4.13), в которой с результатом тоже происходит похожее преобразование, но контролируемое коэффициентом регуляризации α. В некотором смысле получается, что оценка Джеймса — Штейна — это «правильный» способ приготовить гребневую регрессию, ведь в формуле (4.13) мы не знаем α и вынуждены его так или иначе подбирать. Читатели, знакомые с методами глубокого обучения, могут увидеть

здесь параллели со взаимосвязью между L_2-регуляризацией в нейронных сетях (добавлением к функции ошибки обычного регуляризатора вида $\lambda\|\boldsymbol{w}\|^2$) и методом сжатия весов (weight decay), в котором вместо регуляризации веса умножаются на константу меньше единицы. Легко видеть, что для обычного стохастического градиентного спуска эти методы абсолютно эквивалентны, но про сжатие весов пришлось вспомнить, когда в широкое употребление вошли адаптивные алгоритмы градиентного спуска. Например, для очень популярного алгоритма Adam [306] есть не менее популярный вариант AdamW, который как раз модифицирует Adam так, чтобы результат соответствовал сжатию весов, а не добавлению L_2-регуляризатора [351, 352].

Здесь есть и более глубокие взаимосвязи и другие интересные результаты, но в них я уже углубляться не буду, а просто оставлю пару ссылок [152, 163]. Нам пора переходить к завершающему разделу этой главы.

4.5. Case study: линейная регрессия и коронавирус

Но все это умопомрачение рушилось перед доводами разума. Совершенно верно, слово «чума» было произнесено, совершенно верно, как раз в ту самую минуту просвистел бич и сразил одну или две жертвы. Ну и что же — ещё не поздно остановить его. Главное — это ясно осознать то, что должно быть осознано, прогнать прочь бесплодные видения и принять надлежащие меры. И тогда-де чума остановится: ведь человек не может представить себе чуму или представляет её неверно. Если она остановится, что всего вероятнее, тогда всё образуется. В противном случае люди узнают, что такое чума и нет ли средства сначала ужиться с ней, чтобы уж затем одолеть.

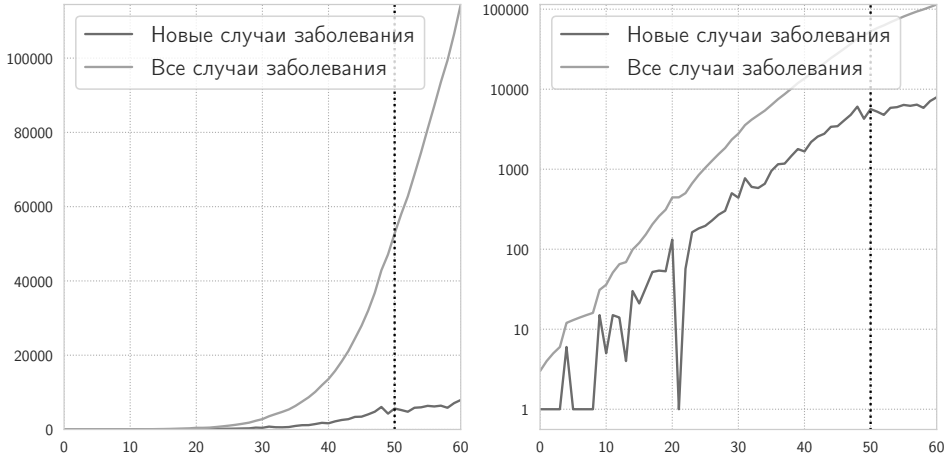
Альбер Камю. Чума

4.5.1. Постановка задачи

Линейная регрессия — одна из основных моделей машинного обучения. Как вы уже видели, она способна обучать даже нелинейные зависимости, хотя, конечно, самой по себе линейной регрессии для этого требуется заранее знать форму этих зависимостей. Более того, мы уже обсудили, как провести полный байесовский вывод в линейной регрессии, то есть построить предсказательное распределение и оценить, насколько точны были наши предсказания. Пора проверить силы линейной регрессии на деле.

В этом разделе мы рассмотрим один конкретный пример, который, думаю, свеж в памяти у многих читателей: попробуем проанализировать эпидемию коронавируса COVID-19, которая существенно изменила нашу жизнь в 2020 году. Представьте себе, что сейчас май 2020 года и вы принимаете решения о том, как противостоять эпидемии COVID-19 в России. Для этого у вас есть, по сути, только данные за последние два месяца — эпидемия пришла в Россию в начале марта[1]. Вот они, изображены на рис. 4.43, *а*; там показаны и новые случаи заболевания, и общее их число. Я взял посуточные данные (более точных нет, да и бессмысленны они, скорее всего) начиная с 3 марта 2020 года, когда число заболевших в России впервые превысило 2 и начало расти. Пунктирная линия на рис. 4.43 отделяет первые 50 дней от последних десяти — в дальнейшем мы будем использовать первые 50 точек данных для обучения, а на последних десяти тестировать. На случай, если сайт с информацией о COVID-19 вдруг изменится, приведу данные здесь, это всего лишь 60 чисел (я привожу число новых случаев,

[1] Данные я взял с сайта https://ourworldindata.org/coronavirus-source-data.



(а) Линейная шкала (б) Логарифмическая шкала

Рис. 4.43. Данные о новых случаях заражения и общем числе заражённых

заменив для удобства нули на единицы, а общее число случаев можно получить как их кумулятивную сумму, начиная с 2 в первый день):

```
x_all = np.array([ 1, 1, 1, 1, 6, 1, 1, 1, 1, 15, 5, 15, 14, 4,\
  30, 21, 33, 52, 54, 53, 132, 1, 57, 163, 182, 196, 228, 270, 302,\
  501, 440, 771, 601, 582, 658, 954, 1154, 1175, 1459, 1786, 1667,\
  2186, 2558, 2774, 3388, 3448, 4070, 4785, 6060, 4268, 5642, 5236,\
  4774, 5849, 5966, 6361, 6198, 6411, 5841, 7099, 7933])

x_all_tot = 2+np.cumsum(x_all)
x_train, x_train_tot = x_all[:50], x_all_tot[:50]
x_test, x_test_tot = x_all[50:], x_all_tot[50:]
```

Первое, что можно сразу же заметить, взглянув на рис. 4.43, *а*, — данные выглядят как практически идеальная экспонента. Это впечатление легко подтвердить, нарисовав те же кривые на логарифмической шкале, как на рис. 4.43, *б*: хотя к концу двухмесячного интервала кривые начинают немного загибаться вниз, всё равно выглядит, как будто они будут хорошо описываться прямыми линиями.

4.5.2. Линейная регрессия на логарифмической шкале

Поэтому вполне естественно с прямых линий и начать. Давайте обучим линейную регрессию, использовав логарифм числа новых случаев и логарифм общего числа случаев как целевые переменные. Теоретически здесь не должно быть разницы, какую из двух переменных брать, ведь экспонента — это как раз функция, которая сохраняется после взятия производной:

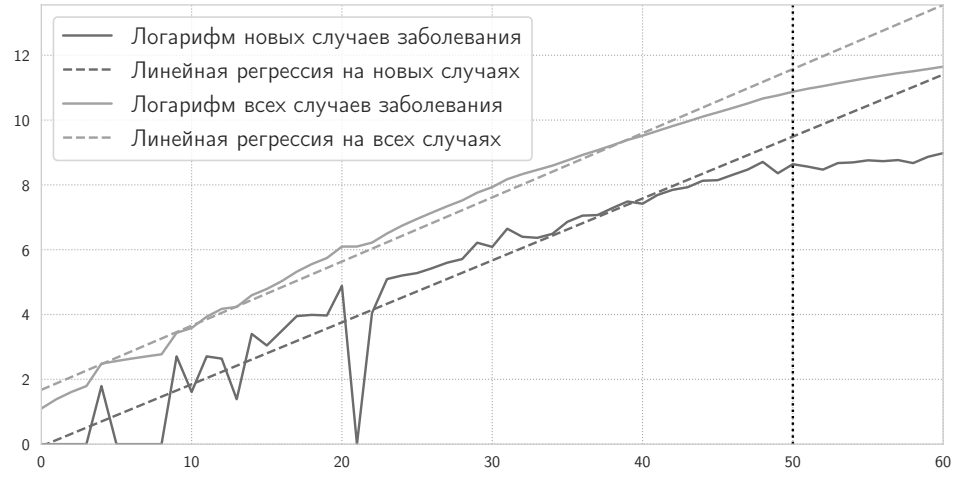


Рис. 4.44. Линейная регрессия на логарифмической шкале

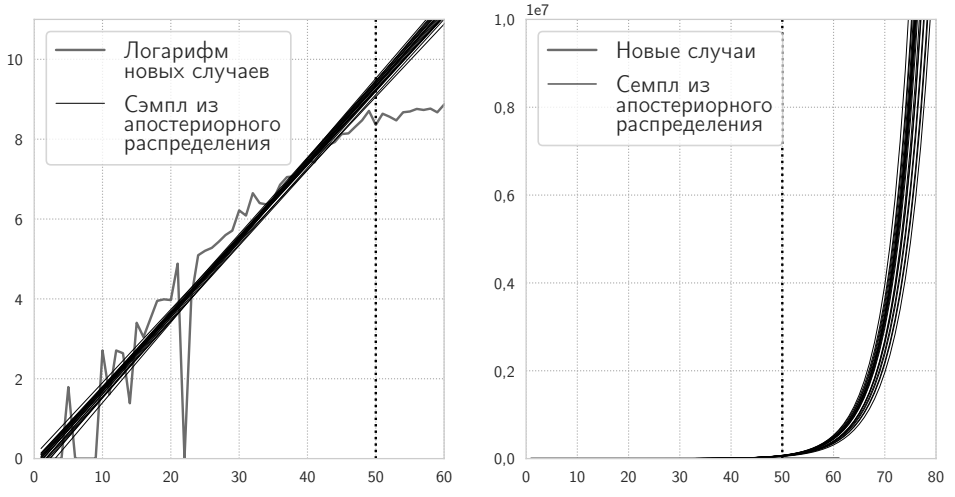
```
xs = np.arange(1, len(x_train)+1).reshape(-1, 1)
m = linear_model.LinearRegression(fit_intercept=True).fit( xs, np.log(x_train))
m_tot = linear_model.LinearRegression(fit_intercept=True).fit( \
  xs, np.log(x_train_tot))
```

Результат (сразу в логарифмах) показан на рис. 4.44. Хотя опять же видно, что кривые начинают загибаться вниз в конце рассматриваемого интервала, в целом выглядит так, как будто наши предположения подтверждаются. Линейные регрессии хорошо описывают данные; они ожидаемо идут параллельно друг другу с одинаковыми наклонами: у меня получилось, что коэффициенты w_1 у регрессий на новых случаях и всех случаях заражения лежат между 0,19 и 0,20.

Более того, мы можем провести байесовский вывод, как в разделе 4.4, и найти апостериорное распределение на веса этих линейных регрессий. В дальнейшем будем обучаться только на числе новых случаев `x_train`, ведь мы уже убедились, что общее число случаев нам ничего нового не скажет.

Начнём с достаточно широких априорных распределений и будем постепенно применять процедуру `bayesian_update`:

```
def bayesian_update(mu, sigma, x, y, sigma_noise=.5):
    x_matrix = x.reshape(1, -1)
    sigma_n = np.linalg.inv(np.linalg.inv(sigma) +\
      (1 / (sigma_noise ** 2)) * np.matmul(np.transpose(x_matrix), x_matrix) )
    mu_n = np.matmul(sigma_n, np.matmul(np.linalg.inv(sigma),\
      np.transpose(mu)) + (1 / (sigma_noise ** 2)) * \
      np.matmul(np.transpose(x_matrix), np.array([y])))
    return mu_n, sigma_n
```



(а) Логарифмическая шкала (б) Линейная шкала

Рис. 4.45. Семплы из апостериорного распределения линейной регрессии, построенной на логарифмической шкале

```
mu_0 = np.zeros(2)
sigma_0 = 10 * np.identity(2)
mu, sigma = mu_0, sigma_0
for x,y in zip(np.arange(1, len(x_train)+1), np.log(x_train)):
    mu, sigma = bayesian_update( mu, sigma, np.array([1, x]), y )
```

В результате получатся приблизительно такие значения:

$$\boldsymbol{\mu} = \begin{pmatrix} -0{,}255166 \\ 0{,}191113 \end{pmatrix}, \qquad \Sigma = \begin{pmatrix} 0{,}02057 & -0{,}000611 \\ -0{,}000611 & 0{,}000024 \end{pmatrix}.$$

Апостериорное распределение получается весьма точным, ведь 50 точек — это очень много данных для обучения прямой в размерности один, то есть для обучения двух параметров. И если мы посемплируем несколько прямых из этого апостериорного распределения, пучок получится очень плотным; он показан на рис. 4.45, *а*.

Казалось бы, всё хорошо, модель обучена... но есть ли в ней хоть какой-нибудь смысл? Прямые линии, которые мы нарисовали, стремительно уходят на бесконечность и очень быстро превышают и население России, и население всего мира, и число атомов во Вселенной...

Для иллюстрации рис. 4.45, *б* показывает те же самые семплы из апостериорного распределения, но уже в линейной шкале и продолженные ещё буквально

на два десятка точек вправо. Согласитесь, что экспонента выглядит всё-таки чересчур безжалостно. С ней ещё и, собственно, ничего не поделаешь: даже если какие-то меры смогут уменьшить показатель экспоненты (вес w_1), всё равно, пока он положительный, ничего, по сути, не изменится.

К счастью, настоящие эпидемии не развиваются по экспоненте; как минимум, число людей, подверженных новой болезни, не бесконечно, и эпидемия неизбежно выйдет на плато насыщения. Общее число случаев заражения в реальных эпидемиях имеет *сигмоидальную* форму, то есть это кривая, начинающаяся от нуля, монотонно возрастающая и асимптотически приближающаяся к некоторому уровню насыщения. Именно этот уровень нам важнее всего было бы узнать: он показывает ожидаемое число людей, которые подвергнутся действию эпидемии, если всё будет продолжаться как сейчас.

Мы подробно поговорим о сигмоидах, в том числе о главном их представителе, логистическом сигмоиде, в разделе 5.2, а сейчас нам нужно просто выбрать из таких кривых одну, которую мы смогли бы обучить.

4.5.3. Обучаем функцию распределения гауссиана

Хотя общая сигмоидальная форма нужной кривой сомнений не вызывает, из общих соображений совершенно не очевидно, какой именно нужно взять сигмоид. Поэтому давайте положимся на авторитет: работу [251] со статистическим анализом COVID-19, которая, как говорилось в прессе весной 2020 года, стала одним из основных источников для принятия решений в правительстве США. Статья эта вышла в конце марта 2020 года, и данных там примерно столько же, сколько у нас в этом примере (в статье анализируются данные сразу во многих разрезах для разных территорий и так далее, но это как раз не очень принципиально).

В качестве сигмоида, описывающего динамику эпидемии, в работе [251] предлагается использовать функцию распределения одномерного гауссиана:

$$\Psi(a) = \int_{-\infty}^{a} \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2\sigma^2}(x-\mu)^2} \mathrm{d}x.$$

Функции распределения вообще представляют собой неисчерпаемый источник сигмоидальных функций, ведь все они монотонно возрастают от нуля к единице. И функция распределения именно гауссиана тоже прекрасно подходит на роль сигмоида — в главе 5 мы обсудим пробит-регрессию, которая тоже использует её в таком качестве.

Правда, эпидемиологического обоснования для этой формы сигмоида я в работе [251] не нашёл; авторы пишут так: «Другие сигмоидальные функции (альтернативные Ψ) рассматривались, но не смогли описать данные столь же хорошо». Что ж, будем использовать лучшее из того, что есть.

Но как обучить параметры такой сложной функции?

Во-первых, заметим, что нам нужен не сам гауссиан, а что-то ему пропорциональное, ведь мы хотим обучить как раз константу насыщения сигмоида. Во-вторых, эту мультипликативную константу можно тоже внести в показатель экспоненты как свободный член, и там получится просто квадратичная функция общего вида. Поэтому давайте перепишем интересующую нас функцию следующим образом:

$$\text{TotalCases}(n) \sim \int_{-\infty}^{n} e^{w_0 + w_1 x + w_2 x^2} \mathrm{d}x, \tag{4.38}$$

и в этой модели нам нужно обучить параметры $\boldsymbol{w}$. В [251] используется чуть более сложная, но, по большому счёту, эквивалентная параметризация.

Всё равно выглядит сложно, но теперь уже проще увидеть, как к такой модели подступиться. Функция из (4.38) превратится в самую обычную экспоненту, если взять от неё производную. Но ведь для производной у нас тоже есть данные — мы можем считать её дискретным аналогом ежедневное число новых случаев! Поэтому модель (4.38) после взятия производной и логарифмирования превращается в уже знакомую нам линейную регрессию с квадратичными признаками:

$$\text{NewCases}(n) \sim e^{w_0 + w_1 n + w_2 n^2}, \quad \log \text{NewCases} \sim w_0 + w_1 n + w_2 n^2. \tag{4.39}$$

Давайте обучим такую модель:

```
mu_0 = np.zeros(3)
sigma_0 = 10 * np.identity(3)
mu, sigma = mu_0, sigma_0
for x,y in zip(np.arange(1, len(x_train)+1), np.log(x_train)):
    mu, sigma = bayesian_update( mu, sigma, np.array([ 1, x, x ** 2 ]), y)
```

Регрессия с квадратичными признаками тоже никак не может пожаловаться на недостаток данных: 50 точек и для трёх параметров более чем достаточно. Так что если мы визуализируем результат, как показано на рис. 4.46, мы увидим очень плотный пучок парабол, которые, казалось бы, все очень неплохо описывают данные и даже начинают загибаться вниз, как того требует тестовое множество. На первый взгляд всё выглядит очень хорошо: в апостериорном распределении будет практически невозможно встретить $w_2 \geq 0$, то есть все эти параболы уходят на минус бесконечность, и после взятия экспоненты и интегрирования (то есть взятия кумулятивной суммы) они действительно превратятся в сигмоиды, как мы и хотели.

4.5.4. Анализ результатов и выводы

Но действительно ли этот анализ позволяет сделать какие бы то ни было выводы? Давайте проанализируем, какие получаются предсказания о переменной, которая нас больше всего интересует: об общем ожидаемом числе людей, которые заболеют в течение всей эпидемии. Именно от этого числа зависит, нужно ли

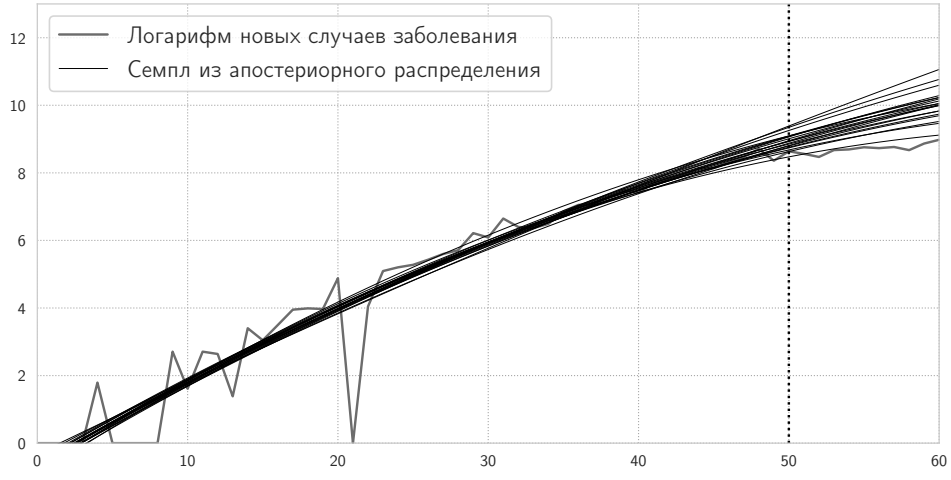


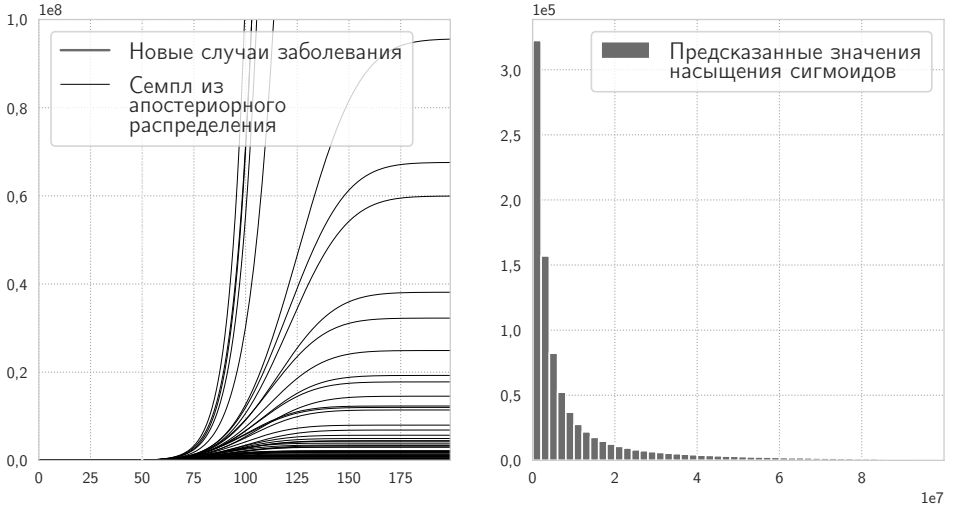
Рис. 4.46. Линейная регрессия с квадратичными признаками на логарифмической шкале

принимать серьёзные меры, которые нарушат жизнь людей и будут стоить дорого, или можно «перетерпеть» эпидемию, а существующая система здравоохранения справится и так.

Сделаем это довольно прямолинейно: посемплируем много кривых из апостериорного распределения и построим гистограмму кумулятивных сумм. В коде ниже я взял миллион семплов и довёл каждый из них до дня $T + 2000$, чтобы все сигмоиды уже наверняка вышли на насыщение:

```
xs, N = np.arange(2000), 1000000

w_sample = np.random.multivariate_normal(mu, sigma, N)
cumul_preds = np.array([ np.cumsum(\
  np.exp(np.dot(w, np.array([ 1, x, x ** 2 ])))  ) for w in w_sample ])
total_cases = np.array([cp[-1] for cp in cumul_preds if cp[-1] < 100000000])
```

Результат показан на рис. 4.47: слева показаны несколько примеров получающихся сигмоидальных кривых, а справа — гистограмма значений насыщения. Выглядит логично... но опять, к сожалению, дисперсия у этого предсказания получается безнадёжно большой. В моём эксперименте медиана предсказанных значений получилась около 3,3 миллиона человек, 90-й процентиль — около 30 миллионов, а 99-й процентиль превышает 85 миллионов потенциально заболевших. Что делать с такими предсказаниями? Три миллиона человек в течение всей эпидемии наверняка удастся разместить в уже существующих больницах,



(а) Несколько примеров (б) Гистограмма значений насыщения

Рис. 4.47. Семплы из апостериорного распределения линейной регрессии с квадратичными признаками

если учесть, что подавляющее большинство из них будут болеть легко или вообще бессимптомно, но 30 или тем более 80 миллионов заболевших — это катастрофа вселенских масштабов...

А если попробовать использовать эти предсказания в предположении, что мы хотим принять те или иные меры по предотвращению эпидемии, то как они помогут нам решить, какие именно меры нужны? Наш анализ выдаёт некоторые абстрактные параметры $\boldsymbol{w}$, которые по имеющимся данным можно оценить достаточно точно... но как узнать, насколько они изменятся от локдауна той или иной степени жёсткости или каких-то других мер? Кажется, что наш анализ всё-таки чего-то недоговаривает.

Для задачи распространения эпидемий можно построить и более сложную, «настоящую» статистическую модель, которая мотивирует и появление сигмоидальных функций в этом контексте, и позволит провести байесовский вывод для обучения своих параметров (см., например, работу [162]); такой вывод потребует методов, выходящих за рамки нашей книги, поэтому к этим моделям мы вернёмся позже. Но, конечно, и такая модель будет всего лишь упрощением, которому для реального применения потребуется ещё много разных дополнений и исследований, уже не лежащих в области машинного обучения. Так что вывод из этого примера получается, как говорят по-английски, bittersweet: с одной стороны, мы увидели, что машинное обучение не всемогуще и иногда (на самом деле не так уж редко) однозначный вывод сделать просто не получается. Но, с другой

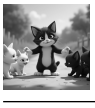
стороны, — и это кажется мне очень позитивным наблюдением — машинное обучение, в котором мы понимаем сделанные предположения и даже в этих предположениях стараемся строить полноценные предсказательные распределения, не станет обманывать нас и вселять ложную уверенность. При правильном подходе к моделированию мы сможем распознать чрезмерно большую дисперсию предсказаний и не станем слишком полагаться на выводы модели. Как вы увидели на этом примере, для этого даже не обязательно иметь в точности правильные вероятностные предположения, хотя оставшаяся неопределённость формы сигмоида (мы выбрали её скорее из соображений удобства) делает наши выводы *ещё* менее надёжными.

На этом несколько противоречивом примере мы заканчиваем разговор о линейной регрессии. В следующей главе нас ждёт вторая, пожалуй, даже более важная и часто встречающаяся постановка задач обучения с учителем: задачи классификации.

Глава 5
Классификация

TL;DR

Мы начинаем с раздела 5.1, в котором обсуждаем, как должны быть устроены разделяющие поверхности, и в качестве примера рассматриваем классический геометрический метод классификации — дискриминант Фишера. Затем переходим к вероятностям, где начинаем с оптимального байесовского классификатора, рассматриваем гауссовские модели для классов и приходим к линейному и квадратичному дискриминантному анализу.



Раздел 5.2 посвящён одной из важнейших моделей классификации: логистической регрессии. Мы обсудим, откуда она берётся, каков её вероятностный смысл и как её обучать градиентным спуском, а также какие ещё сигмоиды, кроме логистического, можно было бы использовать.

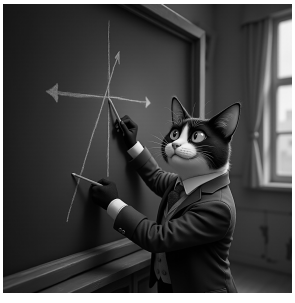
Полный байесовский вывод и предсказательное распределение в логистической регрессии заслуживают отдельного раздела 5.3. В нём мы введём и подробно обсудим метод лапласовских аппроксимаций, а затем применим его к логистической регрессии.

Раздел 5.4 — это развёрнутый пример задачи классификации на классическом наборе данных, известном как «ирисы Фишера». Мы обсудим историю и свойства этого датасета, а затем сравним разные классификаторы на нём и на его подмножествах.

Раздел 5.5 посвящён нескольким общим замечаниям, которые важны для задач классификации независимо от конкретной модели: что делать с несбалансированными классами, как калибровать классификаторы, откуда брать вероятности, если их нет, и чем отличаются проспективные и ретроспективные исследования.

Наконец, в разделе 5.6 мы вернёмся к вероятностной сути классификации и рассмотрим простейшую полноценную порождающую модель — наивный Байес — а также получающийся из неё наивный байесовский классификатор, который мы затем применим к категоризации текстов.

5.1. Постановка задачи, геометрия и вероятности



На древних страницах [энциклопедии под названием «Небесная империя благодетельных знаний»] написано, что животные делятся на а) принадлежащих Императору, б) набальзамированных, в) прирученных, г) сосунков, д) сирен, е) сказочных, ж) отдельных собак, з) включённых в эту классификацию, и) бегающих как сумасшедшие, к) бесчисленных, л) нарисованных тончайшей кистью из верблюжьей шерсти, м) прочих, н) разбивших цветочную вазу, о) похожих издали на мух.

Хорхе Луис Борхес. Аналитический язык Джона Уилкинса (пер. Е. Лысенко)

Наиболее заметный пример первого рода являет Аристотель, который своей диалектикой испортил естественную философию, так как построил мир из категорий... Он всегда больше заботился о том, чтобы иметь на всё ответ и словами высказать что-либо положительное, чем о внутренней истине вещей.

Фрэнсис Бэкон. Новый Органон (пер. Н. А. Фёдорова)

5.1.1. Геометрия классификации: разделяющие поверхности

В разделе 1.3 мы говорили о том, что такое задача классификации: это задача обучения с учителем, в которой целевая переменная дискретна, а не непрерывна. Проще говоря, в задачах классификации мы хотим не предсказать какое-то число, а определить (классифицировать) поданный на вход пример в один из нескольких классов: отличить котиков от пёсиков на фотографиях, выделить фонему, которая реализуется в данном звуковом отрывке, понять, положительный или отрицательный отзыв написали к нашему товару, и так далее.

Такая постановка на первый взгляд кажется даже проще, чем задача регрессии: вместо того чтобы предсказывать число, что, скорее всего, невозможно сделать точно, в классификации нужно всего лишь не ошибиться при выборе из нескольких вариантов. Но в задачах классификации возникают свои сложности. Первая из них — как определить функцию ошибки? В задачах регрессии мы часто можем без проблем использовать квадрат отклонения, который мотивирован предположением о нормальном распределении ошибки (вспомните раздел 4.3). Эта функция ошибки является одновременно и нашей конечной целью при решении задачи регрессии, и очень удобной функцией для оптимизации: она выпуклая и даже допускает точное решение.

Конечная цель в задаче классификации, разумеется, состоит в том, чтобы научиться как можно точнее классифицировать входящие примеры, то есть минимизировать *ошибку классификации* (classification error). Если на вход к нам поступает набор данных $D = \{(\boldsymbol{x}_n, \boldsymbol{y}_n)\}_{n=1}^N$, где в каждой паре $\boldsymbol{x}$ — это признаки входа, а $\boldsymbol{y}$ — это ответ, то ошибку классификации обученной нами функции f на этом датасете можно определить как

$$\text{CE}(f) = \frac{1}{N}\sum_{i=1}^{N} [\![f(\boldsymbol{x}_i) = y_i]\!] .$$

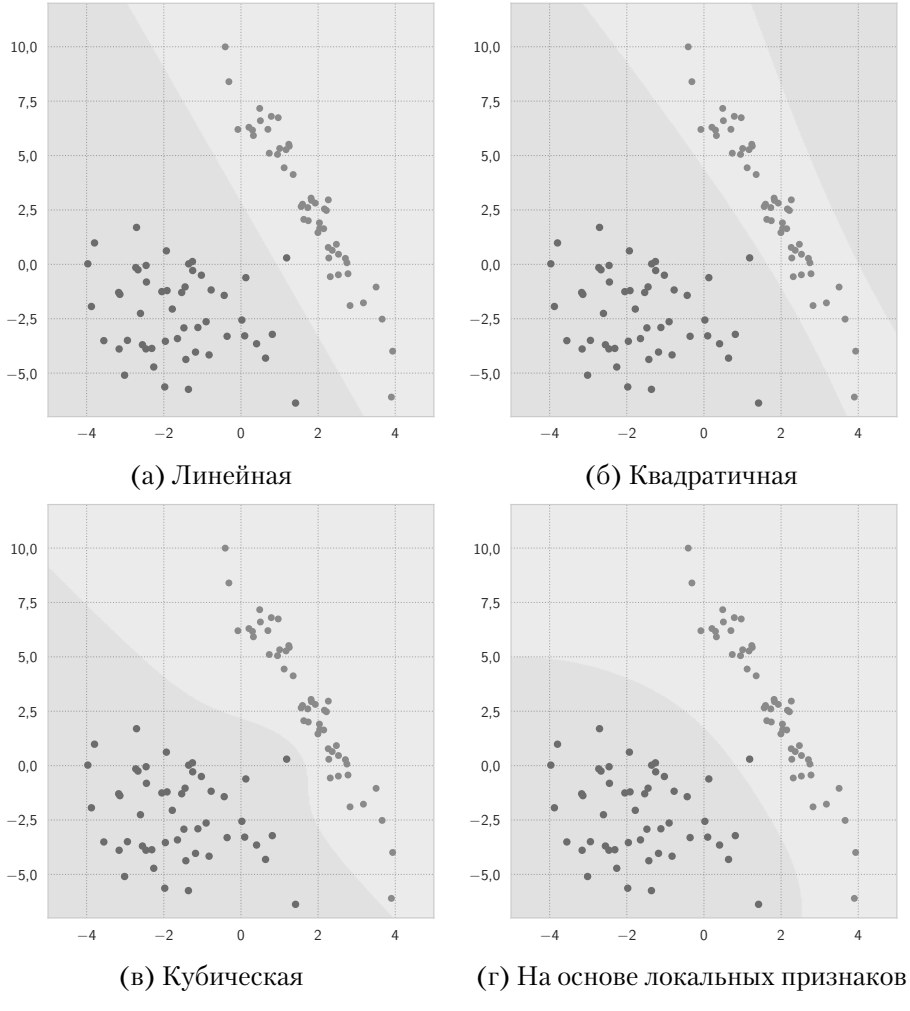
Но вот незадача: оптимизировать такую функцию напрямую будет практически невозможно! Дело в том, что эта функция кусочно-постоянная: если ответ, выданный функцией f, не меняется ни на одном из конечного числа примеров, то и $\text{CE}(f)$ не изменится. А кусочно-постоянные функции оптимизировать совершенно непонятно как: ни градиентный спуск, ни более сложные методы не работают, потому что градиент везде, где вообще существует, строго равен нулю.

Поэтому для классификации нам нужно придумать какую-то другую целевую функцию, которую можно было бы оптимизировать. По сути, методы классификации различаются между собой тем, какие именно целевые функции они используют, и в этой главе мы ещё вернёмся к этой мысли. Но сначала давайте обсудим геометрическую сторону вопроса. Конечно, в этой главе мы лишь наметим основные классические подходы к решению задач классификации. Гораздо подробнее обо всех этих методах можно прочесть, например, в книгах [157, 201, 246], а также в наших основных источниках [66, 419]. Но надеюсь, что некоторую интуицию выработать получится.

Каждый раз, когда вы строите классификатор, то есть функцию, делающую предсказания о том, какому классу принадлежат те или иные точки пространства, фактически вы делите пространство на части, соответствующие разным классам. Такое разделение можно задавать при помощи функций, предсказывающих вероятности или меры принадлежности классам, а можно — при помощи *разделяющих поверхностей* (decision boundaries), границ между классами.

Разделяющие поверхности могут иметь разную форму. На рис. 5.1 показаны разные разделяющие поверхности для одних и тех же двух классов: линейная, квадратичная, кубическая и «следующая за данными», то есть построенная на основе локальных гауссовских признаков. Думать о классификации в терминах разделяющих поверхностей зачастую удобнее и нагляднее, и понимание их геометрии необходимо для того, чтобы развивать методы классификации, поэтому классификацию невозможно обсуждать без её геометрического смысла.

Главный пример здесь — *линейная* разделяющая поверхность. Как вы видели в подразделе 4.1.4, к линейным моделям часто сводятся и нелинейные зависимости, и в классификации получится та же история. Поэтому давайте обсудим геометрию линейных разделяющих поверхностей чуть более подробно.



(а) Линейная (б) Квадратичная

(в) Кубическая (г) На основе локальных признаков

Рис. 5.1. Примеры разделяющих поверхностей

Всё, что нам понадобится, проиллюстрировано на рис. 5.2, *а*:

- линейная разделяющая поверхность — это гиперплоскость в пространстве признаков, то есть множество точек, определяющееся линейным уравнением $\boldsymbol{w}^\top \boldsymbol{x} + w_0 = 0$, где $\boldsymbol{w}$ — это вектор нормали к гиперплоскости;
- эта гиперплоскость делит пространство на две части: с одной стороны $\boldsymbol{w}^\top \boldsymbol{x} + w_0 > 0$, с другой — $\boldsymbol{w}^\top \boldsymbol{x} + w_0 < 0$; знак выражения $\boldsymbol{w}^\top \boldsymbol{x} + w_0$ и будет правилом классификации, если нужен бинарный ответ, а не вероятности принадлежности классам;

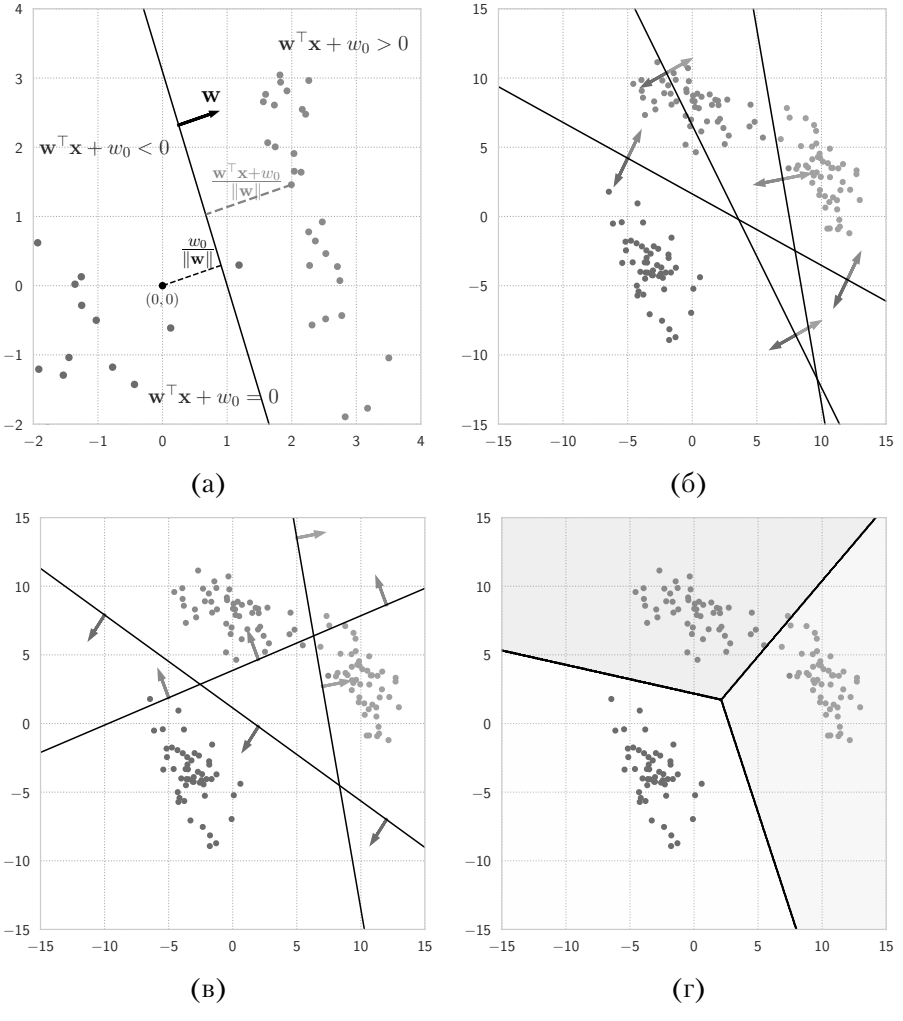


Рис. 5.2. Геометрия линейных разделяющих поверхностей: (а) уравнения гиперплоскости; (б) разделение на три класса методом «каждый против каждого»; (в) методом «каждый против остальных»; (г) на основе линейной оценочной функции

- в машинном обучении иногда потребуется понимать, как найти расстояние от точки $\boldsymbol{x}$ до гиперплоскости; для этого нужно подставить $\boldsymbol{x}$ в уравнение гиперплоскости, а результат нормировать на длину вектора нормали: $d(\boldsymbol{x}, \boldsymbol{w}^\top\boldsymbol{x} + w_0 = 0) = \frac{1}{\|\boldsymbol{w}\|}\left(\boldsymbol{w}^\top\boldsymbol{x} + w_0\right)$; например, расстояние от гиперплоскости до начала координат равно $\frac{w_0}{\|\boldsymbol{w}\|}$.

Здесь, к сожалению, даёт о себе знать то досадное обстоятельство, что рисовать по-настоящему наглядные картинки у меня получается только в двумерном пространстве. Поэтому все рисунки в этой главе выполнены в размерности 2, а значит, на всех рисунках прямые (аффинные подпространства размерности 1) и гиперплоскости (аффинные подпространства *ко*размерности 1) — это одно и то же. Придётся обычным текстом попросить вас держать в голове, что разделяющие поверхности — это поверхности, многообразия размерности $d-1$, а, скажем, векторы нормали к ним — это всегда обычные векторы. Путать эти два типа объектов, несмотря на то что в двумерном случае они совпадают, отнюдь не рекомендуется, и в тексте я постараюсь продолжать называть гиперплоскости гиперплоскостями, даже когда на самом деле это обычные прямые.

5.1.2. Геометрия классификации для нескольких классов

Следующая задача, геометрический смысл которой оказывается весьма любопытным, — это обобщение линейных разделяющих поверхностей на несколько классов. Когда бинарная классификация уступает место K различным классам $C_1, \ldots, C_K$, становится непонятно: как теперь проводить эти самые линейные разделяющие поверхности? На ум приходят два варианта:

- можно провести $K(K-1)$ гиперплоскости, каждая из которых разделяет одну пару классов C_k и C_l; этот подход обычно называют *one-vs-one*: например, в стандартном пакете `scikit-learn` у некоторых классификаторов есть параметр `multi_class` или `decision_function_shape` со значением `'ovo'`; пример такого разделения показан на рис. 5.2, *б*, где цветные стрелочки показывают, какие классы находятся по обе стороны той или иной гиперплоскости;
- а можно провести K гиперплоскостей, каждая из которых отделяет один из классов C_k от точек из классов $C_1, \ldots, C_{k-1}, C_{k+1}, \ldots, C_K$; этот подход известен как *one-vs-rest*, и соответствующее значение параметра определяется как `'ovr'`; пример разделения one-vs-rest показан на рис. 5.2, *в*, где у каждой гиперплоскости теперь остались цветные стрелочки только в одном направлении, а другая сторона соответствует точкам всех остальных классов.

Можно порассуждать о сравнительных характеристиках этих подходов. Например, в большинстве случаев one-vs-rest выглядит предпочтительнее, ведь нужно обучать всего K поверхностей, а не $K(K-1)$, что и вовсе квадратичное число. Но бывают подходы к классификации (например, метод опорных векторов, о котором мы не будем говорить в этой книге), которые плохо масштабируются с числом точек данных N, и тогда может оказаться даже эффективнее обучать квадратичное число поверхностей методом one-vs-one, ведь данные для каждой поверхности будут состоять только из двух классов. Ещё один случай, когда one-vs-one может оказаться предпочтительнее, — это случай, в котором ваш классификатор плохо умеет обрабатывать классы с сильно различающимися размерами (числом точек), ведь в методе one-vs-rest классы в бинарных задачах наверняка получатся очень разного размера.

Но, как видно на рис. 5.2, *б*, *в*, в обоих подходах возникают и принципиальные проблемы, которые не сводятся к вычислительной сложности. На рисунках есть весьма значительные части, которые непонятно как классифицировать! В подходе one-vs-one на рис. 5.2, *б* мы видим подозрительный треугольник в центре графика, в котором реализуется парадокс Кондорсе[1]: первый класс побеждает второй, второй побеждает третий, а третий побеждает первый. Какие-то варианты выхода из этого положения предложить можно, но концептуально неясно, какие из них будут лучше других. Треугольник при других вариантах взаимного расположения классов может превратиться и в более заметную часть плоскости. А на рис. 5.2, *в* видно, что дела в методе one-vs-rest обстоят ещё хуже: возникают целые большие куски пространства, уходящие на бесконечность, где сразу два класса побеждают в классификации, и между ними, опять же, совершенно непонятно как выбирать.

Было бы замечательно, если бы наша линейная разделяющая поверхность выглядела как на рис. 5.2, *г*: если три разделяющих луча сходятся в одной точке, то все геометрические трудности пропадают. Но как этого добиться?

Чтобы получить картинку на рис. 5.2, *г*, удобно будет перейти от параметризации самой разделяющей поверхности к параметризации некоторой *оценки принадлежности* (score) того или иного класса. Вместо того чтобы искать линейную разделяющую поверхность вида $\boldsymbol{w}^\top \boldsymbol{x} + w_0 = 0$ между какими-то множествами точек, давайте введём для каждого класса C_k число a_k, которое линейным образом будет оценивать то, «насколько точка принадлежит классу».

Последнюю фразу я записал в кавычках и без слова «вероятность», потому что a_k будут совершенно произвольными числами, ведь мы хотим моделировать их линейными функциями:

$$a_k(\boldsymbol{x}) = \boldsymbol{w}_k^\top \boldsymbol{x} + w_{k0}.$$

А затем будем относить новую точку $\boldsymbol{x}$ к тому классу, у которого $a_k(\boldsymbol{x})$ максимально. При таком подходе всё получается в точности как на рис. 5.2, *г*. Во-первых, разделяющие поверхности — это гиперплоскости, потому что уравнение $a_k(\boldsymbol{x}) = a_l(\boldsymbol{x})$ линейно. А во-вторых, теперь эти гиперплоскости сходятся вместе по три, ведь если $a_k(\boldsymbol{x}) = a_l(\boldsymbol{x})$ и $a_l(\boldsymbol{x}) = a_s(\boldsymbol{x})$, то и $a_k(\boldsymbol{x}) = a_s(\boldsymbol{x})$!

[1] *Мари-Жан-Антуан-Николя де Карита, маркиз де Кондорсе* (Marie Jean Antoine Nicolas de Caritat, Marquis of Condorcet, 1743–1794) — французский философ, математик и политический деятель. Отучившись в иезуитской школе, Кондорсе поступил в Наваррский коллеж, где ему прочили военную карьеру, но предпочёл уйти в науку. Кондорсе занимался теорией интегрирования, небесной механикой и другими науками, но наиболее известен как человек, инициировавший применение математических методов в общественных науках: в работе «Рассуждения о применении анализа к оценке выборов большинством голосов» [110] он заложил основы теории голосований (теории коллективного выбора) и рассмотрел свой знаменитый парадокс, а после революции принял новую власть, основал первую республиканскую газету во Франции и стал президентом Национального конвента, где многое сделал для общественного образования во Франции.

Замечу здесь мимоходом, что получающиеся разделяющие поверхности имеют и интересные математические свойства. В случае линейных оценок принадлежности наше правило принятия решений выглядит как максимум из нескольких линейных функций.

Такие объекты относятся к так называемой *тропической геометрии* — разделу алгебраической геометрии, изучающему свойства алгебры, в которой роль сложения играет минимум (или максимум, это, разумеется, неважно), а роль умножения играет сложение:

$$x \oplus y = \min\{x, y\}, \qquad x \otimes y = x + y.$$

В такой алгебре многочлен, то есть сумма нескольких мономов, становится как раз минимумом из нескольких линейных функций. Например, многочлен $x \otimes x \oplus 2 \otimes x \otimes y$ в тропической алгебре можно записать в обычных обозначениях как $\min\{2x, x + y + 2\}$. Соответственно, получающиеся у нас разделяющие поверхности, в которых линейные лучи и отрезки сходятся в узлах по три, — это полиномиальные кривые в тропической геометрии.

Своё странное название тропическая геометрия получила в честь бразильского математика венгерского происхождения Имре Саймона, который заложил основы этого раздела математики [544]. Тропическая геометрия имеет много интересных приложений и уже представляет собой хорошо развитый раздел математики [368, 393, 554], но нам в неё углубляться не потребуется.

Выработав геометрическую интуицию, давайте посмотрим, как её реализовать. В этом разделе мы начнём с классических методов классификации, которые работают в основном геометрически, хотя, безусловно, имеют и некоторый вероятностный смысл. Как мы уже обсуждали, различаться между собой методы классификации будут главным образом тем, какую функцию ошибки они будут оптимизировать вместо кусочно-постоянной ошибки классификации.

Можно было бы, конечно, начать с уже очень хорошо знакомой нам функции ошибки: минимизации среднеквадратичного отклонения, которое прекрасно сработало для линейной регрессии в главе 4. В случае классификации (для простоты будем пока рассматривать случай двух классов) нужно будет выбрать одно целевое значение для класса C_1, скажем $t_n = 1$, и другое целевое значение для класса C_2, скажем $t_n = 0$, и минимизировать функцию

$$\mathrm{RSS}(\boldsymbol{w}) = \sum\nolimits_{n=1}^{N} \left(t_n - \boldsymbol{w}^\top \boldsymbol{x}_n\right)^2.$$

Геометрически это будет значить, что мы ищем такое направление, в проекции на которое все точки класса C_1 концентрируются максимально плотно вокруг единицы, а все точки класса C_2 — вокруг нуля. А разделяющей поверхностью будет тогда, естественно, гиперплоскость $\boldsymbol{w}^\top \boldsymbol{x} = \frac{1}{2}$.

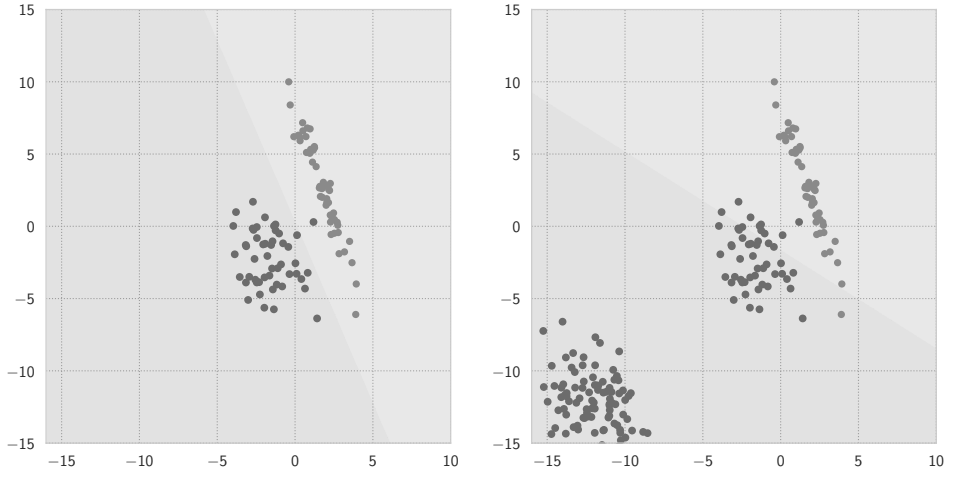


Рис. 5.3. Пример классификатора, оптимизирующего среднеквадратичное отклонение: (а) классификатор работает хорошо; (б) добавление точек, которые классифицируются правильно, делает классификатор хуже

В чём здесь проблема? Во-первых, у метода наименьших квадратов в случае классификации есть принципиальная беда с вероятностными предположениями: мы знаем (вспомните раздел 4.3), что среднеквадратичное отклонение соответствует нормально распределённому шуму. Но какой может быть нормально распределённый шум у случайной величины t_n, если у неё всего два возможных значения?! Это не окончательное соображение: мы можем отнести этот шум к распределению $\boldsymbol{x}_n$, а не t_n; но всё равно минимизация среднеквадратичного отклонения будет работать с вероятностной точки зрения только в том случае, если оба класса представляют собой гауссианы.

Да и интуитивно понятно, что среднеквадратичное отклонение вряд ли может хорошо работать для классификации: рассмотрим пример, изображённый на рис. 5.3. В примере слева (см. рис. 5.3, *а*) всё в порядке: точки обоих классов порождены из гауссовских распределений, и среднеквадратичное отклонение работает адекватно. Но посмотрите, что происходит справа (см. рис. 5.3, *б*): мы добавили точки, которые классифицируются исходной моделью *абсолютно правильно*, с очень высокой степенью уверенности в правильном ответе... но модель от этого внезапно решила измениться и начала гораздо хуже работать даже на тренировочном датасете!

Дело в том, что метод наименьших квадратов «хочет», чтобы в обучаемой проекции точки одного класса не просто лежали с одной стороны от нуля, а все проецировались как можно ближе к одной и той же точке; он минимизирует ошибку

и с одной стороны от целевого значения, и с другой, что и приводит к подобным эксцессам. Поэтому метод наименьших квадратов мы для классификации применять не будем[1].

5.1.3. Линейный дискриминант Фишера

Чтобы мотивировать следующий метод, вернёмся к геометрическому пониманию классификации, которое уже обсуждалось выше: если мы строим линейную разделяющую поверхность (гиперплоскость), это значит, что мы хотим спроецировать точки в размерность 1 (на нормаль разделяющей гиперплоскости) так, чтобы в этой размерности 1 они хорошо разделялись. Давайте рассмотрим классификацию с этих позиций и попробуем формализовать, «что такое хорошо».

Рассмотрим два класса $\mathcal{C}_1$ и $\mathcal{C}_2$ с N_1 и N_2 точками. Первая идея состоит в том, чтобы найти серединный перпендикуляр между центрами кластеров

$$\boldsymbol{m}_1 = \frac{1}{N_1}\sum_{\boldsymbol{x}_n \in \mathcal{C}_1} \boldsymbol{x}_n \quad \text{и} \quad \boldsymbol{m}_2 = \frac{1}{N_2}\sum_{\boldsymbol{x}_n \in \mathcal{C}_2} \boldsymbol{x}_n,$$

то есть максимизировать величину

$$\left|\boldsymbol{w}^\top (\boldsymbol{m}_2 - \boldsymbol{m}_1)\right|$$

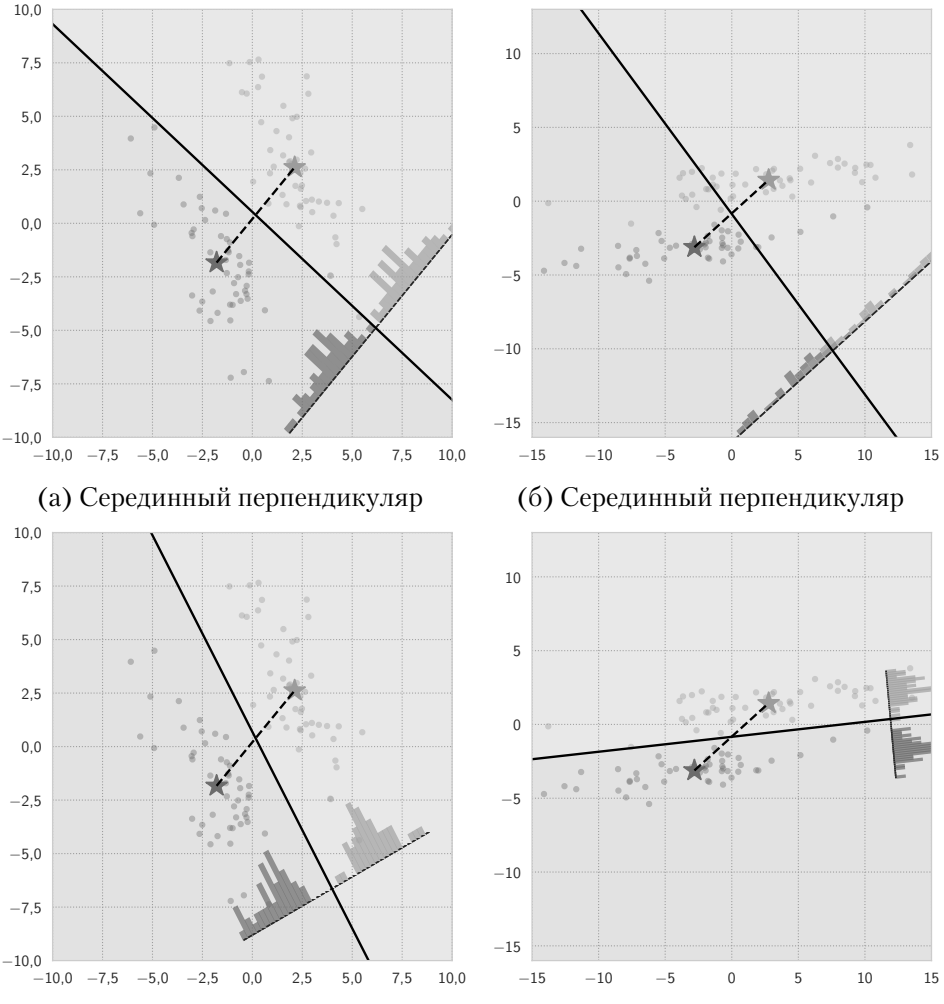
по $\boldsymbol{w}$. Здесь ещё нужно добавить ограничение $\|\boldsymbol{w}\| = 1$, иначе целевая функция будет увеличиваться до бесконечности простым умножением на константу.

Ответ очевиден: скалярное произведение с единичным вектором — это, по сути, длина проекции на направление, и она, конечно, будет максимизироваться при проекции на само направление $\boldsymbol{m}_2 - \boldsymbol{m}_1$. Поэтому разделяющей поверхностью при таком подходе будет гиперплоскость, являющаяся серединным перпендикуляром отрезка, соединяющего центры двух классов. На рис. 5.4, *а* приведён пример, в котором оба класса порождены из двумерного нормального распределения, и они прекрасно разделяются серединным перпендикуляром.

Но в целом этот метод работает не очень-то хорошо. Давайте посмотрим на рис. 5.4, *б*; в этом случае очевидно, что между двумя классами *есть* хорошая разделяющая гиперплоскость, но она не имеет ничего общего с серединным перпендикуляром между центрами.

---

[1] Любопытно, что метод наименьших квадратов для классификации в некотором смысле «вернулся в игру» в последние годы в контексте порождающих моделей в глубоком обучении. Оказалось, что при обучении порождающих состязательных сетей (generative adversarial networks, GAN) вполне разумной функцией ошибки для дискриминатора, который, казалось бы, решает задачу классификации, является именно квадрат отклонения от некоторого целевого значения. Этот подход так и называется — Least Squares GAN (LSGAN) [414], и это одна из, наверное, двух наиболее популярных функций ошибки для порождающих состязательных сетей, наряду с Wasserstein GAN [29]. У LSGAN есть даже некоторый вероятностный смысл, хотя он, как обычно бывает в GAN'ах, не очень релевантен практике; но нашему текущему изложению до GAN'ов пока ещё очень далеко, так что не будем развивать тему.



(а) Серединный перпендикуляр (б) Серединный перпендикуляр

(в) Дискриминант Фишера (г) Дискриминант Фишера

Рис. 5.4. Линейный дискриминант Фишера: (а) серединный перпендикуляр работает; (б) серединный перпендикуляр не работает, хотя классы по-прежнему гауссовские; (в) дискриминант Фишера работает примерно так же, как серединный перпендикуляр; (г) дискриминант Фишера работает, а серединный перпендикуляр — нет

Более того, легко видеть, что так может получиться даже в, казалось бы, самом простом случае — на рис. 5.4, *б* точки обоих классов были порождены из обычных гауссианов, причём одинаковой формы, а серединный перпендикуляр всё равно не работает. В чём здесь подвох, и как с ним справиться?

Мы уже говорили выше о геометрической сути классификации; она состоит в том, чтобы найти направление, при проекции на которое точки хорошо разделяются. И действительно, выбор направления, при проекции на которое центры классов будут как можно дальше друг от друга, — это шаг в нужном направлении.

Но это не единственный критерий! На рис. 5.4, *б* расстояние между центрами в проекции действительно максимально, но зато у обоих классов в этом направлении довольно большая *дисперсия* — и в результате классы в этой проекции имеют существенное пересечение и разделяются плохо.

Значит, нужно минимизировать перекрытие классов, оптимизируя и проекцию расстояния, и дисперсию. Выборочные дисперсии в проекции можно подсчитать обычным образом, и это будет квадратичная функция от $\boldsymbol{w}$ (обратите внимание, что теперь m_1 и m_2 — это числа, а не векторы, речь идёт о центрах в проекции):

$$s_1 = \sum_{\boldsymbol{x}_n \in \mathcal{C}_1} \left(\boldsymbol{w}^\top \boldsymbol{x}_n - m_1 \right)^2, \qquad s_2 = \sum_{\boldsymbol{x}_n \in \mathcal{C}_2} \left(\boldsymbol{w}^\top \boldsymbol{x}_n - m_2 \right)^2,$$

где

$$m_1 = \frac{1}{N_1} \sum_{\boldsymbol{x}_n \in \mathcal{C}_1} \boldsymbol{w}^\top \boldsymbol{x}_n, \qquad m_2 = \frac{1}{N_2} \sum_{\boldsymbol{x}_n \in \mathcal{C}_2} \boldsymbol{w}^\top \boldsymbol{x}_n.$$

Итак, у нас появились две разные целевые функции, каждую из которых мы хотели бы оптимизировать по $\boldsymbol{w}$:

- максимизировать расстояние между классами $\left| \boldsymbol{w}^\top (\boldsymbol{m}_2 - \boldsymbol{m}_1) \right|$ и
- минимизировать суммарную дисперсию в проекции $s_1 + s_2$.

Что делать, как объединить две функции в одну задачу оптимизации?

В такой ситуации можно применить достаточно естественный трюк: давайте будем минимизировать *отношение* двух целевых функций, поставив в числитель то, что мы хотим сделать поменьше, а в знаменатель — то, что мы хотим сделать побольше. Отношение в таких случаях обычно работает лучше, чем разность, потому что разность требует точного совпадения масштабов двух величин (если из тысяч вычитать единицы, единицы окажутся совершенно неважны), а в отношении (или, что то же самое, в разности логарифмов) это превратится в глобальную константу и не помешает оптимизации. Но всё равно обычно лучше сделать так, чтобы, так сказать, «единицы измерения» этих величин совпадали, да и модуль оптимизировать неудобно, поэтому в числитель мы поместим не $\left| \boldsymbol{w}^\top (\boldsymbol{m}_2 - \boldsymbol{m}_1) \right|$, а квадрат этой величины.

В итоге получается так называемый *критерий Фишера*[1], или *линейный дискриминант Фишера* (Fischer's linear discriminant):

$$J(\boldsymbol{w}) = \frac{\boldsymbol{w}^\top (\boldsymbol{m}_2 - \boldsymbol{m}_1)(\boldsymbol{m}_2 - \boldsymbol{m}_1)^\top \boldsymbol{w}}{s_1^2 + s_2^2} = \frac{\boldsymbol{w}^\top S_B \boldsymbol{w}}{\boldsymbol{w}^\top S_W \boldsymbol{w}},$$

где

$$S_B = (\boldsymbol{m}_2 - \boldsymbol{m}_1)(\boldsymbol{m}_2 - \boldsymbol{m}_1)^\top,$$
$$S_W = \sum_{\boldsymbol{x}_n \in \mathcal{C}_1} (\boldsymbol{x}_n - \boldsymbol{m}_1)(\boldsymbol{x}_n - \boldsymbol{m}_1)^\top + \sum_{\boldsymbol{x}_n \in \mathcal{C}_2} (\boldsymbol{x}_n - \boldsymbol{m}_2)(\boldsymbol{x}_n - \boldsymbol{m}_2)^\top.$$

Матрицы S_B и S_W, которые мы здесь явно выделили и обозначили, носят название матриц межклассовой ковариации (between-class covariance) S_B и внутриклассовой ковариации (within-class covariance) S_W. Название матрицы внутриклассовой ковариации S_W понятно, это просто взвешенная сумма двух матриц ковариаций. Название матрицы S_B не так легко объяснить, но оказывается, что сумма этих двух матриц — это в точности полная матрица ковариаций датасета (с точностью до $\frac{1}{N}$):

$$S_B + S_W = S = \sum_{n=1}^{N} (\boldsymbol{x}_n - \boldsymbol{m})(\boldsymbol{x}_n - \boldsymbol{m})^\top, \quad \text{где} \quad \boldsymbol{m} = \frac{1}{N}\sum_{n=1}^{N} \boldsymbol{x}_n.$$

Для доказательства этого факта нужно просто аккуратно раскрыть скобки, так что его мы оставим читателю в качестве упражнения; именно этот результат объясняет смысл слов «межклассовая ковариация» — это та часть полной ковариации, которая не объясняется разбросом данных внутри каждого класса.

Это один из многочисленных примеров одного и того же трюка, который вы ещё увидите в этой книге и в контексте калибровки в подразделе 5.5.2, и в контексте разложения ожидаемой ошибки предсказания в подразделе 6.2.4, и уже видели при обращении с правдоподобием нормального распределения в подразделе 4.1.3, да и вообще в контексте гауссианов.

[1] *Рональд Эйлмер Фишер* (Sir Ronald Aylmer Fisher, 1890–1962) — английский статистик, генетик и эволюционист. Как и Фрэнсис Гальтон, Фишер развивал статистику в основном в её биологических и генетических приложениях; Ричард Докинз называл Фишера «величайшим из последователей Дарвина». Но, конечно, созданные им методы были общими и не ограничивались биологией. В частности, Фишер разработал дисперсионный анализ (ANOVA) [184], пропагандировал и развивал метод максимального правдоподобия [21], внёс важный вклад в статистически корректный дизайн экспериментов, фактически введя идею нулевой гипотезы на примере дегустации чая с молоком [183], впервые обработал знаменитый датасет о цветах ирисов [185] и многое другое. В наше время Фишер стал спорной фигурой из-за своих взглядов на расовые различия — он несомненно был евгеником и в 1950 году даже опубликовал своё особое мнение против Декларации о расе ЮНЕСКО. В результате в 2020 году кембриджский Колледж Гонвилл-энд-Киз снял посвящённый Фишеру витраж из знаменитой серии Марии Макклафферти.

В общем виде трюк выглядит так. Предположим, что нас интересует сумма квадратов отклонений того или иного набора величин $\{a_1,a_2,\ldots,a_N\}$ от среднего; сейчас неважно, откуда они взялись и какие о них есть вероятностные предположения, важно только, что нас интересует

$$\text{TotalSS} = \sum_{n=1}^{N} (a_n - \bar{a})^2, \quad \text{где} \quad \bar{a} = \frac{1}{N}\sum_{n=1}^{N} a_n.$$

Предположим также, что мы можем разбить набор этих величин на некоторое множество разных групп $S_1,\ldots,S_I$; опять же совершенно неважно, откуда они взялись, важно только, что они не пересекаются, $S_i \cap S_j = \emptyset$, и покрывают все индексы, $S_1 \cup \ldots \cup S_I = \{1,\ldots,N\}$.

Тогда мы можем подсчитать сумму квадратов отклонений от средних *внутри* каждой группы,

$$\text{GroupSS} = \sum_{i=1}^{I} \sum_{n\in S_i} (a_n - \bar{a}_i)^2, \quad \text{где} \quad \bar{a}_i = \frac{1}{|S_i|}\sum_{n\in S_i} a_n.$$

Обозначим через ResidualSS остаточную сумму квадратов отклонений внутригрупповых средних от глобального среднего, взвешенную размерами групп:

$$\text{ResidualSS} = \sum_{i=1}^{I} |S_i|\,(\bar{a}_i - \bar{a})^2.$$

Тогда утверждение звучит так:

$$\text{TotalSS} = \text{GroupSS} + \text{ResidualSS}. \tag{5.1}$$

Для его доказательства не нужно ничего знать о природе величин a, о каких-то распределениях и вообще о теории вероятностей, это просто алгебраический факт, доказывающийся аккуратным раскрытием всех скобочек.

В первой строчке ниже мы в каждой скобке прибавляем и вычитаем именно тот $\bar{a}_{i(n)}$, к которому относится a_n, то есть $i(n)$ определяется как индекс, для которого $n \in S_{i(n)}$:

$$\begin{aligned}
\text{TotalSS} &= \sum_{n=1}^{N} (a_n - \bar{a})^2 = \sum_{n=1}^{N} \Big(a_n - \bar{a} + \bar{a}_{i(n)} - \bar{a}_{i(n)}\Big)^2 \\
&= \sum_{n=1}^{N} \Big(a_n - \bar{a}_{i(n)}\Big)^2 + 2\sum_{n=1}^{N} \Big(a_n - \bar{a}_{i(n)}\Big)\Big(\bar{a}_{i(n)} - \bar{a}\Big) + \sum_{n=1}^{N} \Big(\bar{a}_{i(n)} - \bar{a}\Big)^2 \\
&= \sum_{n=1}^{N} \Big(a_n - \bar{a}_{i(n)}\Big)^2 + \sum_{n=1}^{N} \Big(\bar{a}_{i(n)} - \bar{a}\Big)^2 = \text{GroupSS} + \text{ResidualSS},
\end{aligned}$$

потому что

$$\sum_{n=1}^{N}(a_n-\bar{a}_i)(\bar{a}_i-\bar{a})=\sum_{i=1}^{I}\left((\bar{a}_i-\bar{a})\sum_{n\in S_i}(a_n-\bar{a}_i)\right)=0,$$

ведь во внутренней сумме теперь стоит $\sum_{n\in S_i}(a_n-\bar{a}_i)=\sum_{n\in S_i}a_n-N\bar{a}_i=0$.

Теперь можно оптимизировать критерий Фишера. Чтобы минимизировать $J(\boldsymbol{w})$ по $\boldsymbol{w}$, начнём с того, что его продифференцируем:

$$\nabla_{\boldsymbol{w}}J(\boldsymbol{w})=\frac{(\boldsymbol{w}^\top S_W\boldsymbol{w})\cdot\nabla_{\boldsymbol{w}}(\boldsymbol{w}^\top S_B\boldsymbol{w})-(\boldsymbol{w}^\top S_B\boldsymbol{w})\cdot\nabla_{\boldsymbol{w}}(\boldsymbol{w}^\top S_W\boldsymbol{w})}{\left(\boldsymbol{w}^\top S_W\boldsymbol{w}\right)^2}.$$

Мы знаем (вспомните подраздел 4.1.2), что $\nabla_{\boldsymbol{w}}(\boldsymbol{w}^\top A\boldsymbol{w})=2A\boldsymbol{w}$ для симметричной матрицы A, а матрицы ковариаций, ясное дело, симметричны. Приравнивая $\nabla_{\boldsymbol{w}}J(\boldsymbol{w})$ к нулю, в итоге мы получаем, что $J(\boldsymbol{w})$ оптимизируется при

$$\left(\boldsymbol{w}^\top S_B\boldsymbol{w}\right)S_W\boldsymbol{w}=\left(\boldsymbol{w}^\top S_W\boldsymbol{w}\right)S_B\boldsymbol{w}.$$

Как решить такое уравнение? На первый взгляд выглядит сложно — к матричным полиномиальным уравнениям жизнь нас вряд ли готовила. И действительно, в общем виде решить такое уравнение было бы нелегко, но здесь приходят на помощь специальные свойства нашей конкретной задачи. Во-первых, в лице вектора $\boldsymbol{w}$ мы на самом деле ищем направление, на которое будем проецировать точки, то есть его длина для нас принципиального значения не имеет. А значит, вместо уравнения третьей степени можно искать просто такой $\boldsymbol{w}$, для которого

$$S_W\boldsymbol{w}\propto S_B\boldsymbol{w}.$$

Даже это было бы не так легко сделать, но теперь приходит на помощь специальная форма матрицы S_B. Так как $S_B=(\boldsymbol{m}_2-\boldsymbol{m}_1)(\boldsymbol{m}_2-\boldsymbol{m}_1)^\top$, вектор $S_B\boldsymbol{w}$ всегда будет коллинеарен $(\boldsymbol{m}_2-\boldsymbol{m}_1)$; геометрически матрица ранга 1 — это матрица проекции на некоторое одномерное подпространство, а формально это можно увидеть, просто переставив скобки:

$$S_B\boldsymbol{w}=\left((\boldsymbol{m}_2-\boldsymbol{m}_1)(\boldsymbol{m}_2-\boldsymbol{m}_1)^\top\right)\boldsymbol{w}=(\boldsymbol{m}_2-\boldsymbol{m}_1)\cdot\left((\boldsymbol{m}_2-\boldsymbol{m}_1)^\top\boldsymbol{w}\right),$$

и теперь в скобках стоит число, матрица размера 1×1.

Поэтому получается, что

$$\boldsymbol{w}\propto S_W^{-1}(\boldsymbol{m}_2-\boldsymbol{m}_1). \tag{5.2}$$

В результате формула (5.2) позволяет выбрать направление проекции, и остаётся только разделить данные в этой проекции, в размерности 1. Рисунок 5.4, *б*

показывает, как работает линейный дискриминант Фишера на том же примере; как видите, результат гораздо лучше отвечает изначальной интуиции.

Сделаю ещё одно любопытное — и на первый взгляд немного странное — замечание. Оказывается, что линейный дискриминант Фишера в случае двух классов эквивалентен нехитрой модификации метода наименьших квадратов, который мы забраковали выше. В методе наименьших квадратов мы оптимизируем функцию $\text{RSS}(\boldsymbol{w}) = \sum_{n=1}^{N} \left(t_n - \boldsymbol{w}^\top \boldsymbol{x}_n\right)^2$. Давайте предположим, что и $\boldsymbol{x}_n$, и t_n центрированы, то есть $\sum_{n=1}^{N} \boldsymbol{x}_n = 0$ и $\sum_{n=1}^{N} t_n = 0$; это значит, что векторы признаков нужно центрировать линейным преобразованием (что и само по себе часто оказывается полезным преобразованием на шаге предобработки данных), а в качестве целевых значений выбрать $\frac{N}{N_1}$ для класса C_1 и $-\frac{N}{N_2}$ для класса $\mathcal{C}_2$. Из подраздела 4.1.2 мы уже знаем, какой будет ответ в данном случае:

$$\boldsymbol{w}^* = \left(\boldsymbol{X}^\top \boldsymbol{X}\right)^{-1} \boldsymbol{X}^\top \boldsymbol{t}.$$

Но если $\boldsymbol{x}_n$ центрированы, то

$$\boldsymbol{X}^\top \boldsymbol{X} = S = \sum_{n=1}^{N} (\boldsymbol{x}_n - \boldsymbol{m}) (\boldsymbol{x}_n - \boldsymbol{m})^\top,$$

и можно подсчитать, что

$$\boldsymbol{X}^\top \boldsymbol{t} = \sum_{n=1}^{N} \boldsymbol{x}_n t_n = \frac{N}{N_1} \sum_{\boldsymbol{x}_n \in C_1} \boldsymbol{x}_n - \frac{N}{N_2} \sum_{\boldsymbol{x}_n \in C_2} \boldsymbol{x}_n = N (\boldsymbol{m}_1 - \boldsymbol{m}_2).$$

Вот и получается, что

$$\boldsymbol{w}^* = \left(\boldsymbol{X}^\top \boldsymbol{X}\right)^{-1} \boldsymbol{X}^\top \boldsymbol{t} = S^{-1} N (\boldsymbol{m}_1 - \boldsymbol{m}_2) \propto S^{-1} (\boldsymbol{m}_1 - \boldsymbol{m}_2).$$

Осталось заметить, что условие $S_W \boldsymbol{w} \propto S_B \boldsymbol{w}$, которое мы обсуждали выше, эквивалентно полученному сейчас условию $S\boldsymbol{w} \propto S_B \boldsymbol{w}$, ведь $S = S_W + S_B$.

Но всё это пока было применимо исключительно к бинарной классификации — мы довольно активно пользовались формулой $S = S_W + S_B$, в которой не всё обобщается на несколько классов очевидным образом. Классический подход к обобщению дискриминанта Фишера на несколько классов $C_1, \ldots, C_K$ носит название *множественного дискриминантного анализа* (multiple discriminant analysis, MDA). Как перенести на несколько классов матрицу внутриклассовой ковариации, очевидно:

$$S_W = \sum_{k=1}^{K} S_k, \quad \text{где} \quad S_k = \sum_{\boldsymbol{x}_n \in \mathcal{C}_k} (\boldsymbol{x}_n - \boldsymbol{m}_k) (\boldsymbol{x}_n - \boldsymbol{m}_k)^\top.$$

Но что делать с матрицей S_B? Её обобщение вовсе не так уж естественно: объединить несколько разных средних $\boldsymbol{m}_k$ в одно-единственное число можно многими разными способами. На помощь здесь приходит то же соображение, что и выше: раз $S = S_W + S_B$, значит, логично определить S_B как

$$S_B = S - S_W.$$

И теперь можно заметить, что

$$\begin{aligned} S &= \sum_{n=1}^{N} (\boldsymbol{x}_n - \boldsymbol{m})(\boldsymbol{x}_n - \boldsymbol{m})^\top = \sum_{k=1}^{K} \sum_{\boldsymbol{x}_n \in \mathcal{C}_k} (\boldsymbol{x}_n - \boldsymbol{m})(\boldsymbol{x}_n - \boldsymbol{m})^\top = \\ &= \sum_{k=1}^{K} \sum_{\boldsymbol{x}_n \in \mathcal{C}_k} (\boldsymbol{x}_n - \boldsymbol{m}_k + \boldsymbol{m}_k - \boldsymbol{m})(\boldsymbol{x}_n - \boldsymbol{m}_k + \boldsymbol{m}_k - \boldsymbol{m})^\top = \\ &= \sum_{k=1}^{K} \sum_{\boldsymbol{x}_n \in \mathcal{C}_k} \Big((\boldsymbol{x}_n - \boldsymbol{m}_k)(\boldsymbol{x}_n - \boldsymbol{m}_k)^\top + (\boldsymbol{m}_k - \boldsymbol{m})(\boldsymbol{m}_k - \boldsymbol{m})^\top \Big) = \\ &= S_W + \sum\nolimits_{k=1}^{K} N_k (\boldsymbol{m}_k - \boldsymbol{m})(\boldsymbol{m}_k - \boldsymbol{m})^\top, \end{aligned}$$

то есть естественно будет обобщить матрицу межклассовой ковариации как

$$S_B = \sum_{k=1}^{K} N_k (\boldsymbol{m}_k - \boldsymbol{m})(\boldsymbol{m}_k - \boldsymbol{m})^\top.$$

Что теперь делать с этими матрицами? Ответ следует опять же из геометрической интуиции того, что такое линейная классификация. В случае нескольких классов, $K > 2$, мы уже ищем не прямую (подпространство размерности 1), вдоль которой данные хорошо разделяются, а подпространство размерности $K-1$. Введём дискриминантные функции $\boldsymbol{w}_j$, $j = 1, \ldots, K-1$, которые будут, по сути, проецировать точки на это подпространство: мы подсчитаем значения

$$y_{jn} = \boldsymbol{w}_j^\top \boldsymbol{x}_n$$

и составим из них векторы $\boldsymbol{y}_n$, $n = 1, \ldots, N$, размерности $K-1$, то есть

$$\hat{y}_n = W \boldsymbol{x}_n$$

для матрицы W размера $(K-1) \times d$, составленной из векторов весов $\boldsymbol{w}_k$, а это и есть формула проекции на подпространство.

Что будет значить, что в этом подпространстве точки «хорошо разделяются»? Да тот же самый критерий Фишера, только в подпространстве размерности $K-1$.

Мы можем точно так же определить средние классов в проекциях $\hat{\boldsymbol{m}}_k$, глобальное среднее $\hat{\boldsymbol{m}}$ и матрицы

$$\hat{S}_W = \sum_{k=1}^{K} \sum_{\boldsymbol{y} \in C_k} (\boldsymbol{y}_n - \hat{\boldsymbol{m}}_k)(\boldsymbol{y}_n - \hat{\boldsymbol{m}}_k)^\top,$$
$$\hat{S}_B = \sum_{k=1}^{K} N_k (\hat{\boldsymbol{m}}_k - \hat{\boldsymbol{m}})(\hat{\boldsymbol{m}}_k - \hat{\boldsymbol{m}})^\top.$$

Теперь легко показать (оставлю это вам в качестве упражнения), что матрицы W можно вынести за все эти скобки:

$$\hat{S}_W = W^\top S_W W, \qquad \hat{S}_B = W^\top S_B W.$$

Мы хотим найти такую матрицу проекции W, которая будет в некотором смысле максимизировать межклассовую ковариацию и минимизировать внутриклассовую. В качестве скалярной оценки, к которой теперь нужно свести матрицы $\hat{S}_W$ и $\hat{S}_B$, логично взять их определитель, ведь это произведение собственных чисел, то есть «дисперсий» вдоль главных направлений, объём эллипсоида, заданного этой матрицей. Обобщение критерия Фишера получается в виде

$$J(W) = \frac{\det \hat{S}_B}{\det \hat{S}_W} = \frac{\det \left(W^\top S_B W \right)}{\det \left(W^\top S_W W \right)}.$$

Решать эту задачу подробно мы не будем (см., например, [157, 201, 390]), а ответ в ней вполне ожидаем — оптимальная матрица W состоит из обобщённых собственных векторов, которые соответствуют наибольшим собственным числам λ_j среди решений уравнения $S_B \boldsymbol{w}_j = \lambda_j S_W \boldsymbol{w}_j$.

На этом я завершаю рассказ о линейном дискриминанте Фишера, который опирается на геометрическую интуицию задач классификации и реализует её, наверное, самым прямым и концептуально чистым образом. В этом разделе я ни разу не ссылался ни на какие вероятностные предположения, но уже пора.

5.1.4. Порождающие модели для классификации: LDA и QDA

Геометрия геометрией, но мы в этой книге всё-таки обычно рассматриваем машинное обучение как раздел прикладной теории вероятностей. Придать классификации вероятностный смысл на самом деле несложно: для данного вектора признаков $\boldsymbol{x}$ мы хотели бы оценить вероятности того, какому из нескольких классов $C_1, C_2, \ldots, C_K$ принадлежит точка $\boldsymbol{x}$, то есть найти апостериорные вероятности $p(C_k|\boldsymbol{x})$. Первый шаг здесь, как и почти всегда в машинном обучении, состоит в том, чтобы применить теорему Байеса:

$$p\left(C_k|\boldsymbol{x}\right) = \frac{p\left(\boldsymbol{x}|C_k\right)p\left(C_k\right)}{p\left(\boldsymbol{x}\right)} = \frac{p\left(\boldsymbol{x}|C_k\right)p\left(C_k\right)}{p\left(\boldsymbol{x}|C_1\right)p\left(C_1\right) + \ldots + p\left(\boldsymbol{x}|C_K\right)p\left(C_K\right)}. \qquad (5.3)$$

Обратите внимание, что фигурирующие в формуле (5.3) априорные вероятности классов $p\left(C_k\right)$ представляют собой именно вероятности, по одному числу на класс. В задачах классификации не нужно мучительно выбирать априорные распределения, они всего лишь показывают сравнительные веса разных классов, и обычно их либо принимают равными (то есть выбирают равномерное априорное распределение), либо оценивают по соотношению точек разных классов в обучающей выборке.

Интересная же часть формулы (5.3) — это распределения $p\left(\boldsymbol{x}|C_k\right)$. Чтобы их оценить, уже действительно придётся выбрать те или иные распределения для различных классов, то есть сформулировать *порождающие модели*, семплирование из которых будет давать точки данных в каждом классе. А сама формула (5.3) показывает, как из порождающей модели сделать *дискриминирующую* (discriminative), которая будет выражать, наоборот, вероятности классов $p\left(C_k|\boldsymbol{x}\right)$.

Дуальность между порождающими и дискриминирующими моделями мы обсудим в разделе 5.6, хотя в целом современные порождающие модели — это отдельная большая тема, которой придётся подождать как минимум до следующей книги. Сейчас просто отмечу, что если предположения наших порождающих моделей $p\left(\boldsymbol{x}|C_k\right)$ в точности соответствуют действительности, то формула (5.3) даёт нам *оптимальный байесовский классификатор* — лучший возможный классификатор с точки зрения минимизации доли ошибок классификации. Мы вернёмся к этой мысли в разделе 6.2, когда будем обсуждать статистическую теорию принятия решений.

Таким образом, при таком подходе классификация сводится к обучению отдельной порождающей модели $p\left(\boldsymbol{x}|C_k\right)$ для каждого класса. Конечно, это не совсем равноценный обмен: в общем случае проще обучать модель для вероятностей классов, чем для порождения многомерных точек данных сложной структуры. Но если предположить, что $p\left(\boldsymbol{x}|C_k\right)$ достаточно простые, этот подход вполне может сработать. Давайте попробуем найти такие достаточно сильные предположения.

Самые простые и удобные распределения для порождения многомерных точек в $\mathbb{R}^d$ — это, разумеется, гауссианы. Давайте рассмотрим нормальные распределения для классов:

$$p\left(\boldsymbol{x}|C_k\right) = \mathcal{N}\left(\boldsymbol{x}|\boldsymbol{\mu}_k,\Sigma_k\right).$$

Для удобства сначала предположим, что классов всего два. Какая тогда получится разделяющая поверхность между этими двумя классами? Разделяющая поверхность — это геометрическое место точек[1], для которых вероятности двух классов равны, $p\left(C_1|\boldsymbol{x}\right) = p\left(C_2|\boldsymbol{x}\right)$. По формуле (5.3), это эквивалентно

[1] Спорим, вы вряд ли ожидали встретить этот термин в книге по машинному обучению...

$$\ln p\left(\boldsymbol{x}|C_1\right)p\left(C_1\right) = \ln p\left(\boldsymbol{x}|C_2\right)p\left(C_2\right).$$

Теперь можно подставить собственно логарифм правдоподобия, привести подобные и сократить лишнее:

$$-\frac{1}{2}\ln 2\pi - \frac{1}{2}\ln\det\Sigma_1 - \frac{1}{2}\left(\boldsymbol{x}-\boldsymbol{\mu}_1\right)^\top\Sigma_1^{-1}\left(\boldsymbol{x}-\boldsymbol{\mu}_1\right) + \ln p\left(C_1\right) =$$
$$= -\frac{1}{2}\ln 2\pi - \frac{1}{2}\ln\det\Sigma_2 - \frac{1}{2}\left(\boldsymbol{x}-\boldsymbol{\mu}_2\right)^\top\Sigma_2^{-1}\left(\boldsymbol{x}-\boldsymbol{\mu}_2\right) + \ln p\left(C_2\right),$$

то есть

$$-\frac{1}{2}\left(\boldsymbol{x}-\boldsymbol{\mu}_1\right)^\top\Sigma_1^{-1}\left(\boldsymbol{x}-\boldsymbol{\mu}_1\right) + \frac{1}{2}\left(\boldsymbol{x}-\boldsymbol{\mu}_2\right)^\top\Sigma_2^{-1}\boldsymbol{\mu}_2 - \frac{1}{2}\ln\frac{\det\Sigma_1}{\det\Sigma_2} + \ln\frac{p\left(\mathcal{C}_1\right)}{p\left(\mathcal{C}_2\right)} = 0,$$

и мы получаем квадратное уравнение относительно $\boldsymbol{x}$:

$$-\frac{1}{2}\boldsymbol{x}^\top\left(\Sigma_1^{-1}-\Sigma_2^{-1}\right)\boldsymbol{x} + \boldsymbol{x}^\top\left(\Sigma_1^{-1}\boldsymbol{\mu}_1 - \Sigma_2^{-1}\boldsymbol{\mu}_2\right) -$$
$$-\frac{1}{2}\boldsymbol{\mu}_1^\top\Sigma_1^{-1}\boldsymbol{\mu}_1 + \frac{1}{2}\boldsymbol{\mu}_2^\top\Sigma_2^{-1}\boldsymbol{\mu}_2 - \frac{1}{2}\ln\frac{\det\Sigma_1}{\det\Sigma_2} + \ln\frac{p\left(\mathcal{C}_1\right)}{p\left(\mathcal{C}_2\right)} = 0. \quad (5.4)$$

Из формулы (5.4) можно сделать сразу несколько интересных выводов. Во-первых, разделяющая поверхность между любыми гауссианами будет квадратичной и мы без проблем можем выписать её формулу. Такой метод построения разделяющей поверхности известен как *квадратичный дискриминантный анализ* (quadratic discriminative analysis, QDA).

Во-вторых, очевидно, что мы можем сделать разделяющую поверхность (5.4) линейной, если положим $\Sigma_1 = \Sigma_2$. При этом получается, что мы предполагаем не только общую гауссовскую форму всех классов, но и то, что у всех классов одинаковая матрица ковариаций. Это то же самое предположение *гомоскедастичности*, которое мы обсуждали в разделе 4.3 в контексте линейной регрессии. В результате тоже получается очень популярная классическая модель, *линейный дискриминантный анализ* (linear discriminative analysis, LDA[1]). Линейный дискриминантный анализ даёт результаты, очень похожие на дискриминант Фишера (см. подраздел 5.1.3), и тоже восходит к работам сэра Рональда [185].

В-третьих, формула (5.4) показывает, какое воздействие на разделяющую поверхность оказывают априорные распределения. В (5.4) за это отвечает слагаемое $\ln\frac{p(\mathcal{C}_1)}{p(\mathcal{C}_2)}$, и мы видим, что, если мы изменим априорную вероятность одного из классов по сравнению с другим, это приведёт к тому, что разделяющая поверхность просто сдвинется на константу в сторону менее вероятного класса.

[1] Это LDA не нужно путать с латентным размещением Дирихле (latent Dirichlet allocation, LDA) [67, 68], тематической моделью, до которой мы в этой книге не доберёмся, но о которой, надеюсь, когда-нибудь ещё поговорим.

В-четвёртых, всё это прекрасно обобщается на случай нескольких классов. Мы уже обсуждали геометрическую сторону мультиклассовой задачи классификации в разделе 5.1, и здесь мы видим ту же самую ситуацию. Каждый класс будет описываться величиной, пропорциональной $p(C_k|\boldsymbol{x})$:

$$a_k = p(\boldsymbol{x}|C_k)\,p(C_k)\,,$$

и разделяющая поверхность между каждой парой классов C_k и C_l будет иметь вид $a_k = a_l$ и описываться формулой (5.4) с соответствующими $\boldsymbol{\mu}_k, \Sigma_k$ и $\boldsymbol{\mu}_l, \Sigma_l$.

Кроме того, не будем забывать, что это всего лишь формула разделяющей поверхности, а на самом деле придуманная нами модель способна на большее: оптимальный байесовский классификатор позволяет не только дать бинарный ответ (с какой стороны от разделяющей поверхности находится точка $\boldsymbol{x}$), но и оценить апостериорную вероятность $p(C_k|\boldsymbol{x})$.

Может показаться, что у нас всё готово. Но на самом деле мы пока научились только оценивать вероятности $p(C_k|\boldsymbol{x})$ при известных $p(\boldsymbol{x}|C_k)$, то есть в нашем случае при известных $\boldsymbol{\mu}_k, \Sigma_k$. А откуда они возьмутся? Для квадратичного дискриминантного анализа ответ совсем простой: нужно взять точки из класса C_k и обучить параметры гауссиана, то есть положить

$$\boldsymbol{\mu}_k = \bar{\boldsymbol{x}} = \frac{1}{N_k}\sum_{\boldsymbol{x}_n \in C_k} \boldsymbol{x}_n, \qquad \Sigma_k = \frac{1}{N_k}\sum_{\boldsymbol{x}_n \in C_k} (\boldsymbol{x}_n - \bar{\boldsymbol{x}})\,(\boldsymbol{x}_n - \bar{\boldsymbol{x}})^\top,$$

где N_k — число точек в классе C_k.

А для линейного дискриминантного анализа нужно не просто обучить два гауссиана, а обучить их так, чтобы матрица ковариаций у них была одна и та же. Иначе говоря, на самом деле мы максимизируем правдоподобие следующей модели:

$$p(D|\boldsymbol{\mu}_1,\boldsymbol{\mu}_2,\Sigma) = \prod_{\boldsymbol{x}_n \in C_1} \mathcal{N}(\boldsymbol{x}_n|\boldsymbol{\mu}_1, \Sigma) \prod_{\boldsymbol{x}_n \in C_2} \mathcal{N}(\boldsymbol{x}_n|\boldsymbol{\mu}_2, \Sigma)$$

по параметрам $\boldsymbol{\mu}_1,\boldsymbol{\mu}_2,\Sigma$. Ясно, что делать дальше: взять логарифм, выписать правдоподобия и привести подобные члены. Поскольку вывод этот на самом деле практически ничем не отличается от вывода для одного гауссиана, оставим его читателю в качестве упражнения и просто запишем вполне ожидаемый ответ:

$$\boldsymbol{\mu}_1^* = \frac{1}{N_1}\sum_{\boldsymbol{x}_n \in C_1} \boldsymbol{x}_n, \qquad \boldsymbol{\mu}_2^* = \frac{1}{N_2}\sum_{\boldsymbol{x}_n \in C_2} \boldsymbol{x}_n, \qquad \Sigma^* = \frac{N_1}{N_1+N_2}S_1 + \frac{N_2}{N_1+N_2}S_2,$$

где

$$S_1 = \frac{1}{N_1}\sum_{\boldsymbol{x}_n \in \mathcal{C}_1} (\boldsymbol{x}_n - \boldsymbol{\mu}_1)\,(\boldsymbol{x}_n - \boldsymbol{\mu}_1)^\top, \qquad S_2 = \frac{1}{N_2}\sum_{\boldsymbol{x}_n \in \mathcal{C}_2} (\boldsymbol{x}_n - \boldsymbol{\mu}_2)\,(\boldsymbol{x}_n - \boldsymbol{\mu}_2)^\top.$$

Давайте теперь увидим всё это на примере. Для этого породим два класса из настоящих гауссианов, чтобы вероятностные предположения QDA выполнялись в точности, но с разными матрицами ковариаций, чтобы для LDA это не было верно:

```
def sample_mixture(N, pi, mu1, sigma1, mu2, sigma2):
    z = np.array( np.random.rand(N) < pi, dtype=int)
    res = np.zeros((N, 2))
    res[np.where(z == 1)] = np.random.multivariate_normal(mu1, sigma1, np.sum(z))
    res[np.where(z == 0)] = np.random.multivariate_normal(mu2, sigma2, N-np.sum(z))
    return z, res

mu1, sigma1 = np.array([-1, -.5]), np.array([[1, -.5], [-.5, 2]])
mu2, sigma2 = np.array([.5, 1]), np.array([[2, .5], [.5, .5]])
z, x = sample_mixture(100, 0.7, mu1, sigma1, mu2, sigma2)
```

Этот код порождает наборы точек из двух гауссианов, проиллюстрированных на рис. 5.5, *а*, и получаются датасеты, похожие на изображённые на рис. 5.5, *б*.

Линейный и квадратичный дискриминантный анализ, конечно же, реализованы в пакете `scikit-learn`, так что не будем пытаться записывать формулы вручную, а просто вызовем соответствующие процедуры обучения:

```
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.discriminant_analysis import QuadraticDiscriminantAnalysis

lda = LinearDiscriminantAnalysis(store_covariance=True)
z_lda = lda.fit(x, z).predict(x)

qda = QuadraticDiscriminantAnalysis(store_covariance=True)
z_qda = qda.fit(x, z).predict(x)
```

В результате линейный дискриминантный анализ обучит разделяющую поверхность, подобную изображённой на рис. 5.6, *а*. На этом графике заливка показывает решения, которые принимает обученная модель: к синему или красному классу отнести точку; толстая белая линия показывает собственно разделяющую поверхность, а тонкие белые линии — поверхности, соответствующие вероятностям $p\,(C_1|\boldsymbol{x}) \in \{0{,}2, 0{,}4, 0{,}6, 0{,}8\}$. Как мы и ожидали, LDA строит линейную разделяющую поверхность и обучает двумерные гауссианы с центрами, похожими на настоящие (они на рис. 5.6, *а* показаны крестиками), и одинаковыми матрицами ковариаций, что тоже видно в форме эллипсов на рис. 5.6, *а*.

А квадратичный дискриминантный анализ, разумеется, всё делает правильно. Его предположения в точности соответствуют порождающему процессу, из которого были получены точки, так что и центры, и матрицы ковариаций QDA обучает довольно точно — они показаны на рис. 5.6, *б*.

Обратите внимание, кстати, что QDA в нашем случае рисует прекрасную разделяющую поверхность, практически оптимальную... но это гиперболическая поверхность, и у неё есть вторая ветвь с другой стороны! Она даже видна слева на

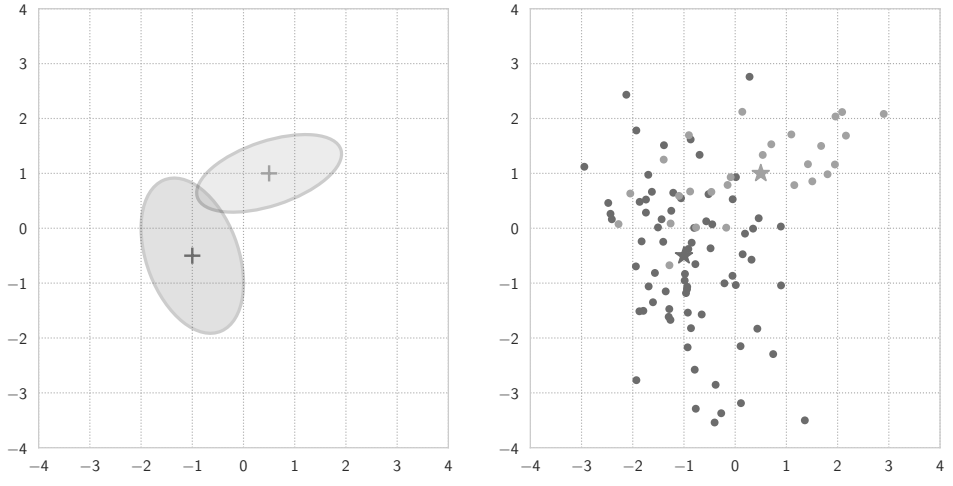
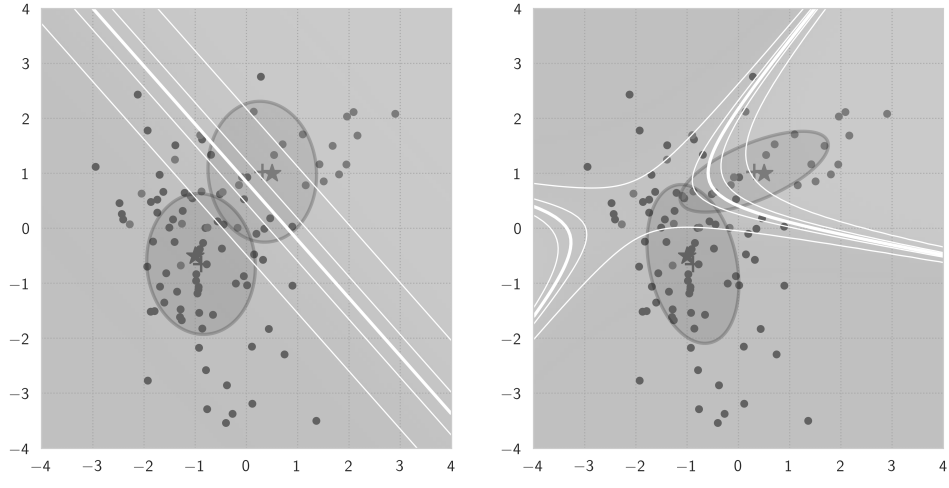


Рис. 5.5. Точки, порождённые из двух гауссианов: (а) параметры распределений; (б) получающиеся датасеты

рис. 5.6, *б*. Выглядит странно, но если наши предположения о нормально порождённых классах соответствуют действительности, то всё верно: слева плотность второго класса (с центром правее на рисунках) снова станет выше, чем у первого класса (с центром левее). На практике, конечно, получить такую разделяющую поверхность немного удивительно, но, опять же, если это действительно гауссианы, то речь в этой второй ветке идёт о сравнении очень близких к нулю вероятностей (плотности гауссианов убывают экспоненциально), и ничего страшного в этом нет, потому что точки там всё равно, скорее всего, не появятся никогда. А если это всё-таки не гауссианы и мы можем ожидать появление точек из первого класса в той далёкой области, значит, не нужно было использовать QDA, ведь его предположения не выполняются. В каком-то смысле это ещё одно подтверждение универсального тезиса о том, что в машинном обучении интерполировать гораздо проще, чем экстраполировать...

LDA и QDA — это примеры очень простых порождающих моделей для классификации. Главный и самый естественный способ усложнить и обобщить этот подход — это перейти к более сложным порождающим моделям, например перейти от обучения гауссианов к обучению смесей гауссианов

$$p\left(\boldsymbol{x}|C_k\right) = \alpha_{1k}\mathcal{N}\left(\boldsymbol{x}|\boldsymbol{\mu}_{1k}, \Sigma_{1k}\right) + \ldots + \alpha_{M_k,k}\mathcal{N}\left(\boldsymbol{x}\middle|\boldsymbol{\mu}_{M_k,k}, \Sigma_{M_k,k}\right), \quad \sum_{i=1}^{M_k}\alpha_{ik} = 1,$$



(а) LDA (б) QDA

Рис. 5.6. Результаты линейного и квадратичного дискриминантного анализа

для каждого класса. Мы пока не умеем обучать параметры смеси гауссианов — для этого потребуется EM-алгоритм, который выходит за рамки этой книги, — но сам классификатор после этого обучения останется тем же оптимальным байесовским:

$$p\left(C_k|\boldsymbol{x}\right) \propto p\left(C_k\right) p\left(\boldsymbol{x}|C_k\right).$$

Были, впрочем, и другие интересные попытки обобщить или улучшить LDA и QDA. Во-первых, в задачах очень высоких размерностей, когда размерность вектора признаков становится больше, чем число точек (такие случаи часто бывают, например, в микробиологии), обучать даже гауссиан становится сложно, ведь эмпирическая матрица ковариаций становится вырожденной.

Поэтому желательно как-то его регуляризовать; например, в работе [233] предлагается регуляризовать матрицу ковариаций каждого класса в QDA или общую матрицу ковариаций в LDA, приближая её к единичной:

$$\Sigma = \alpha\hat{\Sigma} + (1-\alpha)\mathbf{I}.$$

Другой аналогичный подход — *регуляризованный дискриминантный анализ* (regularized discriminant analysis, RDA) всё того же Джерома Фридмана [199]. Здесь мы тоже регуляризуем матрицы ковариаций отдельных классов в квадратичном дискриминантном анализе, но стягиваем их не к единичной, а к общей матрице ковариаций всех точек данных:

$$\Sigma_k = \alpha\hat{\Sigma}_k + (1-\alpha)\hat{\Sigma},$$

где

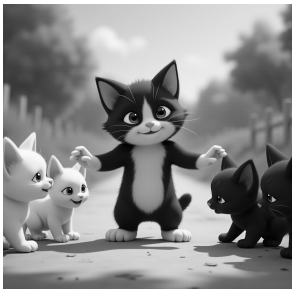
$$\hat{\Sigma}_k = \frac{1}{N_k} \sum_{\boldsymbol{x}_n \in C_k} (\boldsymbol{x}_n - \bar{\boldsymbol{x}}_k)(\boldsymbol{x}_n - \bar{\boldsymbol{x}}_k)^\top, \qquad \hat{\Sigma} = \frac{1}{N} \sum_{n=1}^{N} (\boldsymbol{x}_n - \bar{\boldsymbol{x}})(\boldsymbol{x}_n - \bar{\boldsymbol{x}})^\top.$$

Этот метод разумно применять в случаях, когда размерность данных высока и/или классов в датасете много и в каждом отдельном классе точек маловато, чтобы надёжно обучить независимые гауссианы, а во всём датасете целиком уже вполне достаточно.

Обращаю ваше внимание на то, что работа [233] вышла в 2007 году, когда глубокое обучение уже, казалось бы, победоносно шагало по планете — но в ней классики статистики Хэсти и Тибширани обсуждали и улучшали линейный дискриминантный анализ! Регуляризованный дискриминантный анализ тоже используют и модифицируют до сих пор — вот, например, работа с одной из ведущих конференций по искусственному интеллекту SIGKDD 2006 [652], где RDA используют для маленьких датасетов высокой размерности, а вот новый вариант RDA, появившийся в 2017 году [258]. Методы, которые мы обсуждаем в этой книге, не теряют свою актуальность со временем, у них просто лучше определяется область применимости.

И последнее замечание: в разделе 1.3 мы отнесли порождающие модели к обучению без учителя. И действительно, в данном случае мы обучаем совместное распределение $p(\boldsymbol{x}, y)$ по набору данных $D = \{(\boldsymbol{x}, y)\}$, а не условное распределение $p(y|\boldsymbol{x})$. Однако условное распределение из него легко получается в виде оптимального байесовского классификатора. Но если бы мы обучили только условное распределение $p(y|\boldsymbol{x})$, то есть получили *дискриминирующую* модель (discriminative model), а не порождающую, из неё было бы весьма сложно извлечь порождающие модели хоть для всех $\boldsymbol{x}$, хоть для отдельных классов. В следующем разделе мы как раз такую модель и рассмотрим.

5.2. Логистическая регрессия



Но прежде чем я вам назову эту выгоду, я хочу себя компрометировать лично и потому дерзко объявляю, что все эти прекрасные системы, все эти теории разъяснения человечеству настоящих, нормальных его интересов с тем, чтоб оно, необходимо стремясь достигнуть этих интересов, стало бы тотчас же добрым и благородным, покамест, по моему мненью, одна логистика! Да-с, логистика!

Фёдор Достоевский. Записки из подполья

5.2.1. Как из линейной функции получить вероятности

Мы уже обсудили довольно много разных методов классификации: и выработали геометрическую интуицию, и построили самый настоящий оптимальный байесовский классификатор, жаль только — на очень уж сильных предположениях. Этот оптимальный классификатор получался у нас как порождающая модель и требовал знания того, откуда именно были порождены точки во всех классах; такое предположение редко можно назвать реалистичным.

Оказывается, что можно создать классификатор, имеющий вероятностный смысл, и без настолько разрушительных предположений, разве что порождающей модели из него не получится. Для этого давайте начнём с бинарной классификации и рассмотрим... ну да, как вы наверняка уже догадались, рассмотрим теорему Байеса:

$$p\left(C_1|\boldsymbol{x}\right) = \frac{p\left(\boldsymbol{x}|C_1\right)p\left(C_1\right)}{p\left(\boldsymbol{x}\right)} = \frac{p\left(\boldsymbol{x}|C_1\right)p\left(C_1\right)}{p\left(\boldsymbol{x}|C_1\right)p\left(C_1\right) + p\left(\boldsymbol{x}|C_2\right)p\left(C_2\right)}. \qquad (5.5)$$

Как построить *линейную* модель для классификации? Мы бы хотели построить модель для $p\left(C_1|\boldsymbol{x}\right)$, в которой участвует линейная функция от вектора признаков $\boldsymbol{x}$ (и к ней добавляется тот или иной случайный шум, как в линейной регрессии). Самой простой идеей могла бы стать модель $p\left(C_1|\boldsymbol{x}\right) \sim \boldsymbol{w}^\top\boldsymbol{x}$ (здесь мы опять переходим от возможной аффинной модели со свободным членом к линейной, добавив одну фиктивную размерность, как в подразделе 4.1.2). Но довольно очевидно, что моделировать вероятность линейной функцией напрямую — это плохая идея: вероятность должна быть ограничена отрезком $[0,1]$, а линейная функция неизбежно продолжается до бесконечности в обе стороны, и при таком подходе у нас получались бы отрицательные или большие единицы вероятности.

Формула (5.5) намекает на то, как можно попробовать решить эту проблему. В правой части формулы (5.5) стоят сразу несколько разных переменных, а мы хотели бы моделировать линейной функцией что-нибудь одно. Поэтому давайте разделим числитель и знаменатель на $p\left(\boldsymbol{x}|C_1\right)p\left(C_1\right)$:

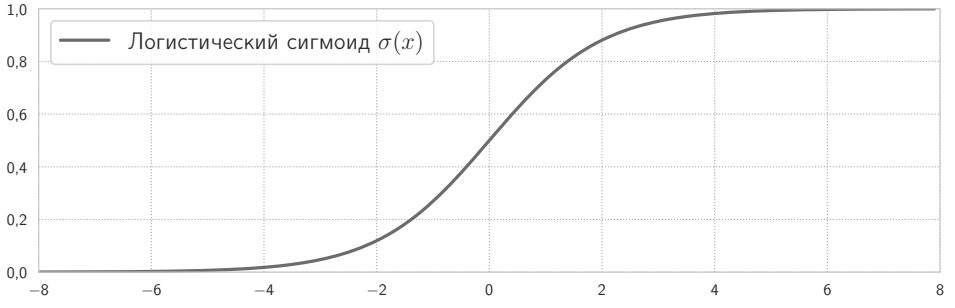


Рис. 5.7. Логистический сигмоид $\sigma(a) = 1/\left(1 + e^{-a}\right)$

$$p\left(C_1|\boldsymbol{x}\right) = \frac{p\left(\boldsymbol{x}|C_1\right)p\left(C_1\right)}{p\left(\boldsymbol{x}|C_1\right)p\left(C_1\right) + p\left(\boldsymbol{x}|C_2\right)p\left(C_2\right)} = \frac{1}{1 + \frac{p(\boldsymbol{x}|C_2)p(C_2)}{p(\boldsymbol{x}|C_1)p(C_1)}}.$$

В качестве следующей попытки можно предложить модель

$$\frac{p\left(\boldsymbol{x}|C_2\right)p\left(C_2\right)}{p\left(\boldsymbol{x}|C_1\right)p\left(C_1\right)} \sim \boldsymbol{w}^\top \boldsymbol{x}.$$

Величина $\frac{p(\boldsymbol{x}|C_2)p(C_2)}{p(\boldsymbol{x}|C_1)p(C_1)}$ совпадает с $\frac{p(C_2|\boldsymbol{x})}{p(C_1|\boldsymbol{x})}$ (знаменатели в двух формулах (5.5) сократятся). Она называется *шансами* (odds) и имеет вполне понятный бытовой смысл: когда говорят «шансы пять к двум» или «один к ста», имеют в виду именно эту величину, отношение апостериорных вероятностей двух классов. Это решает проблему с вероятностями больше единицы, но вот отрицательными шансы всё-таки быть не могут.

Теперь остался только один, понятный и естественный шаг: чтобы из величины со значениями из $(0,\infty)$ сделать величину со значениями из $(-\infty,\infty)$, достаточно просто прологарифмировать! Мы приходим к такому выражению:

$$p\left(C_1|\boldsymbol{x}\right) = \frac{1}{1 + e^{-\log\frac{p(\boldsymbol{x}|C_1)p(C_1)}{p(\boldsymbol{x}|C_2)p(C_2)}}},$$

и теперь уже действительно можно смело моделировать

$$\log\frac{p\left(\boldsymbol{x}|C_1\right)p\left(C_1\right)}{p\left(\boldsymbol{x}|C_2\right)p\left(C_2\right)} \sim \boldsymbol{w}^\top \boldsymbol{x}.$$

Величина $\log\frac{p(\boldsymbol{x}|C_1)p(C_1)}{p(\boldsymbol{x}|C_2)p(C_2)}$ называется соответственно логарифмом шансов (log-odds; по-русски, увы, столь же кратко и ёмко не получается), и это самая естественная величина, которую можно в контексте классификации моделировать

линейной функцией (кстати, в разделе 6.6 мы придадим использованному здесь слову «естественный» и формальный смысл). Получилось, что апостериорная вероятность через эту величину выражается функцией

$$\sigma(a) = \frac{1}{1+e^{-a}}.$$

Функция σ называется *логистическим сигмоидом* (logistic sigmoid); уже говорилось, что слово «сигмоид» означает, что график этой функции (который, кстати, изображён на рис. 5.7) монотонно возрастает от одной горизонтальной асимптоты до другой, то есть напоминает букву S. Этой букве вообще как-то не везёт в математике: с одной стороны её сжали до знака интеграла[1], а с другой, как видите, растянули аж до бесконечности. Так что разных сигмоидов на свете очень много, а этот конкретный называется логистическим, как и вся регрессия. Обратная же функция называется *логитом* (logit):

$$\operatorname{logit}(p) = \sigma^{-1}(p) = \log\frac{p}{1-p} \text{ для } p \in (0,1).$$

История и первая мотивация логистического сигмоида — это отдельная интересная история. Давайте рассмотрим простейшую модель роста популяции в виде дифференциального уравнения. Обозначим во время t размер популяции некоторого вида (кроликов, людей, бактерий...) через $x(t)$, и, как обычно в дифференциальных уравнениях, обозначим $\dot{x}(t) = \frac{\mathrm{d}x}{\mathrm{d}t}$. Самым простым предположением здесь будет предположение о том, что скорость роста популяции $\dot{x}(t)$ растёт пропорционально своему значению $x(t)$ (больше кроликов смогут делать пропорционально больше новых кроликов):

$$\dot{x}(t) = \beta x(t).$$

У этого дифференциального уравнения, разумеется, только одно семейство решений — экспоненциальный рост

$$x(t) = Ae^{\beta t}, \quad \text{где} \quad A = x(0).$$

Это разумная модель на каких-то начальных этапах роста популяции, пока у вида нет никаких ограничителей для роста (как получалось у кроликов в Австралии, пока их не стали изводить целенаправленно, или у коронавируса на начальном этапе пандемии, как мы обсуждали в разделе 4.5). Но если мы будем пытаться продолжать предсказания этой модели и экстраполировать их всё дальше

[1] Если быть точным, знак интеграла $\int$ Лейбниц изобразил по аналогии с буквой ſ («длинная s», с которой в латыни начинается слово ſumma, «сумма»). Но «длинная s» — это та же самая буква, что и «короткая», просто до XIX века написание s употреблялось только на конце слов; а в готическом шрифте и сейчас такая же ситуация.

и дальше, мы неизбежно сначала придём к мальтузианской[1] катастрофе, а затем и вовсе заместим все атомы во Вселенной австралийскими кроликами; нечто подобное получалось у нас в экспоненциальной модели развития эпидемии в разделе 4.5. Очевидно, экспоненциальный рост не может продолжаться вечно.

Это было ясно ещё в начале XIX века бельгийскому астроному и статистику Адольфу Кетле и его ученику Пьеру-Франсуа Ферхюльсту[2]. Они добавили в базовую модель неограниченного экспоненциального роста дополнительное слагаемое, которое представляло собой постепенно увеличивающееся сопротивление этому росту:

$$\dot{x}(t) = \beta x(t) - \phi(x(t)).$$

Они исследовали разные формы функции ϕ и в качестве наиболее естественной и удобной формы выбрали квадратичную функцию от $x(t)$. Тогда само дифференциальное уравнение можно переписать как

$$\dot{x}(t) = \beta x(t)\left(\Omega - x(t)\right),$$

и теперь в нём явным образом виден *уровень насыщения* популяции Ω: когда $x(t)$ превышает Ω, производная становится отрицательной и $x(t)$ возвращается к значению Ω. Если теперь переобозначить $p(t) = x(t)/\Omega$, мы получим уравнение

$$\dot{p}(t) = \beta p(t)\left(1 - p(t)\right),$$

решением которого будет семейство функций (проверьте!)

$$p(t) = \frac{e^{\alpha+\beta t}}{1 + e^{\alpha+\beta t}} = \frac{1}{1 + e^{-(\alpha+\beta t)}}$$

[1] *Томас Роберт Мальтус* (Thomas Robert Malthus, 1766–1832) — английский священник и учёный. Всю жизнь служил священником, занимал кафедру профессора истории и политической экономии, жил очень скромно, но отказывался от государственных должностей. Главная книга Мальтуса — *An Essay on the Principle of Population* [371], в которой он развил теорию о том, что народонаселение растёт в геометрической прогрессии, а ресурсы Земли ограничены, так что нас всех ждёт крайняя бедность, если срочно что-то не сделать с рождаемостью. Доиндустриальные общества действительно могут попасть в так называемую «мальтузианскую ловушку» (malthusian trap) чрезмерной рождаемости, но применительно к популяции Земли теория Мальтуса была слишком пессимистична насчёт технического прогресса и, конечно, не могла учитывать тогда ещё незаметный демографический переход, из-за которого развитые страны сейчас беспокоятся скорее о депопуляции.

[2] *Пьер-Франсуа Ферхюльст* (Pierre François Verhulst, 1804–1849) — французский математик. Известен Ферхюльст главным образом работами, которые мы здесь и обсуждаем [620–622]. К сожалению, он много болел, не так много успел и умер молодым. В 1830 году Ферхюльст был на лечении в Италии, когда услышал о революции в Париже и объявлении независимости Бельгии. На него это произвело такое впечатление, что Ферхюльст написал проект демократической конституции для Папской области и показал его нескольким кардиналам. Кардиналы заинтересовались, проект похвалили, но полицию на всякий случай всё-таки вызвали; Ферхюльст забаррикадировался в своей квартире, но в итоге вынужден был сдаться силам правопорядка и был депортирован из страны.

для любой константы α. Эту функцию Ферхюльст и назвал *логистической*. В своих работах [620, 622] он применил этот метод к населению Франции, Бельгии, Эссекса и России, используя данные вплоть до 1833 года. Ферхюльст обучил логистические кривые (ровно по трём точкам каждую, что, конечно, не очень надёжно) и получил вполне правдоподобные результаты: по его подсчётам, уровень насыщения населения Бельгии составляет 6,6 миллиона человек, а Франции — 40 миллионов. Конечно, сейчас в Бельгии живёт более 11 миллионов человек, а во Франции — более 65, но, согласитесь, с первой половины XIX века технический прогресс изрядно ускорился и в результате интенсивность сельского хозяйства и возможная плотность населения сильно увеличились.

У логистического сигмоида есть несколько простых, но важных свойств, которые нам ещё помогут в этой главе. Во-первых, он симметричен относительно нуля:

$$\sigma(-a) = \frac{1}{1+e^{a}} = \frac{e^{-a}}{e^{-a}+1} = 1 - \sigma(a),$$

а во-вторых, производную логистического сигмоида можно красиво выразить через сам сигмоид:

$$\sigma'(a) = -\frac{-e^{-a}}{(1+e^{-a})^2} = \frac{1}{1+e^{-a}}\frac{e^{-a}}{1+e^{-a}} = \sigma(a)\left(1-\sigma(a)\right).$$

Из этой формулы сразу видно, что значение производной стремится к нулю и на плюс бесконечности, где $\sigma(a) \to 1$, и на минус бесконечности, где $\sigma(a) \to 0$.

5.2.2. Максимизация правдоподобия

Теперь можно возвращаться к нашей модели. *Логистическая регрессия* — это модель, в которой

$$p\left(C_1|\boldsymbol{x}\right) = \sigma(\boldsymbol{w}^\top \boldsymbol{x}), \qquad p\left(C_2|\boldsymbol{x}\right) = 1 - \sigma(\boldsymbol{w}^\top \boldsymbol{x}).$$

Это значит, что общее правдоподобие модели для некоторого набора данных $D = \{(\boldsymbol{x}_n, t_n)\}_{n=1}^{N}$, где $t_n = 1$, если $\boldsymbol{x}_n \in C_1$, и $t_n = 0$, если $\boldsymbol{x}_n \in C_2$, будет выглядеть как

$$p\left(D|\boldsymbol{w}\right) = \prod_{n=1}^{N} p\left(y_n|\boldsymbol{x}_n, \boldsymbol{w}\right) = \prod_{n=1}^{N} p\left(C_1|\boldsymbol{x}\right)^{t_n} p\left(C_2|\boldsymbol{x}\right)^{1-t_n} =$$

$$= \prod_{n=1}^{N} \sigma\left(\boldsymbol{w}^\top \boldsymbol{x}\right)^{t_n} \left(1 - \sigma\left(\boldsymbol{w}^\top \boldsymbol{x}\right)\right)^{1-t_n}. \quad (5.6)$$

Обратите внимание, что в формуле (5.6) мы сделали важный трюк, представив выбор из двух альтернатив в мультипликативной форме. Это можно записать чуть более общо: если нам нужно выбрать значение a_1 при некотором условии A_1, выбрать значение a_2 при некотором условии A_2 и так далее до a_K при условии A_K, эту последовательность **elif** (в предположении, что разные A_k не могут выполняться одновременно) можно записать как

$$a_1^{[\![A_1]\!]} a_2^{[\![A_2]\!]} \cdot \ldots \cdot a_K^{[\![A_K]\!]},$$

где $[\![A]\!]$ обозначает индикатор условия A, то есть равен единице, если A выполняется, и нулю в противном случае. Это, разумеется, не единственный способ записать такую цепочку условий. Можно, например, сделать всё то же самое в аддитивной форме: выражение

$$[\![A_1]\!]\, a_1 + [\![A_2]\!]\, a_2 + \ldots + [\![A_K]\!]\, a_k$$

было бы абсолютно эквивалентно. Но нам нужно записать правдоподобие модели, которое представляет собой большое произведение и от которого мы потом захотим брать логарифм, поэтому мультипликативная форма гораздо удобнее.

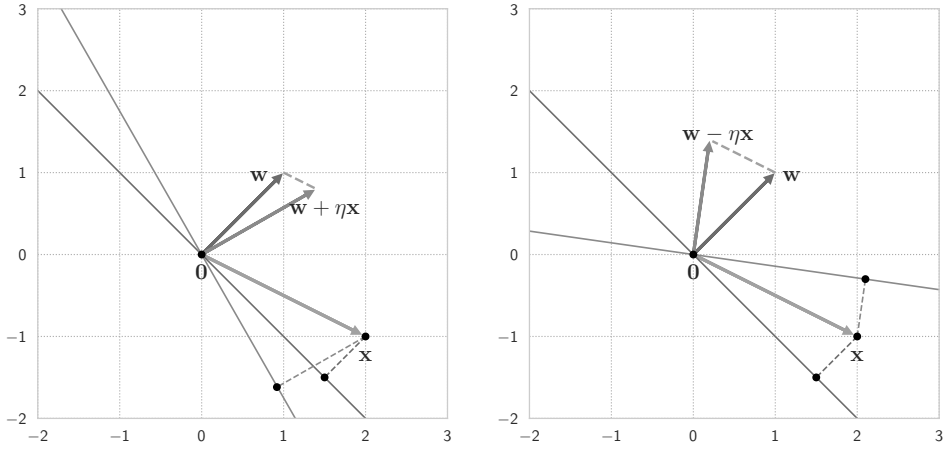
Теперь выходит, что задача поиска параметров максимального правдоподобия — это задача максимизации по весам модели $\boldsymbol{w}$ логарифма правдоподобия

$$\ln p\left(D|\boldsymbol{w}\right) = \sum_{n=1}^{N} \left[t_n \ln \sigma\left(\boldsymbol{w}^\top \boldsymbol{x}_n\right) + \left(1 - t_n\right) \ln\left(1 - \sigma\left(\boldsymbol{w}^\top \boldsymbol{x}_n\right)\right)\right].$$

Обратите внимание, что, хотя эта задача на вид очень похожа на максимизацию правдоподобия линейной регрессии, которую мы проводили в подразделе 4.1.2, на самом деле она стала принципиально сложнее. В линейной регрессии мы максимизировали квадратичную функцию, а здесь никуда не денешься от сигмоидов, которые делают аналитическое решение крайне маловероятным.

Но давайте не будем отчаиваться; целевая функция всё-таки непрерывна, так что градиентный спуск так или иначе должен сработать. Оказывается, что градиент в данном случае благодаря свойствам логистического сигмоида имеет очень простую и логичную форму:

$$\begin{aligned}
\nabla_{\boldsymbol{w}} \ln p\left(D|\boldsymbol{w}\right) &= \sum_{n=1}^{N} \left[\frac{t_n \sigma'\left(\boldsymbol{w}^\top \boldsymbol{x}_n\right)}{\sigma\left(\boldsymbol{w}^\top \boldsymbol{x}_n\right)} \boldsymbol{x}_n - \frac{\left(1 - t_n\right) \sigma'\left(\boldsymbol{w}^\top \boldsymbol{x}_n\right)}{1 - \sigma\left(\boldsymbol{w}^\top \boldsymbol{x}_n\right)} \boldsymbol{x}_n \right] = \\
&= \sum_{n=1}^{N} \left[\frac{t_n \sigma\left(\boldsymbol{w}^\top \boldsymbol{x}_n\right)\left(1 - \sigma\left(\boldsymbol{w}^\top \boldsymbol{x}_n\right)\right)}{\sigma\left(\boldsymbol{w}^\top \boldsymbol{x}_n\right)} - \frac{\left(1 - t_n\right) \sigma\left(\boldsymbol{w}^\top \boldsymbol{x}_n\right)\left(1 - \sigma\left(\boldsymbol{w}^\top \boldsymbol{x}_n\right)\right)}{1 - \sigma\left(\boldsymbol{w}^\top \boldsymbol{x}_n\right)} \right] \boldsymbol{x}_n = \\
&= \sum_{n=1}^{N} \left[t_n \left(1 - \sigma\left(\boldsymbol{w}^\top \boldsymbol{x}_n\right)\right) - \left(1 - t_n\right) \sigma\left(\boldsymbol{w}^\top \boldsymbol{x}_n\right) \right] \boldsymbol{x}_n = \sum_{n=1}^{N} \left(t_n - \sigma\left(\boldsymbol{w}^\top \boldsymbol{x}_n\right)\right) \boldsymbol{x}_n.
\end{aligned}$$



(а) $\boldsymbol{x}$ классифицирован правильно (б) $\boldsymbol{x}$ классифицирован неправильно

Рис. 5.8. Градиент логарифма правдоподобия логистической регрессии:
(а) влияние примера, в котором ответ правильный;
(б) влияние примера, в котором ответ неправильный

Давайте посмотрим на то, что говорит нам эта формула, с геометрической точки зрения. Каждое слагаемое этой суммы — это влияние одной точки данных $(\boldsymbol{x}_n, t_n)$ на общий градиент логарифма правдоподобия. Оказывается, что это влияние коллинеарно самому вектору $\boldsymbol{x}_n$, причём вот с каким коэффициентом:

- если модель дала правильный ответ, то есть $\sigma\left(\boldsymbol{w}^\top \boldsymbol{x}_n\right)$ близко к t_n (например, $t_n = 1$ и $\sigma\left(\boldsymbol{w}^\top \boldsymbol{x}_n\right) > \frac{1}{2}$), то величина $\left(t_n - \sigma\left(\boldsymbol{w}^\top \boldsymbol{x}_n\right)\right)$ будет мала по модулю и будет слегка «доворачивать» вектор градиента ещё дальше в нужную сторону (рис. 5.8, *а*, где расстояние от точки $\boldsymbol{x}$ до плоскости стало больше после доворачивания);
- а если модель ошиблась, то есть $\sigma\left(\boldsymbol{w}^\top \boldsymbol{x}_n\right)$ совсем не похожа на t_n (например, $t_n = 1$ и $\sigma\left(\boldsymbol{w}^\top \boldsymbol{x}_n\right) < \frac{1}{2}$), то величина $\left(t_n - \sigma\left(\boldsymbol{w}^\top \boldsymbol{x}_n\right)\right)$ будет велика по модулю и будет стараться повернуть вектор градиента так, чтобы всё-таки классифицировать правильно; это, конечно, может не получиться сделать за один шаг в зависимости от скорости обучения, но тенденция несомненна (на рис. 5.8, *б* всё получилось сразу).

Таким образом, один способ обучать логистическую регрессию — это просто градиентный спуск. Но можно организовать дело и более эффективно. Как мы обсуждали в разделе 3.5, на практике всегда лучше переходить от методов

оптимизации первого порядка к методам второго порядка, если есть такая возможность. В случае линейной регрессии целевая функция была квадратичной, а значит, оптимизация методом Ньютона сразу давала окончательный ответ, который был приведён в формуле (4.2) для максимизации правдоподобия и в формулах (4.23, 4.24) для максимизации апостериорного распределения.

С логистической регрессией так просто не получится, но метод Ньютона всё равно применить можно. Для этого нужно найти гессиан логарифма правдоподобия, то есть матрицу вторых производных:

$$\boldsymbol{H} = \nabla_{\boldsymbol{w}} \nabla_{\boldsymbol{w}} \ln p\left(D|\boldsymbol{w}\right) = \sum_{n=1}^{N} \sigma_n(1-\sigma_n)\boldsymbol{x}_n\boldsymbol{x}_n^\top = \boldsymbol{X}^\top \boldsymbol{R}\boldsymbol{X},$$

где $\sigma_n = \sigma(\boldsymbol{w}^\top \boldsymbol{x}_n)$, а $\boldsymbol{R}$ — это диагональная матрица

$$\boldsymbol{R} = \begin{pmatrix} \sigma_1(1-\sigma_1) & 0 & \dots & 0 \\ 0 & \sigma_2(1-\sigma_2) & \dots & 0 \\ \vdots & \vdots & & \vdots \\ 0 & 0 & \dots & \sigma_N(1-\sigma_N) \end{pmatrix}.$$

В итоге получается следующая формула шага оптимизации:

$$\boldsymbol{w}^{\text{new}} = \boldsymbol{w}^{\text{old}} - \left(\boldsymbol{X}^\top \boldsymbol{R}\boldsymbol{X}\right)^{-1} \boldsymbol{X}^\top (\boldsymbol{\sigma} - \boldsymbol{t}) = \left(\boldsymbol{X}^\top \boldsymbol{R}\boldsymbol{X}\right)^{-1} \boldsymbol{X}^\top \boldsymbol{R}\boldsymbol{z}, \qquad (5.7)$$

где $\boldsymbol{\sigma}$ — вектор текущих предсказаний модели, то есть вектор, состоящий из σ_n, а через $\boldsymbol{z}$ мы переобозначили

$$\boldsymbol{z} = \boldsymbol{X}\boldsymbol{w}^{\text{old}} - \boldsymbol{R}^{-1}\left(\boldsymbol{\sigma} - \boldsymbol{t}\right).$$

Обратите внимание, что у нас получилось как бы решение задачи минимизации квадратического отклонения (как в формуле (4.2)), но с матрицей весов $\boldsymbol{R}$, умножающей каждое из этих отклонений на вес $\sigma_n(1-\sigma_n)$, который максимален для ситуаций, когда мы менее всего уверены в ответе, $\sigma_n = \frac{1}{2}$, и стремится к нулю, когда наши ответы становятся уверенными.

Кроме того, теперь у нас не получится дойти до ответа методом Ньютона за один шаг: формулу (5.7) нужно применять итеративно, до сходимости. Отсюда и название: это метод *итеративных перевзвешенных наименьших квадратов* (iterative reweighted least squares, IRLS).

Тот же алгоритм, разумеется, можно применить и ко многим другим вероятностным моделям. Для успеха нужно лишь, чтобы гессиан можно было подсчитать, а целевая функция оставалась выпуклой; эти требования выполняются и в других случаях, о которых можно прочесть, например, в обзорах [137, 227].

5.2.3. Другие сигмоиды и пробит-регрессия

И ещё одно замечание, которое имеет самостоятельную ценность, но вдобавок ещё и несколько неожиданным образом пригодится нам уже в следующем разделе. Логистический сигмоид — это очень естественный способ преобразовать произвольное вещественное число в вероятность: он, как вы видели выше, следует из теоремы Байеса и имеет естественную вероятностную интерпретацию. Но это, конечно, не единственный метод! Разных сигмоидов можно придумать очень много, и теоретически для каждого из них можно построить свой собственный метод классификации.

Более того, сигмоиды имеют естественную вероятностную интерпретацию. Функция от одной переменной, которая монотонно возрастает от нуля до единицы по мере того, как её аргумент растёт от $-\infty$ до ∞, — это в точности функция распределения для некоторого распределения вероятностей на $\mathbb{R}$! Поэтому одна из самых интересных альтернатив логистическому сигмоиду — это функция распределения стандартного гауссиана

$$\Phi(a) = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^{a} e^{-\frac{x^2}{2}} \mathrm{d}x.$$

Мы уже использовали её в разделе 4.5 для выделения признаков для линейной регрессии. А сейчас увидим, что на этой функции можно построить классификационную модель точно так же, как на логистическом сигмоиде, моделируя

$$p\left(C_1|\boldsymbol{x}\right) = \Phi\left(\boldsymbol{w}^\top \boldsymbol{x}\right)$$

и получая общее правдоподобие вида

$$p\left(D|\boldsymbol{w}\right) = \prod_{n=1}^{N} \Phi\left(\boldsymbol{w}^\top \boldsymbol{x}\right)^{t_n} \left(1 - \Phi\left(\boldsymbol{w}^\top \boldsymbol{x}\right)\right)^{1-t_n}.$$

Эта модель называется *пробит-регрессией* (probit regression). Это название появилось в работе Честера Блисса *The Method of Probits*, вышедшей в *Science* в 1934 году [69][1], в которой автор предлагал метод обработки данных, например, токсикологического характера: какой процент насекомых будет убит той или иной дозой инсектицида[2]. Такие данные тоже имеют сигмоидальную форму

[1] Впрочем, Блисс ссылается и на работы предшественников, которые, в свою очередь, можно проследить как минимум до трудов основоположника психофизиологии Густава Фехнера [177]. Не будем дальше углубляться в историю, а интересующимся подробностями рекомендую заглянуть в работу [123].

[2] В качестве примера, кстати, Блисс исследовал смертность тлей от никотинового спрея, так что от дурной человеческой привычки исторически страдали не только лошади.

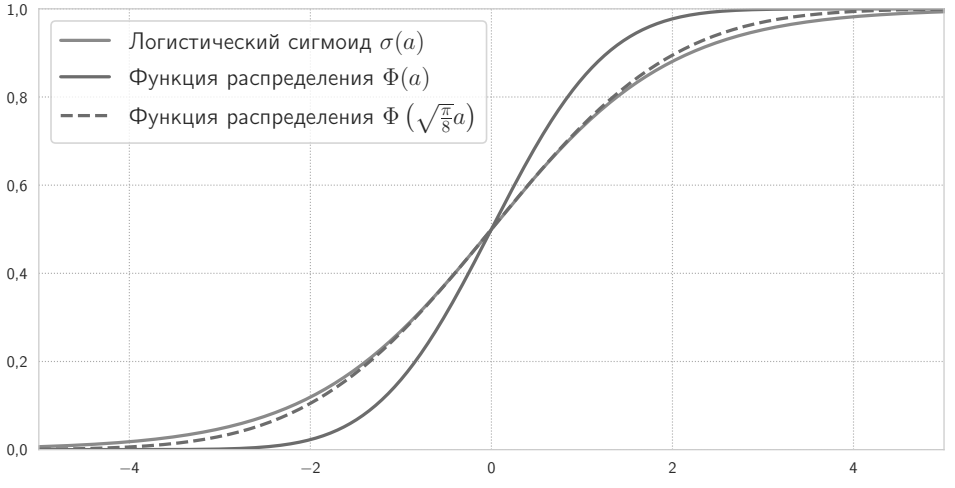


Рис. 5.9. Сравнение логистического сигмоида $\sigma(a)$ и функции распределения стандартного гауссиана с подходящей нормировкой

(от 0 % убитых насекомых до 100 % по оси ординат), и Блисс предлагал преобразование, способное превратить эту сигмоиду в прямую, для чего отображал согласно специальным таблицам (по сути, таблицам значений функции Φ^{-1}) проценты убитых в промежуточные значения, которые называл *probability units*, или сокращённо *probits*. Соответственно, в статистике принято называть функцию $\text{probit}(p) = \Phi^{-1}(p)$ *пробит-функцией*.

Пробит-регрессию можно также представить в виде модели со скрытыми переменными. Введём вспомогательную случайную переменную с нормально распределённым шумом:

$$t' = \boldsymbol{w}^\top \boldsymbol{x} + \epsilon, \quad \text{где} \quad \epsilon \sim \mathcal{N}\left(\epsilon|0{,}1\right).$$

Тогда

$$p\left(t' > 0\right) = p\left(\epsilon > -\boldsymbol{w}^\top \boldsymbol{x}\right) = p\left(\epsilon < \boldsymbol{w}^\top \boldsymbol{x}\right) = \Phi(\boldsymbol{w}^\top \boldsymbol{x}),$$

потому что $\mathcal{N}\left(\epsilon|0{,}1\right)$ симметрично. А значит, $p\left(C_1|\boldsymbol{x}\right) = p\left(t' > 0\middle|\boldsymbol{x}\right)$, и мы можем использовать вспомогательную переменную t' вместо того, чтобы иметь дело с $\Phi(\boldsymbol{w}^\top \boldsymbol{x})$ напрямую. Впрочем, в случае пробит-регрессии это не так уж важно.

Пробит-регрессия была очень популярна в статистике в 1940-е и 1950-е годы и стала фактически основной моделью для обработки данных бинарной классификации как в биомедицинских приложениях, так и в экономике [16, 180]. Но

затем, во многом благодаря трудам Джозефа Берксона [60, 61], учёные постепенно осознали преимущества логистической регрессии (главным образом аналитические преимущества — вы уже видели, что логистический сигмоид обладает многими удобными свойствами).

А по сути логистический сигмоид и функция Φ очень похожи. На рис. 5.9 изображены эти две кривые, но ещё вдобавок изображён график пробит-функции, в которой подобран такой коэффициент, чтобы σ и Φ были как можно более похожи друг на друга. Для этого давайте аппроксимируем $\sigma(a)$ через пробит-функцию так, чтобы приравнять и значение в нуле — оно, правда, уже по умолчанию равно $\frac{1}{2}$, — и наклон в нуле, то есть значение производной. Производная сигмоида в нуле

$$\sigma'(0) = \sigma(0)\,(1 - \sigma(0)) = \frac{1}{4},$$

то есть нам нужно перемасштабировать аргумент $\Phi(a)$ так, чтобы приравнять

$$\Phi'(\lambda a) = \lambda \frac{1}{\sqrt{2\pi}} e^{-\frac{\lambda^2 a^2}{2}}$$

в нуле к $\frac{1}{4}$; получается, что

$$\sigma(a) \approx \Phi(\lambda a) \quad \text{для} \quad \lambda = \sqrt{\frac{\pi}{8}},$$

то есть на рис. 5.9 изображён график функции $\Phi(\sqrt{\frac{\pi}{8}}a)$. Как видите, после такого линейного преобразования (которое для наших моделей совершенно несущественно — веса можно просто разделить на ту же константу) разницу уже совсем трудно увидеть; Φ всего лишь немного быстрее приближается к своим асимптотам.

Более того, логистическую и пробит-регрессию можно рассматривать как части одного и того же семейства — *обобщённых линейных моделей* (generalized linear models, GLM). О них мы поговорим в разделе 6.7, а сейчас пора двигаться дальше. На очереди — полноценный байесовский вывод в линейной регрессии, который позволит нам поговорить ещё об одной очень полезной идее.

5.3. Байесовский вывод в логистической регрессии

Угрюмы скал решённые отвесы.
Пространство молкнет... Отдых от труда!
И близко разуму раскрытые завесы...
И кто-то, вещий, шепчет: Навсегда.

Всё в мире ясно, понято, раскрыто...
Земля и небо — формула, скелет,
В котором всё исчислено и слито,
И прежнего обмана больше нет.

Юргис Балтрушайтис. Приближение

5.3.1. Лапласовская аппроксимация

В разделе 4.4 мы провели полноценный байесовский вывод для задачи линейной регрессии. Тогда оказалось, что даже байесовские предсказания можно подсчитать точно, аналитически, потому что в обычной линейной регрессии всё так или иначе сводится к свёрткам гауссианов, а это дело не то чтобы простое, но чисто техническое и уж точно находящееся в пределах человеческих сил.

С логистической регрессией так не выйдет — логистические сигмоиды всё испортят и никакие интегралы аналитически взять не получится. Но мы всё равно не будем сдаваться. Для этого начнём с небольшого лирического отступления; нам понадобится одна изящная и важная на практике идея: *лапласовская аппроксимация* распределений вероятностей.

Предположим, что мы хотим приблизить сложное распределение вероятностей при помощи более простого. В машинном обучении такие задачи возникают сплошь и рядом, и обычно речь идёт о том, как приблизить апостериорное распределение модели со скрытыми переменными, у которого всегда масса локальных максимумов и крайне сложная структура.

Подробное рассмотрение таких методов в задачу этой книги не входит (хотя я не оставляю надежды, что о вариационных приближениях мы с вами когда-нибудь ещё поговорим), поэтому здесь мы зададимся более простым вопросом: что, если мы хотим приблизить распределение вероятностей чем-нибудь более простым *локально*, в окрестности одного его локального максимума? Или, что то же самое, как аппроксимировать унимодальное распределение, то есть такое распределение, у которого, по нашему предположению, только один «важный» локальный максимум, а «хвостами» можно пренебречь? Давайте для простоты пока говорить об одномерных распределениях:

$$p(z) = \frac{1}{Z} f(z), \qquad z \in \mathbb{R}.$$

Кстати, здесь мы будем предполагать, что сами значения плотности $p(z)$ мы можем и не знать, а знать только значения некоторой функции $f(z)$, *пропорциональной* $p(z)$. Так часто бывает в машинном обучении: интересующие нас распределения часто получаются как апостериорные распределения, подсчитанные по формуле $p(\boldsymbol{\theta}|D) \propto p(\boldsymbol{\theta})\, p(D|\boldsymbol{\theta})$, и если правая часть в этой формуле обычно дана буквально по построению модели, то нормировочную константу найти бывает не так легко.

Во-первых, давайте сразу зафиксируем, *чем* будем аппроксимировать $p(z)$. Поскольку речь идёт об унимодальном распределении, крайне естественно будет искать приближение в форме гауссиана. Но заметим, что для этого недостаточно просто найти среднее и дисперсию (матрицу ковариаций) всего распределения $p(z)$: мы хотим получить именно *локальную* аппроксимацию, то есть средним гауссиана должна быть интересующая нас точка максимума распределения $p(z)$, а не его глобальный центр тяжести.

Как вообще ищут локальные аппроксимации? Все мы знаем, по сути, ровно один метод: разложить функцию в окрестности нужной точки по формуле Тейлора[1]. Но формула Тейлора даёт полиномиальные аппроксимации, а в данном случае нам нужно приблизить одно распределение вероятностей другим. Если мы возьмём, например, первые два члена ряда Тейлора функции $f(z)$, мы получим некую параболу, значения которой рано или поздно станут отрицательными (мы говорим о максимуме, поэтому понятно, что ветви параболы будут направлены вниз), что вряд ли нас устраивает.

Здесь помогает достаточно стандартный трюк, который мы уже не раз применяли: давайте прологарифмируем! У гауссиана в экспоненте стоит как раз квадратичная функция, так что локальная аппроксимация гауссианом — это в точности аппроксимация логарифма плотности некоторой квадратичной функцией.

Формально говоря, давайте рассмотрим плотность нашего распределения как функцию одной непрерывной переменной $p(z) = \frac{1}{Z} f(z)$ и предположим, что интересующий нас максимум достигается в точке z_0. Для того чтобы аппроксимировать $f(z)$ функцией с положительными значениями, рассмотрим функцию $\ln f(z)$ и разложим её в ряд Тейлора:

$$\ln f(z) \approx \ln f(z_0) + \frac{\partial \ln f}{\partial z} \,|_{z=z_0}\, (\ln f(z) - \ln f(z_0)) - \frac{1}{2} A (z - z_0)^2,$$

[1] *Брук Тейлор* (Brook Taylor, 1685–1731) — английский математик, один из немногих англичан (наряду разве что с Исааком Ньютоном), кто мог выдерживать конкуренцию с правившей в то время бал в математике семьёй Бернулли. Свою знаменитую формулу он опубликовал в труде *Methodus Incrementorum Directa et Inversa*, изданном в 1715 году. К сожалению, Тейлор писал весьма кратко и путано, и его публикации долго оставались непонятыми — вот и этот труд был мало кому известен до 1772 года, когда его заметил и по достоинству оценил Лагранж. В конце жизни Тейлор отошёл от математики и занялся... пусть будет религиозной философией, хотя это очень мягко сказано: две его (оставшиеся неизданными) рукописи носили названия *On the Jewish Sacrifices* и *On the Lawfulness of Eating Blood*.

где слагаемое первой степени пропадает, потому что z_0 — это точка максимума, в которой производная равна нулю, а константа A равна

$$-\frac{d^2}{dz^2}\ln f(z)\mid_{z=z_0}.$$

И теперь мы приближаем

$$f(z)\approx f(z_0)e^{-\frac{A}{2}(z-z_0)^2};$$

после нормализации (упражнение для вас: найдите нормировочную константу!) это будет как раз гауссиан. Обратите внимание, что исходная нормировочная константа Z нам не потребовалась, ведь всё равно в результате мы хотим получить распределение вероятностей, и нормировку нужно делать заново.

Лапласовскую аппроксимацию можно напрямую обобщить и на многомерные распределения: для исходной плотности $p(\boldsymbol{z})=\frac{1}{Z}f(\boldsymbol{z})$ мы получаем

$$f(\boldsymbol{z})\approx f(\boldsymbol{z}_0)e^{-\frac{1}{2}(\boldsymbol{z}-\boldsymbol{z}_0)^\top A(\boldsymbol{z}-\boldsymbol{z}_0)},$$

где

$$A=-\nabla\nabla\ln f(\boldsymbol{z})\mid_{z=z_0}.$$

5.3.2. Обобщённое нормальное распределение и формула Стирлинга

Давайте приведём пример лапласовской аппроксимации. Приближать будем новое для нас и интересное само по себе распределение: *обобщённое нормальное распределение* (skew normal distribution), которое чем-то похоже на нормальное, но имеет дополнительную степень свободы и благодаря ей может быть несимметричным. Плотность стандартного обобщённого нормального распределения определяется как

$$p\left(x|\lambda\right)=2p_{\mathcal{N}}\left(x\right)\Phi(\lambda x),$$

где $p_{\mathcal{N}}$ — это плотность стандартного нормального распределения, а Φ — уже знакомая нам пробит-функция, то есть

$$p\left(x|\lambda\right)=\frac{2}{\sqrt{2\pi}}e^{-\frac{x^2}{2}}\int_{-\infty}^{\lambda x}\frac{1}{\sqrt{2\pi}}e^{-\frac{t^2}{2}}\mathrm{d}t. \tag{5.8}$$

Параметр λ здесь как раз контролирует степень несимметричности: при $\lambda=0$ мы получаем стандартное нормальное распределение. К выражению (5.8) можно ещё добавить среднее и дисперсию, сделав стандартную замену переменных $x\mapsto\frac{x-\mu}{\sigma}$, и тогда мы получим плотность обобщённого нормального распределения с тремя параметрами, как ожидалось:

$$p\left(x|\mu,\sigma,\lambda\right)=\frac{2}{\sigma\sqrt{2\pi}}e^{-\frac{(x-\mu)^2}{2\sigma^2}}\int_{-\infty}^{\lambda\left(\frac{x-\mu}{\sigma}\right)}\frac{1}{\sqrt{2\pi}}e^{-\frac{t^2}{2}}\mathrm{d}t. \tag{5.9}$$

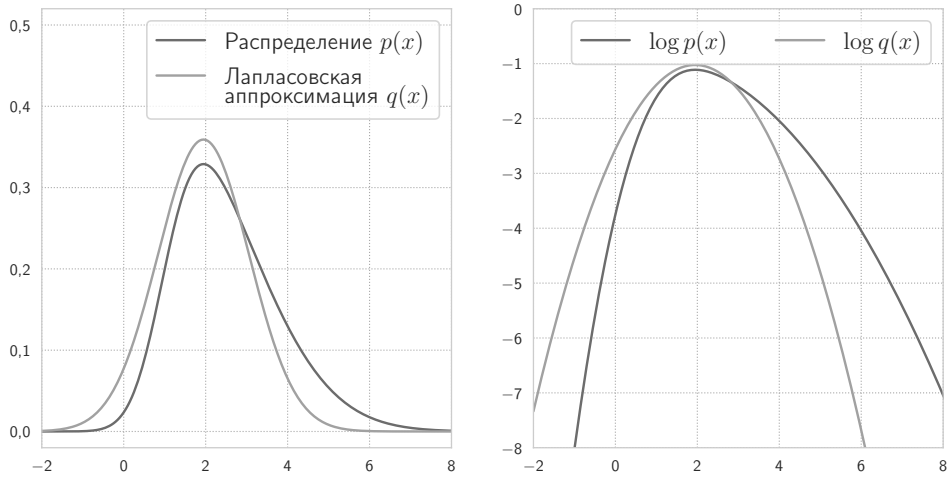


Рис. 5.10. Лапласовская аппроксимация к обобщённому нормальному распределению: (а) плотность распределения $p(x)$ и плотность его приближения $q(x)$; (б) то же самое на логарифмической шкале

Обобщённое нормальное распределение давно известно [40] и применяется во вполне естественных случаях: когда требуется колоколообразное распределение на $\mathbb{R}$ с одним максимумом, но нормальное не подходит, потому что данные очевидно несимметричны [32, 41, 412].

Такое распределение можно найти в пакете `scipy.stats`; на рис. 5.10, *а* показана плотность обобщённого нормального распределения с параметрами $\mu = 1$, $\sigma = 2$, $\lambda = 3$:

```
from scipy.stats import skewnorm
...
ax.plot(xs, skewnorm.pdf(xs, 3, loc=1, scale=2), label=r'Распределение $p(x)$')
```

Теперь нужно реализовать лапласовскую аппроксимацию. Давайте сделаем это численно, так, как мы могли бы справиться и с большинством реальных ситуаций. Сначала нужно найти точку максимума $x_{\max} = \arg\max_x p(x)$:

```
xmax = scipy.optimize.minimize( \
  lambda x: -skewnorm.pdf(x, 3, loc=1, scale=2), x0=0).x[0]
```

Для аппроксимации нужно также знать значение второй производной функции $\log p(x)$ в точке $x_{\max}$. Для численного поиска производных можно воспользоваться пакетом `numdifftools`, где есть и вторая производная (в виде гессиана):

```
def logdensity(x, a=3, loc=1, scale=2):
```

```
    return skewnorm.logpdf(x, a, loc=loc, scale=scale)

import numdifftools as nd
A = -nd.Hessian(logdensity)(xmax)[0,0]
```

И теперь всё готово для того, чтобы определить и нарисовать приближение $q(x) = \sqrt{\frac{A}{2\pi}} e^{-\frac{A}{2}(x-x_{\max})^2}$:

```
def qdensity_func(x):
    return np.sqrt(A/(2*np.pi))*np.exp(-A*((x - xmax) ** 2)/2.)
qdensity = np.vectorize(qdensity_func)
...
ax.plot(xs, qdensity(xs), label=r'Лапласовская аппроксимация$q(x)$')
```

Результат показан на рис. 5.10, *а*, а на рис. 5.10, *б* мы показываем те же кривые в логарифмической шкале, где видно, что лапласовская аппроксимация похожа на приближение в точке максимума двумя членами формулы Тейлора. Обратите внимание, что именно похожа, а не в точности совпадает, потому что для того, чтобы $q(x)$ было распределением вероятностей, нам пришлось это приближение перенормировать, и в точке $x_{\max}$ функции $p(x)$ и $q(x)$ не совпадают.

Сам Лаплас предложил этот метод для приближённого вычисления интегралов вида

$$\int_a^b e^{Mh(x)} \mathrm{d}x,$$

где M — большое число (чем больше показатель экспоненты, тем точнее будет аппроксимация), а у функции $h(x)$ есть единственный глобальный максимум в точке $x_0 \in (a,b)$. Тогда мы точно так же можем приблизить

$$h(x) \approx h(x_0) + h'(x_0)(x - x_0) + \frac{1}{2}h''(x_0)(x - x_0)^2 = h(x_0) + \frac{1}{2}h''(x_0)(x - x_0)^2,$$

где $h''(x_0) < 0$, потому что x_0 — точка максимума. В итоге мы получаем аппроксимацию

$$\int_a^b e^{Mh(x)} \mathrm{d}x \approx e^{Mh(x_0)} \int_a^b e^{-\frac{1}{2}M|h''(x_0)|(x-x_0)^2},$$

и теперь интеграл справа — это интеграл от гауссиана, который можно, в свою очередь, приблизить интегралом по всей оси, предполагая, что «хвосты» гауссиана станут экспоненциально маленькими за пределами отрезка $[a,b]$ и не повлияют существенно на результат:

$$\int_a^b e^{Mh(x)} \mathrm{d}x \approx e^{Mh(x_0)} \int_{-\infty}^{\infty} e^{-\frac{1}{2}M|h''(x_0)|(x-x_0)^2} = \sqrt{\frac{2\pi}{M|h''(x_0)|}} e^{Mh(x_0)}.$$

Эта аппроксимация становится всё более точной при $M \to \infty$.

При помощи лапласовской аппроксимации можно получить, например, формулу Стирлинга для асимптотики факториала, ведь факториал — это частный случай гамма-функции:

$$N! = \Gamma(N+1) = \int_0^\infty e^{-x} x^N \mathrm{d}x.$$

Сделаем замену переменных $x = Nz$:

$$N! = \int_0^\infty e^{-Nz}(Nz)^N \cdot N\mathrm{d}z = N^{N+1}\int_0^\infty e^{-Nz} z^N \mathrm{d}z = N^{N+1}\int_0^\infty e^{N(\ln z - z)}\mathrm{d}z.$$

И теперь мы получили ту самую форму интеграла $\int_a^b e^{Mh(x)}\mathrm{d}x$, к которой можно применить лапласовскую аппроксимацию, где $M = N$,

$$h(z) = \ln z - z, \quad h'(z) = \frac{1}{z} - 1, \quad h''(z) = -\frac{1}{z^2}.$$

Видно, что максимум здесь достигается в точке $z_0 = 1$, в ней $h''(1) = -1$, и мы получаем формулу Стирлинга

$$N! \approx N^{N+1}\sqrt{\frac{2\pi}{N}}e^{-N} = N^N e^{-N}\sqrt{2\pi N}.$$

*5.3.3. Предсказательное распределение в логистической регрессии

Теперь, вооружившись лапласовской аппроксимацией, давайте обработаем логистическую регрессию по-байесовски. Это, безусловно, раздел «со звёздочкой», для понимания оставшегося материала он не обязателен, и при первом чтении (да и втором) вывод вполне можно пропустить.

Возьмём для логистической регрессии нормальное априорное распределение:

$$p(\boldsymbol{w}) = \mathcal{N}(\boldsymbol{w} \mid \boldsymbol{\mu}_0, \Sigma_0).$$

Тогда апостериорное распределение будет выглядеть как

$$p\left(\boldsymbol{w}|\boldsymbol{t}\right) \propto p\left(\boldsymbol{w}\right) p\left(\boldsymbol{t}|\boldsymbol{w}\right),$$

то есть

$$\ln p\left(\boldsymbol{w}|\boldsymbol{t}\right) = -\frac{1}{2}\left(\boldsymbol{w} - \boldsymbol{\mu}_0\right)^\top \Sigma_0^{-1}\left(\boldsymbol{w} - \boldsymbol{\mu}_0\right) + \\ + \sum_{n=1}^{N}\left(t_n \ln y_n + (1 - t_n)\ln(1 - y_n)\right) + \mathrm{Const}, \quad (5.10)$$

где мы для краткости обозначили $y_n = \sigma(\boldsymbol{w}^\top \boldsymbol{x}_n)$.

Чтобы применить к апостериорному распределению (5.10) лапласовскую аппроксимацию, нужно сначала найти максимум $\boldsymbol{w}_{\text{MAP}}$ (у нас нет аналитической формулы для вычисления $\boldsymbol{w}_{\text{MAP}}$, но численно его найти несложно), а затем находим матрицу ковариаций, то есть матрицу вторых производных

$$\Sigma_N = -\nabla\nabla \ln p\left(\boldsymbol{w}|\boldsymbol{t}\right) = \Sigma_0^{-1} + \sum_{n=1}^{N} y_n(1-y_n)\boldsymbol{x}_n\boldsymbol{x}_n^\top.$$

Наше лапласовское приближение — это гауссиан

$$q\left(\boldsymbol{w}\right) = \mathcal{N}\left(\boldsymbol{w}|\boldsymbol{w}_{\text{MAP}}, \Sigma_N\right).$$

Теперь можно описать байесовское предсказание:

$$p\left(\mathcal{C}_1|\boldsymbol{x},\boldsymbol{t}\right) = \int p\left(\mathcal{C}_1|\boldsymbol{x},\boldsymbol{w}\right) p\left(\boldsymbol{w}|\boldsymbol{t}\right) \mathrm{d}\boldsymbol{w} \approx \int \sigma(\boldsymbol{w}^\top\boldsymbol{x}) q\left(\boldsymbol{w}\right) \mathrm{d}\boldsymbol{w}. \tag{5.11}$$

Заметим, что $\sigma(\boldsymbol{w}^\top\boldsymbol{x})$ зависит от $\boldsymbol{w}$ только через его проекцию на $\boldsymbol{x}$. Это позволит нам сделать второй трюк в доказательстве: свести интеграл к одномерному. Для этого обозначим $a = \boldsymbol{w}^\top\boldsymbol{x}$ и перепишем

$$\sigma(\boldsymbol{w}^\top\boldsymbol{x}) = \int \delta(a - \boldsymbol{w}^\top\boldsymbol{x})\sigma(a)\mathrm{d}a,$$

где δ — это дельта-функция, то есть такая обобщённая функция, что

$$\int \delta(x-a)f(x)\mathrm{d}\boldsymbol{x} = f(a).$$

Таким образом, формула (5.11) превращается в

$$p\left(\mathcal{C}_1|\boldsymbol{x},\boldsymbol{t}\right) \approx \int \sigma(\boldsymbol{w}^\top\boldsymbol{x}) q\left(\boldsymbol{w}\right) d\boldsymbol{w} = \int \sigma(a) p\left(a\right) \mathrm{d}a,$$

где

$$p\left(a\right) = \int \delta(a - \boldsymbol{w}^\top\boldsymbol{x}) q\left(\boldsymbol{w}\right) \mathrm{d}\boldsymbol{w}.$$

Одномерное распределение $p\left(a\right)$ — это маргинализация гауссиана $q\left(\boldsymbol{w}\right)$, где мы интегрируем по всему, что ортогонально $\boldsymbol{x}$. Значит, $p\left(a\right)$ — тоже гауссиан, и мы можем найти его моменты:

$$\mu_a = \mathbb{E}\left[a\right] = \int a p\left(a\right) \mathrm{d}a = \int q\left(\boldsymbol{w}\right) \boldsymbol{w}^\top\boldsymbol{x}\mathrm{d}\boldsymbol{w} = \boldsymbol{w}_{\text{MAP}}^\top\boldsymbol{x},$$

$$\sigma_a^2 = \int \Big(a^2 - \mathbb{E}\left[a\right]\Big)^2 p\left(a\right) \mathrm{d}a = \int q\left(\boldsymbol{w}\right) \Big((\boldsymbol{w}^\top\boldsymbol{x})^2 - (\boldsymbol{\mu}_N^\top\boldsymbol{x})^2\Big)^2 \mathrm{d}\boldsymbol{w} = \boldsymbol{x}^\top\Sigma_N\boldsymbol{x}.$$

Итого мы получили, что

$$p\left(C_1|\boldsymbol{t}\right) \approx \int \sigma(a) p\left(a\right) \mathrm{d}a = \int \sigma(a) \mathcal{N}\left(a\middle|\mu_a,\sigma_a^2\right) \mathrm{d}a.$$

Таким образом, при помощи лапласовской аппроксимации и трюков с дельта-функцией и маргинализацией мы приблизили предсказательное распределение логистической регрессии одномерным интегралом $\int \sigma(a) \mathcal{N}\left(a\middle|\mu_a,\sigma_a^2\right) \mathrm{d}a$.

Даже этот интеграл так просто не взять, потому что сигмоид — сложная функция. Но можно взять приближённо! Для этого давайте аппроксимируем $\sigma(a)$ через пробит-функцию так, чтобы приравнять и значение, и наклон в нуле; мы уже делали это в предыдущем разделе, и получалось, что

$$\sigma(a) \approx \Phi(\lambda a) \quad \text{для} \quad \lambda = \sqrt{\frac{\pi}{8}}.$$

А если мы перейдём к пробит-функции, то получится, что нам нужно подсчитать свёртку пробита с гауссианом $\int \Phi(\lambda a) \mathcal{N}(a \mid \mu_a,\sigma_a^2) \mathrm{d}a$. Здесь нам потребуется ещё одно, последнее усилие, а заодно и полезное упражнение из теории вероятностей. Заметим, что

$$\begin{aligned}\Phi(\lambda a) &= \int_{-\infty}^{\lambda a} \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}x^2} \mathrm{d}x = [\text{сделаем замену } x = \lambda x] = \\ &= \int_{-\infty}^{a} \frac{1}{\lambda\sqrt{2\pi}} e^{-\frac{1}{2\lambda^2}x^2} \mathrm{d}x = p\left(X \le a\right), \quad \text{где } X \sim \mathcal{N}\left(0, \frac{1}{\lambda^2}\right). \end{aligned} \tag{5.12}$$

И тогда получается, что

$$\int \Phi(\lambda a) \mathcal{N}(a \mid \mu_a,\sigma_a^2) \mathrm{d}a = \int p\left(X \le a\right) \mathcal{N}(a \mid \mu_a,\sigma_a^2) \mathrm{d}a = p\left(X \le Y\right),$$

где $X \sim \mathcal{N}\left(0, \frac{1}{\lambda^2}\right)$ и $Y \sim \mathcal{N}\left(\mu_a, \sigma_a^2\right)$. Но теперь можно заметить, что

$$p\left(X \le Y\right) = p\left(X - Y \le 0\right) = p\left(X - Y + \mu_a \le \mu_a\right),$$

и здесь

$$X - Y + \mu_a \sim \mathcal{N}\left(0, \sigma_a^2 + \frac{1}{\lambda^2}\right).$$

В результате всех вычислений опять получилась вероятность того же вида, что в формуле (5.12): $p\left(X \le a\right)$ для нормально распределённой величины с нулевым средним. Так что можно воспользоваться формулой (5.12) и получить, что исходная свёртка будет просто другим пробитом:

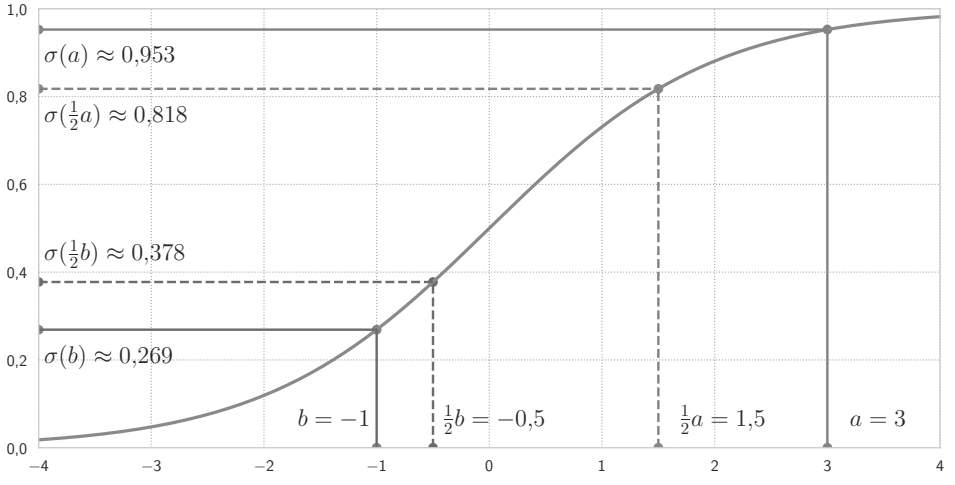


Рис. 5.11. Как уменьшается уверенность в ответе при переходе к байесовскому предсказательному распределению

$$\int \Phi(\lambda a)\mathcal{N}(a \mid \mu_a,\sigma_a^2)\mathrm{d}a = \Phi\left(\frac{\mu_a}{\sqrt{\frac{1}{\lambda^2}+\sigma_a^2}}\right).$$

В итоге получается аппроксимация

$$\int \sigma(a)\mathcal{N}(a \mid \mu,\sigma^2)\mathrm{d}a \approx \sigma\left(\kappa(\sigma^2)\mu\right), \quad \text{где} \quad \kappa(\sigma^2) = \frac{1}{\sqrt{1+\frac{\pi}{8}\sigma^2}}.$$

Собирая всё вместе, мы получаем предсказательное распределение в следующем виде:

$$p\left(\mathcal{C}_1|\boldsymbol{x},\boldsymbol{t}\right) = \sigma\left(\kappa(\sigma_a^2)\mu_a\right),$$

где

$$\mu_a = \boldsymbol{w}_{\mathrm{MAP}}^\top \boldsymbol{x}, \qquad \sigma_a^2 = \boldsymbol{x}^\top \Sigma_N \boldsymbol{x}, \qquad \kappa(\sigma^2) = \frac{1}{\sqrt{1+\frac{\pi}{8}\sigma^2}}.$$

Разумеется, разделяющая поверхность от этого, как и предсказание в линейной регрессии (вспомните раздел 4.4) не изменится: $p\left(C_1|\boldsymbol{x},\boldsymbol{t}\right) = \frac{1}{2}$ задаётся уравнением $\mu_a = 0$, и бинарный ответ будет тем же, как если бы мы просто использовали $\boldsymbol{w}_{\mathrm{MAP}}$ и не задумывались об этих сложных байесовских материях.

Разница здесь (опять же как и в разделе 4.4) заключается не в самом ответе, а в нашей *степени уверенности* в этом ответе. Обратите внимание, что мы умножаем аргумент сигмоида на величину $\kappa(\sigma_a^2)$, которая всегда меньше единицы. Это

значит, что мы сохраняем знак аргумента сигмоида, но по модулю приближаем его к нулю, то есть приближаем получающуюся из этого сигмоида вероятность к $\frac{1}{2}$, и наши предсказания становятся менее уверенными. Этот эффект проиллюстрирован на рис. 5.11, где предполагается, что $\kappa(\sigma_a^2) = \frac{1}{2}$, и показаны два примера, с положительным аргументом $a = 3$ и отрицательным аргументом $b = -1$. В обоих случаях, как видите, уменьшение аргумента по модулю приводит к тому, что значение сигмоида приближается к $\frac{1}{2}$.

Мы видим, что байесовский подход к предсказаниям в логистической регрессии не меняет точечный ответ, но делает нас менее уверенными в нём. Интегрирование по апостериорному распределению позволяет «рассмотреть все варианты» и лучше понять, насколько мы действительно что-то знаем, можем заключить из данных. Тот же эффект мы видели и в разделе 2.4 на примере обучения параметра монетки, и в подразделе 4.4.3, где мы рассматривали предсказательное распределение линейной регрессии. Такое уточнение уверенности в ответе — это важный «бонус» от использования байесовских методов.

5.4. Ирисы Фишера

Она целовала каждый венчик холодных чёрных цветов и улыбалась дрожащими губами:
— Так не уходят! Нет! Это не прощальные цветы. Они чёрные оттого, что он тоскует, а тоскует оттого, что любит! Осталось ещё два дня свободы, — нельзя терять ни минуты.
— Евгений! — говорила она в телефон. — Евгений! Прости меня! Это неправда, я не люблю другого. Я наклеветала на себя, чтобы отомстить тебе. Отчего я одумалась? От чёрных цветов. Чёрный ирис сказал мне, что ты тоскуешь и любишь. Я люблю чёрный ирис! Понимаешь? Чёрный ирис! Только чёрный ирис! Нет, нет, я не сумасшедшая, я только счастливая.

Тэффи. Чёрный ирис

5.4.1. Набор данных

В этом разделе я приведу развёрнутый пример задачи классификации на совсем небольшом, но знаменитом датасете параметров цветков ириса. Этот набор данных имеет долгую и славную историю. Большинство источников называет его «ирисами Фишера» (Fisher Iris Dataset), потому что именно Рональд Фишер в своей статье 1936 года *The use of multiple measurements in taxonomic problems* [185] впервые опубликовал и обработал этот датасет, и именно с лёгкой руки Фишера этот датасет вошёл в практику статистики и стал стандартной иллюстрацией методов классификации на много десятилетий вперёд.

Но вернее было бы назвать его «ирисами Андерсона», потому что собрал эти данные известный биолог Эдгар Андерсон [24][1]. Андерсон хотел найти в ирисах подтверждение эволюционных теорий; в работе [24] он пишет: «В простом предположении, что если текущие теории верны, то свидетельства продолжающейся эволюции должны найтись в любой группе растений, я путешествовал по миру, обращая пристальное внимание на вариативность в ирисах. В любой теории эволюции индивидуальные различия постепенно, со временем накапливаются и превращаются в различия между видами... если эта теория верна, то мы сможем найти начало этих накапливающихся различий в видах, существующих в наши дни. Поэтому я как можно детальнее изучил все доступные мне ирисы,

[1] Кстати, статья Андерсона вышла в журнале под названием *Bulletin of the American Iris Society*; это общество существует с 1920 года и до сих пор выпускает журнал, а также присуждает два десятка ежегодных наград.

Таблица 5.1. Несколько строк данных из датасета об ирисах

	SepalLength	SepalWidth	PetalLength	PetalWidth	Species
0	5,1	3,5	1,4	0,2	Iris setosa
1	4,9	3,0	1,4	0,2	Iris setosa
2	4,7	3,2	1,3	0,2	Iris setosa
3	4,6	3,1	1,5	0,2	Iris setosa
4	5,0	3,6	1,4	0,2	Iris setosa

(а) (б) (в)

Рис. 5.12. Ирисы: (а) *Iris setosa*; (б) *Iris versicolor*; (в) *Iris virginica*

измеряя цветок за цветком, сидя на корточках с книгой для записей и линейкой в горных долинах и кипарисовых болотах, на озёрных пляжах и в английских парках... Но я не нашёл другой такой возможности, как на поле между Ile Verte и Trois Pistoles. Там можно миля за милей собирать сколько угодно ирисов и собрать сотню целых цветков *Iris versicolor* и *Iris setosa canadensis*, каждый с нового растения, но все с одного и того же пастбища, собранные в один и тот же день и измеренные одним и тем же исследователем с одними и теми же инструментами». Андерсон сделал некоторые выводы об эволюции ирисов из своих данных, но по-настоящему популяризировал этот датасет всё-таки Рональд Фишер.

Давайте воспользуемся этим случаем, чтобы поговорить не только о классификации, но и немного о визуализации данных. Начнём с того, что загрузим датасет[1] и посмотрим, что там содержится.

```
import pandas as pd
iris = pd.read_csv("IRIS.csv")
iris.head()
```

Датасет по современным меркам совсем маленький, всего 150 строк, а после команды `head` должно получиться то, что мы видим в табл. 5.1. Первый столбец —

[1] Он есть практически везде — и в UCI Machine Learning Repository [155], и на *Kaggle*, и во многих библиотеках...

это просто номера строк. Последний столбец — правильный ответ, категориальная переменная, в которой записан вид ирисов; именно их мы и будем классифицировать. У него три возможных значения: *Iris setosa*, *Iris versicolor* и *Iris virginica*, по 50 экземпляров каждого вида. Примеры самих цветов ирисов показаны на рис. 5.12; как видите, они действительно различаются, хотя и похожи.

Четыре столбца в таблице между первым и последним — это те самые признаки цветочков, которые Эдгар Андерсон измерял, сидя на корточках с линейкой:

- длина чашелистика (SepalLength), то есть зелёных листов снизу чашечки цветка;
- ширина чашелистика (SepalWidth);
- длина лепестка (PetalLength);
- ширина лепестка (PetalWidth).

5.4.2. Предварительный анализ данных

Давайте начнём с того, что изобразим статистику различных признаков в трёх имеющихся классах; такого рода исследования иногда называют предварительным, или разведочным анализом (exploratory analysis).

В нашем случае статистика показана на рис. 5.13. Каждый из этих графиков получен при помощи следующего кода:

```
sns.boxplot(x="Species", y="SepalWidth", data=iris, ax=ax)
sns.stripplot(x="Species", y="SepalWidth", data=iris, ax=ax, jitter=True)
```

Здесь `sns` обозначает стандартный пакет `seaborn` (его обычно импортируют как `sns`), `ax` — это ось графика, на котором мы рисуем. График вида `stripplot` изображает просто отдельные точки данных по указанным осям; параметр `jitter` добавляет небольшой случайный шум по горизонтали, чтобы точки лучше визуально отделялись друг от друга.

А `boxplot` — это более интересный, хотя и ещё более стандартный вид статистического графика. Полностью его называют *box-and-whiskers plot*, и на русский это иногда переводят буквально как «ящик с усами», хотя используется и более наукообразный термин «диаграмма размаха». Этот вид графика показывает важнейшие одномерные статистики:

- медиану или среднее (это линия в середине «ящика»);
- первый и третий квартили, то есть значения на 25 %-ном и 75 %-ном уровнях в датасете (это границы «ящика»);
- полный размах данных в виде «усов», которые простираются от минимума до максимума.

Иногда «ящики с усами» ограничивают свои «усы» более консервативными значениями, чем настоящие максимумы и минимумы (часто величиной $1{,}5 \cdot \mathrm{IQR}$, где IQR, interquartile range, — это расстояние между третьим и первым квартилем), и отмечают выходящие за эти пределы значения отдельно, как выбросы.

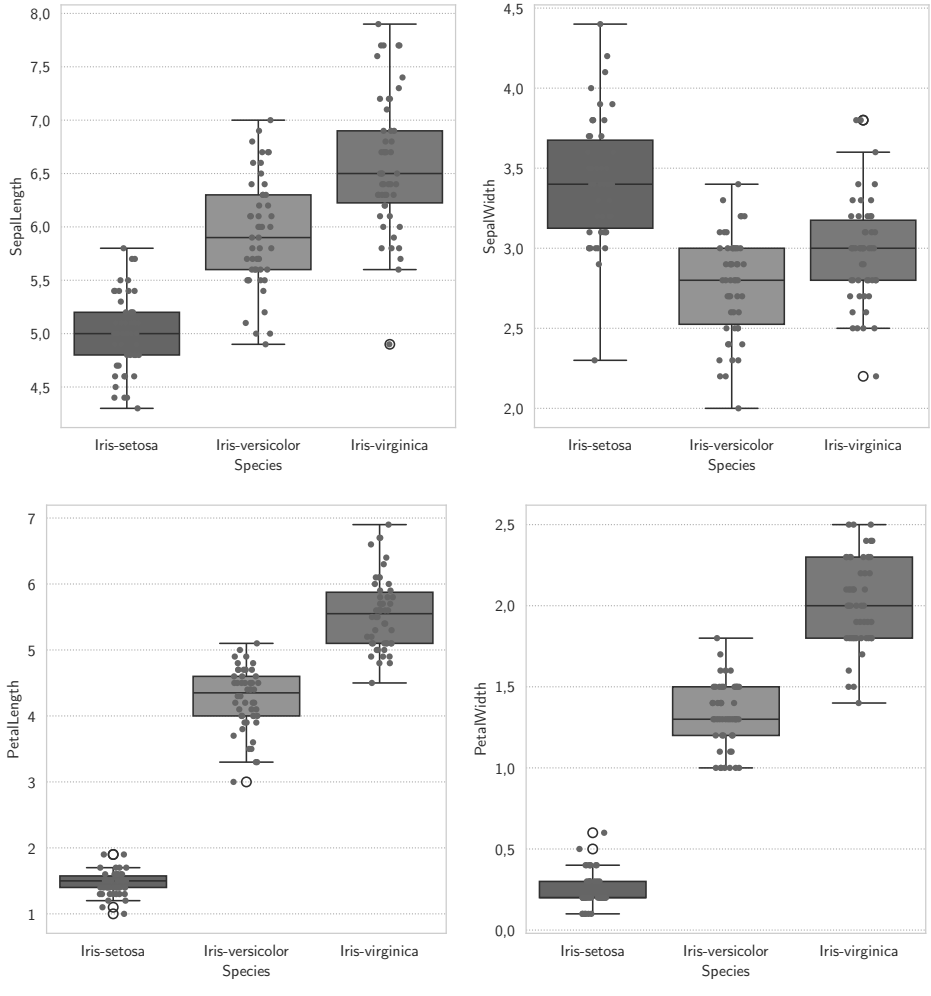


Рис. 5.13. Статистика признаков в различных классах

Этот тип графиков был впервые введён Мэри Элеанор Спир[1] в 1952 году, а затем улучшен и популяризирован Джоном Тьюки[2] в 1970-х годах.

[1] *Мэри Элеанор Спир* (Mary Eleanor Hunt Spear, 1897–1986) — американский специалист по анализу и визуализации данных, пионер многих методов визуализации. В первой половине XX века она работала на различные государственные агентства США, а затем основала свою собственную студию по визуализации данных. Две книги Мэри Спир, *Charting Statistics* [551] и *Practical Charting Techniques* [552], стали классическими источниками по визуализации данных.

[2] *Джон Тьюки* (John Wilder Tukey, 1915–2000) — американский математик и статистик. В 1965 году Тьюки вместе с Джеймсом Кули изобрёл алгоритм Кули — Тьюки для быстрого преобразования Фурье [117, 118]; впоследствии, впрочем, выяснилось, что на самом деле его придумал Гаусс

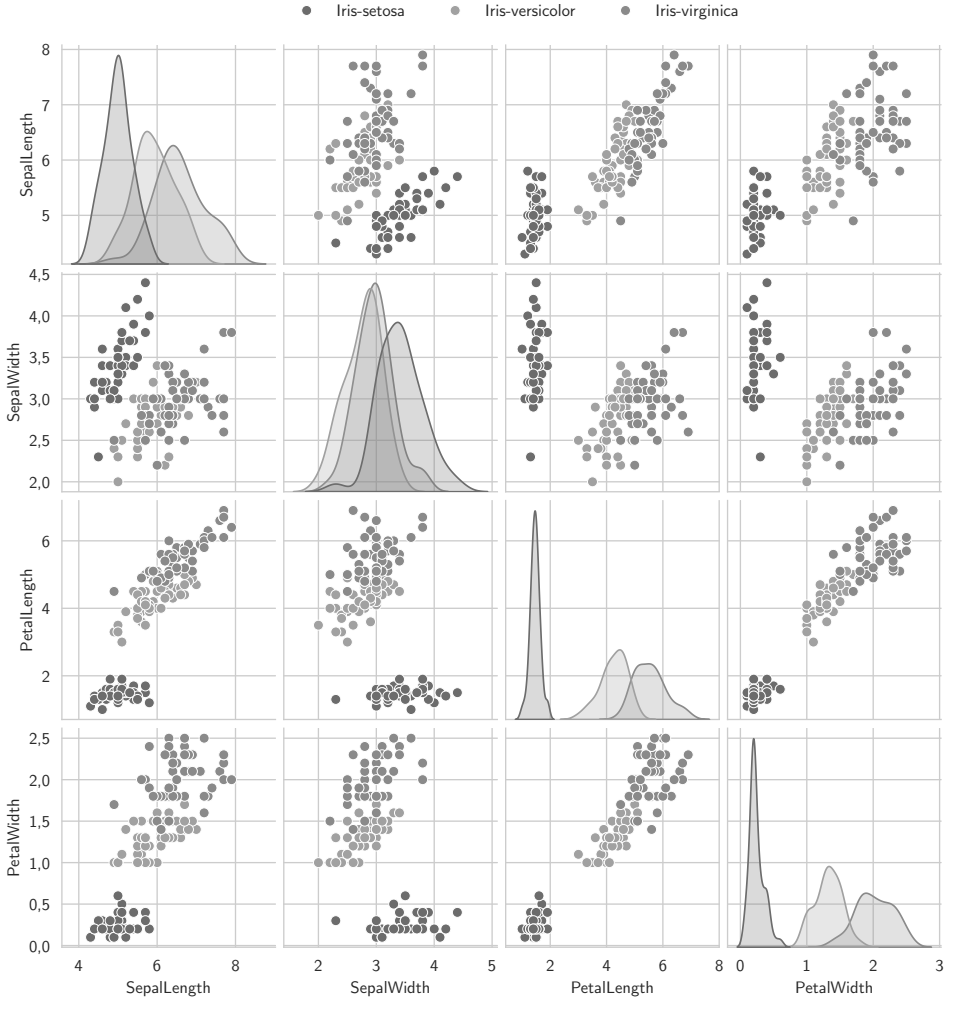


Рис. 5.14. Попарные визуализации переменных в датасете ирисов

Но вернёмся к нашему цветочному датасету. Рисунок 5.13 уже достаточно убедительно показывает, чего мы можем ожидать: мы видим, что форма лепестков (признаки PetalWidth и PetalLength) весьма уверенно отличает *Iris setosa* от

ещё в начале XIX века, но работа Карла Фридриха осталась неизвестной, да и названо в честь Гаусса было уже много чего и так. Тьюки принадлежит ряд важных результатов в математической статистике, но самый известный его труд — книга *Exploratory Data Analysis* [604], где он и популяризировал «ящик с усами», и вообще дал массу советов, полезных и по сей день.

двух других видов ирисов. Чтобы ещё более чётко это увидеть, давайте нарисуем ещё один популярный вид статистической визуализации данных — набор графиков, которые в виде квадратной решётки показывают взаимоотношения между всеми парами признаков:

```
sns.pairplot(iris, hue="Species")
```

Результат этой команды показан на рис. 5.14; графики на диагонали показывают эмпирические плотности четырёх признаков (эти графики фактически показывают то же самое, что рис. 5.13), а графики вне диагонали показывают «срезы» данных по всем парам переменных; разумеется, здесь нет нужды рисовать квадрат полностью, хватило бы и треугольника. На этой картинке мы тоже видим, что *Iris setosa* можно практически идеально отделить от двух других классов и по длине, и по ширине лепестков. Поэтому в бинарной классификации, чтобы задача была не совсем уж тривиальной, мы будем отличать друг от друга *Iris versicolor* и *Iris virginica*.

Но это всё пока была исключительно визуализация, тот самый разведочный анализ данных; пора переходить к классификации. Для этого превратим данные в массивы `numpy` и закодируем названия классов числами (для этого есть стандартный класс `LabelEncoder`):

```
from sklearn.preprocessing import StandardScaler, LabelEncoder
ds = iris.to_numpy()
X, y = ds[:, :-1], ds[:, -1]
le = LabelEncoder().fit(y)
y = le.transform(y)
```

Ещё одно стандартное преобразование, которое точно не повредит, а некоторым классификаторам может помочь, — это нормализация численных признаков, то есть приведение их среднего к нулю, а дисперсии — к единице линейным преобразованием. Конечно, если классификатор всё равно обучает произвольную линейную функцию, как логистическая регрессия, от нормализации ничего не изменится, но для других методов результат может существенно улучшиться. Для этого тоже есть уже готовый класс `StandardScaler`:

```
X_sc = StandardScaler().fit_transform(X)
```

5.4.3. Сравнение классификаторов

Теперь можно обучать различные классификаторы и смотреть, хорошо ли они работают. Давайте для исходной задачи попробуем сразу логистическую регрессию:

```
from sklearn.linear_model import LogisticRegression
logregr = LogisticRegression()
logregr.fit(X_sc, y)
```

Проанализируем получившийся результат. Оценим качество работы модели, подсчитав точность классификации, и выпишем коэффициенты получившихся линейных функций:

```
def get_accuracy(model, X, y):
    return np.sum(model.predict(X) == y) / y.size

print("Точность: %.4f" % (get_accuracy(logregr, X, y))
print("Коэффициенты: %s" % logregr.coef_)
print("Свободные члены: %s" % logregr.intercept_)
```

```
Точность: 0.9733
Коэффициенты: [[-1.0749604  1.1558120 -1.9309383 -1.8203946]
 [ 0.5883315 -0.3604190 -0.365196  -0.8203440]
 [ 0.4866289 -0.7953930  2.2961348  2.6407387]]
Свободные члены: [-0.2116720  2.0771413 -1.8654692]
```

Это значит, что обученная нами логистическая регрессия в качестве a_k берёт для трёх классов следующие значения:

$$a_1 = -0{,}2116720 - 1{,}0749604x_1 + 1{,}1558120x_2 - 1{,}9309383x_3 - 1{,}8203946x_4,$$
$$a_2 = 2{,}0771413 + 0{,}5883315x_1 - 0{,}3604190x_2 - 0{,}365196x_3 - 0{,}8203440x_4,$$
$$a_3 = -1{,}8654692 + 0{,}4866289x_1 - 0{,}7953930x_2 + 2{,}2961348x_3 + 2{,}6407387x_4$$

(точность в семь знаков здесь, конечно, смысла не имеет), а затем, как мы обсуждали в разделе 5.2, использует в качестве вероятностей классов значения, пропорциональные e^{a_k}.

Точность в 97 с лишним процентов для классификации на три класса выглядит неплохо, но мы получили это значение метрики категорически нечестным образом: мы обучали и тестировали модель на одном и том же наборе точек! Давайте вместо этого обучим десяток моделей на случайных подмножествах датасета и усредним результаты. Для этого пригодится функция `train_test_split` из пакета `sklearn.model_selection`:

```
from sklearn.model_selection import train_test_split
res = []
for _ in range(10):
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=.2)
    train_sc = StandardScaler().fit(X_train)
    X_train_sc = train_sc.transform(X_train)
    X_test_sc = train_sc.transform(X_test)
    logregr.fit(X_train_sc, y_train)
    res.append(get_accuracy(logregr, X_test_sc, y_test))
print("Точности: %s" % "".join(["%.2f" % acc for acc in res]))
print("Средняя точность: %.4f" % np.mean(res))
```

В результате выполнения этого кода должно получиться примерно следующее (числа могут немного отличаться, ведь мы выбираем случайные подмножества):

```
Точности: 1.000 0.933 0.967 0.967 0.967 1.000 0.967 0.967 0.967 0.900
Средняя точность: 0.9633
```

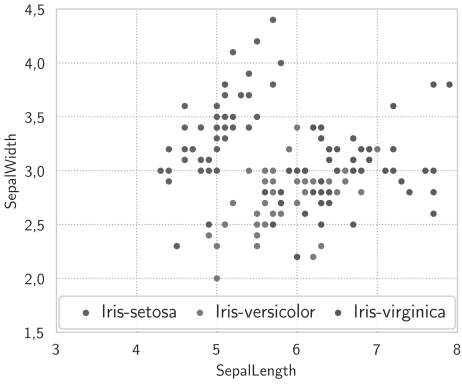
Так что и при честном тестировании датасет с ирисами оказывается несложным и поддаётся логистической регрессии практически идеально.

Чтобы сравнить разные классификаторы визуально, придётся ограничиться двумя признаками, чтобы мы смогли изобразить результат на плоскости. Возьмём первые два признака, длину и ширину чашелистника, потому что по ним распределение выглядит более интересным (см. рис. 5.14), и обучим все те модели, которые мы в этой главе рассматривали. Результат показан на рис. 5.15; вот что можно любопытного сказать об этом результате:

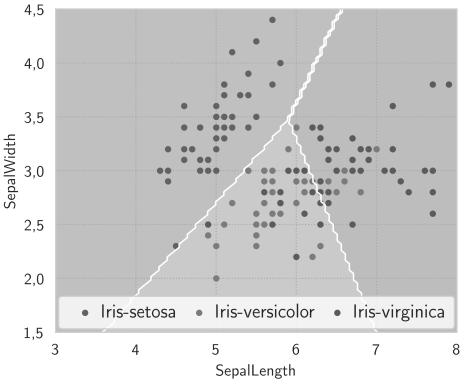
- с отделением класса *Iris setosa* хорошо справляются практически все модели;
- поскольку классы *Iris versicolor* и *Iris virginica* по этим двум переменным разделяются плохо, мы видим те или иные проблемы у всех классификаторов;
- логистическая регрессия и LDA дают, наверное, самые разумные разделяющие поверхности;
- у меня не получилось выбрать число ближайших соседей k так, чтобы из метода ближайших соседей получились действительно хорошие гладкие разделяющие поверхности; показанные на рис. 5.15, *д*, *е* $k = 3$ и $k = 9$, наверное, выглядят лучше других значений, но граница у них всё равно весьма «рваная»;
- а вот с QDA получился интересный эффект: квадратичная функция, соответствующая классу *Iris virginica*, разумно отделяется от других классов там, где есть точки данных, но снова начинает выигрывать у *Iris setosa* в области малых значений *SepalLength*, где точек данных уже нет.

Последний эффект выглядит очень странно, но попробуйте представить себе возможные пересечения квадратичных поверхностей, и вы тут же убедитесь, что такое поведение действительно весьма ожидаемо для QDA. К сожалению, это ещё один случай, подтверждающий важное общее наблюдение: многие модели машинного обучения хорошо годятся для *интерполяции*, то есть для ситуаций, когда мы хотим предсказывать значения возле существующих точек данных, но не очень хороши для *экстраполяции* и могут иметь странное поведение «на бесконечности», то есть, практически говоря, на значениях, существенно отдаляющихся от имеющихся в датасете точек.

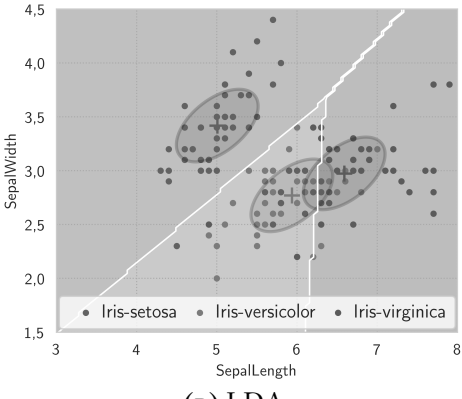
На этом рассмотрение ирисов в этой главе заканчивается. В следующем разделе мы рассмотрим несколько важных тем, которые относятся к любой задаче классификации.



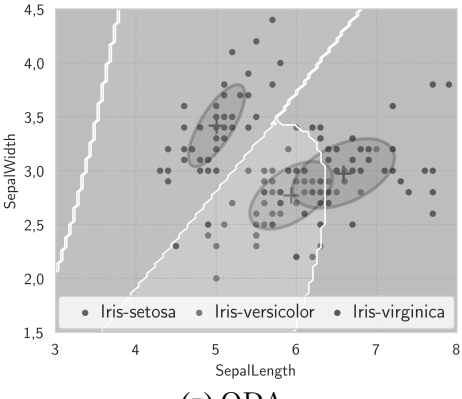
(а) Исходные данные



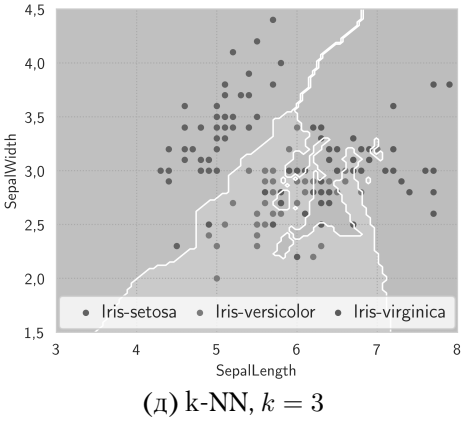
(б) Логистическая регрессия



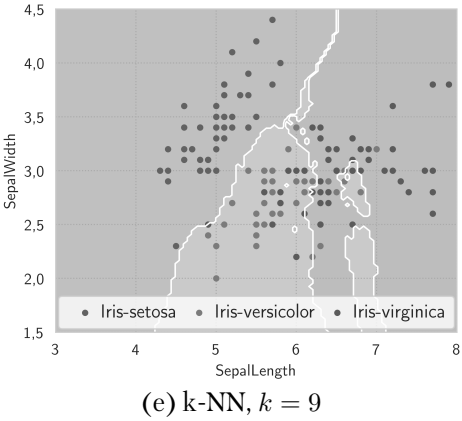
(в) LDA



(г) QDA



(д) k-NN, $k = 3$



(е) k-NN, $k = 9$

Рис. 5.15. Разделяющие поверхности для ирисов: (а) данные; (б) логистическая регрессия; (в) LDA; (г) QDA; (д) метод ближайших соседей с $k = 3$; (е) с $k = 9$

5.5. Общие замечания о классификаторах

Его изначальный метод был относительно прост. Изучить испанский язык, вновь проникнуться католической верой, воевать с маврами или с турками, позабыть европейскую историю с 1602 по 1918 год, стать Мигелем Сервантесом. Пьер Менар взялся за эту затею (я знаю, что он почти свободно владел испанским семнадцатого века), но потом от неё отказался как от слишком простой.

Хорхе Луис Борхес. Пьер Менар, автор «Дон Кихота» (пер. М. Былинкиной)

5.5.1. Несбалансированные классы и перевзвешенная ошибка

Вернёмся к началу главы, а именно к теореме Байеса (5.3). Мы обсуждали её в применении к классификации ещё в главе 2, где в подразделах 2.2.2 и 2.2.3 рассмотрели множество видов ошибок, возникающих в обычном бинарном классификаторе, и немного поговорили о том, как настроить баланс между разными видами этих ошибок. Теперь мы можем лучше понять, что тогда происходило в примере с медицинским тестом, и обобщить это на новые ситуации.

Начнём с того, что в примерах, которые мы до сих пор рассматривали, в классах обучающей выборки точек было примерно поровну. Что произойдёт, если это окажется не так и классы будут сильно несбалансированы; например, что если в C_1 в 20 раз больше точек, чем в C_2? Согласитесь, что в ситуациях, когда так происходит, именно более редкий класс обычно является нашей целью. Например, мы можем классифицировать:

- поведение людей по данным носимых устройств с медицинскими целями, пытаясь заметить момент травмы или несчастного случая [10];
- медицинские снимки, в которых нужно выявить признаки раковой опухоли или другой болезни [172], или анализ данных экспрессии генов с теми же целями [250];
- логи поведения пользователей в онлайн-системе, пытаясь выявить недобросовестных пользователей, ботов или мошенников [52, 161];
- данные наблюдения за промышленной системой, в которых может быть нужно выявить, например, дефектные детали, получающиеся на конвейере [202];
- аномальные значения в большой базе данных, например искать места разлива нефти в океане на основе спутниковых снимков [322];
- и так далее и тому подобное; очевидно, что во всех таких случаях поведение «по умолчанию» будет составлять подавляющее большинство получаемых

данных, но нас интересуют как раз те редкие случаи, когда происходит что-то незапланированное или мы таки находим иголку в стоге сена.

Такие задачи возникают постоянно, и существует много методов, позволяющих справиться с несбалансированными классами [248, 249, 320, 569]. Давайте кратко пройдёмся по самым популярным из них; далее мы будем для простоты предполагать, что решаем задачу бинарной классификации, и обучающая выборка состоит из точек, принадлежащих классу $C_{\min}$ (от слова *minority*), в котором мало точек, и классу C_{maj} (от слова *majority*), в котором их во много раз больше.

Прежде всего замечу, что методы, основанные на порождающих моделях, которые мы обсудили в подразделе 5.1.4 (там мы обсуждали только LDA и QDA, но можно было бы обучать и более сложные распределения вероятностей для каждого класса, просто мы пока не умеем этого делать), вовсе не страдают от несбалансированных классов! Конечно, обучить порождающее распределение на маленьком классе $C_{\min}$ получится хуже, оценки параметров будут менее точными, но с этим в любом случае придётся смириться — если данных больше нет, то ничего не поделаешь.

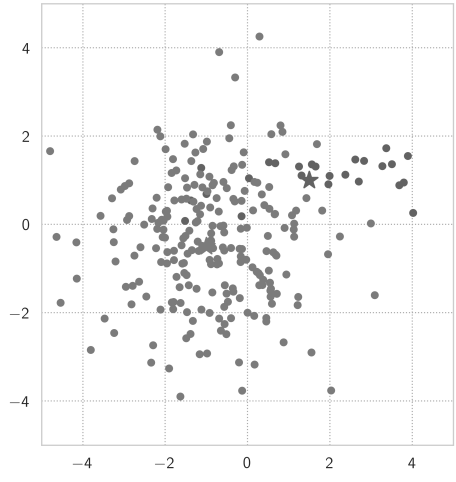
А в остальном порождающие модели обучают распределение каждого класса отдельно и независимо, а собственно баланс между классами, относительное число точек в каждом из них, обучается в априорном распределении $p(C_k)$ — мы уже обсуждали, что для классификатора на базе порождающих моделей это распределение представляет собой отдельный набор параметров, которые обучаются по числу точек в каждом классе.

На рис. 5.16, *а* показан пример датасета, порождённого из тех же распределений, что на рис. 5.5 и 5.6, но с сильным дисбалансом классов, 20:1 в пользу C_{maj}. А на рис. 5.16, *б* показано, как с этим дисбалансом справляется QDA, — как видите, обучить весьма точные параметры распределений для каждого класса этому методу ничто не помешало, да и априорные распределения обучились верно: в переменной `qda.priors_` лежит что-то вроде

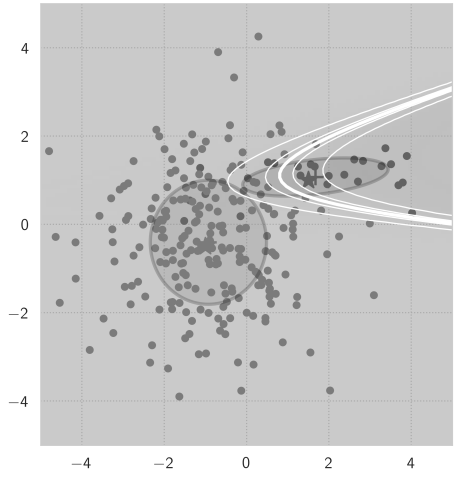
```
array([0.954, 0.046])
```

А вот для чисто геометрических подходов к классификации обучение классов не будет независимым и несбалансированные классы — это неприятное обстоятельство: даже если, например, мы по-прежнему разделяем два гауссиана, но в одном из них гораздо больше точек, чем в другом, эти точки должны «занимать большую площадь».

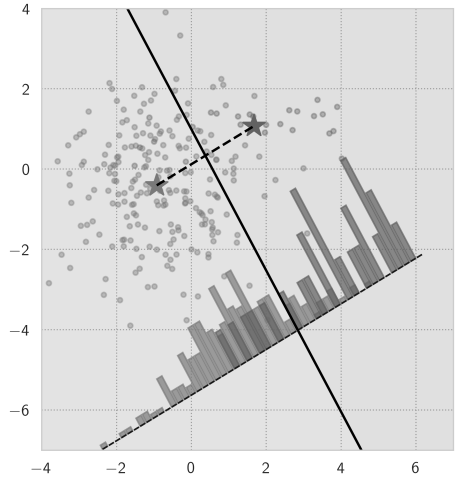
На рис. 5.16, *в* видно, что дискриминант Фишера фактически никак не реагирует на несбалансированность классов, по-прежнему строя разделяющую поверхность где-то посередине между центрами классов. Метод ближайших соседей (см. рис. 5.16, *г*) здесь вполне справляется, если хорошо выбрать число соседей k, но ближайшие соседи всегда хорошо работают в низких размерностях, а общего решения задачи не дают — об этом мы ещё поговорим в разделе 6.1. Что же можно сделать с несбалансированными классами?



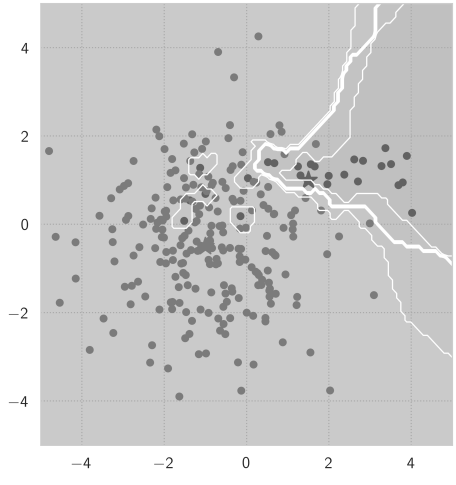
(а) Данные



(б) QDA



(в) Дискриминант Фишера



(г) kNN, $k = 3$

Рис. 5.16. Результаты различных классификаторов для несбалансированных классов

Во-первых, если данных в классе C_{maj} слишком много, можно просто не брать их все для построения классификатора, а обучаться только на части C_{maj}; по-английски это называется *undersampling* («недосемплирование»). Главный недостаток этого метода очевиден: мы не используем все имеющиеся данные и можем что-то пропустить; поэтому есть методы (в которые мы не будем углубляться), где обучаются отдельные модели для того, чтобы решить, какие точки убирать из данных безопасно, а какие лучше оставить [323, 350].

С другой стороны, можно не убирать данные из C_{maj}, а, наоборот, добавлять их в $C_{\min}$, переиспользуя одни и те же точки данных несколько раз; это, соответственно, называется *oversampling* («пересемплирование»). В результате такого подхода получится сбалансированный датасет, в котором в $C_{\min}$ точек формально столько же, сколько в C_{maj}, но значения этих точек повторяются столько раз, сколько нужно (как правило, не поровну, а случайной выборкой с замещением).

Здесь главной проблемой станет, наоборот, оверфиттинг: если использовать одни и те же точки данных много раз, то они могут приобрести слишком большой вес и классификатору придётся обращать на них слишком много внимания. Поэтому вместо прямого переиспользования часто применяют методы, где новые точки для $C_{\min}$ тем или иным способом порождаются, а не просто копируются.

Здесь мы уже заходим в область *синтетических данных* для машинного обучения, о которой можно сказать очень много [437]. Поэтому здесь я просто приведу один из самых простых и популярных алгоритмов такого порождения — SMOTE (synthetic minority oversampling technique) [546]:

- для каждого примера $\boldsymbol{x} \in C_{\min}$ рассмотрим множество её k ближайших соседей (скорее всего, многие из них будут из C_{maj});
- чтобы создать новую синтетическую точку $\boldsymbol{x}'$, выберем из этого множества случайного соседа $\boldsymbol{x}_{\text{nei}}$ точки $\boldsymbol{x}$, выберем случайный вес α и построим выпуклую комбинацию с таким весом:

$$\boldsymbol{x}' = \alpha \boldsymbol{x}_{\text{nei}} + (1 - \alpha)\boldsymbol{x}.$$

При таком подходе мы как будто «расширяем» точку $\boldsymbol{x}$, добавляя точки, которые находятся «на полпути» до её ближайших соседей, и тем самым превращая каждую точку $C_{\min}$ в целый кластер точек, занимающий ненулевой объём в пространстве. Есть и адаптивные варианты SMOTE, которые более сложным образом решают, какие точки выбирать в качестве $\boldsymbol{x}$ и как выбирать α [11, 237, 632].

Но нас сейчас больше интересуют методы, имеющие вероятностную природу. В целом все они сводятся к тому, чтобы изменить априорные вероятности классов $p(C_k)$ в теореме Байеса (5.3), но в случае дискриминирующих моделей, в частности логистической регрессии, здесь есть о чём поговорить.

Вспомним, что бинарная логистическая регрессия моделирует логарифм отношения апостериорных вероятностей классов (логарифм шансов) при помощи линейной функции; нам сейчас будет удобнее записать её со свободным членом:

$$\log \frac{p(\boldsymbol{x}|C_1)\,p(C_1)}{p(\boldsymbol{x}|C_2)\,p(C_2)} \sim \boldsymbol{w}^\top \boldsymbol{x} + w_0, \qquad p(C_1|\boldsymbol{x}) \sim \frac{1}{1 + e^{-(\boldsymbol{w}^\top \boldsymbol{x} + w_0)}}. \tag{5.13}$$

Что произойдёт, если изменить априорные вероятности классов $p(C_1)$ и $p(C_2)$, точнее, их соотношение $\frac{p(C_1)}{p(C_2)}$?

Из формулы (5.13) это непосредственно очевидно: если отношение $\frac{p(C_1)}{p(C_2)}$ изменится на какую-то мультипликативную константу, скажем увеличится в α раз, то в линейной функции $\boldsymbol{w}^\top \boldsymbol{x} + w_0$ свободный член сместится на логарифм этой величины, а сами веса не изменятся:

$$\log \frac{p\left(\boldsymbol{x}|C_1\right) p\left(C_1\right)}{p\left(\boldsymbol{x}|C_2\right) p\left(C_2\right)} = \log \frac{p\left(\boldsymbol{x}|C_1\right)}{p\left(\boldsymbol{x}|C_2\right)} + \log \frac{p\left(C_1\right)}{p\left(C_2\right)} \sim \boldsymbol{w}^\top \boldsymbol{x} + w_0 + \log \alpha. \qquad (5.14)$$

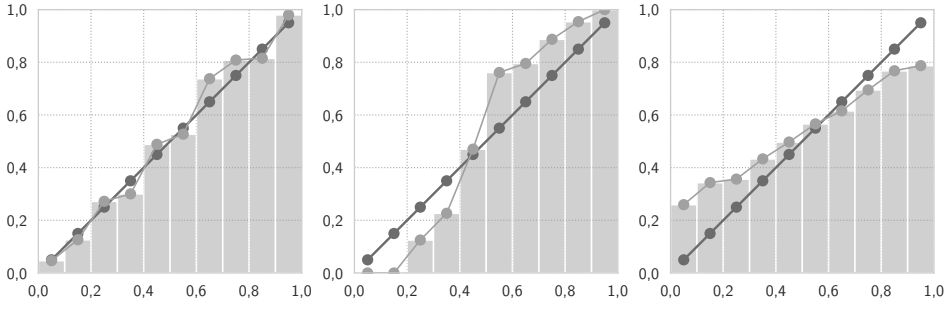
Так что разделяющая поверхность логистической регрессии для учёта несбалансированных классов должна сместиться в сторону более маленького класса на константу, причём даже известно на какую: на $\log \frac{p(C_1)}{p(C_2)}$. Более того, при выводе формулы (5.14) мы нигде не пользовались собственно линейной моделью (5.13) в том смысле, что вместо линейной зависимости $\boldsymbol{w}^\top \boldsymbol{x} + w_0$ там могла бы стоять любая другая. Это значит, что результат (5.14) верен для любого вероятностного классификатора, независимо от того, как именно мы получаем оценку логарифма шансов: хоть линейной моделью, хоть нейросетью на основе трансформера. Получается очень удобно: какой бы сложной ни была разделяющая поверхность, её преобразование при переходе к несбалансированным классам будет одним и тем же и очень простым — нужно просто сдвинуть её на константу, которую тоже легко оценить.

В подразделе 5.5.4 мы найдём этому результату ещё одно важное и интересное применение. А сейчас переходим ко второй части плана: к калибровке классификаторов.

5.5.2. Калибровка классификаторов

Много раз, начиная с раздела 2.2 и далее, мы говорили о том, что в задачах машинного обучения полезно уметь не только выдавать хороший ответ, но и уметь оценивать свою уверенность в этом ответе. Я отмечал, что в идеале на выходе процесса обучения получается модель, которая выдаёт не только точечные предсказания — число в случае регрессии или номер класса в случае классификации, — но целое предсказательное распределение.

Однако как оценить, насколько хорошо модель машинного обучения это делает? Базовая идея довольно проста: представьте себе, что вы предсказываете результаты каких-либо событий, примерно как уже упоминавшийся в этой книге (в подразделе 1.5.5) Джордж Гэллап предсказывал результаты выборов президента США. Предположим, что в сделавших его знаменитыми выборах 1936 года он оценил вероятность победы Франклина Рузвельта в 70 %, а Альфа Лэндона — в 30 %. Рузвельт победил, и это обеспечило репутацию Гэллапа, потому что Гэллап «сделал ставку», противоположную результатам других опросов; но если бы победил Лэндон, это ведь тоже никак не противоречило бы выданной оценке вероятностей: события с вероятностью 30 % случаются сплошь и рядом. Да и события с вероятностью 1 % и даже 0,1 % происходят с нами каждый день.



(а) Хорошо откалиброванный (б) Недостаточно уверенный (в) Слишком уверенный

Рис. 5.17. Калибровка классификаторов: (а) хорошо откалиброванный классификатор, (б) недостаточно уверенный классификатор, (в) слишком уверенный классификатор

Подождите, но ведь этак получается, что прогнозы вообще невозможно опровергнуть? Вряд ли удастся найти серьёзного прогнозиста, который бы стал явно противоречить правилу Кромвеля (вспомните раздел 2.2) и выдал бы предсказание в 100 %, которое можно было бы фальсифицировать напрямую.

Действительно, одна ошибка в вероятностном предсказании ничего не значит. Чтобы оценить качество прогнозов и, например, сравнить друг с другом разных прогнозистов, одного события недостаточно, нужно собрать целую выборку. В идеальном случае события, для которых была предсказана вероятность около 20 %, должны в этой выборке происходить в среднем один раз из пяти; те, для которых предсказывалась вероятность в 50 %, — каждый второй раз и так далее.

Такая оценка называется *калибровкой* (calibration) классификаторов [101]. Говорят, что классификатор *хорошо откалиброван* (well-calibrated), если вероятности, которые он предсказывает, подтверждаются на практике, то есть совпадают с истинными вероятностями соответствующих событий: для классификатора $\boldsymbol{h}$, выдающего вектор вероятностей K классов $\boldsymbol{h}(\boldsymbol{x}) \in [0,1]^d$, должно быть верно, что

$$p\left(\boldsymbol{x} \in C_k | \boldsymbol{h}(\boldsymbol{x}) = q\right) = q. \tag{5.15}$$

На рис. 5.17 условно показаны три варианта, которые могут встретиться по отношению к калибровке на практике; на графиках по горизонтальной оси отложены предсказанные классификатором вероятности, столбцами и соответствующей ломаной линией показаны результаты тестирования классификатора (в реальной жизни или новых условиях), а прямая диагональная линия показывает теоретически идеальную калибровку, когда событие с предсказанной вероятностью 0,1 появляется на практике ровно в 10 % случаев, событие с вероятностью 0,2 — в 20 % случаев и так далее. И даже для классификатора, у которого нет смещения в ту или иную сторону, могут возникнуть такие ситуации:

- если классификатор хорошо откалиброван (well-calibrated; см. рис. 5.17, *а*), для него эмпирическая кривая будет похожа на диагональную прямую;
- если классификатор недостаточно уверен в своих предсказаниях (underconfident; см. рис. 5.17, *б*), то его эмпирические результаты будут образовывать кривую, проходящую ближе к нулю и единице, чем диагональ: когда такой классификатор предсказывает, скажем, $p = 0{,}6$, на самом деле событие происходит в 80 % случаев;
- а если классификатор слишком уверен в своих предсказаниях (overconfident; см. рис. 5.17, *в*), то его реальные эмпирические результаты будут «отставать» в степени уверенности от предсказаний: классификатор уже предсказывает 0,95, а в реальности событие происходит в 70–80 % случаев.

Как численно оценить качество калибровки? Есть два стандартных подхода. Во-первых, можно подсчитать хорошо нам знакомый логарифм правдоподобия для правильных ответов с точки зрения предсказаний классификатора: если для тестового примера $\boldsymbol{x}$ обученный классификатор предсказал вектор вероятностей классов $\boldsymbol{y}$ при истинном векторе (one-hot представлении правильного ответа) $\boldsymbol{t}$, то чем выше $-\sum_{k=1}^{K} t_k \log y_k$, тем лучше.

Для оценки качества калибровки на выборке $\{(\boldsymbol{x}_n, \boldsymbol{t}_n)\}_{n=1}^{N}$ можно эти числа просто усреднить, получив метрику

$$\varphi_{\text{LL}} = -\frac{1}{N}\sum_{n=1}^{N}\sum_{k=1}^{K} t_{nk} \log y_{nk}.$$

На практике, впрочем, чаще используется так называемый *показатель Брайера* (Brier score), предложенный Гленом Брайером в 1950 году [78]:

$$\varphi_{\text{BS}} = \frac{1}{N}\sum_{n=1}^{N}\sum_{k=1}^{K} (t_{nk} - y_{nk})^2 . \tag{5.16}$$

Его свойства мы в основном и будем рассматривать ниже.

Здесь уже ясно, что говорить о хорошо или плохо откалиброванном классификаторе на одном примере не имеет никакого смысла: чтобы оценить калибровку, нужна довольно большая тестовая выборка из разных примеров, для которых классификатор будет давать разные предсказания, чтобы по этой выборке можно было построить такую эмпирическую кривую.

С этой проблемой регулярно сталкиваются люди, делающие публичные прогнозы: если ты предсказал, что «кандидат А победит на выборах с вероятностью 80 %», а победил на самом деле кандидат Б, люди станут думать, что ты плохой прогнозист, хотя это совершенно ничего не значит, события с вероятностью 20 % происходят сплошь и рядом.

Кстати, не случайно статья Брайера [78] была опубликована именно в журнале *Monthly Weather Review*: впервые о калибровке классификаторов заговорили применительно к прогнозам погоды [417, 418]. Ещё в XVIII веке Джон Дальтон[1] [129], а в начале XIX века Жан-Батист Ламарк[2] [328] формулировали свои предсказания в терминах вероятностей, иногда даже выражая их численно. Но считается, что в явном виде сформулировал и поставил задачу калибровки вероятностных предсказаний австралийский метеоролог Уильям Кук. В 1906 году Кук писал об этом так [115, 116]: «...всякий, кто должен выдавать ежедневные прогнозы, знает, что иногда они уверены в своих прогнозах, а иногда сильно сомневаются в том, какая будет погода. Мне кажется, что уверенность или неуверенность — это очень важная часть прогноза, и её нужно уметь выражать. Было бы несправедливо по отношению к прогнозисту присваивать равный вес всем его предсказаниям... более научно и более честно было бы разрешить иногда говорить: "Я совсем не уверен в завтрашней погоде" <...> а иногда — "Погода завтра почти наверняка будет такова"».

Кук предложил снабжать прогнозы весами, которые бы отражали значения вероятностей; табл. 5.2 показывает описания его весов и результаты проведённой им калибровки, то есть результаты прогнозов в двух австралийских округах, разделённых по этим весам [115]. Как видите, эмпирические вероятности предсказанных событий действительно убывают вместе с весами, хотя текстовые описания Кука, пожалуй, занижают реальные вероятности.

А в 1920 году американский метеоролог Клив Халленбек опубликовал результаты калибровки вероятностных предсказаний дождя, которые уже выглядели вполне современно (и сама калибровка оказалась сделана великолепно — предсказанные вероятности очень хорошо сходятся с результатами экспериментов); они показаны в табл. 5.3 [234]. Халленбек отмечает, что указывать вероятности просили сами потребители прогноза погоды: «...запрос на такие прогнозы существовал уже несколько лет, ведь обычный вопрос от фермеров этого округа звучал так: "Каковы *шансы*, что пойдёт дождь?"» [234].

---

[1] *Джон Дальтон* (John Dalton, 1766–1844) — английский физик, химик, метеоролог. Всем известны его исследования цветовой слепоты, основанные в том числе на собственном опыте, но это была лишь одна из первых работ в долгой и славной карьере: Дальтон открыл закон парциальных давлений, закон растворимости газов в жидкостях, обнаружил явление полимеризации, 57 лет вёл метеорологический дневник, где зафиксировал более 200 000 наблюдений. Но главное — именно Дальтон заложил основы атомной теории строения вещества, определив относительные атомные веса нескольких элементов, да и вообще введя понятие атомного веса. Когда в 1940 году бóльшая часть работ Дальтона была уничтожена при бомбардировке Манчестера, Айзек Азимов написал: «На войне умирают не только живые».

[2] *Жан-Батист Пьер Антуан де Моне, шевалье де Ламарк* (Jean-Baptiste Pierre Antoine de Monet, chevalier de Lamarck, 1744–1829) — французский учёный, естествоиспытатель, биолог. Ламарк был первым, кто попытался создать стройную теорию эволюции — и создал её, пусть впоследствии и оказалось ложным его базовое предположение о том, что эволюцией движет внутренне присущее «живому веществу» стремление к совершенствованию и упражнение органов. Впрочем, сейчас многие учёные говорят, что не всё так однозначно: например, CRISPR-кассеты в ДНК бактерий меняют генетический материал именно при жизни, «обучаясь» на ДНК бактериофагов.

Таблица 5.2. Веса уверенности Уильяма Кука и результаты его экспериментов [115]

Вес	Описание	Юго-Восточный округ		Округ Голдфилдс	
		Верно	Нет	Верно	Нет
5	Можно положиться с абсолютной уверенностью	435	5	240	5
4	Можно положиться, но один раз из десяти прогноз может быть неверен	573	38	337	22
3	Сомнительно: может быть неверно четыре раза из десяти	131	38	102	25
2	Возможно, но маловероятно	24	18	13	11
1	Есть лишь небольшая возможность	6	5	5	3

Таблица 5.3. Результаты экспериментов Клива Халленбека [234]

Предсказанная вероятность, %	Число прогнозов	Число дождливых дней	Истинный процент
Выше 90	1	1	100
От 80 до 89	1	1	100
От 70 до 79	7	6	86
От 60 до 69	15	11	75
От 50 до 59	13	8	62
От 40 до 49	15	8	53
От 30 до 39	18	6	33
От 20 до 29	31	7	23
От 10 до 19	22	4	18
От 0 до 9	0	–	–

Но вернёмся к показателю Брайера (5.16). Хотя квадрат разности здесь выглядит немного странно (мы вычитаем друг из друга вероятности), он имеет содержательный смысл. Дальше я для простоты буду говорить о бинарном случае, когда классов два, и показатель Брайера проще уменьшить в два раза (слагаемые в сумме по двум классам получаются равными друг другу) и определить как

$$\varphi_{\mathrm{BS}} = \frac{1}{N}\sum_{n=1}^{N}(t_n - y_n)^2\,. \tag{5.17}$$

Во-первых, это среднеквадратичное отклонение калибровочной кривой нашего классификатора (как на рис. 5.17) от диагонали. Во-вторых, его можно разложить на ещё более интересные компоненты. Разложение показателя Брайера [416, 541] пользуется идеей разделения суммы квадратов отклонений на группы, выраженной в формуле (5.1). Дело в том, что на практике вероятностные прогнозы почти никогда не представляют собой вещественные числа, а делятся на

несколько дискретных категорий, как в табл. 5.2 и 5.3. Давайте присвоим каждой из таких категорий некоторое численное значение вероятности p_i; например, категорию «От 50 до 59» свернём в предсказание $p_i = 0{,}55$. Иначе говоря, каждый прогноз y_n на практике обычно представляет собой одно из нескольких дискретных значений предсказанной вероятности p_i, $y_n \in \{p_1, \ldots, p_M\}$.

Пусть S_i — это множество прогнозов, которые оказались равны p_i, $S_i = \{n \mid y_n = p_i\}$, и $|S_i| = N_i$; пусть также T_i — это общее число событий, которые действительно произошли после прогнозов из категории p_i, то есть $T_i = \sum_{n:y_n=p_i} t_n$, и $\bar{t}_i$ — эмпирическая вероятность $\bar{t}_i = \frac{T_i}{N_i}$. Давайте в таких обозначениях построим разложение φ_{BS}:

$$\varphi_{\text{BS}} = \frac{1}{N}\sum_{n=1}^{N}(t_n - y_n)^2 = \frac{1}{N}\sum_{i=1}^{M}\sum_{n\in S_i}(t_n - p_i)^2\,.$$

Воспользуемся результатом (5.1) внутри скобки:

$$\sum\nolimits_{n\in S_i}(t_n - p_i)^2 = \sum\nolimits_{n\in S_i}(t_n - \bar{t}_i)^2 + N_i\,(\bar{t}_i - p_i)^2\,,$$

а значит,

$$\varphi_{\text{BS}} = \sum_{i=1}^{M}\frac{N_i}{N}(\bar{t}_i - p_i)^2 + \frac{1}{N}\sum_{n=1}^{N}(t_n - \bar{t}_i)^2\,. \tag{5.18}$$

Теперь воспользуемся результатом (5.1) ещё раз для глобального среднего $\bar{t} = \frac{1}{N}\sum t_n$:

$$\frac{1}{N}\sum_{n=1}^{N}(t_n - \bar{t})^2 = \frac{1}{N}\sum_{n=1}^{N}(t_n - \bar{t}_i)^2 + \sum_{i=1}^{M}\frac{N_i}{N}(\bar{t}_i - \bar{t})^2\,. \tag{5.19}$$

Заметим, что слева в (5.19) стоит просто эмпирическая дисперсия бросаний монетки, то есть $\bar{t}(1-\bar{t})$, и подставим (5.19) в (5.18). Получится то самое разложение:

$$\varphi_{\text{BS}} = \text{REL} - \text{RES} + \text{UNC}, \tag{5.20}$$

$$\text{где}\quad \text{REL} = \sum_{i=1}^{M}\frac{N_i}{N}(\bar{t}_i - p_i)^2\,,\quad \text{RES} = \sum_{i=1}^{M}\frac{N_i}{N}(\bar{t}_i - \bar{t})^2\,,\quad \text{UNC} = \bar{t}(1-\bar{t}).$$

Слагаемые в разложении (5.20) можно проинтерпретировать и обсудить:

- *надёжность* (reliability) REL — это как раз и есть мера качества калибровки классификатора: это (взвешенное) среднее отклонение предсказаний классификатора p_i от эмпирических вероятностей, которые на самом деле получаются в тех случаях, когда классификатор предсказывает p_i;

- *разрешение* (resolution) RES — это мера того, насколько наш классификатор вообще пытается делать разные предсказания на разных входах, то есть насколько высоко разрешение его предсказаний вероятности; в частности, в крайнем случае, когда классификатор всегда выдаёт $\bar{t}$, его разрешение будет равно нулю;
- наконец, *неопределённость* (uncertainty) UNC — это шум в данных, дисперсия целевой случайной величины, которую мы никак не контролируем: значение этого слагаемого никак не зависит от классификатора.

Часто второе и третье слагаемое объединяют в так называемую *детализацию* (refinement) REF

$$\varphi_{\text{BS}} = \text{REL} + \text{REF}, \quad \text{где} \quad \text{REF} = \sum_{i=1}^{I} \frac{N_i}{N} \bar{t}_i \, (1 - \bar{t}_i) \, ; \tag{5.21}$$

оставлю вам в качестве упражнения доказать, что REF = UNC − RES.

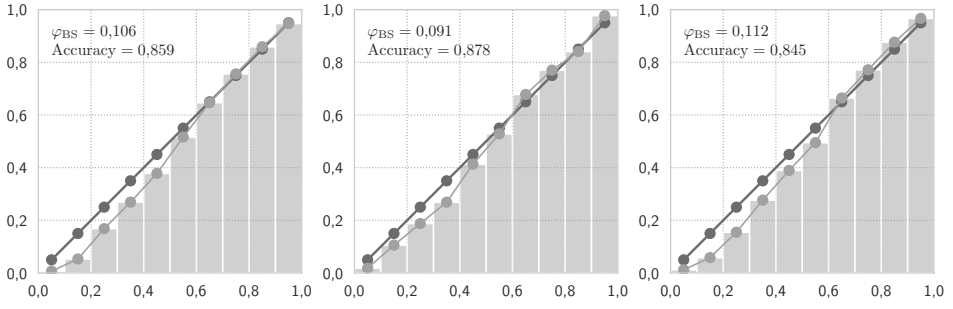
В разложениях (5.20) и (5.21) заложен баланс между надёжностью и разрешением: если сделать классификатор более детализированным, то разрешение увеличится (это хорошо, потому что в разложении (5.20) оно со знаком минус), но хорошую калибровку сделать будет сложнее. И наоборот, более грубый классификатор с меньшим числом значений откалибровать будет проще. Эти разложения очень похожи на разложение на смещение, дисперсию и шум, о котором мы будем более подробно говорить в подразделе 6.2.4.

Проиллюстрируем происходящее на конкретном примере. В подразделе 5.1.4 мы вводили датасет двумерной бинарной классификации; в нём каждый класс порождался из соответствующего нормального распределения (рис. 5.5). Давайте воспользуемся этим датасетом; во-первых, создадим для него тестовый набор данных из 5000 точек, на котором будем проверять калибровку классификаторов (разумеется, на практике вряд ли удастся найти тестовый набор в 50 раз больше обучающей выборки, но сейчас мы иллюстрируем теоретический результат, так что для наглядности лучше всё максимально сгладить):

```
mu1, sigma1 = np.array([-1, -.5]), np.array([[1, -.5], [-.5, 2]])
mu2, sigma2 = np.array([.5, 1]), np.array([[2, .5], [.5, .5]])
z, x = sample_mixture(100, 0.7, mu1, sigma1, mu2, sigma2)
test_z, test_x = sample_mixture(5000, 0.7, mu1, sigma1, mu2, sigma2)
```

Обучим модель, скажем тот же линейный дискриминантный анализ, и превратим предсказанные этой моделью вероятности в проценты. Рассмотрим, как выше, десять возможных значений предсказаний вероятности, от 5 до 95 %, так что будем округлять результат до этих десяти значений:

```
lda = LinearDiscriminantAnalysis(store_covariance=True)
y_lda = lda.fit(x, z).predict_proba(test_x)[:,1]
y_lda_d = np.array(100*np.round(y_lda+0.05, decimals=1)-5, dtype=int)
```



(а) LDA (б) QDA (в) Логистическая регрессия

Рис. 5.18. Калибровка разных классификаторов на примере из подраздела 5.1.4

Теперь в массиве `y_lda_d` лежат целочисленные значения процентов:

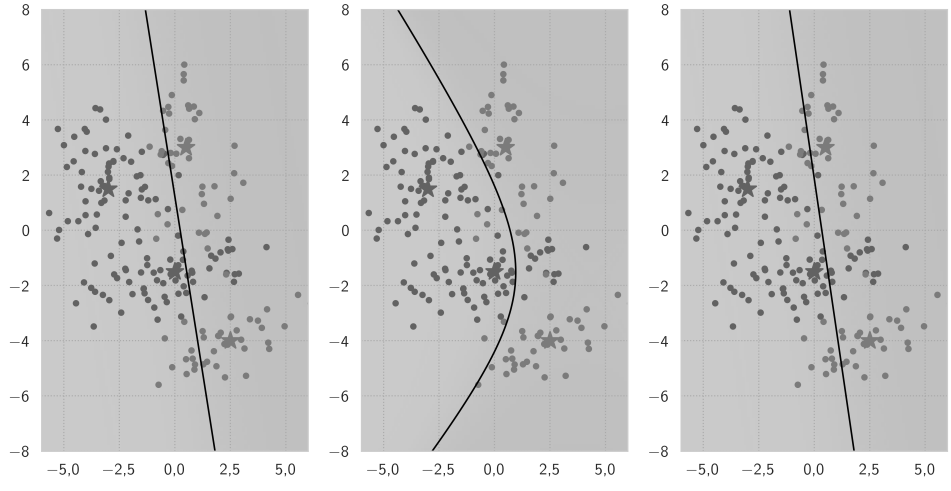
```
array([15, 95,  5, ..., 95, 45, 65])
```

Осталось только их подсчитать и изобразить в виде калибровочной кривой; разумеется, то же самое можно проделать и для любой другой модели, выдающей вероятностные предсказания. На рис. 5.18 показаны полученные результаты для LDA, QDA и логистической регрессии. Как видите, задача оказалась слишком простой: QDA здесь идеально калиброван по определению, потому что он в точности соответствует вероятностным предположениям, из которых получен датасет, но и LDA, и логистическая регрессия тоже оказались прекрасно калиброваны, разве что можно заметить некоторый недостаток уверенности в предсказаниях в нижней части спектра.

Показатели Брайера (которые также показаны на рис. 5.18 вместе с точностью предсказаний на тестовом множестве) в данном случае тоже практически не отличаются; отмечу всё же, что QDA выигрывает, пусть и совсем немного, то есть показатель Брайера соответствует точности предсказаний, и на достаточно большом тестовом наборе он также оказался способен выделить классификатор, который действительно делает в точности правильные предположения. Обратите внимание, что, хотя калибровка QDA близка к идеальной, его точность вовсе не стремится к единице, ведь в данных есть и просто случайный шум, против которого любой классификатор будет бессилен; в подразделе 6.2.4 мы это подробно обсудим.

Чтобы привести не такой идеальный пример (он нам и в следующей главе пригодится), давайте перейдём от разделения гауссианов к разделению нескольких *смесей* гауссианов (Gaussian mixtures). Породим точки каждого класса C_k не из распределения $\mathcal{N}(\boldsymbol{x}|\boldsymbol{\mu}_k, \Sigma_k)$, а из выпуклой комбинации двух гауссианов

$$p(\boldsymbol{x}|\pi, \boldsymbol{\mu}_{k1}, \Sigma_{k1}, \boldsymbol{\mu}_{k2}, \Sigma_{k2}) = \pi\mathcal{N}(\boldsymbol{x}|\boldsymbol{\mu}_{k1}, \Sigma_{k1}) + (1-\pi)\mathcal{N}(\boldsymbol{x}|\boldsymbol{\mu}_{k2}, \Sigma_{k2}).$$



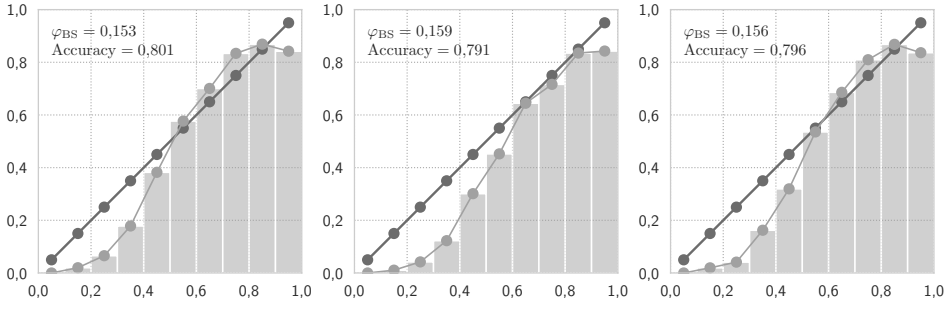
(а) LDA (б) QDA (в) Логистическая регрессия

Рис. 5.19. Классификаторы, разделяющие две смеси гауссианов

В коде это может выглядеть примерно так:

```
def sample_mixtures(N, pi, mus1, sigmas1, mus2, sigmas2):
    z = np.array( np.random.rand(N) < pi, dtype=int)
    res = np.zeros((N, 2))
    for i in range(N):
        if z[i] == 1:
            m = np.random.randint(mus1.shape[0])
            res[i] = np.random.multivariate_normal(mus1[m, :], sigmas1[m])
        else:
            m = np.random.randint(mus2.shape[0])
            res[i] = np.random.multivariate_normal(mus2[m, :], sigmas2[m])
    return z, res

mus1, sigmas1 = np.array([[0, -1.5], [-3, 1.5]]), \
  1.5*np.array([[[4, .5], [.5, .5]], [[1, -.5], [-.5, 2]]])
mus2, sigmas2 = np.array([[.5, 3], [2.5, -4]]), \
  1.5*np.array([[[1, -.5], [-.5, 2]], [[2, .5], [.5, .5]]])
zmix, xmix = sample_mixtures(200, 0.6, mus1, sigmas1, mus2, sigmas2)
test_zmix, test_xmix = sample_mixture(5000, 0.6, mu1, sigma1, mu2, sigma2)
```

Полученный набор точек (`xmix`, `zmix`) и стандартные классификаторы на нём показаны на рис. 5.19. Как видите, результат здесь уже совсем не такой идеальный. А кривые калибровки и соответствующие значения показателей Брайера и точности на тестовом множестве изображены на рис. 5.20.



(а) LDA (б) QDA (в) Логистическая регрессия

Рис. 5.20. Калибровка разных классификаторов на примере разделения смесей гауссианов

Здесь об идеальной калибровке говорить не приходится. Показатель Брайера снова в целом соответствует точности на тестовом множестве: по обоим критериям QDA теперь проигрывает линейным классификаторам, а они друг от друга практически не отличаются. В данном случае более сложные вероятностные предположения QDA только помешали, потому что на этот раз они не соответствовали действительности, хотя предположения LDA здесь тем более не выполняются. Кривая калибровки также позволяет увидеть, где именно и в какую сторону ошибаются классификаторы: все они довольно хорошо откалиброваны, когда предсказывают второй класс, то есть когда предсказанная вероятность больше 0,5 (это тот класс, который расположен в среднем левее на рис. 5.19), и не так хороши для первого. На рис. 5.19 это видно в том, что, если классификатор достаточно уверен во втором классе (точки значительно левее разделяющей поверхности), первый там вряд ли может появиться, а вот наоборот вполне может быть из-за «языка» второго класса, заходящего на «территорию» первого.

Всё это пока было разговором о том, как формально определить, что такое хорошо калиброванный классификатор[1]; но как добиться этой светлой цели? Один (не слишком, правда, интересный) ответ на этот вопрос даёт разложение (5.20):

[1] Без аллюзии на сборник пьес великого Иоганна Себастьяна здесь обойтись трудно, и отмечу в скобках, что аналогия эта имеет и вполне содержательный смысл. «Хорошо темперированный» у Баха в современных терминах означает равномерно темперированный клавир (здесь есть тонкости, но мы их опустим), то есть такой клавишный инструмент, у которого каждая октава делится на равные интервалы, в отличие от, например, чистого строя, который порождается математически наиболее простыми интервалами чистой октавы (1:2), чистой квинты (2:3) и большой терции (4:5). Равномерно темперированный строй позволяет транспонировать музыкальное произведение в любую тональность, не потеряв благозвучия; а хорошо калиброванный классификатор позволяет, как мы обсудим в подразделе 5.5.3, в том числе и «транспонировать» классификатор в другую цену ошибки, сохранив качество классификации.

чтобы получить идеальную калибровку, можно просто максимально снизить разрешение и всегда выдавать одну и ту же глобальную вероятность. Это, конечно, формально верно, но по сути издевательство: разложение (5.20) для того и нужно, чтобы понимать, что для снижения ожидаемой ошибки классификатора нужно не только снижать первое слагаемое (5.20) (надёжность REL), но и не слишком повышать второе (разрешение RES).

Есть и более содержательные ответы на вопрос о том, как откалибровать классификатор [101]. Задачу можно поставить так: если мы уже получили некоторый классификатор, который задачу в целом решает, но откалиброван не очень хорошо, как на рис. 5.18, мы можем обучить дополнительное преобразование, которое превратит эту кривую в идеальную диагональ. Например, можно попробовать обучить параметризованный логистический сигмоид:

$$\sigma(s;\mu,\alpha) = \frac{1}{1+e^{-\alpha(s-\mu)}}, \tag{5.22}$$

где s — это оценка (score), которую даёт классификатор, а μ и α — параметры масштабирования, которые мы, собственно, и обучим. Такой подход называется *масштабированием Платта* (Platt scaling), потому что впервые его предложил ещё в 1999 году Джон Платт [471]. Предложил, кстати, не для калибровки вероятностных классификаторов, а для того, чтобы преобразовывать в вероятности выходы классификаторов, которые вероятностными вовсе не являются, например выходы метода опорных векторов; именно поэтому этот метод называется «масштабированием». Впрочем, метод опорных векторов мы в этой книге обсуждать не будем, да и в масштабирование Платта и другие методы калибровки углубляться не станем — я просто оставлю ещё несколько интересных ссылок [86, 345, 434].

«Но подождите, — скажет на этом месте неискушённый (или вполне искушённый, но желающий немного потроллить автора) читатель, — зачем нам вообще беспокоиться о калибровке, если сами вероятности нам не нужны и мы в итоге всё равно решаем задачу классификации?» Да, абстрактно звучит неплохо, когда при выданной вероятности 20 % событие произойдёт ровно в одном случае из пяти, но ведь ответ на бинарную задачу всё равно будет отрицательным? Зачем же нужна хорошая калибровка? Давайте об этом и поговорим.

5.5.3. Как изменить классификатор в новых условиях

Кроме чувства глубокого удовлетворения от точных оценок вероятностей, хорошая калибровка нужна для того, чтобы вообще *позволять рассматривать* наш классификатор как выдающий оценки вероятностей. На практике этот отнюдь не всегда разумно! Например, наивный байесовский классификатор, которому будет посвящён раздел 5.6, тоже формально выдаёт оценки вероятностей классов, но в реальности наивный Байес за счёт своих избыточно сильных предположений почти всегда оказывается откалиброван отвратительно. Даже в ситуациях,

Таблица 5.4. Матрица цен ошибок классификации

	Правильный ответ C_0	**Правильный ответ** C_1
Предсказан C_0	c_{00}	c_{00}
Предсказан C_1	c_{00}	c_{00}

когда наивный Байес хорошо работает по прямому назначению, как классификатор, выдаваемые им числа лучше не принимать всерьёз как оценки вероятностей.

Но если вероятности всё-таки оцениваются разумно, то оказывается, что с полученным классификатором мы можем сделать много интересного; хорошая калибровка позволяет изменить правило классификации так, чтобы учесть «перекосы» как в данных, так и в целевой функции для классификатора [167, 190, 255]. Давайте разберём эти эффекты последовательно; для простоты я буду говорить о бинарной классификации, и обратите внимание, что весь этот раздел относится к вероятностной классификации в целом, а не конкретно к задаче калибровки классификатора.

Во-первых, давайте вернёмся к целевой функции, функции ошибки классификации. В подразделе 2.2.3 мы обсуждали, что даже у простейшего бинарного классификатора можно найти массу разных способов измерять качество, то есть много разных видов ошибок: чувствительность, специфичность, точность и так далее. По сути, все они сводились к разным способам свести в одно число табличку из четырёх видов исходов: истинно положительные (предсказан верно, скажем, класс C_1), истинно отрицательные (предсказан верно класс C_0 — здесь удобнее будет обозначить их нулём и единицей), ложноположительные (предсказан класс C_1, а на самом деле C_0) и ложноотрицательные (предсказан класс C_0, а на самом деле C_1).

Так вот, в жизни часто встречаются ситуации, когда разные виды ошибок имеют *разную цену*. Представьте, например, что вы, как в подразделе 2.2.3, строите медицинский тест, который должен определять наличие некоторой потенциально опасной болезни; например, вы распознаёте рак кожи по фотографии. Тогда:

- ложное срабатывание теста (ложноположительный результат) будет, конечно, не бесплатным: человек обеспокоится, расстроится и пойдёт делать другой, более надёжный тест (скажем, биопсию), который уже покажет истинное положение дел; ценой такой ошибки будет некоторый стресс для пациента плюс цена более точного теста (очевидно, она выше, чем цена нашего теста, иначе зачем наш вообще нужен);
- но ложноотрицательный результат будет гораздо хуже: он приведёт к тому, что человек успокоится и не будет делать более точные тесты, а болезнь будет спокойно прогрессировать; цена этой ошибки, боюсь, для пациента вообще ничем сверху не ограничена.

Такие соображения хотелось бы как-то учесть при конструировании классификаторов. В общем виде это значит, что нам дана некоторая матрица штрафов, цен ошибок (cost matrix), как показано в табл. 5.4. Если классификатор выдаёт оценки вероятностей $p(C_0|\boldsymbol{x})$ и $p(C_1|\boldsymbol{x})$, это значит, что его легко адаптировать к любой матрице штрафов. Для матрицы, показанной в табл. 5.4, мы хотим предсказывать C_1 тогда и только тогда, когда ожидаемый штраф от этого решения не больше, чем от предсказания C_0, то есть когда

$$p(C_0|\boldsymbol{x})\,c_{10} + p(C_1|\boldsymbol{x})\,c_{11} \le p(C_0|\boldsymbol{x})\,c_{00} + p(C_1|\boldsymbol{x})\,c_{01}. \tag{5.23}$$

Если мы обозначим предсказанную вероятность через $p(C_1|\boldsymbol{x}) = \hat{p}$, а значит, $p(C_0|\boldsymbol{x}) = 1 - \hat{p}$, то из условия (5.23) сразу получим, что порог вероятности $\hat{p}^*$, выше которого нужно предсказывать класс C_1, то есть значение $\hat{p}^*$, при котором неравенство (5.23) превращается в равенство, составляет

$$\hat{p}^* = \frac{c_{10} - c_{00}}{c_{10} - c_{00} + c_{01} - c_{11}}. \tag{5.24}$$

Заметим, что естественные условия на матрицу цен из табл. 5.4 требуют, чтобы цена неправильного ответа была выше, чем правильного: $c_{10} > c_{00}$ и $c_{01} > c_{11}$. А значит, числитель и знаменатель в формуле (5.24) будут положительными числами. Таким образом, если наш классификатор умел выдавать разумные оценки вероятностей, то его ответы легко адаптировать к любой матрице цен ошибок.

Во-вторых, у вероятностного классификатора можно изменить *априорные вероятности* классов. Часто бывает так, что при обучении априорные вероятности классов $p(C_0)$ и $p(C_1)$ одни, скажем, $p(C_1) = \pi$, $p(C_0) = 1 - \pi$ (например, часто бывает, что выборка для обучения сбалансирована, и $p(C_0) = p(C_1) = \frac{1}{2}$), а при реальном применении классификатора оказывается, что $p(C_1) = \pi' \ne \pi$. Что делать в такой ситуации?

Чтобы получить однозначный ответ, нужно сделать дополнительное вероятностное предположение о том, что этот сдвиг в априорных вероятностях не влияет на другие характеристики распределений в классах; формально говоря, нужно предположить, что $p(\boldsymbol{x}|C_0)$ и $p(\boldsymbol{x}|C_1)$ не меняются при переходе от π к π'. Но если это так, то дальше всё получается очень просто, по теореме Байеса:

$$\hat{p} = p(C_1|\boldsymbol{x}) = \frac{p(\boldsymbol{x}|C_1)\,p(C_1)}{p(\boldsymbol{x})} = \pi\frac{p(\boldsymbol{x}|C_1)}{p(\boldsymbol{x}|C_1)\,\pi + p(\boldsymbol{x}|C_0)\,(1-\pi)}. \tag{5.25}$$

Как и в логистической регрессии, разделим числитель и знаменатель на $p(\boldsymbol{x}|C_1)$ и перейдём к шансам $\alpha = \frac{p(\boldsymbol{x}|C_0)}{p(\boldsymbol{x}|C_1)}$:

$$\hat{p} = \frac{\pi}{\pi + \alpha(1-\pi)}. \tag{5.26}$$

Аналогично в популяции с новой априорной вероятностью π' получится, что

$$\hat{p}' = \pi' \frac{p\left(\boldsymbol{x}|C_1\right)}{p\left(\boldsymbol{x}|C_1\right)\pi' + p\left(\boldsymbol{x}|C_0\right)\left(1-\pi'\right)} = \frac{\pi'}{\pi' + \alpha(1-\pi')}. \tag{5.27}$$

Теперь в формуле (5.26) можно выразить α через $\hat{p}$ и π:

$$\pi = \hat{p}\pi + \alpha\hat{p}(1-\pi), \quad \text{то есть} \quad \alpha = \frac{\pi\left(1-\hat{p}\right)}{\hat{p}\left(1-\pi\right)}, \tag{5.28}$$

а затем подставить результат (5.28) в уравнение (5.27):

$$\hat{p}' = \frac{\pi'}{\pi' + \frac{\pi(1-\hat{p})(1-\pi')}{\hat{p}(1-\pi)}} = \hat{p}\frac{\pi'\left(1-\pi\right)}{\pi'\left(\hat{p}-\pi\right) + \pi\left(1-\hat{p}\right)}. \tag{5.29}$$

Мы получили простую формулу, при помощи которой можно перевзвесить выходы вероятностного классификатора в случае, когда априорные вероятности классов изменились. Важно отметить, что результат (5.29) не делает никаких предположений о том, как именно классификатор пришёл к своей оценке $\hat{p}$; он работает для любой формы правдоподобия и любого вида вероятностного классификатора в целом. В подразделе 5.5.4 мы ещё поговорим о подобном эффекте и о том, к чему он приводит, если π' неизвестно.

В-третьих, отметим, что, даже если классификатор не даёт оценок вероятностей напрямую, часто можно эмулировать изменившиеся априорные вероятности, перебалансировав классы. Иначе говоря, можно попробовать провести процедуру, обратную той, что мы рассматривали в подразделе 5.5.1: не убирать эффект от несбалансированных классов, а искусственно внести его, чтобы использовать «во благо». Как и в подразделе 5.5.1, этого эффекта можно достичь, либо изменяя семплирование точек в классах, либо, если классификатор это позволяет, изменяя вес, с которым каждая точка учитывается при обучении.

Кстати, в работе [167], которой во многом следует этот раздел, известный статистик и специалист по машинному обучению Чарльз Элкан формулирует эти результаты в явном виде: не просто в терминах изменения априорных вероятностей, а буквально в терминах того, насколько следует изменить число отрицательных примеров в обучающей выборке. Изменяя соотношение классов в выборке, мы можем изменить ту самую априорную вероятность $\pi = p\left(C_1\right)$. Даже если имеющийся классификатор не откалиброван выдавать вероятностные оценки, а просто хорошо умеет проводить разделяющую поверхность для одного-единственного порога $\hat{p}^*$, чаще всего $\hat{p}^* = \frac{1}{2}$, за счёт изменения обучающей выборки мы можем контролировать π для нашего классификатора и, меняя π, тем самым фактически менять порог его срабатывания!

Для того чтобы понять, как нужно изменить π, достаточно воспользоваться уже полученной формулой (5.29), только выразить теперь из неё π':

$$\hat{p}'\left(\pi'\left(\hat{p}-\pi\right)+\pi\left(1-\hat{p}\right)\right)=\hat{p}\pi'\left(1-\pi\right), \tag{5.30}$$

а значит,

$$\pi'=\frac{\hat{p}'\pi\left(1-\hat{p}\right)}{\hat{p}\left(1-\pi\right)+\hat{p}'\left(\pi-\hat{p}\right)}. \tag{5.31}$$

Уравнение (5.31) показывает, каким должно быть новое априорное распределение. Предположим, как часто бывает в жизни, что число положительных примеров Pos (например, настоящих меланом в выборке изображений) фиксировано и больше их не станет, а вот отрицательных примеров (обычных родинок) мы можем добавить сколько угодно, меняя число Neg. В наших обозначениях

$$\pi=\frac{\text{Pos}}{\text{Pos}+\text{Neg}}=\frac{1}{1+\frac{\text{Neg}}{\text{Pos}}},\qquad \pi'=\frac{\text{Pos}}{\text{Pos}+\text{Neg}'}=\frac{1}{1+\frac{\text{Neg}'}{\text{Pos}}},$$

то есть $\frac{\text{Neg}'}{\text{Pos}}=\frac{1-\pi'}{\pi'}$, и при помощи уравнения (5.31) мы можем оценить, во сколько раз нужно изменить число отрицательных примеров:

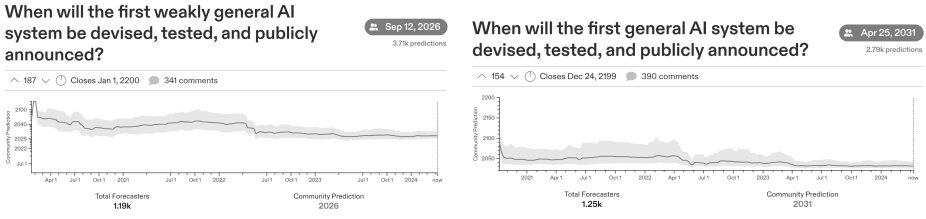
$$\begin{aligned}\frac{\text{Neg}'}{\text{Neg}}=\frac{1-\pi'}{\pi'}\frac{\pi}{1-\pi}=\\ =\frac{\pi}{1-\pi}\frac{\hat{p}\left(1-\pi\right)+\hat{p}'\left(\pi-\hat{p}\right)-\hat{p}'\pi\left(1-\hat{p}\right)}{\hat{p}'\pi\left(1-\hat{p}\right)}=\\ =\frac{\pi}{1-\pi}\frac{\hat{p}\left(1-\pi\right)-\hat{p}'\hat{p}+\hat{p}'\pi\hat{p}}{\hat{p}'\pi\left(1-\hat{p}\right)}=\frac{\hat{p}\left(1-\hat{p}'\right)}{\hat{p}'\left(1-\hat{p}\right)}.\end{aligned} \tag{5.32}$$

Получился красивый результат, который вполне можно использовать и на практике, подбирая размер выборки под нужный порог срабатывания.

Все эти рассуждения показывают (надеюсь, убедительно), что хорошо калиброванный классификатор — это очень полезная вещь, которая сообщает гораздо больше информации, чем просто бинарный результат классификации, и может быть модифицирована в разных направлениях.

Хорошая калибровка — это важная цель и для прогнозистов, которые делают практические предсказания будущего. По результатам одного предсказания оценить качество прогнозов невозможно, но у «профессиональных прогнозистов», которые постоянно делают те или иные ставки на наступление различных событий, накапливается достаточная статистика для оценки и уточнения своей калибровки. Существуют сообщества людей, которые ставят живые деньги на различные события будущего, и речь здесь вовсе не о ставках на результаты спортивных событий, а о прогнозах событий, зачастую важных для развития всего человечества; такие сообщества называются *рынками предсказаний* (prediction markets).

Постепенно на таких рынках выделяются особенно успешные *superforecasters* («сверхпредсказатели») — люди или коллективы, чьи предсказания устойчиво



(а) (б)

Рис. 5.21. Когда появится искусственный интеллект? Мнение пользователей *Metaculus* на 9 апреля 2024 года

оказываются более точными, чем даже усреднённые прогнозы других людей[1]. Для более обстоятельного знакомства с основными принципами такого прогнозирования я рекомендую популярную книгу Филипа Тетлока и Дэна Гарднера *Superforecasters* [588], а на рис. 5.21 привожу примеры текущих прогнозов с крупнейшего портала такого рода — *Metaculus*. Как видите, профессиональные предсказатели очень оптимистичны по поводу развития искусственного интеллекта, и, как показывают графики, со временем они становятся только оптимистичнее: ответы на оба вопроса стали существенно меньше (то есть ближе к настоящему времени) за последний год (я пишу эти строки в апреле 2024 года). Разумеется, в таких вопросах самое главное — это определить, что такое *general AI system* и *weakly general AI system*, и к формализации, принятой на *Metaculus*, можно придраться, но это долгий разговор, который мы сейчас вести не будем.

Так вот, оказывается, что хорошая калибровка очень важна для профессионального прогнозиста, хотя опять же каждый конкретный вопрос задаётся бинарным образом. Тетлок и Гарднер анализируют результаты «сверхпредсказателей» и показывают, что все они очень хорошо откалиброваны в своих предсказаниях. Более того, есть даже зависимость между тем, насколько точно предсказатель указывает вероятности, и тем, насколько верны будут его прогнозы: люди, которые указывают вероятности в десятках процентов ($10, 20\,\%, \ldots$), будут в среднем менее точны в прогнозах, чем люди, оценивающие вероятность до

[1] Здесь у читателя может возникнуть вопрос, почему «даже» — что хорошего в усреднении оценок, которые заведомо неточны? Оказывается, что усреднение даже плохих оценок часто приближает нас к истине; этот эффект называется *wisdom of the crowd* («мудрость толпы») [571]. О мудрости толпы в контексте принятия решений писал ещё Аристотель. А статистический эффект был замечен, когда в 1906 году на ярмарке в Плимуте Фрэнсис Гальтон подсчитал медиану оценок веса свежезабитого быка среди 800 посетителей (угадывание веса туши было популярным ярмарочным развлечением) и очень удивился, получив величину, точную до 1 %. Весь смысл рынка предсказаний состоит не в перераспределении денежных средств между участниками, а в том, чтобы приблизиться к истине за счёт разумного усреднения мнений людей, которые деньгами мотивированы выдавать наилучшую возможную для себя оценку.

Таблица 5.5. Общий вид результата исследования о вреде курения.

	Болезни нет, $y = 0$	**Болезнь есть,** $y = 1$
Не курит, $x = 0$	n_{00}	n_{01}
Курит, $x = 1$	n_{10}	n_{11}

пяти процентов ($5, 10, 15\,\%, \ldots$), а последние будут менее точны, чем люди, опускающиеся на уровень единиц процентов [588].

Можно попытаться улучшить свою собственную калибровку, например ведя дневник сделанных предсказаний, где каждое предсказание будет снабжено вероятностью. Затем, когда результаты станут известны, можно будет перечитать дневник и проанализировать, с чем были связаны самые грубые ошибки. Даже если вы не хотите становиться «сверхпредсказателем», всегда полезно подумать, какую именно вероятность вы готовы присвоить тому или иному своему мнению или прогнозу — уже само это когнитивное усилие, скорее всего, повысит точность ваших оценок.

На этом мы заканчиваем разговор о калибровке классификаторов и её приложениях. В последнем разделе вас ждёт пример классификатора, который, во-первых, совершенно не является геометрическим по своей природе, а во-вторых, позволит нам поговорить в целом о порождающих моделях в машинном обучении и их соотношении с моделями дискриминирующими, которые мы в основном рассматривали до сих пор. Нас ждёт наивный байесовский классификатор, но сначала — обещанный дополнительный раздел.

**5.5.4. Проспективные и ретроспективные исследования*

Этот раздел «со звёздочкой» в некотором формальном смысле целиком описывается формулой (5.14), но для его мотивации нам придётся зайти с немного другой стороны.

Представьте себе, что вы хотите всё-таки рассеять сомнения сэра Рональда Фишера, о которых мы говорили в подразделе 4.3.3, и доказать, что курение вредно для здоровья. Вы хотите взять некоторую выборку курильщиков, сравнить её с контрольной группой не-курильщиков (будем считать, что все выборки в наших воображаемых исследованиях идеально репрезентативны и единственный различающий их фактор — это собственно курение) и понять, увеличивается ли риск, скажем, онкологических заболеваний.

В результате исследования получится таблица, подобная табл. 5.5: один бинарный признак x (курит человек или нет) и одна бинарная целевая переменная y (развилось онкологическое заболевание или нет) дают четыре возможных варианта. Давайте обозначим через n_{ij} число людей в выборке, попавших в соответствующий вариант, а через $n = n_{00} + n_{01} + n_{10} + n_{11}$ — общее число людей в нашем исследовании.

На этом месте можно определить эмпирические оценки всевозможных вероятностей — для каждого события по отдельности:

$$\pi_{00} = \frac{n_{00}}{n}, \quad \pi_{01} = \frac{n_{01}}{n}, \quad \pi_{10} = \frac{n_{10}}{n}, \quad \pi_{11} = \frac{n_{11}}{n},$$

для общей вероятности болезни в выборке

$$\pi_{1*} = \frac{n_{1*}}{n}, \quad \text{где} \quad n_{1*} = n_{10} + n_{11},$$

и для вероятности курения в выборке

$$\pi_{*1} = \frac{n_{*1}}{n}, \quad \text{где} \quad n_{*1} = n_{01} + n_{11},$$

а также их дополнения $\pi_{0*} = 1 - \pi_{1*}$, $\pi_{*0} = 1 - \pi_{*1}$.

На первый взгляд кажется, что нам нужно сравнить две вероятности:

$$p\,(d = 1|x = 0) = \frac{n_{01}}{n_{0*}} = \frac{\pi_{01}}{\pi_{0*}} \quad \text{и} \quad p\,(d = 1|x = 1) = \frac{n_{11}}{n_{1*}} = \frac{\pi_{11}}{\pi_{1*}}. \tag{5.33}$$

Если $p\,(d = 1|x = 1)$ будет существенно больше, чем $p\,(d = 1|x = 0)$, то мы сделаем вывод о том, что курение действительно является фактором риска; а какие-то более сложные вероятностные рассуждения могут потребоваться разве что для того, чтобы определить, что в данном случае означает слово «существенно».

А если признак x будет не бинарным, а более сложным — например, у нас будет целый вектор признаков $\boldsymbol{x}$, — мы можем попробовать обучить ту или иную модель для того, чтобы выразить зависимость целевой переменной от $\boldsymbol{x}$, пропустив некоторую функцию $\eta(\boldsymbol{x})$, выражающую «склонности» (affinities) вектора признаков $\boldsymbol{x}$ к разным классам в виде произвольных вещественных чисел, через, скажем, логистический сигмоид:

$$\sigma\,(\eta(\boldsymbol{x})) = p\,(d = 1|x = 0) = \frac{\pi_{01}}{\pi_{0*}}, \quad 1 - \sigma\,(\eta(\boldsymbol{x})) = p\,(d = 1|x = 1) = \frac{\pi_{11}}{\pi_{1*}}. \tag{5.34}$$

Дальше можно будет обучать параметры модели $\eta(\boldsymbol{x})$; например, для линейной модели $\eta(\boldsymbol{x}) = \boldsymbol{w}^\top \boldsymbol{x}$ получится хорошо знакомая нам логистическая регрессия.

Это выглядит довольно просто, и всё это действительно теоретически можно обеспечить... но вряд ли получится в реальном исследовании! Дело в том, что в формуле (5.33) мы нигде не указали важное и на практике обычно вовсе не тривиальное условие: люди, для которых мы записали это условие, *попали в выборку исследования*. Давайте обозначим это условие через $s = 1$ (от слова *sample*); оценки из формулы (5.33) на самом деле выглядят так:

$$p\,(d = 1|x = 0, s = 1) = \frac{n_{01}}{n_{0*}}, \qquad p\,(d = 1|x = 1, s = 1) = \frac{n_{11}}{n_{1*}}. \tag{5.35}$$

А оценить нам нужно вовсе не вероятности из выборки — не очень интересный был бы результат, если бы он зависел от конкретной выборки! — а глобальные вероятности $p(d=1|x=0)$ и $p(d=1|x=1)$ во всей популяции.

Чтобы условие $s=1$ оказалось неважным, можно брать выборку для исследования из общей популяции людей, без учёта рассматриваемых переменных (в нашем случае курения и онкологических заболеваний), так, чтобы переменные потом получались сами собой.

Если у нас получится обеспечить независимость выборки s и переменных d и x, то всё действительно будет именно так просто, как в формуле (5.34):

$$p(d=1|x=1,s=1)=\frac{p(d=1,x=1,s=1)}{p(x=1,s=1)}= \\ =\frac{p(d=1,x=1)\,p(s=1)}{p(x=1)\,p(s=1)}=p(d=1|x=1).$$

Такие исследования называют *проспективными* (prospective studies). В тех случаях, когда проспективное исследование можно устроить на практике, это действительно отличный вариант, но представьте себе, как это выглядит в реальности. Вам нужно взять выборку людей, не глядя ни на курение, ни на онкологию, и надеяться, что там окажется достаточно много положительных значений этих переменных, чтобы можно было сделать выводы («надеяться» здесь не самое удачное слово, но, надеюсь, вы меня поняли правильно). А лучше, чтобы не зависеть от возрастной смертности, вообще взять репрезентативную выборку новорождённых младенцев и проследить их судьбу в течение всей жизни, подсчитав тех, кто завёл себе вредную привычку, и тех, кто в итоге умер от рака.

Как вы понимаете, сделать такое исследование в большинстве случаев практически нереально, особенно если изучаемые факторы тоже достаточно редки — курильщиков ещё хотя бы, к сожалению, относительно много, а что делать, если вы изучаете влияние какой-нибудь редко встречающейся мутации? Разумеется, на практике вы выберете достаточно большое множество людей с этим редким признаком, а потом уже доберёте к ним остаток выборки в виде так называемой *контрольной группы*: людей, не обладающих этим признаком.

Такие исследования называют *ретроспективными*, и в этих исследованиях уже нельзя сказать, что переменная попадания в выборку s и переменные d и x являются независимыми.

Давайте посмотрим, что означают отношения между элементами табл. 5.5 в ретроспективных исследованиях. Воспользуемся теоремой Байеса:

$$\frac{\pi_{01}}{\pi_{0*}}=p(d=1|x=0,s=1)= \\ =\frac{p(d=1|x=0)\,p(s=1|x=0,d=1)}{p(d=1|x=0)\,p(s=1|x=0,d=1)+p(d=0|x=0)\,p(s=1|x=0,d=0)}. \quad (5.36)$$

В формуле (5.36) уже участвуют величина $p = p(d = 1|x = 0)$, которую нам действительно хочется найти, и её дополнение $p(d = 0|x = 0) = 1 - p$. Но они выражаются через неизвестные вероятности того, как участники выборки представляют больную и здоровую части всей популяции; давайте обозначим эти неизвестные вероятности через

$$\pi_0 = p(s = 1|x = 0, d = 0), \qquad \pi_1 = p(s = 1|x = 0, d = 1).$$

Тогда формула (5.36) выглядит как

$$\frac{\pi_{01}}{\pi_{00} + \pi_{01}} = \frac{\pi_1 p}{\pi_1 p + \pi_0(1 - p)},$$

то есть $\pi_{01}(\pi_1 p + \pi_0(1 - p)) = \pi_1 p(\pi_{00} + \pi_{01})$, и после алгебраических преобразований можно выразить

$$p = \frac{\pi_0 \pi_{01}}{\pi_{00}\pi_1 + \pi_{01}\pi_0}. \tag{5.37}$$

Как и прежде, мы хотели бы обучить p, а в случае более сложной модели — ввести какую-нибудь модель вида $\sigma(\eta(\boldsymbol{x})) = p = p(d = 1|x = 0)$. Но в формуле (5.37) всё ещё участвуют неизвестные величины π_0 и π_1, которые никак не следуют из табл. 5.5! Что же делать?

Оказывается, что для случая логистического сигмоида в модели $\sigma(\eta(\boldsymbol{x}))$ здесь можно провернуть интересный трюк. Давайте подставим модель $\sigma(\eta(\boldsymbol{x})) = p$ обратно в формулу (5.36):

$$\frac{\pi_{01}}{\pi_{0*}} = \frac{\pi_1 \sigma(\eta(\boldsymbol{x}))}{\pi_1 \sigma(\eta(\boldsymbol{x})) + \pi_0(1 - \sigma(\eta(\boldsymbol{x})))} = \\ = \frac{\frac{\pi_1}{1+e^{-\eta(\boldsymbol{x})}}}{\frac{\pi_1}{1+e^{-\eta(\boldsymbol{x})}} + \frac{\pi_0 e^{-\eta(\boldsymbol{x})}}{1+e^{-\eta(\boldsymbol{x})}}} = \frac{\pi_1}{\pi_1 + \pi_0 e^{-\eta(\boldsymbol{x})}} = \frac{1}{1 + e^{-\log\frac{\pi_1}{\pi_0} - \eta(\boldsymbol{x})}}.$$

У нас получилось, что модель $\sigma(\eta^*(\boldsymbol{x}))$, обученная на данных из таблицы, то есть моделирующая логистическим сигмоидом оценки вероятностей $\frac{\pi_{01}}{\pi_{0*}}$ и $\frac{\pi_{11}}{\pi_{1*}}$, отличается от модели, которая моделирует нужные вероятности $p(d = 1|x = 0)$ и $p(d = 1|x = 1)$ таким же логистическим сигмоидом, на константу:

$$\eta^*(\boldsymbol{x}) = \eta(\boldsymbol{x}) + \log\frac{\pi_1}{\pi_0}. \tag{5.38}$$

И действительно, из результатов ретроспективного исследования мы не можем оценить сами значения π_1 и π_0: они не говорят нам ничего о том, насколько распространена была исследуемая болезнь в популяции, ведь мы выбрали больных и здоровых людей для исследования сами, в пропорциях, никак не связанных с реальной встречаемостью болезни.

Но при этом мы *можем* оценить в логистической регрессии $\eta(\boldsymbol{x})$ всё, кроме свободного члена! А это значит, что мы можем оценить, как признаки $\boldsymbol{x}$ (например, курение) влияют на целевую переменную d (например, развитие онкологических заболеваний).

Этот эффект напоминает формулу (5.14) и результат (5.29), но здесь, как видите, мы содержательно пользовались формой логистического сигмоида: для абстрактной модели вероятности $p\,(d = 1|x = 0)$ описанный выше трюк не прошёл бы. Такой эффект часто встречается на практике; главное — суметь распознать, что ваша постановка задачи представляет собой ретроспективное, а не проспективное исследование.

А нам пора переходить к следующему разделу. Вы уже видели и порождающие модели для классификации, например LDA и QDA, из которых может получиться оптимальный байесовский классификатор, и модели, из которых распределение вероятностей точек в кластерах $p\,(\boldsymbol{x})$ никак не получится, например логистическую регрессию. В заключительном разделе этой главы мы обсудим связь между этими двумя типами моделей, а также поговорим ещё об одной важной порождающей модели — о наивном байесовском классификаторе.

5.6. Порождающие модели и наивный Байес

От наивного требуется, чтобы природа одержала в нём победу над искусством... Наивное — это детскость, которая проявилась там, где её уже не ждут, и поэтому наивное в строгом смысле слова не может быть приписано настоящему детству. Но в обоих случаях — в наивном нечаянного и наивном образа мыслей — природа должна быть права, а искусство неправым.

Иоганн Кристоф Фридрих фон Шиллер. О наивной и сентиментальной поэзии (пер. И. Сац)

5.6.1. Порождающие и дискриминирующие модели

Начнём этот заключительный раздел главы о классификации с того, что снова вернёмся к разговору, который начинали ещё в разделе 1.3. Там мы выделили три типа задач искусственного интеллекта: обучение с учителем, без учителя и с подкреплением. Я упоминал, что при обучении с учителем модели обычно приближают условное распределение $p(y|\boldsymbol{x})$ для некоторой целевой переменной y и данных $\boldsymbol{x}$, а при обучении без учителя модели приближают распределение $p(\boldsymbol{x})$ некоторых объектов $\boldsymbol{x}$. Одно из ключевых различий здесь в том, что если мы знаем $p(\boldsymbol{x})$ в подходящей форме, мы можем *порождать* новые объекты из этого распределения, а распределение $p(y|\boldsymbol{x})$ такой возможности точно не предоставит.

В целом модели в машинном обучении делятся на два типа:

- *дискриминирующие модели* (discriminative models), решая задачу обучения с учителем, обучают параметры условного распределения $p(y|\boldsymbol{x})$;
- *порождающие модели* (generative models) обучают параметры совместного распределения $p(\boldsymbol{x}, y)$; при этом никто не мешает решать и ту же задачу обучения с учителем: если нужно классифицировать, можно получить условное распределение из совместного, причём для этого обычно делать и вовсе ничего не нужно, ведь они отличаются на константу $p(\boldsymbol{x})$:

$$p(y|\boldsymbol{x}) = \frac{p(\boldsymbol{x},y)}{p(\boldsymbol{x})} \propto p(\boldsymbol{x},y)\,.$$

В чём разница? Как мы уже много раз видели в этой книге, в задачах обучения с учителем на практике обычно $\boldsymbol{x}$ — это очень сложный объект (изображение, текст, звуковой файл и так далее), а y — простой: как правило, дискретная метка класса, число или вектор из нескольких предсказываемых величин. Чтобы подчеркнуть эту разницу, я даже буду писать в этом разделе в такого рода общих формулах $\boldsymbol{x}$ жирным шрифтом — как вектор, а y обычным — как число, хотя

в реальности y в классификации на несколько классов тоже является вектором. Поэтому в обучении распределения $p(\boldsymbol{x},y) = p(y|\boldsymbol{x})\,p(\boldsymbol{x})$ обычно более сложная часть — это обучение $p(\boldsymbol{x})$, а обучение $p(y|\boldsymbol{x})$ — задача более простая. Например, в классификации изображений хорошо обучить распределение $p(y|\boldsymbol{x})$ — это научиться различать по внешнему виду картины Малевича, Ван Гога и Репина, а хорошо обучить $p(\boldsymbol{x})$ — это научиться рисовать новые картины как Ван Гог; сами понимаете, что из этого является более сложной задачей.

Казалось бы, зачем тогда вообще обучать $p(\boldsymbol{x})$? Для многих практических задач этого действительно не нужно. Например:

- в задаче сегментации медицинских снимков врачу нужно найти на снимке подозрительный узелок (nodule), который может оказаться раковой опухолью; ему совершенно не нужно порождать новые медицинские снимки;
- в задаче анализа тональности (то есть классификации текстов на позитивные, негативные и нейтральные) требуется получить модель, которая сможет выдавать распределение на этих трёх классах, или на отрезке $[0,1]$, если мы захотим представлять тональность непрерывной переменной, порождать текст здесь не требуется;
- а в задаче, например, распознавания речи как раз нужно уметь порождать текст, здесь это целевая переменная, но текст нужно порождать с очень подробным и сильным условием (звуком), и сам звук модели распознавания речи порождать не умеют, он всегда считается заданным на входе.

И так далее и тому подобное; в большинстве практических случаев дискриминирующей модели будет вполне достаточно.

Однако в наше время на переднем крае искусственного интеллекта зачастую обнаруживаются именно порождающие модели. Во-первых, это красиво: существуют классы задач, где требуется именно создавать новые объекты. Так, языковые модели семейства GPT [275, 331, 450] порождают тексты уже почти совсем как люди, а в чём-то и лучше, модели на основе обращения диффузионных процессов в латентном пространстве автокодировщиков создают изображения, практически неотличимые от настоящих [169, 260], синтезаторы речи на основе авторегрессионных моделей или состязательных порождающих сетей весьма жизненно читают тексты [536, 638] и так далее.

Но мне представляется, что здесь есть и более глубокая причина. Порождающие модели часто оказываются важны для того, чтобы сделать следующий шаг и в задачах, где порождение не является конечной целью. Например, обработку естественных языков (да и другие разделы машинного обучения) в наши дни уже невозможно представить себе без архитектур, основанных на трансформерах (Transformers) [34], которые обычно служат для того, чтобы собственно «прочитать» текст и перевести его в некоторое скрытое представление, а также для того, чтобы породить новый текст с некоторым условием. И процесс обучения таких моделей оказывается именно порождающим. Так, семейство GPT — это в точности порождающие языковые модели, которые предсказывают следующее

слово в тексте на основе предыдущих [275], но и обучение моделей из семейства BERT (Bidirectional Encoder Representations) тоже выглядит весьма похожим образом: они обучаются «заполнять пробелы» в тексте [63]. Иначе говоря, чтобы обучиться хорошо решать дискриминативные задачи, в обработке естественных языков требуется сначала научиться писать тексты, и это позволит обучить хорошие скрытые представления.

Другой яркий пример приходит из обучения с подкреплением, которое в этой книге мы более подробно рассматривать не будем (но, надеюсь, ещё доберёмся до него в следующих книгах). В 2017 году появилась модель AlphaZero, которая абсолютно с нуля обучалась играть в шахматы, го и сёги лучше, чем самые лучшие на свете компьютерные программы и тем более люди [214]. Ключевым компонентом этой модели было построение роллаутов, то есть случайных вариантов развития партии, и деревьев поиска на их основе. В AlphaZero трудно говорить о порождающих моделях. Но следующим шагом, моделью, которая смогла улучшить результаты AlphaZero, стала MuZero, которая строит модель окружающей среды и обучает вероятности перехода в ней, а затем строит деревья поиска на основе порождённых самой моделью «галлюцинаций» [379]. Это уже вполне можно расценивать как порождающую модель, и это ещё одна демонстрация того же принципа: чтобы научиться по-настоящему хорошо выбирать следующий ход или следующее действие в игре (что формально является всего лишь задачей классификации, дискриминативной по своей сути), нужно сначала научиться «представлять себе» последствия этого хода, то есть выработать понимание того, как работает окружающая среда, как она будет реагировать, а это понимание уже, по сути, представляет собой задачу порождающего моделирования.

Впрочем, это лишь лирическое отступление, не имеющее статуса математического доказательства. А как на деле обучать порождающие модели? На самом высоком уровне ничего нового для нас здесь не будет; обычно порождающие модели максимизируют правдоподобие:

$$\boldsymbol{\theta}^* = \arg\max \prod_{\boldsymbol{x}\in D} p_{\text{model}}(\boldsymbol{x}, \boldsymbol{\theta}) = \arg\max \sum_{\boldsymbol{x}\in D} \log p_{\text{model}}(\boldsymbol{x}; \boldsymbol{\theta}),$$

где $\boldsymbol{\theta}$ — параметры модели, которая выдаёт распределение p_{model}. Как мы уже знаем из подраздела 3.4.3, на максимизацию правдоподобия можно посмотреть и по-другому: это то же самое, что минимизировать дивергенцию Кульбака — Лейблера между распределением данных p_{data} и распределением модели p_{model}:

$$\boldsymbol{\theta}^* = \arg\min \mathrm{KL}(p_{\text{data}} \| p_{\text{model}}) = \arg\min - \int p_{\text{data}}(\boldsymbol{x}) \ln \frac{p_{\text{model}}(\boldsymbol{x})}{p_{\text{data}}(\boldsymbol{x})} \mathrm{d}x,$$

потому что p_{data} для нас дано в ощущениях через набор данных D, и, по сути, p_{data} представляет собой дискретное равномерное распределение.

5.6.2. Вероятностные предположения наивного Байеса

Теперь давайте наполним эту общую суть конкретными примерами, а заодно и поговорим о разнице между порождающими и дискриминирующими моделями; дальнейшее изложение во многом следует [578, 579]. Начнём с, наверное, самой простой порождающей модели на свете: *наивного байесовского классификатора* (naïve Bayes classifier).

Наивный Байес – это модель, решающая задачу классификации, то есть её конечная цель состоит в том, чтобы выдавать вероятности классов $p\left(y|\boldsymbol{x}\right)$. Но делает это наивный Байес, обучая порождающую модель $p\left(\boldsymbol{x},y\right)$, в которой делает очень сильное предположение о том, что признаки, являющиеся компонентами вектора $\boldsymbol{x} = \left(x_1 \cdots x_d \right)$ являются условно независимыми при условии метки класса y:

$$p\left(y,\boldsymbol{x}\right) = p\left(y\right) \prod_{i=1}^{d} p\left(x_i|y\right). \tag{5.39}$$

Именно предположение (5.39) имеется в виду, когда в машинном обучении употребляют слово «наивный» (naïve). С точки зрения порождающей модели это предположение редко бывает реалистичным. Например, пусть мы хотим разделить входящий поток новостей по темам «Спорт», «Политика» и «Финансы», а признаками при этом служат слова, из которых новости состоят. Тогда наивное вероятностное предположение будет состоять в том, что если мы уже знаем, что новость спортивная, то дальше слова в ней будут порождаться независимо друг от друга, невзирая ни на правила русского языка, ни на стилистику текста, ни даже на зависимости между словами вроде «Криштиану» и «Роналду» (если мы не объединим их в один токен при предобработке, но это уже другая история).

Кстати, именно так — для классификации текстов по словам — и применялся наивный Байес в течение многих десятилетий. Начало его истории найти трудно, потому что по сути это всего лишь применение теоремы Байеса с дополнительным предположением о независимости, и в некотором смысле можно сказать, что наивным байесовским классификатором пользовались ещё во времена самого Байеса. В современном виде наивный Байес уже точно применяли в начале 1960-х годов как в медицинском контексте [2, 614], так и для обработки естественных языков, то есть классификации текстов [377].

Предположения слишком сильны, поэтому вряд ли можно ожидать, что такая модель сможет обучиться писать реалистичные тексты. Но, с другой стороны, при таких сильных предположениях даже порождающую модель обучить довольно просто. Если в нашей задаче K классов, $y \in \{1,\ldots,K\}$, и каждый x_i представляет собой дискретную переменную с M значениями, $x_i \in \{1,\ldots,M\}$ (в примере с классификацией текстов M является размером словаря, то есть это довольно большое число, порядка десятков тысяч), то параметры наивного Байеса представляют собой вероятности

$$\theta_{imk} = p\left(x_i = m|y = k\right), \qquad \theta_k = p(y = k).$$

Обратите внимание, что здесь мы для пущей общности рассматриваем параметры наивного Байеса с большей детализацией, чем обычно в обработке естественных языков: для наивного байесовского классификатора текстов обычно всё равно, на какой позиции стоит то или иное слово, мы просто достаём его из «мешка». Поэтому, когда мы перейдём к практическому примеру о текстах, от θ_{imk} останется только θ_{mk}, и я даже не буду там использовать обозначения с буквой θ, чтобы вас не запутывать.

Параметры θ_{imk} можно обучить как обычные вероятности игральных костей, по статистикам, собранным в обучающей выборке:

$$\hat{\theta}_{imk} = \frac{\#\left\{\text{слов } m \text{ на позиции } i \text{ в документах класса } k\right\} + 1}{\#\left\{\text{слов на позиции } i \text{ в документах класса } k\right\} + M}. \tag{5.40}$$

Обратите внимание на сглаживание в оценке (5.40) — это *байесовское сглаживание*, которое в данном случае представляет собой то же самое правило Лапласа, которое мы обсуждали в разделе 2.4. В наивном байесовском классификаторе сглаживание особенно важно из-за того самого правила Кромвеля, которое упоминалось в разделе 2.2: если не сглаживать выражения вроде (5.40), то в реальных приложениях часто будут обучаться $\hat{\theta}_{imk}$, строго равные нулю, ведь для этого будет достаточно, чтобы какое-то слово случайно не встретилось в каком-то конкретном классе. И тогда всякий новый текст с этим словом будет получать строго нулевое правдоподобие этого класса: если так получилось, что у вас в спортивных новостях никогда ранее не встречалось слово «несомненно», то текст «Несомненно, Криштиану Роналду — один из величайших футболистов в истории, наряду с Лео Месси» тоже никак не получится классифицировать как спортивный. Так что сглаживать нужно не забывать.

Естественно, в применениях наивного байесовского классификатора для обработки текстов не делают разницу между позициями слов, то есть вместо θ_{imk} в нём фигурируют просто θ_{mk}, и в итоге параметры обучаются так:

$$\hat{\theta}_{imk} = \frac{\#\left\{\text{слов } m \text{ в документах класса } k\right\} + 1}{\#\left\{\text{слов в документах класса } k\right\} + M}, \quad \hat{\theta}_n = \frac{\#\left\{\text{документов класса } k\right\}}{N}.$$

Но отмечу между прочим, что в других приложениях различать разные компоненты вектора $\boldsymbol{x}$ может быть важно — это могут быть и вовсе элементы разной природы. Кстати, для наивного Байеса неважно, какие именно компоненты у вектора $\boldsymbol{x}$, лишь бы мы умели обучать каждый x_i по отдельности; например, часть x_i могут быть непрерывными, а часть — дискретными.

В любом случае модель наивного байесовского классификатора получается несложной, и когда модель обучена, семплировать из совместного распределения $p\left(\boldsymbol{x},y\right)$ легко: сначала бросаем кубик с вероятностями θ_k, выбирая класс, а потом последовательно бросаем кубики с вероятностями θ_{imk} для каждой позиции $i = 1,\ldots,L$. Отмечу, что для наивного Байеса, порождающего документы из слов

с общими весами θ_{mk}, ещё нужно как-то выбрать длину документа, задав какое-то распределение на числах или подбрасывая каждый раз монетку, не пора ли останавливаться (что по сути означает экспоненциальное распределение). Нам сейчас длины документов не очень важны, и мы будем предполагать, что длина каждого документа просто фиксирована заранее.

5.6.3. Правдоподобие и сравнение с логистической регрессией

Теперь давайте запишем общее правдоподобие наивного байесовского классификатора, причём запишем его в форме, которая явно подчеркнёт зависимость от параметров θ:

$$\begin{aligned} p\left(D|\boldsymbol{\theta}\right) &= \prod\nolimits_{(\boldsymbol{x},y)\in D} p(y) \prod\nolimits_{k=1}^{K} p\left(x_k|y\right) = \\ &= \prod\nolimits_{(\boldsymbol{x},y)\in D} \prod\nolimits_{n=1}^{N} \left(\theta_n \prod\nolimits_{m=1}^{M} \prod\nolimits_{k=1}^{K} \theta_{kmn}^{[\![x_k=m]\!]}\right)^{[\![y=n]\!]} = \\ &= e^{\sum_{(\boldsymbol{x},y)\in D}\left(\sum_{n=1}^{N}[\![y=n]\!]\theta_n + \sum_{n=1}^{N}\sum_{m=1}^{M}\sum_{k=1}^{K}[\![y=n]\!][\![x_k=m]\!]\theta_{kmn}\right)}. \end{aligned}$$

Иными словами, наивный байесовский классификатор делает предположение о том, что логарифм правдоподобия

$$\log p\left(\boldsymbol{x},y|\boldsymbol{\theta}\right) = \sum_{n=1}^{N} [\![y=n]\!]\,\theta_n + \sum_{n=1}^{N}\sum_{m=1}^{M}\sum_{k=1}^{K} [\![y=n]\!]\,[\![x_k=m]\!]\,\theta_{kmn}$$

является линейной функцией от обучаемых параметров, коэффициенты которой представляют собой некоторые признаки точки данных, в данном случае индикаторы того, какие значения принимают $\boldsymbol{x}$ и y.

Знаем ли мы дискриминирующую модель, которая делает похожие предположения? Конечно, знаем: это логистическая регрессия! Как мы подробно обсуждали в разделе 5.2, логистическая регрессия использует функцию softmax для того, чтобы превратить линейную функцию от признаков в распределение вероятностей. Иначе говоря, логистическая регрессия моделирует

$$p\left(y=n|\boldsymbol{x};\boldsymbol{\theta}\right) = \frac{e^{\sum_{k=1}^{K}\theta_{nk}x_k}}{\sum_{n'=1}^{N} e^{\sum_{k=1}^{K}\theta_{n'k}x_k}} = \frac{1}{Z(\boldsymbol{x})}\exp\left(\sum_{k=1}^{K}\theta_{nk}x_k\right),$$

где $Z(\boldsymbol{x})$ — нормировочная константа.

Так что в данном случае тоже получается линейная функция в логарифме правдоподобия:

$$\log p\left(y=n|\boldsymbol{x};\boldsymbol{\theta}\right) = \sum_{k=1}^{K}\theta_{nk}x_k - \log Z(\boldsymbol{x}).$$

Вот какая получается картина: логистическая регрессия делает по сути то же самое предположение о взаимосвязи между весами и правдоподобием, что наивный Байес; оба метода строят логарифм правдоподобия как линейную комбинацию весов, и между ними несомненно прослеживается некоторое соответствие [39].

Ключевых различий здесь два:

- во-первых, в логистической регрессии участвуют произвольно выделенные признаки x_k, а в наивном байесовском классификаторе форма признаков выглядит жёстко фиксированной в виде различных индикаторов — здесь выигрывает логистическая регрессия, она получается более выразительной;
- во-вторых, в правдоподобии логистической регрессии участвует константа $Z(\boldsymbol{x})$, которая хоть и не влияет на оптимизацию (в ней нет θ), но мешает нам собственно подсчитать $\log p\left(y = n|\boldsymbol{x};\boldsymbol{\theta}\right)$ — это отличие в пользу наивного Байеса, собственно та ключевая причина, по которой наивный Байес является порождающей моделью, а логистическая регрессия — нет.

Впрочем, оказывается, что для этого самого простого случая оба различия несущественны. Во-первых, константу $Z(\boldsymbol{x})$ в большинстве случаев здесь подсчитать несложно — это нормировочная константа по классам, то есть её вычисление включает в себя всего лишь суммирование K чисел.

Во-вторых, признаки в наивном Байесе хоть и выглядят фиксированными, но на самом деле носят очень общий характер — мы перебираем всевозможные комбинации значений x_k, а сами x_k столь же произвольны, как и в логистической регрессии.

Поэтому можно доказать, что обе модели описывают одно и то же семейство распределений $p\left(\boldsymbol{x},y\right)$:

- если обучать наивный байесовский классификатор так, чтобы максимизировать именно условное правдоподобие, получится логистическая регрессия;
- а если интерпретировать логистическую регрессию как порождающую модель с

$$p\left(y,\boldsymbol{x}\right) = \frac{e^{\sum_{k=1}^{K}\theta_k f_k(y,\boldsymbol{x})}}{\sum_{y',\boldsymbol{x}'} e^{\sum_{k=1}^{K}\theta_k f_k(y',\boldsymbol{x}')}},$$

  то получится то же распределение, что из наивного Байеса.

В такой ситуации говорят, что наивный байесовский классификатор и логистическая регрессия образуют *порождающе-дискриминирующую пару* (generative-discriminative pair); про такие пары есть целая наука, в которую мы в этой книге не будем углубляться [578].

Замечу только одно важное общее соображение, которое иллюстрирует, откуда у дискриминирующих моделей появляется больше свободы, чем у порождающих, при одних и тех же предположениях. Предположим, что у нас есть порождающая модель с параметрами $\boldsymbol{\theta}$:

$$p_g\left(\boldsymbol{y},\boldsymbol{x}|\boldsymbol{\theta}\right) = p_g\left(\boldsymbol{y}|\boldsymbol{\theta}\right) p_g\left(\boldsymbol{x}|\boldsymbol{y},\boldsymbol{\theta}\right).$$

Её можно переписать как

$$p_g\left(\boldsymbol{y},\boldsymbol{x}|\boldsymbol{\theta}\right) = p_g\left(\boldsymbol{x}|\boldsymbol{\theta}\right) p_g\left(\boldsymbol{y}|\boldsymbol{x},\boldsymbol{\theta}\right),$$

где

$$p_g\left(\boldsymbol{x}|\boldsymbol{\theta}\right) = \sum_{\boldsymbol{y}} p_g\left(\boldsymbol{y},\boldsymbol{x}|\boldsymbol{\theta}\right), \quad p_g\left(\boldsymbol{y}|\boldsymbol{x},\boldsymbol{\theta}\right) = \frac{p_g\left(\boldsymbol{y},\boldsymbol{x}|\boldsymbol{\theta}\right)}{p_g\left(\boldsymbol{x}|\boldsymbol{\theta}\right)}.$$

А соответствующая ей дискриминирующая модель должна иметь априорное распределение $p(\boldsymbol{x})$, которое может получиться из неё, то есть

$$p_d\left(\boldsymbol{x}\right) = p_d\left(\boldsymbol{x}|\boldsymbol{\theta}'\right) = \sum_{\boldsymbol{y}} p_g\left(\boldsymbol{y},\boldsymbol{x}|\boldsymbol{\theta}'\right),$$

и условное распределение, которое тоже должно получаться как

$$p_d\left(\boldsymbol{y}|\boldsymbol{x},\boldsymbol{\theta}\right) = \frac{p_g\left(\boldsymbol{y},\boldsymbol{x}|\boldsymbol{\theta}\right)}{p_g\left(\boldsymbol{x}|\boldsymbol{\theta}\right)},$$

то есть

$$p_d\left(\boldsymbol{y},\boldsymbol{x}\right) = p_d\left(\boldsymbol{x}|\boldsymbol{\theta}'\right) p_d\left(\boldsymbol{y}|\boldsymbol{x},\boldsymbol{\theta}\right).$$

Выглядит почти точно так же, но важная разница в том, что у дискриминирующей модели теперь вовсе не обязательно $\boldsymbol{\theta} = \boldsymbol{\theta}'$, и дискриминирующая модель получается более выразительной. В результате дискриминирующая модель будет хуже моделировать $p\left(\boldsymbol{x}\right)$ (которое в задаче классификации нам, в общем-то, и не нужно) и более выразительно моделировать $p\left(\boldsymbol{y}|\boldsymbol{x}\right)$ (что, впрочем, увеличивает и риск оверфиттинга).

5.6.4. Пример классификации текстов

Давайте рассмотрим конкретный пример применения наивного байесовского классификатора к классификации текстов на естественном языке. Для этого воспользуемся датасетом новостного портала *Lenta.ru*, который есть в открытом доступе[1] и нередко используется для примеров обработки естественных языков на русском языке. Загрузим датасет в `pandas` и оставим для простоты только три класса — «Спорт», «Культура» и «Наука и техника»:

[1] https://github.com/yutkin/Lenta.Ru-News-Dataset.

```
all_data = pd.read_csv("lenta-ru-news.csv")
chosenlabels = ["Спорт", "Культура", "Наукаитехника"]
data = all_data[all_data["topic"].isin(chosenlabels)]
data = data[np.array([x is not np.nan for x in data["text"]])]
```

После этого в таблице `data` должно остаться 171 345 строк.

Далее давайте превратим метки классов в числа от 0 до 2 и разобьём данные на обучающую и тестовую выборки; я отложил в тестовую выборку 25 000 строк данных:

```
ylabels = {chosenlabels[i] : i for i in range(len(chosenlabels))}
Xtext, y = data["text"], np.array([ylabels[x] for x in data["topic"]])
test_set_size = 25000
test_set = np.sort(np.random.choice(len(y), size=test_set_size))
train_set = np.array([ i for i in range(len(y)) if not i in test_set ])
```

Измерять качество будем самым прямолинейным образом:

```
def accuracy(y_pred, y_true):
    return sum(y_pred == y_true) / len(y_true)
```

Теперь начинается интересная часть примера. Имеющиеся тексты нужно токенизировать, то есть превратить в список признаков, а затем обучить модели наивного байесовского классификатора. Для этого сначала воспользуемся стандартным классом `CountVectorizer`, который превращает каждое слово в признак и заменяет документ на вектор счётчиков соответствующих признаков. Для такой токенизации нужно задать два гиперпараметра:

- `min_df`, минимальную частоту документов (minimal document frequency), то есть нижнюю границу того, в какой доле документов должен встретиться тот или иной токен; это нужно для того, чтобы не включать одноразовые токены, которые могут только добавить оверфиттинг;
- `max_df`, максимальную частоту документов (maximal document frequency), которая, напротив, ограничивает слишком часто встречающиеся слова, которые скорее всего являются служебными и не несут тематической информации, но на случайную разницу в распределении которых классификатор тоже мог бы обучиться.

После токенизации мы обучаем наивный байесовский классификатор. Здесь кроется ещё одно интересное замечание: оказывается, даже в таком простом случае можно интерпретировать наивный байесовский классификатор по-разному! Базовое предположение сохраняется — мы будем предполагать, что признаки условно независимы при условии целевой переменной (класса), — но есть как минимум два разных подхода к тому, как эти признаки определять.

В *мультиномиальном наивном Байесе* (multinomial naive Bayes) мы строим порождающую модель так: документ — это последовательность событий, каждое из которых — это случайный выбор одного слова из словаря, того самого «мешка слов» (bag of words).

Таким образом, в мультиномиальном наивном Байесе мы для порождения каждого слова как бы «бросаем кубик» с тысячами граней, соответствующих словам из словаря. При подсчёте правдоподобие документа мы перемножаем вероятности того, что из мешка достали (на кубике выпали) те самые слова, которые в нём встретились, а наивное предположение здесь в том, что мешок у каждой темы один, и мы достаём из мешка заданной темы разные слова независимо друг от друга. Получается мультиномиальная порождающая модель, которая учитывает число повторений каждого слова, но не учитывает, каких слов *нет* в документе.

Математически говоря, для некоторого словаря $V = \{w_t\}_{t=1}^{|V|}$ документ d_i — это вектор длины $|d_i|$, состоящий из слов, каждое из которых «вынуто» из словаря с вероятностью $p\left(w_t|c_j\right)$. Правдоподобие принадлежности документа d_i классу c_j составляет

$$p(d_i \mid c_j) = p\left(|d_i|\right)|d_i|! \prod_{t=1}^{|V|} \frac{1}{N_{it}!} p\left(w_t|c_j\right)^{N_{it}},$$

где N_{it} — число вхождений w_t в d_i.

Для обучения такого классификатора нужно обучить вероятности $p\left(w_t|c_j\right)$, и если дан набор документов $D = \{d_i\}_{i=1}^{|D|}$, которые уже распределены по классам c_j (возможно, даже вероятностно распределены), дан словарь $V = \{w_t\}_{t=1}^{|V|}$ и мы знаем число вхождений N_{it}, то можно подсчитать апостериорные оценки вероятностей того, что то или иное слово встречается в том или ином классе (не забываем сглаживание по правилу Лапласа!):

$$p\left(w_t|c_j\right) = \frac{\sum_{i=1}^{|D|} N_{it} p\left(c_j|d_i\right) + 1}{\sum_{s=1}^{|V|}\sum_{i=1}^{|D|} N_{is} p\left(c_j|d_i\right) + |V|}.$$

Априорные вероятности классов можно подсчитать как

$$p\left(c_j\right) = \frac{1}{|D|}\sum\nolimits_{i=1}^{|D|} p\left(c_j|d_i\right),$$

то есть просто по числу появлений классов в обучающей выборке. Тогда классификация в мультиномиальном наивном Байесе будет происходить как

$$\begin{aligned} c &= \arg\max_j p\left(c_j\right) p\left(d_i|c_j\right) = \\ &= \arg\max_j \left(\frac{1}{|D|}\sum\nolimits_{i=1}^{|D|} p\left(c_j|d_i\right)\right) p\left(|d_i|\right)|d_i|! \prod\nolimits_{t=1}^{|V|} \frac{1}{N_{it}!} p\left(w_t|c_j\right)^{N_{it}} = \\ &= \operatorname{argmax}_j \left(\log\left(\sum\nolimits_{i=1}^{|D|} p\left(c_j|d_i\right)\right) + \sum\nolimits_{t=1}^{|V|} N_{it} \log p\left(w_t|c_j\right)\right). \end{aligned}$$

Но есть и другой подход. В *многомерном наивном Байесе* (multivariate naive Bayes) документ моделируется как вектор бинарных атрибутов, показывающих, встретилось ли в документе то или иное слово. Иначе говоря, документ — это теперь не последовательность результатов бросков гигантского кубика, а последовательность результатов бросков монеток, по одной монетке для каждого слова в словаре. Когда мы подсчитываем правдоподобие документа, мы перемножаем вероятности того, что встретилось каждое слово из документа, и вероятности того, что *не* встретилось каждое (словарное) слово, которого там нет.

Получается модель многомерных испытаний Бернулли. Наивное предположение здесь состоит в том, что события «встретилось ли слово» предполагаются независимыми при условии заданной темы. Если $V = \{w_t\}_{t=1}^{|V|}$ — это словарь, то документ d_i — это вектор длины $|V|$, состоящий из битов B_{it}; $B_{it} = 1$ тогда и только тогда, когда слово w_t встречается в документе d_i.

Правдоподобие принадлежности d_i классу c_j в таком случае выглядит как

$$p\left(d_i|c_j\right) = \prod_{t=1}^{|V|}\left(B_{it}p\left(w_t|c_j\right) + \left(1 - B_{it}\right)\left(1 - p\left(w_t|c_j\right)\right)\right).$$

Для обучения такого классификатора нужно обучить вероятности $p\left(w_t|c_j\right)$, то есть обучить по одной монетке для каждого слова из словаря и каждой темы.

Если дан набор документов $D = \{d_i\}_{i=1}^{|D|}$, которые уже распределены по классам c_j (как и выше, здесь можно использовать и «мягкие», вероятностные принадлежности документов классам), дан словарь $V = \{w_t\}_{t=1}^{|V|}$, и мы знаем биты B_{it} (знаем документы), мы можем подсчитать оптимальные оценки вероятностей того, что то или иное слово встречается в том или ином классе (опять же не забывая лапласовское сглаживание):

$$p\left(w_t|c_j\right) = \frac{1 + \sum_{i=1}^{|D|} B_{it}p\left(c_j|d_i\right)}{2 + \sum_{i=1}^{|D|} p\left(c_j|d_i\right)}.$$

Априорные вероятности классов считаются так же, как в мультиномиальном случае, $p\left(c_j\right) = \frac{1}{|D|}\sum_{i=1}^{|D|} p\left(c_j|d_i\right)$, и классификация в многомерном наивном Байесе происходит как

$$\begin{aligned} c &= \arg\max_j p\left(c_j\right) p\left(d_i|c_j\right) = \\ &= \arg\max_j \left[\left(\frac{1}{|D|}\sum_{i=1}^{|D|} p\left(c_j|d_i\right)\right)\prod_{t=1}^{|V|}\left(B_{it}p\left(w_t|c_j\right) + (1 - B_{it})(1 - p\left(w_t|c_j\right))\right)\right] = \\ &= \arg\max_j \left[\log\sum_{i=1}^{|D|} p\left(c_j|d_i\right) + \sum_{t=1}^{|V|}\log\left(B_{it}p\left(w_t|c_j\right) + (1 - B_{it})(1 - p\left(w_t|c_j\right))\right)\right]. \end{aligned}$$

Замечу, что в многомерном наивном Байесе теряется информация о том, сколько именно раз то или иное слово появилось в документе: вместо вектора счётчиков теперь документ — это вектор бинарных событий. Зато многомерный наивный Байес в явном виде учитывает события о том, что тот или иной признак *не* встретился.

Теперь оказывается, что мы можем получить массу разных результатов на одном и том же датасете, варьируя для каждой из этих двух моделей гиперпараметры токенизации `min_df` и `max_df`.

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB, BernoulliNB

range_maxdf = np.arange(0.05, 0.25, 0.025)
range_mindf = np.arange(0., 0.02, 0.0025)

results = {}
print("min_df\tmax_df\tfeatures\tBernoulliNB\tMultinomialNB")
for max_df in range_maxdf:
    for min_df in range_mindf:
        vectorizer = CountVectorizer(min_df=min_df, max_df=max_df)
        X_train = vectorizer.fit_transform(np.array(Xtext)[train_set])
        X_test = vectorizer.transform(np.array(Xtext)[test_set])
        y_train, y_test = y[train_set], y[test_set]
        model_nbm = MultinomialNB(fit_prior=True)
        model_nbb = BernoulliNB(fit_prior=True)
        model_nbm.fit(X_train, y_train)
        model_nbb.fit(X_train, y_train)
        acc_nbb, acc_nbm = accuracy(model_nbb.predict(X_test), y_test), \
          accuracy(model_nbm.predict(X_test), y_test)
        print("%.4f\t%.4f\t%d\t\t%.3f\t\t%.3f" % \
         (min_df, max_df, len(vectorizer.vocabulary_), acc_nbb, acc_nbm))
        results[(min_df, max_df)] = \
         (acc_nbb, acc_nbm, len(vectorizer.vocabulary_))
```

Этот код будет заполнять словарь `results` результатами точности на тестовом наборе, полученными для мультиномиального и многомерного наивных байесовских классификаторов для разных значений `min_df` и `max_df`. Должно получиться что-то в таком духе (у вас результаты не совпадут, потому что мы выбирали тестовый набор случайно, но общий смысл должен сохраниться):

```
min_df  max_df  features    BernoulliNB MultinomialNB
0.0000  0.0500  501521      0.980       0.983
0.0025  0.0500  6917        0.974       0.977
0.0050  0.0500  3447        0.971       0.973
0.0075  0.0500  2149        0.969       0.971
0.0100  0.0500  1517        0.965       0.966
...
```

Таблица 5.6. Примеры слов с разными значениями веса idf в датасете Lenta.ru

Слово	idf	Слово	idf	Слово	idf	Слово	idf
Самые популярные слова							
на	1,074	из	1,541	этом	1,845	по	1,331
за	1,704	как	1,855	что	1,411	года	1,720
об	1,955	не	1,498	сообщает	1,721	также	1,964
Слова из середины списка							
задач	6,050	льда	6,051	снят	6,052	поддерживает	6,050
назначения	6,051	условий	6,052	татьяна	6,050	позволил	6,051
обнаружен	6,053	джексон	6,051	живых	6,052	сведения	6,053
Примеры редких слов							
1829	10,218	трехмесячный	10,279	штранцля	10,343	уимблдону	10,412
ботсваны	10,218	бактериофаги	10,343	8799	10,412	авианесущем	10,486
гризманна	10,218	колесница	10,343	копейкин	10,412	гастроэнтерита	10,486

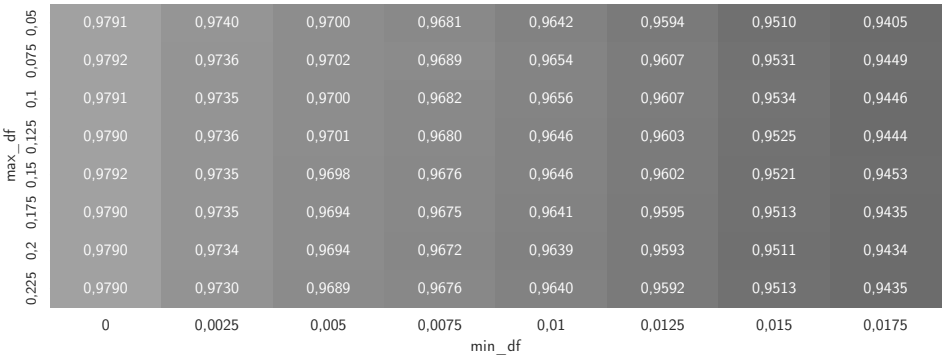
На рис. 5.22, *а* наглядно показаны все результаты для многомерного наивного Байеса, а на рис. 5.22, *б* — для мультиномиального. Отмечу, что, хотя в данном случае, на датасете *Lenta.ru*, классификаторы явно «хотят» оставить себе побольше признаков и дают лучшие результаты для малых `min_df` и больших `max_df`, эти зависимости всё же не вполне монотонные, а в общем случае при слишком больших словарях часто будет возникать оверфиттинг.

Но и это ещё не всё — давайте для полноты картины сделаем ещё одно интересное преобразование. В наивном байесовском классификаторе используются счётчики числа вхождений тех или иных слов в документ — в информационном поиске частоту вхождения слова в документ традиционно называют tf (term frequency). Но слова бывают разные, и, даже не глядя на темы, априорно, важность того или иного слова можно условно оценить его редкостью: понятно, что слова «сказать», «первый» или «данные» будут в новостной выборке высокочастотными, но при этом совершенно бесполезными, хотя, возможно, и не перейдут порога отсечения `max_df`.

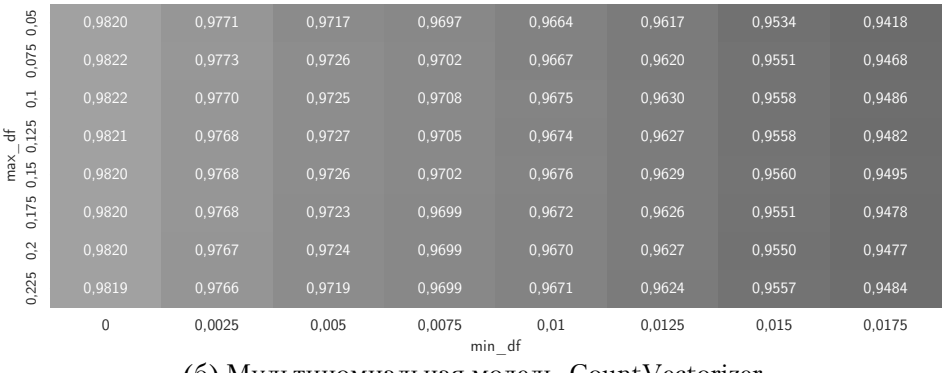
Поэтому было бы хорошо учитывать в наших счётчиках не только частоту встречаемости слов в конкретном документе, но и то, насколько часто они встречаются во всех документах вообще. Для этого обычно дополняют частоты tf весами idf (inverse document frequency), которые имеют обратную зависимость от того, сколько документов содержат то или иное слово:

$$\text{idf}(w, D) = \log \frac{|D|}{\#\{d \in D \mid w \in d\}}.$$

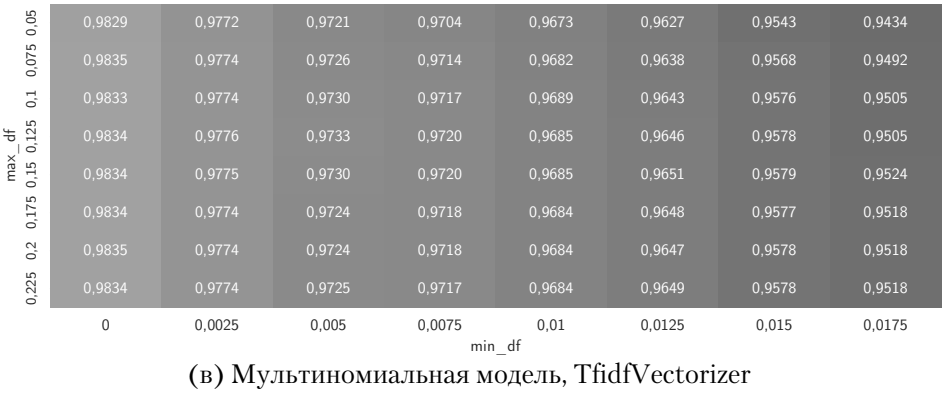
Чем чаще слово встречается по всей выборке текстов, тем меньше становится вес idf. Таблица 5.6 показывает несколько примеров числа документов df, где



(а) Многомерная модель, CountVectorizer



(б) Мультиномиальная модель, CountVectorizer



(в) Мультиномиальная модель, TfidfVectorizer

Рис. 5.22. Точность различных вариантов наивного байесовского классификатора на тестовом наборе для категоризации текстов в датасете новостей Lenta.ru

встречается слово, и соответствующих весов idf на примере датасета Lenta.ru. Как видите, самые популярные слова, очевидно, не нужно учитывать вообще (для

этого и используют `min_df`), а среди самых редких слов много странных, но встречаются и весьма тематические (я выбрал несколько примеров не по порядку, потому что значения idf, соответствующие малым числам документов, начинают повторяться и список, например, слов с idf = 10,343 занимает много сотен позиций).

Конечно, слова всё равно останутся неравноценными: слово «Криштиану» важнее для тематической классификации, чем слово «несомненно», хотя по частоте в околоспортивной выборке соответствующих лет Криштиану мог бы и победить. Далеко не все слова из середины списка в табл. 5.6 исполнены глубокого тематического смысла. Однако это уже будет задача для собственно классификатора (например, наивного Байеса), и переходом к весам tf-idf мы зачастую можем существенно упростить ему эту задачу.

Веса tf-idf появились в информационном поиске очень давно: перевзвешивание через idf встречается ещё в начале 1970-х годов в работах одного из пионеров информационного поиска и обработки естественных языков Карен Спарк Джонс [550] и сразу же прочно вошло в практику [375, 484]. Например, оно является важной частью одной из самых популярных функций ранжирования, BM25 (от слов best matching), которая упорядочивает набор документов по соответствию заданному текстовому запросу. Хотя функция BM25 появилась и вошла в практику ещё в 1980–1990-х годах на основе вероятностного подхода к понятию релевантности, который разрабатывался с 1970-х годов [497–499], этот метод до сих пор частенько появляется даже в самых современных работах по информационному поиску как неожиданно сильный метод (baseline) [518]. О методах ранжирования мы, правда, в этой книге говорить не будем.

Да и в других задачах обработки естественных языков перевзвешивание через веса idf всегда было очень полезным и потому популярным методом предобработки текстов: сделать это преобразование обычно ничего не стоит, а результаты улучшаются, порой заметно. Так, например, опубликованный в 2015 году обзор более чем 200 рекомендательных систем для научной литературы показал, что более 80 % тех систем, что вообще использовали тексты статей, использовали и веса tf-idf [491]. Были и попытки проанализировать веса idf теоретически [496], но в них мы углубляться не будем. В последнее время популярность таких методов предобработки падает, потому что всё больший упор делается на системы поиска, строящие представление запросов и документов в латентном пространстве при помощи глубоких нейросетевых моделей, обычно основанных на тех же трансформерах. Но наивному Байесу веса tf-idf точно могут помочь, так что давайте их применим и в нашем случае; в коде ничего не изменится, только вместо `CountVectorizer` нужно будет подставить другой класс `TfidfVectorizer`.

На рис. 5.22, *в* показан результат в мультиномиальном случае — обратите внимание, что точность стала выше, то есть веса tf-idf действительно помогли в данном случае. А вот многомерный случай с весами tf-idf здесь не показан, потому

что он в точности совпадает с рис. 5.22, *а*: многомерный наивный Байес не обращает внимания на число вхождений и оставляет только сам факт наличия или отсутствия признака, так что перевзвешивание числа вхождений ни на что не повлияет. Впрочем, всё равно рекомендуем читателю получить соответствующие результаты и убедиться в этом совпадении на практике.

И последнее замечание о наивном байесовском классификаторе. Дело в том, что наивный Байес на первый взгляд действительно очень наивный: настолько сильные предположения о независимости ну никак не могут выполняться. В случае классификации текстов мы предполагаем, что слова в тексте на естественном языке появляются независимо друг от друга, если мы всего лишь знаем тему текста, — да, это, пожалуй, действительно странно звучит. Наивный Байес из-за этого даже называли иногда *Байесом для идиотов* (Idiot's Bayes) [238].

В частности, наивность этого предположения означает, что наивный Байес очень плох для оценок вероятностей; в терминах подраздела 5.5.2 можно сказать, что наивный Байес практически никогда не будет хорошо откалиброван. Тот самый Криштиану Роналду, если специально искусственно не объединить его в один токен, будет в нашей модели представлен сразу двумя словами, и каждое из них наверняка будет указывать на тему «Спорт» с высокой вероятностью, а значит, «Криштиану Роналду» укажет на тему «Спорт» вдвое сильнее, причём вдвое на логарифмической шкале. Но на самом деле, конечно, «Криштиану Роналду» — это, по сути, один токен, и, хотя он действительно указывает на тему «Спорт», он не должен давать сразу два множителя по сравнению с упоминанием просто «Криштиану», ведь эти слова (по крайней мере в русском языке) очень тесно скоррелированы.

Соответственно, оценки вероятностей, которые даёт наивный байесовский классификатор, будут крайне экстремальными по сравнению с «реалистичными»: наивный Байес почти всегда будет очень сильно уверен в одном из ответов и практически никогда не выдаст распределения классов, похожего на равномерное. Этот эффект хорошо известен, и использовать наивный Байес для оценки вероятностей не надо [57, 408], да и переносить на задачи регрессии не рекомендуется [424].

Но ведь этот неприятный эффект может и не иметь прямого влияния на собственно результат классификации? Под результатом я здесь имею в виду тот класс, который будет побеждать в наивном байесовском классификаторе, то есть ошибку классификации *zero-one*, которая при правильном ответе равна нулю, а при неправильном — единице. Вполне возможно, что зависимости между признаками, которые мешают получить разумные оценки вероятностей, будут «взаимно сокращаться» и не мешать хорошим результатам классификации; например, грамматические зависимости между словами в естественном языке будут общими для разных тем и, по идее, могут и не помешать тематической классификации наивным методом. И действительно, есть целый ряд интересных результатов, которые показывают, что наивный Байес совсем не такой наивный, как

может показаться, и относительно такой ошибки он может оказаться даже оптимальным в весьма широком классе задач и начальных условий [147, 238, 326, 660]; мы в эти результаты углубляться не будем.

А нам на этом пора завершать главу, посвящённую основам классификации. Мы рассмотрели сразу несколько интересных моделей, от чисто геометрического линейного дискриминанта Фишера до имеющей безупречную вероятностную подоплёку логистической регрессии, которая заставила нас изучить несколько весьма изящных аппроксимаций, а также наивного Байеса, который привёл нас к более общему разговору о связи между порождающими и дискриминирующими моделями. В следующей, последней главе этой книги вас ждут ещё несколько общих сюжетов, которые пронизывают всё машинное обучение.

Глава 6
Несколько важных сюжетов

TL;DR

В разделе 6.1 мы начинаем с того, что обсуждаем метод ближайших соседей, а затем выясняем, почему это не всё, что нам может понадобиться в машинном обучении. Это приводит нас к анализу эффекта проклятия размерности в разных его формах.

Раздел 6.2 посвящён одной из важнейших тем машинного обучения: статистической теории принятия решений. Мы обсудим функцию регрессии, оценим ожидаемую ошибку предсказания и дойдём до разложения на смещение, дисперсию и шум.

В разделе 6.3 мы обсуждаем так называемые ядерные методы: начнём с эквивалентного ядра для линейной регрессии, а затем обобщим полученные результаты до описания ядерных методов в целом.

Раздел 6.4 — последний в этой книге развёрнутый пример. Мы применим методы байесовского вывода к обучению параметров нормального распределения; окажется, что в этом кроется много интересных результатов, вовсе не сводящихся к тому выводу, который мы делали в главе 4.

В разделе 6.5 обсуждаются два важных сюжета, связанных с оценкой знаменателя теоремы Байеса $p(D)$. Мы начнём с эмпирического Байеса, полезного для подбора гиперпараметров, а затем перейдём к байесовскому выбору моделей, в том числе информационным критериям BIC, TIC и AIC.

Раздел 6.6 вводит и рассматривает понятие экспоненциального семейства распределений. Это очень правильная степень обобщения: вы увидите, что в экспоненциальное семейство вкладываются почти все интересные нам распределения, а с другой стороны, для него можно провести байесовский вывод в общем виде.

И завершает книгу раздел 6.7, где аналогичное обобщение применяется к линейной и логистической регрессии. Оказывается, это лишь частные случаи обобщённых линейных моделей, и для всего семейства таких моделей тоже можно получить много интересных результатов.

6.1. Ближайшие соседи и проклятие размерности

Конечно, — продолжал Манилов, — другое дело, если бы соседство было хорошее, если бы, например, такой человек, с которым бы в некотором роде можно было поговорить о любезности, о хорошем обращении, следить какую-нибудь этакую науку, чтобы этак расшевелило душу, дало бы, так сказать, паренье этакое...

Николай Гоголь. Мёртвые души

6.1.1. Метод ближайших соседей

В этой главе мы обсудим несколько разных сюжетов, которые, надеюсь, дадут вам более глубокое понимание того, что вообще происходит в байесовском выводе. И начнём мы с сюжета, который отчасти объясняет, зачем всё это наше машинное обучение вообще нужно и нельзя ли обойтись чем-то попроще.

Давайте рассмотрим метод, о котором вы, скорее всего, слышали и раньше: *метод ближайших соседей* (nearest neighbors). Его начали применять ещё в начале 1950-х годов; первыми, по всей видимости, были статистики из университета Беркли Эвелин Фокс и Джозеф Ходжес, а в середине 1960-х уже стэнфордские учёные Томас Кавер и Питер Харт изучили его статистические свойства, и с тех пор метод прочно вошёл в арсенал специалистов по анализу данных [122].

Суть метода очень проста: предположим, что мы бы хотели классифицировать точки на M классов на основе набора данных $D = \{(\boldsymbol{x}_1, t_1), \ldots, (\boldsymbol{x}_N, t_N)\}$, где $\boldsymbol{x}_n \in \mathcal{X}$, $t_n \in \{1, \ldots, M\}$. Давайте не будем вводить никаких весов и никаких предположений (это, конечно, иллюзия, предположения здесь есть, но об этом позже), а просто сохраним данные и для точки $\boldsymbol{x}$, которую хотим классифицировать, найдём некоторое число k её *ближайших соседей*, то есть точек $\boldsymbol{x}_n$ из датасета D, которые находятся ближе всего к новой точке $\boldsymbol{x}$. Здесь возникает первое (совсем мягкое) предположение: чтобы найти ближайших соседей, нужно, чтобы на точках $\boldsymbol{x}$ было задано расстояние, то есть чтобы $\mathcal{X}$ было некоторым метрическим пространством (если вы не знаете, что это такое, не пугайтесь, эта теория нам здесь не понадобится). Кроме того, здесь возникает первая развилка, гиперпараметр метода ближайших соседей: нужно выбрать число k.

Если и расстояние $d(\boldsymbol{x}_n, \boldsymbol{x})$, и число k заданы, то у нас появляется хорошо определённое множество ближайших соседей

$$\mathrm{NN}_k(\boldsymbol{x}, D) = \{\boldsymbol{x}_n \in D \mid d(\boldsymbol{x}_n, \boldsymbol{x}) \text{ попадает в } k \text{ наименьших значений расстояний } d(\boldsymbol{x}_l, \boldsymbol{x}), l = 1, \ldots, N\};$$

простите, что записал определение словами, но суть ясна, а формально вводить какие-то упорядоченные списки значений расстояний нам сейчас ни к чему.

Затем возникает вторая развилка: на основе этого множества ближайших соседей $\mathrm{NN}_k(\boldsymbol{x}, D)$ нужно сделать предсказание $\hat{t}$ о том, что происходит в точке $\boldsymbol{x}$. В случае классификации это, скорее всего, будет голосование между классами ближайших соседей:

$$\hat{t} = \arg\max_{m=1,\ldots,M} \#\{\boldsymbol{x}_n \in \mathrm{NN}_k(\boldsymbol{x}, D) \mid t_n = m\},$$

где t_n — это номер класса, в который попадает пример t_n в имеющемся датасете; для задач классификации обычно разумно взять нечётное значение k, чтобы избежать равного разделения голосов.

Это голосование можно организовать и не с равными голосами всех участников, а с некоторыми весами w_n:

$$\hat{t} = \arg\max_{m=1,\ldots,M} \sum_{\boldsymbol{x}_n \in \mathrm{NN}_k(\boldsymbol{x}, D):\ t_n = m} w_n.$$

Веса мы определяем сами, и, скорее всего, мы захотим определить их так, чтобы они были выше у точек, лежащих ближе к точке $\boldsymbol{x}$. Например, можно перевзвесить голоса точек величинами, обратными расстоянию до $\boldsymbol{x}$:

$$\hat{t} = \arg\max_{m=1,\ldots,M} \sum_{\boldsymbol{x}_n \in \mathrm{NN}_k(\boldsymbol{x}, D):\ t_n = m} \frac{1}{d(\boldsymbol{x}_n, \boldsymbol{x})}.$$

Методом ближайших соседей можно пытаться решать и задачи регрессии, когда элементами датасета являются пары $(\boldsymbol{x}_n, y_n)$ для $y_n \in \mathbb{R}$. Здесь вместо голосования появится усреднение по ближайшим соседям:

$$\hat{y} = \frac{1}{k} \sum_{\boldsymbol{x}_n \in \mathrm{NN}_k(\boldsymbol{x}, D)} y_n.$$

Его тоже можно проводить с некоторыми весами w_n, но теперь, чтобы результат имел смысл, нужно следить за тем, чтобы получившаяся комбинация была выпуклой:

$$\hat{y} = \sum_{\boldsymbol{x}_n \in \mathrm{NN}_k(\boldsymbol{x}, D)} w_n y_n, \quad \text{где} \sum_{\boldsymbol{x}_n \in \mathrm{NN}_k(\boldsymbol{x}, D)} w_n = 1.$$

А в самом общем виде метод ближайших соседей является частным случаем метода, который предсказывает значение в точке $\boldsymbol{x}$ на основе взвешенной суммы значений во *всех* точках, $\sum_{n=1}^{N} w_n y_n$ для некоторых весов w_n, для которых

$\sum_{n=1}^{N} w_n = 1$; чтобы эта формула работала для классификации, придётся переопределить t_n как one-hot представление. Наш базовый метод ближайших соседей в этих обозначениях определяет $w_n = \frac{1}{k}$ для k ближайших соседей и $w_n = 0$ для всех остальных точек.

Кроме весов и числа k, не забывайте, что метод ближайших соседей существенно зависит ещё и от метрики, которую мы вводим как расстояние между точками. Здесь много вариантов, которые пригождаются в разных ситуациях:

- в обычном евклидовом пространстве, когда $\boldsymbol{x}_n \in \mathbb{R}^d$, чаще всего рассматривают стандартное евклидово расстояние (Euclidean distance), или L_2-расстояние, получающееся по теореме Пифагора:

$$d(\boldsymbol{x}_1,\boldsymbol{x}_2) = \sqrt{(\boldsymbol{x}_1 - \boldsymbol{x}_2)^\top (\boldsymbol{x}_1 - \boldsymbol{x}_2)} = \sqrt{\sum\nolimits_{i=1}^{d} (x_{1i} - x_{2i})^2};$$

- его можно обобщить до L_p-*расстояния*, которое ещё называют *расстоянием Минковского*[1] (Minkowski distance)

$$d_p(\boldsymbol{x}_1,\boldsymbol{x}_2) = \left(\sum\nolimits_{i=1}^{d} |x_{1i} - x_{2i}|^p\right)^{1/p};$$

  мы уже кратко обсуждали свойства таких метрик с геометрической точки зрения в подразделе 4.2.4;
- его другой частный случай, L_1-расстояние

$$d_1(\boldsymbol{x}_1,\boldsymbol{x}_2) = \sum\nolimits_{i=1}^{d} |x_{1i} - x_{2i}|,$$

  имеет своё собственное название: *расстояние городских кварталов* (Manhattan distance, taxicab distance); это интересная метрика со своими собственными применениями, из которых нам, конечно, более всего интересно уже обсуждавшееся в подразделе 4.2.4 приложение к регуляризации, где L_1-расстояние добавляет разреженности в самых разных контекстах [149];

[1] *Герман Минковский* (Hermann Minkowski, 1864–1909) — немецкий математик, происходивший из крайне интеллектуальной еврейской семьи: его прадед Борух бен Яаков Шкловер служил даяном (религиозным судьёй), писал труды по астрономии, анатомии и тригонометрии и перевёл Евклида на иврит, его отец Ицхок бен Аарон был Карлинским ребе и комментатором Талмуда, а его старший брат Оскар был врачом, описавшим болезнь Минковского — Шоффара. Сам Герман Минковский наиболее знаменит своими работами по геометрии: он разработал основы геометрической теории чисел и развил теорию квадратичных форм, а главное, разработал раздел геометрии, который сам назвал «геометродинамикой», — четырёхмерную модель кинематики теории относительности. Вышедшая в 1909 году работа Минковского *Raum und Zeit* («Пространство и время») [397] оказала большое влияние на Эйнштейна и заложила основы теории относительности. Изучал он и L_p-пространства с L_p-метриками, где доказал неравенство Минковского [398].

- если нужно подсчитать похожесть между двумя векторами в линейном пространстве с точки зрения их направления, имеет смысл использовать *косинусную похожесть* (cosine similarity)

$$\cos(\boldsymbol{x}_1,\boldsymbol{x}_2) = \frac{\boldsymbol{x}_1^\top \boldsymbol{x}_2}{\|\boldsymbol{x}_1\| \cdot \|\boldsymbol{x}_2\|};$$

  косинус изменяется от -1 до 1 и указывает на похожесть, а нам нужно наоборот, поэтому в качестве расстояния используют *косинусное расстояние* (cosine distance)

$$d_{\cos}(\boldsymbol{x}_1,\boldsymbol{x}_2) = 1 - \cos(\boldsymbol{x}_1,\boldsymbol{x}_2);$$

  эти метрики часто используются в информационном поиске и классической обработке естественных языков [112, 484, 509, 586];
- в дискретном пространстве, где векторы состоят из битов или других категориальных значений, то есть это скорее *строки* символов, чем векторы чисел, логично использовать *расстояние Хэмминга* (Hamming distance), которое подсчитывает число компонентов, где векторы различаются:

$$d_{\text{Ham}}(\boldsymbol{x}_1,\boldsymbol{x}_2) = \sum\nolimits_{i=1}^{d} [\![x_{1i} \neq x_{2i}]\!];$$

  вы наверняка слышали о расстоянии Хэмминга в контексте теории кодирования и криптографии, где Ричард Хэмминг его и ввёл в 1950 году [236], но оно оказывается очень полезным в любых алгоритмах на строках, в том числе и при поиске ближайших соседей;
- а если речь идёт о подмножествах некоторого объемлющего множества, а не векторах, то можно использовать так называемый *коэффициент Жаккара* (Jaccard index, Jaccard similarity)

$$J(X_1,X_2) = \frac{|A \cap B|}{|A \cup B|} = \frac{|A \cap B|}{|A| + |B| - |A \cap B|};$$

  в машинном обучении эту метрику применяют для сравнения площадей и объёмов, например при оценке похожести прямоугольников в методах распознавания объектов [282, 319, 339, 484, 586].

Метод ближайших соседей — это первый (и последний) в нашей книге пример *непараметрического* метода. Непараметрическая статистика использует методы, которые не делают предположений о том, что интересующее нас распределение относится к тому или иному параметрическому семейству, и не пытается оценивать конкретные параметры вроде среднего и дисперсии [567]. В классической статистике сюда можно отнести методы, которые проверяют принадлежность семейству распределений, но не ищут их параметры, например критерий согласия Колмогорова, или методы, которые и вовсе не пытаются работать

с конкретными семействами распределений, например критерий однородности Смирнова, который проверяет, принадлежат ли две независимые выборки одному и тому же (неизвестно какому!) закону распределения. А в машинном обучении непараметрические методы обычно работают со всем датасетом, не пытаясь сжимать его до фиксированного числа параметров; например, распределение вероятностей можно описать через гистограмму выборки точек, то есть через эмпирическое распределение, и это будет непараметрическим описанием. Мы не будем углубляться в непараметрическую статистику, а для интересующегося читателя я оставлю несколько ссылок [119, 130, 635].

Хорошо ли работает метод ближайших соседей? Он, конечно же, есть в универсальной библиотеке `scikit-learn`:

```
from sklearn import neighbors
knn = neighbors.KNeighborsClassifier( \
  n_neighbors=k, algorithm='ball_tree').fit(X, y)
```

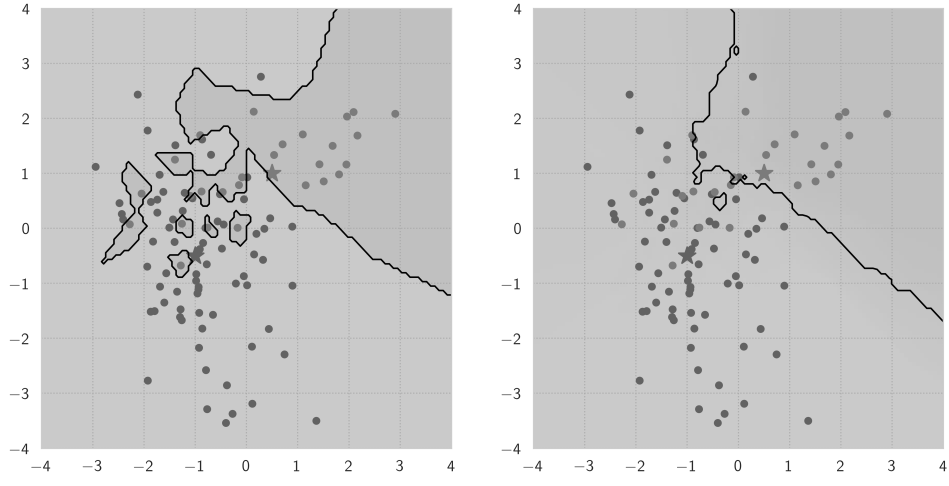
Здесь немного таинственно выглядит параметр `algorithm='ball_tree'`, но об этом чуть позже, а пока давайте посмотрим на результаты.

Если попробовать запустить метод ближайших соседей на данных, которые мы рассматривали в главе 5, мы получим прекрасные результаты, показанные на рис. 6.1. Конечно, в этом методе всегда нужно подбирать гиперпараметр k, но это оказывается не так уж сложно: разумеется, совсем маленькие значения k будут вести к оверфиттингу, как на рис. 6.1, *а* (при $k = 1$ метод буквально рисует диаграмму Вороного между точками), но при «разумных» значениях k будет, скорее всего, получаться отличная разделяющая поверхность, как на рис. 6.1, *б*.

Более того, метод ближайших соседей по своей природе локален, то есть он будет «следовать за данными» и не будет делать строгих предположений о том, как датасет глобально устроен. Это позволяет надеяться, что и более сложные случаи он сможет обработать. На рис. 6.2 показано разделение двух классов, между которыми уже не получится провести хорошую линейную разделяющую поверхность; каждый из двух классов был порождён как смесь двух гауссианов, центры которых обозначены звёздочками. Выясняется, что метод ближайших соседей и здесь справляется отлично! Для $k = 1$ получается очевидный оверфиттинг (см. рис. 6.2, *а*), а для $k = 99$, то есть для значений k, сравнимых с N (здесь у нас два класса по 200 точек в каждом), разделяющая поверхность, напротив, получается слишком «зарегуляризованной» (см. рис. 6.2, *г*). Но есть много разных значений k, для которых всё получается: и $k = 5$ (см. рис. 6.2, *б*), и $k = 11$ (см. рис. 6.2, *в*) дают превосходные разделяющие поверхности.

А что насчёт регрессии? Да тоже выглядит неплохо: на рис. 6.3 показано, как выглядят предсказания метода ближайших соседей для той же задачи регрессии, которую мы много раз решали в главе 4:

```
knnm = neighbors.KNeighborsRegressor( \
  n_neighbors=k, algorithm='ball_tree').fit(xd.reshape(-1, 1), data)
```



(а) $k = 1$ (б) $k = 11$

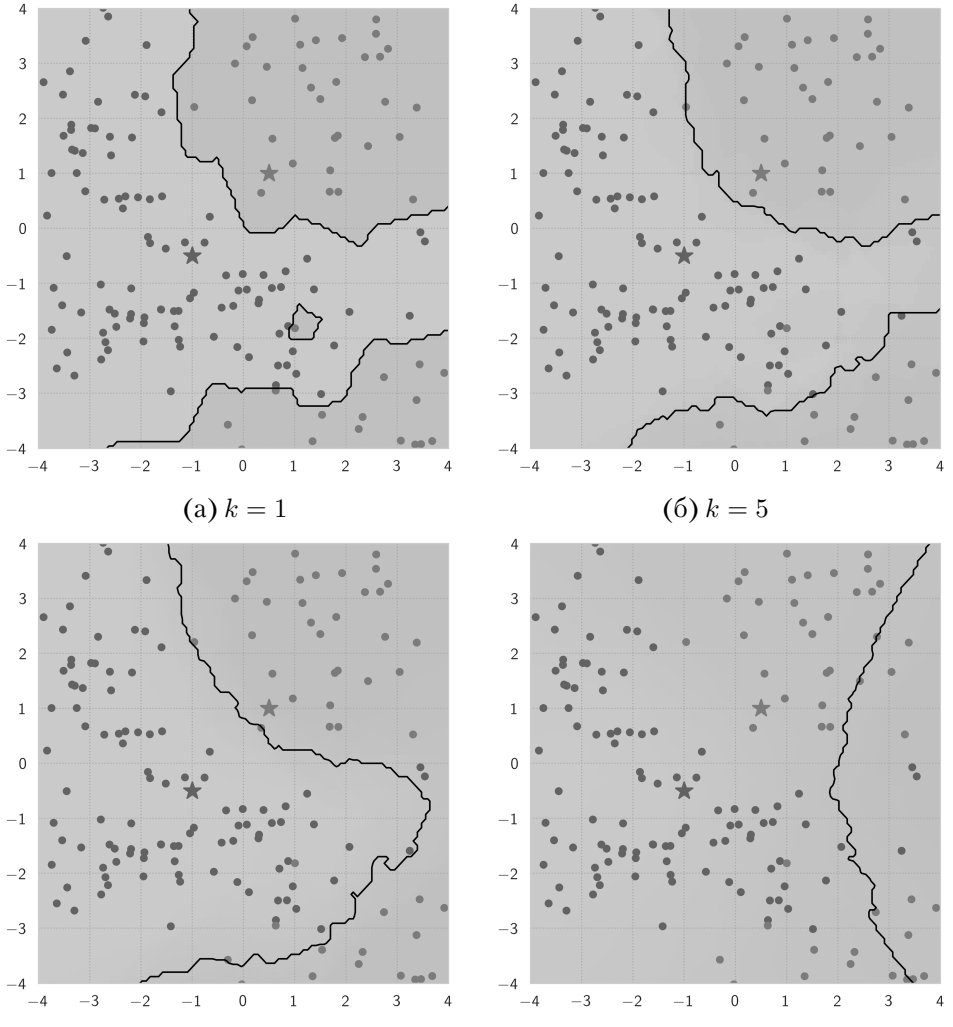
Рис. 6.1. Метод ближайших соседей для классификации на данных из главы 5

Для базовой задачи, из 11 точек, даже $k = 1$ вряд ли будет давать серьёзный оверфиттинг (см. рис. 6.3, *а*); если увеличивать число точек, то опять будет нужно подобрать гиперпараметр k, но предсказания для подходящих k будут всё равно выглядеть хорошо (см. рис. 6.3, *б*). Экстраполяцией трендов метод ближайших соседей, естественно, заниматься не умеет, и на бесконечность его предсказания и влево, и вправо продолжаются горизонтально; но вы уже видели точно такую же картину в подразделе 4.1.4, когда мы обсуждали гауссовские признаки, и так будет с любыми локальными методами, так что ничего страшного в этом нет.

6.1.2. В чём проблема с ближайшими соседями? Проклятие размерности

На этом месте у вас может возникнуть резонный вопрос: а зачем нам вообще нужно что-то другое? Зачем нужна вся та наука, которую мы старательно развивали в предыдущих главах? Вероятностные методы делают массу предположений, которые могут на практике оказаться неверными, а здесь у нас появляется непараметрический метод, который мало того что не требует никаких предположений, так ещё и работает прекрасно! Может быть, нам можно продолжать пользоваться методом ближайших соседей и забыть всё остальное машинное обучение как страшный сон — или всё-таки здесь есть какой-то подвох?

Подвох, конечно, найдётся. Начнём с не самой принципиальной проблемы: вычислительной. Во-первых, непараметрические методы часто требуют запомнить весь датасет, потому что нам могут потребоваться из него любые точки; для метода ближайших соседей это точно верно, ведь любые точки могут оказаться

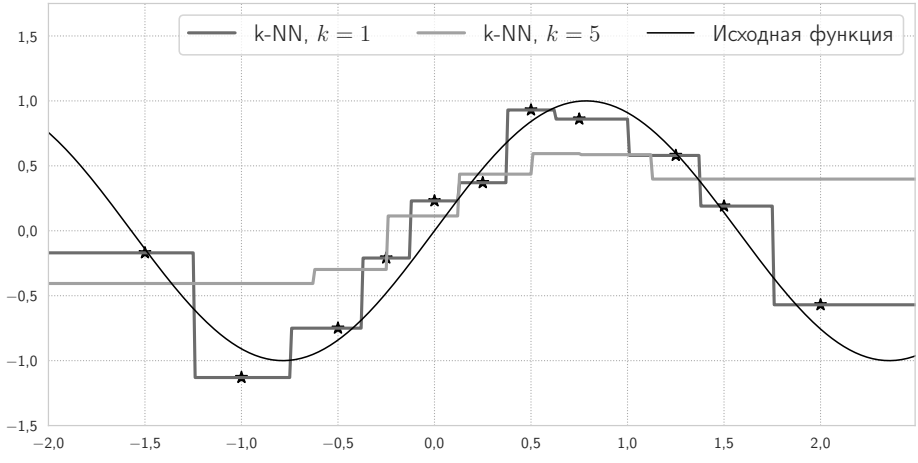


(а) $k = 1$ (б) $k = 5$

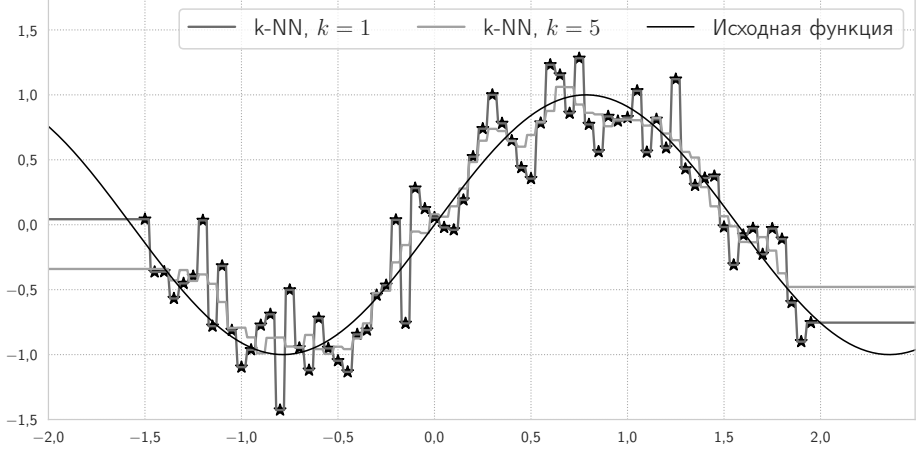
(в) $k = 11$ (г) $k = 99$

Рис. 6.2. Метод ближайших соседей для классификации

ближайшими. Во-вторых, здесь появляется задача скорее из области вычислительной геометрии: как найти ближайших соседей? К этому как раз относился таинственный параметр `algorithm='ball_tree'` — это один из алгоритмов быстрого поиска ближайших соседей [325, 446]. Алгоритм шарового дерева (ball tree) строит дерево, состоящее из подмножеств (кластеров) точек данных, и работает этот алгоритм так:



(а) 11 точек данных



(б) 70 точек данных

Рис. 6.3. Метод ближайших соседей для регрессии на примере из главы 4

- в корне дерева находится множество всех точек, и оно характеризуется центроидом, равным среднему всех точек данных;
- выберем в качестве центра первого кластера точку, наиболее далеко отстоящую от центра, а в качестве центра второго кластера — точку, наиболее далеко отстоящую от первой точки;
- разделим все точки на две части (кластера), относя каждую к ближайшему к ней центру (никакой балансировки кластеров не требуется);

- повторим весь процесс рекурсивно в обоих кластерах-потомках до тех пор, пока не достигнем некоторой заданной глубины дерева.

Такое дерево позволяет проиндексировать точки данных и затем быстро найти подмножество точек, лежащих неподалёку от заданной новой точки, и искать ближайших соседей среди них.

Другой подход к построению дают *k-d-деревья* (k-d-trees) [59], которые тоже строят дерево аналогичным образом, но каждый раз используют только одну координату:

- в корне дерева находится множество всех точек; предположим, что мы упорядочили d компонентов каждого вектора $\boldsymbol{x}$ по номеру;
- первое разделение проведём по первой координате: выберем у значений $x_{11}, x_{21}, \ldots, x_{N1}$ медиану (чтобы сбалансировать дерево) и отнесём к левому потомку корня точки с первой координатой, меньшей медианы, а к правому — с большей;
- второе разделение проходит по второй координате: теперь мы выбираем медианы тех значений x_{i2}, которые попали в левого и правого потомка соответственно; обратите внимание, что эти медианы совершенно не обязаны совпадать;
- и так далее, опять же, до некоторой заданной глубины дерева.

Для небольших размерностей k-d-деревья работают очень хорошо, но с ростом размерности предпочтительными становятся шаровые деревья. Это лишь два примера; есть целая наука на стыке машинного обучения и вычислительной геометрии, в которой улучшаются алгоритмы и структуры данных для поиска ближайших соседей [325, 555, 653]. Обычно речь идёт о приближённых алгоритмах [30, 277, 452], потому что в реальных задачах k редко бывает близким к единице, и один-два пропущенных ближайших соседа не изменят общую картину так уж существенно. В любом случае, вычислительные проблемы с обобщением метода ближайших соседей на миллионы точек, находящихся в пространствах размерности несколько сотен или тысяч, всё ещё остаются.

Есть и более принципиальная причина, по которой метод ближайших соседей не может исчерпывать всё, что нам нужно от машинного обучения. Однако проиллюстрировать его так же наглядно, как на рис. 6.2 и 6.3, не получится: в низких размерностях, которые я мог бы нарисовать для вас, метод ближайших соседей *действительно* работает прекрасно! Он начинает «ломаться» в высоких размерностях — и не только он.

Дело в том, что в высоких размерностях (хотя бы несколько десятков) даже обычное евклидово пространство $\mathbb{R}^d$ начинает вести себя не совсем так, как мы привыкли; многие наши интуитивные представления, сформированные в размерностях 2 и 3, в высоких размерностях перестают работать. Этот эффект известен как *проклятие размерности* (curse of dimensionality), и во многом именно он ответственен за то, что машинное обучение выглядит так, как мы видим в этой

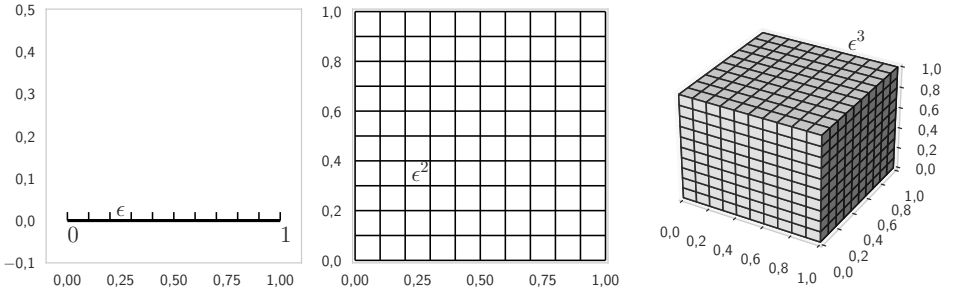
книге. Давайте рассмотрим несколько разных (но связанных друг с другом) аспектов проклятия размерности.

Первый, самый простой аспект: в высоких размерностях мы теряем возможность перебирать области пространства так же эффективно, как это получалось в низких. Например, в машинном обучении, как вы уже не раз убеждались, очень важной операцией является взятие интеграла: и поиск математических ожиданий или других моментов всяческих распределений, и поиск предсказательных распределений у нас всё время сводились к тому, чтобы научиться брать тот или иной интеграл. Несколько раз, например в разделе 4.4, у нас получалось взять такой интеграл аналитически. В разделе 5.3 нам пришлось прибегнуть к хитрой системе аппроксимаций, чтобы оценить предсказательное распределение в логистической регрессии. Но мы ни разу не ссылались на самую простую мысль — любой интеграл можно ведь подсчитать численно! Например, методом трапеций или методом прямоугольников: выбрать достаточно маленький ϵ, разбить отрезок интегрирования $[a,b]$ на интервалы длины ϵ точками $x_0 = a, x_1 = a + \epsilon, \ldots, x_n = b$, а потом приближать интеграл как

$$\int_a^b f(x)\mathrm{d}x \approx \sum_{i=0}^{n-1} \frac{f(x_i) + f(x_{i+1})}{2} (x_{i+1} - x_i) = \\ = \epsilon \left(\frac{f(a) + f(b)}{2} + \sum_{i=1}^{n-1} f(x_i) \right), \quad (6.1)$$

где второе равенство выполняется, если весь отрезок $[a,b]$ разделён на равные интервалы длины $\epsilon = \frac{b-a}{n}$. Это был приближённый подсчёт, а не точный, но разве это важно для машинного обучения, где нам в любом случае приходится приближать дискретное распределение данных, полученных с не известным заранее шумом?

И действительно, формула (6.1) прекрасно работает; она могла бы «закрыть» все реальные потребности машинного обучения... но только в очень небольших размерностях. Чтобы покрыть единичный отрезок интервалами длины ϵ и найти интеграл методом трапеций, в размерности $d = 1$ нужно $\frac{1}{\epsilon}$ раз вычислить значение функции f (см. рис. 6.4, *а*), в размерности $d = 2$ нужно это сделать $\frac{1}{\epsilon^2}$ раз (см. рис. 6.4, *б*), в размерности $d = 3$ — уже $\frac{1}{\epsilon^3}$ раз (см. рис. 6.4, *в*), и так далее. Число узлов сетки, покрывающей тот же объём с тем же шагом, растёт по мере роста размерности экспоненциально, и очень быстро приближённый подсчёт интегралов становится непрактичным. Учтите ещё, что в машинном обучении мы часто хотим подсчитать математическое ожидание по апостериорному распределению, а оно получается перемножением большого числа правдоподобий точек данных. Поэтому апостериорное распределение у немаленького датасета, скорее всего, имеет очень острые пики (локальные максимумы) или хребты в своём



(а) $d = 1$ (б) $d = 2$ (в) $d = 3$

Рис. 6.4. Проклятие размерности: сетка с шагом ϵ в разных размерностях

ландшафте, и надеяться, что мы сможем использовать достаточно мелкую сетку в пространстве высокой размерности, чтобы эти пики и хребты «уловить», не приходится.

Второй аспект связан с первым, но уже имеет более прямое отношение к методу ближайших соседей. С ростом размерности растёт не только число гиперкубиков, которыми можно было бы покрыть тот же заданный объём, но и расстояние между двумя случайно выбранными точками. Если взять две случайные точки x_1 и x_2 равномерно и независимо на отрезке $[a,b]$ длины $l = b - a$ в размерности $d = 1$ (как на рис. 6.4, *а*), то ожидаемое расстояние между ними составит

$$\mathbb{E}\left[|x_1 - x_2|\right] = \int_a^b \int_a^b |x_1 - x_2|\, p(x_1)p(x_2)\mathrm{d}x_1\mathrm{d}x_2 = \frac{1}{l^2}\int_a^b \int_a^b |x_1 - x_2|\,\mathrm{d}x_1\mathrm{d}x_2 =$$
$$= \frac{1}{l^2}\int_a^b \int_a^{x_1} (x_1 - x_2)\,\mathrm{d}x_2\mathrm{d}x_1 + \frac{1}{l^2}\int_a^b \int_{x_1}^b (x_2 - x_1)\,\mathrm{d}x_2\mathrm{d}x_1 =$$
$$= \left(l^3/6 + l^3/6\right)/l^2 = l/3,$$

а ожидаемый квадрат расстояния будет равен

$$\mathbb{E}\left[(x_1 - x_2)^2\right] = \frac{1}{l^2}\int_a^b \int_a^b (x_1 - x_2)\,\mathrm{d}x_1\mathrm{d}x_2 = \frac{1}{l^2}\int_a^b \int_a^b x_1^2\mathrm{d}x_1\mathrm{d}x_2 +$$
$$+ \frac{2}{l^2}\int_a^b \int_a^b x_1x_2\mathrm{d}x_1\mathrm{d}x_2 + \frac{1}{l^2}\int_a^b \int_a^b x_2^2\mathrm{d}x_1\mathrm{d}x_2 = \frac{1}{l^2}\left(\frac{l^3}{3} + 2\frac{l^2}{2}\frac{l^2}{2} + \frac{l^3}{3}\right) = \frac{7}{6}l^2.$$

А при повышении размерности это ожидаемое расстояние будет расти практически линейно. Проще всего это увидеть для L_2-метрики: квадрат евклидова расстояния между двумя точками буквально равен сумме квадратов разностей

их координат, а равномерный выбор точки в кубе со стороной l эквивалентен равномерному и независимому выбору каждой её координаты на соответствующем отрезке, так что в размерности d мы получим в точности

$$\mathbb{E}\left[\|\boldsymbol{x}_1 - \boldsymbol{x}_2\|_2^2\right] = \sum_{i=1}^{d} \mathbb{E}\left[(\boldsymbol{x}_{1i} - \boldsymbol{x}_{2i})^2\right] = \left(\frac{7}{6}l^2\right) \cdot d.$$

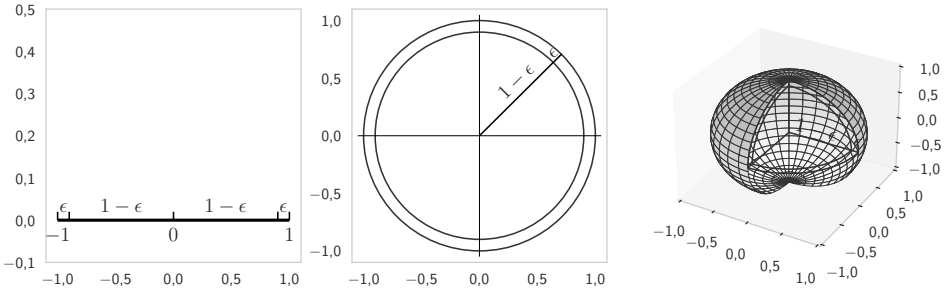
Что это значит для метода ближайших соседей? Это значит, что чем выше размерность, тем дальше будут от точки $\boldsymbol{x}$ те самые ближайшие соседи; конечно, в более высокой размерности можно предполагать и датасет большего размера, но в данном случае это поможет, только если датасет будет совсем уж большим, до полной непрактичности (можете в качестве сложного упражнения попытаться оценить, сколько нужно точек, чтобы убрать коэффициент d в ожидании для ближайшего соседа). Идея здесь такая: чтобы точки оказались «далеко» друг от друга, условно говоря, на расстоянии больше ϵ, достаточно, чтобы в *любой* координате векторы оказались «далеко» друг от друга, а это с ростом размерности становится всё более и более вероятно. Это значит, что у новой точки и у её ближайших соседей из датасета почти наверняка найдутся некоторые компоненты, в которых они непохожи друг на друга; а также наверняка найдётся довольно много компонент, где они похожи. Но загвоздка в том, что мы не знаем заранее, какие компоненты важны для нашей целевой функции (для того, что мы пытаемся предсказать), а какие менее важны, — в том и состоит обычно задача машинного обучения в высокой размерности, чтобы найти или построить важные компоненты входных векторов, отличить их от неважных. Конечно, сейчас я провёл не слишком формальное рассуждение, но надеюсь, что оно в достаточной мере объясняет этот эффект проклятия размерности.

6.1.3. Эффект кожуры апельсина

Третий аспект кажется мне самым интересным; я бы назвал его эффектом кожуры апельсина, потому что здесь будет удобнее рассуждать не о гиперкубах, а о единичных шарах в пространствах разной размерности. Давайте рассмотрим шар радиуса 1 в разных размерностях и предположим, что этот шар — это апельсин, и его кожура имеет толщину ϵ, то есть занимает внешнее кольцо толщины ϵ в единичном шаре. Какую долю объёма будет занимать кожура?

В размерности $d = 1$ всё просто: единичный шар становится отрезком $[-1, 1]$, кожура превращается в два отрезка длины ϵ по его краям, и доля объёма, очевидно, равна ϵ (см. рис. 6.5, *а*). В размерности $d = 2$ объём (площадь) кожуры можно вычислить как разность между площадями внешнего и внутреннего кругов (см. рис. 6.5, *б*):

$$\frac{S_{\text{кожура}}}{S_{\text{шар}}} = \frac{\pi - \pi\,(1-\epsilon)^2}{\pi} = 1 - (1-\epsilon)^2\,.$$



(а) $d = 1$ (б) $d = 2$ (в) $d = 3$

Рис. 6.5. Проклятие размерности: эффект кожуры апельсина толщины ϵ в разных размерностях

В размерности $d = 3$ нужно уже вычитать друг из друга объёмы внешнего и внутреннего шаров (см. рис. 6.5, *в*):

$$\frac{V_{\text{кожура}}}{V_{\text{шар}}} = \frac{\frac{4}{3}\pi - \frac{4}{3}\pi\,(1-\epsilon)^2}{\frac{4}{3}\pi} = 1 - (1-\epsilon)^3\,.$$

И так далее — какой бы ни была константа при πr^d у объёма шара размерности d (это, кстати, весьма нетривиальный вопрос [545]), в отношении она сократится, и останется доля объёма кожуры, равная

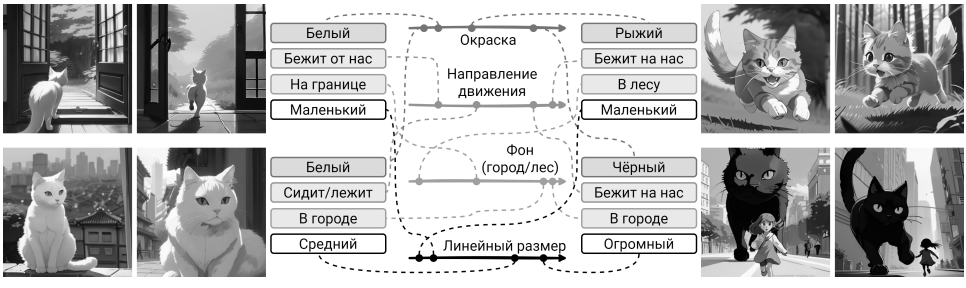
$$1 - (1-\epsilon)^d\,. \tag{6.2}$$

Плохие новости: покупать апельсины в высоких размерностях нет никакого смысла! Уже в размерности $d = 10$ кожура толщиной $\epsilon = \frac{1}{10}$ будет занимать более 65 % объёма, а в размерности $d = 100$ и вовсе более 99,99 %. Но если для апельсинов это очевидно плохие новости, то почему же это плохие новости для машинного обучения и/или конкретно метода ближайших соседей?

Во-первых, это ещё раз подчёркивает то, что расстояния между точками растут с ростом размерности: все точки оказываются сконцентрированы в тонкой внешней оболочке, и расстояния между ними, очевидно, будут немаленькими. Но эффект кожуры приводит к ещё одной плохой новости: оказывается, что в высоких размерностях практически все точки находятся *на границе* датасета. Это, опять же, интуитивно очевидно: чтобы точка оказалась на границе (в «кожуре»), достаточно, чтобы она оказалась на границе хотя бы по одной размерности, и вероятность этого растёт экспоненциально с ростом размерности; именно эту вероятность мы и подсчитали в формуле (6.2).

А модели машинного обучения, которые мы обычно рассматриваем, особенно локальные модели вроде метода ближайших соседей, хороши в *интер*поляции,

(а) Одна размерность объекта высокой сложности



(б) Несколько размерностей объекта высокой сложности

Рис. 6.6. Иллюстрация к проклятию размерности:
(а) если бы картинки с котиками были одномерными; (б) но нет

то есть хорошо предсказывают точки, у которых ближайшие соседи найдутся со всех сторон, но не слишком хороши в *экстра*поляции, то есть в продолжении зависимостей на точки, чьи координаты выходят за пределы высокой концентрации точек датасета. Оказывается, что в высоких размерностях мы почти *всегда* занимаемся экстраполяцией по крайней мере по части компонент вектора $\boldsymbol{x}$ — но, как мы уже обсуждали выше, мы не можем знать заранее, какие именно компоненты важны для целевой функции.

Этот аспект проклятия размерности проиллюстрирован на рис. 6.6. Если бы картинки с котиками образовывали одномерное многообразие, то есть различались между собой только одним параметром — например, какого размера на них котик, — большинство таких изображений находились бы «между» другими уже имеющимися в датасете картинками, и только коты экстремальных габаритов оказывались бы выбросами (см. рис. 6.6, *а*). Но в реальности картинка с котиком — это объект очень высокой размерности, в том числе не только формально по числу пикселей, но и «семантически» по числу содержательных различий, и любая картинка по каким-то шкалам да окажется на краю (см. рис. 6.6, *б*).

Может показаться, что эффект кожуры апельсина касается только равномерного распределения точек в единичном шаре, а в реальности мы будем скорее рассматривать распределения, сконцентрированные около нуля, например экспоненциально убывающий гауссиан. К сожалению, эффект никуда не пропадает. Давайте подсчитаем распределение квадрата расстояния от точки $\boldsymbol{x}$, взятой

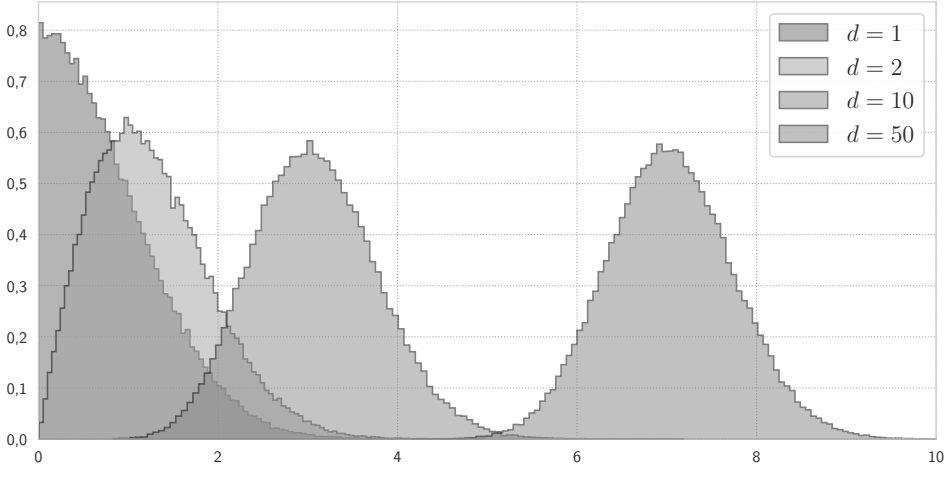


Рис. 6.7. Распределение расстояний от нуля до точек $\boldsymbol{x}$, взятых из стандартного гауссиана, $\boldsymbol{x} \sim \mathcal{N}(\mathbf{O}, \mathbf{I})$, в разных размерностях

из стандартного гауссиана, $\boldsymbol{x} \sim \mathcal{N}(\mathbf{O}, \mathbf{I})$, до нуля; обозначим его через r^2. Здесь, опять же, всё интуитивно очевидно: квадрат расстояния до нуля складывается из квадратов координат,

$$r^2 = x_1^2 + x_2^2 + \ldots + x_d^2,$$

а каждая координата в стандартном нормальном распределении получается из независимого стандартного одномерного гауссиана: $x_i \sim \mathcal{N}(0, 1)$. Поэтому

$$\mathbb{E}\left[r^2\right] = d\mathbb{E}\left[x_i^2\right] = d\sigma^2 = d, \qquad \mathrm{Var}\left[r^2\right] = d\mathrm{Var}\left[x_i^2\right] = d \cdot 2\sigma^4 = 2d,$$

то есть ожидаемое расстояние будет расти с ростом d.

Распределение самого расстояния, а не его квадрата, аналитически найти не так просто [203], поэтому давайте сделаем это эмпирически: посемплируем много точек из многомерного гауссиана и нарисуем получившуюся гистограмму расстояний до нуля. Результат показан на рис. 6.7; как видите, с ростом размерности точки, взятые из стандартного гауссиана, в целом всё дальше отстоят от нуля и становятся существенно отделены от него.

Иначе говоря, нормально распределённые точки тоже оказываются сконцентрированы в тонкой «кожуре» своего «апельсина», этот эффект разве что немного замедляется по сравнению с равномерным распределением. Так что от проклятия размерности нас не спасёт и L_2-регуляризация (то есть нормальные априорные распределения). Из этого же соображения вытекают ещё и другие неприятные эффекты проклятия размерности для более сложных методов байесовского

вывода, в частности для методов, которые оценивают предсказательное распределение путём семплирования из апостериорного, но об этих методах мы в этой книге говорить не будем [66, 419].

Итак, в этом разделе мы начали с того, что рассмотрели метод ближайших соседей. Сначала показалось, что он решает все проблемы машинного обучения с необычайной лёгкостью без всяких предположений, разве что с некоторыми сугубо вычислительными трудностями. Но при ближайшем рассмотрении оказалось, что метод ближайших соседей прекрасно работает только в низких размерностях, а в высоких — в которых и происходит действие основных практических задач машинного обучения — тут же ломается. Эффект, из-за которого он ломается, известен как проклятие размерности, и он мешает далеко не только методу ближайших соседей: выяснилось, что в высоких размерностях мы и интегралы не можем численно брать, и даже более того, мы в высоких размерностях зачастую обречены экстраполировать, а не интерполировать, что тоже отнюдь не помогает делу. В общем, наука наша всё-таки оказалась нужна. А в следующем разделе вы узнаете, в частности, и то, какие предположения метод ближайших соседей делает на самом деле.

6.2. Статистическая теория принятия решений

Что заставило его принять такое решение – неизвестно: ибо он, собственно говоря, не был зол, а так, скотина.

Михаил Салтыков-Щедрин. Медведь на воеводстве

6.2.1. Функция регрессии

В этом разделе нас ждёт ещё один важный сюжет, который так или иначе относится практически ко всем моделям машинного обучения. Мы поговорим о наилучших теоретически возможных моделях, о свойствах функционалов, которые разные модели должны минимизировать, и о том, из чего складывается ошибка практически любой модели машинного обучения, а значит, тем самым и о том, в какую сторону можно эти модели изменять и настраивать.

Начнём с обычной регрессии, которой уже была посвящена целая глава: пусть у нашей модели непрерывный вещественный вход $\boldsymbol{x} \in \mathbb{R}^d$ и одномерный непрерывный вещественный выход $y \in \mathbb{R}$. У них есть некоторое истинное совместное распределение $p(\boldsymbol{x},y)$, которое описывает данные, — это распределение нам неизвестно. Обратите внимание, что в главе 4 мы всё время рассматривали только условное распределение $p\,(y|\boldsymbol{x})$, отмахиваясь от $p(\boldsymbol{x})$ тем, что мы обучаем дискриминирующую модель и значения $\boldsymbol{x}$ всегда предполагаются известными. Для регрессионной модели в этом параграфе это всё ещё так, и обучаем мы по-прежнему $p\,(y|\boldsymbol{x})$, но для того, чтобы оценить ошибку, придётся подключить и $p(\boldsymbol{x})$ тоже: модель должна хорошо работать не везде (так, скорее всего, не получится), а *там, где встречаются точки* $\boldsymbol{x}$.

В задаче регрессии мы хотим найти функцию $f(\boldsymbol{x})$, которая лучше всего предсказывает y. Введём *функцию потерь* (loss function) $L(y,f(\boldsymbol{x}))$, которая наказывает за ошибки. Как и в главе 4, давайте предположим, что шум в данных близок к нормальному, и тем самым возьмём квадратичную функцию потерь

$$L\left(y,f(\boldsymbol{x})\right) = \left(y - f(\boldsymbol{x})\right)^2.$$

Тогда каждой функции f можно сопоставить *ожидаемую ошибку предсказания* (expected prediction error, EPE):

$$\mathrm{EPE}(f) = \mathbb{E}_{p(\boldsymbol{x},y)}\left[\left(y - f(\boldsymbol{x})\right)^2\right] = \iint \left(y - f(\boldsymbol{x})\right)^2 p\left(\boldsymbol{x},y\right) \mathrm{d}\boldsymbol{x}\mathrm{d}y.$$

Теперь самая лучшая функция предсказания $\hat{f}$ — это та функция, которая минимизирует функционал $\mathrm{EPE}(f)$:

$$\hat{f} = \arg\min \mathrm{EPE}(f).$$

Оказывается, что если мы не накладываем никаких дополнительных ограничений на f, то идеальную функцию $\hat{f}$ найти очень легко. Давайте перепишем интеграл как

$$\mathrm{EPE}(f) = \iint (y - f(\boldsymbol{x}))^2 \, p(\boldsymbol{x}, y) \, \mathrm{d}\boldsymbol{x}\mathrm{d}y = \iint (y - f(\boldsymbol{x}))^2 \, p(y|\boldsymbol{x}) \, p(\boldsymbol{x})\mathrm{d}\boldsymbol{x}\mathrm{d}y = \\ = \int \left(\int (y - f(\boldsymbol{x}))^2 \, p(y|\boldsymbol{x}) \, \mathrm{d}y \right) p(\boldsymbol{x})\mathrm{d}\boldsymbol{x}, \quad (6.3)$$

выделив внутренний интеграл по y. Внутренний интеграл в формуле (6.3) зависит от $f(\boldsymbol{x})$ только в одной (текущей) точке $\boldsymbol{x}$, а это значит, что теперь можно минимизировать EPE поточечно:

$$\hat{f}(\boldsymbol{x}) = \arg\min_a \int (y - a)^2 \, p(y|\boldsymbol{x}) \, \mathrm{d}y = \\ = \arg\min_a \mathbb{E}_{p(y|\boldsymbol{x})} \left[(y - a)^2 \right] = \mathbb{E}_{p(y|\boldsymbol{x})} [y]. \quad (6.4)$$

Это решение называется *функцией регрессии* и является наилучшим предсказанием y в любой точке $\boldsymbol{x}$.

То же самое можно сделать и для задач классификации. Давайте рассмотрим не непрерывную целевую переменную y, а дискретную переменную c с k возможными значениями $c_1, \ldots, c_k$, соответствующими k классам. Задача та же: построить функцию $g(\boldsymbol{x})$ с дискретными значениями, $g(\boldsymbol{x}) \in \{c_1, \ldots, c_k\}$, которая как можно лучше предсказывает y.

Для задач классификации нам придётся изменить определение того, что такое «как можно лучше». Давайте введём функцию потерь, равную 1 за каждый неверный ответ и 0 за верный:

$$L(c, f(\boldsymbol{x})) = [\![f(\boldsymbol{x}) \neq c]\!].$$

Это самая простая и прямолинейная функция ошибки классификации; в реальных моделях, которые мы рассматривали, например, в главе 5, её оптимизировать обычно не получается, потому что производная от неё везде равна нулю и никакие методы оптимизации не работают; но для нашего теоретического разговора она сейчас подходит идеально. Рассмотрим функционал ожидаемой ошибки предсказания

$$\mathrm{EPE}(f) = \mathbb{E}_{p(\boldsymbol{x},y)} [L(c, f(\boldsymbol{x}))]$$

и перепишем его как выше:

$$\mathrm{EPE}(f) = \iint [\![f(\boldsymbol{x}) \neq c]\!]\, p(\boldsymbol{x}, c) \mathrm{d}\boldsymbol{x}\mathrm{d}c = \iint [\![f(\boldsymbol{x}) \neq c]\!]\, p\left(c|\boldsymbol{x}\right) p(\boldsymbol{x}) \mathrm{d}\boldsymbol{x}\mathrm{d}c = \\ = \int \left(\sum_{i=1}^{k} [\![f(\boldsymbol{x}) \neq c_i]\!]\, p\left(c_i|\boldsymbol{x}\right) \right) p(\boldsymbol{x}) \mathrm{d}\boldsymbol{x}. \quad (6.5)$$

Опять достаточно оптимизировать внутренний интеграл в (6.5) поточечно, и теперь получается, что этот внутренний интеграл — это сумма величин $p\left(c_i|\boldsymbol{x}\right)$, из которой мы выбором $f(\boldsymbol{x})$ имеем право вычеркнуть одно слагаемое. И если мы хотим минимизировать сумму, нам, разумеется, нужно вычеркнуть из неё самое большое слагаемое:

$$\hat{f}(\boldsymbol{x}) = \arg \min_{c \in \{c_1, \ldots, c_k\}} \sum_{i=1}^{k} [\![c \neq c_i]\!]\, p\left(c_i|\boldsymbol{x}\right) = \arg \max_{c \in \{c_1, \ldots, c_k\}} p\left(c|\boldsymbol{x}\right).$$

Получился очень простой и логичный результат: наилучшим классификатором с точки зрения этой функции ошибки является тот, который для точки $\boldsymbol{x}$ выбирает самый вероятный класс согласно распределению $p\left(c|\boldsymbol{x}\right)$. Такая функция называется *оптимальным байесовским классификатором*, который мы уже обсуждали в подразделе 5.1.4. Если распределение $p\left(c|\boldsymbol{x}\right)$ известно, то его можно и нужно построить, а задача обычно заключается в том, чтобы сделать подходящие предположения по поводу $p\left(c|\boldsymbol{x}\right)$ и оценить его.

6.2.2. Анализ метода ближайших соседей

Кажется, что пока мы совсем ничего умного не сказали, но уже на этом этапе из нашего рассмотрения можно сделать некие практические выводы. Например, давайте запишем ещё раз оптимальное решение задачи классификации, не подставляя пока что конкретную функцию L:

$$\hat{f}(\boldsymbol{x}) = \arg \min_{c \in \{c_1, \ldots, c_k\}} \sum_{i=1}^{k} L\left(c, c_i\right) p\left(c_i|\boldsymbol{x}\right) = \arg \max_{c \in \{c_1, \ldots, c_k\}} L\left(c, c_i\right) p\left(c_i|\boldsymbol{x}\right).$$

Что, если функция $L\left(c, c_i\right)$ не равна нулю и единице, а выражает что-нибудь более содержательное? Пусть для простоты классов всего два, c_1 и c_2. Конечно, трудно представить себе ситуацию, в которой мы будем наказывать за неправильный ответ, то есть $L(c_1,c_1) = L(c_2,c_2) = 0$, в общем-то, всегда.

Но вот $L(c_1, c_2)$ и $L(c_2, c_1)$ вовсе не обязаны совпадать! Эти значения функции ошибки представляют собой «штраф», который мы хотим наложить на модель за ошибки первого рода (false positives) и ошибки второго рода (false negatives), а они вполне могут оказаться несимметричны; мы подробно обсуждали эти ошибки и разные метрики качества классификации в разделе 2.2.

В подразделе 5.5.3 мы уже обсуждали, что вероятностные классификаторы достаточно легко перевзвесить, чтобы адаптировать к новым ценам ошибок. В наших текущих терминах это тоже легко сделать: если модель (как большинство классификаторов) оценивает $p(c_1|\boldsymbol{x})$ и $p(c_2|\boldsymbol{x})$, то достаточно просто выдавать в качестве ответа не тот класс, у которого больше вероятность $p(c_i|\boldsymbol{x})$, а тот, у которого больше величина $L(c_{3-i}, c_i)p(c_i|\boldsymbol{x})$.

Например, если $L(c_2, c_1) = 1000L(c_1, c_2)$, то, значит, класс c_1 нужно выдавать только в том случае, если $p(c_1|\boldsymbol{x}) > 1000p(c_2|\boldsymbol{x})$, то есть если $p(c_1|\boldsymbol{x}) > \frac{1000}{1001}$. Конечно, и без вычислений было понятно, что нужно сдвинуть принимаемые решения в сторону того, чтобы чаще отвечать c_2, а не c_1. Но теперь мы можем численно оценить, *насколько* их нужно сдвинуть, и использовать это понимание на практике.

Но более интересно нам сейчас более теоретическое замечание: наш разговор о функции регрессии и оптимальном байесовском классификаторе позволяет нам лучше понять, что делает метод ближайших соседей, который мы подробно рассматривали в разделе 6.1. Там упоминалось, что на первый взгляд он не делает никаких предположений, но на самом деле оказывается, что неявно делает; давайте разберёмся, какие именно.

Это будет проще увидеть на примере задачи регрессии — формула (6.4) говорит, что мы хотим построить функцию

$$\hat{f}(\boldsymbol{x}) = \mathbb{E}_{p(y|\boldsymbol{x})}[y]$$

по данным. Как оценивать ожидания, хорошо известно — достаточно усреднить несколько значений (семплов) y для данного $\boldsymbol{x}$:

$$\hat{f}(\boldsymbol{x}) = \mathbb{E}_{p(y|\boldsymbol{x})}[y] \approx \frac{1}{k}\sum_{i=1}^{k} y_i, \quad \text{где } y_i \sim p(y|\boldsymbol{x}). \tag{6.6}$$

Это оценка с очень хорошими свойствами, в таком приближении никаких особенных проблем и сильных допущений нет. Но беда в том, что такой выборки у нас тоже нет — формула (6.6) фактически говорит, что нам нужно иметь в датасете несколько разных значений y для каждого значения $\boldsymbol{x}$, в котором мы захотим предсказывать результат. Поэтому метод ближайших соседей приближает эту выборку при помощи того, что есть, заменяя $y_r \sim p(y|\boldsymbol{x})$ на значения в ближайших соседях

$$f_{\text{kNN}}(\boldsymbol{x}) = \frac{1}{k}\sum_{i=1}^{R} y_i, \quad \text{где } (\boldsymbol{x}_i, y_i) \in D, \boldsymbol{x}_i \in \text{kNN}(\boldsymbol{x}).$$

Получается, что метод ближайших соседей строит функцию регрессии напрямую (непараметрически), пользуясь сразу двумя приближениями: мы сначала

аппроксимируем ожидание через среднее, а затем среднее в точке через среднее в ближайших имеющихся точках.

Иначе говоря, метод ближайших соседей предполагает, что в окрестности $\boldsymbol{x}$ функция $y(\boldsymbol{x})$ меняется не слишком быстро на том масштабе расстояний, в который укладывается заданное число ближайших соседей новой точки $\boldsymbol{x}$. Именно это предположение и «ломается» при переходе к высоким размерностям, как вы видели в разделе 6.1.

6.2.3. Минимизация ожидаемой ошибки предсказания

Вернёмся к началу нашего анализа. Рассмотрим совместное распределение данных $p(\boldsymbol{x},y)$ и квадратичную функцию потерь $L(y,f(\boldsymbol{x})) = (y - f(\boldsymbol{x}))^2$. Мы знаем, что оптимальная оценка — это функция регрессии

$$\hat{f}(\boldsymbol{x}) = \mathbb{E}_{p(y|\boldsymbol{x})}[y] = \int yp(y|\boldsymbol{x})\,\mathrm{d}y.$$

Но в жизни мы, скорее всего, не сможем построить $\hat{f}(\boldsymbol{x})$ напрямую; нам придётся пользоваться некоторой моделью $f(\boldsymbol{x})$, которая была обучена на некотором датасете D, порождённом из того же самого распределения данных $p(\boldsymbol{x},y)$. Давайте попробуем подсчитать ожидаемую ошибку для $f(\boldsymbol{x})$:

$$\mathrm{EPE}(f) = \mathbb{E}_{p(\boldsymbol{x},y)}\left[(y - f(\boldsymbol{x}))^2\right] = \iint (y - f(\boldsymbol{x}))^2\, p(\boldsymbol{x},y)\,\mathrm{d}\boldsymbol{x}\mathrm{d}y. \tag{6.7}$$

Перепишем выражение (6.7) в другой форме, прибавив и вычтя в скобках $\hat{f}(\boldsymbol{x})$ (это стандартный трюк, смысл которого станет ясен прямо сейчас):

$$\begin{aligned}\mathrm{EPE}(f) &= \iint \left(y - \hat{f}(\boldsymbol{x}) + \hat{f}(\boldsymbol{x}) - f(\boldsymbol{x})\right)^2 p(\boldsymbol{x},y)\,\mathrm{d}\boldsymbol{x}\mathrm{d}y = \\ &= \iint \left(y - \hat{f}(\boldsymbol{x})\right)^2 p(\boldsymbol{x},y)\,\mathrm{d}\boldsymbol{x}\mathrm{d}y + 2\iint \left(y - \hat{f}(\boldsymbol{x})\right)\left(\hat{f}(\boldsymbol{x}) - f(\boldsymbol{x})\right) p(\boldsymbol{x},y)\,\mathrm{d}\boldsymbol{x}\mathrm{d}y + \\ &\qquad + \iint \left(\hat{f}(\boldsymbol{x}) - f(\boldsymbol{x})\right)^2 p(\boldsymbol{x},y)\,\mathrm{d}\boldsymbol{x}\mathrm{d}y.\end{aligned}$$

Теперь можно заметить, что

$$\begin{aligned}&\iint \left(y - \hat{f}(\boldsymbol{x})\right)\left(\hat{f}(\boldsymbol{x}) - f(\boldsymbol{x})\right) p(\boldsymbol{x},y)\,\mathrm{d}\boldsymbol{x}\mathrm{d}y = \\ &\qquad \int \left(\int \left(y - \mathbb{E}_{p(y|\boldsymbol{x})}[y]\right) p(y|\boldsymbol{x})\,\mathrm{d}y\right)\left(\hat{f}(\boldsymbol{x}) - f(\boldsymbol{x})\right) p(\boldsymbol{x})\mathrm{d}\boldsymbol{x} = 0,\end{aligned}$$

ведь внутри интеграл по y уже от первой степени разности, а не второй. В этом и был смысл нашего нехитрого трюка.

Итого получилось, что

$$\mathrm{EPE}(f) = \mathbb{E}_{p(\boldsymbol{x},y)}\left[\left(y - \hat{f}(\boldsymbol{x})\right)^2\right] + \mathbb{E}_{p(\boldsymbol{x},y)}\left[\left(f(\boldsymbol{x}) - \hat{f}(\boldsymbol{x})\right)^2\right]. \qquad (6.8)$$

Ожидаемая ошибка предсказания разделилась на две неотрицательные части, причём в первой из них вообще никак не участвует f, то есть мы её никак не контролируем. Как видно из формулы (6.8), эта часть, по сути, представляет собой просто среднюю дисперсию шума в данных, отклонение конкретных точек y от своего собственного ожидания.

Это слагаемое называется *шумом* (noise), и это одна из частей того результата, к которому мы стремимся. Для двух других частей придётся присмотреться ко второму слагаемому.

Само по себе выражение $\mathbb{E}_{p(\boldsymbol{x},y)}\left[\left(f(\boldsymbol{x}) - \hat{f}(\boldsymbol{x})\right)^2\right]$, получившееся в формуле (6.8), ничего нового нам не говорит: мы и так знали, что хотим приблизиться к функции регрессии $\hat{f}(\boldsymbol{x})$, а это просто среднеквадратичное отклонение нашей модели $f(\boldsymbol{x})$ от него по распределению $p(\boldsymbol{x},y)$.

Но давайте вспомним, что функция $f(\boldsymbol{x})$ — это не какая-то абстрактная функция, придуманная из головы, а модель машинного обучения $f(\boldsymbol{x}; D)$, которая была обучена на датасете $D \sim p(\boldsymbol{x},y)$ из N точек (здесь и далее размер датасета лучше зафиксировать, иначе придётся говорить много дополнительных слов о том, откуда этот размер возьмётся).

От датасета здесь многое зависит. Представьте себе, например, обычную одномерную линейную регрессию из главы 4, в которой мы пытаемся построить прямую по трём точкам ($N = 3$). Пусть все наши предположения идеально соответствуют действительности: точки данных и в самом деле порождены вокруг некоторой прямой $y = w_0 + w_1 x$ с нормально распределённым шумом. Но тогда получается, что, например, с вероятностью $\frac{1}{8}$ *все три* точки данных будут лежать выше исходной прямой — а это значит, что и обученная нами модель, даже самая лучшая, пройдёт выше. А с вероятностью ещё $\frac{1}{8}$ все три точки окажутся ниже исходной прямой... Это, конечно, крайний случай, но в общем понятно, что ошибка модели будет зависеть от того, насколько нам повезло с данными, даже если все предположения модели абсолютно верны.

Поэтому логично ввести в рассмотрение ещё и среднее предсказание нашей модели *по наборам данных данного размера*; обозначим его через

$$\mathbb{E}_D f(\boldsymbol{x}) = \mathbb{E}_{D \sim p(\boldsymbol{x},y)}\left[f(\boldsymbol{x}; D)\right].$$

Заметим, что данные, по нашим предположениям, приходят из всё того же распределения $p(\boldsymbol{x}, y)$, по которому мы здесь везде берём ожидания.

И теперь можно применить такой же трюк, как выше; прибавим и вычтем $\mathbb{E}_D f(\boldsymbol{x})$ внутри скобки:

$$\mathbb{E}_{p(\boldsymbol{x},y)}\left[\left(f(\boldsymbol{x})-\hat{f}(\boldsymbol{x})\right)^2\right]=$$
$$=\iint\left(\hat{f}(\boldsymbol{x})-\mathbb{E}_D f(\boldsymbol{x})+\mathbb{E}_D f(\boldsymbol{x})-f(\boldsymbol{x};D)\right)^2 p(\boldsymbol{x},y)\,\mathrm{d}\boldsymbol{x}\mathrm{d}y=$$
$$\iint\left(\hat{f}(\boldsymbol{x})-\mathbb{E}_D f(\boldsymbol{x})\right)^2 p(\boldsymbol{x},y)\,\mathrm{d}\boldsymbol{x}\mathrm{d}y+$$
$$+2\iint\left(\hat{f}(\boldsymbol{x})-\mathbb{E}_D f(\boldsymbol{x})\right)\left(\mathbb{E}_D f(\boldsymbol{x})-f(\boldsymbol{x};D)\right)p(\boldsymbol{x},y)\,\mathrm{d}\boldsymbol{x}\mathrm{d}y+$$
$$+\iint\left(\mathbb{E}_D f(\boldsymbol{x})-f(\boldsymbol{x};D)\right)^2 p(\boldsymbol{x},y)\,\mathrm{d}\boldsymbol{x}\mathrm{d}y.$$

Снова по тем же соображениям, применённым теперь к усреднению $f(\boldsymbol{x})$ вокруг её собственного ожидания по распределению $p(\boldsymbol{x},y)$,

$$\iint\left(\hat{f}(\boldsymbol{x})-\mathbb{E}_D f(\boldsymbol{x})\right)\left(\mathbb{E}_D f(\boldsymbol{x})-f(\boldsymbol{x};D)\right)p(\boldsymbol{x},y)\,\mathrm{d}\boldsymbol{x}\mathrm{d}y=0.$$

Теперь уже всё готово для основного результата этого параграфа.

6.2.4. Разложение на смещение, дисперсию и шум

Собрав вместе все результаты предыдущего раздела, мы получаем разложение ожидаемой ошибки предсказания на три слагаемых:

$$\mathrm{EPE}(f)=\mathrm{Bias}^2+\mathrm{Variance}+\mathrm{Noise},$$

где

$$(\mathrm{Bias})^2=\mathbb{E}_{p(\boldsymbol{x},y)}\left[\left(\hat{f}(\boldsymbol{x})-\mathbb{E}_D f(\boldsymbol{x})\right)^2\right],$$
$$\mathrm{Variance}=\mathbb{E}_{p(\boldsymbol{x},y)}\left[\left(\mathbb{E}_D f(\boldsymbol{x})-f(\boldsymbol{x};D)\right)^2\right],$$
$$\mathrm{Noise}=\mathbb{E}_{p(\boldsymbol{x},y)}\left[\left(\hat{f}(\boldsymbol{x})-y\right)^2\right].$$

Этот результат называется *разложением на смещение, дисперсию и шум* (bias-variance-noise decomposition). Шум (noise) мы никак не контролируем, он не зависит от f. А два других слагаемых говорят нам, что общая ошибка складывается из таких двух частей, как:

- смещение (bias) — это то, насколько похожа на правду наша модель в идеальном случае, если рассмотреть результаты, усреднённые по огромному числу датасетов (то есть взять $\mathbb{E}_D f(\boldsymbol{x})$), и сравнить их с правильным ответом $\hat{f}(\boldsymbol{x})$; если смещение велико, то результаты обученных моделей будут непохожи на $\hat{f}(\boldsymbol{x})$, даже если они очень хорошо сходятся друг с другом;

- дисперсия (variance) — это то, насколько каждая конкретная модель, обученная на конкретном датасете D, будет отличаться от своего среднего; если дисперсия велика, то даже при идеальном ожидании $\mathbb{E}_D f(\boldsymbol{x})$ мы никогда не доберёмся до этого ожидания в реальной жизни, ведь на самом деле у нас будет только один датасет и не будет никакого способа узнать $\mathbb{E}_D f(\boldsymbol{x})$.

В реальности, скорее всего, это не две независимые шкалы, значение на каждой из которых нужно минимизировать, а выбор, компромисс: очень низкое смещение будет сбалансировано относительно высокой дисперсией, и наоборот.

Этот компромисс легко увидеть уже даже на примере линейной регрессии, которую мы подробно рассматривали в главе 4. Давайте проведём такую процедуру: будем набрасывать много датасетов из фиксированного числа точек, каждый из которых будет порождён вокруг одного и того же правильного ответа $\sin(2x)$, но с новым случайным шумом, на каждом из них обучим модель полиномиальной регрессии, а затем усредним результаты.

Рассмотрим тот же пример для линейной регрессии, выделив полиномиальные признаки и определив функцию `train_model` так, чтобы она могла обучать полиномиальную регрессию с регуляризацией:

```
orig = lambda x : np.sin(2*x)
xd = np.array([-3, -2, -1, -0.5, 0, 0.5, 1, 1.5, 2.5, 3, 4]) / 2
num_points = len(xd)
xd_d = np.vstack([xd ** i for i in range(1, num_points+1)]).transpose()

def train_model(xs, ys, a=0, use_lasso=False):
    if a == 0:
        return linear_model.LinearRegression(fit_intercept=True).fit(xs, ys)
    else:
        if use_lasso:
            return linear_model.Lasso(alpha=a, fit_intercept=True).fit(xs, ys)
        else:
            return linear_model.Ridge(alpha=a, fit_intercept=True).fit(xs, ys)
```

И теперь давайте порождать новые датасеты, обучать на них модели разных степеней и рисовать результаты (как обычно, само рисование я опущу, но `xs` — это плотное множество точек, которое мы и будем изображать, а `xs_d` — извлечённые из них полиномиальные признаки):

```
d, N, alpha = 1, 100, 0
res = []
for _ in range(N):
    cur_data = orig(xd) + np.random.normal(0, .25, num_points)
    cur_model = train_model(xd_d[:, :d], cur_data, alpha, False)
    res.append(cur_model.predict( xs_d[:, :d] ))
res_mean = np.mean( res, axis=0 )
```

Меняя значение `d`, можно получить `N` графиков моделей для разных датасетов, а в `res_mean` получится их среднее значение, то есть как раз эмпирическая оценка $\mathbb{E}_D f(\boldsymbol{x})$.

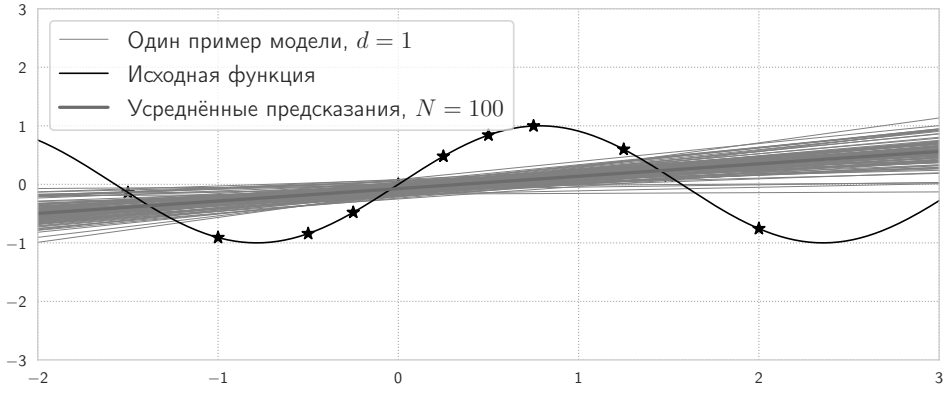
На рис. 6.8 показан результат такого эксперимента для трёх разных значений степени `d`. Как видите, наша интуиция по поводу компромисса между смещением и дисперсией блестяще подтвердилась: по мере того как модель становится более выразительной, то есть с ростом степени многочлена d, смещение модели становится всё меньше, то есть она способна всё точнее описывать период синусоиды.

Но эта точность достигается в идеальном случае на датасете стремящегося к бесконечности размера, а в реальности вместе с ростом выразительности растёт и дисперсия (пучки кривых становятся более «широкими»), то есть модель становится всё более зависима от конкретной реализации случайного шума в датасете. На рис. 6.8, *в* мы видим, что усреднённое предсказание всё равно имеет «странные» пики на краях диапазона, но это артефакт усреднения: для многочлена десятой степени даже $N = 100$ недостаточно, чтобы результаты действительно усреднились, но при $N \to \infty$ всё сойдётся довольно близко к истинной кривой.

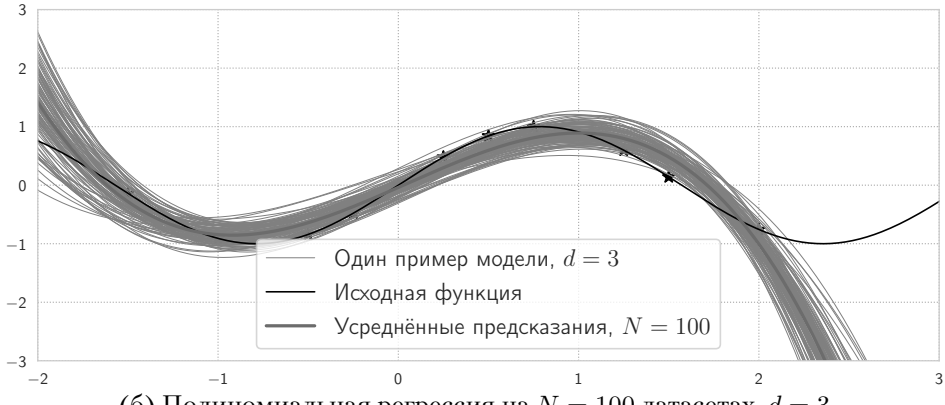
Тот же эффект можно увидеть и при добавлении регуляризации. На рис. 6.9 показаны такие же графики для $d = 10$ с возрастающим коэффициентом L_2-регуляризации $\alpha\|\boldsymbol{w}\|^2$. Как видите, при росте α пучки кривых становятся всё более плотными, то есть дисперсия уменьшается, но это достигается за счёт постепенно возрастающего смещения: с большим регуляризатором модель уже не столько описывает точки данных, сколько пытается сделать коэффициенты регрессии поменьше.

Теперь давайте рассмотрим на практике главный результат этого раздела — не просто компромисс, а собственно разложение ожидаемой ошибки на смещение, дисперсию и шум. Для этого мы, как и выше, обучим много одинаковых моделей на случайных датасетах и посмотрим на их отклонения от собственных средних, а также на отклонения этих средних от истинного значения функции регрессии, причём в качестве модели будем использовать нашу старую знакомую, гребневую регрессию с полиномиальными признаками. Но откуда возьмётся само значение ожидаемой ошибки, чтобы его сравнить со смещением и дисперсией? Для этого давайте введём в рассмотрение некоторое тестовое множество из достаточно большого числа точек, порождённое по тому же распределению, и будем оценивать ошибку на нём:

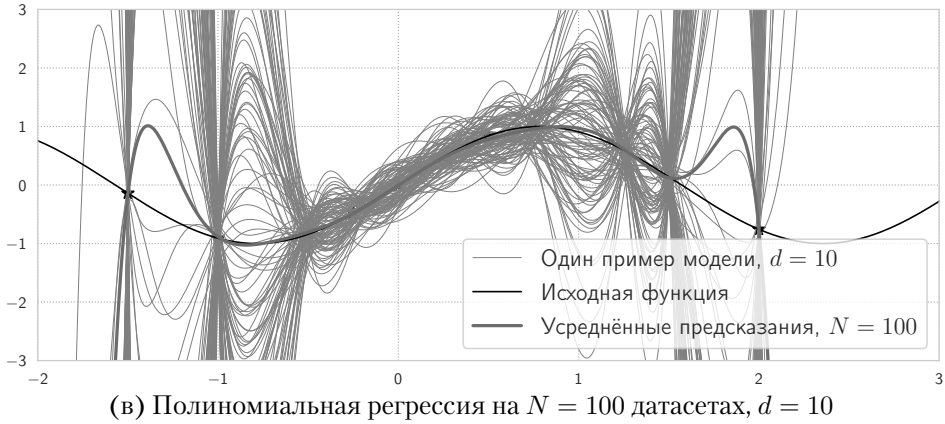
```
test_size = 50
test_x = np.random.rand(test_size) * (2 + 1.5) - 1.5
test_xs = np.vstack([test_x ** i for i in range(1, 12)]).transpose()
test_y = orig(test_xs[:,0]) + np.random.normal(0, .25, test_size)

def test_set_error(model, d):
    return np.mean( (test_y - model.predict(test_xs[:, :d])) ** 2 )
```



(а) Линейная регрессия на $N = 100$ датасетах, $d = 1$

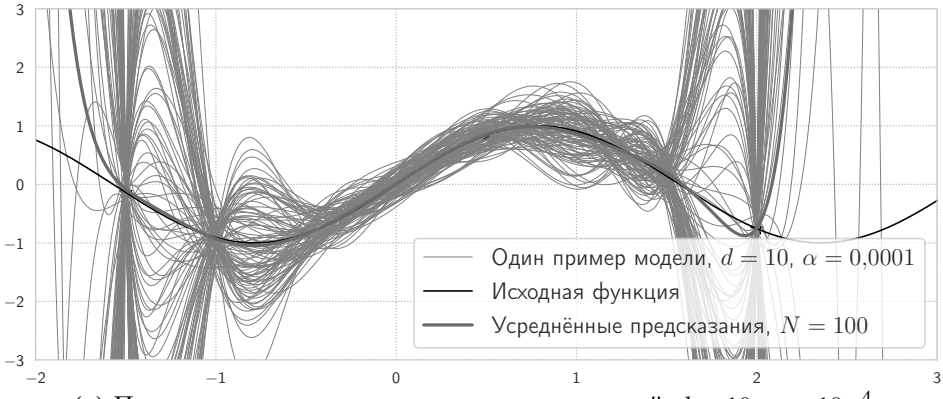


(б) Полиномиальная регрессия на $N = 100$ датасетах, $d = 3$

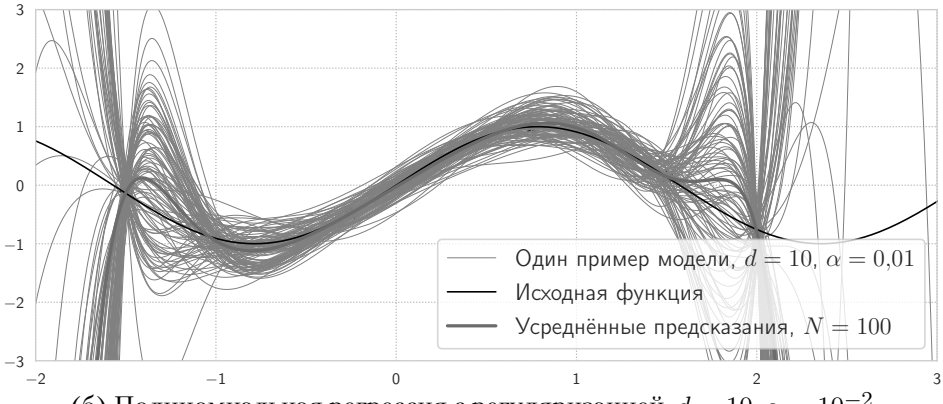


(в) Полиномиальная регрессия на $N = 100$ датасетах, $d = 10$

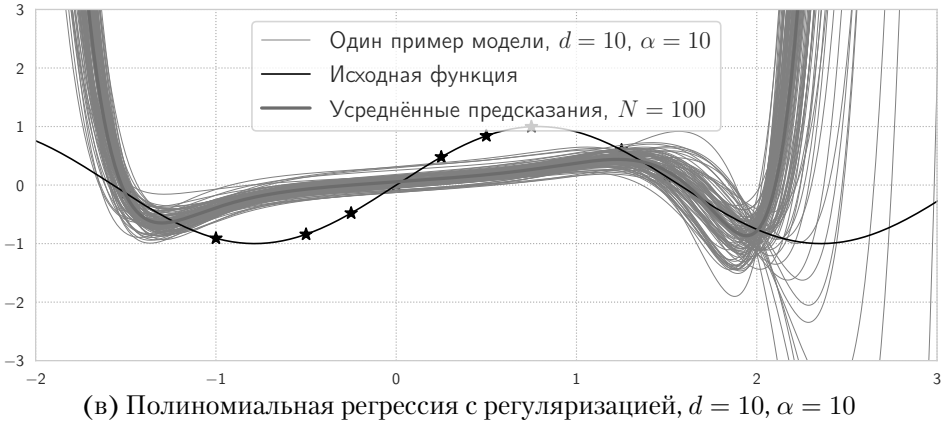
Рис. 6.8. Усреднение полиномиальных моделей разной степени



(а) Полиномиальная регрессия с регуляризацией, $d = 10$, $\alpha = 10^{-4}$



(б) Полиномиальная регрессия с регуляризацией, $d = 10$, $\alpha = 10^{-2}$



(в) Полиномиальная регрессия с регуляризацией, $d = 10$, $\alpha = 10$

Рис. 6.9. Усреднение полиномиальных моделей с разной силой регуляризации

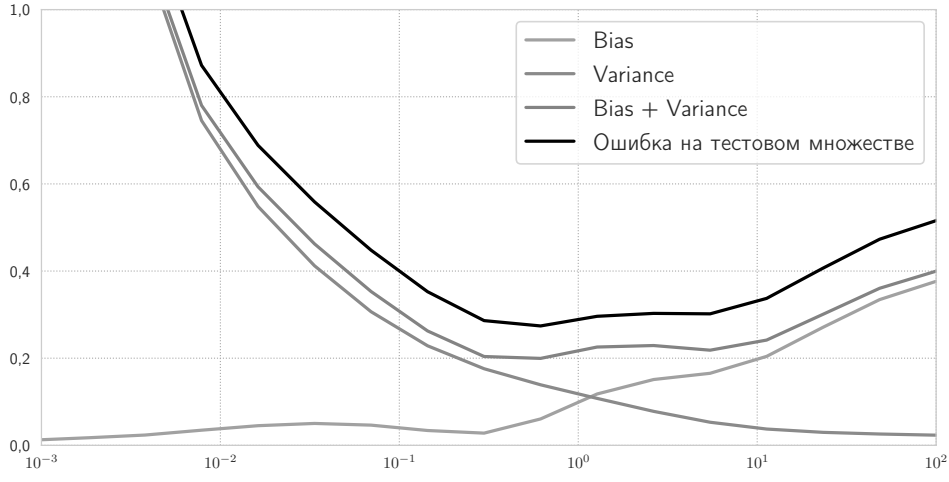


Рис. 6.10. Практическая иллюстрация разложения bias-variance-noise

Построим несколько последовательных экспериментов с возрастающим коэффициентом регуляризации α, в каждом из которых будем обучать 5000 моделей на случайно порождённых датасетах и подсчитывать оценки тех самых смещения и дисперсии.

```
N = 5000
alphas = np.logspace(-4, 2, num=20)
errors, biases, variances = [], [], []

for alpha in alphas:
    res, res_preds, res_test = [], [], []
    for _ in range(N):
        cur_data = orig(xd) + np.random.normal(0, .25, num_points)
        cur_model = train_model(xd_d, cur_data, alpha, use_lasso=False)
        res.append(test_set_error(cur_model, xd_d.shape[1]))
        res_test.append(cur_model.predict(test_set_xs))
        res_preds.append(cur_model.predict( xs_d ))
    res_test = np.array(res_test)
    avg_preds = np.mean(res_test, axis=0)
    errors.append(np.mean(res))
    biases.append(np.mean((avg_preds-orig(test_set_x))**2))
    variances.append(np.mean((res_test-avg_preds)**2))
    print("alpha = %.6f\tmean error = %.6f\n bias = %.6f\tvariance = %.6f" %
      (alpha, errors[-1], biases[-1], variances[-1]))
```

В результате запуска этого кода мы начнём постепенно получать вполне ожидаемые значения:

```
alpha = 0.000100        mean error = 9.381947
  bias = 0.000446        variance = 9.330813
alpha = 0.000207        mean error = 6.110273
  bias = 0.003616        variance = 6.055797
alpha = 0.000428        mean error = 3.646019
  bias = 0.002268        variance = 3.593182
...
```

А весь результат целиком показан в виде графика на рис. 6.10. Как видите, общая идея разложения bias-variance-noise блестяще подтвердилась: мы действительно видим баланс между смещением и дисперсией. Пока регуляризатор слабый, смещения практически нет, но дисперсия очень велика. Это мы можем увидеть и на рис. 6.8, *в*, где показаны несколько многочленов десятой степени, обученных без регуляризации, и результат их усреднения. Когда регуляризатор становится сильнее, дисперсия снижается, но начинает расти смещение; этот эффект тоже можно довести до абсурда слишком сильной регуляризацией, как видно на рис. 6.9, *в*. А где-то между этими крайностями достигается минимум суммы bias + variance.

На рис. 6.10 приведён ещё и график средней ошибки на тестовом множестве. Как видно, разложение снова блестяще подтверждается: кривая ошибки на тестовом множестве практически точно повторяет форму кривой bias + variance, а разница между ними — это и есть дисперсия шума в данных.

Итак, в этом разделе мы формализовали и подробно рассмотрели один из ключевых общих результатов машинного обучения: разложение на смещение, дисперсию и шум. В машинном обучении постоянно приходится искать наилучший баланс между смещением и дисперсией, и много важных результатов обобщают и продолжают разложение на смещение, дисперсию и шум, а также используют эту идею для того, чтобы улучшить разные модели машинного обучения, подобрать в них гиперпараметры и так далее [148, 213, 256, 402, 611]. А мы переходим к следующему сюжету — ядерным методам.

6.3. Эквивалентные ядра и ядерные методы

Звучал булат, картечь визжала,
Рука бойцов колоть устала,
И ядрам пролетать мешала
Гора кровавых тел.

Михаил Лермонтов. Бородино

6.3.1. Другой взгляд на предсказания линейной регрессии

В этом разделе я дам немного другой взгляд на уже хорошо знакомые вам модели машинного обучения, поместив их в контекст *ядерных методов* (kernel methods), важного класса моделей и алгоритмов машинного обучения [268, 516, 532]. Можете считать этот раздел небольшим трейлером более подробного разговора о ядрах, который должен содержать и детальное обсуждение метода опорных векторов; такой разговор мы в этой книге провести не успеем.

А сейчас давайте вернёмся к линейной регрессии и вспомним, как выглядят там байесовские предсказания (мы их получили в разделе 4.4):

$$p\left(y|\boldsymbol{x},D\right)=\mathcal{N}\left(y\middle|\mu_{\text{pred}}(\boldsymbol{x}),\sigma_{\text{pred}}(\boldsymbol{x})\right),\quad\text{где}\quad\mu_{\text{pred}}(\boldsymbol{x})=\boldsymbol{\mu}_N^\top\boldsymbol{x},\,\sigma^2_{\text{pred}}(\boldsymbol{x})=\sigma^2_N(\boldsymbol{x}).$$

Давайте вспомним, что при априорном распределении с нулевым средним $\boldsymbol{\mu}_N=\frac{1}{\sigma^2}\Sigma_N\boldsymbol{X}^\top\boldsymbol{t}$, и перепишем среднее апостериорного распределения в другой форме:

$$y\left(\boldsymbol{x},\boldsymbol{\mu}_N\right)=\boldsymbol{\mu}_N^\top\boldsymbol{x}=\boldsymbol{x}^\top\left(\frac{1}{\sigma^2}\Sigma_N\boldsymbol{X}^\top\boldsymbol{t}\right)=\sum_{n=1}^{N}\boldsymbol{x}^\top\left(\frac{1}{\sigma^2}\Sigma_N\boldsymbol{x}_n\right)t_n.$$

Это значит, что само предсказание можно записать как

$$y\left(\boldsymbol{x},\boldsymbol{\mu}_N\right)=\sum_{n=1}^{N}k\left(\boldsymbol{x},\boldsymbol{x}_n\right)t_n,$$

где

$$k\left(\boldsymbol{x},\boldsymbol{x}_n\right)=\frac{1}{\sigma^2}\boldsymbol{x}^\top\Sigma_N\boldsymbol{x}_n,\quad\Sigma_N=\left(\Sigma_0^{-1}+\frac{1}{\sigma^2}\boldsymbol{X}^\top\boldsymbol{X}\right)^{-1}.$$

Получилось, что мы предсказываем значение в следующей точке как *линейную комбинацию* значений t_n в известных точках, а коэффициенты этой комбинации зависят от точек $\boldsymbol{x}_n$ и $\boldsymbol{x}$, но не зависят от t_n. Это совсем не интуитивно

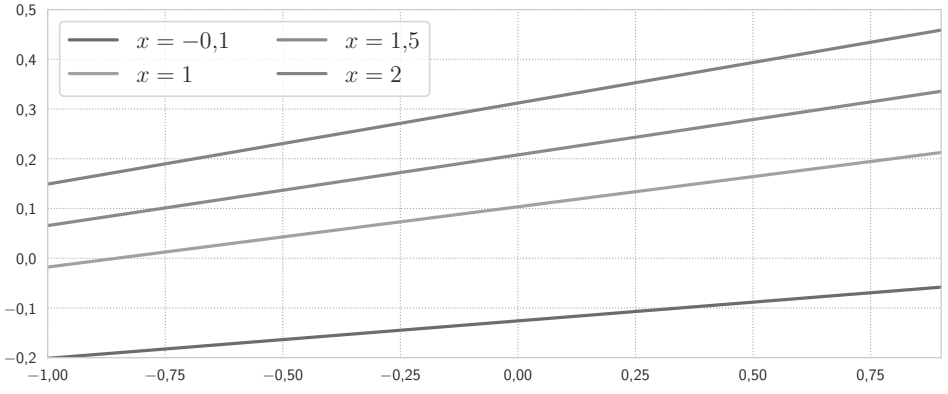


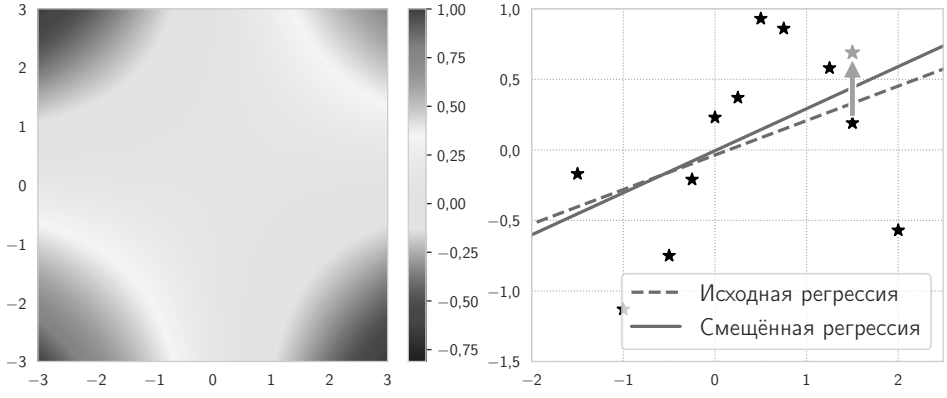
Рис. 6.11. Предсказания в нескольких точках x как функция от значения y в данных в точке $x_n = 0{,}86$ в примере из главы 4

очевидный факт: когда мы меняем какую-то точку данных t_n, это изменение не напрямую отражается на предсказании $y(\boldsymbol{x}, \boldsymbol{\mu}_N)$, а «пропускается» через весьма сложный процесс обучения линейной регрессии, от которого линейности как-то совсем не ожидаешь. И тем не менее, каждая пара точек $\boldsymbol{x}$ и $\boldsymbol{x}_n$ через функцию $k(\boldsymbol{x},\boldsymbol{x}_n)$ задаёт коэффициент, с которым изменение данных t_n в точке $\boldsymbol{x}_n$ именно *линейным* образом повлияет на предсказание в точке $\boldsymbol{x}$.

Это легко проверить экспериментально — давайте вернёмся к нашему примеру из главы 4, начнём двигать одну из точек данных и нарисуем, как меняется предсказание в какой-нибудь другой точке. Код ниже предполагает, что `xd_d` и `data` заданы как в подразделе 4.1.2, для линейной регрессии с исходной функцией $y = \frac{1}{2}x - \frac{1}{2}$ (эти данные были изображены на рис. 4.3):

```
def get_one_prediction(x_pred, cur_y, d=1, data_ind=7, data=data):
    new_data = np.copy(data)
    new_data[data_ind] = cur_y
    cur_model = linear_model.LinearRegression(fit_intercept=True).\
        fit( xd_d[:, :d], new_data )
    return cur_model.predict(np.array([x_pred]).reshape(1,-1))[0]

one_pred = {}
for x_pred in [-0.1, 1, 1.5, 2]:
    one_pred[x_pred] = [ get_one_prediction(x_pred, y) for y in ys ]
```

Если теперь нарисовать массивы из словаря `one_pred`, получатся идеальные прямые, как показано на рис. 6.11.



(а) (б)

Рис. 6.12. Эквивалентное ядро $k(\boldsymbol{x},\boldsymbol{x}_n) = \frac{1}{\sigma^2}\boldsymbol{x}^\top \Sigma_N \boldsymbol{x}_n$ в примере из главы 4: (а) двумерная функция $k(x, x_n)$; (б) пример смещения линейной регрессии

6.3.2. Эквивалентное ядро в линейной регрессии

Функция $k(\boldsymbol{x},\boldsymbol{x}_n) = \frac{1}{\sigma^2}\boldsymbol{x}^\top \Sigma_N \boldsymbol{x}_n$ называется *эквивалентным ядром* (equivalent kernel). В данном случае она имеет довольно любопытную форму; давайте построим график этой функции, вспомнив процедуру байесовского вывода, которую мы реализовывали в разделе 4.4:

```
def bayesian_update(mu, sigma, x, y, sigma_noise=.5):
    x_matrix = np.array([[1, x]])
    sgm_n = np.linalg.inv(np.linalg.inv(sigma) + \
      (1/(sigma_noise**2)) * np.matmul(np.transpose(x_matrix), x_matrix) )
    mu_n = np.matmul(sgm_n, np.matmul(np.linalg.inv(sigma), np.transpose(mu))+\
      (1/(sigma_noise**2)) * np.matmul(np.transpose(x_matrix), np.array([y])))
    return mu_n, sgm_n

mu, sigma = np.zeros(2), np.identity(2)
for x,y in zip(xd, data):
    mu, sigma = bayesian_update(mu, sigma, x, y)

def my_kernel(x, xn, sigma_noise=.5):
    return np.dot(np.array([1, x]), np.matmul(sigma, \
             np.transpose(np.array([1, xn])))) / (sigma_noise ** 2)
Z = np.array([[ my_kernel(x, xn) for x in xs] for xn in xs])
```

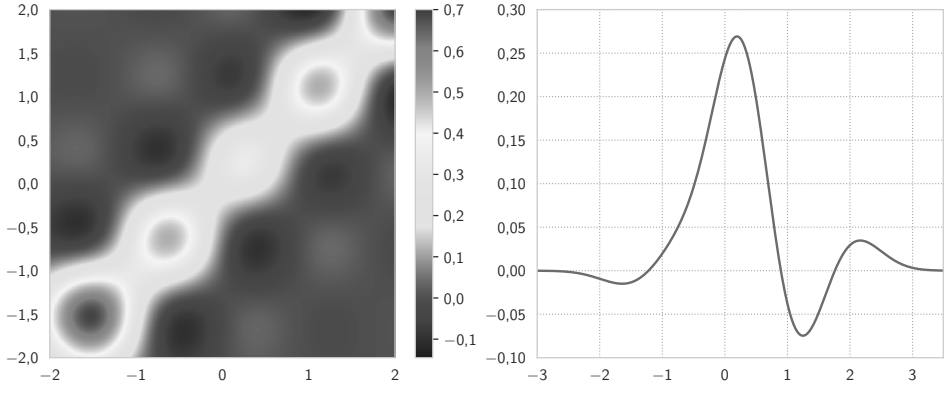
На рис. 6.12, а изображён полученный массив `Z` для `xs`, плотно заметающего интервал $[-3, 3]$, в виде трёхмерного графика для всё той же одномерной ли-

нейной регрессии из подраздела 4.1.2 с $\boldsymbol{\mu}_0 = \mathbf{O}$ и $\Sigma_0 = \mathbf{I}$. Обратите внимание, что влияние значений в одной точке на предсказания в другой может быть как с положительным коэффициентом, так и с отрицательным. Это, опять же, звучит контринтуитивно: казалось бы, если в целом значения данных поднимаются, то и предсказания должны увеличиваться. Но этот эффект тоже легко объяснить, ведь одна прямая не может оставаться всё время выше другой, если они не параллельны: на рис. 6.12, *б* показано, как при смещении одной из точек у линейной регрессии образуется «рычаг» и предсказанные значения левее точки «оси вращения» начинают уменьшаться.

В качестве ещё одной иллюстрации давайте построим аналогичный график для регрессии с гауссовскими признаками. Для этого придётся немножко помучиться с преобразованием массивов в матрицы и обратно, но, по сути, это выглядит точно так же; в коде ниже `bayesian_update_phi` и `my_kernel_phi` в целом совпадают с `bayesian_update` и `my_kernel`, но сначала выделяют из `x` признаки:

```
def bayesian_update_phi(mu, sigma, x, y, phi, sigma_noise=.5):
    x_matrix = np.array([phi(x)])
    sgm_n = np.linalg.inv(np.linalg.inv(sigma) + \
      (1/(sigma_noise**2)) * np.matmul(np.transpose(x_matrix), x_matrix))
    mu_n = np.matmul(sgm_n, np.matmul(np.linalg.inv(sigma), np.transpose(mu))+\
      (1/(sigma_noise**2)) * np.matmul(np.transpose(x_matrix), np.array([y])))
    return mu_n, sgm_n

gf = GaussianFeatures(5)
gf.fit(gauss_xd[:, np.newaxis], gauss_yd)

def my_phi(x):
    return gf.transform(np.array([[x]]))[0]

g_mu, g_sigma = np.zeros(5), np.identity(5)
for x,y in zip(xd, data):
    g_mu, g_sigma = bayesian_update_phi(g_mu, g_sigma, x, y, my_phi)

def my_kernel_phi(x, xn, phi, sigma_noise = 0.5):
    return np.dot(phi(x), np.matmul(g_sigma, \
            np.transpose(np.array([phi(xn)]))))[0] / (sigma_noise**2)

g_Z = np.array([[ my_kernel_phi(x, xn, my_phi) for x in xs] for xn in xs])
```

Результат показан на рис. 6.13, *а*, а на рис. 6.13, *б* изображён один «срез» двумерной функции k_{Gauss} для $x_n = 0$, то есть функция $k_{\text{Gauss}}(x, 0)$. Как видите, теперь зависимость влияния точки от её расположения стала гораздо более интересной, а её смысл в целом соответствует сути локальных признаков: если точка x находится рядом с x_n (около нуля на рис. 6.13, *б* и около главной диагонали на рис. 6.13, *а*), то её влияние велико, а затем начинает убывать по модулю, хоть и неравномерно. Ровно в нуле максимум не получается потому, что признаков



(а) (б)

Рис. 6.13. Эквивалентное ядро $k_{\text{Gauss}}(\boldsymbol{x},\boldsymbol{x}_n) = \frac{1}{\sigma^2}\varphi(x)^\top \Sigma_N \varphi(x_n)$ с пятью гауссовскими признаками $\varphi(x)$: (а) двумерная функция $k_{\text{Gauss}}(x, x_n)$; (б) срез функции $k_{\text{Gauss}}(x, x_n)$ для $x_n = 0$

больше одного, и после обучения у них появляются нетривиальные коэффициенты, так что с одной стороны влияние может быть больше, чем с другой. Но обратите внимание, что хотя все сложные зависимости в линейной регрессии с признаками приводят к тому, что ядро $k(\boldsymbol{x},\boldsymbol{x}_n)$ становится сложнее, но суть эквивалентного ядра не меняется: каждая точка данных по-прежнему оказывает линейное воздействие на предсказания в какой-либо другой фиксированной точке.

6.3.3. Ядерные методы

То, что у нас получилось с линейной регрессией, — это частный случай очень общего подхода, другого взгляда на машинное обучение: *ядерных методов* (kernel methods). Давайте кратко обрисуем на примере, о чём этот раздел машинного обучения говорит, а более близкое знакомство отложим на будущее.

Предположим, что мы решаем задачу обучения с учителем, то есть у нас есть некий набор данных $D = \{(x_1,y_1), (x_2,y_2), \ldots, (x_N,y_N)\}$, где $x_n \in \mathcal{X}$ и $y_n \in \mathcal{Y}$; так выглядят и задачи регрессии, и задачи классификации. Цель машинного обучения — научиться предсказывать $y \in \mathcal{Y}$ для данного $x \in \mathcal{X}$ так, чтобы это было связано с имеющимися данными, то есть так, чтобы (и здесь появляется новый взгляд) пара (x, y) была *похожа* на пары (x_n, y_n), которые уже есть в D. Похожесть в пространстве $\mathcal{Y}$ обычно задать легко; например, в случае бинарной классификации $\mathcal{Y}$ и вовсе состоит из двух значений, например $\mathcal{Y} = \{\pm 1\}$. Интересная часть здесь состоит в том, чтобы задать похожесть в пространстве $\mathcal{X}$; для этого нам нужна некоторая функция похожести

$$k : \mathcal{X} \times \mathcal{X} \to \mathbb{R}, \quad (x, x') \mapsto k(x,x').$$

Эта функция похожести и называется *ядром* (kernel).

Главное техническое требование к функции k состоит в том, что она должна задавать скалярное произведение в некотором пространстве $\mathcal{H}$, которое иногда называют *пространством признаков* (feature space). Иначе говоря,

$$k(x,x') = \Phi(x)^\top \Phi(x')$$

для некоторого выделяющего признаки отображения $\Phi : \mathcal{X} \to \mathcal{H}$. Если это требование выполнено (а оно выполнено довольно часто, но опять же технические детали я сейчас опущу), то мы можем начать строить алгоритмы в пространстве признаков $\mathcal{H}$, используя при этом функцию $k(x,x')$ вместо того, чтобы реально переходить в это пространство, а оно, скорее всего, будет иметь очень, очень высокую размерность.

Например, предположим, что мы строим простейший бинарный классификатор в пространстве $\mathcal{H}$: ищем центроиды обоих классов в данных и проводим серединный перпендикуляр между ними. Это значит, что мы ищем

$$\boldsymbol{c}_1 = \frac{1}{N_1} \sum_{n:y_n=1} \Phi(x_n), \qquad \boldsymbol{c}_2 = \frac{1}{N_2} \sum_{n:y_n=-1} \Phi(x_n), \tag{6.9}$$

где $N_1 = \#\{n : y_n = 1\}$, $N_2 = \#\{n : y_n = -1\}$, а потом строим предсказания вида

$$y(x) = \operatorname{sign}\left(\boldsymbol{c}_1^\top \Phi(x) - \boldsymbol{c}_2^\top \Phi(x) - b\right), \quad \text{где} \quad b = \frac{\|\boldsymbol{c}_1\|^2 - \|\boldsymbol{c}_2\|^2}{2}; \tag{6.10}$$

мы обсуждали такой подход, когда вели разговор о геометрической сути классификации в разделе 5.1.

А теперь главное: давайте подставим $\boldsymbol{c}_1$ и $\boldsymbol{c}_2$ из (6.9) в (6.10) и распишем скалярное произведение по линейности:

$$\begin{aligned} y(x) &= \operatorname{sign}\left(\frac{1}{N_1} \sum_{n:y_n=1} \Phi(x_n)^\top \Phi(x) - \frac{1}{N_1} \sum_{n:y_n=1} \Phi(x_n)^\top \Phi(x) - b\right) = \\ &= \operatorname{sign}\left(\frac{1}{N_1} \sum_{n:y_n=1} k(x_n, x) - \frac{1}{N_1} \sum_{n:y_n=1} k(x_n, x) - b\right), \end{aligned}$$

где

$$b = \frac{1}{2}\left(\frac{1}{N_1^2} \sum_{m,n:y_n=y_m=1} k(x_m, x_n) - \frac{1}{N_2^2} \sum_{m,n:y_n=y_m=-1} k(x_m, x_n)\right).$$

Получилось, что мы обучили классификатор в пространстве $\mathcal{H}$ и выразили полученное правило классификации при помощи ядра $k(x,x')$, вообще *ни разу не записывая* ни одного вектора из $\mathcal{H}$ — все операции проводятся над элементами исходного пространства $\mathcal{X}$. Это значит, что пространство $\mathcal{H}$ может иметь гигантскую размерность, а может быть и вовсе бесконечномерным — всё равно всё будет замечательно работать, если только мы сможем скалярное произведение в этом пространстве «свернуть» в функцию k, действующую в исходном пространстве $\mathcal{X}$. Этот эффект называется *ядерным трюком* (kernel trick), и на нём основан и нелинейный метод опорных векторов, и гауссовские процессы, и многое другое; но здесь я оставлю ссылки на основные книги и обзоры по ядерным методам [268, 516, 532], а сам пока остановлюсь.

6.4. Case study: байесовский вывод для гауссиана

Алексей Максимович настаивал на своём:
— И не видя, я мог прийти путем умозаключений к тому же самому выводу... Не верите, вот спросите Преподобного — он не станет говорить неправду.
— Ещё не научился, — подал голос Абзац.
— Совершенно верно, — согласился я. — Да зачем врать, когда можно говорить правду!

Сергей Григорьев. Кругосветка

6.4.1. Выводы для фиксированного среднего и фиксированной точности

В этом разделе мы рассмотрим последний в этой книге развёрнутый пример: проведём полноценный байесовский вывод для нормального распределения, решив все задачи байесовского вывода. Мы найдём семейство сопряжённых априорных распределений для гауссиана и построим как апостериорное, так и предсказательное распределения.

Но позвольте, разве не это мы делали в разделе 4.4? То, да не совсем. Дело в том, что в разделе 4.4 мы обучали фактически только *один* параметр гауссиана $\mathcal{N}(\boldsymbol{x}|\boldsymbol{\mu},\Sigma)$, а именно среднее $\boldsymbol{\mu}$. А теперь мы попробуем сделать это для обоих параметров. Для простоты в этом разделе мы будем рассматривать только одномерный гауссиан $\mathcal{N}(x|\mu,\sigma^2)$, а также будем записывать его не через дисперсию σ^2, а через точность $\tau = \frac{1}{\sigma^2}$:

$$\mathcal{N}(x|\mu,\tau) = \sqrt{\frac{\tau}{2\pi}} e^{-\frac{\tau}{2}(x-\mu)^2}.$$

К полному байесовскому выводу мы придём в три приёма[1]. В каждом из них нам потребуется логарифм правдоподобия для набора данных $D = \{x_1,\ldots,x_n\}$. Давайте сначала запишем его полностью:

$$\log p(x_1,\ldots,x_n|\mu,\tau) = \frac{n}{2}\log\tau - \frac{n}{2}\log 2\pi - \frac{\tau}{2}\sum_{i=1}^{n}(x_i-\mu)^2. \tag{6.11}$$

[1] Как известно, «Бог любит троицу», и именно с третьей попытки или после третьего испытания в сказках обычно происходит всё самое главное. Можно было бы подумать, что дело тут во влиянии христианства, но тройка была магическим числом задолго до Христа: уже Аристотель писал, что «Триада есть число целого, ибо содержит начало, середину и конец», а трискелион встречается ещё на артефактах эпохи неолита.

Первый шаг — давайте сделаем вывод для фиксированного τ, то есть для правдоподобия

$$p(x|\mu) \propto e^{-\frac{\tau}{2}(x-\mu)^2}.$$

Это уже действительно в точности совпадает с тем, что мы делали в разделе 4.4, так что много подробностей нам здесь не понадобится.

Сопряжённым априорным распределением будет тоже гауссиан

$$p(\mu) = \mathcal{N}(\mu|\mu_0,\tau_0) = \sqrt{\frac{\tau_0}{2\pi}} e^{-\frac{\tau_0}{2}(\mu-\mu_0)^2},$$

и логарифм апостериорного распределения для набора данных $D = \{x_1,\ldots,x_n\}$ запишется как

$$\log p(\mu|x_1,\ldots,x_n) = \text{Const} + \log p(\mu) + \log p(x_1,\ldots,x_n|\mu) = \\ = \text{Const} - \frac{\tau_0}{2}(\mu-\mu_0)^2 - \sum_{i=1}^{n}\frac{\tau}{2}(x_i-\mu)^2.$$

Здесь мы воспользовались формулой (6.11), но не полностью, а только вторым слагаемым: в первом не встречается μ, и для нас сейчас это константа.

Теперь мы видим, что у нас снова получается нормальное распределение, ведь логарифм апостериорного распределения является квадратичной функцией от μ. А чтобы найти его параметры, нужно выделить полный квадрат, в точности так же, как в разделе 4.4. В выводе ниже мы снова нещадно заносим в константу всё, что не зависит от μ:

$$\log p(\mu|x_1,\ldots,x_n) = \\ = \text{Const} - \frac{\tau_0}{2}\left(\mu^2 - 2\mu\mu_0 + \mu_0^2\right) - \frac{\tau}{2}\left(\sum_{i=1}^{n}x_i^2 - 2\mu\sum_{i=1}^{n}x_i + n\mu^2\right) = \\ = \text{Const} - \frac{1}{2}\left(\tau_0\mu^2 - 2\mu_0\tau_0\mu - 2\mu\tau\sum_{i=1}^{n}x_i + n\tau\mu^2\right) = \\ = \text{Const} - \frac{\tau_0+n\tau}{2}\left(\mu - \frac{\tau_0\mu_0 + \tau\sum_{i=1}^{n}x_i}{\tau_0+n\tau}\right)^2.$$

Иначе говоря, в этом выводе мы получили нормальное апостериорное распределение

$$p(\mu|x_1,\ldots,x_n) = \mathcal{N}(\mu|\mu_n,\tau_n), \quad \text{где} \\ \tau_n = \tau_0 + n\tau, \qquad \mu_n = \frac{\tau_0\mu_0 + \tau\sum_{i=1}^{n}x_i}{\tau_0+n\tau}. \tag{6.12}$$

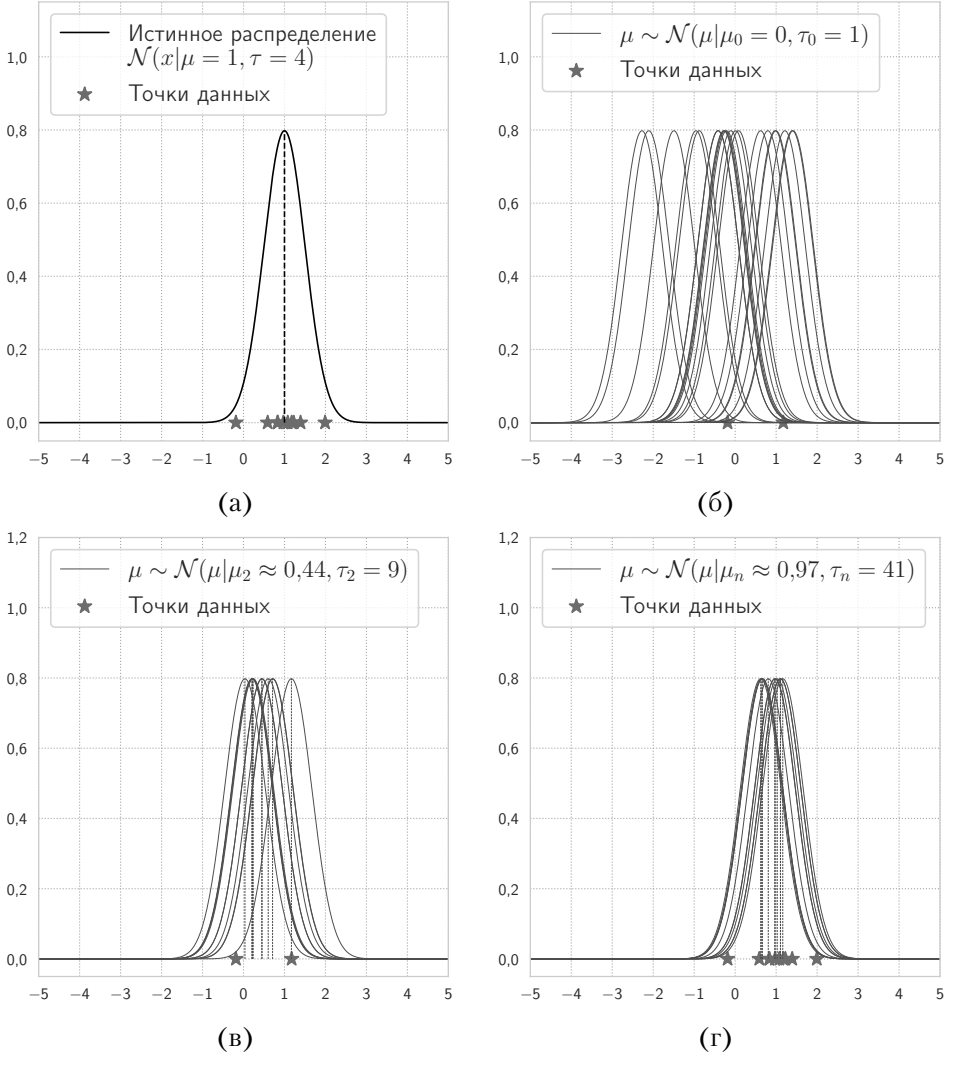


Рис. 6.14. Пример обучения гауссиана по μ с фиксированным τ: (а) истинное распределение и выборка точек данных; (б) выборка из априорного распределения $\mathcal{N}(\mu|\mu_0 = 0, \tau_0 = 1)$ и две точки данных; (в) выборка из апостериорного распределения $\mathcal{N}(\mu|\mu_2, \tau_2)$ и новые точки данных; (г) выборка из апостериорного распределения $\mathcal{N}(\mu|\mu_n, \tau_n)$ для $n = 10$

Обратите внимание, насколько удобнее получаются формулы в терминах τ, а не σ: по τ байесовский вывод получается почти аддитивным, нужно просто добавить τ нужное число раз, а по σ формула была бы куда более громоздкой:

$$\frac{1}{\sigma_n^2} = \frac{1}{\sigma_0^2} + \frac{n}{\sigma^2}, \quad \text{то есть} \quad \sigma_n^2 = \frac{\sigma_0^2\sigma^2}{\sigma^2 + n\sigma_0^2}.$$

На рис. 6.14 проиллюстрирован получившийся у нас вывод. Десять точек данных были взяты из распределения $\mathcal{N}(x|\mu = 1, \tau = 4)$ (см. рис. 6.14, *а*), то есть $\sigma = \frac{1}{2}$, и в течение всего вывода значение $\tau = 4$ было зафиксировано. Мы начали с априорного распределения $\mathcal{N}(\mu|\mu_0 = 0, \tau_0 = 1)$ (см. рис. 6.14, *б*) и провели байесовский вывод по формуле (6.12) для двух точек данных. Результат — выборка из апостериорного распределения $\mathcal{N}(\mu|\mu_2, \tau_2)$ — показан на рис. 6.14, *в*, а на рис. 6.14, *г* показана выборка из апостериорного распределения $\mathcal{N}(\mu|\mu_n, \tau_n)$, полученного добавлением остальных восьми точек данных. Как видите, апостериорное распределение на μ постепенно уточняется и приближается к правильному ответу $\mu = 1$, как мы и ожидали.

В разделе 4.4 мы на этом и остановились, но теперь это лишь первый шаг. Второй шаг будет симметричен: давайте теперь зафиксируем μ и сделаем вывод по τ. Эта постановка задачи выглядит несколько искусственной: пожалуй, в жизни нам действительно вряд ли придётся обучать гауссианы с фиксированным средним и переменной дисперсией. Но это даст нам идеи, нужные для дальнейшего более общего вывода.

Какое здесь взять априорное распределение $p(\tau)$? Запишем сначала правдоподобие (6.11) как функцию от τ:

$$\log p(x_1, \ldots, x_n|\tau) = \text{Const} + \frac{n}{2}\log\tau - \frac{\tau}{2}\sum_{i=1}^{n}(x_i - \mu)^2,$$

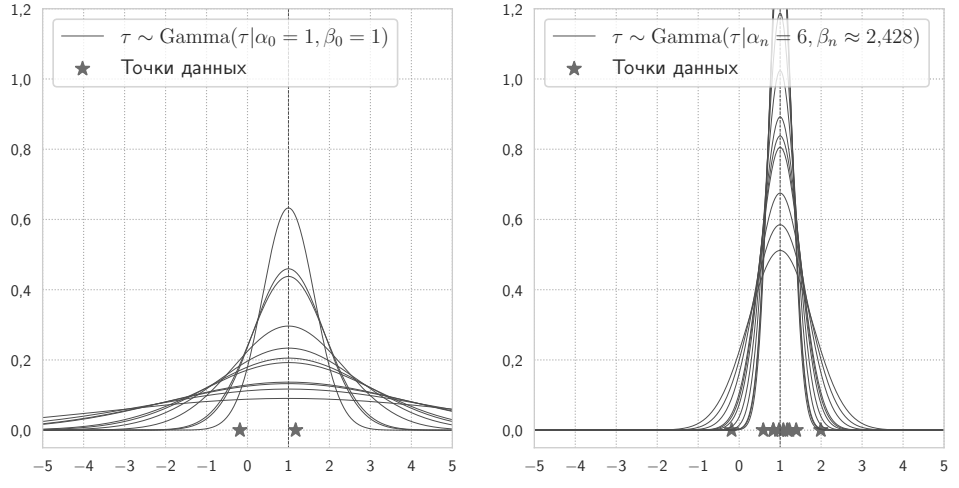
то есть правдоподобие — это функция вида $\text{Const} \cdot \tau^a e^{-b\tau}$. Это сразу намекает на *гамма-распределение*, которое мы обсуждали в разделе 3.2. И действительно, давайте выберем

$$p(\tau) = \text{Gamma}(\tau|\alpha_0, \beta_0) = \frac{\beta_0^{\alpha_0}}{\Gamma(\alpha_0)}\tau^{\alpha_0 - 1}e^{-\beta_0\tau}$$

и запишем логарифм апостериорного распределения $\log p(\tau|x_1, \ldots, x_n)$.

Как и раньше, воспользуемся формулой (6.11) и будем заносить в константу всё, что не зависит от τ:

$$\begin{aligned}\log p(\tau|x_1, \ldots, x_n) &= \text{Const} + \log p(\tau) + \log p(x_1, \ldots, x_n|\tau) = \\ &= \text{Const} - (\alpha_0 - 1)\log\tau - \beta_0\tau + \frac{n}{2}\log\tau - \sum_{i=1}^{n}\frac{\tau}{2}(x_i - \mu)^2 = \\ &= \text{Const} - \left(\alpha_0 + \frac{n}{2} - 1\right)\log\tau - \left(\beta_0 + \frac{1}{2}\sum_{i=1}^{n}(x_i - \mu)^2\right)\tau.\end{aligned}$$



(а) (б)

Рис. 6.15. Пример обучения гауссиана по τ с фиксированным μ:
(а) выборка из априорного распределения $\mathrm{Gamma}\,(\tau|\alpha_0 = 1, \beta_0 = 1)$ и точки данных;
(б) выборка из апостериорного распределения $\mathrm{Gamma}\,(\tau|\alpha_n, \beta_n)$ для $n = 10$

У нас получилось, что логарифм апостериорного распределения опять выглядит как $a \log \tau - b\tau$, а значит, семейство гамма-распределений действительно является сопряжённым для этого семейства правдоподобий (гауссианы с параметром τ для фиксированного μ):

$$p\,(\tau|x_1, \ldots, x_n) = \mathrm{Gamma}\,(\tau|\alpha_n, \beta_n)\,, \quad \text{где} \tag{6.13}$$

$$\alpha_n = \alpha_0 + \frac{n}{2}, \qquad \beta_n = \beta_0 + \frac{1}{2}\sum_{i=1}^{n}(x_i - \mu)^2\,. \tag{6.14}$$

И снова обратите внимание, насколько проще все формулы выглядят в терминах τ: у нас получился понятный вид правдоподобия, соответствующий широко известному распределению, с которым удобно работать.

Если бы мы попытались записать всё то же самое в терминах σ, мы бы, конечно, в итоге пришли, по сути, к тому же самому, но весьма окольным путём. Про саму величину σ рассуждать совсем неудобно, а для σ^2 сопряжённым априорным распределением пришлось бы выбирать *обратное гамма-распределение* (inverse gamma distribution), которое определяется как распределение случайной величины, обратная к которой распределена по гамма-распределению... подождите, но может, проще будет к этой обратной и перейти? Именно это мы и сделали, перейдя к τ вместо σ.

Пример на этот раз показан на рис. 6.15. Мы использовали то же истинное распределение и те же точки данных, что на рис. 6.14, *а*, но теперь зафиксировали $\mu = 1$ и обновили τ по формулам (6.13); показанная на рис. 6.15, *б* выборка из апостериорного распределения уже выглядит близкой к истинному распределению.

6.4.2. Вывод для среднего и точности одновременно

Но всё это были лишь предварительные приготовления. Теперь давайте сделаем последний и самый главный шаг: попробуем построить сопряжённое априорное распределение сразу по *двум* параметрам, $p(\tau,\mu)$. У нас есть хорошие кандидаты на $p(\tau)$ и $p(\mu)$ по отдельности — может быть, можно просто объединить их в априорное распределение

$$p(\mu,\tau) = p(\tau)\,p(\mu) = \text{Gamma}(\tau|\alpha_0,\beta_0)\,\mathcal{N}(\mu|\mu_0,\tau_0)?$$

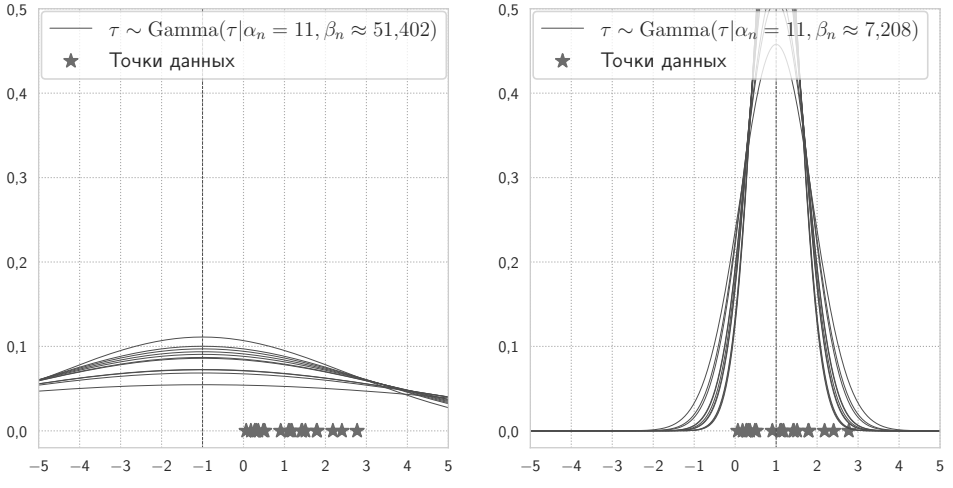
Давайте попробуем записать для такого априорного распределения байесовский вывод:

$$\log p(\tau,\mu|x_1,\ldots,x_n) = \text{Const} + \log p(\tau) + \log p(\mu) + \log p(x_1,\ldots,x_n|\mu) =$$
$$= \text{Const} - (\alpha_0 - 1)\log\tau - \beta_0\tau - \frac{\tau_0}{2}\left(\mu^2 - 2\mu\mu_0 + \mu_0^2\right) + \frac{n}{2}\log\tau - \frac{\tau}{2}\sum_{i=1}^{n}(x_i - \mu)^2\,.$$

Что-то не сходится, правда? В логарифме правдоподобия $\log p(x_1,\ldots,x_n|\mu,\tau)$ действительно есть слагаемое $\frac{n}{2}\log\tau$, которое отлично войдёт в гамма-распределение, и оно действительно представляет собой квадратичную функцию от μ, что хорошо подошло бы к гауссовскому априорному распределению на μ, но есть загвоздка: в логарифме правдоподобия τ и μ умножаются друг на друга. Иначе говоря, $\log p(x_1,\ldots,x_n|\mu,\tau)$ не раскладывается в сумму функции от μ и функции от τ, а значит, в апостериорном распределении $p(\mu,\tau|x_1,\ldots,x_n)$ случайные величины μ и τ будут *зависимы* и семейство априорных распределений с независимыми τ и μ никак не сможет быть сопряжённым этому семейству правдоподобий.

Этот эффект легко понять и интуитивно: представьте, что вы обучаете параметры гауссиана по набору точек, изображённому на рис. 6.16. Если мы зафиксируем $\mu = -1$, то в результате обучится большая дисперсия (маленькое τ), как на рис. 6.16, *а*, а если зафиксировать $\mu = 1$, то дисперсия будет гораздо меньше (а τ — больше), как на рис. 6.16, *б*. Это значит, что, даже если до получения данных μ и τ были независимы, первая же точка данных x устанавливает между ними зависимость.

Теперь ясно, что дело не в конкретных формах априорных распределений, а в предположении о независимости $p(\mu,\tau) = p(\tau)\,p(\mu)$. Но, к счастью, оказывается, что достаточно совсем немного изменить выбранное нами $p(\mu,\tau)$, чтобы



(а) (б)

Рис. 6.16. Обучение гауссиана по τ с двумя разными фиксированными μ и априорным распределением $\text{Gamma}\,(\tau|\alpha_0 = 1, \beta_0 = 1)$: (а) выборка из апостериорного распределения для $\mu = -1$; (б) выборка из апостериорного распределения для $\mu = 1$

всё получилось. Нам нужно учесть в логарифме правдоподобия величину вида точности τ, умноженной на квадратичную функцию от μ. Давайте запишем

$$p\,(\mu,\tau) = p\,(\tau)\,p\,(\mu|\tau)$$

и в качестве $p\,(\tau)$ опять возьмём гамма-распределение $p\,(\tau) = \text{Gamma}\,(\tau|\alpha_0,\beta_0)$, а в качестве $p\,(\mu|\tau)$ — всё тот же гауссиан, но теперь с точностью, пропорциональной τ:

$$p\,(\mu|\tau) = \mathcal{N}\,(\mu|\mu_0, \lambda_0\tau) = \sqrt{\frac{\lambda_0\tau}{2\pi}}e^{-\frac{\lambda_0\tau}{2}(\mu-\mu_0)^2}.$$

Теперь в нашем априорном распределении найдётся куда вписать $\mu\tau$ из логарифма правдоподобия. Давайте убедимся, что байесовский вывод проходит (мы по-прежнему убираем в константу всё что можно, но теперь можно только слагаемые, не зависящие ни от μ, ни от τ):

$$\begin{aligned}\log p\,(\tau,\mu|x_1,\ldots,x_n) &= \text{Const} + \log p\,(\tau) + \log p\,(\mu|\tau) + \log p\,(x_1,\ldots,x_n|\mu) = \\ &= \text{Const} - (\alpha_0 - 1)\log\tau - \beta_0\tau + \frac{1}{2}\log\tau - \frac{\lambda_0\tau_0}{2}\,(\mu-\mu_0)^2 + \frac{n}{2}\log\tau - \frac{\tau}{2}\sum_{i=1}^{n}(x_i-\mu)^2 = \\ &= \text{Const} + \left(\alpha_0 + \frac{n}{2} - 1\right)\log\tau + \frac{1}{2}\log\tau - \frac{\tau}{2}\left(\lambda_0\,(\mu-\mu_0)^2 + \sum_{i=1}^{n}(x_i-\mu)^2\right). \quad (6.15)\end{aligned}$$

Здесь мы оставили одно слагаемое с логарифмом $\frac{1}{2}\log\tau$ «на будущее», потому что оно позже понадобится для нового гауссиана по μ. Теперь нужно выделить полный квадрат в последней скобке:

$$\lambda_0(\mu-\mu_0)^2+\sum_{i=1}^{n}(x_i-\mu)^2=\lambda_0\mu^2-2\lambda_0\mu_0\mu+\lambda_0\mu_0^2+\sum_{i=1}^{n}x_i^2-2\mu\sum_{i=1}^{n}x_i+n\mu^2=$$
$$=(\lambda_0+n)\left(\mu-\frac{\lambda_0\mu_0+\sum_{i=1}^{n}x_i}{\lambda_0+n}\right)^2+\lambda_0\mu_0^2+\sum_{i=1}^{n}x_i^2-\frac{(\lambda_0\mu_0+\sum_{i=1}^{n}x_i)^2}{\lambda_0+n}. \tag{6.16}$$

Подставим результат (6.16) в выражение для логарифма апостериорного распределения (6.15):

$$\log p(\tau,\mu|x_1,\ldots,x_n)=\text{Const}+\left(\alpha_0+\frac{n}{2}-1\right)\log\tau+$$
$$+\frac{1}{2}\log\tau-\frac{(\lambda_0+n)\tau}{2}\left(\mu-\frac{\lambda_0\mu_0+\sum_{i=1}^{n}x_i}{\lambda_0+n}\right)^2+$$
$$+\frac{\tau}{2}\left(\lambda_0\mu_0^2+\sum_{i=1}^{n}x_i^2-\frac{(\lambda_0\mu_0+\sum_{i=1}^{n}x_i)^2}{\lambda_0+n}\right). \tag{6.17}$$

Видно, что теперь у нас всё получилось: вторая строчка в формуле (6.17) — это с точностью до константы логарифм плотности гауссиана $\mathcal{N}(\mu|\mu_n,\lambda_n\tau)$ с

$$\lambda_n=\lambda_0+n,\qquad \mu_n=\frac{\lambda_0\mu_0+\sum_{i=1}^{n}x_i}{\lambda_0+n}, \tag{6.18}$$

а первая и третья строчки представляют собой логарифм плотности гамма-распределения $\text{Gamma}(\tau|\alpha_n,\beta_n)$ для

$$\alpha_n=\alpha_0+\frac{n}{2},\qquad \beta_n=\lambda_0\mu_0^2+\sum_{i=1}^{n}x_i^2-\frac{(\lambda_0\mu_0+\sum_{i=1}^{n}x_i)^2}{\lambda_0+n}. \tag{6.19}$$

В итоге семейством сопряжённых априорных распределений для одномерного гауссиана с двумя неизвестными параметрами μ и τ является семейство распределений вида

$$p(\mu,\tau)=p(\tau)\,p(\mu|\tau)=\text{Gamma}(\tau|\alpha_0,\beta_0)\,\mathcal{N}(\mu|\mu_0,\lambda_0\tau),$$

и после байесовского вывода для набора данных $D=\{x_1,\ldots,x_n\}$ получается распределение из того же семейства

$$p(\mu,\tau|x_1,\ldots,x_n)=\text{Gamma}(\tau|\alpha_n,\beta_n)\,\mathcal{N}(\mu|\mu_n,\lambda_n\tau),$$

параметры которого подсчитаны в формулах (6.18) и (6.19).

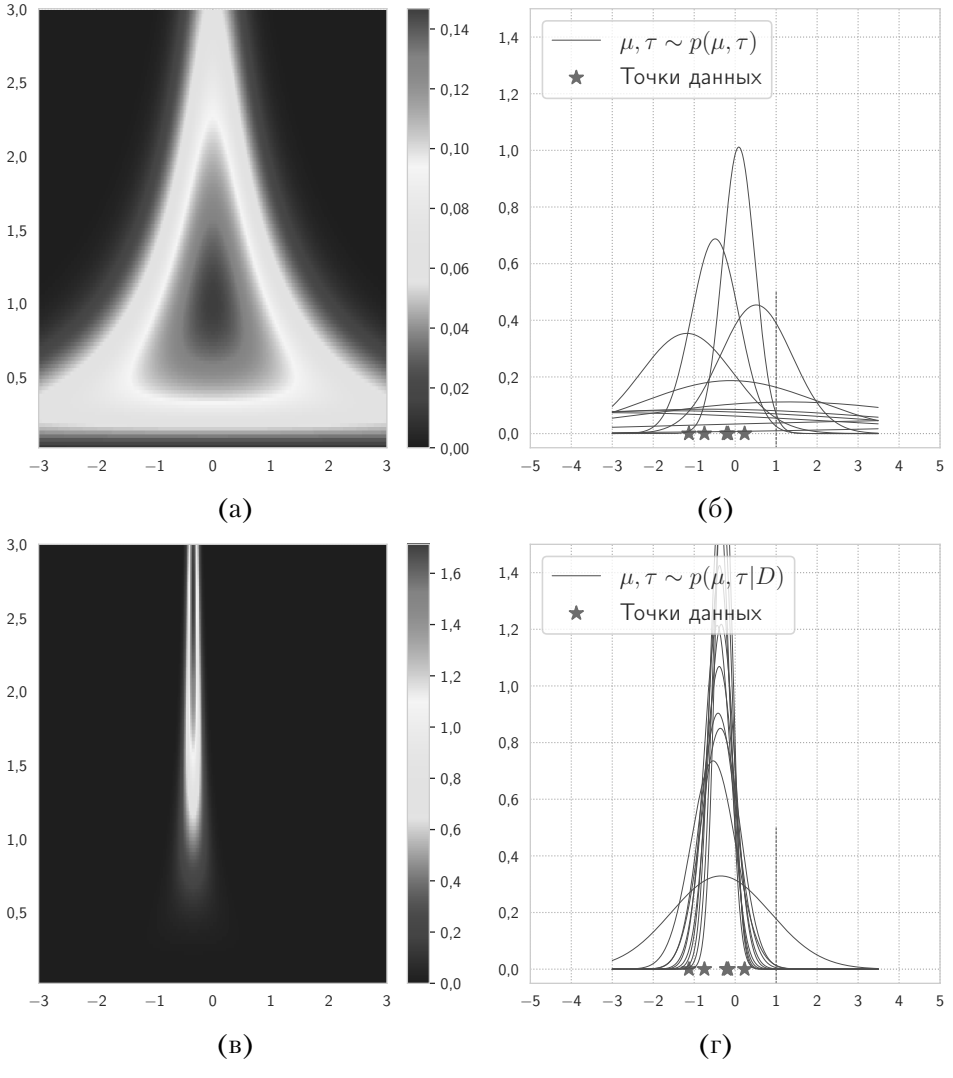


Рис. 6.17. Обучение гауссиана по μ и τ: (а) априорное распределение $p(\mu,\tau) = \text{Gamma}(\tau|\alpha_0,\beta_0)\mathcal{N}(\mu|\mu_0,\lambda_0\tau)$; (б) выборка из априорного распределения и данные; (в) апостериорное распределение $p(\mu,\tau|D) = \text{Gamma}(\tau|\alpha_n,\beta_n)\mathcal{N}(\mu|\mu_n,\lambda_n\tau)$; (г) выборка из апостериорного распределения и данные

Пример полного обучения гауссиана показан на рис. 6.17. Теперь у нас двумерное распределение $p(\mu,\tau)$, так что опять, как в разделе 4.4, придётся прибегнуть к двумерным тепловым картам. На рис. 6.17, *а* показано априорное распределение $p(\mu,\tau)$, произведение гамма-распределения и гауссиана, а рис. 6.17, *б*

показывает примеры гауссианов, параметры которых взяты из этого распределения, — как видите, оно весьма разнообразно. Однако после обучения на пяти точках, показанных на рис. 6.17, *б*, апостериорное распределение $p(\mu,\tau|D)$ становится гораздо более сосредоточенным. Примеры гауссианов, семплированных из этого распределения, показаны на рис. 6.17, *г* — они тоже стали задаваться гораздо более однозначно.

6.4.3. Маргинализация апостериорного распределения

Ещё один очень интересный вывод об апостериорном распределении для одномерного гауссиана получится, если мы попробуем маргинализовать это распределение, то есть попытаемся понять, что нам говорит апостериорное распределение $p(\mu,\tau|D)$ отдельно про μ или отдельно про τ.

Давайте маргинализуем апостериорное распределение по τ, вынося за интеграл всё, что от τ не зависит, и пряча в константу всё, что не зависит от μ:

$$\begin{aligned} p(\mu|D) = \int_0^\infty p(\mu,\tau|D)\,\mathrm{d}\tau = \int_0^\infty \mathrm{Gamma}(\tau|\alpha_n,\beta_n)\,\mathcal{N}(\mu|\mu_n,\lambda_n\tau)\,\mathrm{d}\tau = \\ = \mathrm{Const}\cdot\int_0^\infty \tau^{\alpha_n-1}e^{-\beta_n\tau}\sqrt{\tau}e^{-\frac{\lambda_n\tau}{2}(\mu-\mu_n)^2}\mathrm{d}\tau = \\ = \mathrm{Const}\cdot\int_0^\infty \tau^{\left(\alpha_n+\frac{1}{2}\right)-1}e^{-\left(\beta_n+\frac{\lambda_n}{2}(\mu-\mu_n)^2\right)\tau}\mathrm{d}\tau. \end{aligned}$$

У нас получился интеграл от величины, пропорциональной плотности гамма-распределения, так что результатом интегрирования будет нормировочная константа этого самого распределения:

$$\begin{aligned} p(\mu|D) \propto \left(\beta_n + \frac{\lambda_n}{2}(\mu-\mu_n)^2\right)^{-\frac{\alpha_n+1}{2}} \Gamma\left(\frac{\alpha_n+1}{2}\right) \propto \\ \propto \left(1+\frac{\lambda_n}{2\beta_n}(\mu-\mu_n)^2\right)^{-\frac{\alpha_n+1}{2}}. \end{aligned} \quad (6.20)$$

Выходит, что после маргинализации по τ в ответе получается уже не гауссиан, как всё время до сих пор получалось в распределениях на μ, а распределение Стьюдента! Иначе говоря, вместо распределения с экспоненциально затухающими «лёгкими хвостами» получилось распределение гораздо более пологое, с полиномиальными «тяжёлыми хвостами», что опять подтверждает общий принцип: переход к байесовскому выводу обычно не слишком меняет точечные оценки, но делает нашу уверенность в выводах меньше. В данном случае «меньше» — это уже даже не просто то же самое распределение с увеличенной дисперсией, как получалось в главе 4 с линейной регрессией и главе 5 с логистической, а распределение из совсем другого семейства, плотность которого имеет другую

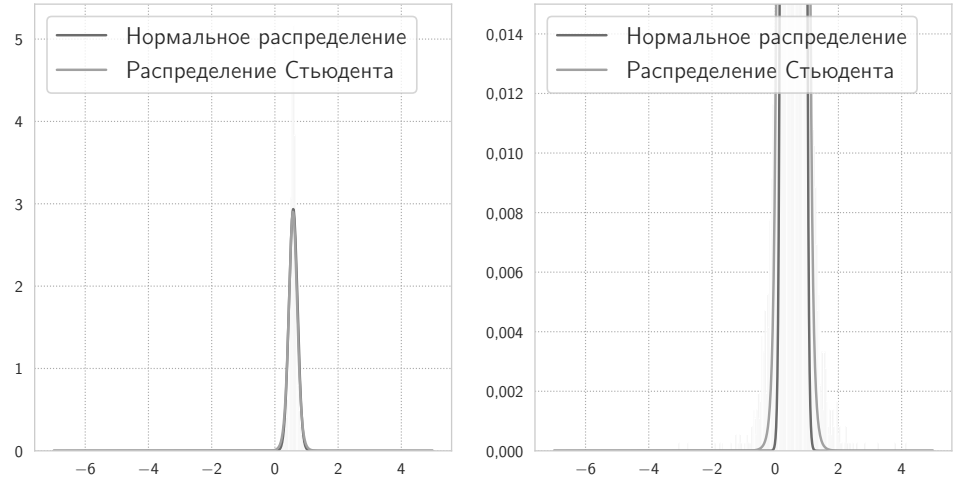


Рис. 6.18. Распределение значений μ в выборке из апостериорного распределения $p(\mu,\tau|D)$ и обученных по ней нормального распределения и распределения Стьюдента: (а) кривые целиком; (б) «хвосты»

асимптотику и которое гораздо более склонно к выбросам. Результат (6.20) — это и есть тот самый вывод, который я упоминал в разделе 3.2: из этих соображений Хельмерт [252, 253, 534] и Люрот [358] ввели распределение Стьюдента ещё в 1870-х годах.

Проиллюстрировать этот результат на практике несложно: у нас уже есть параметры апостериорного распределения, показанного на рис. 6.17, *в*: $\alpha_n = 3{,}5$, $\beta_n \approx 1{,}59$, $\mu_n \approx 0{,}58$, $\lambda_n = 6$, и мы умеем семплировать из этого распределения, так что для того, чтобы получить выборку из маргинала $p(\mu|D) = \int p(\mu,\tau|D)\,\mathrm{d}\tau$, нужно просто выбросить из этой выборки τ. Давайте возьмём выборку побольше, чтобы в ней наверняка видны были «хвосты», а затем обучим на ней для сравнения нормальное распределение и распределение Стьюдента. Параметры нормального распределения — это просто среднее и дисперсия полученного массива значений μ, а для обучения распределения Стьюдента воспользуемся стандартным методом из `scipy.stats`:

```
def sample_mt(alpha, beta, mu, lmb):
    tau = np.random.gamma(shape=alpha, scale=1./beta)
    m = np.random.normal(loc=mu, scale=1./(lmb*tau))
    return m, tau

mus = np.array([ sample_mt(a, b, m, l)[0] for _ in range(100000) ])
theta = st.t.fit(mus)
```

Результат изображён на рис. 6.18. На рис. 6.18, *а* уже видно, что распределение Стьюдента подходит лучше, но «хвосты» не видны, они слишком маленькие, так что рис. 6.18, *б* показывает те же кривые в более крупном масштабе. Видно, что распределение Стьюдента описывает полученную выборку значений μ практически идеально, а нормальное распределение затухает гораздо быстрее, чем нужно, что подтверждает наши теоретические выкладки.

6.4.4. Предсказательное распределение для гауссиана

А теперь третья, самая интересная задача байесовского вывода: давайте найдём предсказательное распределение для неизвестных параметров нормального распределения. Для этого сначала проведём вспомогательное вычисление — подсчитаем свёртку двух гауссианов аналогично формуле (4.28), но в терминах τ:

$$\int_{-\infty}^{\infty}\mathcal{N}\left(x|\mu,\tau\right)\mathcal{N}\left(\mu|\mu_0,\tau_0\right)\mathrm{d}\mu=\frac{\sqrt{\tau\tau_0}}{2\pi}\int_{-\infty}^{\infty}e^{-\frac{\tau}{2}(x-\mu)^2-\frac{\tau_0}{2}(\mu-\mu_0)^2}\mathrm{d}\mu=$$

$$=\frac{\sqrt{\tau\tau_0}}{2\pi}e^{-\frac{1}{2}\left(x^2\tau+\mu_0^2\tau_0-\frac{(x\tau+\mu_0\tau_0)^2}{\tau+\tau_0}\right)}\int_{-\infty}^{\infty}e^{-\frac{\tau+\tau_0}{2}\left(\mu-\frac{x\tau+\mu_0\tau_0}{\tau+\tau_0}\right)^2}\mathrm{d}\mu=$$

$$=\frac{\sqrt{\tau\tau_0}}{2\pi}e^{-\frac{1}{2}\left(x^2\tau+\mu_0^2\tau_0-\frac{(x\tau+\mu_0\tau_0)^2}{\tau+\tau_0}\right)}\sqrt{\frac{2\pi}{\tau+\tau_0}}=$$

$$=\sqrt{\frac{\tau\tau_0}{2\pi\left(\tau+\tau_0\right)}}e^{-\frac{1}{2}\left(x^2\left(\tau-\frac{\tau^2}{\tau+\tau_0}\right)-2x\frac{\tau\tau_0\mu_0}{\tau+\tau_0}+\mu_0^2\tau_0-\frac{\mu_0^2\tau_0^2}{\tau+\tau_0}\right)},$$

то есть получается вполне ожидаемый результат:

$$\int_{-\infty}^{\infty}\mathcal{N}\left(x|\mu,\tau\right)\mathcal{N}\left(\mu|\mu_0,\tau_0\right)\mathrm{d}\mu=\mathcal{N}\left(x\middle|\mu_0,\frac{\tau\tau_0}{\tau+\tau_0}\right). \tag{6.21}$$

Теперь можно подсчитать предсказательное распределение по (6.21):

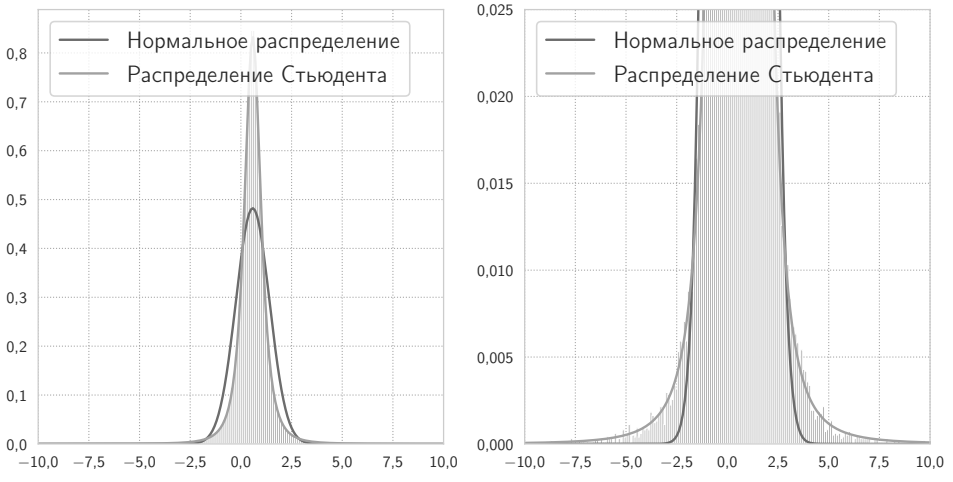
$$p\left(x|D\right)=\int_0^{\infty}\int_{-\infty}^{\infty}p\left(x|\mu,\tau\right)p\left(\mu,\tau|D\right)\mathrm{d}\mu\mathrm{d}\tau=$$

$$=\int_0^{\infty}\int_{-\infty}^{\infty}\mathcal{N}\left(x|\mu,\tau\right)\mathrm{Gamma}\left(\tau|\alpha_n,\beta_n\right)\mathcal{N}\left(\mu|\mu_n,\lambda_n\tau\right)\mathrm{d}\mu\mathrm{d}\tau=$$

$$=\int_0^{\infty}\mathrm{Gamma}\left(\tau|\alpha_n,\beta_n\right)\left[\int_{-\infty}^{\infty}\mathcal{N}\left(x|\mu,\tau\right)\mathcal{N}\left(\mu|\mu_n,\lambda_n\tau\right)\mathrm{d}\mu\right]\mathrm{d}\tau=$$

$$=\int_0^{\infty}\mathrm{Gamma}\left(\tau|\alpha_n,\beta_n\right)\mathcal{N}\left(x\middle|\mu_n,\frac{\lambda_n}{\lambda_n+1}\tau\right)\mathrm{d}\tau,$$

а затем воспользоваться результатом (6.20), заменив в нём λ_n на $\frac{\lambda_n}{\lambda_n+1}$:



(а) (б)

Рис. 6.19. Распределение значений x в выборке из предсказательного распределения $p(x|D)$ и обученных по ней нормального распределения и распределения Стьюдента: (а) кривые целиком; (б) «хвосты»

$$p(x|D) \propto \left(1 + \frac{\lambda_n}{2\beta_n(\lambda_n+1)}(\mu-\mu_n)^2\right)^{-\frac{\alpha_n+1}{2}}. \tag{6.22}$$

Иначе говоря, предсказательное распределение для новых точек гауссиана, в котором не известно ни μ, ни σ, — это опять же вовсе не гауссиан, а распределение Стьюдента, то есть распределение с асимптотически гораздо более пологими «хвостами».

Этот результат тоже легко проиллюстрировать — нужно взять ту же выборку из апостериорного распределения, из которой мы выше брали параметры μ, и использовать её как параметры нормальных распределений:

```
mts = np.array([ sample_mt(a, b, m, l) for _ in range(100000) ])
new_xs = np.array([ np.random.normal(loc=params[0], scale=1./params[1]) \
  for params in mts ])
theta = st.t.fit(new_xs)
```

Результат показан на рис. 6.19, снова полностью на рис. 6.19, *а* и в укрупнённом виде на рис. 6.19, *б*. Мы снова видим идеально подходящую кривую распределения Стьюдента и совсем не так хорошо подходящую кривую нормального распределения.

На этом наш анализ одномерного гауссиана заканчивается; конечно, аналогичные результаты можно было бы получить и в многомерном случае [66, 419].

Мы уже не раз видели, как при байесовском усреднении в предсказательном распределении наша уверенность уменьшается; в разделе 2.4 появилось сглаживание по Лапласу, в подразделе 4.4.3 увеличилась дисперсия нормального распределения в предсказаниях линейной регрессии, а в подразделе 5.3.3 стали менее уверенными предсказания регрессии логистической. Но сейчас эффект вышел на новый уровень: оказалось, что в байесовском предсказании получается совсем другое распределение, с другой асимптотикой убывания плотности, а не просто с увеличенной дисперсией. Этот эффект может возникнуть и в более сложных моделях.

Далее мы переходим от самого́ байесовского вывода, который всё-таки происходит в рамках одной модели, к мета-задаче: как выбрать модель из нескольких вариантов? Выбор модели может происходить как в форме подбора гиперпараметров, так и просто в форме сравнения моделей разной структуры; в следующем разделе мы рассмотрим оба варианта.

6.5. Оценки $\mathbf{p(D)}$: эмпирический Байес и сравнение моделей

Бердяевская критика современной действительности и выставленных ею идеалов так сурова, его требования так велики, что, казалось бы, те действия, которые он проповедует, должны быть воистину чудесны, сверхэмпиричны. Но, увы, лишь только ему пришлось столкнуться с реальной действительностью, как «мистический реалист» превращается в самого обыкновенного эмпирика.

Д. М. Философов. На распутье

6.5.1. Маргинальное правдоподобие и подбор гиперпараметров

Мы много говорили о том, как обучать параметры моделей машинного обучения, но пока совершенно не затрагивали другой естественный вопрос: откуда возьмутся *гипер*параметры? Давайте обсудим это на примере всё той же линейной регрессии. У гребневой регрессии, которую мы обсуждали в разделе 4.4, два гиперпараметра: точность априорного распределения α (само априорное распределение пусть имеет вид $p(\boldsymbol{w}|\alpha) = \mathcal{N}(\boldsymbol{w}|\mathbf{O}, \alpha\mathbf{I})$) и точность распределения шума β; нам будет удобнее работать с ней, а не с дисперсией шума σ^2, которая фигурировала в главе 4. Логарифм плотности апостериорного распределения линейной регрессии, то есть формула (4.21), в этих обозначениях имеет вид

$$\ln p(\boldsymbol{w}|\boldsymbol{t}, \boldsymbol{X}, \alpha, \beta) = -\frac{\beta}{2}\sum_{n=1}^{N}\left(t_n - \boldsymbol{w}^\top \boldsymbol{x}_n\right)^2 - \frac{\alpha}{2}\boldsymbol{w}^\top\boldsymbol{w} + \text{Const}.$$

Откуда же взять значения α и β? Идея простая и изящная: давайте попробуем сравнивать друг с другом значения $p(D|\alpha, \beta)$, то есть значения знаменателя теоремы Байеса

$$p(\boldsymbol{w}|\boldsymbol{t}, \boldsymbol{X}, \alpha, \beta) = \frac{p(\boldsymbol{w}|\alpha)\, p(\boldsymbol{t}|\boldsymbol{X}, \boldsymbol{w}, \beta)}{p(\boldsymbol{t}|\boldsymbol{X}, \alpha, \beta)}. \tag{6.23}$$

Знаменатель этот называется *маргинальным правдоподобием* (marginal likelihood) или *функцией свидетельства* (evidence function). Полностью байесовский вывод для гиперпараметров подразумевал бы введение априорного распределения $p(\alpha,\beta)$ на гиперпараметры α и β и поиск ещё и апостериорного распределения $p(\alpha,\beta|\boldsymbol{t}, \boldsymbol{X})$. А *эмпирический метод Байеса* (empirical Bayes), который мы

обсуждаем в этом разделе, подразумевает просто максимизацию функции маргинального правдоподобия $p(\boldsymbol{t}|\boldsymbol{X}, \alpha, \beta)$ по гиперпараметрам, то есть как бы максимизацию правдоподобия «на втором уровне»; иногда эмпирический метод Байеса так и называют: метод максимизации правдоподобия второго типа (type II maximum likelihood, MLE-II). Он является немного «нечестным», потому что использует для подбора гиперпараметров тот же датасет $D = (\boldsymbol{X}, \boldsymbol{t})$, который потом будет использоваться для обучения самих параметров $\boldsymbol{w}$. Но зачастую выбора нет, а эмпирический метод Байеса всё-таки оказывается полезным.

Мы пока только один раз встречались с вычислением маргинального правдоподобия: в разделе 2.4 мы подсчитали $p(D)$ для монетки, что помогло нам построить предсказательное распределение. Вообще говоря, считать константу интегрированием, как мы делали тогда, было не обязательно: мы могли просто найти величины, пропорциональные $p(\text{орёл}|D)$ и $p(\text{решка}|D)$, по отдельности, а затем нормировать результат. Но на самом деле подсчёт $p(D)$ действительно очень похож на вычисление предсказательного распределения. В случае линейной регрессии нам нужно взять интеграл

$$p(\boldsymbol{t}|\boldsymbol{X}, \alpha, \beta) = \int p(\boldsymbol{w}|\alpha)\, p(\boldsymbol{t}|\boldsymbol{X}, \boldsymbol{w}, \beta)\, \mathrm{d}\boldsymbol{w} = \\ = \left(\frac{\alpha}{2\pi}\right)^{\frac{d}{2}} \left(\frac{\beta}{2\pi}\right)^{\frac{N}{2}} \int e^{-\frac{\beta}{2}(\boldsymbol{t}-\boldsymbol{X}\boldsymbol{w})^\top(\boldsymbol{t}-\boldsymbol{X}\boldsymbol{w}) - \frac{\alpha}{2}\boldsymbol{w}^\top\boldsymbol{w}} \mathrm{d}\boldsymbol{w}. \quad (6.24)$$

Для этого нужно, в точности как в разделе 4.4, выделить полный квадрат в экспоненте. Мы могли бы буквально воспользоваться формулами (4.23) и (4.24), но нам теперь нельзя забывать про все константы, не зависящие от $\boldsymbol{w}$, поэтому придётся вывод вкратце повторить:

$$-\frac{\beta}{2}(\boldsymbol{t}-\boldsymbol{X}\boldsymbol{w})^\top(\boldsymbol{t}-\boldsymbol{X}\boldsymbol{w}) - \frac{\alpha}{2}\boldsymbol{w}^\top\boldsymbol{w} = \\ = -\frac{\beta}{2}\boldsymbol{t}^\top\boldsymbol{t} + \beta\boldsymbol{w}^\top\boldsymbol{X}^\top\boldsymbol{t} - \frac{1}{2}\boldsymbol{w}^\top\left(\beta\boldsymbol{X}^\top\boldsymbol{X} + \alpha\mathbf{I}\right)\boldsymbol{w} = \\ = -\frac{1}{2}(\boldsymbol{w}-\boldsymbol{\mu}_N)^\top\mathbf{A}(\boldsymbol{w}-\boldsymbol{\mu}_N) - \frac{\beta}{2}\boldsymbol{t}^\top\boldsymbol{t} + \frac{1}{2}\boldsymbol{\mu}_N^\top\mathbf{A}\boldsymbol{\mu}_N,$$

где, как и в формулах (4.23) и (4.24),

$$\mathbf{A} = \beta\boldsymbol{X}^\top\boldsymbol{X} + \alpha\mathbf{I}, \qquad \boldsymbol{\mu}_N = \beta\mathbf{A}^{-1}\boldsymbol{X}^\top\boldsymbol{t}.$$

Теперь мы можем проинтегрировать

$$\int e^{-\frac{1}{2}(\boldsymbol{w}-\boldsymbol{\mu}_N)^\top\mathbf{A}(\boldsymbol{w}-\boldsymbol{\mu}_N)}\mathrm{d}\boldsymbol{w} = \sqrt{\frac{(2\pi)^d}{\det\mathbf{A}}}$$

и получить в итоге

$$p\left(\boldsymbol{t}|\boldsymbol{X},\alpha,\beta\right)=\left(\frac{\alpha}{2\pi}\right)^{\frac{d}{2}}\left(\frac{\beta}{2\pi}\right)^{\frac{N}{2}}\sqrt{\frac{(2\pi)^d}{\det\mathbf{A}}}e^{-\frac{\beta}{2}\boldsymbol{t}^\top\boldsymbol{t}+\frac{1}{2}\boldsymbol{\mu}_N^\top\mathbf{A}\boldsymbol{\mu}_N},$$

то есть

$$\begin{aligned}\log p\left(\boldsymbol{t}|\boldsymbol{X},\alpha,\beta\right)=\\ =\frac{d}{2}\log\alpha+\frac{N}{2}\log\beta-\frac{1}{2}\log\det\mathbf{A}-\frac{\beta}{2}\boldsymbol{t}^\top\boldsymbol{t}+\frac{1}{2}\boldsymbol{\mu}_N^\top\mathbf{A}\boldsymbol{\mu}_N-\frac{N}{2}\log 2\pi.\end{aligned}\quad(6.25)$$

На этом можно было бы остановиться, но давайте преобразуем выражение (6.25) ещё и к другой форме. Заметим, что

$$\beta\boldsymbol{\mu}_N^\top\boldsymbol{X}^\top\boldsymbol{t}=\boldsymbol{\mu}_N^\top\mathbf{A}\left(\beta\mathbf{A}^{-1}\boldsymbol{X}^\top\boldsymbol{t}\right)=\boldsymbol{\mu}_N^\top\mathbf{A}\boldsymbol{\mu}_N=\alpha\boldsymbol{\mu}_N^\top\boldsymbol{\mu}_N+\beta\boldsymbol{\mu}_N^\top\boldsymbol{X}^\top\boldsymbol{X}\boldsymbol{\mu}_N,$$

а значит,

$$\begin{aligned}-\frac{\beta}{2}\boldsymbol{t}^\top\boldsymbol{t}+\frac{1}{2}\boldsymbol{\mu}_N^\top\mathbf{A}\boldsymbol{\mu}_N=-\frac{\beta}{2}\boldsymbol{t}^\top\boldsymbol{t}+\frac{\alpha}{2}\boldsymbol{\mu}_N^\top\boldsymbol{\mu}_N+\frac{\beta}{2}\boldsymbol{\mu}_N^\top\boldsymbol{X}^\top\boldsymbol{X}\boldsymbol{\mu}_N=\\ =-\tfrac{\beta}{2}\left(\boldsymbol{t}^\top\boldsymbol{t}-2\boldsymbol{\mu}_N^\top\boldsymbol{X}^\top\boldsymbol{t}+\boldsymbol{\mu}_N^\top\boldsymbol{X}^\top\boldsymbol{X}\boldsymbol{\mu}_N\right)-\beta\boldsymbol{\mu}_N^\top\boldsymbol{X}^\top\boldsymbol{t}+\beta\boldsymbol{\mu}_N^\top\boldsymbol{X}^\top\boldsymbol{X}\boldsymbol{\mu}_N+\tfrac{\alpha}{2}\boldsymbol{\mu}_N^\top\boldsymbol{\mu}_N=\\ =-\frac{\beta}{2}\left(\boldsymbol{t}-\boldsymbol{X}\boldsymbol{\mu}_N\right)^\top\left(\boldsymbol{t}-\boldsymbol{X}\boldsymbol{\mu}_N\right)-\boldsymbol{\mu}_N^\top\mathbf{A}\boldsymbol{\mu}_N+\boldsymbol{\mu}_N^\top\mathbf{A}\boldsymbol{\mu}_N-\frac{\alpha}{2}\boldsymbol{\mu}_N^\top\boldsymbol{\mu}_N=\\ =-\frac{\beta}{2}\left(\boldsymbol{t}-\boldsymbol{X}\boldsymbol{\mu}_N\right)^\top\left(\boldsymbol{t}-\boldsymbol{X}\boldsymbol{\mu}_N\right)-\frac{\alpha}{2}\boldsymbol{\mu}_N^\top\boldsymbol{\mu}_N,\end{aligned}$$

и мы теперь можем переписать (6.25) как

$$\begin{aligned}\log p\left(\boldsymbol{t}|\boldsymbol{X},\alpha,\beta\right)=\frac{d}{2}\log\alpha+\frac{N}{2}\log\beta-\frac{1}{2}\log\det\mathbf{A}-\\ -\frac{\beta}{2}\left(\boldsymbol{t}-\boldsymbol{X}\boldsymbol{\mu}_N\right)^\top\left(\boldsymbol{t}-\boldsymbol{X}\boldsymbol{\mu}_N\right)-\frac{\alpha}{2}\boldsymbol{\mu}_N^\top\boldsymbol{\mu}_N-\frac{N}{2}\log 2\pi.\end{aligned}\quad(6.26)$$

Теперь в нашем результате (6.26) опять появляется общая форма «энергии», которая уже встречалась в формуле (6.24):

$$E(\boldsymbol{w})=-\frac{\beta}{2}\left(\boldsymbol{t}-\boldsymbol{X}\boldsymbol{w}\right)^\top\left(\boldsymbol{t}-\boldsymbol{X}\boldsymbol{w}\right)-\frac{\alpha}{2}\boldsymbol{w}^\top\boldsymbol{w},$$

а выше мы фактически доказали формулу преобразования таких «энергий» друг в друга при смене аргумента:

$$E(\boldsymbol{w})=E(\boldsymbol{\mu}_n)+\frac{1}{2}\left(\boldsymbol{w}-\boldsymbol{\mu}_N\right)^\top\mathbf{A}\left(\boldsymbol{w}-\boldsymbol{\mu}_N\right).\quad(6.27)$$

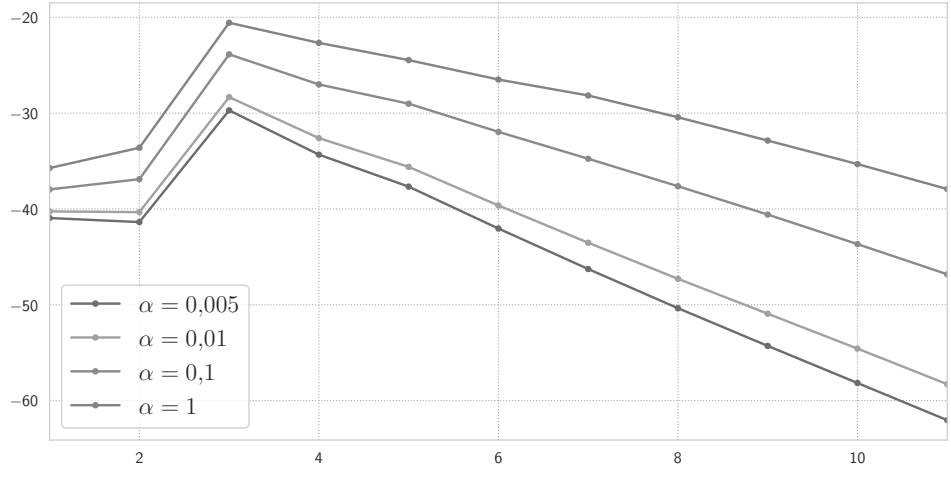


Рис. 6.20. Значения $\log p(\boldsymbol{t}|\boldsymbol{X}, \alpha, \beta)$ как функция от степени полиномиальной регрессии для нескольких разных значений α и $\beta = \frac{1}{4}$

Теперь мы можем использовать формулы (6.25) и (6.26) для того, чтобы аналитически находить значения $p(\boldsymbol{t}|\boldsymbol{X}, \alpha, \beta)$ для разных вариантов линейной регрессии и тем самым сравнивать их между собой.

```
def bayesian_update(mu, Sigma, X, t, beta=4):
    Sigma_inv = np.linalg.inv(Sigma)
    Sigma_n_inv = Sigma_inv + beta * X.T.dot(X)
    Sigma_n = np.linalg.inv(Sigma_n_inv)
    mu_n = Sigma_n.dot( Sigma_inv.dot(mu.T) + beta * X.T.dot(t) )
    return mu_n, Sigma_n

def log_marginal_likelihood(X, t, alpha, beta):
    N, M = X.shape
    mu, sigma = bayesian_update(np.zeros(M), \
                (1/alpha) * np.identity(M), X, t, beta=beta)
    A = np.linalg.inv(sigma)
    return 0.5 * ( M * np.log(alpha) + N * np.log(beta) - \
                np.log(np.linalg.det(A)) - \
                beta * np.sum((t - X.dot(mu)) ** 2) - \
                alpha * np.sum(mu ** 2) - \
                N * np.log(2 * np.pi) )
```

Напрямую перебирать $p(\boldsymbol{t}|\boldsymbol{X}, \alpha, \beta)$ разумно для гиперпараметров с дискретными множествами значений; в нашем случае это можно сделать, например, для разных значений степени переменной d.

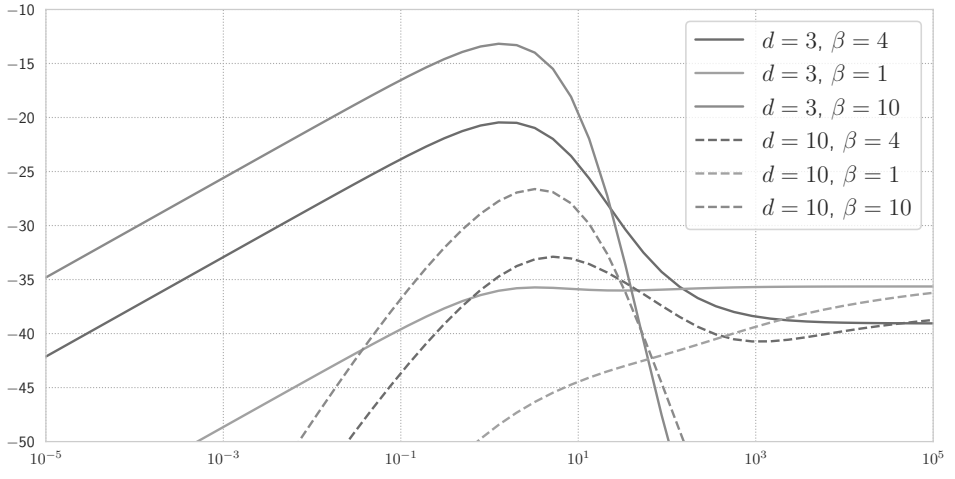


Рис. 6.21. Значения $\log p\left(\boldsymbol{t}|\boldsymbol{X}, \alpha, \beta\right)$ как функция от гиперпараметра α для нескольких разных значений d и β

На рис. 6.20 показано, что будет получаться в нашем вечном примере полиномиальной регрессии, если постепенно увеличивать степень полинома d. Для всех показанных на рис. 6.20 значений α мы видим одно и то же поведение кривой: переход от $d = 1$ к $d = 2$ уменьшает $p\left(\boldsymbol{t}|\boldsymbol{X}, \alpha, \beta\right)$, затем наилучшей степенью многочлена оказывается $d = 3$, что мы и ожидаем интуитивно от аппроксимации одного периода синусоиды, а потом с ростом d значения $p\left(\boldsymbol{t}|\boldsymbol{X}, \alpha, \beta\right)$ стабильно уменьшаются.

А что делать с непрерывными параметрами α и β? Можно, конечно, попробовать покрыть их значения сеткой с достаточно малым шагом — рис. 6.20 намекает, что мы вряд ли можем ожидать монотонной или выпуклой зависимости, так что сетка понадобится. На рис. 6.21 показан пример такой зависимости для нескольких вариантов d и β и для переменного гиперпараметра α. Для разумных случаев вроде $d = 3$, $\beta = 4$ видно, что по α у логарифма маргинального правдоподобия есть чёткий максимум, нужно его только найти; да и по β, судя по взаимному расположению кривых, максимум скорее всего найдётся.

Интересный эффект виден на рис. 6.21 для некоторых комбинаций d и β, например для случая заведомого оверфиттинга $d = 10$, $\beta = 4$: логарифм маргинального правдоподобия продолжает расти с ростом α даже в область астрономических значений. Это значит, что в таком случае с точки зрения $p\left(\boldsymbol{t}|\boldsymbol{X}, \alpha, \beta\right)$ выгоднее зарегуляризовать веса полиномиальной регрессии до предела, устремив точность к бесконечности, то есть, по сути, приравняв веса к нулю. Сейчас мы просто отметим этот эффект, но когда-нибудь (не в этой книге) ещё вернёмся к нему, когда будем обсуждать метод релевантных векторов (relevance vector

machines, RVM): в нём беспредельное увеличение точности α используется для того, чтобы обнулить не все, а *некоторые* веса и добавить в обучаемую модель эффект *разреженности*.

Можно теперь искать максимум для $d = 3$, подбирая α и β по сетке, но в данном случае можно оптимизировать и более эффективным образом. Главная загвоздка здесь — определитель матрицы $\mathbf{A}$, который из формулы (6.26) никуда не денется и который зависит и от α, и от β каким-то на первый взгляд очень сложным образом. Но мы сможем справиться с определителем, если вспомним из курса алгебры, что он равен произведению собственных чисел матрицы $\mathbf{A}$[1]. Давайте обозначим через λ_i собственные числа матрицы $\beta \boldsymbol{X}^\top \boldsymbol{X}$:

$$\left(\beta \boldsymbol{X}^\top \boldsymbol{X}\right) \boldsymbol{u}_i = \lambda_i \boldsymbol{u}_i$$

для некоторых собственных векторов $\boldsymbol{u}_i$. Тогда $\boldsymbol{u}_i$ будут являться и собственными векторами матрицы $\mathbf{A}$, с собственными числами $\alpha + \lambda_i$:

$$\mathbf{A}\boldsymbol{u}_i = \left(\beta \boldsymbol{X}^\top \boldsymbol{X} + \alpha \mathbf{I}\right) \boldsymbol{u}_i = \lambda_i \boldsymbol{u}_i + \alpha \boldsymbol{u}_i.$$

Теперь давайте оптимизировать логарифм маргинального правдоподобия $\log p(\boldsymbol{t}|\boldsymbol{X}, \alpha, \beta)$. Возьмём производную по α от логарифма определителя $\mathbf{A}$:

$$\frac{\partial \log \det \mathbf{A}}{\partial \alpha} = \frac{\partial \log \prod_i (\alpha + \lambda_i)}{\partial \alpha} = \sum_i \frac{\partial \log(\alpha + \lambda_i)}{\partial \alpha} = \sum_i \frac{1}{\alpha + \lambda_i},$$

и вся производная будет равна

$$\frac{\partial \log p(\boldsymbol{t}|\boldsymbol{X}, \alpha, \beta)}{\partial \alpha} = \frac{M}{2\alpha} - \frac{1}{2}\boldsymbol{\mu}_N^\top \boldsymbol{\mu}_n - \frac{1}{2}\sum_i \frac{1}{\alpha + \lambda_i}.$$

Если приравнять производную к нулю, мы получим уравнение вида

$$\alpha \boldsymbol{\mu}_N^\top \boldsymbol{\mu}_N = M - \alpha \sum_i \frac{1}{\alpha + \lambda_i} = \sum_i \frac{\lambda_i}{\alpha + \lambda_i}.$$

Решить его аналитически не получится не только потому, что правая его часть зависит от α, но и потому, что $\boldsymbol{\mu}_N$ тоже зависит от α, причём не самым тривиальным образом. Так что, если мы хотим оптимизировать α, нам придётся использовать метод простых итераций: начать с какого-нибудь $\alpha^{(0)}$, а потом на очередном шаге оптимизации обновлять

[1] Чтобы это увидеть, достаточно вспомнить, что собственные числа λ_i — это корни характеристического многочлена с учётом кратности, то есть он представляется в виде произведения мономов $(x - \lambda_i)$. Но при этом характеристический многочлен определяется как $\det(\mathbf{A} - x\mathbf{I})$, то есть при $x = 0$ он равен $\det \mathbf{A}$, с одной стороны, и произведению λ_i (с точностью до знака), с другой стороны.

$$\alpha^{(k+1)} = \frac{1}{\boldsymbol{\mu}_N\left(\alpha^{(k)}\right)^\top \boldsymbol{\mu}_N\left(\alpha^{(k)}\right)} \sum_i \frac{\lambda_i}{\alpha^{(k)} + \lambda_i}. \tag{6.28}$$

Аналогичный трюк можно провернуть и с оптимизацией по β. Для этого заметим, что λ_i — это собственные числа матрицы $\beta \boldsymbol{X}^\top \boldsymbol{X}$, а значит, они пропорциональны β. Поэтому $\frac{\partial \lambda_i}{\partial \beta} = \frac{\lambda_i}{\beta}$, и дальше мы аналогичным образом получаем

$$\frac{\partial \log \det \mathbf{A}}{\partial \beta} = \frac{\partial \log \prod_i (\alpha + \lambda_i)}{\partial \beta} = \sum\nolimits_i \frac{\partial \log(\alpha + \lambda_i)}{\partial \beta} = \frac{1}{\beta} \sum\nolimits_i \frac{\lambda_i}{\alpha + \lambda_i},$$

$$\frac{\partial \log p\left(\boldsymbol{t} | \boldsymbol{X}, \alpha, \beta\right)}{\partial \beta} = \frac{N}{2\beta} - \frac{1}{2} \sum\nolimits_{n=1}^{N} \left(t_n - \boldsymbol{\mu}_n^\top \boldsymbol{x}_n\right)^2 - \frac{1}{2\beta} \sum\nolimits_i \frac{\lambda_i}{\alpha + \lambda_i}.$$

Таким образом, метод простых итераций для β будет выглядеть как

$$\beta^{(k+1)} = \frac{N - \sum_i \frac{\lambda_i}{\alpha^{(k)} + \lambda_i}}{\sum_{n=1}^{N} \left(t_n - \boldsymbol{\mu}_n\left(\alpha^{(k)}, \beta^{(k)}\right)^\top \boldsymbol{x}_n\right)^2}.$$

Здесь не нужно будет каждый раз пересчитывать собственные числа λ_i, потому что матрица $\boldsymbol{X}^\top \boldsymbol{X}$ фиксирована, и достаточно их подсчитать один раз для $\beta = 1$, а затем просто умножать на новые значения β.

В коде это будет выглядеть примерно так:

```
def empirical_iteration(X, t, XtX, XtT, lambdas, alpha, beta):
    N, d = X.shape
    cur_lambdas = beta * lambdas
    gamma = np.sum(cur_lambdas / (alpha + cur_lambdas))
    A = beta * XtX + alpha * np.identity(d)
    mu = beta * np.linalg.inv(A).dot(XtT)
    a = gamma / np.sum(mu ** 2)
    b = (N - gamma) / np.sum( (t - X.dot(mu)) ** 2)
    return a, b

def fit_eb(X, t, max_iter=400, eps=0.0001):
    XtX = X.T.dot(X)
    XtT = X.T.dot(t)
    lambdas = np.linalg.eigvalsh(XtX)
    alpha_0, beta_0 = 1, 1
    alpha, beta = [alpha_0], [beta_0]
    logmargs = [log_marginal_likelihood(X, t, alpha=alpha_0, beta=beta_0)]
    for _ in range(max_iter):
        a, b = empirical_iteration(X, t, XtX, XtT, lambdas, alpha[-1], beta[-1])
        alpha.append(a)
        beta.append(b)
        logmargs.append(log_marginal_likelihood(X, t, alpha=a, beta=b))
        if np.isclose(alpha[-1], alpha[-2], eps) and \
```

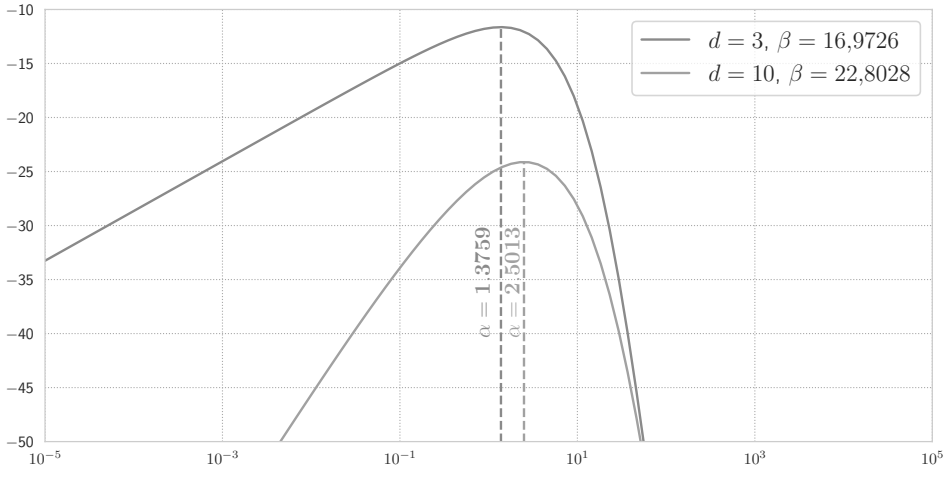


Рис. 6.22. Значения $\log p\,(\boldsymbol{t}|\boldsymbol{X}, \alpha, \beta)$ как функция от гиперпараметра α и результат численной итеративной оптимизации

```
            np.isclose(beta[-1], beta[-2], eps):
        break
return alpha, beta, logmargs
```

Теперь `get_ab_by_degree` будет принимать на вход привычную нам матрицу полиномиальных признаков $\boldsymbol{X}$, целевую переменную $\boldsymbol{t}$ и степень d и выдавать последовательность параметров $\alpha^{(k)}$ и $\beta^{(k)}$, начинающуюся с $\alpha^{(0)} = \beta^{(0)} = 1$. На рис. 6.22 показаны два результата такой оптимизации: для $d = 3$ и для $d = 10$.

6.5.2. Оценка $\mathbf{p}(\mathbf{D})$ как метод сравнения моделей

Оценка $p\,(D)$ может оказаться полезной не только для выбора гиперпараметров: она не ограничена случаем, когда модели имеют сходную структуру и один и тот же набор гиперпараметров. В жизни часто нужно сравнивать модели совсем разной структуры, или модели хоть и похожие, но, например, с разным числом параметров.

Наш избитый пример полиномиальной регрессии здесь отлично иллюстрирует проблему: по мере того как степень многочлена d растёт, модели с точки зрения правдоподобия данных становятся только лучше, ведь многочлен меньшей степени является частным случаем многочлена большей степени. Но в какой-то момент всё-таки нужно остановиться, потому что с ростом выразительности модели мы неизбежно ударимся в оверфиттинг. Как выбрать этот момент, если большого тестового множества точек, на котором можно было бы сравнить реальное качество предсказаний, под рукой нет?

Идея *байесовского сравнения моделей*, по сути, та же, что и в эмпирическом методе Байеса. Если мы хотим сравнить несколько моделей $\mathcal{M}_1, \ldots, \mathcal{M}_K$ с наборами параметров $\boldsymbol{\theta}_1, \ldots, \boldsymbol{\theta}_K$ на наборе данных D, мы на самом деле хотим сравнить между собой вероятности $p(\mathcal{M}_k|D)$. Они по теореме Байеса пропорциональны

$$p(\mathcal{M}_k|D) \propto p(\mathcal{M}_k)\, p(D|\mathcal{M}_k)\,.$$

Здесь $p(\mathcal{M}_k)$ — это априорные вероятности моделей, которые мы в этом разделе будем предполагать равномерными; если у вас есть основания предпочитать одну модель другой, просто перевзвесьте результаты априорными вероятностями $p(\mathcal{M}_k)$. А $p(D|\mathcal{M}_k)$ — это как раз в точности знаменатель теоремы Байеса, применённой к параметрам моделей:

$$p(\boldsymbol{\theta}_k|D, \mathcal{M}_k) = \frac{p(\boldsymbol{\theta}_k|\mathcal{M}_k)\, p(D|\boldsymbol{\theta}_k, \mathcal{M}_k)}{p(D|\mathcal{M}_k)}.$$

Обратите внимание, что в понятие «модели» здесь входит и априорное распределение, то есть сравнение моделей может помочь в том числе и сравнить разные регуляризаторы и разные значения гиперпараметров.

Как оценить $p(D)$ (я опускаю условие $\mathcal{M}_k$, потому что, начиная с этого места, мы всё время будем говорить об одной модели, а потом просто сравнивать полученные результаты для разных моделей)? Прежде чем переходить к «настоящим», применимым на практике методам, давайте проиллюстрируем основную идею, использовав заведомо нереалистичное приближение.

Для начала предположим, что параметр только один, $\theta \in \mathbb{R}$, а также что (здесь реализм нас покидает) априорное и апостериорное распределения равномерны:

- $p(\theta) = \frac{1}{\Delta\theta_{\text{pri}}}$ на некотором интервале $[a, a + \Delta\theta_{\text{pri}}]$ длиной $\Delta\theta_{\text{pri}}$;
- $p(\theta|D) = \frac{1}{\Delta\theta_{\text{post}}}$ на интервале $[b, b + \Delta\theta_{\text{post}}]$ длиной $\Delta\theta_{\text{post}}$, который, очевидно, входит в интервал априорного распределения, $\Delta\theta_{\text{post}} \leq \Delta\theta_{\text{pri}}$.

То, что апостериорное распределение равномерно, означает в данном случае, что $\theta_{\text{ML}} = \theta_{\text{MAP}}$, и правдоподобие тоже было равномерным на том же интервале, но с другой константой, равной максимуму правдоподобия $p(D|\theta_{\text{ML}})$; всё это проиллюстрировано на рис. 6.23, *а*. Поэтому оценить $p(D)$ несложно:

$$p(D) = \int p(\boldsymbol{\theta})\, p(\boldsymbol{\theta}|D)\, \mathrm{d}\boldsymbol{\theta} = \int_a^{a+\Delta\theta_{\text{pri}}} \frac{1}{\Delta\theta_{\text{pri}}} p(\boldsymbol{\theta}|D)\, \mathrm{d}\boldsymbol{\theta} =$$
$$= \int_b^{b+\Delta\theta_{\text{post}}} \frac{1}{\Delta\theta_{\text{pri}}} p(D|\theta_{\text{ML}})\, \mathrm{d}\boldsymbol{\theta} = p(D|\theta_{\text{ML}})\, \frac{\Delta\theta_{\text{post}}}{\Delta\theta_{\text{pri}}}.$$

Прологарифмировав, получим окончательный результат:

$$\log p(D) = \log p(D|\theta_{\text{ML}}) - \log \frac{\Delta\theta_{\text{pri}}}{\Delta\theta_{\text{post}}}. \tag{6.29}$$

Я перевернул дробь во втором слагаемом (6.29), чтобы подчеркнуть, что по сути $\log p(D)$ — это логарифм максимума правдоподобия за вычетом некоторого штрафа, ведь $\log \frac{\Delta\theta_{\text{pri}}}{\Delta\theta_{\text{post}}} \geq 0$.

А если $\boldsymbol{\theta}$ — это всё-таки вектор длины d, то мы столь же нереалистично предположим, что эти равномерные распределения одинаково и независимо появляются в каждой компоненте $\boldsymbol{\theta}$, и тогда результат (6.29) превращается в

$$\log p(D) = \log p(D|\boldsymbol{\theta}_{\text{ML}}) - d \log \frac{\Delta\theta_{\text{pri}}}{\Delta\theta_{\text{post}}}. \tag{6.30}$$

Что значит этот результат? Оказалось, что оценка качества модели складывается из:

- собственно максимума правдоподобия, который наверняка будет увеличиваться по мере того, как модель становится более выразительной, — в (слегка абсурдном) пределе этого процесса наибольшее правдоподобие будет достигнуто моделью, которая просто запомнит весь датасет D и обучит равномерное распределение на его точках, то самое распределение p_{data}, к которому, как мы выяснили в разделе 5.6, мы стремимся, когда максимизируем правдоподобие;
- и штрафа $d \log \frac{\Delta\theta_{\text{pri}}}{\Delta\theta_{\text{post}}}$, который как раз должен не позволить нам дойти до этого абсурдного предела.

Что происходит, когда модель становится более выразительной? Чаще всего это значит, что мы увеличиваем число параметров d; например, увеличиваем степень многочлена в полиномиальной регрессии). А в пределах одной размерности увеличение выразительности значит, что, скорее всего, априорное распределение становится более «широким», расширяя множество допустимых значений $\boldsymbol{\theta}$; в наших терминах это значит, что $\Delta\theta_{\text{pri}}$ увеличивается. С другой стороны, для более выразительных моделей с тем же датасетом апостериорное распределение становится более «узким», то есть $\Delta\theta_{\text{post}}$ уменьшается. Как ни крути, всё это приводит к тому, что штраф в формуле (6.30) увеличивается.

Можно посмотреть на такое сравнение моделей и с другой стороны: представим себе собственно *распределение* $p(D|\mathcal{M})$, то есть рассмотрим маргинальное правдоподобие датасета как распределение на пространстве датасетов. Это пространство очень высокой размерности — оно состоит из всевозможных наборов из N точек $(\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N)$. Чем будут отличаться друг от друга разные модели и, соответственно, разные распределения?

Абсолютно неточная, но выражающая базовую интуицию иллюстрация на этот счёт представлена на рис. 6.23, *б*. Более выразительные модели (такие, как

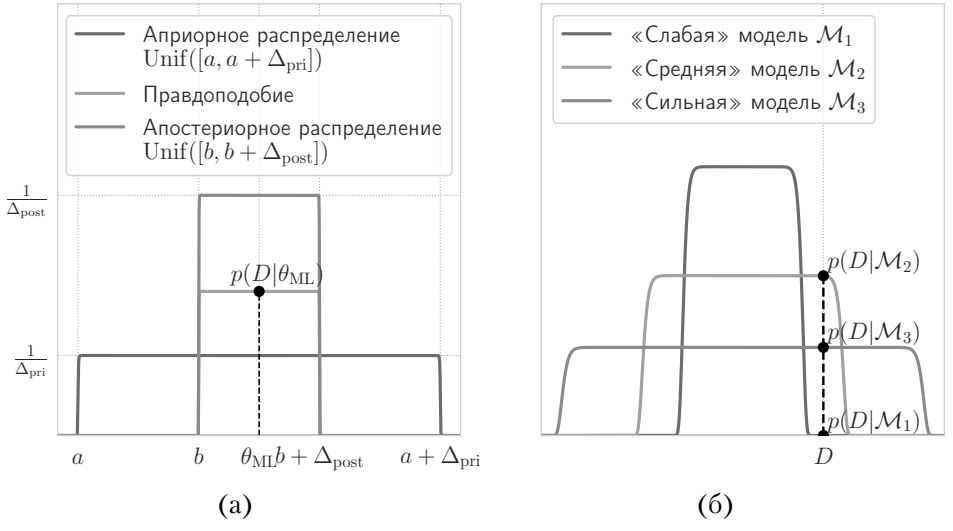


Рис. 6.23. Иллюстрации к байесовскому выбору моделей: (а) случай равномерных априорного и апостериорного распределений; (б) значения $p\,(D|\mathcal{M})$ для моделей разной выразительности в пространстве датасетов

модель $\mathcal{M}_3$ на рис. 6.23, *б*) могут породить больше разных датасетов заданного размера; с точки зрения обучения это значит, что более выразительная модель, например полиномиальная регрессия большей степени, может хорошо описать больше разных наборов точек. Но из этого следует, что в среднем на каждый датасет у более выразительной модели будет приходиться меньшее значение плотности ($p\,(D|\mathcal{M}_3)$ на рис. 6.23, *б*), ведь его нужно распространить на бóльшую область пространства датасетов. А значит, если менее выразительная модель ($\mathcal{M}_2$ на рис. 6.23, *б*) тоже сможет хорошо описать заданный датасет D, то по критерию $p\,(D|\mathcal{M})$ она выиграет у более выразительной, чего мы и хотели добиться. С другой стороны, если модель ещё менее выразительна ($\mathcal{M}_1$ на рис. 6.23, *б*), то она просто не сможет описать D достаточно хорошо, и её правдоподобие будет низким.

6.5.3. Байесовский информационный критерий

Но это были рассуждения, которые хоть и дают интуитивное представление о происходящем, не очень применимы на практике: откуда мы возьмём значения $\Delta\theta_{\mathrm{pri}}$ и $\Delta\theta_{\mathrm{post}}$ в формуле (6.30), как определим, «хорошо» ли модель описывает тот или иной датасет? Давайте попробуем операционализировать подобные рассуждения, применив более разумный способ аппроксимации участвующих в байесовском выводе распределений. Вернёмся к исходной постановке: нам нужно оценить интеграл

$$p(D) = \int p(\boldsymbol{\theta})\, p(\boldsymbol{\theta}|D)\, \mathrm{d}\boldsymbol{\theta} = \int p(\boldsymbol{\theta})\, e^{\ell(\boldsymbol{\theta})} \mathrm{d}\boldsymbol{\theta},$$

где $\ell(\boldsymbol{\theta}) = \log p(\boldsymbol{\theta}|D)$.

Давайте применим уже знакомый нам по разделу 5.3 метод: лапласовскую аппроксимацию в окрестности точки максимума правдоподобия $\boldsymbol{\theta}_{\mathrm{ML}}$:

$$\ell(\boldsymbol{\theta}) \approx \ell(\boldsymbol{\theta}_{\mathrm{ML}}) - \frac{N}{2} (\boldsymbol{\theta} - \boldsymbol{\theta}_{\mathrm{ML}})^\top J(\boldsymbol{\theta}_{\mathrm{ML}}) (\boldsymbol{\theta} - \boldsymbol{\theta}_{\mathrm{ML}}),$$

где

$$J(\boldsymbol{\theta}_{\mathrm{ML}}) = -\frac{1}{N} \left. \frac{\partial^2 \log p(\boldsymbol{\theta}|D)}{\partial \boldsymbol{\theta} \partial \boldsymbol{\theta}^\top} \right|_{\boldsymbol{\theta}_{\mathrm{ML}}}.$$

Аналогично можно и априорное распределение разложить в окрестности точки $\boldsymbol{\theta}_{\mathrm{ML}}$, только там не пропадёт член первого порядка, поэтому давайте им и ограничимся:

$$p(\boldsymbol{\theta}) \approx p(\boldsymbol{\theta}_{\mathrm{ML}}) + (\boldsymbol{\theta} - \boldsymbol{\theta}_{\mathrm{ML}})^\top \left. \nabla_{\boldsymbol{\theta}} p(\boldsymbol{\theta}) \right|_{\boldsymbol{\theta}_{\mathrm{ML}}}.$$

Итого получается, что

$$p(D) \approx$$
$$\int \left(p(\boldsymbol{\theta}_{\mathrm{ML}}) + (\boldsymbol{\theta} - \boldsymbol{\theta}_{\mathrm{ML}})^\top \left. \nabla_{\boldsymbol{\theta}} p(\boldsymbol{\theta}) \right|_{\boldsymbol{\theta}_{\mathrm{ML}}} \right) e^{\ell(\boldsymbol{\theta}_{\mathrm{ML}}) - \frac{N}{2} (\boldsymbol{\theta} - \boldsymbol{\theta}_{\mathrm{ML}})^\top J(\boldsymbol{\theta}_{\mathrm{ML}}) (\boldsymbol{\theta} - \boldsymbol{\theta}_{\mathrm{ML}})} \mathrm{d}\boldsymbol{\theta}.$$

Но теперь можно заметить, что

$$\int (\boldsymbol{\theta} - \boldsymbol{\theta}_{\mathrm{ML}})\, e^{-\frac{N}{2} (\boldsymbol{\theta} - \boldsymbol{\theta}_{\mathrm{ML}})^\top J(\boldsymbol{\theta}_{\mathrm{ML}}) (\boldsymbol{\theta} - \boldsymbol{\theta}_{\mathrm{ML}})} \mathrm{d}\boldsymbol{\theta} = \mathbf{O},$$

потому что это величина, пропорциональная ожиданию $(\boldsymbol{\theta} - \boldsymbol{\theta}_{\mathrm{ML}})$ по гауссиану со средним $(\boldsymbol{\theta} - \boldsymbol{\theta}_{\mathrm{ML}})$ и матрицей ковариаций $J(\boldsymbol{\theta}_{\mathrm{ML}})^{-1}$. А значит, наша аппроксимация превращается в

$$p(D) \approx e^{\ell(\boldsymbol{\theta}_{\mathrm{ML}})} p(\boldsymbol{\theta}_{\mathrm{ML}}) \int e^{-\frac{N}{2} (\boldsymbol{\theta} - \boldsymbol{\theta}_{\mathrm{ML}})^\top J(\boldsymbol{\theta}_{\mathrm{ML}}) (\boldsymbol{\theta} - \boldsymbol{\theta}_{\mathrm{ML}})} \mathrm{d}\boldsymbol{\theta}.$$

Интеграл теперь можно взять — из него получится нормировочная константа для того же самого гауссиана:

$$p(D) \approx e^{\ell(\boldsymbol{\theta}_{\mathrm{ML}})} p(\boldsymbol{\theta}_{\mathrm{ML}})\, (2\pi)^{\frac{d}{2}}\, N^{-\frac{d}{2}}\, (\det J(\boldsymbol{\theta}_{\mathrm{ML}}))^{-\frac{1}{2}}, \tag{6.31}$$

или после логарифмирования:

$$\log p(D) \approx \ell(\boldsymbol{\theta}_{\mathrm{ML}}) + \log p(\boldsymbol{\theta}_{\mathrm{ML}}) - \frac{d}{2} \log N - \frac{1}{2} \log \det J(\boldsymbol{\theta}_{\mathrm{ML}}) + \frac{d}{2} \log(2\pi). \tag{6.32}$$

Дополнительные слагаемые $\log p(\boldsymbol{\theta}_{\mathrm{ML}}) - \frac{d}{2}\log N - \frac{1}{2}\log\det J(\boldsymbol{\theta}_{\mathrm{ML}}) + \frac{d}{2}\log(2\pi)$, которые уточняют максимум правдоподобия в (6.32), называют *фактором Оккама*[1], потому что этот штраф как раз представляет собой численное выражение бритвы Оккама, предпочитающей более простые объяснения.

Если теперь в формуле (6.32) выбросить из правой части всё то, что не растёт с ростом числа точек данных N, а потом умножить на -2, то получится *байесовский информационный критерий* (Bayesian information criterion, BIC), он же *критерий Шварца* (Schwartz criterion):

$$\mathrm{BIC}(\mathcal{M}) = -2\log p(D|\boldsymbol{\theta}_{\mathrm{ML}}, \mathcal{M}) + d\log N, \tag{6.33}$$

где d — это размерность вектора $\boldsymbol{\theta}$, или число *свободных параметров* в модели $\mathcal{M}$. Замечу, что в случае совсем маленьких датасетов, если $\log N$ исчисляется буквально единицами (например, в нашем извечном примере линейной регрессии с $N = 10$), оставлять только $\frac{d}{2}\log N$, то есть выбрасывать в том числе слагаемое $\frac{d}{2}\log(2\pi)$, может быть преждевременным; но в реальной жизни, конечно, N обычно достаточно велико, чтобы не обращать на это внимания.

Байесовский информационный критерий был разработан Гидеоном Шварцем в 1978 году [517], и поэтому иногда его также называют *информационным критерием Шварца* (Schwarz information criterion). Однако есть и другие подходы к статистическому анализу выбора моделей, которые тоже называются информационными критериями; в следующем разделе мы их рассмотрим.

*6.5.4. Информационные критерии Такеучи и Акаике

Байесовский информационный критерий — это только один из возможных вариантов того, как можно подойти к выбору моделей с учётом числа свободных параметров. Другой подход, который мы сейчас рассмотрим, более «статистический» на вид, но в результате даёт очень похожие результаты, а также несколько обобщает весь разговор о различных *информационных критериях* [313, 563]; он несколько более сложен идейно и математически, поэтому пусть этот раздел будет «со звёздочкой».

[1] *Уильям Оккам* (William of Ockham, ок. 1285–1347) — английский философ, францисканский монах; Оккам — это не фамилия, а название деревни, где Уильям предположительно родился. Оккам всем известен исключительно своей бритвой, но на самом деле он был важной фигурой в развитии схоластики, учеником Дунса Скота и учителем Буридана, автором множества книг, в том числе основополагающей *Summa Logicae*, где он выписал формулы де Моргана и даже обсудил возможность построения тернарной логики. Оккам много писал о натурфилософии, эпистемологии, политической теории, а в философии был одним из основателей номинализма — учения о том, что абстрактные сущности и универсалии на самом деле не существуют, а являются просто метками, что прямо противоречило идеям Платона. Что же до бритвы, здесь не стоит забывать, что для Оккама единственной необходимой сущностью был Бог, и рассуждения о неумножении сущностей использованы Оккамом для того, чтобы отвергнуть закон достаточного основания и отрицать идеи Фомы Аквинского: по мнению Оккама, человеческий разум не способен доказать бессмертие души или существование Бога.

Как и раньше, предположим, что у нас есть данные $\mathcal{X} = \{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N\}$, которые были получены из некоторого истинного распределения данных $p_{\text{data}}(\boldsymbol{x})$, а мы пытаемся приблизить их некоторой параметрической моделью $p(\boldsymbol{x}|\boldsymbol{\theta})$, $\boldsymbol{\theta} \in \mathbb{R}^d$. Предположим также, что мы обучили модель методом максимального правдоподобия, получив распределение $p(\boldsymbol{x}|\boldsymbol{\theta}_{\text{ML}})$. Давайте попробуем оценить, насколько модель $p(\boldsymbol{x}|\boldsymbol{\theta}_{\text{ML}})$ отличается от неизвестного истинного распределения $p_{\text{data}}(\boldsymbol{x})$. Для этого оценим KL-дивергенцию между ними:

$$\begin{aligned}\text{KL}\left(p_{\text{data}} \| p(\boldsymbol{x}|\boldsymbol{\theta}_{\text{ML}})\right) = \mathbb{E}_{p_{\text{data}}(\boldsymbol{x})}\left[\log \frac{p_{\text{data}}(\boldsymbol{x})}{p(\boldsymbol{x}|\boldsymbol{\theta}_{\text{ML}})}\right] = \\ = \mathbb{E}_{p_{\text{data}}(\boldsymbol{x})}\left[\log p_{\text{data}}(\boldsymbol{x})\right] - \mathbb{E}_{p_{\text{data}}(\boldsymbol{x})}\left[\log p(\boldsymbol{x}|\boldsymbol{\theta}_{\text{ML}})\right].\end{aligned}$$

Поскольку энтропия распределения данных $\mathbb{E}_{p_{\text{data}}(\boldsymbol{x})}\left[\log p_{\text{data}}(\boldsymbol{x})\right]$ нам неподвластна, модель будет тем лучше, чем больше будет $\mathbb{E}_{p_{\text{data}}(\boldsymbol{x})}\left[\log p(\boldsymbol{x}|\boldsymbol{\theta}_{\text{ML}})\right]$, и для оценки расхождения между $p(\boldsymbol{x}|\boldsymbol{\theta}_{\text{ML}})$ и $p_{\text{data}}(\boldsymbol{x})$ нам нужно получить хорошую оценку ожидаемого логарифма правдоподобия. С этой точки зрения становится понятно, почему во всех критериях ключевую роль играет логарифм правдоподобия в точке его максимума, ведь это как раз выборочная оценка ожидания:

$$\mathbb{E}_{p_{\text{data}}(\boldsymbol{x})}\left[\log p(\boldsymbol{x}|\boldsymbol{\theta}_{\text{ML}})\right] = \int p_{\text{data}}(\boldsymbol{x}) \log p(\boldsymbol{x}|\boldsymbol{\theta}_{\text{ML}})\, \mathrm{d}\boldsymbol{x} \approx \frac{1}{N}\sum_{n=1}^{N} \log p(\boldsymbol{x}_n|\boldsymbol{\theta}_{\text{ML}}),$$

потому что точки $\boldsymbol{x}_n$, по нашему предположению, взяты из распределения данных $p_{\text{data}}(\boldsymbol{x})$. Однако эта оценка будет смещённой как минимум потому, что мы обучаем параметры максимального правдоподобия $\boldsymbol{\theta}_{\text{ML}}$ на основе того же датасета $\boldsymbol{X}$, который используется как выборка в этой оценке.

Если истинная модель p_{data} тоже принадлежит семейству $p(\boldsymbol{x}|\boldsymbol{\theta})$ с некоторым истинным параметром $\boldsymbol{\theta}_0$, то

$$\boldsymbol{\theta}_0 = \arg\max_{\boldsymbol{\theta}} \mathbb{E}_{p_{\text{data}}(\boldsymbol{x})}\left[\log p(\boldsymbol{x}|\boldsymbol{\theta})\right],$$

но это ожидание берётся по всему распределению, а для конкретного датасета $\boldsymbol{X}$ максимум будет другим.

Величину $\boldsymbol{\theta}_0$ можно считать «истинной» гипотезой максимального правдоподобия; при некоторых условиях регулярности (не будем в это углубляться, за полными доказательствами отошлю вас, например, к [313]) можно доказать, что:

- $\boldsymbol{\theta}_{\text{ML}}(\boldsymbol{X}) \to \boldsymbol{\theta}_0$ при $N \to \infty$;
- для $\boldsymbol{\theta}_{\text{ML}}(\boldsymbol{X})$ верна асимптотическая нормальность, то есть распределение величины $\sqrt{N}\left(\boldsymbol{\theta}_{\text{ML}} - \boldsymbol{\theta}_0\right)$ сходится по вероятности к нормальному распределению $\mathcal{N}\left(\mathbf{O}, I(\boldsymbol{\theta}_0)^{-1}\right)$, где $I(\boldsymbol{\theta})$ — это матрица информации Фишера, которую мы обсуждали в разделе 3.4,

$$I(\boldsymbol{\theta}) = \int p\left(\boldsymbol{x}|\boldsymbol{\theta}\right) \frac{\partial \log p\left(\boldsymbol{x}|\boldsymbol{\theta}\right)}{\partial \boldsymbol{\theta}} \frac{\partial \log p\left(\boldsymbol{x}|\boldsymbol{\theta}\right)}{\partial \boldsymbol{\theta}^{\top}} \mathrm{d}\boldsymbol{x}. \tag{6.34}$$

Более того, хотя приведённые выше формулы предполагают, что $p_{\text{data}}(\boldsymbol{x}) = p\left(\boldsymbol{x}|\boldsymbol{\theta}_0\right)$, аналогичные результаты можно получить и если $p_{\text{data}}(\boldsymbol{x})$ не принадлежит параметрическому семейству $p\left(\boldsymbol{x}|\boldsymbol{\theta}\right)$. В дальнейшем мы обозначим через $\boldsymbol{\theta}_0$ максимум ожидания логарифма правдоподобия по p_{data}, то есть решение системы уравнений

$$\int p_{\text{data}}(\boldsymbol{x}) \frac{\partial \log p\left(\boldsymbol{x}|\boldsymbol{\theta}\right)}{\partial \boldsymbol{\theta}} \mathrm{d}\boldsymbol{x} = \mathbf{O},$$

и будем предполагать, что данные $\boldsymbol{X}$, по которым строится оценка $\boldsymbol{\theta}_{\text{ML}}$, получены из распределения $p_{\text{data}}(\boldsymbol{x})$. Тогда при тех же условиях можно доказать, что:

- $\boldsymbol{\theta}_{\text{ML}}(\boldsymbol{X}) \to \boldsymbol{\theta}_0$ при $N \to \infty$;
- распределение величины $\sqrt{N}\left(\boldsymbol{\theta}_{\text{ML}} - \boldsymbol{\theta}_0\right)$ сходится по вероятности к нормальному распределению

$$\sqrt{N}\left(\boldsymbol{\theta}_{\text{ML}} - \boldsymbol{\theta}_0\right) \to_{N\to\infty} \mathcal{N}\left(\mathbf{O}, J^{-1}(\boldsymbol{\theta}_0) I(\boldsymbol{\theta}_0) J^{-1}(\boldsymbol{\theta}_0)\right), \tag{6.35}$$

где $I(\boldsymbol{\theta})$ — это та же матрица информации Фишера (6.34), только по распределению p_{data},

$$I(\boldsymbol{\theta}) = \int p_{\text{data}}(\boldsymbol{x}) \frac{\partial \log p\left(\boldsymbol{x}|\boldsymbol{\theta}\right)}{\partial \boldsymbol{\theta}} \frac{\partial \log p\left(\boldsymbol{x}|\boldsymbol{\theta}\right)}{\partial \boldsymbol{\theta}^{\top}} \mathrm{d}\boldsymbol{x}, \tag{6.36}$$

а $J(\boldsymbol{\theta})$ — это ожидание матрицы вторых производных

$$J(\boldsymbol{\theta}) = -\int p_{\text{data}}(\boldsymbol{x}) \frac{\partial^2 \log p\left(\boldsymbol{x}|\boldsymbol{\theta}\right)}{\partial \boldsymbol{\theta} \partial \boldsymbol{\theta}^{\top}} \mathrm{d}\boldsymbol{x}. \tag{6.37}$$

Иначе говоря, элементом на позиции (i,j) у матрицы $I(\boldsymbol{\theta})$ будет ожидание произведения $\frac{\partial \log p(\boldsymbol{x}|\boldsymbol{\theta})}{\partial \theta_i}$ и $\frac{\partial \log p(\boldsymbol{x}|\boldsymbol{\theta})}{\partial \theta_j}$, а у матрицы $J(\boldsymbol{\theta})$ — ожидание второй производной $\frac{\partial^2 \log p(\boldsymbol{x}|\boldsymbol{\theta})}{\partial \boldsymbol{\theta}_i \partial \boldsymbol{\theta}_j}$.

И если всё-таки $p_{\text{data}}(\boldsymbol{x}) = p\left(\boldsymbol{x}|\boldsymbol{\theta}_0\right)$, то можно будет заметить, что

$$\frac{\partial^2 \log p\left(\boldsymbol{x}|\boldsymbol{\theta}\right)}{\partial \boldsymbol{\theta}_i \partial \boldsymbol{\theta}_j} = \frac{\partial}{\partial \theta_i}\left(\frac{1}{p\left(x|\boldsymbol{\theta}\right)} \frac{\partial p\left(\boldsymbol{x}|\boldsymbol{\theta}\right)}{\partial \theta_j}\right) =$$

$$= \frac{1}{p\left(x|\boldsymbol{\theta}\right)} \frac{\partial^2 p\left(\boldsymbol{x}|\boldsymbol{\theta}\right)}{\partial \boldsymbol{\theta}_i \partial \boldsymbol{\theta}_j} - \frac{\partial \log p\left(\boldsymbol{x}|\boldsymbol{\theta}\right)}{\partial \theta_i} \frac{\partial \log p\left(\boldsymbol{x}|\boldsymbol{\theta}\right)}{\partial \theta_j},$$

а в ожидании по $p_{\text{data}}(\boldsymbol{x}) = p\left(\boldsymbol{x}|\boldsymbol{\theta}_0\right)$ при подстановке $\boldsymbol{\theta} = \boldsymbol{\theta}_0$ получается, что

$$\mathbb{E}_{p(\boldsymbol{x}|\boldsymbol{\theta}_0)}\left[\frac{1}{p(\boldsymbol{x}|\boldsymbol{\theta}_0)}\frac{\partial^2 p(\boldsymbol{x}|\boldsymbol{\theta}_0)}{\partial\boldsymbol{\theta}_i\partial\boldsymbol{\theta}_j}\right] = \int \frac{p(\boldsymbol{x}|\boldsymbol{\theta}_0)}{p(\boldsymbol{x}|\boldsymbol{\theta}_0)}\frac{\partial^2 p(\boldsymbol{x}|\boldsymbol{\theta}_0)}{\partial\boldsymbol{\theta}_i\partial\boldsymbol{\theta}_j}\mathrm{d}\boldsymbol{x} = \\ = \frac{\partial^2}{\partial\boldsymbol{\theta}_i\partial\boldsymbol{\theta}_j}\int p(\boldsymbol{x}|\boldsymbol{\theta}_0) = 0,$$

а значит, в этом случае $I(\boldsymbol{\theta}_0) = J(\boldsymbol{\theta}_0)$ и мы приходим к формуле (6.34).

В целом можно сказать, что различные информационные критерии для сравнения моделей направлены на то, чтобы оценить смещение выборочной оценки для величины $\mathbb{E}_{p_{\text{data}}(\boldsymbol{x})}[\log p(\boldsymbol{x}|\boldsymbol{\theta}_{\text{ML}})]$

$$b(p_{\text{data}}) = \mathbb{E}_{D\sim p_{\text{data}}}\left[\log p(D|\boldsymbol{\theta}_{\text{ML}}) - N\mathbb{E}_{p_{\text{data}}(\boldsymbol{x})}[\log p(\boldsymbol{x}|\boldsymbol{\theta}_{\text{ML}})]\right], \qquad (6.38)$$

где мы взяли ожидание по датасетам D, выбранным по истинному распределению p_{data}, а $\boldsymbol{\theta}_{\text{ML}} = \boldsymbol{\theta}_{\text{ML}}(D)$ — это гипотеза максимального правдоподобия для модели, обученной на том же самом датасете D.

Давайте проиллюстрируем этот эффект на практическом примере; разумеется, самом простом, чтобы всё происходило в размерности 1 и, как следствие, было очень наглядно. Давайте определим обычную линейную регрессию, причём без свободного члена (с одним параметром w), и зададим датасет из пяти точек, порождённый на основе прямой линии $y = -\frac{1}{2}x$ с гауссовским шумом, дисперсия которого равна $\sigma = \frac{1}{2}$:

```
true_w, sigma = -.5, .5
orig = lambda x : true_w*x
num_points = 5
xd = np.array([-3, -1, 0, 0.5, 2.5]) / 2
data = orig(xd) + np.random.normal(0, sigma, num_points)
m = linear_model.LinearRegression(fit_intercept=False).fit( \
  xd.reshape(-1, 1), data )
```

На рис. 6.24, *а* показана получающаяся линейная регрессия. Я честным образом породил этот датасет из приведённого выше кода, но выбрал из нескольких порождений то, которое давало большее отклонение обученной прямой от истинной, чтобы остальные графики получались более наглядными. В итоге получился вектор точек данных

$$\boldsymbol{y} = \begin{pmatrix}1{,}33 & 0{,}71 & -0{,}56 & -0{,}7 & -1{,}18\end{pmatrix},$$

и после обучения гипотеза максимального правдоподобия для этого датасета получилась равной $w_{\text{ML}} = -0{,}23$.

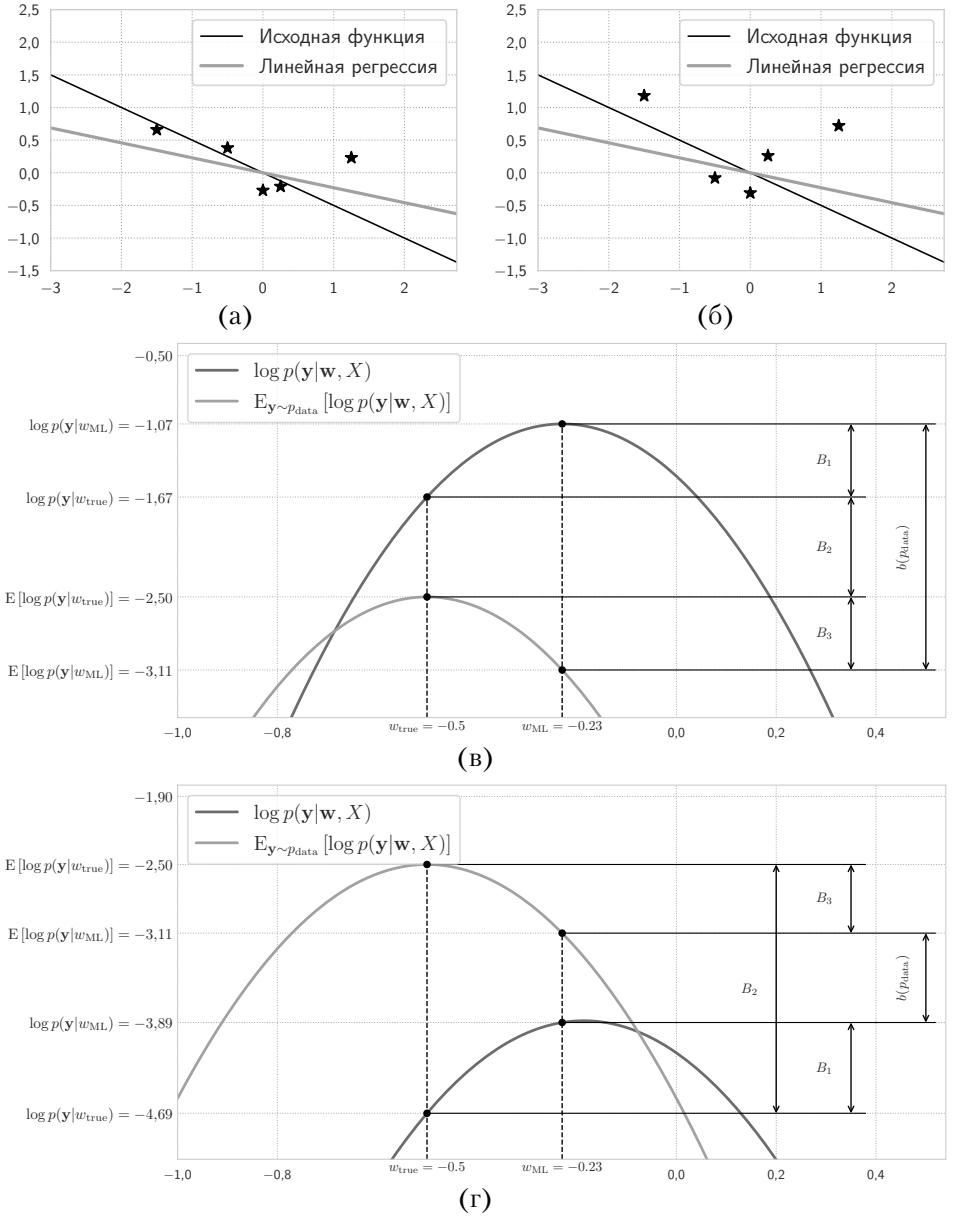


Рис. 6.24. Эффект смещения выборочной оценки $\mathbb{E}_{p_{\mathrm{data}}(\boldsymbol{x})}[\log p(\boldsymbol{x}|\boldsymbol{\theta}_{\mathrm{ML}})]$: (а, б) данные, порождённые из $y = -\frac{1}{2}x$ и обученная на них линейная регрессия; (в, г) разница между $\mathbb{E}_{p_{\mathrm{data}}(\boldsymbol{y})}[\log p(\boldsymbol{y}|w, X)]$ и $\log p(\boldsymbol{y}|w, X)$ как функциями от w

Следующий шаг — оценить правдоподобие датасета и ожидаемое правдоподобие датасетов в зависимости от параметров. Правдоподобие данных как функция от весов $\boldsymbol{w}$ в линейной регрессии выглядит как

$$\log p\left(\boldsymbol{y}|\boldsymbol{w},X\right) = -\frac{1}{2\sigma^2}\sum_{n=1}^{N}\left(y_n - \boldsymbol{w}^\top \boldsymbol{x}_n\right)^2,$$

а чтобы найти ожидаемое правдоподобие датасетов из N точек, семплированных из p_{data}, можно просто подсчитать это ожидание:

$$\begin{aligned}
\mathbb{E}_{p_{\text{data}}(\boldsymbol{y})}\left[\log p\left(\boldsymbol{y}|\boldsymbol{w},X\right)\right] &= \mathbb{E}\left[-\frac{1}{2\sigma^2}\sum_{n=1}^{N}\left(y_n - \boldsymbol{w}^\top \boldsymbol{x}_n\right)^2\right] = \\
&= -\frac{1}{2\sigma^2}\sum_{n=1}^{N}\left(\mathbb{E}\left[y_n^2\right] - 2\left(\boldsymbol{w}^\top \boldsymbol{x}_n\right)\mathbb{E}\left[y_n\right] + \left(\boldsymbol{w}^\top \boldsymbol{x}_n\right)^2\right) = \\
&= -\frac{1}{2\sigma^2}\sum_{n=1}^{N}\left(\left(\boldsymbol{w}_{\text{true}}^\top \boldsymbol{x}_n\right)^2 + \sigma^2 - 2\left(\boldsymbol{w}^\top \boldsymbol{x}_n\right)\left(\boldsymbol{w}_{\text{true}}^\top \boldsymbol{x}_n\right) + \left(\boldsymbol{w}^\top \boldsymbol{x}_n\right)^2\right) = \\
&= -\frac{N}{2} - \frac{1}{2\sigma^2}\sum_{n=1}^{N}\left(\left(\boldsymbol{w} - \boldsymbol{w}_{\text{true}}\right)^\top \boldsymbol{x}_n\right)^2,
\end{aligned}$$

где я подставил параметры истинного распределения p_{data} как $\mathbb{E}\left[y_n\right] = \boldsymbol{w}_{\text{true}}^\top \boldsymbol{x}_n$ и $\mathbb{E}\left[y_n^2\right] = \mathbb{E}\left[y_n\right]^2 + \sigma^2$.

В нашем же конкретном случае всё ещё проще, потому что в нашей линейной регрессии нет свободного члена, и обе величины легко запрограммировать как функции от w:

```
def log_likelihood(w, xd=xd, data=data, sigma=sigma):
    return - np.sum((data - w*xd)**2) / (2 * (sigma**2))

def expected_log_l(w, xd=xd, data=data, sigma=sigma):
    return - len(data) / 2. - np.sum(((w-true_w)*xd) ** 2) / (2 * (sigma**2))
```

Результат показан на рис. 6.24, *в*. Как видно, есть существенная разница как между точками w_{ML} и w_{true}, так и между максимумами правдоподобия на имеющемся датасете и ожидаемого правдоподобия; последняя разница — это и есть смещение $b(p_{\text{data}})$ из (6.38). Если мы сможем оценить $b(p_{\text{data}})$, то информационный критерий можно будет построить, умножив на -2 аналогично байесовскому информационному критерию, как

$$\mathrm{IC}\,(D,\boldsymbol{\theta}) = -2\,(\text{логарифм правдоподобия } D \text{ в } \boldsymbol{\theta}_{\mathrm{ML}} - \text{оценка смещения}) =$$
$$= -2\sum_{n=1}^{N}\log p\,(\boldsymbol{x}_n|\boldsymbol{\theta}_{\mathrm{ML}}) + 2\cdot(\text{оценка } b(p_{\mathrm{data}}))\,. \quad (6.39)$$

Давайте попробуем оценить $b(p_{\mathrm{data}})$. Для этого разложим его на три слагаемых так, как показано на рис. 6.24, *б*, а именно на:

- разницу между $\log p\,(D|\boldsymbol{\theta}_{\mathrm{ML}})$ и $\log p\,(D|\boldsymbol{\theta}_0)$ (B_1 на рис. 6.24, *б*),
- между $\log p\,(D|\boldsymbol{\theta}_0)$ и $\mathbb{E}_{p_{\mathrm{data}}}[\log p\,(D|\boldsymbol{\theta}_0)]$ (B_2 на рис. 6.24, *б*) и
- между $\mathbb{E}_{p_{\mathrm{data}}}[\log p\,(D|\boldsymbol{\theta}_0)]$ и $\mathbb{E}_{p_{\mathrm{data}}}[\log p\,(D|\boldsymbol{\theta}_{\mathrm{ML}})]$ (B_3 на рис. 6.24, *б*).

На рисунке разложение показано для конкретного датасета, а при выводе мы будем оценивать $b(p_{\mathrm{data}})$ в ожидании:

$$\begin{aligned} b(p_{\mathrm{data}}) =& \mathbb{E}_{D\sim p_{\mathrm{data}}}\left[\log p\,(D|\boldsymbol{\theta}_{\mathrm{ML}}) - N\mathbb{E}_{p_{\mathrm{data}}(\boldsymbol{x})}[\log p\,(\boldsymbol{x}|\boldsymbol{\theta}_{\mathrm{ML}})]\right] = \\ &= \mathbb{E}_{D\sim p_{\mathrm{data}}}[\log p\,(D|\boldsymbol{\theta}_{\mathrm{ML}}) - \log p\,(D|\boldsymbol{\theta}_0)] + \\ &+ \mathbb{E}_{D\sim p_{\mathrm{data}}}\left[\log p\,(D|\boldsymbol{\theta}_0) - N\mathbb{E}_{p_{\mathrm{data}}(\boldsymbol{x})}[\log p\,(\boldsymbol{x}|\boldsymbol{\theta}_0)]\right] + \\ &+ \mathbb{E}_{D\sim p_{\mathrm{data}}}\left[N\mathbb{E}_{p_{\mathrm{data}}(\boldsymbol{x})}[\log p\,(\boldsymbol{x}|\boldsymbol{\theta}_0)] - N\mathbb{E}_{p_{\mathrm{data}}(\boldsymbol{x})}[\log p\,(\boldsymbol{x}|\boldsymbol{\theta}_{\mathrm{ML}})]\right] = \\ =& B_1 + B_2 + B_3. \end{aligned}$$

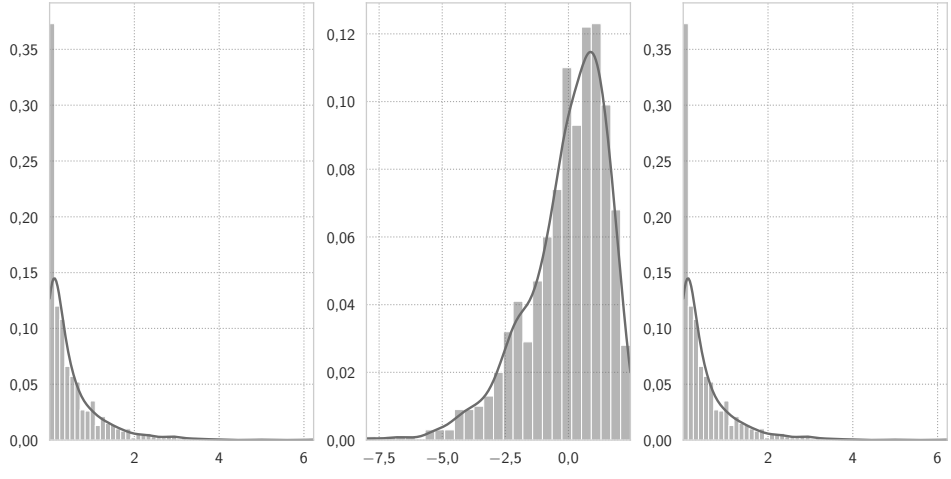
Не забудьте, что $\boldsymbol{\theta}_{\mathrm{ML}} = \boldsymbol{\theta}_{\mathrm{ML}}(D)$ зависит от датасета, так что в B_3 ожидание по D тоже вовсе не бессмысленно.

Проще всего оценить B_2, потому что в нём нет $\boldsymbol{\theta}_{\mathrm{ML}}$:

$$B_2 = \mathbb{E}_{D\sim p_{\mathrm{data}}}\left[\log p\,(D|\boldsymbol{\theta}_0) - N\mathbb{E}_{p_{\mathrm{data}}(\boldsymbol{x})}[\log p\,(\boldsymbol{x}|\boldsymbol{\theta}_0)]\right] =$$
$$\mathbb{E}_{D\sim p_{\mathrm{data}}}\left[\sum_{n=1}^{N}\log p\,(\boldsymbol{x}_n|\boldsymbol{\theta}_0)\right] - N\mathbb{E}_{p_{\mathrm{data}}(\boldsymbol{x})}[\log p\,(\boldsymbol{x}|\boldsymbol{\theta}_0)] = 0. \quad (6.40)$$

Это не значит, что B_2 всегда равно нулю, и в графике на рис. 6.24, *в* нет ошибки; для конкретного датасета D значение B_2 будет ненулевым, но в ожидании получится ноль. При этом значения B_1 и B_3 всегда неотрицательные, потому что они показывают отклонение двух кривых логарифма правдоподобия от своих же соответствующих максимумов. Например, при данных, которые плохо подходят под заданную модель, логарифм даже гипотезы максимального правдоподобия $\log p\,(D|\boldsymbol{\theta}_{\mathrm{ML}})$ может оказаться ниже, чем ожидание $\mathbb{E}_{p_{\mathrm{data}}}[\log p\,(D|\boldsymbol{\theta}_0)]$, именно за счёт большого отрицательного B_2. Такой пример тоже показан на графиках: на рис. 6.24, *б* показана линейная регрессия с данными

$$\boldsymbol{y} = \begin{pmatrix}1{,}18 & -0{,}08 & -0{,}31 & 0{,}26 & 0{,}72\end{pmatrix},$$



(а) B_1 (б) B_2 (в) B_3

Рис. 6.25. Эмпирическое распределение частей оценки $b(p_{\text{data}})$ в случае линейной регрессии с одним параметром: (а) B_1, (б) B_2, (в) B_3

для которых $w_{\text{ML}} = -0{,}19$, но в данных есть плохо описываемые выбросы. Тогда в результате на рис. 6.24, *г* мы увидим большой отрицательный B_2, но всё равно, конечно, положительные B_1 и B_3.

Можно и эмпирически посмотреть на распределения B_1, B_2 и B_3 — для случая нашей линейной регрессии это совсем просто:

```
def get_B(data, xd=xd.reshape(-1, 1)):
    m = linear_model.LinearRegression(fit_intercept=False).fit(xd, data)
    logl_w0 = log_likelihood(true_w, data=data)
    logl_w = log_likelihood(m.coef_[0], data=data)
    expl_w0 = expected_log_l(true_w, data=data)
    expl_w = expected_log_l(m.coef_[0], data=data)
    B1 = logl_w - logl_w0
    B2 = logl_w0 - expl_w0
    B3 = expl_w0 - expl_w
    return [B1, B2, B3]

B = np.vstack([get_B(orig(xd)+np.random.normal(0, sigma, num_points)) \
  for _ in range(1000)])
```

Гистограммы полученных значений показаны на рис. 6.25; в нашем случае распределения B_1 и B_3 неотрицательны (и совпадают, потому что это распределения отклонений двух парабол одной и той же формы), а распределение B_2 довольно широкое и несимметричное, хотя его среднее равно нулю.

Но вернёмся к теоретическим оценкам. Чтобы оценить величину B_3, рассмотрим функцию

$$\eta(\boldsymbol{\theta}_{\text{ML}}) = \mathbb{E}_{p_{\text{data}}(\boldsymbol{x})} \left[\log p\left(\boldsymbol{x}|\boldsymbol{\theta}_{\text{ML}}\right)\right]$$

и разложим её по формуле Тейлора в окрестности точки $\boldsymbol{\theta}_0$ до второго члена. Поскольку $\boldsymbol{\theta}_0$ — это точка максимума функции $\eta(\boldsymbol{\theta}_{\text{ML}})$, член первого порядка обратится в ноль, и мы получим

$$\eta(\boldsymbol{\theta}_{\text{ML}}) \approx \eta(\boldsymbol{\theta}_0) - \frac{1}{2}\left(\boldsymbol{\theta}_{\text{ML}} - \boldsymbol{\theta}_0\right)^\top J(\boldsymbol{\theta}_0)\left(\boldsymbol{\theta}_{\text{ML}} - \boldsymbol{\theta}_0\right), \tag{6.41}$$

где

$$J(\boldsymbol{\theta}_0) = -\mathbb{E}_{p_{\text{data}}(\boldsymbol{x})}\left[\left.\frac{\partial^2 \log p\left(\boldsymbol{x}|\boldsymbol{\theta}\right)}{\partial\boldsymbol{\theta}\partial\boldsymbol{\theta}^\top}\right|_{\boldsymbol{\theta}_0}\right] = -\int p_{\text{data}}(\boldsymbol{x})\left.\frac{\partial^2 \log p\left(\boldsymbol{x}|\boldsymbol{\theta}\right)}{\partial\boldsymbol{\theta}\partial\boldsymbol{\theta}^\top}\right|_{\boldsymbol{\theta}_0}\mathrm{d}\boldsymbol{x}.$$

А B_3 — это ожидание $\eta(\boldsymbol{\theta}_0) - \eta(\boldsymbol{\theta}_{\text{ML}})$ по распределению $p_{\text{data}}(D)$:

$$\begin{aligned}
B_3 &= \mathbb{E}_{D\sim p_{\text{data}}}\left[N\mathbb{E}_{p_{\text{data}}(\boldsymbol{x})}\left[\log p\left(\boldsymbol{x}|\boldsymbol{\theta}_0\right)\right] - N\mathbb{E}_{p_{\text{data}}(\boldsymbol{x})}\left[\log p\left(\boldsymbol{x}|\boldsymbol{\theta}_{\text{ML}}\right)\right]\right] = \\
&= \frac{N}{2}\mathbb{E}_{D\sim p_{\text{data}}}\left[\left(\boldsymbol{\theta}_{\text{ML}} - \boldsymbol{\theta}_0\right)^\top J(\boldsymbol{\theta}_0)\left(\boldsymbol{\theta}_{\text{ML}} - \boldsymbol{\theta}_0\right)\right] = \\
&= \frac{N}{2}\mathbb{E}_{D\sim p_{\text{data}}}\left[\mathrm{Tr}\left(J(\boldsymbol{\theta}_0)\left(\boldsymbol{\theta}_{\text{ML}} - \boldsymbol{\theta}_0\right)\left(\boldsymbol{\theta}_{\text{ML}} - \boldsymbol{\theta}_0\right)^\top\right)\right] = \\
&= \frac{N}{2}\mathrm{Tr}\left(J(\boldsymbol{\theta}_0)\mathbb{E}_{D\sim p_{\text{data}}}\left[\left(\boldsymbol{\theta}_{\text{ML}} - \boldsymbol{\theta}_0\right)\left(\boldsymbol{\theta}_{\text{ML}} - \boldsymbol{\theta}_0\right)^\top\right]\right).
\end{aligned}$$

Теперь давайте вместо ожидания матрицы ковариаций по датасету D подставим асимптотический результат (6.35):

$$B_3 = \frac{N}{2}\mathrm{Tr}\left(J(\boldsymbol{\theta}_0)\frac{1}{N}J(\boldsymbol{\theta}_0)^{-1}I(\boldsymbol{\theta}_0)J(\boldsymbol{\theta}_0)^{-1}\right) = \frac{1}{2}\mathrm{Tr}\left(I(\boldsymbol{\theta}_0)J(\boldsymbol{\theta}_0)^{-1}\right). \tag{6.42}$$

Осталось оценить B_1; для этого нужно провернуть аналогичный трюк с $\ell(\boldsymbol{\theta}) = \log p\left(\mathcal{X}|\boldsymbol{\theta}\right)$, разложив его вокруг своего максимума $\boldsymbol{\theta}_{\text{ML}}$:

$$\ell(\boldsymbol{\theta}) = \ell(\boldsymbol{\theta}_{\text{ML}}) + \frac{1}{2}\left(\boldsymbol{\theta} - \boldsymbol{\theta}_{\text{ML}}\right)^\top \left.\frac{\partial^2\ell(\boldsymbol{\theta})}{\partial\boldsymbol{\theta}\partial\boldsymbol{\theta}^\top}\right|_{\boldsymbol{\theta}_{\text{ML}}}\left(\boldsymbol{\theta} - \boldsymbol{\theta}_{\text{ML}}\right).$$

Мы знаем, что $\boldsymbol{\theta}_{\text{ML}}$ сходится к $\boldsymbol{\theta}_0$ при $N \to \infty$; а по закону больших чисел можно получить, что

$$-\frac{1}{N}\left.\frac{\partial^2\ell(\boldsymbol{\theta})}{\partial\boldsymbol{\theta}\partial\boldsymbol{\theta}^\top}\right|_{\boldsymbol{\theta}_{\text{ML}}} = -\frac{1}{N}\sum_{n=1}^{N}\left.\frac{\partial^2 \log p\left(\boldsymbol{x}_n|\boldsymbol{\theta}\right)}{\partial\boldsymbol{\theta}\partial\boldsymbol{\theta}^\top}\right|_{\boldsymbol{\theta}_{\text{ML}}} \longrightarrow J(\boldsymbol{\theta}_0).$$

Следовательно, в нашу оценку можно подставить

$$\ell(\boldsymbol{\theta}_{\mathrm{ML}}) - \ell(\boldsymbol{\theta}_0) \approx -\frac{N}{2}\left(\boldsymbol{\theta} - \boldsymbol{\theta}_{\mathrm{ML}}\right)^{\top} J(\boldsymbol{\theta}_0)\left(\boldsymbol{\theta} - \boldsymbol{\theta}_{\mathrm{ML}}\right),$$

а затем и оценить B_1 так же, как оценивали B_3:

$$\begin{aligned} B_1 = \mathbb{E}_{D\sim p_{\mathrm{data}}}\left[\log p\left(D|\boldsymbol{\theta}_{\mathrm{ML}}\right) - \log p\left(D|\boldsymbol{\theta}_0\right)\right] = \\ = \frac{N}{2}\mathbb{E}_{D\sim p_{\mathrm{data}}}\left[\left(\boldsymbol{\theta} - \boldsymbol{\theta}_{\mathrm{ML}}\right)^{\top} J(\boldsymbol{\theta}_0)\left(\boldsymbol{\theta} - \boldsymbol{\theta}_{\mathrm{ML}}\right)\right] = \\ = \frac{N}{2}\mathrm{Tr}\left(J(\boldsymbol{\theta}_0)\mathbb{E}_{D\sim p_{\mathrm{data}}}\left[\left(\boldsymbol{\theta} - \boldsymbol{\theta}_{\mathrm{ML}}\right)^{\top}\left(\boldsymbol{\theta} - \boldsymbol{\theta}_{\mathrm{ML}}\right)\right]\right), \end{aligned}$$

то есть

$$B_3 = \frac{1}{2}\mathrm{Tr}\left(I(\boldsymbol{\theta}_0)J(\boldsymbol{\theta}_0)^{-1}\right). \tag{6.43}$$

Осталось только объединить (6.40), (6.42) и (6.43):

$$b(p_{\mathrm{data}}) = B_1 + B_2 + B_3 = \mathrm{Tr}\left(I(\boldsymbol{\theta}_0)J(\boldsymbol{\theta}_0)^{-1}\right). \tag{6.44}$$

В выражении (6.44) участвуют величины $I(\boldsymbol{\theta}_0)$ и $J(\boldsymbol{\theta}_0)$, которые нам неизвестны, поскольку зависят от неизвестного распределения p_{data}. Если заменить их на какие-нибудь оценки $\hat{I}$ и $\hat{J}$, всё это приведёт нас к так называемому *информационному критерию Такеучи* (Takeuchi information criterion, TIC) [564, 585]:

$$\mathrm{TIC} = -2\sum_{n=1}^{N}\log p\left(\boldsymbol{x}_n|\boldsymbol{\theta}_{\mathrm{ML}}\right) + 2\mathrm{Tr}\left(\hat{I}\hat{J}^{-1}\right). \tag{6.45}$$

В качестве оценок $\hat{I}$ и $\hat{J}$ здесь можно (и, видимо, нужно, потому что другого способа оценить эти матрицы не видно) подставить просто усреднённые значения по датасету в точке максимума правдоподобия:

$$\begin{aligned} \hat{I}_{i,j} &= \frac{1}{N}\sum\nolimits_{n=1}^{N}\frac{\partial \log p\left(\boldsymbol{x}_n|\boldsymbol{\theta}\right)}{\partial\theta_i}\frac{\partial \log p\left(\boldsymbol{x}_n|\boldsymbol{\theta}\right)}{\partial\theta_j}\bigg|_{\boldsymbol{\theta}_{\mathrm{ML}}}, \\ \hat{J}_{i,j} &= \frac{1}{N}\sum\nolimits_{n=1}^{N}\frac{\partial^2 \log p\left(\boldsymbol{x}_n|\boldsymbol{\theta}\right)}{\partial\theta_i\partial\theta_j}\bigg|_{\boldsymbol{\theta}_{\mathrm{ML}}}. \end{aligned}$$

А если можно всё-таки предполагать, что истинное распределение данных p_{data} лежит в параметрическом семействе $p\left(\boldsymbol{x}|\boldsymbol{\theta}\right)$, то есть $p_{\mathrm{data}}(\boldsymbol{x}) = p\left(\boldsymbol{x}|\boldsymbol{\theta}_0\right)$, то, как мы обсуждали выше, $I(\boldsymbol{\theta}_0) = J(\boldsymbol{\theta}_0)$, и информационный критерий Такеучи

приобретает гораздо более простую форму, известную как *информационный критерий Акаике* (Akaike information criterion, AIC) [17–19]:

$$\mathrm{AIC} = -2\sum_{n=1}^{N}\log p\left(\boldsymbol{x}_n|\boldsymbol{\theta}_{\mathrm{ML}}\right) + 2\mathrm{Tr}\left(\mathbf{I}_d\right) = -2\sum_{n=1}^{N}\log p\left(\boldsymbol{x}_n|\boldsymbol{\theta}_{\mathrm{ML}}\right) + 2d. \quad (6.46)$$

Любопытно, кстати, что впервые свой информационный критерий японский статистик Хиротугу Акаике представил в 1971 году на Втором международном симпозиуме по теории информации, проходившем в СССР, точнее, на армянском курорте Цахкадзор [17]. Правда, в этом докладе содержалась только неформальная идея информационного критерия. Полностью метод AIC был представлен в опубликованной в 1974 году статье [18], которая стала одной из самых высокоцитируемых работ в истории, получив более 14 000 ссылок в *Web of Science* и, по подсчётам 2014 года, войдя в топ-100 по цитируемости не в информатике, а *вообще* в науке [613].

Давайте попробуем применить разработанные нами информационные критерии к той самой полиномиальной регрессии, которую мы уже совсем замучили на протяжении этой книги. В ней логарифм правдоподобия выглядит как

$$\ell(\boldsymbol{w}) = -\frac{N}{2}\log\left(2\pi\sigma^2\right) - \frac{1}{2\sigma^2}\sum_{n=1}^{N}\left(t_n - \boldsymbol{w}^\top\boldsymbol{x}_n\right)^2. \quad (6.47)$$

Здесь можно поступить двумя разными способами: можно, как мы поначалу делали в главе 4, считать σ^2 константой и расценивать только $\boldsymbol{w}$ как параметры модели с гипотезой максимального правдоподобия $\boldsymbol{w}_{\mathrm{ML}} = \left(\boldsymbol{X}^\top\boldsymbol{X}\right)^{-1}\boldsymbol{X}^\top\boldsymbol{t}$, а можно и σ^2 оценивать как параметр модели, найдя гипотезу максимального правдоподобия

$$\sigma_{\mathrm{ML}}{}^2 = \frac{1}{N}\sum_{n=1}^{N}\left(t_n - \boldsymbol{w}_{\mathrm{ML}}^\top\boldsymbol{x}_n\right)^2.$$

Давайте в этом разделе для разнообразия рассмотрим именно второй вариант. Тогда при подстановке гипотезы максимального правдоподобия мы из выражения (6.47) получим

$$\ell(\boldsymbol{w}_{\mathrm{ML}}) = -\frac{N}{2}\log\left(2\pi\sigma_{\mathrm{ML}}{}^2\right) - \frac{N}{2},$$

а число параметров в модели составит $d+2$, где d — степень многочлена. Поэтому критерий Акаике будет выглядеть как

$$\mathrm{AIC}(d) = N\left(\log(2\pi) + \log\sigma_{\mathrm{ML}}{}^2 + 1\right) + 2(d+2),$$

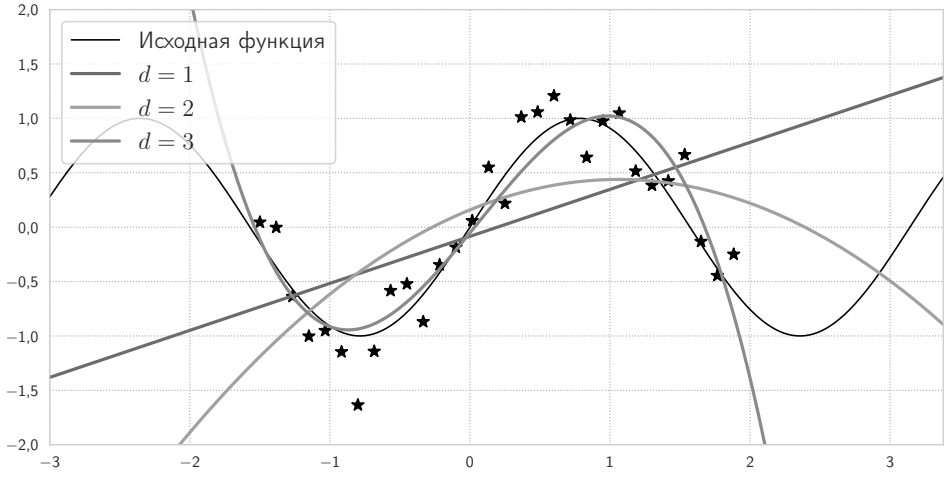


Рис. 6.26. Пример полиномиальной регрессии

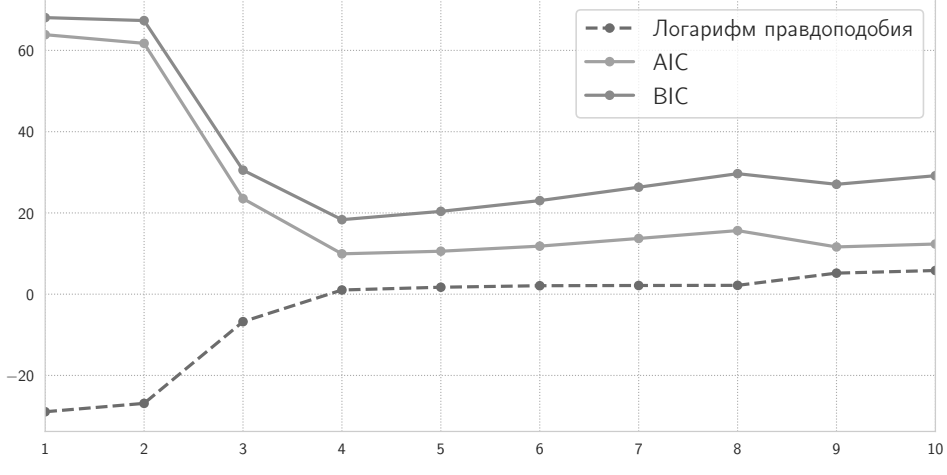


Рис. 6.27. Значения логарифма правдоподобия, AIC и BIC для примера на рис. 6.26

а байесовский информационный критерий — как

$$\mathrm{BIC}(d) = N\left(\log(2\pi) + \log {\sigma_{\mathrm{ML}}}^2 + 1\right) + (d+2)\log N.$$

На рис. 6.26 показан пример нашей обычной задачи полиномиальной регрессии, а на рис. 6.27 — значения информационных критериев вместе со значениями логарифма правдоподобия для разных полиномиальных моделей. В данном случае, как видите, оптимальные значения и AIC, и BIC достигаются при $d = 5$, а общая форма этих информационных критериев весьма характерна: они быстро убывают от $d = 2$ к $d = 3$, потому что многочлен степени 3 уже гораздо лучше описывает данные, а после $d = 5$ начинают медленно возрастать, указывая на то, что дополнительные степени свободы уже не нужны.

Информационные критерии дают разные способы скорректировать оверфиттинг, который получился бы из чистой оценки максимального правдоподобия. Но это, конечно, не единственный возможный подход к выбору моделей. Вот, например, два широко известных метода, о которых мы не говорили:

- принцип *минимальной длины описания* (minimum description length, MDL) рассматривает выбор моделей с теоретико-информационной точки зрения и пытается применить «очень продвинутую бритву Оккама», предпочитая модели, которые могут записать данные максимально коротко [493]; это связано с понятием *колмогоровской сложности*, но, конечно, на практике использует другие аппроксимации, потому что колмогоровскую сложность подсчитать на реальных данных возможным не представляется [425, 494];
- метод *минимизации структурного риска* (structural risk minimization, SRM) даёт другой подход к тому, как сбалансировать сложность модели и точность описания обучающей выборки; этот метод был предложен советскими учёными Владимиром Вапником и Алексеем Червоненкисом в середине 1970-х годов [676] и представляет собой добавление регуляризатора к модели, а суть метода заключается в том, как правильно выбрать коэффициент регуляризации [565, 566, 616].

Но их мы подробнее рассматривать не будем, а перейдём к заключительным разделам, обобщающим многое из того, что вы увидели в этой книге.

6.6. Экспоненциальное семейство

У Марьи, жены брата Кирьяка, было шестеро детей, у Фёклы, жены брата Дениса, ушедшего в солдаты, — двое; и когда Николай, войдя в избу, увидел все семейство, все эти большие и маленькие тела, которые шевелились на полатях, в люльках и во всех углах, и когда увидел, с какою жадностью старик и бабы ели чёрный хлеб, макая его в воду, то сообразил, что напрасно он сюда приехал, больной, без денег да ещё с семьей, — напрасно!

Антон Чехов. Мужики

6.6.1. Определение и примеры

Близится конец книги, и в конце хочется так или иначе подвести итог всему, о чём мы в этой книге говорили. Поэтому последние два раздела будут посвящены... нет, не подведению итогов, этим мы займёмся в заключении, а *обобщению* всего того, о чём шла речь в большей части книги.

Во-первых, введём понятие, обобщающее многие распределения вероятностей, которые часто встречаются в машинном обучении. Мы уже несколько раз видели общий паттерн: найти правдоподобие, посмотреть на его форму (точнее, обычно на форму его логарифма) и догадаться, как должно выглядеть семейство сопряжённых априорных распределений. Это выглядит как достаточно несложная процедура, которая должна обобщаться; и действительно, как мы сейчас увидим, её можно обобщить по крайней мере на очень широкий класс распределений.

Этот класс носит название *экспоненциального семейства* распределений (exponential family). Начнём с определения: параметрическое семейство распределений принадлежит экспоненциальному семейству, если оно имеет вид

$$p(\boldsymbol{x}|\theta) = h(\boldsymbol{x})e^{\boldsymbol{\eta}(\boldsymbol{\theta})^\top \boldsymbol{t}(\boldsymbol{x}) - a(\boldsymbol{\theta})} = h(\boldsymbol{x})g(\boldsymbol{\theta})e^{\boldsymbol{\eta}(\boldsymbol{\theta})^\top \boldsymbol{t}(\boldsymbol{x})} \tag{6.48}$$

для некоторого параметра $\boldsymbol{\theta}$; здесь $g(\boldsymbol{\theta}) = e^{-a(\boldsymbol{\theta})}$, просто иногда удобнее записывать (6.48) в логарифмической форме, а иногда нет. Векторная функция $\boldsymbol{t}(\boldsymbol{x})$ выделяет *достаточные статистики* (sufficient statistics; ниже вы увидите, почему они так называются), и она играет роль извлечения из $\boldsymbol{x}$ признаков, которые затем нужно будет скалярно умножить на функцию от параметров.

Если $\boldsymbol{\eta}(\boldsymbol{\theta}) = \boldsymbol{\theta}$, то такая параметризация называется *естественной*, а $\boldsymbol{\theta}$ в таком случае называется *естественным параметром* (natural parameter):

$$p(\boldsymbol{x}|\boldsymbol{\theta}) = h(\boldsymbol{x})e^{\boldsymbol{\theta}^\top \boldsymbol{t}(\boldsymbol{x}) - a(\boldsymbol{\theta})} = h(\boldsymbol{x})g(\boldsymbol{\theta})e^{\boldsymbol{\theta}^\top \boldsymbol{t}(\boldsymbol{x})}. \tag{6.49}$$

Как видите, определение (6.49) выглядит очень общим. Основное и, по сути, единственное предположение здесь в том, как $\boldsymbol{\theta}$ и $\boldsymbol{x}$ разделяются в этом определении: в экспоненте они связаны друг с другом линейно, а вне экспоненты полностью разнесены по функциям $h(\boldsymbol{x})$ и $g(\boldsymbol{\theta})$, то есть единственная зависимость между $\boldsymbol{x}$ и $\boldsymbol{\theta}$ — это скалярное произведение в экспоненте. Главная прелесть экспоненциального семейства в том, что оно определяет очень правильную степень общности: с одной стороны, в него естественным образом погружаются практически все интересующие нас распределения, а с другой стороны, лежащие в его основе предположения всё же позволяют сделать интересные общие замечания о распределениях, принадлежащих к экспоненциальному семейству.

Начнём с первого пункта: давайте покажем, что уже знакомые и привычные для нас распределения вероятностей, как дискретные, так и непрерывные, принадлежат экспоненциальному семейству. Например, распределение Бернулли, то есть бросание монетки. Давайте сразу рассмотрим его естественное обобщение, биномиальное распределение (см. раздел 3.1)

$$\text{Binom}\,(k|n,p) = \binom{n}{k} p^k\,(1-p)^{n-k}.$$

Его можно переписать как

$$\text{Binom}\,(k|n,p) = \binom{n}{k} e^{k\log p + (n-k)\log(1-p)} = \binom{n}{k} e^{k\log\frac{p}{1-p} + n\log(1-p)}.$$

В итоге получается, что биномиальное распределение принадлежит экспоненциальному семейству, и его естественный параметр — это

$$\theta = \log\frac{p}{1-p}, \qquad p = \frac{e^\theta}{1+e^\theta},$$

то есть в точности те самые log-odds, которые мы подробно обсуждали в разделе 5.2! Функции из формулы (6.49) для биномиального распределения определяются как $t(k) = k$ (выделять признаки не потребовалось), $h(k) = \binom{n}{k}$, и

$$a(\theta) = -n\log(1-p) = n\log(1+e^\theta), \text{ то есть } g(\theta) = e^{n\log(1-p)} = (1+e^\theta)^{-n}.$$

Аналогично мультиномиальное распределение, то есть распределение бросков кубика

$$\text{Mult}\,(\boldsymbol{x}|n,p_1,p_2,\ldots,p_k) = \begin{cases} \frac{n!}{x_1!x_2!\ldots x_k!}p_1^{x_1}p_2^{x_2}\ldots p_k^{x_k}, & \text{если } \sum_{i=1}^k x_i = n, \\ 0 & \text{в противном случае,} \end{cases}$$

можно переписать в соответствующем экспоненциальному семейству виде как (будем везде считать, что $\sum_{i=1}^k x_i = n$)

$$\text{Mult}\,(\boldsymbol{x}|n, p_1, p_2, \ldots, p_k) = \frac{n!}{x_1!x_2!\ldots x_k!} e^{\sum_{i=1}^k x_i \log p_i},$$

то есть на первый взгляд кажется, что в экспоненциальном семействе здесь

$$\boldsymbol{t}(\boldsymbol{x}) = \boldsymbol{x}, \quad \boldsymbol{\theta} = \log \boldsymbol{p}, \quad a(\boldsymbol{\theta}) = 0, \quad h(\boldsymbol{x}) = \frac{n!}{x_1!x_2!\ldots x_k!}.$$

Но такое представление, как вы увидите ниже, ведёт к техническим трудностям из-за того, что $a(\boldsymbol{\theta}) = 0$. В данном случае это происходит потому, что мы получаем представление в пространстве векторов $\boldsymbol{\theta} \in \mathbb{R}^k$ с дополнительным условием $\sum_i \boldsymbol{\theta}_i = 1$ вместо минимального представления в пространстве векторов $\boldsymbol{\theta} \in \mathbb{R}^{k-1}$. Давайте запишем минимальное представление, выразив p_k и x_k через остальные p_i и x_i соответственно:

$$e^{\sum_{i=1}^k x_i \log p_i} = e^{\sum_{i=1}^{k-1} x_i \log p_i + \left(n - \sum_{i=1}^{k-1} x_i\right) \log\left(1 - \sum_{i=1}^{k-1} p_i\right)} =$$

$$= e^{\sum_{i=1}^{k-1} x_i \log\left(\frac{p_i}{1-\sum_{i=1}^{k-1} p_i}\right) + n \log\left(1 - \sum_{i=1}^{k-1} p_i\right)}.$$

Таким образом, в итоге достаточные статистики выглядят так же, $\boldsymbol{t}(\boldsymbol{x}) = \boldsymbol{x}$, но естественный параметр мультиномиального распределения в минимальном представлении получается равным

$$\theta_i = \log\left(\frac{p_i}{1 - \sum_{i=1}^{k-1} p_i}\right) = \log\frac{p_i}{p_k}, \quad p_i = \frac{e^{\theta_i}}{\sum_{j=1}^k e^{\theta_j}},$$

и теперь

$$a(\boldsymbol{\theta}) = -n \log\left(1 - \sum_{i=1}^{k-1} p_i\right) = n \log\left(\sum_{j=1}^{k} e^{\theta_j}\right).$$

Обратите внимание, что в обратном выражении для p_i через $\boldsymbol{\theta}$ у нас опять получилась как раз та самая softmax-функция, которую мы использовали в логистической регрессии в разделе 5.2. Это, конечно, далеко не случайно: получается, что в качестве вектора весов мы обучали как раз естественный параметр мультиномиального распределения.

С распределением Пуассона совсем нет вопросов: его форма «по умолчанию»

$$p\,(x|\lambda) = \frac{1}{x!}\lambda^x e^{-\lambda} = \frac{1}{x!} e^{x \log \lambda - \lambda}$$

сразу же принадлежит экспоненциальному семейству с естественным параметром, равным логарифму интенсивности:

$$t(x) = x, \quad \theta = \log\lambda, \quad h(x) = \frac{1}{x!}, \quad a(\theta) = \lambda = e^\theta.$$

А редкий пример встречающегося в главе 3 распределения, которое *не* принадлежит экспоненциальному семейству, — это гипергеометрическое распределение

$$p\left(x|N,n,K\right)=\frac{1}{\binom{N}{n}}\binom{K}{x}\binom{N-K}{n-x};$$

его к форме (6.48) преобразовать никак не получится.

Теперь перейдём к непрерывным распределениям. Первое и главное из них, разумеется, нормальное распределение. Давайте преобразуем одномерный гауссиан к виду (6.49):

$$\begin{aligned}\mathcal{N}\left(x\middle|\mu,\sigma^2\right)&=\frac{1}{\sqrt{2\pi\sigma^2}}e^{-\frac{1}{2\sigma^2}(x-\mu)^2}=\\&=\frac{1}{\sqrt{2\pi\sigma^2}}e^{-\frac{1}{2\sigma^2}x^2+\frac{\mu}{\sigma^2}x-\frac{\mu^2}{2\sigma^2}}=\frac{1}{\sqrt{2\pi}}e^{\binom{x^2}{x}^\top\binom{-1/2\sigma^2}{\mu/\sigma^2}-\frac{\mu^2}{2\sigma^2}-\log\sigma}.\end{aligned}$$

Иначе говоря, получается, что одномерное нормальное распределение имеет две достаточные статистики, $\boldsymbol{t}(x)=\binom{x^2}{x}$; это для нас вполне ожидаемо после всего того, что мы успели узнать в этой книге о гауссианах. Естественный параметр, соответственно, тоже имеет размерность два:

$$\boldsymbol{\theta}=\begin{pmatrix}\theta_1\\\theta_2\end{pmatrix}=\begin{pmatrix}-1/2\sigma^2\\\mu/\sigma^2\end{pmatrix}=\begin{pmatrix}-\tau/2\\\mu\tau\end{pmatrix};$$

а остальные функции выглядят как

$$h(x)=\frac{1}{\sqrt{2\pi}},\qquad a(\boldsymbol{\theta})=\frac{\mu^2}{2\sigma^2}+\log\sigma=\frac{\mu^2\tau}{2}-\frac{1}{2}\log\tau=-\frac{\theta_2^2}{4\theta_1^2}-\frac{1}{2}\log(-2\theta_1).$$

С логнормальным распределением теперь разобраться очень просто: у него все функции точно такие же, только в $\boldsymbol{t}(x)$ появляются логарифмы:

$$\boldsymbol{t}(x)=\begin{pmatrix}(\log x)^2\\\log x\end{pmatrix}.$$

Без особого труда в экспоненциальное семейство помещается и многомерный гауссиан, который доставил нам немало проблем в главе 4. Правда, нам теперь нужно вписать в вектор $\boldsymbol{\theta}$ и вектор среднего $\boldsymbol{\mu}$, и матрицу ковариаций Σ, так что в формулах ниже мы будем обозначать через $\operatorname{vec}(A)$ матрицу A, развёрнутую в «плоский» вектор, например в виде последовательной конкатенации всех строк матрицы A; я не буду подробно описывать каждый раз, как именно это происходит, достаточно предполагать, что это происходит одинаково для всех вхождений операции $\operatorname{vec}(\cdot)$.

Запишем плотность многомерного гауссиана в соответствующем виде:

$$\mathcal{N}(\boldsymbol{x}|\boldsymbol{\mu},\Sigma) = \frac{1}{\sqrt{2\pi\det\Sigma}} e^{-\frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu})^\top\Sigma^{-1}(\boldsymbol{x}-\boldsymbol{\mu})} = \\ = e^{-\frac{1}{2}\left(\boldsymbol{x}^\top\Sigma^{-1}\boldsymbol{x}-2\boldsymbol{x}^\top\Sigma^{-1}\mu+\boldsymbol{\mu}^\top\Sigma^{-1}\boldsymbol{\mu}+\log(2\pi\det\Sigma)\right)}.$$

Теперь нужно представить $\boldsymbol{x}^\top\Sigma^{-1}\boldsymbol{x}$ в виде скалярного произведения; здесь как раз и пригодится операция $\operatorname{vec}(\cdot)$:

$$\boldsymbol{x}^\top\Sigma^{-1}\boldsymbol{x} = \sum\nolimits_{i,j=1}^{d}\left(\Sigma^{-1}\right)_{ij} x_i x_j = \operatorname{vec}\left(\boldsymbol{x}\boldsymbol{x}^\top\right)^\top \operatorname{vec}\left(\Sigma^{-1}\right).$$

В итоге получается, что многомерный гауссиан можно вложить в экспоненциальное семейство при помощи следующих функций:

$$\boldsymbol{t}(\boldsymbol{x}) = \begin{pmatrix}\operatorname{vec}\left(\boldsymbol{x}\boldsymbol{x}^\top\right)\\ \boldsymbol{x}\end{pmatrix}, \qquad \boldsymbol{\theta} = \begin{pmatrix}-\frac{1}{2}\operatorname{vec}\left(\Sigma^{-1}\right)\\ \Sigma^{-1}\boldsymbol{\mu}\end{pmatrix},$$
$$h(\boldsymbol{x}) = 1, \qquad a(\boldsymbol{\theta}) = \boldsymbol{\mu}^\top\Sigma^{-1}\boldsymbol{\mu} + \log\left(2\pi\det\Sigma\right).$$

Разве что переписывать $a(\boldsymbol{\theta})$ как явное выражение от компонентов вектора $\boldsymbol{\theta}$ не просите, ничего хорошего там не получится. Все полученные выше результаты подытожены в табл. 6.1, а мы будем двигаться дальше.

6.6.2. Моменты достаточных статистик

Итак, мы поняли, что многие важные и известные распределения вкладываются в экспоненциальное семейство, но точно так же можно было бы сказать, что они вкладываются в семейство *всех* распределений. Чтобы абстракция была полезной, нужно ещё, чтобы можно было установить какие-то общие результаты об этой абстракции. Давайте вернёмся к общей форме экспоненциального семейства (6.48) и попробуем такие результаты получить.

Первый интересный результат будет касаться среднего и дисперсии распределений из экспоненциального семейства; далее будем предполагать, что речь идёт о распределении с естественным параметром, то есть мы будем вводить распределение по формуле (6.49). Давайте начнём с очевидного равенства: поскольку интеграл от любого распределения равен единице,

$$a(\boldsymbol{\theta}) = \log\int h(\boldsymbol{x}) e^{\boldsymbol{\theta}^\top\boldsymbol{t}(\boldsymbol{x})}\mathrm{d}\boldsymbol{x}. \tag{6.50}$$

Теперь возьмём в выражении (6.50) градиент по $\boldsymbol{\theta}$ слева и справа. Ниже мы лихо заносим градиент под знак интеграла — так, конечно, можно делать не всегда, но в данном случае можно, и я не буду подробно останавливаться на том, почему именно; кроме того, мы опять воспользуемся выражением (6.50):

Таблица 6.1. Стандартные распределения как часть экспоненциального семейства

Распределение	**Биномиальное**	**Мультиномиальное**	**Пуассона**
Плотность	$\binom{n}{x} p^x (1-p)^{n-x}$	$\frac{n!}{x_1!\ldots x_k!} p_1^{x_1} \ldots p_k^{x_k}$	$\frac{1}{x!}\lambda^x e^{-\lambda}$
Достаточные статистики $\boldsymbol{t}(\boldsymbol{x})$	x	$\boldsymbol{x}$	x
Естественный параметр $\boldsymbol{\theta}$	$\log \frac{p}{1-p}$	$\theta_i = \log\left(\frac{p_i}{1-\sum_{i=1}^{k-1} p_i}\right)$	$\log\lambda$
Функция $a(\boldsymbol{\theta})$	$n\log(1+e^{\theta})$	$n\log\left(\sum_{j=1}^{k} e^{\theta_j}\right)$	e^{θ}
Функция $h(\boldsymbol{x})$	$\binom{n}{x}$	$\frac{n!}{x_1!\ldots x_k!}$	$\frac{1}{x!}$
Распределение	**Нормальное**	**Многомерное нормальное**	
Плотность	$\frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{1}{2\sigma^2}(x-\mu)^2}$	$\frac{1}{\sqrt{(2\pi)^d \det\Sigma}} e^{-\frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu})^\top \Sigma^{-1}(\boldsymbol{x}-\boldsymbol{\mu})}$	
Достаточные статистики $\boldsymbol{t}(\boldsymbol{x})$	$\begin{pmatrix} x^2 \\ x \end{pmatrix}$	$\begin{pmatrix} \operatorname{vec}\left(\boldsymbol{x}\boldsymbol{x}^\top\right) \\ \boldsymbol{x} \end{pmatrix}$	
Естественный параметр $\boldsymbol{\theta}$	$\begin{pmatrix} -\frac{1}{2\sigma^2} \\ \frac{\mu}{\sigma^2} \end{pmatrix} = \begin{pmatrix} -\frac{\tau}{2} \\ \mu\tau \end{pmatrix}$	$\begin{pmatrix} -\frac{1}{2}\operatorname{vec}\left(\Sigma^{-1}\right) \\ \Sigma^{-1}\boldsymbol{\mu} \end{pmatrix}$	
Функция $a(\boldsymbol{\theta})$	$-\frac{\theta_2^2}{4\theta_1^2} - \frac{1}{2}\log(-2\theta_1)$	$\boldsymbol{\mu}^\top\Sigma^{-1}\boldsymbol{\mu} + \log(2\pi\det\Sigma)$	
Функция $h(\boldsymbol{x})$	$\frac{1}{\sqrt{2\pi}}$	1	

$$\nabla_{\boldsymbol{\theta}} a(\boldsymbol{\theta}) = \nabla_{\boldsymbol{\theta}} \log \int h(\boldsymbol{x}) e^{\boldsymbol{\theta}^\top \boldsymbol{t}(\boldsymbol{x})} \mathrm{d}\boldsymbol{x} = \frac{\int \nabla_{\boldsymbol{\theta}} h(\boldsymbol{x}) e^{\boldsymbol{\theta}^\top \boldsymbol{t}(\boldsymbol{x})} \mathrm{d}\boldsymbol{x}}{\int h(\boldsymbol{x}) e^{\boldsymbol{\theta}^\top \boldsymbol{t}(\boldsymbol{x})} \mathrm{d}\boldsymbol{x}} =$$

$$= \frac{\int h(\boldsymbol{x}) \nabla_{\boldsymbol{\theta}} e^{\boldsymbol{\theta}^\top \boldsymbol{t}(\boldsymbol{x})} \mathrm{d}\boldsymbol{x}}{e^{a(\boldsymbol{\theta})}} = \frac{\int h(\boldsymbol{x}) e^{\boldsymbol{\theta}^\top \boldsymbol{t}(\boldsymbol{x})} \boldsymbol{t}(\boldsymbol{x}) \mathrm{d}\boldsymbol{x}}{e^{a(\boldsymbol{\theta})}} =$$

$$= \int \boldsymbol{t}(\boldsymbol{x}) \left(h(\boldsymbol{x}) e^{\boldsymbol{\theta}^\top \boldsymbol{t}(\boldsymbol{x}) - a(\boldsymbol{\theta})} \right) \mathrm{d}\boldsymbol{x}.$$

В результате мы получили, что $\nabla_{\boldsymbol{\theta}} a(\boldsymbol{\theta})$ — это математическое ожидание достаточных статистик исходного распределения:

$$\mathbb{E}\left[\boldsymbol{t}(\boldsymbol{x})\right] = \nabla_{\boldsymbol{\theta}} a(\boldsymbol{\theta}). \tag{6.51}$$

Это очень мощный результат, который нередко пригождается и в машинном обучении (правда, в этой книге мы уже не успеем им воспользоваться). Именно из-за результата (6.51) функцию $a(\boldsymbol{\theta})$ часто называют *кумулянтом* (cumulant) распределений в экспоненциальном семействе: сама она «аккумулирует» в себе распределение и служит нормировочной константой, а из её градиента получается математическое ожидание достаточных статистик.

Кроме того, для минимальных представлений распределений из экспоненциального семейства, когда в $\boldsymbol{\theta}$ нет лишних параметров, уравнение (6.51) можно обратить, то есть выразить $\boldsymbol{\theta}$ через $\mathbb{E}[\boldsymbol{t}(\boldsymbol{x})]$. В таких случаях вполне естественно будет рассмотреть не только каноническую параметризацию распределения с параметром $\boldsymbol{\theta}$, но и так называемую *параметризацию средним* (mean parametrization), в которой параметром будет

$$\boldsymbol{\mu} = \mathbb{E}[\boldsymbol{t}(\boldsymbol{x})] = \nabla_{\boldsymbol{\theta}} a(\boldsymbol{\theta}). \tag{6.52}$$

Например, привычная нам параметризация гауссиана именно такова, да и во многих других распределениях нам привычно считать его среднее одним из параметров, даже если естественный параметр оказывается другим.

Более того, результат (6.51) можно продолжить и на последующие моменты распределений в экспоненциальном семействе. Давайте рассмотрим матрицу вторых производных $a(\boldsymbol{\theta})$; для простоты давайте запишем её в координатах, а не в матричной форме, то есть найдём $\frac{\partial^2 a}{\partial\theta_i \partial\theta_j}$.

В приведённом ниже выводе мы дважды пользуемся результатом (6.51), в начале и в конце вывода, а также снова бесстрашно заносим производную под знак интеграла:

$$\begin{aligned}
\frac{\partial^2 a}{\partial\theta_i\partial\theta_j} = \frac{\partial \mathbb{E}[t_i(\boldsymbol{x})]}{\partial\theta_j} &= \frac{\partial}{\partial\theta_j}\int t_i(\boldsymbol{x}) h(\boldsymbol{x}) e^{\boldsymbol{\theta}^\top \boldsymbol{t}(\boldsymbol{x}) - a(\boldsymbol{\theta})}\mathrm{d}\boldsymbol{x} = \\
&= \int t_i(\boldsymbol{x}) h(\boldsymbol{x}) \frac{\partial e^{\boldsymbol{\theta}^\top \boldsymbol{t}(\boldsymbol{x}) - a(\boldsymbol{\theta})}}{\partial\theta_j}\mathrm{d}\boldsymbol{x} = \\
&= \int t_i(\boldsymbol{x}) h(\boldsymbol{x}) e^{\boldsymbol{\theta}^\top \boldsymbol{t}(\boldsymbol{x}) - a(\boldsymbol{\theta})}\left(t_j(\boldsymbol{x}) - \frac{\partial a(\boldsymbol{\theta})}{\partial\theta_j}\right)\mathrm{d}\boldsymbol{x} = \\
&= \int t_i(\boldsymbol{x})\left(t_j(\boldsymbol{x}) - \mathbb{E}\left[t_j(\boldsymbol{x})\right]\right) h(\boldsymbol{x}) e^{\boldsymbol{\theta}^\top \boldsymbol{t}(\boldsymbol{x}) - a(\boldsymbol{\theta})}\mathrm{d}\boldsymbol{x} = \\
&= \mathbb{E}\left[t_i(\boldsymbol{x}) t_j(\boldsymbol{x})\right] - \mathbb{E}[t_i(\boldsymbol{x})]\,\mathbb{E}\left[t_j(\boldsymbol{x})\right].
\end{aligned}$$

Иначе говоря, у нас получилось, что матрица вторых производных (гессиан) функции $a(\boldsymbol{\theta})$ — это в точности матрица ковариаций вектора достаточных статистик $\boldsymbol{t}(\boldsymbol{x})$:

$$\operatorname{Var}[\boldsymbol{t}(\boldsymbol{x})] = \mathbf{H}(a(\boldsymbol{\theta})) = \left(\frac{\partial^2 a}{\partial\theta_i\partial\theta_j}\right)_{i,j=1}^{k}. \tag{6.53}$$

Аналогичные результаты можно получить и для последующих моментов.

6.6.3. Максимизация правдоподобия

Первый результат об экспоненциальном семействе был общим и для нас вспомогательным, но второй уже приведёт нас ближе к задачам машинного обучения.

Давайте попробуем записать логарифм правдоподобия нескольких точек данных для некоторого распределения из экспоненциального семейства:

$$p\left(\boldsymbol{x}_1,\ldots,\boldsymbol{x}_N|\theta\right) = \prod_{n=1}^{N} h(\boldsymbol{x}_n)e^{\boldsymbol{\eta}(\boldsymbol{\theta})^\top \boldsymbol{t}(\boldsymbol{x}_n)-a(\boldsymbol{\theta})}, \tag{6.54}$$

$$\log p\left(\boldsymbol{x}_1,\ldots,\boldsymbol{x}_N|\theta\right) = \sum_{n=1}^{N} \log h(\boldsymbol{x}_n) + \boldsymbol{\eta}(\boldsymbol{\theta})^\top \left(\sum_{n=1}^{N} \boldsymbol{t}(\boldsymbol{x}_n)\right) - Na(\boldsymbol{\theta}). \tag{6.55}$$

Как видите, у нас получилось, что всё, что в логарифме правдоподобия зависит от $\boldsymbol{\theta}$, содержит $\boldsymbol{x}_n$ только в виде $\sum_{n=1}^{N} \boldsymbol{t}(\boldsymbol{x}_n)$. Иначе говоря, для того, чтобы найти гипотезу максимального правдоподобия для распределения из экспоненциального семейства, достаточно сохранять суммы $\sum_{n=1}^{N} \boldsymbol{t}(\boldsymbol{x}_n)$ и число N, а сами точки $\boldsymbol{x}_n$ можно «забывать», они не понадобятся; собственно, именно поэтому значения $\boldsymbol{t}(\boldsymbol{x}_n)$ называются *достаточными статистиками*. Например, как мы уже давно знаем, для одномерного гауссиана достаточно хранить $\sum_{n=1}^{N} x_n$ и $\sum_{n=1}^{N} x_n^2$, и из них можно найти гипотезу максимального правдоподобия и для параметра μ, и для σ^2.

То же самое можно сказать другими словами: в вероятностной модели, правдоподобие которой лежит в экспоненциальном семействе, вектор параметров $\boldsymbol{\theta}$ и данные $\boldsymbol{x}_1,\ldots,\boldsymbol{x}_n$ условно независимы при условии известных достаточных статистик $\boldsymbol{t}(\boldsymbol{x})$.

Более того, и для обратного утверждения существует так называемая *теорема Купмана — Питмана — Дармуа* (Pitman — Koopman — Darmois theorem)[1], которая утверждает, что при некоторых условиях регулярности экспоненциальное семейство — это *единственное* семейство распределений с конечным набором достаточных статистик, то есть с набором достаточных статистик, размер которого не зависит от числа точек данных N.

Записанный выше логарифм правдоподобия (6.55) показывает и то, что саму гипотезу максимального правдоподобия найти обычно несложно — нужно взять от (6.55) градиент по $\boldsymbol{\theta}$ и приравнять его к нулю, то есть решить уравнение

$$\nabla_{\boldsymbol{\theta}} a(\boldsymbol{\theta}) = \frac{1}{N} \nabla_{\boldsymbol{\theta}} \boldsymbol{\eta}(\boldsymbol{\theta})^\top \left(\sum_{n=1}^{N} \boldsymbol{t}(\boldsymbol{x}_n)\right). \tag{6.56}$$

Учитывая, что обычно речь идёт о естественном параметре, то есть $\boldsymbol{\eta}(\boldsymbol{\theta}) = \boldsymbol{\theta}$, и тогда в правой части формулы (6.56) нет никакой зависимости от $\boldsymbol{\theta}$, уравнение (6.56) решить обычно получается достаточно легко, или аналитически, или

[1] Она называется так потому, что трое учёных — Жорж Питман, Бернард Купман и Эдвин Джеймс Питман — независимо получили разные варианты этого утверждения примерно одновременно, в середине 1930-х годов [132, 314, 470]. Понять, почему в русском и английском языках авторы идут в разном порядке, мне не удалось.

хотя бы численно. Мы уже много раз в этой книге находили гипотезы максимального правдоподобия для распределений из экспоненциального семейства (особенно в разделах 3.1 и 3.2), и это каждый раз оказывалось несложно.

А в терминах параметризации средним у нас и вовсе сразу же получается ответ: если $\boldsymbol{\eta}(\boldsymbol{\theta}) = \boldsymbol{\theta}$, то

$$\boldsymbol{\mu} = \nabla_{\boldsymbol{\theta}} a(\boldsymbol{\theta}) = \frac{1}{N}\sum\nolimits_{n=1}^{N} \boldsymbol{t}(\boldsymbol{x}_n). \tag{6.57}$$

6.6.4. Сопряжённые априорные и предсказательные распределения

Третий интересный результат об экспоненциальном семействе сразу же помещает нас в контекст байесовского вывода: оказывается, что есть универсальный способ найти семейство сопряжённых априорных распределений для любого распределения из экспоненциального семейства. Это не очень сложно увидеть, пристально посмотрев на формулу (6.55) для логарифма правдоподобия датасета: логарифм сопряжённого априорного распределения тоже должен иметь примерно такую же форму и достаточно параметров, чтобы в них поместились N и достаточные статистики.

Формально говоря, для семейства распределений с параметром $\boldsymbol{\theta}$, заданного в (6.48), семейством сопряжённых априорных распределений будет семейство

$$p\left(\boldsymbol{\theta}|\boldsymbol{\chi}, \nu\right) = f(\boldsymbol{\chi},\nu) e^{\boldsymbol{\chi}^\top \boldsymbol{\eta}(\boldsymbol{\theta}) - \nu a(\boldsymbol{\theta})}, \tag{6.58}$$

где $\boldsymbol{\chi}$ и ν — гиперпараметры, функция $a(\boldsymbol{\theta})$ та же самая, что в исходном распределении, а функция $f(\boldsymbol{\chi}, \nu)$ — это нормировочная константа.

Давайте проверим, что (6.58) действительно задаёт сопряжённое априорное распределение, а заодно и найдём апостериорное распределение в этом случае:

$$p\left(\boldsymbol{\theta}|\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N, \boldsymbol{\chi}_0, \nu_0\right) \propto p\left(\boldsymbol{\theta}|\boldsymbol{\chi}_0, \nu_0\right) p\left(\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N|\boldsymbol{\theta}\right).$$

Подставляя результаты (6.55) и (6.58), мы получим

$$\begin{aligned}\log p\left(\boldsymbol{\theta}|\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N, \boldsymbol{\chi}_0, \nu_0\right) = \sum\nolimits_{n=1}^{N} \log h(\boldsymbol{x}_n) + \boldsymbol{\eta}(\boldsymbol{\theta})^\top \left(\sum\nolimits_{n=1}^{N} \boldsymbol{t}(\boldsymbol{x}_n)\right) - \\ - N a(\boldsymbol{\theta}) + \log f(\boldsymbol{\chi}_0, \nu_0) + \boldsymbol{\chi}_0^\top \boldsymbol{\eta}(\boldsymbol{\theta}) - \nu_0 a(\boldsymbol{\theta}) + \text{Const} = \\ = \text{Const} + \left(\boldsymbol{\chi}_0 + \sum\nolimits_{n=1}^{N} \boldsymbol{t}(\boldsymbol{x}_n)\right)^\top \boldsymbol{\eta}(\boldsymbol{\theta}) - (N + \nu_0)\, a(\boldsymbol{\theta}),\end{aligned}$$

и в итоге у нас получилось апостериорное распределение $p\left(\boldsymbol{\theta}|\boldsymbol{\chi}_N, \nu_N\right)$ того же вида, что (6.58), но с параметрами

$$\nu_N = \nu_0 + N, \qquad \boldsymbol{\chi}_N = \boldsymbol{\chi}_0 + \sum\nolimits_{n=1}^{N} \boldsymbol{t}(\boldsymbol{x}_n). \tag{6.59}$$

Здесь мы снова видим, что для пересчёта апостериорного распределения достаточно знать не сами значения $\boldsymbol{x}_n$, а только $\sum_{n=1}^{N} \boldsymbol{t}(\boldsymbol{x}_n)$, то есть те самые достаточные статистики.

Теоретически мы могли бы начать книгу с этого вывода, а затем получить все сопряжённые априорные распределения, которые встречались в предыдущих главах, из формулы (6.58). Но в жизни всё-таки полезно думать о разных распределениях как о разных сущностях и понимать их свойства, да и поиск сопряжённых априорных распределений до сих пор не представлял особых трудностей. А формула (6.58) оказывается полезной в случаях, когда напрямую придумать сопряжённое априорное распределение сложно; например, именно так обычно получают сопряжённое априорное распределение для распределения Висхарта, которое является многомерным обобщением гамма-распределения.

И даже более того, для сопряжённого априорного распределения к распределению из экспоненциального семейства можно и предсказательное распределение найти в общем виде! Вывод похож на все выводы предсказательных распределений, которые вы уже видели в этой книге; на последнем шаге мы сворачиваем интеграл по распределению (6.58) к его нормировочной константе:

$$
\begin{aligned}
p\left(\boldsymbol{x}|\boldsymbol{x}_1,\ldots,\boldsymbol{x}_n,\boldsymbol{\chi}_0,\nu_0\right) &= \int p\left(\boldsymbol{x}|\boldsymbol{\theta}\right) p\left(\boldsymbol{\theta}|\boldsymbol{x}_1,\ldots,\boldsymbol{x}_N,\boldsymbol{\chi}_0,\nu_0\right) \mathrm{d}\boldsymbol{\theta} = \\
&= \int h(\boldsymbol{x}) e^{\boldsymbol{t}(\boldsymbol{x})^\top \boldsymbol{\eta}(\boldsymbol{\theta}) - a(\boldsymbol{\theta})} f(\boldsymbol{\chi}_N,\nu_N) e^{\boldsymbol{\chi}_N^\top \boldsymbol{\eta}(\boldsymbol{\theta}) - \nu_N a(\boldsymbol{\theta})} \mathrm{d}\boldsymbol{\theta} = \\
&= f(\boldsymbol{\chi}_N,\nu_N) \int h(\boldsymbol{x}) e^{(\boldsymbol{t}(\boldsymbol{x})+\boldsymbol{\chi}_N)^\top \boldsymbol{\eta}(\boldsymbol{\theta}) - (\nu_N+1)a(\boldsymbol{\theta})} = \frac{f(\boldsymbol{\chi}_N,\nu_N)}{f(\boldsymbol{\chi}_N + \boldsymbol{t}(\boldsymbol{x}),\nu_N+1)}.
\end{aligned}
$$

Получилось, что предсказательное распределение — это отношение нормировочных констант для сопряжённого априорного распределения (6.48) с разными параметрами; сравните это, например, с выводом в разделе 2.4.

Наконец, отмечу, не углубляясь в детали, что для распределений из экспоненциального семейства можно вычислить и матрицу информации Фишера, а значит, найти априорные распределения Джеффриса, о которых мы говорили в подразделе 3.4.5. Более того, есть общие результаты о том, что априорные распределения Джеффриса в моделях из экспоненциального семейства уменьшают смещение максимальных апостериорных оценок по сравнению с оценками максимального правдоподобия, убирая из этого смещения самый асимптотически большой член [181, 370, 634]. Но это уже выходит далеко за рамки нашего вводного курса, так что ограничусь ссылками, а мы с вами будем двигаться к обобщённым линейным моделям.

6.7. Обобщённые линейные модели

Мозги туземцев туги на обобщения, и необходимо, чтобы факты повторялись часто, пока явится настоящее понимание.

Джек Лондон. Мудрость снежной тропы (пер. Е. Гуро)

Нам ещё рано гнать искусства, они у нас ещё своего дела не сделали. Чтобы образовать нацию, не довольно выстроить университеты. Умы, чтобы быть готовыми к восприятию научных истин, нуждаются в предварительной культуре. Процессы обобщения и отвлечения не сразу даются мозгу; они должны быть подготовлены. Обобщения, представляемые искусством, легче воспринимаются и постигаются и, практикуя ум, подготовляют его к научным отвлечениям...

Александр Островский. Мысли о драматическом искусстве

6.7.1. Определение GLM

В предыдущем разделе мы обобщили многое из того, что вы узнали о разных распределениях и байесовском выводе для этих распределений. А здесь давайте попробуем обобщить происходящее в линейных моделях, которые рассматривались в главе 4 для задачи регрессии и в главе 5, точнее, в разделе 5.2 для классификации. Кажется, что и линейная, и логистическая регрессия имеют одну и ту же форму: и там и там мы строим предсказания как

$$\hat{y} = h(\boldsymbol{w}^\top \boldsymbol{x}),$$

только для разных функций h: $h(a) = a$ для линейной регрессии и $h(a) = \sigma(a)$ для логистической регрессии (в бинарном случае).

И действительно, такое обобщение существует; оно носит название *обобщённых линейных моделей* (generalized linear models, GLM) и уже давно изучается в статистике и используется в многочисленных приложениях [158, 369, 385]. Теперь, когда вы знакомы с экспоненциальным семейством, прийти к обобщённым линейным моделям будет очень легко.

Начнём с того, что определим линейную функцию от входов, $c = \boldsymbol{w}^\top \boldsymbol{x}$, а затем определим среднее интересующего нас распределения μ как функцию от c.

Обычно задают обратную к ней функцию

$$c = g(\mu), \quad \text{то есть} \quad \mu = g^{-1}(c);$$

g называется *функцией связи* (link function).

В качестве g можно выбрать практически любую обратимую функцию. Например, как мы уже обсудили выше, в линейной регрессии $g = g^{-1} = \mathrm{id}$, а в бинарной логистической регрессии

$$g^{-1}(a) = \sigma(a), \quad \text{то есть} \quad g(\mu) = \log\frac{\mu}{1-\mu}.$$

Чтобы получить обобщённую линейную модель, нужно определить одномерное распределение $p(y|\boldsymbol{x},\boldsymbol{w})$ со средним μ, которое бы использовало $g^{-1}(\boldsymbol{w}^\top\boldsymbol{x})$ как достаточную статистику. Правда, в данном случае мы хотим научиться контролировать не только среднее, но и дисперсию на выходе. Поэтому давайте вместо общего вида экспоненциального семейства, о котором шла речь в разделе 6.6, будем рассматривать распределение вида

$$p\left(y\middle|\theta,\sigma^2\right) = h(y,\sigma^2)e^{\frac{y\theta-a(\theta)}{\sigma^2}}, \tag{6.60}$$

где σ^2 — параметр разброса (dispersion parameter); это стандартное обозначение, но оно может несколько ввести в заблуждение, потому что мы будем относиться к σ^2 как к константе, а параметром является только θ.

Само семейство распределений вида (6.60) иногда называют *дисперсным экспоненциальным семейством* (overdispersed exponential family); это, конечно, не обобщение (6.48), а его частный случай, в котором $t(y) = \frac{y}{\sigma^2}$, а кумулянт равен $\frac{1}{\sigma^2}a(\theta)$. Тогда доказанный нами в разделе 6.6 результат (6.52) сразу даёт среднее

$$\mathbb{E}\left[\frac{y}{\sigma^2}\right] = \frac{\partial\left(\frac{1}{\sigma^2}a(\theta)\right)}{\partial\theta}, \quad \text{то есть} \quad \mathbb{E}[y] = \frac{\partial a(\theta)}{\partial\theta},$$

а результат (6.53) даёт дисперсию

$$\mathrm{Var}\left[\frac{y}{\sigma^2}\right] = \frac{1}{\sigma^4}\mathrm{Var}[y] = \frac{\partial^2\left(\frac{1}{\sigma^2}a(\theta)\right)}{\partial\theta^2}, \quad \text{то есть} \quad \mathrm{Var}[y] = \sigma^2\frac{\partial^2 a(\theta)}{\partial\theta^2}.$$

Осталось только договориться, что такое здесь θ; обобщённые линейные модели потому и называются линейными, что $\theta = \boldsymbol{w}^\top\boldsymbol{x}$. Это значит, что функция связи здесь определяется как

$$g^{-1}(\boldsymbol{w}^\top\boldsymbol{x}) = \mu = \mathbb{E}[y|\boldsymbol{x},\boldsymbol{w}] = a'(\boldsymbol{w}^\top\boldsymbol{x}), \qquad g(\mu) = \boldsymbol{w}^\top\boldsymbol{x}. \tag{6.61}$$

6.7.2. Примеры и максимизация правдоподобия

Как и в предыдущем параграфе, давайте начнём с того, что погрузим в это обобщение уже знакомые нам примеры. Так, для линейной регрессии

$$p\left(y\middle|\mu,\sigma^2\right) = e^{-\frac{(y-\mu)^2}{2\sigma^2}-\frac{1}{2}\log\left(2\pi\sigma^2\right)} = e^{\frac{y\mu-\frac{1}{2}\mu^2}{\sigma^2}-\left(\frac{y^2}{2\sigma^2}+\frac{1}{2}\log\left(2\pi\sigma^2\right)\right)},$$

поэтому для линейной регрессии логично положить

$$g = g^{-1} = \mathrm{id}, \quad \theta = \mu = \boldsymbol{w}^\top\boldsymbol{x}, \quad a(\theta) = -\frac{1}{2}\mu^2, \quad h(y,\sigma^2) = e^{-\left(\frac{y^2}{2\sigma^2}+\frac{1}{2}\log\left(2\pi\sigma^2\right)\right)}.$$

Второй естественный пример — испытания Бернулли. Здесь средним выступает вероятность выпадения орла $\mu = \mathbb{E}\left[y|\boldsymbol{x}\right]$, и, как мы уже выясняли в разделе 6.6,

$$p\left(y|\mu\right) = \mu^y\left(1-\mu\right)^{1-y} = e^{y\log\frac{\mu}{1-\mu}+\log(1-\mu)},$$

то есть в данном случае можно положить

$$\sigma^2 = 1, \quad \theta = \log\frac{\mu}{1-\mu}, \quad a(\theta) = -\log(1-\mu), \quad h(y,\sigma^2) = 1.$$

Вот мы и пришли к вложению логистической регрессии в обобщённые линейные модели:

$$\mu = \frac{e^\theta}{1+e^\theta}, \quad a(\theta) = -\log\left(1-\frac{e^\theta}{1+e^\theta}\right) = \log\left(1+e^\theta\right),$$

а значит,

$$g^{-1}(\theta) = a'(\theta) = \frac{e^\theta}{1+e^\theta} = \sigma(\theta), \quad \text{и} \quad g(\mu) = \log\frac{\mu}{1-\mu} = \sigma^{-1}(\mu).$$

Но эти два примера мы знали и так; давайте же получим что-нибудь новое! Ниже я приведу два примера, которые покажут, как использовать обобщённые линейные модели для того, чтобы обработать ситуации, которые у нас до сих пор обрабатывать не получалось. Но для того, чтобы извлечь из этих примеров что-то полезное, давайте сначала поговорим о том, как вести вывод в обобщённых линейных моделях.

Оказывается, что вести его и в общем случае можно точно так же, как мы это делали для логистической регрессии, — всё будет не совсем так же просто, как для линейной регрессии, но и чего-то более сложного, чем раздел 5.2, нас тоже не ожидает. Логарифм правдоподобия обобщённой линейной модели (6.60), где $\theta = \boldsymbol{w}^\top\boldsymbol{x}$, выглядит как

$$\log p\left(y\middle|\boldsymbol{x},\boldsymbol{w},\sigma^2\right)=\log h(y,\sigma^2)+\frac{y\boldsymbol{w}^\top\boldsymbol{x}-a(\boldsymbol{w}^\top\boldsymbol{x})}{\sigma^2},$$

и, соответственно, логарифм правдоподобия набора данных $D=\{(\boldsymbol{x}_n,y_n)\}_{n=1}^N$ равен

$$\ell(\boldsymbol{w})=\log p\left(y_1,\ldots,y_N\middle|\boldsymbol{x}_1,\ldots,\boldsymbol{x}_N,\boldsymbol{w},\sigma^2\right)=$$
$$=\text{Const}+\frac{1}{\sigma^2}\sum_{n=1}^N\left(y_n\boldsymbol{w}^\top\boldsymbol{x}_n-a(\boldsymbol{w}^\top\boldsymbol{x}_n)\right). \quad (6.62)$$

От обеих частей формулы (6.62) теперь можно взять градиент:

$$\nabla_{\boldsymbol{w}}\ell(\boldsymbol{w})=\frac{1}{\sigma^2}\sum_{n=1}^N\left(y_n-a'(\boldsymbol{w}^\top\boldsymbol{x}_n)\right)\boldsymbol{x}_n=\frac{1}{\sigma^2}\sum_{n=1}^N(y_n-\mu_n)\,\boldsymbol{x}_n, \quad (6.63)$$

где мы подставили $\mu_n=\mathbb{E}\,[y_n|\boldsymbol{x}_n,\boldsymbol{w}]=a'(\boldsymbol{w}^\top\boldsymbol{x}_n)$ согласно определению (6.61). Иначе говоря, мы снова, как и в разделе 5.2, получили в градиенте сумму входных векторов $\boldsymbol{x}_n$, взвешенных их ошибками, то есть отклонениями от ожидаемого среднего.

Результат (6.63) можно использовать напрямую в (стохастическом) градиентном спуске. А можно, опять же в точности как в разделе 5.2, сделать следующий шаг и перейти к методу второго порядка:

$$\mathbf{H}\,(\ell)=-\frac{1}{\sigma^2}\sum_{n=1}^N\frac{\partial\mu_n}{\partial\theta_n}\boldsymbol{x}_n\boldsymbol{x}_n^\top=-\frac{1}{\sigma^2}\boldsymbol{X}^\top\mathbf{S}\boldsymbol{X},$$

где матрица $\mathbf{S}$ — это диагональная матрица весов, составленная из производных обратной функции связи:

$$\mathbf{S}=\text{diag}\left(\frac{\partial\mu_1}{\partial\theta_1},\frac{\partial\mu_2}{\partial\theta_2},\ldots,\frac{\partial\mu_N}{\partial\theta_N}\right).$$

Аналогично разделу 5.2 и в полном соответствии с формулой (5.7) в данном случае мы получим новый вариант метода итеративных перевзвешенных наименьших квадратов:

$$\boldsymbol{w}^{\text{new}}=\boldsymbol{w}^{\text{old}}-\left(\boldsymbol{X}^\top\mathbf{S}\boldsymbol{X}\right)^{-1}\boldsymbol{X}^\top(\boldsymbol{\mu}-\boldsymbol{y})=\left(\boldsymbol{X}^\top\mathbf{S}\boldsymbol{X}\right)^{-1}\boldsymbol{X}^\top\mathbf{S}\boldsymbol{z}, \quad (6.64)$$

где $\boldsymbol{z}=\boldsymbol{X}\boldsymbol{w}^{\text{old}}-\mathbf{S}^{-1}(\boldsymbol{\mu}-\boldsymbol{y})$. Таким образом, уравнение (6.64) можно использовать для того, чтобы найти гипотезу максимального правдоподобия в любой

обобщённой линейной модели. В байесовский вывод мы здесь углубляться не будем; его тоже можно провести, и он тоже, конечно, как и в разделе 5.2, получится только приближённым, но здесь уже разумнее будет использовать общие методы приближённого вывода — методы Монте-Карло для марковских цепей (MCMC) или вариационные приближения, — до которых мы в этой книге не дойдём.

6.7.3. Пуассоновская регрессия

А теперь, вооружившись алгоритмом (6.64), можно и что-нибудь новенькое получить. Первый новый пример — *пуассоновская регрессия* (Poisson regression). Предположим, что целевая переменная y в нашей модели имеет смысл числа неких происходящих независимо друг от друга событий: звонки в кол-центр, «лайки» под новым постом в социальной сети, число мутаций на данном участке ДНК и так далее. Иначе говоря, целевую переменную было бы неплохо описать распределением Пуассона, которое мы рассматривали в разделе 3.1. Поскольку среднее здесь совпадает с интенсивностью λ, давайте сразу переобозначим её через μ:

$$p(y|\mu) = \frac{1}{y!}\mu^y e^{-\mu}, \quad \text{то есть} \quad \log p(y|\mu) = y\log\mu - \mu - \log(y!).$$

Мы видим, что естественным параметром здесь является $\theta = \log\mu$, и, приняв, как обычно, $\theta = \boldsymbol{w}^\top\boldsymbol{x}$, мы получим обобщённую линейную модель

$$\log p(y|\boldsymbol{x},\boldsymbol{w}) = y\boldsymbol{w}^\top\boldsymbol{x} - e^{\boldsymbol{w}^\top\boldsymbol{x}} - \log(y!),$$

то есть в данном случае $\sigma^2 = 1$, $a(\theta) = e^\theta = e^{\boldsymbol{w}^\top\boldsymbol{x}}$, $h(y,\sigma^2) = \frac{1}{y!}$. И теперь мы уже умеем обучать эту модель или градиентным спуском с использованием формулы (6.63), или методом второго порядка по формуле (6.64).

Пуассоновскую регрессию разработал и впервые начал активно применять Владислав Борткевич[1]. В своей книге «Закон малых чисел» [73] он показывает, что события с низкой частотой в большой популяции следуют пуассоновскому распределению, даже если их вероятности различаются, а в качестве примера приводится ныне знаменитый датасет о прусских кавалеристах (Prussian horse-kicking data): сколько прусских солдат в каждом из 14 кавалерийских полков погибли от того, что их лягнула лошадь, за 20 лет наблюдений. Честно говоря,

[1] *Владислав Иосифович Борткевич* (Wladyslaw Bortkiewicz, 1868–1931) — экономист и статистик. Отнести его к какой-то одной стране затруднительно: по национальности Борткевич был поляком, родился и вырос в Санкт-Петербурге, где его отец преподавал артиллерию и математику, там же начал научную работу, был автором трудов о денежной реформе 1895–1897 годов, а с 1901 года и до конца жизни Борткевич жил в Германии и работал в Берлинском университете. В статистике Борткевич популяризировал распределение Пуассона, которому посвятил книгу «Закон малых чисел» [73]. А в политэкономии он обнаружил одну из главных проблем марксистской теории: проблему трансформации, то есть внутреннее противоречие в трудовой теории стоимости Карла Маркса (у него получалось, что одни и те же товары на одном и том же рынке покупаются то по стоимостям, то по ценам производства), которая долго считалась «проклятием» марксистской экономики.

Таблица 6.2. Данные о подмастерьях, мигрирующих в Эдинбург

Регион	Расстояние до Эдинбурга	Число подмастерий	Население (тыс.)	Степень урбанизации	Направление от Эдинбурга	Регион	Расстояние до Эдинбурга	Число подмастерий	Население (тыс.)	Степень урбанизации	Направление от Эдинбурга
Midlothian	21	225	56	18,8	S	Ayr	110	2	84	26,4	W
West Lothian	24	22	18	37,9	W	Kircudbright	110	0	29	11,3	S
East Lothian	33	44	30	43,4	S	Kincardine	125	1	26	12,3	N
Kinross	33	3	7	30,3	N	Bute	132	0	12	43,6	W
Fife	36	41	94	41,3	N	Aberdeen	156	3	123	23,1	N
Peebles	41	9	9	29,3	S	Wigtown	157	0	22	23,2	S
Clackmannan	41	2	11	47,4	N	Banff	159	4	36	12,9	N
Selkirk	52	5	5	41,9	S	Moray	174	2	27	27,6	N
Lanark	54	23	147	68,1	W	Nairn	175	0	8	45,3	N
Berwick	56	11	31	15,2	S	Argyll	179	1	72	12,7	W
Roxburgh	67	9	34	31,8	S	Inverness	234	7	74	10,8	N
Stirling	71	13	51	31,1	W	Ross	274	4	55	10,7	N
Perth	78	26	126	14,4	N	Caithness	274	1	23	28,3	N
Dunbarton	79	0	21	27,3	W	Sutherland	283	0	23	10,3	N
Angus	85	5	99	55,3	N	Orkney	366	0	29	9,0	N
Dumfries	86	3	55	25,9	S	Shetland	491	1	22	7,7	N
Renfrew	92	1	78	69,9	W						

классические примеры применения и пуассоновской регрессии, и вообще распределения Пуассона не слишком аппетитны: в книге Борткевича ещё приводятся данные о самоубийствах среди детей, а стандартные примеры включают датасет о бомбардировке Лондона немецкой авиацией, обработанный британским статистиком Р. Д. Кларком [100], аварии на дорогах [292] и даже случаи смерти от рака среди переживших бомбардировки Хиросимы и Нагасаки [131].

Давайте выберем что-нибудь менее депрессивное: рассмотрим классический датасет из области исторической географии, который был обработан пуассоновской регрессией ещё в 1980-х годах [353]. Он настолько маленький, что целиком умещается в табл. 6.2. Датасет иллюстрирует постепенную урбанизацию и центростремительное движение людей в регионе; он содержит данные о миграции подмастерий из разных графств Шотландии в её столицу и (в то время) крупнейший город Эдинбург в последней четверти XVIII века. В датасете указаны данные о миграции из 33 графств Шотландии (вторая колонка), а также приводятся географические данные о провинциях: расстояние до Эдинбурга, население графства в тысячах человек, уровень урбанизации (процент людей, живущих в городах) и направление от Эдинбурга, которое может принимать три значения — на север, на запад и на юг[1].

[1] К востоку от Эдинбурга расположено Северное море, из которого в XVIII веке в Британию уже прибывало не так много серьёзно настроенных гостей, как примерно тысячелетием ранее.

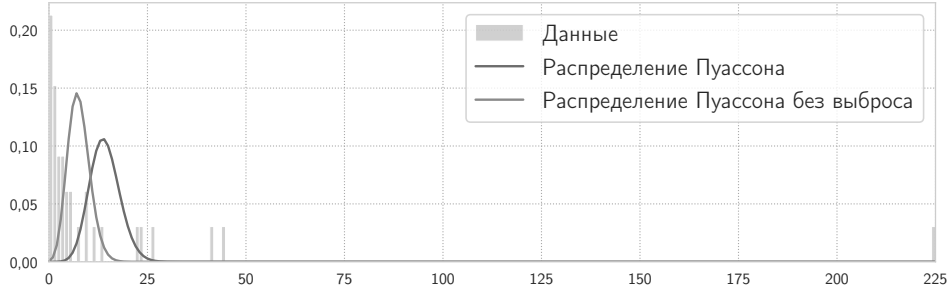


Рис. 6.28. Распределение прибывших подмастерий и обученные на нём распределения Пуассона

Для обработки этих данных нам понадобится модуль `statsmodels.api`, в котором есть реализация обучения обобщённых линейных моделей. Давайте создадим датасет, в котором закодируем категориальную переменную (направление) через one-hot кодирование:

```
from sklearn.preprocessing import OneHotEncoder
import statsmodels.api as sm

ss = [ s.strip().split() for s in open("apprentice.dat") ]
names = np.array([" ".join(x[:-5]) for x in ss])
d = np.array([[float(y) for y in x[-5:]] for x in ss])
X = np.hstack([ d[:, [0, 2, 3]], OneHotEncoder().
    fit_transform(d[:, 4].reshape(-1, 1)).toarray() ])
y = np.array(d[:, 1])
```

В итоге получились матрица признаков `X` и вектор значений целевой переменной `y`; вот несколько первых строк матрицы `X`:

```
[[ 21.   56.   18.8   0.    0.    1. ]
 [ 24.   18.   37.9   0.    1.    0. ]
 [ 33.   30.   43.4   0.    0.    1. ]
 ...
```

Число прибывших — это суммарное число неких единичных случайных событий, так что вполне логично, что это величина, чьё распределение похоже на пуассоновское. Однако распределение прибывших само по себе не очень хорошо описывается пуассоновским распределением: на рис. 6.28 показано распределение целевой переменной `y` и обученные на нём пуассоновские распределения. Кстати, «обучение» пуассоновского распределения заключается в том, чтобы найти среднее значение данных — оно и будет оценкой единственного параметра λ. Можно попробовать выкинуть из датасета «явный выброс» в графстве

Мидлотиан (ближайшем к Эдинбургу), но это, как видно на рис. 6.28, тоже не помогает.

Это вполне естественно; мы ожидаем, что не весь датасет будет описываться одним пуассоновским распределением, а что его интенсивность λ будет разной в зависимости от признаков: логично, что число прибывших подмастерий будет тем больше, чем больше общее население графства, и тем меньше, чем дальше графство находится от Эдинбурга и чем больше в нём степень урбанизации (крестьяне скорее поедут подмастерьями в ближайший город в своём графстве, если он есть и достаточного размера, а из города будут ехать уже не подмастерья).

Чтобы обучить обобщённую линейную модель, достаточно вызвать

```
model = sm.GLM(y, sm.add_constant(data, prepend=True), \
    family=sm.families.Poisson()).fit()
```

Здесь `y` — целевая переменная, `data` — признаки, представленные как матрица (например, `X`), а функция `sm.add_constant` добавляет к ним ещё один столбец из единиц, чтобы класс `sm.GLM` обучил модель со свободным членом. После обучения можно вызвать

```
print(model.summary())
```

и увидеть набор важных характеристик обученной модели.

Например, для пуассоновской регрессии по логарифму расстояния до Эдинбурга (логарифму первого столбца матрицы `X`) мы получим такую табличку:

```
                 Generalized Linear Model Regression Results
==============================================================================
Dep. Variable:                      y   No. Observations:                   33
Model:                            GLM   Df Residuals:                       31
Model Family:                 Poisson   Df Model:                            1
Link Function:                    Log   Scale:                          1.0000
Method:                          IRLS   Log-Likelihood:                -244.15
Date:                Fri, 07 Oct 2022   Deviance:                       393.96
Time:                        07:53:21   Pearson chi2:                     482.
No. Iterations:                     6   Pseudo R-squ. (CS):              1.000
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
const         11.7936      0.337     35.014      0.000      11.133      12.454
x1            -2.3072      0.095    -24.337      0.000      -2.493      -2.121
==============================================================================
```

В этой строке переменная всего одна, поэтому обозначение `x1` не вызывает затруднений, но дальше мы будем сообщать модели более «говорящие» имена переменных. В результатах выше для нас главное в том, что влияние расстояния получилось отрицательным, как мы и ожидали, но отклонение даже на тренировочном наборе остаётся весьма большим. Прологарифмировать расстояние

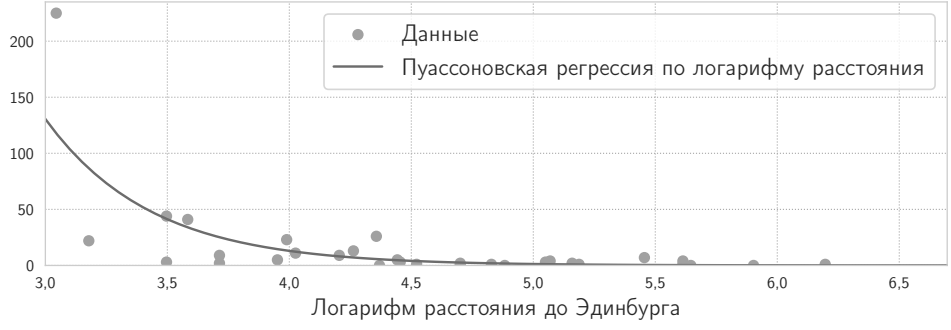


Рис. 6.29. Пуассоновская регрессия, обученная на признаке (логарифма) расстояния до Эдинбурга

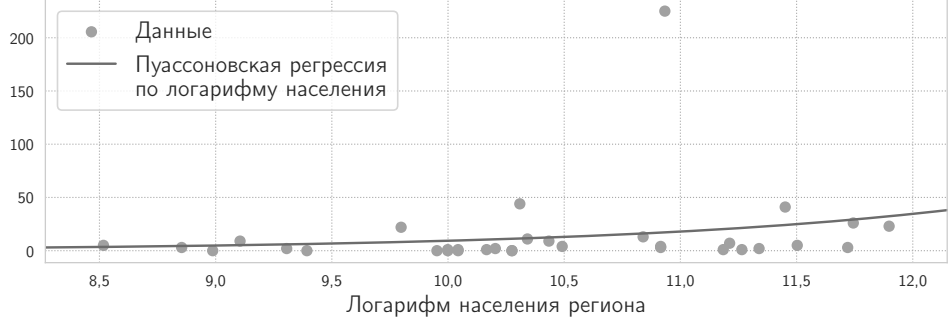


Рис. 6.30. Пуассоновская регрессия, обученная на признаке (логарифма) населения графства

здесь логично из общих соображений, и авторы [353] мотивируют логарифмирование расстояния и населения так называемой гравитационной моделью миграции. В этой модели считается, что степень миграционной активности подчиняется чему-то вроде ньютоновского закона притяжения: она пропорциональна характеристикам значимости географических пунктов, например размерам их населения, и обратно пропорциональна квадрату расстояния. Иначе говоря, размеры населения и расстояние совмещаются друг с другом мультипликативно, а значит, в разумной линейной модели будут участвовать их логарифмы.

На рис. 6.29 показана регрессия по логарифму расстояния, а на рис. 6.30 — ещё одна пуассоновская регрессия, обученная по логарифму населения графств; там, что тоже вполне логично, ситуация обратная, и коэффициент при численности населения получается положительным. Однако и эта одномерная регрессия

объясняет не так уж много, так что давайте попробуем обучиться на всей матрице X. Мы снова прологарифмируем расстояние и население, а степень урбанизации u — это процент, так что в качестве признака для линейной модели логично использовать $\sigma^{-1}\left(\frac{u}{100}\right) = \log\left(\frac{u}{100-u}\right)$. В итоге получается модель такого вида:

```
                 Generalized Linear Model Regression Results
==============================================================================
Dep. Variable:                      y   No. Observations:                   33
Model:                            GLM   Df Residuals:                       27
Model Family:                 Poisson   Df Model:                            5
Link Function:                    Log   Scale:                          1.0000
Method:                          IRLS   Log-Likelihood:                -86.710
Date:                Fri, 07 Oct 2022   Deviance:                       79.073
Time:                        08:51:47   Pearson chi2:                     90.1
No. Iterations:                     6   Pseudo R-squ. (CS):             1.000
Covariance Type:            nonrobust
==============================================================================
                         coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
const                  0.1780      0.704      0.253      0.800      -1.201       1.557
Log distance          -2.0739      0.094    -22.080      0.000      -2.258      -1.890
Log population         0.9682      0.078     12.405      0.000       0.815       1.121
Logit urbanization    -0.3652      0.085     -4.299      0.000      -0.532      -0.199
North                 -0.0685      0.279     -0.246      0.806      -0.614       0.477
West                  -0.1721      0.260     -0.662      0.508      -0.681       0.337
South                  0.4186      0.220      1.906      0.057      -0.012       0.849
==============================================================================
```

Нарисовать модель с шестью признаками я уже не смогу, но видно, что она вполне отвечает нашей интуиции: по расстоянию (x1) и коэффициенту урбанизации (x3) коэффициенты отрицательные, а по населению (x2) положительный. Любопытно, что направление от Эдинбурга оказывает серьёзное влияние только в одном случае: из южных шотландских графств люди едут охотнее.

Модель, однако, всё ещё не слишком хорошо описывает данные, так что авторы работы [353] идут дальше и рассматривают признаки второго порядка, точнее, взаимодействия между численными признаками и категориальным направлением. Хотя строить на таком небольшом датасете квадратичную модель попахивает оверфиттингом, добавить взаимодействие между численными признаками и направлением выглядит разумно. Формально это значит, что мы добавим в качестве признаков произведения логарифма населения и категориального признака, а также произведения логарифма расстояния и категориального признака; а в реальности это приведёт к тому, что мы сможем обучить в модели отдельные коэффициенты при численном признаке для каждого значения категориального. Например, если добавить взаимодействие направления с логарифмом расстояния (что выглядит самым логичным), получится что-то вроде

```
                 Generalized Linear Model Regression Results
==============================================================================
Dep. Variable:                      y   No. Observations:                   33
Model:                            GLM   Df Residuals:                       25
Model Family:                 Poisson   Df Model:                           7
Link Function:                    Log   Scale:                          1.0000
Method:                          IRLS   Log-Likelihood:                -73.068
Date:                Fri, 07 Oct 2022   Deviance:                       51.789
Time:                        08:52:09   Pearson chi2:                     46.2
No. Iterations:                     6   Pseudo R-squ. (CS):              1.000
Covariance Type:            nonrobust
==============================================================================
                         coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
const                  0.6305      0.757      0.833      0.405      -0.853       2.114
Log distance          -1.5207      0.079    -19.364      0.000      -1.675      -1.367
Log population         0.8969      0.090      9.934      0.000       0.720       1.074
Logit urbanization    -0.1514      0.093     -1.629      0.103      -0.334       0.031
North                 -2.7014      0.665     -4.061      0.000      -4.005      -1.398
West                   0.7996      0.694      1.151      0.250      -0.561       2.161
South                  2.5322      0.704      3.595      0.000       1.152       3.913
Log dist North         0.2157      0.147      1.472      0.141      -0.071       0.503
Log dist West         -0.7158      0.201     -3.566      0.000      -1.109      -0.322
Log dist South        -1.0206      0.158     -6.472      0.000      -1.330      -0.712
==============================================================================
```

Разумеется, качество модели на тренировочном наборе, выраженное в этих таблицах показателями deviance и χ^2, растёт с добавлением новых признаков. Но не добавляем ли мы оверфиттинг? Давайте попробуем поэкспериментировать с тренировочным и тестовым множеством:

```
from sklearn.model_selection import train_test_split

def one_experiment(X, y, test_size=0.1):
    X_trn, X_test, y_trn, y_test = train_test_split(X, y, test_size=test_size)
    model_poisson = sm.GLM(y_trn, X_trn, family=sm.families.Poisson()).fit()
    predictions = model_poisson.predict(X_test)
    return np.sqrt(np.mean((predictions - y_test) ** 2))
```

Если запустить эту процедуру много раз и усреднить, для модели только на логарифме расстояния среднеквадратичное отклонение получится около 25,2, для модели на логарифмах расстояния и населения — около 17,5, для модели со всеми шестью признаками — около 8, а вот для приведённой выше модели со взаимодействиями между расстоянием и направлением — уже более 47. Таким

образом, хотя добавление таких взаимодействий выглядит естественным и часто применяется на практике, например в исходной работе [353], в данном случае это, по всей видимости, ведёт к оверфиттингу. Поэтому анализ пуассоновской регрессии я в данном случае закончу на том, что порекомендую использовать модель с шестью признаками: тремя численными и one-hot представлением категориального направления.

6.7.4. Отрицательная биномиальная регрессия

Однако у пуассоновской регрессии есть один важный недостаток: распределение Пуассона часто оказывается недостаточно выразительным. Как мы уже обсуждали в разделе 3.1, у распределения Пуассона дисперсия совпадает со средним, а в реальных данных это зачастую не так, и хотелось бы иметь похожее на пуассоновское распределение с переменной дисперсией. К счастью, там же, в разделе 3.1, мы обсудили и ответ на этот запрос — отрицательное биномиальное распределение

$$\text{NegBinom}\,(k|r,p) = \binom{k+r-1}{r-1}(1-p)^k\,p^r.$$

Отрицательная биномиальная регрессия (negative binomial regression) — это обобщённая линейная модель с отрицательным биномиальным распределением в качестве функции связи. Мы знаем из раздела 3.1, что среднее отрицательного биномиального распределения составляет

$$\mu = \frac{r(1-p)}{p},$$

и с этим параметром мы можем переписать распределение как

$$p\,(y|\mu) = \frac{1}{y!}\mu^y e^{-\mu}, \quad \text{то есть} \quad \log p\,(y|\mu) = y\log\mu - \mu - \log\,(y!)\,.$$

Мы видим, что естественным параметром здесь является $\theta = \log\mu$, и, снова приняв $\theta = \boldsymbol{w}^\top\boldsymbol{x}$, мы получим обобщённую линейную модель

$$\log p\,(y|\boldsymbol{x},\boldsymbol{w}) = y\boldsymbol{w}^\top\boldsymbol{x} - e^{\boldsymbol{w}^\top\boldsymbol{x}} - \log\,(y!)\,,$$

то есть в данном случае $\sigma^2 = 1$, $a(\theta) = e^\theta = e^{\boldsymbol{w}^\top\boldsymbol{x}}$, $h(y,\sigma^2) = \frac{1}{y!}$. И теперь мы уже умеем обучать эту модель или градиентным спуском с использованием формулы (6.63), или методом второго порядка по формуле (6.64).

Отрицательная биномиальная регрессия применяется на практике даже чаще, чем пуассоновская: у неё больше выразительная сила, а лишний параметр редко ведёт к оверфиттингу. К датасету о миграции подмастерий её применить

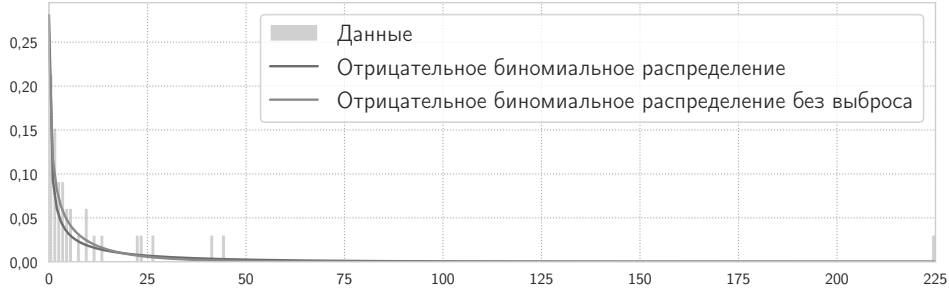


Рис. 6.31. Распределение прибывших подмастерий и обученные на нём отрицательные биномиальные распределения

Рис. 6.32. Отрицательная биномиальная регрессия, обученная на признаке (логарифма) расстояния до Эдинбурга

ничем не сложнее, чем пуассоновскую. На рис. 6.31 показаны отрицательные биномиальные распределения, обученные без признаков, просто на данных о миграциях; как видите, они куда больше похожи на имеющиеся данные, чем распределения Пуассона (см. рис. 6.28).

Можно обучить и отрицательную биномиальную регрессию той же процедурой, только с другим семейством распределений:

```
model = sm.GLM(y, sm.add_constant(data, prepend=True), \
    family=sm.families.NegativeBinomial()).fit()
```

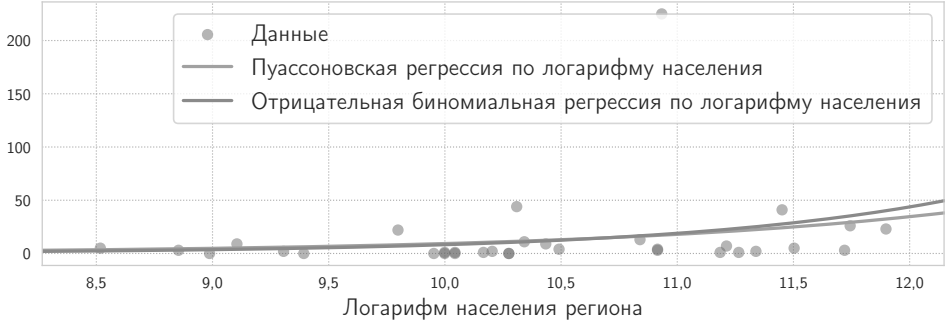


Рис. 6.33. Пуассоновская регрессия, обученная на признаке (логарифма) населения графства

По рис. 6.32 и 6.33, правда, уже труднее заметить разницу между отрицательной биномиальной и пуассоновской регрессиями; точнее говоря, труднее оценить, в нужную ли сторону эта разница. А эксперименты с тренировочным и тестовым множеством, аналогичные показанным выше, дают результаты хуже: отрицательная биномиальная регрессия на логарифме расстояния даёт среднеквадратичное отклонение около 40, на логарифмах расстояния и населения — около 15,5, для модели со всеми шестью признаками — около 11,5, а для модели со взаимодействиями между расстоянием и направлением опять начинается серьёзный оверфиттинг — ошибка превышает 150.

Итак, в этом разделе мы рассмотрели краткое введение в обобщённые линейные модели; конечно, это только начало, и о таких моделях написаны целые книги [245, 369, 385, 646]. Они дают общий взгляд на многое из того, о чём мы говорили в этой книге. А книга на этом подошла к концу — впереди осталось только краткое заключение.

Заключение

> Владеть велосипедом — это значит уметь вначале разглядеть его, потом коснуться его. Но касания явно недостаточно; нужно уметь забраться на него, чтобы совершить прогулку. Однако эта бесцельная прогулка сама по себе также недостаточна; нужно использовать велосипед для того, чтобы совершать поездки. А это отсылает нас к использованиям более длительным, более совершенным, к долгим путешествиям по Франции... В конечном счете... достаточно отдать деньги, чтобы велосипед принадлежал мне, но нужна вся моя жизнь, чтобы реализовать это владение; именно это я хорошо чувствую, приобретая предмет; владение является предприятием, которое смерть оставляет всегда незавершённым.
>
> *Жан-Поль Сартр. Бытие и ничто*

Эта книга заканчивается, но я надеюсь, что это не последняя моя встреча с читателями. Вы наверняка не раз замечали в тексте фразы вроде «до такой-то темы мы в этой книге не доберёмся» — они как раз и выражают мою надежду на то, что появятся и следующие тома этого учебника по машинному обучению. Во всяком случае, из упоминавшегося во введении курса по машинному обучению, который я сейчас каждый год читаю в Санкт-Петербургском государственном университете, эта книга покрывает только первый семестр (пусть и в сильно расширенной версии — конечно, не всё из того, что описано в этой книге, я успеваю рассказать за 14–15 лекций), а всего в курсе четыре семестра лекций.

В заключение хочется вернуться к одной из тем введения и немного поговорить о том, что происходит в искусственном интеллекте прямо сейчас. Во введении мы обсуждали «горький урок» Ричарда Саттона: в машинном обучении нередко происходит так, что очередной скачок вычислительных возможностей «поглощает» многие изящные идеи и сложные модели, и оказывается, что модели более простые (но, возможно, более сложные с инженерной точки зрения) работают лучше. Кажется, что мир, в котором основной новостью науки неизбежно становится новая версия большой языковой модели от *OpenAI*, подтверждает этот вывод.

Однако мне кажется, что это никоим образом не делает, например, эту книгу нерелевантной. Новые модели и новые идеи будут продолжать появляться, некоторые старые идеи будут отмирать со временем и поглощаться новыми, но основы вероятностного понимания машинного обучения никуда не денутся. И большая языковая модель GPT-4, и на данный момент ещё не существующая GPT-5 всё равно занимаются в основном именно предсказанием распределения вероятностей следующего токена, и, насколько бы сложную архитектуру они ни имели, на последнем слое у них всё равно, скорее всего, находится функция softmax, которую мы обсуждали в главе 5. В книге мы развиваем вероятностную интуицию

машинного обучения на примерах, в которых мы можем сделать чёткий вывод, часто можем даже что-то аналитически подсчитать, но интуиция переносится и на более сложные модели.

Конечно, развитие искусственного интеллекта может привести к тому, что станут нерелевантными все книги по машинному обучению, а то и все книги вообще: возможно, исследования в области искусственного интеллекта модели скоро будут вести сами, а возможно, они заменят и вообще всю продуктивную человеческую деятельность. Но тут уж ничего не поделаешь, приходится принимать этот риск, тем более что всё-таки в ближайшие годы я настолько радикальных изменений не ожидаю.

И последнее, что я хотел сказать. Основная тема этой книги — вероятностный вывод — важна не только для машинного обучения, но и просто для повседневной жизни. В главе 2 мы обсуждали, что люди плохо умеют рассуждать о вероятностях. Но это всё-таки приходится делать! Вероятностную природу имеет не только машинное обучение, но и буквально любой процесс, происходящий в окружающем нас мире: редко у нас имеется настолько полная информация, чтобы места случайностям не оставалось вовсе.

Поэтому даже идеальный логик (разумеется, несуществующий, как всё идеальное), умеющий делать выводы любой сложности и полностью использующий всю имеющуюся информацию до самого последнего бита[1], вынужден будет делать вероятностные выводы и оперировать именно тем математическим аппаратом, который мы развивали в этой книге. Жизненные задачи почти без исключений заключаются не в том, чтобы сделать строгий формальный вывод, построив логический силлогизм или математическое доказательство, а в том, чтобы оценить вероятности разных сценариев развития событий и попытаться найти такое воздействие на окружающий мир, чтобы вероятности желаемых сценариев стали как можно выше.

Конечно, мы с вами никогда не станем идеальными машинами по вычислению вероятностей, это бесконечный путь саморазвития, но мне кажется очень важным знать о его существовании и сделать по нему хотя бы несколько первых шагов.

Надеюсь, что эта книга смогла хоть немного поспособствовать развитию вашей вероятностной интуиции, а также рассказать о том, как основные модели и методы машинного обучения выстраиваются в единое здание. В основании этого здания лежит теорема Байеса и основные задачи байесовского вывода, которые мы решали в разных ситуациях на протяжении всей книги.

В следующих книгах, если они будут, мы поговорим о более сложных вероятностных моделях (в частности, о языке графических моделей и приближённом вероятностном выводе) и специальных постановках задач (например, о задачах

[1] Рекомендую, кстати, попытаться пофантазировать о том, на что мог бы быть способен такой идеальный логик — это полезное упражнение, и поверьте, пределы возможного в использовании получаемой нами информации отстоят от наших человеческих способностей *очень* далеко...

ранжирования или коллаборативной фильтрации), а затем и об искусственных нейронных сетях, которые в последние годы стали основным инструментом машинного обучения. Но это будет уже при следующей встрече, а сейчас — до свидания!

Сергей Николенко,
май 2024 года

Литература

1. 3D Molecular Representations Based on the Wave Transform for Convolutional Neural Networks / D. Kuzminykh et al. // Molecular Pharmaceutics, 2018, vol. 15, no. 10. — P. 4378–4385.
2. A Mathematical Approach to Medical Diagnosis: Application to Congenital Heart Disease / H. R. Warner et al. // JAMA, 07 1961, vol. 177, no. 3. — P. 177–183.
3. *Aad G. et al.* Observation of a New Particle in the Search for the Standard Model Higgs Boson with the ATLAS Detector at the LHC // Phys. Lett. B, 2012, vol. 716. — P. 1–29.
4. *Aad G. et al.* A Detailed Map of Higgs Boson Interactions by the ATLAS Experiment Ten Years after the Discovery // Nature, 2022, vol. 607, no. 7917. — P. 52–59, [Erratum: Nature 612, E24 (2022)].
5. *Aaronson S.* Is P Versus NP Formally Independent? // Bull. EATCS, 2003, vol. 81. — P. 109–136.
6. *Aaronson S.* P =? NP // Open Problems in Mathematics / Springer, 2016. — P. 1–122.
7. *Aaronson S.* P=?NP // Electron. Colloquium Comput. Complex., 2017, vol. TR17-004. https://eccc.weizmann.ac.il/report/2017/004.
8. Accurate and Transferable Multitask Prediction of Chemical Properties with an Atoms-in-molecules Neural Network / R. Zubatyuk et al. // Science Advances, 2019, vol. 5, no. 8, eaav6490.
9. *Aczél J.* A Short Course on Functional Equations, Springer, 1987.
10. Adaptive Weighted Imbalance Learning with Application to Abnormal Activity Recognition / X. Gao et al. // Neurocomputing, 2016, vol. 173. — P. 1927–1935.
11. ADASYN: Adaptive Synthetic Sampling Approach for Imbalanced Learning / H. He et al. // 2008 IEEE International Joint Conference on Neural Networks (IEEE World Congress on Computational Intelligence), 2008. — P. 1322–1328.
12. *Adcock C.* Asset Pricing and Portfolio Selection Based on the Multivariate Extended Skew-Student-t Distribution // Ann. Oper. Res., 2002, vol. 176. — P. 221–234.
13. Advancing Mathematics by Guiding Human Intuition with AI / A. Davies et al. // Nature, 2021, vol. 600. — P. 70 – 74.
14. *Agarwal D., Chen B.-C., Elango P.* Collaborative Filtering for Implicit Feedback Datasets // Proc. 18th International Conference on World Wide Web, New York, NY, USA: ACM, 2009. — P. 21–30.
15. *Agterberg F. P.* Geomathematics, Mathematical Background and Geo-Science Applications, Elsevier Scientific Publishing Company, Amsterdam, 1974.
16. *Aitchison J., Brown J. A. C.* The Lognormal Distribution, Cambridge Univ. Press, 1957.
17. *Akaike H.* Information Theory and an Extension of the Maximum Likelihood Principle // Proc. 2nd International Symposium on Information Theory, Tsahkadsor, Armenia, USSR, September 2–8, 1971, Budapest: Akadémiai Kiado, 1973. — P. 267–281.
18. *Akaike H.* A New Look at the Statistical Model Identification // IEEE Transactions on Automatic Control, 1974, vol. 19, no. 6. — P. 716–723.
19. *Akaike H.* Prediction and entropy // A Celebration of Statistics, Springer, 1985. — P. 1–24.
20. *Alaba S., Ball J.* A Survey on Deep-Learning-Based LiDAR 3D Object Detection for Autonomous Driving // Sensors, 12 2022, vol. 22. — P. 9577.
21. *Aldrich J.* R.A. Fisher and the Making of Maximum Likelihood 1912–1922 // Statistical Science, 1997, vol. 12, no. 3. — P. 162–176.
22. *Alexander S.* The Control Group is out of Control, 2014. https://slatestarcodex.com/2014/04/28/the-control-group-is-out-of-control.

23. *Anaya-Isaza A., Mera-Jiménez L., Zequera-Diaz M.* An Overview of Deep Learning in Medical Imaging // Informatics in Medicine Unlocked, 2021, vol. 26. — Article 100723.
24. *Anderson E.* The Irises of the Gaspé Peninsula // Bulletin of the American Iris Society, 1935, vol. 59. — P. 2–5.
25. *Appel K., Haken W.* Every Planar Map is Four Colorable. I: Discharging // Illinois J. Math., 1977, vol. 21. — P. 429–490.
26. *Arbuthnot J.* II. An Argument for Divine Providence, taken from the Constant Regularity Observ'd in the Births of both Sexes. By Dr. John Arbuthnott, Physitian in Ordinary to Her Majesty, and Fellow of the College of Physitians and the Royal Society // Philosophical Transactions of the Royal Society of London, 1710, vol. 27, no. 328. — P. 186–190.
27. *Archana R., Jeevaraj P. S. E.* Deep Learning Models for Digital Image Processing: a Review // Artif. Intell. Rev., Jan. 2024, vol. 57, no. 1. — 33 P.
28. Are Replication Rates the Same across Academic Fields? Community Forecasts from the DARPA SCORE Programme / M. Gordon et al. // Royal Society Open Science, 2020, vol. 7, no. 7. — Article 200566.
29. *Arjovsky M., Chintala S., Bottou L.* Wasserstein GAN, 2017.
30. *Arya S., Mount D. M.* Approximate Nearest Neighbor Queries in Fixed Dimensions // Proc. Fourth Annual ACM-SIAM Symposium on Discrete Algorithms, USA: Society for Industrial and Applied Mathematics, 1993. — P. 271–280.
31. *Asch S. E.* Studies of Independence and Conformity: I. A Minority of One against a Unanimous Majority // Psychological Monographs: General and Applied, 1956, vol. 70, no. 9. — P. 1–70.
32. *Ashour S. K., Abdel-hameed M. A.* Approximate Skew Normal Distribution // Journal of Advanced Research, 2010, vol. 1, no. 4. — P. 341–350.
33. *Athey S., Imbens G. W.* The State of Applied Econometrics: Causality and Policy Evaluation // Journal of Economic Perspectives, May 2017, vol. 31, no. 2. — P. 3–32.
34. Attention Is All You Need / A. Vaswani et al. // arXiv preprint, 2017. http://arxiv.org/abs/1706.03762.
35. *Auerbach F.* Das Gesetz der Bevölkerungskonzentration // Petermann's Geographische Mitteilungen, 1913, vol. 59. — P. 74–76.
36. *Authors not Disclosed.* The QED Manifesto // Lecture Notes in Artificial Intelligence, 1994, vol. 814. — P. 238–251.
37. Automated Extraction of Chemical Synthesis Actions from Experimental Procedures / A. C. Vaucher et al. // Nature communications, 2020, vol. 11, no. 1. — P. 1–11.
38. *Axtell R. L.* Zipf Distribution of U.S. Firm Sizes // Science, 2001, vol. 293, no. 5536. — P. 1818–1820.
39. *Azeraf E., Monfrini E., Pieczynski W.* Using the Naive Bayes as a Discriminative Model // 2021 13th International Conference on Machine Learning and Computing, New York, NY, USA: ACM, 2021. — P. 106–110.
40. *Azzalini A.* A Class of Distributions Which Includes the Normal Ones // Scandinavian Journal of Statistics, 1985. — P. 171–178.
41. *Azzalini A., Capitanio A.* Statistical Applications of the Multivariate Skew Normal Distribution // Journal of the Royal Statistical Society Series B: Statistical Methodology, Sep. 1999, vol. 61, no. 3. — P. 579–602.
42. *Bak A., Vavilov N.* Structure Of Hyperbolic Unitary Groups - I. Elementary Subgroups // Algebra Colloquium, 05 1999, vol. 7.
43. *Banks W. P., Isham E. A.* We Infer Rather Than Perceive the Moment We Decided to Act // Psychological Science, 2009, vol. 20, no. 1. — P. 17–21.
44. *Bargh J. A., Chen M., Burrows L.* Automaticity of Social Behavior: Direct Effects of Trait Construct and Stereotype-Activation on Action // Journal of Personality and Social Psychology, 1996, vol. 71, no. 2. — P. 230–244.

45. *Barnes T. J.* A History of Regression: Actors, Networks, Machines, and Numbers // Environment and Planning A: Economy and Space, 1998, vol. 30, no. 2. — P. 203–223.
46. *Bass S. D., De Roeck A., Kado M.* The Higgs Boson Implications and Prospects for Future Discoveries // Nature Reviews Physics, Jul. 2021, vol. 3, no. 9. — P. 608–624.
47. *Bayes T.* Divine Benevolence: Or, An Attempt to Prove that the Principal End of the Divine Providence and Government is the Happiness of His Creatures, John Noon, London, 1731.
48. *Bayes T.* An Introduction to the Doctrine of Fluxions, John Noon, London, 1736.
49. *Bayes T.* An Essay Towards Solving a Problem in the Doctrine of Chances // Phil. Trans. of the Royal Soc. of London, 1763, vol. 53. — P. 370–418.
50. Bayesian Data Analysis / A. Gelman et al., CRC Press, 2003.
51. *Beck N., King G., Zeng L.* Improving Quantitative Studies of International Conflict: A Conjecture // American Political Science Review, 03 2000, vol. 94. — P. 21–35.
52. Behavioral Analysis of Insider Threat: A Survey and Bootstrapped Prediction in Imbalanced Data / A. Azaria et al. // IEEE Transactions on Computational Social Systems, 2014, vol. 1, no. 2. — P. 135–155.
53. Behavioral Priming: It's All in the Mind, but Whose Mind? / S. Doyen et al. // PloS one, 01 2012, vol. 7. — P. e29081.
54. *Bell S. J., Kampman O. P.* Perspectives on Machine Learning from Psychology's Reproducibility Crisis // ArXiv preprint abs/2104.08878, 2021. https://arxiv.org/abs/2104.08878.
55. *Bellhouse D. R.* The Reverend Thomas Bayes, FRS: A Biography to Celebrate the Tercentenary of His Birth // Statistical Science, 2004, vol. 19, no. 1. — P. 3–43.
56. *Bem D. J.* Feeling the Future: Experimental Evidence for Anomalous Retroactive Influences on Cognition and Affect. // Journal of Personality and Social Psychology, 2011, vol. 100, no. 3. — P. 407.
57. *Bennet P. N.* Assessing the Calibration of Naive Bayes' Posterior Estimates, School of Computer Science, Carnegie Mellon University, 2000.
58. *Benoit C.* Note sur une méthode de résolution des équations normales provenant de l'application de la méthode des moindres carrés a un systeme d'équations linéaires en nombre inférieur a celui des inconnues (Procédé du Commandant Cholesky) // Bulletin géodésique, 1924, vol. 2, no. 1. — P. 67–77.
59. *Bentley J. L.* Multidimensional Binary Search Trees Used for Associative Searching // Commun. ACM, sep 1975, vol. 18, no. 9. — P. 509–517.
60. *Berkson J.* Application of the Logistic Function to Bio-Assay // Journal of the American Statistical Association, 1944, vol. 39, no. 227. — P. 357–365.
61. *Berkson J.* Why I Prefer Logits to Probits // Biometrics, 1951, vol. 7, no. 4. — P. 327–339.
62. *Bernoulli D.* Exposition of a New Theory on the Measurement of Risk // Econometrica, 1954, vol. 22, no. 1. — P. 23–36.
63. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding / J. Devlin et al. // Proc. 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers), Minneapolis, Minnesota: Association for Computational Linguistics, 2019. — P. 4171–4186.
64. *Bierlaire M.* Optimization: Principles and Algorithms, 2nd ed., Lausanne: EPFL Press, 2018.
65. Big Data in Radio Astronomy: Scientific Data Processing for Advanced Radio Telescopes, Elsevier, 2020.
66. *Bishop C. M.* Pattern Recognition and Machine Learning, Springer, 2006.
67. *Blei D. M.* Introduction to Probabilistic Topic Models // Comm. ACM, 2011.

68. *Blei D. M., Ng A. Y., Jordan M. I.* Latent Dirichlet Allocation // Journal of Machine Learning Research, 2003, vol. 3, no. 4–5. — P. 993–1022.
69. *Bliss C. I.* The Method of Probits // Science, 1934, vol. 79, no. 2037. — P. 38–39.
70. *Blunck H., Vahrenhold J.* In-Place Randomized Slope Selection // Algorithms and Complexity, Berlin, Heidelberg: Springer, 2006. — P. 30–41.
71. *Blyth C.* On Minimax Statistical Decision Procedures and Their Admissibility // The Annals of Mathematical Statistics, 1951, vol. 22. — P. 22–42.
72. *Bond R., Smith P.* Culture and Conformity: A Meta-Analysis of Studies Using Asch's (1952b, 1956) Line Judgment Task // Psych. Bull., 1996, vol. 119. — P. 111–137.
73. *Bortkiewicz L.* Das Gesetz der kleinen Zahlen, Leipzig: B.G. Teubner, 1898.
74. *Box G. E. P., Wilson K. B.* On the Experimental Attainment of Optimum Conditions // J. Royal Statistical Society: Series B, 1951, vol. 13, no. 1. — P. 1–38.
75. Breast Tumor Cellularity Assessment using Deep Neural Networks / A. Rakhlin et al. // ICCV 2019 Workshop "Visual Recognition for Medical Images", 2019.
76. *Breiman L.* Random Forests // Mach. Learn., Oct. 2001, vol. 45, no. 1. — P. 5–32.
77. *Breusch T. S., Pagan A. R.* A Simple Test for Heteroscedasticity and Random Coefficient Variation // Econometrica, 1979, vol. 47, no. 5. — P. 1287–1294.
78. *Brier G. W.* Verification of Forecasts Expressed in Terms of Probability // Monthly Weather Review, 1950, vol. 78, no. 1. — P. 1–3.
79. *Brönnimann H., Chazelle B.* Optimal Slope Selection via Cuttings // Computational Geometry, 1998, vol. 10, no. 1. — P. 23–29.
80. *Brownee J.* Generative Adversarial Networks with Python, Machine Learning Mastery, 2019.
81. *Bubeck S.* Convex Optimization: Algorithms and Complexity // Found. Trends Mach. Learn., 2015, vol. 8, no. 3–4. — P. 231–357.
82. *Buchanan B. G., Shortliffe E. H.* Rule Based Expert Systems: The Mycin Experiments of the Stanford Heuristic Programming Project. The Addison-Wesley Series in Artificial Intelligence, Boston, MA, USA: Addison-Wesley Longman Publishing Co., Inc., 1984.
83. *Buehler R., Griffin D., Ross M.* Exploring the "Planning Fallacy": Why People Underestimate Their Task Completion Times // Journal of Personality and Social Psychology, 09 1994, vol. 67. — P. 366–381.
84. *Bundy A.* Automated Theorem Provers: a Practical Tool for the Working Mathematician? // Ann. Math. Artif. Intel., Jan. 2011, vol. 61, no. 1. — P. 3–14.
85. *Burger J. M.* Replicating Milgram: Would People Still Obey Today? // The American Psychologist, January 2009, vol. 64, no. 1. — P. 1–11.
86. *Böken B.* On the Appropriateness of Platt Scaling in Classifier Calibration // Information Systems, 2021, vol. 95. — Article 101641.
87. *Calin B. P., Louis T. A.* Bayesian Methods for Data Analysis, CRC Press, 2008.
88. *Camps-Valls G.* Machine Learning in Remote Sensing Data Processing // 2009 IEEE International Workshop on Machine Learning for Signal Processing, 2009. — P. 1–6.
89. *Capelle K.* A Bird's-Eye View of Density-Functional Theory // Brazilian Journal of Physics, 2006, vol. 36. — P. 1318–1343.
90. *Carleo G., Troyer M.* Solving the Quantum Many-Body Problem with Artificial Neural Networks // Science, Feb. 2017, vol. 355, no. 6325. — P. 602–606.
91. *Carothers A. D.* Capture-Recapture Methods Applied to a Population with Known Parameters // Journal of Animal Ecology, 1973, vol. 42, no. 1. — P. 125–146.
92. Machine Learning in Chemistry / Ed. by H. M. Cartwright. Theoretical and Computational Chemistry Series, The Royal Society of Chemistry, 2020. — P001-546 P.
93. *Casscells W., Schoenberger A., Graboys T. B.* Interpretation by Physicians of Clinical Laboratory Results // New England Journal of Medicine, 1978, vol. 299, no. 18. — P. 999–1001, PMID: 692627.

94. Chemical Space as a Source for New Drugs / J.-L. Reymond et al. // Med. Chem. Commun., 2010, vol. 1. — P. 30–38.
95. *Chen S., Liu J.* Statistical Applications of the Poisson-Binomial and conditional Bernoulli Distributions // Statistica Sinica, 10 1997, vol. 7. — P. 875–892.
96. *Chiles J.-P., Desassis N.* Fifty Years of Kriging // Handbook of Mathematical Geosciences: Fifty Years of IAMG / Ed. by B. Daya Sagar et al., Cham: Springer International Publishing, 2018. — P. 589–612.
97. *Choi S., Cichocki A., Amari S.* Local stability Analysis of Flexible Independent Component Analysis Algorithm // 2000 IEEE International Conference on Acoustics, Speech, and Signal Processing. Proceedings (Cat. No.00CH37100) / vol. 6, 2000. — P. 3426–3429.
98. *Chum O., Matas J.* Matching with PROSAC — Progressive Sample Consensus // 2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR'05) / vol. 1, 2005. — P. 220–226 vol. 1.
99. *Citroni M.* Computation Sparks Chemical Discovery // Nature Communications, 09 2020, vol. 11. — P. 4811.
100. *Clarke R. D.* An Application of the Poisson Distribution // Journal of the Institute of Actuaries, 1946, vol. 72, no. 3. — P. 481.
101. Classifier Calibration: a Survey on How to Assess and Improve Predicted Class Probabilities / T. Silva Filho et al. // Machine Learning, May 2023, vol. 112, no. 9. — P. 3211–3260.
102. *Hong W., Ding M. et al.* CogVideo: Large-scale Pretraining for Text-to-Video Generation via Transformers, 2022.
103. *Cohen B.* The Hot Hand: The Mystery and Science of Streaks, HarperCollins Publishers, 2020.
104. *Collins L., Drewett R., Ferguson R.* Markov Models in Geography // Journal of the Royal Statistical Society: Series D (The Statistician), 1974, vol. 23, no. 3–4. — P. 179–209.
105. A Comprehensive Survey and Analysis of Generative Models in Machine Learning / G. M. Harshvardhan et al. // Computer Science Review, 2020, vol. 38. — Article 100285.
106. A Comprehensive Survey of LIDAR-Based 3D Object Detection Methods with Deep Learning for Autonomous Driving / G. Zamanakos et al. // Computers & Graphics, 2021, vol. 99. — P. 153–181.
107. A Comprehensive Survey of Visual SLAM Algorithms / A. Macario Barros et al. // Robotics, 2022, vol. 11, no. 1.
108. Computational Design of Molecules for an All-Quinone Redox Flow Battery / S. Er et al. // Chem. Sci., 2015, vol. 6. — P. 885–893.
109. Computer Vision Algorithms and Hardware Implementations: A Survey / X. Feng et al. // Integration, 2019, vol. 69. — P. 309–320.
110. *Condorcet J.-A. N. d. C. M. d.* Essai sur l'application de l'analyse la probabilité des décisions rendues la pluralité des voix, Paris, 1785.
111. *Gulati A., Qin J. et al.* Conformer: Convolution-Augmented Transformer for Speech Recognition, 2020.
112. *Connor R.* A Tale of Four Metrics // Similarity Search and Applications, Cham: Springer International Publishing, 2016. — P. 210–217.
113. *Conrad L. I.* The Western Medical Tradition 800 BC to AD 1800, Cambridge: Cambridge Univ. Press, 2011.
114. *Conselice C. J.* The Relationship between Stellar Light Distributions of Galaxies and Their Formation Histories // The Astrophysical Journal Supplement Series, 2003, vol. 147, no. 1. — P. 1–28.
115. *Cooke W. E.* Forecasts and Verifications in Western Australia // Monthly Weather Review, 1906, vol. 34. — P. 23–24.
116. *Cooke W. E.* Weighting Forecasts // Monthly Weather Review, 1906, vol. 34. — P. 274–275.

117. *Cooley J., Lewis P., Welch P.* Historical Notes on the Fast Fourier Transform // IEEE Trans. on Audio and Electroacoustics, 1967, vol. 15, no. 2. — P. 76–79.
118. *Cooley J. W., Tukey J. W.* An Algorithm for the Machine Calculation of Complex Fourier Series // Mathematics of Computation, 1965, vol. 19, no. 90. — P. 297–301.
119. *Corder G., Foreman D.* Nonparametric Statistics: A Step-by-Step Approach, Wiley, 2014.
120. The Cornucopia of Meaningful Leads: Applying Deep Adversarial Autoencoders for New Molecule Development in Oncology / A. Kadurin et al. // Oncotarget, 2017, vol. 8. — P. 10883–10890.
121. *Council N. R.* Language and Machines: Computers in Translation and Linguistics, Washington, DC: The National Academies Press, 1966. https://nap.nationalacademies.org/catalog/9547/language-and-machines-computers-in-translation-and-linguistics.
122. *Cover T., Hart P.* Nearest Neighbor Pattern Classification // IEEE Transactions on Information Theory, 1967, vol. 13, no. 1. — P. 21–27.
123. *Cramer J. S.* The Early Origins of the Logit Model // Studies in History and Philosophy of Science Part C: Studies in History and Philosophy of Biological and Biomedical Sciences, 2004, vol. 35, no. 4. — P. 613–626.
124. *Cressie N. A. C.* The Origins of Kriging // Mathematical Geology, 1990, vol. 22. — P. 239–252.
125. *Crevier D.* AI: The Tumultuous Search for Artificial Intelligence, New York, NY: BasicBooks, 1993.
126. *D'Abrusco R., Longo G., Walton N. A.* Quasar Candidates Selection in the Virtual Observatory Era // Monthly Notices of the Royal Astronomical Society, 06 2009, vol. 396, no. 1. — P. 223–262.
127. *Dacey M., Krumbein W.* Markovian Models in Stratigraphic Analysis // Journal of the International Association for Mathematical Geology, 1970, vol. 2. — P. 175–191.
128. *Dahn B. I.* Robbins Algebras Are Boolean: A Revision of McCune's Computer-Generated Solution of Robbins Problem // Journal of Algebra, 1998, vol. 208, no. 2. — P. 526–532.
129. *Dalton J.* Meteorological Essays, Richardson, Phillips and Pennington, 1793.
130. *Daniel W.* Applied Nonparametric Statistics. Duxbury Advanced Series in Statistics and Decision Sciences, PWS-KENT Pub., 1990. https://books.google.ru/books?id=0hPvAAAAMAAJ.
131. *Darby S. C., Nakashima E., Kato H.* A Parallel Analysis of Cancer Mortality Among Atomic Bomb Survivors and Patients With Ankylosing Spondylitis Given X-Ray Therapy2 // JNCI: Journal of the National Cancer Institute, 07 1985, vol. 75, no. 1. — P. 1–21.
132. *Darmois G.* Sur les lois de probabilitéa Estimation exhaustive // CR Acad. Sci. Paris, 1935, vol. 260, no. 1265. — P. 85.
133. *Davies P., Gather U.* The Breakdown Point—Examples and Counterexamples // REVSTAT – Statistical Journal Volume, 04 2007, vol. 5. — P. 1–17.
134. Deep Learning for Scene Classification: A Survey / D. Zeng et al. // ArXiv preprint abs/2101.10531, 2021. https://arxiv.org/abs/2101.10531.
135. Deep Neural Networks for Acoustic Modeling in Speech Recognition: The Shared Views of Four Research Groups / G. Hinton et al. // IEEE Signal Processing Magazine, Nov. 2012, vol. 29, no. 6. — P. 82–97.
136. DeepCycle Reconstructs a Cyclic Cell Cycle Trajectory from Unsegmented Cell Images using Convolutional Neural Networks / L. Rappez et al. // Molecular Systems Biology, 2020, vol. 16, no. 10. — P. e9474.
137. *del Pino G.* The Unifying Role of Iterative Generalized Least Squares in Statistical Algorithms // Statistical Science, 1989, vol. 4, no. 4. — P. 394–403.
138. Delving Deep Into Label Smoothing / C.-B. Zhang et al. // IEEE Transactions on Image Processing, 2021, vol. 30. — P. 5984–5996.

139. *Denison S., Xu F.* Integrating Physical Constraints in Statistical Inference by 11-Month-Old Infants // Cognitive Science, 2010, vol. 34, no. 5. — P. 885–908.
140. Design of Efficient Molecular Organic Light-Emitting Diodes by a High-Throughput Virtual Screening and Experimental Approach / R. Gomez-Bombarelli et al. // Nature Materials, Oct. 2016, vol. 15, no. 10. — P. 1120–1127.
141. Diagnostic Accuracy of Deep Learning in Medical Imaging: a Systematic Review and Meta-Analysis / R. Aggarwal et al. // NPJ Digital Medicine, 12 2021, vol. 4.
142. *Dietz E. J.* A Comparison of Robust Estimators in Simple Linear Regression // Communications in Statistics — Simulation and Computation, 1987, vol. 16. — P. 1209–1227.
143. *Dietz E. J.* Teaching Regression in a Nonparametric Statistics Course // The American Statistician, 1989, vol. 43, no. 1. — P. 35–40.
144. *DiMaggio P., Nag M., Blei D.* Exploiting Affinities between Topic Modeling and the Sociological Perspective on Culture: Application to Newspaper Coverage of U.S. Government Arts Funding // Poetics, 2013, vol. 41, no. 6. — P. 570–606.
145. Discovering Faster Matrix Multiplication Algorithms with Reinforcement Learning / A. Fawzi et al. // Nature, 10 2022, vol. 610. — P. 47–53.
146. *Doersch C.* Tutorial on Variational Autoencoders // ArXiv e-prints, Jun. 2016.
147. *Domingos P., Pazzani M.* On the Optimality of the Simple Bayesian Classifier under Zero-One Loss // Machine Learning — ML, 01 1997, vol. 29. — P. 103–130.
148. *Domingos P. M.* A Unifeid Bias-Variance Decomposition and its Applications // Proc. 17th International Conference on Machine Learning (ICML 2000), Stanford University, Stanford, CA, USA, June 29 — July 2, 2000, Morgan Kaufmann, 2000. — P. 231–238.
149. *Donoho D. L.* For Most Large Underdetermined Systems of Linear Equations the Minimal ℓ_1-norm Solution is Also the Sparsest Solution // Communications on Pure and Applied Mathematics, 2006, vol. 59, no. 6. — P. 797–829.
150. *Donoho D. L., Huber P. J.* The Notion of Breakdown Point // A Festschrift for Erich Lehmann / Ed. by P. J. Bickel et al., Wadsworth, Belmont, CA, 1983. — P. 157–184.
151. *Dramsch J. S.* Chapter One — 70 Years of Machine Learning in Geoscience in Review // Machine Learning in Geosciences / Elsevier, 2020. — P. 1–55.
152. *Draper N. R., Nostrand R. C. V.* Ridge Regression and James-Stein Estimation: Review and Comments // Technometrics, 1979, vol. 21, no. 4. — P. 451–466.
153. *Dreyfus S. E.* The Computational Solution of Optimal Control Problems with Time Lag // IEEE Transactions on Automatic Control, 1973, vol. 18, no. 4. — P. 383–385.
154. druGAN: An Advanced Generative Adversarial Autoencoder Model for de Novo Generation of New Molecules with Desired Molecular Properties in Silico / A. Kadurin et al. // Molecular Pharmaceutics, 2017, vol. 14, no. 9. — P. 3098–3104.
155. *Dua D., Graff C.* UCI Machine Learning Repository, 2017. http://archive.ics.uci.edu/ml.
156. *Duchi J., Hazan E., Singer Y.* Adaptive Subgradient Methods for Online Learning and Stochastic Optimization // Journal of Machine Learning Research, Jul. 2011, vol. 12. — P. 2121–2159.
157. *Duda R., Hart P., Stork D.* Pattern Classification, Wiley, 2000.
158. *Dunn P. K., Smyth G. K.* Generalized Linear Models With Examples in R, New York: Springer, 2018.
159. *Easley D., Kleinberg J.* Networks, Crowds, and Markets: Reasoning About a Highly Connected World, Cambridge Univ. Press, 2010.
160. *Ebden M.* Gaussian Processes: A Quick Introduction, 2015.
161. Effective Detection of Sophisticated Online Banking Fraud on Extremely Imbalanced Data / W. Wei et al. // World Wide Web, 2013, vol. 16. — P. 449–475.
162. Efficient Data Augmentation for Fitting Stochastic Epidemic Models to Prevalence Data / J. Fintzi et al. // Journal of Computational and Graphical Statistics, 2017, vol. 26, no. 4. — P. 918–929.

163. *Efron B., Hastie T.* James–Stein Estimation and Ridge Regression // Computer Age Statistical Inference: Algorithms, Evidence, and Data Science, Institute of Mathematical Statistics Monographs. Cambridge University Press, 2016. — P. 91–107.
164. *Efron B., Morris C.* Stein's Estimation Rule and Its Competitors — an Empirical Bayes Approach // Journal of the American Statistical Association, 1973, vol. 68. — P. 117–130.
165. *Efron B., Morris C.* Data Analysis using Stein's Estimator and Its Generalizations // Journal of the American Statistical Association, 1975, vol. 70. — P. 311–319.
166. *Efron B., Morris C.* Stein's Paradox in Statistics // Scientific American, 1977, vol. 236. — P. 119–127.
167. *Elkan C.* The Foundations of Cost-Sensitive Learning // Proc. 17th International Joint Conference on Artificial Intelligence — Volume 2, San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 2001. — P. 973–978.
168. *Emmons R., McCullough M.* Counting Blessings Versus Burdens: An Experimental Investigation of Gratitude and Subjective Well-Being in Daily Life // Journal of Personality and Social Psychology, 02 2003, vol. 84. — P. 377–389.
169. *Esser P., Rombach R., Ommer B.* Taming Transformers for High-Resolution Image Synthesis // Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), June 2021. — P. 12873–12883.
170. *Estoup J.-B.* Gammes sténographiques. Recueil de textes choisis pour l'acquisition méthodique de la vitesse, Institut sténographique, 1912.
171. Evaluating Large Language Models Trained on Code / M. Chen et al. // ArXiv preprint abs/2107.03374, 2021. https://arxiv.org/abs/2107.03374.
172. Evolutionary Undersampling Boosting for Imbalanced Classification of Breast Cancer Malignancy / B. Krawczyk et al. // Applied Soft Computing, 2016, vol. 38. — P. 714–726.
173. Fast Collapsed Gibbs Sampling for Latent Dirichlet Allocation / I. Porteous et al. // Proc. 14th ACM SIGKDD Intl. Conf. on Knowledge Discovery and Data Mining, New York, NY, USA: ACM, 2008. — P. 569–577.
174. *Rekesh D., Koluguri N. R. et al.* Fast Conformer with Linearly Scalable Attention for Efficient Speech Recognition, 2023.
175. Faster Sorting Algorithms Discovered Using Deep Reinforcement Learning / D. Mankowitz et al. // Nature, 06 2023, vol. 618. — P. 257–263.
176. *Fateman R. J.* A Review of Macsyma // IEEE Trans. Knowl. Data Eng., 1989, vol. 1, no. 1. — P. 133–145.
177. *Fechner G. T.* Elemente der Psychophysik, Leipzig: Breitkopf und Hartel, 1860.
178. Feeling the future: A Meta-Analysis of 90 Experiments on the Anomalous Anticipation of Random Future Events / D. Bem et al. // F1000Research, 10 2015, vol. 4. — P. 1188.
179. *Feickert M., Nachman B.* A Living Review of Machine Learning for Particle Physics, 2021. https://iml-wg.github.io/HEPML-LivingReview/.
180. *Finney D.* Probit Analysis, 3rd (1st ed. 1947) edition, Cambridge Univ. Press, 1971.
181. *Firth D.* Bias Reduction, the Jeffreys Prior and GLIM // Advances in GLIM and Statistical Modelling, New York, NY: Springer New York, 1992. — P. 91–100.
182. *Fischler M. A., Bolles R. C.* Random Sample Consensus: a Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography // Commun. ACM, jun 1981, vol. 24, no. 6. — P. 381–395.
183. *Fisher R.* The Design of Experiments, Macmillan, 1935.
184. *Fisher R. A.* On the "Probable Error" of a Coefficient of Correlation Deduced from a Small Sample // Metron, 1921, vol. 1. — P. 1–32.
185. *Fisher R. A.* The Use of Multiple Measurements in Taxonomic Problems // Annals of Eugenics, 1936, vol. 7, no. 2. — P. 179–188.
186. *Fisher R. A.* Dangers of Cigarette-smoking // BMJ, 1957, vol. 2, no. 5035. — P. 43.

187. *Fisher R. A.* Cigarettes, Cancer, and Statistics // The Centennial Review of Arts & Science, 1958, vol. 2. — P. 151–166.
188. *Fisher R. A.* Lung Cancer and Cigarettes? // Nature, 1958, vol. 182. — P. 108.
189. *Fisher R. A., Russell E. J.* On the Mathematical Foundations of Theoretical Statistics // Philosophical Transactions of the Royal Society of London. Series A, Containing Papers of a Mathematical or Physical Character, 1922, vol. 222, no. 594–604. — P. 309–368.
190. *Flach P.* Classification in Context: Adapting to Changes in Class and Cost Distribution // 1st International Workshop on Learning over Multiple Contexts, ECML-PKDD, Springer, 2014.
191. *Foix S., Alenya G., Torras C.* Lock-in Time-of-Flight (ToF) Cameras: A Survey // IEEE Sensors Journal, 2011, vol. 11, no. 9. — P. 1917–1926.
192. *for the Women's Health Initiative Investigators W. G.* Risks and Benefits of Estrogen Plus Progestin in Healthy Postmenopausal Women: Principal Results From the Women's Health Initiative Randomized Controlled Trial // JAMA, 07 2002, vol. 288, no. 3. — P. 321–333.
193. *Fortnow L.* The Status of the P versus NP Problem. // Commun. ACM, 2009, vol. 52, no. 9. — P. 78–86.
194. *Madan N., Moegelmose A. et al.* Foundation Models for Video Understanding: A Survey, 2024.
195. *Francis G.* The Psychology of Replication and Replication in Psychology // Perspectives on Psychological Science, 2012, vol. 7, no. 6. — P. 585–594.
196. *Frank I. E., Friedman J. H.* A Statistical View of Some Chemometrics Regression Tools // Technometrics, 1993, vol. 35, no. 2. — P. 109–135.
197. *Freund Y.* Boosting a Weak Learning Algorithm by Majority // Information and Computation, 1995, vol. 121, no. 2. — P. 256–285.
198. *Freund Y., Schapire R. E.* A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting // Journal of Computer and System Sciences, 1997, vol. 55, no. 1. — P. 119–139.
199. *Friedman J. H.* Regularized Discriminant Analysis // Journal of the American Statistical Association, 1989, vol. 84, no. 405. — P. 165–175.
200. *Friend R., Rafferty Y., Bramel D.* A Puzzling Misinterpretation of the Asch "Conformity" Study // European Journal of Social Psychology, 1990, vol. 20, no. 1. — P. 29–44.
201. *Fukunaga K.* Introduction to Statistical Pattern Classification, USA: Academic Press, 1990.
202. Fuzzy-Rough imbalanced Learning for the Diagnosis of High Voltage Circuit Breaker Maintenance: The SMOTE-FRST-2T Algorithm / E. Ramentol et al. // Engineering Applications of Artificial Intelligence, 2016, vol. 48. — P. 134–139.
203. *Gaboune B., Laporte G., Soumis F.* Expected Distances between Two Uniformly Distributed Random Points in Rectangles and Rectangular Parallelpipeds // The Journal of the Operational Research Society, 1993, vol. 44, no. 5. — P. 513–519.
204. *Gage P.* A New Algorithm for Data Compression, 1994.
205. *Galton F.* Regression Towards Mediocrity in Hereditary Stature // The Journal of the Anthropological Institute of Great Britain and Ireland, Jan. 1886, vol. 15. — P. 246–263.
206. *Gangemi A., Presutti V.* Ontology Design Patterns // Handbook on Ontologies / Ed. by S. Staab, R. Studer, Berlin, Heidelberg: Springer Berlin Heidelberg, 2009. — P. 221–243.
207. *Gaskell T. J., George B. J.* A Bayesian Modification of the Lincoln Index // Journal of Applied Ecology, 1972, vol. 9, no. 2. — P. 377–384.
208. *Gauss J. C. F.* Theoria Combinationis Observationum Erroribus Minimis Obnoxiae, Gottingen, 1825.
209. *Gavrilovich M., Nikolenko S. I., Vavilov N. A.* Structure of Chevalley Groups: the Proof from the Book // Journal of Mathematical Sciences, 2007, vol. 140, no. 5. — P. 626–645.

210. *Gazey W.J., Staley M.J.* Population Estimation from Mark-Recapture Experiments Using a Sequential Bayes Algorithm // Ecology, 1986, vol. 67, no. 4. — P. 941–951.
211. *Gelman A.* Too Good to be True // Slate, 2013.
212. *Gelman A., Loken E.* The Garden of Forking Paths: Why Multiple Comparisons can be a Problem, even When There is no "Fishing Expedition" or "p-hacking" and the Research Hypothesis was Posited Ahead of Time, http://www.stat.columbia.edu/~gelman/research/unpublished/p_hacking.pdf, 2013.
213. *Geman S., Bienenstock E., Doursat R.* Neural Networks and the Bias/Variance Dilemma // Neural Computation, 01 1992, vol. 4, no. 1. — P. 1–58.
214. A General Reinforcement Learning Algorithm that masters chess, shogi, and Go through self-play / D. Silver et al. // Science, 2018, vol. 362, no. 6419. — P. 1140–1144.
215. Generative Adversarial Networks / I. J. Goodfellow et al. // ArXiv e-prints, Jun. 2014.
216. Geoscience-aware Deep Learning: A New paradigm for Remote Sensing / Y. Ge et al. // Science of Remote Sensing, 2022, vol. 5. — P. 100047.
217. *Gibrat R.* Les Inégalités économiques, Paris, France, 1931.
218. *Giger M. L.* Machine Learning in Medical Imaging // Journal of the American College of Radiology, 2018, vol. 15, no. 3, Part B. — P. 512–520, Data Science: Big Data Machine Learning and Artificial Intelligence.
219. *Gigerenzer G.* Reckoning with Risk: Learning to Live with Uncertainty, Penguin, 2003.
220. *Gigerenzer G., Hoffrage U.* How to Improve Bayesian Reasoning Without Instruction: Frequency Formats // Psychological Review, 1995, vol. 102, no. 4. — P. 684–704.
221. *Gilovich T., Vallone R., Tversky A.* The hot hand in basketball: On the misperception of random sequences // Cognitive Psychology, 1985, vol. 17, no. 3. — P. 295–314.
222. *Girshick M., Savage L.* Bayes and Minimax Estimates for quadratic loss Functions // Proc. of the 2nd Berkeley Symposium on Mathematical Statistics and Probability / vol. 1, University of California Press, 1951. — P. 53–73.
223. *Goerigk L., Mehta N.* A Trip to the Density Functional Theory Zoo: Warnings and Recommendations for the User // Australian Journal of Chemistry, 01 2019, vol. 72, no. 8. — P. 563–573.
224. *Gonthier G.* A Computer-Checked Proof of the Four Color Theorem: Tech. rep.: Inria, Mar. 2023. https://inria.hal.science/hal-04034866.
225. *Goodfellow I.J.* NIPS 2016 Tutorial: Generative Adversarial Networks // ArXiv preprint abs/1701.00160, 2017. http://arxiv.org/abs/1701.00160.
226. Google's Neural Machine Translation System: Bridging the Gap between Human and Machine Translation / Y. Wu et al. // ArXiv preprint abs/1609.08144, 2016. http://arxiv.org/abs/1609.08144.
227. *Green P.J.* Iteratively Reweighted Least Squares for Maximum Likelihood Estimation, and some Robust and Resistant Alternatives // Journal of the Royal Statistical Society. Series B (Methodological), 1984, vol. 46, no. 2. — P. 149–192.
228. *Greenbaum A.* Iterative Methods for Solving Linear Systems, USA: Society for Industrial and Applied Mathematics, 1997.
229. *Griggs R. A.* The Disappearance of Independence in Textbook Coverage of Asch's Social Pressure Experiments // Teaching of Psychology, 2015, vol. 42, no. 2. — P. 137–142.
230. *Guarino N., Oberle D., Staab S.* What Is an Ontology? // Handbook on Ontologies / Ed. by S. Staab, R. Studer, Berlin, Heidelberg: Springer Berlin Heidelberg, 2009. — P. 1–17.
231. *Gugerty L.* Newell and Simon's Logic Theorist: Historical Background and Impact on Cognitive Modeling // Proc. Human Factors and Ergonomics Society Annual Meeting, 2006, vol. 50, no. 9. — P. 880–884.
232. *Gundersen O. E., Kjensmo S.* State of the Art: Reproducibility in Artificial Intelligence // Proc. AAAI Conference on Artificial Intelligence, Apr. 2018, vol. 32, no. 1.

233. *Guo Y., Hastie T., Tibshirani R.* Regularized Linear discriminant Analysis and its Application in Microarrays // Biostatistics, 04 2006, vol. 8, no. 1. — P. 86–100.
234. *Hallenbeck C.* Forecasting Precipitation in Percentages of Probability // Monthly Weather Review, 1920, vol. 48, no. 11. — P. 645–647.
235. *Halpern J. Y.* Reasoning about uncertainty, Cambridge, Massachusetts: MIT Press, 2003.
236. *Hamming R. W.* Error Detecting and Error Correcting Codes // The Bell System Technical Journal, 1950, vol. 29, no. 2. — P. 147–160.
237. *Han H., Wang W.-Y., Mao B.-H.* Borderline-SMOTE: a New Over-Sampling Method in Imbalanced Data Sets Learning // International Conference on Intelligent Computing, 2005. — P. 878–887.
238. *Hand D. J., Yu K.* Idiot's Bayes: Not So Stupid after All? // International Statistical Review / Revue Internationale de Statistique, 2001, vol. 69, no. 3. — P. 385–398.
239. *Haney C., Banks C., Zimbardo P. G.* A Study of Prisoners and Guards in a Simulated Prison // Naval Research Review, 1973, no. 30. — P. 4–17.
240. *Hanneman R. A., Riddle M.* Introduction to Social Network Methods, Riverside, CA: University of California, Riverside, 2005.
241. *Harnad S.* Minds, Machines and Searle // Journal of Experimental & Theoretical Artificial Intelligence, 1989, vol. 1. — P. 5–25.
242. *Harnad S.* Correlation Vs. Causality: How/Why the Mind-Body Problem is Hard // Journal of Consciousness Studies, 2000, vol. 7, no. 4. — P. 54–61.
243. *Harnad S.* Minds, Machines and Turing: The Indistinguishability of Indistinguishables // Journal of Logic, Language, and Information, 2000, vol. 9, no. 4. — P. 425–445.
244. *Harnad S.* Minds, Machines, and Searle 2: What's Right and Wrong about the Chinese Room Argument // Views into the Chinese Room: New Essays on Searle and Artificial Intelligence / Oxford University Press, 2002.
245. *Hastie T., Tibshirani R.* Generalized Additive Models, Chapman & Hall/CRC, 1990.
246. *Hastie T., Tibshirani R., Friedman J.* The Elements of Statistical Learning. Springer Series in Statistics, New York, NY, USA: Springer New York Inc., 2001.
247. *Hayes B.* Gauss's Day of Reckoning // American Scientist, 2006, vol. 94, no. 3. — P. 200. https://www.americanscientist.org/article/gausss-day-of-reckoning.
248. *He H., Garcia E. A.* Learning from Imbalanced Data // IEEE Transactions on Knowledge and Data Engineering, 2009, vol. 21, no. 9. — P. 1263–1284.
249. *He H., Ma Y.* Imbalanced Learning: Foundations, Algorithms, and Applications, 2013.
250. *He H., Shen X.* A Ranked Subspace Learning Method for Gene Expression Data Classification // IC-AI, 2007. — P. 358–364.
251. *health service utilization forecasting team I. C.-., Murray C. J.* Forecasting COVID-19 Impact on Hospital Bed-Days, ICU-Days, Ventilator-Days and Deaths by US State in the Next 4 Months // medRxiv, 2020.
252. *Helmert F. R.* Über die Berechnung des wahrscheinlichen Fehlers aus einer endlichen Anzahl wahrer Beobachtungsfehler // Zeitschrift für Mathematik und Physik, 1875, vol. 20. — P. 300–303.
253. *Helmert F. R.* Über die Wahrscheinlichkeit der Potenzsummen der Beobachtungsfehler und uber einige damit in Zusammenhang stehende Fragen // Zeitschrift für Mathematik und Physik, 1876, vol. 21. — P. 192–218.
254. *Hernan M. A., Robins J. M.* Causal Inference: What If, Boca Raton: Chapman & Hall/CRC, 2020.
255. *Hernandez-Orallo J., Flach P., Ferri C.* Brier curves: a New Cost-based visualisation of Classifier performance // Proceedings of the 28th International Conference on Machine Learning, Madison, WI, USA: Omnipress, 2011. — P. 585–592.

256. *Hero A., Fessler J., Usman M.* Exploring Estimator Bias-Variance Tradeoffs Using the Uniform CR bound // IEEE Transactions on Signal Processing, 1996, vol. 44, no. 8. — P. 2026–2041.
257. *Hestenes M. R., Stiefel E.* Methods of Conjugate Gradients for Solving Linear Systems // Journal of Research of the National Bureau of Standards, 1952, vol. 49. — P. 409–436.
258. *Ramey J. A., Stein C. K. et al.* High-Dimensional Regularized Discriminant Analysis, 2017.
259. *Rombach R., Blattmann A. et al.* High-Resolution Image Synthesis with Latent Diffusion Models, 2021.
260. High-Resolution Image Synthesis With Latent Diffusion Models / R. Rombach et al. // Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), June 2022. — P. 10684–10695.
261. *Hill R.* Multiple sudden infant deaths – coincidence or beyond coincidence? // Paediatric and Perinatal Epidemiology, 2004, vol. 18, no. 5. — P. 320–326.
262. HMM-Based Audio Keyword Generation / M. Xu et al. // Advances in Multimedia Information Processing — PCM 2004, Berlin, Heidelberg: Springer Berlin Heidelberg, 2005. — P. 566–574.
263. *Hodges Jr. J. L., Lehmann E. L.* Some Applications of the Cramer-Rao Inequality // Proc. Second Berkeley Symposium on Mathematical Statistics and Probability / vol. 1, University of California Press, 1951. — P. 13–22.
264. *Hoerl A. E.* Optimum Solution of Many Variables Equations // Chemical Engineering Progress, 1959, vol. 55. — P. 69–78.
265. *Hoerl A. E.* Ridge Analysis // Chemical Engineering Progress Symposium Series, 1964, vol. 60. — P. 67–77.
266. *Hoerl A. E., Kennard R. W.* Ridge Regression: Applications to Nonorthogonal Problems // Technometrics, 1970, vol. 12, no. 1. — P. 69–82.
267. *Hoerl A. E., Kennard R. W.* Ridge Regression: Biased Estimation for Nonorthogonal Problems // Technometrics, 1970, vol. 12, no. 1. — P. 55–67.
268. *Hofmann T., Schölkopf B., Smola A. J.* Kernel Methods in Machine Learning // The Annals of Statistics, 2008, vol. 36, no. 3. — P. 1171–1220.
269. *Höhle M., Held L.* Bayesian Estimation of the Size of a Population: Tech. Rep. Technical Report SFB 386, No. 399: Department of Statistics, University of Munich, 2006.
270. *Hutchins J.* From First Conception to First Demonstration: the Nascent Years of Machine Translation, 1947–1954. A Chronology // Machine Translation, 1997, vol. 12, no. 3. — P. 195–252.
271. *Hutchins W. J.* The Georgetown-IBM Experiment Demonstrated in January 1954 // Machine Translation: From Real Users to Research, Berlin, Heidelberg: Springer Berlin Heidelberg, 2004. — P. 102–114.
272. *Ho J., Chan W. et al.* Imagen Video: High Definition Video Generation with Diffusion Models, 2022.
273. Imaging volition: what the brain can tell us about the will / M. Brass et al. // Experimental brain Research, 2013, vol. 229. — P. 301–312.
274. *Betker J., Goh G. et al.* Improving Image Generation with Better Captions, OpenAI, https://cdn.openai.com/papers/dall-e-3.pdf.
275. *Radford A., Narasimhan K. et al.* Improving Language Understanding by Generative Pre-Training, 2018. https://s3-us-west-2.amazonaws.com/openai-assets/research-covers/language-unsupervised/language_understanding_paper.pdf.
276. Improving Reproducibility in Machine Learning Research (a Report from the NeurIPS 2019 Reproducibility Program) / J. Pineau et al. // J. Mach. Learn. Res., Jan. 2021, vol. 22, no. 1. — 20 P.

277. *Indyk P., Motwani R.* Approximate Nearest Neighbors: Towards Removing the Curse of Dimensionality // Proc. Thirtieth Annual ACM Symposium on Theory of Computing, New York, NY, USA: ACM, 1998. — P. 604–613.
278. Initial Evidence for Probabilistic Reasoning in a Grey Parrot (Psittacus erithacus) / K. A. Clements et al. // Journal of Comparative Psycholog, 2018, vol. 132, no. 2. — P. 166–177.
279. *Ioannidis J. P. A.* Why Most Published Research Findings Are False // PLOS Medicine, 08 2005, vol. 2, no. 8.
280. *Iso-Ahola S. E.* Replication and the Establishment of Scientific Truth // Frontiers in Psychology, 2020, vol. 11.
281. *da Silva M. H. M., da Silva J. V. S. et al.* ISP Meets Deep Learning: A Survey on Deep Learning Methods for Image Signal Processing, 2023.
282. *Jaccard P.* The Distribution of the Flora in the Alpine Zone. 1 // New Phytologist, 1912, vol. 11, no. 2. — P. 37–50.
283. *Jakoby B.* Mark and Recapture: Statistical Treatment, Wikipedia, 2017. https://en.wikipedia.org/wiki/Talk:Mark_and_recapture.
284. *James W., Stein C.* Estimation with Quadratic Loss // Proc. Fourth Berkeley Symposium on Mathematical Statistics and Probability / vol. 1, University of California Press, 1961. — P. 361–379.
285. *Jaynes E. T.* Prior Probabilities // IEEE Transactions on Systems Science and Cybernetics, 1968, vol. SSC-4. — P. 227–241.
286. *Jaynes E. T.* Probability Theory: The Logic of Science, Cambridge Univ. Press, 2003.
287. *Jeffreys H.* Theory of Probability, Oxford: Oxford University Press, 1939.
288. *Jeffreys H.* An Invariant Form for the Prior Probability in Estimation Problems // Proc. Royal Society of London. Series A. Mathematical and Physical Sciences, 1946, vol. 186, no. 1007. — P. 453–461.
289. *Jiao L., Zhao J.* A Survey on the New Generation of Deep Learning in Image Processing // IEEE Access, 2019, vol. 7. — P. 172 231–172 263.
290. *Johns L.* The Prosecutor's Fallacy: Conditional Probability in the Courtroom, 2019. https://towardsdatascience.com/the-prosecutors-fallacy-cb0da4e9c039.
291. *Johnson R. W.* Estimating the Size of a Population // Teaching Statistics, 1994, vol. 16, no. 2. — P. 50–52.
292. *Jovanis P. P., Chang H.-L.* Modeling the Relationship of Accidents to Miles Traveled // Transportation Research Record, 1986.
293. *Kahneman D.* Thinking, Fast and Slow, Farrar, Straus and Giroux, 2011.
294. *Kahneman D., Diener E., Schwarz N.* Well-Being: The Foundations of Hedonic Psychology, Russell Sage Foundation, 1999.
295. *Kahneman D., Slovic P., Tversky A.* Judgment under Uncertainty: Heuristics and Biases, Cambridge Univ. Press, 1982.
296. *Kahneman D., Smith V. L.* Daniel Kahneman: Biographical, 2002. https://www.nobelprize.org/prizes/economic-sciences/2002/kahneman/biographical/.
297. *Kahneman D., Tversky A.* Prospect Theory: An Analysis of Decision under Risk // Econometrica, 1979, vol. 47, no. 2. — P. 263–291.
298. *Kahneman D., Tversky A.* Intuitive Prediction: Biases and Corrective Procedures // Judgment under Uncertainty: Heuristics and Biases / Ed. by D. Kahneman et al., Cambridge Univ. Press, 1982. — P. 414–421.
299. *Kahneman D., Tversky A.* Extensional versus intuitive Reasoning: The Conjunction Fallacy in Probability judgment // Psychological Review, 1983, vol. 90, no. 4. — P. 293–315.
300. *Kahneman D., Tversky A.* Choices, values, and frames, Cambridge Univ. Press, 2000.
301. *Kant I.* Kritik der reinen Vernunft, J. F. Hartknoch, Riga, 1781.
302. *Kant I.* Kritik der praktischen Vernunft, J. F. Hartknoch, Riga, 1788.

303. *Kant I.* Kritik der Urteilskraft, Verlag Lagarde und Friedrich, Berlin, 1790.
304. *Katz M. J., Sharir M.* Optimal slope Selection via expanders // Information Processing Letters, 1993, vol. 47, no. 3. — P. 115–122.
305. *Katz Y.* Noam Chomsky on Where Artificial Intelligence Went Wrong, The Atlantic, https://www.theatlantic.com/technology/archive/2012/11/noam-chomsky-on-where-artificial-intelligence-went-wrong/261637/?single_page=true, 2012.
306. *Kingma D., Ba J.* Adam: A Method for Stochastic Optimization // arXiv preprint arXiv:1412.6980, 2014.
307. *Kingma D. P., Welling M.* Auto-Encoding Variational Bayes // 2nd International Conference on Learning Representations, ICLR 2014, Banff, AB, Canada, April 14–16, 2014, Conference Track Proceedings, 2014. http://arxiv.org/abs/1312.6114.
308. *Kleinberg J. M., Liang A., Mullainathan S.* The Theory is Predictive, but is it Complete? An Application to Human Perception of Randomness // ArXiv preprint abs/1706.06974, 2017. http://arxiv.org/abs/1706.06974.
309. *Kleinrock L.* Queueing Systems: Volume I – Theory, New York: Wiley Interscience, 1975. — 417 p.
310. *Kleinrock L.* Queueing Systems: Volume II – Computer Applications, New York: Wiley Interscience, 1976. — 576 p.
311. *Klir G. J., St. Clair U. H., Yuan B.* Fuzzy Set Theory: Foundations and Applications, Englewood Cliffs, NJ: Prentice Hall, 1997.
312. *Koltsova O., Koltsov S., Nikolenko S. I.* Communities of Co-Commenting in the Russian LiveJournal and Their Topical Coherence // Internet Research, 2016, vol. 26, no. 3. — P. 710–732.
313. *Konishi S., Kitagawa G.* Information Criteria and Statistical Modeling. Springer Series in Statistics, New York, NY, USA: Springer New York Inc., 2001.
314. *Koopman B. O.* On Distributions admitting a sufficient Statistic // Transactions of the American Mathematical Society, 1936, vol. 39, no. 3. — P. 399–409.
315. *Koren Y.* Factorization Meets the Neighborhood: a Multifaceted Collaborative Filtering Model // Proc. 14th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, Las Vegas, Nevada, USA: IEEE Computer Society, 2008. — P. 426–434.
316. *Koren Y., Bell R. M.* Advances in Collaborative Filtering // Recommender Systems Handbook / Springer, 2011. — P. 145–186.
317. *Kornilowicz A.* Elementary Number Theory Problems. Part IX // Formalized Mathematics, 2023, vol. 31, no. 1. — P. 161–169.
318. *Kornilowicz A.* Elementary Number Theory Problems. Part X – Diophantine Equations // Formalized Mathematics, 2023, vol. 31, no. 1. — P. 171–180.
319. *Kosub S.* A Note on the Triangle Inequality for the Jaccard Distance // Pattern Recognition Letters, Apr. 2019, vol. 120. — P. 36–38.
320. *Krawczyk B.* Learning from Imbalanced Data: Open Challenges and Future Directions // Progress in Artificial Intelligence, 2016, vol. 5. — P. 221–232.
321. *Krumbein W. C., Dacey M.* Markov Chains and Embedded Markov Chains in Geology // J. International Association for Mathematical Geology, 1969, vol. 1. — P. 79–96.
322. *Kubat M., Holte R. C., Matwin S.* Machine Learning for the Detection of Oil Spills in Satellite Radar Images // Machine Learning, 1998, vol. 30. — P. 195–215.
323. *Kubat M., Matwin S. et al.* Addressing the Curse of Imbalanced Training Sets: One-Sided Selection // ICML / vol. 97, 1997. — P. 179.
324. *Kullback S., Leibler R. A.* On Information and Sufficiency // The Annals of Mathematical Statistics, 1951, vol. 22, no. 1. — P. 79–86.
325. *Kumar N., Zhang L., Nayar S.* What Is a Good Nearest Neighbors Algorithm for Finding Similar Patches in Images? // Computer Vision – ECCV 2008, Berlin, Heidelberg: Springer Berlin Heidelberg, 2008. — P. 364–378.

326. *Kuncheva L. I.* On the optimality of Na ve Bayes with Dependent Binary Features // Pattern Recognition Letters, 2006, vol. 27, no. 7. — P. 830–837.
327. *Lagay F.* The Legacy of Humoral Medicine // Virtual Mentor, 2002, vol. 4, no. 7. — P. 206–208.
328. *Lamarck J. B.* Annuaire Meteorologique, Vols. 1–11, Chez l'Auteur, 1800–1811.
329. *Lamel L., Kassel R., Seneff S.* Speech Database Development: Design and Analysis of the Acoustic-Phonetic Corpus // Proc. DARPA Speech Recognition Workshop, 01 1989.
330. Language Models are Few-Shot Learners / T. Brown et al. // Advances in Neural Information Processing Systems / vol. 33, Curran Associates, Inc., 2020. — P. 1877–1901.
331. Language Models are Unsupervised Multitask Learners / A. Radford et al., 2019.
332. *Lau H. C., Rogers R. D., Passingham R. E.* On Measuring the Perceived Onsets of Spontaneous Actions // Journal of Neuroscience, 2006, vol. 26, no. 27. — P. 7265–7271.
333. *Lavazza A.* Free Will and Neuroscience: From Explaining Freedom Away to New Ways of Operationalizing and Measuring It // Frontiers in Human Neuroscience, 2016, vol. 10.
334. *Lawlor D. A., Davey Smith G., Ebrahim S.* Commentary: The Hormone Replacement-Coronary Heart Disease Conundrum: Is This the Death of Observational Epidemiology? // International Journal of Epidemiology, 05 2004, vol. 33, no. 3. — P. 464–467.
335. *Le Texier T.* Debunking the Stanford Prison Experiment // American Psychologist, 08 2019, vol. 74.
336. The Lean Theorem Prover (System Description). / L. M. de Moura et al. // CADE / LNCS vol. 9195, Springer, 2015. — P. 378–388.
337. *Lefkowitz M.* Professor's Perceptron Paved the Way for AI — 60 Years Too Soon, Cornell Chronicle. https://news.cornell.edu/stories/2019/09/professors-perceptron-paved-way-ai-60-years-too-soon, 2019.
338. Letters to the Editor // The American Statistician, 1975, vol. 29, no. 1. — P. 67–71.
339. *Levandowsky M.* Distance between Sets // Nature, 1971, vol. 234, no. 5323. — P. 34–35.
340. *Li C.-D., Tan D.-R., Jiang F.-J.* Applications of Neural Networks to the Studies of Phase Transitions of Two-Dimensional Potts Models // Annals of Physics, Apr. 2018, vol. 391. — P. 312–331.
341. *Li W.* Random Texts Exhibit Zipf's-Law-Like Word Frequency Distribution // IEEE Transactions on Information Theory, 1992, vol. 38, no. 6. — P. 1842–1845.
342. *Liao S., Lan S., Zachariah A. G.* EVA-GAN: Enhanced Various Audio Generation via Scalable Generative Adversarial Networks, 2024.
343. *Libbrecht M. W., Noble W. S.* Machine Learning Applications in Genetics and Genomics // Nature Reviews Genetics, 2015, vol. 16, no. 6. — P. 321–332.
344. *Libet B., Gleason C.* Time of Conscious Intention to Act in Relation to Onset of Cerebral Activity (Readiness-Potential) // Brain, vol. 106, no. 3. — P. 623–642.
345. *Lin H.-T., Lin C.-J., Weng R. C.* A Note on Platt's Probabilistic outputs for support vector Machines // Mach. Learn., Oct. 2007, vol. 68, no. 3. — P. 267–276.
346. *Lincoln F. C.* Calculating Waterfowl Abundance on the Basis of Banding Returns // Circular, 1930, vol. 118.
347. *Lindley D.* Making Decisions, Wiley, 1991.
348. *Linnainmaa S.*, The Representation of the Cumulative Rounding Error of an Algorithm as a Taylor Expansion of the Local Rounding Errors, Master's thesis, Univ. Helsinki, 1970.
349. *Liu H.* Feature Selection // Encyclopedia of Machine Learning / Ed. by C. Sammut, G. I. Webb, Boston, MA: Springer US, 2010. — P. 402–406.
350. *Liu X.-Y., Wu J., Zhou Z.-H.* Exploratory Undersampling for Class-Imbalance Learning // IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics), 2008, vol. 39, no. 2. — P. 539–550.
351. *Loshchilov I., Hutter F.* Fixing Weight Decay Regularization in Adam // ArXiv preprint abs/1711.05101, 2017. http://arxiv.org/abs/1711.05101.

352. *Loshchilov I., Hutter F.* Decoupled Weight Decay Regularization // International Conference on Learning Representations, 2019. https://openreview.net/forum?id=Bkg6RiCqY7.
353. *Lovett A. A., Flowerdew R.* Analysis of Count Data Using Poisson Regression // The Professional Geographer, 1989, vol. 41. — P. 190–198.
354. *Lowe D. G.* Object Recognition from Local Scale-Invariant Features // Proc. Seventh IEEE International Conference on Computer Vision / vol. 2, 1999. — P. 1150–1157 vol.2.
355. *Lowe D. G.* Distinctive Image Features from Scale-Invariant Keypoints // International Journal of Computer Vision, 2004, vol. 60, no. 2. — P. 91–110.
356. *Luenberger D. G., Ye Y.* Linear and Nonlinear Programming. International Series in Optimization Research & Management Science, 4th edition edition, Springer, 2016.
357. *Lundervold A. S., Lundervold A.* An Overview of Deep Learning in Medical Imaging Focusing on MRI // Zeitschrift für Medizinische Physik, 2019, vol. 29, no. 2. — P. 102–127, Special Issue: Deep Learning in Medical Physics.
358. *Lüroth J.* Vergleichung von zwei Werthen des wahrscheinlichen Fehlers // Astronomische Nachrichten, 1876, vol. 87, no. 14. — P. 209–220.
359. *Lusinchi D.* "President" Landon and the 1936 Literary Digest Poll: Were Automobile and Telephone Owners to Blame? // Social Science History, 2012, vol. 36, no. 1. — P. 23–54.
360. *Lyons L.* Discovering the Significance of 5 Sigma, 2013.
361. *Lyons L.* Five Sigma Revisited // CERN Courier, 2023. https://cerncourier.com/a/five-sigma-revisited/.
362. *Lyons R.* Understanding Digital Signal Processing, 3rd edition: Pearson Education, Prentice Hall, 2010.
363. Machine Learning and the Physical Sciences / G. Carleo et al. // Reviews of Modern Physics, Dec. 2019, vol. 91, no. 4.
364. Machine Learning for Data-Driven Discovery in Solid Earth Geoscience / K. J. Bergen et al. // Science, 2019, vol. 363, no. 6433. — Article eaau0323.
365. Machine Learning for Medical Imaging / B. J. Erickson et al. // RadioGraphics, 2017, vol. 37, no. 2. — P. 505–515, PMID: 28212054.
366. Machine Learning in Geosciences and Remote Sensing / D. J. Lary et al. // Geoscience Frontiers, 2016, vol. 7, no. 1. — P. 3–10, Special Issue: Progress of Machine Learning in Geosciences.
367. *MacKay D. J.* Information Theory, Inference and Learning Algorithms, Cambridge Univ. Press, 2003.
368. *Maclagan D., Sturmfels B.* Introduction to Tropical Geometry, American Mathematical Society, 2015.
369. *Madsen H., Thyregod P.* Introduction to General and Generalized Linear Models, Chapman & Hall/CRC, 2011.
370. *Magis D.* A Note on Weighted Likelihood and Jeffreys Modal Estimation of Proficiency Levels in Polytomous Item Response Models // Psychometrika, 2015, vol. 80. — P. 200–204.
371. *Malthus T. R.* An Essay on the Principle of Population, J. Johnson, London, 1798.
372. *Mandelbrot B.* An Informational Theory of the Statistical Structure of Language // Communication Theory / Ed. by W. Jackson, Betterworths, 1953.
373. *Mandelbrot B.* Information Theory and Psycholinguistics // Scientific Psychology / Ed. by B. Wolman, E. Nagel, 1965.
374. *Mandelbrot B.* How Long Is the Coast of Britain? Statistical Self-Similarity and Fractional Dimension // Science, 1967, vol. 156, no. 3775. — P. 636–638.
375. *Manning C. D., Raghavan P., Schütze H.* Introduction to Information Retrieval, Cambridge Univ. Press, 2008.

376. *Marlow W. H.* Factorial Distributions // The Annals of Mathematical Statistics, 1965, vol. 36, no. 3. — P. 1066–1068.
377. *Maron M. E.* Automatic Indexing: An Experimental Inquiry // J. ACM, Jul. 1961, vol. 8, no. 3. — P. 404–417.
378. *Marquis de la Place.* Mécanique céleste, Hillard, Gray, Little, and Wilkins, 1829.
379. Mastering Atari, Go, Chess and Shogi by Planning with a Learned Model / J. Schrittwieser et al. // Nature, 2020, vol. 588. — P. 604–609.
380. Mastering the game of Go with Deep Neural Networks and tree search / D. Silver et al. // Nature, 2016, vol. 529, no. 7587. — P. 484–489.
381. *Matas J., Chum O.* Randomized RANSAC with Td,d test // Image and Vision Computing, 09 2004, vol. 22. — P. 837–842.
382. Mathematical Discoveries from Program Search with Large Language Models / B. Romera-Paredes et al. // Nature, 12 2023, vol. 625.
383. *Matousek J., Mount D. M., Netanyahu N. S.* Efficient Randomized Algorithms for the Repeated Median Line Estimator // Proc. 4th SODA, SIAM, 1993. — P. 74–82.
384. *The Coq Development team.* The Coq Proof Assistant Reference Manual, 2004. http://coq.inria.fr.
385. *McCullagh P., Nelder J.* Generalized Linear Models, Chapman & Hall/CRC, 1989.
386. *McCulloch W., Pitts W.* A Logical Calculus of the Ideas Immanent in Nervous Activity // Bulletin of Mathematical Biophysics, 1943, vol. 7. — P. 115–133.
387. *McCulloh I., Armstrong H., Johnson A.* Social Network Analysis with Applications, Wiley, 2013.
388. *McCune W.* Solution of the Robbins Problem // Journal of Automated Reasoning, 1997, vol. 19. — P. 263–276.
389. *McCune W., Wos L.* Otter-the CADE-13 competition incarnations // Journal of Automated Reasoning, 1997, vol. 18. — P. 211–220.
390. *McLachlan G. J.* Discriminant Analysis and Statistical Pattern Recognition, New York, NY: Wiley Interscience, 1992.
391. Measuring the Predictability of Life Outcomes with a Scientific Mass Collaboration / M. J. Salganik et al. // Proc. National Academy of Sciences, 2020, vol. 117, no. 15. — P. 8398–8403.
392. *Mermin N. D.* What's Wrong with This Pillow? // Physics Today, 1989. — P. 9.
393. *Mikhalkin G.* Tropical Geometry and Its Applications // International Congress of Mathematicians Madrid 2006. Vol. II. Invited Lectures, 2007. — P. 827–852.
394. *Milgram S.* Behavioural Study of Obedience // Journal of abnormal Psychology, 1963, vol. 67. — P. 371–378.
395. *Miller J. B., Sanjurjo A.* A Cold Shower for the Hot Hand Fallacy: Robust Evidence that Belief in the Hot Hand is Justified, IGIER Working Paper No. 518, 2018. https://ssrn.com/abstract=2450479.
396. *Miller J. B., Sanjurjo A.* Surprised by the Hot Hand Fallacy? A Truth in the Law of Small Numbers // Econometrica, 2018, vol. 86, no. 6. — P. 2019–2047.
397. *Minkowski H.* Raum und Zeit // Jahresbericht der Deutschen Mathematiker-Vereinigung, 1909, vol. 18. — P. 75–88.
398. *Minkowski H.* Geometrie der Zahlen. In 2 Lieferungen. II. (Schlu -) Lieferung., Leipzig: B. G. Teubner. viii, 241–256, 1910.
399. *Minsky M.* Steps Toward Artificial Intelligence // Computers and Thought / McGraw-Hill, New York, 1963. — P. 406–450.
400. *Minsky M.* Matter, Mind and Models // IFIP Congress, Spartan Books, 1965. — P. 45–50.
401. *Minsky M., Papert S.* Perceptrons, Cambridge, MA: MIT Press, 1969.
402. *Neal B., Mittal S. et al.* A Modern Take on the Bias-Variance Tradeoff in Neural Networks, 2019. https://openreview.net/forum?id=HkgmzhC5F7.

403. *Moivre A. D.* De mensura sortis, seu, de probabilitate eventuum in ludis a casu fortuito pendentibus // Philosophical Transactions of the Royal Society of London, 1710, vol. 27, no. 329. — P. 213–264.
404. *Moivre A. D.* The doctrine of Chances; or, a Method of calculating the Probability of events in play, London, 1718.
405. Molecular Generation for Desired Transcriptome Changes with Adversarial Autoencoders / R. Shayakhmetov et al. // Frontiers in Pharmacology, 2020, vol. 11.
406. *Polykovskiy D., Zhebrak A. et al.* Molecular Sets (MOSES): A Benchmarking Platform for Molecular Generation Models, Arxiv preprint arXiv:1811.12823 [cs.CV], 2018.
407. *Molina M., Garip F.* Machine Learning for Sociology // Annual Review of Sociology, 2019, vol. 45, no. 1. — P. 27–45.
408. *Monti S., Cooper G. F.* A Bayesian Network Classifier That Combines a Finite Mixture Model and a Naïve Bayes Model // Proc. Fifteenth Conference on Uncertainty in Artificial Intelligence, San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 1999. — P. 447–456.
409. *Morgan S. L., Winship C.* Counterfactuals and Causal Inference: Methods and Principles for Social Research, Cambridge Univ. Press, 2014.
410. *Motulsky H. J.* Common Misconceptions about Data Analysis and Statistics // Pharmacology Research & Perspectives, 2015, vol. 3, no. 1. — P. e00093.
411. *Moura L. D., Ullrich S.* The Lean 4 Theorem Prover and Programming Language // Proc. CADE 28, Springer, 2021. — P. 625–635.
412. *Mudholkar G. S., Hutson A. D.* The Epsilon–Skew–Normal Distribution for Analyzing Near-Normal Data // Journal of Statistical Planning and Inference, 2000, vol. 83, no. 2. — P. 291–309. https://www.sciencedirect.com/science/article/pii/S0378375899000968.
413. *Mullainathan S., Spiess J.* Machine Learning: An Applied Econometric Approach // Journal of Economic Perspectives, May 2017, vol. 31, no. 2. — P. 87–106. https://www.aeaweb.org/articles?id=10.1257/jep.31.2.87.
414. Multi-class Generative Adversarial Networks with the L2 Loss Function / X. Mao et al. // ArXiv preprint abs/1611.04076, 2016. http://arxiv.org/abs/1611.04076.
415. *Munroe R.* XKCD 882: Significant, 2005. https://xkcd.com/882/.
416. *Murphy A. H.* A New Vector Partition of the Probability Score // Journal of Applied Meteorology and Climatology, 1973, vol. 12, no. 4. — P. 595–600.
417. *Murphy A. H.* The Early History of Probability Forecasts: Some Extensions and Clarifications // Weather and Forecasting, 1998, vol. 13, no. 1. — P. 5–15.
418. *Murphy A. H., Winkler R. L.* Probability Forecasting in Meterology // Journal of the American Statistical Association, 1984, vol. 79, no. 387. — P. 489–500.
419. *Murphy K. P.* Machine Learning: a Probabilistic Perspective, Cambridge Univ. Press, 2013.
420. *Myers R. H., Montgomery D. C., Anderson-Cook C. M.* Response Surface Methodology: Process and Product Optimization Using Designed Experiments, Wiley, 2016.
421. Myopia and Ambient Lighting at Night / G. E. Quinn et al. // Nature, 1999, vol. 399, no. 6732. — P. 113–114.
422. *Müller R., Kornblith S., Hinton G.* When Does Label Smoothing Help?, 2020.
423. ∇DFT: Large-Scale Conformational Energy and Hamiltonian Prediction Benchmark and Dataset / K. Khrabrov et al. // Physical Chemistry Chemical Physics, 2022, vol. 24. — P. 25 853—25 863.
424. Naive Bayes for Regression / E. Frank et al. // Mach. Learn., 2000, vol. 41. — P. 5–25.
425. *Nannen V.* A Short Introduction to Model Selection, Kolmogorov Complexity and Minimum Description Length (MDL), 2010.
426. *Nash J.* Real Algebraic Manifolds // Ann. Math., 1952, vol. 56, no. 3. — P. 405–421.
427. *Nash J. F.* Non-Cooperative Games: Ph.D. thesis / Princeton University, 1950.

428. *Naumowicz A., Kornilowicz A.* A Brief Overview of Mizar // Theorem Proving in Higher Order Logics, Berlin, Heidelberg: Springer Berlin Heidelberg, 2009. — P. 67–72.
429. *Nederpelt R. P., Geuvers J. H., de Vrijer R. C.* Selected Papers on Automath, Elsevier, Amsterdam, 1994, vol. 133 of *Studies in Logic*.
430. *Nesterov Y.* Introductory Lectures on Convex Optimization, Springer, 2004.
431. *Newell A.* You Can't Play 20 Questions with Nature and Win: Projective Comments on the Papers of this Symposium // Visual Information Processing / Ed. by W. G. Chase, 1973. — P. 283–308.
432. *Newell A., Shaw J. C., Simon H. A.* Elements of a Theory of Human Problem Solving // Psychological Review, 1958, vol. 65, no. 3. — P. 151–166.
433. *Newell A., Simon H.* The Logic Theory Machine–A Complex Information Processing System // IRE Transactions on Information Theory, 1956, vol. 2, no. 3. — P. 61–79.
434. *Niculescu-Mizil A., Caruana R.* Predicting Good Probabilities with Supervised Learning // Proc. 22nd International Conference on Machine Learning, New York, NY, USA: ACM, 2005. — P. 625–632.
435. *Nikolenko S., Semenov N.* Chow Ring Structure Made Simple, 2006. ArXiv preprint Math/0606335.
436. *Nikolenko S. I.* SVD-LDA: Topic Modeling for Full-Text Recommender Systems // Proc. 14th Mexican International Conference on Artificial Intelligence / vol. 9414 of Lecture Notes in Artificial Intelligence, Springer, 2015. — P. 67–79.
437. *Nikolenko S. I.* Synthetic Data for Deep Learning, Springer, 2021.
438. *Nikolenko S. I., Koltsova O., Koltsov S.* Topic Modelling for Qualitative Studies // Journal of Information Science, 2017, vol. 43, no. 1. — P. 88–102.
439. *Nikolenko S. I., Semenov N., Zainoulline K.* Motivic Decomposition of Anisotropic Varieties of Type F4 into Generalized Rost Motives // J. K-Theory, 2009, vol. 3. — P. 85–102.
440. *Nipkow T., Wenzel M., Paulson L. C.* Isabelle/HOL: A Proof Assistant for Higher-Order Logic, Berlin, Heidelberg: Springer-Verlag, 2002.
441. *Nocedal J., Wright S. J.* Numerical Optimization. Springer Series in Operations Research and Financial Engineering, Springer, 2006.
442. *Nocedal J., Wright S. J.* Numerical Optimization. Springer Series in Operations Research and Financial Engineering, Springer, New York, 2006.
443. *Norvig P.* On Chomsky and the Two Cultures of Statistical Learning, https://norvig.com/chomsky.html, 2011.
444. Numerical Optimization: Theoretical and Practical Aspects / J.-F. Bonnans et al., Springer, Berlin Heidelberg, 2013.
445. *O'Donnell T.* History of Life Insurance, American Conservation Co., Chicago, 1936.
446. *Omohundro S.* Five Balltree Construction Algorithms: Tech. Rep. Technical Report 89-063: USAF School of Aviation Medicine, Randolph Field, Texas, 1989.
447. Ontology Visualization Methods and Tools: a Survey of the State of the Art / M. Dud et al. // The Knowledge Engineering Review, 2018, vol. 33. — P. e10.
448. Ontology Visualization Methods—a Survey / A. Katifori et al. // ACM Comput. Surv., Nov. 2007, vol. 39, no. 4. — P. 10–es.
449. *Open Science Collaboration.* Estimating the Reproducibility of Psychological Science // Science, 2015, vol. 349, no. 6251. — P. aac4716.
450. *OpenAI.* GPT-4 Technical Report, 2023.
451. *Oppenheim A., Schafer R.* Discrete-Time Signal Processing, 3rd edition: Pearson Education, Prentice Hall, 2009.
452. An Optimal Algorithm for Approximate Nearest Neighbor Searching Fixed Dimensions / S. Arya et al. // J. ACM, Nov. 1998, vol. 45, no. 6. — P. 891–923.

453. Paediatric Bone Age Assessment Using Deep Convolutional Neural Networks / V. I. Iglovikov et al. // Deep Learning in Medical Image Analysis and Multimodal Learning for Clinical Decision Support, Springer, 2018. — P. 300–308.
454. *Papert S.* The Summer Vision Project, MIT Artificial Intelligence Group Vision Memo no. 100, 1966. https://people.csail.mit.edu/brooks/idocs/AIM-100.pdf.
455. *Pashler H., Coburn N., Harris C. R.* Priming of Social Distance? Failure to Replicate Effects on Social and Food Judgments // PLOS ONE, 08 2012, vol. 7, no. 8. — P. 1–6.
456. *Pavelle R., Wang P. S.* MACSYMA from F to G // Journal of Symbolic Computation, 1985, vol. 1, no. 1. — P. 69–100.
457. *Pearl J.* Causal Inference in Statistics: An Overview // Statistics Surveys, 2009, vol. 3. — P. 96–146.
458. *Pearl J.* Causality: Models, Reasoning and Inference, Cambridge Univ. Press, 2009.
459. *Pearl J., Glymour M., Jewell N. P.* Causal Inference in Statistics: A Primer, Wiley, 2016.
460. *Pearson K. A., Palafox L., Griffith C. A.* Searching for Exoplanets Using Artificial Intelligence // Monthly Notices of the Royal Astronomical Society, Oct 2017, vol. 474, no. 1. — P. 478–491.
461. *Perry G.* Behind the Shock Machine: the Untold Story of the Notorious Milgram Psychology Experiments, The New Press, 2012.
462. *Peters J., Janzing D., Schölkopf B.* Elements of Causal Inference: Foundations and Learning Algorithms, MIT Press, 2017.
463. *Petersen C. G. J.* The Yearly Immigration of Young Plaice Into the Limfjord From the German Sea // Report of the Danish Biological Station, 1995, vol. 6. — P. 5–84.
464. *Phillips J. D., Stanovsky D.* Automated Theorem Proving in Loop Theory // Proc. CICM Workshop on Empirically Successful Automated Reasoning in Mathematics, 2008. — P. 42–54.
465. *Piaget J.* Judgment and Reasoning in the Child. International Library of Psychology, Philosophy and Scientific Method, 1959.
466. *Piaget J.* Le langage et la pensée chez l'enfant: études sur la logique de l'enfant. Actualités pédagogiques et psychologiques, Delachaux et Niestlé, 2002.
467. *Piaget J.* La représentation du monde chez l'enfant. Quadrige, Humensis, 2018.
468. *Piaget J., Inhelder B.* Psychology Of The Child. Basic Books, 1969.
469. *Pickover C. A.* Artifical Intelligence: An Illustrated History: From Medieval Robots to Neural Networks, Sterling, 2019.
470. *Pitman E. J. G.* Sufficient Statistics and Intrinsic Accuracy // Mathematical Proc. Cambridge Philosophical society, 1936, vol. 32, no. 4. — P. 567–579.
471. *Platt J. C.* Probabilistic Outputs for Support Vector Machines and Comparisons to Regularized Likelihood Methods // Advances in Large Margin Classifiers, MIT Press, 1999. — P. 61–74.
472. *Poisson S. D.* Recherches sur la probabilité des jugements en matière criminelle et en matière civile, Paris, 1837.
473. *Poisson S. D.* English Translation of Poisson's "Recherches sur la probabilité des jugements en matière criminelle et en matière civile" / "Researches into the Probabilities of Judgements in Criminal and Civil Cases", 2019. https://arxiv.org/abs/1902.02782.
474. Prediction Policy Problems / J. Kleinberg et al. // American Economic Review, May 2015, vol. 105, no. 5. — P. 491–495.
475. *Preston F. W., Henderson J. H.* Fourier Series Characterization of Cyclic Sediments for Stratigraphic Correlation // Symposium on cyclic sedimentation: Kansas Geological Survey, Bulletin / 1954. — P. 415–425.
476. A primer on Deep Learning in Genomics / J. Zou et al. // Nature Genetics, 11 2018, vol. 51.

477. *Prince S.* Computer Vision: Models, Learning, and Inference, Cambridge Univ. Press, 2012.
478. *Proakis J., Manolakis D.* Digital Signal Processing, 4th edition: Pearson Education, Prentice Hall, 2006.
479. Progressive Growing of GANs for Improved Quality, Stability, and Variation / T. Karras et al. // ArXiv preprint abs/1710.10196, 2017. http://arxiv.org/abs/1710.10196.
480. *McCarthy J., Minsky M. et al.* A Proposal for the Dartmouth Summer Research Project on Artificial Intelligence, 1955. http://www-formal.stanford.edu/jmc/history/dartmouth/dartmouth.html.
481. *Zhang Y., Qin J. et al.* Pushing the Limits of Semi-Supervised Learning for Automatic Speech Recognition, 2022.
482. *Puthenpura S., Sinha N.* Modified Maximum Likelihood Method for the Robust Estimation of System Parameters from Very Noisy Data // Automatica, 1986, vol. 22, no. 2. — P. 231–235.
483. PyTorch: An Imperative Style, High-Performance Deep Learning Library / A. Paszke et al. // Advances in Neural Information Processing Systems 32 / Curran Associates, Inc., 2019. — P. 8024–8035.
484. *Rajaraman A., Ullman J. D.* Data Mining // Mining of Massive Datasets, Cambridge Univ. Press, 2011. — P. 1–17.
485. *Ramscar M.* Learning and the Replicability of Priming Effects // Current Opinion in Psychology, 2016, vol. 12. — P. 80–84.
486. *Ramscar M., Shaoul C., Baayen R. H.* Why Many Priming Results Don't (and Won't) Replicate: A Quantitative Analysis, Eberhard Karls University, Germany, 2015.
487. Randomized Trial of Estrogen Plus Progestin for Secondary Prevention of Coronary Heart Disease in Postmenopausal Women / S. Hulley et al. // JAMA, 08 1998, vol. 280, no. 7. — P. 605–613.
488. *Rasmussen C. E., Williams C. K. I.* Gaussian Processes For Machine Learning, MIT Press, 2006.
489. Reading Tea Leaves: How Humans Interpret Topic Models / J. Chang et al. // Advances in Neural Information Processing Systems, 2009, vol. 20.
490. Realistic Atomistic Structure of Amorphous Silicon from Machine-Learning-Driven Molecular Dynamics / V. L. Deringer et al. // The Journal of Physical chemistry letters, 2018, vol. 9, no. 11. — P. 2879–2885.
491. Research-Paper Recommender Systems: A Literature Survey / J. Beel et al. // International Journal of Digital Libraries, Nov. 2016, vol. 17, no. 4. — P. 305–338.
492. *Reymond J.-L.* The Chemical Space Project // Accounts of Chemical Research, 2015, vol. 48. — P. 722–730.
493. *Rissanen J.* Modeling by Shortest Data Description // Automatica, 1978, vol. 14, no. 5. — P. 465–471.
494. *Rissanen J.* Information and Complexity in Statistical Modeling, 1 edition, Springer Publishing Company, Incorporated, 2007.
495. *Robert C. P., Chopin N., Rousseau J.* Harold Jeffreys's Theory of Probability Revisited // Statistical Science, 2009, vol. 24, no. 2. — P. 141–172.
496. *Robertson S.* Understanding Inverse Document Frequency: On Theoretical Arguments for IDF // Journal of Documentation, 2004, vol. 60, no. 5. — P. 503–520.
497. *Robertson S., Zaragoza H.* The Probabilistic Relevance Framework: BM25 and Beyond // Found. Trends Inf. Retr., Apr. 2009, vol. 3, no. 4. — P. 333–389.
498. *Robertson S. E., Jones K. S.* Relevance Weighting of Search Terms // Journal of the American Society for Information Science, 1976, vol. 27, no. 3. — P. 129–146.
499. *Robertson S. E., Walker S.* Some Simple Effective Approximations to the 2-Poisson Model for Probabilistic Weighted Retrieval // SIGIR '94, Springer, 1994. — P. 232–241.

500. Robust Speech Recognition via Large-Scale Weak Supervision / A. Radford et al. // Proc. 40th International Conference on Machine Learning, JMLR.org, 2023. — 27 p.
501. *Roriz R., Cabral J., Gomes T.* Automotive LiDAR Technology: A Survey // IEEE Transactions on Intelligent Transportation Systems, 2022, vol. 23, no. 7. — P. 6282–6297.
502. *Rosheim M. E.* Leonardo's Knight // Leonardo's Lost Robots, Berlin, Heidelberg: Springer Berlin Heidelberg, 2006. — P. 69–113.
503. *Ross S.* A First Course in Probability, Pearson Prentice Hall, 2010.
504. *Rossmo D. K.* Criminal Investigative Failures, CRC Press Taylor & Francis Group, 2009.
505. *Rudin W.* Principles of Mathematical Analysis, McGraw Hill, 1964.
506. *Ruggles R., Brodie H.* An Empirical Approach to Economic Intelligence in World War II // Journal of the American Statistical Association, 1947, vol. 42, no. 237. — P. 72–91.
507. *Rumelhart D. E., Hinton G. E., Williams R. J.* Learning Internal Representations by Error Propagation // Parallel Distributed Processing. Vol 1: Foundations / Cambridge, MA, USA: MIT Press, 1986.
508. *Rumelhart D. E., Hinton G. E., Williams R. J.* Learning Representations by Back-Propagating Errors // Neurocomputing: Foundations of Research / Cambridge, MA, USA: MIT Press, 1988. — P. 696–699.
509. *Russell S. J., Norvig P.* Artificial Intelligence: A Modern Approach, Hoboken: Pearson, 4th edition, 2021.
510. *Sadinle M.* Transformed Logit Confidence Intervals for Small Populations in Single Capture–Recapture Estimation // Communications in Statistics - Simulation and Computation, 2009, vol. 38, no. 9. — P. 1909–1924.
511. *Ghojogh B., Nekoei H. et al.* Sampling Algorithms, from Survey Sampling to Monte Carlo Methods: Tutorial and Literature Review, 2020.
512. *Samworth R. J.* Stein's Paradox // Eureka, 2012, vol. 62. — P. 38–41.
513. *Sandmo A.* Chapter 1 — The Principal Problem in Political Economy: Income Distribution in the History of Economic Thought // Handbook of Income Distribution / Elsevier, 2015. — P. 3–65.
514. *Sathe S., Aggarwal S., Tang J.* Gene Expression and Protein Function: A Survey of Deep Learning Methods // SIGKDD Explor. Newsl., 2019, vol. 21, no. 2. — P. 23–38.
515. *Schapire R. E., Freund Y.* Boosting: Foundations and Algorithms, MIT Press, 2012.
516. *Schölkopf B., Smola A. J., Bach F.* Learning with Kernels: Support Vector Machines, Regularization, Optimization, and Beyond, MIT Press, 2018.
517. *Schwarz G.* Estimating the Dimension of a Model // The Annals of Statistics, 1978, vol. 6, no. 2. — P. 461–464.
518. Searching by Code: a New SearchBySnippet Dataset and SnippeR Retrieval Model for Searching by Code Snippets / I. Sedykh et al. // Proc. 2024 Joint Intl. Conf. on Computational Linguistics, Language Resources and Evaluation, 2024.
519. *Searle J. R.* Speech Acts: An Essay in the Philosophy of Language, Cambridge Univ. Press, 1969.
520. *Searle J. R.* Minds, Brains, and Programs // Behavioral and Brain Sciences, 1980, vol. 3. — P. 417–424.
521. *Searle J. R.* Intentionality: An Essay in the Philosophy of Mind, Cambridge Univ. Press, 1983.
522. *Searle J. R.* Is the Brain a Digital Computer? // Proceedings and Addresses of the American Philosophical Association, 1990, vol. 64, no. 3. — P. 21–37.
523. *Searle J. R.* The Rediscovery of the Mind, MIT Press, 1992.
524. *Searle J. R., Vanderveken D.* Foundations of Illocutionary Logic, Cambridge Univ. Press, 1985.
525. *Sen P. K.* Estimates of the Regression Coefficient Based on Kendall's Tau // Journal of the American Statistical Association, 1968, vol. 63, no. 324. — P. 1379–1389.

526. *Sennrich R., Haddow B., Birch A.* Neural Machine Translation of Rare Words with Subword Units // Proc. 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), Berlin, Germany: Association for Computational Linguistics, 2016. — P. 1715–1725.
527. *Shanahan P. E., Trewartha D., Detmold W.* Machine Learning Action Parameters in Lattice Quantum Chromodynamics // Physical Review D, May 2018, vol. 97, no. 9.
528. *Shannon C. E.* A Symbolic Analysis of Relay and Switching Circuits // Transactions of the American Institute of Electrical Engineers, 1938, vol. 57, no. 12. — P. 713–723.
529. *Shannon C. E.* A Mathematical Theory of Communication // Bell System Technical Journal, July & October 1948, vol. 27. — P. 379–423 & 623–656.
530. *Shannon C. E.* Communication Theory of Secrecy Systems // Bell System Technical Journal, 1949, vol. 28, no. 4. — P. 656–715.
531. *Sharifi K., Leon-Garcia A.* Estimation of Shape Parameter for Generalized Gaussian Distributions in Subband Decompositions of Video // IEEE Transactions on Circuits and Systems for Video Technology, 1995, vol. 5, no. 1. — P. 52–56.
532. *Shawe-Taylor J., Cristianini N.* Kernel Methods for Pattern Analysis, Cambridge Univ. Press, 2004.
533. *Shen D., Wu G., Suk H.* Deep Learning in Medical Image Analysis // Annual Review of Biomedical Engineering, Jun. 2017, Vol. 19. — P. 221–248.
534. *Sheynin O.* Helmert's Work in the Theory of Errors // Archive for History of Exact Sciences, 1995, vol. 49. — P. 73–104.
535. *Sheynin O.* History of Statistics, Berlin: NG Verlag Berlin, 2012.
536. *Shibuya T., Takida Y., Mitsufuji Y.* BigVSAN: Enhancing GAN-based Neural Vocoders with Slicing Adversarial Network // ArXiv preprint 2309.02836, 2023. https://arxiv.org/abs/2309.02836.
537. *Shoda Y., Mischel W., Peake P. K.* Predicting Adolescent Cognitive and Self-Regulatory Competencies from Preschool Delay of Gratification: Identifying Diagnostic Conditions // Developmental Psychology, 1990, vol. 26. — P. 978–986.
538. *Shoesmith E.* The Continental Controversy over Arbuthnot's Argument for Divine Providence // Historia Mathematica, 1987, vol. 14, no. 2. — P. 133–146.
539. *Shortliffe E. H., Buchanan B. G.* A Model of Inexact Reasoning in Medicine // Mathematical biosciences, 1975, vol. 23. — P. 351–379.
540. *Siegel A. F.* Robust Regression using Repeated Medians // Biometrika, 04 1982, vol. 69, no. 1. — P. 242–244.
541. *Siegert S.* Simplifying and Generalising Murphy's Brier Score Decomposition // Quarterly Journal of the Royal Meteorological Society, 2017, vol. 143, no. 703. — P. 1178–1183.
542. *Sierpinski W., Sierpinski W.* 250 Problems in Elementary Number Theory. Modern Analytic and Computational Methods in Science and Mathematics, American Elsevier Publishing Company, 1970.
543. *Simmons J. P., Nelson L. D., Simonsohn U.* False-Positive Psychology: Undisclosed Flexibility in Data Collection and Analysis Allows Presenting Anything as Significant // Psychological Science, 2011, vol. 22, no. 11. — P. 1359–1366.
544. *Simon I.* Recognizable Sets with Multiplicities in the Tropical Semiring // Mathematical Foundations of Computer Science 1988, Springer, 1988. — P. 107–120.
545. *Smith D. J., Vamanamurthy M. K.* How Small Is a Unit Ball? // Mathematics Magazine, 1989, vol. 62, no. 2. — P. 101–107.
546. SMOTE: Synthetic Minority Over-Sampling Technique / N. V. Chawla et al. // Journal of Artificial Intelligence Research, 2002, vol. 16. — P. 321–357.
547. *Smyth P., Welling M., Asuncion A.* Asynchronous Distributed Learning of Topic Models // Advances in Neural Information Processing Systems, 2008, vol. 21.

548. *Soleimanitaleb Z., Keyvanrad M. A.* Single Object Tracking: A Survey of Methods, Datasets, and Evaluation Metrics, 2022.
549. Sora: A Review on Background, Technology, Limitations, and Opportunities of Large Vision Models / Liu Y., Zhang K. et al. // 2024.
550. *Sparck Jones K.* A Statistical Interpretation of Term Specificity and Its Application in Retrieval // Journal of documentation, 1972, vol. 28, no. 1. — P. 11–21.
551. *Spear M. E.* Charting Statistics, McGraw Hill, 1952.
552. *Spear M. E.* Practical Charting Techniques, McGraw Hill, 1969.
553. SpeechFormer++: A Hierarchical Efficient Framework for Paralinguistic Speech Processing / W. Chen et al. // IEEE/ACM Transactions on Audio, Speech, and Language Processing, 2023, vol. 31. — P. 775–788.
554. *Speyer D., Sturmfels B.* Tropical Mathematics // Mathematics Magazine, 2009, vol. 82, no. 3. — P. 163–173.
555. *Sproull R. F.* Refinements to nearest-neighbor searching in k-dimensional trees // Algorithmica, 1991, vol. 6. — P. 579–589.
556. *Stampfer M. J., Colditz G. A.* Estrogen Replacement Therapy and Coronary Heart Disease: A Quantitative Assessment of the Epidemiologic Evidence // Preventive Medicine, 1991, vol. 20, no. 1. — P. 47–63.
557. *Stanton J. M.* Galton, Pearson, and the Peas: A Brief History of Linear Regression for Statistics Instructors // Journal of Statistics Education, 2001, vol. 9, no. 3. — P. null.
558. *Stein C.* Inadmissibility of the Usual Estimator for the Mean of a Multivariate Normal Distribution // Proc. Third Berkeley Symposium on Mathematical Statistics and Probability / vol. 1, University of California Press, 1956. — P. 197–206.
559. *Stepanov A., Vavilov N.* Decomposition of Transvections: A Theme with Variations // K-theory, 2000, vol. 19, no. 2. — P. 109–153.
560. *Steuer A. E., Brockbals L., Kraemer T.* Metabolomic Strategies in Biomarker Research–New Approach for Indirect Identification of Drug Consumption and Sample Manipulation in Clinical and Forensic Toxicology? // Frontiers in Chemistry, 2019, vol. 7.
561. *Stigler S. M.* Science and Social Structure: a Festschrift for Robert K. Merton, New York Academy of Sciences, 1980.
562. *Stigler S. M.* The 1988 Neyman Memorial Lecture: A Galtonian Perspective on Shrinkage Estimators // Statistical Science, 1990, vol. 5, no. 1. — P. 147–155.
563. *Stoica P., Selen Y.* Model-Order Selection: a Review of Information Criterion Rules // IEEE Signal Processing Magazine, 2004, vol. 21, no. 4. — P. 36–47.
564. *Stone M.* An Asymptotic Equivalence of Choice of Model by Cross-Validation and Akaike's Criterion // Journal of the Royal Statistical Society: Series B (Methodological), 1977, vol. 39, no. 1. — P. 44–47.
565. Structural Risk Minimization for Character Recognition / I. Guyon et al. // Advances in Neural Information Processing Systems, vol. 4, Morgan-Kaufmann, 1991.
566. Structural Risk Minimization over Data-dependent hierarchies / J. Shawe-Taylor et al. // IEEE Transactions on Information Theory, 1998, vol. 44, no. 5. — P. 1926–1940.
567. *Stuart A., Ord K.* Kendall's Advanced Theory of Statistics / Ed. by E. Arnold, 5th edition, 1991, vol. 2, Classical Inference and Relationship.
568. *Student.* The Probable Error of a Mean // Biometrika, 1908, vol. 6, no. 2/3. — P. 1–25.
569. *Sun Y., Wong A. K., Kamel M. S.* Classification of Imbalanced Data: A Review // International Journal of Pattern recognition and Artificial Intelligence, 2009, vol. 23, no. 04. — P. 687–719.
570. *Sure Y., Staab S., Studer R.* Ontology Engineering Methodology // Handbook on Ontologies / Ed. by S. Staab, R. Studer, Berlin, Heidelberg: Springer Berlin Heidelberg, 2009. — P. 135–152.
571. *Surowiecki J.* The Wisdom of Crowds, Doubleday, 2004.

572. A Survey of Collaborative Filtering-Based Recommender Systems: From Traditional Methods to Hybrid Methods Based on Social Networks / R. Chen et al. // IEEE Access, 2018, vol. 6. — P. 64301–64320.
573. A Survey of Modern Deep Learning based Object Detection Models / S. S. A. Zaidi et al. // ArXiv preprint abs/2104.11892, 2021. https://arxiv.org/abs/2104.11892.
574. A Survey on Active Simultaneous Localization and Mapping: State of the Art and New Frontiers / J. A. Placed et al. // IEEE Transactions on Robotics, 2023, vol. 39, no. 3. — P. 1686–1705.
575. A Survey on Deep Learning Based Approaches for Scene Understanding in Autonomous Driving / Z. Guo et al. // Electronics, 2021, vol. 10, no. 4.
576. Survey on Deep Learning-Based Point Cloud Compression / M. Quach et al. // Frontiers in Signal Processing, 2022, vol. 2.
577. A Survey on Neural Recommendation: From Collaborative Filtering to Content and Context Enriched Recommendation / L. Wu et al. // ArXiv preprint abs/2104.13030, 2021. https://arxiv.org/abs/2104.13030.
578. *Sutton C., McCallum A.* An Introduction to Conditional Random Fields for Relational Learning // Introduction to Statistical Relational Learning, 2007.
579. *Sutton C., McCallum A.* An Introduction to Conditional Random Fields // Found. Trends Mach. Learn., Apr. 2012, vol. 4, no. 4. — P. 267–373.
580. *Sutton R. S.* The Bitter Lesson, 2019. http://www.incompleteideas.net/IncIdeas/BitterLesson.html.
581. *Sutton R. S., Barto A. G.* Reinforcement Learning: An Introduction, Cambridge, MA, 1998.
582. *Sutton R. S., Barto A. G.* Reinforcement Learning: An Introduction, MIT press, 2018. http://incompleteideas.net/book/the-book-2nd.html.
583. *Synnott J., Dietzel D., Ioannou M.* A Review of the Polygraph: History, Methodology and Current Status // Crime Psychology Review, 2015, vol. 1, no. 1. — P. 59–83.
584. *Szeliski R.* Computer Vision: Algorithms and Applications. Texts in Computer Science, Springer, London, 2011.
585. *Takeuchi K.* Distribution of Informational Statistics and a Criterion of Model Fitting // Suri Kagaku, 1976, vol. 153. — P. 12–18.
586. *Tan P.-N., Steinbach M., Kumar V.* Introduction to Data Mining, Addison-Wesley, 2005.
587. *Taraldsen G., Lindqvist B. H.* Improper Priors Are Not Improper // The American Statistician, 2010, vol. 64, no. 2. — P. 154–158.
588. *Tetlock P. E., Gardner D.* Superforecasting: The Art and Science of Prediction, Crown Publishers, 2015.
589. *Theil H.* A Rank-Invariant Method of Linear and Polynomial Regression Analysis I // Proc. Royal Netherlands Academy of Arts and Sciences, 1950, vol. 53. — P. 386–392.
590. *Theil H.* A Rank-Invariant Method of Linear and Polynomial Regression Analysis II // Proc. Royal Netherlands Academy of Arts and Sciences, 1950, vol. 53. — P. 521–525.
591. *Theil H.* A Rank-Invariant Method of Linear and Polynomial Regression Analysis III // Proc. Royal Netherlands Academy of Arts and Sciences, 1950, vol. 53. — P. 1397–1412.
592. *Wang X., Dang X. et al.* Theil-Sen Estimators in a Multiple Linear Regression Model, Preprint, https://home.olemiss.edu/~xdang/papers/MTSE.pdf, 2009.
593. *Thompson W. C., Schumann E. L.* Interpretation of Statistical Evidence in Criminal Trials: The Prosecutor's Fallacy and the Defense Attorney's Fallacy // Law and Human Behavior, 1987, vol. 11, no. 3. — P. 167–187.
594. *Tibshirani R.* Regression Shrinkage and Selection via the Lasso // Journal of the Royal Statistical Society, 1996, vol. 58, no. 1. — P. 267–288.
595. *The New York Times.* New Navy Device Learns by Doing; Psychologist Shows Embryo of Computer Designed to Read and Grow Wiser, July 8, 1958. — P. 25.

596. *Tkatchenko A.* Machine Learning for Chemical Discovery // Nature Communications, 08 2020, vol. 11. — P. 4125.
597. *Tokdar S. T.* STA 114: Statistics, Duke University. Lecture Notes, https://www2.stat.duke.edu/courses/Fall11/sta114/, 2011.
598. *Tomczak J. M.* Deep Generative Modeling, Springer, 2022.
599. *Tordoff B., Murray D.* Guided-MLESAC: Faster Image Transform Estimation by using matching priors // IEEE Transactions on Pattern Analysis and Machine Intelligence, 2005, vol. 27, no. 10. — P. 1523–1535.
600. *Torr P.* Bayesian Model Estimation and Selection for Epipolar Geometry and Generic Manifold Fitting // Intl. J. Computer Vision, 10 2002, vol. 50. — P. 35–61.
601. *Torr P., Zisserman A.* MLESAC: A New Robust Estimator with Application to Estimating Image Geometry // Computer Vision and Image Understanding, 08 2000, vol. 78. — P. 138–156.
602. Training Language Models to Follow Instructions with Human Feedback / L. Ouyang et al. // Advances in Neural Information Processing Systems, 2022.
603. *Truitt E.* The Garden of Earthly Delights: Mahaut of Artois and the Automata at Hesdin // Medieval Feminist Forum: A Journal of Gender and Sexuality, 2010, vol. 46, no. 1. — P. 74–79.
604. *Tukey J. W.* Exploratory Data Analysis, Addison Wesley, 1977.
605. *Turing A.* Intelligent Machinery (1948) // The Essential Turing / Oxford University Press, 2004.
606. *Turing A. M.* Computing Machinery and Intelligence // Mind, 1950, vol. 59, no. 236. — P. 433–460.
607. *Tutubalina E., Nikolenko S. I.* Inferring Sentiment-Based Priors in Topic Models // Proc. 14th Mexican International Conference on Artificial Intelligence / vol. 9414 of Lecture Notes in Computer Science, Springer, 2015. — P. 92–104.
608. *Tversky A.* Preference, Belief, and Similarity: Selected Writings / Ed. by E. Shafir, Massachusets Institute of Technology, 2004.
609. *Tversky A., Kahneman D.* Belief in the Law of Small Numbers // Psychological Bulletin, 1971. — P. 105–110.
610. Unifying Machine Learning and Quantum Chemistry with a Deep Neural Network for Molecular Wavefunctions / K. T. Schütt et al. // Nature Communications, November 2019, vol. 10. — P. 5024.
611. *Valentini G., Dietterich T.* Bias-Variance Analysis of Support Vector Machines for the Development of SVM-Based Ensemble Methods // Journal of Machine Learning Research, 07 2004, vol. 5. — P. 725–775.
612. *Van den Oord A., Vinyals O., Kavukcuoglu k.* Neural Discrete Representation Learning // Advances in Neural Information Processing Systems / vol. 30, Curran Associates, Inc., 2017.
613. *Van Noorden R., Maher B., Nuzzo R.* The top 100 papers // Nature, 10 2014, vol. 514. — P. 550–553.
614. *Van Woerkom A. J., Brodman K.* Statistics for a Diagnostic Model // Biometrics, 1961, vol. 17, no. 2. — P. 299–318.
615. *Van der Plas J.* Python Data Science Handbook: Essential Tools for Working with Data, 1st edition, O'Reilly Media, Inc., 2016.
616. *Vapnik V.* Principles of Risk Minimization for Learning Theory // Advances in Neural Information Processing Systems / vol. 4, Morgan-Kaufmann, 1991.
617. *Vavilov N., Eugene P.* Chevalley Groups Over Commutative Rings I. Elementary Calculations // Acta Applicandae Mathematicae, 05 1999, vol. 45.
618. *Vavilov N. A.* Structure of Chevalley groups over commutative rings // Nonassociative algebras and related topics, 1991. — P. 219–335.

619. *Vavilov N. A., Nikolenko S. I.* A2-proof of Structure Theorems for Chevalley groups of type F4 // St. Petersburg Mathematical Journal, 2009, vol. 20, no. 4. — P. 527–551.
620. *Verhulst P.-F.* Notice sur la loi que la population suit dans son accroissement // Correspondance mathématique et physique, 1838, vol. 10. — P. 113–121.
621. *Verhulst P.-F.* Traité élémentaire des fonctions elliptiques: ouvrage destiné a faire suite aux traités élémentaires de calcul intégral, Bruxelles: Hayez, 1841.
622. *Verhulst P.-F.* Recherches mathématiques sur la loi d'accroissement de la population // Nouveaux Mémoires de l'Académie Royale des Sciences et Belles-Lettres de Bruxelles, 1845, vol. 18. — P. 1–42.
623. *Tang Y., Bi J. et al.* Video Understanding with Large Language Models: A Survey, 2024.
624. *Vigen T.* Spurious Scholar. https://tylervigen.com/spurious-scholar, 2024.
625. *Villoutreix P.* What Machine Learning Can Do for Developmental Biology // Development, 01 2021, vol. 148, no. 1, dev188474.
626. Vision: Myopia and Ambient Night-Time Lighting / K. Zadnik et al. // Nature, 2000, vol. 404, no. 6774. — P. 143–144.
627. Vision: Myopia and Ambient Night-Time Lighting / J. Gwiazda et al. // Nature, 2000, vol. 404, no. 6774. — P. 144.
628. Visual Object Tracking: A Survey / F. Chen et al. // Computer Vision and Image Understanding, 2022, vol. 222. — P. 103508.
629. *Volz A. G.* A Soviet Estimate of German Tank Production // The Journal of Slavic Military Studies, 2008, vol. 21, no. 3. — P. 588–590.
630. *Vos Savant M.* Ask Marilyn // Parade, September 1990, vol. 16.
631. *Walsh S. M., Willman B., Jerjen H.* The Invisibles: A Detection Algorithm to Trace the Faintest Milky Way Satellites // The Astronomical Journal, 2009, vol. 137, no. 1. — P. 450–469.
632. *Wang B. X., Japkowicz N.* Imbalanced Data Set Learning with Synthetic Samples // Proc. IRIS Machine Learning workshop / vol. 19, 2004. — P. 435.
633. *Wang H., Suter D.* Robust Adaptive-Scale Parametric Model Estimation for Computer Vision // IEEE Transactions on Pattern Analysis and Machine Intelligence, 2004, vol. 26, no. 11. — P. 1459–1474.
634. *Warm T. A.* Weighted Likelihood Estimation of Ability in Item Response Theory // Psychometrika, 1989, vol. 54, no. 3. — P. 427–450.
635. *Wasserman L.* All of Nonparametric Statistics. Springer, 2006.
636. *Wasserstein R. L., Lazar N. A.* The ASA Statement on p-Values: Context, Process, and Purpose // The American Statistician, 2016, vol. 70, no. 2. — P. 129–133.
637. *Watts T. W., Duncan G. J., Quan H.* Revisiting the Marshmallow Test: A Conceptual Replication Investigating Links Between Early Delay of Gratification and Later Outcomes // Psychological Science, 2018, vol. 29, no. 7. — P. 1159–1177, PMID: 29799765.
638. WaveNet: A Generative Model for Raw Audio / A. van den Oord et al. // ArXiv preprint abs/1609.03499, 2016.
639. *Weaver W.* Translation // Machine Translation of Languages / Ed. by W. N. Locke, D. A. Booth, 1955. — P. 15–23.
640. *Werbos P. J.* Applications of Advances in Nonlinear Sensitivity Analysis // Proc. 10th IFIP Conference, 31.8 - 4.9, NYC, 1981. — P. 762–770.
641. *Werlinger B.* Auto-GPT: An Autonomous GPT-4 Experiment, 2023. https://github.com/Significant-Gravitas/Auto-GPT.
642. *Wiedijk F.* The QED Manifesto Revisited // Studies in Logic, Grammar and Rhetoric, 2007, vol. 10, no. 23. — P. 121–133.
643. *Wikipedia contributors.* Correlation Does Not Imply Causation — Wikipedia, The Free Encyclopedia, 2024. https://en.wikipedia.org/w/index.php?title=Correlation_does_not_imply_causation&oldid=1199296100.

644. *Wilcox R.* A Note on the Theil-Sen Regression Estimator When the Regressor Is Random and the Error Term Is Heteroscedastic // Biometrical Journal, 1998, vol. 40, no. 3. — P. 261–268.
645. *Willingham E.* Of Lice and Men: An Itchy History, Scientific American, https://blogs.scientificamerican.com/guest-blog/of-lice-and-men-an-itchy-history/, 2011.
646. *Wood S.* Generalized Additive Models: An Introduction, Chapman & Hall/CRC, 2006.
647. The Worst of Both Worlds: A Comparative Analysis of Errors in Learning from Data in Psychology and Machine Learning / J. Hullman et al. // Proc. 2022 AAAI/ACM Conference on AI, Ethics, and Society, New York, NY, USA: ACM, 2022. — P. 335–348.
648. *Wu X., Xu K., Hall P.* A Survey of Image Synthesis and Editing with Generative Adversarial Networks // Tsinghua Science and Technology, 2017, vol. 22, no. 6. — P. 660–674.
649. *Wyse L.* Audio Spectrogram Representations for Processing with Convolutional Neural Networks // ArXiv preprint abs/1706.09559, 2017. http://arxiv.org/abs/1706.09559.
650. *Xu F., Garcia V.* Intuitive Statistics by 8-Month-Old Infants // Proc. National Academy of Sciences, 2008, vol. 105, no. 13. — P. 5012–5015.
651. *Yates F., Mather K.* Ronald Aylmer Fisher, 1890–1962 // Biographical Memoirs of Fellows of the Royal Society, 1963, vol. 9. — P. 91–129.
652. *Ye J., Wang T.* Regularized Discriminant Analysis for High Dimensional, Low Sample Size Data // Proc. 12th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, New York, NY, USA: ACM, 2006. — P. 454–463.
653. *Yianilos P. N.* Data Structures and Algorithms for Nearest Neighbor Search in General Metric Spaces // Proc. Fourth Annual ACM-SIAM Symposium on Discrete Algorithms, USA: Society for Industrial and Applied Mathematics, 1993. — P. 311–321.
654. *Yilmaz A., Javed O., Shah M.* Object Tracking: a Survey. ACM Comput Surv // ACM Comput. Surv., 12 2006, vol. 38.
655. *Youssef R., Aniss M., Jamal C.* Machine Learning and Deep Learning in Remote Sensing and Urban Application: A Systematic Review and Meta-Analysis // Proc. 4th Intl. Conf. on Geo-IT and Water Resources 2020, Geo-IT and Water Resources 2020, New York, NY, USA: ACM, 2020. — 5 p.
656. *Yu S., Ma J.* Deep Learning for Geophysics: Current and Future Trends // Reviews of Geophysics, 2021, vol. 59, no. 3. — Article e2021RG000742.
657. *Zanette D. H.* Zipf's Law and the Creation of Musical Context // ArXiv preprint cs.CL/0406015, 2004. http://arxiv.org/abs/cs/0406015.
658. *Zeiler M. D.* ADADELTA: An Adaptive Learning Rate Method // ArXiv preprint abs/1212.5701, 2012. http://arxiv.org/abs/1212.5701.
659. Zero-Shot Text-to-Image Generation / A. Ramesh et al. // International Conference on Machine Learning, 2021. — P. 8821–8831.
660. *Zhang H.* The Optimality of Naive Bayes // Proc. 17th International Florida Artificial Intelligence Research Society Conference, FLAIRS 2004 / vol. 2, 01 2004.
661. *Zhang I. Y., Xu X.* On the Top Rung of Jacob's Ladder of Density Functional Theory: Toward resolving the dilemma of SIE and NCE // WIREs Computational Molecular Science, 2021, vol. 11, no. 1. — P. e1490.
662. *Zhou S. K., Greenspan H., Shen D.* Deep Learning for Medical Image Analysis, Elsevier, 2017.
663. *Zhou W., Serfling R.* Multivariate spatial U-quantiles: A Bahadur–Kiefer Representation, a Theil — Sen Estimator for Multiple Regression, and a Robust Dispersion Estimator // Journal of Statistical Planning and Inference, 2008, vol. 138, no. 6. — P. 1660–1678.
664. *Zipf G. K.* Selected Studies of the Principle of Relative Frequency in Language, Harvard University Press, 1932.
665. *Zipf G. K.* The Psychobiology of Language, Houghton-Mifflin, 1935.

666. *Zipf G. K.* Human Behavior and the Principle of Least Effort, Cambridge, Massachusetts: Addison-Wesley, 1949.
667. Zipf's Law Holds for Phrases, Not Words / J. R. Williams et al. // Nature Scientific Reports, 2015, vol. 5. — P. 12209.
668. *Zou H., Hastie T.* Regularization and Variable Selection via the Elastic Net // J. Royal Statistical Society: Series B, 2005, vol. 67, no. 2. — P. 301–320.
669. *Zue V., Seneff S., Glass J.* Speech Database Development at MIT: Timit and Beyond // Speech Communication, 1990, vol. 9, no. 4. — P. 351–356.
670. *Zukerman M.* Introduction to Queueing Theory and Stochastic Teletraffic Models, 2023.
671. *Баудолино.* Как работает феномен «горячей руки»? — 2021. https://www.sports.ru/tribuna/blogs/bankshot/2877553.html.
672. *Бергсон А.* Творческая эволюция. — М.: Канон-пресс, 1998.
673. *Вавилов Н. А.* Конкретная теория групп. — 2020. http://mathscinet.ru/files/VavilovN.pdf.
674. *Вавилов Н. А.* Не совсем наивная линейная алгебра. — 2020. http://alexei.stepanov.spb.ru/students/VavilovLA.pdf.
675. *Вавилов Н. А.* Не совсем наивная теория множеств. — 2020. http://patryshev.com/books/set-int.pdf.
676. *Вапник В. Н., Червоненкис А. Я.* Теория распознавания образов. — М.: Наука, 1974.
677. *Канеман Д.* Думай медленно... решай быстро. — М.: АСТ, 2014.
678. *Кант И.* Сочинения в 6 т. — М.: Мысль, 1965.
679. *Кант И.* Критика чистого разума. — М.: Мысль, 1994.
680. *Капица С. П.* Поль Дирак // Природа, 1985, т. 3. — С. 63.
681. *Кингман Дж.* Пуассоновские процессы. — М.: МЦНМО, 2007. — 136 с.
682. *Клейнрок Л.* Теория массового обслуживания. — М.: Машиностроение, 1979. — 432 с.
683. *Бернштейн С. Н.* Теория вероятностей. — М.-Л.: Гостехиздат, 1946.
684. *Колмогоров А. Н.* Автоматы и жизнь // Возможное и невозможное в кибернетике / под ред. Пекелис В. Д., Берг А. И., Кольман, Э. Я. — М.: Наука, 1964.
685. *Николенко С. И., Кадурин А. А., Архангельская Е. О.* Глубокое обучение: погружение в мир нейронных сетей. — СПб.: Питер, 2007. — 476 с.
686. *Николенко С. И., Тулупьев А. Л.* Самообучающиеся системы. — М.: МЦНМО, 2009. — 288 с.
687. *Панов Д. Ю.* Автоматический перевод. — М.: Изд-во Академии наук СССР, 1958.
688. *Пиковер К.* Искусственный интеллект: иллюстрированная история. — М.: Синдбад, 2021.
689. *Тулупьев А. Л., Николенко С. И., Сироткин А. В.* Байесовские сети: логико-вероятностный подход. — СПб.: Наука, 2006. — 608 с.
690. *Тулупьев А. Л., Сироткин А. В., Николенко С. И.* Байесовские сети доверия: логико-вероятностный вывод в ациклических направленных графах. — СПб.: Изд-во С.-Петербургского ун-та, 2009. — 400 с.
691. *Феллер В.* Введение в теорию вероятностей и её приложения. — М.: Мир, 1984.
692. *Фихтенгольц Г. М.* Курс дифференциального и интегрального исчисления. — М.: Наука, Гл. ред. физ.-мат. лит., 1969.
693. *Ширяев А. Н.* Вероятность. — М.: Наука, 1989.
694. *Ширяев А., Эрлих И., Яськов П.* Вероятность в теоремах и задачах (с доказательствами и решениями). — М.: МЦНМО, 2013.